

All Python solutions

Count: 3169

1. 1.py

Path: LeetCode-main/solutions/1. Two Sum/1.py

```
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        numToIndex = {}

        for i, num in enumerate(nums):
            if target - num in numToIndex:
                return numToIndex[target - num], i
            numToIndex[num] = i
```

2. 10.py

Path: LeetCode-main/solutions/10. Regular Expression Matching/10.py

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m = len(s)
        n = len(p)
        # dp[i][j] := True if s[0..i) matches p[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return j >= 0 and p[j] == '.' or s[i] == p[j]

        for j, c in enumerate(p):
            if c == '*' and dp[0][j - 1]:
                dp[0][j + 1] = True

        for i in range(m):
            for j in range(n):
                if p[j] == '*':
                    # The minimum index of '*' is 1.
                    noRepeat = dp[i + 1][j - 1]
                    doRepeat = isMatch(i, j - 1) and dp[i][j + 1]
                    dp[i + 1][j + 1] = noRepeat or doRepeat
                elif isMatch(i, j):
                    dp[i + 1][j + 1] = dp[i][j]

        return dp[m][n]
```

3. 100.py

Path: LeetCode-main/solutions/100. Same Tree/100.py

```

class Solution:
    def isSameTree(self, p: TreeNode | None, q: TreeNode | None) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and
                self.isSameTree(p.left, q.left) and
                self.isSameTree(p.right, q.right))

```

4. 1001.py

Path: LeetCode-main/solutions/1001. Grid Illumination/1001.py

```

class Solution:
    def gridIllumination(
        self,
        n: int,
        lamps: list[list[int]],
        queries: list[list[int]],
    ) -> list[int]:
        ans = []
        rows = collections.Counter()
        cols = collections.Counter()
        diag1 = collections.Counter()
        diag2 = collections.Counter()
        lampsSet = set()

        for i, j in lamps:
            if (i, j) not in lampsSet:
                lampsSet.add((i, j))
                rows[i] += 1
                cols[j] += 1
                diag1[i + j] += 1
                diag2[i - j] += 1

        for i, j in queries:
            if rows[i] or cols[j] or diag1[i + j] or diag2[i - j]:
                ans.append(1)
            for y in range(max(0, i - 1), min(n, i + 2)):
                for x in range(max(0, j - 1), min(n, j + 2)):
                    if (y, x) in lampsSet:
                        lampsSet.remove((y, x))
                        rows[y] -= 1
                        cols[x] -= 1
                        diag1[y + x] -= 1
                        diag2[y - x] -= 1
            else:
                ans.append(0)

        return ans

```

5. 1002.py

Path: LeetCode-main/solutions/1002. Find Common Characters/1002.py

```

class Solution:
    def commonChars(self, words: list[str]) -> list[str]:
        return functools.reduce(lambda a, b: a & b,
                                map(collections.Counter, words)).elements()

```

6. 1003.py

Path: LeetCode-main/solutions/1003. Check If Word Is Valid After Substitutions/1003.py

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        for c in s:
            if c == 'c':
                if len(stack) < 2 or stack[-2] != 'a' or stack[-1] != 'b':
                    return False
                stack.pop()
                stack.pop()
            else:
                stack.append(c)

        return not stack
```

7. 1004.py

Path: LeetCode-main/solutions/1004. Max Consecutive Ones III/1004.py

```
class Solution:
    def longestOnes(self, nums: list[int], k: int) -> int:
        ans = 0

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                k -= 1
            while k < 0:
                if nums[l] == 0:
                    k += 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

8. 1005.py

Path: LeetCode-main/solutions/1005. Maximize Sum Of Array After K Negations/1005.py

```
class Solution:
    def largestSumAfterKNegations(self, nums: list[int], k: int) -> int:
        nums.sort()

        for i, num in enumerate(nums):
            if num > 0 or k == 0:
                break
            nums[i] = -num
            k -= 1

        return sum(nums) - (k % 2) * min(nums) * 2
```

9. 1006.py

Path: LeetCode-main/solutions/1006. Clumsy Factorial/1006.py

```
class Solution:
    def clumsy(self, n: int) -> int:
        if n == 1:
            return 1
        if n == 2:
            return 2
        if n == 3:
            return 6
        if n == 4:
            return 7
        if n % 4 == 1:
            return n + 2
        if n % 4 == 2:
            return n + 2
        if n % 4 == 3:
            return n - 1
        return n + 1
```

10. 1007.py

Path: LeetCode-main/solutions/1007. Minimum Domino Rotations For Equal Row/1007.py

```
class Solution:
    def minDominoRotations(self, tops: list[int], bottoms: list[int]) -> int:
        for num in range(1, 7):
            if all(num in pair for pair in zip(tops, bottoms)):
                return len(tops) - max(tops.count(num), bottoms.count(num))
        return -1
```

11. 1008.py

Path: LeetCode-main/solutions/1008. Construct Binary Search Tree from Preorder Traversal/1008.py

```
class Solution:
    def bstFromPreorder(self, preorder: list[int]) -> TreeNode | None:
        root = TreeNode(preorder[0])
        stack = [root]

        for i in range(1, len(preorder)):
            parent = stack[-1]
            child = TreeNode(preorder[i])
            # Adjust the parent.
            while stack and stack[-1].val < child.val:
                parent = stack.pop()
            # Create parent-child link according to BST property.
            if parent.val > child.val:
                parent.left = child
            else:
                parent.right = child
            stack.append(child)

        return root
```

12. 1009.py

Path: LeetCode-main/solutions/1009. Complement of Base 10 Integer/1009.py

```
class Solution:
    def bitwiseComplement(self, n: int) -> int:
        mask = 1
        while mask < n:
            mask = (mask << 1) + 1
        return mask ^ n
```

13. 101.py

Path: LeetCode-main/solutions/101. Symmetric Tree/101.py

```
class Solution:
    def isSymmetric(self, root: TreeNode | None) -> bool:
        def isSymmetric(p: TreeNode | None, q: TreeNode | None) -> bool:
            if not p or not q:
                return p == q
            return (p.val == q.val and
                    isSymmetric(p.left, q.right) and
                    isSymmetric(p.right, q.left))

        return isSymmetric(root, root)
```

14. 1010.py

Path: LeetCode-main/solutions/1010. Pairs of Songs With Total Durations Divisible by 60/1010.py

```
class Solution:
    def numPairsDivisibleBy60(self, time: list[int]) -> int:
        ans = 0
        count = [0] * 60

        for t in time:
            t %= 60
            ans += count[(60 - t) % 60]
            count[t] += 1

        return ans
```

15. 1011.py

Path: LeetCode-main/solutions/1011. Capacity To Ship Packages Within D Days/1011.py

```

class Solution:
    def shipWithinDays(self, weights: list[int], days: int) -> int:
        def shipDays(shipCapacity: int) -> int:
            shipDays = 1
            capacity = 0
            for weight in weights:
                if capacity + weight > shipCapacity:
                    shipDays += 1
                    capacity = weight
                else:
                    capacity += weight
            return shipDays

        l = max(weights)
        r = sum(weights)
        return bisect.bisect_left(range(l, r), True,
                                   key=lambda m: shipDays(m) <= days) + 1

```

16. 1012.py

Path: LeetCode-main/solutions/1012. Numbers With Repeated Digits/1012.py

```

class Solution:
    def numDupDigitsAtMostN(self, n: int) -> int:
        return n - self._countSpecialNumbers(n)

    # Same as 2376. Count Special Integers
    def _countSpecialNumbers(self, n: int) -> int:
        s = str(n)

        @functools.lru_cache(None)
        def dp(i: int, used: int, tight: bool) -> int:
            """
            Returns the number of special integers, considering the i-th digit, where
            `used` is the bitmask of the used digits, and `tight` indicates if the
            current digit is tightly bound.
            """
            if i == len(s):
                return 1

            res = 0
            maxDigit = int(s[i]) if tight else 9

            for d in range(maxDigit + 1):
                # `d` is used.
                if used >> d & 1:
                    continue
                # Use `d` now.
                nextTight = tight and (d == maxDigit)
                if used == 0 and d == 0: # Don't count leading 0s as used.
                    res += dp(i + 1, used, nextTight)
                else:
                    res += dp(i + 1, used | 1 << d, nextTight)

            return res

        return dp(0, 0, True) - 1 # - 0

```

17. 1013.py

Path: LeetCode-main/solutions/1013. Partition Array Into Three Parts With Equal Sum/1013.py

```
class Solution:
    def canThreePartsEqualSum(self, arr: list[int]) -> bool:
        summ = sum(arr)
        if summ % 3 != 0:
            return False

        average = summ // 3
        partCount = 0
        partSum = 0

        for a in arr:
            partSum += a
            if partSum == average:
                partCount += 1
                partSum = 0

        # edge case: arr = [0, 0, 0, 0] . partCount = 4.
        return partCount >= 3
```

18. 1014.py

Path: LeetCode-main/solutions/1014. Best Sightseeing Pair/1014.py

```
class Solution:
    def maxScoreSightseeingPair(self, values: list[int]) -> int:
        ans = 0
        bestPrev = 0

        for value in values:
            ans = max(ans, value + bestPrev)
            bestPrev = max(bestPrev, value) - 1

        return ans
```

19. 1015.py

Path: LeetCode-main/solutions/1015. Smallest Integer Divisible by K/1015.py

```
class Solution:
    def smallestRepunitDivByK(self, k: int) -> int:
        if k % 10 not in {1, 3, 7, 9}:
            return -1

        seen = set()
        n = 0

        for length in range(1, k + 1):
            n = (n * 10 + 1) % k
            if n == 0:
                return length
            if n in seen:
                return -1
            seen.add(n)

        return -1
```

20. 1016.py

Path: LeetCode-main/solutions/1016. Binary String With Substrings Representing 1 To N/1016.py

```
class Solution:
    def queryString(self, s: str, n: int) -> bool:
        if n > 1511:
            return False

        for i in range(n, n // 2, -1):
            if format(i, 'b') not in s:
                return False

        return True
```

21. 1017.py

Path: LeetCode-main/solutions/1017. Convert to Base -2/1017.py

```
class Solution:
    def baseNeg2(self, n: int) -> str:
        ans = []

        while n != 0:
            ans.append(str(n % 2))
            n = -(n >> 1)

        return ''.join(reversed(ans)) if ans else '0'
```

22. 1018.py

Path: LeetCode-main/solutions/1018. Binary Prefix Divisible By 5/1018.py

```
class Solution:
    def prefixesDivBy5(self, nums: list[int]) -> list[bool]:
        ans = []
        curr = 0

        for num in nums:
            curr = (curr * 2 + num) % 5
            ans.append(curr % 5 == 0)

        return ans
```

23. 1019.py

Path: LeetCode-main/solutions/1019. Next Greater Node In Linked List/1019.py


```
class Solution:
    def nextLargerNodes(self, head: ListNode) -> list[int]:
        ans = []
        stack = []

        while head:
            while stack and head.val > ans[stack[-1]]:
                index = stack.pop()
                ans[index] = head.val
            stack.append(len(ans))
            ans.append(head.val)
            head = head.next

        for i in stack:
            ans[i] = 0

        return ans
```

24. 102.py

Path: LeetCode-main/solutions/102. Binary Tree Level Order Traversal/102.py

```
class Solution:
    def levelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            currLevel = []
            for _ in range(len(q)):
                node = q.popleft()
                currLevel.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(currLevel)

        return ans
```

25. 1021.py

Path: LeetCode-main/solutions/1021. Remove Outermost Parentheses/1021.py

```

class Solution:
    def removeOuterParentheses(self, s: str) -> str:
        ans = []
        opened = 0

        for c in s:
            if c == '(':
                opened += 1
                if opened > 1:
                    ans.append(c)
            else: # c == ')'
                opened -= 1
                if opened > 0:
                    ans.append(c)

        return ''.join(ans)

```

26. 1022.py

Path: LeetCode-main/solutions/1022. Sum of Root To Leaf Binary Numbers/1022.py

```

class Solution:
    def sumRootToLeaf(self, root: TreeNode | None) -> int:
        ans = 0

        def dfs(root: TreeNode | None, val: int) -> None:
            nonlocal ans
            if not root:
                return
            val = val * 2 + root.val
            if not root.left and not root.right:
                ans += val
            dfs(root.left, val)
            dfs(root.right, val)

        dfs(root, 0)
        return ans

```

27. 1023.py

Path: LeetCode-main/solutions/1023. Camelcase Matching/1023.py

```

class Solution:
    def camelMatch(self, queries: list[str], pattern: str) -> list[bool]:
        def isMatch(query: str) -> bool:
            j = 0
            for c in query:
                if j < len(pattern) and c == pattern[j]:
                    j += 1
                elif c.isupper():
                    return False
            return j == len(pattern)

        return [isMatch(query) for query in queries]

```

28. 1024.py

Path: LeetCode-main/solutions/1024. Video Stitching/1024.py

```
class Solution:
    def videoStitching(self, clips: list[list[int]], time: int) -> int:
        ans = 0
        end = 0
        farthest = 0

        clips.sort()

        i = 0
        while farthest < time:
            while i < len(clips) and clips[i][0] <= end:
                farthest = max(farthest, clips[i][1])
                i += 1
            if end == farthest:
                return -1
            ans += 1
            end = farthest

        return ans
```

29. 1025.py

Path: LeetCode-main/solutions/1025. Divisor Game/1025.py

```
class Solution:
    def divisorGame(self, n: int) -> bool:
        return n % 2 == 0
```

30. 1027.py

Path: LeetCode-main/solutions/1027. Longest Arithmetic Subsequence/1027.py

```
class Solution:
    def longestArithSeqLength(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        # dp[i][k] := the length of the longest arithmetic subsequence of nums[0..i]
        # with k = diff + 500
        dp = [[0] * 1001 for _ in range(n)]

        for i in range(n):
            for j in range(i):
                k = nums[i] - nums[j] + 500
                dp[i][k] = max(2, dp[j][k] + 1)
            ans = max(ans, dp[i][k])

        return ans
```

31. 1028.py

Path: LeetCode-main/solutions/1028. Recover a Tree From Preorder Traversal/1028.py

```

class Solution:
    def recoverFromPreorder(self, traversal: str) -> TreeNode | None:
        i = 0

        def recoverFromPreorder(depth: int) -> TreeNode | None:
            nonlocal i
            nDashes = 0
            while i + nDashes < len(traversal) and traversal[i + nDashes] == '-':
                nDashes += 1
            if nDashes != depth:
                return None

            i += depth
            start = i
            while i < len(traversal) and traversal[i].isdigit():
                i += 1

            return TreeNode(int(traversal[start:i]),
                            recoverFromPreorder(depth + 1),
                            recoverFromPreorder(depth + 1))

        return recoverFromPreorder(0)

```

32. 1029.py

Path: LeetCode-main/solutions/1029. Two City Scheduling/1029.py

```

class Solution:
    def twoCitySchedCost(self, costs: list[list[int]]) -> int:
        n = len(costs) // 2

        # How much money can we save if we fly a person to A instead of B?
        # To save money, we should
        #   1. Fly the person with the maximum saving to A.
        #   2. Fly the person with the minimum saving to B.

        # Sort `costs` in ascending order by the money saved if we fly a person to
        # B instead of A.
        costs.sort(key=lambda x: x[0] - x[1])
        return sum(costs[i][0] + costs[i + n][1] for i in range(n))

```

33. 103-2.py

Path: LeetCode-main/solutions/103. Binary Tree Zigzag Level Order Traversal/103-2.py

```

class Solution:
    def zigzagLevelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])
        isLeftToRight = True

        while q:
            size = len(q)
            currLevel = [0] * size
            for i in range(size):
                node = q.popleft()
                index = i if isLeftToRight else size - i - 1
                currLevel[index] = node.val
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(currLevel)
            isLeftToRight = not isLeftToRight

        return ans

```

34. 103.py

Path: LeetCode-main/solutions/103. Binary Tree Zigzag Level Order Traversal/103.py

```

class Solution:
    def zigzagLevelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        dq = collections.deque([root])
        isLeftToRight = True

        while dq:
            currLevel = []
            for _ in range(len(dq)):
                if isLeftToRight:
                    node = dq.popleft()
                    currLevel.append(node.val)
                    if node.left:
                        dq.append(node.left)
                    if node.right:
                        dq.append(node.right)
                else:
                    node = dq.pop()
                    currLevel.append(node.val)
                    if node.right:
                        dq.appendleft(node.right)
                    if node.left:
                        dq.appendleft(node.left)
            ans.append(currLevel)
            isLeftToRight = not isLeftToRight

        return ans

```

35. 1030.py

Path: LeetCode-main/solutions/1030. Matrix Cells in Distance Order/1030.py

```
class Solution:
    def allCellsDistOrder(self, rows: int, cols: int, rCenter: int, cCenter: int) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        ans = []
        q = collections.deque([(rCenter, cCenter)])
        seen = {(rCenter, cCenter)}

        while q:
            i, j = q.popleft()
            ans.append([i, j])
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == rows or y < 0 or y == cols:
                    continue
                if (x, y) in seen:
                    continue
                seen.add((x, y))
                q.append((x, y))

        return ans
```

36. 1031.py

Path: LeetCode-main/solutions/1031. Maximum Sum of Two Non-Overlapping Subarrays/1031.py

```
class Solution:
    def maxSumTwoNoOverlap(
        self,
        nums: list[int],
        firstLen: int,
        secondLen: int,
    ) -> int:
        def helper(l: int, r: int) -> int:
            n = len(nums)
            left = [0] * n
            summ = 0

            for i in range(n):
                summ += nums[i]
                if i >= l:
                    summ -= nums[i - l]
                if i >= l - 1:
                    left[i] = max(left[i - 1], summ) if i > 0 else summ

            right = [0] * n
            summ = 0

            for i in reversed(range(n)):
                summ += nums[i]
                if i <= n - r - 1:
                    summ -= nums[i + r]
                if i <= n - r:
                    right[i] = max(right[i + 1], summ) if i < n - 1 else summ

            return max(left[i] + right[i + 1] for i in range(n - 1))

        return max(helper(firstLen, secondLen), helper(secondLen, firstLen))
```

37. 1032.py

Path: LeetCode-main/solutions/1032. Stream of Characters/1032.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class StreamChecker:
    def __init__(self, words: list[str]):
        self.root = TrieNode()
        self.letters = []
        for word in words:
            self._insert(word)

    def query(self, letter: str) -> bool:
        self.letters.append(letter)
        node = self.root
        for c in reversed(self.letters):
            if c not in node.children:
                return False
            node = node.children[c]
            if node.isWord:
                return True
        return False

    def _insert(self, word: str) -> None:
        node = self.root
        for c in reversed(word):
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

```

38. 1033.py

Path: LeetCode-main/solutions/1033. Moving Stones Until Consecutive/1033.py

```

class Solution:
    def numMovesStones(self, a: int, b: int, c: int) -> list[int]:
        nums = sorted([a, b, c])

        if nums[2] - nums[0] == 2:
            return [0, 0]
        return [1 if min(nums[1] - nums[0], nums[2] - nums[1]) <= 2 else 2,
                nums[2] - nums[0] - 2]

```

39. 1034.py

Path: LeetCode-main/solutions/1034. Coloring A Border/1034.py


```

class Solution:
    def colorBorder(
        self,
        grid: list[list[int]],
        r0: int,
        c0: int,
        color: int
    ) -> list[list[int]]:
        def dfs(i: int, j: int, startColor: int) -> None:
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return
            if grid[i][j] != startColor:
                return

            grid[i][j] = -startColor
            dfs(i + 1, j, startColor)
            dfs(i - 1, j, startColor)
            dfs(i, j + 1, startColor)
            dfs(i, j - 1, startColor)

            # If this cell is already on the boarder, it must be painted later.
            if i == 0 or i == len(grid) - 1 or j == 0 or j == len(grid[0]) - 1:
                return

            if (abs(grid[i + 1][j]) == startColor and
                abs(grid[i - 1][j]) == startColor and
                abs(grid[i][j + 1]) == startColor and
                abs(grid[i][j - 1]) == startColor):
                grid[i][j] = startColor

        dfs(r0, c0, grid[r0][c0])

        for i, row in enumerate(grid):
            for j, num in enumerate(row):
                if num < 0:
                    grid[i][j] = color

        return grid

```

40. 1035.py

Path: LeetCode-main/solutions/1035. Uncrossed Lines/1035.py

```

class Solution:
    def maxUncrossedLines(self, nums1: list[int], nums2: list[int]) -> int:
        m = len(nums1)
        n = len(nums2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                dp[i][j] = (dp[i - 1][j - 1] + 1
                            if nums1[i - 1] == nums2[j - 1]
                            else max(dp[i - 1][j], dp[i][j - 1]))

        return dp[m][n]

```

41. 1036.py

Path: LeetCode-main/solutions/1036. Escape a Large Maze/1036.py

```
class Solution:
    def isEscapePossible(
        self,
        blocked: list[list[int]],
        source: list[int],
        target: list[int]
    ) -> bool:
        def dfs(i: int, j: int, target: list[int], seen: set) -> bool:
            if i < 0 or i >= 10**6 or j < 0 or j >= 10**6:
                return False
            if (i, j) in blocked or (i, j) in seen:
                return False
            seen.add((i, j))
            return (len(seen) > (1 + 199) * 199 // 2 or [i, j] == target or
                    dfs(i + 1, j, target, seen) or
                    dfs(i - 1, j, target, seen) or
                    dfs(i, j + 1, target, seen) or
                    dfs(i, j - 1, target, seen))

        blocked = set(tuple(b) for b in blocked)
        return (dfs(source[0], source[1], target, set()) and
                dfs(target[0], target[1], source, set()))
```

42. 1037.py

Path: LeetCode-main/solutions/1037. Valid Boomerang/1037.py

```
class Solution:
    def isBoomerang(self, points: list[list[int]]) -> bool:
        return ((points[1][0] - points[0][0]) * (points[2][1] - points[1][1]) !=
                (points[1][1] - points[0][1]) * (points[2][0] - points[1][0]))
```

43. 1039.py

Path: LeetCode-main/solutions/1039. Minimum Score Triangulation of Polygon/1039.py

```
class Solution:
    def minScoreTriangulation(self, values: list[int]) -> int:
        n = len(values)
        dp = [[0] * n for _ in range(n)]

        for j in range(2, n):
            for i in range(j - 2, -1, -1):
                dp[i][j] = math.inf
                for k in range(i + 1, j):
                    dp[i][j] = min(dp[i][j], dp[i][k] + values[i]
                                    * values[k] * values[j] + dp[k][j])

        return dp[0][n - 1]
```

44. 104.py

Path: LeetCode-main/solutions/104. Maximum Depth of Binary Tree/104.py

```
class Solution:
    def maxDepth(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

45. 1040.py

Path: LeetCode-main/solutions/1040. Moving Stones Until Consecutive II/1040.py

```
class Solution:
    def numMovesStonesII(self, stones: list[int]) -> list[int]:
        n = len(stones)
        minMoves = n

        stones.sort()

        l = 0
        for r, stone in enumerate(stones):
            while stone - stones[l] + 1 > n:
                l += 1
            alreadyStored = r - l + 1
            if alreadyStored == n - 1 and stone - stones[l] + 1 == n - 1:
                minMoves = 2
            else:
                minMoves = min(minMoves, n - alreadyStored)

        return [minMoves, max(stones[n - 1] - stones[1] - n + 2, stones[n - 2] - stones[0] - n + 2)]
```

46. 1041.py

Path: LeetCode-main/solutions/1041. Robot Bounded In Circle/1041.py

```
class Solution:
    def isRobotBounded(self, instructions: str) -> bool:
        x = 0
        y = 0
        d = 0
        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]

        for instruction in instructions:
            if instruction == 'G':
                x += directions[d][0]
                y += directions[d][1]
            elif instruction == 'L':
                d = (d + 3) % 4
            else:
                d = (d + 1) % 4

        return (x, y) == (0, 0) or d > 0
```

47. 1042.py

Path: LeetCode-main/solutions/1042. Flower Planting With No Adjacent/1042.py

```

class Solution:
    def gardenNoAdj(self, n: int, paths: list[list[int]]) -> list[int]:
        ans = [0] * n # ans[i] := 1, 2, 3, or 4
        graph = [[] for _ in range(n)]

        for x, y in paths:
            u = x - 1
            v = y - 1
            graph[u].append(v)
            graph[v].append(u)

        for u in range(n):
            used = functools.reduce(operator.or_, (1 << ans[v] for v in graph[u]), 0)
            ans[u] = next(type_
                           for type_ in range(1, 5)
                           if not (used >> type_ & 1))

        return ans

```

48. 1043.py

Path: LeetCode-main/solutions/1043. Partition Array for Maximum Sum/1043.py

```

class Solution:
    def maxSumAfterPartitioning(self, arr: list[int], k: int) -> int:
        n = len(arr)
        dp = [0] * (n + 1)

        for i in range(1, n + 1):
            mx = -math.inf
            for j in range(1, min(i, k) + 1):
                mx = max(mx, arr[i - j])
                dp[i] = max(dp[i], dp[i - j] + mx * j)

        return dp[n]

```

49. 1044.py

Path: LeetCode-main/solutions/1044. Longest Duplicate Substring/1044.py

```

class Solution:
    def longestDupSubstring(self, s: str) -> str:
        BASE = 26
        HASH = 1_000_000_007
        bestStart = -1
        l = 1
        r = len(s)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        # k := the length of the substring to be hashed
        def getStart(k: int) -> int | None:
            maxPow = pow(BASE, k - 1, HASH)
            hashToStart = collections.defaultdict(list)
            h = 0

            # Compute the hash value of s[:k].
            for i in range(k):
                h = (h * BASE + val(s[i])) % HASH
            hashToStart[h].append(0)

            # Compute the rolling hash by Rabin Karp.
            for i in range(k, len(s)):
                startIndex = i - k + 1
                h = (h - maxPow * val(s[startIndex])) % HASH
                h = (h * BASE + val(s[i])) % HASH
                if h in hashToStart:
                    currSub = s[startIndex:startIndex + k]
                    for start in hashToStart[h]:
                        if s[start:start + k] == currSub:
                            return startIndex
                hashToStart[h].append(startIndex)

        while l < r:
            m = (l + r) // 2
            start: int | None = getStart(m)
            if start:
                bestStart = start
                l = m + 1
            else:
                r = m

        if bestStart == -1:
            return ''
        if getStart(1):
            return s[bestStart:bestStart + 1]
        return s[bestStart:bestStart + 1 - 1]

```

50. 1046.py

Path: LeetCode-main/solutions/1046. Last Stone Weight/1046.py

```

class Solution:
    def lastStoneWeight(self, stones: list[int]) -> int:
        pq = [-stone for stone in stones]
        heapq.heapify(pq)

        while len(pq) >= 2:
            n1 = -heapq.heappop(pq)
            n2 = -heapq.heappop(pq)
            if n1 != n2:
                heapq.heappush(pq, -(n1 - n2))

        return 0 if not pq else -pq[0]

```

51. 1048-2.py

Path: LeetCode-main/solutions/1048. Longest String Chain/1048-2.py

```

class Solution:
    def longestStrChain(self, words: list[str]) -> int:
        dp = {}

        for word in sorted(words, key=len):
            dp[word] = max(dp.get(word[:i] + word[i + 1:], 0) +
                          1 for i in range(len(word)))

        return max(dp.values())

```

52. 1048.py

Path: LeetCode-main/solutions/1048. Longest String Chain/1048.py

```

class Solution:
    def longestStrChain(self, words: list[str]) -> int:
        wordsSet = set(words)

        @functools.lru_cache(None)
        def dp(s: str) -> int:
            """Returns the longest chain where s is the last word."""
            ans = 1
            for i in range(len(s)):
                pred = s[:i] + s[i + 1:]
                if pred in wordsSet:
                    ans = max(ans, dp(pred) + 1)
            return ans

        return max(dp(word) for word in words)

```

53. 1049.py

Path: LeetCode-main/solutions/1049. Last Stone Weight II/1049.py

```

class Solution:
    def lastStoneWeightII(self, stones: list[int]) -> int:
        summ = sum(stones)
        s = 0
        dp = [True] + [False] * summ

        for stone in stones:
            for w in range(summ // 2 + 1)[::-1]:
                if w >= stone:
                    dp[w] = dp[w] or dp[w - stone]
                if dp[w]:
                    s = max(s, w)

        return summ - 2 * s

```

54. 105.py

Path: LeetCode-main/solutions/105. Construct Binary Tree from Preorder and Inorder Traversal/105.py

```

class Solution:
    def buildTree(
        self,
        preorder: list[int],
        inorder: list[int],
    ) -> TreeNode | None:
        inToIndex = {num: i for i, num in enumerate(inorder)}

        def build(
            preStart: int,
            preEnd: int,
            inStart: int,
            inEnd: int,
        ) -> TreeNode | None:
            if preStart > preEnd:
                return None

            rootVal = preorder[preStart]
            rootInIndex = inToIndex[rootVal]
            leftSize = rootInIndex - inStart

            root = TreeNode(rootVal)
            root.left = build(preStart + 1, preStart + leftSize,
                             inStart, rootInIndex - 1)
            root.right = build(preStart + leftSize + 1,
                              preEnd, rootInIndex + 1, inEnd)

            return root

        return build(0, len(preorder) - 1, 0, len(inorder) - 1)

```

55. 1051.py

Path: LeetCode-main/solutions/1051. Height Checker/1051.py

```
class Solution:
    def heightChecker(self, heights: list[int]) -> int:
        ans = 0
        currentHeight = 1
        count = [0] * 101

        for height in heights:
            count[height] += 1

        for height in heights:
            while count[currentHeight] == 0:
                currentHeight += 1
            if height != currentHeight:
                ans += 1
                count[currentHeight] -= 1

        return ans
```

56. 1052.py

Path: LeetCode-main/solutions/1052. Grumpy Bookstore Owner/1052.py

```
class Solution:
    def maxSatisfied(
        self,
        customers: list[int],
        grumpy: list[int],
        X: int,
    ) -> int:
        satisfied = sum(c for i, c in enumerate(customers) if grumpy[i] == 0)
        madeSatisfied = 0
        windowSatisfied = 0

        for i, customer in enumerate(customers):
            if grumpy[i] == 1:
                windowSatisfied += customer
            if i >= X and grumpy[i - X] == 1:
                windowSatisfied -= customers[i - X]
            madeSatisfied = max(madeSatisfied, windowSatisfied)

        return satisfied + madeSatisfied
```

57. 1053.py

Path: LeetCode-main/solutions/1053. Previous Permutation With One Swap/1053.py


```

class Solution:
    def prevPermOpt1(self, arr: list[int]) -> list[int]:
        n = len(arr)
        l = n - 2
        r = n - 1

        while l >= 0 and arr[l] <= arr[l + 1]:
            l -= 1
        if l < 0:
            return arr
        while arr[r] >= arr[l] or arr[r] == arr[r - 1]:
            r -= 1
        arr[l], arr[r] = arr[r], arr[l]

        return arr

```

58. 1054.py

Path: LeetCode-main/solutions/1054. Distant Barcodes/1054.py

```

class Solution:
    def rearrangeBarcodes(self, barcodes: list[int]) -> list[int]:
        ans = [0] * len(barcodes)
        count = collections.Counter(barcodes)
        i = 0 # ans' index
        maxNum = max(count, key=count.get)

        def fillAns(num: int) -> None:
            nonlocal i
            while count[num]:
                ans[i] = num
                i = i + 2 if i + 2 < len(barcodes) else 1
                count[num] -= 1

        fillAns(maxNum)
        for num in count.keys():
            fillAns(num)

        return ans

```

59. 1056.py

Path: LeetCode-main/solutions/1056. Confusing Number/1056.py

```

class Solution:
    def confusingNumber(self, n: int) -> bool:
        s = str(n)
        rotated = {'0': '0', '1': '1', '6': '9', '8': '8', '9': '6'}
        rotatedNum = []

        for c in s[::-1]:
            if c not in rotated:
                return False
            rotatedNum.append(rotated[c])

        return ''.join(rotatedNum) != s

```

60. 1057.py

Path: LeetCode-main/solutions/1057. Campus Bikes/1057.py

```
class Solution:
    def assignBikes(
        self,
        workers: list[list[int]],
        bikes: list[list[int]],
    ) -> list[int]:
        ans = [-1] * len(workers)
        usedBikes = [False] * len(bikes)
        # buckets[k] := (i, j), where k = dist(workers[i], bikes[j])
        buckets = [[] for _ in range(2001)]

        def dist(p1: list[int], p2: list[int]) -> int:
            return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

        for i, worker in enumerate(workers):
            for j, bike in enumerate(bikes):
                buckets[dist(worker, bike)].append((i, j))

        for k in range(2001):
            for i, j in buckets[k]:
                if ans[i] == -1 and not usedBikes[j]:
                    ans[i] = j
                    usedBikes[j] = True

        return ans
```

61. 1058.py

Path: LeetCode-main/solutions/1058. Minimize Rounding Error to Meet Target/1058.py

[illegible]

62. 106.py

Path: LeetCode-main/solutions/106. Construct Binary Tree from Inorder and Postorder Traversal/106.py

```
class Solution:
    def buildTree(
        self,
        inorder: list[int],
        postorder: list[int],
    ) -> TreeNode | None:
        inToIndex = {num: i for i, num in enumerate(inorder)}

        def build(
            inStart: int,
            inEnd: int,
            postStart: int,
            postEnd: int,
        ) -> TreeNode | None:
            if inStart > inEnd:
                return None

            rootVal = postorder[postEnd]
            rootInIndex = inToIndex[rootVal]
            leftSize = rootInIndex - inStart

            root = TreeNode(rootVal)
            root.left = build(inStart, rootInIndex - 1, postStart,
                             postStart + leftSize - 1)
            root.right = build(rootInIndex + 1, inEnd, postStart + leftSize,
                               postEnd - 1)

            return root

        return build(0, len(inorder) - 1, 0, len(postorder) - 1)
```

63. 1062.py

Path: LeetCode-main/solutions/1062. Longest Repeating Substring/1062.py

```
class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        ans = 0
        # dp[i][j] := the number of repeating characters of s[0..i) and s[0..j)
        dp = [[0] * (n + 1) for _ in range(n + 1)]

        for i in range(1, n + 1):
            for j in range(i + 1, n + 1):
                if s[i - 1] == s[j - 1]:
                    dp[i][j] = 1 + dp[i - 1][j - 1]
                    ans = max(ans, dp[i][j])

        return ans
```

64. 1063.py

Path: LeetCode-main/solutions/1063. Number of Valid Subarrays/1063.py

```

class Solution:
    def validSubarrays(self, nums: list[int]) -> int:
        # For each `num` in `nums`, each element x in the stack can be the leftmost
        # element s.t. [x, num] forms a valid subarray, so the size of the stack is
        # the number of valid subarrays ending in the current number.
        #
        # e.g. nums = [1, 3, 2]
        # num = 1, stack = [1] -> valid subarray is [1]
        # num = 3, stack = [1, 3] -> valid subarrays are [1, 3], [3]
        # num = 2, stack = [1, 2] -> valid subarrays are [1, 3, 2], [2]
        ans = 0
        stack = []

        for num in nums:
            while stack and stack[-1] > num:
                stack.pop()
            stack.append(num)
            ans += len(stack)

        return ans

```

65. 1064.py

Path: LeetCode-main/solutions/1064. Fixed Point/1064.py

```

class Solution:
    def fixedPoint(self, arr: list[int]) -> int:
        l = 0
        r = len(arr) - 1

        # Since arr[i] is strictly increasing, arr[i] - i will also be increasing.
        # Therefore, binary search `arr` for the first arr[i] - i = 0.
        while l < r:
            m = (l + r) // 2
            if arr[m] - m >= 0:
                r = m
            else:
                l = m + 1

        return l if arr[l] == l else -1

```

66. 1065.py

Path: LeetCode-main/solutions/1065. Index Pairs of a String/1065.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Solution:
    def indexPairs(self, text: str, words: list[str]) -> list[list[int]]:
        ans = []
        root = TrieNode()

        for word in words:
            node: TrieNode = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.isWord = True

        # Scan each text[i..j].
        for i in range(len(text)):
            node: TrieNode = root
            for j in range(i, len(text)):
                c = text[j]
                if c not in node.children:
                    break
                node = node.children[c]
                if node.isWord:
                    ans.append([i, j])

        return ans

```

67. 1066.py

Path: LeetCode-main/solutions/1066. Campus Bikes II/1066.py

```

class Solution:
    def assignBikes(
        self,
        workers: list[list[int]],
        bikes: list[list[int]],
    ) -> int:
        def dist(p1: list[int], p2: list[int]) -> int:
            return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

        @functools.lru_cache(None)
        def dp(workerIndex: int, used: int) -> int:
            """
            Returns the minimum Manhattan distances to assign bikes to
            workers[workerIndex..n), where `used` is the bitmask of the used bikes.
            """
            if workerIndex == len(workers):
                return 0
            return min(
                (dist(workers[workerIndex],
                    bike) + dp(workerIndex + 1, used | 1 << i) for i,
                    bike in enumerate(bikes) if not used >> i & 1),
                default=math.inf)

        return dp(0, 0)

```

68. 107.py

Path: LeetCode-main/solutions/107. Binary Tree Level Order Traversal II/107.py

```
class Solution:
    def levelOrderBottom(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            currLevel = []
            for _ in range(len(q)):
                node = q.popleft()
                currLevel.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(currLevel)

        return ans[::-1]
```

69. 1071-2.py

Path: LeetCode-main/solutions/1071. Greatest Common Divisor of Strings/1071-2.py

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        if str1 + str2 != str2 + str1:
            return ''
        g = math.gcd(len(str1), len(str2))
        return str1[:g]
```

70. 1071.py

Path: LeetCode-main/solutions/1071. Greatest Common Divisor of Strings/1071.py

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        for sz in range(min(len(str1), len(str2)), 0, -1):
            if self._isDivisible(str1, str2, sz):
                return str1[:sz]
        return ''

    def _isDivisible(self, str1: str, str2: str, sz: int) -> bool:
        """Returns True if str1 and str2 are divisible by str1[0..sz)."""
        if len(str1) % sz > 0 or len(str2) % sz > 0:
            return False
        gcd = str1[:sz]
        return str1.replace(gcd, '') == '' and str2.replace(gcd, '') == ''
```

71. 1072.py

Path: LeetCode-main/solutions/1072. Flip Columns For Maximum Number of Equal Rows/1072.py

```
class Solution:
    def maxEqualRowsAfterFlips(self, matrix: list[list[int]]) -> int:
        patterns = [tuple(a ^ row[0] for a in row) for row in matrix]
        return max(Counter(patterns).values())
```

72. 1073.py

Path: LeetCode-main/solutions/1073. Adding Two Negabinary Numbers/1073.py

```
class Solution:
    def addNegabinary(self, arr1: list[int], arr2: list[int]) -> list[int]:
        ans = []
        carry = 0

        while carry != 0 or arr1 or arr2:
            if arr1:
                carry += arr1.pop()
            if arr2:
                carry += arr2.pop()
            ans.append(carry & 1)
            carry = -(carry >> 1)

        while len(ans) > 1 and ans[-1] == 0:
            ans.pop()

        return ans[::-1]
```

73. 1074.py

Path: LeetCode-main/solutions/1074. Number of Submatrices That Sum to Target/1074.py

```
class Solution:
    def numSubmatrixSumTarget(self, matrix: list[list[int]], target: int) -> int:
        m = len(matrix)
        n = len(matrix[0])
        ans = 0

        # Transfer each row in the matrix to the prefix sum.
        for row in matrix:
            for i in range(1, n):
                row[i] += row[i - 1]

        for baseCol in range(n):
            for j in range(baseCol, n):
                prefixCount = collections.Counter({0: 1})
                summ = 0
                for i in range(m):
                    if baseCol > 0:
                        summ -= matrix[i][baseCol - 1]
                    summ += matrix[i][j]
                    ans += prefixCount[summ - target]
                    prefixCount[summ] += 1

        return ans
```

74. 1078.py

Path: LeetCode-main/solutions/1078. Occurrences After Bigram/1078.py

```
class Solution:
    def findOccurrences(self, text: str, first: str, second: str) -> list[str]:
        words = text.split()
        return [c for a, b, c in zip(words, words[1:], words[2:]) if a == first and b == second]
```

75. 1079.py

Path: LeetCode-main/solutions/1079. Letter Tile Possibilities/1079.py

```
class Solution:
    def numTilePossibilities(self, tiles: str) -> int:
        count = collections.Counter(tiles)

        def dfs(count: dict[int, int]) -> int:
            possibleSequences = 0

            for k, v in count.items():
                if v == 0:
                    continue
                # Put c in the current position. We only care about the number of possible
                # sequences of letters but don't care about the actual combination.
                count[k] -= 1
                possibleSequences += 1 + dfs(count)
                count[k] += 1

            return possibleSequences

        return dfs(count)
```

76. 108.py

Path: LeetCode-main/solutions/108. Convert Sorted Array to Binary Search Tree/108.py

```
class Solution:
    def sortedArrayToBST(self, nums: list[int]) -> TreeNode | None:
        def build(l: int, r: int) -> TreeNode | None:
            if l > r:
                return None
            m = (l + r) // 2
            return TreeNode(nums[m],
                            build(l, m - 1),
                            build(m + 1, r))

        return build(0, len(nums) - 1)
```

77. 1080.py

Path: LeetCode-main/solutions/1080. Insufficient Nodes in Root to Leaf Paths/1080.py


```

class Solution:
    def sufficientSubset(
        self,
        root: TreeNode | None,
        limit: int
    ) -> TreeNode | None:
        if not root:
            return None
        if not root.left and not root.right:
            return None if root.val < limit else root
        root.left = self.sufficientSubset(root.left, limit - root.val)
        root.right = self.sufficientSubset(root.right, limit - root.val)
        return None if not root.left and not root.right else root

```

78. 1081.py

Path: LeetCode-main/solutions/1081. Smallest Subsequence of Distinct Characters/1081.py

```

class Solution:
    def smallestSubsequence(self, text: str) -> str:
        ans = []
        count = collections.Counter(text)
        used = [False] * 26

        for c in text:
            count[c] -= 1
            if used[ord(c) - ord('a')]:
                continue
            while ans and ans[-1] > c and count[ans[-1]] > 0:
                used[ord(ans[-1]) - ord('a')] = False
                ans.pop()
            ans.append(c)
            used[ord(ans[-1]) - ord('a')] = True

        return ''.join(ans)

```

79. 1085.py

Path: LeetCode-main/solutions/1085. Sum of Digits in the Minimum Number/1085.py

```

class Solution:
    def sumOfDigits(self, nums: list[int]) -> int:
        return sum(int(d) for d in str(min(nums))) & 1 ^ 1

```

80. 1086.py

Path: LeetCode-main/solutions/1086. High Five/1086.py

```

class Solution:
    def highFive(self, items: list[list[int]]) -> list[list[int]]:
        idToScores = collections.defaultdict(list)

        for id, score in items:
            heapq.heappush(idToScores[id], score)
            if len(idToScores[id]) > 5:
                heapq.heappop(idToScores[id])

        return [[id, sum(scores) // 5] for id, scores in sorted(idToScores.items())]

```

81. 1087.py

Path: LeetCode-main/solutions/1087. Brace Expansion/1087.py

```

class Solution:
    def expand(self, s: str) -> list[str]:
        ans = []

        def dfs(i: int, path: list[str]) -> None:
            if i == len(s):
                ans.append(''.join(path))
                return
            if s[i] == '{':
                nextRightBraceIndex = s.find('}', i)
                for c in s[i + 1:nextRightBraceIndex].split(','):
                    path.append(c)
                    dfs(nextRightBraceIndex + 1, path)
                    path.pop()
            else: # s[i] != '{'
                path.append(s[i])
                dfs(i + 1, path)
                path.pop()

        dfs(0, [])
        return sorted(ans)

```

82. 1088.py

Path: LeetCode-main/solutions/1088. Confusing Number II/1088.py

```

class Solution:
    def confusingNumberII(self, n: int) -> int:
        digitToRotated = [(0, 0), (1, 1), (6, 9), (8, 8), (9, 6)]

        def dfs(num: int, rotatedNum: int, unit: int) -> int:
            ans = 0 if num == rotatedNum else 1
            # Add one more digit
            for digit, rotated in digitToRotated:
                if digit == 0 and num == 0:
                    continue
                nextNum = num * 10 + digit
                if nextNum > n:
                    break
                ans += dfs(nextNum, rotated * unit + rotatedNum, unit * 10)
            return ans

        return dfs(0, 0, 1)

```

83. 1089.py

Path: LeetCode-main/solutions/1089. Duplicate Zeros/1089.py

```
class Solution:
    def duplicateZeros(self, arr: list[int]) -> None:
        zeros = arr.count(0)
        i = len(arr) - 1
        j = len(arr) + zeros - 1

        while i < j:
            if j < len(arr):
                arr[j] = arr[i]
            if arr[i] == 0:
                j -= 1
            if j < len(arr):
                arr[j] = arr[i]
            i -= 1
            j -= 1
```

84. 109-2.py

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109-2.py

```
class Solution:
    def sortedListToBST(self, head: ListNode | None) -> TreeNode | None:
        arr = []

        # Construct the array.
        curr = head
        while curr:
            arr.append(curr.val)
            curr = curr.next

        def helper(l: int, r: int) -> TreeNode | None:
            if l > r:
                return None
            m = (l + r) // 2
            root = TreeNode(arr[m])
            root.left = helper(l, m - 1)
            root.right = helper(m + 1, r)
            return root

        return helper(0, len(arr) - 1)
```

85. 109-3.py

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109-3.py

```

class Solution:
    def sortedListToBST(self, head: ListNode | None) -> TreeNode | None:
        def helper(l: int, r: int) -> TreeNode | None:
            nonlocal head
            if l > r:
                return None

            m = (l + r) // 2

            # Simulate inorder traversal: recursively form the left half.
            left = helper(l, m - 1)

            # Once the left half is traversed, process the current node.
            root = TreeNode(head.val)
            root.left = left

            # Maintain the invariance.
            head = head.next

            # Simulate inorder traversal: recursively form the right half.
            root.right = helper(m + 1, r)
            return root

        return helper(0, self._getLength(head) - 1)

    def _getLength(self, head: ListNode | None) -> int:
        length = 0
        curr = head
        while curr:
            length += 1
            curr = curr.next
        return length

```

86. 109.py

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109.py

```

class Solution:
    def sortedListToBST(self, head: ListNode) -> TreeNode:
        def findMid(head: ListNode) -> ListNode:
            prev = None
            slow = head
            fast = head

            while fast and fast.next:
                prev = slow
                slow = slow.next
                fast = fast.next.next
            prev.next = None

            return slow

        if not head:
            return None
        if not head.next:
            return TreeNode(head.val)

        mid = findMid(head)
        root = TreeNode(mid.val)
        root.left = self.sortedListToBST(head)
        root.right = self.sortedListToBST(mid.next)
        return root

```

87. 1093.py

Path: LeetCode-main/solutions/1093. Statistics from a Large Sample/1093.py

```

class Solution:
    def sampleStats(self, count: list[int]) -> list[float]:
        minimum = next((i for i, num in enumerate(count) if num), None)
        maximum = next((i for i, num in reversed(
            list(enumerate(count))) if num), None)
        n = sum(count)
        mean = sum(i * c / n for i, c in enumerate(count))
        mode = count.index(max(count))

        numCount = 0
        leftMedian = 0
        for i, c in enumerate(count):
            numCount += c
            if numCount >= n / 2:
                leftMedian = i
                break

        numCount = 0
        rightMedian = 0
        for i, c in reversed(list(enumerate(count))):
            numCount += c
            if numCount >= n / 2:
                rightMedian = i
                break

        return [minimum, maximum, mean, (leftMedian + rightMedian) / 2, mode]

```

88. 1095.py

Path: LeetCode-main/solutions/1095. Find in Mountain Array/1095.py

```
# """
# This is MountainArray's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class MountainArray:
#   def get(self, index: int) -> int:
#   def length(self) -> int:

class Solution:
    def findInMountainArray(
        self,
        target: int,
        mountain_arr: 'MountainArray',
    ) -> int:
        n = mountain_arr.length()
        peakIndex = self.peakIndexInMountainArray(mountain_arr, 0, n - 1)

        leftIndex = self.searchLeft(mountain_arr, target, 0, peakIndex)
        if mountain_arr.get(leftIndex) == target:
            return leftIndex

        rightIndex = self.searchRight(mountain_arr, target, peakIndex + 1, n - 1)
        if mountain_arr.get(rightIndex) == target:
            return rightIndex

        return -1

# 852. Peak Index in a Mountain Array
def peakIndexInMountainArray(self, A: 'MountainArray', l: int, r: int) -> int:
    while l < r:
        m = (l + r) // 2
        if A.get(m) < A.get(m + 1):
            l = m + 1
        else:
            r = m
    return l

def searchLeft(self, A: 'MountainArray', target: int, l: int, r: int) -> int:
    while l < r:
        m = (l + r) // 2
        if A.get(m) < target:
            l = m + 1
        else:
            r = m
    return l

def searchRight(self, A: 'MountainArray', target: int, l: int, r: int) -> int:
    while l < r:
        m = (l + r) // 2
        if A.get(m) > target:
            l = m + 1
        else:
            r = m
    return l
```

89. 1096.py

Path: LeetCode-main/solutions/1096. Brace Expansion II/1096.py

```

class Solution:
    def braceExpansionII(self, expression: str) -> list[str]:
        def merge(groups: list[list[str]], group: list[str]) -> None:
            if not groups[-1]:
                groups[-1] = group
            return

            groups[-1] = [word1 + word2 for word1 in groups[-1]
                           for word2 in group]

        def dfs(s: int, e: int) -> list[str]:
            groups = [[]]
            layer = 0

            for i in range(s, e + 1):
                c = expression[i]
                if c == '{':
                    layer += 1
                    if layer == 1:
                        left = i + 1
                    elif c == '}':
                        layer -= 1
                        if layer == 0:
                            group = dfs(left, i - 1)
                            merge(groups, group)
                    elif c == ',' and layer == 0:
                        groups.append([])
                    elif layer == 0:
                        merge(groups, [c])

            return sorted(list({word for group in groups for word in group}))

        return dfs(0, len(expression) - 1)

```

90. 1099.py

Path: LeetCode-main/solutions/1099. Two Sum Less Than K/1099.py

```

class Solution:
    def twoSumLessThanK(self, nums: list[int], k: int) -> int:
        if len(nums) < 2:
            return -1

        ans = -1 # Note the constrathat nums[i] > 0.
        l = 0
        r = len(nums) - 1

        nums.sort()

        while l < r:
            if nums[l] + nums[r] < k:
                ans = max(ans, nums[l] + nums[r])
                l += 1
            else:
                r -= 1

        return ans

```

91. 11.py

Path: LeetCode-main/solutions/11. Container With Most Water/11.py

```
class Solution:
    def maxArea(self, height: list[int]) -> int:
        ans = 0
        l = 0
        r = len(height) - 1

        while l < r:
            minHeight = min(height[l], height[r])
            ans = max(ans, minHeight * (r - l))
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1

        return ans
```

92. 110-2.py

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110-2.py

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        ans = True

        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root and sets ans to false if root unbalanced."""
            nonlocal ans
            if not root or not ans:
                return 0
            left = maxDepth(root.left)
            right = maxDepth(root.right)
            if abs(left - right) > 1:
                ans = False
            return max(left, right) + 1

        maxDepth(root)
        return ans
```

93. 110-3.py

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110-3.py


```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root if root is balanced; otherwise, returns -1."""
            if not root:
                return 0

            left = maxDepth(root.left)
            if left == -1:
                return -1
            right = maxDepth(root.right)
            if right == -1:
                return -1
            if abs(left - right) > 1:
                return -1

            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return maxDepth(root) != -1
```

94. 110.py

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110.py

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        if not root:
            return True

        def maxDepth(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return (abs(maxDepth(root.left) - maxDepth(root.right)) <= 1 and
                self.isBalanced(root.left) and
                self.isBalanced(root.right))
```

95. 1100.py

Path: LeetCode-main/solutions/1100. Find K-Length Substrings With No Repeated Characters/1100.py

```
class Solution:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        ans = 0
        unique = 0
        count = collections.Counter()

        for i, c in enumerate(s):
            count[c] += 1
            if count[c] == 1:
                unique += 1
            if i >= k:
                count[s[i - k]] -= 1
                if count[s[i - k]] == 0:
                    unique -= 1
                if unique == k:
                    ans += 1

        return ans
```

96. 1101.py

Path: LeetCode-main/solutions/1101. The Earliest Moment When Everyone Become Friends/1101.py

```

class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def getCount(self) -> int:
        return self.count

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def earliestAcq(self, logs: list[list[int]], n: int) -> int:
        uf = UnionFind(n)

        # Sort `logs` by timestamp.
        logs.sort(key=lambda x: x[0])

        for timestamp, x, y in logs:
            uf.unionByRank(x, y)
            if uf.getCount() == 1:
                return timestamp

        return -1

```

97. 1103.py

Path: LeetCode-main/solutions/1103. Distribute Candies to People/1103.py

```

class Solution:
    def distributeCandies(self, candies: int, n: int) -> list[int]:
        ans = [0] * n
        rows = int((-n + (n**2 + 8 * n**2 * candies)**0.5) / (2 * n**2))
        accumN = rows * (rows - 1) * n // 2

        for i in range(n):
            ans[i] = accumN + rows * (i + 1)

        givenCandies = (n**2 * rows**2 + n * rows) // 2
        candies -= givenCandies
        lastGiven = rows * n
        i = 0

        while candies > 0:
            lastGiven += 1
            actualGiven = min(lastGiven, candies)
            candies -= actualGiven
            ans[i] += actualGiven
            i += 1

        return ans

```

98. 1104.py

Path: LeetCode-main/solutions/1104. Path In Zigzag Labelled Binary Tree/1104.py

```

class Solution:
    def pathInZigZagTree(self, label: int) -> list[int]:
        def boundarySum(level: int):
            return 2**level + 2**(level + 1) - 1

        ans = []

        for l in range(21):
            if 2**l > label:
                level = l - 1
                break

        if level % 2 == 1:
            label = boundarySum(level) - label

        for l in reversed(range(level + 1)):
            ans.append(label if l % 2 == 0 else boundarySum(l) - label)
            label //= 2

        return ans[::-1]

```

99. 1105.py

Path: LeetCode-main/solutions/1105. Filling Bookcase Shelves/1105.py

```

class Solution:
    def minHeightShelves(self, books: list[list[int]], shelfWidth: int) -> int:
        # dp[i] := the minimum height to place the first i books
        dp = [0] + [math.inf] * len(books)

        for i in range(len(books)):
            sumThickness = 0
            maxHeight = 0
            # Place books[j..i] on a new shelf.
            for j in range(i, -1, -1):
                thickness, height = books[j]
                sumThickness += thickness
                if sumThickness > shelfWidth:
                    break
                maxHeight = max(maxHeight, height)
                dp[i + 1] = min(dp[i + 1], dp[j] + maxHeight)

        return dp[-1]

```

100. 1106.py

Path: LeetCode-main/solutions/1106. Parsing A Boolean Expression/1106.py

```

class Solution:
    def parseBoolExpr(self, expression: str) -> bool:
        def dfs(s: int, e: int) -> list[str]:
            if s == e:
                return True if expression[s] == 't' else False

            exps = []
            layer = 0

            for i in range(s, e + 1):
                c = expression[i]
                if layer == 0 and c in '!&|':
                    op = c
                elif c == '(':
                    layer += 1
                    if layer == 1:
                        left = i + 1
                elif c == ')':
                    layer -= 1
                    if layer == 0:
                        exps.append(dfs(left, i - 1))
                elif c == ',' and layer == 1:
                    exps.append(dfs(left, i - 1))
                    left = i + 1

            if op == '|':
                return functools.reduce(operator.or_, exps)
            if op == '&':
                return functools.reduce(operator.and_, exps)
            if op == '!':
                return not exps[0]

            return dfs(0, len(expression) - 1)

```

101. 1108.py

Path: LeetCode-main/solutions/1108. Defanging an IP Address/1108.py

```
class Solution:
    def defangIPaddr(self, address: str) -> str:
        return address.replace('.', '[.]')
```

102. 1109.py

Path: LeetCode-main/solutions/1109. Corporate Flight Bookings/1109.py

```
class Solution:
    def corpFlightBookings(self, bookings: list[list[int]], n: int) -> list[int]:
        ans = [0] * n

        for booking in bookings:
            ans[booking[0] - 1] += booking[2]
            if booking[1] < n:
                ans[booking[1]] -= booking[2]

        for i in range(1, n):
            ans[i] += ans[i - 1]

        return ans
```

103. 111-2.py

Path: LeetCode-main/solutions/111. Minimum Depth of Binary Tree/111-2.py

```
class Solution:
    def minDepth(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        q = collections.deque([root])

        step = 1
        while q:
            for _ in range(len(q)):
                node = q.popleft()
                if not node.left and not node.right:
                    return step
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            step += 1
```

104. 111.py

Path: LeetCode-main/solutions/111. Minimum Depth of Binary Tree/111.py

```
class Solution:
    def minDepth(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        if not root.left:
            return self.minDepth(root.right) + 1
        if not root.right:
            return self.minDepth(root.left) + 1
        return min(self.minDepth(root.left), self.minDepth(root.right)) + 1
```

105. 1110.py

Path: LeetCode-main/solutions/1110. Delete Nodes And Return Forest/1110.py

```
class Solution:
    def delNodes(self, root: TreeNode, to_delete: list[int]) -> list[TreeNode]:
        ans = []
        toDeleteSet = set(to_delete)

        def dfs(root: TreeNode, isRoot: bool) -> TreeNode:
            if not root:
                return None

            deleted = root.val in toDeleteSet
            if isRoot and not deleted:
                ans.append(root)

            # If root is deleted, both children have the possibility to be a new root
            root.left = dfs(root.left, deleted)
            root.right = dfs(root.right, deleted)
            return None if deleted else root

        dfs(root, True)
        return ans
```

106. 1111.py

Path: LeetCode-main/solutions/1111. Maximum Nesting Depth of Two Valid Parentheses Strings/1111.py

```
class Solution:
    def maxDepthAfterSplit(self, seq: str) -> list[int]:
        ans = []
        depth = 1

        # Put all odd-depth parentheses in one group and even-depth parentheses in the other group.
        for c in seq:
            if c == '(':
                depth += 1
                ans.append(depth % 2)
            else:
                ans.append(depth % 2)
                depth -= 1

        return ans
```

107. 1114.py

Path: LeetCode-main/solutions/1114. Print in Order/1114.py

```
from threading import Lock

class Foo:
    def __init__(self):
        self.firstDone = Lock()
        self.secondDone = Lock()
        self.firstDone.acquire()
        self.secondDone.acquire()

    def first(self, printFirst: 'Callable[[], None]' ) -> None:
        printFirst()
        self.firstDone.release()

    def second(self, printSecond: 'Callable[[], None]' ) -> None:
        self.firstDone.acquire()
        printSecond()
        self.secondDone.release()

    def third(self, printThird: 'Callable[[], None]' ) -> None:
        self.secondDone.acquire()
        printThird()
```

108. 1115.py

Path: LeetCode-main/solutions/1115. Print FooBar Alternately/1115.py

```
from threading import Semaphore

class FooBar:
    def __init__(self, n):
        self.n = n
        self.fooSemaphore = Semaphore(1)
        self.barSemaphore = Semaphore(0)

    def foo(self, printFoo: 'Callable[[], None]' ) -> None:
        for _ in range(self.n):
            self.fooSemaphore.acquire()
            printFoo()
            self.barSemaphore.release()

    def bar(self, printBar: 'Callable[[], None]' ) -> None:
        for _ in range(self.n):
            self.barSemaphore.acquire()
            printBar()
            self.fooSemaphore.release()
```

109. 1116.py

Path: LeetCode-main/solutions/1116. Print Zero Even Odd/1116.py


```

from threading import Semaphore

class ZeroEvenOdd:
    def __init__(self, n):
        self.n = n
        self.zeroSemaphore = Semaphore(1)
        self.evenSemaphore = Semaphore(0)
        self.oddSemaphore = Semaphore(0)

    # printNumber(x) outputs "x", where x is an integer.
    def zero(self, printNumber: 'Callable[[int], None]') -> None:
        for i in range(self.n):
            self.zeroSemaphore.acquire()
            printNumber(0)
            (self.oddSemaphore if i & 2 == 0 else self.evenSemaphore).release()

    def even(self, printNumber: 'Callable[[int], None]') -> None:
        for i in range(2, self.n + 1, 2):
            self.evenSemaphore.acquire()
            printNumber(i)
            self.zeroSemaphore.release()

    def odd(self, printNumber: 'Callable[[int], None]') -> None:
        for i in range(1, self.n + 1, 2):
            self.oddSemaphore.acquire()
            printNumber(i)
            self.zeroSemaphore.release()

```

110. 1118.py

Path: LeetCode-main/solutions/1118. Number of Days in a Month/1118.py

```

class Solution:
    def numberOfDays(self, year: int, month: int) -> int:
        days = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

        def isLeapYear(year: int) -> bool:
            return year % 4 == 0 and year % 100 != 0 or year % 400 == 0
        return 29 if month == 2 and isLeapYear(year) else days[month]

```

111. 1119.py

Path: LeetCode-main/solutions/1119. Remove Vowels from a String/1119.py

```

class Solution:
    def removeVowels(self, s: str) -> str:
        return re.sub('a|e|i|o|u', '', s)

```

112. 112.py

Path: LeetCode-main/solutions/112. Path Sum/112.py

```

class Solution:
    def hasPathSum(self, root: TreeNode, summ: int) -> bool:
        if not root:
            return False
        if root.val == summ and not root.left and not root.right:
            return True
        return (self.hasPathSum(root.left, summ - root.val) or
                self.hasPathSum(root.right, summ - root.val))

```

113. 1120.py

Path: LeetCode-main/solutions/1120. Maximum Average Subtree/1120.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    summ: int
    count: int
    maxAverage: int

class Solution:
    def maximumAverageSubtree(self, root: TreeNode | None) -> float:
        def maximumAverage(root: TreeNode | None) -> T:
            if not root:
                return T(0, 0, 0)

            left = maximumAverage(root.left)
            right = maximumAverage(root.right)

            summ = root.val + left.summ + right.summ
            count = 1 + left.count + right.count
            maxAverage = max(summ / count, left.maxAverage, right.maxAverage)
            return T(summ, count, maxAverage)

        return maximumAverage(root).maxAverage

```

114. 1121.py

Path: LeetCode-main/solutions/1121. Divide Array Into Increasing Sequences/1121.py

```

class Solution:
    def canDivideIntoSubsequences(self, nums: list[int], k: int) -> bool:
        # Find the number with the maxFreq, we need at least maxFreq * k elements
        # e.g. nums = [1, 2, 2, 3, 4], we have maxFreq = 2 (two 2s), so we have to
        # Split nums into two subsequences say k = 3, the minimum length of nums is 2 x
        # 3 = 6, which is impossible if len(nums) = 5
        return len(nums) >= k * max(Counter(nums).values())

```

115. 1122.py

Path: LeetCode-main/solutions/1122. Relative Sort Array/1122.py

```

class Solution:
    def relativeSortArray(self, arr1: list[int], arr2: list[int]) -> list[int]:
        ans = []
        count = [0] * 1001

        for a in arr1:
            count[a] += 1

        for a in arr2:
            while count[a] > 0:
                ans.append(a)
                count[a] -= 1

        for num in range(1001):
            for _ in range(count[num]):
                ans.append(num)

        return ans

```

116. 1124.py

Path: LeetCode-main/solutions/1124. Longest Well-Performing Interval/1124.py

```

class Solution:
    def longestWPI(self, hours: list[int]) -> int:
        ans = 0
        prefix = 0
        dict = {}

        for i in range(len(hours)):
            prefix += 1 if hours[i] > 8 else -1
            if prefix > 0:
                ans = i + 1
            else:
                if prefix not in dict:
                    dict[prefix] = i
                if prefix - 1 in dict:
                    ans = max(ans, i - dict[prefix - 1])

        return ans

```

117. 1128.py

Path: LeetCode-main/solutions/1128. Number of Equivalent Domino Pairs/1128.py

```

class Solution:
    def numEquivDominoPairs(self, dominoes: list[list[int]]) -> int:
        ans = 0
        count = collections.Counter()

        for domino in dominoes:
            key = min(domino[0], domino[1]) * 10 + max(domino[0], domino[1])
            ans += count[key]
            count[key] += 1

        return ans

```

118. 1129.py

Path: LeetCode-main/solutions/1129. Shortest Path with Alternating Colors/1129.py

```
from enum import Enum

class Color(Enum):
    INIT = 0
    RED = 1
    BLUE = 2

class Solution:
    def shortestAlternatingPaths(
        self,
        n: int,
        redEdges: list[list[int]],
        blueEdges: list[list[int]],
    ) -> list[int]:
        ans = [-1] * n
        graph = [[] for _ in range(n)] # graph[u] := [(v, edgeColor)]
        q = collections.deque([(0, Color.INIT)]) # [(u, prevColor)]

        for u, v in redEdges:
            graph[u].append((v, Color.RED))

        for u, v in blueEdges:
            graph[u].append((v, Color.BLUE))

        step = 0
        while q:
            for _ in range(len(q)):
                u, prevColor = q.popleft()
                if ans[u] == -1:
                    ans[u] = step
                for i, (v, edgeColor) in enumerate(graph[u]):
                    if v == -1 or edgeColor == prevColor:
                        continue
                    q.append((v, edgeColor))
                    graph[u][i] = (-1, edgeColor) # Mark (u, v) as used.
                step += 1

        return ans
```

119. 113.py

Path: LeetCode-main/solutions/113. Path Sum II/113.py

```

class Solution:
    def pathSum(self, root: TreeNode, summ: int) -> list[list[int]]:
        ans = []

        def dfs(root: TreeNode, summ: int, path: list[int]) -> None:
            if not root:
                return
            if root.val == summ and not root.left and not root.right:
                ans.append(path + [root.val])
                return

            dfs(root.left, summ - root.val, path + [root.val])
            dfs(root.right, summ - root.val, path + [root.val])

        dfs(root, summ, [])
        return ans

```

120. 1130-2.py

Path: LeetCode-main/solutions/1130. Minimum Cost Tree From Leaf Values/1130-2.py

```

class Solution:
    def mctFromLeafValues(self, arr: list[int]) -> int:
        ans = 0
        stack = [math.inf]

        for a in arr:
            while stack and stack[-1] <= a:
                mid = stack.pop()
                # Multiply mid with next greater element in the array,
                # On the left (stack[-1]) or on the right (current number a)
                ans += min(stack[-1], a) * mid
            stack.append(a)

        return ans + sum(a * b for a, b in zip(stack[1:], stack[2:]))

```

121. 1130-3.py

Path: LeetCode-main/solutions/1130. Minimum Cost Tree From Leaf Values/1130-3.py

```

class Solution:
    def mctFromLeafValues(self, arr: list[int]) -> int:
        ans = 0

        while len(arr) > 1:
            i = arr.index(min(arr))
            ans += min(arr[i - 1:i] + arr[i + 1:i + 2]) * arr.pop(i)

        return ans

```

122. 1130.py

Path: LeetCode-main/solutions/1130. Minimum Cost Tree From Leaf Values/1130.py

```

class Solution:
    def mctFromLeafValues(self, arr: list[int]) -> int:
        n = len(arr)
        # dp[i][j] := the minimum cost of arr[i..j]
        dp = [[0] * n for _ in range(n)]
        # maxVal[i][j] := the maximum value of arr[i..j]
        maxVal = [[0] * n for _ in range(n)]

        for i in range(n):
            maxVal[i][i] = arr[i]

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                maxVal[i][j] = max(maxVal[i][j - 1], maxVal[i + 1][j])

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                dp[i][j] = math.inf
                for k in range(i, j):
                    dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j] +
                                    maxVal[i][k] * maxVal[k + 1][j])

        return dp[0][-1]

```

123. 1131.py

Path: LeetCode-main/solutions/1131. Maximum of Absolute Value Expression/1131.py

```

class Solution:
    def maxAbsValExpr(self, arr1: list[int], arr2: list[int]) -> int:
        n = len(arr1)
        a = [arr1[i] + arr2[i] + i for i in range(n)]
        b = [arr1[i] + arr2[i] - i for i in range(n)]
        c = [arr1[i] - arr2[i] + i for i in range(n)]
        d = [arr1[i] - arr2[i] - i for i in range(n)]
        return max(map(lambda x: max(x) - min(x), (a, b, c, d)))

```

124. 1133.py

Path: LeetCode-main/solutions/1133. Largest Unique Number/1133.py

```

class Solution:
    def largestUniqueNumber(self, nums: list[int]) -> int:
        return max([num for num, freq in collections.Counter(nums).items()
                    if freq == 1], default=-1)

```

125. 1134.py

Path: LeetCode-main/solutions/1134. Armstrong Number/1134.py

```
class Solution:
    def isArmstrong(self, n: int) -> bool:
        s = str(n)
        k = len(s)
        return sum(pow(int(c), k) for c in s) == n
```

126. 1135.py

Path: LeetCode-main/solutions/1135. Connecting Cities With Minimum Cost/1135.py

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def minimumCost(self, n: int, connections: list[list[int]]) -> int:
        ans = 0
        uf = UnionFind(n + 1)

        # Sort by cost.
        connections.sort(key=lambda x: x[2])

        for u, v, cost in connections:
            if uf.find(u) == uf.find(v):
                continue
            uf.unionByRank(u, v)
            ans += cost

        root = uf.find(1)
        if any(uf.find(i) != root for i in range(1, n + 1)):
            return -1

        return ans
```

127. 1136-2.py

Path: LeetCode-main/solutions/1136. Parallel Courses/1136-2.py

```
class Solution:
    def minimumSemesters(self, n: int, relations: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n

        # Build the graph.
        for u, v in relations:
            graph[u - 1].append(v - 1)
            inDegrees[v - 1] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        step = 0
        while q:
            for _ in range(len(q)):
                u = q.popleft()
                n -= 1
                for v in graph[u]:
                    inDegrees[v] -= 1
                    if inDegrees[v] == 0:
                        q.append(v)
            step += 1

        return step if n == 0 else -1
```

128. 1136.py

Path: LeetCode-main/solutions/1136. Parallel Courses/1136.py


```

from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def minimumSemesters(self, n: int, relations: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]
        states = [State.INIT] * n
        depth = [1] * n

        for u, v in relations:
            graph[u - 1].append(v - 1)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            for v in graph[u]:
                if hasCycle(v):
                    return True
            depth[u] = max(depth[u], 1 + depth[v])
            states[u] = State.VISITED
            return False

        if any(hasCycle(i) for i in range(n)):
            return -1
        return max(depth)

```

129. 1137-2.py

Path: LeetCode-main/solutions/1137. N-th Tribonacci Number/1137-2.py

```

class Solution:
    def tribonacci(self, n: int) -> int:
        dp = [0, 1, 1]

        for i in range(3, n + 1):
            dp[i % 3] = sum(dp)

        return dp[n % 3]

```

130. 1137.py

Path: LeetCode-main/solutions/1137. N-th Tribonacci Number/1137.py

```

class Solution:
    def tribonacci(self, n: int) -> int:
        if n < 2:
            return n

        dp = [0, 1, 1]

        for _ in range(3, n + 1):
            dp[0], dp[1], dp[2] = dp[1], dp[2], sum(dp)

        return dp[2]

```

131. 1139.py

Path: LeetCode-main/solutions/1139. Largest 1-Bordered Square/1139.py

```

class Solution:
    def largest1BorderedSquare(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        # leftOnes[i][j] := consecutive 1s in the left of grid[i][j]
        leftOnes = [[0] * n for _ in range(m)]
        # topOnes[i][j] := consecutive 1s in the top of grid[i][j]
        topOnes = [[0] * n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    leftOnes[i][j] = 1 if j == 0 else 1 + leftOnes[i][j - 1]
                    topOnes[i][j] = 1 if i == 0 else 1 + topOnes[i - 1][j]

        for sz in range(min(m, n), 0, -1):
            for i in range(m - sz + 1):
                for j in range(n - sz + 1):
                    x = i + sz - 1
                    y = j + sz - 1
                    # If grid[i..x][j..y] has all 1s on its border.
                    if min(
                        leftOnes[i][y],
                        leftOnes[x][y],
                        topOnes[x][j],
                        topOnes[x][y]) >= sz:
                        return sz * sz

        return 0

```

132. 114-2.py

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114-2.py

```
class Solution:
    def flatten(self, root: TreeNode | None) -> None:
        if not root:
            return

        stack = [root]

        while stack:
            root = stack.pop()
            if root.right:
                stack.append(root.right)
            if root.left:
                stack.append(root.left)
            if stack:
                root.right = stack[-1]
            root.left = None
```

133. 114-3.py

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114-3.py

```
class Solution:
    def flatten(self, root: TreeNode | None) -> None:
        if not root:
            return

        while root:
            if root.left:
                # Find the rightmost root
                rightmost = root.left
                while rightmost.right:
                    rightmost = rightmost.right
                # Rewire the connections
                rightmost.right = root.right
                root.right = root.left
                root.left = None
            # Move on to the right side of the tree
            root = root.right
```

134. 114.py

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114.py

```

class Solution:
    def flatten(self, root: TreeNode | None) -> None:
        if not root:
            return

        self.flatten(root.left)
        self.flatten(root.right)

        left = root.left # flattened left
        right = root.right # flattened right

        root.left = None
        root.right = left

        # Connect the original right subtree to the end of the new right subtree.
        rightmost = root
        while rightmost.right:
            rightmost = rightmost.right
        rightmost.right = right

```

135. 1143.py

Path: LeetCode-main/solutions/1143. Longest Common Subsequence/1143.py

```

class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m = len(text1)
        n = len(text2)
        # dp[i][j] := the length of LCS(text1[0..i], text2[0..j])
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                dp[i + 1][j + 1] = (1 + dp[i][j] if text1[i] == text2[j]
                                     else max(dp[i][j + 1], dp[i + 1][j]))

        return dp[m][n]

```

136. 1144.py

Path: LeetCode-main/solutions/1144. Decrease Elements To Make Array Zigzag/1144.py

```

class Solution:
    def movesToMakeZigzag(self, nums: list[int]) -> int:
        decreasing = [0] * 2

        for i, num in enumerate(nums):
            l = nums[i - 1] if i > 0 else 1001
            r = nums[i + 1] if i + 1 < len(nums) else 1001
            decreasing[i % 2] += max(0, num - min(l, r) + 1)

        return min(decreasing[0], decreasing[1])

```

137. 1146.py

Path: LeetCode-main/solutions/1146. Snapshot Array/1146.py

```

class SnapshotArray:
    def __init__(self, length: int):
        self.snaps = [[[0, 0]] for _ in range(length)]
        self.snap_id = 0

    def set(self, index: int, val: int) -> None:
        snap = self.snaps[index][-1]
        if snap[0] == self.snap_id:
            snap[1] = val
        else:
            self.snaps[index].append([self.snap_id, val])

    def snap(self) -> int:
        self.snap_id += 1
        return self.snap_id - 1

    def get(self, index: int, snap_id: int) -> int:
        i = bisect.bisect_left(self.snaps[index], [snap_id + 1]) - 1
        return self.snaps[index][i][1]

```

138. 115-2.py

Path: LeetCode-main/solutions/115. Distinct Subsequences/115-2.py

```

class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        dp = [1] + [0] * n

        for i in range(1, m + 1):
            for j in range(n, 1 - 1, -1):
                if s[i - 1] == t[j - 1]:
                    dp[j] += dp[j - 1]

        return dp[n]

```

139. 115.py

Path: LeetCode-main/solutions/115. Distinct Subsequences/115.py

```

class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m + 1):
            dp[i][0] = 1

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if s[i - 1] == t[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
                else:
                    dp[i][j] = dp[i - 1][j]

        return dp[m][n]

```

140. 1150.py

Path: LeetCode-main/solutions/1150. Check If a Number Is Majority Element in a Sorted Array/1150.py

```
class Solution:
    def isMajorityElement(self, nums: list[int], target: int) -> bool:
        n = len(nums)
        i = bisect.bisect_left(nums, target)
        return i + n // 2 < n and nums[i + n // 2] == target
```

141. 1151.py

Path: LeetCode-main/solutions/1151. Minimum Swaps to Group All 1's Together/1151.py

```
class Solution:
    def minSwaps(self, data: list[int]) -> int:
        k = data.count(1)
        ones = 0 # the number of ones in the window
        maxOnes = 0 # the maximum number of ones in the window

        for i, num in enumerate(data):
            if i >= k and data[i - k]:
                ones -= 1
            if num:
                ones += 1
            maxOnes = max(maxOnes, ones)

        return k - maxOnes
```

142. 1152.py

Path: LeetCode-main/solutions/1152. Analyze User Website Visit Pattern/1152.py

```
class Solution:
    def mostVisitedPattern(
        self,
        username: list[str],
        timestamp: list[int],
        website: list[str],
    ) -> list[str]:
        userToSites = collections.defaultdict(list)

        # Sort websites of each user by timestamp.
        for user, _, site in sorted(
            zip(username, timestamp, website),
            key=lambda x: x[1]):
            userToSites[user].append(site)

        # For each of three websites, count its frequency.
        patternCount = collections.Counter()

        for user, sites in userToSites.items():
            patternCount.update(Counter(set(itertools.combinations(sites, 3))))

        return max(sorted(patternCount), key=patternCount.get)
```

143. 1153.py

Path: LeetCode-main/solutions/1153. String Transforms Into Another String/1153.py

```
class Solution:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True

        mappings = {}

        for a, b in zip(str1, str2):
            if mappings.get(a, b) != b:
                return False
            mappings[a] = b

        # No letter in the str1 maps to > 1 letter in the str2 and there is at
        # least one temporary letter can break any loops.
        return len(set(str2)) < 26
```

144. 1154.py

Path: LeetCode-main/solutions/1154. Day of the Year/1154.py

```
class Solution:
    def dayOfYear(self, date: str) -> int:
        def isLeapYear(year: int) -> bool:
            return (year % 4 == 0 and year % 100 != 0) or year % 400 == 0

        year = int(date[:4])
        month = int(date[5:7])
        day = int(date[8:])
        days = [31, 29 if isLeapYear(year) else 28, 31, 30, 31, 31, 30, 31, 30, 31]

        return sum(days[:month - 1]) + day
```

145. 1155.py

Path: LeetCode-main/solutions/1155. Number of Dice Rolls With Target Sum/1155.py

```
class Solution:
    def numRollsToTarget(self, n: int, k: int, target: int) -> int:
        MOD = 1_000_000_007
        dp = [1] + [0] * target

        for _ in range(n): # n dices
            newDp = [0] * (target + 1)
            for i in range(1, k + 1): # numbers 1, 2, ..., k
                for t in range(i, target + 1): # all the possible targets
                    newDp[t] += dp[t - i]
                newDp[t] %= MOD
            dp = newDp

        return dp[target]
```

146. 1156.py

Path: LeetCode-main/solutions/1156. Swap For Longest Repeated Character Substring/1156.py

```
class Solution:
    def maxRepOpt1(self, text: str) -> int:
        count = collections.Counter(text)
        groups = [[c, len(list(group))]
                   for c, group in itertools.groupby(text)]
        ans = max(min(length + 1, count[c]) for c, length in groups)

        for i in range(1, len(groups) - 1):
            if groups[i - 1][0] == groups[i + 1][0] and groups[i][1] == 1:
                ans = max(
                    ans,
                    min(groups[i - 1][1] + groups[i + 1][1] + 1, count
                        [groups[i - 1][0]]))

        return ans
```

147. 1157.py

Path: LeetCode-main/solutions/1157. Online Majority Element In Subarray/1157.py

```
class MajorityChecker:
    def __init__(self, arr: list[int]):
        self.arr = arr
        self.TIMES = 20 # 2^TIMES >> |arr|
        self.numToIndices = collections.defaultdict(list)

        for i, a in enumerate(self.arr):
            self.numToIndices[a].append(i)

    def query(self, left: int, right: int, threshold: int) -> int:
        for _ in range(self.TIMES):
            randIndex = random.randint(left, right)
            num = self.arr[randIndex]
            indices = self.numToIndices[num]
            l = bisect.bisect_left(indices, left)
            r = bisect.bisect_right(indices, right)
            if r - l >= threshold:
                return num

        return -1
```

148. 116-2.py

Path: LeetCode-main/solutions/116. Populating Next Right Pointers in Each Node/116-2.py


```

class Solution:
    def connect(self, root: 'Node') -> 'Node':
        node = root # the node that is above the current needling

        while node and node.left:
            dummy = Node(0) # a dummy node before needling
            # Needle the children of the node.
            needle = dummy
            while node:
                needle.next = node.left
                needle = needle.next
                needle.next = node.right
                needle = needle.next
                node = node.next
            node = dummy.next # Move the node to the next level.

        return root

```

149. 116.py

Path: LeetCode-main/solutions/116. Populating Next Right Pointers in Each Node/116.py

```

class Solution:
    def connect(self, root: 'Node | None') -> 'Node | None':
        if not root:
            return None

        def connectTwoNodes(p, q) -> None:
            if not p:
                return
            p.next = q
            connectTwoNodes(p.left, p.right)
            connectTwoNodes(q.left, q.right)
            connectTwoNodes(p.right, q.left)

        connectTwoNodes(root.left, root.right)
        return root

```

150. 1160.py

Path: LeetCode-main/solutions/1160. Find Words That Can Be Formed by Characters/1160.py

```

class Solution:
    def countCharacters(self, words: list[str], chars: str) -> int:
        ans = 0
        count = collections.Counter(chars)

        for word in words:
            tempCount = count.copy()
            for c in word:
                tempCount[c] -= 1
                if tempCount[c] < 0:
                    ans -= len(word)
                    break
            ans += len(word)

        return ans

```

151. 1161-2.py

Path: LeetCode-main/solutions/1161. Maximum Level Sum of a Binary Tree/1161-2.py

```
class Solution:
    def maxLevelSum(self, root: TreeNode | None) -> int:
        # levelSums[i] := the sum of level (i + 1) (1-indexed)
        levelSums = []

        def dfs(root: TreeNode | None, level: int) -> None:
            if not root:
                return
            if len(levelSums) == level:
                levelSums.append(0)
            levelSums[level] += root.val
            dfs(root.left, level + 1)
            dfs(root.right, level + 1)

        dfs(root, 0)
        return 1 + levelSums.index(max(levelSums))
```

152. 1161.py

Path: LeetCode-main/solutions/1161. Maximum Level Sum of a Binary Tree/1161.py

```
class Solution:
    def maxLevelSum(self, root: TreeNode | None) -> int:
        ans = 0
        maxLevelSum = -math.inf
        q = collections.deque([root])

        level = 1
        while q:
            levelSum = 0
            for _ in range(len(q)):
                node = q.popleft()
                levelSum += node.val
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            if levelSum > maxLevelSum:
                maxLevelSum = levelSum
                ans = level
            level += 1

        return ans
```

153. 1162.py

Path: LeetCode-main/solutions/1162. As Far from Land as Possible/1162.py

```
class Solution:
    def maxDistance(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        q = collections.deque()
        water = 0

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 0:
                    water += 1
                else:
                    q.append((i, j))

        if water == 0 or water == m * n:
            return -1

        ans = 0
        d = 0

        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                ans = d
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if grid[x][y] > 0:
                        continue
                    q.append((x, y))
                    grid[x][y] = 2 # Mark as visited.
                d += 1

        return ans
```

154. 1163.py

Path: LeetCode-main/solutions/1163. Last Substring in Lexicographical Order/1163.py

```

class Solution:
    def lastSubstring(self, s: str) -> str:
        i = 0
        j = 1
        k = 0 # the number of the same letters of s[i..n) and s[j..n)

        while j + k < len(s):
            if s[i + k] == s[j + k]:
                k += 1
            elif s[i + k] > s[j + k]:
                # Skip s[j..j + k) and advance to s[j + k + 1] to find a possible
                # lexicographically larger substring since s[i..i + k) == s[j..j + k)
                # and s[i + k] > s[j + k).
                j = j + k + 1
                k = 0
            else:
                # Skip s[i..i + k) and advance to s[i + k + 1] or s[j] to find a
                # possible lexicographically larger substring since
                # s[i..i + k) == s[j..j + k) and s[i + k] < s[j + k).
                # Note that it's unnecessary to explore s[i + k + 1..j) if
                # i + k + 1 < j since they are already explored by j.
                i = max(i + k + 1, j)
                j = i + 1
                k = 0

        return s[i:]

```

155. 1165.py

Path: LeetCode-main/solutions/1165. Single-Row Keyboard/1165.py

```

class Solution:
    def calculateTime(self, keyboard: str, word: str) -> int:
        letterToIndex = {c: i for i, c in enumerate(keyboard)}
        return (letterToIndex[word[0]] +
                sum(abs(letterToIndex[a] - letterToIndex[b])
                    for a, b in itertools.pairwise(word)))

```

156. 1166.py

Path: LeetCode-main/solutions/1166. Design File System/1166.py

```

class TrieNode:
    def __init__(self, value: int = 0):
        self.children: dict[str, TrieNode] = {}
        self.value = value

class FileSystem:
    def __init__(self):
        self.root = TrieNode()

    def createPath(self, path: str, value: int) -> bool:
        node: TrieNode = self.root
        subpaths = path.split('/')

        for i in range(1, len(subpaths) - 1):
            if subpaths[i] not in node.children:
                return False
            node = node.children[subpaths[i]]

        if subpaths[-1] in node.children:
            return False
        node.children[subpaths[-1]] = TrieNode(value)
        return True

    def get(self, path: str) -> int:
        node: TrieNode = self.root
        for subpath in path.split('/')[1:]:
            if subpath not in node.children:
                return -1
            node = node.children[subpath]
        return node.value

```

157. 1167.py

Path: LeetCode-main/solutions/1167. Minimum Cost to Connect Sticks/1167.py

```

class Solution:
    def connectSticks(self, sticks: list[int]) -> int:
        ans = 0
        heapq.heapify(sticks)

        while len(sticks) > 1:
            x = heapq.heappop(sticks)
            y = heapq.heappop(sticks)
            ans += x + y
            heapq.heappush(sticks, x + y)

        return ans

```

158. 1168.py

Path: LeetCode-main/solutions/1168. Optimize Water Distribution in a Village/1168.py

```

class Solution:
    def minCostToSupplyWater(
        self,
        n: int,
        wells: list[int],
        pipes: list[list[int]],
    ) -> int:
        ans = 0
        graph = [[] for _ in range(n + 1)]
        minHeap = [] # (d, u)

        for u, v, w in pipes:
            graph[u].append((v, w))
            graph[v].append((u, w))

        # Connect virtual 0 with nodes 1 to n.
        for i, well in enumerate(wells):
            graph[0].append((i + 1, well))
            heapq.heappush(minHeap, (well, i + 1))

        mst = {0}

        while len(mst) < n + 1:
            d, u = heapq.heappop(minHeap)
            if u in mst:
                continue
            # Add the new vertex.
            mst.add(u)
            ans += d
            # Expand if possible.
            for v, w in graph[u]:
                if v not in mst:
                    heapq.heappush(minHeap, (w, v))

        return ans

```

159. 1169.py

Path: LeetCode-main/solutions/1169. Invalid Transactions/1169.py

```

class Solution:
    def invalidTransactions(self, transactions: list[str]) -> list[str]:
        ans = []
        nameToTrans = collections.defaultdict(list)

        for t in transactions:
            name, time, amount, city = t.split(',')
            time, amount = int(time), int(amount)
            nameToTrans[name].append({'time': time, 'city': city})

        for t in transactions:
            name, time, amount, city = t.split(',')
            time, amount = int(time), int(amount)
            if amount > 1000:
                ans.append(t)
            elif name in nameToTrans:
                for sameName in nameToTrans[name]:
                    if abs(sameName['time'] - time) <= 60 and sameName['city'] != city:
                        ans.append(t)
                        break

        return ans

```

160. 117.py

Path: LeetCode-main/solutions/117. Populating Next Right Pointers in Each Node II/117.py

```

class Solution:
    def connect(self, root: 'Node') -> 'Node':
        node = root # the node that is above the current needling

        while node:
            dummy = Node(0) # a dummy node before needling
            # Needle the children of the node.
            needle = dummy
            while node:
                if node.left: # Needle the left child.
                    needle.next = node.left
                    needle = needle.next
                if node.right: # Needle the right child.
                    needle.next = node.right
                    needle = needle.next
                node = node.next
            node = dummy.next # Move the node to the next level.

        return root

```

161. 1170.py

Path: LeetCode-main/solutions/1170. Compare Strings by Frequency of the Smallest Character/1170.py

```

class Solution:
    def numSmallerByFrequency(
        self,
        queries: list[str],
        words: list[str],
    ) -> list[int]:
        ans = []
        wordsFreq = sorted([word.count(min(word)) for word in words])

        for q in queries:
            count = q.count(min(q))
            index = bisect.bisect(wordsFreq, count)
            ans.append(len(words) - index)

        return ans

```

162. 1171.py

Path: LeetCode-main/solutions/1171. Remove Zero Sum Consecutive Nodes from Linked List/1171.py

```

class Solution:
    def removeZeroSumSublists(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        prefix = 0
        prefixToNode = {0: dummy}

        while head:
            prefix += head.val
            prefixToNode[prefix] = head
            head = head.next

        prefix = 0
        head = dummy

        while head:
            prefix += head.val
            head.next = prefixToNode[prefix].next
            head = head.next

        return dummy.next

```

163. 1172.py

Path: LeetCode-main/solutions/1172. Dinner Plate Stacks/1172.py


```

class DinnerPlates:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.stacks = []
        self.minHeap = [0] # the minimum indices of the stacks to push

    def push(self, val: int) -> None:
        index = self.minHeap[0]
        # Add a new stack on demand.
        if index == len(self.stacks):
            self.stacks.append([])
        # Push the new value.
        self.stacks[index].append(val)
        # If the stack pushed is full, remove its candidacy from `minHeap`.
        if len(self.stacks[index]) == self.capacity:
            heapq.heappop(self.minHeap)
        # If `minHeap` is empty, the next available stack index is |stacks|.
        if not self.minHeap:
            heapq.heappush(self.minHeap, len(self.stacks))

    def pop(self) -> int:
        # Remove empty stacks from the back.
        while self.stacks and not self.stacks[-1]:
            self.stacks.pop()
        if not self.stacks:
            return -1
        return self.popAtStack(len(self.stacks) - 1)

    def popAtStack(self, index: int) -> int:
        if index >= len(self.stacks) or not self.stacks[index]:
            return -1
        # If the stack is going to have space, add its candidacy to `minHeap`.
        if len(self.stacks[index]) == self.capacity:
            heapq.heappush(self.minHeap, index)
        return self.stacks[index].pop()

```

164. 1175.py

Path: LeetCode-main/solutions/1175. Prime Arrangements/1175.py

```

class Solution:
    def numPrimeArrangements(self, n: int) -> int:
        MOD = 1_000_000_007

        def factorial(n: int) -> int:
            fact = 1
            for i in range(2, n + 1):
                fact = fact * i % MOD
            return fact

        count = self._countPrimes(n)
        return factorial(count) * factorial(n - count) % MOD

    def _countPrimes(self, n: int) -> int:
        isPrime = [False] * 2 + [True] * (n - 1)
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n + 1, i):
                    isPrime[j] = False
        return sum(isPrime)

```

165. 1176.py

Path: LeetCode-main/solutions/1176. Diet Plan Performance/1176.py

```
class Solution:
    def dietPlanPerformance(
        self,
        calories: list[int],
        k: int,
        lower: int,
        upper: int,
    ) -> int:
        ans = 0
        summ = 0

        for i, calorie in enumerate(calories):
            summ += calorie
            if i < k - 1:
                continue
            if i >= k:
                summ -= calories[i - k]
            if summ < lower:
                ans -= 1
            elif summ > upper:
                ans += 1

        return ans
```

166. 1177.py

Path: LeetCode-main/solutions/1177. Can Make Palindrome from Substring/1177.py

```
class Solution:
    def canMakePaliQueries(self, s: str, queries: list[list[int]]) -> list[bool]:
        dp = [0] * (len(s) + 1)

        for i in range(1, len(s) + 1):
            dp[i] = dp[i - 1] ^ 1 << ord(s[i - 1]) - ord('a')

        return [
            (dp[right + 1] ^ dp[left]).bit_count() // 2 <= k
            for left, right, k in queries
        ]
```

167. 1178.py

Path: LeetCode-main/solutions/1178. Number of Valid Words for Each Puzzle/1178.py

```

class Solution:
    def findNumOfValidWords(
        self,
        words: list[str],
        puzzles: list[str],
    ) -> list[int]:
        ans = []
        binaryCount = collections.Counter()

        for word in words:
            mask = 0
            for c in word:
                mask |= 1 << ord(c) - ord('a')
            binaryCount[mask] += 1

        for puzzle in puzzles:
            valid = 0
            n = len(puzzle) - 1
            for i in range(1 << n):
                mask = 1 << ord(puzzle[0]) - ord('a')
                for j in range(n):
                    if i >> j & 1:
                        mask |= 1 << ord(puzzle[j + 1]) - ord('a')
                if mask in binaryCount:
                    valid += binaryCount[mask]
            ans.append(valid)

        return ans

```

168. 118.py

Path: LeetCode-main/solutions/118. Pascal's Triangle/118.py

```

class Solution:
    def generate(self, numRows: int) -> list[list[int]]:
        ans = []

        for i in range(numRows):
            ans.append([1] * (i + 1))

        for i in range(2, numRows):
            for j in range(1, len(ans[i]) - 1):
                ans[i][j] = ans[i - 1][j - 1] + ans[i - 1][j]

        return ans

```

169. 1180.py

Path: LeetCode-main/solutions/1180. Count Substrings with Only One Distinct Letter/1180.py

```

class Solution:
    def countLetters(self, s: str) -> int:
        ans = 0
        dp = 0 # the length of the running letter
        letter = '@' # the running letter

        for c in s:
            if c == letter:
                dp += 1
            else:
                dp = 1
                letter = c
            ans += dp # Add the number of substrings ending in the current letter.

        return ans

```

170. 1182.py

Path: LeetCode-main/solutions/1182. Shortest Distance to Target Color/1182.py

```

class Solution:
    def shortestDistanceColor(
        self,
        colors: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        NUM_COLOR = 3
        n = len(colors)
        ans = []
        # left[i][c] := the closest index of color c in index i to the left
        left = [[0] * (NUM_COLOR + 1) for _ in range(n)]
        # right[i][c] := the closest index of color c in index i to the right
        right = [[0] * (NUM_COLOR + 1) for _ in range(n)]

        colorToLatestIndex = [0, -1, -1, -1] # 0-indexed, -1 means N/A
        for i, color in enumerate(colors):
            colorToLatestIndex[color] = i
            for c in range(1, NUM_COLOR + 1):
                left[i][c] = colorToLatestIndex[c]

        colorToLatestIndex = [0, -1, -1, -1] # Reset.
        for i in range(n - 1, -1, -1):
            colorToLatestIndex[colors[i]] = i
            for c in range(1, NUM_COLOR + 1):
                right[i][c] = colorToLatestIndex[c]

        for i, c in queries:
            leftDist = math.inf if left[i][c] == -1 else i - left[i][c]
            rightDist = math.inf if right[i][c] == -1 else right[i][c] - i
            minDist = min(leftDist, rightDist)
            ans.append(-1 if minDist == math.inf else minDist)

        return ans

```

171. 1183-2.py

Path: LeetCode-main/solutions/1183. Maximum Number of Ones/1183-2.py

```
class Solution:
    def maximumNumberOfOnes(
        self,
        width: int,
        height: int,
        sideLength: int,
        maxOnes: int,
    ) -> int:
        subCount = []

        def getCount(length: int, index: int) -> int:
            return (length - index - 1) // sideLength + 1

        for i in range(sideLength):
            for j in range(sideLength):
                subCount.append(getCount(width, i) * getCount(height, j))

        return sum(sorted(subCount, reverse=True)[:maxOnes])
```

172. 1183.py

Path: LeetCode-main/solutions/1183. Maximum Number of Ones/1183.py

```
class Solution:
    def maximumNumberOfOnes(
        self,
        width: int,
        height: int,
        sideLength: int,
        maxOnes: int,
    ) -> int:
        submatrix = [[0] * sideLength for _ in range(sideLength)]

        for i in range(width):
            for j in range(height):
                submatrix[i % sideLength][j % sideLength] += 1

        return sum(heapq.nlargest(maxOnes, [a for row in submatrix for a in row]))
```

173. 1184.py

Path: LeetCode-main/solutions/1184. Distance Between Bus Stops/1184.py

```

class Solution:
    def distanceBetweenBusStops(
        self,
        distance: list[int],
        start: int, destination: int,
    ) -> int:
        clockwise = 0
        counterclockwise = 0

        if start > destination:
            start, destination = destination, start

        for i, d in enumerate(distance):
            if i >= start and i < destination:
                clockwise += d
            else:
                counterclockwise += d

        return min(clockwise, counterclockwise)

```

174. 1185.py

Path: LeetCode-main/solutions/1185. Day of the Week/1185.py

```

class Solution:
    def dayOfTheWeek(self, day: int, month: int, year: int) -> str:
        def isLeapYear(year: int) -> bool:
            return (year % 4 == 0 and year % 100 != 0) or year % 400 == 0

        week = ["Sunday", "Monday", "Tuesday",
                "Wednesday", "Thursday", "Friday", "Saturday"]
        days = [31, 29 if isLeapYear(year) else 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
        count = 0

        for i in range(1971, year):
            count += 366 if i % 4 == 0 else 365
        for i in range(month - 1):
            count += days[i]
        count += day

        return week[(count + 4) % 7]

```

175. 1186-2.py

Path: LeetCode-main/solutions/1186. Maximum Subarray Sum with One Deletion/1186-2.py

```

class Solution:
    # Similar to 53. Maximum Subarray
    def maximumSum(self, arr: list[int]) -> int:
        ans = -math.inf
        zero = -math.inf # no deletion
        one = -math.inf # <= 1 deletion

        for a in arr:
            one = max(a, one + a, zero)
            zero = max(a, zero + a)
            ans = max(ans, one)

        return ans

```

176. 1186.py

Path: LeetCode-main/solutions/1186. Maximum Subarray Sum with One Deletion/1186.py

```

class Solution:
    # Similar to 53. Maximum Subarray
    def maximumSum(self, arr: list[int]) -> int:
        # dp[0][i] := the maximum sum subarray ending in i (no deletion)
        # dp[1][i] := the maximum sum subarray ending in i (at most 1 deletion)
        dp = [[0] * len(arr) for _ in range(2)]

        dp[0][0] = arr[0]
        dp[1][0] = arr[0]
        for i in range(1, len(arr)):
            dp[0][i] = max(arr[i], dp[0][i - 1] + arr[i])
            dp[1][i] = max(arr[i], dp[1][i - 1] + arr[i], dp[0][i - 1])

        return max(dp[1])

```

177. 1187.py

Path: LeetCode-main/solutions/1187. Make Array Strictly Increasing/1187.py

```

class Solution:
    def makeArrayIncreasing(self, arr1: list[int], arr2: list[int]) -> int:
        # dp[i] := the minimum steps to reach i at previous round
        dp = {-1: 0}

        arr2.sort()

        for a in arr1:
            newDp = collections.defaultdict(lambda: math.inf)
            for val, steps in dp.items():
                # It's possible to use the value in the arr1.
                if a > val:
                    newDp[a] = min(newDp[a], steps)
                # Also try the value in the arr2.
                i = bisect_right(arr2, val)
                if i < len(arr2):
                    newDp[arr2[i]] = min(newDp[arr2[i]], steps + 1)
            if not newDp:
                return -1
            dp = newDp

        return min(dp.values())

```

178. 1188.py

Path: LeetCode-main/solutions/1188. Design Bounded Blocking Queue/1188.py

```
from threading import Semaphore

class BoundedBlockingQueue:
    def __init__(self, capacity: int):
        self.q = collections.deque()
        self.enqueueSemaphore = Semaphore(capacity)
        self.dequeueSemaphore = Semaphore(0)

    def enqueue(self, element: int) -> None:
        self.enqueueSemaphore.acquire()
        self.q.append(element)
        self.dequeueSemaphore.release()

    def dequeue(self) -> int:
        self.dequeueSemaphore.acquire()
        element = self.q.popleft()
        self.enqueueSemaphore.release()
        return element

    def size(self) -> int:
        return len(self.q)
```

179. 1189.py

Path: LeetCode-main/solutions/1189. Maximum Number of Balloons/1189.py

```
class Solution:
    def maxNumberOfBalloons(self, text: str) -> int:
        count = collections.Counter(text)
        return min(
            count['b'],
            count['a'],
            count['l'] // 2, count['o'] // 2, count['n'])
```

180. 119.py

Path: LeetCode-main/solutions/119. Pascal's Triangle II/119.py

```
class Solution:
    def getRow(self, rowIndex: int) -> list[int]:
        ans = [1] * (rowIndex + 1)

        for i in range(2, rowIndex + 1):
            for j in range(1, i):
                ans[i - j] += ans[i - j - 1]

        return ans
```

181. 1190-2.py

Path: LeetCode-main/solutions/1190. Reverse Substrings Between Each Pair of Parentheses/1190-2.py

```
class Solution:
    def reverseParentheses(self, s: str) -> str:
        ans = []
        stack = []
        pair = {}

        for i, c in enumerate(s):
            if c == '(':
                stack.append(i)
            elif c == ')':
                j = stack.pop()
                pair[i] = j
                pair[j] = i

        i = 0
        d = 1
        while i < len(s):
            if s[i] in '()':
                i = pair[i]
                d = -d
            else:
                ans.append(s[i])
                i += d

        return ''.join(ans)
```

182. 1190.py

Path: LeetCode-main/solutions/1190. Reverse Substrings Between Each Pair of Parentheses/1190.py

```
class Solution:
    def reverseParentheses(self, s: str) -> str:
        stack = []
        ans = []

        for c in s:
            if c == '(':
                stack.append(len(ans))
            elif c == ')':
                # Reverse the corresponding substring between ().
                j = stack.pop()
                ans[j:] = ans[j:][::-1]
            else:
                ans.append(c)

        return ''.join(ans)
```

183. 1191.py

Path: LeetCode-main/solutions/1191. K-Concatenation Maximum Sum/1191.py

```
class Solution:
    def kConcatenationMaxSum(self, arr: list[int], k: int) -> int:
        MOD = 1_000_000_007
        sz = len(arr) * (1 if k == 1 else 2)
        summ = sum(arr)
        # The concatenated array will be [arr1, arr2, ..., arrk].
        # If sum(arr) > 0 and k > 2, then arr2, ..., arr(k - 1) should be included.
        # Equivalently, maxSubarraySum is from arr1 and arrk.
        if summ > 0 and k > 2:
            return (self._kadane(arr, sz) + summ * (k - 2)) % MOD
        return self._kadane(arr, sz) % MOD

    def _kadane(self, arr: list[int], sz: int) -> int:
        ans = 0
        summ = 0
        for i in range(sz):
            a = arr[i % len(arr)]
            summ = max(a, summ + a)
            ans = max(ans, summ)
        return ans
```

184. 1195.py

Path: LeetCode-main/solutions/1195. Fizz Buzz Multithreaded/1195.py

```

from threading import Semaphore

class FizzBuzz:
    def __init__(self, n: int):
        self.n = n
        self.fizzSemaphore = Semaphore(0)
        self.buzzSemaphore = Semaphore(0)
        self.fizzbuzzSemaphore = Semaphore(0)
        self.numberSemaphore = Semaphore(1)

    # printFizz() outputs "fizz"
    def fizz(self, printFizz: 'Callable[[], None]') -> None:
        for i in range(1, self.n + 1):
            if i % 3 == 0 and i % 5 != 0:
                self.fizzSemaphore.acquire()
                printFizz()
                self.numberSemaphore.release()

    # printBuzz() outputs "buzz"
    def buzz(self, printBuzz: 'Callable[[], None]') -> None:
        for i in range(1, self.n + 1):
            if i % 5 == 0 and i % 3 != 0:
                self.buzzSemaphore.acquire()
                printBuzz()
                self.numberSemaphore.release()

    # printFizzBuzz() outputs "fizzbuzz"
    def fizzbuzz(self, printFizzBuzz: 'Callable[[], None]') -> None:
        for i in range(1, self.n + 1):
            if i % 15 == 0:
                self.fizzbuzzSemaphore.acquire()
                printFizzBuzz()
                self.numberSemaphore.release()

    # printNumber(x) outputs "x", where x is an integer.
    def number(self, printNumber: 'Callable[[int], None]') -> None:
        for i in range(1, self.n + 1):
            self.numberSemaphore.acquire()
            if i % 15 == 0:
                self.fizzbuzzSemaphore.release()
            elif i % 3 == 0:
                self.fizzSemaphore.release()
            elif i % 5 == 0:
                self.buzzSemaphore.release()
            else:
                printNumber(i)
                self.numberSemaphore.release()

```

185. 1196.py

Path: LeetCode-main/solutions/1196. How Many Apples Can You Put into the Basket/1196.py

```

class Solution:
    def maxNumberOfApples(self, weight: list[int]) -> int:
        summ = 0

        for i, w in enumerate(sorted(weight)):
            summ += w
            if summ > 5000:
                return i

        return len(weight)

```

186. 1198.py

Path: LeetCode-main/solutions/1198. Find Smallest Common Element in All Rows/1198.py

```

class Solution:
    def smallestCommonElement(self, mat: list[list[int]]) -> int:
        MAX = 10000
        count = [0] * (MAX + 1)

        for row in mat:
            for a in row:
                count[a] += 1
                if count[a] == len(mat):
                    return a

        return -1

```

187. 1199.py

Path: LeetCode-main/solutions/1199. Minimum Time to Build Blocks/1199.py

```

class Solution:
    def minBuildTime(self, blocks: list[int], split: int) -> int:
        minHeap = blocks.copy()
        heapq.heapify(minHeap)

        while len(minHeap) > 1:
            heapq.heappop(minHeap) # the minimum
            x = heapq.heappop(minHeap) # the second minimum
            heapq.heappush(minHeap, x + split)

        return minHeap[0]

```

188. 12-2.py

Path: LeetCode-main/solutions/12. Integer to Roman/12-2.py

```

class Solution:
    def intToRoman(self, num: int) -> str:
        M = ['', 'M', 'MM', 'MMM']
        C = ['', 'C', 'CC', 'CCC', 'CD', 'D', 'DC', 'DCC', 'DCCC', 'CM']
        X = ['', 'X', 'XX', 'XXX', 'XL', 'L', 'LX', 'LXX', 'LXXX', 'XC']
        I = ['', 'I', 'II', 'III', 'IV', 'V', 'VI', 'VII', 'VIII', 'IX']
        return M[num // 1000] + C[num % 1000 // 100] + X[num % 100 // 10] + I[num % 10]

```

189. 12.py

Path: LeetCode-main/solutions/12. Integer to Roman/12.py

```
class Solution:
    def intToRoman(self, num: int) -> str:
        valueSymbols = [(1000, 'M'), (900, 'CM'),
                        (500, 'D'), (400, 'CD'),
                        (100, 'C'), (90, 'XC'),
                        (50, 'L'), (40, 'XL'),
                        (10, 'X'), (9, 'IX'),
                        (5, 'V'), (4, 'IV'),
                        (1, 'I')]

        ans = []

        for value, symbol in valueSymbols:
            if num == 0:
                break
            count, num = divmod(num, value)
            ans.append(symbol * count)

        return ''.join(ans)
```

190. 120.py

Path: LeetCode-main/solutions/120. Triangle/120.py

```
class Solution:
    def minimumTotal(self, triangle: list[list[int]]) -> int:
        for i in range(len(triangle) - 2, -1, -1):
            for j in range(i + 1):
                triangle[i][j] += min(triangle[i + 1][j],
                                      triangle[i + 1][j + 1])

        return triangle[0][0]
```

191. 1200.py

Path: LeetCode-main/solutions/1200. Minimum Absolute Difference/1200.py

```
class Solution:
    def minimumAbsDifference(self, arr: list[int]) -> list[list[int]]:
        ans = []
        mn = math.inf

        arr.sort()

        for a, b in itertools.pairwise(arr):
            diff = b - a
            if diff < mn:
                mn = diff
                ans = []
            if diff == mn:
                ans.append([a, b])

        return ans
```

192. 1201.py

Path: LeetCode-main/solutions/1201. Ugly Number III/1201.py

```
class Solution:
    def nthUglyNumber(self, n: int, a: int, b: int, c: int) -> int:
        ab = a * b // math.gcd(a, b)
        ac = a * c // math.gcd(a, c)
        bc = b * c // math.gcd(b, c)
        abc = a * bc // math.gcd(a, bc)
        return bisect.bisect_left(range(2 * 10**9), n,
                                   key=lambda m: m // a + m // b + m // c
                                   - m // ab - m // ac - m // bc
                                   + m // abc)
```

193. 1202.py

Path: LeetCode-main/solutions/1202. Smallest String With Swaps/1202.py

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def smallestStringWithSwaps(self, s: str, pairs: list[list[int]]) -> str:
        uf = UnionFind(len(s))
        indexToLetters = collections.defaultdict(list)

        for a, b in pairs:
            uf.unionByRank(a, b)

        for i, c in enumerate(s):
            indexToLetters[uf.find(i)].append(c)

        for key in indexToLetters.keys():
            indexToLetters[key].sort(reverse=True)

        return ''.join(indexToLetters[uf.find(i)].pop()
                        for i in range(len(s)))
```

194. 1206.py

Path: LeetCode-main/solutions/1206. Design Skiplist/1206.py

```
from dataclasses import dataclass

@dataclass
class Node:
    val: int = -1
    next: 'Node' = None
    down: 'Node' = None

class Skiplist:
    def __init__(self):
        self.dummy = Node()

    def search(self, target: int) -> bool:
        node = self.dummy
        while node:
            while node.next and node.next.val < target:
                node = node.next
            if node.next and node.next.val == target:
                return True
            # Move to the next level
            node = node.down
        return False

    def add(self, num: int) -> None:
        # Collect nodes that are before the insertion point.
        nodes = []
        node = self.dummy
        while node:
            while node.next and node.next.val < num:
                node = node.next
            nodes.append(node)
            # Move to the next level
            node = node.down

        shouldInsert = True
        down = None
        while shouldInsert and nodes:
            node = nodes.pop()
            node.next = Node(num, node.next, down)
            down = node.next
            shouldInsert = random.getrandbits(1) == 0

        # Create a topmost new level dummy that points to the existing dummy.
        if shouldInsert:
            self.dummy = Node(-1, None, self.dummy)

    def erase(self, num: int) -> bool:
        node = self.dummy
        found = False
        while node:
            while node.next and node.next.val < num:
                node = node.next
            if node.next and node.next.val == num:
                # Delete the node
                node.next = node.next.next
                found = True
            # Move to the next level
            node = node.down
        return found
```

```
# Move to the node s.t. node.next.val >= target
def _advance(self, node: Node, target: int) -> None:
    while node.next and node.next.val < target:
        node = node.next
```

195. 1207.py

Path: LeetCode-main/solutions/1207. Unique Number of Occurrences/1207.py

```
class Solution:
    def uniqueOccurrences(self, arr: list[int]) -> bool:
        count = collections.Counter(arr)
        occurrences = set()

        for value in count.values():
            if value in occurrences:
                return False
            occurrences.add(value)

        return True
```

196. 1208.py

Path: LeetCode-main/solutions/1208. Get Equal Substrings Within Budget/1208.py

```
class Solution:
    def equalSubstring(self, s: str, t: str, maxCost: int) -> int:
        j = 0
        for i in range(len(s)):
            maxCost -= abs(ord(s[i]) - ord(t[i]))
            if maxCost < 0:
                maxCost += abs(ord(s[j]) - ord(t[j]))
                j += 1

        return len(s) - j
```

197. 1209.py

Path: LeetCode-main/solutions/1209. Remove All Adjacent Duplicates in String II/1209.py

```
class Solution:
    def removeDuplicates(self, s: str, k: int) -> str:
        stack = []

        for c in s:
            if not stack or stack[-1][0] != c:
                stack.append([c, 1])
            else: # stack[-1][0] == c
                stack[-1][1] += 1
                if stack[-1][1] == k:
                    stack.pop()

        return ''.join(c * count for c, count in stack)
```


198. 121.py

Path: LeetCode-main/solutions/121. Best Time to Buy and Sell Stock/121.py

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sellOne = 0
        holdOne = -math.inf

        for price in prices:
            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)

        return sellOne
```

199. 1210.py

Path: LeetCode-main/solutions/1210. Minimum Moves to Reach Target with Rotations/1210.py

```

from enum import IntEnum


class Pos(IntEnum):
    HORIZONTAL = 0
    VERTICAL = 1


class Solution:
    def minimumMoves(self, grid: list[list[int]]) -> int:
        n = len(grid)
        ans = 0
        # the state of (x, y, pos)
        # pos := 0 (horizontal) / 1 (vertical)
        q = collections.deque([(0, 0, Pos.HORIZONTAL)])
        seen = {(0, 0, Pos.HORIZONTAL)}

        def canMoveRight(x: int, y: int, pos: Pos) -> bool:
            if pos == Pos.HORIZONTAL:
                return y + 2 < n and not grid[x][y + 2]
            return y + 1 < n and not grid[x][y + 1] and not grid[x + 1][y + 1]

        def canMoveDown(x: int, y: int, pos: Pos) -> bool:
            if pos == Pos.VERTICAL:
                return x + 2 < n and not grid[x + 2][y]
            return x + 1 < n and not grid[x + 1][y] and not grid[x + 1][y + 1]

        def canRotateClockwise(x: int, y: int, pos: Pos) -> bool:
            return (pos == Pos.HORIZONTAL and x + 1 < n and
                    not grid[x + 1][y + 1] and not grid[x + 1][y])

        def canRotateCounterclockwise(x: int, y: int, pos: Pos) -> bool:
            return (pos == Pos.VERTICAL and y + 1 < n and
                    not grid[x + 1][y + 1] and not grid[x][y + 1])

        while q:
            for _ in range(len(q)):
                x, y, pos = q.popleft()
                if x == n - 1 and y == n - 2 and pos == Pos.HORIZONTAL:
                    return ans
                if canMoveRight(x, y, pos) and (x, y + 1, pos) not in seen:
                    q.append((x, y + 1, pos))
                    seen.add((x, y + 1, pos))
                if canMoveDown(x, y, pos) and (x + 1, y, pos) not in seen:
                    q.append((x + 1, y, pos))
                    seen.add((x + 1, y, pos))
                newPos = Pos.VERTICAL if pos == Pos.HORIZONTAL else Pos.HORIZONTAL
                if ((canRotateClockwise(x, y, pos) or
                     canRotateCounterclockwise(x, y, pos)) and
                    (x, y, newPos) not in seen):
                    q.append((x, y, newPos))
                    seen.add((x, y, newPos))
            ans += 1

        return -1

```

200. 1214.py

Path: LeetCode-main/solutions/1214. Two Sum BSTs/1214.py

```

class BSTIterator:
    def __init__(self, root: TreeNode | None, leftToRight: bool):
        self.stack = []
        self.leftToRight = leftToRight
        self._pushUntilNone(root)

    def hasNext(self) -> bool:
        return len(self.stack) > 0

    def next(self) -> int:
        node = self.stack.pop()
        if self.leftToRight:
            self._pushUntilNone(node.right)
        else:
            self._pushUntilNone(node.left)
        return node.val

    def _pushUntilNone(self, root: TreeNode | None):
        while root:
            self.stack.append(root)
            root = root.left if self.leftToRight else root.right

class Solution:
    def twoSumBSTs(
        self,
        root1: TreeNode | None,
        root2: TreeNode | None,
        target: int,
    ) -> bool:
        bst1 = BSTIterator(root1, True)
        bst2 = BSTIterator(root2, False)

        l = bst1.next()
        r = bst2.next()
        while True:
            summ = l + r
            if summ == target:
                return True
            if summ < target:
                if not bst1.hasNext():
                    return False
                l = bst1.next()
            else:
                if not bst2.hasNext():
                    return False
                r = bst2.next()

```

201. 1215-2.py

Path: LeetCode-main/solutions/1215. Stepping Numbers/1215-2.py

```

class Solution:
    def countSteppingNumbers(self, low: int, high: int) -> list[int]:
        ans = [0] if low == 0 else []

        def dfs(curr: int) -> None:
            if curr > high:
                return
            if curr >= low:
                ans.append(curr)

            lastDigit = curr % 10
            if lastDigit > 0:
                dfs(curr * 10 + lastDigit - 1)
            if lastDigit < 9:
                dfs(curr * 10 + lastDigit + 1)

        for i in range(1, 9 + 1):
            dfs(i)

        ans.sort()
        return ans

```

202. 1215.py

Path: LeetCode-main/solutions/1215. Stepping Numbers/1215.py

```

class Solution:
    def countSteppingNumbers(self, low: int, high: int) -> list[int]:
        ans = [0] if low == 0 else []
        q = collections.deque(list(range(1, 10)))

        while q:
            curr = q.popleft()
            if curr > high:
                continue
            if curr >= low:
                ans.append(curr)
            lastDigit = curr % 10
            if lastDigit > 0:
                q.append(curr * 10 + lastDigit - 1)
            if lastDigit < 9:
                q.append(curr * 10 + lastDigit + 1)

        return ans

```

203. 1216.py

Path: LeetCode-main/solutions/1216. Valid Palindrome III/1216.py

```

class Solution:
    def isValidPalindrome(self, s: str, k: int) -> bool:
        return len(s) - self._longestPalindromeSubseq(s) <= k

    # Same as 516. Longest Palindromic Subsequence
    def _longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        # dp[i][j] := the length of LPS(s[i..j])
        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                if s[i] == s[j]:
                    dp[i][j] = 2 + dp[i + 1][j - 1]
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

        return dp[0][n - 1]

```

204. 1217.py

Path: LeetCode-main/solutions/1217. Play with Chips/1217.py

```

class Solution:
    def minCostToMoveChips(self, position: list[int]) -> int:
        count = [0, 0]
        for p in position:
            count[p % 2] += 1
        return min(count[0], count[1])

```

205. 1218.py

Path: LeetCode-main/solutions/1218. Longest Arithmetic Subsequence of Given Difference/1218.py

```

class Solution:
    def longestSubsequence(self, arr: list[int], difference: int) -> int:
        ans = 0
        lengthAt = {}

        for a in arr:
            lengthAt[a] = lengthAt.get(a - difference, 0) + 1
            ans = max(ans, lengthAt[a])

        return ans

```

206. 1219.py

Path: LeetCode-main/solutions/1219. Path with Maximum Gold/1219.py

```

class Solution:
    def getMaximumGold(self, grid: list[list[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if i < 0 or j < 0 or i == len(grid) or j == len(grid[0]):
                return 0
            if grid[i][j] == 0:
                return 0

            gold = grid[i][j]
            grid[i][j] = 0 # Mark as visited.
            maxPath = max(dfs(i + 1, j), dfs(i - 1, j),
                          dfs(i, j + 1), dfs(i, j - 1))
            grid[i][j] = gold
            return gold + maxPath

        return max(dfs(i, j)
                   for i in range(len(grid))
                   for j in range(len(grid[0])))

```

207. 122.py

Path: LeetCode-main/solutions/122. Best Time to Buy and Sell Stock II/122.py

```

class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sell = 0
        hold = -math.inf

        for price in prices:
            sell = max(sell, hold + price)
            hold = max(hold, sell - price)

        return sell

```

208. 1220.py

Path: LeetCode-main/solutions/1220. Count Vowels Permutation/1220.py

```

class Solution:
    def countVowelPermutation(self, n: int) -> int:
        MOD = 1_000_000_007
        dp = {'a': 1, 'e': 1, 'i': 1, 'o': 1, 'u': 1}

        for _ in range(n - 1):
            newDp = {'a': dp['e'] + dp['i'] + dp['u'],
                     'e': dp['a'] + dp['i'],
                     'i': dp['e'] + dp['o'],
                     'o': dp['i'],
                     'u': dp['i'] + dp['o']}
            dp = newDp

        return sum(dp.values()) % MOD

```

209. 1222.py

Path: LeetCode-main/solutions/1222. Queens That Can Attack the King/1222.py

```

class Solution:
    def queensAttacktheKing(self, queens: list[list[int]],
                           king: list[int]) -> list[list[int]]:

        ans = []
        queens = {(i, j) for i, j in queens}

        for d in [
            [-1, -1],
            [-1, 0],
            [-1, 1],
            [0, -1],
            [0, 1],
            [1, -1],
            [1, 0],
            [1, 1]]:
            i = king[0] + d[0]
            j = king[1] + d[1]
            while 0 <= i < 8 and 0 <= j < 8:
                if (i, j) in queens:
                    ans.append([i, j])
                    break
                i += d[0]
                j += d[1]

        return ans

```

210. 1223.py

Path: LeetCode-main/solutions/1223. Dice Roll Simulation/1223.py

```

class Solution:
    def dieSimulator(self, n: int, rollMax: list[int]) -> int:
        MAX_ROLLS = 15
        MOD = 1_000_000_007

        dp = [[[0] * (MAX_ROLLS + 1) for j in range(6)] for i in range(n + 1)]

        for num in range(6):
            dp[1][num][1] = 1

        for i in range(2, n + 1):
            for currNum in range(6):
                for prevNum in range(6):
                    for k in range(1, 15 + 1):
                        if prevNum != currNum:
                            dp[i][currNum][1] = (
                                dp[i][currNum][1] + dp[i - 1][prevNum][k]) % MOD
                        elif k < rollMax[currNum]:
                            dp[i][currNum][k + 1] = dp[i - 1][currNum][k]

        ans = 0

        for num in range(6):
            for k in range(1, 15 + 1):
                ans += dp[n][num][k]

        return ans % MOD

```

211. 1224.py

Path: LeetCode-main/solutions/1224. Maximum Equal Frequency/1224.py

```
class Solution:
    def maxEqualFreq(self, nums: list[int]) -> int:
        ans = 0
        maxFreq = 0
        count = collections.Counter()
        freq = collections.Counter()

        for i, num in enumerate(nums):
            freq[count[num]] -= 1
            count[num] += 1
            freq[count[num]] += 1
            maxFreq = max(maxFreq, count[num])
            if maxFreq == 1 or maxFreq * freq[maxFreq] == i or (maxFreq - 1) * (
                freq[maxFreq - 1] + 1) == i:
                ans = i + 1

        return ans
```

212. 1227.py

Path: LeetCode-main/solutions/1227. Airplane Seat Assignment Probability/1227.py

```
class Solution:
    def nthPersonGetsNthSeat(self, n: int) -> float:
        return 1 if n == 1 else 0.5
```

213. 1228.py

Path: LeetCode-main/solutions/1228. Missing Number In Arithmetic Progression/1228.py

```
class Solution:
    def missingNumber(self, arr: list[int]) -> int:
        n = len(arr)
        delta = (arr[-1] - arr[0]) // n
        l = 0
        r = n - 1

        while l < r:
            m = (l + r) // 2
            if arr[m] == arr[0] + m * delta:
                l = m + 1
            else:
                r = m

        return arr[0] + l * delta
```

214. 1229.py

Path: LeetCode-main/solutions/1229. Meeting Scheduler/1229.py


```

class Solution:
    def minAvailableDuration(
        self,
        slots1: list[list[int]],
        slots2: list[list[int]],
        duration: int,
    ) -> list[int]:
        slots1.sort()
        slots2.sort()

        i = 0 # slots1's index
        j = 0 # slots2's index

        while i < len(slots1) and j < len(slots2):
            start = max(slots1[i][0], slots2[j][0])
            end = min(slots1[i][1], slots2[j][1])
            if start + duration <= end:
                return [start, start + duration]
            if slots1[i][1] < slots2[j][1]:
                i += 1
            else:
                j += 1

        return []

```

215. 123.py

Path: LeetCode-main/solutions/123. Best Time to Buy and Sell Stock III/123.py

```

class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sellTwo = 0
        holdTwo = -math.inf
        sellOne = 0
        holdOne = -math.inf

        for price in prices:
            sellTwo = max(sellTwo, holdTwo + price)
            holdTwo = max(holdTwo, sellOne - price)
            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)

        return sellTwo

```

216. 1230-2.py

Path: LeetCode-main/solutions/1230. Toss Strange Coins/1230-2.py

```

class Solution:
    def probabilityOfHeads(self, prob: list[float], target: int) -> float:
        # dp[j] := the probability of tossing the coins so far with j heads
        dp = [1.0] + [0] * len(prob)

        for p in prob:
            for j in range(target, -1, -1):
                dp[j] = (dp[j - 1] * p if j > 0 else 0) + dp[j] * (1 - p)

        return dp[target]

```

217. 1230.py

Path: LeetCode-main/solutions/1230. Toss Strange Coins/1230.py

```
class Solution:
    def probabilityOfHeads(self, prob: list[float], target: int) -> float:
        # dp[i][j] := the probability of tossing the first i coins with j heads
        dp = [[0] * (target + 1) for _ in range(len(prob) + 1)]
        dp[0][0] = 1.0

        for i in range(1, len(prob) + 1):
            for j in range(target + 1):
                dp[i][j] = ((dp[i - 1][j - 1] * prob[i - 1] if j > 0 else 0) +
                           dp[i - 1][j] * (1 - prob[i - 1]))

        return dp[len(prob)][target]
```

218. 1231.py

Path: LeetCode-main/solutions/1231. Divide Chocolate/1231.py

```
class Solution:
    def maximizeSweetness(self, sweetness: list[int], k: int) -> int:
        l = len(sweetness) // (k + 1)
        r = sum(sweetness) // (k + 1)

        def canEat(m: int) -> bool:
            """
            Returns True if can eat m sweetness (the minimum sweetness of each piece).
            """
            pieces = 0
            summ = 0 # the running sum
            for s in sweetness:
                summ += s
                if summ >= m:
                    pieces += 1
                    summ = 0
            return pieces > k

        while l < r:
            m = (l + r) // 2
            if canEat(m):
                l = m + 1
            else:
                r = m

        return l if canEat(l) else l - 1
```

219. 1232.py

Path: LeetCode-main/solutions/1232. Check If It Is a Straight Line/1232.py

```
class Solution:
    def checkStraightLine(self, coordinates: list[list[int]]) -> bool:
        x0, y0, x1, y1 = *coordinates[0], *coordinates[1]
        dx = x1 - x0
        dy = y1 - y0

        return all((x - x0) * dy == (y - y0) * dx for x, y in coordinates)
```

220. 1233.py

Path: LeetCode-main/solutions/1233. Remove Sub-Folders from the Filesystem/1233.py

```
class Solution:
    def removeSubfolders(self, folder: list[str]) -> list[str]:
        ans = []
        prev = ""

        folder.sort()

        for f in folder:
            if len(prev) > 0 and f.startswith(prev) and f[len(prev)] == '/':
                continue
            ans.append(f)
            prev = f

        return ans
```

221. 1234.py

Path: LeetCode-main/solutions/1234. Replace the Substring for Balanced String/1234.py

```
class Solution:
    def balancedString(self, s: str) -> int:
        ans = len(s)
        count = collections.Counter(s)
        j = 0

        for i, c in enumerate(s):
            count[c] -= 1
            while j < len(s) and all(count[c] <= len(s) // 4 for c in 'QWER'):
                ans = min(ans, i - j + 1)
                count[s[j]] += 1
                j += 1

        return ans
```

222. 1235-2.py

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235-2.py

```

class Solution:
    def jobScheduling(
        self,
        startTime: list[int],
        endTime: list[int],
        profit: list[int],
    ) -> int:
        # dp[i] := the maximum profit to schedule jobs[i..n)
        dp = [0] * (len(startTime) + 1)
        jobs = sorted([(s, e, p) for s, e, p in zip(startTime, endTime, profit)])

        for i in range(len(startTime)):
            startTime[i] = jobs[i][0]

        for i in reversed(range(len(startTime))):
            j = bisect.bisect_left(startTime, jobs[i][1])
            dp[i] = max(jobs[i][2] + dp[j], dp[i + 1])

        return dp[0]

```

223. 1235-3.py

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235-3.py

```

class Solution:
    def jobScheduling(
        self,
        startTime: list[int],
        endTime: list[int],
        profit: list[int],
    ) -> int:
        maxProfit = 0
        jobs = sorted([(s, e, p) for s, e, p in zip(startTime, endTime, profit)])
        minHeap = [] # (endTime, profit)

        # Will use binary search to find the first available startTime
        for i in range(len(startTime)):
            startTime[i] = jobs[i][0]

        for s, e, p in jobs:
            while minHeap and s >= minHeap[0][0]:
                maxProfit = max(maxProfit, heapq.heappop(minHeap)[1])
            heapq.heappush(minHeap, (e, p + maxProfit))

        return max(maxProfit, max(p for _, p in minHeap))

```

224. 1235.py

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235.py

```

class Solution:
    def jobScheduling(
        self,
        startTime: list[int],
        endTime: list[int],
        profit: list[int],
    ) -> int:
        jobs = sorted([(s, e, p) for s, e, p in zip(startTime, endTime, profit)])

        # Will use binary search to find the first available startTime
        for i in range(len(startTime)):
            startTime[i] = jobs[i][0]

        @functools.lru_cache(None)
        def dp(i: int) -> int:
            """Returns the maximum profit to schedule jobs[i..n)."""
            if i == len(startTime):
                return 0
            j = bisect.bisect_left(startTime, jobs[i][1])
            return max(jobs[i][2] + dp(j), dp(i + 1))

        return dp(0)

```

225. 1236.py

Path: LeetCode-main/solutions/1236. Web Crawler/1236.py

```

# """
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class HtmlParser(object):
#     def getUrls(self, url: str) -> list[str]:

class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> list[str]:
        q = collections.deque([startUrl])
        seen = {startUrl}
        hostname = startUrl.split('/')[2]

        while q:
            currUrl = q.popleft()
            for url in htmlParser.getUrls(currUrl):
                if url in seen:
                    continue
                if hostname in url:
                    q.append(url)
                    seen.add(url)

        return seen

```

226. 1237.py

Path: LeetCode-main/solutions/1237. Find Positive Integer Solution for a Given Equation/1237.py

```

class Solution:
    def findSolution(self, customfunction: 'CustomFunction', z: int) -> list[list[int]]:
        ans = []
        x = 1
        y = 1000

        while x <= 1000 and y >= 1:
            f = customfunction.f(x, y)
            if f < z:
                x += 1
            elif f > z:
                y -= 1
            else:
                ans.append([x, y])
                x += 1
                y -= 1

        return ans

```

227. 1238.py

Path: LeetCode-main/solutions/1238. Circular Permutation in Binary Representation/1238.py

```

class Solution:
    def circularPermutation(self, n: int, start: int) -> list[int]:
        return [start ^ i ^ i >> 1 for i in range(1 << n)]

```

228. 124.py

Path: LeetCode-main/solutions/124. Binary Tree Maximum Path Sum/124.py

```

class Solution:
    def maxPathSum(self, root: TreeNode | None) -> int:
        ans = -math.inf

        def maxPathSumDownFrom(root: TreeNode | None) -> int:
            """
            Returns the maximum path sum starting from the current root, where
            root.val is always included.
            """
            nonlocal ans
            if not root:
                return 0

            l = max(0, maxPathSumDownFrom(root.left))
            r = max(0, maxPathSumDownFrom(root.right))
            ans = max(ans, root.val + l + r)
            return root.val + max(l, r)

        maxPathSumDownFrom(root)
        return ans

```

229. 1240.py

Path: LeetCode-main/solutions/1240. Tiling a Rectangle with the Fewest Squares/1240.py

```

class Solution:
    def tilingRectangle(self, n: int, m: int) -> int:
        @functools.lru_cache(None)
        def dp(heights: int) -> int:
            minHeight = min(heights)
            if minHeight == n: # All filled.
                return 0

            ans = m * n
            heightsList = list(heights)
            start = heightsList.index(minHeight)

            # Try to put square of different size that doesn't exceed the width/height.
            for sz in range(1, min(m - start + 1, n - minHeight + 1)):
                # heights[start..start + sz) must has the same height.
                if heights[start + sz - 1] != minHeight:
                    break
                # Put a square of size `sz` to cover heights[start..start + sz).
                heightsList[start:start + sz] = [minHeight + sz] * sz
                ans = min(ans, dp(tuple(heightsList)))

            return 1 + ans

        return dp(tuple([0] * m))

```

230. 1243.py

Path: LeetCode-main/solutions/1243. Array Transformation/1243.py

```

class Solution:
    def transformArray(self, arr: list[int]) -> list[int]:
        if len(arr) < 3:
            return arr

        ans = []

        while ans != arr:
            ans = arr[:]
            for i in range(1, len(arr) - 1):
                if ans[i - 1] > ans[i] < ans[i + 1]:
                    arr[i] += 1
                elif ans[i - 1] < ans[i] > ans[i + 1]:
                    arr[i] -= 1

        return ans

```

231. 1244.py

Path: LeetCode-main/solutions/1244. Design A Leaderboard/1244.py

```
class Leaderboard:
    def __init__(self):
        self.idToScore = collections.Counter()

    def addScore(self, playerId: int, score: int) -> None:
        self.idToScore[playerId] += score

    def top(self, K: int) -> int:
        return sum(score for _, score in self.idToScore.most_common(K))

    def reset(self, playerId: int) -> None:
        del self.idToScore[playerId]
```

232. 1246.py

Path: LeetCode-main/solutions/1246. Palindrome Removal/1246.py

```
class Solution:
    def minimumMoves(self, arr: list[int]) -> int:
        n = len(arr)
        # dp[i][j] := the minimum number of moves to remove all numbers from arr[i..j]
        dp = [[n] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for i in range(n - 1):
            dp[i][i + 1] = 1 if arr[i] == arr[i + 1] else 2

        for d in range(2, n):
            for i in range(n - d):
                j = i + d
                # Remove arr[i] and arr[j] within the move of removing
                # arr[i + 1..j - 1]
                if arr[i] == arr[j]:
                    dp[i][j] = dp[i + 1][j - 1]
                # Try all the possible partitions.
                for k in range(i, j):
                    dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j])

        return dp[0][n - 1]
```

233. 1247.py

Path: LeetCode-main/solutions/1247. Minimum Swaps to Make Strings Equal/1247.py


```

class Solution:
    def minimumSwap(self, s1: str, s2: str) -> int:
        # ('xx', 'yy') = (2 'xy's) . 1 swap
        # ('yy', 'xx') = (2 'yx's) . 1 swap
        # ('xy', 'yx') = (1 'xy' and 1 'yx') . 2 swaps
        xy = 0 # the number of indices i's s.t. s1[i] = 'x' and s2[i] 'y'
        yx = 0 # the number of indices i's s.t. s1[i] = 'y' and s2[i] 'x'

        for a, b in zip(s1, s2):
            if a == b:
                continue
            if a == 'x':
                xy += 1
            else:
                yx += 1

        if (xy + yx) % 2 == 1:
            return -1
        return xy // 2 + yx // 2 + (0 if xy % 2 == 0 else 2)

```

234. 1248.py

Path: LeetCode-main/solutions/1248. Count Number of Nice Subarrays/1248.py

```

class Solution:
    def numberOfSubarrays(self, nums: list[int], k: int) -> int:
        def numberOfSubarraysAtMost(k: int) -> int:
            ans = 0
            l = 0
            r = 0

            while r <= len(nums):
                if k >= 0:
                    ans += r - l
                    if r == len(nums):
                        break
                    if nums[r] & 1:
                        k -= 1
                    r += 1
                else:
                    if nums[l] & 1:
                        k += 1
                    l += 1
            return ans

        return numberOfSubarraysAtMost(k) - numberOfSubarraysAtMost(k - 1)

```

235. 1249.py

Path: LeetCode-main/solutions/1249. Minimum Remove to Make Valid Parentheses/1249.py

```

class Solution:
    def minRemoveToMakeValid(self, s: str) -> str:
        stack = [] # unpaired '(' indices
        chars = list(s)

        for i, c in enumerate(chars):
            if c == '(':
                stack.append(i) # Record the unpaired '(' index.
            elif c == ')':
                if stack:
                    stack.pop() # Find a pair
                else:
                    chars[i] = '*' # Mark the unpaired ')' as '*'.

        # Mark the unpaired '(' as '*'.
        while stack:
            chars[stack.pop()] = '*'

        return ''.join(chars).replace('*', '')

```

236. 125.py

Path: LeetCode-main/solutions/125. Valid Palindrome/125.py

```

class Solution:
    def isPalindrome(self, s: str) -> bool:
        l = 0
        r = len(s) - 1

        while l < r:
            while l < r and not s[l].isalnum():
                l += 1
            while l < r and not s[r].isalnum():
                r -= 1
            if s[l].lower() != s[r].lower():
                return False
            l += 1
            r -= 1

        return True

```

237. 1252-2.py

Path: LeetCode-main/solutions/1252. Cells with Odd Values in a Matrix/1252-2.py

```
class Solution:
    def oddCells(self, m: int, n: int, indices: list[list[int]]) -> int:
        # rows[i] and cols[i] :=
        # 1. True (flipped even times)
        # 2. False (flipped odd times)
        rows = [False] * m
        cols = [False] * n

        for r, c in indices:
            rows[r] ^= True
            cols[c] ^= True

        oddRowCount = rows.count(True)
        oddColsCount = cols.count(True)
        evenRowCount = m - oddRowCount
        evenColsCount = n - oddColsCount
        return oddRowCount * evenColsCount + oddColsCount * evenRowCount
```

238. 1252.py

Path: LeetCode-main/solutions/1252. Cells with Odd Values in a Matrix/1252.py

```
class Solution:
    def oddCells(self, m: int, n: int, indices: list[list[int]]) -> int:
        # rows[i] and cols[i] :=
        # true (flipped even times) / false (flipped odd times)
        rows = [False] * m
        cols = [False] * n

        for r, c in indices:
            rows[r] ^= True
            cols[c] ^= True

        return sum(rows[i] ^ cols[j]
                    for i in range(m)
                    for j in range(n))
```

239. 1253.py

Path: LeetCode-main/solutions/1253. Reconstruct a 2-Row Binary Matrix/1253.py

```
class Solution:
    def reconstructMatrix(self, upper: int, lower: int, colsum: list[int]) -> list[list[int]]:
        if upper + lower != sum(colsum):
            return []
        if min(upper, lower) < colsum.count(2):
            return []

        ans = [[0] * len(colsum) for _ in range(2)]

        for j, c in enumerate(colsum):
            if c == 2:
                ans[0][j] = 1
                ans[1][j] = 1
                upper -= 1
                lower -= 1

        for j, c in enumerate(colsum):
            if c == 1 and upper > 0:
                ans[0][j] = 1
                c -= 1
                upper -= 1
            if c == 1 and lower > 0:
                ans[1][j] = 1
                lower -= 1

        return ans
```

240. 1254.py

Path: LeetCode-main/solutions/1254. Number of Closed Islands/1254.py

```
class Solution:
    def closedIsland(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if grid[i][j] == 1:
                return
            grid[i][j] = 1
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        # Remove the lands connected to the edge.
        for i in range(m):
            for j in range(n):
                if i * j == 0 or i == m - 1 or j == n - 1:
                    if grid[i][j] == 0:
                        dfs(i, j)

        ans = 0

        # Reduce to 200. Number of Islands
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 0:
                    dfs(i, j)
                    ans += 1

        return ans
```

241. 1255.py

Path: LeetCode-main/solutions/1255. Maximum Score Words Formed by Letters/1255.py

```

class Solution:
    def maxScoreWords(
        self,
        words: list[str],
        letters: list[str],
        score: list[int],
    ) -> int:
        count = collections.Counter(letters)

        def useWord(i: int) -> int:
            isValid = True
            earned = 0
            for c in words[i]:
                count[c] -= 1
                if count[c] < 0:
                    isValid = False
            earned += score[ord(c) - ord('a')]
            return earned if isValid else -1

        def unuseWord(i: int) -> None:
            for c in words[i]:
                count[c] += 1

        def dfs(s: int) -> int:
            """Returns the maximum score you can get from words[s..n)."""
            ans = 0
            for i in range(s, len(words)):
                earned = useWord(i)
                if earned > 0:
                    ans = max(ans, earned + dfs(i + 1))
                unuseWord(i)
            return ans

        return dfs(0)

```

242. 1256.py

Path: LeetCode-main/solutions/1256. Encode Number/1256.py

```

class Solution:
    def encode(self, num: int) -> str:
        return bin(num + 1)[3:]

```

243. 1258.py

Path: LeetCode-main/solutions/1258. Synonymous Sentences/1258.py

```

from sortedcontainers import SortedSet

class Solution:
    def generateSentences(
        self,
        synonyms: list[list[str]],
        text: str,
    ) -> list[str]:
        ans = SortedSet()
        graph = collections.defaultdict(list)
        q = collections.deque([text])

        for s, t in synonyms:
            graph[s].append(t)
            graph[t].append(s)

        while q:
            u = q.popleft()
            ans.add(u)
            words = u.split()
            for i, word in enumerate(words):
                for synonym in graph[word]:
                    # Replace words[i] with its synonym.
                    words[i] = synonym
                    newText = ' '.join(words)
                    if newText not in ans:
                        q.append(newText)

        return list(ans)

```

244. 1259.py

Path: LeetCode-main/solutions/1259. Handshakes That Don't Cross/1259.py

```

class Solution:
    def numberOfWays(self, numPeople: int) -> int:
        MOD = 1_000_000_007
        # dp[i] := the number of ways i handshakes could occur s.t. none of the
        # handshakes cross
        dp = [1] + [0] * (numPeople // 2)

        for i in range(1, numPeople // 2 + 1):
            for j in range(i):
                dp[i] += dp[j] * dp[i - 1 - j]
                dp[i] %= MOD

        return dp[numPeople // 2]

```

245. 126.py

Path: LeetCode-main/solutions/126. Word Ladder II/126.py

```

class Solution:
    def findLadders(self, beginWord: str, endWord: str, wordList: list[str]) -> list[list[str]]:
        wordSet = set(wordList)
        if endWord not in wordSet:
            return []

        # {"hit": ["hot"], "hot": ["dot", "lot"], ...}

```

```

graph: dict[str, list[str]] = collections.defaultdict(list)

# Build the graph from the beginWord to the endWord.
if not self._bfs(beginWord, endWord, wordSet, graph):
    return []

ans = []
self._dfs(graph, beginWord, endWord, [beginWord], ans)
return ans

def _bfs(
    self,
    beginWord: str,
    endWord: str,
    wordSet: set[str],
    graph: dict[str, list[str]],
) -> bool:
    currentLevelWords = {beginWord}

    while currentLevelWords:
        for word in currentLevelWords:
            wordSet.discard(word)
            nextLevelWords = set()
            reachEndWord = False
            for parent in currentLevelWords:
                for child in self._getChildren(parent, wordSet):
                    if child in wordSet:
                        nextLevelWords.add(child)
                        graph[parent].append(child)
                    if child == endWord:
                        reachEndWord = True
            if reachEndWord:
                return True
            currentLevelWords = nextLevelWords

    return False

def _getChildren(self, parent: str, wordSet: set[str]) -> list[str]:
    children = []
    s = list(parent)

    for i, cache in enumerate(s):
        for c in string.ascii_lowercase:
            if c == cache:
                continue
            s[i] = c
            child = ''.join(s)
            if child in wordSet:
                children.append(child)
            s[i] = cache

    return children

def _dfs(
    self,
    graph: dict[str, list[str]],
    word: str,
    endWord: str,
    path: list[str],
    ans: list[list[str]],
) -> None:
    if word == endWord:
        ans.append(path.copy())
        return

```



```

for child in graph.get(word, []):
    path.append(child)
    self._dfs(graph, child, endWord, path, ans)
    path.pop()

```

246. 127-2.py

Path: LeetCode-main/solutions/126. Word Ladder II/127-2.py

```

class Solution:
    def findLadders(self, beginWord: str, endWord: str, wordList: list[str]) -> list[list[str]]:
        wordSet = set(wordList)
        if endWord not in wordList:
            return []

        # {"hit": ["hot"], "hot": ["dot", "lot"], ...}
        graph: dict[str, list[str]] = collections.defaultdict(list)

        # Build the graph from the beginWord to the endWord.
        if not self._bfs(beginWord, endWord, wordSet, graph):
            return []

        ans = []

        self._dfs(graph, beginWord, endWord, [beginWord], ans)
        return ans

    def _bfs(
        self,
        beginWord: str,
        endWord: str,
        wordSet: set[str],
        graph: dict[str, list[str]],
    ) -> bool:
        q1 = {beginWord}
        q2 = {endWord}
        backward = False

        while q1 and q2:
            for word in q1:
                wordSet.discard(word)
            for word in q2:
                wordSet.discard(word)
            # Always expand the smaller queue.
            if len(q1) > len(q2):
                q1, q2 = q2, q1
                backward = not backward
            q = set()
            reachEndWord = False
            for parent in q1:
                for child in self._getChildren(parent, wordSet, q2):
                    if child in wordSet or child in q2:
                        q.add(child)
                    if backward:
                        graph[child].append(parent)
                    else:
                        graph[parent].append(child)
                    if child in q2:
                        reachEndWord = True
            if reachEndWord:
                return True
            q1 = q

```

```

return False

def _getChildren(self, parent: str, wordSet: set[str], q2) -> list[str]:
    children = []
    s = list(parent)

    for i, cache in enumerate(s):
        for c in string.ascii_lowercase:
            if c == cache:
                continue
            s[i] = c
            child = ''.join(s)
            if child in wordSet or child in q2:
                children.append(child)
            s[i] = cache

    return children

def _dfs(
    self,
    graph: dict[str, list[str]],
    word: str,
    endWord: str,
    path: list[str],
    ans: list[list[str]],
) -> None:
    if word == endWord:
        ans.append(path.copy())
        return

    for child in graph.get(word, []):
        path.append(child)
        self._dfs(graph, child, endWord, path, ans)
        path.pop()

```

247. 1260.py

Path: LeetCode-main/solutions/1260. Shift 2D Grid/1260.py

```

class Solution:
    def shiftGrid(self, grid: list[list[int]], k: int) -> list[list[int]]:
        m = len(grid)
        n = len(grid[0])
        ans = [[0] * n for _ in range(m)]

        k %= m * n

        for i in range(m):
            for j in range(n):
                index = (i * n + j + k) % (m * n)
                x = index // n
                y = index % n
                ans[x][y] = grid[i][j]

        return ans

```

248. 1261.py

Path: LeetCode-main/solutions/1261. Find Elements in a Contaminated Binary Tree/1261.py

```

class FindElements:
    def __init__(self, root: TreeNode | None):
        self.vals = set()
        self.dfs(root, 0)

    def find(self, target: int) -> bool:
        return target in self.vals

    def dfs(self, root: TreeNode | None, val: int) -> None:
        if not root:
            return

        root.val = val
        self.vals.add(val)
        self.dfs(root.left, val * 2 + 1)
        self.dfs(root.right, val * 2 + 2)

```

249. 1263.py

Path: LeetCode-main/solutions/1263. Minimum Moves to Move a Box to Their Target Location/1263.py

```

class Solution:
    def minPushBox(self, grid: list[list[str]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 'B':
                    box = (i, j)
                elif grid[i][j] == 'S':
                    player = (i, j)
                elif grid[i][j] == 'T':
                    target = (i, j)

        def isInvalid(playerX: int, playerY: int) -> bool:
            return (playerX < 0 or playerX == m or playerY < 0 or playerY == n or
                    grid[playerX][playerY] == '#')

        def canGoTo(
            playerX: int,
            playerY: int,
            fromX: int,
            fromY: int,
            boxX: int,
            boxY: int
        ) -> bool:
            """Returns True if (playerX, playerY) can go to (fromX, fromY)."""
            q = collections.deque([(playerX, playerY)])
            seen = {(playerX, playerY)}

            while q:
                i, j = q.popleft()
                if i == fromX and j == fromY:
                    return True
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if isInvalid(x, y):
                        continue
                    if (x, y) in seen:
                        continue
                    if grid[x][y] == 'B':
                        continue
                    q.append((x, y))
                    seen.add((x, y))

```

```

        continue
    if x == boxX and y == boxY:
        continue
    q.append((x, y))
    seen.add((x, y))

    return False

# (boxX, boxY, playerX, playerY)
q = collections.deque([(box[0], box[1], player[0], player[1])])
seen = {(box[0], box[1], player[0], player[1])}

step = 0
while q:
    for _ in range(len(q)):
        boxX, boxY, playerX, playerY = q.popleft()
        if boxX == target[0] and boxY == target[1]:
            return step
    for k, (dx, dy) in enumerate(DIRS):
        nextBoxX = boxX + dx
        nextBoxY = boxY + dy
        if isValid(nextBoxX, nextBoxY):
            continue
        if (nextBoxX, nextBoxY, boxX, boxY) in seen:
            continue
        fromX = boxX + DIRS[(k + 2) % 4][0]
        fromY = boxY + DIRS[(k + 2) % 4][1]
        if isValid(fromX, fromY):
            continue
        if canGoTo(playerX, playerY, fromX, fromY, boxX, boxY):
            q.append((nextBoxX, nextBoxY, boxX, boxY))
            seen.add((nextBoxX, nextBoxY, boxX, boxY))
    step += 1

return -1

```

250. 1266.py

Path: LeetCode-main/solutions/1266. Minimum Time Visiting All Points/1266.py

```

class Solution:
    def minTimeToVisitAllPoints(self, points: list[list[int]]) -> int:
        ans = 0

        for i in range(1, len(points)):
            ans += max(abs(points[i][0] - points[i - 1][0]),
                       abs(points[i][1] - points[i - 1][1]))

        return ans

```

251. 1267.py

Path: LeetCode-main/solutions/1267. Count Servers that Communicate/1267.py

```
class Solution:
    def countServers(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = 0
        rows = [0] * m
        cols = [0] * n

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    rows[i] += 1
                    cols[j] += 1

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1 and (rows[i] > 1 or cols[j] > 1):
                    ans += 1

        return ans
```

252. 1268.py

Path: LeetCode-main/solutions/1268. Search Suggestions System/1268.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def suggestedProducts(
        self,
        products: list[str],
        searchWord: str
    ) -> list[list[str]]:
        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        def search(node: TrieNode | None) -> list[str]:
            res: list[str] = []
            dfs(node, res)
            return res

        def dfs(node: TrieNode | None, res: list[str]) -> None:
            if len(res) == 3:
                return
            if not node:
                return
            if node.word:
                res.append(node.word)
            for c in string.ascii_lowercase:
                if c in node.children:
                    dfs(node.children[c], res)

        for product in products:
            insert(product)

        node = root

        for c in searchWord:
            if not node or c not in node.children:
                node = None
                ans.append([])
                continue
            node = node.children[c]
            ans.append(search(node))

        return ans

```

253. 1269.py

Path: LeetCode-main/solutions/1269. Number of Ways to Stay in the Same Place After Some Steps/1269.py

```

class Solution:
    def numWays(self, steps: int, arrLen: int) -> int:
        MOD = 1_000_000_007

        # dp[i] := the number of ways to stay at index i
        dp = [0] * min(steps // 2 + 1, arrLen)
        dp[0] = 1

        for _ in range(steps):
            newDp = [0] * min(steps // 2 + 1, arrLen)
            for i, ways in enumerate(dp):
                if ways > 0:
                    for dx in (-1, 0, 1):
                        nextIndex = i + dx
                        if 0 <= nextIndex < len(dp):
                            newDp[nextIndex] += ways
                            newDp[nextIndex] %= MOD
            dp = newDp

        return dp[0]

```

254. 127.py

Path: LeetCode-main/solutions/127. Word Ladder/127.py

```

class Solution:
    def ladderLength(
        self,
        beginWord: str,
        endWord: str,
        wordList: list[str],
    ) -> int:
        wordSet = set(wordList)
        if endWord not in wordSet:
            return 0

        q = collections.deque([beginWord])

        step = 1
        while q:
            for _ in range(len(q)):
                wordList = list(q.popleft())
                for i, cache in enumerate(wordList):
                    for c in string.ascii_lowercase:
                        wordList[i] = c
                        word = ''.join(wordList)
                        if word == endWord:
                            return step + 1
                        if word in wordSet:
                            q.append(word)
                            wordSet.remove(word)
                wordList[i] = cache
            step += 1

        return 0

```

255. 1271.py

Path: LeetCode-main/solutions/1271. Hexspeak/1271.py

```
class Solution:
    def toHexspeak(self, num: str) -> str:
        s = hex(int(num)).upper()[2:].translate(str.maketrans('01', 'OI'))
        return 'ERROR' if any(c.isdigit() for c in s) else s
```

256. 1272.py

Path: LeetCode-main/solutions/1272. Remove Interval/1272.py

```
class Solution:
    def removeInterval(self, intervals: list[list[int]],
                       toBeRemoved: list[int]) -> list[list[int]]:
        ans = []

        for a, b in intervals:
            if a >= toBeRemoved[1] or b <= toBeRemoved[0]:
                ans.append([a, b])
            else: # a < toBeRemoved[1] and b > toBeRemoved[0]
                if a < toBeRemoved[0]:
                    ans.append([a, toBeRemoved[0]])
                if b > toBeRemoved[1]:
                    ans.append([toBeRemoved[1], b])

        return ans
```

257. 1274.py

Path: LeetCode-main/solutions/1274. Number of Ships in a Rectangle/1274.py


```

# """
# This is Sea's API interface.
# You should not implement it, or speculate about its implementation
# """
# class Sea(object):
#     def hasShips(self, topRight: 'Point', bottomLeft: 'Point') -> bool:
#         pass
#
# class Point(object):
#     def __init__(self, x: int, y: int):
#         self.x = x
#         self.y = y

class Solution(object):
    def countShips(
        self,
        sea: 'Sea',
        topRight: 'Point',
        bottomLeft: 'Point',
    ) -> int:
        if topRight.x < bottomLeft.x or topRight.y < bottomLeft.y:
            return 0
        if not sea.hasShips(topRight, bottomLeft):
            return 0

        # sea.hashShips(topRight, bottomLeft) == True
        if topRight.x == bottomLeft.x and topRight.y == bottomLeft.y:
            return 1

        mx = (topRight.x + bottomLeft.x) // 2
        my = (topRight.y + bottomLeft.y) // 2
        ans = 0
        # the top-right
        ans += self.countShips(sea, topRight, Point(mx + 1, my + 1))
        # the bottom-right
        ans += self.countShips(sea, Point(topRight.x, my),
                               Point(mx + 1, bottomLeft.y))
        # the top-left
        ans += self.countShips(sea, Point(mx, topRight.y),
                               Point(bottomLeft.x, my + 1))
        # the bottom-left
        ans += self.countShips(sea, Point(mx, my), bottomLeft)
        return ans

```

258. 1275.py

Path: LeetCode-main/solutions/1275. Find Winner on a Tic Tac Toe Game/1275.py

```

class Solution:
    def tictactoe(self, moves: list[list[int]]) -> str:
        row = [[0] * 3 for _ in range(2)]
        col = [[0] * 3 for _ in range(2)]
        diag1 = [0] * 2
        diag2 = [0] * 2
        i = 0

        for r, c in moves:
            row[i][r] += 1
            col[i][c] += 1
            diag1[i] += r == c
            diag2[i] += r + c == 2
            if 3 in (row[i][r], col[i][c], diag1[i], diag2[i]):
                return 'A' if i == 0 else 'B'
            i ^= 1

        return 'Draw' if len(moves) == 9 else 'Pending'

```

259. 1276.py

Path: LeetCode-main/solutions/1276. Number of Burgers with No Waste of Ingredients/1276.py

```

class Solution:
    def numOfBurgers(self, tomatoSlices: int, cheeseSlices: int) -> list[int]:
        if tomatoSlices % 2 == 1 or tomatoSlices < 2 * cheeseSlices or tomatoSlices > cheeseSlices * 4:
            return []

        jumboBurgers = (tomatoSlices - 2 * cheeseSlices) // 2

        return [jumboBurgers, cheeseSlices - jumboBurgers]

```

260. 1277.py

Path: LeetCode-main/solutions/1277. Count Square Submatrices with All Ones/1277.py

```

class Solution:
    def countSquares(self, matrix: list[list[int]]) -> int:
        for i in range(len(matrix)):
            for j in range(len(matrix[0])):
                if matrix[i][j] == 1 and i > 0 and j > 0:
                    matrix[i][j] += min(matrix[i - 1][j - 1],
                                         matrix[i - 1][j], matrix[i][j - 1])
        return sum(map(sum, matrix))

```

261. 1279.py

Path: LeetCode-main/solutions/1279. Traffic Light Controlled Intersection/1279.py

```

class TrafficLight:
    def __init__(self):
        self.canPassRoadId = 1 # 1 := road A, 2 := road B

    def carArrived(
        self,
        # ID of the car
        carId: int,
        # ID of the road the car travels on. Can be 1 (road A) or 2 (road B).
        roadId: int,
        # direction of the car
        direction: int,
        # Use turnGreen() to turn light to green on current road.
        turnGreen: Callable[[], None],
        # Use crossCar() to make car cross the intersection.
        crossCar: Callable[[], None]
    ) -> None:
        if roadId != self.canPassRoadId:
            self.canPassRoadId = roadId
            turnGreen()
            crossCar()

```

262. 128.py

Path: LeetCode-main/solutions/128. Longest Consecutive Sequence/128.py

```

class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        ans = 0
        seen = set(nums)

        for num in seen:
            # `num` is the start of a sequence.
            if num - 1 in seen:
                continue
            length = 0
            while num in seen:
                num += 1
                length += 1
            ans = max(ans, length)

        return ans

```

263. 1281.py

Path: LeetCode-main/solutions/1281. Subtract the Product and Sum of Digits of an Integer/1281.py

```

class Solution:
    def subtractProductAndSum(self, n: int) -> int:
        prod = 1
        summ = 0

        while n > 0:
            prod *= n % 10
            summ += n % 10
            n //= 10

        return prod - summ

```

264. 1282.py

Path: LeetCode-main/solutions/1282. Group the People Given the Group Size They Belong To/1282.py

```
class Solution:
    def groupThePeople(self, groupSizes: list[int]) -> list[list[int]]:
        ans = []
        groupSizeToIndices = defaultdict(list)

        for i, groupSize in enumerate(groupSizes):
            groupSizeToIndices[groupSize].append(i)

        for groupSize, indices in groupSizeToIndices.items():
            groupIndices = []
            for index in indices:
                groupIndices.append(index)
            if len(groupIndices) == groupSize:
                ans.append(groupIndices.copy())
                groupIndices.clear()

        return ans
```

265. 1283.py

Path: LeetCode-main/solutions/1283. Find the Smallest Divisor Given a Threshold/1283.py

```
class Solution:
    def smallestDivisor(self, nums: list[int], threshold: int) -> int:
        l = 1
        r = max(nums)

        while l < r:
            m = (l + r) // 2
            if sum((num - 1) // m + 1 for num in nums) <= threshold:
                r = m
            else:
                l = m + 1

        return l
```

266. 1284.py

Path: LeetCode-main/solutions/1284. Minimum Number of Flips to Convert Binary Matrix to Zero Matrix/1284.py

```

class Solution:
    def minFlips(self, mat: list[list[int]]) -> int:
        m = len(mat)
        n = len(mat[0])
        hash = self._getHash(mat, m, n)
        if hash == 0:
            return 0

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        step = 0
        q = collections.deque([hash])
        seen = {hash}

        while q:
            step += 1
            for _ in range(len(q)):
                curr = q.popleft()
                for i in range(m):
                    for j in range(n):
                        next = curr ^ 1 << (i * n + j)
                        # Flie the four neighbors.
                        for dx, dy in DIRS:
                            x = i + dx
                            y = j + dy
                            if x < 0 or x == m or y < 0 or y == n:
                                continue
                            next ^= 1 << (x * n + y)
                        if next == 0:
                            return step
                        if next in seen:
                            continue
                        q.append(next)
                        seen.add(next)

        return -1

    def _getHash(self, mat: list[list[int]], m: int, n: int) -> int:
        hash = 0
        for i in range(m):
            for j in range(n):
                if mat[i][j]:
                    hash |= 1 << (i * n + j)
        return hash

```

267. 1287.py

Path: LeetCode-main/solutions/1287. Element Appearing More Than 25% In Sorted Array/1287.py

```

class Solution:
    def findSpecialInteger(self, arr: list[int]) -> int:
        n = len(arr)
        quarter = n // 4

        for i in range(n - quarter):
            if arr[i] == arr[i + quarter]:
                return arr[i]

```

268. 1288.py

Path: LeetCode-main/solutions/1288. Remove Covered Intervals/1288.py

```
class Solution:
    def removeCoveredIntervals(self, intervals: list[list[int]]) -> int:
        ans = 0
        prevEnd = 0

        for _, end in sorted(intervals, key=lambda x: (x[0], -x[1])):
            # The current interval is not covered by the previous one.
            if prevEnd < end:
                prevEnd = end
                ans += 1

        return ans
```

269. 1289.py

Path: LeetCode-main/solutions/1289. Minimum Falling Path Sum II/1289.py

```
class Solution:
    def minFallingPathSum(self, grid: list[list[int]]) -> int:
        n = len(grid)

        for i in range(1, n):
            (firstMinNum, firstMinIndex), (secondMinNum, _) = sorted(
                {(a, i) for i, a in enumerate(grid[i - 1])}[:2]
            )
            for j in range(n):
                if j == firstMinIndex:
                    grid[i][j] += secondMinNum
                else:
                    grid[i][j] += firstMinNum

        return min(grid[-1])
```

270. 129.py

Path: LeetCode-main/solutions/129. Sum Root to Leaf Numbers/129.py

```
class Solution:
    def sumNumbers(self, root: TreeNode | None) -> int:
        ans = 0

        def dfs(root: TreeNode | None, path: int) -> None:
            nonlocal ans
            if not root:
                return
            if not root.left and not root.right:
                ans += path * 10 + root.val
                return
            dfs(root.left, path * 10 + root.val)
            dfs(root.right, path * 10 + root.val)

        dfs(root, 0)
        return ans
```

271. 1290.py

Path: LeetCode-main/solutions/1290. Convert Binary Number in a Linked List to Integer/1290.py

```
class Solution:
    def getDecimalValue(self, head: ListNode) -> int:
        ans = 0

        while head:
            ans = ans * 2 + head.val
            head = head.next

        return ans
```

272. 1291.py

Path: LeetCode-main/solutions/1291. Sequential Digits/1291.py

```
class Solution:
    def sequentialDigits(self, low: int, high: int) -> list[int]:
        ans = []
        q = collections.deque([num for num in range(1, 10)])

        while q:
            num = q.popleft()
            if num > high:
                return ans
            if low <= num and num <= high:
                ans.append(num)
            lastDigit = num % 10
            if lastDigit < 9:
                q.append(num * 10 + lastDigit + 1)

        return ans
```

273. 1292.py

Path: LeetCode-main/solutions/1292. Maximum Side Length of a Square with Sum Less than or Equal to Threshold/1292.py

```

class Solution:
    def maxSideLength(self, mat: list[list[int]], threshold: int) -> int:
        m = len(mat)
        n = len(mat[0])
        ans = 0
        prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                prefix[i + 1][j + 1] = (mat[i][j] + prefix[i][j + 1] +
                                         prefix[i + 1][j] - prefix[i][j])

        def squareSum(r1: int, c1: int, r2: int, c2: int) -> int:
            return prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1] - prefix[r2 + 1][c1] + prefix[r1][c1]

        for i in range(m):
            for j in range(n):
                for length in range(ans, min(m - i, n - j)):
                    if squareSum(i, j, i + length, j + length) > threshold:
                        break
                    ans = max(ans, length + 1)

        return ans

```

274. 1293.py

Path: LeetCode-main/solutions/1293. Shortest Path in a Grid with Obstacles Elimination/1293.py

```

class Solution:
    def shortestPath(self, grid: list[list[int]], k: int) -> int:
        m = len(grid)
        n = len(grid[0])
        if m == 1 and n == 1:
            return 0

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        q = collections.deque([(0, 0, k)])
        seen = {(0, 0, k)}

        step = 0
        while q:
            step += 1
            for _ in range(len(q)):
                i, j, eliminate = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if x == m - 1 and y == n - 1:
                        return step
                    if grid[x][y] == 1 and eliminate == 0:
                        continue
                    newEliminate = eliminate - grid[x][y]
                    if (x, y, newEliminate) in seen:
                        continue
                    q.append((x, y, newEliminate))
                    seen.add((x, y, newEliminate))

        return -1

```


275. 1295.py

Path: LeetCode-main/solutions/1295. Find Numbers with Even Number of Digits/1295.py

```
class Solution:
    def findNumbers(self, nums: list[int]) -> int:
        ans = 0

        for num in nums:
            if 9 < num < 100 or 999 < num < 10000 or num == 100000:
                ans += 1

        return ans
```

276. 1296.py

Path: LeetCode-main/solutions/1296. Divide Array in Sets of K Consecutive Numbers/1296.py

```
class Solution:
    def isPossibleDivide(self, nums: list[int], k: int) -> bool:
        count = collections.Counter(nums)

        for start in sorted(count):
            value = count[start]
            if value > 0:
                for i in range(start, start + k):
                    count[i] -= value
                    if count[i] < 0:
                        return False

        return True
```

277. 1297.py

Path: LeetCode-main/solutions/1297. Maximum Number of Occurrences of a Substring/1297.py

```
class Solution:
    def maxFreq(self, s: str, maxLetters: int, minSize: int, maxSize: int) -> int:
        # Greedily consider strings with `minSize`, so ignore `maxSize`.
        ans = 0
        letters = 0
        count = collections.Counter()
        substringCount = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            if count[c] == 1:
                letters += 1
            while letters > maxLetters or r - l + 1 > maxSize:
                count[s[l]] -= 1
                if count[s[l]] == 0:
                    letters -= 1
                l += 1
            if r - l + 1 == minSize:
                sub = s[l:l + minSize]
                substringCount[sub] += 1
                ans = max(ans, substringCount[sub])

        return ans
```

278. 1298.py

Path: LeetCode-main/solutions/1298. Maximum Candies You Can Get from Boxes/1298.py

```

class Solution:
    def maxCandies(
        self,
        status: list[int],
        candies: list[int],
        keys: list[list[int]],
        containedBoxes: list[list[int]],
        initialBoxes: list[int],
    ) -> int:
        ans = 0
        q = collections.deque()
        reachedClosedBoxes = [0] * len(status)

        def pushBoxesIfPossible(boxes: list[int]) -> None:
            for box in boxes:
                if status[box]:
                    q.append(box)
                else:
                    reachedClosedBoxes[box] = True

        pushBoxesIfPossible(initialBoxes)

        while q:
            currBox = q.popleft()

            # Add the candies.
            ans += candies[currBox]

            # Push `reachedClosedBoxes` by `key` obtained in this turn and change
            # their statuses.
            for key in keys[currBox]:
                if not status[key] and reachedClosedBoxes[key]:
                    q.append(key)
                    status[key] = 1 # boxes[key] is now open

            # Push the boxes contained in `currBox`.
            pushBoxesIfPossible(containedBoxes[currBox])

        return ans

```

279. 1299.py

Path: LeetCode-main/solutions/1299. Replace Elements with Greatest Element on Right Side/1299.py

```

class Solution:
    def replaceElements(self, arr: list[int]) -> list[int]:
        maxOfRight = -1
        for i in reversed(range(len(arr))):
            arr[i], maxOfRight = maxOfRight, max(maxOfRight, arr[i])
        return arr

```

280. 13.py

Path: LeetCode-main/solutions/13. Roman to Integer/13.py

```

class Solution:
    def romanToInt(self, s: str) -> int:
        ans = 0
        roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50,
                  'C': 100, 'D': 500, 'M': 1000}

        for a, b in zip(s, s[1:]):
            if roman[a] < roman[b]:
                ans -= roman[a]
            else:
                ans += roman[a]

        return ans + roman[s[-1]]

```

281. 130-2.py

Path: LeetCode-main/solutions/130. Surrounded Regions/130-2.py

```

class Solution:
    def solve(self, board: list[list[str]]) -> None:
        if not board:
            return

        m = len(board)
        n = len(board[0])

        def dfs(i: int, j: int) -> None:
            """Marks the grids with 'O' that stretch from the four sides to '*'."""
            if i < 0 or i == m or j < 0 or j == n:
                return
            if board[i][j] != 'O':
                return
            board[i][j] = '*'
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        for i in range(m):
            for j in range(n):
                if i * j == 0 or i == m - 1 or j == n - 1:
                    dfs(i, j)

        for row in board:
            for i, c in enumerate(row):
                row[i] = 'O' if c == '*' else 'X'

```

282. 130.py

Path: LeetCode-main/solutions/130. Surrounded Regions/130.py

```

class Solution:
    def solve(self, board: list[list[str]]) -> None:
        if not board:
            return

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(board)
        n = len(board[0])
        q = collections.deque()

        for i in range(m):
            for j in range(n):
                if i * j == 0 or i == m - 1 or j == n - 1:
                    if board[i][j] == 'O':
                        q.append((i, j))
                        board[i][j] = '*'

        # Mark the grids that stretch from the four sides with '*'.
        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if board[x][y] != 'O':
                    continue
                q.append((x, y))
                board[x][y] = '*'

        for row in board:
            for i, c in enumerate(row):
                row[i] = 'O' if c == '*' else 'X'

```

283. 1300.py

Path: LeetCode-main/solutions/1300. Sum of Mutated Array Closest to Target/1300.py

```

class Solution:
    def findBestValue(self, arr: list[int], target: int) -> int:
        prefix = 0

        arr.sort()

        for i, a in enumerate(arr):
            ans = round((target - prefix) / (len(arr) - i))
            if ans <= a:
                return ans
            prefix += a

        return arr[-1]

```

284. 1301.py

Path: LeetCode-main/solutions/1301. Number of Paths with Max Score/1301.py

```

class Solution:
    def pathsWithMaxScore(self, board: list[str]) -> list[int]:
        MOD = 1_000_000_007
        DIRS = ((0, 1), (1, 0), (1, 1))
        n = len(board)
        # dp[i][j] := the maximum sum from (n - 1, n - 1) to (i, j)
        dp = [[-1] * (n + 1) for _ in range(n + 1)]
        # count[i][j] := the number of paths to get dp[i][j] from (n - 1, n - 1) to
        # (i, j)
        count = [[0] * (n + 1) for _ in range(n + 1)]

        dp[0][0] = 0
        dp[n - 1][n - 1] = 0
        count[n - 1][n - 1] = 1

        for i in reversed(range(n)):
            for j in reversed(range(n)):
                if board[i][j] == 'S' or board[i][j] == 'X':
                    continue
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if dp[i][j] < dp[x][y]:
                        dp[i][j] = dp[x][y]
                        count[i][j] = count[x][y]
                    elif dp[i][j] == dp[x][y]:
                        count[i][j] += count[x][y]
                    count[i][j] %= MOD

                # If there's path(s) from 'S' to (i, j) and the cell is not 'E'.
                if dp[i][j] != -1 and board[i][j] != 'E':
                    dp[i][j] += int(board[i][j])
                    dp[i][j] %= MOD

        return [dp[0][0], count[0][0]]

```

285. 1304.py

Path: LeetCode-main/solutions/1304. Find N Unique Integers Sum up to Zero/1304.py

```

class Solution:
    def sumZero(self, n: int) -> list[int]:
        return list(range(1 - n, n, 2))

```

286. 1307.py

Path: LeetCode-main/solutions/1307. Verbal Arithmetic Puzzle/1307.py

```

class Solution:
    def isSolvable(self, words: list[str], result: str) -> bool:
        words.append(result)
        rows = len(words)
        cols = max(map(len, words))
        letterToDigit = {}
        usedDigit = [False] * 10

        def dfs(row: int, col: int, summ: int) -> bool:
            if col == cols:
                return summ == 0
            if row == rows:
                return summ % 10 == 0 and dfs(0, col + 1, summ // 10)

            word = words[row]
            if col >= len(word):
                return dfs(row + 1, col, summ)

            letter = word[~col]
            sign = -1 if row == rows - 1 else 1

            if letter in letterToDigit and (
                letterToDigit[letter] > 0 or col < len(word) - 1):
                return dfs(row + 1, col, summ + sign * letterToDigit[letter])

            for digit, used in enumerate(usedDigit):
                if not used and (digit > 0 or col < len(word) - 1):
                    letterToDigit[letter] = digit
                    usedDigit[digit] = True
                    if dfs(row + 1, col, summ + sign * digit):
                        return True
                    usedDigit[digit] = False
                    if letter in letterToDigit:
                        del letterToDigit[letter]

            return False

        return dfs(0, 0, 0)

```

287. 1309.py

Path: LeetCode-main/solutions/1309. Decrypt String from Alphabet to Integer Mapping/1309.py

```

class Solution:
    def freqAlphabets(self, s: str) -> str:
        ans = ''
        i = 0

        while i < len(s):
            if i + 2 < len(s) and s[i + 2] == '#':
                ans += chr(int(s[i:i + 2]) + ord('a') - 1)
                i += 3
            else:
                ans += chr(int(s[i]) + ord('a') - 1)
                i += 1

        return ans

```

288. 131.py

Path: LeetCode-main/solutions/131. Palindrome Partitioning/131.py

```
class Solution:
    def partition(self, s: str) -> list[list[str]]:
        ans = []

        def isPalindrome(s: str) -> bool:
            return s == s[::-1]

        def dfs(s: str, j: int, path: list[str], ans: list[list[str]]) -> None:
            if j == len(s):
                ans.append(path)
                return

            for i in range(j, len(s)):
                if isPalindrome(s[j: i + 1]):
                    dfs(s, i + 1, path + [s[j: i + 1]], ans)

        dfs(s, 0, [], ans)
        return ans
```

289. 1310.py

Path: LeetCode-main/solutions/1310. XOR Queries of a Subarray/1310.py

```
class Solution:
    def xorQueries(self, arr: list[int], queries: list[list[int]]) -> list[int]:
        ans = []
        xors = [0] * (len(arr) + 1)

        for i, a in enumerate(arr):
            xors[i + 1] = xors[i] ^ a

        for left, right in queries:
            ans.append(xors[left] ^ xors[right + 1])

        return ans
```

290. 1311.py

Path: LeetCode-main/solutions/1311. Get Watched Videos by Your Friends/1311.py


```

class Solution:
    def watchedVideosByFriends(
        self,
        watchedVideos: list[list[str]],
        friends: list[list[int]],
        id: int,
        level: int,
    ) -> list[str]:
        seen = [False] * 100
        seen[id] = True
        q = collections.deque([id])
        count = collections.Counter()

        for _ in range(level):
            for _ in range(len(q)):
                curr = q.popleft()
                for friend in friends[curr]:
                    if not seen[friend]:
                        seen[friend] = True
                        q.append(friend)

        for friend in q:
            for video in watchedVideos[friend]:
                count[video] += 1

        return sorted(count, key=lambda video: (count[video], video))

```

291. 1312.py

Path: LeetCode-main/solutions/1312. Minimum Insertion Steps to Make a String Palindrome/1312.py

```

class Solution:
    def minInsertions(self, s: str) -> int:
        return len(s) - self._longestPalindromeSubseq(s)

    # Same as 516. Longest Palindromic Subsequence
    def _longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        # dp[i][j] := the length of LPS(s[i..j])
        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                if s[i] == s[j]:
                    dp[i][j] = 2 + dp[i + 1][j - 1]
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

        return dp[0][n - 1]

```

292. 1313.py

Path: LeetCode-main/solutions/1313. Decompress Run-Length Encoded List/1313.py

```
class Solution:
    def decompressRLElist(self, nums: list[int]) -> list[int]:
        ans = []

        for i in range(0, len(nums), 2):
            ans += [nums[i + 1]] * nums[i]

        return ans
```

293. 1314.py

Path: LeetCode-main/solutions/1314. Matrix Block Sum/1314.py

```
class Solution:
    def matrixBlockSum(self, mat: list[list[int]], k: int) -> list[list[int]]:
        m = len(mat)
        n = len(mat[0])
        ans = [[0] * n for _ in range(m)]
        prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                prefix[i + 1][j + 1] = (mat[i][j] + prefix[i][j + 1] +
                                         prefix[i + 1][j] - prefix[i][j])

        for i in range(m):
            for j in range(n):
                r1 = max(0, i - k) + 1
                c1 = max(0, j - k) + 1
                r2 = min(m - 1, i + k) + 1
                c2 = min(n - 1, j + k) + 1
                ans[i][j] = (prefix[r2][c2] - prefix[r2][c1 - 1] -
                             prefix[r1 - 1][c2] + prefix[r1 - 1][c1 - 1])

        return ans
```

294. 1316.py

Path: LeetCode-main/solutions/1316. Distinct Echo Substrings/1316.py

```

class Solution:
    def distinctEchoSubstrings(self, text: str) -> int:
        seen = set()

        for k in range(1, len(text) // 2 + 1): # the target length
            same = 0
            l = 0
            for r in range(k, len(text)):
                if text[l] == text[r]:
                    same += 1
                else:
                    same = 0
            if same == k:
                seen.add(text[l - k + 1:l + 1])
                # Move the window thus leaving a letter behind, so we need to
                # decrease the counter.
                same -= 1
                l += 1

        return len(seen)

```

295. 1317.py

Path: LeetCode-main/solutions/1317. Convert Integer to the Sum of Two No-Zero Integers/1317.py

```

class Solution:
    def getNoZeroIntegers(self, n: int) -> list[int]:
        for A in range(n):
            B = n - A
            if '0' not in str(A) and '0' not in str(B):
                return A, B

```

296. 1318.py

Path: LeetCode-main/solutions/1318. Minimum Flips to Make a OR b Equal to c/1318.py

```

class Solution:
    def minFlips(self, a: int, b: int, c: int) -> int:
        MAX_BIT = 30
        ans = 0

        for i in range(MAX_BIT):
            if c >> i & 1:
                ans += (a >> i & 1) == 0 and (b >> i & 1) == 0
            else: # (c >> i & 1) == 0
                ans += (a >> i & 1) + (b >> i & 1)

        return ans

```

297. 132.py

Path: LeetCode-main/solutions/132. Palindrome Partitioning II/132.py

```

class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        # isPalindrome[i][j] := True if s[i..j] is a palindrome
        isPalindrome = [[True] * n for _ in range(n)]
        # dp[i] := the minimum cuts needed for a palindrome partitioning of s[0..i]
        dp = [n] * n

        for l in range(2, n + 1):
            i = 0
            for j in range(l - 1, n):
                isPalindrome[i][j] = s[i] == s[j] and isPalindrome[i + 1][j - 1]
                i += 1

        for i in range(n):
            if isPalindrome[0][i]:
                dp[i] = 0
                continue

            # Try all the possible partitions.
            for j in range(i):
                if isPalindrome[j + 1][i]:
                    dp[i] = min(dp[i], dp[j] + 1)

        return dp[-1]

```

298. 1320.py

Path: LeetCode-main/solutions/1320. Minimum Distance to Type a Word Using Two Fingers/1320.py

```

class Solution:
    def minimumDistance(self, word: str) -> int:
        def dist(a: int, b: int) -> int:
            if a == 26: # the first hovering state
                return 0
            x1, y1 = a // 6, a % 6
            x2, y2 = b // 6, b % 6
            return abs(x1 - x2) + abs(y1 - y2)

        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> int:
            """
            Returns the minimum distance to type the `word`, where the left finger is
            on the i-th letter, the right finger is on the j-th letter, and the
            words[0..k) have been written.
            """
            if k == len(word):
                return 0
            nxt = ord(word[k]) - ord('A')
            moveLeft = dist(i, nxt) + dp(nxt, j, k + 1)
            moveRight = dist(j, nxt) + dp(i, nxt, k + 1)
            return min(moveLeft, moveRight)

        return dp(26, 26, 0)

```

299. 1323.py

Path: LeetCode-main/solutions/1323. Maximum 69 Number/1323.py

```
class Solution:
    def maximum69Number(self, num: int) -> int:
        return int(str(num).replace('6', '9', 1))
```

300. 1324.py

Path: LeetCode-main/solutions/1324. Print Words Vertically/1324.py

```
class Solution:
    def printVertically(self, s: str) -> list[str]:
        ans = []
        words = s.split()
        maxLength = max(len(word) for word in words)

        for i in range(maxLength):
            row = []
            for word in words:
                row.append(word[i] if i < len(word) else ' ')
            ans.append(''.join(row).rstrip())

        return ans
```

301. 1325.py

Path: LeetCode-main/solutions/1325. Delete Leaves With a Given Value/1325.py

```
class Solution:
    def removeLeafNodes(
        self,
        root: TreeNode | None,
        target: int,
    ) -> TreeNode | None:
        if not root:
            return None
        root.left = self.removeLeafNodes(root.left, target)
        root.right = self.removeLeafNodes(root.right, target)
        return None if self._isLeaf(root) and root.val == target else root

    def _isLeaf(self, root: TreeNode | None) -> bool:
        return not root.left and not root.right
```

302. 1326.py

Path: LeetCode-main/solutions/1326. Minimum Number of Taps to Open to Water a Garden/1326.py

```
class Solution:
    def minTaps(self, n: int, ranges: list[int]) -> int:
        nums = [0] * (n + 1)

        for i, range_ in enumerate(ranges):
            l = max(0, i - range_)
            r = min(n, range_ + i)
            nums[l] = max(nums[l], r - l)

        ans = 0
        end = 0
        farthest = 0

        for i in range(n):
            farthest = max(farthest, i + nums[i])
            if i == end:
                ans += 1
                end = farthest

        return ans if end == n else -1
```

303. 1328.py

Path: LeetCode-main/solutions/1328. Break a Palindrome/1328.py

```
class Solution:
    def breakPalindrome(self, palindrome: str) -> str:
        if len(palindrome) == 1:
            return ''

        ans = list(palindrome)

        for i in range(len(palindrome) // 2):
            if palindrome[i] != 'a':
                ans[i] = 'a'
                return ''.join(ans)

        ans[-1] = 'b'
        return ''.join(ans)
```

304. 1329.py

Path: LeetCode-main/solutions/1329. Sort the Matrix Diagonally/1329.py

```
class Solution:
    def diagonalSort(self, mat: list[list[int]]) -> list[list[int]]:
        m = len(mat)
        n = len(mat[0])

        count = collections.defaultdict(list)

        for i in range(m):
            for j in range(n):
                count[i - j].append(mat[i][j])

        for value in count.values():
            value.sort(reverse=1)

        for i in range(m):
            for j in range(n):
                mat[i][j] = count[i - j].pop()

        return mat
```

305. 133-2.py

Path: LeetCode-main/solutions/133. Clone Graph/133-2.py

```
class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
        if node in self.map:
            return self.map[node]

        newNode = Node(node.val, [])
        self.map[node] = newNode

        for neighbor in node.neighbors:
            self.map[node].neighbors.append(self.cloneGraph(neighbor))

        return newNode

    map = {}
```

306. 133.py

Path: LeetCode-main/solutions/133. Clone Graph/133.py

```

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None

        q = collections.deque([node])
        map = {node: Node(node.val)}

        while q:
            u = q.popleft()
            for v in u.neighbors:
                if v not in map:
                    map[v] = Node(v.val)
                    q.append(v)
                map[u].neighbors.append(map[v])

        return map[node]

```

307. 1330.py

Path: LeetCode-main/solutions/1330. Reverse Subarray To Maximize Array Value/1330.py

```

class Solution:
    def maxValueAfterReverse(self, nums: list[int]) -> int:
        mn = math.inf
        mx = -math.inf

        for a, b in zip(nums, nums[1:]):
            mn = min(mn, max(a, b))
            mx = max(mx, min(a, b))
            diff = max(0, (mx - mn) * 2)

        for a, b in zip(nums, nums[1:]):
            headDiff = -abs(a - b) + abs(nums[0] - b)
            tailDiff = -abs(a - b) + abs(nums[-1] - a)
            diff = max(diff, headDiff, tailDiff)

        return sum(abs(a - b) for a, b in zip(nums, nums[1:])) + diff

```

308. 1331.py

Path: LeetCode-main/solutions/1331. Rank Transform of an Array/1331.py

```

class Solution:
    def arrayRankTransform(self, arr: list[int]) -> list[int]:
        rank = {}

        for a in sorted(arr):
            if a not in rank:
                rank[a] = len(rank) + 1

        return map(rank.get, arr)

```

309. 1332.py

Path: LeetCode-main/solutions/1332. Remove Palindromic Subsequences/1332.py


```
class Solution:
    def removePalindromeSub(self, s: str) -> int:
        return 1 if s == s[::-1] else 2
```

310. 1333.py

Path: LeetCode-main/solutions/1333. Filter Restaurants by Vegan-Friendly, Price and Distance/1333.py

```
class Solution:
    def filterRestaurants(
        self,
        restaurants: list[list[int]],
        veganFriendly: int,
        maxPrice: int,
        maxDistance: int,
    ) -> list[int]:
        restaurants.sort(key=lambda x: (-x[1], -x[0]))
        return [i for i, _, v, p, d in restaurants
                if v >= veganFriendly and p <= maxPrice and d <= maxDistance]
```

311. 1334.py

Path: LeetCode-main/solutions/1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance/1334.py

```

class Solution:
    def findTheCity(
        self,
        n: int,
        edges: list[list[int]],
        distanceThreshold: int,
    ) -> int:
        ans = -1
        minCitiesCount = n
        dist = self._floydWarshall(n, edges, distanceThreshold)

        for i in range(n):
            citiesCount = sum(dist[i][j] <= distanceThreshold for j in range(n))
            if citiesCount <= minCitiesCount:
                ans = i
                minCitiesCount = citiesCount

        return ans

    def _floydWarshall(
        self,
        n: int,
        edges: list[list[int]],
        distanceThreshold: int,
    ) -> list[list[int]]:
        dist = [[distanceThreshold + 1] * n for _ in range(n)]

        for i in range(n):
            dist[i][i] = 0

        for u, v, w in edges:
            dist[u][v] = w
            dist[v][u] = w

        for k in range(n):
            for i in range(n):
                for j in range(n):
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

        return dist

```

312. 1335.py

Path: LeetCode-main/solutions/1335. Minimum Difficulty of a Job Schedule/1335.py

```

class Solution:
    def minDifficulty(self, jobDifficulty: list[int], d: int) -> int:
        n = len(jobDifficulty)
        if d > n:
            return -1

        # dp[i][k] := the minimum difficulty to schedule the first i jobs in k days
        dp = [[math.inf] * (d + 1) for _ in range(n + 1)]
        dp[0][0] = 0

        for i in range(1, n + 1):
            for k in range(1, d + 1):
                maxDifficulty = 0 # max(job[j + 1..i])
                for j in range(i - 1, k - 2, -1): # 1-based
                    maxDifficulty = max(maxDifficulty, jobDifficulty[j]) # 0-based
                dp[i][k] = min(dp[i][k], dp[j][k - 1] + maxDifficulty)

        return dp[n][d]

```

313. 1337.py

Path: LeetCode-main/solutions/1337. The K Weakest Rows in a Matrix/1337.py

```

class Solution:
    def kWeakestRows(self, mat: list[list[int]], k: int) -> list[int]:
        rowSums = [(sum(row), i) for i, row in enumerate(mat)]
        return [i for _, i in sorted(rowSums)[:k]]

```

314. 1338.py

Path: LeetCode-main/solutions/1338. Reduce Array Size to The Half/1338.py

```

class Solution:
    def minSetSize(self, arr: list[int]) -> int:
        count = collections.Counter(arr).most_common()
        count.sort(key=lambda x: -x[1])

        summ = 0
        for i, (_, freq) in enumerate(count):
            summ += freq
            if summ >= len(arr) // 2:
                return i + 1

```

315. 134.py

Path: LeetCode-main/solutions/134. Gas Station/134.py

```

class Solution:
    def canCompleteCircuit(self, gas: list[int], cost: list[int]) -> int:
        ans = 0
        net = 0
        summ = 0

        # Try to start from each index.
        for i in range(len(gas)):
            net += gas[i] - cost[i]
            summ += gas[i] - cost[i]
            if summ < 0:
                summ = 0
                ans = i + 1 # Start from the next index.

        return -1 if net < 0 else ans

```

316. 1340.py

Path: LeetCode-main/solutions/1340. Jump Game V/1340.py

```

class Solution:
    def maxJumps(self, arr: list[int], d: int) -> int:
        n = len(arr)
        # dp[i] := the maximum jumps starting from arr[i]
        dp = [1] * n
        # a decreasing stack that stores indices
        stack = []

        for i in range(n + 1):
            while stack and (i == n or arr[stack[-1]] < arr[i]):
                indices = [stack.pop()]
                while stack and arr[stack[-1]] == arr[indices[0]]:
                    indices.append(stack.pop())
                for j in indices:
                    if i < n and i - j <= d:
                        # Can jump from i to j.
                        dp[i] = max(dp[i], dp[j] + 1)
                if stack and j - stack[-1] <= d:
                    # Can jump from stack[-1] to j
                    dp[stack[-1]] = max(dp[stack[-1]], dp[j] + 1)
            stack.append(i)

        return max(dp)

```

317. 1342.py

Path: LeetCode-main/solutions/1342. Number of Steps to Reduce a Number to Zero/1342.py

```

class Solution:
    def numberOfSteps(self, num: int) -> int:
        if num == 0:
            return 0
        subtractSteps = num.bit_count()
        divideSteps = num.bit_length() - 1
        return subtractSteps + divideSteps

```

318. 1343.py

Path: LeetCode-main/solutions/1343. Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold/1343.py

```
class Solution:
    def numOfSubarrays(self, arr: list[int], k: int, threshold: int) -> int:
        ans = 0
        windowSum = 0

        for i in range(len(arr)):
            windowSum += arr[i]
            if i >= k:
                windowSum -= arr[i - k]
            if i >= k - 1 and windowSum // k >= threshold:
                ans += 1

        return ans
```

319. 1344.py

Path: LeetCode-main/solutions/1344. Angle Between Hands of a Clock/1344.py

```
class Solution:
    def angleClock(self, hour: int, minutes: int) -> float:
        hourAngle = (hour % 12) * 30 + minutes * 0.5
        minuteAngle = minutes * 6
        ans = abs(hourAngle - minuteAngle)

        return min(ans, 360 - ans)
```

320. 1345.py

Path: LeetCode-main/solutions/1345. Jump Game IV/1345.py

```

class Solution:
    def minJumps(self, arr: list[int]) -> int:
        n = len(arr)
        # {num: indices}
        graph = collections.defaultdict(list)
        step = 0
        q = collections.deque([0])
        seen = {0}

        for i, a in enumerate(arr):
            graph[a].append(i)

        while q:
            for _ in range(len(q)):
                i = q.popleft()
                if i == n - 1:
                    return step
                seen.add(i)
                u = arr[i]
                if i + 1 < n:
                    graph[u].append(i + 1)
                if i - 1 >= 0:
                    graph[u].append(i - 1)
                for v in graph[u]:
                    if v in seen:
                        continue
                    q.append(v)
                graph[u].clear()
            step += 1

```

321. 1346.py

Path: LeetCode-main/solutions/1346. Check If N and Its Double Exist/1346.py

```

class Solution:
    def checkIfExist(self, arr: list[int]) -> bool:
        seen = set()

        for a in arr:
            if a * 2 in seen or a % 2 == 0 and a // 2 in seen:
                return True
            seen.add(a)

        return False

```

322. 1347.py

Path: LeetCode-main/solutions/1347. Minimum Number of Steps to Make Two Strings Anagram/1347.py

```

class Solution:
    def minSteps(self, s: str, t: str) -> int:
        count = collections.Counter(s)
        count.subtract(collections.Counter(t))
        return sum(abs(value) for value in count.values()) // 2

```

323. 1348.py

Path: LeetCode-main/solutions/1348. Tweet Counts Per Frequency/1348.py

```
from sortedcontainers import SortedList

class TweetCounts:
    def __init__(self):
        self.tweetNameToTimes = collections.defaultdict(SortedList)

    def recordTweet(self, tweetName: str, time: int) -> None:
        self.tweetNameToTimes[tweetName].add(time)

    def getTweetCountsPerFrequency(self, freq: str, tweetName: str,
                                   startTime: int, endTime: int) -> list[int]:
        counts = []
        times = self.tweetNameToTimes[tweetName]
        chunk = 60 if freq == 'minute' else 3600 if freq == 'hour' else 86400

        # i := startTime of each chunk
        for i in range(startTime, endTime + 1, chunk):
            j = min(i + chunk, endTime + 1) # EndTime of each chunk
            counts.append(bisect_left(times, j) - bisect_left(times, i))

        return counts
```

324. 1349.py

Path: LeetCode-main/solutions/1349. Maximum Students Taking Exam/1349.py

```

class Solution:
    def maxStudents(self, seats: list[list[str]]) -> int:
        m = len(seats)
        n = len(seats[0])
        DIRS = ((-1, -1), (0, -1), (1, -1), (-1, 1), (0, 1), (1, 1))
        seen = [[0] * n for _ in range(m)]
        match = [[-1] * n for _ in range(m)]

        def dfs(i: int, j: int, sessionId: int) -> int:
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if seats[x][y] != '.' or seen[x][y] == sessionId:
                    continue
                seen[x][y] = sessionId
                if match[x][y] == -1 or dfs(*divmod(match[x][y], n), sessionId):
                    match[x][y] = i * n + j
                    match[i][j] = x * n + y
                    return 1
            return 0

        def hungarian() -> int:
            count = 0
            for i in range(m):
                for j in range(n):
                    if seats[i][j] == '.' and match[i][j] == -1:
                        sessionId = i * n + j
                        seen[i][j] = sessionId
                        count += dfs(i, j, sessionId)
            return count

        return sum(seats[i][j] == '.'
                    for i in range(m)
                    for j in range(n)) - hungarian()

```

325. 135.py

Path: LeetCode-main/solutions/135. Candy/135.py

```

class Solution:
    def candy(self, ratings: list[int]) -> int:
        n = len(ratings)

        ans = 0
        l = [1] * n
        r = [1] * n

        for i in range(1, n):
            if ratings[i] > ratings[i - 1]:
                l[i] = l[i - 1] + 1

        for i in range(n - 2, -1, -1):
            if ratings[i] > ratings[i + 1]:
                r[i] = r[i + 1] + 1

        for a, b in zip(l, r):
            ans += max(a, b)

        return ans

```

326. 1351.py

Path: LeetCode-main/solutions/1351. Count Negative Numbers in a Sorted Matrix/1351.py

```
class Solution:
    def countNegatives(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = 0
        i = m - 1
        j = 0

        while i >= 0 and j < n:
            if grid[i][j] < 0:
                ans += n - j
                i -= 1
            else:
                j += 1

        return ans
```

327. 1352.py

Path: LeetCode-main/solutions/1352. Product of the Last K Numbers/1352.py

```
class ProductOfNumbers:
    def __init__(self):
        self.prefix = [1]

    def add(self, num: int) -> None:
        if num == 0:
            self.prefix = [1]
        else:
            self.prefix.append(self.prefix[-1] * num)

    def getProduct(self, k: int) -> int:
        return 0 if k >= len(self.prefix) else self.prefix[-1] // self.prefix[len(self.prefix) - k - 1]
```

328. 1353.py

Path: LeetCode-main/solutions/1353. Maximum Number of Events That Can Be Attended/1353.py

```

class Solution:
    def maxEvents(self, events: list[list[int]]) -> int:
        ans = 0
        minHeap = []
        i = 0 # events' index

        events.sort(key=lambda x: x[0])

        while minHeap or i < len(events):
            # If no events are available to attend today, let time flies to the next
            # available event.
            if not minHeap:
                d = events[i][0]
            # All the events starting from today are newly available.
            while i < len(events) and events[i][0] == d:
                heapq.heappush(minHeap, events[i][1])
                i += 1
            # Greedily attend the event that'll end the earliest since it has higher
            # chance can't be attended in the future.
            heapq.heappop(minHeap)
            ans += 1
            d += 1
            # Pop the events that can't be attended.
            while minHeap and minHeap[0] < d:
                heapq.heappop(minHeap)

        return ans

```

329. 1354.py

Path: LeetCode-main/solutions/1354. Construct Target Array With Multiple Sums/1354.py

```

class Solution:
    def isPossible(self, target: list[int]) -> bool:
        if len(target) == 1:
            return target[0] == 1

        summ = sum(target)
        maxHeap = [-num for num in target]
        heapq.heapify(maxHeap)

        while -maxHeap[0] > 1:
            mx = -heapq.heappop(maxHeap)
            restSum = summ - mx
            # Only occurs if n == 2.
            if restSum == 1:
                return True
            updated = mx % restSum
            # updated == 0 (invalid) or didn't change.
            if updated == 0 or updated == mx:
                return False
            heapq.heappush(maxHeap, -updated)
            summ = summ - mx + updated

        return True

```

330. 1356.py

Path: LeetCode-main/solutions/1356. Sort Integers by The Number of 1 Bits/1356.py

```
class Solution:
    def sortByBits(self, arr: list[int]) -> list[int]:
        return sorted(arr, key=lambda x: (x.bit_count(), x))
```

331. 1357.py

Path: LeetCode-main/solutions/1357. Apply Discount Every n Orders/1357.py

```
class Cashier:
    def __init__(
        self,
        n: int,
        discount: int,
        products: list[int],
        prices: list[int],
    ):
        self.n = n
        self.discount = discount
        self.productToPrice = dict(zip(products, prices))
        self.count = 0

    def getBill(self, product: list[int], amount: list[int]) -> float:
        self.count += 1
        total = sum(self.productToPrice[p] * amount[i]
                     for i, p in enumerate(product))
        if self.count % self.n == 0:
            return total * (1 - self.discount / 100)
        return total
```

332. 1358-2.py

Path: LeetCode-main/solutions/1358. Number of Substrings Containing All Three Characters/1358-2.py

```
class Solution:
    # Similar to 3. Longest SubWithout Repeating Characters
    def numberOfSubstrings(self, s: str) -> int:
        ans = 0
        # lastSeen[c] := the index of the last time c appeared
        lastSeen = {c: -1 for c in 'abc'}

        for i, c in enumerate(s):
            lastSeen[c] = i
            # s[0..i], s[1..i], s[min(lastSeen)..i] are satisfied strings.
            ans += 1 + min(lastSeen.values())

        return ans
```

333. 1358.py

Path: LeetCode-main/solutions/1358. Number of Substrings Containing All Three Characters/1358.py

```

class Solution:
    # Similar to 3. Longest SubWithout Repeating Characters
    def numberOfSubstrings(self, s: str) -> int:
        ans = 0
        count = {c: 0 for c in 'abc'}

        l = 0
        for c in s:
            count[c] += 1
            while min(count.values()) > 0:
                count[s[l]] -= 1
                l += 1
            # s[0..r], s[1..r], ..., s[l - 1..r] are satisfied strings.
            ans += 1

        return ans

```

334. 1359.py

Path: LeetCode-main/solutions/1359. Count All Valid Pickup and Delivery Options/1359.py

```

class Solution:
    def countOrders(self, n: int) -> int:
        MOD = 1_000_000_007
        ans = 1

        for i in range(1, n + 1):
            ans = ans * i * (i * 2 - 1) % MOD

        return ans

```

335. 136.py

Path: LeetCode-main/solutions/136. Single Number/136.py

```

class Solution:
    def singleNumber(self, nums: list[int]) -> int:
        return functools.reduce(operator.xor, nums, 0)

```

336. 1360.py

Path: LeetCode-main/solutions/1360. Number of Days Between Two Dates/1360.py

```

class Solution:
    def daysBetweenDates(self, date1: str, date2: str) -> int:
        days = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

        def isLeapYear(year: int) -> bool:
            return year % 4 == 0 and year % 100 != 0 or year % 400 == 0

        def daysFrom1971(date: str) -> int:
            year, month, day = map(int, date.split('-'))
            return (365 * (year - 1971) + sum(map(isLeapYear, range(1971, year))) +
                    sum(days[:month]) + day + (month > 2 and isLeapYear(year)))

        return abs(daysFrom1971(date1) - daysFrom1971(date2))

```

337. 1362.py

Path: LeetCode-main/solutions/1362. Closest Divisors/1362.py

```
class Solution:
    def closestDivisors(self, num: int) -> list[int]:
        for root in reversed(range(math.isqrt(num + 2) + 1)):
            for cand in [num + 1, num + 2]:
                if cand % root == 0:
                    return [root, cand // root]
```

338. 1363.py

Path: LeetCode-main/solutions/1363. Largest Multiple of Three/1363.py

```
class Solution:
    def largestMultipleOfThree(self, digits: list[int]) -> str:
        ans = ''
        mod1 = [1, 4, 7, 2, 5, 8]
        mod2 = [2, 5, 8, 1, 4, 7]
        count = collections.Counter(digits)
        summ = sum(digits)

        while summ % 3 != 0:
            for digit in (mod1 if summ % 3 == 1 else mod2):
                if count[digit]:
                    count[digit] -= 1
                    summ -= digit
                    break

        for digit in reversed(range(10)):
            ans += str(digit) * count[digit]

        return '0' if len(ans) and ans[0] == '0' else ans
```

339. 1365.py

Path: LeetCode-main/solutions/1365. How Many Numbers Are Smaller Than the Current Number/1365.py

```
class Solution:
    def smallerNumbersThanCurrent(self, nums: list[int]) -> list[int]:
        MAX = 100
        count = collections.Counter(nums)

        for i in range(1, MAX + 1):
            count[i] += count[i - 1]

        return [0 if num == 0 else count[num - 1]
                for num in nums]
```

340. 1366.py

Path: LeetCode-main/solutions/1366. Rank Teams by Votes/1366.py

```

from dataclasses import dataclass

@dataclass
class Team:
    name: str
    rank: list[int]

    def __init__(self, name: str, teamSize: int):
        self.name = name
        self.rank = [0] * teamSize

class Solution:
    def rankTeams(self, votes: list[str]) -> str:
        teamSize = len(votes[0])
        teams = [Team(chr(ord('A') + i), teamSize) for i in range(26)]

        for vote in votes:
            for i in range(teamSize):
                teams[ord(vote[i]) - ord('A')].rank[i] += 1

        teams.sort(key=lambda x: (x.rank, -ord(x.name)), reverse=True)
        return ''.join(team.name for team in teams[:teamSize])

```

341. 1367.py

Path: LeetCode-main/solutions/1367. Linked List in Binary Tree/1367.py

```

class Solution:
    def isSubPath(self, head: ListNode | None, root: TreeNode | None) -> bool:
        if not root:
            return False
        return (self._isContinuousSubPath(head, root) or
                self.isSubPath(head, root.left) or
                self.isSubPath(head, root.right))

    def _isContinuousSubPath(
        self,
        head: ListNode | None,
        root: TreeNode | None,
    ) -> bool:
        if not head:
            return True
        if not root:
            return False
        return (head.val == root.val and
                (self._isContinuousSubPath(head.next, root.left) or
                 self._isContinuousSubPath(head.next, root.right)))

```

342. 1368.py

Path: LeetCode-main/solutions/1368. Minimum Cost to Make at Least One Valid Path in a Grid/1368.py

```

class Solution:
    def minCost(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        DIRS = ((0, 1), (0, -1), (1, 0), (-1, 0))
        dp = [[-1] * n for _ in range(m)]
        q = collections.deque()

        def dfs(i: int, j: int, cost: int) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if dp[i][j] != -1:
                return
            dp[i][j] = cost
            q.append((i, j))
            dx, dy = DIRS[grid[i][j] - 1]
            dfs(i + dx, j + dy, cost)

        dfs(0, 0, 0)

        cost = 0
        while q:
            cost += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for dx, dy in DIRS:
                    dfs(i + dx, j + dy, cost)

        return dp[-1][-1]

```

343. 137-2.py

Path: LeetCode-main/solutions/137. Single Number II/137-2.py

```

class Solution:
    def singleNumber(self, nums: list[int]) -> int:
        ones = 0
        twos = 0

        for num in nums:
            ones ^= (num & ~twos)
            twos ^= (num & ~ones)

        return ones

```

344. 137.py

Path: LeetCode-main/solutions/137. Single Number II/137.py

```

class Solution:
    def singleNumber(self, nums: list[int]) -> int:
        ones = 0
        twos = 0

        for num in nums:
            ones ^= num & ~twos
            twos ^= num & ~ones

        return ones

```

345. 1370.py

Path: LeetCode-main/solutions/1370. Increasing Decreasing String/1370.py

```
class Solution:
    def sortString(self, s: str) -> str:
        ans = []
        count = collections.Counter(s)

        while count:
            for chars in string.ascii_lowercase, reversed(string.ascii_lowercase):
                ans += [c for c in chars if c in count]
                count -= dict.fromkeys(count, 1)

        return ''.join(ans)
```

346. 1371.py

Path: LeetCode-main/solutions/1371. Find the Longest Substring Containing Vowels in Even Counts/1371.py

```
class Solution:
    def findTheLongestSubstring(self, s: str) -> int:
        VOWELS = 'aeiou'
        ans = 0
        prefix = 0 # the binary prefix
        prefixToIndex = {0: -1}

        for i, c in enumerate(s):
            index = VOWELS.find(c)
            if index != -1:
                prefix ^= 1 << index
            prefixToIndex.setdefault(prefix, i)
            ans = max(ans, i - prefixToIndex[prefix])

        return ans
```

347. 1372.py

Path: LeetCode-main/solutions/1372. Longest ZigZag Path in a Binary Tree/1372.py


```
from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    leftMax: int
    rightMax: int
    subtreeMax: int

class Solution:
    def longestZigZag(self, root: TreeNode | None) -> int:
        def dfs(root: TreeNode | None) -> T:
            if not root:
                return T(-1, -1, -1)
            left = dfs(root.left)
            right = dfs(root.right)
            leftZigZag = left.rightMax + 1
            rightZigZag = right.leftMax + 1
            subtreeMax = max(leftZigZag, rightZigZag,
                             left.subtreeMax, right.subtreeMax)
            return T(leftZigZag, rightZigZag, subtreeMax)

        return dfs(root).subtreeMax
```

348. 1373.py

Path: LeetCode-main/solutions/1373. Maximum Sum BST in Binary Tree/1373.py

```

from dataclasses import dataclass

@dataclass
class T:
    isBST: bool | None = False
    mx: int | None = None
    mn: int | None = None
    summ: int | None = None

class Solution:
    def maxSumBST(self, root: TreeNode | None) -> int:
        self.ans = 0

        def traverse(root: TreeNode | None) -> T:
            if not root:
                return T(True, -math.inf, math.inf, 0)

            left: T = traverse(root.left)
            right: T = traverse(root.right)

            if not left.isBST or not right.isBST:
                return T()
            if root.val <= left.mx or root.val >= right.mn:
                return T()

            # The `root` is a valid BST.
            summ = root.val + left.summ + right.summ
            self.ans = max(self.ans, summ)
            return T(True, max(root.val, right.mx), min(root.val, left.mn), summ)

        traverse(root)
        return self.ans

```

349. 1374.py

Path: LeetCode-main/solutions/1374. Generate a String With Characters That Have Odd Counts/1374.py

```

class Solution:
    def generateTheString(self, n: int) -> str:
        s = 'a' * n
        if n % 2 == 0:
            s = s[:-1] + 'b'
        return s

```

350. 1375.py

Path: LeetCode-main/solutions/1375. Number of Times Binary String Is Prefix-Aligned/1375.py

```

class Solution:
    def numTimesAllBlue(self, flips: list[int]) -> int:
        ans = 0
        rightmost = 0

        for i, flip in enumerate(flips):
            rightmost = max(rightmost, flip)
            # max(flips[0..i]) = rightmost = i + 1,
            # so flips[0..i] is a permutation of 1, 2, ..., i + 1.
            if rightmost == i + 1:
                ans += 1

        return ans

```

351. 1377.py

Path: LeetCode-main/solutions/1377. Frog Position After T Seconds/1377.py

```

class Solution:
    def frogPosition(
        self,
        n: int,
        edges: list[list[int]],
        t: int,
        target: int,
    ) -> float:
        tree = [[] for _ in range(n + 1)]
        q = collections.deque([1])
        seen = [False] * (n + 1)
        prob = [0] * (n + 1)

        prob[1] = 1
        seen[1] = True

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        for _ in range(t):
            for _ in range(len(q)):
                a = q.popleft()
                nChildren = sum(not seen[b] for b in tree[a])
                for b in tree[a]:
                    if seen[b]:
                        continue
                    seen[b] = True
                    prob[b] = prob[a] / nChildren
                    q.append(b)
                if nChildren > 0:
                    prob[a] = 0

        return prob[target]

```

352. 1379.py

Path: LeetCode-main/solutions/1379. Find a Corresponding Node of a Binary Tree in a Clone of That Tree/1379.py

```

class Solution:
    def getTargetCopy(
        self,
        original: TreeNode,
        cloned: TreeNode,
        target: TreeNode,
    ) -> TreeNode:
        ans = None

        def dfs(original: TreeNode, cloned: TreeNode) -> None:
            nonlocal ans
            if ans:
                return
            if not original:
                return
            if original == target:
                ans = cloned
                return

            dfs(original.left, cloned.left)
            dfs(original.right, cloned.right)

        dfs(original, cloned)
        return ans

```

353. 138.py

Path: LeetCode-main/solutions/138. Copy List with Random Pointer/138.py

```

class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None
        if head in self.map:
            return self.map[head]

        newNode = Node(head.val)
        self.map[head] = newNode
        newNode.next = self.copyRandomList(head.next)
        newNode.random = self.copyRandomList(head.random)
        return newNode

        map = {}

```

354. 1380.py

Path: LeetCode-main/solutions/1380. Lucky Numbers in a Matrix/1380.py

```

class Solution:
    def luckyNumbers(self, matrix: list[list[int]]) -> list[int]:
        for row in matrix:
            minIndex = row.index(min(row))
            if row[minIndex] == max(list(zip(*matrix))[minIndex]):
                return [row[minIndex]]
        return []

```

355. 1381.py

Path: LeetCode-main/solutions/1381. Design a Stack With Increment Operation/1381.py

```
class CustomStack:
    def __init__(self, maxSize: int):
        self.maxSize = maxSize
        self.stack = []
        # pendingIncrements[i] := the pending increment for stack[0..i].
        self.pendingIncrements = []

    def push(self, x: int) -> None:
        if len(self.stack) == self.maxSize:
            return
        self.stack.append(x)
        self.pendingIncrements.append(0)

    def pop(self) -> int:
        if not self.stack:
            return -1
        if len(self.stack) > 1:
            self.pendingIncrements[-2] += self.pendingIncrements[-1]
        return self.stack.pop() + self.pendingIncrements.pop()

    def increment(self, k: int, val: int) -> None:
        if not self.stack:
            return
        i = min(k - 1, len(self.stack) - 1)
        self.pendingIncrements[i] += val
```

356. 1382.py

Path: LeetCode-main/solutions/1382. Balance a Binary Search Tree/1382.py

```
class Solution:
    def balanceBST(self, root: TreeNode | None) -> TreeNode | None:
        nums = []

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return
            inorder(root.left)
            nums.append(root.val)
            inorder(root.right)

        inorder(root)

        # Same as 108. Convert Sorted Array to Binary Search Tree
        def build(l: int, r: int) -> TreeNode | None:
            if l > r:
                return None
            m = (l + r) // 2
            return TreeNode(nums[m],
                            build(l, m - 1),
                            build(m + 1, r))

        return build(0, len(nums) - 1)
```

357. 1383.py

Path: LeetCode-main/solutions/1383. Maximum Performance of a Team/1383.py

```

class Solution:
    # Similar to 857. Minimum Cost to Hire K Workers
    def maxPerformance(
        self,
        n: int,
        speed: list[int],
        efficiency: list[int],
        k: int,
    ) -> int:
        MOD = 1_000_000_007
        ans = 0
        speedSum = 0
        # (efficiency[i], speed[i]) sorted by efficiency[i] in descending order
        A = sorted([(e, s) for s, e in zip(speed, efficiency)], reverse=True)
        minHeap = []

        for e, s in A:
            heapq.heappush(minHeap, s)
            speedSum += s
            if len(minHeap) > k:
                speedSum -= heapq.heappop(minHeap)
            ans = max(ans, speedSum * e)

        return ans % MOD

```

358. 1385.py

Path: LeetCode-main/solutions/1385. Find the Distance Value Between Two Arrays/1385.py

```

class Solution:
    def findTheDistanceValue(
        self,
        arr1: list[int],
        arr2: list[int],
        d: int,
    ) -> int:
        ans = 0

        arr2.sort()

        for a in arr1:
            i = bisect.bisect_left(arr2, a)
            if ((i == len(arr2) or arr2[i] - a > d) and
                (i == 0 or a - arr2[i - 1] > d)):
                ans += 1

        return ans

```

359. 1386.py

Path: LeetCode-main/solutions/1386. Cinema Seat Allocation/1386.py

```

class Solution:
    def maxNumberOfFamilies(self, n: int, reservedSeats: list[list[int]]) -> int:
        ans = 0
        rowToSeats = collections.Counter()

        for row, seat in reservedSeats:
            rowToSeats[row] |= 1 << (seat - 1)

        for seats in rowToSeats.values():
            if (seats & 0b011111110) == 0:
                # Can fit 2 four-person groups.
                ans += 2
            elif ((seats & 0b011110000) == 0 or
                  (seats & 0b0001111000) == 0 or
                  (seats & 0b0000011110) == 0):
                # Can fit 1 four-person group.
                ans += 1

        # Any empty row can fit 2 four-person groups.
        return ans + (n - len(rowToSeats)) * 2

```

360. 1387.py

Path: LeetCode-main/solutions/1387. Sort Integers by The Power Value/1387.py

```

class Solution:
    def getKth(self, lo: int, hi: int, k: int) -> int:
        return sorted([(self._getPow(i), i) for i in range(lo, hi + 1)])[k - 1][1]

    def _getPow(self, n: int) -> int:
        if n == 1:
            return 0
        if n % 2 == 0:
            return 1 + self._getPow(n // 2)
        return 1 + self._getPow(n * 3 + 1)

```

361. 1388.py

Path: LeetCode-main/solutions/1388. Pizza With 3n Slices/1388.py

```

class Solution:
    def maxSizeSlices(self, slices: list[int]) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> int:
            """
            Returns the maximum the sum of slices if you can pick k slices from
            slices[i..j].
            """
            if k == 1:
                return max(slices[i:j])
            # Note that j - i is not the number of all the left slices. Since you
            # Might have chosen not to take a slice in a previous step, there would be
            # Leftovers outside [i:j]. If you take slices[i], one of the slices your
            # Friends take will be outside of [i:j], so the length of [i:j] is reduced
            # By 2 instead of 3. Therefore, the minimum # Is 2 * k - 1 (the last step only
            # Requires one slice).
            if j - i < 2 * k - 1:
                return -math.inf
            return max(slices[i] + dp(i + 2, j, k - 1),
                      dp(i + 1, j, k))

        k = len(slices) // 3
        return max(dp(0, len(slices) - 1, k),
                  dp(1, len(slices), k))

```

362. 1389.py

Path: LeetCode-main/solutions/1389. Create Target Array in the Given Order/1389.py

```

class Solution:
    def createTargetArray(self, nums, index):
        ans = []
        for num, i in zip(nums, index):
            ans.insert(i, num)
        return ans

```

363. 139-2.py

Path: LeetCode-main/solutions/139. Word Break/139-2.py

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(i: int) -> bool:
            """Returns True if s[i..n) can be segmented."""
            if i == len(s):
                return True
            return any(s[i:j] in wordSet and wordBreak(j)
                      for j in range(i + 1, len(s) + 1))

        return wordBreak(0)

```

364. 139-3.py

Path: LeetCode-main/solutions/139. Word Break/139-3.py


```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i) can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i):
                # s[0..j) can be segmented and s[j..i) is in `wordSet`, so s[0..i) can
                # be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]

```

365. 139-4.py

Path: LeetCode-main/solutions/139. Word Break/139-4.py

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        maxLength = len(max(wordDict, key=len))
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i) can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i - 1, -1, -1):
                if i - j > maxLength:
                    break
                # s[0..j) can be segmented and s[j..i) is in the wordSet, so s[0..i)
                # can be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]

```

366. 139.py

Path: LeetCode-main/solutions/139. Word Break/139.py

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> bool:
            """Returns True if s can be segmented."""
            if s in wordSet:
                return True
            return any(s[:i] in wordSet and wordBreak(s[i:]) for i in range(len(s)))

        return wordBreak(s)

```

367. 1390.py

Path: LeetCode-main/solutions/1390. Four Divisors/1390.py

```
class Solution:
    def sumFourDivisors(self, nums: list[int]) -> int:
        ans = 0

        for num in nums:
            divisor = 0
            for i in range(2, math.isqrt(num) + 1):
                if num % i == 0:
                    if divisor == 0:
                        divisor = i
                    else:
                        divisor = 0
                        break
            if divisor > 0 and divisor * divisor < num:
                ans += 1 + num + divisor + num // divisor

        return ans
```

368. 1392.py

Path: LeetCode-main/solutions/1392. Longest Happy Prefix/1392.py

```
class Solution:
    def longestPrefix(self, s: str) -> str:
        BASE = 26
        HASH = 8_417_508_174_513
        n = len(s)
        maxLength = 0
        pow = 1
        prefixHash = 0 # the hash of s[0..i]
        suffixHash = 0 # the hash of s[j..n)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        j = n - 1
        for i in range(n - 1):
            prefixHash = (prefixHash * BASE + val(s[i])) % HASH
            suffixHash = (val(s[j]) * pow + suffixHash) % HASH
            pow = pow * BASE % HASH
            if prefixHash == suffixHash:
                maxLength = i + 1
                j -= 1

        return s[:maxLength]
```

369. 1394.py

Path: LeetCode-main/solutions/1394. Find Lucky Integer in an Array/1394.py

```
class Solution:
    def findLucky(self, arr: list[int]) -> int:
        count = [0] * (len(arr) + 1)

        for a in arr:
            if a <= len(arr):
                count[a] += 1

        for i in range(len(arr), 0, -1):
            if count[i] == i:
                return i

        return -1
```

370. 1395.py

Path: LeetCode-main/solutions/1395. Count Number of Teams/1395.py

```
class Solution:
    def numTeams(self, rating: list[int]) -> int:
        ans = 0

        for i in range(1, len(rating) - 1):
            # Calculate soldiers on the left with less//greater ratings.
            leftLess = 0
            leftGreater = 0
            for j in range(i):
                if rating[j] < rating[i]:
                    leftLess += 1
                elif rating[j] > rating[i]:
                    leftGreater += 1
            # Calculate soldiers on the right with less//greater ratings.
            rightLess = 0
            rightGreater = 0
            for j in range(i + 1, len(rating)):
                if rating[j] < rating[i]:
                    rightLess += 1
                elif rating[j] > rating[i]:
                    rightGreater += 1
            ans += leftLess * rightGreater + leftGreater * rightLess

        return ans
```

371. 1396.py

Path: LeetCode-main/solutions/1396. Design Underground System/1396.py

```
class UndergroundSystem:
    def __init__(self):
        # {id: (stationName, time)}
        self.checkIns = {}
        # {route: (numTrips, totalTime)}
        self.checkOuts = collections.defaultdict(lambda: [0, 0])

    def checkIn(self, id: int, stationName: str, t: int) -> None:
        self.checkIns[id] = (stationName, t)

    def checkOut(self, id: int, stationName: str, t: int) -> None:
        startStation, startTime = self.checkIns.pop(id)
        route = (startStation, stationName)
        self.checkOuts[route][0] += 1
        self.checkOuts[route][1] += t - startTime

    def getAverageTime(self, startStation: str, endStation: str) -> float:
        numTrips, totalTime = self.checkOuts[(startStation, endStation)]
        return totalTime / numTrips
```

372. 1397.py

Path: LeetCode-main/solutions/1397. Find All Good Strings/1397.py

```

class Solution:
    def findGoodStrings(self, n: int, s1: str, s2: str, evil: str) -> int:
        MOD = 1_000_000_007
        evilLPS = self._getLPS(evil)

        @functools.lru_cache(None)
        def getNextMatchedEvilCount(j: int, currChar: str) -> int:
            """
            Returns the number of next matched evil count, where there're j matches
            with `evil` and the current letter is ('a' + j).
            """
            while j > 0 and evil[j] != currChar:
                j = evilLPS[j - 1]
            return j + 1 if evil[j] == currChar else j

        @functools.lru_cache(None)
        def dp(i: int, matchedEvilCount: int, isS1Prefix: bool, isS2Prefix: bool) -> int:
            """
            Returns the number of good strings for s[i..n), where there're j matches
            with `evil`, `isS1Prefix` indicates if the current letter is tightly bound
            for `s1` and `isS2Prefix` indicates if the current letter is tightly bound
            for `s2`.
            """
            # s[0..i) contains `evil`, so don't consider any ongoing strings.
            if matchedEvilCount == len(evil):
                return 0
            # Run out of strings, so contribute one.
            if i == n:
                return 1
            ans = 0
            minCharIndex = ord(s1[i]) if isS1Prefix else ord('a')
            maxCharIndex = ord(s2[i]) if isS2Prefix else ord('z')
            for charIndex in range(minCharIndex, maxCharIndex + 1):
                c = chr(charIndex)
                nextMatchedEvilCount = getNextMatchedEvilCount(matchedEvilCount, c)
                ans += dp(i + 1, nextMatchedEvilCount,
                        isS1Prefix and c == s1[i],
                        isS2Prefix and c == s2[i])
            ans %= MOD
            return ans

        return dp(0, 0, True, True)

    def _getLPS(self, pattern: str) -> list[int]:
        """
        Returns the lps array, where lps[i] is the length of the longest prefix of
        pattern[0..i] which is also a suffix of this substring.
        """
        lps = [0] * len(pattern)
        j = 0
        for i in range(1, len(pattern)):
            while j > 0 and pattern[j] != pattern[i]:
                j = lps[j - 1]
            if pattern[i] == pattern[j]:
                lps[i] = j + 1
                j += 1
        return lps

```

373. 1399.py

Path: LeetCode-main/solutions/1399. Count Largest Group/1399.py

```

class Solution:
    def countLargestGroup(self, n: int) -> int:
        count = [0] * (9 * 4 + 1)
        for i in range(1, n + 1):
            count[self._getDigitSum(i)] += 1
        return count.count(max(count))

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))

```

374. 14.py

Path: LeetCode-main/solutions/14. Longest Common Prefix/14.py

```

class Solution:
    def longestCommonPrefix(self, strs: list[str]) -> str:
        if not strs:
            return ''

        for i in range(len(strs[0])):
            for j in range(1, len(strs)):
                if i == len(strs[j]) or strs[j][i] != strs[0][i]:
                    return strs[0][:i]

        return strs[0]

```

375. 140.py

Path: LeetCode-main/solutions/140. Word Break II/140.py

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> list[str]:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> list[str]:
            ans = []

            # 1 <= len(prefix) < len(s)
            for i in range(1, len(s)):
                prefix = s[:i]
                suffix = s[i:]
                if prefix in wordSet:
                    for word in wordBreak(suffix):
                        ans.append(prefix + ' ' + word)

            # `wordSet` contains the whole string s, so don't add any space.
            if s in wordSet:
                ans.append(s)

            return ans

        return wordBreak(s)

```

376. 1400.py

Path: LeetCode-main/solutions/1400. Construct K Palindrome Strings/1400.py

```
class Solution:
    def canConstruct(self, s: str, k: int) -> bool:
        # If |s| < k, we cannot construct k strings from the s.
        # If the number of letters that have odd counts > k, the minimum number of
        # palindromic strings we can construct is > k.
        return sum(freq & 1
                    for freq in collections.Counter(s).values()) <= k <= len(s)
```

377. 1402.py

Path: LeetCode-main/solutions/1402. Reducing Dishes/1402.py

```
class Solution:
    def maxSatisfaction(self, satisfaction: list[int]) -> int:
        ans = 0
        sumSatisfaction = 0

        for s in sorted(satisfaction, reverse=True):
            sumSatisfaction += s
            if sumSatisfaction <= 0:
                return ans
            ans += sumSatisfaction

        return ans
```

378. 1403.py

Path: LeetCode-main/solutions/1403. Minimum Subsequence in Non-Increasing Order/1403.py

```
class Solution:
    def minSubsequence(self, nums: list[int]) -> list[int]:
        ans = []
        maxHeap = [-num for num in nums]
        heapq.heapify(maxHeap)
        half = sum(nums) // 2

        while half >= 0:
            ans.append(-maxHeap[0])
            half += heapq.heappop(maxHeap)

        return ans
```

379. 1404.py

Path: LeetCode-main/solutions/1404. Number of Steps to Reduce a Number in Binary Representation to One/1404.py

```

class Solution:
    def numSteps(self, s: str) -> int:
        ans = 0
        chars = list(s)

        # All the trailing 0s can be popped by 1 step.
        while chars[-1] == '0':
            chars.pop()
            ans += 1

        if ''.join(chars) == '1':
            return ans

        # `s` is now odd, so add 1 to `s` and cost 1 step.
        # All the 1s will become 0s and can be popped by 1 step.
        # All the 0s will become 1s and can be popped by 2 steps (adding 1 then
        # dividing by 2).
        return ans + 1 + sum(1 if c == '1' else 2 for c in chars)

```

380. 1406-2.py

Path: LeetCode-main/solutions/1406. Stone Game III/1406-2.py

```

class Solution:
    def stoneGameIII(self, stoneValue: list[int]) -> str:
        n = len(stoneValue)
        # dp[i] := the maximum relative score Alice can make with stoneValue[i..n)
        dp = [-math.inf] * n + [0]

        for i in reversed(range(n)):
            summ = 0
            for j in range(i, i + 3):
                if j == n:
                    break
                summ += stoneValue[j]
                dp[i] = max(dp[i], summ - dp[j + 1])

        score = dp[0]
        if score == 0:
            return 'Tie'
        return 'Alice' if score > 0 else 'Bob'

```

381. 1406.py

Path: LeetCode-main/solutions/1406. Stone Game III/1406.py


```
class Solution:
    def stoneGameIII(self, stoneValue: list[int]) -> str:
        @functools.lru_cache(None)
        def dp(i: int) -> int:
            """
            Returns the maximum relative score Alice can make with stoneValue[i..n).
            """
            if i == len(stoneValue):
                return 0

            res = -math.inf
            summ = 0

            for j in range(i, i + 3):
                if j == len(stoneValue):
                    break
                summ += stoneValue[j]
                res = max(res, summ - dp(j + 1))

            return res

        score = dp(0)
        if score == 0:
            return 'Tie'
        return 'Alice' if score > 0 else 'Bob'
```

382. 1408-2.py

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408-2.py

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node: TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())

    def search(self, word: str) -> bool:
        node: TrieNode = self.root
        for c in word:
            if c not in node.children:
                return False
            node = node.children[c]
        return True

class Solution:
    def stringMatching(self, words: list[str]) -> list[str]:
        ans = []
        trie = Trie()

        for word in sorted(words, key=lambda x: -len(x)):
            if trie.search(word):
                ans.append(word)
            for i in range(len(word)):
                trie.insert(word[i:])

        return ans
```

383. 1408-3.py

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408-3.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.count = 0

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node: TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
            node.count += 1

    def search(self, word: str) -> bool:
        node: TrieNode = self.root
        for c in word:
            if c not in node.children:
                return False
            node = node.children[c]
        return node.count > 1

class Solution:
    def stringMatching(self, words: list[str]) -> list[str]:
        trie = Trie()

        for word in words:
            for i in range(len(word)):
                trie.insert(word[i:])

        return [word for word in words if trie.search(word)]

```

384. 1408.py

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408.py

```

class Solution:
    def stringMatching(self, words: list[str]) -> list[str]:
        ans = []
        for a in words:
            for b in words:
                if len(a) < len(b) and b.find(a) != -1:
                    ans.append(a)
                    break
        return ans

```

385. 1409.py

Path: LeetCode-main/solutions/1409. Queries on a Permutation With Key/1409.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    def processQueries(self, queries: list[int], m: int) -> list[int]:
        ans = []
        # Map [-m, m] to [0, 2 * m].
        tree = FenwickTree(2 * m + 1)
        numToIndex = {num: num + m for num in range(1, m + 1)}

        for num in range(1, m + 1):
            tree.add(num + m, 1)

        nextEmptyIndex = m # Map 0 to m.

        for query in queries:
            index = numToIndex[query]
            ans.append(tree.get(index - 1))
            # Move `query` from `index` to `nextEmptyIndex`.
            tree.add(index, -1)
            tree.add(nextEmptyIndex, 1)
            numToIndex[query] = nextEmptyIndex
            nextEmptyIndex -= 1

        return ans

```

386. 141.py

Path: LeetCode-main/solutions/141. Linked List Cycle/141.py

```

class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True

        return False

```

387. 1410.py

Path: LeetCode-main/solutions/1410. HTML Entity Parser/1410.py

```
class Solution:
    def entityParser(self, text: str) -> str:
        entityToChar = {'&quot;': '"', '&apos;': '\'',
                        '&gt;': '>', '&lt;': '<', '&frasl;': '/'}

        for entity, c in entityToChar.items():
            text = text.replace(entity, c)

        # Process '&' in last.
        return text.replace('&', '&')
```

388. 1413.py

Path: LeetCode-main/solutions/1413. Minimum Value to Get Positive Step by Step Sum/1413.py

```
class Solution:
    def minStartValue(self, nums: list[int]) -> int:
        summ = 0
        minSum = 0

        for num in nums:
            summ += num
            minSum = min(minSum, summ)

        return 1 - minSum
```

389. 1414-2.py

Path: LeetCode-main/solutions/1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K/1414-2.py

```
class Solution:
    def findMinFibonacciNumbers(self, k: int) -> int:
        ans = 0
        a = 1 # F_1
        b = 1 # F_2

        while b <= k:
            # a, b = F_{i + 1}, F_{i + 2}
            # -> a, b = F_{i + 2}, F_{i + 3}
            a, b = b, a + b

        while a > 0:
            if a <= k:
                k -= a
                ans += 1
            # a, b = F_{i + 2}, F_{i + 3}
            # -> a, b = F_{i + 1}, F_{i + 2}
            a, b = b - a, a

        return ans
```

390. 1414.py

Path: LeetCode-main/solutions/1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K/1414.py

```
class Solution:
    def findMinFibonacciNumbers(self, k: int) -> int:
        if k < 2: # k == 0 or k == 1
            return k

        a = 1 # F_1
        b = 1 # F_2

        while b <= k:
            # a, b = F_{i + 1}, F_{i + 2}
            # -> a, b = F_{i + 2}, F_{i + 3}
            a, b = b, a + b

        return 1 + self.findMinFibonacciNumbers(k - a)
```

391. 1415.py

Path: LeetCode-main/solutions/1415. The k-th Lexicographical String of All Happy Strings of Length n/1415.py

```
class Solution:
    def getHappyString(self, n: int, k: int) -> str:
        nextLetters = {'a': 'bc', 'b': 'ac', 'c': 'ab'}
        q = collections.deque(['a', 'b', 'c'])

        while len(q[0]) != n:
            u = q.popleft()
            for nextLetter in nextLetters[u[-1]]:
                q.append(u + nextLetter)

        return '' if len(q) < k else q[k - 1]
```

392. 1417.py

Path: LeetCode-main/solutions/1417. Reformat The String/1417.py

```
class Solution:
    def reformat(self, s: str) -> str:
        A = [c for c in s if c.isalpha()]
        B = [c for c in s if c.isdigit()]

        if len(A) < len(B):
            A, B = B, A
        if len(A) - len(B) > 1:
            return ''

        ans = []

        for i in range(len(B)):
            ans.append(A[i])
            ans.append(B[i])

        if len(A) == len(B) + 1:
            ans.append(A[-1])
        return ''.join(ans)
```

393. 1419.py

Path: LeetCode-main/solutions/1419. Minimum Number of Frogs Croaking/1419.py

```
class Solution:
    def minNumberOfFrogs(self, croakOfFrogs: str) -> int:
        CROAK = 'croak'
        ans = 0
        frogs = 0
        count = [0] * 5

        for c in croakOfFrogs:
            count[CROAK.index(c)] += 1
            if any(count[i] > count[i - 1] for i in range(1, 5)):
                return -1
            if c == 'c':
                frogs += 1
            elif c == 'k':
                frogs -= 1
            ans = max(ans, frogs)

        return ans if frogs == 0 else -1
```

394. 142.py

Path: LeetCode-main/solutions/142. Linked List Cycle II/142.py

```
class Solution:
    def detectCycle(self, head: ListNode) -> ListNode:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                slow = head
                while slow != fast:
                    slow = slow.next
                    fast = fast.next
                return slow

        return None
```

395. 1420-2.py

Path: LeetCode-main/solutions/1420. Build Array Where You Can Find The Maximum Exactly K Comparisons/1420-2.py

```

class Solution:
    def numOfArrays(self, n: int, m: int, k: int) -> int:
        MOD = 1_000_000_007

        # dp[i][j][k] := the number of ways to build an array of length i, where j
        # is the maximum number and k is the `search_cost`
        dp = [[[0] * (k + 1) for j in range(m + 1)] for _ in range(n + 1)]
        # prefix[i][j][k] := sum(dp[i][x][k]), where 1 <= x <= j
        prefix = [[[0] * (k + 1) for j in range(m + 1)] for _ in range(n + 1)]

        for j in range(1, m + 1):
            dp[1][j][1] = 1
            prefix[1][j][1] = j

        for i in range(2, n + 1): # for each length
            for j in range(1, m + 1): # for each max value
                for cost in range(1, k + 1): # for each cost
                    # 1. Appending any of [1, j] in the i-th position doesn't change the
                    #    maximum and cost.
                    # 2. Appending j in the i-th position makes j the new max and cost 1.
                    dp[i][j][cost] = (j * dp[i - 1][j][cost] +
                                       prefix[i - 1][j - 1][cost - 1]) % MOD
                    prefix[i][j][cost] = (dp[i][j][cost] + prefix[i][j - 1][cost]) % MOD

        return sum(dp[n][j][k] for j in range(1, m + 1)) % MOD

```

396. 1420.py

Path: LeetCode-main/solutions/1420. Build Array Where You Can Find The Maximum Exactly K Comparisons/1420.py

```

class Solution:
    def numOfArrays(self, n: int, m: int, k: int) -> int:
        MOD = 1_000_000_007

        # dp[i][j][k] := the number of ways to build an array of length i, where j
        # is the maximum number and k is `search_cost`
        dp = [[[0] * (k + 1) for j in range(m + 1)] for _ in range(n + 1)]

        for j in range(1, m + 1):
            dp[1][j][1] = 1

        for i in range(2, n + 1): # for each length
            for j in range(1, m + 1): # for each max value
                for cost in range(1, k + 1): # for each cost
                    # 1. Appending any of [1, j] in the i-th position doesn't change the
                    #    maximum and cost.
                    dp[i][j][cost] = j * dp[i - 1][j][cost] % MOD
                    # 2. Appending j in the i-th position makes j the new max and cost 1.
                    for prevMax in range(1, j):
                        dp[i][j][cost] += dp[i - 1][prevMax][cost - 1]
                        dp[i][j][cost] %= MOD

        return sum(dp[n][j][k] for j in range(1, m + 1)) % MOD

```

397. 1422.py

Path: LeetCode-main/solutions/1422. Maximum Score After Splitting a String/1422.py


```

class Solution:
    def maxScore(self, s: str) -> int:
        ans = 0
        zeros = 0
        ones = s.count('1')

        for i in range(len(s) - 1):
            if s[i] == '0':
                zeros += 1
            else:
                ones -= 1
            ans = max(ans, zeros + ones)

        return ans

```

398. 1423.py

Path: LeetCode-main/solutions/1423. Maximum Points You Can Obtain from Cards/1423.py

```

class Solution:
    def maxScore(self, cardPoints: list[int], k: int) -> int:
        n = len(cardPoints)
        summ = sum(cardPoints)
        windowSum = sum(cardPoints[:n - k])
        ans = summ - windowSum

        for i in range(k):
            windowSum -= cardPoints[i]
            windowSum += cardPoints[i + n - k]
            ans = max(ans, summ - windowSum)

        return ans

```

399. 1425.py

Path: LeetCode-main/solutions/1425. Constrained Subsequence Sum/1425.py

```

class Solution:
    def constrainedSubsetSum(self, nums: list[int], k: int) -> int:
        # dp[i] := the maximum the sum of non-empty subsequences in nums[0..i]
        dp = [0] * len(nums)
        # dq stores dp[i - k], dp[i - k + 1], ..., dp[i - 1] whose values are > 0
        # in decreasing order.
        dq = collections.deque()

        for i, num in enumerate(nums):
            if dq:
                dp[i] = max(dq[0], 0) + num
            else:
                dp[i] = num
            while dq and dq[-1] < dp[i]:
                dq.pop()
            dq.append(dp[i])
            if i >= k and dp[i - k] == dq[0]:
                dq.popleft()

        return max(dp)

```

400. 1426.py

Path: LeetCode-main/solutions/1426. Counting Elements/1426.py

```
class Solution:
    def countElements(self, arr: list[int]) -> int:
        count = collections.Counter(arr)
        return sum(freq
                    for a, freq in count.items()
                    if count[a + 1] > 0)
```

401. 1427.py

Path: LeetCode-main/solutions/1427. Perform String Shifts/1427.py

```
class Solution:
    def stringShift(self, s: str, shift: list[list[int]]) -> str:
        move = 0

        for direction, amount in shift:
            if direction == 0:
                move -= amount
            else:
                move += amount

        move %= len(s)
        return s[-move:] + s[:-move]
```

402. 1428-2.py

Path: LeetCode-main/solutions/1428. Leftmost Column with at Least a One/1428-2.py

```
# """
# This is BinaryMatrix's API interface.
# You should not implement it, or speculate about its implementation
# """
# class BinaryMatrix(object):
#     def get(self, row: int, col: int) -> int:
#     def dimensions(self) -> list[int]:

class Solution:
    def leftMostColumnWithOne(self, binaryMatrix: 'BinaryMatrix') -> int:
        m, n = binaryMatrix.dimensions()
        ans = -1
        i = 0
        j = n - 1

        while i < m and j >= 0:
            if binaryMatrix.get(i, j):
                ans = j
                j -= 1
            else:
                i += 1

        return ans
```

403. 1428.py

Path: LeetCode-main/solutions/1428. Leftmost Column with at Least a One/1428.py

```
# """
# This is BinaryMatrix's API interface.
# You should not implement it, or speculate about its implementation
# """
# class BinaryMatrix(object):
#     def get(self, row: int, col: int) -> int:
#     def dimensions(self) -> list[int]:

class Solution:
    def leftMostColumnWithOne(self, binaryMatrix: 'BinaryMatrix') -> int:
        m, n = binaryMatrix.dimensions()
        ans = -1
        l = 0
        r = n - 1

        while l <= r:
            mid = (l + r) // 2
            if any(binaryMatrix.get(i, mid) for i in range(m)):
                ans = mid
                r = mid - 1
            else:
                l = mid + 1

        return ans
```

404. 1429.py

Path: LeetCode-main/solutions/1429. First Unique Number/1429.py

```
class FirstUnique:
    def __init__(self, nums: list[int]):
        self.seen = set()
        self.unique = {}
        for num in nums:
            self.add(num)

    def showFirstUnique(self) -> int:
        return next(iter(self.unique), -1)

    def add(self, value: int) -> None:
        if value not in self.seen:
            self.seen.add(value)
            self.unique[value] = 1
        elif value in self.unique:
            # We have added this value before, and this is the second time we're
            # adding it. So, erase the value from `unique`.
            self.unique.pop(value)
```

405. 143.py

Path: LeetCode-main/solutions/143. Reorder List/143.py

```

class Solution:
    def reorderList(self, head: ListNode) -> None:
        def findMid(head: ListNode):
            prev = None
            slow = head
            fast = head

            while fast and fast.next:
                prev = slow
                slow = slow.next
                fast = fast.next.next
            prev.next = None

            return slow

        def reverse(head: ListNode) -> ListNode:
            prev = None
            curr = head

            while curr:
                next = curr.next
                curr.next = prev
                prev = curr
                curr = next

            return prev

        def merge(l1: ListNode, l2: ListNode) -> None:
            while l2:
                next = l1.next
                l1.next = l2
                l1 = l2
                l2 = next

        if not head or not head.next:
            return

        mid = findMid(head)
        reversed = reverse(mid)
        merge(head, reversed)

```

406. 1430.py

Path: LeetCode-main/solutions/1430. Check If a String Is a Valid Sequence from Root to Leaves Path in a Binary Tree/1430.py

```

class Solution:
    def isValidSequence(self, root: TreeNode | None, arr: list[int]) -> bool:
        def isValidSequence(root: TreeNode | None, i: int) -> bool:
            if not root:
                return False
            if i == len(arr) - 1:
                return root.val == arr[i] and not root.left and not root.right
            return root.val == arr[i] and (
                isValidSequence(root.left, i + 1) or
                isValidSequence(root.right, i + 1))

        return isValidSequence(root, 0)

```

407. 1431.py

Path: LeetCode-main/solutions/1431. Kids With the Greatest Number of Candies/1431.py

```
class Solution:
    def kidsWithCandies(
        self,
        candies: list[int],
        extraCandies: int,
    ) -> list[bool]:
        maxCandy = max(candies)
        return [candy + extraCandies >= maxCandy for candy in candies]
```

408. 1432.py

Path: LeetCode-main/solutions/1432. Max Difference You Can Get From Changing an Integer/1432.py

```
class Solution:
    def maxDiff(self, num: int) -> int:
        s = str(num)

        def firstNot(s: str, t: str) -> int:
            for i, c in enumerate(s):
                if all(c != d for d in t):
                    return i
            return 0

        firstNot9 = firstNot(s, '9')
        firstNot01 = firstNot(s, '01')
        a = s.replace(s[firstNot9], '9')
        b = s.replace(s[firstNot01], '1' if firstNot01 == 0 else '0')
        return int(a) - int(b)
```

409. 1433-2.py

Path: LeetCode-main/solutions/1433. Check If a String Can Break Another String/1433-2.py

```
class Solution:
    def checkIfCanBreak(self, s1: str, s2: str) -> bool:
        count = collections.Counter(s1)
        count.subtract(collections.Counter(s2))

        for a, b in itertools.pairwise(string.ascii_lowercase):
            count[b] += count[a]

        return (all(value <= 0 for value in count.values()) or
                all(value >= 0 for value in count.values()))
```

410. 1433.py

Path: LeetCode-main/solutions/1433. Check If a String Can Break Another String/1433.py

```

class Solution:
    def checkIfCanBreak(self, s1: str, s2: str) -> bool:
        count1 = collections.Counter(s1)
        count2 = collections.Counter(s2)

        def canBreak(count1: dict[str, int], count2: dict[str, int]) -> bool:
            """Returns True if count1 can break count2."""
            diff = 0
            for c in string.ascii_lowercase:
                diff += count2[c] - count1[c]
                # count2 is alphabetically greater than count1.
                if diff < 0:
                    return False
            return True

        return canBreak(count1, count2) or canBreak(count2, count1)

```

411. 1436.py

Path: LeetCode-main/solutions/1436. Destination City/1436.py

```

class Solution:
    def destCity(self, paths: list[list[str]]) -> str:
        count = collections.Counter()

        for a, b in paths:
            count[a] += 1

        for a, b in paths:
            if b in count:
                count[b] -= 1
                if count[b] == 0:
                    del count[b]
            else:
                return b

```

412. 1437.py

Path: LeetCode-main/solutions/1437. Check If All 1's Are at Least Length K Places Away/1437.py

```

class Solution:
    def kLengthApart(self, nums: list[int], k: int) -> bool:
        if k == 0:
            return True

        n = len(nums)
        curr = 0
        next = 1

        while curr < n and next < n:
            if nums[next] == 1:
                if nums[curr] == 1 and next - curr <= k:
                    return False
                curr = next
            next += 1

        return True

```

413. 144-2.py

Path: LeetCode-main/solutions/144. Binary Tree Preorder Traversal/144-2.py

```
class Solution:
    def preorderTraversal(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        stack = [root]

        while stack:
            node = stack.pop()
            ans.append(node.val)
            if node.right:
                stack.append(node.right)
            if node.left:
                stack.append(node.left)

        return ans
```

414. 144.py

Path: LeetCode-main/solutions/144. Binary Tree Preorder Traversal/144.py

```
class Solution:
    def preorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []

        def preorder(root: TreeNode | None) -> None:
            if not root:
                return

            ans.append(root.val)
            preorder(root.left)
            preorder(root.right)

        preorder(root)
        return ans
```

415. 1441.py

Path: LeetCode-main/solutions/1441. Build an Array With Stack Operations/1441.py

```

class Solution:
    def buildArray(self, target: list[int], n: int) -> list[str]:
        ans = []
        i = 0 # Target pointer
        num = 1 # Curr num

        while i < len(target):
            t = target[i]
            if t == num:
                ans.append('Push')
                i += 1
            else:
                ans.append('Push')
                ans.append('Pop')
                num += 1

        return ans

```

416. 1442.py

Path: LeetCode-main/solutions/1442. Count Triplets That Can Form Two Arrays of Equal XOR/1442.py

```

class Solution:
    def countTriplets(self, arr: list[int]) -> int:
        ans = 0
        xors = [0]
        prefix = 0

        for i, a in enumerate(arr):
            prefix ^= a
            xors.append(prefix)

        for j in range(1, len(arr)):
            for i in range(0, j):
                xors_i = xors[j] ^ xors[i]
                for k in range(j, len(arr)):
                    xors_k = xors[k + 1] ^ xors[j]
                    if xors_i == xors_k:
                        ans += 1

        return ans

```

417. 1444.py

Path: LeetCode-main/solutions/1444. Number of Ways of Cutting a Pizza/1444.py


```

class Solution:
    def ways(self, pizza: list[str], k: int) -> int:
        MOD = 1_000_000_007
        M = len(pizza)
        N = len(pizza[0])
        prefix = [[0] * (N + 1) for _ in range(M + 1)]

        for i in range(M):
            for j in range(N):
                prefix[i + 1][j + 1] = ((pizza[i][j] == 'A') + prefix[i][j + 1] +
                                         prefix[i + 1][j] - prefix[i][j])

        def hasApple(row1: int, row2: int, col1: int, col2: int) -> bool:
            """Returns True if pizza[row1..row2][col1..col2] has apple."""
            return (prefix[row2][col2] - prefix[row1][col2] -
                    prefix[row2][col1] + prefix[row1][col1]) > 0

        @functools.lru_cache(None)
        def dp(m: int, n: int, k: int) -> int:
            """Returns the number of ways to cut pizza[m..M][n..N] with k cuts."""
            if k == 0:
                return 1 if hasApple(m, M, n, N) else 0

            res = 0

            for i in range(m + 1, M): # Cut horizontally.
                if hasApple(m, i, n, N) and hasApple(i, M, n, N):
                    res += dp(i, n, k - 1)

            for j in range(n + 1, N): # Cut vertically.
                if hasApple(m, M, n, j) and hasApple(m, M, j, N):
                    res += dp(m, j, k - 1)

            return res % MOD

        return dp(0, 0, k - 1)

```

418. 1446.py

Path: LeetCode-main/solutions/1446. Consecutive Characters/1446.py

```

class Solution:
    def maxPower(self, s: str) -> int:
        ans = 1
        count = 1

        for i in range(1, len(s)):
            count = count + 1 if s[i] == s[i - 1] else 1
            ans = max(ans, count)

        return ans

```

419. 1447.py

Path: LeetCode-main/solutions/1447. Simplified Fractions/1447.py

```
class Solution:
    def simplifiedFractions(self, n: int) -> list[str]:
        ans = []
        for denominator in range(2, n + 1):
            for numerator in range(1, denominator):
                if math.gcd(denominator, numerator) == 1:
                    ans.append(str(numerator) + '/' + str(denominator))
        return ans
```

420. 145-2.py

Path: LeetCode-main/solutions/145. Binary Tree Postorder Traversal/145-2.py

```
class Solution:
    def postorderTraversal(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        stack = [root]

        while stack:
            node = stack.pop()
            ans.append(node.val)
            if node.left:
                stack.append(node.left)
            if node.right:
                stack.append(node.right)

        return ans[::-1]
```

421. 145.py

Path: LeetCode-main/solutions/145. Binary Tree Postorder Traversal/145.py

```
class Solution:
    def postorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []

        def postorder(root: TreeNode | None) -> None:
            if not root:
                return

            postorder(root.left)
            postorder(root.right)
            ans.append(root.val)

        postorder(root)
        return ans
```

422. 1451.py

Path: LeetCode-main/solutions/1451. Rearrange Words in a Sentence/1451.py

```
class Solution:
    def arrangeWords(self, text: str) -> str:
        words = text.split()
        count = collections.defaultdict(list)

        for word in words:
            count[len(word)].append(word.lower())

        c2 = OrderedDict(sorted(count.items()))

        ans = []

        for l in c2:
            for word in c2[l]:
                ans.append(word)

        ans[0] = ans[0].capitalize()

        return ' '.join(ans)
```

423. 1452.py

Path: LeetCode-main/solutions/1452. People Whose List of Favorite Companies Is Not a Subset of Another List/1452.py

```
class Solution:
    def peopleIndexes(self, favoriteCompanies: list[list[str]]) -> list[int]:
        ans = []
        n = len(favoriteCompanies)
        companies = [set(comp) for comp in favoriteCompanies]

        for i in range(n):
            find = False
            for j in range(n):
                if i == j:
                    continue
                if companies[i].issubset(companies[j]):
                    find = True
                    break
            if not find:
                ans.append(i)

        return ans
```

424. 1453.py

Path: LeetCode-main/solutions/1453. Maximum Number of Darts Inside of a Circular Dartboard/1453.py

```

class Point:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

class Solution:
    def numPoints(self, darts: list[list[int]], r: int) -> int:
        ERR = 1e-6
        ans = 1
        points = [Point(x, y) for x, y in darts]

        def dist(p: Point, q: Point) -> float:
            return ((p.x - q.x)**2 + (p.y - q.y)**2)**0.5

        def getCircles(p: Point, q: Point) -> list[Point]:
            if dist(p, q) - 2.0 * r > ERR:
                return []
            m = Point((p.x + q.x) / 2, (p.y + q.y) / 2)
            distCM = (r**2 - (dist(p, q) / 2)**2)**0.5
            alpha = math.atan2(p.y - q.y, q.x - p.x)
            return [Point(m.x - distCM * math.sin(alpha), m.y - distCM * math.cos(alpha)),
                    Point(m.x + distCM * math.sin(alpha), m.y + distCM * math.cos(alpha))]

        for i, j in itertools.combinations(points, 2):
            for c in getCircles(i, j):
                count = 0
                for point in points:
                    if dist(c, point) - r <= ERR:
                        count += 1
                ans = max(ans, count)

        return ans

```

425. 1455.py

Path: LeetCode-main/solutions/1455. Check If a Word Occurs As a Prefix of Any Word in a Sentence/1455.py

```

class Solution:
    def isPrefixOfWord(self, sentence: str, searchWord: str) -> int:
        words = sentence.split()

        for i, word in enumerate(words):
            if word.startswith(searchWord):
                return i + 1

        return -1

```

426. 1456.py

Path: LeetCode-main/solutions/1456. Maximum Number of Vowels in a Substring of Given Length/1456.py

```

class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        ans = 0
        mx = 0
        VOWELS = 'aeiou'

        for i, c in enumerate(s):
            if c in VOWELS:
                mx += 1
            if i >= k and s[i - k] in VOWELS:
                mx -= 1
            ans = max(ans, mx)

        return ans

```

427. 1457.py

Path: LeetCode-main/solutions/1457. Pseudo-Palindromic Paths in a Binary Tree/1457.py

```

class Solution:
    def pseudoPalindromicPaths(self, root: TreeNode | None) -> int:
        ans = 0

        def dfs(root: TreeNode | None, path: int) -> None:
            nonlocal ans
            if not root:
                return
            if not root.left and not root.right:
                path ^= 1 << root.val
                if path & (path - 1) == 0:
                    ans += 1
                return

            dfs(root.left, path ^ 1 << root.val)
            dfs(root.right, path ^ 1 << root.val)

        dfs(root, 0)
        return ans

```

428. 1458.py

Path: LeetCode-main/solutions/1458. Max Dot Product of Two Subsequences/1458.py

```

class Solution:
    def maxDotProduct(self, A: list[int], B: list[int]) -> int:
        m = len(A)
        n = len(B)
        # dp[i][j] := the maximum dot product of the two subsequences nums[0..i)
        # and nums2[0..j)
        dp = [[-math.inf] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                dp[i + 1][j + 1] = max(dp[i][j + 1], dp[i + 1][j],
                                         max(0, dp[i][j]) + A[i] * B[j])

        return dp[m][n]

```

429. 146.py

Path: LeetCode-main/solutions/146. LRU Cache/146.py

```
class Node:
    def __init__(self, key: int, value: int):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.keyToNode = {}
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.join(self.head, self.tail)

    def get(self, key: int) -> int:
        if key not in self.keyToNode:
            return -1

        node = self.keyToNode[key]
        self.remove(node)
        self.moveToHead(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.keyToNode:
            node = self.keyToNode[key]
            node.value = value
            self.remove(node)
            self.moveToHead(node)
            return

        if len(self.keyToNode) == self.capacity:
            lastNode = self.tail.prev
            del self.keyToNode[lastNode.key]
            self.remove(lastNode)

        self.moveToHead(Node(key, value))
        self.keyToNode[key] = self.head.next

    def join(self, node1: Node, node2: Node):
        node1.next = node2
        node2.prev = node1

    def moveToHead(self, node: Node):
        self.join(node, self.head.next)
        self.join(self.head, node)

    def remove(self, node: Node):
        self.join(node.prev, node.next)
```

430. 1460.py

Path: LeetCode-main/solutions/1460. Make Two Arrays Equal by Reversing Subarrays/1460.py

```
class Solution:
    def canBeEqual(self, target: list[int], arr: list[int]) -> bool:
        return collections.Counter(arr) == collections.Counter(target)
```

431. 1461.py

Path: LeetCode-main/solutions/1461. Check If a String Contains All Binary Codes of Size K/1461.py

```
class Solution:
    def hasAllCodes(self, s: str, k: int) -> bool:
        n = 1 << k
        if len(s) < n:
            return False

        # used[i] := True if i is a substring of `s`
        used = [0] * n

        windowStr = 0 if k == 1 else int(s[0:k - 1], 2)
        for i in range(k - 1, len(s)):
            # Include the s[i].
            windowStr = (windowStr << 1) + int(s[i])
            # Discard the s[i - k].
            windowStr &= n - 1
            used[windowStr] = True

        return all(u for u in used)
```

432. 1462-2.py

Path: LeetCode-main/solutions/1462. Course Schedule IV/1462-2.py

```
class Solution:
    def checkIfPrerequisite(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
        queries: list[list[int]],
    ) -> list[bool]:
        # isPrerequisite[i][j] := True if course i is a prerequisite of course j.
        isPrerequisite = [[False] * numCourses for _ in range(numCourses)]

        for u, v in prerequisites:
            isPrerequisite[u][v] = True

        for k in range(numCourses):
            for i in range(numCourses):
                for j in range(numCourses):
                    isPrerequisite[i][j] = (isPrerequisite[i][j] or (
                        isPrerequisite[i][k] and isPrerequisite[k][j]))

        return [isPrerequisite[u][v] for u, v in queries]
```

433. 1462.py

Path: LeetCode-main/solutions/1462. Course Schedule IV/1462.py

```

class Solution:
    def checkIfPrerequisite(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
        queries: list[list[int]],
    ) -> list[bool]:
        graph = [[] for _ in range(numCourses)]
        # isPrerequisite[i][j] := True if course i is a prerequisite of course j.
        isPrerequisite = [[False] * numCourses for _ in range(numCourses)]

        for u, v in prerequisites:
            graph[u].append(v)

        # DFS from every course.
        for i in range(numCourses):
            self._dfs(graph, i, isPrerequisite[i])

        return [isPrerequisite[u][v] for u, v in queries]

    def _dfs(self, graph: list[list[int]], u: int, used: list[bool]) -> None:
        for v in graph[u]:
            if used[v]:
                continue
            used[v] = True
            self._dfs(graph, v, used)

```

434. 1464.py

Path: LeetCode-main/solutions/1464. Maximum Product of Two Elements in an Array/1464.py

```

class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        max1 = 0
        max2 = 0

        for num in nums:
            if num > max1:
                max2, max1 = max1, num
            elif num > max2:
                max2 = num

        return (max1 - 1) * (max2 - 1)

```

435. 1465.py

Path: LeetCode-main/solutions/1465. Maximum Area of a Piece of Cake After Horizontal and Vertical Cuts/1465.py


```
class Solution:
    def maxArea(
        self,
        h: int,
        w: int,
        horizontalCuts: list[int],
        verticalCuts: list[int],
    ) -> int:
        MOD = 1_000_000_007
        # the maximum gap of each direction
        maxGapX = max(b - a
                       for a, b in itertools.pairwise(
                           [0] + sorted(horizontalCuts) + [h]))
        maxGapY = max(b - a
                       for a, b in itertools.pairwise(
                           [0] + sorted(verticalCuts) + [w]))
        return maxGapX * maxGapY % MOD
```

436. 1467.py

Path: LeetCode-main/solutions/1467. Probability of a Two Boxes Having The Same Number of Distinct Balls/1467.py

```

from enum import Enum

class BoxCase(Enum):
    EQUAL_DISTANT_BALLS = 0
    EQUAL_BALLS = 1

class Solution:
    def getProbability(self, balls: list[int]) -> float:
        n = sum(balls) // 2
        fact = [1, 1, 2, 6, 24, 120, 720]

        def cases(
            i: int,
            ballsCountA: int,
            ballsCountB: int,
            colorsCountA: int,
            colorsCountB: int,
            boxCase: BoxCase) -> float:
            if ballsCountA > n or ballsCountB > n:
                return 0
            if i == len(balls):
                return (1 if boxCase == BoxCase.EQUAL_BALLS
                        else colorsCountA == colorsCountB)

            ans = 0.0

            # balls taken from A for `balls[i]`
            for ballsTakenA in range(balls[i] + 1):
                ballsTakenB = balls[i] - ballsTakenA
                newcolorsCountA = colorsCountA + (ballsTakenA > 0)
                newcolorsCountB = colorsCountB + (ballsTakenB > 0)
                ans += (cases(i + 1,
                             ballsCountA + ballsTakenA,
                             ballsCountB + ballsTakenB,
                             newcolorsCountA, newcolorsCountB, boxCase) /
                       (fact[ballsTakenA] * fact[ballsTakenB]))

            return ans

        return (cases(0, 0, 0, 0, 0, BoxCase.EQUAL_DISTANT_BALLS) /
                cases(0, 0, 0, 0, 0, BoxCase.EQUAL_BALLS))

```

437. 147.py

Path: LeetCode-main/solutions/147. Insertion Sort List/147.py

```
class Solution:
    def insertionSortList(self, head: ListNode | None) -> ListNode | None:
        dummy = ListNode(0)
        prev = dummy # the last and thus largest of the sorted list

        while head: # the current inserting node
            next = head.next # Cache the next inserting node.
            if prev.val >= head.val:
                prev = dummy # Move `prev` to the front.
            while prev.next and prev.next.val < head.val:
                prev = prev.next
            head.next = prev.next
            prev.next = head
            head = next # Update the current inserting node.

        return dummy.next
```

438. 1470.py

Path: LeetCode-main/solutions/1470. Shuffle the Array/1470.py

```
class Solution:
    def shuffle(self, nums: list[int], n: int) -> list[int]:
        ans = []
        for a, b in zip(nums[:n], nums[n:]):
            ans.append(a)
            ans.append(b)
        return ans
```

439. 1471.py

Path: LeetCode-main/solutions/1471. The k Strongest Values in an Array/1471.py

```
class Solution:
    def getStrongest(self, arr: list[int], k: int) -> list[int]:
        arr.sort()

        ans = []
        median = arr[(len(arr) - 1) // 2]
        l = 0
        r = len(arr) - 1

        for _ in range(k):
            if median - arr[l] > arr[r] - median:
                ans.append(arr[l])
                l += 1
            else:
                ans.append(arr[r])
                r -= 1

        return ans
```

440. 1472-2.py

Path: LeetCode-main/solutions/1472. Design Browser History/1472-2.py

```

class BrowserHistory:
    def __init__(self, homepage: str):
        self.history = []
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.history.append(url)
        self.future = []

    def back(self, steps: int) -> str:
        while len(self.history) > 1 and steps > 0:
            self.future.append(self.history.pop())
            steps -= 1
        return self.history[-1]

    def forward(self, steps: int) -> str:
        while self.future and steps > 0:
            self.history.append(self.future.pop())
            steps -= 1
        return self.history[-1]

```

441. 1472-3.py

Path: LeetCode-main/solutions/1472. Design Browser History/1472-3.py

```

class Node:
    def __init__(self, url: str):
        self.prev = None
        self.next = None
        self.url = url

class BrowserHistory:
    def __init__(self, homepage: str):
        self.curr = Node(homepage)

    def visit(self, url: str) -> None:
        self.curr.next = Node(url)
        self.curr.next.prev = self.curr
        self.curr = self.curr.next

    def back(self, steps: int) -> str:
        while self.curr.prev and steps > 0:
            self.curr = self.curr.prev
            steps -= 1
        return self.curr.url

    def forward(self, steps: int) -> str:
        while self.curr.next and steps > 0:
            self.curr = self.curr.next
            steps -= 1
        return self.curr.url

```

442. 1472.py

Path: LeetCode-main/solutions/1472. Design Browser History/1472.py

```

class BrowserHistory:
    def __init__(self, homepage: str):
        self.urls = []
        self.index = -1
        self.lastIndex = -1
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.index += 1
        if self.index < len(self.urls):
            self.urls[self.index] = url
        else:
            self.urls.append(url)
        self.lastIndex = self.index

    def back(self, steps: int) -> str:
        self.index = max(0, self.index - steps)
        return self.urls[self.index]

    def forward(self, steps: int) -> str:
        self.index = min(self.lastIndex, self.index + steps)
        return self.urls[self.index]

```

443. 1474.py

Path: LeetCode-main/solutions/1474. Delete N Nodes After M Nodes of a Linked List/1474.py

```

class Solution:
    def deleteNodes(
        self,
        head: ListNode | None,
        m: int,
        n: int,
    ) -> ListNode | None:
        curr = head
        prev = None # prev.next == curr

        while curr:
            # Set the m-th node as `prev`.
            for _ in range(m):
                if not curr:
                    break
                prev = curr
                curr = curr.next
            # Set the (m + n + 1)-th node as `curr`.
            for _ in range(n):
                if not curr:
                    break
                curr = curr.next
            # Delete the nodes [m + 1..n - 1].
            prev.next = curr

        return head

```

444. 1475.py

Path: LeetCode-main/solutions/1475. Final Prices With a Special Discount in a Shop/1475.py

```
class Solution:
    def finalPrices(self, prices: list[int]) -> list[int]:
        ans = prices.copy()
        stack = []

        for i, price in enumerate(prices):
            # stack[-1] := i in the problem description.
            while stack and prices[stack[-1]] >= price:
                ans[stack.pop()] -= price
            stack.append(i)

        return ans
```

445. 1476.py

Path: LeetCode-main/solutions/1476. Subrectangle Queries/1476.py

```
class SubrectangleQueries:
    def __init__(self, rectangle: list[list[int]]):
        self.rectangle = rectangle
        self.updates = []

    def updateSubrectangle(self, row1: int, col1: int, row2: int, col2: int,
                           newValue: int) -> None:
        self.updates.append((row1, col1, row2, col2, newValue))

    def getValue(self, row: int, col: int) -> int:
        for r1, c1, r2, c2, v in reversed(self.updates):
            if r1 <= row <= r2 and c1 <= col <= c2:
                return v
        return self.rectangle[row][col]
```

446. 148.py

Path: LeetCode-main/solutions/148. Sort List/148.py

```

class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        def split(head: ListNode, k: int) -> ListNode:
            while k > 1 and head:
                head = head.next
                k -= 1
            rest = head.next if head else None
            if head:
                head.next = None
            return rest

        def merge(l1: ListNode, l2: ListNode) -> tuple:
            dummy = ListNode(0)
            tail = dummy

            while l1 and l2:
                if l1.val < l2.val:
                    l1, l2 = l2, l1
                tail.next = l1
                l1 = l1.next
                tail = tail.next
            tail.next = l1 if l1 else l2
            while tail.next:
                tail = tail.next

            return dummy.next, tail

        length = 0
        curr = head
        while curr:
            length += 1
            curr = curr.next

        dummy = ListNode(0, head)

        k = 1
        while k < length:
            curr = dummy.next
            tail = dummy
            while curr:
                l = curr
                r = split(l, k)
                curr = split(r, k)
                mergedHead, mergedTail = merge(l, r)
                tail.next = mergedHead
                tail = mergedTail
            k *= 2

        return dummy.next

```

447. 1480.py

Path: LeetCode-main/solutions/1480. Running Sum of 1d Array/1480.py

```

class Solution:
    def runningSum(self, nums: list[int]) -> list[int]:
        return itertools.accumulate(nums)

```

448. 1481.py

Path: LeetCode-main/solutions/1481. Least Number of Unique Integers after K Removals/1481.py

```
class Solution:
    def findLeastNumOfUniqueInts(self, arr: list[int], k: int) -> int:
        minHeap = list(collections.Counter(arr).values())
        heapq.heapify(minHeap)

        # Greedily remove the k least frequent numbers to have the least number of unique integers.
        while k > 0:
            k -= heapq.heappop(minHeap)

        return len(minHeap) + (1 if k < 0 else 0)
```

449. 1482.py

Path: LeetCode-main/solutions/1482. Minimum Number of Days to Make m Bouquets/1482.py

```
class Solution:
    def minDays(self, bloomDay: list[int], m: int, k: int) -> int:
        if len(bloomDay) < m * k:
            return -1

        def getBouquetCount(waitingDays: int) -> int:
            """
            Returns the number of bouquets (k flowers needed) can be made after the
            `waitingDays`.
            """
            bouquetCount = 0
            requiredFlowers = k
            for day in bloomDay:
                if day > waitingDays:
                    # Reset `requiredFlowers` since there was not enough adjacent flowers.
                    requiredFlowers = k
                else:
                    requiredFlowers -= 1
                    if requiredFlowers == 0:
                        # Use k adjacent flowers to make a bouquet.
                        bouquetCount += 1
                        requiredFlowers = k
            return bouquetCount

        l = min(bloomDay)
        r = max(bloomDay)

        while l < r:
            mid = (l + r) // 2
            if getBouquetCount(mid) >= m:
                r = mid
            else:
                l = mid + 1

        return l
```

450. 1483.py

Path: LeetCode-main/solutions/1483. Kth Ancestor of a Tree Node/1483.py


```

class TreeAncestor:
    def __init__(self, n: int, parent: list[int]):
        self.maxLevel = n.bit_length()
        # dp[i][j] := node i's 2^j-th ancestor
        self.dp = [[0] * self.maxLevel for _ in range(n)]

        # Node i's 2^0 ancestor is its direct parent
        for i in range(n):
            self.dp[i][0] = parent[i]

        for j in range(1, self.maxLevel):
            for i in range(n):
                if self.dp[i][j - 1] == -1: # There's no such ancestor
                    self.dp[i][j] = -1
                else: # A(i, 2^j) = A(A(i, 2^{j - 1}), 2^{j - 1})
                    self.dp[i][j] = self.dp[self.dp[i][j - 1]][j - 1]

    def getKthAncestor(self, node: int, k: int) -> int:
        for j in range(self.maxLevel):
            if node == -1:
                break
            if k >> j & 1:
                node = self.dp[node][j]
        return node

```

451. 1486.py

Path: LeetCode-main/solutions/1486. XOR Operation in an Array/1486.py

```

class Solution:
    def xorOperation(self, n: int, start: int) -> int:
        return functools.reduce(operator.xor,
                                [start + 2 * i for i in range(n)])

```

452. 1487.py

Path: LeetCode-main/solutions/1487. Making File Names Unique/1487.py

```

class Solution:
    def getFolderNames(self, names: list[str]) -> list[str]:
        ans = []
        nameToSuffix = {}

        for name in names:
            if name in nameToSuffix:
                suffix = nameToSuffix[name] + 1
                newName = self._getName(name, suffix)
                while newName in nameToSuffix:
                    suffix += 1
                    newName = self._getName(name, suffix)
                nameToSuffix[name] = suffix
                nameToSuffix[newName] = 0
                ans.append(newName)
            else:
                nameToSuffix[name] = 0
                ans.append(name)

        return ans

    def _getName(self, name: str, suffix: int) -> str:
        return name + '(' + str(suffix) + ')'

```

453. 1488.py

Path: LeetCode-main/solutions/1488. Avoid Flood in The City/1488.py

```

from sortedcontainers import SortedSet

class Solution:
    def avoidFlood(self, rains: list[int]) -> list[int]:
        ans = [-1] * len(rains)
        lakeIdToFullDay = {}
        emptyDays = SortedSet() # indices of rains[i] == 0

        for i, lakeId in enumerate(rains):
            if lakeId == 0:
                emptyDays.add(i)
                continue

            # The lake was full in a previous day. Greedily find the closest day
            # to make the lake empty.
            if lakeId in lakeIdToFullDay:
                fullDay = lakeIdToFullDay[lakeId]
                emptyDayIndex = emptyDays.bisect_right(fullDay)
                if emptyDayIndex == len(emptyDays): # Not found.
                    return []

                # Empty the lake at this day.
                emptyDay = emptyDays[emptyDayIndex]
                ans[emptyDay] = lakeId
                emptyDays.discard(emptyDay)

            # The lake with `lakeId` becomes full at the day `i`.
            lakeIdToFullDay[lakeId] = i

        # Empty an arbitrary lake if there are remaining empty days.
        for emptyDay in emptyDays:
            ans[emptyDay] = 1

        return ans

```

454. 1489.py

Path: LeetCode-main/solutions/1489. Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree/1489.py

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def findCriticalAndPseudoCriticalEdges(self, n: int, edges: list[list[int]]) -> list[list[int]]:
        criticalEdges = []
        pseudoCriticalEdges = []

        # Record the index information, so edges[i] := (u, v, weight, index).
        for i in range(len(edges)):
            edges[i].append(i)

        # Sort by the weight.
        edges.sort(key=lambda x: x[2])

        def getMSTWeight(
            firstEdge: list[int],
            deletedEdgeIndex: int) -> int | float:
            mstWeight = 0
            uf = UnionFind(n)

            if firstEdge:
                uf.unionByRank(firstEdge[0], firstEdge[1])
                mstWeight += firstEdge[2]

            for u, v, weight, index in edges:
                if index == deletedEdgeIndex:
                    continue
                if uf.find(u) == uf.find(v):
                    continue
                uf.unionByRank(u, v)
                mstWeight += weight

            root = uf.find(0)
            if any(uf.find(i) != root for i in range(n)):
                return math.inf

            return mstWeight

        mstWeight = getMSTWeight([], -1)
```

```

for edge in edges:
    index = edge[3]
    # Deleting the `edge` increases the weight of the MST or makes the MST
    # invalid.
    if getMSTWeight([], index) > mstWeight:
        criticalEdges.append(index)
    # If an edge can be in any MST, we can always add `edge` to the edge set.
    elif getMSTWeight(edge, -1) == mstWeight:
        pseudoCriticalEdges.append(index)

return [criticalEdges, pseudoCriticalEdges]

```

455. 149.py

Path: LeetCode-main/solutions/149. Max Points on a Line/149.py

```

class Solution:
    def maxPoints(self, points: list[list[int]]) -> int:
        ans = 0

        def gcd(a: int, b: int) -> int:
            return a if b == 0 else gcd(b, a % b)

        def getSlope(p: list[int], q: list[int]) -> tuple[int, int]:
            dx = p[0] - q[0]
            dy = p[1] - q[1]
            if dx == 0:
                return (0, p[0])
            if dy == 0:
                return (p[1], 0)
            d = gcd(dx, dy)
            return (dx // d, dy // d)

        for i, p in enumerate(points):
            slopeCount = collections.defaultdict(int)
            samePoints = 1
            maxPoints = 0 # the maximum number of points with the same slope
            for j in range(i + 1, len(points)):
                q = points[j]
                if p == q:
                    samePoints += 1
                else:
                    slope = getSlope(p, q)
                    slopeCount[slope] += 1
                    maxPoints = max(maxPoints, slopeCount[slope])
            ans = max(ans, samePoints + maxPoints)

        return ans

```

456. 1491.py

Path: LeetCode-main/solutions/1491. Average Salary Excluding the Minimum and Maximum Salary/1491.py

```

class Solution:
    def average(self, salary: list[int]) -> float:
        return (sum(salary) - max(salary) - min(salary)) / (len(salary) - 2)

```

457. 1492.py

Path: LeetCode-main/solutions/1492. The kth Factor of n/1492.py

```
class Solution:
    def kthFactor(self, n: int, k: int) -> int:
        # If i is a divisor of n, then n // i is also a divisor of n. So, we can
        # find all the divisors of n by processing the numbers <= sqrt(n).
        factor = 1
        i = 0 # the i-th factor

        while factor < math.isqrt(n):
            if n % factor == 0:
                i += 1
                if i == k:
                    return factor
                factor += 1

        factor = n // factor
        while factor >= 1:
            if n % factor == 0:
                i += 1
                if i == k:
                    return n // factor
                factor -= 1

        return -1
```

458. 1493-2.py

Path: LeetCode-main/solutions/1493. Longest Subarray of 1's After Deleting One Element/1493-2.py

```
class Solution:
    def longestSubarray(self, nums: list[int]) -> int:
        l = 0
        zeros = 0

        for num in nums:
            if num == 0:
                zeros += 1
            if zeros > 1:
                if nums[l] == 0:
                    zeros -= 1
                l += 1

        return len(nums) - l - 1
```

459. 1493.py

Path: LeetCode-main/solutions/1493. Longest Subarray of 1's After Deleting One Element/1493.py

```

class Solution:
    def longestSubarray(self, nums: list[int]) -> int:
        ans = 0
        zeros = 0

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                zeros += 1
            while zeros == 2:
                if nums[l] == 0:
                    zeros -= 1
                l += 1
            ans = max(ans, r - l)

        return ans

```

460. 1494.py

Path: LeetCode-main/solutions/1494. Parallel Courses II/1494.py

```

class Solution:
    def minNumberOfSemesters(
        self,
        n: int,
        relations: list[list[int]],
        k: int,
    ) -> int:
        # dp[i] := the minimum number of semesters to take the courses, where i is
        # the bitmask of the taken courses
        dp = [n] * (1 << n)
        # prereq[i] := bitmask of all dependencies of course i
        prereq = [0] * n

        for prevCourse, nextCourse in relations:
            prereq[nextCourse - 1] |= 1 << prevCourse - 1

        dp[0] = 0 # Don't need time to finish 0 course.

        for i in range(1 << n):
            # the bitmask of all the courses can be taken
            coursesCanBeTaken = 0
            # Can take the j-th course if i contains all of j's prerequisites.
            for j in range(n):
                if (i & prereq[j]) == prereq[j]:
                    coursesCanBeTaken |= 1 << j
            # Don't take any course which is already taken.
            # (i represents set of courses that are already taken)
            coursesCanBeTaken &= ~i
            # Enumerate every bitmask subset of `coursesCanBeTaken`.
            s = coursesCanBeTaken
            while s:
                if s.bit_count() <= k:
                    # Any combination of courses (if <= k) can be taken now.
                    # i | s := combining courses taken with courses can be taken.
                    dp[i | s] = min(dp[i | s], dp[i] + 1)
                    s = (s - 1) & coursesCanBeTaken

        return dp[-1]

```

461. 1497.py

Path: LeetCode-main/solutions/1497. Check If Array Pairs Are Divisible by k/1497.py

```
class Solution:
    def canArrange(self, arr: list[int], k: int) -> bool:
        count = [0] * k

        for a in arr:
            a %= k
            count[a if a >= 0 else a + k] += 1

        return (count[0] % 2 == 0 and
                all(count[i] == count[k - i]
                    for i in range(1, k // 2 + 1)))
```

462. 1498.py

Path: LeetCode-main/solutions/1498. Number of Subsequences That Satisfy the Given Sum Condition/1498.py

```
class Solution:
    def numSubseq(self, nums: list[int], target: int) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        ans = 0

        nums.sort()

        l = 0
        r = n - 1
        while l <= r:
            if nums[l] + nums[r] <= target:
                ans += pow(2, r - l, MOD)
                l += 1
            else:
                r -= 1

        return ans % MOD
```

463. 1499-2.py

Path: LeetCode-main/solutions/1499. Max Value of Equation/1499-2.py

```

class Solution:
    def findMaxValueOfEquation(self, points: list[list[int]], k: int) -> int:
        ans = -math.inf
        maxQ = collections.deque() # (y - x, x)

        for x, y in points:
            # Remove the invalid points, xj - xi > k
            while maxQ and x - maxQ[0][1] > k:
                maxQ.popleft()
            if maxQ:
                ans = max(ans, x + y + maxQ[0][0])
            # Remove the points that contribute less value and have a bigger x.
            while maxQ and y - x >= maxQ[-1][0]:
                maxQ.pop()
            maxQ.append((y - x, x))

        return ans

```

464. 1499.py

Path: LeetCode-main/solutions/1499. Max Value of Equation/1499.py

```

class Solution:
    def findMaxValueOfEquation(self, points: list[list[int]], k: int) -> int:
        ans = -math.inf
        maxHeap = [] # (y - x, x)

        for x, y in points:
            while maxHeap and x + maxHeap[0][1] > k:
                heapq.heappop(maxHeap)
            if maxHeap:
                ans = max(ans, x + y - maxHeap[0][0])
            heapq.heappush(maxHeap, (x - y, -x))

        return ans

```

465. 15.py

Path: LeetCode-main/solutions/15. 3Sum/15.py


```

class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        if len(nums) < 3:
            return []

        ans = []

        nums.sort()

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the
            # remaining numbers in [i + 1, n - 1].
            l = i + 1
            r = len(nums) - 1
            while l < r:
                summ = nums[i] + nums[l] + nums[r]
                if summ == 0:
                    ans.append((nums[i], nums[l], nums[r]))
                    l += 1
                    r -= 1
                    while nums[l] == nums[l - 1] and l < r:
                        l += 1
                    while nums[r] == nums[r + 1] and l < r:
                        r -= 1
                elif summ < 0:
                    l += 1
                else:
                    r -= 1

        return ans

```

466. 150.py

Path: LeetCode-main/solutions/150. Evaluate Reverse Polish Notation/150.py

```

class Solution:
    def evalRPN(self, tokens: list[str]) -> int:
        stack = []
        op = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b),
        }

        for token in tokens:
            if token in op:
                b = stack.pop()
                a = stack.pop()
                stack.append(op[token](a, b))
            else:
                stack.append(int(token))

        return stack.pop()

```

467. 1500.py

Path: LeetCode-main/solutions/1500. Design a File Sharing System/1500.py

```
from sortedcontainers import SortedSet

class FileSharing:
    def __init__(self, m: int):
        self.userToChunks: dict[int, SortedSet[int]] = {}
        self.chunkToUsers: dict[int, SortedSet[int]] = {}
        self.availableUserIds: list[int] = []

    def join(self, ownedChunks: list[int]) -> int:
        userId = (heapq.heappop(self.availableUserIds) if self.availableUserIds
                  else len(self.userToChunks) + 1)
        self.userToChunks[userId] = SortedSet(ownedChunks)
        for chunk in ownedChunks:
            self.chunkToUsers.setdefault(chunk, SortedSet()).add(userId)
        return userId

    def leave(self, userID: int) -> None:
        if userID not in self.userToChunks:
            return
        for chunk in self.userToChunks[userID]:
            self.chunkToUsers[chunk].discard(userID)
            if not self.chunkToUsers[chunk]:
                del self.chunkToUsers[chunk]
        del self.userToChunks[userID]
        heapq.heappush(self.availableUserIds, userID)

    def request(self, userID: int, chunkID: int) -> list[int]:
        if chunkID not in self.chunkToUsers:
            return []
        userIds = list(self.chunkToUsers[chunkID])
        self.userToChunks[userID].add(chunkID)
        self.chunkToUsers[chunkID].add(userID)
        return userIds
```

468. 1504-2.py

Path: LeetCode-main/solutions/1504. Count Submatrices With All Ones/1504-2.py

```

class Solution:
    def numSubmat(self, mat: list[list[int]]) -> int:
        ans = 0
        hist = [0] * len(mat[0])

        for row in mat:
            for i, num in enumerate(row):
                hist[i] = 0 if num == 0 else hist[i] + 1
            ans += self._count(hist)

        return ans

    def _count(self, hist: list[int]) -> int:
        # submatrices[i] := the number of submatrices, where the i-th column is the
        # right border
        submatrices = [0] * len(hist)
        stack = []

        for i, h in enumerate(hist):
            while stack and hist[stack[-1]] >= h:
                stack.pop()
            if stack:
                prevIndex = stack[-1]
                submatrices[i] = submatrices[prevIndex] + h * (i - prevIndex)
            else:
                submatrices[i] = h * (i + 1)
            stack.append(i)

        return sum(submatrices)

```

469. 1504.py

Path: LeetCode-main/solutions/1504. Count Submatrices With All Ones/1504.py

```

class Solution:
    def numSubmat(self, mat: list[list[int]]) -> int:
        m = len(mat)
        n = len(mat[0])
        ans = 0

        for baseRow in range(m):
            row = [1] * n
            for i in range(baseRow, m):
                for j in range(n):
                    row[j] &= mat[i][j]
                ans += self._count(row)

        return ans

    def _count(self, row: list[int]) -> int:
        res = 0
        length = 0
        for num in row:
            length = 0 if num == 0 else length + 1
            res += length
        return res

```

470. 1506.py

Path: LeetCode-main/solutions/1506. Find Root of N-Ary Tree/1506.py

```
class Solution:
    def findRoot(self, tree: list['Node']) -> 'Node':
        sum = 0

        for node in tree:
            sum ^= node.val
            for child in node.children:
                sum ^= child.val

        for node in tree:
            if node.val == sum:
                return node
```

471. 1507.py

Path: LeetCode-main/solutions/1507. Reformat Date/1507.py

```
class Solution:
    def reformatDate(self, date: str) -> str:
        monthToNumString = {
            'Jan': '01', 'Feb': '02', 'Mar': '03', 'Apr': '04',
            'May': '05', 'Jun': '06', 'Jul': '07', 'Aug': '08',
            'Sep': '09', 'Oct': '10', 'Nov': '11', 'Dec': '12',
        }
        day, month, year = date.split()
        day = day[:-2] if len(day) == 4 else '0' + day[:-2]
        return f'{year}-{monthToNumString[month]}-{day}'
```

472. 1509.py

Path: LeetCode-main/solutions/1509. Minimum Difference Between Largest and Smallest Value in Three Moves/1509.py

```
class Solution:
    def minDifference(self, nums: list[int]) -> int:
        n = len(nums)
        if n < 5:
            return 0

        ans = math.inf

        nums.sort()

        # 1. Change nums[0..i) to nums[i].
        # 2. Change nums[n - 3 + i..n) to nums[n - 4 + i].
        for i in range(4):
            ans = min(ans, nums[n - 4 + i] - nums[i])

        return ans
```

473. 151.py

Path: LeetCode-main/solutions/151. Reverse Words in a String/151.py

```
class Solution:
    def reverseWords(self, s: str) -> str:
        return ' '.join(reversed(s.split()))
```

474. 1514.py

Path: LeetCode-main/solutions/1514. Path with Maximum Probability/1514.py

```
class Solution:
    def maxProbability(
        self,
        n: int,
        edges: list[list[int]],
        succProb: list[float],
        start: int,
        end: int,
    ) -> float:
        graph = [[] for _ in range(n)] # {a: [(b, probability_ab)]}
        maxHeap = [(-1.0, start)] # (the probability to reach u, u)
        seen = [False] * n

        for i, ((u, v), prob) in enumerate(zip(edges, succProb)):
            graph[u].append((v, prob))
            graph[v].append((u, prob))

        while maxHeap:
            prob, u = heapq.heappop(maxHeap)
            prob *= -1
            if u == end:
                return prob
            if seen[u]:
                continue
            seen[u] = True
            for nextNode, edgeProb in graph[u]:
                if seen[nextNode]:
                    continue
                heapq.heappush(maxHeap, (-prob * edgeProb, nextNode))

        return 0
```

475. 1515.py

Path: LeetCode-main/solutions/1515. Best Position for a Service Centre/1515.py

```
class Solution:
    def getMinDistSum(self, positions: list[list[int]]) -> float:
        def distSum(a: float, b: float) -> float:
            return sum(math.sqrt((a - x)**2 + (b - y)**2)
                        for x, y in positions)

        ERR = 1e-6
        currX = 50
        currY = 50
        ans = distSum(currX, currY)
        step = 1

        while step > ERR:
            shouldDecreaseStep = True
            for dx, dy in [(0, step), (0, -step), (step, 0), (-step, 0)]:
                x = currX + dx
                y = currY + dy
                newDistSum = distSum(x, y)
                if newDistSum < ans:
                    ans = newDistSum
                    currX = x
                    currY = y
                    shouldDecreaseStep = False
            if shouldDecreaseStep:
                step /= 10

        return ans
```

476. 1516.py

Path: LeetCode-main/solutions/1516. Move Sub-Tree of N-Ary Tree/1516.py

```

class Solution:
    def moveSubTree(self, root: 'Node', p: 'Node', q: 'Node') -> 'Node':
        if p in q.children:
            return root

        # Create a dummy Node for the case when root == p
        dummy = Node(None, [root])

        # Get each parent of p and q
        pParent = self._getParent(dummy, p)
        qParent = self._getParent(p, q)

        # Get p's original index in p's parent
        pIndex = pParent.children.index(p)
        pParent.children.pop(pIndex)

        q.children.append(p)

        # If q is in the p's subtree, qParent != None
        if qParent:
            qParent.children.remove(q)
            pParent.children.insert(pIndex, q)

        return dummy.children[0]

    def _getParent(self, root: 'Node', target: 'Node') -> Optional['Node']:
        for child in root.children:
            if child == target:
                return root
            res = self._getParent(child, target)
            if res:
                return res
        return None

```

477. 152.py

Path: LeetCode-main/solutions/152. Maximum Product Subarray/152.py

```

class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        ans = nums[0]
        dpMin = nums[0] # the minimum so far
        dpMax = nums[0] # the maximum so far

        for i in range(1, len(nums)):
            num = nums[i]
            prevMin = dpMin # dpMin[i - 1]
            prevMax = dpMax # dpMax[i - 1]
            if num < 0:
                dpMin = min(prevMax * num, num)
                dpMax = max(prevMin * num, num)
            else:
                dpMin = min(prevMin * num, num)
                dpMax = max(prevMax * num, num)

            ans = max(ans, dpMax)

        return ans

```

478. 1521.py

Path: LeetCode-main/solutions/1521. Find a Value of a Mysterious Function Closest to Target/1521.py

```
class Solution:
    def closestToTarget(self, arr: list[int], target: int) -> int:
        ans = math.inf
        dp = set() # all the values of subarrays that end in the current number

        for num in arr:
            # Extend each subarray that ends in the previous number. Due to
            # monotonicity of the AND operation, the size of `dp` will be at most
            # num.bit_count() + 1.
            dp = {num} | {val & num for val in dp}
            ans = min(ans, min(abs(target - val) for val in dp))

        return ans
```

479. 1522.py

Path: LeetCode-main/solutions/1522. Diameter of N-Ary Tree/1522.py

```
class Solution:
    def diameter(self, root: 'Node') -> int:
        ans = 0

        def maxDepth(root: 'Node') -> int:
            """Returns the maximum depth of the subtree rooted at `root`."""
            nonlocal ans
            maxSubDepth1 = 0
            maxSubDepth2 = 0
            for child in root.children:
                maxSubDepth = maxDepth(child)
                if maxSubDepth > maxSubDepth1:
                    maxSubDepth2 = maxSubDepth1
                    maxSubDepth1 = maxSubDepth
                elif maxSubDepth > maxSubDepth2:
                    maxSubDepth2 = maxSubDepth
            ans = max(ans, maxSubDepth1 + maxSubDepth2)
            return 1 + maxSubDepth1

        maxDepth(root)
        return ans
```

480. 1525.py

Path: LeetCode-main/solutions/1525. Number of Good Ways to Split a String/1525.py


```

class Solution:
    def numSplits(self, s: str) -> int:
        n = len(s)
        ans = 0
        seen = set()
        prefix = [0] * n
        suffix = [0] * n

        for i in range(n):
            seen.add(s[i])
            prefix[i] = len(seen)

        seen.clear()

        for i in reversed(range(n)):
            seen.add(s[i])
            suffix[i] = len(seen)

        for i in range(n - 1):
            if prefix[i] == suffix[i + 1]:
                ans += 1

        return ans

```

481. 1526.py

Path: LeetCode-main/solutions/1526. Minimum Number of Increments on Subarrays to Form a Target Array/1526.py

```

class Solution:
    def minNumberOperations(self, target: list[int]) -> int:
        ans = target[0]

        for a, b in zip(target, target[1:]):
            if a < b:
                ans += b - a

        return ans

```

482. 153.py

Path: LeetCode-main/solutions/153. Find Minimum in Rotated Sorted Array/153.py

```

class Solution:
    def findMin(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] < nums[r]:
                r = m
            else:
                l = m + 1

        return nums[l]

```

483. 1531.py

Path: LeetCode-main/solutions/1531. String Compression II/1531.py

```
class Solution:
    def getLengthOfOptimalCompression(self, s: str, k: int) -> int:
        def getLength(maxFreq: int) -> int:
            """Returns the length to compress `maxFreq`."""
            if maxFreq == 1:
                return 1 # c
            if maxFreq < 10:
                return 2 # [1-9]c
            if maxFreq < 100:
                return 3 # [1-9][0-9]c
            return 4 # [1-9][0-9][0-9]c

        @functools.lru_cache(None)
        def dp(i: int, k: int) -> int:
            """Returns the length of optimal dp of s[i..n) with at most k deletion."""
            if k < 0:
                return math.inf
            if i == len(s) or len(s) - i <= k:
                return 0

            ans = math.inf
            maxFreq = 0 # the maximum frequency in s[i..j]
            count = collections.Counter()

            # Make letters in s[i..j] be the same.
            # Keep the letter that has the maximum frequency in this range and remove
            # the other letters.
            for j in range(i, len(s)):
                count[s[j]] += 1
                maxFreq = max(maxFreq, count[s[j]])
                ans = min(ans, getLength(maxFreq) +
                        dp(j + 1, k - (j - i + 1 - maxFreq)))

            return ans

        return dp(0, k)
```

484. 1533-2.py

Path: LeetCode-main/solutions/1533. Find the Index of the Large Integer/1533-2.py

```

# """
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
# class ArrayReader(object):
#     # Compares the sum of arr[l..r] with the sum of arr[x..y]
#     # return 1 if sum(arr[l..r]) > sum(arr[x..y])
#     # return 0 if sum(arr[l..r]) == sum(arr[x..y])
#     # return -1 if sum(arr[l..r]) < sum(arr[x..y])
#     def compareSub(self, l: int, r: int, x: int, y: int) -> int:
#
#
#     # Returns the length of the array
#     def length(self) -> int:
#
#
class Solution:
    def getIndex(self, reader: 'ArrayReader') -> int:
        l = 0
        r = reader.length() - 1

        while l < r:
            m = (l + r) // 2
            res = (reader.compareSub(l, m, m + 1, r) if (r - l + 1) % 2 == 0
                  else reader.compareSub(l, m, m, r))
            if res == -1:
                l = m + 1
            else: # res == 1 or res == 0
                r = m

        return l

```

485. 1533.py

Path: LeetCode-main/solutions/1533. Find the Index of the Large Integer/1533.py

```

# """
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
# class ArrayReader(object):
#     # Compares the sum of arr[l..r] with the sum of arr[x..y]
#     # return 1 if sum(arr[l..r]) > sum(arr[x..y])
#     # return 0 if sum(arr[l..r]) == sum(arr[x..y])
#     # return -1 if sum(arr[l..r]) < sum(arr[x..y])
#     def compareSub(self, l: int, r: int, x: int, y: int) -> int:
#
#
#     # Returns the length of the array
#     def length(self) -> int:
#
#
class Solution:
    def getIndex(self, reader: 'ArrayReader') -> int:
        l = 0
        r = reader.length() - 1

        while l < r:
            m = (l + r) // 2
            if (r - l) % 2 == 0:
                res = reader.compareSub(l, m - 1, m + 1, r)
                if res == 0:
                    return m
                if res == 1:
                    r = m - 1
                else: # res == -1
                    l = m + 1
            else:
                res = reader.compareSub(l, m, m + 1, r)
                # res is either 1 or -1.
                if res == 1:
                    r = m
                else: # res == -1
                    l = m + 1

        return l

```

486. 1534.py

Path: LeetCode-main/solutions/1534. Count Good Triplets/1534.py

```

class Solution:
    def countGoodTriplets(self, arr: list[int], a: int, b: int, c: int) -> int:
        return sum(abs(arr[i] - arr[j]) <= a and
                   abs(arr[j] - arr[k]) <= b and
                   abs(arr[i] - arr[k]) <= c
                   for i, j, k in itertools.combinations(range(len(arr)), 3))

```

487. 1535.py

Path: LeetCode-main/solutions/1535. Find the Winner of an Array Game/1535.py

```
class Solution:
    def getWinner(self, arr: list[int], k: int) -> int:
        ans = arr[0]
        wins = 0

        i = 1
        while i < len(arr) and wins < k:
            if arr[i] > ans:
                ans = arr[i]
                wins = 1
            else:
                wins += 1
            i += 1

        return ans
```

488. 1536.py

Path: LeetCode-main/solutions/1536. Minimum Swaps to Arrange a Binary Grid/1536.py

```
class Solution:
    def minSwaps(self, grid: list[list[int]]) -> int:
        n = len(grid)
        ans = 0

        # suffixZeros[i] := the number of suffix zeros in the i-th row
        suffixZeros = [n if 1 not in row else row[::-1].index(1) for row in grid]

        for i in range(n):
            neededZeros = n - 1 - i
            # Get the first row with suffix zeros >= `neededZeros` in suffixZeros[i:n).
            j = next((j for j in range(i, n) if suffixZeros[j] >= neededZeros), -1)
            if j == -1:
                return -1

            # Move the rows[j] to the rows[i].
            for k in range(j, i, -1):
                suffixZeros[k] = suffixZeros[k - 1]
            ans += j - i

        return ans
```

489. 1537.py

Path: LeetCode-main/solutions/1537. Get the Maximum Score/1537.py

```

class Solution:
    def maxSum(self, nums1: list[int], nums2: list[int]) -> int:
        # Keep the running the sum of `nums1` and `nums2` before the next rendezvous.
        # Since `nums1` and `nums2` are increasing, move forward on the smaller one
        # to ensure we don't miss any rendezvous. When meet rendezvous, choose the
        # better path.
        ans = 0
        sum1 = 0 # sum(nums1) in (the prevoious rendezvous, the next rendezvous)
        sum2 = 0 # sum(nums2) in (the prevoious rendezvous, the next rendezvous)
        i = 0 # nums1's index
        j = 0 # nums2's index

        while i < len(nums1) and j < len(nums2):
            if nums1[i] < nums2[j]:
                sum1 += nums1[i]
                i += 1
            elif nums1[i] > nums2[j]:
                sum2 += nums2[j]
                j += 1
            else: # An rendezvous happens.
                ans += max(sum1, sum2) + nums1[i]
                sum1 = 0
                sum2 = 0
                i += 1
                j += 1

        while i < len(nums1):
            sum1 += nums1[i]
            i += 1

        while j < len(nums2):
            sum2 += nums2[j]
            j += 1

        return (ans + max(sum1, sum2)) % (10**9 + 7)

```

490. 1538.py

Path: LeetCode-main/solutions/1538. Guess the Majority in a Hidden Array/1538.py

```

# """
# This is the ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
# class ArrayReader(object):
#     # Compares 4 different elements in the array
#     # Returns 4 if the values of the 4 elements are the same (0 or 1).
#     # Returns 2 if three elements have a value equal to 0 and one element has
#     #     value equal to 1 or vice versa.
#     # Returns 0 if two element have a value equal to 0 and two elements have a
#     #     value equal to 1.
#     def query(self, a: int, b: int, c: int, d: int) -> int:
#
#
#     # Returns the length of the array
#     def length(self) -> int:
#
#
class Solution:
    def guessMajority(self, reader: 'ArrayReader') -> int:
        n = reader.length()
        query0123 = reader.query(0, 1, 2, 3)
        query1234 = reader.query(1, 2, 3, 4)
        zeros = 1 # the number of numbers that are same as `nums[0]`
        nonZeros = 0 # the number of numbers that are different from `nums[0]`
        indexNot0 = -1 # any index i s.t. nums[i] != nums[0]

        # Find which group nums[1..3] belong to.
        for i in range(1, 4):
            abcd = [0] + [num for num in [1, 2, 3] if num != i] + [4]
            if reader.query(*abcd) == query1234: # nums[i] == nums[0]
                zeros += 1
            else:
                nonZeros += 1
                indexNot0 = i

        # Find which group nums[4..n] belong to.
        for i in range(4, n):
            if reader.query(1, 2, 3, i) == query0123: # nums[i] == nums[0]
                zeros += 1
            else:
                nonZeros += 1
                indexNot0 = i

        if zeros == nonZeros:
            return -1
        if zeros > nonZeros:
            return 0
        return indexNot0

```

491. 154.py

Path: LeetCode-main/solutions/154. Find Minimum in Rotated Sorted Array II/154.py

```
class Solution:
    def findMin(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] == nums[r]:
                r -= 1
            elif nums[m] < nums[r]:
                r = m
            else:
                l = m + 1

        return nums[l]
```

492. 1540-2.py

Path: LeetCode-main/solutions/1540. Can Convert String in K Moves/1540-2.py

```
class Solution:
    def canConvertString(self, s: str, t: str, k: int) -> bool:
        if len(s) != len(t):
            return False

        # e.g. s = "aab", t = "bbc", so shiftCount[1] = 3
        # 1. a -> b, need 1 move.
        # 2. a -> b, need 1 + 26 moves.
        # 3. b -> c, need 1 + 26 * 2 moves.
        shiftCount = [0] * 26

        for a, b in zip(s, t):
            shift = (ord(b) - ord(a) + 26) % 26
            if shift == 0:
                continue
            if shift + 26 * shiftCount[shift] > k:
                return False
            shiftCount[shift] += 1

        return True
```

493. 1540.py

Path: LeetCode-main/solutions/1540. Can Convert String in K Moves/1540.py


```

class Solution:
    def canConvertString(self, s: str, t: str, k: int) -> bool:
        if len(s) != len(t):
            return False

        # e.g. s = "aab", t = "bbc", so shiftCount[1] = 3
        # 1. a -> b, need 1 move.
        # 2. a -> b, need 1 + 26 moves.
        # 3. b -> c, need 1 + 26 * 2 moves.
        shiftCount = [0] * 26

        for a, b in zip(s, t):
            shiftCount[(ord(b) - ord(a) + 26) % 26] += 1

        for shift in range(1, 26):
            if shift + 26 * (shiftCount[shift] - 1) > k:
                return False

        return True

```

494. 1541.py

Path: LeetCode-main/solutions/1541. Minimum Insertions to Balance a Parentheses String/1541.py

```

class Solution:
    def minInsertions(self, s: str) -> int:
        neededRight = 0 # Increment by 2 for each '('.
        missingLeft = 0 # Increment by 1 for each missing '('.
        missingRight = 0 # Increment by 1 for each missing ')'.

        for c in s:
            if c == '(':
                if neededRight % 2 == 1:
                    # e.g. '()(...'
                    missingRight += 1
                    neededRight -= 1
                neededRight += 2
            else: # c == ')'
                neededRight -= 1
                if neededRight < 0:
                    # e.g. '())...'
                    missingLeft += 1
                    neededRight += 2

        return neededRight + missingLeft + missingRight

```

495. 1542.py

Path: LeetCode-main/solutions/1542. Find Longest Awesome Substring/1542.py

```

class Solution:
    def longestAwesome(self, s: str) -> int:
        ans = 0
        prefix = 0 # the binary prefix
        prefixToIndex = [len(s)] * 1024
        prefixToIndex[0] = -1

        for i, c in enumerate(s):
            prefix ^= 1 << int(c)
            ans = max(ans, i - prefixToIndex[prefix])
            for j in range(10):
                ans = max(ans, i - prefixToIndex[prefix ^ 1 << j])
            prefixToIndex[prefix] = min(prefixToIndex[prefix], i)

        return ans

```

496. 1544.py

Path: LeetCode-main/solutions/1544. Make The String Great/1544.py

```

class Solution:
    def makeGood(self, s: str) -> str:
        ans = []
        for c in s:
            if ans and self._is_bad_pair(ans[-1], c):
                ans.pop()
            else:
                ans.append(c)
        return ''.join(ans)

    def _is_bad_pair(self, a: str, b: str) -> bool:
        return a != b and a.lower() == b.lower()

```

497. 1545.py

Path: LeetCode-main/solutions/1545. Find Kth Bit in Nth Binary String/1545.py

```

class Solution:
    def findKthBit(self, n: int, k: int) -> str:
        if n == 1:
            return '0'
        midIndex = pow(2, n - 1) # 1-indexed
        if k == midIndex:
            return '1'
        if k < midIndex:
            return self.findKthBit(n - 1, k)
        return '1' if self.findKthBit(n - 1, midIndex * 2 - k) == '0' else '0'

```

498. 1546.py

Path: LeetCode-main/solutions/1546. Maximum Number of Non-Overlapping Subarrays With Sum Equals Target/1546.py

```

class Solution:
    def maxNonOverlapping(self, nums: list[int], target: int) -> int:
        # Ending the subarray ASAP always has a better result.
        ans = 0
        prefix = 0
        prefixes = {0}

        # Greedily find the subarrays that equal to the target.
        for num in nums:
            # Check if there is a subarray ends in here and equals to the target.
            prefix += num
            if prefix - target in prefixes:
                # Find one and discard all the prefixes that have been used.
                ans += 1
                prefix = 0
                prefixes = {0}
            else:
                prefixes.add(prefix)

        return ans

```

499. 1547-2.py

Path: LeetCode-main/solutions/1547. Minimum Cost to Cut a Stick/1547-2.py

```

class Solution:
    def minCost(self, n: int, cuts: list[int]) -> int:
        arr = sorted([0] + cuts + [n])

        dp = [[0] * len(arr) for _ in range(len(arr))]

        for d in range(2, len(arr)):
            for i in range(len(arr) - d):
                j = i + d
                dp[i][j] = math.inf
                for k in range(i + 1, j):
                    dp[i][j] = min(dp[i][j], arr[j] - arr[i] + dp[i][k] + dp[k][j])

        return dp[0][len(arr) - 1]

```

500. 1547.py

Path: LeetCode-main/solutions/1547. Minimum Cost to Cut a Stick/1547.py

```

class Solution:
    def minCost(self, n: int, cuts: list[int]) -> int:
        A = sorted([0] + cuts + [n])

        @functools.lru_cache(None)
        def dp(i, j):
            if j - i <= 1:
                return 0

            return min(A[j] - A[i] + dp(i, k) + dp(k, j) for k in range(i + 1, j))

        return dp(0, len(A) - 1)

```

501. 1548.py

Path: LeetCode-main/solutions/1548. The Most Similar Path in a Graph/1548.py

```
class Solution:
    def mostSimilar(self, n: int, roads: list[list[int]], names: list[str],
                    targetPath: list[str]) -> list[int]:
        # cost[i][j] := the minimum cost to start from names[i] in path[j]
        cost = [[-1] * len(targetPath) for _ in range(len(names))]
        # next[i][j] := the best next of names[i] in path[j]
        next = [[0] * len(targetPath) for _ in range(len(names))]
        graph = [[] for _ in range(n)]

        for u, v in roads:
            graph[u].append(v)
            graph[v].append(u)

        minDist = math.inf
        start = 0

        def dfs(nameIndex: int, pathIndex: int) -> int:
            if cost[nameIndex][pathIndex] != -1:
                return cost[nameIndex][pathIndex]

            editDist = names[nameIndex] != targetPath[pathIndex]
            if pathIndex == len(targetPath) - 1:
                return editDist

            minDist = math.inf

            for v in graph[nameIndex]:
                dist = dfs(v, pathIndex + 1)
                if dist < minDist:
                    minDist = dist
                    next[nameIndex][pathIndex] = v

            cost[nameIndex][pathIndex] = editDist + minDist
            return editDist + minDist

        for i in range(n):
            dist = dfs(i, 0)
            if dist < minDist:
                minDist = dist
                start = i

        ans = []

        while len(ans) < len(targetPath):
            ans.append(start)
            start = next[start][len(ans) - 1]

        return ans
```

502. 155.py

Path: LeetCode-main/solutions/155. Min Stack/155.py

```

class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, x: int) -> None:
        mn = x if not self.stack else min(self.stack[-1][1], x)
        self.stack.append([x, mn])

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]

```

503. 1550.py

Path: LeetCode-main/solutions/1550. Three Consecutive Odds/1550.py

```

class Solution:
    def threeConsecutiveOdds(self, arr: list[int]) -> bool:
        count = 0
        for a in arr:
            count = 0 if a % 2 == 0 else count + 1
            if count == 3:
                return True
        return False

```

504. 1551.py

Path: LeetCode-main/solutions/1551. Minimum Operations to Make Array Equal/1551.py

```

class Solution:
    def minOperations(self, n: int) -> int:
        def arr(self, i: int) -> int:
            """Returns the i-th element of `arr`, where 1 <= i <= n."""
            return (i - 1) * 2 + 1

        # median := median of arr
        # diffs[i] := median - arr[i] where i <= i <= n // 2
        # ans := sum(diffs)
        # e.g.
        # n = 5, arr = [1, 3, 5, 7, 9], diffs = [4, 2]
        # ans = (4 + 2) * 2 // 2 = 6
        # n = 6, arr = [1, 3, 5, 7, 9, 11], diffs = [5, 3, 1]
        # ans = (5 + 1) * 3 // 2 = 9
        halfSize = n // 2
        median = (arr(n) + arr(1)) // 2
        firstDiff = median - arr(1)
        lastDiff = median - arr(halfSize)
        return (firstDiff + lastDiff) * halfSize // 2

```

505. 1552.py

Path: LeetCode-main/solutions/1552. Magnetic Force Between Two Balls/1552.py

```
class Solution:
    def maxDistance(self, position: list[int], m: int) -> int:
        position.sort()

        l = 1
        r = position[-1] - position[0]

        def numBalls(force: int) -> int:
            balls = 0
            prevPosition = -force
            for pos in position:
                if pos - prevPosition >= force:
                    balls += 1
                    prevPosition = pos
            return balls

        while l < r:
            mid = r - (r - l) // 2
            if numBalls(mid) >= m:
                l = mid
            else:
                r = mid - 1

        return l
```

506. 1553.py

Path: LeetCode-main/solutions/1553. Minimum Number of Days to Eat N Oranges/1553.py

```
class Solution:
    @functools.lru_cache(None)
    def minDays(self, n: int) -> int:
        if n <= 1:
            return n
        return 1 + min(self.minDays(n // 3) + n % 3,
                       self.minDays(n // 2) + n % 2)
```

507. 1554-2.py

Path: LeetCode-main/solutions/1554. Strings Differ by One Character/1554-2.py

```

class Solution:
    def differByOne(self, dict: list[str]) -> bool:
        BASE = 26
        HASH = 1_000_000_007
        m = len(dict[0])

        def val(c: str) -> int:
            return ord(c) - ord('a')

        def getHash(s: str) -> int:
            """Returns the hash of `s`. Assume the length of `s` is m.

            e.g. getHash(s) = 26^(m - 1) * s[0] + 26^(m - 2) * s[1] + ... + s[m - 1].
            """
            hash = 0
            for c in s:
                hash = (hash * BASE + val(c))
            return hash

        wordToHash = [getHash(word) for word in dict]

        # Compute the hash without each letter.
        # e.g. hash of "abc" = 26^2 * 'a' + 26 * 'b' + 'c'
        #   newHash of "a*c" = hash - 26 * 'b'
        coefficient = 1
        for j in range(m - 1, -1, -1):
            newHashToIndices = collections.defaultdict(list)
            for i, (word, hash) in enumerate(zip(dict, wordToHash)):
                newHash = (hash - coefficient * val(word[j]) % HASH + HASH) % HASH
                if any(word[: j] == dict[index][: j] and word[j + 1:] ==
                    dict[index][j + 1:] for index in newHashToIndices[newHash]):
                    return True
                newHashToIndices[newHash].append(i)
            coefficient = coefficient * BASE % HASH

        return False

```

508. 1556.py

Path: LeetCode-main/solutions/1556. Thousand Separator/1556.py

```

class Solution:
    def thousandSeparator(self, n: int) -> str:
        return f'{n:,}'.replace(',', '.')

```

509. 1557.py

Path: LeetCode-main/solutions/1557. Minimum Number of Vertices to Reach All Nodes/1557.py

```
class Solution:
    def findSmallestSetOfVertices(
        self,
        n: int,
        edges: list[list[int]],
    ) -> list[int]:
        inDegrees = [0] * n

        for _, v in edges:
            inDegrees[v] += 1

        return [i for i, d in enumerate(inDegrees) if d == 0]
```

510. 1558.py

Path: LeetCode-main/solutions/1558. Minimum Numbers of Function Calls to Make Target Array/1558.py

```
class Solution:
    def minOperations(self, nums: list[int]) -> int:
        mx = max(nums)
        return (sum(num.bit_count() for num in nums) +
                (0 if mx == 0 else mx.bit_length() - 1))
```

511. 156-2.py

Path: LeetCode-main/solutions/156. Binary Tree Upside Down/156-2.py

```
class Solution:
    def upsideDownBinaryTree(self, root: TreeNode | None) -> TreeNode | None:
        prevRoot = None
        prevRightChild = None

        while root:
            nextRoot = root.left # Cache the next root.
            root.left = prevRightChild
            prevRightChild = root.right
            root.right = prevRoot
            prevRoot = root # Record the previous root.
            root = nextRoot # Update the root.

        return prevRoot
```

512. 156.py

Path: LeetCode-main/solutions/156. Binary Tree Upside Down/156.py


```

class Solution:
    def upsideDownBinaryTree(self, root: TreeNode | None) -> TreeNode | None:
        if not root or not root.left:
            return root

        newRoot = self.upsideDownBinaryTree(root.left)
        root.left.left = root.right # 2's left = 3 (root's right)
        root.left.right = root # 2's right = 1 (root)
        root.left = None
        root.right = None
        return newRoot

```

513. 1560.py

Path: LeetCode-main/solutions/1560. Most Visited Sector in a Circular Track/1560.py

```

class Solution:
    def mostVisited(self, n: int, rounds: list[int]) -> list[int]:
        # 1. if start <= end, [start, end] is the most visited.
        #
        #      s ----- n
        # 1 ----- n
        # 1 ----- e
        #
        # 2. if start > end, [1, end] and [start, n] are the most visited.
        #
        #      s -- n
        # 1 ----- n
        # 1 ----- e
        start = rounds[0]
        end = rounds[-1]
        if start <= end:
            return range(start, end + 1)
        return list(range(1, end + 1)) + list(range(start, n + 1))

```

514. 1561.py

Path: LeetCode-main/solutions/1561. Maximum Number of Coins You Can Get/1561.py

```

class Solution:
    def maxCoins(self, piles: list[int]) -> int:
        return sum(sorted(piles)[len(piles) // 3::2])

```

515. 1562.py

Path: LeetCode-main/solutions/1562. Find Latest Group of Size M/1562.py

```

class Solution:
    def findLatestStep(self, arr: list[int], m: int) -> int:
        if len(arr) == m:
            return len(arr)

        ans = -1
        step = 0
        # sizes[i] := the size of the group starting from i or ending in i
        # (1-indexed)
        sizes = [0] * (len(arr) + 2)

        for i in arr:
            step += 1
            # In the previous step, there exists a group with a size of m.
            if sizes[i - 1] == m or sizes[i + 1] == m:
                ans = step - 1
            head = i - sizes[i - 1]
            tail = i + sizes[i + 1]
            sizes[head] = tail - head + 1
            sizes[tail] = tail - head + 1

        return ans

```

516. 1564-2.py

Path: LeetCode-main/solutions/1564. Put Boxes Into the Warehouse I/1564-2.py

```

class Solution:
    def maxBoxesInWarehouse(self, boxes: list[int], warehouse: list[int]) -> int:
        i = 0 # warehouse's index

        for box in sorted(boxes, reverse=True):
            if i < len(warehouse) and warehouse[i] >= box:
                i += 1

        return i

```

517. 1564.py

Path: LeetCode-main/solutions/1564. Put Boxes Into the Warehouse I/1564.py

```

class Solution:
    def maxBoxesInWarehouse(self, boxes: list[int], warehouse: list[int]) -> int:
        realWarehouse = [warehouse[0]]

        for i in range(1, len(warehouse)):
            realWarehouse.append(min(realWarehouse[-1], warehouse[i]))

        boxes.sort()
        i = 0 # boxes' index
        for height in reversed(realWarehouse):
            if i < len(boxes) and boxes[i] <= height:
                i += 1

        return i

```

518. 1566.py

Path: LeetCode-main/solutions/1566. Detect Pattern of Length M Repeated K or More Times/1566.py

```
class Solution:
    def containsPattern(self, arr: list[int], m: int, k: int) -> bool:
        count = 0
        for i in range(m, len(arr)):
            count = count + 1 if arr[i] == arr[i - m] else 0
            if count == m * k - m:
                return True
        return False
```

519. 1567.py

Path: LeetCode-main/solutions/1567. Maximum Length of Subarray With Positive Product/1567.py

```
class Solution:
    def getMaxLen(self, nums: list[int]) -> int:
        ans = 0
        # the maximum length of subarrays ending in `num` with a negative product
        neg = 0
        # the maximum length of subarrays ending in `num` with a positive product
        pos = 0

        for num in nums:
            pos = 0 if num == 0 else pos + 1
            neg = 0 if num == 0 or neg == 0 else neg + 1
            if num < 0:
                pos, neg = neg, pos
            ans = max(ans, pos)

        return ans
```

520. 1568.py

Path: LeetCode-main/solutions/1568. Minimum Number of Days to Disconnect Island/1568.py

```

class Solution:
    def minDays(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])

        def dfs(grid: list[list[int]], i: int, j: int, seen: set[tuple[int, int]]):
            seen.add((i, j))
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if grid[x][y] == 0 or (x, y) in seen:
                    continue
                dfs(grid, x, y, seen)

        def disconnected(grid: list[list[int]]) -> bool:
            islandsCount = 0
            seen = set()
            for i in range(m):
                for j in range(n):
                    if grid[i][j] == 0 or (i, j) in seen:
                        continue
                    if islandsCount > 1:
                        return True
                    islandsCount += 1
                    dfs(grid, i, j, seen)
            return islandsCount != 1

        if disconnected(grid):
            return 0

        # Try to remove 1 land.
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    grid[i][j] = 0
                    if disconnected(grid):
                        return 1
                    grid[i][j] = 1

        # Remove 2 lands.
        return 2

```

521. 157.py

Path: LeetCode-main/solutions/157. Read N Characters Given Read4/157.py

```

"""
The read4 API is already defined for you.
    def read4(buf4: list[str]) -> int:

# Below is an example of how the read4 API can be called.
file = File("abcdefghijk") # File is "abcdefghijk", initially file pointer (fp) points to 'a'
buf4 = [' '] * 4 # Create buffer with enough space to store characters
read4(buf4) # Read4 returns 4. Now buf = ['a','b','c','d'], fp points to 'e'
read4(buf4) # Read4 returns 4. Now buf = ['e','f','g','h'], fp points to 'i'
read4(buf4) # Read4 returns 3. Now buf = ['i','j','k',...], fp points to end of file
"""

```

```

class Solution:
    def read(self, buf: list[str], n: int) -> int:
        buf4 = [' '] * 4
        i4 = 0 # buf4's index
        n4 = 0 # buf4's size
        i = 0 # buf's index

        while i < n:
            if i4 == n4: # All the characters in the buf4 are consumed.
                i4 = 0 # Reset the buf4's index.
                n4 = read4(buf4) # Read <= 4 characters from the file to the buf4.
                if n4 == 0: # Reach the EOF.
                    return i
            buf[i] = buf4[i4]
            i += 1
            i4 += 1

        return i

```

522. 1572.py

Path: LeetCode-main/solutions/1572. Matrix Diagonal Sum/1572.py

```

class Solution:
    def diagonalSum(self, mat: list[list[int]]) -> int:
        n = len(mat)
        ans = 0

        for i in range(n):
            ans += mat[i][i]
            ans += mat[n - 1 - i][i]

        return ans if n % 2 == 0 else ans - mat[n // 2][n // 2]

```

523. 1573.py

Path: LeetCode-main/solutions/1573. Number of Ways to Split a String/1573.py

```
class Solution:
    def numWays(self, s: str) -> int:
        MOD = 1_000_000_007
        ones = s.count('1')
        if ones % 3 != 0:
            return 0
        if ones == 0:
            n = len(s)
            return (n - 1) * (n - 2) // 2 % MOD

        s1End = -1
        s2Start = -1
        s2End = -1
        s3Start = -1
        onesSoFar = 0

        for i, c in enumerate(s):
            if c == '1':
                onesSoFar += 1
            if s1End == -1 and onesSoFar == ones // 3:
                s1End = i
            elif s2Start == -1 and onesSoFar == ones // 3 + 1:
                s2Start = i
            if s2End == -1 and onesSoFar == ones // 3 * 2:
                s2End = i
            elif s3Start == -1 and onesSoFar == ones // 3 * 2 + 1:
                s3Start = i

        return (s2Start - s1End) * (s3Start - s2End) % MOD
```

524. 1574.py

Path: LeetCode-main/solutions/1574. Shortest Subarray to be Removed to Make Array Sorted/1574.py

```

class Solution:
    def findLengthOfShortestSubarray(self, arr: list[int]) -> int:
        n = len(arr)
        l = 0
        r = n - 1

        # arr[0..l] is non-decreasing.
        while l < n - 1 and arr[l + 1] >= arr[l]:
            l += 1
        # arr[r..n - 1] is non-decreasing.
        while r > 0 and arr[r - 1] <= arr[r]:
            r -= 1
        # Remove arr[l + 1..n - 1] or arr[0..r - 1].
        ans = min(n - 1 - l, r)

        # Since arr[0..l] and arr[r..n - 1] are non-decreasing, we place pointers
        # at the rightmost indices, l and n - 1, and greedily shrink them toward
        # the leftmost indices, 0 and r, respectively. By removing arr[i + 1..j],
        # we ensure that `arr` becomes non-decreasing.
        i = l
        j = n - 1
        while i >= 0 and j >= r and j > i:
            if arr[i] <= arr[j]:
                j -= 1
            else:
                i -= 1
            ans = min(ans, j - i)

        return ans

```

525. 1575-2.py

Path: LeetCode-main/solutions/1575. Count All Possible Routes/1575-2.py

```
class Solution:
    def countRoutes(
        self,
        locations: list[int],
        start: int,
        finish: int,
        fuel: int,
    ) -> int:
        MOD = 1_000_000_007
        n = len(locations)
        # dp[i][j] := the number of ways to reach the `finish` city from the i-th
        # city with `j` fuel
        dp = [[0] * (fuel + 1) for _ in range(n)]

        for f in range(fuel + 1):
            dp[finish][f] = 1

        for f in range(fuel + 1):
            for i in range(n):
                for j in range(n):
                    if i == j:
                        continue
                    requiredFuel = abs(locations[i] - locations[j])
                    if requiredFuel <= f:
                        dp[i][f] += dp[j][f - requiredFuel]
                        dp[i][f] %= MOD

        return dp[start][fuel]
```

526. 1575.py

Path: LeetCode-main/solutions/1575. Count All Possible Routes/1575.py


```

class Solution:
    def countRoutes(
        self,
        locations: list[int],
        start: int,
        finish: int,
        fuel: int,
    ) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(i: int, fuel: int) -> int:
            """
            Returns the number of ways to reach the `finish` city from the i-th city
            with `fuel` fuel.
            """
            if fuel < 0:
                return 0

            res = 1 if i == finish else 0
            for j in range(len(locations)):
                if j == i:
                    continue
                res += dp(j, fuel - abs(locations[i] - locations[j]))
                res %= MOD

            return res

        return dp(start, fuel)

```

527. 1576.py

Path: LeetCode-main/solutions/1576. Replace All ?'s to Avoid Consecutive Repeating Characters/1576.py

```

class Solution:
    def modifyString(self, s: str) -> str:
        ans = []

        def nextAvailable(ans: list[int], s: str, i: int) -> str:
            c = 'a'
            while ((i > 0 and ans[i - 1] == c) or
                    (i + 1 < len(s) and c == s[i + 1])):
                c = chr(ord(c) + 1)
            return c

        for i, c in enumerate(s):
            if c == '?':
                ans.append(nextAvailable(ans, s, i))
            else:
                ans.append(c)

        return ''.join(ans)

```

528. 1577.py

Path: LeetCode-main/solutions/1577. Number of Ways Where Square of Number Is Equal to Product of Two Numbers/1577.py

```

class Solution:
    def numTriplets(self, nums1: list[int], nums2: list[int]) -> int:
        def countTriplets(A: list[int], B: list[int]):
            """Returns the number of triplet (i, j, k) if A[i]^2 == B[j] * B[k]."""
            res = 0
            count = collections.Counter(B)

            for a in A:
                target = a * a
                for b, freq in count.items():
                    if target % b > 0 or target // b not in count:
                        continue
                    if target // b == b:
                        res += freq * (freq - 1)
                    else:
                        res += freq * count[target // b]

            return res // 2

        return countTriplets(nums1, nums2) + countTriplets(nums2, nums1)

```

529. 1578.py

Path: LeetCode-main/solutions/1578. Minimum Time to Make Rope Colorful/1578.py

```

class Solution:
    def minCost(self, colors: str, neededTime: list[int]) -> int:
        ans = 0
        maxNeededTime = neededTime[0]

        for i in range(1, len(colors)):
            if colors[i] == colors[i - 1]:
                ans += min(maxNeededTime, neededTime[i])
                # For each continuous group, Bob needs to remove every balloon except
                # the one with the maximum `neededTime`. So, he should hold the balloon
                # with the highest `neededTime` in his hand.
                maxNeededTime = max(maxNeededTime, neededTime[i])
            else:
                # If the current balloon is different from the previous one, discard
                # the balloon from the previous group and hold the new one in hand.
                maxNeededTime = neededTime[i]

        return ans

```

530. 1579.py

Path: LeetCode-main/solutions/1579. Remove Max Number of Edges to Keep Graph Fully Traversable/1579.py

```

class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1
        return True

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def maxNumEdgesToRemove(self, n: int, edges: list[list[int]]) -> int:
        alice = UnionFind(n)
        bob = UnionFind(n)
        requiredEdges = 0

        # Greedily put type 3 edges in the front.
        for type_, u, v in sorted(edges, reverse=True):
            u -= 1
            v -= 1
            if type_ == 3: # Can be traversed by Alice and Bob.
                # Note that we should use | instead of or because if the first
                # expression is True, short-circuiting will skip the second
                # expression.
                if alice.unionByRank(u, v) | bob.unionByRank(u, v):
                    requiredEdges += 1
            elif type_ == 2: # Can be traversed by Bob.
                if bob.unionByRank(u, v):
                    requiredEdges += 1
            else: # type == 1 Can be traversed by Alice.
                if alice.unionByRank(u, v):
                    requiredEdges += 1

        return (len(edges) - requiredEdges
                if alice.count == 1 and bob.count == 1
                else -1)

```

531. 158.py

Path: LeetCode-main/solutions/158. Read N Characters Given Read4 II - Call multiple times/158.py

```
# The read4 API is already defined for you.
# Def read4(buf4: list[str]) -> int:

class Solution:
    def read(self, buf: list[str], n: int) -> int:
        i = 0 # buf's index

        while i < n:
            if self.i4 == self.n4: # All the characters in the buf4 are consumed.
                self.i4 = 0 # Reset the buf4's index.
                # Read <= 4 characters from the file to the buf4.
                self.n4 = read4(self.buf4)
                if self.n4 == 0: # Reach the EOF.
                    return i
            buf[i] = self.buf4[self.i4]
            i += 1
            self.i4 += 1

        return i

buf4 = [' '] * 4
i4 = 0 # buf4's index
n4 = 0 # buf4's size
```

532. 1580.py

Path: LeetCode-main/solutions/1580. Put Boxes Into the Warehouse II/1580.py

```
class Solution:
    def maxBoxesInWarehouse(self, boxes: list[int], warehouse: list[int]) -> int:
        l = 0
        r = len(warehouse) - 1

        for box in sorted(boxes, reverse=True):
            if l > r:
                return len(warehouse)
            if box <= warehouse[l]:
                l += 1
            elif box <= warehouse[r]:
                r -= 1

        return l + (len(warehouse) - r - 1)
```

533. 1582.py

Path: LeetCode-main/solutions/1582. Special Positions in a Binary Matrix/1582.py

```

class Solution:
    def numSpecial(self, mat: list[list[int]]) -> int:
        m = len(mat)
        n = len(mat[0])
        ans = 0
        rowOnes = [0] * m
        colOnes = [0] * n

        for i in range(m):
            for j in range(n):
                if mat[i][j] == 1:
                    rowOnes[i] += 1
                    colOnes[j] += 1

        for i in range(m):
            for j in range(n):
                if mat[i][j] == 1 and rowOnes[i] == 1 and colOnes[j] == 1:
                    ans += 1

        return ans

```

534. 1583.py

Path: LeetCode-main/solutions/1583. Count Unhappy Friends/1583.py

```

class Solution:
    def unhappyFriends(
        self,
        n: int,
        preferences: list[list[int]],
        pairs: list[list[int]],
    ) -> int:
        ans = 0
        matches = [0] * n
        prefer = [{ } for _ in range(n)]

        for x, y in pairs:
            matches[x] = y
            matches[y] = x

        for i in range(n):
            for j in range(n - 1):
                prefer[i][preferences[i][j]] = j

        for x in range(n):
            for u in prefer[x].keys():
                y = matches[x]
                v = matches[u]
                if prefer[x][u] < prefer[x][y] and prefer[u][x] < prefer[u][v]:
                    ans += 1
                    break

        return ans

```

535. 1584.py

Path: LeetCode-main/solutions/1584. Min Cost to Connect All Points/1584.py

```

class Solution:
    def minCostConnectPoints(self, points: list[int]) -> int:
        # dist[i] := the minimum distance to connect the points[i]
        dist = [math.inf] * len(points)
        ans = 0

        for i in range(len(points) - 1):
            for j in range(i + 1, len(points)):
                # Try to connect the points[i] with the points[j].
                dist[j] = min(dist[j], abs(points[i][0] - points[j][0]) +
                              abs(points[i][1] - points[j][1]))

                # Swap the points[j] (the point with the minimum distance) with the
                # points[i + 1].
                if dist[j] < dist[i + 1]:
                    points[j], points[i + 1] = points[i + 1], points[j]
                    dist[j], dist[i + 1] = dist[i + 1], dist[j]
            ans += dist[i + 1]

        return ans

```

536. 1585.py

Path: LeetCode-main/solutions/1585. Check If String Is Transformable With Substring Sort Operations/1585.py

```

class Solution:
    def isTransformable(self, s: str, t: str) -> bool:
        if collections.Counter(s) != collections.Counter(t):
            return False

        positions = [collections.deque() for _ in range(10)]

        for i, c in enumerate(s):
            positions[int(c)].append(i)

        # For each digit in `t`, check if we can put this digit in `s` at the same
        # position as `t`. Ensure that all the left digits are equal to or greater
        # than it. This is because the only operation we can perform is sorting in
        # ascending order. If there is a digit to the left that is smaller than it,
        # we can never move it to the same position as in `t`. However, if all the
        # digits to its left are equal to or greater than it, we can move it one
        # position to the left until it reaches the same position as in `t`.
        for c in t:
            d = int(c)
            front = positions[d].popleft()
            for smaller in range(d):
                if positions[smaller] and positions[smaller][0] < front:
                    return False

        return True

```

537. 1586.py

Path: LeetCode-main/solutions/1586. Binary Search Tree Iterator II/1586.py

```

class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.prevsAndCurr = []
        self.nexts = []
        self._pushLeftsUntilNull(root)

    def hasNext(self) -> bool:
        return len(self.nexts) > 0

    def next(self) -> int:
        root, fromNext = self.nexts.pop()
        if fromNext:
            self._pushLeftsUntilNull(root.right)
        self.prevsAndCurr.append(root)
        return root.val

    def hasPrev(self) -> bool:
        return len(self.prevsAndCurr) > 1

    def prev(self) -> int:
        self.nexts.append((self.prevsAndCurr.pop(), False))
        return self.prevsAndCurr[-1].val

    def _pushLeftsUntilNull(self, root):
        while root:
            self.nexts.append((root, True))
            root = root.left

```

538. 1588.py

Path: LeetCode-main/solutions/1588. Sum of All Odd Length Subarrays/1588.py

```

class Solution:
    def sumOddLengthSubarrays(self, arr: list[int]) -> int:
        ans = 0
        # Maintain two sums of subarrays ending in the previous index.
        # Each time we meet a new number, we'll consider 'how many times' it should
        # contribute to the newly built subarrays by calculating the number of
        # previous even/odd-length subarrays.
        prevEvenSum = 0 # the sum of even-length subarrays
        prevOddSum = 0 # the sum of odd-length subarrays

        for i, a in enumerate(arr):
            # (i + 1) // 2 := the number of previous odd-length subarrays.
            currEvenSum = prevOddSum + ((i + 1) // 2) * a
            # i // 2 + 1 := the number of previous even-length subarrays
            # (including 0).
            currOddSum = prevEvenSum + (i // 2 + 1) * a
            ans += currOddSum
            prevEvenSum = currEvenSum
            prevOddSum = currOddSum

        return ans

```

539. 1589.py

Path: LeetCode-main/solutions/1589. Maximum Sum Obtained of Any Permutation/1589.py

```

class Solution:
    def maxSumRangeQuery(self, nums: list[int], requests: list[list[int]]) -> int:
        MOD = 1_000_000_007
        ans = 0
        # count[i] := the number of times nums[i] has been requested
        count = [0] * len(nums)

        for start, end in requests:
            count[start] += 1
            if end + 1 < len(nums):
                count[end + 1] -= 1

        for i in range(1, len(nums)):
            count[i] += count[i - 1]

        for num, c in zip(sorted(nums), sorted(count)):
            ans += num * c
            ans %= MOD

        return ans

```

540. 159.py

Path: LeetCode-main/solutions/159. Longest Substring with At Most Two Distinct Characters/159.py

```

class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        ans = 0
        distinct = 0
        count = [0] * 128

        l = 0
        for r, c in enumerate(s):
            count[ord(c)] += 1
            if count[ord(c)] == 1:
                distinct += 1
            while distinct == 3:
                count[ord(s[l])] -= 1
                if count[ord(s[l])] == 0:
                    distinct -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

541. 1590.py

Path: LeetCode-main/solutions/1590. Make Sum Divisible by P/1590.py


```
class Solution:
    def minSubarray(self, nums: list[int], p: int) -> int:
        summ = sum(nums)
        remainder = summ % p
        if remainder == 0:
            return 0

        ans = len(nums)
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += num
            prefix %= p
            target = (prefix - remainder + p) % p
            if target in prefixToIndex:
                ans = min(ans, i - prefixToIndex[target])
            prefixToIndex[prefix] = i

        return -1 if ans == len(nums) else ans
```

542. 1591.py

Path: LeetCode-main/solutions/1591. Strange Printer II/1591.py

```

from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def isPrintable(self, targetGrid: list[list[int]]) -> bool:
        MAX_COLOR = 60
        m = len(targetGrid)
        n = len(targetGrid[0])

        # graph[u] := {v1, v2} means v1 and v2 cover u
        graph = [set() for _ in range(MAX_COLOR + 1)]

        for color in range(1, MAX_COLOR + 1):
            # Get the rectangle of the current color.
            minI = m
            minJ = n
            maxI = -1
            maxJ = -1
            for i in range(m):
                for j in range(n):
                    if targetGrid[i][j] == color:
                        minI = min(minI, i)
                        minJ = min(minJ, j)
                        maxI = max(maxI, i)
                        maxJ = max(maxJ, j)

            # Add any color covering the current as the children.
            for i in range(minI, maxI + 1):
                for j in range(minJ, maxJ + 1):
                    if targetGrid[i][j] != color:
                        graph[color].add(targetGrid[i][j])

        states = [State.INIT] * (MAX_COLOR + 1)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            if any(hasCycle(v) for v in graph[u]):
                return True
            states[u] = State.VISITED
            return False

        return not (any(hasCycle(i) for i in range(1, MAX_COLOR + 1)))

```

543. 1595.py

Path: LeetCode-main/solutions/1595. Minimum Cost to Connect Two Groups of Points/1595.py

```

class Solution:
    def connectTwoGroups(self, cost: list[list[int]]) -> int:
        # minCosts[j] := the minimum cost of connecting group2's point j
        minCosts = [min(col) for col in zip(*cost)]

        @functools.lru_cache(None)
        def dp(i: int, mask: int) -> int:
            """
            Returns the minimum cost to connect group1's points[i..n) with group2's
            points, where `mask` is the bitmask of the connected points in group2.
            """
            if i == len(cost):
                # All the points in group 1 are connected, so greedily assign the
                # minimum cost for the unconnected points of group2.
                return sum(minCost for j, minCost in enumerate(minCosts)
                           if (mask >> j & 1) == 0)
            return min(cost[i][j] + dp(i + 1, mask | 1 << j)
                       for j in range(len(cost[0])))

        return dp(0, 0)

```

544. 16.py

Path: LeetCode-main/solutions/16. 3Sum Closest/16.py

```

class Solution:
    def threeSumClosest(self, nums: list[int], target: int) -> int:
        ans = nums[0] + nums[1] + nums[2]

        nums.sort()

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the
            # remaining numbers in [i + 1, n - 1].
            l = i + 1
            r = len(nums) - 1
            while l < r:
                summ = nums[i] + nums[l] + nums[r]
                if summ == target:
                    return summ
                if abs(summ - target) < abs(ans - target):
                    ans = summ
                if summ < target:
                    l += 1
                else:
                    r -= 1

        return ans

```

545. 160.py

Path: LeetCode-main/solutions/160. Intersection of Two Linked Lists/160.py

```
class Solution:
    def getIntersectionNode(
        self,
        headA: ListNode,
        headB: ListNode,
    ) -> ListNode | None:
        a = headA
        b = headB

        while a != b:
            a = a.next if a else headB
            b = b.next if b else headA

        return a
```

546. 1602.py

Path: LeetCode-main/solutions/1602. Find Nearest Right Node in Binary Tree/1602.py

```
class Solution:
    def findNearestRightNode(
        self,
        root: TreeNode,
        u: TreeNode,
    ) -> TreeNode | None:
        ans = None
        targetDepth = -1

        def dfs(root: TreeNode, depth: int) -> None:
            nonlocal ans
            nonlocal targetDepth
            if not root:
                return
            if root == u:
                targetDepth = depth
                return
            if depth == targetDepth and not ans:
                ans = root
                return
            dfs(root.left, depth + 1)
            dfs(root.right, depth + 1)

        dfs(root, 0)
        return ans
```

547. 1603.py

Path: LeetCode-main/solutions/1603. Design Parking System/1603.py

```
class ParkingSystem:
    def __init__(self, big: int, medium: int, small: int):
        self.count = [big, medium, small]

    def addCar(self, carType: int) -> bool:
        self.count[carType - 1] -= 1
        return self.count[carType - 1] >= 0
```

548. 1604.py

Path: LeetCode-main/solutions/1604. Alert Using Same Key-Card Three or More Times in a One Hour Period/1604.py

```
class Solution:
    def alertNames(self, keyName: list[str], keyTime: list[str]) -> list[str]:
        nameToMinutes = collections.defaultdict(list)

        for name, time in zip(keyName, keyTime):
            minutes = self._getMinutes(time)
            nameToMinutes[name].append(minutes)

        return sorted([name for name, minutes in nameToMinutes.items()
                        if self._hasAlert(minutes)])

    def _hasAlert(self, minutes: list[int]) -> bool:
        if len(minutes) > 70:
            return True
        minutes.sort()
        for i in range(2, len(minutes)):
            if minutes[i - 2] + 60 >= minutes[i]:
                return True
        return False

    def _getMinutes(self, time: str) -> int:
        h, m = map(int, time.split(':'))
        return 60 * h + m
```

549. 1605.py

Path: LeetCode-main/solutions/1605. Find Valid Matrix Given Row and Column Sums/1605.py

```
class Solution:
    def restoreMatrix(self, rowSum: list[int],
                      colSum: list[int]) -> list[list[int]]:
        m = len(rowSum)
        n = len(colSum)
        ans = [[0] * n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                ans[i][j] = min(rowSum[i], colSum[j])
                rowSum[i] -= ans[i][j]
                colSum[j] -= ans[i][j]

        return ans
```

550. 1608.py

Path: LeetCode-main/solutions/1608. Special Array With X Elements Greater Than or Equal X/1608.py

```

class Solution:
    def specialArray(self, nums: list[int]) -> int:
        nums.sort()

        if nums[0] >= len(nums):
            return len(nums)

        for i, (a, b) in enumerate(itertools.pairwise(nums)):
            count = len(nums) - i - 1
            if a < count and b >= count:
                return count

        return -1

```

551. 1609.py

Path: LeetCode-main/solutions/1609. Even Odd Tree/1609.py

```

class Solution:
    def isEvenOddTree(self, root: TreeNode | None) -> bool:
        q = collections.deque([root])
        isEven = True

        while q:
            prevVal = -math.inf if isEven else math.inf
            for _ in range(sz):
                node = q.popleft()
                if isEven and (node.val % 2 == 0 or node.val <= prevVal):
                    return False # invalid case on even level
                if not isEven and (node.val % 2 == 1 or node.val >= prevVal):
                    return False # invalid case on odd level
                prevVal = node.val
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            isEven = not isEven

        return True

```

552. 161.py

Path: LeetCode-main/solutions/161. One Edit Distance/161.py

```

class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        m = len(s)
        n = len(t)
        if m > n: # Make sure that |s| <= |t|.
            return self.isOneEditDistance(t, s)

        for i in range(m):
            if s[i] != t[i]:
                if m == n:
                    return s[i + 1:] == t[i + 1:] # Replace s[i] with t[i].
                return s[i:] == t[i + 1:] # Delete t[i].

        return m + 1 == n # Delete t[-1].

```

553. 1610.py

Path: LeetCode-main/solutions/1610. Maximum Number of Visible Points/1610.py

```
class Solution:
    def visiblePoints(
        self,
        points: list[list[int]],
        angle: int,
        location: list[int],
    ) -> int:
        posX, posY = location
        maxVisible = 0
        same = 0
        A = []

        for x, y in points:
            if x == posX and y == posY:
                same += 1
            else:
                A.append(math.atan2(y - posY, x - posX))

        A.sort()
        A = A + [a + 2.0 * math.pi for a in A]

        angleInRadians = math.pi * (angle / 180)

        l = 0
        for r in range(len(A)):
            while A[r] - A[l] > angleInRadians:
                l += 1
            maxVisible = max(maxVisible, r - l + 1)

        return maxVisible + same
```

554. 1611.py

Path: LeetCode-main/solutions/1611. Minimum One Bit Operations to Make Integers Zero/1611.py

```

class Solution:
    def minimumOneBitOperations(self, n: int) -> int:
        # Observation: e.g. n = 2^2
        #      100 (2^2 needs 2^3 - 1 ops)
        # op1 -> 101
        # op2 -> 111
        # op1 -> 110
        # op2 -> 010 (2^1 needs 2^2 - 1 ops)
        # op1 -> 011
        # op2 -> 001 (2^0 needs 2^1 - 1 ops)
        # op1 -> 000
        #
        # So 2^k needs 2^(k + 1) - 1 ops. Note this is reversible, i.e., 0 -> 2^k
        # also takes 2^(k + 1) - 1 ops.

        # e.g. n = 1XXX, our first goal is to change 1XXX -> 1100.
        # - If the second bit is 1, you only need to consider the cost of turning
        #   the last 2 bits to 0.
        # - If the second bit is 0, you need to add up the cost of flipping the
        #   second bit from 0 to 1.
        # XOR determines the cost minimumOneBitOperations(1XXX^1100) accordingly.
        # Then, 1100 -> 0100 needs 1 op. Finally, 0100 -> 0 needs 2^3 - 1 ops.
        if n == 0:
            return 0

        # x is the largest 2^k <= n.
        # x | x >> 1 -> x >> 1 needs 1 op.
        #   x >> 1 -> 0       needs x = 2^k - 1 ops.
        x = 1 << n.bit_length() - 1
        return self.minimumOneBitOperations(n ^ (x | x >> 1)) + 1 + x - 1

```

555. 1612.py

Path: LeetCode-main/solutions/1612. Check If Two Expression Trees are Equivalent/1612.py

```

class Solution:
    def checkEquivalence(self, root1: 'Node', root2: 'Node') -> bool:
        count = collections.Counter()

        def dfs(root: 'Node', add: int) -> None:
            if not root:
                return
            if 'a' <= root.val <= 'z':
                count[root.val] += add
            dfs(root.left, add)
            dfs(root.right, add)

        dfs(root1, 1)
        dfs(root2, -1)
        return all(value == 0 for value in count.values())

```

556. 1614.py

Path: LeetCode-main/solutions/1614. Maximum Nesting Depth of the Parentheses/1614.py


```
class Solution:
    def maxDepth(self, s: str) -> int:
        ans = 0
        opened = 0

        for c in s:
            if c == '(':
                opened += 1
                ans = max(ans, opened)
            elif c == ')':
                opened -= 1

        return ans
```

557. 1615.py

Path: LeetCode-main/solutions/1615. Maximal Network Rank/1615.py

```

class Solution:
    def maximalNetworkRank(self, n: int, roads: list[list[int]]) -> int:
        degrees = [0] * n

        for u, v in roads:
            degrees[u] += 1
            degrees[v] += 1

        # Find the first maximum and the second maximum degrees.
        maxDegree1 = 0
        maxDegree2 = 0
        for degree in degrees:
            if degree > maxDegree1:
                maxDegree2 = maxDegree1
                maxDegree1 = degree
            elif degree > maxDegree2:
                maxDegree2 = degree

        # There can be multiple nodes with `maxDegree1` or `maxDegree2`.
        # Find the counts of such nodes.
        countMaxDegree1 = 0
        countMaxDegree2 = 0
        for degree in degrees:
            if degree == maxDegree1:
                countMaxDegree1 += 1
            elif degree == maxDegree2:
                countMaxDegree2 += 1

        if countMaxDegree1 == 1:
            # 1. If there is only one node with degree = `maxDegree1`, then we'll
            # need to use the node with degree = `maxDegree2`. The answer in general
            # will be (maxDegree1 + maxDegree2), but if the two nodes that we're
            # considering are connected, then we'll have to subtract 1.
            edgeCount = (self._getEdgeCount(roads, degrees, maxDegree1, maxDegree2) +
                         self._getEdgeCount(roads, degrees, maxDegree2, maxDegree1))
            return maxDegree1 + maxDegree2 - (countMaxDegree2 == edgeCount)
        else:
            # 2. If there are more than one node with degree = `maxDegree1`, then we
            # can consider `maxDegree1` twice, and we don't need to use `maxDegree2`.
            # The answer in general will be 2 * maxDegree1, but if the two nodes that
            # we're considering are connected, then we'll have to subtract 1.
            edgeCount = self._getEdgeCount(roads, degrees, maxDegree1, maxDegree1)
            maxPossibleEdgeCount = countMaxDegree1 * (countMaxDegree1 - 1) // 2
            return 2 * maxDegree1 - (maxPossibleEdgeCount == edgeCount)

    def _getEdgeCount(
        self,
        roads: list[list[int]],
        degrees: list[int],
        degreeU: int, degreeV: int,
    ) -> int:
        """
        Returns the number of edges (u, v) where degrees[u] == degreeU and
        degrees[v] == degreeV.
        """
        edgeCount = 0
        for u, v in roads:
            if degrees[u] == degreeU and degrees[v] == degreeV:
                edgeCount += 1
        return edgeCount

```

Path: LeetCode-main/solutions/1616. Split Two Strings to Make Palindrome/1616.py

```
class Solution:
    def checkPalindromeFormation(self, a: str, b: str) -> bool:
        return self._check(a, b) or self._check(b, a)

    def _check(self, a: str, b: str) -> bool:
        i, j = 0, len(a) - 1
        while i < j:
            if a[i] != b[j]:
                # a[0:i] + a[i..j] + b[j + 1:] or
                # a[0:i] + b[i..j] + b[j + 1:]
                return self._isPalindrome(a, i, j) or self._isPalindrome(b, i, j)
            i += 1
            j -= 1
        return True

    def _isPalindrome(self, s: str, i: int, j: int) -> bool:
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True
```

559. 1617.py

Path: LeetCode-main/solutions/1617. Count Subtrees With Max Distance Between Cities/1617.py

```

class Solution:
    def countSubgraphsForEachDiameter(
        self,
        n: int,
        edges: list[list[int]],
    ) -> list[int]:
        maxMask = 1 << n
        dist = self._floydWarshall(n, edges)
        ans = [0] * (n - 1)

        # mask := the subset of the cities
        for mask in range(maxMask):
            maxDist = self._getMaxDist(mask, dist, n)
            if maxDist > 0:
                ans[maxDist - 1] += 1

        return ans

    def _floydWarshall(self, n: int, edges: list[list[int]]) -> list[list[int]]:
        dist = [[n] * n for _ in range(n)]

        for i in range(n):
            dist[i][i] = 0

        for u, v in edges:
            dist[u - 1][v - 1] = 1
            dist[v - 1][u - 1] = 1

        for k in range(n):
            for i in range(n):
                for j in range(n):
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

        return dist

    def _getMaxDist(self, mask: int, dist: list[list[int]], n: int) -> int:
        maxDist = 0
        edgeCount = 0
        cityCount = 0
        for u in range(n):
            if (mask >> u) & 1 == 0: # u is not in the subset.
                continue
            cityCount += 1
        for v in range(u + 1, n):
            if (mask >> v) & 1 == 0: # v is not in the subset.
                continue
            if dist[u][v] == 1: # u and v are connected.
                edgeCount += 1
            maxDist = max(maxDist, dist[u][v])

        return maxDist if edgeCount == cityCount - 1 else 0

```

560. 1618.py

Path: LeetCode-main/solutions/1618. Maximum Font to Fit a Sentence in a Screen/1618.py

```

# """
# This is FontInfo's API interface.
# You should not implement it, or speculate about its implementation
# """
# class FontInfo(object):
#     Return the width of char ch when fontSize is used.
#     def getWidth(self, fontSize: int, ch: str) -> int:
#         pass
#
#     def getHeight(self, fontSize: int) -> int:
#         pass
class Solution:
    def maxFont(
        self,
        text: str,
        w: int,
        h: int,
        fonts: list[int],
        fontInfo: 'FontInfo',
    ) -> int:
        count = collections.Counter(text)
        l = 0
        r = len(fonts) - 1

        while l < r:
            m = (l + r + 1) // 2
            if fontInfo.getHeight(
                fonts[m]) <= h and self._getWidthSum(
                    count, fonts[m],
                    fontInfo) <= w:
                l = m
            else:
                r = m - 1

        return fonts[l] if self._getWidthSum(count, fonts[l], fontInfo) <= w else -1

    def _getWidthSum(
        self,
        count: list[int],
        font: int,
        fontInfo: 'FontInfo',
    ) -> int:
        width = 0
        for c in string.ascii_lowercase:
            width += count[c] * fontInfo.getWidth(font, c)
        return width

```

561. 1619.py

Path: LeetCode-main/solutions/1619. Mean of Array After Removing Some Elements/1619.py

```

class Solution:
    def trimMean(self, arr: list[int]) -> float:
        arr.sort()
        offset = len(arr) // 20
        return mean(arr[offset:-offset])

```

562. 162.py

Path: LeetCode-main/solutions/162. Find Peak Element/162.py

```
class Solution:
    def findPeakElement(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] >= nums[m + 1]:
                r = m
            else:
                l = m + 1

        return l
```

563. 1620.py

Path: LeetCode-main/solutions/1620. Coordinate With Maximum Network Quality/1620.py

```
class Solution:
    def bestCoordinate(self, towers: list[list[int]], radius: int) -> list[int]:
        MAX = 50
        n = len(towers)
        ans = [0] * 2
        maxQuality = 0

        def dist(tower: list[int], i: int, j: int) -> float:
            """Returns the distance between the tower and the coordinate."""
            return math.sqrt((tower[0] - i)**2 + (tower[1] - j)**2)

        for i in range(MAX + 1):
            for j in range(MAX + 1):
                qualitySum = 0
                for tower in towers:
                    q = tower[2]
                    d = dist(tower, i, j)
                    if d <= radius:
                        qualitySum += int(q / (1 + d))
                if qualitySum > maxQuality:
                    maxQuality = qualitySum
                    ans = [i, j]

        return ans
```

564. 1621.py

Path: LeetCode-main/solutions/1621. Number of Sets of K Non-Overlapping Line Segments/1621.py

```

class Solution:
    def numberOfSets(self, n: int, k: int) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(i: int, k: int, drawing: bool) -> int:
            if k == 0: # Find a way to draw k segments.
                return 1
            if i == n: # Reach the end.
                return 0
            if drawing:
                # 1. Keep drawing at i and move to i + 1.
                # 2. Stop at i so decrease k. We can start from i for the next segment.
                return (dp(i + 1, k, True) + dp(i, k - 1, False)) % MOD
            # 1. Skip i and move to i + 1.
            # 2. Start at i and move to i + 1.
            return (dp(i + 1, k, False) + dp(i + 1, k, True)) % MOD

        return dp(0, k, False)

```

565. 1622.py

Path: LeetCode-main/solutions/1622. Fancy Sequence/1622.py

```

class Fancy:
    def __init__(self):
        self.MOD = 1_000_000_007
        # For each `val` in `vals`, it actually represents a * val + b.
        self.vals = []
        self.a = 1
        self.b = 0

        # To undo a * val + b and get the original value, we append (val - b) // a.
        # By Fermat's little theorem:
        # a^(p - 1) ≡ 1 (mod p)
        # a^(p - 2) ≡ a^(-1) (mod p)
        # So, (val - b) / a ≡ (val - b) * a^(p - 2) (mod p)
        def append(self, val: int) -> None:
            x = (val - self.b + self.MOD) % self.MOD
            self.vals.append(x * pow(self.a, self.MOD - 2, self.MOD))

        # If the value is a * val + b, then the value after adding by `inc` will be
        # a * val + b + inc. So, we adjust b to b + inc.
        def addAll(self, inc: int) -> None:
            self.b = (self.b + inc) % self.MOD

        # If the value is a * val + b, then the value after multiplying by `m` will
        # be a * m * val + b * m. So, we adjust a to a * m and b to b * m.
        def multAll(self, m: int) -> None:
            self.a = (self.a * m) % self.MOD
            self.b = (self.b * m) % self.MOD

        def getIndex(self, idx: int) -> int:
            return (-1 if idx >= len(self.vals)
                    else (self.a * self.vals[idx] + self.b) % self.MOD)

```

566. 1624.py

Path: LeetCode-main/solutions/1624. Largest Substring Between Two Equal Characters/1624.py

```
class Solution:
    def maxLengthBetweenEqualCharacters(self, s: str) -> int:
        ans = -1
        lastSeen = {}

        for i, c in enumerate(s):
            if c not in lastSeen:
                lastSeen[c] = i
            else:
                ans = max(ans, i - lastSeen[c] - 1)

        return ans
```

567. 1626.py

Path: LeetCode-main/solutions/1626. Best Team With No Conflicts/1626.py

```
from dataclasses import dataclass

@dataclass
class Player:
    age: int
    score: int

class Solution:
    def bestTeamScore(self, scores: list[int], ages: list[int]) -> int:
        n = len(scores)
        players = [Player(age, score) for age, score in zip(ages, scores)]
        # dp[i] := the maximum score of choosing the players[0..i] with the
        # players[i] being selected
        dp = [0] * n

        # Sort by age descending, then by score descending
        players.sort(key=lambda x: (-x.age, -x.score))

        for i in range(n):
            # For each player, choose it first
            dp[i] = players[i].score
            # players[j].age >= players[i].age since we sort in descending order.
            # So, we only have to check that players[j].score >= players[i].score.
            for j in range(i):
                if players[j].score >= players[i].score:
                    dp[i] = max(dp[i], dp[j] + players[i].score)

        return max(dp)
```

568. 1627.py

Path: LeetCode-main/solutions/1627. Graph Connectivity With Threshold/1627.py


```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return False
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def areConnected(
        self,
        n: int,
        threshold: int,
        queries: list[list[int]],
    ) -> list[bool]:
        uf = UnionFind(n + 1)

        for z in range(threshold + 1, n + 1):
            for x in range(z * 2, n + 1, z):
                uf.unionByRank(z, x)

        return [uf.find(a) == uf.find(b) for a, b in queries]
```

569. 1628.py

Path: LeetCode-main/solutions/1628. Design an Expression Tree With Evaluate Function/1628.py

```

from abc import ABC, abstractmethod

"""
This is the interface for the expression tree Node.
You should not remove it, and you can define some classes to implement it.
"""

class Node(ABC):
    @abstractmethod
    # define your fields here
    def evaluate(self) -> int:
        pass

class ExpNode(Node):
    op = {
        '+': lambda a, b: a + b,
        '-': lambda a, b: a - b,
        '*': lambda a, b: a * b,
        '/': lambda a, b: int(a / b),
    }

    def __init__(
        self,
        val: str,
        left: Optional['ExpNode'],
        right: Optional['ExpNode'],
    ):
        self.val = val
        self.left = left
        self.right = right

    def evaluate(self) -> int:
        if not self.left and not self.right:
            return int(self.val)
        return ExpNode.op[self.val](self.left.evaluate(), self.right.evaluate())

"""
This is the TreeBuilder class.
You can treat it as the driver code that takes the postfix input
and returns the expression tree representing it as a Node.
"""

class TreeBuilder(object):
    def buildTree(self, postfix: list[str]) -> 'Node':
        stack: list[ExpNode | None] = []

        for val in postfix:
            if val in '+-*/*':
                right = stack.pop()
                left = stack.pop()
                stack.append(ExpNode(val, left, right))
            else:
                stack.append(ExpNode(val, None, None))

        return stack.pop()

```

Path: LeetCode-main/solutions/1629. Slowest Key/1629.py

```
class Solution:
    def slowestKey(self, releaseTimes: list[int], keysPressed: str) -> str:
        ans = keysPressed[0]
        maxDuration = releaseTimes[0]

        for i in range(1, len(keysPressed)):
            duration = releaseTimes[i] - releaseTimes[i-1]
            if duration > maxDuration or (
                duration == maxDuration and keysPressed[i] > ans):
                ans = keysPressed[i]
                maxDuration = duration

        return ans
```

571. 163.py

Path: LeetCode-main/solutions/163. Missing Ranges/163.py

```
class Solution:
    def findMissingRanges(
        self,
        nums: list[int],
        lower: int,
        upper: int,
    ) -> list[list[int]]:
        def getRange(lo: int, hi: int) -> list[int]:
            if lo == hi:
                return [lo, lo]
            return [lo, hi]

        if not nums:
            return [getRange(lower, upper)]

        ans = []

        if nums[0] > lower:
            ans.append(getRange(lower, nums[0] - 1))

        for prev, curr in zip(nums, nums[1:]):
            if curr > prev + 1:
                ans.append(getRange(prev + 1, curr - 1))

        if nums[-1] < upper:
            ans.append(getRange(nums[-1] + 1, upper))

        return ans
```

572. 1630.py

Path: LeetCode-main/solutions/1630. Arithmetic Subarrays/1630.py

```
class Solution:
    def checkArithmeticSubarrays(
        self,
        nums: list[int],
        l: list[int],
        r: list[int],
    ) -> list[bool]:
        return [self._isArithmetic(nums, a, b) for a, b in zip(l, r)]

    def _isArithmetic(self, nums: list[int], l: int, r: int) -> bool:
        if r - l < 2:
            return True

        numsSet = set()
        mn = math.inf
        mx = -math.inf

        for i in range(l, r+1):
            mn = min(mn, nums[i])
            mx = max(mx, nums[i])
            numsSet.add(nums[i])

        if (mx - mn) % (r - l) != 0:
            return False

        interval = (mx - mn) // (r - l)
        return all(mn + k * interval in numsSet
                   for k in range(1, r - l + 1))
```

573. 1631.py

Path: LeetCode-main/solutions/1631. Path With Minimum Effort/1631.py

```

class Solution:
    def minimumEffortPath(self, heights: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heights)
        n = len(heights[0])
        # diff[i][j] := the maximum absolute difference to reach (i, j)
        diff = [[math.inf] * n for _ in range(m)]
        seen = set()

        minHeap = [(0, 0, 0)] # (d, i, j)
        diff[0][0] = 0

        while minHeap:
            d, i, j = heapq.heappop(minHeap)
            if i == m - 1 and j == n - 1:
                return d
            seen.add((i, j))
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if (x, y) in seen:
                    continue
                newDiff = abs(heights[i][j] - heights[x][y])
                maxDiff = max(diff[i][j], newDiff)
                if diff[x][y] > maxDiff:
                    diff[x][y] = maxDiff
                    heapq.heappush(minHeap, (diff[x][y], x, y))

```

574. 1632.py

Path: LeetCode-main/solutions/1632. Rank Transform of a Matrix/1632.py

```

class UnionFind:
    def __init__(self):
        self.id = {}

    def union(self, u: int, v: int) -> None:
        self.id.setdefault(u, u)
        self.id.setdefault(v, v)
        i = self._find(u)
        j = self._find(v)
        if i != j:
            self.id[i] = j

    def getGroupIdToValues(self) -> dict[int, list[int]]:
        groupIdToValues = collections.defaultdict(list)
        for u in self.id.keys():
            groupIdToValues[self._find(u)].append(u)
        return groupIdToValues

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def matrixRankTransform(self, matrix: list[list[int]]) -> list[list[int]]:
        m = len(matrix)
        n = len(matrix[0])
        ans = [[0] * n for _ in range(m)]
        # {val: [(i, j)]}
        valToGrids = collections.defaultdict(list)
        # rank[i] := the maximum rank of the row or column so far
        maxRankSoFar = [0] * (m + n)

        for i, row in enumerate(matrix):
            for j, val in enumerate(row):
                valToGrids[val].append((i, j))

        for _, grids in sorted(valToGrids.items()):
            uf = UnionFind()
            for i, j in grids:
                # Union i-th row with j-th col.
                uf.union(i, j + m)
            for values in uf.getGroupIdToValues().values():
                # Get the maximum rank of all the included rows and columns.
                maxRank = max(maxRankSoFar[i] for i in values)
                for i in values:
                    # Update all the rows and columns to maxRank + 1.
                    maxRankSoFar[i] = maxRank + 1
            for i, j in grids:
                ans[i][j] = maxRankSoFar[i]

        return ans

```

575. 1634.py

Path: LeetCode-main/solutions/1634. Add Two Polynomials Represented as Linked Lists/1634.py

```

# Definition for polynomial singly-linked list.
# class PolyNode:
#     def __init__(self, x=0, y=0, next=None):
#         self.coefficient = x
#         self.power = y
#         self.next = next

class Solution:
    def addPoly(self, poly1: 'PolyNode', poly2: 'PolyNode') -> 'PolyNode':
        dummy = PolyNode()
        curr = dummy
        p = poly1 # poly1's pointer
        q = poly2 # poly2's pointer

        while p and q:
            if p.power > q.power:
                curr.next = PolyNode(p.coefficient, p.power)
                curr = curr.next
                p = p.next
            elif p.power < q.power:
                curr.next = PolyNode(q.coefficient, q.power)
                curr = curr.next
                q = q.next
            else: # p.power == q.power
                sumCoefficient = p.coefficient + q.coefficient
                if sumCoefficient != 0:
                    curr.next = PolyNode(sumCoefficient, p.power)
                    curr = curr.next
                p = p.next
                q = q.next

        while p:
            curr.next = PolyNode(p.coefficient, p.power)
            curr = curr.next
            p = p.next

        while q:
            curr.next = PolyNode(q.coefficient, q.power)
            curr = curr.next
            q = q.next

        return dummy.next

```

576. 1636.py

Path: LeetCode-main/solutions/1636. Sort Array by Increasing Frequency/1636.py

```
from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    num: int
    freq: int

    def __lt__(self, other):
        if self.freq == other.freq:
            return self.num > other.num
        return self.freq < other.freq

class Solution:
    def frequencySort(self, nums: list[int]) -> list[int]:
        ans = []
        heap = []

        for num, freq in collections.Counter(nums).items():
            heapq.heappush(heap, T(num, freq))

        while len(heap) > 0:
            num = heap[0].num
            freq = heapq.heappop(heap).freq
            ans.extend([num] * freq)

        return ans
```

577. 1637.py

Path: LeetCode-main/solutions/1637. Widest Vertical Area Between Two Points Containing No Points/1637.py

```
class Solution:
    def maxWidthOfVerticalArea(self, points: list[list[int]]) -> int:
        xs = sorted([x for x, _ in points])
        return max(b - a for a, b in itertools.pairwise(xs))
```

578. 1638.py

Path: LeetCode-main/solutions/1638. Count Substrings That Differ by One Character/1638.py


```

class Solution:
    def countSubstrings(self, s: str, t: str) -> int:
        ans = 0

        for i in range(len(s)):
            ans += self._count(s, t, i, 0)

        for j in range(1, len(t)):
            ans += self._count(s, t, 0, j)

        return ans

    def _count(self, s: str, t: str, i: int, j: int) -> int:
        """Returns the number of substrings of s[i..n) and t[j:] that differ by one char."""
        res = 0
        # the number of substrings starting at s[i] and t[j] ending in the current
        # index with zero different letter
        dp0 = 0
        # the number of substrings starting at s[i] and t[j] ending in the current
        # index with one different letter
        dp1 = 0

        while i < len(s) and j < len(t):
            if s[i] == t[j]:
                dp0 += 1
            else:
                dp0, dp1 = 0, dp0 + 1
            res += dp1
            i += 1
            j += 1

        return res

```

579. 1639-2.py

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639-2.py

```

class Solution:
    def numWays(self, words: list[str], target: str) -> int:
        MOD = 1_000_000_007
        wordLength = len(words[0])
        # dp[i][j] := the number of ways to form the first i characters of the
        # `target` using the j first characters in each word
        dp = [[0] * (wordLength + 1) for _ in range(len(target) + 1)]
        # counts[j] := the count map of words[i][j], where 0 <= i < |words|
        counts = [collections.Counter() for _ in range(wordLength)]

        for i in range(wordLength):
            for word in words:
                counts[i][word[i]] += 1

        dp[0][0] = 1

        for i in range(len(target) + 1):
            for j in range(wordLength):
                if i < len(target):
                    # Pick the character target[i] from word[j].
                    dp[i + 1][j + 1] = dp[i][j] * counts[j][target[i]]
                    dp[i + 1][j + 1] %= MOD
                # Skip the word[j].
                dp[i][j + 1] += dp[i][j]
                dp[i][j + 1] %= MOD

        return dp[len(target)][wordLength]

```

580. 1639-3.py

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639-3.py

```

class Solution:
    def numWays(self, words: list[str], target: str) -> int:
        MOD = 1_000_000_007
        # dp[i] := the number of ways to form the first i characters of `target`
        dp = [0] * (len(target) + 1)
        dp[0] = 1

        for j in range(len(words[0])):
            count = collections.Counter(word[j] for word in words)
            for i in range(len(target), 0, -1):
                dp[i] += dp[i - 1] * count[target[i - 1]]
                dp[i] %= MOD

        return dp[len(target)]

```

581. 1639.py

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639.py

```

class Solution:
    def numWays(self, words: list[str], target: str) -> int:
        MOD = 1_000_000_007
        wordLength = len(words[0])
        # counts[j] := the count map of words[i][j], where 0 <= i < |words|
        counts = [collections.Counter() for _ in range(wordLength)]

        for i in range(wordLength):
            for word in words:
                counts[i][word[i]] += 1

        @functools.lru_cache(None)
        def dp(i: int, j: int):
            """Returns the number of ways to form target[i..n) using word[j..n)."""
            if i == len(target):
                return 1
            if j == wordLength:
                return 0
            return (dp(i + 1, j + 1) * counts[j][target[i]] + dp(i, j + 1)) % MOD

        return dp(0, 0)

```

582. 164.py

Path: LeetCode-main/solutions/164. Maximum Gap/164.py

```

class Bucket:
    def __init__(self, mn: int, mx: int):
        self.mn = mn
        self.mx = mx

class Solution:
    def maximumGap(self, nums: list[int]) -> int:
        if len(nums) < 2:
            return 0

        mn = min(nums)
        mx = max(nums)
        if mn == mx:
            return 0

        gap = math.ceil((mx - mn) / (len(nums) - 1))
        bucketSize = (mx - mn) // gap + 1
        buckets = [Bucket(math.inf, -math.inf) for _ in range(bucketSize)]

        for num in nums:
            i = (num - mn) // gap
            buckets[i].mn = min(buckets[i].mn, num)
            buckets[i].mx = max(buckets[i].mx, num)

        ans = 0
        prevMax = mn

        for bucket in buckets:
            if bucket.mn == math.inf:
                continue # empty bucket
            ans = max(ans, bucket.mn - prevMax)
            prevMax = bucket.mx

        return ans

```

583. 1640.py

Path: LeetCode-main/solutions/1640. Check Array Formation Through Concatenation/1640.py

```
class Solution:
    def canFormArray(self, arr: list[int], pieces: list[list[int]]) -> bool:
        concatenated = []
        startToPiece = {piece[0]: piece for piece in pieces}

        for a in arr:
            concatenated += startToPiece.get(a, [])

        return concatenated == arr
```

584. 1642.py

Path: LeetCode-main/solutions/1642. Furthest Building You Can Reach/1642.py

```
class Solution:
    def furthestBuilding(
        self,
        heights: list[int],
        bricks: int,
        ladders: int,
    ) -> int:
        minHeap = []

        for i, (a, b) in enumerate(itertools.pairwise(heights)):
            diff = b - a
            if diff <= 0:
                continue
            heapq.heappush(minHeap, diff)
            # If we run out of ladders, greedily use as less bricks as possible.
            if len(minHeap) > ladders:
                bricks -= heapq.heappop(minHeap)
            if bricks < 0:
                return i

        return len(heights) - 1
```

585. 1643.py

Path: LeetCode-main/solutions/1643. Kth Smallest Instructions/1643.py

```

class Solution:
    def kthSmallestPath(self, destination: list[int], k: int) -> str:
        ans = []
        v, h = destination

        for _ in range(h + v):
            # If pick 'H', then we're able to reach 1, 2, ..., availableRank.
            availableRank = math.comb(h + v - 1, v)
            if availableRank >= k: # Should pick 'H'.
                ans.append('H')
                h -= 1
            else: # Should pick 'V'.
                k -= availableRank
                ans.append('V')
                v -= 1

        return ''.join(ans)

```

586. 1644-2.py

Path: LeetCode-main/solutions/1644. Lowest Common Ancestor of a Binary Tree II/1644-2.py

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        def getLCA(root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
            if not root or root == p or root == q:
                return root
            left = getLCA(root.left, p, q)
            right = getLCA(root.right, p, q)
            if left and right:
                return root
            return left or right

        ans = getLCA(root, p, q)
        if ans == p: # Search q in the subtree rooted at p.
            return ans if getLCA(p, q, q) else None
        if ans == q: # Search p in the subtree rooted at q.
            return ans if getLCA(q, p, p) else None
        return ans # (ans != p and ans != q) or ans is None

```

587. 1644.py

Path: LeetCode-main/solutions/1644. Lowest Common Ancestor of a Binary Tree II/1644.py

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        seenP = False
        seenQ = False

        def getLCA(root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
            nonlocal seenP
            nonlocal seenQ
            if not root:
                return None
            # Need to traverse the entire tree to update `seenP` and `seenQ`.
            left = getLCA(root.left, p, q)
            right = getLCA(root.right, p, q)
            if root == p:
                seenP = True
                return root
            if root == q:
                seenQ = True
                return root
            if left and right:
                return root
            return left or right

        lca = getLCA(root, p, q)
        return lca if seenP and seenQ else None

```

588. 1646.py

Path: LeetCode-main/solutions/1646. Get Maximum in Generated Array/1646.py

```

class Solution:
    def getMaximumGenerated(self, n: int) -> int:
        if n == 0:
            return 0
        if n == 1:
            return 1

        nums = [0] * (n + 1)
        nums[1] = 1

        i = 1
        while 2 * i + 1 <= n:
            nums[2 * i] = nums[i]
            nums[2 * i + 1] = nums[i] + nums[i + 1]
            i += 1

        return max(nums)

```

589. 1647.py

Path: LeetCode-main/solutions/1647. Minimum Deletions to Make Character Frequencies Unique/1647.py

```

class Solution:
    def minDeletions(self, s: str) -> int:
        ans = 0
        count = collections.Counter(s)
        usedFreq = set()

        for freq in count.values():
            while freq > 0 and freq in usedFreq:
                freq -= 1 # Delete ('a' + i).
                ans += 1
                usedFreq.add(freq)

        return ans

```

590. 1648.py

Path: LeetCode-main/solutions/1648. Sell Diminishing-Valued Colored Balls/1648.py

```

class Solution:
    def maxProfit(self, inventory: list[int], orders: int) -> int:
        MOD = 1_000_000_007
        ans = 0
        largestCount = 1

        def trapezoid(a: int, b: int) -> int:
            return (a + b) * (a - b + 1) // 2

        for a, b in itertools.pairwise(sorted(inventory, reverse=True) + [0]):
            if a > b:
                # If we are at the last inventory, or inventory[i] > inventory[i + 1].
                # In either case, we will pick inventory[i - largestCount + 1..i].
                pick = a - b
                # We have run out of orders, so we need to recalculate the number of
                # balls that we actually pick for inventory[i - largestCount + 1..i].
                if largestCount * pick >= orders:
                    actualPick, remaining = divmod(orders, largestCount)
                    return (ans +
                            largestCount * trapezoid(a, a - actualPick + 1) +
                            remaining * (a - actualPick)) % MOD
                ans += largestCount * trapezoid(a, a - pick + 1)
                ans %= MOD
                orders -= largestCount * pick
                largestCount += 1

```

591. 165.py

Path: LeetCode-main/solutions/165. Compare Version Numbers/165.py

```

class Solution:
    def compareVersion(self, version1: str, version2: str) -> int:
        levels1 = version1.split('.')
        levels2 = version2.split('.')
        length = max(len(levels1), len(levels2))

        for i in range(length):
            v1 = int(levels1[i]) if i < len(levels1) else 0
            v2 = int(levels2[i]) if i < len(levels2) else 0
            if v1 < v2:
                return -1
            if v1 > v2:
                return 1

        return 0

```

592. 1650.py

Path: LeetCode-main/solutions/1650. Lowest Common Ancestor of a Binary Tree III/1650.py

```

class Solution:
    # Same as 160. Intersection of Two Linked Lists
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        a = p
        b = q

        while a != b:
            a = a.parent if a else q
            b = b.parent if b else p

        return a

```

593. 1652.py

Path: LeetCode-main/solutions/1652. Defuse the Bomb/1652.py

```

class Solution:
    def decrypt(self, code: list[int], k: int) -> list[int]:
        n = len(code)
        ans = [0] * n
        if k == 0:
            return ans

        summ = 0
        start = 1 if k > 0 else n + k # the start of the next k numbers
        end = k if k > 0 else n - 1 # the end of the next k numbers

        for i in range(start, end + 1):
            summ += code[i]

        for i in range(n):
            ans[i] = summ
            summ -= code[start % n]
            start += 1
            end += 1
            summ += code[end % n]

        return ans

```

594. 1653.py

Path: LeetCode-main/solutions/1653. Minimum Deletions to Make String Balanced/1653.py

```
class Solution:
    # Same as 926. Flip String to Monotone Increasing
    def minimumDeletions(self, s: str) -> int:
        dp = 0 # the number of characters to be deleted to make subso far balanced
        countB = 0

        for c in s:
            if c == 'a':
                # 1. Delete 'a'.
                # 2. Keep 'a' and delete the previous 'b's.
                dp = min(dp + 1, countB)
            else:
                countB += 1

        return dp
```

595. 1654.py

Path: LeetCode-main/solutions/1654. Minimum Jumps to Reach Home/1654.py

```
from enum import Enum

class Direction(Enum):
    FORWARD = 0
    BACKWARD = 1

class Solution:
    def minimumJumps(self, forbidden: list[int], a: int, b: int, x: int) -> int:
        furthest = max(x + a + b, max(pos + a + b for pos in forbidden))
        seenForward = {pos for pos in forbidden}
        seenBackward = {pos for pos in forbidden}

        # (direction, position)
        q = collections.deque([(Direction.FORWARD, 0)])

        ans = 0
        while q:
            for _ in range(len(q)):
                dir, pos = q.popleft()
                if pos == x:
                    return ans
                forward = pos + a
                backward = pos - b
                if forward <= furthest and forward not in seenForward:
                    seenForward.add(forward)
                    q.append((Direction.FORWARD, forward))
                # It cannot jump backward twice in a row.
                if dir == Direction.FORWARD and backward >= 0 and backward not in seenBackward:
                    seenBackward.add(backward)
                    q.append((Direction.BACKWARD, backward))
            ans += 1

        return -1
```

596. 1655.py

Path: LeetCode-main/solutions/1655. Distribute Repeating Integers/1655.py

```
class Solution:
    def canDistribute(self, nums: list[int], quantity: list[int]) -> bool:
        freqs = list(collections.Counter(nums).values())
        # validDistribution[i][j] := True if it's possible to distribute the i-th
        # freq into a subset of quantity represented by the bitmask j
        validDistribution = self._getValidDistribution(freqs, quantity)
        n = len(freqs)
        m = len(quantity)
        maxMask = 1 << m
        # dp[i][j] := true if it's possible to distribute freqs[i..n), where j is
        # the bitmask of the selected quantity
        dp = [[False] * maxMask for _ in range(n + 1)]
        dp[n][maxMask - 1] = True

        for i in range(n - 1, -1, -1):
            for mask in range(maxMask):
                dp[i][mask] = dp[i + 1][mask]
                availableMask = ~mask & (maxMask - 1)
                submask = availableMask
                while submask > 0:
                    if validDistribution[i][submask]:
                        dp[i][mask] = dp[i][mask] or dp[i + 1][mask | submask]
                        submask = (submask - 1) & availableMask

        return dp[0][0]

    def _getValidDistribution(self, freqs: list[int],
                             quantity: list[int]) -> list[list[bool]]:
        maxMask = 1 << len(quantity)
        validDistribution = [[False] * maxMask for _ in range(len(freqs))]
        for i, freq in enumerate(freqs):
            for mask in range(maxMask):
                if freq >= self._getQuantitySum(quantity, mask):
                    validDistribution[i][mask] = True
        return validDistribution

    def _getQuantitySum(self, quantity: list[int], mask: int) -> int:
        """Returns the sum of the selected quantity represented by `mask`."""
        return sum(q for i, q in enumerate(quantity) if mask >> i & 1)
```

597. 1656.py

Path: LeetCode-main/solutions/1656. Design an Ordered Stream/1656.py

```
class OrderedStream:
    def __init__(self, n: int):
        self.values = [''] * n
        self.i = 0 # self.values' index (0-indexed)

    def insert(self, idKey: int, value: str) -> list[str]:
        idKey -= 1 # Converts to 0-indexed.
        self.values[idKey] = value
        if idKey > self.i:
            return []
        while self.i < len(self.values) and self.values[self.i]:
            self.i += 1
        return self.values[idKey:self.i]
```

598. 1657.py

Path: LeetCode-main/solutions/1657. Determine if Two Strings Are Close/1657.py

```
class Solution:
    def closeStrings(self, word1: str, word2: str) -> bool:
        if len(word1) != len(word2):
            return False

        count1 = collections.Counter(word1)
        count2 = collections.Counter(word2)
        if count1.keys() != count2.keys():
            return False

        return sorted(count1.values()) == sorted(count2.values())
```

599. 1658.py

Path: LeetCode-main/solutions/1658. Minimum Operations to Reduce X to Zero/1658.py

```
class Solution:
    def minOperations(self, nums: list[int], x: int) -> int:
        targetSum = sum(nums) - x
        if targetSum == 0:
            return len(nums)
        maxLen = self._maxSubArrayLen(nums, targetSum)
        return -1 if maxLen == -1 else len(nums) - maxLen

    # Same as 325. Maximum Size Subarray Sum Equals k
    def _maxSubArrayLen(self, nums: list[int], k: int) -> int:
        res = -1
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += num
            target = prefix - k
            if target in prefixToIndex:
                res = max(res, i - prefixToIndex[target])
            # No need to check the existence of the prefix since it's unique.
            prefixToIndex[prefix] = i

        return res
```

600. 1659.py

Path: LeetCode-main/solutions/1659. Maximize Grid Happiness/1659.py

```
class Solution:
    def getMaxGridHappiness(
        self,
        m: int,
        n: int,
        introvertsCount: int,
        extrovertsCount: int,
    ) -> int:
        def getPlacementCost(
            i: int,
            j: int,
```

```

    j: int,
    inMask: int,
    exMask: int,
    diff: int,
) -> int:
    """Calculates the cost based on left and up neighbors.

    The `diff` parameter represents the happiness change due to the current
    placed person in (i, j). We add `diff` each time we encounter a neighbor
    (left or up) who is already placed.

    1. If the neighbor is an introvert, we subtract 30 from cost.
    2. If the neighbor is an extrovert, we add 20 to from cost.
    """
    cost = 0
    if i > 0:
        if (1 << (n - 1)) & inMask:
            cost += diff - 30
        if (1 << (n - 1)) & exMask:
            cost += diff + 20
    if j > 0:
        if 1 & inMask:
            cost += diff - 30
        if 1 & exMask:
            cost += diff + 20
    return cost

@functools.lru_cache(None)
def dp(
    pos: int, inMask: int, exMask: int, inCount: int, exCount: int
) -> int:
    # `inMask` is the placement of introvert people in the last n cells.
    # e.g. if we have m = 2, n = 3, i = 1, j = 1, then inMask = 0b101 means
    #
    # ? 1 0
    # 1 x ? (x := current position)
    i, j = divmod(pos, n)
    if i == m:
        return 0

    shiftedInMask = (inMask << 1) & ((1 << n) - 1)
    shiftedExMask = (exMask << 1) & ((1 << n) - 1)

    skip = dp(pos + 1, shiftedInMask, shiftedExMask, inCount, exCount)
    placeIntrovert = (
        120 + getPlacementCost(i, j, inMask, exMask, -30) +
        dp(pos + 1, shiftedInMask + 1, shiftedExMask, inCount - 1, exCount)
        if inCount > 0
        else -math.inf
    )
    placeExtrovert = (
        40 + getPlacementCost(i, j, inMask, exMask, 20) +
        dp(pos + 1, shiftedInMask, shiftedExMask + 1, inCount, exCount - 1)
        if exCount > 0
        else -math.inf
    )
    return max(skip, placeIntrovert, placeExtrovert)

return dp(0, 0, 0, introvertsCount, extrovertsCount)

```

601. 166.py

Path: LeetCode-main/solutions/166. Fraction to Recurring Decimal/166.py

```

class Solution:
    def fractionToDecimal(self, numerator: int, denominator: int) -> str:
        if numerator == 0:
            return '0'

        ans = ''

        if (numerator < 0) ^ (denominator < 0):
            ans += '-'

        numerator = abs(numerator)
        denominator = abs(denominator)
        ans += str(numerator // denominator)

        if numerator % denominator == 0:
            return ans

        ans += '.'
        dict = {}

        remainder = numerator % denominator
        while remainder:
            if remainder in dict:
                ans = ans[:dict[remainder]] + '(' + ans[dict[remainder]:] + ')'
                break
            dict[remainder] = len(ans)
            remainder *= 10
            ans += str(remainder // denominator)
            remainder %= denominator

        return ans

```

602. 1660.py

Path: LeetCode-main/solutions/1660. Correct a Binary Tree/1660.py

```

class Solution:
    def __init__(self):
        self.seen = set()

    def correctBinaryTree(self, root: TreeNode | None) -> TreeNode | None:
        if root == None:
            return None
        if root.right and root.right.val in self.seen:
            return None
        self.seen.add(root.val)
        root.right = self.correctBinaryTree(root.right)
        root.left = self.correctBinaryTree(root.left)
        return root

```

603. 1662.py

Path: LeetCode-main/solutions/1662. Check If Two String Arrays are Equivalent/1662.py

```

class Solution:
    def arrayStringsAreEqual(self, word1: list[str], word2: list[str]) -> bool:
        i = 0 # word1's index
        j = 0 # word2's index
        a = 0 # word1[i]'s index
        b = 0 # word2[j]'s index

        while i < len(word1) and j < len(word2):
            if word1[i][a] != word2[j][b]:
                return False
            a += 1
            if a == len(word1[i]):
                i += 1
                a = 0
            b += 1
            if b == len(word2[j]):
                j += 1
                b = 0

        return i == len(word1) and j == len(word2)

```

604. 1663.py

Path: LeetCode-main/solutions/1663. Smallest String With A Given Numeric Value/1663.py

```

class Solution:
    def getSmallestString(self, n: int, k: int) -> str:
        ans = []

        for i in range(n):
            remainingLetters = n - 1 - i
            rank = max(1, k - remainingLetters * 26)
            ans.append(chr(ord('a') + rank - 1))
            k -= rank

        return ''.join(ans)

```

605. 1665.py

Path: LeetCode-main/solutions/1665. Minimum Initial Energy to Finish Tasks/1665.py

```

class Solution:
    def minimumEffort(self, tasks: list[list[int]]) -> int:
        ans = 0
        prevSaved = 0

        for actual, minimum in sorted(tasks, key=lambda x: x[0] - x[1]):
            if prevSaved < minimum:
                ans += minimum - prevSaved
                prevSaved = minimum - actual
            else:
                prevSaved -= actual

        return ans

```

606. 1668.py

Path: LeetCode-main/solutions/1668. Maximum Repeating Substring/1668.py

```
class Solution:
    def maxRepeating(self, sequence: str, word: str) -> int:
        ans = 1
        while word * ans in sequence:
            ans += 1
        return ans - 1
```

607. 1669.py

Path: LeetCode-main/solutions/1669. Merge In Between Linked Lists/1669.py

```
class Solution:
    def mergeInBetween(
        self,
        list1: ListNode,
        a: int,
        b: int,
        list2: ListNode,
    ) -> ListNode:
        nodeBeforeA = list1
        for i in range(a - 1):
            nodeBeforeA = nodeBeforeA.next

        nodeB = nodeBeforeA.next
        for i in range(b - a):
            nodeB = nodeB.next

        nodeBeforeA.next = list2
        lastNodeInList2 = list2

        while lastNodeInList2.next:
            lastNodeInList2 = lastNodeInList2.next

        lastNodeInList2.next = nodeB.next
        nodeB.next = None
        return list1
```

608. 167.py

Path: LeetCode-main/solutions/167. Two Sum II - Input array is sorted/167.py

```
class Solution:
    def twoSum(self, numbers: list[int], target: int) -> list[int]:
        l = 0
        r = len(numbers) - 1

        while l < r:
            summ = numbers[l] + numbers[r]
            if summ == target:
                return [l + 1, r + 1]
            if summ < target:
                l += 1
            else:
                r -= 1
```

609. 1670.py

Path: LeetCode-main/solutions/1670. Design Front Middle Back Queue/1670.py

```
class FrontMiddleBackQueue:
    def __init__(self):
        self.frontQueue = collections.deque()
        self.backQueue = collections.deque()

    def pushFront(self, val: int) -> None:
        self.frontQueue.appendleft(val)
        self._moveFrontToBackIfNeeded()

    def pushMiddle(self, val: int) -> None:
        if len(self.frontQueue) == len(self.backQueue):
            self.backQueue.appendleft(val)
        else:
            self.frontQueue.append(val)

    def pushBack(self, val: int) -> None:
        self.backQueue.append(val)
        self._moveBackToFrontIfNeeded()

    def popFront(self) -> int:
        if self.frontQueue:
            x = self.frontQueue.popleft()
            self._moveBackToFrontIfNeeded()
            return x
        if self.backQueue:
            return self.backQueue.popleft()
        return -1

    def popMiddle(self) -> int:
        if not self.frontQueue and not self.backQueue:
            return -1
        if len(self.frontQueue) + 1 == len(self.backQueue):
            return self.backQueue.popleft()
        return self.frontQueue.pop()

    def popBack(self) -> int:
        if self.backQueue:
            x = self.backQueue.pop()
            self._moveFrontToBackIfNeeded()
            return x
        return -1

    def _moveFrontToBackIfNeeded(self) -> None:
        if len(self.frontQueue) - 1 == len(self.backQueue):
            self.backQueue.appendleft(self.frontQueue.pop())

    def _moveBackToFrontIfNeeded(self) -> None:
        if len(self.frontQueue) + 2 == len(self.backQueue):
            self.frontQueue.append(self.backQueue.popleft())
```

610. 1671.py

Path: LeetCode-main/solutions/1671. Minimum Number of Removals to Make Mountain Array/1671.py


```

class Solution:
    def minimumMountainRemovals(self, nums: list[int]) -> int:
        left = self._lengthOfLIS(nums)
        right = self._lengthOfLIS(nums[::-1])[::-1]
        maxMountainSeq = 0

        for l, r in zip(left, right):
            if l > 1 and r > 1:
                maxMountainSeq = max(maxMountainSeq, l + r - 1)

        return len(nums) - maxMountainSeq

# Similar to 300. Longest Increasing Subsequence
def _lengthOfLIS(self, nums: list[int]) -> list[int]:
    # tails[i] := the minimum tail of all the increasing subsequences having
    # length i + 1
    tails = []
    # dp[i] := the length of LIS ending in nums[i]
    dp = []
    for num in nums:
        if not tails or num > tails[-1]:
            tails.append(num)
        else:
            tails[bisect.bisect_left(tails, num)] = num
        dp.append(len(tails))
    return dp

```

611. 1672.py

Path: LeetCode-main/solutions/1672. Richest Customer Wealth/1672.py

```

class Solution:
    def maximumWealth(self, accounts: list[list[int]]) -> int:
        return max(map(sum, accounts))

```

612. 1673.py

Path: LeetCode-main/solutions/1673. Find the Most Competitive Subsequence/1673.py

```

class Solution:
    def mostCompetitive(self, nums: list[int], k: int) -> list[int]:
        ans = []

        for i, num in enumerate(nums):
            # If |ans| - 1 + |nums[i..n]| >= k, then it means we still have enough
            # numbers, and we can safely pop an element from ans.
            while ans and ans[-1] > nums[i] and len(ans) - 1 + len(nums) - i >= k:
                ans.pop()
            if len(ans) < k:
                ans.append(nums[i])

        return ans

```

613. 1674.py

Path: LeetCode-main/solutions/1674. Minimum Moves to Make Array Complementary/1674.py

```

class Solution:
    def minMoves(self, nums: list[int], limit: int) -> int:
        n = len(nums)
        ans = n
        # delta[i] := the number of moves needed when target goes from i - 1 to i
        delta = [0] * (limit * 2 + 2)

        for i in range(n // 2):
            a = nums[i]
            b = nums[n - 1 - i]
            delta[min(a, b) + 1] -= 1
            delta[a + b] -= 1
            delta[a + b + 1] += 1
            delta[max(a, b) + limit + 1] += 1

        # Initially, we need `moves` when the target is 2.
        moves = n
        for i in range(2, limit * 2 + 1):
            moves += delta[i]
            ans = min(ans, moves)

        return ans

```

614. 1675.py

Path: LeetCode-main/solutions/1675. Minimize Deviation in Array/1675.py

```

class Solution:
    def minimumDeviation(self, nums: list[int]) -> int:
        ans = math.inf
        mn = math.inf
        maxHeap = []

        for num in nums:
            evenNum = num if num % 2 == 0 else num * 2
            heapq.heappush(maxHeap, -evenNum)
            mn = min(mn, evenNum)

        while maxHeap[0] % 2 == 0:
            mx = -heapq.heappop(maxHeap)
            ans = min(ans, mx - mn)
            mn = min(mn, mx // 2)
            heapq.heappush(maxHeap, -mx // 2)

        return min(ans, -maxHeap[0] - mn)

```

615. 1676.py

Path: LeetCode-main/solutions/1676. Lowest Common Ancestor of a Binary Tree IV/1676.py

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        nodes: 'list[TreeNode]',
    ) -> 'TreeNode':
        nodes = set(nodes)

        def lca(root: 'TreeNode') -> 'TreeNode':
            if not root:
                return None
            if root in nodes:
                return root
            left = lca(root.left)
            right = lca(root.right)
            if left and right:
                return root
            return left or right

        return lca(root)

```

616. 1678.py

Path: LeetCode-main/solutions/1678. Goal Parser Interpretation/1678.py

```

class Solution:
    def interpret(self, command: str) -> str:
        return command.replace('()', 'o').replace('(al)', 'a')

```

617. 1679.py

Path: LeetCode-main/solutions/1679. Max Number of K-Sum Pairs/1679.py

```

class Solution:
    def maxOperations(self, nums: list[int], k: int) -> int:
        count = collections.Counter(nums)
        return sum(min(count[num], count[k - num])
                    for num in count) // 2

```

618. 168.py

Path: LeetCode-main/solutions/168. Excel Sheet Column Title/168.py

```

class Solution:
    def convertToTitle(self, n: int) -> str:
        return (self.convertToTitle((n - 1) // 26) + chr(ord('A') + (n - 1) % 26)
                if n
                else '')

```

619. 1680-2.py

Path: LeetCode-main/solutions/1680. Concatenation of Consecutive Binary Numbers/1680-2.py

```

class Solution:
    def concatenatedBinary(self, n: int) -> int:
        MOD = 1_000_000_007
        ans = 0
        numberOfBits = 0

        for i in range(1, n + 1):
            if i.bit_count() == 1:
                numberOfBits += 1
            ans = ((ans << numberOfBits) + i) % MOD

        return ans

```

620. 1680.py

Path: LeetCode-main/solutions/1680. Concatenation of Consecutive Binary Numbers/1680.py

```

class Solution:
    def concatenatedBinary(self, n: int) -> int:
        MOD = 1_000_000_007
        ans = 0

        def numberOfBits(n: int) -> int:
            return int(math.log2(n)) + 1

        for i in range(1, n + 1):
            ans = ((ans << numberOfBits(i)) + i) % MOD

        return ans

```

621. 1681.py

Path: LeetCode-main/solutions/1681. Minimum Incompatibility/1681.py

```

class Solution:
    def __init__(self):
        self.MAX_NUM = 16

    def minimumIncompatibility(self, nums: list[int], k: int) -> int:
        MAX_COMPATIBILITY = (16 - 1) * (16 // 2)
        n = len(nums)
        subsetSize = n // k
        maxMask = 1 << n
        incompatibilities = self._getIncompatibilities(nums, subsetSize)

        # dp[i] := the minimum possible sum of incompatibilities of the subset
        # of numbers represented by the bitmask i
        dp = [MAX_COMPATIBILITY] * maxMask
        dp[0] = 0

        for mask in range(1, maxMask):
            # The number of 1s in `mask` isn't a multiple of `subsetSize`.
            if mask.bit_count() % subsetSize != 0:
                continue
            # https://cp-algorithms.com/algebra/all-submasks.html
            submask = mask
            while submask > 0:
                if incompatibilities[submask] != -1: # valid submask
                    dp[mask] = min(dp[mask], dp[mask - submask] +
                                   incompatibilities[submask])
                submask = submask & (submask - 1)

```

```

        submask = (submask - 1) & mask

    return dp[-1] if dp[-1] != MAX_COMPATIBILITY else -1

def _getIncompatibilities(
    self,
    nums: list[int],
    subsetSize: int,
) -> list[int]:
    """
    Returns an incompatibilities array where
    * incompatibilities[i] := the incompatibility of the subset of numbers
      represented by the bitmask i
    * incompatibilities[i] := -1 if the number of 1s in the bitmask i is not
      `subsetSize`
    """
    maxMask = 1 << len(nums)
    incompatibilities = [-1] * maxMask
    for mask in range(maxMask):
        if mask.bit_count() == subsetSize and self._isUnique(nums, mask, subsetSize):
            incompatibilities[mask] = self._getIncompatibility(nums, mask)
    return incompatibilities

def _isUnique(self, nums: list[int], mask: int, subsetSize: int) -> bool:
    """Returns True if the numbers selected by `mask` are unique."""
    used = 0
    for i, num in enumerate(nums):
        if mask >> i & 1:
            used |= 1 << num
    return used.bit_count() == subsetSize

def _getIncompatibility(self, nums: list[int], mask: int) -> int:
    """
    Returns the incompatibility of the selected numbers represented by the
    `mask`.
    """
    mn = self.MAX_NUM
    mx = 0
    for i, num in enumerate(nums):
        if mask >> i & 1:
            mx = max(mx, num)
            mn = min(mn, num)
    return mx - mn

```

622. 1682-2.py

Path: LeetCode-main/solutions/1682. Longest Palindromic Subsequence II/1682-2.py

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        # dp[i][j][k] := the length of LPS(s[i..j]), where the previous letter is
        # ('a' + k).
        dp = [[[0] * 27 for _ in range(n)] for _ in range(n)]

        for d in range(1, n):
            for i in range(n - d):
                for k in range(27):
                    j = i + d
                    if s[i] == s[j] and s[i] != chr(ord('a') + k):
                        dp[i][j][k] = dp[i + 1][j - 1][ord(s[i]) - ord('a')] + 2
                    else:
                        dp[i][j][k] = max(dp[i + 1][j][k], dp[i][j - 1][k])

        return dp[0][n - 1][26]

```

623. 1682.py

Path: LeetCode-main/solutions/1682. Longest Palindromic Subsequence II/1682.py

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)

        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> int:
            """
            Returns the length of LPS(s[i..j]), where the previous letter is
            ('a' + k).
            """
            if i >= j:
                return 0
            if s[i] == s[j] and s[i] != chr(ord('a') + k):
                return dp(i + 1, j - 1, ord(s[i]) - ord('a')) + 2
            return max(dp(i + 1, j, k), dp(i, j - 1, k))

        return dp(0, n - 1, 26)

```

624. 1684.py

Path: LeetCode-main/solutions/1684. Count the Number of Consistent Strings/1684.py

```

class Solution:
    def countConsistentStrings(self, allowed: str, words: list[str]) -> int:
        return sum(all(c in allowed for c in word)
                    for word in words)

```

625. 1685.py

Path: LeetCode-main/solutions/1685. Sum of Absolute Differences in a Sorted Array/1685.py

```

class Solution:
    def getSumAbsoluteDifferences(self, nums: list[int]) -> list[int]:
        prefix = list(itertools.accumulate(nums))
        suffix = list(itertools.accumulate(nums[::-1]))[::-1]
        return [num * (i + 1) - prefix[i] + suffix[i] - num * (len(nums) - i)
                for i, num in enumerate(nums)]

```

626. 1687.py

Path: LeetCode-main/solutions/1687. Delivering Boxes from Storage to Ports/1687.py

```

class Solution:
    def boxDelivering(
        self,
        boxes: list[list[int]],
        portsCount: int,
        maxBoxes: int,
        maxWeight: int,
    ) -> int:
        n = len(boxes)
        # dp[i] := the minimum trips to deliver boxes[0..i] and return to the
        # storage
        dp = [0] * (n + 1)
        trips = 2
        weight = 0

        l = 0
        for r in range(n):
            weight += boxes[r][1]

            # The current box is different from the previous one, need to make one
            # more trip.
            if r > 0 and boxes[r][0] != boxes[r - 1][0]:
                trips += 1

            # Loading boxes[l] in the previous turn is always no bad than loading it
            # in this turn
            while r - l + 1 > maxBoxes or weight > maxWeight or (
                l < r and dp[l + 1] == dp[l]):
                weight -= boxes[l][1]
                if boxes[l][0] != boxes[l + 1][0]:
                    trips -= 1
                l += 1

            # min trips to deliver boxes[0..r]
            # = min trips to deliver boxes[0..l] + trips to deliver boxes[l..r]
            dp[r + 1] = dp[l] + trips

        return dp[n]

```

627. 1688.py

Path: LeetCode-main/solutions/1688. Count of Matches in Tournament/1688.py

```

class Solution:
    def numberOfMatches(self, n: int) -> int:
        return n - 1

```

628. 1689.py

Path: LeetCode-main/solutions/1689. Partitioning Into Minimum Number Of Deci-Binary Numbers/1689.py

```
class Solution:
    def minPartitions(self, n: str) -> int:
        return int(max(n))
```

629. 169.py

Path: LeetCode-main/solutions/169. Majority Element/169.py

```
class Solution:
    def majorityElement(self, nums: list[int]) -> int:
        ans = None
        count = 0

        for num in nums:
            if count == 0:
                ans = num
            count += (1 if num == ans else -1)

        return ans
```

630. 1694.py

Path: LeetCode-main/solutions/1694. Reformat Phone Number/1694.py

```
class Solution:
    def reformatNumber(self, number: str) -> str:
        ans = []

        number = number.replace("-", "").replace(" ", "")

        i = 0 # number's index
        while i + 4 < len(number):
            ans.append(number[i:i + 3] + '-')
            i += 3

        countFinalDigits = len(number) - i
        if countFinalDigits < 4:
            ans.append(number[i:])
        else: # countFinalDigits == 4
            ans.append(number[i:i + 2] + '-' + number[i + 2:])

        return ''.join(ans)
```

631. 1695.py

Path: LeetCode-main/solutions/1695. Maximum Erasure Value/1695.py


```

class Solution:
    def maximumUniqueSubarray(self, nums: list[int]) -> int:
        ans = 0
        score = 0
        seen = set()

        l = 0
        for r, num in enumerate(nums):
            while num in seen:
                score -= nums[l]
                seen.remove(nums[l])
                l += 1
            seen.add(nums[r])
            score += nums[r]
            ans = max(ans, score)

        return ans

```

632. 1696.py

Path: LeetCode-main/solutions/1696. Jump Game VI/1696.py

```

class Solution:
    def maxResult(self, nums: list[int], k: int) -> int:
        # Stores dp[i] within the bounds.
        maxQ = collections.deque([0])
        # dp[i] := the maximum score to consider nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]

        for i in range(1, len(nums)):
            # Pop the index if it's out-of-bounds.
            if maxQ[0] + k < i:
                maxQ.popleft()
            dp[i] = dp[maxQ[0]] + nums[i]
            # Pop indices that won't be chosen in the future.
            while maxQ and dp[maxQ[-1]] <= dp[i]:
                maxQ.pop()
            maxQ.append(i)

        return dp[-1]

```

633. 1698.py

Path: LeetCode-main/solutions/1698. Number of Distinct Substrings in a String/1698.py

```

class Solution:
    def countDistinct(self, s: str) -> int:
        BASE = 26
        HASH = 1_000_000_007

        n = len(s)
        ans = 0
        pow = [1] + [0] * n    # pow[i] := BASE^i
        hashes = [0] * (n + 1) # hashes[i] := the hash of s[0..i)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        for i in range(1, n + 1):
            pow[i] = pow[i - 1] * BASE % HASH
            hashes[i] = (hashes[i - 1] * BASE + val(s[i - 1])) % HASH

        def getHash(l: int, r: int) -> int:
            """Returns the hash of s[l..r)."""
            hash = (hashes[r] - hashes[l] * pow[r - l]) % HASH
            return hash + HASH if hash < 0 else hash

        for length in range(1, n + 1):
            seen = set()
            for i in range(n - length + 1):
                seen.add(getHash(i, i + length))
            ans += len(seen)

        return ans

```

634. 17-2.py

Path: LeetCode-main/solutions/17. Letter Combinations of a Phone Number/17-2.py

```

class Solution:
    def letterCombinations(self, digits: str) -> list[str]:
        if not digits:
            return []

        ans = ['']
        digitToLetters = ['', '', 'abc', 'def', 'ghi',
                          'jkl', 'mno', 'pqrs', 'tuv', 'wxyz']

        for d in digits:
            temp = []
            for s in ans:
                for c in digitToLetters[int(d)]:
                    temp.append(s + c)
            ans = temp

        return ans

```

635. 17.py

Path: LeetCode-main/solutions/17. Letter Combinations of a Phone Number/17.py

```

class Solution:
    def letterCombinations(self, digits: str) -> list[str]:
        if not digits:
            return []

        digitToLetters = ['', '', 'abc', 'def', 'ghi',
                           'jkl', 'mno', 'pqrs', 'tuv', 'wxyz']

        ans = []

        def dfs(i: int, path: list[str]) -> None:
            if i == len(digits):
                ans.append(''.join(path))
                return

            for letter in digitToLetters[int(digits[i])]:
                path.append(letter)
                dfs(i + 1, path)
                path.pop()

        dfs(0, [])
        return ans

```

636. 170.py

Path: LeetCode-main/solutions/170. Two Sum III - Data structure design/170.py

```

class TwoSum:
    def __init__(self):
        self.count = collections.Counter()

    def add(self, number: int) -> None:
        self.count[number] += 1

    def find(self, value: int) -> bool:
        for key, freq in self.count.items():
            remain = value - key
            if key == remain and freq > 1:
                return True
            if key != remain and remain in self.count:
                return True

        return False

```

637. 1700.py

Path: LeetCode-main/solutions/1700. Number of Students Unable to Eat Lunch/1700.py

```

class Solution:
    def countStudents(self, students: list[int], sandwiches: list[int]) -> int:
        count = collections.Counter(students)

        for i, sandwich in enumerate(sandwiches):
            if count[sandwich] == 0:
                return len(sandwiches) - i
            count[sandwich] -= 1

        return 0

```

638. 1702.py

Path: LeetCode-main/solutions/1702. Maximum Binary String After Change/1702.py

```
class Solution:
    def maximumBinaryString(self, binary: str) -> str:
        # e.g. binary = '100110'
        # Do Operation 2 -> '100011'
        # Do Operation 1 -> '111011'
        # So, the index of the only '0' is prefixOnes + zeros - 1.
        zeros = binary.count('0')
        prefixOnes = binary.find('0')

        # Make the entire string as 1s.
        ans = ['1'] * len(binary)

        # Make the only '0' if necessary.
        if prefixOnes != -1:
            ans[prefixOnes + zeros - 1] = '0'
        return ''.join(ans)
```

639. 1704.py

Path: LeetCode-main/solutions/1704. Determine if String Halves Are Alike/1704.py

```
class Solution:
    def halvesAreAlike(self, s: str) -> bool:
        VOWELS = 'aeiouAEIOU'
        aVowelsCount = sum(c in VOWELS for c in s[:len(s) // 2])
        bVowelsCount = sum(c in VOWELS for c in s[len(s) // 2:])
        return aVowelsCount == bVowelsCount
```

640. 1705.py

Path: LeetCode-main/solutions/1705. Maximum Number of Eaten Apples/1705.py

```
class Solution:
    def eatenApples(self, apples: list[int], days: list[int]) -> int:
        n = len(apples)
        ans = 0
        minHeap = [] # (the rotten day, the number of apples)

        i = 0
        while i < n or minHeap:
            # Remove the rotten apples.
            while minHeap and minHeap[0][0] <= i:
                heapq.heappop(minHeap)
            # Add today's apples.
            if i < n and apples[i] > 0:
                heapq.heappush(minHeap, (i + days[i], apples[i]))
            # Eat one apple today.
            if minHeap:
                rottenDay, numApples = heapq.heappop(minHeap)
                if numApples > 1:
                    heapq.heappush(minHeap, (rottenDay, numApples - 1))
                ans += 1
            i += 1

        return ans
```

641. 1706.py

Path: LeetCode-main/solutions/1706. Where Will the Ball Fall/1706.py

```
class Solution:
    def findBall(self, grid: list[list[int]]) -> list[int]:
        m = len(grid)
        n = len(grid[0])
        # dp[i] := status of the i-th column
        # -1 := empty, 0 := b0, 1 := b1, ...
        dp = [i for i in range(n)]
        # ans[i] := the i-th ball's final positio
        ans = [-1] * n

        for i in range(m):
            newDp = [-1] * n
            for j in range(n):
                # out-of-bounds
                if j + grid[i][j] < 0 or j + grid[i][j] == n:
                    continue
                if (grid[i][j] == 1 and grid[i][j + 1] == -1 or
                    grid[i][j] == -1 and grid[i][j - 1] == 1):
                    continue
                newDp[j + grid[i][j]] = dp[j]
            dp = newDp

        for i, ball in enumerate(dp):
            if ball != -1:
                ans[ball] = i

        return ans
```

642. 1707.py

Path: LeetCode-main/solutions/1707. Maximum XOR With an Element From Array/1707.py

```
from dataclasses import dataclass

class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2

class BitTrie:
    def __init__(self, maxBit: int):
        self.maxBit = maxBit
        self.root = TrieNode()

    def insert(self, num: int) -> None:
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = num >> i & 1
            if not node.children[bit]:
                node.children[bit] = TrieNode()
            node = node.children[bit]

    def getMaxXor(self, num: int) -> int:
        maxXor = 0
        node = self.root
        for i in range(self.maxBit, -1, -1):
```

```

bit = num >> i & 1
toggleBit = bit ^ 1
if node.children[toggleBit]:
    maxXor = maxXor | 1 << i
    node = node.children[toggleBit]
elif node.children[bit]:
    node = node.children[bit]
else: # There's nothing in the Bit Trie.
    return 0
return maxXor

```

```
@dataclass(frozen=True)
```

```
class IndexedQuery:
```

```
    queryIndex: int
```

```
    x: int
```

```
    m: int
```

```
    def __iter__(self):
```

```
        yield self.queryIndex
```

```
        yield self.x
```

```
        yield self.m
```

```
class Solution:
```

```
    def maximizeXor(self, nums: list[int], queries: list[list[int]]) -> list[int]:
```

```
        ans = [-1] * len(queries)
```

```
        maxBit = int(math.log2(max(max(nums), max(x for x, _ in queries))))
```

```
        bitTrie = BitTrie(maxBit)
```

```
        nums.sort()
```

```
        i = 0 # nums' index
```

```
        for queryIndex, x, m in sorted([IndexedQuery(i, x, m)
                                         for i, (x, m) in enumerate(queries)],
                                         key=lambda x: x.m):
```

```
            while i < len(nums) and nums[i] <= m:
```

```
                bitTrie.insert(nums[i])
```

```
                i += 1
```

```
            if i > 0 and nums[i - 1] <= m:
```

```
                ans[queryIndex] = bitTrie.getMaxXor(x)
```

```
        return ans
```

643. 1708.py

Path: LeetCode-main/solutions/1708. Largest Subarray Length K/1708.py

```
class Solution:
```

```
    def largestSubarray(self, nums: list[int], k: int) -> list[int]:
```

```
        mx = max(nums[:len(nums) - k + 1])
```

```
        i = nums.index(mx)
```

```
        return nums[i:i + k]
```

644. 171.py

Path: LeetCode-main/solutions/171. Excel Sheet Column Number/171.py

```
class Solution:
    def titleToNumber(self, columnTitle: str) -> int:
        return functools.reduce(lambda subtotal, c:
                                subtotal * 26 + ord(c) - ord('@'), columnTitle, 0)
```

645. 1710.py

Path: LeetCode-main/solutions/1710. Maximum Units on a Truck/1710.py

```
class Solution:
    def maximumUnits(self, boxTypes: list[list[int]], truckSize: int) -> int:
        ans = 0

        for boxes, units in sorted(boxTypes, key=lambda x: -x[1]):
            if boxes >= truckSize:
                return ans + truckSize * units
            ans += boxes * units
            truckSize -= boxes

        return ans
```

646. 1711.py

Path: LeetCode-main/solutions/1711. Count Good Meals/1711.py

```
class Solution:
    def countPairs(self, deliciousness: list[int]) -> int:
        MOD = 10**9 + 7
        MAX_BIT = 20 + 1
        ans = 0
        count = collections.Counter()

        for d in deliciousness:
            for i in range(MAX_BIT + 1):
                power = 1 << i
                ans += count[power - d]
                ans %= MOD
                count[d] += 1

        return ans
```

647. 1712-2.py

Path: LeetCode-main/solutions/1712. Ways to Split Array Into Three Subarrays/1712-2.py

```
class Solution:
    def waysToSplit(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        ans = 0
        prefix = list(itertools.accumulate(nums))

        j = 0
        k = 0
        for i in range(n - 2):
            # Find the first index j s.t.
            # left = prefix[i] <= mid = prefix[j] - prefix[i]
            j = max(j, i + 1)
            while j < n - 1 and prefix[i] > prefix[j] - prefix[i]:
                j += 1
            # Find the first index k s.t.
            # mid = prefix[k] - prefix[i] > right = prefix[-1] - prefix[k]
            k = max(k, j)
            while k < n - 1 and prefix[k] - prefix[i] <= prefix[-1] - prefix[k]:
                k += 1
            ans += k - j
            ans %= MOD

        return ans
```

648. 1712.py

Path: LeetCode-main/solutions/1712. Ways to Split Array Into Three Subarrays/1712.py


```

class Solution:
    def waysToSplit(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        ans = 0
        prefix = list(itertools.accumulate(nums))

        def firstGreaterEqual(i: int) -> int:
            """Finds the first index j s.t.
            Mid = prefix[j] - prefix[i] >= left = prefix[i]
            """
            l = i + 1
            r = n - 1
            while l < r:
                m = (l + r) // 2
                if prefix[m] - prefix[i] >= prefix[i]:
                    r = m
                else:
                    l = m + 1
            return l

        def firstGreater(i: int) -> int:
            """Finds the first index k s.t.
            mid = prefix[k] - prefix[i] > right = prefix[-1] - prefix[k]
            """
            l = i + 1
            r = n - 1
            while l < r:
                m = (l + r) // 2
                if prefix[m] - prefix[i] > prefix[-1] - prefix[m]:
                    r = m
                else:
                    l = m + 1
            return l

        for i in range(n - 2):
            j = firstGreaterEqual(i)
            if j == n - 1:
                break
            mid = prefix[j] - prefix[i]
            right = prefix[-1] - prefix[j]
            if mid > right:
                continue
            k = firstGreater(i)
            ans = (ans + k - j) % MOD

        return ans

```

649. 1713.py

Path: LeetCode-main/solutions/1713. Minimum Operations to Make a Subsequence/1713.py

```

class Solution:
    def minOperations(self, target: list[int], arr: list[int]) -> int:
        indices = []
        numToIndex = {num: i for i, num in enumerate(target)}

        for a in arr:
            if a in numToIndex:
                indices.append(numToIndex[a])

        return len(target) - self._lengthOfLIS(indices)

# Same as 300. Longest Increasing Subsequence
def _lengthOfLIS(self, nums: list[int]) -> int:
    # tails[i] := the minimum tail of all the increasing subsequences having
    # length i + 1
    tails = []
    for num in nums:
        if not tails or num > tails[-1]:
            tails.append(num)
        else:
            tails[bisect.bisect_left(tails, num)] = num
    return len(tails)

```

650. 1714.py

Path: LeetCode-main/solutions/1714. Sum Of Special Evenly-Spaced Elements In Array/1714.py

```

class Solution:
    def solve(self, nums: list[int], queries: list[list[int]]) -> list[int]:
        MOD = 10**9 + 7
        n = len(nums)
        sqrtN = int(n**0.5)
        # prefix[x][y] = sum(nums[x + ay]), where a >= 0 and x + ay < n
        # Set prefix[i][j] to nums[i] to indicate the sequence starts with nums[i].
        prefix = [[num] * sqrtN for num in nums]

        for x in range(n - 1, -1, -1):
            for y in range(1, sqrtN):
                if x + y < n:
                    prefix[x][y] += prefix[x + y][y]
                    prefix[x][y] %= MOD

        return [prefix[x][y] if y < sqrtN
                else sum(nums[x:y]) % MOD
                for x, y in queries]

```

651. 1716.py

Path: LeetCode-main/solutions/1716. Calculate Money in Leetcode Bank/1716.py

```

class Solution:
    def totalMoney(self, n: int) -> int:
        def trapezoid(a: int, b: int) -> int:
            """Returns sum(a..b)."""
            return (a + b) * (b - a + 1) // 2

        weeks = n // 7
        firstWeek = trapezoid(1, 7)
        lastFullWeek = trapezoid(1 + weeks - 1, 7 + weeks - 1)
        remainingDays = trapezoid(1 + weeks, n % 7 + weeks)
        return (firstWeek + lastFullWeek) * weeks // 2 + remainingDays

```

652. 1717.py

Path: LeetCode-main/solutions/1717. Maximum Score From Removing Substrings/1717.py

```

class Solution:
    def maximumGain(self, s: str, x: int, y: int) -> int:
        # The assumption that gain('ab') > gain('ba') while removing 'ba' first is
        # optimal is contradicted. Only 'b(ab)a' satisfies the condition of
        # preventing two 'ba' removals, but after removing 'ab', we can still
        # remove one 'ba', resulting in a higher gain. Thus, removing 'ba' first is
        # not optimal.
        return (self._gain(s, 'ab', x, 'ba', y) if x > y else
                self._gain(s, 'ba', y, 'ab', x))

    # Returns the points gained by first removing sub1 ('ab' | 'ba') from s with
    # point1, then removing sub2 ('ab' | 'ba') from s with point2.
    def _gain(self, s: str, sub1: str, point1: int, sub2: str, point2: int) -> int:
        points = 0
        stack1 = []
        stack2 = []

        # Remove 'sub1' from s with point1 gain.
        for c in s:
            if stack1 and stack1[-1] == sub1[0] and c == sub1[1]:
                stack1.pop()
                points += point1
            else:
                stack1.append(c)

        # Remove 'sub2' from s with point2 gain.
        for c in stack1:
            if stack2 and stack2[-1] == sub2[0] and c == sub2[1]:
                stack2.pop()
                points += point2
            else:
                stack2.append(c)

        return points

```

653. 1718.py

Path: LeetCode-main/solutions/1718. Construct the Lexicographically Largest Valid Sequence/1718.py

```

class Solution:
    def constructDistancedSequence(self, n: int) -> list[int]:
        ans = [0] * (2 * n - 1)

        def dfs(i: int, mask: int) -> bool:
            if i == len(ans):
                return True
            if ans[i] > 0:
                return dfs(i + 1, mask)

            # Greedily fill in `ans` in descending order.
            for num in range(n, 0, -1):
                if (mask >> num & 1) == 1:
                    continue
                if num == 1:
                    ans[i] = num
                    if dfs(i + 1, mask | 1 << num):
                        return True
                    ans[i] = 0
                else: # num in [2, n]
                    if i + num >= len(ans) or ans[i + num] > 0:
                        continue
                    ans[i] = num
                    ans[i + num] = num
                    if dfs(i + 1, mask | 1 << num):
                        return True
                    ans[i + num] = 0
                    ans[i] = 0

            return False

        dfs(0, 0)
        return ans

```

654. 1719.py

Path: LeetCode-main/solutions/1719. Number Of Ways To Reconstruct A Tree/1719.py

```

class Solution:
    def checkWays(self, pairs: list[list[int]]) -> int:
        MAX = 501
        graph = collections.defaultdict(list)
        degrees = [0] * MAX
        connected = [[False] * MAX for _ in range(MAX)]

        for u, v in pairs:
            graph[u].append(v)
            graph[v].append(u)
            degrees[u] += 1
            degrees[v] += 1
            connected[u][v] = True
            connected[v][u] = True

        # For each node, sort its children by degrees in descending order.
        for _, children in graph.items():
            children.sort(key=lambda x: -degrees[x])

        # Find the root with a degree that equals to n - 1.
        root = next((i for i, d in enumerate(degrees) if d == len(graph) - 1), -1)
        if root == -1:
            return 0

        hasMoreThanOneWay = False

        def dfs(u: int, ancestors: list[int], seen: list[bool]) -> bool:
            """
            Returns True if each node rooted at u is connected to all of its
            ancestors.
            """
            nonlocal hasMoreThanOneWay
            seen[u] = True
            for ancestor in ancestors:
                if not connected[u][ancestor]:
                    return False
            ancestors.append(u)
            for v in graph[u]:
                if seen[v]:
                    continue
                if degrees[v] == degrees[u]:
                    hasMoreThanOneWay = True
                if not dfs(v, ancestors, seen):
                    return False
            ancestors.pop()
            return True

        if not dfs(root, [], [False] * MAX):
            return 0
        return 2 if hasMoreThanOneWay else 1

```

655. 172.py

Path: LeetCode-main/solutions/172. Factorial Trailing Zeroes/172.py

```

class Solution:
    def trailingZeroes(self, n: int) -> int:
        return 0 if n == 0 else n // 5 + self.trailingZeroes(n // 5)

```

656. 1720.py

Path: LeetCode-main/solutions/1720. Decode XORed Array/1720.py

```
class Solution:
    def decode(self, encoded: list[int], first: int) -> list[int]:
        ans = [first]

        for e in encoded:
            ans.append(e ^ ans[-1])

        return ans
```

657. 1721.py

Path: LeetCode-main/solutions/1721. Swapping Nodes in a Linked List/1721.py

```
class Solution:
    def swapNodes(self, head: ListNode | None, k: int) -> ListNode | None:
        p = None # Points the k-th node from the beginning.
        q = None # Points the k-th node from the end.

        curr = head
        while curr:
            if q:
                q = q.next
            k -= 1
            if k == 0:
                p = curr
                q = head
            curr = curr.next

        p.val, q.val = q.val, p.val
        return head
```

658. 1722.py

Path: LeetCode-main/solutions/1722. Minimize Hamming Distance After Swap Operations/1722.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def minimumHammingDistance(
        self,
        source: list[int],
        target: list[int],
        allowedSwaps: list[list[int]],
    ) -> int:
        n = len(source)
        ans = 0
        uf = UnionFind(n)
        groupIdToCount = [collections.Counter() for _ in range(n)]

        for a, b in allowedSwaps:
            uf.unionByRank(a, b)

        for i in range(n):
            groupIdToCount[uf.find(i)][source[i]] += 1

        for i in range(n):
            groupId = uf.find(i)
            count = groupIdToCount[groupId]
            if target[i] not in count:
                ans += 1
            else:
                count[target[i]] -= 1
                if count[target[i]] == 0:
                    del count[target[i]]

        return ans

```

659. 1723.py

Path: LeetCode-main/solutions/1723. Find Minimum Time to Finish All Jobs/1723.py

```

class Solution:
    def minimumTimeRequired(self, jobs: list[int], k: int) -> int:
        ans = sum(jobs)
        times = [0] * k # times[i] := accumulate time of workers[i]

        # Assign the most time-consuming job first.
        jobs.sort(reverse=True)

        def dfs(s: int) -> None:
            nonlocal ans
            if s == len(jobs):
                ans = min(ans, max(times))
                return
            for i in range(k):
                # There is no need to explore assigning jobs[s] to workers[i] further as
                # it would not yield better results.
                if times[i] + jobs[s] >= ans:
                    continue
                times[i] += jobs[s]
                dfs(s + 1)
                times[i] -= jobs[s]
                # It's always non-optimal to have a worker with no jobs.
                if times[i] == 0:
                    return

        dfs(0)
        return ans

```

660. 1725.py

Path: LeetCode-main/solutions/1725. Number Of Rectangles That Can Form The Largest Square/1725.py

```

class Solution:
    def countGoodRectangles(self, rectangles: list[list[int]]) -> int:
        minSides = [min(x, y) for x, y in rectangles]
        return minSides.count(max(minSides))

```

661. 1726.py

Path: LeetCode-main/solutions/1726. Tuple with Same Product/1726.py

```

class Solution:
    def tupleSameProduct(self, nums: list[int]) -> int:
        # nums of ways to arrange (a, b) = 2
        # nums of ways to arrange (c, d) = 2
        # nums of ways to arrange (a, b), (c, d) = 2^3 = 8
        ans = 0
        count = collections.Counter()

        for i in range(len(nums)):
            for j in range(i):
                prod = nums[i] * nums[j]
                ans += count[prod] * 8
                count[prod] += 1

        return ans

```


662. 1727.py

Path: LeetCode-main/solutions/1727. Largest Submatrix With Rearrangements/1727.py

```
class Solution:
    def largestSubmatrix(self, matrix: list[list[int]]) -> int:
        ans = 0
        hist = [0] * len(matrix[0])

        for row in matrix:
            # Accumulate the histogram if possible.
            for i, num in enumerate(row):
                hist[i] = 0 if num == 0 else hist[i] + 1

            # Get the sorted histogram.
            sortedHist = sorted(hist)

            # Greedily calculate the answer.
            for i, h in enumerate(sortedHist):
                ans = max(ans, h * (len(row) - i))

        return ans
```

663. 1728.py

Path: LeetCode-main/solutions/1728. Cat and Mouse II/1728.py

```
class Solution:
    def canMouseWin(self, grid: list[str], catJump: int, mouseJump: int) -> bool:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        nFloors = 0
        cat = 0 # cat's position
        mouse = 0 # mouse's position

        def hash(i: int, j: int) -> int:
            return i * n + j

        for i in range(m):
            for j in range(n):
                if grid[i][j] != '#':
                    nFloors += 1
                if grid[i][j] == 'C':
                    cat = hash(i, j)
                elif grid[i][j] == 'M':
                    mouse = hash(i, j)

        @functools.lru_cache(None)
        def dp(cat: int, mouse: int, turn: int) -> bool:
            """
            Returns True if the mouse can win, where the cat is on (i / 8, i % 8), the
            mouse is on (j / 8, j % 8), and the turns is k.
            """
            # We already search the whole touchable grid.
            if turn == nFloors * 2:
                return False

            if turn % 2 == 0:
                # the mouse's turn
                i = mouse // n
                j = mouse % n
```

```

for dx, dy in DIRS:
    for jump in range(mouseJump + 1):
        x = i + dx * jump
        y = j + dy * jump
        if x < 0 or x == m or y < 0 or y == n:
            break
        if grid[x][y] == '#':
            break
        # The mouse eats the food, so the mouse wins.
        if grid[x][y] == 'F':
            return True
        if dp(cat, hash(x, y), turn + 1):
            return True
    # The mouse can't win, so the mouse loses.
    return False
else:
    # the cat's turn
    i = cat // n
    j = cat % n
    for dx, dy in DIRS:
        for jump in range(catJump + 1):
            x = i + dx * jump
            y = j + dy * jump
            if x < 0 or x == m or y < 0 or y == n:
                break
            if grid[x][y] == '#':
                break
            # The cat eats the food, so the mouse loses.
            if grid[x][y] == 'F':
                return False
            nextCat = hash(x, y)
            # The cat catches the mouse, so the mouse loses.
            if nextCat == mouse:
                return False
            if not dp(nextCat, mouse, turn + 1):
                return False
    # The cat can't win, so the mouse wins.
    return True

return dp(cat, mouse, 0)

```

664. 173-2.py

Path: LeetCode-main/solutions/173. Binary Search Tree Iterator/173-2.py

```

class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.stack = []
        self._pushLeftsUntilNull(root)

    def next(self) -> int:
        root = self.stack.pop()
        self._pushLeftsUntilNull(root.right)
        return root.val

    def hasNext(self) -> bool:
        return self.stack

    def _pushLeftsUntilNull(self, root: TreeNode | None) -> None:
        while root:
            self.stack.append(root)
            root = root.left

```

665. 173.py

Path: LeetCode-main/solutions/173. Binary Search Tree Iterator/173.py

```
class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.i = 0
        self.vals = []
        self._inorder(root)

    def next(self) -> int:
        self.i += 1
        return self.vals[self.i - 1]

    def hasNext(self) -> bool:
        return self.i < len(self.vals)

    def _inorder(self, root: TreeNode | None) -> None:
        if not root:
            return
        self._inorder(root.left)
        self.vals.append(root.val)
        self._inorder(root.right)
```

666. 1730.py

Path: LeetCode-main/solutions/1730. Shortest Path to Get Food/1730.py

```
class Solution:
    def getFood(self, grid: list[list[str]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        ans = 0
        q = collections.deque([self._getStartLocation(grid)])

        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if grid[x][y] == 'X':
                        continue
                    if grid[x][y] == '#':
                        return ans + 1
                    q.append((x, y))
                    grid[x][y] = 'X' # Mark as visited.
                ans += 1

        return -1

    def _getStartLocation(self, grid: list[list[str]]) -> tuple[int, int]:
        for i, row in enumerate(grid):
            for j, cell in enumerate(row):
                if cell == '*':
                    return (i, j)
```

667. 1732.py

Path: LeetCode-main/solutions/1732. Find the Highest Altitude/1732.py

```
class Solution:
    def largestAltitude(self, gain: list[int]) -> int:
        ans = 0
        currAltitude = 0
        for g in gain:
            currAltitude += g
            ans = max(ans, currAltitude)
        return ans
```

668. 1733.py

Path: LeetCode-main/solutions/1733. Minimum Number of People to Teach/1733.py

```
class Solution:
    def minimumTeachings(
        self,
        n: int,
        languages: list[list[int]],
        friendships: list[list[int]],
    ) -> int:
        languageSets = [set(languages) for languages in languages]
        needTeach = set()
        languageCount = collections.Counter()

        # Find friends that can't communicate.
        for u, v in friendships:
            if not languageSets[u - 1] & languageSets[v - 1]:
                needTeach.add(u - 1)
                needTeach.add(v - 1)

        # Find the most popular language.
        for u in needTeach:
            for language in languageSets[u]:
                languageCount[language] += 1

        # Teach the most popular language to people don't understand.
        return len(needTeach) - max(languageCount.values(), default=0)
```

669. 1734.py

Path: LeetCode-main/solutions/1734. Decode XORed Permutation/1734.py

```

class Solution:
    def decode(self, encoded: list[int]) -> list[int]:
        # Our goal is to find the value of a1, which will allow us to decode a2, a3,
        # ..., an. This can be achieved by performing XOR operation between each
        # element in `encoded` and a1.
        #
        # e.g. n = 3, perm = [a1, a2, a3] is a permutation of [1, 2, 3].
        #         encoded = [a1^a2, a2^a3]
        # accumulatedEncoded = [a1^a2, a1^a3]
        # a1 = (a1^a2)^(a1^a3)^(a1^a2^a3)
        # a2 = a1^(a1^a2)
        # a3 = a2^(a2^a3)
        n = len(encoded) + 1
        nXors = functools.reduce(operator.xor, [i for i in range(1, n + 1)])

        # Instead of constructing the array, we can track of the running XOR value
        # of `accumulatedEncoded`.
        xors = 0 # xors(accumulatedEncoded)

        for encode in encoded:
            runningXors ^= encode
            xors ^= runningXors

        ans = [xors ^ nXors]

        for encode in encoded:
            ans.append(ans[-1] ^ encode)

        return ans

```

670. 1735.py

Path: LeetCode-main/solutions/1735. Count Ways to Make Array With Product/1735.py

```

class Solution:
    def waysToFillArray(self, queries: list[list[int]]) -> list[int]:
        MOD = 1_000_000_007
        MAX = 10_000
        minPrimeFactors = self._sieveEratosthenes(MAX + 1)

        @functools.lru_cache(None)
        def fact(i: int) -> int:
            return 1 if i <= 1 else i * fact(i - 1) % MOD

        @functools.lru_cache(None)
        def inv(i: int) -> int:
            return pow(i, MOD - 2, MOD)

        @functools.lru_cache(None)
        def nCk(n: int, k: int) -> int:
            return fact(n) * inv(fact(k)) * inv(fact(n - k)) % MOD

        ans = []

        for n, k in queries:
            res = 1
            for freq in self._getPrimeFactorsCount(k, minPrimeFactors).values():
                res = res * nCk(n - 1 + freq, freq) % MOD
            ans.append(res)

        return ans

    def _sieveEratosthenes(self, n: int) -> list[int]:
        """Gets the minimum prime factor of i, where 1 < i <= n."""
        minPrimeFactors = [i for i in range(n + 1)]
        for i in range(2, int(n**0.5) + 1):
            if minPrimeFactors[i] == i: # `i` is prime.
                for j in range(i * i, n, i):
                    minPrimeFactors[j] = min(minPrimeFactors[j], i)
        return minPrimeFactors

    def _getPrimeFactorsCount(self, num: int, minPrimeFactors: list[int]) -> dict[int, int]:
        count = collections.Counter()
        while num > 1:
            divisor = minPrimeFactors[num]
            while num % divisor == 0:
                num //= divisor
                count[divisor] += 1
        return count

```

671. 1736.py

Path: LeetCode-main/solutions/1736. Latest Time by Replacing Hidden Digits/1736.py

```

class Solution:
    def maximumTime(self, time: str) -> str:
        ans = list(time)
        if time[0] == '?':
            ans[0] = '2' if time[1] == '?' or time[1] < '4' else '1'
        if time[1] == '?':
            ans[1] = '3' if ans[0] == '2' else '9'
        if time[3] == '?':
            ans[3] = '5'
        if time[4] == '?':
            ans[4] = '9'
        return ''.join(ans)

```

672. 1739.py

Path: LeetCode-main/solutions/1739. Building Boxes/1739.py

```

class Solution:
    def minimumBoxes(self, n: int) -> int:
        nBoxes = 0
        nextTouchings = 0 # j
        currLevelBoxes = 0 # 1 + 2 + ... + j

        # Find the minimum j s.t. `nBoxes` = 1 + (1 + 2) + ... + (1 + 2 + ... + j)
        # >= n
        while nBoxes < n:
            nextTouchings += 1
            currLevelBoxes += nextTouchings
            nBoxes += currLevelBoxes

        # If nBoxes = n, the answer is `currLevelBoxes` = 1 + 2 + ... + j.
        if nBoxes == n:
            return currLevelBoxes

        # Otherwise, need to remove the boxes in the current level and rebuild it.
        nBoxes -= currLevelBoxes
        currLevelBoxes -= nextTouchings
        nextTouchings = 0

        while nBoxes < n:
            nextTouchings += 1
            nBoxes += nextTouchings

        return currLevelBoxes + nextTouchings

```

673. 174.py

Path: LeetCode-main/solutions/174. Dungeon Game/174.py

```
class Solution:
    def calculateMinimumHP(self, dungeon: list[list[int]]) -> int:
        m = len(dungeon)
        n = len(dungeon[0])
        dp = [math.inf] * (n + 1)
        dp[n - 1] = 1

        for i in reversed(range(m)):
            for j in reversed(range(n)):
                dp[j] = min(dp[j], dp[j + 1]) - dungeon[i][j]
                dp[j] = max(dp[j], 1)

        return dp[0]
```

674. 1740.py

Path: LeetCode-main/solutions/1740. Find Distance in a Binary Tree/1740.py

```
class Solution:
    def findDistance(self, root: TreeNode, p: int, q: int) -> int:
        def getLCA(root, p, q):
            if not root or root.val == p or root.val == q:
                return root

            l = getLCA(root.left, p, q)
            r = getLCA(root.right, p, q)

            if l and r:
                return root
            return l or r

        def dist(lca, target):
            if not lca:
                return 10000
            if lca.val == target:
                return 0
            return 1 + min(dist(lca.left, target), dist(lca.right, target))

        lca = getLCA(root, p, q)
        return dist(lca, p) + dist(lca, q)
```

675. 1742.py

Path: LeetCode-main/solutions/1742. Maximum Number of Balls in a Box/1742.py


```

class Solution:
    def countBalls(self, lowLimit: int, highLimit: int) -> int:
        maxDigitSum = 9 * 5 # 99999
        ans = 0
        count = [0] * (maxDigitSum + 1)

        for num in range(lowLimit, highLimit + 1):
            digitSum = self._getDigitSum(num)
            count[digitSum] += 1
            ans = max(ans, count[digitSum])

        return ans

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))

```

676. 1743.py

Path: LeetCode-main/solutions/1743. Restore the Array From Adjacent Pairs/1743.py

```

class Solution:
    def restoreArray(self, adjacentPairs: list[list[int]]) -> list[int]:
        ans = []
        numToAdjs = collections.defaultdict(list)

        for a, b in adjacentPairs:
            numToAdjs[a].append(b)
            numToAdjs[b].append(a)

        for num, adjs in numToAdjs.items():
            if len(adjs) == 1:
                ans.append(num)
                ans.append(adjs[0])
                break

        while len(ans) < len(adjacentPairs) + 1:
            tail = ans[-1]
            prev = ans[-2]
            adjs = numToAdjs[tail]
            if adjs[0] == prev: # adjs[0] is already used
                ans.append(adjs[1])
            else:
                ans.append(adjs[0])

        return ans

```

677. 1744.py

Path: LeetCode-main/solutions/1744. Can You Eat Your Favorite Candy on Your Favorite Day?/1744.py

```

class Solution:
    def canEat(
        self,
        candiesCount: list[int],
        queries: list[list[int]]
    ) -> list[bool]:
        prefix = list(itertools.accumulate(candiesCount, initial=0))
        return [prefix[t] // c <= d < prefix[t + 1] for t, d, c in queries]

```

678. 1745-2.py

Path: LeetCode-main/solutions/1745. Palindrome Partitioning IV/1745-2.py

```
class Solution:
    def checkPartitioning(self, s: str) -> bool:
        n = len(s)
        # dp[i][j] := true if s[i..j] is a palindrome
        dp = [[False] * (n + 1) for _ in range(n + 1)]

        for i in range(n):
            dp[i][i] = True

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                if s[i] == s[j]:
                    dp[i][j] = i + 1 > j - 1 or dp[i + 1][j - 1]

        for i, j in itertools.combinations(range(n), 2):
            if dp[0][i] and dp[i + 1][j] and dp[j + 1][n - 1]:
                return True

        return False
```

679. 1745.py

Path: LeetCode-main/solutions/1745. Palindrome Partitioning IV/1745.py

```
class Solution:
    def checkPartitioning(self, s: str) -> bool:
        @functools.lru_cache(None)
        def isPalindrome(i: int, j: int) -> bool:
            """Returns True if s[i..j] is a palindrome."""
            if i > j:
                return True
            if s[i] == s[j]:
                return isPalindrome(i + 1, j - 1)
            return False

        n = len(s)
        return any(isPalindrome(0, i) and
                   isPalindrome(i + 1, j) and
                   isPalindrome(j + 1, n - 1)
                   for i in range(n)
                   for j in range(i + 1, n - 1))
```

680. 1746.py

Path: LeetCode-main/solutions/1746. Maximum Subarray Sum After One Operation/1746.py

```
class Solution:
    def maxSumAfterOperation(self, nums: list[int]) -> int:
        ans = -math.inf
        regular = 0
        squared = 0

        for num in nums:
            squared = max(num**2, regular + num**2, squared + num)
            regular = max(num, regular + num)
            ans = max(ans, squared)

        return ans
```

681. 1748.py

Path: LeetCode-main/solutions/1748. Sum of Unique Elements/1748.py

```
class Solution:
    def sumOfUnique(self, nums: list[int]) -> int:
        return sum(num
                    for num, freq in collections.Counter(nums).items()
                    if freq == 1)
```

682. 1749-2.py

Path: LeetCode-main/solutions/1749. Maximum Absolute Sum of Any Subarray/1749-2.py

```
class Solution:
    def maxAbsoluteSum(self, nums):
        summ = 0
        maxPrefix = 0
        minPrefix = 0

        for num in nums:
            summ += num
            maxPrefix = max(maxPrefix, summ)
            minPrefix = min(minPrefix, summ)

        return maxPrefix - minPrefix
```

683. 1749.py

Path: LeetCode-main/solutions/1749. Maximum Absolute Sum of Any Subarray/1749.py

```
class Solution:
    def maxAbsoluteSum(self, nums):
        ans = -math.inf
        maxSum = 0
        minSum = 0

        for num in nums:
            maxSum = max(num, maxSum + num)
            minSum = min(num, minSum + num)
            ans = max(ans, maxSum, -minSum)

        return ans
```

684. 1750.py

Path: LeetCode-main/solutions/1750. Minimum Length of String After Deleting Similar Ends/1750.py

```
class Solution:
    def minimumLength(self, s: str) -> int:
        i = 0
        j = len(s) - 1

        while i < j and s[i] == s[j]:
            c = s[i]
            while i <= j and s[i] == c:
                i += 1
            while i <= j and s[j] == c:
                j -= 1

        return j - i + 1
```

685. 1751.py

Path: LeetCode-main/solutions/1751. Maximum Number of Events That Can Be Attended II/1751.py

```
class Solution:
    def maxValue(self, events: list[list[int]], k: int) -> int:
        events.sort()

        @functools.lru_cache(None)
        def dp(i: int, k: int) -> int:
            """
            Returns the maximum sum of values that you can receive by attending
            events[i..n), where k is the maximum number of attendance.
            """
            if k == 0 or i == len(events):
                return 0

            # Binary search `events` to find the first index j
            # s.t. events[j][0] > events[i][1].
            j = bisect.bisect(events, [events[i][1], math.inf, math.inf], i + 1)
            return max(events[i][2] + dp(j, k - 1), dp(i + 1, k))

        return dp(0, k)
```

686. 1754.py

Path: LeetCode-main/solutions/1754. Largest Merge Of Two Strings/1754.py

```
class Solution:
    def largestMerge(self, word1: str, word2: str) -> str:
        if not word1:
            return word2
        if not word2:
            return word1
        if word1 > word2:
            return word1[0] + self.largestMerge(word1[1:], word2)
        return word2[0] + self.largestMerge(word1, word2[1:])
```

687. 1755.py

Path: LeetCode-main/solutions/1755. Closest Subsequence Sum/1755.py

```
class Solution:
    def minAbsDifference(self, nums: list[int], goal: int) -> int:
        n = len(nums) // 2
        ans = math.inf
        lSums = []
        rSums = []

        def dfs(arr: list[int], i: int, path: int, sums: list[int]) -> None:
            if i == len(arr):
                sums.append(path)
                return
            dfs(arr, i + 1, path + arr[i], sums)
            dfs(arr, i + 1, path, sums)

        dfs(nums[:n], 0, 0, lSums)
        dfs(nums[n:], 0, 0, rSums)
        rSums.sort()

        for lSum in lSums:
            i = bisect_left(rSums, goal - lSum)
            if i < len(rSums): # 2^n
                ans = min(ans, abs(goal - lSum - rSums[i]))
            if i > 0:
                ans = min(ans, abs(goal - lSum - rSums[i - 1]))

        return ans
```

688. 1756.py

Path: LeetCode-main/solutions/1756. Design Most Recently Used Queue/1756.py

```
from sortedcontainers import SortedList

class MRUQueue:
    def __init__(self, n: int):
        # [(priority value, actual value)]
        self.q = SortedList((i, i) for i in range(1, n + 1))

    def fetch(self, k: int) -> int:
        _, num = self.q.pop(k - 1)
        if self.q:
            maxPriority = self.q[-1][0]
            self.q.add((maxPriority + 1, num))
        else:
            self.q.add((0, num))
        return num
```

689. 1758.py

Path: LeetCode-main/solutions/1758. Minimum Changes To Make Alternating Binary String/1758.py

```
class Solution:
    def minOperations(self, s: str) -> int:
        # the cost to make s "1010"
        cost10 = sum(int(c) == i % 2 for i, c in enumerate(s))
        # the cost to make s "0101"
        cost01 = len(s) - cost10
        return min(cost10, cost01)
```

690. 1759.py

Path: LeetCode-main/solutions/1759. Count Number of Homogenous Substrings/1759.py

```
class Solution:
    def countHomogenous(self, s: str) -> int:
        MOD = 1_000_000_007
        ans = 0
        count = 0
        currentChar = '@'

        for c in s:
            count = count + 1 if c == currentChar else 1
            currentChar = c
            ans += count
            ans %= MOD

        return ans
```

691. 1760.py

Path: LeetCode-main/solutions/1760. Minimum Limit of Balls in a Bag/1760.py

```
class Solution:
    def minimumSize(self, nums: list[int], maxOperations: int) -> int:
        def numOperations(m: int) -> int:
            """Returns the number of operations required to make m penalty."""
            return sum((num - 1) // m for num in nums)

        l = 1
        r = max(nums)
        return bisect.bisect_left(
            range(l, r),
            True, key=lambda m: numOperations(m) <= maxOperations) + 1
```

692. 1761.py

Path: LeetCode-main/solutions/1761. Minimum Degree of a Connected Trio in a Graph/1761.py

```

class Solution:
    def minTrioDegree(self, n: int, edges: list[list[int]]) -> int:
        ans = math.inf
        graph = [set() for _ in range(n)]
        degrees = [0] * n

        for u, v in edges:
            u -= 1
            v -= 1
            # Store the mapping from `min(u, v)` to `max(u, v)` to speed up.
            graph[min(u, v)].add(max(u, v))
            degrees[u] += 1
            degrees[v] += 1

        for u in range(n):
            for v in graph[u]:
                for w in graph[v]:
                    if w in graph[v]:
                        ans = min(ans, degrees[u] + degrees[v] + degrees[w] - 6)

        return -1 if ans == math.inf else ans

```

693. 1762.py

Path: LeetCode-main/solutions/1762. Buildings With an Ocean View/1762.py

```

class Solution:
    def findBuildings(self, heights: list[int]) -> list[int]:
        stack = []

        for i, height in enumerate(heights):
            while stack and heights[stack[-1]] <= height:
                stack.pop()
            stack.append(i)

        return stack

```

694. 1763.py

Path: LeetCode-main/solutions/1763. Longest Nice Substring/1763.py

```

class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        if len(s) < 2:
            return ''

        seen = set(s)

        for i, c in enumerate(s):
            # If both upper and lower case letters exists in the string, keep moving,
            # else take the erroneous character as a partition and check for its left
            # and right parts to be nice strings.
            if c.swapcase() not in seen:
                prefix = self.longestNiceSubstring(s[:i])
                suffix = self.longestNiceSubstring(s[i + 1:])
                return max(prefix, suffix, key=len)

        return s

```

695. 1764.py

Path: LeetCode-main/solutions/1764. Form Array by Concatenating Subarrays of Another Array/1764.py

```
class Solution:
    def canChoose(self, groups: list[list[int]], nums: list[int]) -> bool:
        i = 0 # groups' index
        j = 0 # nums' index

        while i < len(groups) and j < len(nums):
            if self._isMatch(groups[i], nums, j):
                j += len(groups[i])
                i += 1
            else:
                j += 1

        return i == len(groups)

    # Returns True if group == nums[j..j + |group|].
    def _isMatch(self, group: list[int], nums: list[int], j: int) -> bool:
        if j + |group| > len(nums):
            return False
        for i, g in enumerate(group):
            if g != nums[j + i]:
                return False
        return True
```

696. 1765.py

Path: LeetCode-main/solutions/1765. Map of Highest Peak/1765.py

```
class Solution:
    def highestPeak(self, isWater: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(isWater)
        n = len(isWater[0])
        ans = [[-1] * n for _ in range(m)]
        q = collections.deque()

        for i in range(m):
            for j in range(n):
                if isWater[i][j] == 1:
                    q.append((i, j))
                    ans[i][j] = 0

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if ans[x][y] != -1:
                    continue
                ans[x][y] = ans[i][j] + 1
                q.append((x, y))

        return ans
```

697. 1766.py

Path: LeetCode-main/solutions/1766. Tree of Coprimes/1766.py

```
class Solution:
    def getCoprimes(self, nums: list[int], edges: list[list[int]]) -> list[int]:
        MAX = 50
        ans = [-1] * len(nums)
        tree = [[] for _ in range(len(nums))]
        # stacks[i] := (node, depth)s of nodes with value i
        stacks = [[] for _ in range(MAX + 1)]

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def getAncestor(u: int) -> int:
            maxNode = -1
            maxDepth = -1
            for i, stack in enumerate(stacks):
                if stack and stack[-1][1] > maxDepth and math.gcd(nums[u], i) == 1:
                    maxNode, maxDepth = stack[-1]
            return maxNode

        def dfs(u: int, prev: int, depth: int) -> int:
            ans[u] = getAncestor(u)
            stacks[nums[u]].append((u, depth))

            for v in tree[u]:
                if v != prev:
                    dfs(v, u, depth + 1)

            stacks[nums[u]].pop()

        dfs(0, -1, 0)
        return ans
```

698. 1768.py

Path: LeetCode-main/solutions/1768. Merge Strings Alternately/1768.py

```
class Solution:
    def mergeAlternately(self, word1: str, word2: str) -> str:
        return ''.join(a + b for a, b in zip_longest(word1, word2, fillvalue=''))
```

699. 1770.py

Path: LeetCode-main/solutions/1770. Maximum Score from Performing Multiplication Operations/1770.py

```

class Solution:
    def maximumScore(self, nums: list[int], multipliers: list[int]) -> int:
        @functools.lru_cache(2000)
        def dp(s: int, i: int) -> int:
            """Returns the maximum score of nums[s..e] and multipliers[i]."""
            if i == len(multipliers):
                return 0

            # The number of nums picked on the start side is s.
            # The number of nums picked on the end side is i - s.
            # So, e = n - (i - s) - 1.
            e = len(nums) - (i - s) - 1
            pickStart = nums[s] * multipliers[i] + dp(s + 1, i + 1)
            pickEnd = nums[e] * multipliers[i] + dp(s, i + 1)
            return max(pickStart, pickEnd)

        return dp(0, 0)

```

700. 1776.py

Path: LeetCode-main/solutions/1776. Car Fleet II/1776.py

```

class Solution:
    def getCollisionTimes(self, cars: list[list[int]]) -> list[float]:
        ans = []
        stack = [] # (pos, speed, collisionTime)

        def getCollisionTime(
            car: tuple[int, int, int],
            pos: int, speed: int) -> float:
            return (car[0] - pos) / (speed - car[1])

        for pos, speed in reversed(cars):
            while stack and (
                speed <= stack[-1][1] or getCollisionTime(stack[-1],
                                                            pos, speed) >=
                    stack[-1][2]):
                stack.pop()
            if stack:
                collisionTime = getCollisionTime(stack[-1], pos, speed)
                stack.append((pos, speed, collisionTime))
                ans.append(collisionTime)
            else:
                stack.append((pos, speed, math.inf))
                ans.append(-1)

        return ans[::-1]

```

701. 1780.py

Path: LeetCode-main/solutions/1780. Check if Number is a Sum of Powers of Three/1780.py

```

class Solution:
    def checkPowersOfThree(self, n: int) -> bool:
        while n > 1:
            n, r = divmod(n, 3)
            if r == 2:
                return False
        return True

```

702. 1781.py

Path: LeetCode-main/solutions/1781. Sum of Beauty of All Substrings/1781.py

```
class Solution:
    def beautySum(self, s: str) -> int:
        ans = 0

        for i in range(len(s)):
            count = collections.Counter()
            for j in range(i, len(s)):
                count[s[j]] += 1
                ans += max(count.values()) - min(count.values())

        return ans
```

703. 1782.py

Path: LeetCode-main/solutions/1782. Count Pairs Of Nodes/1782.py

```

class Solution:
    def countPairs(
        self,
        n: int,
        edges: list[list[int]],
        queries: list[int],
    ) -> list[int]:
        ans = [0] * len(queries)

        # count[i] := the number of edges of node i
        count = [0] * (n + 1)

        # shared[i][j] := the number of edges incident to i or j, where i < j
        shared = [collections.Counter() for _ in range(n + 1)]

        for u, v in edges:
            count[u] += 1
            count[v] += 1
            shared[min(u, v)][max(u, v)] += 1

        sortedCount = sorted(count)

        for k, query in enumerate(queries):
            i = 1
            j = n
            while i < j:
                if sortedCount[i] + sortedCount[j] > query:
                    # sortedCount[i] + sortedCount[j] > query
                    # sortedCount[i + 1] + sortedCount[j] > query
                    # ...
                    # sortedCount[j - 1] + sortedCount[j] > query
                    # So, there are (j - 1) - i + 1 = j - i pairs > query
                    ans[k] += j - i
                    j -= 1
                else:
                    i += 1
            for i in range(1, n + 1):
                for j, sh in shared[i].items():
                    if count[i] + count[j] > query and count[i] + count[j] - sh <= query:
                        ans[k] -= 1

        return ans

```

704. 1784.py

Path: LeetCode-main/solutions/1784. Check if Binary String Has at Most One Segment of Ones/1784.py

```

class Solution:
    def checkOnesSegment(self, s: str) -> bool:
        return '01' not in s

```

705. 1786.py

Path: LeetCode-main/solutions/1786. Number of Restricted Paths From First to Last Node/1786.py

```

class Solution:
    def countRestrictedPaths(self, n: int, edges: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u - 1].append((v - 1, w))
            graph[v - 1].append((u - 1, w))

        return self._dijkstra(graph, 0, n - 1)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        dst: int,
    ) -> int:
        MOD = 10**9 + 7
        # ways[i] := the number of restricted path from i to n
        ways = [0] * len(graph)
        # dist[i] := the distance to the last node of i
        dist = [math.inf] * len(graph)

        ways[dst] = 1
        dist[dst] = 0
        minHeap = [(dist[dst], dst)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))
                if dist[v] < dist[u]:
                    ways[u] += ways[v]
                    ways[u] %= MOD

        return ways[src]

```

706. 1787.py

Path: LeetCode-main/solutions/1787. Make the XOR of All Segments Equal to Zero/1787.py

```

class Solution:
    def minChanges(self, nums: list[int], k: int) -> int:
        MAX = 1024
        n = len(nums)
        # counts[i] := the counter that maps at the i-th position
        counts = [collections.Counter() for _ in range(k)]
        # dp[i][j] := the minimum number of elements to change s.t. XOR(nums[i..k - 1]) is j
        dp = [[n] * MAX for _ in range(k)]

        for i, num in enumerate(nums):
            counts[i % k][num] += 1

        def countAt(i: int) -> int:
            return n // k + (1 if n % k > i else 0)

        # Initialize the DP array.
        for j in range(MAX):
            dp[k - 1][j] = countAt(k - 1) - counts[k - 1][j]

        for i in range(k - 2, -1, -1):
            # The worst-case scenario is changing all the i-th position numbers to a
            # non-existent value in the current bucket.
            changeAll = countAt(i) + min(dp[i + 1])
            for j in range(MAX):
                dp[i][j] = changeAll
                for num, freq in counts[i].items():
                    # the cost to change every number in the i-th position to `num`
                    cost = countAt(i) - freq
                    dp[i][j] = min(dp[i][j], dp[i + 1][j ^ num] + cost)

        return dp[0][0]

```

707. 1788.py

Path: LeetCode-main/solutions/1788. Maximize the Beauty of the Garden/1788.py

```

class Solution:
    def maximumBeauty(self, flowers: list[int]) -> int:
        ans = -math.inf
        prefix = 0
        flowerToPrefix = collections.defaultdict(int)

        for flower in flowers:
            if flower in flowerToPrefix:
                ans = max(ans, prefix - flowerToPrefix[flower] + flower * 2)
            prefix += max(0, flower)
            flowerToPrefix.setdefault(flower, prefix)

        return ans

```

708. 179.py

Path: LeetCode-main/solutions/179. Largest Number/179.py

```

class LargerStrKey(str):
    def __lt__(x: str, y: str) -> bool:
        return x + y > y + x

class Solution:
    def largestNumber(self, nums: list[int]) -> str:
        return ''.join(sorted(map(str, nums), key=LargerStrKey)).rstrip('0') or '0'

```

709. 1790.py

Path: LeetCode-main/solutions/1790. Check if One String Swap Can Make Strings Equal/1790.py

```

class Solution:
    # Similar to 859. Buddy Strings
    def areAlmostEqual(self, s1: str, s2: str) -> bool:
        diffIndices = [i for i, (a, b) in enumerate(zip(s1, s2))
                        if a != b]
        return not diffIndices or (len(diffIndices) == 2 and
                                    s1[diffIndices[0]] == s2[diffIndices[1]] and
                                    s1[diffIndices[1]] == s2[diffIndices[0]])

```

710. 1791.py

Path: LeetCode-main/solutions/1791. Find Center of Star Graph/1791.py

```

class Solution:
    def findCenter(self, edges: list[list[int]]) -> int:
        if edges[0][0] in edges[1]:
            return edges[0][0]
        return edges[0][1]

```

711. 1792.py

Path: LeetCode-main/solutions/1792. Maximum Average Pass Ratio/1792.py

```

class Solution:
    def maxAverageRatio(
        self,
        classes: list[list[int]],
        extraStudents: int,
    ) -> float:
        def extraPassRatio(pas: int, total: int) -> float:
            """Returns the extra pass ratio if a brilliant student joins."""
            return (pas + 1) / (total + 1) - pas / total

        maxHeap = [(-extraPassRatio(pas, total), pas, total)
                    for pas, total in classes]
        heapq.heapify(maxHeap)

        for _ in range(extraStudents):
            _, pas, total = heapq.heappop(maxHeap)
            heapq.heappush(
                maxHeap, (-extraPassRatio(pas + 1, total + 1), pas + 1, total + 1))

        return sum(pas / total for _, pas, total in maxHeap) / len(maxHeap)

```

712. 1793-2.py

Path: LeetCode-main/solutions/1793. Maximum Score of a Good Subarray/1793-2.py

```
class Solution:
    def maximumScore(self, nums: list[int], k: int) -> int:
        n = len(nums)
        ans = nums[k]
        mn = nums[k]
        i = k
        j = k

        # Greedily expand the window and decrease the minimum as slow as possible.
        while i > 0 or j < n - 1:
            if i == 0:
                j += 1
            elif j == n - 1:
                i -= 1
            elif nums[i - 1] < nums[j + 1]:
                j += 1
            else:
                i -= 1
            mn = min(mn, nums[i], nums[j])
            ans = max(ans, mn * (j - i + 1))

        return ans
```

713. 1793.py

Path: LeetCode-main/solutions/1793. Maximum Score of a Good Subarray/1793.py

```
class Solution:
    # Similar to 84. Largest Rectangle in Histogram
    def maximumScore(self, nums: list[int], k: int) -> int:
        ans = 0
        stack = []

        for i in range(len(nums) + 1):
            while stack and (i == len(nums) or nums[stack[-1]] > nums[i]):
                h = nums[stack.pop()]
                w = i - stack[-1] - 1 if stack else i
                if (not stack or stack[-1] + 1 <= k) and i - 1 >= k:
                    ans = max(ans, h * w)
                stack.append(i)

        return ans
```

714. 1794.py

Path: LeetCode-main/solutions/1794. Count Pairs of Equal Substrings With Minimum Difference/1794.py


```

class Solution:
    def countQuadruples(self, s1: str, s2: str) -> int:
        # To minimize j - a, the length of the substring should be 1. This is
        # because for substrings with a size greater than 1, a will decrease,
        # causing j - a to become larger.
        ans = 0
        diff = math.inf # diff := j - a
        firstJ = {}
        lastA = {}

        for j in range(len(s1) - 1, -1, -1):
            firstJ[s1[j]] = j

        for a in range(len(s2)):
            lastA[s2[a]] = a

        for c in string.ascii_lowercase:
            if c not in firstJ or c not in lastA:
                continue
            if firstJ[c] - lastA[c] < diff:
                diff = firstJ[c] - lastA[c]
                ans = 0
            if firstJ[c] - lastA[c] == diff:
                ans += 1

        return ans

```

715. 1796.py

Path: LeetCode-main/solutions/1796. Second Largest Digit in a String/1796.py

```

class Solution:
    def secondHighest(self, s: str) -> int:
        maxDigit = -1
        secondMaxDigit = -1

        for c in s:
            if c.isdigit():
                d = int(c)
                if d > maxDigit:
                    secondMaxDigit = maxDigit
                    maxDigit = d
                elif maxDigit > d > secondMaxDigit:
                    secondMaxDigit = d

        return secondMaxDigit

```

716. 1797.py

Path: LeetCode-main/solutions/1797. Design Authentication Manager/1797.py

```

from sortedcontainers import SortedSet

class AuthenticationManager:
    def __init__(self, timeToLive: int):
        self.timeToLive = timeToLive
        self.tokenIdToExpiryTime = {}
        self.times = SortedSet()

    def generate(self, tokenId: str, currentTime: int) -> None:
        self.tokenIdToExpiryTime[tokenId] = currentTime
        self.times.add(currentTime)

    def renew(self, tokenId: str, currentTime: int) -> None:
        if (tokenId not in self.tokenIdToExpiryTime or
            currentTime >= self.tokenIdToExpiryTime[tokenId] + self.timeToLive):
            return
        self.times.remove(self.tokenIdToExpiryTime[tokenId])
        self.tokenIdToExpiryTime[tokenId] = currentTime
        self.times.add(currentTime)

    def countUnexpiredTokens(self, currentTime: int) -> int:
        i = self.times.bisect_left(currentTime - self.timeToLive + 1)
        # Remove expired tokens.
        for _ in range(i):
            self.times.remove(self.times[0])
        return len(self.times)

```

717. 1798.py

Path: LeetCode-main/solutions/1798. Maximum Number of Consecutive Values You Can Make/1798.py

```

class Solution:
    def getMaximumConsecutive(self, coins: list[int]) -> int:
        ans = 1 # the next value we want to make

        for coin in sorted(coins):
            if coin > ans:
                return ans
            ans += coin

        return ans

```

718. 1799.py

Path: LeetCode-main/solutions/1799. Maximize Score After N Operations/1799.py

```
class Solution:
    def maxScore(self, nums: list[int]) -> int:
        n = len(nums) // 2

        @functools.lru_cache(None)
        def dp(k: int, mask: int) -> int:
            """
            Returns the maximum score you can receive after performing the k to n
            operations, where `mask` is the bitmask of the chosen numbers.
            """
            if k == n + 1:
                return 0

            res = 0

            for i, j in itertools.combinations(range(len(nums)), 2):
                chosenMask = 1 << i | 1 << j
                if (mask & chosenMask) == 0:
                    res = max(res, k * math.gcd(nums[i], nums[j])
                              + dp(k + 1, mask | chosenMask))

            return res

        return dp(1, 0)
```

719. 18.py

Path: LeetCode-main/solutions/18. 4Sum/18.py

```

class Solution:
    def fourSum(self, nums: list[int], target: int):
        ans = []

        def nSum(
            l: int, r: int, target: int, n: int, path: list[int],
            ans: list[list[int]]) -> None:
            """Finds n numbers that add up to the target in [l, r]."""
            if r - l + 1 < n or n < 2 or target < nums[l] * n or target > nums[r] * n:
                return
            if n == 2:
                while l < r:
                    summ = nums[l] + nums[r]
                    if summ == target:
                        ans.append(path + [nums[l], nums[r]])
                        l += 1
                        while nums[l] == nums[l - 1] and l < r:
                            l += 1
                    elif summ < target:
                        l += 1
                    else:
                        r -= 1
                return

            for i in range(l, r + 1):
                if i > l and nums[i] == nums[i - 1]:
                    continue

                nSum(i + 1, r, target - nums[i], n - 1, path + [nums[i]], ans)

        nums.sort()
        nSum(0, len(nums) - 1, target, 4, [], ans)
        return ans

```

720. 1800.py

Path: LeetCode-main/solutions/1800. Maximum Ascending Subarray Sum/1800.py

```

class Solution:
    def maxAscendingSum(self, nums: list[int]) -> int:
        ans = 0
        sum = nums[0]

        for i in range(1, len(nums)):
            if nums[i] > nums[i - 1]:
                sum += nums[i]
            else:
                ans = max(ans, sum)
                sum = nums[i]

        return max(ans, sum)

```

721. 1805-2.py

Path: LeetCode-main/solutions/1805. Number of Different Integers in a String/1805-2.py

```

class Solution:
    def numDifferentIntegers(self, word: str) -> int:
        return len(set(map(int, re.findall(r'\d+', word))))

```

722. 1805.py

Path: LeetCode-main/solutions/1805. Number of Different Integers in a String/1805.py

```
class Solution:
    def numDifferentIntegers(self, word: str) -> int:
        nums = set()
        curr = []

        for c in word:
            if c.isdigit():
                curr.append(c)
            elif curr:
                nums.add(''.join(self._removeLeadingZeros(curr)))
                curr = []

        if curr:
            nums.add(''.join(self._removeLeadingZeros(curr)))

        return len(nums)

    def _removeLeadingZeros(self, s: str) -> str:
        index = next((i for i, c in enumerate(s) if c != '0'), -1)
        return ['0'] if index == -1 else s[index:]
```

723. 1812.py

Path: LeetCode-main/solutions/1812. Determine Color of a Chessboard Square/1812.py

```
class Solution:
    def squareIsWhite(self, coordinates: str) -> bool:
        letter, digit = coordinates
        return ord(letter) % 2 != int(digit) % 2
```

724. 1813.py

Path: LeetCode-main/solutions/1813. Sentence Similarity III/1813.py

```
class Solution:
    def areSentencesSimilar(self, sentence1: str, sentence2: str) -> bool:
        if len(sentence1) == len(sentence2):
            return sentence1 == sentence2

        words1 = sentence1.split()
        words2 = sentence2.split()
        m, n = map(len, (words1, words2))
        if m > n:
            return self.areSentencesSimilar(sentence2, sentence1)

        i = 0 # words1's index
        while i < m and words1[i] == words2[i]:
            i += 1
        while i < m and words1[i] == words2[i + n - m]:
            i += 1

        return i == m
```

725. 1814.py

Path: LeetCode-main/solutions/1814. Count Nice Pairs in an Array/1814.py

```
class Solution:
    def countNicePairs(self, nums: list[int]) -> int:
        freqs = collections.Counter(num - int(str(num)[::-1]) for num in nums)
        return sum(freq * (freq - 1) // 2 for freq in freqs.values()) % 1000000007
```

726. 1815.py

Path: LeetCode-main/solutions/1815. Maximum Number of Groups Getting Fresh Donuts/1815.py

```
class Solution:
    def maxHappyGroups(self, batchSize: int, groups: list[int]) -> int:
        happy = 0
        freq = [0] * batchSize

        for g in groups:
            g %= batchSize
            if g == 0:
                happy += 1
            elif freq[batchSize - g]:
                freq[batchSize - g] -= 1
                happy += 1
            else:
                freq[g] += 1

        @functools.lru_cache(None)
        def dp(freq: int, remainder: int) -> int:
            """Returns the maximum number of partitions can be formed."""
            ans = 0
            if any(freq):
                for i, f in enumerate(freq):
                    if f:
                        ans = max(ans, dp(freq[:i] + (f - 1,) +
                                           freq[i + 1:], (remainder + i) % batchSize))

                if remainder == 0:
                    ans += 1
            return ans

        return happy + dp(tuple(freq), 0)
```

727. 1816.py

Path: LeetCode-main/solutions/1816. Truncate Sentence/1816.py

```
class Solution:
    def truncateSentence(self, s: str, k: int) -> str:
        return ' '.join(s.split()[:k])
```

728. 1817.py

Path: LeetCode-main/solutions/1817. Finding the Users Active Minutes/1817.py

```

class Solution:
    def findingUsersActiveMinutes(
        self,
        logs: list[list[int]],
        k: int,
    ) -> list[int]:
        idToTimes = collections.defaultdict(set)

        for id, time in logs:
            idToTimes[id].add(time)

        c = collections.Counter(len(times) for times in idToTimes.values())
        return [c[i] for i in range(1, k + 1)]

```

729. 1818.py

Path: LeetCode-main/solutions/1818. Minimum Absolute Sum Difference/1818.py

```

class Solution:
    def minAbsoluteSumDiff(self, nums1: list[int], nums2: list[int]) -> int:
        ans = math.inf
        diffs = [abs(a - b) for a, b in zip(nums1, nums2)]
        sumDiff = sum(diffs)

        nums1.sort()

        for num, diff in zip(nums2, diffs):
            i = bisect.bisect_left(nums1, num)
            if i > 0:
                ans = min(ans, sumDiff - diff + abs(num - nums1[i - 1]))
            if i < len(nums1):
                ans = min(ans, sumDiff - diff + abs(num - nums1[i]))

        return ans % 1_000_000_007

```

730. 1820.py

Path: LeetCode-main/solutions/1820. Maximum Number of Accepted Invitations/1820.py

```

class Solution:
    def maximumInvitations(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = 0
        mates = [-1] * n # mates[i] := the i-th girl's mate

        def canInvite(i: int, seen: list[bool]) -> bool:
            """Returns True if the i-th boy can make an invitation."""
            # The i-th boy asks each girl.
            for j in range(n):
                if not grid[i][j] or seen[j]:
                    continue
                seen[j] = True
                if mates[j] == -1 or canInvite(mates[j], seen):
                    mates[j] = i # Match the j-th girl with i-th boy.
                    return True
            return False

        for i in range(m):
            seen = [False] * n
            if canInvite(i, seen):
                ans += 1

        return ans

```

731. 1822.py

Path: LeetCode-main/solutions/1822. Sign of the Product of an Array/1822.py

```

class Solution:
    def arraySign(self, nums: list[int]) -> int:
        sign = 1

        for num in nums:
            if num == 0:
                return 0
            if num < 0:
                sign = -sign

        return sign

```

732. 1823-2.py

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823-2.py


```

class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        # e.g. n = 4, k = 2.
        # By using 0-indexed notation, we have the following circle:
        #
        # 0 -> 1 -> 2 -> 3 -> 0
        #      x
        #      0 -> 1 -> 2 -> 0
        #
        # After the first round, 1 is removed.
        # So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
        # Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
        # By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
        def f(n: int, k: int) -> int:
            if n == 1:
                return 0
            return (f(n - 1, k) + k) % n

        # Converts back to 1-indexed.
        return f(n, k) + 1

```

733. 1823-3.py

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823-3.py

```

class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        # e.g. n = 4, k = 2.
        # By using 0-indexed notation, we have the following circle:
        #
        # 0 -> 1 -> 2 -> 3 -> 0
        #      x
        #      0 -> 1 -> 2 -> 0
        #
        # After the first round, 1 is removed.
        # So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
        # Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
        # By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
        def f(n: int, k: int) -> int:
            ans = 0 # f(1, k)
            # Computes f(i, k) based on f(i - 1, k).
            for i in range(2, n + 1):
                ans = (ans + k) % i
            return ans

        # Converts back to 1-indexed.
        return f(n, k) + 1

```

734. 1823.py

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823.py

```
class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        # True if i-th friend is left
        friends = [False] * n

        friendCount = n
        fp = 0 # friends' index

        while friendCount > 1:
            for _ in range(k):
                while friends[fp % n]: # The friend is not there.
                    fp += 1 # Point to the next one.
                fp += 1
            friends[(fp - 1) % n] = True
            friendCount -= 1

        fp = 0
        while friends[fp]:
            fp += 1

        return fp + 1
```

735. 1824.py

Path: LeetCode-main/solutions/1824. Minimum Sideway Jumps/1824.py

```
class Solution:
    def minSideJumps(self, obstacles: list[int]) -> int:
        INF = 1e6
        # dp[i] := the minimum jump to reach the i-th lane
        dp = [INF, 1, 0, 1]

        for obstacle in obstacles:
            print(dp)
            if obstacle > 0:
                dp[obstacle] = INF
            for i in range(1, 4): # the current
                if i != obstacle:
                    for j in range(1, 4): # the previous
                        dp[i] = min(dp[i], dp[j] + (0 if i == j else 1))

        return min(dp)
```

736. 1826.py

Path: LeetCode-main/solutions/1826. Faulty Sensor/1826.py

```
class Solution:
    def badSensor(self, sensor1: list[int], sensor2: list[int]) -> int:
        # A -> B, so B is defect
        def canReplace(A, B):
            i = 0 # A's index
            j = 0 # B's index
            droppedValue = -1

            while i < len(A):
                if A[i] == B[j]:
                    i += 1
                    j += 1
                else:
                    droppedValue = A[i]
                    i += 1

            return j == len(B) - 1 and B[-1] != droppedValue

        oneDefect = canReplace(sensor2, sensor1)
        twoDefect = canReplace(sensor1, sensor2)
        if oneDefect and twoDefect:
            return -1
        if not oneDefect and not twoDefect:
            return -1
        return 1 if oneDefect else 2
```

737. 1827.py

Path: LeetCode-main/solutions/1827. Minimum Operations to Make the Array Increasing/1827.py

```
class Solution:
    def minOperations(self, nums: list[int]) -> int:
        ans = 0
        last = 0

        for num in nums:
            ans += max(0, last - num + 1)
            last = max(num, last + 1)

        return ans
```

738. 1828.py

Path: LeetCode-main/solutions/1828. Queries on Number of Points Inside a Circle/1828.py

```
class Solution:
    def countPoints(
        self,
        points: list[list[int]],
        queries: list[list[int]],
    ) -> list[int]:
        ans = []

        for xj, yj, rj in queries:
            count = 0
            for xi, yi in points:
                if (xi - xj)**2 + (yi - yj)**2 <= rj**2:
                    count += 1
            ans.append(count)

        return ans
```

739. 1829.py

Path: LeetCode-main/solutions/1829. Maximum XOR for Each Query/1829.py

```
class Solution:
    def getMaximumXor(self, nums: list[int], maximumBit: int) -> list[int]:
        mx = (1 << maximumBit) - 1
        ans = []
        xors = 0

        for num in nums:
            xors ^= num
            ans.append(xors ^ mx)

        return ans[::-1]
```

740. 1830.py

Path: LeetCode-main/solutions/1830. Minimum Number of Operations to Make String Sorted/1830.py

```

class Solution:
    def makeStringSorted(self, s: str) -> int:
        MOD = 1_000_000_007
        ans = 0
        count = [0] * 26

        @functools.lru_cache(None)
        def fact(i: int) -> int:
            return 1 if i <= 1 else i * fact(i - 1) % MOD

        @functools.lru_cache(None)
        def inv(i: int) -> int:
            return pow(i, MOD - 2, MOD)

        for i, c in enumerate(reversed(s)):
            order = ord(c) - ord('a')
            count[order] += 1
            # count[:order] := s[i] can be any character smaller than c
            # fact(i) := s[i + 1..n - 1] can be any sequence of characters
            perm = sum(count[:order]) * fact(i)
            for j in range(26):
                perm = perm * inv(fact(count[j])) % MOD
            ans = (ans + perm) % MOD

        return ans

```

741. 1832-2.py

Path: LeetCode-main/solutions/1832. Check if the Sentence Is Pangram/1832-2.py

```

class Solution:
    def checkIfPangram(self, sentence: str) -> bool:
        seen = 0

        for c in sentence:
            seen |= 1 << ord(c) - ord('a')

        return seen == (1 << 26) - 1

```

742. 1832.py

Path: LeetCode-main/solutions/1832. Check if the Sentence Is Pangram/1832.py

```

class Solution:
    def checkIfPangram(self, sentence: str) -> bool:
        return len(set(sentence)) == 26

```

743. 1833.py

Path: LeetCode-main/solutions/1833. Maximum Ice Cream Bars/1833.py

```
class Solution:
    def maxIceCream(self, costs: list[int], coins: int) -> int:
        for i, cost in enumerate(sorted(costs)):
            if coins >= cost:
                coins -= cost
            else:
                return i

        return len(costs)
```

744. 1834.py

Path: LeetCode-main/solutions/1834. Single-Threaded CPU/1834.py

```
class Solution:
    def getOrder(self, tasks: list[list[int]]) -> list[int]:
        n = len(tasks)
        A = [[*task, i] for i, task in enumerate(tasks)]
        ans = []
        minHeap = []
        i = 0 # tasks' index
        time = 0 # the current time

        A.sort()

        while i < n or minHeap:
            if not minHeap:
                time = max(time, A[i][0])
            while i < n and time >= A[i][0]:
                heapq.heappush(minHeap, (A[i][1], A[i][2]))
                i += 1
            procTime, index = heapq.heappop(minHeap)
            time += procTime
            ans.append(index)

        return ans
```

745. 1835.py

Path: LeetCode-main/solutions/1835. Find XOR Sum of All Pairs Bitwise AND/1835.py

```
class Solution:
    def getXORSum(self, arr1: list[int], arr2: list[int]) -> int:
        return functools.reduce(
            operator.xor, arr1) & functools.reduce(
            operator.xor, arr2)
```

746. 1836.py

Path: LeetCode-main/solutions/1836. Remove Duplicates From an Unsorted Linked List/1836.py

```

class Solution:
    def deleteDuplicatesUnsorted(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        count = collections.Counter()

        curr = head
        while curr:
            count[curr.val] += 1
            curr = curr.next

        curr = dummy

        while curr:
            while curr.next and curr.next.val in count and count[curr.next.val] > 1:
                curr.next = curr.next.next
            curr = curr.next

        return dummy.next

```

747. 1837.py

Path: LeetCode-main/solutions/1837. Sum of Digits in Base K/1837.py

```

class Solution:
    def sumBase(self, n: int, k: int) -> int:
        ans = 0

        while n > 0:
            ans += n % k
            n //= k

        return ans

```

748. 1838.py

Path: LeetCode-main/solutions/1838. Frequency of the Most Frequent Element/1838.py

```

class Solution:
    def maxFrequency(self, nums: list[int], k: int) -> int:
        ans = 0
        summ = 0

        nums.sort()

        l = 0
        for r, num in enumerate(nums):
            summ += num
            while summ + k < num * (r - l + 1):
                summ -= nums[l]
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

749. 1839.py

Path: LeetCode-main/solutions/1839. Longest Substring Of All Vowels in Order/1839.py

```

class Solution:
    def longestBeautifulSubstring(self, word: str) -> int:
        ans = 0
        count = 1

        l = 0
        for r in range(1, len(word)):
            curr = word[r]
            prev = word[r - 1]
            if curr >= prev:
                if curr > prev:
                    count += 1
                if count == 5:
                    ans = max(ans, r - l + 1)
            else:
                count = 1
                l = r

        return ans

```

750. 1840.py

Path: LeetCode-main/solutions/1840. Maximum Building Height/1840.py

```

class Solution:
    def maxBuilding(self, n: int, restrictions: list[list[int]]) -> int:
        A = sorted(restrictions + [[1, 0]] + [[n, n - 1]])

        for i in range(len(A)):
            dist = A[i][0] - A[i - 1][0]
            A[i][1] = min(A[i][1], A[i - 1][1] + dist)

        for i in reversed(range(len(A) - 1)):
            dist = A[i + 1][0] - A[i][0]
            A[i][1] = min(A[i][1], A[i + 1][1] + dist)

        ans = 0

        for (l, hL), (r, hR) in zip(A, A[1:]):
            ans = max(ans, max(hL, hR) + (r - l - abs(hL - hR)) // 2)

        return ans

```

751. 1842.py

Path: LeetCode-main/solutions/1842. Next Palindrome Using Same Digits/1842.py


```

class Solution:
    def nextPalindrome(self, num: str) -> str:
        def nextPermutation(nums: list[int]) -> bool:
            n = len(nums)

            # From the back to the front, find the first num < nums[i + 1].
            i = n - 2
            while i >= 0:
                if nums[i] < nums[i + 1]:
                    break
                i -= 1

            if i < 0:
                return False

            # From the back to the front, find the first num > nums[i] and swap it
            # with nums[i].
            for j in range(n - 1, i, -1):
                if nums[j] > nums[i]:
                    nums[i], nums[j] = nums[j], nums[i]
                    break

            def reverse(nums, l, r):
                while l < r:
                    nums[l], nums[r] = nums[r], nums[l]
                    l += 1
                    r -= 1

            # Reverse nums[i + 1..n - 1].
            reverse(nums, i + 1, len(nums) - 1)
            return True

        n = len(num)
        arr = [int(num[i]) for i in range(len(num) // 2)]
        if not nextPermutation(arr):
            return ''

        s = ''.join([chr(ord('0') + a) for a in arr])
        if n % 2 == 1:
            return s + num[n // 2] + s[::-1]
        return s + s[::-1]

```

752. 1844.py

Path: LeetCode-main/solutions/1844. Replace All Digits with Characters/1844.py

```

class Solution:
    def replaceDigits(self, s: str) -> str:
        return ''.join(
            c if i % 2 == 0 else chr(ord(s[i - 1]) + int(c))
            for i, c in enumerate(s)
        )

```

753. 1845.py

Path: LeetCode-main/solutions/1845. Seat Reservation Manager/1845.py

```
class SeatManager:
    def __init__(self, n: int):
        self.minHeap = [i + 1 for i in range(n)]

    def reserve(self) -> int:
        return heapq.heappop(self.minHeap)

    def unreserve(self, seatNumber: int) -> None:
        heapq.heappush(self.minHeap, seatNumber)
```

754. 1846.py

Path: LeetCode-main/solutions/1846. Maximum Element After Decreasing and Rearranging/1846.py

```
class Solution:
    def maximumElementAfterDecrementingAndRearranging(
        self, arr: list[int],
    ) -> int:
        arr.sort()
        arr[0] = 1

        for i in range(1, len(arr)):
            arr[i] = min(arr[i], arr[i - 1] + 1)

        return arr[-1]
```

755. 1847.py

Path: LeetCode-main/solutions/1847. Closest Room/1847.py

```

from sortedcontainers import SortedList

class Solution:
    def closestRoom(
        self,
        rooms: list[list[int]],
        queries: list[list[int]],
    ) -> list[int]:
        ans = [0] * len(queries)
        qs = [[*q, i] for i, q in enumerate(queries)]
        roomIds = SortedList()

        rooms.sort(key=lambda x: -x[1])
        qs.sort(key=lambda x: -x[1])

        def searchClosestRoomId(roomIds: SortedList, preferred: int):
            if not roomIds:
                return -1

            candIds = []
            i = roomIds.bisect_right(preferred)
            if i > 0:
                candIds.append(roomIds[i - 1])
            if i < len(roomIds):
                candIds.append(roomIds[i])
            return min(candIds, key=lambda x: abs(x - preferred))

        i = 0 # rooms' index
        for preferred, minSize, index in qs:
            while i < len(rooms) and rooms[i][1] >= minSize:
                roomIds.add(rooms[i][0])
                i += 1
            ans[index] = searchClosestRoomId(roomIds, preferred)

        return ans

```

756. 1848.py

Path: LeetCode-main/solutions/1848. Minimum Distance to the Target Element/1848.py

```

class Solution:
    def getMinDistance(self, nums: list[int], target: int, start: int) -> int:
        ans = math.inf

        for i, num in enumerate(nums):
            if num == target:
                ans = min(ans, abs(i - start))

        return ans

```

757. 1849.py

Path: LeetCode-main/solutions/1849. Splitting a String Into Descending Consecutive Values/1849.py

```
class Solution:
    def splitString(self, s: str) -> bool:
        def isValid(s: str, start: int, prev: int, segment: int) -> bool:
            if start == len(s) and segment > 1:
                return True

            curr = 0
            for i in range(start, len(s)):
                curr = curr * 10 + int(s[i])
                if curr > 999999999:
                    return False
                if (prev == -1 or curr == prev - 1) and isValid(s, i + 1, curr, segment + 1):
                    return True

            return False

        return isValid(s, 0, -1, 0)
```

758. 1850.py

Path: LeetCode-main/solutions/1850. Minimum Adjacent Swaps to Reach the Kth Smallest Number/1850.py

```

class Solution:
    def getMinSwaps(self, num: str, k: int) -> int:
        original = [int(c) for c in num]
        permutated = original.copy()

        for _ in range(k):
            self._nextPermutation(permutated)

        return self._countSteps(original, permutated)

    def _nextPermutation(self, nums: list[int]):
        n = len(nums)

        # From the back to the front, find the first num < nums[i + 1].
        i = n - 2
        while i >= 0:
            if nums[i] < nums[i + 1]:
                break
            i -= 1

        # From the back to the front, find the first num > nums[i] and swap it with nums[i].
        if i >= 0:
            for j in range(n - 1, i, -1):
                if nums[j] > nums[i]:
                    nums[i], nums[j] = nums[j], nums[i]
                    break

        def reverse(nums, l, r):
            while l < r:
                nums[l], nums[r] = nums[r], nums[l]
                l += 1
                r -= 1

        # Reverse nums[i + 1..n - 1].
        reverse(nums, i + 1, len(nums) - 1)

    def _countSteps(self, A: list[int], B: list[int]) -> int:
        count = 0

        j = 0
        for i in range(len(A)):
            j = i
            while A[i] != B[j]:
                j += 1
            while i < j:
                B[j], B[j - 1] = B[j - 1], B[j]
                j -= 1
            count += 1

        return count

```

759. 1852.py

Path: LeetCode-main/solutions/1852. Distinct Numbers in Each Subarray/1852.py

```

class Solution:
    def distinctNumbers(self, nums: list[int], k: int) -> list[int]:
        ans = []
        count = collections.Counter()
        distinct = 0

        for i, num in enumerate(nums):
            count[num] += 1
            if count[num] == 1:
                distinct += 1
            if i >= k:
                count[nums[i - k]] -= 1
                if count[nums[i - k]] == 0:
                    distinct -= 1
            if i >= k - 1:
                ans.append(distinct)

        return ans

```

760. 1854.py

Path: LeetCode-main/solutions/1854. Maximum Population Year/1854.py

```

class Solution:
    def maximumPopulation(self, logs: list[list[int]]) -> int:
        MIN_YEAR = 1950
        MAX_YEAR = 2050
        ans = 0
        maxPopulation = 0
        runningPopulation = 0
        # population[i] := the population of year i
        population = [0] * (MAX_YEAR + 1)

        for birth, death in logs:
            population[birth] += 1
            population[death] -= 1

        for year in range(MIN_YEAR, MAX_YEAR + 1):
            runningPopulation += population[year]
            if runningPopulation > maxPopulation:
                maxPopulation = runningPopulation
                ans = year

        return ans

```

761. 1855-2.py

Path: LeetCode-main/solutions/1855. Maximum Distance Between a Pair of Values/1855-2.py

```

class Solution:
    def maxDistance(self, nums1: list[int], nums2: list[int]) -> int:
        i = 0
        j = 0

        while i < len(nums1) and j < len(nums2):
            if nums1[i] > nums2[j]:
                i += 1
            j += 1

        return 0 if i == j else j - i - 1

```

762. 1855.py

Path: LeetCode-main/solutions/1855. Maximum Distance Between a Pair of Values/1855.py

```

class Solution:
    def maxDistance(self, nums1: list[int], nums2: list[int]) -> int:
        ans = 0
        i = 0
        j = 0

        while i < len(nums1) and j < len(nums2):
            if nums1[i] > nums2[j]:
                i += 1
            else:
                ans = max(ans, j - i)
                j += 1

        return ans

```

763. 1856.py

Path: LeetCode-main/solutions/1856. Maximum Subarray Min-Product/1856.py

```

class Solution:
    def maxSumMinProduct(self, nums: list[int]) -> int:
        ans = 0
        stack = []
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(len(nums) + 1):
            while stack and (i == len(nums) or nums[stack[-1]] > nums[i]):
                minVal = nums[stack.pop()]
                summ = prefix[i] - prefix[stack[-1] + 1] if stack else prefix[i]
                ans = max(ans, minVal * summ)
            stack.append(i)

        return ans % 1_000_000_007

```

764. 1857.py

Path: LeetCode-main/solutions/1857. Largest Color Value in a Directed Graph/1857.py

```

class Solution:
    def largestPathValue(self, colors: str, edges: list[list[int]]) -> int:
        n = len(colors)
        ans = 0
        processed = 0
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n
        q = collections.deque()
        count = [[0] * 26 for _ in range(n)]

        # Build the graph.
        for u, v in edges:
            graph[u].append(v)
            inDegrees[v] += 1

        # Vpology
        for i, degree in enumerate(inDegrees):
            if degree == 0:
                q.append(i)

        while q:
            u = q.popleft()
            processed += 1
            count[u][ord(colors[u]) - ord('a')] += 1
            ans = max(ans, count[u][ord(colors[u]) - ord('a')])
            for v in graph[u]:
                for i in range(26):
                    count[v][i] = max(count[v][i], count[u][i])
                inDegrees[v] -= 1
                if inDegrees[v] == 0:
                    q.append(v)

        return ans if processed == n else -1

```

765. 1858.py

Path: LeetCode-main/solutions/1858. Longest Word With All Prefixes/1858.py


```

class Solution:
    def __init__(self):
        self.root = {}

    def longestWord(self, words: list[str]) -> str:
        ans = ''

        for word in words:
            self.insert(word)

        for word in words:
            if not self.allPrefixed(word):
                continue
            if len(ans) < len(word) or (len(ans) == len(word) and ans > word):
                ans = word

        return ans

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node:
                node[c] = {}
            node = node[c]
        node['isWord'] = True

    def allPrefixed(self, word: str) -> bool:
        node = self.root
        for c in word:
            node = node[c]
            if 'isWord' not in node:
                return False
        return True

```

766. 1859.py

Path: LeetCode-main/solutions/1859. Sorting the Sentence/1859.py

```

class Solution:
    def sortSentence(self, s: str) -> str:
        return ' '.join([word[:-1]
                        for word in sorted(s.split(), key=lambda x: x[-1])])

```

767. 186.py

Path: LeetCode-main/solutions/186. Reverse Words in a String II/186.py

```

class Solution:
    def reverseWords(self, s: list[str]) -> None:
        def reverse(l: int, r: int) -> None:
            while l < r:
                s[l], s[r] = s[r], s[l]
                l += 1
                r -= 1

        def reverseWords(n: int) -> None:
            i = 0
            j = 0

            while i < n:
                while i < j or (i < n and s[i] == ' '): # Skip the spaces.
                    i += 1
                while j < i or (j < n and s[j] != ' '): # Skip the spaces.
                    j += 1
                reverse(i, j - 1) # Reverse the word.

            reverse(0, len(s) - 1) # Reverse the whole string.
            reverseWords(len(s)) # Reverse each word.

```

768. 1860.py

Path: LeetCode-main/solutions/1860. Incremental Memory Leak/1860.py

```

class Solution:
    def memLeak(self, memory1: int, memory2: int) -> list[int]:
        i = 1

        while memory1 >= i or memory2 >= i:
            if memory1 >= memory2:
                memory1 -= i
            else:
                memory2 -= i
            i += 1

        return [i, memory1, memory2]

```

769. 1861.py

Path: LeetCode-main/solutions/1861. Rotating the Box/1861.py

```

class Solution:
    def rotateTheBox(self, box: list[list[str]]) -> list[list[str]]:
        m = len(box)
        n = len(box[0])
        rotated = [['.' * m for _ in range(n)]]

        for i in range(m):
            k = n - 1
            for j in reversed(range(n)):
                if box[i][j] != '.':
                    if box[i][j] == '*':
                        k = j
                    rotated[k][m - i - 1] = box[i][j]
                    k -= 1

        return [''.join(row) for row in rotated]

```

770. 1862.py

Path: LeetCode-main/solutions/1862. Sum of Floored Pairs/1862.py

```
class Solution:
    def sumOfFlooredPairs(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        MAX = max(nums)
        ans = 0
        count = [0] * (MAX + 1)

        for num in nums:
            count[num] += 1

        for i in range(1, MAX + 1):
            count[i] += count[i - 1]

        for i in range(1, MAX + 1):
            if count[i] > count[i - 1]:
                summ = 0
                j = 1
                while i * j <= MAX:
                    lo = i * j - 1
                    hi = i * (j + 1) - 1
                    summ += (count[min(hi, MAX)] - count[lo]) * j
                    j += 1
                ans += summ * (count[i] - count[i - 1])
            ans %= MOD

        return ans
```

771. 1863-2.py

Path: LeetCode-main/solutions/1863. Sum of All Subset XOR Totals/1863-2.py

```
class Solution:
    def subsetXORSum(self, nums: list[int]) -> int:
        return functools.reduce(operator.or_, nums) << len(nums) - 1
```

772. 1863.py

Path: LeetCode-main/solutions/1863. Sum of All Subset XOR Totals/1863.py

```
class Solution:
    def subsetXORSum(self, nums: list[int]) -> int:
        def dfs(i: int, xors: int) -> int:
            if i == len(nums):
                return xors

            x = dfs(i + 1, xors)
            y = dfs(i + 1, nums[i] ^ xors)
            return x + y

        return dfs(0, 0)
```

773. 1864.py

Path: LeetCode-main/solutions/1864. Minimum Number of Swaps to Make the Binary String Alternating/1864.py

```
class Solution:
    def minSwaps(self, s: str) -> int:
        ones = s.count('1')
        zeros = len(s) - ones
        if abs(ones - zeros) > 1:
            return -1

        def countSwaps(curr: str) -> int:
            swaps = 0
            for c in s:
                if c != curr:
                    swaps += 1
                curr = chr(ord(curr) ^ 1)
            return swaps // 2

        if ones > zeros:
            return countSwaps('1')
        if zeros > ones:
            return countSwaps('0')
        return min(countSwaps('1'), countSwaps('0'))
```

774. 1865.py

Path: LeetCode-main/solutions/1865. Finding Pairs With a Certain Sum/1865.py

```
class FindSumPairs:
    def __init__(self, nums1: list[int], nums2: list[int]):
        self.nums1 = nums1
        self.nums2 = nums2
        self.count2 = collections.Counter(nums2)

    def add(self, index: int, val: int) -> None:
        self.count2[self.nums2[index]] -= 1
        self.nums2[index] += val
        self.count2[self.nums2[index]] += 1

    def count(self, tot: int) -> int:
        ans = 0
        for num in self.nums1:
            ans += self.count2[tot - num]
        return ans
```

775. 1866.py

Path: LeetCode-main/solutions/1866. Number of Ways to Rearrange Sticks With K Sticks Visible/1866.py

```

class Solution:
    @functools.lru_cache(None)
    def rearrangeSticks(self, n: int, k: int) -> int:
        if n == k:
            return 1
        if k == 0:
            return 0
        return (self.rearrangeSticks(n - 1, k - 1) +
                self.rearrangeSticks(n - 1, k) * (n - 1)) % self.MOD

MOD = 1_000_000_007

```

776. 1868.py

Path: LeetCode-main/solutions/1868. Product of Two Run-Length Encoded Arrays/1868.py

```

class Solution:
    def findRLEArray(self, encoded1: list[list[int]],
                     encoded2: list[list[int]]) -> list[list[int]]:

        ans = []
        i = 0 # encoded1's index
        j = 0 # encoded2's index

        while i < len(encoded1) and j < len(encoded2):
            mult = encoded1[i][0] * encoded2[j][0]
            minFreq = min(encoded1[i][1], encoded2[j][1])
            if ans and mult == ans[-1][0]:
                ans[-1][1] += minFreq
            else:
                ans.append([mult, minFreq])
            encoded1[i][1] -= minFreq
            encoded2[j][1] -= minFreq
            if encoded1[i][1] == 0:
                i += 1
            if encoded2[j][1] == 0:
                j += 1

        return ans

```

777. 1869.py

Path: LeetCode-main/solutions/1869. Longer Contiguous Segments of Ones than Zeros/1869.py

```
class Solution:
    def checkZeroOnes(self, s: str) -> bool:
        longestOnes = 0
        longestZeros = 0
        currentOnes = 0
        currentZeros = 0

        for c in s:
            if c == '0':
                currentOnes = 0
                currentZeros += 1
                longestZeros = max(longestZeros, currentZeros)
            else:
                currentZeros = 0
                currentOnes += 1
                longestOnes = max(longestOnes, currentOnes)

        return longestOnes > longestZeros
```

778. 187.py

Path: LeetCode-main/solutions/187. Repeated DNA Sequences/187.py

```
class Solution:
    def findRepeatedDnaSequences(self, s: str) -> list[str]:
        ans = set()
        seen = set()

        for i in range(len(s) - 9):
            seq = s[i:i + 10]
            if seq in seen:
                ans.add(seq)
            seen.add(seq)

        return list(ans)
```

779. 1870.py

Path: LeetCode-main/solutions/1870. Minimum Speed to Arrive on Time/1870.py

```

class Solution:
    def minSpeedOnTime(self, dist: list[int], hour: float) -> int:
        ans = -1
        l = 1
        r = int(1e7)

        def time(speed: int) -> float:
            summ = 0
            for i in range(len(dist) - 1):
                summ += math.ceil(dist[i] / speed)
            return summ + dist[-1] / speed

        while l <= r:
            m = (l + r) // 2
            if time(m) > hour:
                l = m + 1
            else:
                ans = m
                r = m - 1

        return ans

```

780. 1871.py

Path: LeetCode-main/solutions/1871. Jump Game VII/1871.py

```

class Solution:
    def canReach(self, s: str, minJump: int, maxJump: int) -> bool:
        count = 0
        dp = [True] + [False] * (len(s) - 1)

        for i in range(minJump, len(s)):
            count += dp[i - minJump]
            if i - maxJump > 0:
                count -= dp[i - maxJump - 1]
            dp[i] = count and s[i] == '0'

        return dp[-1]

```

781. 1872.py

Path: LeetCode-main/solutions/1872. Stone Game VIII/1872.py

```

class Solution:
    def stoneGameVIII(self, stones: list[int]) -> int:
        n = len(stones)
        prefix = list(itertools.accumulate(stones))
        # dp[i] := the maximum score difference the current player can get when the
        # game starts at i, i.e. stones[0..i] are merged into the value prefix[i]
        dp = [-math.inf] * n

        # Must take all when there're only two stones left.
        dp[n - 2] = prefix[-1]

        for i in reversed(range(n - 2)):
            dp[i] = max(dp[i + 1], prefix[i + 1] - dp[i + 1])

        return dp[0]

```

782. 1874.py

Path: LeetCode-main/solutions/1874. Minimize Product Sum of Two Arrays/1874.py

```
class Solution:
    def minProductSum(self, nums1: list[int], nums2: list[int]) -> int:
        return sum([a * b for a, b in zip(sorted(nums1), sorted(nums2, reverse=True))])
```

783. 1876.py

Path: LeetCode-main/solutions/1876. Substrings of Size Three with Distinct Characters/1876.py

```
class Solution:
    def countGoodSubstrings(self, s: str) -> int:
        ans = 0

        for a, b, c in zip(s, s[1:], s[2:]):
            if a == b or a == c or b == c:
                continue
            ans += 1

        return ans
```

784. 1877.py

Path: LeetCode-main/solutions/1877. Minimize Maximum Pair Sum in Array/1877.py

```
class Solution:
    def minPairSum(self, nums: list[int]) -> int:
        nums.sort()
        return max(nums[i] + nums[len(nums) - 1 - i] for i in range(len(nums) // 2))
```

785. 1878.py

Path: LeetCode-main/solutions/1878. Get Biggest Three Rhombus Sums in a Grid/1878.py


```

from sortedcontainers import SortedSet

class Solution:
    def getBiggestThree(self, grid: list[list[int]]) -> list[int]:
        m = len(grid)
        n = len(grid[0])
        sums = SortedSet()

        for i in range(m):
            for j in range(n):
                sz = 0
                while i + sz < m and i - sz >= 0 and j + 2 * sz < n:
                    summ = grid[i][j] if sz == 0 else self._getSum(grid, i, j, sz)
                    sums.add(summ)
                    if len(sums) > 3:
                        sums.pop(0)
                    sz += 1

        return sums

    def _getSum(self, grid: list[list[int]], i: int, j: int, sz: int) -> int:
        """
        Returns the sum of the rhombus, where the top grid is (i, j) and the edge
        size is `sz`.
        """
        x = i
        y = j
        summ = 0

        # Go left down.
        for _ in range(sz):
            x -= 1
            y += 1
            summ += grid[x][y]

        # Go right down.
        for _ in range(sz):
            x += 1
            y += 1
            summ += grid[x][y]

        # Go right up.
        for _ in range(sz):
            x += 1
            y -= 1
            summ += grid[x][y]

        # Go left up.
        for _ in range(sz):
            x -= 1
            y -= 1
            summ += grid[x][y]

        return summ

```

786. 1879.py

Path: LeetCode-main/solutions/1879. Minimum XOR Sum of Two Arrays/1879.py

```

class Solution:
    def minimumXORSum(self, nums1: list[int], nums2: list[int]) -> int:
        @functools.lru_cache(None)
        def dp(mask: int) -> int:
            i = mask.bit_count()
            if i == len(nums1):
                return 0
            return min((nums1[i] ^ nums2[j]) + dp(mask | 1 << j)
                        for j in range(len(nums2)) if not mask >> j & 1)
        return dp(0)

```

787. 188.py

Path: LeetCode-main/solutions/188. Best Time to Buy and Sell Stock IV/188.py

```

class Solution:
    def maxProfit(self, k: int, prices: list[int]) -> int:
        if k >= len(prices) // 2:
            sell = 0
            hold = -math.inf

            for price in prices:
                sell = max(sell, hold + price)
                hold = max(hold, sell - price)

            return sell

        sell = [0] * (k + 1)
        hold = [-math.inf] * (k + 1)

        for price in prices:
            for i in range(k, 0, -1):
                sell[i] = max(sell[i], hold[i] + price)
                hold[i] = max(hold[i], sell[i - 1] - price)

        return sell[k]

```

788. 1880.py

Path: LeetCode-main/solutions/1880. Check if Word Equals Summation of Two Words/1880.py

```

class Solution:
    def isSumEqual(
        self,
        firstWord: str,
        secondWord: str,
        targetWord: str,
    ) -> bool:
        first = self._getNumber(firstWord)
        second = self._getNumber(secondWord)
        target = self._getNumber(targetWord)
        return first + second == target

    def _getNumber(self, word: str) -> int:
        num = 0
        for c in word:
            num = num * 10 + ord(c) - ord('a')
        return num

```

789. 1881.py

Path: LeetCode-main/solutions/1881. Maximum Value after Insertion/1881.py

```
class Solution:
    def maxValue(self, n: str, x: int) -> str:
        isNegative = n[0] == '-'

        for i, c in enumerate(n):
            if not isNegative and int(c) < x or isNegative and int(c) > x:
                return n[:i] + str(x) + n[i:]

        return n + str(x)
```

790. 1882.py

Path: LeetCode-main/solutions/1882. Process Tasks Using Servers/1882.py

```
class Solution:
    def assignTasks(self, servers: list[int], tasks: list[int]) -> list[int]:
        ans = []
        free = [] # (weight, index, freeTime)
        used = [] # (freeTime, weight, index)

        for i, weight in enumerate(servers):
            heapq.heappush(free, (weight, i, 0))

        for i, executionTime in enumerate(tasks): # i := the current time
            # Poll all servers that'll be free at time i.
            while used and used[0][0] <= i:
                curr = heapq.heappop(used)
                heapq.heappush(free, (curr[1], curr[2], curr[0]))
            if free:
                curr = heapq.heappop(free)
                ans.append(curr[1])
                heapq.heappush(used, (i + executionTime, curr[0], curr[1]))
            else:
                curr = heapq.heappop(used)
                ans.append(curr[2])
                heapq.heappush(used, (curr[0] + executionTime, curr[1], curr[2]))

        return ans
```

791. 1883.py

Path: LeetCode-main/solutions/1883. Minimum Skips to Arrive at Meeting On Time/1883.py

```

class Solution:
    def minSkips(self, dist: list[int], speed: int, hoursBefore: int) -> int:
        INF = 10**7
        EPS = 1e-9
        n = len(dist)
        # dp[i][j] := the minimum time, where i is the number of roads we traversed
        # so far and j is the number of skips we did
        dp = [[INF] * (n + 1) for _ in range(n + 1)]
        dp[0][0] = 0

        for i, d in enumerate(dist, 1):
            dp[i][0] = math.ceil(dp[i - 1][0] + d / speed - EPS)
            for j in range(1, i + 1):
                dp[i][j] = min(dp[i - 1][j - 1] + d / speed,
                               math.ceil(dp[i - 1][j] + d / speed - EPS))

        for j, time in enumerate(dp[-1]):
            if time <= hoursBefore:
                return j

        return -1

```

792. 1885.py

Path: LeetCode-main/solutions/1885. Count Pairs in Two Arrays/1885.py

```

class Solution:
    def countPairs(self, nums1: list[int], nums2: list[int]) -> int:
        ans = 0
        arr = sorted([x - y for x, y in zip(nums1, nums2)])

        for i, a in enumerate(arr):
            index = bisect_left(arr, -a + 1)
            ans += len(arr) - max(i + 1, index)

        return ans

```

793. 1886.py

Path: LeetCode-main/solutions/1886. Determine Whether Matrix Can Be Obtained By Rotation/1886.py

```

class Solution:
    def findRotation(self, mat: list[list[int]], target: list[list[int]]) -> bool:
        for _ in range(4):
            if mat == target:
                return True
            mat = [list(x) for x in zip(*mat[::-1])]
        return False

```

794. 1887.py

Path: LeetCode-main/solutions/1887. Reduction Operations to Make the Array Elements Equal/1887.py

```
class Solution:
    def reductionOperations(self, nums: list[int]) -> int:
        ans = 0

        nums.sort()

        for i in range(len(nums) - 1, 0, -1):
            if nums[i] != nums[i - 1]:
                ans += len(nums) - i

        return ans
```

795. 1888.py

Path: LeetCode-main/solutions/1888. Minimum Number of Flips to Make the Binary String Alternating/1888.py

```
class Solution:
    def minFlips(self, s: str) -> int:
        n = len(s)
        # count[0][0] := the number of '0' in the even indices
        # count[0][1] := the number of '0' in the odd indices
        # count[1][0] := the number of '1' in the even indices
        # count[1][1] := the number of '1' in the odd indices
        count = [[0] * 2 for _ in range(2)]

        for i, c in enumerate(s):
            count[int(c)][i % 2] += 1

        # min(make all 0s in the even indices + make all 1s in the odd indices,
        #      make all 1s in the even indices + make all 0s in the odd indices)
        ans = min(count[1][0] + count[0][1], count[0][0] + count[1][1])

        for i, c in enumerate(s):
            count[int(c)][i % 2] -= 1
            count[int(c)][(n + i) % 2] += 1
            ans = min(ans, count[1][0] + count[0][1], count[0][0] + count[1][1])

        return ans
```

796. 1889.py

Path: LeetCode-main/solutions/1889. Minimum Space Wasted From Packaging/1889.py

```

class Solution:
    def minWastedSpace(self, packages: list[int], boxes: list[list[int]]) -> int:
        ans = math.inf

        packages.sort()

        for box in boxes:
            box.sort()
            if box[-1] < packages[-1]:
                continue
            accu = 0
            i = 0
            for b in box:
                j = bisect.bisect(packages, b, i)
                accu += b * (j - i)
                i = j
            ans = min(ans, accu)

        return -1 if ans == math.inf else (ans - sum(packages)) % 1_000_000_007

```

797. 189.py

Path: LeetCode-main/solutions/189. Rotate Array/189.py

```

class Solution:
    def rotate(self, nums: list[int], k: int) -> None:
        k %= len(nums)
        self.reverse(nums, 0, len(nums) - 1)
        self.reverse(nums, 0, k - 1)
        self.reverse(nums, k, len(nums) - 1)

    def reverse(self, nums: list[int], l: int, r: int) -> None:
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l += 1
            r -= 1

```

798. 1891.py

Path: LeetCode-main/solutions/1891. Cutting Ribbons/1891.py

```

class Solution:
    def maxLength(self, ribbons: list[int], k: int) -> int:
        def isCutPossible(length: int) -> bool:
            count = 0
            for ribbon in ribbons:
                count += ribbon // length
            return count >= k

        l = 1
        r = sum(ribbons) // k + 1

        while l < r:
            m = (l + r) // 2
            if not isCutPossible(m):
                r = m
            else:
                l = m + 1

        return l - 1

```

799. 1893-2.py

Path: LeetCode-main/solutions/1893. Check if All the Integers in a Range Are Covered/1893-2.py

```

class Solution:
    def isCovered(self, ranges: list[list[int]], left: int, right: int) -> bool:
        seen = [0] * 52

        for l, r in ranges:
            seen[l] += 1
            seen[r + 1] -= 1

        for i in range(1, 52):
            seen[i] += seen[i - 1]

        return all(seen[i] for i in range(left, right + 1))

```

800. 1893.py

Path: LeetCode-main/solutions/1893. Check if All the Integers in a Range Are Covered/1893.py

```

class Solution:
    def isCovered(self, ranges: list[list[int]], left: int, right: int) -> bool:
        return all(any(l <= i <= r for l, r in ranges) for i in range(
            left, right + 1))

```

801. 1894.py

Path: LeetCode-main/solutions/1894. Find the Student that Will Replace the Chalk/1894.py

```

class Solution:
    def chalkReplacer(self, chalk: list[int], k: int) -> int:
        k %= sum(chalk)
        if k == 0:
            return 0

        for i, c in enumerate(chalk):
            k -= c
            if k < 0:
                return i

```

802. 1895.py

Path: LeetCode-main/solutions/1895. Largest Magic Square/1895.py

```

class Solution:
    def largestMagicSquare(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        # prefixRow[i][j] := the sum of the first j numbers in the i-th row
        prefixRow = [[0] * (n + 1) for _ in range(m)]
        # prefixCol[i][j] := the sum of the first j numbers in the i-th column
        prefixCol = [[0] * (m + 1) for _ in range(n)]

        for i in range(m):
            for j in range(n):
                prefixRow[i][j + 1] = prefixRow[i][j] + grid[i][j]
                prefixCol[j][i + 1] = prefixCol[j][i] + grid[i][j]

        def isMagicSquare(i: int, j: int, k: int) -> bool:
            """Returns True if grid[i..i + k][j..j + k) is a magic square."""
            diag, antiDiag = 0, 0
            for d in range(k):
                diag += grid[i + d][j + d]
                antiDiag += grid[i + d][j + k - 1 - d]
            if diag != antiDiag:
                return False
            for d in range(k):
                if self._getSum(prefixRow, i + d, j, j + k - 1) != diag:
                    return False
                if self._getSum(prefixCol, j + d, i, i + k - 1) != diag:
                    return False
            return True

        def containsMagicSquare(k: int) -> bool:
            """Returns True if the grid contains any magic square of size k x k."""
            for i in range(m - k + 1):
                for j in range(n - k + 1):
                    if isMagicSquare(i, j, k):
                        return True
            return False

        for k in range(min(m, n), 1, -1):
            if containsMagicSquare(k):
                return k

        return 1

    def _getSum(self, prefix: list[list[int]], i: int, l: int, r: int) -> int:
        """Returns sum(grid[i][l..r]) or sum(grid[l..r][i])."""
        return prefix[i][r + 1] - prefix[i][l]

```

803. 1896.py

Path: LeetCode-main/solutions/1896. Minimum Cost to Change the Final Value of Expression/1896.py

```
class Solution:
    def minOperationsToFlip(self, expression: str) -> int:
        stack = [] # [(the expression, the cost to toggle the expression)]

        for e in expression:
            if e in '&|':
                # These aren't expressions, so the cost is meaningless.
                stack.append((e, 0))
                continue
            if e == ')':
                lastPair = stack.pop()
                stack.pop() # Pop '('
            else: # e == '0' or e == '1'
                # Store the '0' or '1'. The cost to change their values is just 1,
                # whether it's changing '0' to '1' or '1' to '0'.
                lastPair = (e, 1)
            if stack and stack[-1][0] in '&|':
                op = stack.pop()[0]
                a, costA = stack.pop()
                b, costB = lastPair
                # Determine the cost to toggle op(a, b).
                if op == '&':
                    if a == '0' and b == '0':
                        # Change '&' to '|' and a|b to '1'.
                        lastPair = ('0', 1 + min(costA, costB))
                    elif a == '0' and b == '1':
                        # Change '&' to '|'.
                        lastPair = ('0', 1)
                    elif a == '1' and b == '0':
                        # Change '&' to '|'.
                        lastPair = ('0', 1)
                    else: # a == '1' and b == '1'
                        # Change a|b to '0'.
                        lastPair = ('1', min(costA, costB))
                else: # op == '|'
                    if a == '0' and b == '0':
                        # Change a|b to '1'.
                        lastPair = ('0', min(costA, costB))
                    elif a == '0' and b == '1':
                        # Change '|' to '&'.
                        lastPair = ('1', 1)
                    elif a == '1' and b == '0':
                        # Change '|' to '&'.
                        lastPair = ('1', 1)
                    else: # a == '1' and b == '1'
                        # Change '|' to '&' and a|b to '0'.
                        lastPair = ('1', 1 + min(costA, costB))
                stack.append(lastPair)

        return stack[-1][1]
```

804. 1897.py

Path: LeetCode-main/solutions/1897. Redistribute Characters to Make All Strings Equal/1897.py

```
class Solution:
    def makeEqual(self, words: list[str]) -> bool:
        return all(c % len(words) == 0
                   for c in collections.Counter(''.join(words)).values())
```

805. 1898.py

Path: LeetCode-main/solutions/1898. Maximum Number of Removable Characters/1898.py

```
class Solution:
    def maximumRemovals(self, s: str, p: str, removable: list[int]) -> int:
        l = 0
        r = len(removable) + 1

        def remove(k: int) -> str:
            removed = [c for c in s]
            for i in range(k):
                removed[removable[i]] = '*'
            return ''.join(removed)

        def isSubsequence(p: str, s: str) -> bool:
            i = 0
            for j, c in enumerate(s):
                if p[i] == s[j]:
                    i += 1
                if i == len(p):
                    return True
            return False

        while l < r:
            m = (l + r) // 2
            removed = remove(m)
            if isSubsequence(p, removed):
                l = m + 1
            else:
                r = m

        return l - 1
```

806. 1899.py

Path: LeetCode-main/solutions/1899. Merge Triplets to Form Target Triplet/1899.py

```
class Solution:
    def mergeTriplets(self, triplets: list[list[int]], target: list[int]) -> bool:
        merged = [0] * len(target)

        for triplet in triplets:
            if all(a <= b for a, b in zip(triplet, target)):
                for i in range(3):
                    merged[i] = max(merged[i], triplet[i])

        return merged == target
```

807. 19.py

Path: LeetCode-main/solutions/19. Remove Nth Node From End of List/19.py

```
class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        slow = head
        fast = head

        for _ in range(n):
            fast = fast.next
        if not fast:
            return head.next

        while fast.next:
            slow = slow.next
            fast = fast.next
        slow.next = slow.next.next

        return head
```

808. 190.py

Path: LeetCode-main/solutions/190. Reverse Bits/190.py

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if n >> i & 1:
                ans |= 1 << 31 - i

        return ans
```

809. 1900.py

Path: LeetCode-main/solutions/1900. The Earliest and Latest Rounds Where Players Compete/1900.py

```

class Solution:
    def earliestAndLatest(self, n: int,
                          firstPlayer: int, secondPlayer: int) -> list[int]:
        @functools.lru_cache(None)
        def dp(l: int, r: int, k: int) -> list[int]:
            """
            Returns the (earliest, latest) pair, the first player is the l-th player
            from the front, the second player is the r-th player from the end, and
            there're k people.
            """
            if l == r:
                return [1, 1]
            if l > r:
                return dp(r, l, k)

            a = math.inf
            b = -math.inf

            # Enumerate all the possible positions.
            for i in range(1, l + 1):
                for j in range(l - i + 1, r - i + 1):
                    if not l + r - k // 2 <= i + j <= (k + 1) // 2:
                        continue
                    x, y = dp(i, j, (k + 1) // 2)
                    a = min(a, x + 1)
                    b = max(b, y + 1)

            return [a, b]

        return dp(firstPlayer, n - secondPlayer + 1, n)

```

810. 1901.py

Path: LeetCode-main/solutions/1901. Find a Peak Element II/1901.py

```

class Solution:
    def findPeakGrid(self, mat: list[list[int]]) -> list[int]:
        l = 0
        r = len(mat) - 1

        while l < r:
            m = (l + r) // 2
            if max(mat[m]) >= max(mat[m + 1]):
                r = m
            else:
                l = m + 1

        return [l, mat[l].index(max(mat[l]))]

```

811. 1903.py

Path: LeetCode-main/solutions/1903. Largest Odd Number in String/1903.py

```

class Solution:
    def largestOddNumber(self, num: str) -> str:
        for i, n in reversed(list(enumerate(num))):
            if int(n) % 2 == 1:
                return num[:i + 1]
        return ''

```

812. 1904.py

Path: LeetCode-main/solutions/1904. The Number of Full Rounds You Have Played/1904.py

```
class Solution:
    def numberOfRounds(self, loginTime: str, logoutTime: str) -> int:
        start = self._getMinutes(loginTime)
        finish = self._getMinutes(logoutTime)
        if start > finish:
            finish += 60 * 24

        return max(0, finish // 15 - (start + 14) // 15)

    def _getMinutes(self, time: str) -> int:
        h, m = map(int, time.split(':'))
        return 60 * h + m
```

813. 1905.py

Path: LeetCode-main/solutions/1905. Count Sub Islands/1905.py

```
class Solution:
    def countSubIslands(
        self,
        grid1: list[list[int]],
        grid2: list[list[int]],
    ) -> int:
        m = len(grid2)
        n = len(grid2[0])

        def dfs(i: int, j: int) -> int:
            if i < 0 or i == m or j < 0 or j == n:
                return 1
            if grid2[i][j] != 1:
                return 1

            grid2[i][j] = 2 # Mark 2 as visited.

            return (dfs(i + 1, j) & dfs(i - 1, j) &
                    dfs(i, j + 1) & dfs(i, j - 1) & grid1[i][j])

        ans = 0

        for i in range(m):
            for j in range(n):
                if grid2[i][j] == 1:
                    ans += dfs(i, j)

        return ans
```

814. 1906.py

Path: LeetCode-main/solutions/1906. Minimum Absolute Difference Queries/1906.py

```

class Solution:
    def minDifference(
        self,
        nums: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        numToIndices = [[] for _ in range(101)]

        for i, num in enumerate(nums):
            numToIndices[num].append(i)

        if len(numToIndices[nums[0]]) == len(nums):
            return [-1] * len(queries)

        ans = []

        for l, r in queries:
            prevNum = -1
            minDiff = 101
            for num in range(1, 101):
                indices = numToIndices[num]
                i = bisect_left(indices, l)
                if i == len(indices) or indices[i] > r:
                    continue
                if prevNum != -1:
                    minDiff = min(minDiff, num - prevNum)
                prevNum = num
            ans.append(-1 if minDiff == 101 else minDiff)

        return ans

```

815. 1908.py

Path: LeetCode-main/solutions/1908. Game of Nim/1908.py

```

class Solution:
    def nimGame(self, piles: list[int]) -> bool:
        return functools.reduce(operator.xor, piles) > 0

```

816. 1909.py

Path: LeetCode-main/solutions/1909. Remove One Element to Make the Array Strictly Increasing/1909.py

```

class Solution:
    def canBeIncreasing(self, nums: list[int]) -> bool:
        removed = False

        for i in range(1, len(nums)):
            if nums[i - 1] >= nums[i]:
                if removed:
                    return False
                removed = True # Remove nums[i - 1].
            if i > 1 and nums[i - 2] >= nums[i]:
                nums[i] = nums[i - 1] # Remove nums[i] instead.

        return True

```

817. 191-2.py

Path: LeetCode-main/solutions/191. Number of 1 Bits/191-2.py

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        return n.bit_count()
```

818. 191.py

Path: LeetCode-main/solutions/191. Number of 1 Bits/191.py

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if (n >> i) & 1:
                ans += 1

        return ans
```

819. 1910.py

Path: LeetCode-main/solutions/1910. Remove All Occurrences of a Substring/1910.py

```
class Solution:
    def removeOccurrences(self, s: str, part: str) -> str:
        n = len(s)
        k = len(part)

        t = [' '] * n
        j = 0 # t's index

        for i, c in enumerate(s):
            t[j] = c
            j += 1
            if j >= k and ''.join(t[j - k:j]) == part:
                j -= k

        return ''.join(t[:j])
```

820. 1911.py

Path: LeetCode-main/solutions/1911. Maximum Alternating Subsequence Sum/1911.py

```
class Solution:
    def maxAlternatingSum(self, nums: list[int]) -> int:
        even = 0 # the maximum alternating sum ending in an even index
        odd = 0 # the maximum alternating sum ending in an odd index

        for num in nums:
            even = max(even, odd + num)
            odd = even - num

        return even
```

821. 1912.py

Path: LeetCode-main/solutions/1912. Design Movie Rental System/1912.py

```
from sortedcontainers import SortedList

class MovieRentingSystem:
    def __init__(self, n: int, entries: list[list[int]]):
        self.unrented = collections.defaultdict(
            SortedList) # {movie: (price, shop)}
        self.shopAndMovieToPrice = {} # {(shop, movie): price}
        self.rented = SortedList() # (price, shop, movie)
        for shop, movie, price in entries:
            self.unrented[movie].add((price, shop))
            self.shopAndMovieToPrice[(shop, movie)] = price

    def search(self, movie: int) -> list[int]:
        return [shop for _, shop in self.unrented[movie][:5]]

    def rent(self, shop: int, movie: int) -> None:
        price = self.shopAndMovieToPrice[(shop, movie)]
        self.unrented[movie].remove((price, shop))
        self.rented.add((price, shop, movie))

    def drop(self, shop: int, movie: int) -> None:
        price = self.shopAndMovieToPrice[(shop, movie)]
        self.unrented[movie].add((price, shop))
        self.rented.remove((price, shop, movie))

    def report(self) -> list[list[int]]:
        return [[shop, movie] for _, shop, movie in self.rented[:5]]
```

822. 1913.py

Path: LeetCode-main/solutions/1913. Maximum Product Difference Between Two Pairs/1913.py

```
class Solution:
    def maxProductDifference(self, nums: list[int]) -> int:
        max1 = -math.inf
        max2 = -math.inf
        min1 = math.inf
        min2 = math.inf

        for num in nums:
            if num > max1:
                max2 = max1
                max1 = num
            elif num > max2:
                max2 = num
            if num < min1:
                min2 = min1
                min1 = num
            elif num < min2:
                min2 = num

        return max1 * max2 - min1 * min2
```


823. 1914.py

Path: LeetCode-main/solutions/1914. Cyclically Rotating a Grid/1914.py

```
class Solution:
    def rotateGrid(self, grid: list[list[int]], k: int) -> list[list[int]]:
        m = len(grid)
        n = len(grid[0])
        t = 0 # the top
        l = 0 # the left
        b = m - 1 # the bottom
        r = n - 1 # the right

        while t < b and l < r:
            elementInThisLayer = 2 * (b - t + 1) + 2 * (r - l + 1) - 4
            netRotations = k % elementInThisLayer
            for _ in range(netRotations):
                topLeft = grid[t][l]
                for j in range(l, r):
                    grid[t][j] = grid[t][j + 1]
                for i in range(t, b):
                    grid[i][r] = grid[i + 1][r]
                for j in range(r, l, -1):
                    grid[b][j] = grid[b][j - 1]
                for i in range(b, t, -1):
                    grid[i][l] = grid[i - 1][l]
                grid[t + 1][l] = topLeft
                t += 1
                l += 1
                b -= 1
                r -= 1

        return grid
```

824. 1915.py

Path: LeetCode-main/solutions/1915. Number of Wonderful Substrings/1915.py

```
class Solution:
    def wonderfulSubstrings(self, word: str) -> int:
        ans = 0
        prefix = 0 # the binary prefix
        count = [0] * 1024 # the binary prefix count
        count[0] = 1 # the empty string ""

        for c in word:
            prefix ^= 1 << ord(c) - ord('a')
            # All the letters occur even number of times.
            ans += count[prefix]
            # `c` occurs odd number of times.
            ans += sum(count[prefix ^ 1 << i] for i in range(10))
            count[prefix] += 1

        return ans
```

825. 1916.py

Path: LeetCode-main/solutions/1916. Count Ways to Build Rooms in an Ant Colony/1916.py

```

class Solution:
    def waysToBuildRooms(self, prevRoom: list[int]) -> int:
        MOD = 1_000_000_007
        graph = collections.defaultdict(list)

        for i, prev in enumerate(prevRoom):
            graph[prev].append(i)

        def dfs(node: int) -> tuple[int, int]:
            if not graph[node]:
                return 1, 1

            ans = 1
            l = 0

            for child in graph[node]:
                temp, r = dfs(child)
                ans = (ans * temp * math.comb(l + r, r)) % MOD
                l += r

            return ans, l + 1

        return dfs(0)[0]

```

826. 1918.py

Path: LeetCode-main/solutions/1918. Kth Smallest Subarray Sum/1918.py

```

class Solution:
    def kthSmallestSubarraySum(self, nums: list[int], k: int) -> int:
        def numSubarrayLessThan(m: int) -> int:
            res = 0
            summ = 0
            l = 0
            for r, num in enumerate(nums):
                summ += num
                while summ > m:
                    summ -= nums[l]
                    l += 1
                res += r - l + 1
            return res

        return bisect.bisect_left(range(sum(nums)), k, key=numSubarrayLessThan)

```

827. 1920.py

Path: LeetCode-main/solutions/1920. Build Array from Permutation/1920.py

```

class Solution:
    def buildArray(self, nums: list[int]) -> list[int]:
        n = len(nums)

        for i, num in enumerate(nums):
            nums[i] += n * (nums[num] % n)

        for i in range(n):
            nums[i] //= n

        return nums

```

828. 1921.py

Path: LeetCode-main/solutions/1921. Eliminate Maximum Number of Monsters/1921.py

```
class Solution:
    def eliminateMaximum(self, dist: list[int], speed: list[int]) -> int:
        for i, arrivalTime in enumerate(
            sorted([(d - 1) // s for d, s in zip(dist, speed)])):
            if i > arrivalTime:
                return i
        return len(dist)
```

829. 1922.py

Path: LeetCode-main/solutions/1922. Count Good Numbers/1922.py

```
class Solution:
    def countGoodNumbers(self, n: int) -> int:
        MOD = 1_000_000_007

        def modPow(x: int, n: int) -> int:
            if n == 0:
                return 1
            if n % 2 == 1:
                return x * modPow(x, n - 1) % MOD
            return modPow(x * x % MOD, n // 2)

        return modPow(4 * 5, n // 2) * (1 if n % 2 == 0 else 5) % MOD
```

830. 1923.py

Path: LeetCode-main/solutions/1923. Longest Common Subpath/1923.py

```

class Solution:
    def __init__(self):
        self.BASE = 165_131
        self.HASH = 8_417_508_174_513

    def longestCommonSubpath(self, n: int, paths: list[list[int]]) -> int:
        l = 0
        r = len(paths[0])

        while l < r:
            m = l + (r - l + 1) // 2
            if self._checkCommonSubpath(paths, m):
                l = m
            else:
                r = m - 1

        return l

    def _checkCommonSubpath(self, paths: list[list[int]], m: int) -> bool:
        """
        Returns True if there's a common subpath of length m for all the paths.
        """
        # Calculate the hash values for subpaths of length m for every path.
        hashSets = [self._rabinKarp(path, m) for path in paths]

        # Check if there is a common subpath of length m.
        for subpathHash in hashSets[0]:
            if all(subpathHash in hashSet for hashSet in hashSets):
                return True

        return False

    def _rabinKarp(self, path: list[int], m: int) -> set[int]:
        """Returns the hash values for subpaths of length m in the path."""
        hashes = set()
        maxPower = 1
        hash = 0

        for i, num in enumerate(path):
            hash = (hash * self.BASE + num) % self.HASH
            if i >= m:
                hash = (hash - path[i - m] * maxPower %
                       self.HASH + self.HASH) % self.HASH
            else:
                maxPower = maxPower * self.BASE % self.HASH
            if i >= m - 1:
                hashes.add(hash)

        return hashes

```

831. 1924.py

Path: LeetCode-main/solutions/1924. Erect the Fence II/1924.py

```

from dataclasses import dataclass

```

```

@dataclass(frozen=True)
class Point:
    x: float
    y: float

```

```

@dataclass(frozen=True)
class Disk:
    center: Point
    radius: float

class Solution:
    def outerTrees(self, trees: list[list[int]]) -> list[float]:
        points = [Point(x, y) for x, y in trees]
        disk = self._welzl(points, 0, [])
        return [disk.center.x, disk.center.y, disk.radius]

    def _welzl(
        self,
        points: list[Point],
        i: int,
        planePoints: list[Point],
    ) -> Disk:
        """Returns the smallest disk that encloses points[i..n).

        https://en.wikipedia.org/wiki/Smallest-disk_problem#Welzl's_algorithm
        """
        if i == len(points) or len(planePoints) == 3:
            return self._trivial(planePoints)
        disk = self._welzl(points, i + 1, planePoints)
        if self._inside(disk, points[i]):
            return disk
        return self._welzl(points, i + 1, planePoints + [points[i]])

    def _trivial(self, planePoints: list[Point]) -> Disk:
        """Returns the smallest disk that encloses `planePoints`."""
        if len(planePoints) == 0:
            return Disk(Point(0, 0), 0)
        if len(planePoints) == 1:
            return Disk(Point(planePoints[0].x, planePoints[0].y), 0)
        if len(planePoints) == 2:
            return self._getDisk(planePoints[0], planePoints[1])

        disk01 = self._getDisk(planePoints[0], planePoints[1])
        if self._inside(disk01, planePoints[2]):
            return disk01

        disk02 = self._getDisk(planePoints[0], planePoints[2])
        if self._inside(disk02, planePoints[1]):
            return disk02

        disk12 = self._getDisk(planePoints[1], planePoints[2])
        if self._inside(disk12, planePoints[0]):
            return disk12

        return self._getDiskFromThree(
            planePoints[0],
            planePoints[1],
            planePoints[2])

    def _getDisk(self, A: Point, B: Point) -> Disk:
        """Returns the smallest disk that encloses the points A and B."""
        x = (A.x + B.x) / 2
        y = (A.y + B.y) / 2
        return Disk(Point(x, y), self._distance(A, B) / 2)

    def _getDiskFromThree(self, A: Point, B: Point, C: Point) -> Disk:
        """Returns the smallest disk that encloses the points A, B, and C."""
        # Calculate midpoints.
        mAB = Point((A.x + B.x) / 2, (A.y + B.y) / 2)

```

```

mBC = Point((B.x + C.x) / 2, (B.y + C.y) / 2)

# Calculate the slopes and the perpendicular slopes.
slopeAB = math.inf if B.x == A.x else (B.y - A.y) / (B.x - A.x)
slopeBC = math.inf if C.x == B.x else (C.y - B.y) / (C.x - B.x)
perpSlopeAB = math.inf if slopeAB == 0 else -1 / slopeAB
perpSlopeBC = math.inf if slopeBC == 0 else -1 / slopeBC

# Calculate the center.
x = (perpSlopeBC * mBC.x - perpSlopeAB * mAB.x +
     mAB.y - mBC.y) / (perpSlopeBC - perpSlopeAB)
y = perpSlopeAB * (x - mAB.x) + mAB.y
center = Point(x, y)
return Disk(center, self._distance(center, A))

def _inside(self, disk: Disk, point: Point) -> bool:
    """Returns True if the point is inside the disk."""
    return disk.radius > 0 and self._distance(disk.center, point) <= disk.radius

def _distance(self, A: Point, B: Point) -> float:
    dx = A.x - B.x
    dy = A.y - B.y
    return math.sqrt(dx**2 + dy**2)

```

832. 1925.py

Path: LeetCode-main/solutions/1925. Count Square Sum Triples/1925.py

```

class Solution:
    def countTriples(self, n: int) -> int:
        ans = 0
        squared = set()

        for i in range(1, n + 1):
            squared.add(i * i)

        for a in squared:
            for b in squared:
                if a + b in squared:
                    ans += 1

        return ans

```

833. 1926.py

Path: LeetCode-main/solutions/1926. Nearest Exit from Entrance in Maze/1926.py

```

class Solution:
    def nearestExit(self, maze: list[list[str]], entrance: list[int]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(maze)
        n = len(maze[0])
        q = collections.deque([(entrance[0], entrance[1])])
        seen = {(entrance[0], entrance[1])}

        step = 1
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if (x, y) in seen or maze[x][y] == '+':
                        continue
                    if x == 0 or x == m - 1 or y == 0 or y == n - 1:
                        return step
                    q.append((x, y))
                    seen.add((x, y))
                step += 1

        return -1

```

834. 1927.py

Path: LeetCode-main/solutions/1927. Sum Game/1927.py

```

class Solution:
    def sumGame(self, num: str) -> bool:
        n = len(num)
        ans = 0.0

        def getExpectation(c: str) -> float:
            return 4.5 if c == '?' else int(c)

        for i in range(n // 2):
            ans += getExpectation(num[i])

        for i in range(n // 2, n):
            ans -= getExpectation(num[i])

        return ans != 0.0

```

835. 1928.py

Path: LeetCode-main/solutions/1928. Minimum Cost to Reach Destination in Time/1928.py

```

class Solution:
    def minCost(
        self,
        maxTime: int,
        edges: list[list[int]],
        passingFees: list[int],
    ) -> int:
        n = len(passingFees)
        graph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        return self._dijkstra(graph, 0, n - 1, maxTime, passingFees)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        dst: int,
        maxTime: int,
        passingFees: list[int],
    ) -> int:
        # cost[i] := the minimum cost to reach the i-th city
        cost = [math.inf] * len(graph)
        # dist[i] := the minimum time to reach the i-th city
        dist = [maxTime + 1] * len(graph)

        cost[src] = passingFees[src]
        dist[src] = 0
        minHeap = [(cost[src], dist[src], src)] # (cost[u], dist[u], u)

        while minHeap:
            currCost, d, u = heapq.heappop(minHeap)
            if u == dst:
                return cost[dst]
            if d > dist[u] and currCost > cost[u]:
                continue
            for v, w in graph[u]:
                if d + w > maxTime:
                    continue
                # Go from u -> v.
                if currCost + passingFees[v] < cost[v]:
                    cost[v] = currCost + passingFees[v]
                    dist[v] = d + w
                    heapq.heappush(minHeap, (cost[v], dist[v], v))
                elif d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (currCost + passingFees[v], dist[v], v))

        return -1

```

836. 1929.py

Path: LeetCode-main/solutions/1929. Concatenation of Array/1929.py

```

class Solution:
    def getConcatenation(self, nums: list[int]) -> list[int]:
        return nums * 2

```


837. 1930.py

Path: LeetCode-main/solutions/1930. Unique Length-3 Palindromic Subsequences/1930.py

```
class Solution:
    def countPalindromicSubsequence(self, s: str) -> int:
        ans = 0
        first = [len(s)] * 26
        last = [-1] * 26

        for i, c in enumerate(s):
            index = ord(c) - ord('a')
            first[index] = min(first[index], i)
            last[index] = i

        for f, l in zip(first, last):
            if f < l:
                ans += len(set(s[f + 1:l]))

        return ans
```

838. 1931.py

Path: LeetCode-main/solutions/1931. Painting a Grid With Three Different Colors/1931.py

```
class Solution:
    def colorTheGrid(self, m: int, n: int) -> int:
        def getColor(mask: int, r: int) -> int:
            return mask >> r * 2 & 3

        def setColor(mask: int, r: int, color: int) -> int:
            return mask | color << r * 2

        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(r: int, c: int, prevColMask: int, currColMask: int) -> int:
            if c == n:
                return 1
            if r == m:
                return dp(0, c + 1, currColMask, 0)

            ans = 0

            # 1 := red, 2 := green, 3 := blue
            for color in range(1, 4):
                if getColor(prevColMask, r) == color:
                    continue
                if r > 0 and getColor(currColMask, r - 1) == color:
                    continue
                ans += dp(r + 1, c, prevColMask, setColor(currColMask, r, color))
                ans %= MOD

            return ans

        return dp(0, 0, 0, 0)
```

839. 1932.py

Path: LeetCode-main/solutions/1932. Merge BSTs to Create Single BST/1932.py

```
class Solution:
    def canMerge(self, trees: list[TreeNode]) -> TreeNode | None:
        valToNode = {} # {val: node}
        count = collections.Counter() # {val: freq}

        for tree in trees:
            valToNode[tree.val] = tree
            count[tree.val] += 1
            if tree.left:
                count[tree.left.val] += 1
            if tree.right:
                count[tree.right.val] += 1

        def isValidBST(tree: TreeNode | None, minNode: TreeNode | None,
                      maxNode: TreeNode | None) -> bool:
            if not tree:
                return True
            if minNode and tree.val <= minNode.val:
                return False
            if maxNode and tree.val >= maxNode.val:
                return False
            if not tree.left and not tree.right and tree.val in valToNode:
                val = tree.val
                tree.left = valToNode[val].left
                tree.right = valToNode[val].right
                del valToNode[val]

            return isValidBST(
                tree.left, minNode, tree) and isValidBST(
                    tree.right, tree, maxNode)

        for tree in trees:
            if count[tree.val] == 1:
                if isValidBST(tree, None, None) and len(valToNode) <= 1:
                    return tree
            return None

        return None
```

840. 1933.py

Path: LeetCode-main/solutions/1933. Check if String Is Decomposable Into Value-Equal Substrings/1933.py

```
class Solution:
    def isDecomposable(self, s: str) -> bool:
        twos = 0

        for _, group in itertools.groupby(s):
            groupLength = len(list(group))
            if groupLength % 3 == 1:
                return False
            if groupLength % 3 == 2:
                twos += 1
            if twos > 1:
                return False

        return twos == 1
```

841. 1935.py

Path: LeetCode-main/solutions/1935. Maximum Number of Words You Can Type/1935.py

```
class Solution:
    def canBeTypedWords(self, text: str, brokenLetters: str) -> int:
        ans = 0
        broken = set(brokenLetters)

        for word in text.split():
            ans += all(c not in broken for c in word)

        return ans
```

842. 1936.py

Path: LeetCode-main/solutions/1936. Add Minimum Number of Rungs/1936.py

```
class Solution:
    def addRungs(self, rungs: list[int], dist: int) -> int:
        ans = 0
        prev = 0

        for rung in rungs:
            ans += (rung - prev - 1) // dist
            prev = rung

        return ans
```

843. 1937.py

Path: LeetCode-main/solutions/1937. Maximum Number of Points with Cost/1937.py

```
class Solution:
    def maxPoints(self, points: list[list[int]]) -> int:
        n = len(points[0])
        # dp[j] := the maximum number of points you can have if points[i][j] is the
        # most recent cell you picked
        dp = [0] * n

        for row in points:
            leftToRight = [0] * n
            runningMax = 0
            for j in range(n):
                runningMax = max(runningMax - 1, dp[j])
                leftToRight[j] = runningMax

            rightToLeft = [0] * n
            runningMax = 0
            for j in range(n - 1, -1, -1):
                runningMax = max(runningMax - 1, dp[j])
                rightToLeft[j] = runningMax

            for j in range(n):
                dp[j] = max(leftToRight[j], rightToLeft[j]) + row[j]

        return max(dp)
```

844. 1938.py

Path: LeetCode-main/solutions/1938. Maximum Genetic Difference Query/1938.py

```
class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2
        self.count = 0

class Trie:
    def __init__(self):
        self.root = TrieNode()
        self.HEIGHT = 17

    def update(self, num: int, val: int) -> None:
        node = self.root
        for i in range(self.HEIGHT, -1, -1):
            bit = (num >> i) & 1
            if not node.children[bit]:
                node.children[bit] = TrieNode()
            node = node.children[bit]
            node.count += val

    def query(self, num: int) -> int:
        ans = 0
        node = self.root
        for i in range(self.HEIGHT, -1, -1):
            bit = (num >> i) & 1
            targetBit = bit ^ 1
            if node.children[targetBit] and node.children[targetBit].count > 0:
                ans += 1 << i
                node = node.children[targetBit]
            else:
                node = node.children[targetBit ^ 1]
        return ans

class Solution:
    def maxGeneticDifference(
        self,
        parents: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        n = len(parents)
        ans = [0] * len(queries)
        rootVal = -1
        tree = [[] for _ in range(n)]
        nodeToQueries = collections.defaultdict(list)  # {node: (index, val)}
        trie = Trie()

        for i, parent in enumerate(parents):
            if parent == -1:
                rootVal = i
            else:
                tree[parent].append(i)

        for i, (node, val) in enumerate(queries):
            nodeToQueries[node].append((i, val))

        def dfs(node: int) -> None:
            trie.update(node, 1)

            # Answer queries for node
            for i, val in nodeToQueries[node]:
```

```
ans[i] = trie.query(val)

for child in tree[node]:
    dfs(child)

trie.update(node, -1)

dfs(rootVal)
return ans
```

845. 1940-2.py

Path: LeetCode-main/solutions/1940. Longest Common Subsequence Between Sorted Arrays/1940-2.py

```
class Solution:
    def longestCommonSubsequence(self, arrays: list[list[int]]) -> list[int]:
        return sorted(functools.reduce(lambda a, b: set(a) & set(b), arrays))
```

846. 1940.py

Path: LeetCode-main/solutions/1940. Longest Common Subsequence Between Sorted Arrays/1940.py

```
class Solution:
    def longestCommonSubsequence(self, arrays: list[list[int]]) -> list[int]:
        MAX = 100
        ans = []
        count = [0] * (MAX + 1)

        for array in arrays:
            for a in array:
                count[a] += 1
                if count[a] == len(arrays):
                    ans.append(a)

        return ans
```

847. 1941.py

Path: LeetCode-main/solutions/1941. Check if All Characters Have Equal Number of Occurrences/1941.py

```
class Solution:
    def areOccurrencesEqual(self, s: str) -> bool:
        return len(set(collections.Counter(s).values())) == 1
```

848. 1942.py

Path: LeetCode-main/solutions/1942. The Number of the Smallest Unoccupied Chair/1942.py

```

class Solution:
    def smallestChair(self, times: list[list[int]], targetFriend: int) -> int:
        nextUnsatChair = 0
        emptyChairs = []
        occupied = [] # (leaving, chair)

        for i in range(len(times)):
            times[i].append(i)

        times.sort(key=lambda x: x[0])

        for arrival, leaving, i in times:
            while len(occupied) > 0 and occupied[0][0] <= arrival:
                unsatChair = heapq.heappop(occupied)[1]
                heapq.heappush(emptyChairs, unsatChair)
            if i == targetFriend:
                return emptyChairs[0] if len(emptyChairs) > 0 else nextUnsatChair
            if len(emptyChairs) == 0:
                heapq.heappush(occupied, (leaving, nextUnsatChair))
                nextUnsatChair += 1
            else:
                emptyChair = heapq.heappop(emptyChairs)
                heapq.heappush(occupied, (leaving, emptyChair))

```

849. 1943.py

Path: LeetCode-main/solutions/1943. Describe the Painting/1943.py

```

from sortedcontainers import SortedDict

class Solution:
    def splitPainting(self, segments: list[list[int]]) -> list[list[int]]:
        ans = []
        prevIndex = 0
        runningMix = 0
        line = SortedDict()

        for start, end, color in segments:
            line[start] = line.get(start, 0) + color
            line[end] = line.get(end, 0) - color

        for i, mix in line.items():
            if runningMix > 0:
                ans.append([prevIndex, i, runningMix])
                runningMix += mix
                prevIndex = i

        return ans

```

850. 1944.py

Path: LeetCode-main/solutions/1944. Number of Visible People in a Queue/1944.py

```

class Solution:
    def canSeePersonsCount(self, heights: list[int]) -> list[int]:
        ans = [0] * len(heights)
        stack = []

        for i, height in enumerate(heights):
            while stack and heights[stack[-1]] <= height:
                ans[stack.pop()] += 1
            if stack:
                ans[stack[-1]] += 1
            stack.append(i)

        return ans

```

851. 1945.py

Path: LeetCode-main/solutions/1945. Sum of Digits of String After Convert/1945.py

```

class Solution:
    def getLucky(self, s: str, k: int) -> int:
        ans = self._convert(s)
        for _ in range(k):
            ans = self._getDigitSum(ans)
        return ans

    def _convert(self, s: str) -> int:
        return int(''.join(str(ord(c) - ord('a') + 1) for c in s))

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))

```

852. 1946.py

Path: LeetCode-main/solutions/1946. Largest Number After Mutating Substring/1946.py

```

class Solution:
    def maximumNumber(self, num: str, change: list[int]) -> str:
        numList = list(num)
        mutated = False

        for i, c in enumerate(numList):
            d = int(c)
            numlist[i] = chr(ord('0') + max(d, change[d]))
            if mutated and d > change[d]:
                return ''.join(numList)
            if d < change[d]:
                mutated = True

        return ''.join(numList)

```

853. 1947.py

Path: LeetCode-main/solutions/1947. Maximum Compatibility Score Sum/1947.py

```
class Solution:
    def maxCompatibilitySum(
        self,
        students: list[list[int]],
        mentors: list[list[int]],
    ) -> int:
        ans = 0

        def dfs(i: int, scoreSum: int, used: list[bool]) -> None:
            nonlocal ans
            if i == len(students):
                ans = max(ans, scoreSum)
                return

            for j, mentor in enumerate(mentors):
                if used[j]:
                    continue
                used[j] = True # The `mentors[j]` is used.
                dfs(i + 1, scoreSum + sum(s == m
                                           for s, m in zip(students[i], mentor)), used)
                used[j] = False

        dfs(0, 0, [False] * len(students))
        return ans
```

854. 1948.py

Path: LeetCode-main/solutions/1948. Delete Duplicate Folders in System/1948.py


```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.deleted = False

class Solution:
    def deleteDuplicateFolder(self, paths: list[list[str]]) -> list[list[str]]:
        ans = []
        root = TrieNode()
        subtreeToNodes: dict[str, list[TrieNode]] = collections.defaultdict(list)

        # Construct the Trie
        for path in sorted(paths):
            node = root
            for s in path:
                node = node.children.setdefault(s, TrieNode())

        # For each subtree, fill in the {subtree encoding: [root]} hash table
        def buildSubtreeToRoots(node: TrieNode) -> str:
            subtree = '(' + ''.join(s + buildSubtreeToRoots(node.children[s])
                                   for s in node.children) + ')'

            if subtree != '()':
                subtreeToNodes[subtree].append(node)
            return subtree

        buildSubtreeToRoots(root)

        # Mark nodes that should be deleted
        for nodes in subtreeToNodes.values():
            if len(nodes) > 1:
                for node in nodes:
                    node.deleted = True

        # Construct the answer array for nodes that haven't been deleted
        def constructPath(node: TrieNode, path: list[str]) -> None:
            for s, child in node.children.items():
                if not child.deleted:
                    constructPath(child, path + [s])
            if path:
                ans.append(path)

        constructPath(root, [])
        return ans

```

855. 1950.py

Path: LeetCode-main/solutions/1950. Maximum of Minimum Values in All Subarrays/1950.py

```

class Solution:
    # Similar to 1950. Maximum of Minimum Values in All Subarrays
    def findMaximums(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [0] * n
        # prevMin[i] := the index k s.t.
        # nums[k] is the previous minimum in nums[0..n)
        prevMin = [-1] * n
        # nextMin[i] := the index k s.t.
        # nums[k] is the next minimum in nums[i + 1..n)
        nextMin = [n] * n
        stack = []

        for i, num in enumerate(nums):
            while stack and nums[stack[-1]] > nums[i]:
                index = stack.pop()
                nextMin[index] = i
            if stack:
                prevMin[i] = stack[-1]
            stack.append(i)

        # For each nums[i], let l = nextMin[i] + 1 and r = nextMin[i] - 1.
        # nums[i] is the minimum in nums[l..r].
        # So, the ans[r - l + 1] will be at least nums[i].
        for num, l, r in zip(nums, prevMin, nextMin):
            sz = r - l - 1
            ans[sz - 1] = max(ans[sz - 1], num)

        # ans[i] should always >= ans[i + 1..n).
        for i in range(n - 2, -1, -1):
            ans[i] = max(ans[i], ans[i + 1])

        return ans

```

856. 1952.py

Path: LeetCode-main/solutions/1952. Three Divisors/1952.py

```

class Solution:
    def isThree(self, n: int) -> bool:
        if n == 1:
            return False
        # The numbers with exactly three divisors are perfect squares of a prime
        # number.
        root = math.isqrt(n)
        return (root**2 == n and
                all(root % i != 0
                    for i in range(2, math.isqrt(root) + 1)))

```

857. 1953.py

Path: LeetCode-main/solutions/1953. Maximum Number of Weeks for Which You Can Work/1953.py

```

class Solution:
    def numberOfWeeks(self, milestones: list[int]) -> int:
        # The best strategy is to pick 'max, nonMax, max, nonMax, ...'.
        summ = sum(milestones)
        nonMax = summ - max(milestones)
        return min(summ, 2 * nonMax + 1)

```

858. 1954.py

Path: LeetCode-main/solutions/1954. Minimum Garden Perimeter to Collect Enough Apples/1954.py

```
class Solution:
    def minimumPerimeter(self, neededApples: int) -> int:
        def numApples(k: int) -> int:
            """Returns the number of apples at the k-th level.

            k := the level making perimeter = 8k
            p(k) := the number of apples at the k-th level on the perimeter
            n(k) := the number of apples at the k-th level not on the perimeter

            p(1) = 1 + 2
            p(2) = 3 + 2 + 3 + 4
            p(3) = 5 + 4 + 3 + 4 + 5 + 6
            p(4) = 7 + 6 + 5 + 4 + 5 + 6 + 7 + 8
            p(k) = k + 2(k+1) + 2(k+2) + ... + 2(k+k-1) + 2k
                  = k + 2k^2 + 2*k(k-1)//2
                  = k + 2k^2 + k^2 - k = 3k^2

            n(k) = p(1) + p(2) + p(3) + ... + p(k)
                  = 3*1^2 + 3*4^2 + 3*9^2 + ... + 3*k^2
                  = 3 * (1 + 4 + 9 + ... + k^2)
                  = 3 * k(k+1)(2k+1)//6 = k(k+1)(2k+1)//2

            So, the number of apples at the k-th level should be
            k(k+1)(2k+1)//2 * 4 = 2k(k+1)(2k+1)
            """
            return 2 * k * (k + 1) * (2 * k + 1)

        return bisect.bisect_left(range(100_000), neededApples,
                                   key=numApples) * 8
```

859. 1955-2.py

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955-2.py

```

class Solution:
    def countSpecialSubsequences(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        # dp[i][j] := the number of increasing subsequences of the first i numbers
        # that end in j
        dp = [[0] * 3 for _ in range(n)]

        if nums[0] == 0:
            dp[0][0] = 1

        for i in range(1, n):
            for ending in range(3):
                dp[i][ending] = dp[i - 1][ending]

            if nums[i] == 0:
                # 1. The number of the previous subsequences that end in 0.
                # 2. Append a 0 to the previous subsequences that end in 0.
                # 3. Start a new subsequence from this 0.
                dp[i][0] = dp[i - 1][0] * 2 + 1
            elif nums[i] == 1:
                # 1. The number of the previous subsequences that end in 1.
                # 2. Append a 1 to the previous subsequences that end in 1.
                # 3. Append a 1 to the previous subsequences that end in 0.
                dp[i][1] = dp[i - 1][1] * 2 + dp[i - 1][0]
            else: # nums[i] == 2
                # 1. The number of the previous subsequences that end in 2.
                # 2. Append a 2 to the previous subsequences that end in 2.
                # 3. Append a 2 to the previous subsequences that end in 1.
                dp[i][2] = dp[i - 1][2] * 2 + dp[i - 1][1]

            for ending in range(3):
                dp[i][ending] %= MOD

        return dp[-1][2]

```

860. 1955-3.py

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955-3.py

```

class Solution:
    def countSpecialSubsequences(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        # dp[j] := the number of increasing subsequences of the numbers so far that
        # end in j
        dp = [0] * 3

        if nums[0] == 0:
            dp[0] = 1

        for i in range(1, n):
            if nums[i] == 0:
                dp[0] = dp[0] * 2 + 1
            elif nums[i] == 1:
                dp[1] = dp[1] * 2 + dp[0]
            else: # nums[i] == 2
                dp[2] = dp[2] * 2 + dp[1]

        for ending in range(3):
            dp[ending] %= MOD

        return dp[2]

```

861. 1955.py

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955.py

```

class Solution:
    def countSpecialSubsequences(self, nums: list[int]) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(i: int, prev: int) -> int:
            """
            Returns the number of increasing subsequences of the first i numbers,
            where the the previous number is j - 1.
            """
            if i == len(nums):
                return prev == 2

            res = 0

            # Don't include `nums[i]`.
            res += dp(i + 1, prev)

            # Include `nums[i]`.
            if nums[i] == prev:
                res += dp(i + 1, prev)
            if prev == -1 and nums[i] == 0:
                res += dp(i + 1, 0)
            if prev == 0 and nums[i] == 1:
                res += dp(i + 1, 1)
            if prev == 1 and nums[i] == 2:
                res += dp(i + 1, 2)

            res %= MOD
            return res

        return dp(0, -1)

```

862. 1956.py

Path: LeetCode-main/solutions/1956. Minimum Time For K Virus Variants to Spread/1956.py

```
class Solution:
    def minDayskVariants(self, points: list[list[int]], k: int) -> int:
        MAX = 100
        ans = math.inf

        for a in range(1, MAX + 1):
            for b in range(1, MAX + 1):
                # Stores the k minimum distances of points that can reach (a, b).
                maxHeap = []
                for x, y in points:
                    heapq.heappush(maxHeap, -abs(x - a) + -abs(y - b))
                    if len(maxHeap) > k:
                        heapq.heappop(maxHeap)
                ans = min(ans, -maxHeap[0])

        return ans
```

863. 1957.py

Path: LeetCode-main/solutions/1957. Delete Characters to Make Fancy String/1957.py

```
class Solution:
    def makeFancyString(self, s: str) -> str:
        ans = []
        for c in s:
            if len(ans) < 2 or ans[-1] != c or ans[-2] != c:
                ans.append(c)
        return ''.join(ans)
```

864. 1958.py

Path: LeetCode-main/solutions/1958. Check if Move is Legal/1958.py

```
class Solution:
    def checkMove(
        self,
        board: list[list[str]],
        rMove: int,
        cMove: int,
        color: str,
    ) -> bool:
        DIRS = ((-1, -1), (-1, 0), (-1, 1), (0, -1),
                (0, 1), (1, -1), (1, 0), (1, 1))

        for dx, dy in DIRS:
            cellsCount = 2
            i = rMove + dx
            j = cMove + dy
            while 0 <= i < 8 and 0 <= j < 8:
                # There are no free cells in between.
                if board[i][j] == '.':
                    break
                # Need >= 3 cells.
                if cellsCount == 2 and board[i][j] == color:
                    break
                # >= 3 cells.
                if board[i][j] == color:
                    return True
                i += dx
                j += dy
                cellsCount += 1

        return False
```

865. 1959.py

Path: LeetCode-main/solutions/1959. Minimum Total Space Wasted With K Resizing Operations/1959.py

```
class Solution:
    def minSpaceWastedKResizing(self, nums: list[int], k: int) -> int:
        MAX = 200_000_000

        @functools.lru_cache(None)
        def dp(i: int, k: int) -> int:
            """
            Returns the minimum space wasted for nums[i..n) if you can resize k times.
            """
            if i == len(nums):
                return 0
            if k == -1:
                return MAX

            res = MAX
            summ = 0
            maxNum = nums[i]

            for j in range(i, len(nums)):
                summ += nums[j]
                maxNum = max(maxNum, nums[j])
                wasted = maxNum * (j - i + 1) - summ
                res = min(res, dp(j + 1, k - 1) + wasted)

            return res

        return dp(0, k)
```

866. 1960-2.py

Path: LeetCode-main/solutions/1960. Maximum Product of the Length of Two Palindromic Substrings/1960-2.py


```

class Solution:
    def maxProduct(self, s: str) -> int:
        BASE = 26
        HASH = 1_000_000_007
        n = len(s)
        pows = [1] # pows[i] := BASE^i % HASH
        # hashL[i] = the hash of the first i letters of s, where hashL[i] =
        # (26^(i - 1) * s[0] + 26^(i - 2) * s[1] + ... + s[i - 1]) % HASH
        hashL = [0]
        # hashR[i] = the hash of the last i letters of s, where hashR[i] =
        # (26^(i - 1) * s[-1] + 26^(i - 2) * s[-2] + ... + s[-i]) % HASH
        hashR = [0]
        # maxLeft[i] := the maximum odd length of palindromes in s[0..i]
        maxLeft = [0] * n
        # maxRight[i] := the maximum odd length of palindromes in s[i..n - 1]
        maxRight = [0] * n

        def val(c: str) -> int:
            return ord(c) - ord('a')

        for _ in range(n):
            pows.append(pows[-1] * BASE % HASH)

        for c in s:
            hashL.append((hashL[-1] * BASE + val(c)) % HASH)

        for c in reversed(s):
            hashR.append((hashR[-1] * BASE + val(c)) % HASH)

        hashR.reverse()

        def getLeftRollingHash(l: int, r: int) -> int:
            """Returns the left rolling hash of s[l..r)."""
            h = (hashL[r] - hashL[l] * pows[r - l]) % HASH
            return h + HASH if h < 0 else h

        def getRightRollingHash(l: int, r: int) -> int:
            """Returns the right rolling hash of s[l..r)."""
            h = (hashR[l] - hashR[r] * pows[r - l]) % HASH
            return h + HASH if h < 0 else h

        def isPalindrome(l: int, r: int) -> bool:
            """Returns True if s[l..r) is a palindrome."""
            return getLeftRollingHash(l, r) == getRightRollingHash(l, r)

        maxLength = 1
        for r in range(n):
            l = (r - maxLength - 2) + 1
            if l >= 0 and isPalindrome(l, r + 1):
                maxLength += 2
            maxLeft[r] = maxLength

        maxLength = 1
        for l in reversed(range(n)):
            r = (l + maxLength + 2) - 1
            if r < n and isPalindrome(l, r + 1):
                maxLength += 2
            maxRight[l] = maxLength

        return max(maxLeft[i - 1] * maxRight[i] for i in range(1, n))

```

Path: LeetCode-main/solutions/1960. Maximum Product of the Length of Two Palindromic Substrings/1960.py

```
class Solution:
    def maxProduct(self, s: str) -> int:
        n = len(s)

        def manacher(s: str) -> list[int]:
            maxExtends = [0] * n
            leftToRight = [1] * n
            center = 0

            for i in range(n):
                r = center + maxExtends[center] - 1
                mirrorIndex = center - (i - center)
                extend = 1 if i > r else min(maxExtends[mirrorIndex], r - i + 1)
                while i - extend >= 0 and i + extend < n and s[i - extend] == s[i + extend]:
                    leftToRight[i + extend] = 2 * extend + 1
                    extend += 1
                maxExtends[i] = extend
                if i + maxExtends[i] >= r:
                    center = i

            for i in range(1, n):
                leftToRight[i] = max(leftToRight[i], leftToRight[i - 1])

            return leftToRight

        # maxLeft[i] := the maximum odd length of palindromes in s[0..i]
        maxLeft = manacher(s)
        # maxRight[i] := the maximum odd length of palindromes in s[i..n - 1]
        maxRight = manacher(s[::-1])[::-1]
        return max(maxLeft[i - 1] * maxRight[i] for i in range(1, n))
```

868. 1961.py

Path: LeetCode-main/solutions/1961. Check If String Is a Prefix of Array/1961.py

```
class Solution:
    def isPrefixString(self, s: str, words: list[str]) -> bool:
        prefix = []
        for word in words:
            prefix.append(word)
            if ''.join(prefix) == s:
                return True
        return False
```

869. 1962.py

Path: LeetCode-main/solutions/1962. Remove Stones to Minimize the Total/1962.py

```
class Solution:
    def minStoneSum(self, piles: list[int], k: int) -> int:
        maxHeap = [-pile for pile in piles]
        heapq.heapify(maxHeap)

        for _ in range(k):
            heapq.heapreplace(maxHeap, maxHeap[0] // 2)

        return -sum(maxHeap)
```

870. 1963.py

Path: LeetCode-main/solutions/1963. Minimum Number of Swaps to Make the String Balanced/1963.py

```
class Solution:
    def minSwaps(self, s: str) -> int:
        # Cancel out all the matched pairs, then we'll be left with ']]]..[[['.
        # The answer is ceil(# of unmatched pairs // 2).
        unmatched = 0

        for c in s:
            if c == '[':
                unmatched += 1
            elif unmatched > 0: # c == ']' and there's a match.
                unmatched -= 1

        return (unmatched + 1) // 2
```

871. 1964.py

Path: LeetCode-main/solutions/1964. Find the Longest Valid Obstacle Course at Each Position/1964.py

```
class Solution:
    # Similar to 300. Longest Increasing Subsequence
    def longestObstacleCourseAtEachPosition(
        self, obstacles: list[int],
    ) -> list[int]:
        ans = []
        # tails[i] := the minimum tail of all the increasing subsequences having
        # length i + 1
        tails = []

        for obstacle in obstacles:
            if not tails or obstacle >= tails[-1]:
                tails.append(obstacle)
                ans.append(len(tails))
            else:
                index = bisect.bisect_right(tails, obstacle)
                tails[index] = obstacle
                ans.append(index + 1)

        return ans
```

872. 1966.py

Path: LeetCode-main/solutions/1966. Binary Searchable Numbers in an Unsorted Array/1966.py

```

class Solution:
    def binarySearchableNumbers(self, nums: list[int]) -> int:
        n = len(nums)
        # prefixMaxs[i] := max(nums[0..i])
        prefixMaxs = [0] * n
        # suffixMins[i] := min(nums[i + 1..n))
        suffixMins = [0] * n

        # Fill in `prefixMaxs`.
        prefixMaxs[0] = -math.inf
        for i in range(1, n):
            prefixMaxs[i] = max(prefixMaxs[i - 1], nums[i - 1])

        # Fill in `suffixMins`.
        suffixMins[n - 1] = math.inf
        for i in range(n - 2, -1, -1):
            suffixMins[i] = min(suffixMins[i + 1], nums[i + 1])

        return sum(prefixMaxs[i] < nums[i] < suffixMins[i] for i in range(n))

```

873. 1967.py

Path: LeetCode-main/solutions/1967. Number of Strings That Appear as Substrings in Word/1967.py

```

class Solution:
    def numOfStrings(self, patterns: list[str], word: str) -> int:
        return sum(pattern in word for pattern in patterns)

```

874. 1968.py

Path: LeetCode-main/solutions/1968. Array With Elements Not Equal to Average of Neighbors/1968.py

```

class Solution:
    def rearrangeArray(self, nums: list[int]) -> list[int]:
        nums.sort()
        for i in range(1, len(nums), 2):
            nums[i], nums[i - 1] = nums[i - 1], nums[i]
        return nums

```

875. 1969.py

Path: LeetCode-main/solutions/1969. Minimum Non-Zero Product of the Array Elements/1969.py

```

class Solution:
    def minNonZeroProduct(self, p: int) -> int:
        MOD = 1_000_000_007
        # Can always turn  $[1..2^p - 1]$  to  $[1, 1, \dots, 2^{p-2}, 2^{p-2}, 2^{p-2}, \dots, 2^{p-2}, 2^{p-2}]$ .
        n = 1 << p
        halfCount = n // 2 - 1
        return pow(n - 2, halfCount, MOD) * ((n - 1) % MOD) % MOD

```

876. 1970.py

Path: LeetCode-main/solutions/1970. Last Day Where You Can Still Cross/1970.py

```

class Solution:
    def latestDayToCross(self, row: int, col: int, cells: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))

        def canWalk(day: int) -> bool:
            matrix = [[0] * col for _ in range(row)]
            for i in range(day):
                x, y = cells[i]
                matrix[x - 1][y - 1] = 1

            q = collections.deque()

            for j in range(col):
                if matrix[0][j] == 0:
                    q.append((0, j))
                    matrix[0][j] = 1

            while q:
                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == row or y < 0 or y == col:
                        continue
                    if matrix[x][y] == 1:
                        continue
                    if x == row - 1:
                        return True
                    q.append((x, y))
                    matrix[x][y] = 1

            return False

        ans = 0
        l = 1
        r = len(cells) - 1

        while l <= r:
            m = (l + r) // 2
            if canWalk(m):
                ans = m
                l = m + 1
            else:
                r = m - 1

        return ans

```

877. 1971.py

Path: LeetCode-main/solutions/1971. Find if Path Exists in Graph/1971.py

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def validPath(
        self,
        n: int,
        edges: list[list[int]],
        source: int,
        destination: int,
    ) -> bool:
        uf = UnionFind(n)

        for u, v in edges:
            uf.unionByRank(u, v)

        return uf.find(source) == uf.find(destination)
```

878. 1973.py

Path: LeetCode-main/solutions/1973. Count Nodes Equal to Sum of Descendants/1973.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    summ: int
    count: int

class Solution:
    def equalToDescendants(self, root: TreeNode | None) -> int:
        def dfs(root: TreeNode | None) -> T:
            if not root:
                return T(0, 0)
            left = dfs(root.left)
            right = dfs(root.right)
            return T(root.val + left.summ + right.summ,
                    left.count + right.count +
                    (1 if root.val == left.summ + right.summ else 0))

        return dfs(root).count

```

879. 1974.py

Path: LeetCode-main/solutions/1974. Minimum Time to Type Word Using Special Typewriter/1974.py

```

class Solution:
    def minTimeToType(self, word: str) -> int:
        moves = 0
        letter = 'a'

        for c in word:
            diff = abs(ord(c) - ord(letter))
            moves += min(diff, 26 - diff)
            letter = c

        return moves + len(word)

```

880. 1975.py

Path: LeetCode-main/solutions/1975. Maximum Matrix Sum/1975.py

```

class Solution:
    def maxMatrixSum(self, matrix: list[list[int]]) -> int:
        absSum = 0
        minAbs = math.inf
        # 0 := even number of negatives
        # 1 := odd number of negatives
        oddNeg = 0

        for row in matrix:
            for num in row:
                absSum += abs(num)
                minAbs = min(minAbs, abs(num))
                if num < 0:
                    oddNeg ^= 1

        return absSum - oddNeg * minAbs * 2

```

881. 1976.py

Path: LeetCode-main/solutions/1976. Number of Ways to Arrive at Destination/1976.py

```
class Solution:
    def countPaths(self, n: int, roads: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in roads:
            graph[u].append((v, w))
            graph[v].append((u, w))

        return self._dijkstra(graph, 0, n - 1)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        dst: int,
    ) -> int:
        MOD = 10**9 + 7
        ways = [0] * len(graph)
        dist = [math.inf] * len(graph)

        ways[src] = 1
        dist[src] = 0
        minHeap = [(dist[src], src)]

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    ways[v] = ways[u]
                    heapq.heappush(minHeap, (dist[v], v))
                elif d + w == dist[v]:
                    ways[v] += ways[u]
                    ways[v] %= MOD

        return ways[dst]
```

882. 1977.py

Path: LeetCode-main/solutions/1977. Number of Ways to Separate Numbers/1977.py


```

class Solution:
    def numberOfCombinations(self, num: str) -> int:
        if num[0] == '0':
            return 0

        MOD = 1_000_000_007
        n = len(num)
        # dp[i][k] := the number of possible lists of integers ending in num[i]
        # with the length of the last number being 1..k
        dp = [[0] * (n + 1) for _ in range(n)]
        # lcs[i][j] := the number of the same digits in num[i..n) and num[j..n)
        lcs = [[0] * (n + 1) for _ in range(n + 1)]

        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                if num[i] == num[j]:
                    lcs[i][j] = lcs[i + 1][j + 1] + 1

        for i in range(n):
            for k in range(1, i + 2):
                dp[i][k] += dp[i][k - 1]
                dp[i][k] %= MOD
                # The last number is num[s..i].
                s = i - k + 1
                if num[s] == '0':
                    # the number of possible lists of integers ending in num[i] with the
                    # length of the last number being k
                    continue
                if s == 0: # the whole string
                    dp[i][k] += 1
                    continue
                if s < k:
                    # The length k is not enough, so add the number of possible lists of
                    # integers in num[0..s - 1].
                    dp[i][k] += dp[s - 1][s]
                    continue
                l = lcs[s - k][s]
                if l >= k or num[s - k + 1] <= num[s + 1]:
                    # Have enough length k and num[s - k..s - 1] <= num[j..i].
                    dp[i][k] += dp[s - 1][k]
                else:
                    # Have enough length k but num[s - k..s - 1] > num[j..i].
                    dp[i][k] += dp[s - 1][k - 1]

        return dp[n - 1][n] % MOD

```

883. 1979.py

Path: LeetCode-main/solutions/1979. Find Greatest Common Divisor of Array/1979.py

```

class Solution:
    def findGCD(self, nums: list[int]) -> int:
        return math.gcd(min(nums), max(nums))

```

884. 198-2.py

Path: LeetCode-main/solutions/198. House Robber/198-2.py

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        prev1 = 0 # dp[i - 1]
        prev2 = 0 # dp[i - 2]

        for num in nums:
            dp = max(prev1, prev2 + num)
            prev2 = prev1
            prev1 = dp

        return prev1
```

885. 198.py

Path: LeetCode-main/solutions/198. House Robber/198.py

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) == 1:
            return nums[0]

        # dp[i] := max money of robbing nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]
        dp[1] = max(nums[0], nums[1])

        for i in range(2, len(nums)):
            dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])

        return dp[-1]
```

886. 1980-2.py

Path: LeetCode-main/solutions/1980. Find Unique Binary String/1980-2.py

```
class Solution:
    def findDifferentBinaryString(self, nums: list[str]) -> str:
        return ''.join('1' if num[i] == '0' else '0' for i, num in enumerate(nums))
```

887. 1980.py

Path: LeetCode-main/solutions/1980. Find Unique Binary String/1980.py

```
class Solution:
    def findDifferentBinaryString(self, nums: list[str]) -> str:
        bitSize = len(nums[0])
        maxNum = 1 << bitSize
        numsSet = {int(num, 2) for num in nums}

        for num in range(maxNum):
            if num not in numsSet:
                return f'{num:0>{bitSize}b}'
```

888. 1981.py

Path: LeetCode-main/solutions/1981. Minimize the Difference Between Target and Chosen Elements/1981.py

```
class Solution:
    def minimizeTheDifference(self, mat: list[list[int]], target: int) -> int:
        minSum = sum(min(row) for row in mat)
        if minSum >= target: # No need to consider any larger combination.
            return minSum - target

        @functools.lru_cache(None)
        def dp(i: int, summ: int) -> int:
            if i == len(mat):
                return abs(summ - target)
            return min(dp(i + 1, summ + num) for num in mat[i])

        return dp(0, 0)
```

889. 1982.py

Path: LeetCode-main/solutions/1982. Find Array Given Subset Sums/1982.py

```
class Solution:
    def recoverArray(self, n: int, sums: list[int]) -> list[int]:
        def recover(sums: list[int]) -> list[int]:
            if len(sums) == 1:
                return []

            count = collections.Counter(sums)
            # Either num or -num must be in the final array.
            # num + sumsExcludingNum = sumsIncludingNum
            # -num + sumsIncludingNum = sumsExcludingNum
            num = sums[1] - sums[0]
            sumsExcludingNum = []
            sumsIncludingNum = []
            chooseSumsExcludingNum = True

            for summ in sums:
                if count[summ] == 0:
                    continue
                count[summ] -= 1
                count[summ + num] -= 1
                sumsExcludingNum.append(summ)
                sumsIncludingNum.append(summ + num)
                if summ + num == 0:
                    chooseSumsExcludingNum = False

            # Choose `sumsExcludingNum` by default since we want to gradually strip
            # `num` from each sum in `sums` to have the final array. However, we should
            # always choose the group of sums with 0 since it's a must-have.
            return ([num] + recover(sumsExcludingNum) if chooseSumsExcludingNum else
                    [-num] + recover(sumsIncludingNum))

        return recover(sorted(sums))
```

890. 1983.py

Path: LeetCode-main/solutions/1983. Widest Pair of Indices With Equal Range Sum/1983.py

```
class Solution:
    def widestPairOfIndices(self, nums1: list[int], nums2: list[int]) -> int:
        ans = 0
        prefix = 0
        prefixToIndex = {0: -1}

        for i, (num1, num2) in enumerate(zip(nums1, nums2)):
            prefix += num1 - num2
            ans = max(ans, i - prefixToIndex.setdefault(prefix, i))

        return ans
```

891. 1984.py

Path: LeetCode-main/solutions/1984. Minimum Difference Between Highest and Lowest of K Scores/1984.py

```
class Solution:
    def minimumDifference(self, nums: list[int], k: int) -> int:
        nums.sort()
        ans = nums[k - 1] - nums[0]

        for i in range(k, len(nums)):
            ans = min(ans, nums[i] - nums[i - k + 1])

        return ans
```

892. 1985.py

Path: LeetCode-main/solutions/1985. Find the Kth Largest Integer in the Array/1985.py

```
class Solution:
    # Similar to 215. Kth Largest Element in an Array
    def kthLargestNumber(self, nums: list[str], k: int) -> str:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, int(num))
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return str(minHeap[0])
```

893. 1986.py

Path: LeetCode-main/solutions/1986. Minimum Number of Work Sessions to Finish the Tasks/1986.py

```

class Solution:
    def minSessions(self, tasks: list[int], sessionTime: int) -> int:
        # Returns True if we can assign tasks[s..n) to `sessions`. Note that `sessions`
        # may be occupied by some tasks.
        def dfs(s: int, sessions: list[int]) -> bool:
            if s == len(tasks):
                return True

            for i, session in enumerate(sessions):
                # Can't assign the tasks[s] to this session.
                if session + tasks[s] > sessionTime:
                    continue
                # Assign the tasks[s] to this session.
                sessions[i] += tasks[s]
                if dfs(s + 1, sessions):
                    return True
                # Backtracking.
                sessions[i] -= tasks[s]
                # If it's the first time we assign the tasks[s] to this session, then future
                # `session`s can't satisfy either.
                if sessions[i] == 0:
                    return False

            return False

        for numSessions in range(1, len(tasks) + 1):
            if dfs(0, [0] * numSessions):
                return numSessions

```

894. 1987.py

Path: LeetCode-main/solutions/1987. Number of Unique Good Subsequences/1987.py

```

class Solution:
    # Similar to 940. Distinct Subsequences II
    def numberOfUniqueGoodSubsequences(self, binary: str) -> int:
        MOD = 1_000_000_007
        # endsIn[i] := the number of subsequence that end in ('0' + i)
        endsIn = {'0': 0, '1': 0}

        for c in binary:
            endsIn[c] = sum(endsIn.values()) % MOD
            # Don't count '0' since we want to avoid the leading zeros case.
            # However, we can always count '1'.
            if c == '1':
                endsIn['1'] += 1

        # Count '0' in the end.
        return (sum(endsIn.values()) + ('0' in binary)) % MOD

```

895. 1989.py

Path: LeetCode-main/solutions/1989. Maximum Number of People That Can Be Caught in Tag/1989.py

```

class Solution:
    def catchMaximumAmountofPeople(self, team: list[int], dist: int) -> int:
        ans = 0
        i = 0 # 0s index
        j = 0 # 1s index

        while i < len(team) and j < len(team):
            if i + dist < j or team[i] != 0:
                # Find the next 0 that can be caught by 1.
                i += 1
            elif j + dist < i or team[j] != 1:
                # Find the next 1 that can catch 0.
                j += 1
            else:
                # team[j] catches team[i], so move both.
                ans += 1
                i += 1
                j += 1

        return ans

```

896. 199-2.py

Path: LeetCode-main/solutions/199. Binary Tree Right Side View/199-2.py

```

class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        ans = []

        def dfs(root: TreeNode | None, depth: int) -> None:
            if not root:
                return

            if depth == len(ans):
                ans.append(root.val)
            dfs(root.right, depth + 1)
            dfs(root.left, depth + 1)

        dfs(root, 0)
        return ans

```

897. 199.py

Path: LeetCode-main/solutions/199. Binary Tree Right Side View/199.py

```
class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            size = len(q)
            for i in range(size):
                root = q.popleft()
                if i == size - 1:
                    ans.append(root.val)
                if root.left:
                    q.append(root.left)
                if root.right:
                    q.append(root.right)

        return ans
```

898. 1991.py

Path: LeetCode-main/solutions/1991. Find the Middle Index in Array/1991.py

```
class Solution:
    def findMiddleIndex(self, nums: list[int]) -> int:
        prefix = 0
        suffix = sum(nums)

        for i, num in enumerate(nums):
            suffix -= num
            if prefix == suffix:
                return i
            prefix += num

        return -1
```

899. 1992.py

Path: LeetCode-main/solutions/1992. Find All Groups of Farmland/1992.py

```
class Solution:
    def findFarmland(self, land: list[list[int]]) -> list[list[int]]:
        ans = []

        def dfs(i: int, j: int, cell: list[int]) -> None:
            if i < 0 or i == len(land) or j < 0 or j == len(land[0]):
                return
            if land[i][j] != 1:
                return
            land[i][j] = 2 # Mark as visited.
            cell[0] = max(cell[0], i)
            cell[1] = max(cell[1], j)
            dfs(i + 1, j, cell)
            dfs(i, j + 1, cell)

        for i in range(len(land)):
            for j in range(len(land[0])):
                if land[i][j] == 1:
                    cell = [i, j]
                    dfs(i, j, cell)
                    ans.append([i, j, *cell])

        return ans
```

900. 1993.py

Path: LeetCode-main/solutions/1993. Operations on Tree/1993.py


```

class Node:
    def __init__(self):
        self.children: list[int] = []
        self.lockedBy = -1

class LockingTree:
    def __init__(self, parent: list[int]):
        self.parent = parent
        self.nodes = [Node() for _ in range(len(parent))]
        for i in range(1, len(parent)):
            self.nodes[parent[i]].children.append(i)

    def lock(self, num: int, user: int) -> bool:
        if self.nodes[num].lockedBy != -1:
            return False
        self.nodes[num].lockedBy = user
        return True

    def unlock(self, num: int, user: int) -> bool:
        if self.nodes[num].lockedBy != user:
            return False
        self.nodes[num].lockedBy = -1
        return True

    def upgrade(self, num: int, user: int) -> bool:
        if self.nodes[num].lockedBy != -1:
            return False
        if not self._anyLockedDescendant(num):
            return False

        # Walk up the hierarchy to ensure that there are no locked ancestors.
        i = num
        while i != -1:
            if self.nodes[i].lockedBy != -1:
                return False
            i = self.parent[i]

        self._unlockDescendants(num)
        self.nodes[num].lockedBy = user
        return True

    def _anyLockedDescendant(self, i: int) -> bool:
        return (self.nodes[i].lockedBy != -1 or
                any(self._anyLockedDescendant(child)
                    for child in self.nodes[i].children))

    def _unlockDescendants(self, i: int) -> None:
        self.nodes[i].lockedBy = -1
        for child in self.nodes[i].children:
            self._unlockDescendants(child)

```

901. 1994.py

Path: LeetCode-main/solutions/1994. The Number of Good Subsets/1994.py

```

class Solution:
    def numberOfGoodSubsets(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
        n = 1 << len(primes)
        # dp[i] := the number of good subsets with set of primes = i bit mask
        dp = [1] + [0] * (n - 1)
        count = collections.Counter(nums)

        for num, freq in count.items():
            if num == 1:
                continue
            if any(num % squared == 0 for squared in [4, 9, 25]):
                continue
            numPrimesMask = sum(1 << i
                               for i, prime in enumerate(primes)
                               if num % prime == 0)
            for primesMask in range(n):
                # Skip since there're common set of primes (becomes invalid subset)
                if primesMask & numPrimesMask > 0:
                    continue
                nextPrimesMask = numPrimesMask | primesMask
                dp[nextPrimesMask] += dp[primesMask] * freq
                dp[nextPrimesMask] %= MOD

        return (1 << count[1]) * sum(dp[1:]) % MOD

```

902. 1995-2.py

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995-2.py

```

class Solution:
    def countQuadruplets(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        count = collections.Counter()

        for c in range(n - 1, 1, -1): # `c` also represents `d`.
            for b in range(c - 1, 0, -1):
                for a in range(b - 1, -1, -1):
                    ans += count[nums[a] + nums[b] + nums[c]]
                    count[nums[c]] += 1 # c := d

        return ans

```

903. 1995-3.py

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995-3.py

```

class Solution:
    def countQuadruplets(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        count = collections.Counter()

        #   nums[a] + nums[b] + nums[c] == nums[d]
        # => nums[a] + nums[b] == nums[d] - nums[c]
        for b in range(n - 1, 0, -1): # `b` also represents `c`.
            for a in range(b - 1, -1, -1):
                ans += count[nums[a] + nums[b]]
            for d in range(n - 1, b, -1):
                count[nums[d] - nums[b]] += 1 # b := c

        return ans

```

904. 1995.py

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995.py

```

class Solution:
    def countQuadruplets(self, nums: list[int]) -> int:
        n = len(nums)
        return sum(nums[a] + nums[b] + nums[c] == nums[d]
                    for a in range(n)
                    for b in range(a + 1, n)
                    for c in range(b + 1, n)
                    for d in range(c + 1, n))

```

905. 1996-2.py

Path: LeetCode-main/solutions/1996. The Number of Weak Characters in the Game/1996-2.py

```

class Solution:
    def numberOfWeakCharacters(self, properties: list[list[int]]) -> int:
        ans = 0
        maxAttack = max(attack for attack, _ in properties)
        # maxDefenses[i] := the maximum defense for the i-th attack
        maxDefenses = [0] * (maxAttack + 2)

        for attack, defense in properties:
            maxDefenses[attack] = max(maxDefenses[attack], defense)

        for i in range(maxAttack, 0, -1):
            maxDefenses[i] = max(maxDefenses[i], maxDefenses[i + 1])

        for attack, defense in properties:
            if maxDefenses[attack + 1] > defense:
                ans += 1

        return ans

```

906. 1996.py

Path: LeetCode-main/solutions/1996. The Number of Weak Characters in the Game/1996.py

```

class Solution:
    def numberOfWeakCharacters(self, properties: list[list[int]]) -> int:
        ans = 0
        maxDefense = 0

        # Sort properties by `attack` in descending order, then by `defense` in
        # ascending order.
        for _, defense in sorted(properties, key=lambda x: (-x[0], x[1])):
            if defense < maxDefense:
                ans += 1
                maxDefense = max(maxDefense, defense)

        return ans

```

907. 1997.py

Path: LeetCode-main/solutions/1997. First Day Where You Have Been in All the Rooms/1997.py

```

class Solution:
    def firstDayBeenInAllRooms(self, nextVisit: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(nextVisit)

        # dp[i] := the number of days to visit room i for the first time
        dp = [0] * n

        # Whenever we visit i, visit times of room[0..i - 1] are all even.
        # Therefore, the rooms before i can be seen as reset and we can safely
        # reuse dp[0..i - 1] as first-time visit to get second-time visit.
        for i in range(1, n):
            # The total days to visit room[i] is the sum of
            # * dp[i - 1]: 1st-time visit room[i - 1]
            # * 1: visit room[nextVisit[i - 1]]
            # * dp[i - 1] - dp[nextVisit[i - 1]]: 2-time visit room[i - 1]
            # * 1: visit room[i]
            dp[i] = (2 * dp[i - 1] - dp[nextVisit[i - 1]] + 2) % MOD

        return dp[-1]

```

908. 1998.py

Path: LeetCode-main/solutions/1998. GCD Sort of an Array/1998.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return False
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def gcdSort(self, nums: list[int]) -> bool:
        mx = max(nums)
        minPrimeFactors = self._sieveEratosthenes(mx + 1)
        uf = UnionFind(mx + 1)

        for num in nums:
            for primeFactor in self._getPrimeFactors(num, minPrimeFactors):
                uf.unionByRank(num, primeFactor)

        for a, b in zip(nums, sorted(nums)):
            # Can't swap nums[i] with sortedNums[i].
            if uf.find(a) != uf.find(b):
                return False

        return True

    def _sieveEratosthenes(self, n: int) -> list[int]:
        """Gets the minimum prime factor of i, where 1 < i <= n."""
        minPrimeFactors = [i for i in range(n + 1)]
        for i in range(2, int(n**0.5) + 1):
            if minPrimeFactors[i] == i: # `i` is prime.
                for j in range(i * i, n, i):
                    minPrimeFactors[j] = min(minPrimeFactors[j], i)
        return minPrimeFactors

    def _getPrimeFactors(self, num: int, minPrimeFactors: list[int]) -> list[int]:
        primeFactors = []
        while num > 1:
            divisor = minPrimeFactors[num]
            primeFactors.append(divisor)
            while num % divisor == 0:
                num //= divisor
        return primeFactors

```

Path: LeetCode-main/solutions/1999. Smallest Greater Multiple Made of Two Digits/1999-2.py

```
class Solution:
    def findInteger(self, k: int, digit1: int, digit2: int) -> int:
        def dfs(x: int) -> int:
            if x > 2**31 - 1:
                return -1
            if x > k and x % k == 0:
                return x
            # Skip if digit1/digit2 and x are zero.
            a = -1 if x + digit1 == 0 else dfs(x * 10 + digit1)
            b = -1 if x + digit2 == 0 else dfs(x * 10 + digit2)
            if a == -1:
                return b
            if b == -1:
                return a
            return min(a, b)

        return dfs(0)
```

910. 1999.py

Path: LeetCode-main/solutions/1999. Smallest Greater Multiple Made of Two Digits/1999.py

```
class Solution:
    def findInteger(self, k: int, digit1: int, digit2: int) -> int:
        minDigit = min(digit1, digit2)
        maxDigit = max(digit1, digit2)
        digits = [minDigit] if minDigit == maxDigit else [minDigit, maxDigit]
        q = collections.deque()

        for digit in digits:
            q.append(digit)

        while q:
            u = q.popleft()
            if u > k and u % k == 0:
                return u
            if u == 0:
                continue
            for digit in digits:
                nextNum = u * 10 + digit
                if nextNum > 2**31 - 1:
                    continue
                q.append(nextNum)

        return -1
```

911. 2.py

Path: LeetCode-main/solutions/2. Add Two Numbers/2.py

```
class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        carry = 0

        while carry or l1 or l2:
            if l1:
                carry += l1.val
                l1 = l1.next
            if l2:
                carry += l2.val
                l2 = l2.next
            curr.next = ListNode(carry % 10)
            carry //= 10
            curr = curr.next

        return dummy.next
```

912. 20.py

Path: LeetCode-main/solutions/20. Valid Parentheses/20.py

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        for c in s:
            if c == '(':
                stack.append(')')
            elif c == '{':
                stack.append('}')
            elif c == '[':
                stack.append(']')
            elif not stack or stack.pop() != c:
                return False

        return not stack
```

913. 200-2.py

Path: LeetCode-main/solutions/200. Number of Islands/200-2.py

```

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        m = len(grid)
        n = len(grid[0])

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if grid[i][j] != '1':
                return

            grid[i][j] = '2' # Mark '2' as visited.
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        ans = 0

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    dfs(i, j)
                    ans += 1

        return ans

```

914. 200.py

Path: LeetCode-main/solutions/200. Number of Islands/200.py

```

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])

        def bfs(r, c):
            q = collections.deque([(r, c)])
            grid[r][c] = '2' # Mark '2' as visited.
            while q:
                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if grid[x][y] != '1':
                        continue
                    q.append((x, y))
                    grid[x][y] = '2' # Mark '2' as visited.

        ans = 0

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    bfs(i, j)
                    ans += 1

        return ans

```

915. 2000.py

Path: LeetCode-main/solutions/2000. Reverse Prefix of Word/2000.py

```
class Solution:
    def reversePrefix(self, word: str, ch: str) -> str:
        i = word.find(ch) + 1
        return word[:i][::-1] + word[i:]
```

916. 2001-2.py

Path: LeetCode-main/solutions/2001. Number of Pairs of Interchangeable Rectangles/2001-2.py

```
class Solution:
    def interchangeableRectangles(self, rectangles: list[list[int]]) -> int:
        ratioCount = collections.Counter()

        def gcd(a: int, b: int) -> int:
            return a if b == 0 else gcd(b, a % b)

        for width, height in rectangles:
            d = gcd(width, height)
            ratioCount[(width // d, height // d)] += 1

        return sum(c * (c - 1) // 2 for c in ratioCount.values())
```

917. 2001.py

Path: LeetCode-main/solutions/2001. Number of Pairs of Interchangeable Rectangles/2001.py

```
class Solution:
    def interchangeableRectangles(self, rectangles: list[list[int]]) -> int:
        ratioCount = collections.Counter()

        for width, height in rectangles:
            ratioCount[width / height] += 1

        return sum(count * (count - 1) // 2
                    for count in ratioCount.values())
```

918. 2002.py

Path: LeetCode-main/solutions/2002. Maximum Product of the Length of Two Palindromic Subsequences/2002.py

```
class Solution:
    def maxProduct(self, s: str) -> int:
        ans = 0

        def isPalindrome(s: str) -> bool:
            i = 0
            j = len(s) - 1
            while i < j:
                if s[i] != s[j]:
                    return False
                i += 1
                j -= 1
            return True

        @functools.lru_cache(None)
        def dfs(i: int, s1: str, s2: str) -> None:
            nonlocal ans
            if i == len(s):
                if isPalindrome(s1) and isPalindrome(s2):
                    ans = max(ans, len(s1) * len(s2))
                return

            dfs(i + 1, s1 + s[i], s2)
            dfs(i + 1, s1, s2 + s[i])
            dfs(i + 1, s1, s2)

        dfs(0, '', '')
        return ans
```

919. 2003.py

Path: LeetCode-main/solutions/2003. Smallest Missing Genetic Value in Each Subtree/2003.py

```

class Solution:
    def smallestMissingValueSubtree(
        self,
        parents: list[int],
        nums: list[int],
    ) -> list[int]:
        n = len(parents)
        ans = [1] * n
        tree = [[] for _ in range(n)]
        seen = set()
        minMiss = 1

        for i in range(1, n):
            tree[parents[i]].append(i)

        def getNode(nums: list[int]) -> int:
            for i, num in enumerate(nums):
                if num == 1:
                    return i
            return -1

        nodeThatsOne = getNode(nums)
        if nodeThatsOne == -1:
            return ans

        u = nodeThatsOne
        prev = -1 # the u that just handled

        def dfs(u: int) -> None:
            seen.add(nums[u])
            for v in tree[u]:
                dfs(v)

        # Upward from `nodeThatsOne` to the root `u`.
        while u != -1:
            for v in tree[u]:
                if v != prev:
                    dfs(v)
            seen.add(nums[u])
            while minMiss in seen:
                minMiss += 1
            ans[u] = minMiss
            prev = u
            u = parents[u]

        return ans

```

920. 2005.py

Path: LeetCode-main/solutions/2005. Subtree Removal Game with Fibonacci Tree/2005.py

```

class Solution:
    def findGameWinner(self, n: int) -> bool:
        return n % 6 != 1

```

921. 2006.py

Path: LeetCode-main/solutions/2006. Count Number of Pairs With Absolute Difference K/2006.py

```
class Solution:
    def countKDifference(self, nums: list[int], k: int) -> int:
        count = collections.Counter(nums)
        return sum(count[i] * count[i - k] for i in range(k + 1, 101))
```

922. 2007.py

Path: LeetCode-main/solutions/2007. Find Original Array From Doubled Array/2007.py

```
class Solution:
    def findOriginalArray(self, changed: list[int]) -> list[int]:
        ans = []
        q = collections.deque()

        for num in sorted(changed):
            if q and num == q[0]:
                q.popleft()
            else:
                q.append(num * 2)
                ans.append(num)

        return [] if q else ans
```

923. 2008-2.py

Path: LeetCode-main/solutions/2008. Maximum Earnings From Taxi/2008-2.py

```
class Solution:
    def maxTaxiEarnings(self, n: int, rides: list[list[int]]) -> int:
        endToStartAndEarns = [[] for _ in range(n + 1)]
        # dp[i] := the maximum dollars you can earn starting at i
        dp = [0] * (n + 1)

        for start, end, tip in rides:
            earn = end - start + tip
            endToStartAndEarns[end].append((start, earn))

        for i in range(1, n + 1):
            dp[i] = dp[i - 1]
            for start, earn in endToStartAndEarns[i]:
                dp[i] = max(dp[i], dp[start] + earn)

        return dp[n]
```

924. 2008.py

Path: LeetCode-main/solutions/2008. Maximum Earnings From Taxi/2008.py

```

class Solution:
    def maxTaxiEarnings(self, n: int, rides: list[list[int]]) -> int:
        startToEndAndEarns = [[] for _ in range(n)]
        # dp[i] := the maximum dollars you can earn starting at i
        dp = [0] * (n + 1)

        for start, end, tip in rides:
            earn = end - start + tip
            startToEndAndEarns[start].append((end, earn))

        for i in range(n - 1, 0, -1):
            dp[i] = dp[i + 1]
            for end, earn in startToEndAndEarns[i]:
                dp[i] = max(dp[i], dp[end] + earn)

        return dp[1]

```

925. 2009.py

Path: LeetCode-main/solutions/2009. Minimum Number of Operations to Make Array Continuous/2009.py

```

class Solution:
    def minOperations(self, nums: list[int]) -> int:
        n = len(nums)
        ans = n
        nums = sorted(set(nums))

        for i, start in enumerate(nums):
            end = start + n - 1
            index = bisect_right(nums, end)
            uniqueLength = index - i
            ans = min(ans, n - uniqueLength)

        return ans

```

926. 201.py

Path: LeetCode-main/solutions/201. Bitwise AND of Numbers Range/201.py

```

class Solution:
    def rangeBitwiseAnd(self, m: int, n: int) -> int:
        return self.rangeBitwiseAnd(m >> 1, n >> 1) << 1 if m < n else m

```

927. 2011.py

Path: LeetCode-main/solutions/2011. Final Value of Variable After Performing Operations/2011.py

```

class Solution:
    def finalValueAfterOperations(self, operations: list[str]) -> int:
        return sum(op[1] == '+' or -1 for op in operations)

```

928. 2012.py

Path: LeetCode-main/solutions/2012. Sum of Beauty in the Array/2012.py

```
class Solution:
    def sumOfBeauties(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        minOfRight = [0] * (n - 1) + [nums[-1]]

        for i in range(n - 2, 1, -1):
            minOfRight[i] = min(nums[i], minOfRight[i + 1])

        maxOfLeft = nums[0]

        for i in range(1, n - 1):
            if maxOfLeft < nums[i] < minOfRight[i + 1]:
                ans += 2
            elif nums[i - 1] < nums[i] < nums[i + 1]:
                ans += 1
            maxOfLeft = max(maxOfLeft, nums[i])

        return ans
```

929. 2013.py

Path: LeetCode-main/solutions/2013. Detect Squares/2013.py

```
class DetectSquares:
    def __init__(self):
        self.pointCount = collections.Counter()

    def add(self, point: list[int]) -> None:
        self.pointCount[tuple(point)] += 1

    def count(self, point: list[int]) -> int:
        x1, y1 = point
        ans = 0
        for (x3, y3), c in self.pointCount.items():
            if x1 != x3 and abs(x1 - x3) == abs(y1 - y3):
                ans += c * self.pointCount[(x1, y3)] * self.pointCount[(x3, y1)]
        return ans
```

930. 2014.py

Path: LeetCode-main/solutions/2014. Longest Subsequence Repeated k Times/2014.py

```

class Solution:
    def longestSubsequenceRepeatedK(self, s: str, k: int) -> str:
        ans = ''
        count = [0] * 26
        possibleChars = []
        # Stores subsequences, where the length grows by 1 each time.
        q = collections.deque([''])

        for c in s:
            count[ord(c) - ord('a')] += 1

        for c in string.ascii_lowercase:
            if count[ord(c) - ord('a')] >= k:
                possibleChars.append(c)

        def isSubsequence(subseq: str, s: str, k: int) -> bool:
            i = 0 # subseq's index
            for c in s:
                if c == subseq[i]:
                    i += 1
                if i == len(subseq):
                    k -= 1
                    if k == 0:
                        return True
                    i = 0
            return False

        while q:
            currSubseq = q.popleft()
            if len(currSubseq) * k > len(s):
                return ans
            for c in possibleChars:
                newSubseq = currSubseq + c
                if isSubsequence(newSubseq, s, k):
                    q.append(newSubseq)
                    ans = newSubseq

        return ans

```

931. 2015.py

Path: LeetCode-main/solutions/2015. Average Height of Buildings in Each Segment/2015.py

```

class Solution:
    def averageHeightOfBuildings(self, buildings: list[list[int]]) -> list[list[int]]:
        ans = []
        events = []

        for start, end, height in buildings:
            events.append((start, height))
            events.append((end, -height))

        prev = 0
        count = 0
        sumHeight = 0

        for curr, height in sorted(events):
            if sumHeight > 0 and curr > prev:
                avgHeight = sumHeight // count
                if ans and ans[-1][1] == prev and avgHeight == ans[-1][2]:
                    ans[-1][1] = curr
                else:
                    ans.append([prev, curr, avgHeight])
            sumHeight += height
            count += 1 if height > 0 else -1
            prev = curr

        return ans

```

932. 2016.py

Path: LeetCode-main/solutions/2016. Maximum Difference Between Increasing Elements/2016.py

```

class Solution:
    def maximumDifference(self, nums: list[int]) -> int:
        ans = -1
        mn = nums[0]

        for i in range(len(nums)):
            if nums[i] > mn:
                ans = max(ans, nums[i] - mn)
            mn = min(mn, nums[i])

        return ans

```

933. 2017.py

Path: LeetCode-main/solutions/2017. Grid Game/2017.py

```

class Solution:
    def gridGame(self, grid: list[list[int]]) -> int:
        n = len(grid[0])
        ans = math.inf
        sumRow0 = sum(grid[0])
        sumRow1 = 0

        for i in range(n):
            sumRow0 -= grid[0][i]
            ans = min(ans, max(sumRow0, sumRow1))
            sumRow1 += grid[1][i]

        return ans

```

934. 2018.py

Path: LeetCode-main/solutions/2018. Check if Word Can Be Placed In Crossword/2018.py

```
class Solution:
    def placeWordInCrossword(self, board: list[list[str]], word: str) -> bool:
        for x in board, zip(*board):
            for row in x:
                for token in ''.join(row).split('#'):
                    for letters in word, word[::-1]:
                        if len(token) == len(letters):
                            if all(c in (' ', letter) for c, letter in zip(token, letters)):
                                return True
        return False
```

935. 2019.py

Path: LeetCode-main/solutions/2019. The Score of Students Solving Math Expression/2019.py

```
class Solution:
    def scoreOfStudents(self, s: str, answers: list[int]) -> int:
        n = len(s) // 2 + 1
        ans = 0
        func = {'+': operator.add, '*': operator.mul}
        dp = [[set() for j in range(n)] for _ in range(n)]

        for i in range(n):
            dp[i][i].add(int(s[i * 2]))

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                for k in range(i, j):
                    op = s[k * 2 + 1]
                    for a in dp[i][k]:
                        for b in dp[k + 1][j]:
                            res = func[op](a, b)
                            if res <= 1000:
                                dp[i][j].add(res)

        correctAnswer = eval(s)

        for answer, freq in collections.Counter(answers).items():
            if answer == correctAnswer:
                ans += 5 * freq
            elif answer in dp[0][n - 1]:
                ans += 2 * freq

        return ans
```

936. 202.py

Path: LeetCode-main/solutions/202. Happy Number/202.py

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def squaredSum(n: int) -> bool:
            summ = 0
            while n > 0:
                summ += pow(n % 10, 2)
                n //= 10
            return summ

        slow = squaredSum(n)
        fast = squaredSum(squaredSum(n))

        while slow != fast:
            slow = squaredSum(slow)
            fast = squaredSum(squaredSum(fast))

        return slow == 1
```

937. 2021.py

Path: LeetCode-main/solutions/2021. Brightest Position on Street/2021.py

```
from sortedcontainers import SortedDict

class Solution:
    def brightestPosition(self, lights: list[list[int]]) -> int:
        ans = math.inf
        maxBrightness = -1
        currBrightness = 0
        line = SortedDict()

        for position, rg in lights:
            start = position - rg
            end = position + rg + 1
            line[start] = line.get(start, 0) + 1
            line[end] = line.get(end, 0) - 1

        for pos, brightness in line.items():
            currBrightness += brightness
            if currBrightness > maxBrightness:
                maxBrightness = currBrightness
                ans = pos

        return ans
```

938. 2022.py

Path: LeetCode-main/solutions/2022. Convert 1D Array Into 2D Array/2022.py

```

class Solution:
    def construct2DArray(self, original: list[int],
                        m: int, n: int) -> list[list[int]]:
        if len(original) != m * n:
            return []

        ans = [[0] * n for _ in range(m)]

        for i, num in enumerate(original):
            ans[i // n][i % n] = num

        return ans

```

939. 2023.py

Path: LeetCode-main/solutions/2023. Number of Pairs of Strings With Concatenation Equal to Target/2023.py

```

class Solution:
    def numOfPairs(self, nums: list[str], target: str) -> int:
        ans = 0
        count = collections.Counter()

        for num in nums:
            k = len(num)
            if target[:k] == num:
                ans += count[target[k:]]
            if target[-k:] == num:
                ans += count[target[:-k]]
            count[num] += 1

        return ans

```

940. 2024.py

Path: LeetCode-main/solutions/2024. Maximize the Confusion of an Exam/2024.py

```

class Solution:
    def maxConsecutiveAnswers(self, answerKey: str, k: int) -> int:
        ans = 0
        maxCount = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(answerKey):
            count[c == 'T'] += 1
            maxCount = max(maxCount, count[c == 'T'])
            while maxCount + k < r - l + 1:
                count[answerKey[l] == 'T'] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

941. 2025.py

Path: LeetCode-main/solutions/2025. Maximum Number of Ways to Partition an Array/2025.py

```

class Solution:
    def waysToPartition(self, nums: list[int], k: int) -> int:
        n = len(nums)
        summ = sum(nums)
        prefix = 0
        # Count of sum(A[0..k)) - sum(A[k..n)) for k in [0..i)
        l = collections.Counter()
        # Count of sum(A[0..k)) - sum(A[k..n)) for k in [i..n)
        r = collections.Counter()

        for pivot in range(1, n):
            prefix += nums[pivot - 1]
            suffix = summ - prefix
            r[prefix - suffix] += 1

        ans = r[0]
        prefix = 0

        for num in nums:
            ans = max(ans, l[k - num] + r[num - k])
            prefix += num
            suffix = summ - prefix
            diff = prefix - suffix
            r[diff] -= 1
            l[diff] += 1

        return ans

```

942. 2027.py

Path: LeetCode-main/solutions/2027. Minimum Moves to Convert String/2027.py

```

class Solution:
    def minimumMoves(self, s: str) -> int:
        ans = 0

        i = 0
        while i < len(s):
            if s[i] == '0':
                i += 1
            else:
                ans += 1
                i += 3

        return ans

```

943. 2028.py

Path: LeetCode-main/solutions/2028. Find Missing Observations/2028.py

```
class Solution:
    def missingRolls(self, rolls: list[int], mean: int, n: int) -> list[int]:
        targetSum = (len(rolls) + n) * mean
        missingSum = targetSum - sum(rolls)
        if missingSum > n * 6 or missingSum < n:
            return []

        ans = [missingSum // n] * n
        for i in range(missingSum % n):
            ans[i] += 1

        return ans
```

944. 2029.py

Path: LeetCode-main/solutions/2029. Stone Game IX/2029.py

```
class Solution:
    def stoneGameIX(self, stones: list[int]) -> bool:
        count = collections.Counter(stone % 3 for stone in stones)
        if count[0] % 2 == 0:
            return min(count[1], count[2]) > 0
        return abs(count[1] - count[2]) > 2
```

945. 203.py

Path: LeetCode-main/solutions/203. Remove Linked List Elements/203.py

```
class Solution:
    def removeElements(self, head: ListNode, val: int) -> ListNode:
        dummy = ListNode(0, head)
        prev = dummy

        while head:
            if head.val != val:
                prev.next = head
                prev = prev.next
            head = head.next
        prev.next = None

        return dummy.next
```

946. 2030.py

Path: LeetCode-main/solutions/2030. Smallest K-Length Subsequence With Occurrences of a Letter/2030.py

```

class Solution:
    def smallestSubsequence(
        self,
        s: str,
        k: int,
        letter: str,
        repetition: int,
    ) -> str:
        stack = [] # running string
        required = repetition
        nLetters = s.count(letter)

        for i, c in enumerate(s):
            # Make sure the length is sufficient:
            # len(stack) := the length of running string
            # len(s) - i := the length of remain chars
            # -1 := we're going to pop a char
            while (stack and stack[-1] > c
                   and len(stack) + len(s) - i - 1 >= k
                   and (stack[-1] != letter or nLetters > required)):
                if stack.pop() == letter:
                    required -= 1
            if len(stack) < k:
                if c == letter:
                    stack.append(c)
                    required -= 1
                elif k - len(stack) > required:
                    stack.append(c)
            if c == letter:
                nLetters -= 1

        return ''.join(stack)

```

947. 2031.py

Path: LeetCode-main/solutions/2031. Count Subarrays With More Ones Than Zeros/2031.py

```

class FenwichTree:
    def __init__(self, n: int):
        self.n = n
        self.sums = [0] * (2 * n + 1)

    def add(self, i: int, delta: int) -> None:
        i += self.n + 1 # re-mapping
        while i < len(self.sums):
            self.sums[i] += delta
            i += i & -i

    def get(self, i: int) -> int:
        i += self.n + 1 # re-mapping
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= i & -i
        return summ

class Solution:
    def subarraysWithMoreZerosThanOnes(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        ans = 0
        prefix = 0
        tree = FenwichTree(len(nums))
        tree.add(0, 1)

        for num in nums:
            prefix += -1 if num == 0 else 1
            ans += tree.get(prefix - 1)
            ans %= MOD
            tree.add(prefix, 1)

        return ans

```

948. 2032.py

Path: LeetCode-main/solutions/2032. Two Out of Three/2032.py

```

class Solution:
    def twoOutOfThree(
        self,
        nums1: list[int],
        nums2: list[int],
        nums3: list[int],
    ) -> list[int]:
        count = collections.Counter()
        for nums in nums1, nums2, nums3:
            count.update(set(nums))
        return [i for i, c in count.items() if c >= 2]

```

949. 2033.py

Path: LeetCode-main/solutions/2033. Minimum Operations to Make a Uni-Value Grid/2033.py

```
class Solution:
    def minOperations(self, grid: list[list[int]], x: int) -> int:
        arr = sorted([a for row in grid for a in row])
        if any((a - arr[0]) % x for a in arr):
            return -1

        ans = 0

        for a in arr:
            ans += abs(a - arr[len(arr) // 2]) // x

        return ans
```

950. 2034.py

Path: LeetCode-main/solutions/2034. Stock Price Fluctuation/2034.py

```
from sortedcontainers import SortedDict

class StockPrice:
    def __init__(self):
        self.timestampToPrice = SortedDict()
        self.pricesCount = SortedDict()

    def update(self, timestamp: int, price: int) -> None:
        if timestamp in self.timestampToPrice:
            prevPrice = self.timestampToPrice[timestamp]
            self.pricesCount[prevPrice] -= 1
            if self.pricesCount[prevPrice] == 0:
                del self.pricesCount[prevPrice]
            self.timestampToPrice[timestamp] = price
            self.pricesCount[price] = self.pricesCount.get(price, 0) + 1

    def current(self) -> int:
        return self.timestampToPrice.peekitem(-1)[1]

    def maximum(self) -> int:
        return self.pricesCount.peekitem(-1)[0]

    def minimum(self) -> int:
        return self.pricesCount.peekitem(0)[0]
```

951. 2035.py

Path: LeetCode-main/solutions/2035. Partition Array Into Two Arrays to Minimize Sum Difference/2035.py


```

class Solution:
    def minimumDifference(self, nums: list[int]) -> int:
        n = len(nums) // 2
        summ = sum(nums)
        goal = summ // 2
        lNums = nums[:n]
        rNums = nums[n:]
        ans = abs(sum(lNums) - sum(rNums))
        lSums = [[] for _ in range(n + 1)]
        rSums = [[] for _ in range(n + 1)]

        def dfs(
            arr: list[int],
            i: int,
            count: int,
            path: int,
            sums: list[list[int]]
        ):
            if i == len(arr):
                sums[count].append(path)
                return
            dfs(arr, i + 1, count + 1, path + arr[i], sums)
            dfs(arr, i + 1, count, path, sums)

        dfs(lNums, 0, 0, 0, lSums)
        dfs(rNums, 0, 0, 0, rSums)

        for lCount in range(n):
            l = lSums[lCount]
            r = rSums[n - lCount]
            r.sort()
            for lSum in l:
                i = bisect_left(r, goal - lSum)
                if i < len(r):
                    sumPartOne = summ - lSum - r[i]
                    sumPartTwo = summ - sumPartOne
                    ans = min(ans, abs(sumPartOne - sumPartTwo))
                if i > 0:
                    sumPartOne = summ - lSum - r[i - 1]
                    sumPartTwo = summ - sumPartOne
                    ans = min(ans, abs(sumPartOne - sumPartTwo))

        return ans

```

952. 2036.py

Path: LeetCode-main/solutions/2036. Maximum Alternating Subarray Sum/2036.py

```

class Solution:
    def maximumAlternatingSubarraySum(self, nums: list[int]) -> int:
        ans = -math.inf
        even = 0 # the subarray sum starting from an even index
        odd = 0 # the subarray sum starting from an odd index

        for i in range(len(nums)):
            if i % 2 == 0: # Must pick.
                even += nums[i]
            else: # Start a fresh subarray or subtract `nums[i]`.
                even = max(0, even - nums[i])
            ans = max(ans, even)

        for i in range(1, len(nums)):
            if i % 2 == 1: # Must pick.
                odd += nums[i]
            else: # Start a fresh subarray or subtract `nums[i]`.
                odd = max(0, odd - nums[i])
            ans = max(ans, odd)

        return ans

```

953. 2037.py

Path: LeetCode-main/solutions/2037. Minimum Number of Moves to Seat Everyone/2037.py

```

class Solution:
    def minMovesToSeat(self, seats: list[int], students: list[int]) -> int:
        return sum(
            abs(seat - student) for seat,
            student in zip(sorted(seats),
                           sorted(students)))

```

954. 2038.py

Path: LeetCode-main/solutions/2038. Remove Colored Pieces if Both Neighbors are the Same Color/2038.py

```

class Solution:
    def winnerOfGame(self, colors: str) -> bool:
        countAAA = 0
        countBBB = 0

        for a, b, c in zip(colors, colors[1:], colors[2:]):
            if 'A' == a == b == c:
                countAAA += 1
            elif 'B' == a == b == c:
                countBBB += 1

        return countAAA > countBBB

```

955. 2039.py

Path: LeetCode-main/solutions/2039. The Time When the Network Becomes Idle/2039.py

```

class Solution:
    def networkBecomesIdle(
        self,
        edges: list[list[int]],
        patience: list[int],
    ) -> int:
        n = len(patience)
        ans = 0
        graph = [[] for _ in range(n)]
        q = collections.deque([0])
        dist = [math.inf] * n # dist[i] := the distance between i and 0
        dist[0] = 0

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        while q:
            for _ in range(len(q)):
                u = q.popleft()
                for v in graph[u]:
                    if dist[v] == math.inf:
                        dist[v] = dist[u] + 1
                        q.append(v)

        for i in range(1, n):
            numResending = (dist[i] * 2 - 1) // patience[i]
            lastResendingTime = patience[i] * numResending
            lastArrivingTime = lastResendingTime + dist[i] * 2
            ans = max(ans, lastArrivingTime)

        return ans + 1

```

956. 204.py

Path: LeetCode-main/solutions/204. Count Primes/204.py

```

class Solution:
    def countPrimes(self, n: int) -> int:
        if n <= 2:
            return 0
        return sum(self._sieveEratosthenes(n))

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime

```

957. 2040.py

Path: LeetCode-main/solutions/2040. Kth Smallest Product of Two Sorted Arrays/2040.py

```

class Solution:
    def kthSmallestProduct(
        self,
        nums1: list[int],
        nums2: list[int],
        k: int,
    ) -> int:
        A1 = [-num for num in nums1 if num < 0][::-1]
        A2 = [num for num in nums1 if num >= 0]
        B1 = [-num for num in nums2 if num < 0][::-1]
        B2 = [num for num in nums2 if num >= 0]

        negCount = len(A1) * len(B2) + len(A2) * len(B1)

        if k > negCount: # Find the (k - negCount)-th positive.
            k -= negCount
            sign = 1
        else:
            k = negCount - k + 1 # Find the (negCount - k + 1)-th abs(negative).
            sign = -1
            B1, B2 = B2, B1

        def numProductNoGreaterThan(A: list[int], B: list[int], m: int) -> int:
            ans = 0
            j = len(B) - 1
            for i in range(len(A)):
                # For each A[i], find the first index j s.t. A[i] * B[j] <= m
                # So numProductNoGreaterThan m for this row will be j + 1
                while j >= 0 and A[i] * B[j] > m:
                    j -= 1
                ans += j + 1
            return ans

        l = 0
        r = 10**10

        while l < r:
            m = (l + r) // 2
            if (numProductNoGreaterThan(A1, B1, m) +
                numProductNoGreaterThan(A2, B2, m) >= k):
                r = m
            else:
                l = m + 1

        return sign * l

```

958. 2042.py

Path: LeetCode-main/solutions/2042. Check if Numbers Are Ascending in a Sentence/2042.py

```
class Solution:
    def areNumbersAscending(self, s: str) -> bool:
        prev = 0

        for token in s.split():
            if token.isdigit():
                num = int(token)
                if num <= prev:
                    return False
                prev = num

        return True
```

959. 2043.py

Path: LeetCode-main/solutions/2043. Simple Bank System/2043.py

```
class Bank:
    def __init__(self, balance: list[int]):
        self.balance = balance

    def transfer(self, account1: int, account2: int, money: int) -> bool:
        if not self._isValid(account2):
            return False
        return self.withdraw(account1, money) and self.deposit(account2, money)

    def deposit(self, account: int, money: int) -> bool:
        if not self._isValid(account):
            return False
        self.balance[account - 1] += money
        return True

    def withdraw(self, account: int, money: int) -> bool:
        if not self._isValid(account):
            return False
        if self.balance[account - 1] < money:
            return False
        self.balance[account - 1] -= money
        return True

    def _isValid(self, account: int) -> bool:
        return 1 <= account <= len(self.balance)
```

960. 2044.py

Path: LeetCode-main/solutions/2044. Count Number of Maximum Bitwise-OR Subsets/2044.py

```

class Solution:
    def countMaxOrSubsets(self, nums: list[int]) -> int:
        ors = functools.reduce(operator.or_, nums)
        ans = 0

        def dfs(i: int, path: int) -> None:
            nonlocal ans
            if i == len(nums):
                if path == ors:
                    ans += 1
                return

            dfs(i + 1, path)
            dfs(i + 1, path | nums[i])

        dfs(0, 0)
        return ans

```

961. 2045.py

Path: LeetCode-main/solutions/2045. Second Minimum Time to Reach Destination/2045.py

```

class Solution:
    def secondMinimum(
        self,
        n: int,
        edges: list[list[int]],
        time: int,
        change: int,
    ) -> int:
        graph = [[] for _ in range(n + 1)]
        q = collections.deque([(1, 0)])
        # minTime[u][0] := the first minimum time to reach the node u
        # minTime[u][1] := the second minimum time to reach the node u
        minTime = [[math.inf] * 2 for _ in range(n + 1)]
        minTime[1][0] = 0

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        while q:
            u, prevTime = q.popleft()
            # Start from green.
            # If `numChangeSignal` is odd, now red.
            # If numChangeSignal is even -> now gree
            numChangeSignal = prevTime // change
            waitTime = (0 if numChangeSignal % 2 == 0
                        else change - (prevTime % change))
            newTime = prevTime + waitTime + time
            for v in graph[u]:
                if newTime < minTime[v][0]:
                    minTime[v][0] = newTime
                    q.append((v, newTime))
                elif minTime[v][0] < newTime < minTime[v][1]:
                    if v == n:
                        return newTime
                    minTime[v][1] = newTime
                    q.append((v, newTime))

```

962. 2046.py

Path: LeetCode-main/solutions/2046. Sort Linked List Already Sorted Using Absolute Values/2046.py

```
class Solution:
    def sortLinkedList(self, head: ListNode | None) -> ListNode | None:
        prev = head
        curr = head.next

        while curr:
            if curr.val < 0:
                prev.next = curr.next
                curr.next = head
                head = curr
                curr = prev.next
            else:
                prev = curr
                curr = curr.next

        return head
```

963. 2047-2.py

Path: LeetCode-main/solutions/2047. Number of Valid Words in a Sentence/2047-2.py

```
class Solution:
    def countValidWords(self, sentence: str) -> int:
        pattern = re.compile(r'^[a-z]*([a-z]-[a-z])?[a-z]*[!.,.]*$')
        return sum(pattern.search(token) != None for token in
                    sentence.strip().split())
```

964. 2047.py

Path: LeetCode-main/solutions/2047. Number of Valid Words in a Sentence/2047.py

```
class Solution:
    def countValidWords(self, sentence: str) -> int:
        def isValid(token: str) -> bool:
            countHyphen = 0
            for i, c in enumerate(token):
                if c.isdigit():
                    return False
                if c == '-':
                    if i == 0 or not token[i - 1].isalpha():
                        return False
                    if i == len(token) - 1 or not token[i + 1].isalpha():
                        return False
                    if countHyphen == 1:
                        return False
                    countHyphen += 1
                if c in ['!', '.', ',']:
                    if i != len(token) - 1:
                        return False
            return True

        return sum(isValid(token) for token in sentence.split())
```

965. 2048.py

Path: LeetCode-main/solutions/2048. Next Greater Numerically Balanced Number/2048.py

```
class Solution:
    def nextBeautifulNumber(self, n: int) -> int:
        def isBalance(num: int) -> bool:
            count = [0] * 10
            while num > 0:
                if num % 10 == 0:
                    return False
                count[num % 10] += 1
                num //= 10
            return all(c == i for i, c in enumerate(count) if c)

        n += 1
        while not isBalance(n):
            n += 1
        return n
```

966. 2049.py

Path: LeetCode-main/solutions/2049. Count Nodes With the Highest Score/2049.py

```
class Solution:
    def countHighestScoreNodes(self, parents: list[int]) -> int:
        tree = [[] for _ in range(len(parents))]

        for i, parent in enumerate(parents):
            if parent == -1:
                continue
            tree[parent].append(i)

        ans = 0
        maxScore = 0

        def dfs(u: int) -> int: # Returns node count
            nonlocal ans
            nonlocal maxScore
            count = 1
            score = 1
            for v in tree[u]:
                childCount = dfs(v)
                count += childCount
                score *= childCount
            score *= len(parents) - count or 1
            if score > maxScore:
                maxScore = score
                ans = 1
            elif score == maxScore:
                ans += 1
            return count

        dfs(0)
        return ans
```

967. 205.py

Path: LeetCode-main/solutions/205. Isomorphic Strings/205.py


```
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        return [*map(s.index, s)] == [*map(t.index, t)]
```

968. 2050.py

Path: LeetCode-main/solutions/2050. Parallel Courses III/2050.py

```
class Solution:
    def minimumTime(
        self,
        n: int,
        relations: list[list[int]],
        time: list[int],
    ) -> int:
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n
        dist = time.copy()

        # Build the graph.
        for a, b in relations:
            u = a - 1
            v = b - 1
            graph[u].append(v)
            inDegrees[v] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        while q:
            u = q.popleft()
            for v in graph[u]:
                dist[v] = max(dist[v], dist[u] + time[v])
                inDegrees[v] -= 1
                if inDegrees[v] == 0:
                    q.append(v)

        return max(dist)
```

969. 2052.py

Path: LeetCode-main/solutions/2052. Minimum Cost to Separate Sentence Into Rows/2052.py

```

class Solution:
    def minimumCost(self, sentence: str, k: int) -> int:
        if len(sentence) <= k:
            return 0

        words = sentence.split()

        # dp[i] := the minimum cost of the first i words
        dp = [0] * (len(words) + 1)

        for i in range(1, len(words) + 1):
            n = len(words[i - 1]) # the length of the current row
            dp[i] = dp[i - 1] + (k - n)**2
            # Gradually add words[j - 1], words[j - 2], ....
            for j in range(i - 1, 0, -1):
                n += len(words[j - 1]) + 1
                if n > k:
                    break
            dp[i] = min(dp[i], dp[j - 1] + (k - n)**2)

        lastRowLen = len(words[-1])
        i = len(words) - 2 # Greedily put words into last row

        while i > 0 and lastRowLen + len(words[i]) + 1 <= k:
            lastRowLen += len(words[i]) + 1
            i -= 1

        return min(dp[i + 1:len(words)])

```

970. 2053.py

Path: LeetCode-main/solutions/2053. Kth Distinct String in an Array/2053.py

```

class Solution:
    def kthDistinct(self, arr: list[str], k: int) -> str:
        count = collections.Counter(arr)

        for a in arr:
            if count[a] == 1:
                k -= 1
            if k == 0:
                return a

        return ''

```

971. 2054.py

Path: LeetCode-main/solutions/2054. Two Best Non-Overlapping Events/2054.py

```

class Solution:
    def maxTwoEvents(self, events: list[list[int]]) -> int:
        ans = 0
        maxValue = 0
        evts = [] # (time, isStart, value)

        for s, e, v in events:
            evts.append((s, 1, v))
            evts.append((e + 1, 0, v))

        # When two events have the same time, the one is not start will be in the front
        evts.sort()

        for _, isStart, value in evts:
            if isStart:
                ans = max(ans, value + maxValue)
            else:
                maxValue = max(maxValue, value)

        return ans

```

972. 2055-2.py

Path: LeetCode-main/solutions/2055. Plates Between Candles/2055-2.py

```

class Solution:
    def platesBetweenCandles(self, s: str, queries: list[list[int]]) -> list[int]:
        n = len(s)
        ans = []
        closestLeftCandle = [0] * n
        closestRightCandle = [0] * n
        candleCount = [0] * n # candleCount[i] := the number of candles in s[0..i]
        candle = -1
        count = 0

        for i, c in enumerate(s):
            if c == '|':
                candle = i
                count += 1
            closestLeftCandle[i] = candle
            candleCount[i] = count

        candle = -1
        for i, c in reversed(list(enumerate(s))):
            if c == '|':
                candle = i
            closestRightCandle[i] = candle

        for left, right in queries:
            l = closestRightCandle[left]
            r = closestLeftCandle[right]
            if l == -1 or r == -1 or l > r:
                ans.append(0)
            else:
                lengthBetweenCandles = r - l + 1
                numCandles = candleCount[r] - candleCount[l] + 1
                ans.append(lengthBetweenCandles - numCandles)

        return ans

```

973. 2055.py

Path: LeetCode-main/solutions/2055. Plates Between Candles/2055.py

```
class Solution:
    def platesBetweenCandles(self, s: str, queries: list[list[int]]) -> list[int]:
        ans = []
        indices = [i for i, c in enumerate(s) if c == '|'] # indices of '|'

        for left, right in queries:
            l = bisect.bisect_left(indices, left)
            r = bisect.bisect_right(indices, right) - 1
            if l < r:
                lengthBetweenCandles = indices[r] - indices[l] + 1
                numCandles = r - l + 1
                ans.append(lengthBetweenCandles - numCandles)
            else:
                ans.append(0)

        return ans
```

974. 2056.py

Path: LeetCode-main/solutions/2056. Number of Valid Move Combinations On Chessboard/2056.py

```

class Solution:
    def countCombinations(
        self,
        pieces: list[str],
        positions: list[list[int]],
    ) -> int:
        n = len(pieces)
        moves = {"rook": [(1, 0), (-1, 0), (0, 1), (0, -1)],
                  "bishop": [(1, 1), (1, -1), (-1, 1), (-1, -1)],
                  "queen": [(1, 0), (-1, 0), (0, 1), (0, -1), (1, 1), (1, -1), (-1, 1), (-1, -1)]}
        hashedBoards = set()

        def getHash(board: list[list[int]]) -> Tuple:
            return tuple([tuple(pos) for pos in board])

        def dfs(
            board: list[list[int]],
            pieceMoves: list[tuple[int, int]],
            activeMask: int,
        ) -> None:
            """Performs a depth-first search to explore all possible board states."""
            if activeMask == 0:
                return
            hashedBoards.add(getHash(board))

            for nextActiveMask in range(1, 1 << n):
                if activeMask & nextActiveMask != nextActiveMask:
                    continue

                # Copy the board.
                nextBoard = [pos.copy() for pos in board]

                # Move the pieces that are active in this turn.
                for i in range(n):
                    if nextActiveMask >> i & 1:
                        nextBoard[i][0] += pieceMoves[i][0]
                        nextBoard[i][1] += pieceMoves[i][1]

                # No two or more pieces occupy the same square.
                if len(set(getHash(nextBoard))) < n:
                    continue

                # Every piece needs to be in the boundary.
                if all(1 <= x <= 8 and 1 <= y <= 8 for x, y in nextBoard):
                    dfs(nextBoard, pieceMoves, nextActiveMask)

            for pieceMoves in itertools.product(*(moves[piece] for piece in pieces)):
                dfs(positions, pieceMoves, (1 << n) - 1)

        return len(hashedBoards)

```

975. 2057.py

Path: LeetCode-main/solutions/2057. Smallest Index With Equal Value/2057.py

```

class Solution:
    def smallestEqual(self, nums: list[int]) -> int:
        return next((i for i, num in enumerate(nums) if i % 10 == num), -1)

```

976. 2058.py

Path: LeetCode-main/solutions/2058. Find the Minimum and Maximum Number of Nodes Between Critical Points/2058.py

```
class Solution:
    def nodesBetweenCriticalPoints(self, head: ListNode | None) -> list[int]:
        minDistance = math.inf
        firstMaIndex = -1
        prevMaIndex = -1
        index = 1
        prev = head # Point to the index 0.
        curr = head.next # Point to the index 1.

        while curr.next:
            if (curr.val > prev.val and curr.val > curr.next.val or
                curr.val < prev.val and curr.val < curr.next.val):
                if firstMaIndex == -1: # Only assign once.
                    firstMaIndex = index
                if prevMaIndex != -1:
                    minDistance = min(minDistance, index - prevMaIndex)
                    prevMaIndex = index
                prev = curr
                curr = curr.next
                index += 1

        if minDistance == math.inf:
            return [-1, -1]
        return [minDistance, prevMaIndex - firstMaIndex]
```

977. 2059.py

Path: LeetCode-main/solutions/2059. Minimum Operations to Convert Number/2059.py

```
class Solution:
    def minimumOperations(self, nums: list[int], start: int, goal: int) -> int:
        q = collections.deque([start])
        seen = {start}

        step = 1
        while q:
            for _ in range(len(q)):
                x = q.popleft()
                for num in nums:
                    for res in (x + num, x - num, x ^ num):
                        if res == goal:
                            return step
                        if res < 0 or res > 1000 or res in seen:
                            continue
                        seen.add(res)
                        q.append(res)
            step += 1

        return -1
```

978. 206-2.py

Path: LeetCode-main/solutions/206. Reverse Linked List/206-2.py

```
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        prev = None
        curr = head

        while curr:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
```

979. 206.py

Path: LeetCode-main/solutions/206. Reverse Linked List/206.py

```
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        if not head or not head.next:
            return head

        newHead = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return newHead
```

980. 2060.py

Path: LeetCode-main/solutions/2060. Check if an Original String Exists Given Two Encoded Strings/2060.py

```

class Solution:
    def possiblyEquals(self, s1: str, s2: str) -> bool:
        def getNums(s: str) -> set[int]:
            nums = {int(s)}
            for i in range(1, len(s)):
                nums |= {x + y for x in getNums(s[:i]) for y in getNums(s[i:])}
            return nums

        def getNextLetterIndex(s: str, i: int) -> int:
            j = i
            while j < len(s) and s[j].isdigit():
                j += 1
            return j

        @functools.lru_cache(None)
        def dp(i: int, j: int, paddingDiff: int) -> bool:
            """
            Returns True if s1[i..n) matches s2[j..n), accounting for the padding
            difference. Here, `paddingDiff` represents the signed padding. A positive
            `paddingDiff` indicates that s1 has an additional number of offset bytes
            compared to s2.
            """
            if i == len(s1) and j == len(s2):
                return paddingDiff == 0
            # Add padding on s1.
            if i < len(s1) and s1[i].isdigit():
                nextLetterIndex = getNextLetterIndex(s1, i)
                for num in getNums(s1[i:nextLetterIndex]):
                    if dp(nextLetterIndex, j, paddingDiff + num):
                        return True
            # Add padding on s2.
            elif j < len(s2) and s2[j].isdigit():
                nextLetterIndex = getNextLetterIndex(s2, j)
                for num in getNums(s2[j:nextLetterIndex]):
                    if dp(i, nextLetterIndex, paddingDiff - num):
                        return True
            # `s1` has more padding, so j needs to catch up.
            elif paddingDiff > 0:
                if j < len(s2):
                    return dp(i, j + 1, paddingDiff - 1)
            # `s2` has more padding, so i needs to catch up.
            elif paddingDiff < 0:
                if i < len(s1):
                    return dp(i + 1, j, paddingDiff + 1)
            # There's no padding difference, so consume the next letter.
            else: # paddingDiff == 0
                if i < len(s1) and j < len(s2) and s1[i] == s2[j]:
                    return dp(i + 1, j + 1, 0)
            return False

        return dp(0, 0, 0)

```

981. 2061.py

Path: LeetCode-main/solutions/2061. Number of Spaces Cleaning Robot Cleaned/2061.py


```
class Solution:
    def numberOfCleanRooms(self, room: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(room)
        n = len(room[0])
        ans = 1
        i = 0
        j = 0
        state = 0 # 0 := right, 1 := down, 2 := left, 3 := up
        seen = {(i, j, state)}
        room[i][j] = 2 # 2 := cleaned

        while True:
            x = i + DIRS[state][0]
            y = j + DIRS[state][1]
            if x < 0 or x == m or y < 0 or y == n or room[x][y] == 1:
                # Turn 90 degrees clockwise.
                state = (state + 1) % 4
            else:
                # Walk to (x, y).
                if room[x][y] == 0:
                    ans += 1
                    room[x][y] = 2
                i = x
                j = y
            if (x, y, state) in seen:
                return ans
            seen.add((x, y, state))
```

982. 2062.py

Path: LeetCode-main/solutions/2062. Count Vowel Substrings of a String/2062.py

```

class Solution:
    def countVowelSubstrings(self, word: str) -> int:
        VOWELS = 'aeiou'

        def countVowelSubstringsAtMost(goal: int) -> int:
            ans = 0
            k = goal
            count = collections.Counter()

            l = 0
            for r, c in enumerate(word):
                if c not in VOWELS: # Fresh start.
                    l = r + 1
                    k = goal
                    count = collections.Counter()
                    continue
                count[c] += 1
                if count[c] == 1:
                    k -= 1
                while k == -1:
                    count[word[l]] -= 1
                    if count[word[l]] == 0:
                        k += 1
                    l += 1
                ans += r - l + 1 # s[l..r], s[l + 1..r], ..., s[r]

            return ans

        return countVowelSubstringsAtMost(5) - countVowelSubstringsAtMost(4)

```

983. 2063-2.py

Path: LeetCode-main/solutions/2063. Vowels of All Substrings/2063-2.py

```

class Solution:
    def countVowels(self, word: str) -> int:
        return sum((i + 1) * (len(word) - i)
                    for i, c in enumerate(word)
                    if c in 'aeiou')

```

984. 2063.py

Path: LeetCode-main/solutions/2063. Vowels of All Substrings/2063.py

```

class Solution:
    def countVowels(self, word: str) -> int:
        # dp[i] := the sum of the number of vowels of word[0..i), ...,
        # word[i - 1..i)
        dp = [0] * (len(word) + 1)

        for i, c in enumerate(word):
            dp[i + 1] = dp[i]
            if c in 'aeiou':
                dp[i + 1] += i + 1

        return sum(dp)

```

985. 2064.py

Path: LeetCode-main/solutions/2064. Minimized Maximum of Products Distributed to Any Store/2064.py

```
class Solution:
    def minimizedMaximum(self, n: int, quantities: list[int]) -> int:
        l = 1
        r = max(quantities)

        def numStores(m: int) -> int:
            return sum((q - 1) // m + 1 for q in quantities)

        while l < r:
            m = (l + r) // 2
            if numStores(m) <= n:
                r = m
            else:
                l = m + 1

        return l
```

986. 2065-2.py

Path: LeetCode-main/solutions/2065. Maximum Path Quality of a Graph/2065-2.py

```
class Solution:
    def maximalPathQuality(
        self,
        values: list[int],
        edges: list[list[int]],
        maxTime: int,
    ) -> int:
        ans = 0
        graph = [[] for _ in range(len(values))]
        # (node, quality, remainingTime, seen)
        q = collections.deque([(0, values[0], maxTime, {0})])

        for u, v, time in edges:
            graph[u].append((v, time))
            graph[v].append((u, time))

        while q:
            u, quality, remainingTime, seen = q.popleft()
            if u == 0:
                ans = max(ans, quality)
            for v, time in graph[u]:
                if time <= remainingTime:
                    q.append(
                        (v, quality + values[v] * (v not in seen),
                         remainingTime - time, seen | set([v])))

        return ans
```

987. 2065.py

Path: LeetCode-main/solutions/2065. Maximum Path Quality of a Graph/2065.py

```

class Solution:
    def maximalPathQuality(
        self,
        values: list[int],
        edges: list[list[int]],
        maxTime: int,
    ) -> int:
        n = len(values)
        ans = 0
        graph = [[] for _ in range(n)]
        seen = [0] * n
        seen[0] = 1

        for u, v, time in edges:
            graph[u].append((v, time))
            graph[v].append((u, time))

        def dfs(u: int, quality: int, remainingTime: int):
            nonlocal ans
            if u == 0:
                ans = max(ans, quality)
            for v, time in graph[u]:
                if time > remainingTime:
                    continue
                newQuality = quality + values[v] * (seen[v] == 0)
                seen[v] += 1
                dfs(v, newQuality, remainingTime - time)
                seen[v] -= 1

        dfs(0, values[0], maxTime)
        return ans

```

988. 2067.py

Path: LeetCode-main/solutions/2067. Number of Equal Count Substrings/2067.py

```

class Solution:
    def equalCountSubstrings(self, s: str, count: int) -> int:
        maxUnique = len(set(s))
        ans = 0

        for unique in range(1, maxUnique + 1):
            windowSize = unique * count
            lettersCount = collections.Counter()
            uniqueCount = 0
            for i, c in enumerate(s):
                lettersCount[c] += 1
                if lettersCount[c] == count:
                    uniqueCount += 1
                if i >= windowSize:
                    lettersCount[s[i - windowSize]] -= 1
                    if lettersCount[s[i - windowSize]] == count - 1:
                        uniqueCount -= 1
                ans += uniqueCount == unique

        return ans

```

989. 2068.py

Path: LeetCode-main/solutions/2068. Check Whether Two Strings are Almost Equivalent/2068.py

```
class Solution:
    def checkAlmostEquivalent(self, word1: str, word2: str) -> bool:
        count = collections.Counter(word1)
        count.subtract(collections.Counter(word2))
        return all(abs(freq) <= 3 for freq in count.values())
```

990. 2069.py

Path: LeetCode-main/solutions/2069. Walking Robot Simulation II/2069.py

```
class Robot:
    def __init__(self, width: int, height: int):
        self.isOrigin = True
        self.i = 0
        self.pos = (((0, 0), 'South')) +
            (((i, 0), 'East') for i in range(1, width)) +
            (((width - 1, j), 'North') for j in range(1, height)) +
            (((i, height - 1), 'West') for i in range(width - 2, -1, -1)) +
            (((0, j), 'South') for j in range(height - 2, 0, -1))

    def step(self, num: int) -> None:
        self.isOrigin = False
        self.i = (self.i + num) % len(self.pos)

    def getPos(self) -> list[int]:
        return self.pos[self.i][0]

    def getDir(self) -> str:
        return 'East' if self.isOrigin else self.pos[self.i][1]
```

991. 207.py

Path: LeetCode-main/solutions/207. Course Schedule/207.py

```

from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def canFinish(self, numCourses: int, prerequisites: list[list[int]]) -> bool:
        graph = [[] for _ in range(numCourses)]
        states = [State.INIT] * numCourses

        for v, u in prerequisites:
            graph[u].append(v)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            if any(hasCycle(v) for v in graph[u]):
                return True
            states[u] = State.VISITED
            return False

        return not any(hasCycle(i) for i in range(numCourses))

```

992. 2070.py

Path: LeetCode-main/solutions/2070. Most Beautiful Item for Each Query/2070.py

```

class Solution:
    def maximumBeauty(
        self,
        items: list[list[int]],
        queries: list[int],
    ) -> list[int]:
        prices, beauties = zip(*sorted(items))
        maxBeautySoFar = [0] * (len(beauties) + 1)

        for i, beauty in enumerate(beauties):
            maxBeautySoFar[i + 1] = max(maxBeautySoFar[i], beauty)

        return [maxBeautySoFar[bisect_right(prices, query)] for query in queries]

```

993. 2071.py

Path: LeetCode-main/solutions/2071. Maximum Number of Tasks You Can Assign/2071.py

```

from sortedcontainers import SortedList

class Solution:
    def maxTaskAssign(
        self,
        tasks: list[int],
        workers: list[int],
        pills: int,
        strength: int,
    ) -> int:
        tasks.sort()
        workers.sort()

        def canComplete(k: int, pillsLeft: int) -> bool:
            """Returns True if we can finish k tasks."""
            # k strongest workers
            sortedWorkers = SortedList(workers[-k:])

            # Out of the k smallest tasks, start from the biggest one.
            for i in reversed(range(k)):
                # Find the first worker that has strength >= tasks[i].
                index = sortedWorkers.bisect_left(tasks[i])
                if index < len(sortedWorkers):
                    sortedWorkers.pop(index)
                elif pillsLeft > 0:
                    # Find the first worker that has strength >= tasks[i] - strength.
                    index = sortedWorkers.bisect_left(tasks[i] - strength)
                    if index < len(sortedWorkers):
                        sortedWorkers.pop(index)
                        pillsLeft -= 1
                else:
                    return False
            else:
                return False

            return True

        ans = 0
        l = 0
        r = min(len(tasks), len(workers))

        while l <= r:
            m = (l + r) // 2
            if canComplete(m, pills):
                ans = m
                l = m + 1
            else:
                r = m - 1

        return ans

```

994. 2073.py

Path: LeetCode-main/solutions/2073. Time Needed to Buy Tickets/2073.py

```

class Solution:
    def timeRequiredToBuy(self, tickets: list[int], k: int) -> int:
        ans = 0

        for i, ticket in enumerate(tickets):
            if i <= k:
                ans += min(ticket, tickets[k])
            else:
                ans += min(ticket, tickets[k] - 1)

        return ans

```

995. 2074.py

Path: LeetCode-main/solutions/2074. Reverse Nodes in Even Length Groups/2074.py

```

class Solution:
    def reverseEvenLengthGroups(self, head: ListNode | None) -> ListNode | None:
        # prev -> (head -> ... -> tail) -> next -> ...
        dummy = ListNode(0, head)
        prev = dummy
        tail = head
        next = head.next
        groupLength = 1

        def getTailAndLength(head: ListNode | None, groupLength: int) -> tuple[ListNode | None, int]:
            length = 1
            tail = head
            while length < groupLength and tail.next:
                tail = tail.next
                length += 1
            return tail, length

        def reverse(head: ListNode | None) -> ListNode | None:
            prev = None
            while head:
                next = head.next
                head.next = prev
                prev = head
                head = next
            return prev

        while True:
            if groupLength % 2 == 1:
                prev.next = head
                prev = tail
            else:
                tail.next = None
                prev.next = reverse(head)
                # Prev -> (tail -> ... -> head) -> next -> ...
                head.next = next
                prev = head
            if not next:
                break
            head = next
            tail, length = getTailAndLength(head, groupLength + 1)
            next = tail.next
            groupLength = length

        return dummy.next

```


996. 2075-2.py

Path: LeetCode-main/solutions/2075. Decode the Slanted Ciphertext/2075-2.py

```
class Solution:
    def decodeCiphertext(self, encodedText: str, rows: int) -> str:
        n = len(encodedText)
        cols = n // rows

        ans = []

        for j in range(cols):
            for i in range(j, n, cols + 1):
                ans.append(encodedText[i])

        return ''.join(ans).rstrip()
```

997. 2075.py

Path: LeetCode-main/solutions/2075. Decode the Slanted Ciphertext/2075.py

```
class Solution:
    def decodeCiphertext(self, encodedText: str, rows: int) -> str:
        n = len(encodedText)
        cols = n // rows

        ans = []
        matrix = [[' ' * cols for _ in range(rows)]

        for i in range(rows):
            for j in range(cols):
                matrix[i][j] = encodedText[i * cols + j]

        for col in range(cols):
            i = 0
            j = col
            while i < rows and j < cols:
                ans.append(matrix[i][j])
                i += 1
                j += 1

        return ''.join(ans).rstrip()
```

998. 2076.py

Path: LeetCode-main/solutions/2076. Process Restricted Friend Requests/2076.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

```

```

class Solution:
    def friendRequests(
        self,
        n: int,
        restrictions: list[list[int]],
        requests: list[list[int]],
    ) -> list[bool]:
        ans = []
        uf = UnionFind(n)

        for u, v in requests:
            pu = uf.find(u)
            pv = uf.find(v)
            isValid = True
            if pu != pv:
                for x, y in restrictions:
                    px = uf.find(x)
                    py = uf.find(y)
                    if (pu, pv) in [(px, py), (py, px)]:
                        isValid = False
                        break
            ans.append(isValid)
            if isValid:
                uf.unionByRank(pu, pv)

        return ans

```

999. 2077.py

Path: LeetCode-main/solutions/2077. Paths in Maze That Lead to Same Room/2077.py

```

class Solution:
    def numberOfPaths(self, n: int, corridors: list[list[int]]) -> int:
        ans = 0
        graph = [[False] * 1001 for _ in range(n + 1)]

        for u, v in corridors:
            graph[u][v] = True
            graph[v][u] = True

        for u, v in corridors:
            for i in range(1, n + 1):
                if graph[u][i] and graph[i][v]:
                    ans += 1

        return ans // 3

```

1000. 2078.py

Path: LeetCode-main/solutions/2078. Two Furthest Houses With Different Colors/2078.py

```

class Solution:
    def maxDistance(self, colors: list[int]) -> int:
        # The maximum distance always includes either the first or the last house.
        n = len(colors)
        i = 0 # the leftmost index, where colors[i] != colors[-1]
        j = n - 1 # the rightmost index, where colors[j] != colors[0]
        while colors[i] == colors[-1]:
            i += 1
        while colors[j] == colors[0]:
            j -= 1
        return max(n - 1 - i, j)

```

1001. 2079.py

Path: LeetCode-main/solutions/2079. Watering Plants/2079.py

```

class Solution:
    def wateringPlants(self, plants: list[int], capacity: int) -> int:
        ans = 0
        currCapacity = 0

        for i, plant in enumerate(plants):
            if currCapacity + plant <= capacity:
                currCapacity += plant
            else:
                currCapacity = plant # Reset
                ans += i * 2

        return ans + len(plants)

```

1002. 208.py

Path: LeetCode-main/solutions/208. Implement Trie (Prefix Tree)/208.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node: TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

    def search(self, word: str) -> bool:
        node = self._find(word)
        return node is not None and node.isWord

    def startsWith(self, prefix: str) -> bool:
        return self._find(prefix) is not None

    def _find(self, prefix: str) -> TrieNode | None:
        node: TrieNode = self.root
        for c in prefix:
            if c not in node.children:
                return None
            node = node.children[c]
        return node

```

1003. 2080.py

Path: LeetCode-main/solutions/2080. Range Frequency Queries/2080.py

```

class RangeFreqQuery:
    def __init__(self, arr: list[int]):
        self.valueToIndices = collections.defaultdict(list)
        for i, a in enumerate(arr):
            self.valueToIndices[a].append(i)

    def query(self, left: int, right: int, value: int) -> int:
        indices = self.valueToIndices[value]
        i = bisect_left(indices, left)
        j = bisect_right(indices, right)
        return j - i

```

1004. 2081.py

Path: LeetCode-main/solutions/2081. Sum of k-Mirror Numbers/2081.py

```

class Solution:
    def kMirror(self, k: int, n: int) -> int:
        ans = 0
        A = ['0']

        def nextKMirror(A: list[str]) -> list[str]:
            for i in range(len(A) // 2, len(A)):
                nextNum = int(A[i]) + 1
                if nextNum < k:
                    A[i] = str(nextNum)
                    A[~i] = str(nextNum)
                    for j in range(len(A) // 2, i):
                        A[j] = '0'
                        A[~j] = '0'
                    return A
            return ['1'] + ['0'] * (len(A) - 1) + ['1']

        for _ in range(n):
            while True:
                A = nextKMirror(A)
                num = int(''.join(A), k)
                if str(num)[::-1] == str(num):
                    break
            ans += num

        return ans

```

1005. 2083.py

Path: LeetCode-main/solutions/2083. Substrings That Begin and End With the Same Letter/2083.py

```

class Solution:
    def numberOfSubstrings(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        for c in s:
            ans += count[c] + 1
            count[c] += 1

        return ans

```

1006. 2085.py

Path: LeetCode-main/solutions/2085. Count Common Words With One Occurrence/2085.py

```

class Solution:
    def countWords(self, words1: list[str], words2: list[str]) -> int:
        count = collections.Counter(words1)

        for word in words2:
            if word in count and count[word] < 2:
                count[word] -= 1

        return sum(value == 0 for value in count.values())

```

1007. 2086.py

Path: LeetCode-main/solutions/2086. Minimum Number of Buckets Required to Collect Rainwater from Houses/2086.py

```
class Solution:
    def minimumBuckets(self, street: str) -> int:
        A = list(street)

        for i, c in enumerate(A):
            if c == 'H':
                if i > 0 and A[i - 1] == 'B':
                    continue
                if i + 1 < len(A) and A[i + 1] == '.':
                    # Always prefer place a bucket in (i + 1) because it enhances the
                    # possibility to collect the upcoming houses.
                    A[i + 1] = 'B'
                elif i > 0 and A[i - 1] == '.':
                    A[i - 1] = 'B'
                else:
                    return -1

        return A.count('B')
```

1008. 2087.py

Path: LeetCode-main/solutions/2087. Minimum Cost Homecoming of a Robot in a Grid/2087.py

```
class Solution:
    def minCost(
        self,
        startPos: list[int],
        homePos: list[int],
        rowCosts: list[int],
        colCosts: list[int],
    ) -> int:
        ans = 0
        i, j = startPos
        x, y = homePos

        while i != x:
            i += 1 if i < x else -1
            ans += rowCosts[i]

        while j != y:
            j += 1 if j < y else -1
            ans += colCosts[j]

        return ans
```

1009. 2088.py

Path: LeetCode-main/solutions/2088. Count Fertile Pyramids in a Land/2088.py

```

class Solution:
    def countPyramids(self, grid: list[list[int]]) -> int:
        # dp[i][j] := the maximum height of the pyramid for which it is the apex
        def count(dp: list[list[int]]) -> int:
            ans = 0
            for i in range(len(dp) - 2, -1, -1):
                for j in range(1, len(dp[0]) - 1):
                    if dp[i][j] == 1:
                        dp[i][j] = min(dp[i + 1][j - 1],
                                         dp[i + 1][j],
                                         dp[i + 1][j + 1]) + 1
                    ans += dp[i][j] - 1
            return ans

        return count(deepcopy(grid)[::-1]) + count(grid)

```

1010. 2089.py

Path: LeetCode-main/solutions/2089. Find Target Indices After Sorting Array/2089.py

```

class Solution:
    def targetIndices(self, nums: list[int], target: int) -> list[int]:
        count = nums.count(target)
        lessThan = sum(num < target for num in nums)
        return [i for i in range(lessThan, lessThan + count)]

```

1011. 209.py

Path: LeetCode-main/solutions/209. Minimum Size Subarray Sum/209.py

```

class Solution:
    def minSubArrayLen(self, target: int, nums: list[int]) -> int:
        ans = math.inf
        summ = 0
        j = 0

        for i, num in enumerate(nums):
            summ += num
            while summ >= target:
                ans = min(ans, i - j + 1)
                summ -= nums[j]
                j += 1

        return 0 if ans == math.inf else ans

```

1012. 2090.py

Path: LeetCode-main/solutions/2090. K Radius Subarray Averages/2090.py

```
class Solution:
    def getAverages(self, nums: list[int], k: int) -> list[int]:
        n = len(nums)
        size = 2 * k + 1
        ans = [-1] * n
        if size > n:
            return ans

        summ = sum(nums[:size])

        for i in range(k, n - k):
            ans[i] = summ // size
            if i + k + 1 < n:
                summ += nums[i + k + 1] - nums[i - k]

        return ans
```

1013. 2091.py

Path: LeetCode-main/solutions/2091. Removing Minimum and Maximum From Array/2091.py

```
class Solution:
    def minimumDeletions(self, nums: list[int]) -> int:
        n = len(nums)
        a = nums.index(min(nums))
        b = nums.index(max(nums))
        if a > b:
            a, b = b, a
        return min(a + 1 + n - b, b + 1, n - a)
```

1014. 2092.py

Path: LeetCode-main/solutions/2092. Find All People With Secret/2092.py


```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def connected(self, u: int, v: int) -> bool:
        return self._find(self.id[u]) == self._find(self.id[v])

    def reset(self, u: int) -> None:
        self.id[u] = u

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findAllPeople(
        self,
        n: int,
        meetings: list[list[int]],
        firstPerson: int,
    ) -> list[int]:
        uf = UnionFind(n)
        timeToPairs = collections.defaultdict(list)

        uf.unionByRank(0, firstPerson)

        for x, y, time in meetings:
            timeToPairs[time].append((x, y))

        for _, pairs in sorted(timeToPairs.items(), key=lambda x: x[0]):
            peopleUnioned = set()
            for x, y in pairs:
                uf.unionByRank(x, y)
                peopleUnioned.add(x)
                peopleUnioned.add(y)
            for person in peopleUnioned:
                if not uf.connected(person, 0):
                    uf.reset(person)

        return [i for i in range(n) if uf.connected(i, 0)]

```

1015. 2093.py

Path: LeetCode-main/solutions/2093. Minimum Cost to Reach City With Discounts/2093.py

```

class Solution:
    def minimumCost(
        self,
        n: int,
        highways: list[list[int]],
        discounts: int,
    ) -> int:
        graph = [[] for _ in range(n)]
        minHeap = [(0, 0, discounts)] # (d, u, leftDiscounts)
        minDiscounts = {}

        for city1, city2, toll in highways:
            graph[city1].append((city2, toll))
            graph[city2].append((city1, toll))

        while minHeap:
            d, u, leftDiscounts = heapq.heappop(minHeap)
            if u == n - 1:
                return d
            if u in minDiscounts and minDiscounts[u] >= leftDiscounts:
                continue
            minDiscounts[u] = leftDiscounts
            for v, w in graph[u]:
                heapq.heappush(minHeap, (d + w, v, leftDiscounts))
                if leftDiscounts > 0:
                    heapq.heappush(minHeap, (d + w // 2, v, leftDiscounts - 1))

        return -1

```

1016. 2094.py

Path: LeetCode-main/solutions/2094. Finding 3-Digit Even Numbers/2094.py

```

class Solution:
    def findEvenNumbers(self, digits: list[int]) -> list[int]:
        ans = []
        count = collections.Counter(digits)

        # Try to construct `abc`.
        for a in range(1, 10):
            for b in range(0, 10):
                for c in range(0, 9, 2):
                    if count[a] > 0 and count[b] > (
                        b == a) and count[c] > (
                            c == a) + (
                                c == b):
                        ans.append(a * 100 + b * 10 + c)

        return ans

```

1017. 2095.py

Path: LeetCode-main/solutions/2095. Delete the Middle Node of a Linked List/2095.py

```

class Solution:
    def deleteMiddle(self, head: ListNode | None) -> ListNode | None:
        dummy = ListNode(0, head)
        slow = dummy
        fast = dummy

        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next

        # Delete the middle node.
        slow.next = slow.next.next
        return dummy.next

```

1018. 2096-2.py

Path: [LeetCode-main/solutions/2096. Step-By-Step Directions From a Binary Tree Node to Another/2096-2.py](#)

```

class Solution:
    def getDirections(
        self,
        root: TreeNode | None,
        startValue: int,
        destValue: int,
    ) -> str:
        def dfs(root: TreeNode | None, val: int, path: list[str]) -> bool:
            """Builds the string in reverse order to avoid creating a new copy."""
            if root.val == val:
                return True
            if root.left and dfs(root.left, val, path):
                path.append('L')
            elif root.right and dfs(root.right, val, path):
                path.append('R')
            return len(path) > 0

        pathToStart = []
        pathToDest = []

        dfs(root, startValue, pathToStart)
        dfs(root, destValue, pathToDest)

        while pathToStart and pathToDest and pathToStart[-1] == pathToDest[-1]:
            pathToStart.pop()
            pathToDest.pop()

        return 'U' * len(pathToStart) + ''.join(reversed(pathToDest))

```

1019. 2096.py

Path: [LeetCode-main/solutions/2096. Step-By-Step Directions From a Binary Tree Node to Another/2096.py](#)

```
class Solution:
    def getDirections(
        self,
        root: TreeNode | None,
        startValue: int,
        destValue: int,
    ) -> str:
        def lca(root: TreeNode | None) -> TreeNode | None:
            if not root or root.val in (startValue, destValue):
                return root
            left = lca(root.left)
            right = lca(root.right)
            if left and right:
                return root
            return left or right

        def dfs(root: TreeNode | None, path: list[str]) -> None:
            if not root:
                return
            if root.val == startValue:
                self.pathToStart = ''.join(path)
            if root.val == destValue:
                self.pathToDest = ''.join(path)
            path.append('L')
            dfs(root.left, path)
            path.pop()
            path.append('R')
            dfs(root.right, path)
            path.pop()

        dfs(lca(root), []) # Only this subtree matters.
        return 'U' * len(self.pathToStart) + ''.join(self.pathToDest)
```

1020. 2097.py

Path: LeetCode-main/solutions/2097. Valid Arrangement of Pairs/2097.py

```
class Solution:
    def validArrangement(self, pairs: list[list[int]]) -> list[list[int]]:
        ans = []
        graph = collections.defaultdict(list)
        outDegree = collections.Counter()
        inDegrees = collections.Counter()

        for start, end in pairs:
            graph[start].append(end)
            outDegree[start] += 1
            inDegrees[end] += 1

        def getStartNode() -> int:
            for u in graph.keys():
                if outDegree[u] - inDegrees[u] == 1:
                    return u
            return pairs[0][0] # Arbitrarily choose a node.

        def euler(u: int) -> None:
            stack = graph[u]
            while stack:
                v = stack.pop()
                euler(v)
            ans.append([u, v])

        euler(getStartNode())
        return ans[::-1]
```

1021. 2098.py

Path: LeetCode-main/solutions/2098. Subsequence of Size K With the Largest Even Sum/2098.py

```

class Solution:
    def largestEvenSum(self, nums: list[int], k: int) -> int:
        nums.sort()
        summ = sum(nums[-k:])
        if summ % 2 == 0:
            return summ

        minOdd = -1
        minEven = -1
        maxOdd = -1
        maxEven = -1

        for i in range(len(nums) - 1, len(nums) - k - 1, -1):
            if nums[i] & 1:
                minOdd = nums[i]
            else:
                minEven = nums[i]

        for i in range(len(nums) - k):
            if nums[i] & 1:
                maxOdd = nums[i]
            else:
                maxEven = nums[i]

        ans = -1

        if maxEven >= 0 and minOdd >= 0:
            ans = max(ans, summ + maxEven - minOdd)
        if maxOdd >= 0 and minEven >= 0:
            ans = max(ans, summ + maxOdd - minEven)
        return ans

```

1022. 2099.py

Path: LeetCode-main/solutions/2099. Find Subsequence of Length K With the Largest Sum/2099.py

```

class Solution:
    def maxSubsequence(self, nums: list[int], k: int) -> list[int]:
        ans = []
        threshold = sorted(nums)[-k]
        larger = sum(num > threshold for num in nums)
        equal = k - larger

        for num in nums:
            if num > threshold:
                ans.append(num)
            elif num == threshold and equal:
                ans.append(num)
                equal -= 1

        return ans

```

1023. 21.py

Path: LeetCode-main/solutions/21. Merge Two Sorted Lists/21.py

```
class Solution:
    def mergeTwoLists(
        self,
        list1: ListNode | None,
        list2: ListNode | None,
    ) -> ListNode | None:
        if not list1 or not list2:
            return list1 if list1 else list2
        if list1.val > list2.val:
            list1, list2 = list2, list1
        list1.next = self.mergeTwoLists(list1.next, list2)
        return list1
```

1024. 210-2.py

Path: LeetCode-main/solutions/210. Course Schedule II/210-2.py

```
class Solution:
    def findOrder(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
    ) -> list[int]:
        ans = []
        graph = [[] for _ in range(numCourses)]
        inDegrees = [0] * numCourses
        q = collections.deque()

        # Build the graph.
        for v, u in prerequisites:
            graph[u].append(v)
            inDegrees[v] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        while q:
            u = q.popleft()
            ans.append(u)
            for v in graph[u]:
                inDegrees[v] -= 1
                if inDegrees[v] == 0:
                    q.append(v)

        return ans if len(ans) == numCourses else []
```

1025. 210.py

Path: LeetCode-main/solutions/210. Course Schedule II/210.py

```

from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def findOrder(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
    ) -> list[int]:
        ans = []
        graph = [[] for _ in range(numCourses)]
        states = [State.INIT] * numCourses

        for v, u in prerequisites:
            graph[u].append(v)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            if any(hasCycle(v) for v in graph[u]):
                return True
            states[u] = State.VISITED
            ans.append(u)
            return False

        if any(hasCycle(i) for i in range(numCourses)):
            return []

        return ans[::-1]

```

1026. 2100.py

Path: LeetCode-main/solutions/2100. Find Good Days to Rob the Bank/2100.py

```

class Solution:
    def goodDaysToRobBank(self, security: list[int], time: int) -> list[int]:
        n = len(security)
        dec = [0] * n # dec[i] := the number of continuous decreasing numbers before i
        inc = [0] * n # inc[i] := the number of continuous increasing numbers after i

        for i in range(1, n):
            if security[i - 1] >= security[i]:
                dec[i] = dec[i - 1] + 1

        for i in range(n - 2, -1, -1):
            if security[i] <= security[i + 1]:
                inc[i] = inc[i + 1] + 1

        return [i for i, (a, b) in enumerate(zip(dec, inc))
                if a >= time and b >= time]

```


1027. 2101.py

Path: LeetCode-main/solutions/2101. Detonate the Maximum Bombs/2101.py

```
class Solution:
    def maximumDetonation(self, bombs: list[list[int]]) -> int:
        n = len(bombs)
        ans = 0
        graph = [[] for _ in range(n)]

        for i, (xi, yi, ri) in enumerate(bombs):
            for j, (xj, yj, rj) in enumerate(bombs):
                if i == j:
                    continue
                if ri**2 >= (xi - xj)**2 + (yi - yj)**2:
                    graph[i].append(j)

        def dfs(u: int, seen: set[int]) -> None:
            for v in graph[u]:
                if v in seen:
                    continue
                seen.add(v)
                dfs(v, seen)

        for i in range(n):
            seen = set([i])
            dfs(i, seen)
            ans = max(ans, len(seen))

        return ans
```

1028. 2102.py

Path: LeetCode-main/solutions/2102. Sequentially Ordinal Rank Tracker/2102.py

```

class Location:
    def __init__(self, name: str, score: int):
        self.name = name
        self.score = score

    def __lt__(self, location):
        if self.score == location.score:
            return self.name > location.name
        return self.score < location.score

class SORTracker:
    def __init__(self):
        self.l = []
        self.r = []
        self.k = 0  # the number of times get() called

    def add(self, name: str, score: int) -> None:
        heapq.heappush(self.l, Location(name, score))
        if len(self.l) > self.k + 1:
            location = heapq.heappop(self.l)
            heapq.heappush(self.r, (-location.score, location.name))

    def get(self) -> str:
        name = self.l[0].name
        if self.r:
            topScore, topName = heapq.heappop(self.r)
            heapq.heappush(self.l, Location(topName, -topScore))
        self.k += 1
        return name

```

1029. 2103.py

Path: LeetCode-main/solutions/2103. Rings and Rods/2103.py

```

class Solution:
    def countPoints(self, rings: str) -> int:
        colors = [0] * 10

        for c, num in zip(rings[::2], rings[1::2]):
            color = 1 if c == 'R' else 2 if c == 'G' else 4
            colors[int(num)] |= color

        return sum(color == 7 for color in colors)

```

1030. 2104.py

Path: LeetCode-main/solutions/2104. Sum of Subarray Ranges/2104.py

```

class Solution:
    def subArrayRanges(self, nums: list[int]) -> int:
        prevGt, nextGt = self._getPrevNext(nums, operator.lt)
        prevLt, nextLt = self._getPrevNext(nums, operator.gt)
        return sum(num * (i - prevGt[i]) * (nextGt[i] - i) -
                    num * (i - prevLt[i]) * (nextLt[i] - i)
                    for i, num in enumerate(nums))

    def _getPrevNext(
        self,
        nums: list[int],
        op: callable
    ) -> tuple[list[int], list[int]]:
        """
        Returns `prev` and `next`, that store the indices of the nearest numbers
        that are smaller or larger than the current number depending on `op`.
        """
        n = len(nums)
        prev = [-1] * n
        next = [n] * n
        stack = []
        for i, num in enumerate(nums):
            while stack and op(nums[stack[-1]], num):
                next[stack.pop()] = i
            if stack:
                prev[i] = stack[-1]
            stack.append(i)
        return prev, next

```

1031. 2105.py

Path: LeetCode-main/solutions/2105. Watering Plants II/2105.py

```

class Solution:
    def minimumRefill(
        self,
        plants: list[int],
        capacityA: int,
        capacityB: int,
    ) -> int:
        ans = 0
        i = 0
        j = len(plants) - 1
        canA = capacityA
        canB = capacityB

        while i < j:
            ans += (canA < plants[i]) + (canB < plants[j])
            if canA < plants[i]:
                canA = capacityA
            if canB < plants[j]:
                canB = capacityB
            canA -= plants[i]
            canB -= plants[j]
            i += 1
            j -= 1

        return ans + (i == j and max(canA, canB) < plants[i])

```

1032. 2106.py

Path: LeetCode-main/solutions/2106. Maximum Fruits Harvested After at Most K Steps/2106.py

```
class Solution:
    def maxTotalFruits(
        self,
        fruits: list[list[int]],
        startPos: int,
        k: int,
    ) -> int:
        ans = 0
        maxRight = max(startPos, fruits[-1][0])
        amounts = [0] * (1 + maxRight)
        for position, amount in fruits:
            amounts[position] = amount
        prefix = list(itertools.accumulate(amounts, initial=0))

        def getFruits(leftSteps: int, rightSteps: int) -> int:
            l = max(0, startPos - leftSteps)
            r = min(maxRight, startPos + rightSteps)
            return prefix[r + 1] - prefix[l]

        # Go right first.
        for rightSteps in range(min(maxRight - startPos, k) + 1):
            leftSteps = max(0, k - 2 * rightSteps) # Turn left
            ans = max(ans, getFruits(leftSteps, rightSteps))

        # Go left first.
        for leftSteps in range(min(startPos, k) + 1):
            rightSteps = max(0, k - 2 * leftSteps) # Turn right
            ans = max(ans, getFruits(leftSteps, rightSteps))

        return ans
```

1033. 2107.py

Path: LeetCode-main/solutions/2107. Number of Unique Flavors After Sharing K Candies/2107.py

```
class Solution:
    def shareCandies(self, candies: list[int], k: int) -> int:
        ans = 0
        count = collections.Counter(candies)
        unique = len(count)

        for i, candy in enumerate(candies):
            count[candy] -= 1
            if count[candy] == 0:
                del count[candy]
                unique -= 1
            if i >= k:
                count[candies[i - k]] += 1
                if count[candies[i - k]] == 1:
                    unique += 1
            if i >= k - 1:
                ans = max(ans, unique)

        return ans
```

1034. 2108.py

Path: LeetCode-main/solutions/2108. Find First Palindromic String in the Array/2108.py

```
class Solution:
    def firstPalindrome(self, words: list[str]) -> str:
        def isPalindrome(s: str) -> bool:
            i = 0
            j = len(s) - 1
            while i < j:
                if s[i] != s[j]:
                    return False
                i += 1
                j -= 1
            return True
        return next((word for word in words if isPalindrome(word)), '')
```

1035. 2109.py

Path: LeetCode-main/solutions/2109. Adding Spaces to a String/2109.py

```
class Solution:
    def addSpaces(self, s: str, spaces: list[int]) -> str:
        ans = []
        j = 0 # spaces' index

        for i, c in enumerate(s):
            if j < len(spaces) and i == spaces[j]:
                ans.append(' ')
                j += 1
            ans.append(c)

        return ''.join(ans)
```

1036. 211.py

Path: LeetCode-main/solutions/211. Add and Search Word - Data structure design/211.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node: TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

    def search(self, word: str) -> bool:
        return self._dfs(word, 0, self.root)

    def _dfs(self, word: str, s: int, node: TrieNode) -> bool:
        if s == len(word):
            return node.isWord
        if word[s] != '.':
            child: TrieNode = node.children.get(word[s], None)
            return self._dfs(word, s + 1, child) if child else False
        return any(self._dfs(word, s + 1, child) for child in node.children.values())

```

1037. 2110.py

Path: LeetCode-main/solutions/2110. Number of Smooth Descent Periods of a Stock/2110.py

```

class Solution:
    def getDescentPeriods(self, prices: list[int]) -> int:
        ans = 1 # prices[0]
        dp = 1

        for i in range(1, len(prices)):
            if prices[i] == prices[i - 1] - 1:
                dp += 1
            else:
                dp = 1
            ans += dp

        return ans

```

1038. 2111.py

Path: LeetCode-main/solutions/2111. Minimum Operations to Make the Array K-Increasing/2111.py

```
class Solution:
    def kIncreasing(self, arr: list[int], k: int) -> int:
        def numReplaced(arr: list[int]) -> int:
            tails = []
            for a in arr:
                if not tails or tails[-1] <= a:
                    tails.append(a)
                else:
                    tails[bisect_right(tails, a)] = a
            return len(arr) - len(tails)

        return sum(numReplaced(arr[i::k]) for i in range(k))
```

1039. 2113.py

Path: LeetCode-main/solutions/2113. Elements in Array After Removing and Replacing Elements/2113.py

```
class Solution:
    def elementInNums(
        self,
        nums: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        n = len(nums)

        def f(time: int, index: int) -> int:
            if time < n: # [0, 1, 2] -> [1, 2] -> [2]
                index += time
                return -1 if index >= n else nums[index]
            else: # [] -> [0] -> [0, 1]
                return -1 if index >= time - n else nums[index]

        return [f(time % (2 * n), index) for time, index in queries]
```

1040. 2114.py

Path: LeetCode-main/solutions/2114. Maximum Number of Words Found in Sentences/2114.py

```
class Solution:
    def mostWordsFound(self, sentences: list[str]) -> int:
        return max(s.count(' ') for s in sentences) + 1
```

1041. 2115.py

Path: LeetCode-main/solutions/2115. Find All Possible Recipes from Given Supplies/2115.py

```
class Solution:
    def findAllRecipes(
        self,
        recipes: list[str],
        ingredients: list[list[str]],
        supplies: list[str],
    ) -> list[str]:
        ans = []
        supplies = set(supplies)
        graph = collections.defaultdict(list)
        inDegrees = collections.Counter()
        q = collections.deque()

        # Build the graph.
        for i, recipe in enumerate(recipes):
            for ingredient in ingredients[i]:
                if ingredient not in supplies:
                    graph[ingredient].append(recipe)
                    inDegrees[recipe] += 1

        # Perform topological sorting.
        for recipe in recipes:
            if inDegrees[recipe] == 0:
                q.append(recipe)

        while q:
            u = q.popleft()
            ans.append(u)
            for v in graph[u]:
                inDegrees[v] -= 1
                if inDegrees[v] == 0:
                    q.append(v)

        return ans
```

1042. 2116.py

Path: LeetCode-main/solutions/2116. Check if a Parentheses String Can Be Valid/2116.py


```

class Solution:
    def canBeValid(self, s: str, locked: str) -> bool:
        if len(s) % 2 == 1:
            return False

        def check(s: str, locked: str, isForward: bool) -> bool:
            changeable = 0
            l = 0
            r = 0

            for c, lock in zip(s, locked):
                if lock == '0':
                    changeable += 1
                elif c == '(':
                    l += 1
                else: # c == ')'
                    r += 1

                if isForward and changeable + l - r < 0:
                    return False
                if not isForward and changeable + r - l < 0:
                    return False

            return True

        return check(s, locked, True) and check(s[::-1], locked[::-1], False)

```

1043. 2117.py

Path: LeetCode-main/solutions/2117. Abbreviating the Product of a Range/2117.py

```

class Solution:
    def abbreviateProduct(self, left: int, right: int) -> str:
        prod = 1.0
        suf = 1
        countDigits = 0
        countZeros = 0

        for num in range(left, right + 1):
            prod *= num
            while prod >= 1.0:
                prod /= 10
                countDigits += 1
            suf *= num
            while suf % 10 == 0:
                suf //= 10
                countZeros += 1
            if suf > 10**8:
                suf %= 10**8

        if countDigits - countZeros <= 10:
            tens = 10**(countDigits - countZeros)
            return str(int(prod * tens + 0.5)) + 'e' + str(countZeros)

        pre = str(int(prod * 10 ** 5))
        suf = str(suf)[-5:]
        return pre + '...' + suf + 'e' + str(countZeros)

```

1044. 2119-2.py

Path: LeetCode-main/solutions/2119. A Number After a Double Reversal/2119-2.py

```
class Solution:
    def isSameAfterReversals(self, num: int) -> bool:
        return num == 0 or num % 10
```

1045. 2119.py

Path: LeetCode-main/solutions/2119. A Number After a Double Reversal/2119.py

```
class Solution:
    def isSameAfterReversals(self, num: int) -> bool:
        def getReversed(num: int) -> int:
            reversed = 0
            while num > 0:
                reversed = reversed * 10 + num % 10
                num //= 10
            return reversed

        reversed1 = getReversed(num)
        reversed2 = getReversed(reversed1)
        return reversed2 == num
```

1046. 212.py

Path: LeetCode-main/solutions/212. Word Search II/212.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
        m = len(board)
        n = len(board[0])
        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        for word in words:
            insert(word)

        def dfs(i: int, j: int, node: TrieNode) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if board[i][j] == '*':
                return

            c = board[i][j]
            if c not in node.children:
                return

            child = node.children[c]
            if child.word:
                ans.append(child.word)
                child.word = None

            board[i][j] = '*'
            dfs(i + 1, j, child)
            dfs(i - 1, j, child)
            dfs(i, j + 1, child)
            dfs(i, j - 1, child)
            board[i][j] = c

        for i in range(m):
            for j in range(n):
                dfs(i, j, root)

        return ans

```

1047. 2120.py

Path: LeetCode-main/solutions/2120. Execution of All Suffix Instructions Staying in a Grid/2120.py

```

class Solution:
    def executeInstructions(
        self,
        n: int,
        startPos: list[int],
        s: str,
    ) -> list[int]:
        moves = {'L': (0, -1), 'R': (0, 1), 'U': (-1, 0), 'D': (1, 0)}
        m = len(s)
        uMost = startPos[0] + 1
        dMost = n - startPos[0]
        lMost = startPos[1] + 1
        rMost = n - startPos[1]

        ans = [0] * m
        reach = {(0, None): m, (None, 0): m}
        x = 0
        y = 0

        for i in reversed(range(m)):
            dx, dy = moves[s[i]]
            x -= dx
            y -= dy
            reach[(x, None)] = i
            reach[(None, y)] = i
            out = min(reach.get((x - uMost, None), math.inf),
                      reach.get((x + dMost, None), math.inf),
                      reach.get((None, y - lMost), math.inf),
                      reach.get((None, y + rMost), math.inf))
            ans[i] = m - i if out == math.inf else out - i - 1

        return ans

```

1048. 2121.py

Path: LeetCode-main/solutions/2121. Intervals Between Identical Elements/2121.py

```

class Solution:
    def getDistances(self, arr: list[int]) -> list[int]:
        prefix = [0] * len(arr)
        suffix = [0] * len(arr)
        numToIndices = collections.defaultdict(list)

        for i, a in enumerate(arr):
            numToIndices[a].append(i)

        for indices in numToIndices.values():
            for i in range(1, len(indices)):
                currIndex = indices[i]
                prevIndex = indices[i - 1]
                prefix[currIndex] += prefix[prevIndex] + i * (currIndex - prevIndex)
            for i in range(len(indices) - 2, -1, -1):
                currIndex = indices[i]
                prevIndex = indices[i + 1]
                suffix[currIndex] += (suffix[prevIndex] +
                                       (len(indices) - i - 1) * (prevIndex - currIndex))

        return [p + s for p, s in zip(prefix, suffix)]

```

1049. 2122.py

Path: LeetCode-main/solutions/2122. Recover the Original Array/2122.py

```
class Solution:
    def recoverArray(self, nums: list[int]) -> list[int]:
        nums = sorted(nums)

        def getArray(x: int, count: collections.Counter) -> list[int]:
            arr = []
            for num in nums:
                if count[num] == 0:
                    continue
                if count[num + x] == 0:
                    return []
                count[num] -= 1
                count[num + x] -= 1
                arr.append(num + x // 2)
            return arr

        count = collections.Counter(nums)

        for i in range(1, len(nums)):
            x = nums[i] - nums[0] # 2 * k
            if x <= 0 or x % 2 == 1:
                continue
            arr = getArray(x, count.copy())
            if arr:
                return arr
```

1050. 2123.py

Path: LeetCode-main/solutions/2123. Minimum Operations to Remove Adjacent Ones in Matrix/2123.py

```

class Solution:
    def minimumOperations(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        seen = [[0] * n for _ in range(m)]
        match = [[-1] * n for _ in range(m)]

        def dfs(i: int, j: int, sessionId: int) -> int:
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if grid[x][y] == 0 or seen[x][y] == sessionId:
                    continue
                seen[x][y] = sessionId
                if match[x][y] == -1 or dfs(*divmod(match[x][y], n), sessionId):
                    match[x][y] = i * n + j
                    match[i][j] = x * n + y
                    return 1
            return 0

        ans = 0

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1 and match[i][j] == -1:
                    sessionId = i * n + j
                    seen[i][j] = sessionId
                    ans += dfs(i, j, sessionId)

        return ans

```

1051. 2124.py

Path: LeetCode-main/solutions/2124. Check if All A's Appears Before All B's/2124.py

```

class Solution:
    def checkString(self, s: str) -> bool:
        return 'ba' not in s

```

1052. 2125.py

Path: LeetCode-main/solutions/2125. Number of Laser Beams in a Bank/2125.py

```

class Solution:
    def numberOfBeams(self, bank: list[str]) -> int:
        ans = 0
        prevOnes = 0

        for row in bank:
            ones = row.count('1')
            if ones:
                ans += prevOnes * ones
                prevOnes = ones

        return ans

```

1053. 2126.py

Path: LeetCode-main/solutions/2126. Destroying Asteroids/2126.py

```
class Solution:
    def asteroidsDestroyed(self, mass: int, asteroids: list[int]) -> bool:
        for asteroid in sorted(asteroids):
            if mass >= asteroid:
                mass += asteroid
            else:
                return False
        return True
```

1054. 2127.py

Path: LeetCode-main/solutions/2127. Maximum Employees to Be Invited to a Meeting/2127.py

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def maximumInvitations(self, favorite: list[int]) -> int:
        n = len(favorite)
        sumComponentsLength = 0 # the component: a -> b -> c <-> x <- y
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n
        maxChainLength = [1] * n

        # Build the graph.
        for i, f in enumerate(favorite):
            graph[i].append(f)
            inDegrees[f] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        while q:
            u = q.popleft()
            for v in graph[u]:
                inDegrees[v] -= 1
                if inDegrees[v] == 0:
                    q.append(v)
                maxChainLength[v] = max(maxChainLength[v], 1 + maxChainLength[u])

        for i in range(n):
            if favorite[favorite[i]] == i:
                # i <-> favorite[i] (the cycle's length = 2)
                sumComponentsLength += maxChainLength[i] + maxChainLength[favorite[i]]

        maxCycleLength = 0 # Cycle: a -> b -> c -> a
        parent = [-1] * n
        seen = set()
        states = [State.INIT] * n

        def findCycle(u: int) -> None:
```

```

nonlocal maxCycleLength
seen.add(u)
states[u] = State.VISITING
for v in graph[u]:
    if v not in seen:
        parent[v] = u
        findCycle(v)
    elif states[v] == State.VISITING:
        # Find the cycle's length.
        curr = u
        cycleLength = 1
        while curr != v:
            curr = parent[curr]
            cycleLength += 1
        maxCycleLength = max(maxCycleLength, cycleLength)
states[u] = State.VISITED

for i in range(n):
    if i not in seen:
        findCycle(i)

return max(sumComponentsLength // 2, maxCycleLength)

```

1055. 2128.py

Path: LeetCode-main/solutions/2128. Remove All Ones With Row and Column Flips/2128.py

```

class Solution:
    def removeOnes(self, grid: list[list[int]]) -> bool:
        revRow = [a ^ 1 for a in grid[0]]
        return all(row == grid[0] or row == revRow for row in grid)

```

1056. 2129.py

Path: LeetCode-main/solutions/2129. Capitalize the Title/2129.py

```

class Solution:
    def capitalizeTitle(self, title: str) -> str:
        return ' '.join(s.lower() if len(s) < 3
                        else s.capitalize() for s in title.split())

```

1057. 213.py

Path: LeetCode-main/solutions/213. House Robber II/213.py


```

class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) < 2:
            return nums[0]

        def rob(l: int, r: int) -> int:
            dp1 = 0
            dp2 = 0

            for i in range(l, r + 1):
                temp = dp1
                dp1 = max(dp1, dp2 + nums[i])
                dp2 = temp

            return dp1

        return max(rob(0, len(nums) - 2),
                   rob(1, len(nums) - 1))

```

1058. 2130.py

Path: LeetCode-main/solutions/2130. Maximum Twin Sum of a Linked List/2130.py

```

class Solution:
    def pairSum(self, head: ListNode | None) -> int:
        def reverseList(head: ListNode) -> ListNode:
            prev = None
            while head:
                next = head.next
                head.next = prev
                prev = head
                head = next
            return prev

        ans = 0
        slow = head
        fast = head

        # `slow` points to the start of the second half.
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # `tail` points to the end of the reversed second half.
        tail = reverseList(slow)

        while tail:
            ans = max(ans, head.val + tail.val)
            head = head.next
            tail = tail.next

        return ans

```

1059. 2131.py

Path: LeetCode-main/solutions/2131. Longest Palindrome by Concatenating Two Letter Words/2131.py

```
class Solution:
    def longestPalindrome(self, words: list[str]) -> int:
        ans = 0
        count = [[0] * 26 for _ in range(26)]

        for a, b in words:
            i = ord(a) - ord('a')
            j = ord(b) - ord('a')
            if count[j][i]:
                ans += 4
                count[j][i] -= 1
            else:
                count[i][j] += 1

        for i in range(26):
            if count[i][i]:
                return ans + 2

        return ans
```

1060. 2132.py

Path: LeetCode-main/solutions/2132. Stamping the Grid/2132.py

```

class Solution:
    def possibleToStamp(
        self,
        grid: list[list[int]],
        stampHeight: int,
        stampWidth: int,
    ) -> bool:
        m = len(grid)
        n = len(grid[0])
        # A[i][j] := the number of 1s in grid[0..i][0..j]
        A = [[0] * (n + 1) for _ in range(m + 1)]
        # B[i][j] := the number of ways to stamp the submatrix in [0..i][0..j]
        B = [[0] * (n + 1) for _ in range(m + 1)]
        # fit[i][j] := true if the stamps can fit with the right-bottom at (i, j)
        fit = [[False] * n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                A[i + 1][j + 1] = A[i + 1][j] + A[i][j + 1] - A[i][j] + grid[i][j]
                if i + 1 >= stampHeight and j + 1 >= stampWidth:
                    x = i - stampHeight + 1
                    y = j - stampWidth + 1
                    if A[i + 1][j + 1] - A[x][j + 1] - A[i + 1][y] + A[x][y] == 0:
                        fit[i][j] = True

        for i in range(m):
            for j in range(n):
                B[i + 1][j + 1] = B[i + 1][j] + B[i][j + 1] - B[i][j] + fit[i][j]

        for i in range(m):
            for j in range(n):
                if not grid[i][j]:
                    x = min(i + stampHeight, m)
                    y = min(j + stampWidth, n)
                    if B[x][y] - B[i][y] - B[x][j] + B[i][j] == 0:
                        return False

        return True

```

1061. 2133.py

Path: LeetCode-main/solutions/2133. Check if Every Row and Column Contains All Numbers/2133.py

```

class Solution:
    def checkValid(self, matrix: list[list[int]]) -> bool:
        return all(min(len(set(row)), len(set(col))) == len(matrix)
                    for row, col in zip(matrix, zip(*matrix)))

```

1062. 2134.py

Path: LeetCode-main/solutions/2134. Minimum Swaps to Group All 1's Together II/2134.py

```

class Solution:
    def minSwaps(self, nums: list[int]) -> int:
        n = len(nums)
        k = nums.count(1)
        ones = 0 # the number of ones in the window
        maxOnes = 0 # the maximum number of ones in the window

        for i in range(n * 2):
            if i >= k and nums[i % n - k]: # Magic in Python :)
                ones -= 1
            if nums[i % n]:
                ones += 1
            maxOnes = max(maxOnes, ones)

        return k - maxOnes

```

1063. 2135.py

Path: LeetCode-main/solutions/2135. Count Words Obtained After Adding a Letter/2135.py

```

class Solution:
    def wordCount(self, startWords: list[str], targetWords: list[str]) -> int:
        def getMask(s: str) -> int:
            mask = 0
            for c in s:
                mask ^= 1 << ord(c) - ord('a')
            return mask

        ans = 0
        seen = set(getMask(w) for w in startWords)

        for targetWord in targetWords:
            mask = getMask(targetWord)
            for c in targetWord:
                # Toggle one character.
                if mask ^ 1 << ord(c) - ord('a') in seen:
                    ans += 1
                    break

        return ans

```

1064. 2136.py

Path: LeetCode-main/solutions/2136. Earliest Possible Day of Full Bloom/2136.py

```

class Solution:
    def earliestFullBloom(self, plantTime: list[int], growTime: list[int]) -> int:
        ans = 0
        time = 0

        for p, g in sorted(
            [(p, g) for (p, g) in zip(plantTime, growTime)],
            key=lambda x: -x[1]):
            time += p
            ans = max(ans, time + g)

        return ans

```

1065. 2137.py

Path: LeetCode-main/solutions/2137. Pour Water Between Buckets to Make Water Levels Equal/2137.py

```
class Solution:
    def equalizeWater(self, buckets: list[int], loss: int) -> float:
        ERR = 1e-5
        PERCENTAGE = (100 - loss) / 100
        l = 0.0
        r = max(buckets)

        def canFill(target: float) -> bool:
            extra = 0
            need = 0
            for bucket in buckets:
                if bucket > target:
                    extra += bucket - target
                else:
                    need += target - bucket
            return extra * PERCENTAGE >= need

        while r - l > ERR:
            m = (l + r) / 2
            if canFill(m):
                l = m
            else:
                r = m

        return l
```

1066. 2138.py

Path: LeetCode-main/solutions/2138. Divide a String Into Groups of Size k/2138.py

```
class Solution:
    def divideString(self, s: str, k: int, fill: str) -> list[str]:
        return [
            s[i:] + fill * (i + k - len(s)) if i + k > len(s)
            else s[i:i + k]
            for i in range(0, len(s), k)
        ]
```

1067. 2139.py

Path: LeetCode-main/solutions/2139. Minimum Moves to Reach Target Score/2139.py

```
class Solution:
    def minMoves(self, target: int, maxDoubles: int) -> int:
        steps = 0

        while target > 1 and maxDoubles:
            if target % 2 == 1:
                target -= 1
            else:
                target //= 2
                maxDoubles -= 1
            steps += 1

        return steps + target - 1
```

1068. 214.py

Path: LeetCode-main/solutions/214. Shortest Palindrome/214.py

```
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        t = s[::-1]

        for i in range(len(t)):
            if s.startswith(t[i:]):
                return t[:i] + s

        return t + s
```

1069. 2140.py

Path: LeetCode-main/solutions/2140. Solving Questions With Brainpower/2140.py

```
class Solution:
    def mostPoints(self, questions: list[list[int]]) -> int:
        n = len(questions)
        # dp[i] := the maximum points starting from questions[i]
        dp = [0] * (n + 1)

        for i in reversed(range(n)):
            points, brainpower = questions[i]
            nextIndex = i + brainpower + 1
            nextPoints = dp[nextIndex] if nextIndex < n else 0
            dp[i] = max(points + nextPoints, dp[i + 1])

        return dp[0]
```

1070. 2141.py

Path: LeetCode-main/solutions/2141. Maximum Running Time of N Computers/2141.py

```
class Solution:
    def maxRunTime(self, n: int, batteries: list[int]) -> int:
        summ = sum(batteries)

        batteries.sort()

        # The maximum battery is greater than the average, so it can last forever.
        # Reduce the problem from size n to size n - 1.
        while batteries[-1] > summ // n:
            summ -= batteries.pop()
            n -= 1

        # If the maximum battery <= average running time, it won't be waste, and so
        # do smaller batteries.
        return summ // n
```

1071. 2143.py

Path: LeetCode-main/solutions/2143. Choose Numbers From Two Arrays in Range/2143.py

```

class Solution:
    def countSubranges(self, nums1: list[int], nums2: list[int]) -> int:
        MOD = 1_000_000_007
        ans = 0
        # {sum, count}, add if choose from nums1, minus if choose from nums2
        dp = collections.Counter()

        for a, b in zip(nums1, nums2):
            newDp = collections.Counter()
            newDp[a] += 1
            newDp[-b] += 1

            for prevSum, count in dp.items():
                # Choose nums1[i]
                newDp[prevSum + a] += count
                newDp[prevSum + a] %= MOD
                # Choose nums2[i]
                newDp[prevSum - b] += count
                newDp[prevSum - b] %= MOD

            dp = newDp
            ans += dp[0]
            ans %= MOD

        return ans

```

1072. 2144.py

Path: LeetCode-main/solutions/2144. Minimum Cost of Buying Candies With Discount/2144.py

```

class Solution:
    def minimumCost(self, cost: list[int]) -> int:
        return sum(cost) - sum(sorted(cost)[-3::-3])

```

1073. 2145-2.py

Path: LeetCode-main/solutions/2145. Count the Hidden Sequences/2145-2.py

```

class Solution:
    def numberOfArrays(
        self,
        differences: list[int],
        lower: int,
        upper: int,
    ) -> int:
        prefix = 0
        mn = 0 # Starts from 0.
        mx = 0 # Starts from 0.

        for d in differences:
            prefix += d
            mn = min(mn, prefix)
            mx = max(mx, prefix)

        return max(0, (upper - lower) - (mx - mn) + 1)

```

1074. 2145.py

Path: LeetCode-main/solutions/2145. Count the Hidden Sequences/2145.py

```
class Solution:
    def numberOfArrays(
        self,
        differences: list[int],
        lower: int,
        upper: int,
    ) -> int:
        prefix = list(itertools.accumulate(differences, initial=0))
        return max(0, (upper - lower) - (max(prefix) - min(prefix)) + 1)
```

1075. 2146.py

Path: LeetCode-main/solutions/2146. K Highest Ranked Items Within a Price Range/2146.py

```
class Solution:
    def highestRankedKItems(
        self,
        grid: list[list[int]],
        pricing: list[int],
        start: list[int],
        k: int
    ) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        low, high = pricing
        row, col = start
        ans = []

        if low <= grid[row][col] <= high:
            ans.append([row, col])
            if k == 1:
                return ans

        q = collections.deque([(row, col)])
        seen = {(row, col)} # Mark as visited.

        while q:
            neighbors = []
            for _ in range(len(q)):
                i, j = q.popleft()
                for t in range(4):
                    x = i + DIRS[t][0]
                    y = j + DIRS[t][1]
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if not grid[x][y] or (x, y) in seen:
                        continue
                    if low <= grid[x][y] <= high:
                        neighbors.append([x, y])
                    q.append((x, y))
                    seen.add((x, y))
            neighbors.sort(key=lambda x: (grid[x[0]][x[1]], x[0], x[1]))
            for neighbor in neighbors:
                if len(ans) < k:
                    ans.append(neighbor)
                if len(ans) == k:
                    return ans

        return ans
```

1076. 2147.py

Path: LeetCode-main/solutions/2147. Number of Ways to Divide a Long Corridor/2147.py

```
class Solution:
    def numberOfWays(self, corridor: str) -> int:
        MOD = 1_000_000_007
        ans = 1
        prevSeat = -1
        numSeats = 0

        for i, c in enumerate(corridor):
            if c == 'S':
                numSeats += 1
                if numSeats > 2 and numSeats % 2 == 1:
                    ans = ans * (i - prevSeat) % MOD
                prevSeat = i

        return ans if numSeats > 1 and numSeats % 2 == 0 else 0
```

1077. 2148.py

Path: LeetCode-main/solutions/2148. Count Elements With Strictly Smaller and Greater Elements/2148.py

```
class Solution:
    def countElements(self, nums: list[int]) -> int:
        mn = min(nums)
        mx = max(nums)
        return sum(mn < num < mx for num in nums)
```

1078. 2149.py

Path: LeetCode-main/solutions/2149. Rearrange Array Elements by Sign/2149.py

```
class Solution:
    def rearrangeArray(self, nums: list[int]) -> list[int]:
        ans = []
        pos = []
        neg = []

        for num in nums:
            (pos if num > 0 else neg).append(num)

        for p, n in zip(pos, neg):
            ans += [p, n]

        return ans
```

1079. 215-2.py

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215-2.py

```

class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        def quickSelect(l: int, r: int, k: int) -> int:
            pivot = nums[r]

            nextSwapped = l
            for i in range(l, r):
                if nums[i] >= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1
            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]

            count = nextSwapped - l + 1 # Number of nums >= pivot
            if count == k:
                return nums[nextSwapped]
            if count > k:
                return quickSelect(l, nextSwapped - 1, k)
            return quickSelect(nextSwapped + 1, r, k - count)

        return quickSelect(0, len(nums) - 1, k)

```

1080. 215-3.py

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215-3.py

```

class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        def quickSelect(l: int, r: int, k: int) -> int:
            randIndex = random.randint(0, r - 1) + 1
            nums[randIndex], nums[r] = nums[r], nums[randIndex]
            pivot = nums[r]

            nextSwapped = l
            for i in range(l, r):
                if nums[i] >= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1
            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]

            count = nextSwapped - l + 1 # Number of nums >= pivot
            if count == k:
                return nums[nextSwapped]
            if count > k:
                return quickSelect(l, nextSwapped - 1, k)
            return quickSelect(nextSwapped + 1, r, k - count)

        return quickSelect(0, len(nums) - 1, k)

```

1081. 215.py

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215.py

```
class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, num)
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return minHeap[0]
```

1082. 2150.py

Path: LeetCode-main/solutions/2150. Find All Lonely Numbers in the Array/2150.py

```
class Solution:
    def findLonely(self, nums: list[int]) -> list[int]:
        count = collections.Counter(nums)
        return [num for num, freq in count.items()
                if freq == 1 and
                   count[num - 1] == 0 and
                   count[num + 1] == 0]
```

1083. 2151-2.py

Path: LeetCode-main/solutions/2151. Maximum Good People Based on Statements/2151-2.py

```
class Solution:
    def maximumGood(self, statements: list[list[int]]) -> int:
        n = len(statements)

        def isValid(mask: int) -> bool:
            for i in range(n):
                # The i-th person is bad, so no need to check.
                if (mask >> i & 1) == 0:
                    continue
                for j in range(n):
                    if statements[i][j] == 2:
                        continue
                    if statements[i][j] != (mask >> j & 1):
                        return False
            return True

        return max(mask.bit_count()
                  for mask in range(1 << n)
                  if isValid(mask))
```

1084. 2151.py

Path: LeetCode-main/solutions/2151. Maximum Good People Based on Statements/2151.py

```
class Solution:
    def maximumGood(self, statements: list[list[int]]) -> int:
        n = len(statements)
        ans = 0

        def isValid(good: list[int]) -> bool:
            for i, g in enumerate(good):
                if not g: # The i-th person is bad, so no need to check.
                    continue
                for j in range(n):
                    if statements[i][j] == 2:
                        continue
                    if statements[i][j] != good[j]:
                        return False
            return True

        def dfs(good: list[int], i: int, count: int) -> None:
            nonlocal ans
            if i == n:
                if isValid(good):
                    ans = max(ans, count)
                return

            good.append(0) # Assume the i-th person is bad.
            dfs(good, i + 1, count)
            good[-1] = 1 # Assume the i-th person is good.
            dfs(good, i + 1, count + 1)
            good.pop()

        dfs([], 0, 0)
        return ans
```

1085. 2152.py

Path: LeetCode-main/solutions/2152. Minimum Number of Lines to Cover Points/2152.py

```

class Solution:
    def minimumLines(self, points: list[list[int]]) -> int:
        n = len(points)
        allCovered = (1 << n) - 1
        maxLines = n // 2 + (n & 1)

        def getSlope(p: list[int], q: list[int]) -> tuple[int, int]:
            dx = p[0] - q[0]
            dy = p[1] - q[1]
            if dx == 0:
                return (0, p[0])
            if dy == 0:
                return (p[1], 0)
            d = gcd(dx, dy)
            x = dx // d
            y = dy // d
            return (x, y) if x > 0 else (-x, -y)

        @functools.lru_cache(None)
        def dfs(covered: int) -> int:
            if covered == allCovered:
                return 0

            ans = maxLines

            for i in range(n):
                if covered >> i & 1:
                    continue
                for j in range(n):
                    if i == j:
                        continue
                    # Connect the points[i] with the points[j].
                    newCovered = covered | 1 << i | 1 << j
                    slope = getSlope(points[i], points[j])
                    # Mark the points covered by this line.
                    for k in range(n):
                        if getSlope(points[i], points[k]) == slope:
                            newCovered |= 1 << k
                    ans = min(ans, 1 + dfs(newCovered))

            return ans

        return dfs(0)

```

1086. 2155-2.py

Path: LeetCode-main/solutions/2154. Keep Multiplying Found Values by Two/2155-2.py

```

class Solution:
    def findFinalValue(self, nums: list[int], original: int) -> int:
        seen = [False] * 1001

        for num in nums:
            seen[num] = True

        while original < 1001 and seen[original]:
            original *= 2

        return original

```

1087. 2155.py

Path: LeetCode-main/solutions/2154. Keep Multiplying Found Values by Two/2155.py

```
class Solution:
    def findFinalValue(self, nums: list[int], original: int) -> int:
        numsSet = set(nums)
        while original in numsSet:
            original *= 2
        return original
```

1088. 2155.py

Path: LeetCode-main/solutions/2155. All Divisions With the Highest Score of a Binary Array/2155.py

```
class Solution:
    def maxScoreIndices(self, nums: list[int]) -> list[int]:
        zeros = nums.count(0)
        ones = len(nums) - zeros
        ans = [0] # the division at index 0
        leftZeros = 0
        leftOnes = 0
        maxScore = ones # `leftZeros` + `rightOnes`

        for i, num in enumerate(nums):
            leftZeros += num == 0
            leftOnes += num == 1
            rightOnes = ones - leftOnes
            score = leftZeros + rightOnes
            if maxScore == score:
                ans.append(i + 1)
            elif maxScore < score:
                maxScore = score
                ans = [i + 1]

        return ans
```

1089. 2156.py

Path: LeetCode-main/solutions/2156. Find Substring With Given Hash Value/2156.py

```
class Solution:
    def subStrHash(
        self,
        s: str,
        power: int,
        modulo: int,
        k: int,
        hashValue: int,
    ) -> str:
        maxPower = pow(power, k, modulo)
        hash = 0

        def val(c: str) -> int:
            return ord(c) - ord('a') + 1

        for i, c in reversed(list(enumerate(s))):
            hash = (hash * power + val(c)) % modulo
            if i + k < len(s):
                hash = (hash - val(s[i + k]) * maxPower) % modulo
            if hash == hashValue:
                bestLeft = i

        return s[bestLeft:bestLeft + k]
```

1090. 2157.py

Path: LeetCode-main/solutions/2157. Groups of Strings/2157.py

```

class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.sz = [1] * n

    def unionBySize(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.sz[i] < self.sz[j]:
            self.sz[j] += self.sz[i]
            self.id[i] = j
        else:
            self.sz[i] += self.sz[j]
            self.id[j] = i
        self.count -= 1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def groupStrings(self, words: list[str]) -> list[int]:
        uf = UnionFind(len(words))

        def getMask(s: str) -> int:
            mask = 0
            for c in s:
                mask |= 1 << ord(c) - ord('a')
            return mask

        def getAddedMasks(mask: int):
            for i in range(26):
                if not (mask >> i & 1):
                    yield mask | 1 << i

        def getDeletedMasks(mask: int):
            for i in range(26):
                if mask >> i & 1:
                    yield mask ^ 1 << i

        maskToIndex = {getMask(word): i for i, word in enumerate(words)}
        deletedMaskToIndex = {}

        for i, word in enumerate(words):
            mask = getMask(word)
            for m in getAddedMasks(mask):
                if m in maskToIndex:
                    uf.unionBySize(i, maskToIndex[m])
            for m in getDeletedMasks(mask):
                if m in maskToIndex:
                    uf.unionBySize(i, maskToIndex[m])
                if m in deletedMaskToIndex:
                    uf.unionBySize(i, deletedMaskToIndex[m])
            else:
                deletedMaskToIndex[m] = i

        return [uf.count, max(uf.sz)]

```


1091. 2158.py

Path: LeetCode-main/solutions/2158. Amount of New Area Painted Each Day/2158.py

```
from sortedcontainers import SortedList

class Solution:
    def amountPainted(self, paint: list[list[int]]) -> list[int]:
        minDay = min(s for s, e in paint)
        maxDay = max(e for s, e in paint)
        ans = [0] * len(paint)
        # Stores the indices of paints that are available now.
        runningIndices = SortedList()
        events = [] # (day, index, type)

        for i, (start, end) in enumerate(paint):
            events.append((start, i, 1)) # 1 := entering
            events.append((end, i, -1)) # -1 := leaving

        events.sort()

        i = 0 # events' index
        for day in range(minDay, maxDay):
            while i < len(events) and events[i][0] == day:
                day, index, type = events[i]
                if type == 1:
                    runningIndices.add(index)
                else:
                    runningIndices.remove(index)
                i += 1
            if runningIndices:
                ans[runningIndices[0]] += 1

        return ans
```

1092. 216.py

Path: LeetCode-main/solutions/216. Combination Sum III/216.py

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> list[list[int]]:
        ans = []

        def dfs(k: int, n: int, s: int, path: list[int]) -> None:
            if k == 0 and n == 0:
                ans.append(path)
                return
            if k == 0 or n < 0:
                return

            for i in range(s, 10):
                dfs(k - 1, n - i, i + 1, path + [i])

        dfs(k, n, 1, [])
        return ans
```

1093. 2160.py

Path: LeetCode-main/solutions/2160. Minimum Sum of Four Digit Number After Splitting Digits/2160.py

```
class Solution:
    def minimumSum(self, num: int) -> int:
        s = sorted(str(num))
        return int(s[0] + s[2]) + int(s[1] + s[3])
```

1094. 2161.py

Path: LeetCode-main/solutions/2161. Partition Array According to Given Pivot/2161.py

```
class Solution:
    def pivotArray(self, nums: list[int], pivot: int) -> list[int]:
        return ([num for num in nums if num < pivot] +
                [num for num in nums if num == pivot] +
                [num for num in nums if num > pivot])
```

1095. 2162.py

Path: LeetCode-main/solutions/2162. Minimum Cost to Set Cooking Time/2162.py

```
class Solution:
    def minCostSetTime(
        self,
        startAt: int,
        moveCost: int,
        pushCost: int,
        targetSeconds: int,
    ) -> int:
        ans = math.inf
        mins = 99 if targetSeconds > 5999 else targetSeconds // 60
        secs = targetSeconds - mins * 60

        def getCost(mins: int, secs: int) -> int:
            cost = 0
            curr = str(startAt)
            for c in str(mins * 100 + secs):
                if c == curr:
                    cost += pushCost
                else:
                    cost += moveCost + pushCost
                    curr = c
            return cost

        while secs < 100:
            ans = min(ans, getCost(mins, secs))
            mins -= 1
            secs += 60

        return ans
```

1096. 2163.py

Path: LeetCode-main/solutions/2163. Minimum Difference in Sums After Removal of Elements/2163.py

```

class Solution:
    def minimumDifference(self, nums: list[int]) -> int:
        n = len(nums) // 3
        ans = math.inf
        leftSum = 0
        rightSum = 0
        maxHeap = [] # Left part, as small as possible
        minHeap = [] # Right part, as big as possible
        # minLeftSum[i] := the minimum of the sum of n nums in nums[0..i)
        minLeftSum = [0] * len(nums)

        for i in range(2 * n):
            heapq.heappush(maxHeap, -nums[i])
            leftSum += nums[i]
            if len(maxHeap) == n + 1:
                leftSum += heapq.heappop(maxHeap)
            if len(maxHeap) == n:
                minLeftSum[i] = leftSum

        for i in range(len(nums) - 1, n - 1, -1):
            heapq.heappush(minHeap, nums[i])
            rightSum += nums[i]
            if len(minHeap) == n + 1:
                rightSum -= heapq.heappop(minHeap)
            if len(minHeap) == n:
                ans = min(ans, minLeftSum[i - 1] - rightSum)

        return ans

```

1097. 2164-2.py

Path: LeetCode-main/solutions/2164. Sort Even and Odd Indices Independently/2164-2.py

```

class Solution:
    def sortEvenOdd(self, nums: list[int]) -> list[int]:
        nums[::2] = sorted(nums[::2])
        nums[1::2] = sorted(nums[1::2])[:-1]
        return nums

```

1098. 2164.py

Path: LeetCode-main/solutions/2164. Sort Even and Odd Indices Independently/2164.py

```
class Solution:
    def sortEvenOdd(self, nums: list[int]) -> list[int]:
        ans = [0] * len(nums)
        evenCount = collections.Counter(nums[::2])
        oddCount = collections.Counter(nums[1::2])

        ansIndex = 0
        for i in range(1, 101):
            while evenCount[i] > 0:
                ans[ansIndex] = i
                ansIndex += 2
                evenCount[i] -= 1

        ansIndex = 1
        for i in range(100, 0, -1):
            while oddCount[i] > 0:
                ans[ansIndex] = i
                ansIndex += 2
                oddCount[i] -= 1

        return ans
```

1099. 2165.py

Path: LeetCode-main/solutions/2165. Smallest Value of the Rearranged Number/2165.py

```
class Solution:
    def smallestNumber(self, num: int) -> int:
        s = sorted(str(abs(num)), reverse=num < 0)
        firstNonZeroIndex = next((i for i, c in enumerate(s) if c != '0'), 0)
        s[0], s[firstNonZeroIndex] = s[firstNonZeroIndex], s[0]
        return int(''.join(s)) * (-1 if num < 0 else 1)
```

1100. 2166.py

Path: LeetCode-main/solutions/2166. Design Bitset/2166.py

```

class Bitset:
    def __init__(self, size: int):
        self.s = ['0'] * size # the original
        self.r = ['1'] * size # the reversed
        self.cnt = 0

    def fix(self, idx: int) -> None:
        if self.s[idx] == '0':
            self.cnt += 1
            self.s[idx] = '1'
            self.r[idx] = '0'

    def unfix(self, idx: int) -> None:
        if self.s[idx] == '1':
            self.cnt -= 1
            self.s[idx] = '0'
            self.r[idx] = '1'

    def flip(self) -> None:
        self.s, self.r = self.r, self.s
        self.cnt = len(self.s) - self.cnt

    def all(self) -> bool:
        return self.cnt == len(self.s)

    def one(self) -> bool:
        return self.cnt

    def count(self) -> int:
        return self.cnt

    def toString(self) -> str:
        return ''.join(self.s)

```

1101. 2167-2.py

Path: LeetCode-main/solutions/2167. Minimum Time to Remove All Cars Containing Illegal Goods/2167-2.py

```

class Solution:
    def minimumTime(self, s: str) -> int:
        n = len(s)
        ans = n
        left = 0 # the minimum time to remove the illegal cars so far

        for i, c in enumerate(s):
            left = min(left + int(c) * 2, i + 1)
            ans = min(ans, left + n - 1 - i)

        return ans

```

1102. 2167.py

Path: LeetCode-main/solutions/2167. Minimum Time to Remove All Cars Containing Illegal Goods/2167.py

```
class Solution:
    def minimumTime(self, s: str) -> int:
        n = len(s)
        # left[i] := the minimum time to remove the illegal cars of s[0..i]
        left = [0] * n
        left[0] = int(s[0])
        # dp[i] := the minimum time to remove the illegal cars of s[0..i] optimally
        # + the time to remove the illegal cars of s[i + 1..n) consecutively
        # Note that the way to remove the illegal cars in the right part
        # doesn't need to be optimal since:
        #   `left | illegal cars | n - 1 - k` will be covered in
        #   `left' | n - 1 - i` later.
        dp = [n] * n
        dp[0] = left[0] + n - 1

        for i in range(1, n):
            left[i] = min(left[i - 1] + int(s[i]) * 2, i + 1)
            dp[i] = min(dp[i], left[i] + n - 1 - i)

        return min(dp)
```

1103. 2168.py

Path: LeetCode-main/solutions/2168. Unique Substrings With Equal Digit Frequency/2168.py

```

class Solution:
    def equalDigitFrequency(self, s: str) -> int:
        BASE = 11
        HASH = 1_000_000_007
        counts: list[dict] = [] # counts[i] := the counter map of s[0..i]
        count = collections.Counter()
        pows = [1] # pows[i] := BASE^i % HASH
        # hash[i] = the hash of the first i letters of s, where hash[i] =
        # (26^(i - 1) * s[0] + 26^(i - 2) * s[1] + ... + s[i - 1]) % HASH
        hash = [0]

        def val(c: str) -> int:
            return int(c) + 1

        for c in s:
            count[c] += 1
            counts.append(count.copy())
            pows.append(pows[-1] * BASE % HASH)
            hash.append((hash[-1] * BASE + val(c)) % HASH)

        def getRollingHash(l: int, r: int) -> int:
            """Returns the rolling hash of s[l..r)."""
            h = (hash[r] - hash[l] * pows[r - l]) % HASH
            return h + HASH if h < 0 else h

        return len({getRollingHash(i, j + 1)
                     for i in range(len(s))
                     for j in range(i, len(s))
                     if self._isSameFreq(counts, i, j)})

        def _isSameFreq(self, counts: list[dict], i: int, j: int) -> bool:
            count = counts[j].copy()
            if i > 0:
                for c, freq in counts[i - 1].items():
                    count[c] -= freq
                    if count[c] == 0:
                        del count[c]
            return min(count.values()) == max(count.values())

```

1104. 2169.py

Path: LeetCode-main/solutions/2169. Count Operations to Obtain Zero/2169.py

```

class Solution:
    def countOperations(self, num1: int, num2: int) -> int:
        ans = 0

        while num1 and num2:
            if num1 < num2:
                num1, num2 = num2, num1
            ans += num1 // num2
            num1 %= num2

        return ans

```

1105. 217.py

Path: LeetCode-main/solutions/217. Contains Duplicate/217.py

```
class Solution:
    def containsDuplicate(self, nums: list[int]) -> bool:
        return len(nums) != len(set(nums))
```

1106. 2170.py

Path: LeetCode-main/solutions/2170. Minimum Operations to Make the Array Alternating/2170.py

```
class T:
    def __init__(self):
        self.count = collections.Counter()
        self.mx = 0
        self.secondMax = 0
        self.maxFreq = 0
        self.secondMaxFreq = 0

class Solution:
    def minimumOperations(self, nums: list[int]) -> int:
        # 0 := odd indices, 1 := even indices
        ts = [T() for _ in range(2)]

        for i, num in enumerate(nums):
            t = ts[i % 2]
            t.count[num] += 1
            freq = t.count[num]
            if freq > t.maxFreq:
                t.maxFreq = freq
                t.mx = num
            elif freq > t.secondMaxFreq:
                t.secondMaxFreq = freq
                t.secondMax = num

        if ts[0].mx == ts[1].mx:
            return len(nums) - max(ts[0].maxFreq + ts[1].secondMaxFreq,
                                   ts[1].maxFreq + ts[0].secondMaxFreq)
        return len(nums) - (ts[0].maxFreq + ts[1].maxFreq)
```

1107. 2171.py

Path: LeetCode-main/solutions/2171. Removing Minimum Number of Magic Beans/2171.py

```
class Solution:
    def minimumRemoval(self, beans: list[int]) -> int:
        n = len(beans)
        summ = sum(beans)
        return min(summ - (n - i) * bean
                   for i, bean in enumerate(sorted(beans)))
```

1108. 2172.py

Path: LeetCode-main/solutions/2172. Maximum AND Sum of Array/2172.py


```

class Solution:
    def maximumANDSum(self, nums: list[int], numSlots: int) -> int:
        n = 2 * numSlots
        nSelected = 1 << n
        # dp[i] := the maximum value, where i is the bitmask of the selected
        # numbers
        dp = [0] * nSelected

        nums += [0] * (n - len(nums))

        for mask in range(1, nSelected):
            selected = mask.bit_count()
            slot = (selected + 1) // 2 # (1, 2) -> 1, (3, 4) -> 2
            for i, num in enumerate(nums):
                if mask >> i & 1: # Assign `nums[i]` to the `slot`-th slot.
                    dp[mask] = max(dp[mask], dp[mask ^ 1 << i] + (slot & num))

        return dp[-1]

```

1109. 2174-2.py

Path: LeetCode-main/solutions/2174. Remove All Ones With Row and Column Flips II/2174-2.py

```

class Solution:
    def removeOnes(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        maxMask = 1 << m * n
        # dp[i] := the minimum number of operations to remove all 1s from the grid,
        # where `i` is the bitmask of the state of the grid
        dp = [math.inf] * maxMask
        dp[0] = 0

        for mask in range(maxMask):
            for i in range(m):
                for j in range(n):
                    if grid[i][j] == 1:
                        newMask = mask
                        # Set the cells in the same row with 0.
                        for k in range(n):
                            newMask &= ~(1 << i * n + k)
                        # Set the cells in the same column with 0.
                        for k in range(m):
                            newMask &= ~(1 << k * n + j)
                        dp[mask] = min(dp[mask], 1 + dp[newMask])

        return dp[self.encode(grid, m, n)]

    def encode(self, grid: list[list[int]], m: int, n: int) -> int:
        encoded = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j]:
                    encoded |= 1 << i * n + j
        return encoded

```

1110. 2174.py

Path: LeetCode-main/solutions/2174. Remove All Ones With Row and Column Flips II/2174.py

```

class Solution:
    def removeOnes(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        @functools.lru_cache(None)
        def dp(mask: int) -> int:
            """
            Returns the minimum number of operations to remove all 1s from the grid,
            where `mask` is the bitmask of the state of the grid.
            """
            if mask == 0:
                return 0
            ans = math.inf
            for i in range(m):
                for j in range(n):
                    if mask >> i * n + j & 1: # grid[i][j] == 1
                        newMask = mask
                        for k in range(n): # Set the cells in the same row with 0.
                            newMask &= ~(1 << i * n + k)
                        for k in range(m): # Set the cells in the same column with 0.
                            newMask &= ~(1 << k * n + j)
                        ans = min(ans, 1 + dp(newMask))
            return ans

        return dp(self.encode(grid, m, n))

    def encode(self, grid: list[list[int]], m: int, n: int) -> int:
        encoded = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j]:
                    encoded |= 1 << i * n + j
        return encoded

```

1111. 2176.py

Path: LeetCode-main/solutions/2176. Count Equal and Divisible Pairs in an Array/2176.py

```

class Solution:
    def countPairs(self, nums: list[int], k: int) -> int:
        ans = 0
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums):
            numToIndices[num].append(i)

        for indices in numToIndices.values():
            gcds = collections.Counter()
            for i in indices:
                gcd_i = math.gcd(i, k)
                for gcd_j, count in gcds.items():
                    if gcd_i * gcd_j % k == 0:
                        ans += count
                gcds[gcd_i] += 1

        return ans

```

1112. 2177.py

Path: LeetCode-main/solutions/2177. Find Three Consecutive Integers That Sum to a Given Number/2177.py

```
class Solution:
    def sumOfThree(self, num: int) -> list[int]:
        if num % 3:
            return []
        x = num // 3
        return [x - 1, x, x + 1]
```

1113. 2178.py

Path: LeetCode-main/solutions/2178. Maximum Split of Positive Even Integers/2178.py

```
class Solution:
    def maximumEvenSplit(self, finalSum: int) -> list[int]:
        if finalSum % 2 == 1:
            return []

        ans = []
        needSum = finalSum
        even = 2

        while needSum - even >= even + 2:
            ans.append(even)
            needSum -= even
            even += 2

        return ans + [needSum]
```

1114. 2179.py

Path: LeetCode-main/solutions/2179. Count Good Triplets in an Array/2179.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    def goodTriplets(self, nums1: list[int], nums2: list[int]) -> int:
        n = len(nums1)
        numToIndex = {num: i for i, num in enumerate(nums1)}
        # Remap each number in `nums2` to the according index in `nums1` as `arr`.
        # So the problem is to find the number of increasing tripets in `arr`.
        arr = [numToIndex[num] for num in nums2]
        # leftSmaller[i] := the number of arr[j] < arr[i], where 0 <= j < i
        leftSmaller = [0] * n
        # rightLarger[i] := the number of arr[j] > arr[i], where i < j < n
        rightLarger = [0] * n
        tree1 = FenwickTree(n) # Calculates `leftSmaller`.
        tree2 = FenwickTree(n) # Calculates `rightLarger`.

        for i, a in enumerate(arr):
            leftSmaller[i] = tree1.get(a)
            tree1.add(a + 1, 1)

        for i, a in reversed(list(enumerate(arr))):
            rightLarger[i] = tree2.get(n) - tree2.get(a)
            tree2.add(a + 1, 1)

        return sum(a * b for a, b in zip(leftSmaller, rightLarger))

```

1115. 218.py

Path: LeetCode-main/solutions/218. The Skyline Problem/218.py

```

class Solution:
    def getSkyline(self, buildings: list[list[int]]) -> list[list[int]]:
        n = len(buildings)
        if n == 0:
            return []
        if n == 1:
            left, right, height = buildings[0]
            return [[left, height], [right, 0]]

        left = self.getSkyline(buildings[:n // 2])
        right = self.getSkyline(buildings[n // 2:])
        return self._merge(left, right)

    def _merge(self, left: list[list[int]],
               right: list[list[int]]) -> list[list[int]]:
        ans = []
        i = 0 # left's index
        j = 0 # right's index
        leftY = 0
        rightY = 0

        while i < len(left) and j < len(right):
            # Choose the powith smaller x
            if left[i][0] < right[j][0]:
                leftY = left[i][1] # Update the ongoing `leftY` .
                self._addPoint(ans, left[i][0], max(left[i][1], rightY))
                i += 1
            else:
                rightY = right[j][1] # Update the ongoing `rightY` .
                self._addPoint(ans, right[j][0], max(right[j][1], leftY))
                j += 1

        while i < len(left):
            self._addPoint(ans, left[i][0], left[i][1])
            i += 1

        while j < len(right):
            self._addPoint(ans, right[j][0], right[j][1])
            j += 1

        return ans

    def _addPoint(self, ans: list[list[int]], x: int, y: int) -> None:
        if ans and ans[-1][0] == x:
            ans[-1][1] = y
            return
        if ans and ans[-1][1] == y:
            return
        ans.append([x, y])

```

1116. 2180.py

Path: LeetCode-main/solutions/2180. Count Integers With Even Digit Sum/2180.py

```

class Solution:
    def countEven(self, num: int) -> int:
        return (num - self._getDigitSum(num) % 2) // 2

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))

```

1117. 2181-2.py

Path: LeetCode-main/solutions/2181. Merge Nodes in Between Zeros/2181-2.py

```
class Solution:
    def mergeNodes(self, head: ListNode | None) -> ListNode | None:
        curr = head.next

        while curr:
            running = curr
            summ = 0
            while running.val > 0:
                summ += running.val
                running = running.next

            curr.val = summ
            curr.next = running.next
            curr = running.next

        return head.next
```

1118. 2181.py

Path: LeetCode-main/solutions/2181. Merge Nodes in Between Zeros/2181.py

```
class Solution:
    def mergeNodes(self, head: ListNode | None) -> ListNode | None:
        if not head:
            return None
        if not head.next.val:
            node = ListNode(head.val)
            node.next = self.mergeNodes(head.next.next)
            return node

        next = self.mergeNodes(head.next)
        next.val += head.val
        return next
```

1119. 2182.py

Path: LeetCode-main/solutions/2182. Construct String With Repeat Limit/2182.py

```

class Solution:
    def repeatLimitedString(self, s: str, repeatLimit: int) -> str:
        ans = ''
        count = collections.Counter(s)

        while True:
            addOne = ans and self._shouldAddOne(ans, count)
            c = self._getLargestChar(ans, count)
            if c == ' ':
                break
            repeats = 1 if addOne else min(count[c], repeatLimit)
            ans += c * repeats
            count[c] -= repeats

        return ans

    def _shouldAddOne(self, ans: str, count: collections.Counter) -> bool:
        for c in reversed(string.ascii_lowercase):
            if count[c]:
                return ans[-1] == c
        return False

    def _getLargestChar(self, ans: str, count: collections.Counter) -> int:
        for c in reversed(string.ascii_lowercase):
            if count[c] and (not ans or ans[-1] != c):
                return c
        return ' '

```

1120. 2183.py

Path: LeetCode-main/solutions/2183. Count Array Pairs Divisible by K/2183.py

```

class Solution:
    def countPairs(self, nums: list[int], k: int) -> int:
        ans = 0
        gcds = collections.Counter()

        for num in nums:
            gcd_i = math.gcd(num, k)
            for gcd_j, count in gcds.items():
                if gcd_i * gcd_j % k == 0:
                    ans += count
            gcds[gcd_i] += 1

        return ans

```

1121. 2184.py

Path: LeetCode-main/solutions/2184. Number of Ways to Build Sturdy Brick Wall/2184.py

```

class Solution:
    def buildWall(self, height: int, width: int, bricks: list[int]) -> int:
        MOD = 1_000_000_007
        # Stores the valid rows in bitmask.
        rows = []
        self._buildRows(width, bricks, 0, rows)

        n = len(rows)
        # dp[i] := the number of ways to build `h` height walls with rows[i] in the bottom
        dp = [1] * n
        # graph[i] := the valid neighbors of rows[i]
        graph = [[] for _ in range(n)]

        for i, a in enumerate(rows):
            for j, b in enumerate(rows):
                if not a & b:
                    graph[i].append(j)

        for _ in range(2, height + 1):
            newDp = [0] * n
            for i in range(n):
                for v in graph[i]:
                    newDp[i] += dp[v]
                    newDp[i] %= MOD
            dp = newDp

        return sum(dp) % MOD

    def _buildRows(
        self,
        width: int,
        bricks: list[int],
        path: int,
        rows: list[int],
    ):
        for brick in bricks:
            if brick == width:
                rows.append(path)
            elif brick < width:
                newWidth = width - brick
                self._buildRows(newWidth, bricks, path | 2 << newWidth, rows)

```

1122. 2185.py

Path: LeetCode-main/solutions/2185. Counting Words With a Given Prefix/2185.py

```

class Solution:
    def prefixCount(self, words: list[str], pref: str) -> int:
        return sum(word.startswith(pref) for word in words)

```

1123. 2186.py

Path: LeetCode-main/solutions/2186. Minimum Number of Steps to Make Two Strings Anagram II/2186.py

```

class Solution:
    def minSteps(self, s: str, t: str) -> int:
        count = collections.Counter(s)
        count.subtract(collections.Counter(t))
        return sum([abs(c) for c in count.values()])

```

1124. 2187.py

Path: LeetCode-main/solutions/2187. Minimum Time to Complete Trips/2187.py

```
class Solution:
    def minimumTime(self, time: list[int], totalTrips: int) -> int:
        l = 1
        r = min(time) * totalTrips

        while l < r:
            m = (l + r) // 2
            if sum(m // t for t in time) >= totalTrips:
                r = m
            else:
                l = m + 1

        return l
```

1125. 2188.py

Path: LeetCode-main/solutions/2188. Minimum Time to Finish the Race/2188.py

```
class Solution:
    def minimumFinishTime(
        self,
        tires: list[list[int]],
        changeTime: int,
        numLaps: int,
    ) -> int:
        # singleTire[i] := the minimum time to finish i laps without changing tire
        singleTire = [math.inf] * (numLaps + 1)
        # dp[i] := the minimum time to finish i laps
        dp = [math.inf] * (numLaps + 1)

        for i, (f, r) in enumerate(tires):
            sumSecs = 0
            rPower = 1
            for j in range(1, numLaps + 1):
                # the time to use the same tire for the next lap >=
                # the time to change a new tire + f
                if f * rPower >= changeTime + f:
                    break
                sumSecs += f * rPower
                rPower *= r
                singleTire[j] = min(singleTire[j], sumSecs)

        dp[0] = 0
        for i in range(1, numLaps + 1):
            for j in range(1, i + 1):
                dp[i] = min(dp[i], dp[i - j] + changeTime + singleTire[j])

        return dp[numLaps] - changeTime
```

1126. 2189.py

Path: LeetCode-main/solutions/2189. Number of Ways to Build House of Cards/2189.py

```

class Solution:
    def houseOfCards(self, n: int) -> int:
        # dp[i] := the number of valid result for i cards
        dp = [1] + [0] * n

        for baseCards in range(2, n + 1, 3):
            for i in range(n, baseCards - 1, -1):
                # Use `baseCards` as the base, so we're left with `i - baseCards` cards.
                dp[i] += dp[i - baseCards]

        return dp[n]

```

1127. 219.py

Path: LeetCode-main/solutions/219. Contains Duplicate II/219.py

```

class Solution:
    def containsNearbyDuplicate(self, nums: list[int], k: int) -> bool:
        seen = set()

        for i, num in enumerate(nums):
            if i > k:
                seen.remove(nums[i - k - 1])
            if num in seen:
                return True
            seen.add(num)

        return False

```

1128. 2190.py

Path: LeetCode-main/solutions/2190. Most Frequent Number Following Key In an Array/2190.py

```

class Solution:
    def mostFrequent(self, nums: list[int], key: int) -> int:
        count = collections.Counter()

        for a, b in itertools.pairwise(nums):
            if a == key:
                count[b] += 1

        return max(count, key=lambda num: count[num])

```

1129. 2191.py

Path: LeetCode-main/solutions/2191. Sort the Jumbled Numbers/2191.py

```

class Solution:
    def sortJumbled(self, mapping: list[int], nums: list[int]) -> list[int]:
        def getMapped(num: int) -> int:
            mapped = []
            for c in str(num):
                mapped.append(str(mapping[int(c)]))
            return int(''.join(mapped))
        A = [(getMapped(num), i, num) for i, num in enumerate(nums)]
        return [num for _, i, num in sorted(A)]

```

1130. 2192-2.py

Path: LeetCode-main/solutions/2192. All Ancestors of a Node in a Directed Acyclic Graph/2192-2.py

```
class Solution:
    def getAncestors(self, n: int, edges: list[list[int]]) -> list[list[int]]:
        ans = [set() for _ in range(n)]
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n

        # Build the graph.
        for u, v in edges:
            graph[u].append(v)
            inDegrees[v] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        while q:
            for _ in range(len(q)):
                u = q.popleft()
                for v in graph[u]:
                    ans[v].add(u)
                    ans[v].update(ans[u])
                    inDegrees[v] -= 1
                    if inDegrees[v] == 0:
                        q.append(v)

        return [sorted(nodes) for nodes in ans]
```

1131. 2192.py

Path: LeetCode-main/solutions/2192. All Ancestors of a Node in a Directed Acyclic Graph/2192.py

```
class Solution:
    def getAncestors(self, n: int, edges: list[list[int]]) -> list[list[int]]:
        ans = [[] for _ in range(n)]
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)

        def dfs(u: int, ancestor: int, seen: set[int]) -> None:
            seen.add(u)
            for v in graph[u]:
                if v in seen:
                    continue
                ans[v].append(ancestor)
                dfs(v, ancestor, seen)

        for i in range(n):
            dfs(i, i, set())

        return ans
```

1132. 2193.py

Path: LeetCode-main/solutions/2193. Minimum Number of Moves to Make Palindrome/2193.py

```

class Solution:
    def minMovesToMakePalindrome(self, s: str) -> int:
        ans = 0
        chars = list(s)

        while len(chars) > 1:
            # Greedily match the last digit.
            i = chars.index(chars[-1])
            if i == len(chars) - 1:
                # s[i] is the middle letter.
                ans += i // 2
            else:
                chars.pop(i)
                ans += i # Swap the matched letter to the left.
                chars.pop()

        return ans

```

1133. 2194.py

Path: LeetCode-main/solutions/2194. Cells in a Range on an Excel Sheet/2194.py

```

class Solution:
    def cellsInRange(self, s: str) -> list[str]:
        ans = []
        startCol, startRow, _, endCol, endRow = s

        for j in range(ord(startCol), ord(endCol) + 1):
            for i in range(int(startRow), int(endRow) + 1):
                ans.append(chr(j) + str(i))

        return ans

```

1134. 2195.py

Path: LeetCode-main/solutions/2195. Append K Integers With Minimal Sum/2195.py

```

class Solution:
    def minimalKSum(self, nums: list[int], k: int) -> int:
        ans = 0
        nums.append(0)
        nums.sort()

        for a, b in zip(nums, nums[1:]):
            if a == b:
                continue
            l = a + 1
            r = min(a + k, b - 1)
            ans += (l + r) * (r - l + 1) // 2
            k -= r - l + 1
            if k == 0:
                return ans

        if k > 0:
            l = nums[-1] + 1
            r = nums[-1] + k
            ans += (l + r) * (r - l + 1) // 2

        return ans

```

1135. 2196.py

Path: LeetCode-main/solutions/2196. Create Binary Tree From Descriptions/2196.py

```
class Solution:
    def createBinaryTree(self, descriptions: list[list[int]]) -> TreeNode | None:
        children = set()
        valToNode = {}

        for p, c, isLeft in descriptions:
            parent = valToNode.setdefault(p, TreeNode(p))
            child = valToNode.setdefault(c, TreeNode(c))
            if isLeft:
                parent.left = child
            else:
                parent.right = child
            children.add(c)

        root = (set(valToNode) - set(children)).pop()
        return valToNode[root]
```

1136. 2197.py

Path: LeetCode-main/solutions/2197. Replace Non-Coprime Numbers in Array/2197.py

```
class Solution:
    def replaceNonCoprimes(self, nums: list[int]) -> list[int]:
        ans = []

        for num in nums:
            while ans and math.gcd(ans[-1], num) > 1:
                num = math.lcm(ans.pop(), num)
            ans.append(num)

        return ans
```

1137. 2198.py

Path: LeetCode-main/solutions/2198. Number of Single Divisor Triplets/2198.py

```

class Solution:
    def singleDivisorTriplet(self, nums: list[int]) -> int:
        ans = 0
        count = collections.Counter(nums)

        def divisible(summ: int, num: int) -> int:
            return summ % num == 0

        for a in range(1, 101):
            if count[a] == 0:
                continue
            for b in range(a, 101):
                if count[b] == 0:
                    continue
                for c in range(b, 101):
                    if count[c] == 0:
                        continue
                    summ = a + b + c
                    if divisible(summ, a) + divisible(summ, b) + divisible(summ, c) != 1:
                        continue
                    if a == b:
                        ans += count[a] * (count[a] - 1) // 2 * count[c]
                    elif b == c:
                        ans += count[b] * (count[b] - 1) // 2 * count[a]
                    else:
                        ans += count[a] * count[b] * count[c]

        return ans * 6

```

1138. 22.py

Path: LeetCode-main/solutions/22. Generate Parentheses/22.py

```

class Solution:
    def generateParenthesis(self, n):
        ans = []

        def dfs(l: int, r: int, s: list[str]) -> None:
            if l == 0 and r == 0:
                ans.append(''.join(s))
            if l > 0:
                s.append('(')
                dfs(l - 1, r, s)
                s.pop()
            if l < r:
                s.append(')')
                dfs(l, r - 1, s)
                s.pop()

        dfs(n, n, [])
        return ans

```

1139. 220-2.py

Path: LeetCode-main/solutions/220. Contains Duplicate III/220-2.py

```

class Solution:
    def containsNearbyAlmostDuplicate(
        self,
        nums: list[int],
        indexDiff: int,
        valueDiff: int,
    ) -> bool:
        if not nums or indexDiff <= 0 or valueDiff < 0:
            return False

        mn = min(nums)
        diff = valueDiff + 1 # In case that `valueDiff` equals 0.
        bucket = {}

        def getKey(num: int) -> int:
            return (num - mn) // diff

        for i, num in enumerate(nums):
            key = getKey(num)
            if key in bucket: # the current bucket
                return True
            # the left adjacent bucket
            if key - 1 in bucket and num - bucket[key - 1] < diff:
                return True
            # the right adjacent bucket
            if key + 1 in bucket and bucket[key + 1] - num < diff:
                return True
            bucket[key] = num
            if i >= indexDiff:
                del bucket[getKey(nums[i - indexDiff])]

        return False

```

1140. 2200.py

Path: LeetCode-main/solutions/2200. Find All K-Distant Indices in an Array/2200.py

```

class Solution:
    def findKDistantIndices(self, nums: list[int], key: int, k: int) -> list[int]:
        n = len(nums)
        ans = []

        j = 0
        for i in range(n):
            # the first index j s.t. nums[j] == key and j >= i - k
            while j < n and (nums[j] != key or j < i - k):
                j += 1
            if j == n:
                break
            if abs(i - j) <= k:
                ans.append(i)

        return ans

```

1141. 2201.py

Path: LeetCode-main/solutions/2201. Count Artifacts That Can Be Extracted/2201.py

```
class Solution:
    def digArtifacts(
        self,
        n: int,
        artifacts: list[list[int]],
        dig: list[list[int]],
    ) -> int:
        digged = set((r, c) for r, c in dig)

        def canExtract(a: list[int]) -> bool:
            for i in range(a[0], a[2] + 1):
                for j in range(a[1], a[3] + 1):
                    if (i, j) not in digged:
                        return False
            return True

        return sum(canExtract(a) for a in artifacts)
```

1142. 2202.py

Path: LeetCode-main/solutions/2202. Maximize the Topmost Element After K Moves/2202.py

```
class Solution:
    def maximumTop(self, nums: list[int], k: int) -> int:
        n = len(nums)
        # After taking k elements, if we're left something, then we return nums[k]
        # Otherwise, return -1.
        if k == 0 or k == 1:
            return -1 if n == k else nums[k]
        # Remove then add even number of times.
        if n == 1:
            return -1 if k & 1 else nums[0]
        # Take min(n, k - 1) elements and put the largest one back.
        mx = max(nums[:min(n, k - 1)])
        if k >= n:
            return mx
        return max(mx, nums[k])
```

1143. 2203.py

Path: LeetCode-main/solutions/2203. Minimum Weighted Subgraph With the Required Paths/2203.py


```

class Solution:
    def minimumWeight(
        self,
        n: int,
        edges: list[list[int]],
        src1: int,
        src2: int,
        dest: int,
    ) -> int:
        graph = [[] for _ in range(n)]
        reversedGraph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))
            reversedGraph[v].append((u, w))

        fromSrc1 = self._dijkstra(graph, src1)
        fromSrc2 = self._dijkstra(graph, src2)
        fromDest = self._dijkstra(reversedGraph, dest)
        minWeight = min(a + b + c for a, b, c in zip(fromSrc1, fromSrc2, fromDest))
        return -1 if minWeight == math.inf else minWeight

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
    ) -> list[int]:
        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)]  # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        return dist

```

1144. 2204.py

Path: LeetCode-main/solutions/2204. Distance to a Cycle in Undirected Graph/2204.py

```

class Solution:
    def distanceToCycle(self, n: int, edges: list[list[int]]) -> list[int]:
        ans = [0] * n
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        NO_RANK = -2

        # The minRank that u can reach with forward edges
        def getRank(u: int, currRank: int, rank: list[int]) -> int:
            if rank[u] != NO_RANK: # The rank is already determined
                return rank[u]

            rank[u] = currRank
            minRank = currRank

            for v in graph[u]:
                # Visited or parent (that's why NO_RANK = -2 instead of -1)
                if rank[v] == len(rank) or rank[v] == currRank - 1:
                    continue
                nextRank = getRank(v, currRank + 1, rank)
                # NextRank should > currRank if there's no cycle
                if nextRank <= currRank:
                    cycle.append(v)
                minRank = min(minRank, nextRank)

            rank[u] = len(rank) # Mark as visited.
            return minRank

        # rank[i] := the minimum node that node i can reach with forward edges
        # Initialize with NO_RANK = -2 to indicate not visited.
        cycle = []
        getRank(0, 0, [NO_RANK] * n)

        q = collections.deque(cycle)
        seen = set(cycle)

        step = 1
        while q:
            for _ in range(len(q)):
                u = q.popleft()
                for v in graph[u]:
                    if v in seen:
                        continue
                    q.append(v)
                    seen.add(v)
                    ans[v] = step
                step += 1

        return ans

```

1145. 2206.py

Path: LeetCode-main/solutions/2206. Divide Array Into Equal Pairs/2206.py

```

class Solution:
    def divideArray(self, nums: list[int]) -> bool:
        return all(value % 2 == 0 for value in collections.Counter(nums).values())

```

1146. 2207.py

Path: LeetCode-main/solutions/2207. Maximize Number of Subsequences in a String/2207.py

```
class Solution:
    def maximumSubsequenceCount(self, text: str, pattern: str) -> int:
        ans = 0
        count0 = 0 # the count of the letter pattern[0]
        count1 = 0 # the count of the letter pattern[1]

        for c in text:
            if c == pattern[1]:
                ans += count0
                count1 += 1
            if c == pattern[0]:
                count0 += 1

        # It is optimal to add pattern[0] at the beginning or add pattern[1] at the
        # end of the text.
        return ans + max(count0, count1)
```

1147. 2208.py

Path: LeetCode-main/solutions/2208. Minimum Operations to Halve Array Sum/2208.py

```
class Solution:
    def halveArray(self, nums: list[int]) -> int:
        halfSum = sum(nums) / 2
        ans = 0
        runningSum = 0.0
        maxHeap = [-num for num in nums]

        heapq.heapify(maxHeap)

        while runningSum < halfSum:
            maxValue = -heapq.heappop(maxHeap) / 2
            runningSum += maxValue
            heapq.heappush(maxHeap, -maxValue)
            ans += 1

        return ans
```

1148. 2209-2.py

Path: LeetCode-main/solutions/2209. Minimum White Tiles After Covering With Carpets/2209-2.py

```

class Solution:
    def minimumWhiteTiles(
        self,
        floor: str,
        numCarpets: int,
        carpetLen: int,
    ) -> int:
        n = len(floor)
        # dp[i][j] := the minimum number of visible white tiles of floor[i..n)
        # after covering at most j carpets
        dp = [[0] * (numCarpets + 1) for _ in range(n + 1)]

        for i in reversed(range(n)):
            dp[i][0] = int(floor[i]) + dp[i + 1][0]

        for i in reversed(range(n)):
            for j in range(1, numCarpets + 1):
                cover = dp[i + carpetLen][j - 1] if i + carpetLen < n else 0
                skip = int(floor[i]) + dp[i + 1][j]
                dp[i][j] = min(cover, skip)

        return dp[0][numCarpets]

```

1149. 2209.py

Path: LeetCode-main/solutions/2209. Minimum White Tiles After Covering With Carpets/2209.py

```

class Solution:
    def minimumWhiteTiles(
        self,
        floor: str,
        numCarpets: int,
        carpetLen: int,
    ) -> int:
        MAX = 1000

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """
            Returns the minimum number of visible white tiles of floor[i..n) after
            covering at most j carpets.
            """
            if j < 0:
                return MAX
            if i >= len(floor):
                return 0
            return min(dp(i + carpetLen, j - 1),
                       dp(i + 1, j) + int(floor[i]))

        return dp(0, numCarpets)

```

1150. 221-2.py

Path: LeetCode-main/solutions/221. Maximal Square/221-2.py

```

class Solution:
    def maximalSquare(self, matrix: list[list[str]]) -> int:
        m = len(matrix)
        n = len(matrix[0])
        dp = [0] * n
        maxLength = 0
        prev = 0 # dp[i - 1][j - 1]

        for i in range(m):
            for j in range(n):
                cache = dp[j]
                if i == 0 or j == 0 or matrix[i][j] == '0':
                    dp[j] = 1 if matrix[i][j] == '1' else 0
                else:
                    dp[j] = min([prev, dp[j], dp[j - 1]]) + 1
                    maxLength = max(maxLength, dp[j])
                prev = cache

        return maxLength * maxLength

```

1151. 221.py

Path: LeetCode-main/solutions/221. Maximal Square/221.py

```

class Solution:
    def maximalSquare(self, matrix: list[list[str]]) -> int:
        m = len(matrix)
        n = len(matrix[0])
        dp = [[0] * n for _ in range(m)]
        maxLength = 0

        for i in range(m):
            for j in range(n):
                if i == 0 or j == 0 or matrix[i][j] == '0':
                    dp[i][j] = 1 if matrix[i][j] == '1' else 0
                else:
                    dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1
                    maxLength = max(maxLength, dp[i][j])

        return maxLength * maxLength

```

1152. 2210.py

Path: LeetCode-main/solutions/2210. Count Hills and Valleys in an Array/2210.py

```

class Solution:
    def countHillValley(self, nums: list[int]) -> int:
        ans = 0
        left = nums[0]

        for i in range(1, len(nums) - 1):
            if (left < nums[i] and nums[i] > nums[i + 1] or # the hill
                left > nums[i] and nums[i] < nums[i + 1]): # the valley
                ans += 1
            left = nums[i]

        return ans

```

1153. 2211.py

Path: LeetCode-main/solutions/2211. Count Collisions on a Road/2211.py

```
class Solution:
    def countCollisions(self, directions: str) -> int:
        l = 0
        r = len(directions) - 1

        while l < len(directions) and directions[l] == 'L':
            l += 1

        while r >= 0 and directions[r] == 'R':
            r -= 1

        return sum(c != 'S' for c in directions[l:r + 1])
```

1154. 2212.py

Path: LeetCode-main/solutions/2212. Maximum Points in an Archery Competition/2212.py

```
class Solution:
    def maximumBobPoints(
        self,
        numArrows: int,
        aliceArrows: list[int],
    ) -> list[int]:
        FULL_MASK = (1 << 12) - 1
        maxPoint = 0
        maxMask = 0

        def getShorableAndPoint(mask: int, leftArrows: int) -> tuple[bool, int]:
            point = 0
            for i in range(12):
                if mask >> i & 1:
                    leftArrows -= aliceArrows[i] + 1
                    point += i
            return leftArrows >= 0, point

        for mask in range(FULL_MASK):
            shorable, point = getShorableAndPoint(mask, numArrows)
            if shorable and point > maxPoint:
                maxPoint = point
                maxMask = mask

        def getBobsArrows(mask: int, leftArrows: int) -> list[int]:
            bobsArrows = [0] * 12
            for i in range(12):
                if mask >> i & 1:
                    bobsArrows[i] = aliceArrows[i] + 1
                    leftArrows -= aliceArrows[i] + 1
            bobsArrows[0] = leftArrows
            return bobsArrows

        return getBobsArrows(maxMask, numArrows)
```

1155. 2214.py

Path: LeetCode-main/solutions/2214. Minimum Health to Beat Game/2214.py

```
class Solution:
    def minimumHealth(self, damage: list[int], armor: int) -> int:
        return 1 + sum(damage) - min(max(damage), armor)
```

1156. 2215.py

Path: LeetCode-main/solutions/2215. Find the Difference of Two Arrays/2215.py

```
class Solution:
    def findDifference(self, nums1: list[int],
                      nums2: list[int]) -> list[list[int]]:
        set1 = set(nums1)
        set2 = set(nums2)
        return [set1 - set2, set2 - set1]
```

1157. 2216.py

Path: LeetCode-main/solutions/2216. Minimum Deletions to Make Array Beautiful/2216.py

```
class Solution:
    def minDeletion(self, nums: list[int]) -> int:
        ans = 0

        for i in range(len(nums) - 1):
            # i - ans := the index after deletion
            if nums[i] == nums[i + 1] and (i - ans) % 2 == 0:
                ans += 1

        # Add one if the length after deletion is odd
        return ans + ((len(nums) - ans) & 1)
```

1158. 2217.py

Path: LeetCode-main/solutions/2217. Find Palindrome With Fixed Length/2217.py

```
class Solution:
    def kthPalindrome(self, queries: list[int], intLength: int) -> list[int]:
        start = pow(10, (intLength + 1) // 2 - 1)
        end = pow(10, (intLength + 1) // 2)
        mul = pow(10, intLength // 2)

        def reverse(num: int) -> int:
            res = 0
            while num:
                res = res * 10 + num % 10
                num //= 10
            return res

        def getKthPalindrome(query: int) -> int:
            prefix = start + query - 1
            return prefix * mul + reverse(prefix)
                if intLength % 2 == 0 else prefix // 10)

        return [-1 if start + query > end else getKthPalindrome(query)
                for query in queries]
```

1159. 2218.py

Path: LeetCode-main/solutions/2218. Maximum Value of K Coins From Piles/2218.py

```
class Solution:
    def maxValueOfCoins(self, piles: list[list[int]], k: int) -> int:
        @functools.lru_cache(None)
        def dp(i: int, k: int) -> int:
            """Returns the maximum value of picking k coins from piles[i..n)."""
            if i == len(piles) or k == 0:
                return 0

            # Pick no coins from the current pile.
            res = dp(i + 1, k)
            val = 0 # the coins picked from the current pile

            # Try to pick 1, 2, ..., k coins from the current pile.
            for j in range(min(len(piles[i]), k)):
                val += piles[i][j]
                res = max(res, val + dp(i + 1, k - j - 1))

            return res

        return dp(0, k)
```

1160. 2219.py

Path: LeetCode-main/solutions/2219. Maximum Sum Score of Array/2219.py

```
class Solution:
    def maximumSumScore(self, nums: list[int]) -> int:
        ans = -math.inf
        prefix = 0
        summ = sum(nums)

        for num in nums:
            prefix += num
            ans = max(ans, prefix, summ - prefix + num)

        return ans
```

1161. 222-2.py

Path: LeetCode-main/solutions/222. Count Complete Tree Nodes/222-2.py


```

class Solution:
    def countNodes(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        left = root
        right = root
        heightL = 0
        heightR = 0

        while left:
            heightL += 1
            left = left.left

        while right:
            heightR += 1
            right = right.right

        if heightL == heightR: # `root` is a complete tree.
            return pow(2, heightL) - 1
        return 1 + self.countNodes(root.left) + self.countNodes(root.right)

```

1162. 222.py

Path: LeetCode-main/solutions/222. Count Complete Tree Nodes/222.py

```

class Solution:
    def countNodes(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        return 1 + self.countNodes(root.left) + self.countNodes(root.right)

```

1163. 2220.py

Path: LeetCode-main/solutions/2220. Minimum Bit Flips to Convert Number/2220.py

```

class Solution:
    def minBitFlips(self, start: int, goal: int) -> int:
        return (start ^ goal).bit_count()

```

1164. 2221.py

Path: LeetCode-main/solutions/2221. Find Triangular Sum of an Array/2221.py

```

class Solution:
    def triangularSum(self, nums: list[int]) -> int:
        for sz in range(len(nums), 0, -1):
            for i in range(sz - 1):
                nums[i] = (nums[i] + nums[i + 1]) % 10
        return nums[0]

```

1165. 2222.py

Path: LeetCode-main/solutions/2222. Number of Ways to Select Buildings/2222.py

```

class Solution:
    def numberOfWays(self, s: str) -> int:
        ans = 0
        # before[i] := the number of i before the current digit
        before = [0] * 2
        # after[i] := the number of i after the current digit
        after = [0] * 2
        after[0] = s.count('0')
        after[1] = len(s) - after[0]

        for c in s:
            num = int(c)
            after[num] -= 1
            if num == 0:
                ans += before[1] * after[1]
            else:
                ans += before[0] * after[0]
            before[num] += 1

        return ans

```

1166. 2223.py

Path: LeetCode-main/solutions/2223. Sum of Scores of Built Strings/2223.py

```

class Solution:
    def sumScores(self, s: str) -> int:
        n = len(s)
        # https://cp-algorithms.com/string/z-function.html#implementation
        z = [0] * n
        # [l, r] := the indices of the rightmost segment match
        l = 0
        r = 0

        for i in range(1, n):
            if i < r:
                z[i] = min(r - i, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] > r:
                l = i
                r = i + z[i]

        return sum(z) + n

```

1167. 2224.py

Path: LeetCode-main/solutions/2224. Minimum Number of Operations to Convert Time/2224.py

```

class Solution:
    def convertTime(self, current: str, correct: str) -> int:
        ops = [60, 15, 5, 1]

        def getMinutes(s: str) -> int:
            return int(s[:2]) * 60 + int(s[3:])

        diff = getMinutes(correct) - getMinutes(current)
        ans = 0

        for op in ops:
            ans += diff // op
            diff %= op

        return ans

```

1168. 2225.py

Path: LeetCode-main/solutions/2225. Find Players With Zero or One Losses/2225.py

```

class Solution:
    def findWinners(self, matches: list[list[int]]) -> list[list[int]]:
        ans = [[] for _ in range(2)]
        lossesCount = collections.Counter()

        for winner, loser in matches:
            if winner not in lossesCount:
                lossesCount[winner] = 0
            lossesCount[loser] += 1

        for player, nLosses in lossesCount.items():
            if nLosses < 2:
                ans[nLosses].append(player)

        return [sorted(ans[0]), sorted(ans[1])]

```

1169. 2226.py

Path: LeetCode-main/solutions/2226. Maximum Candies Allocated to K Children/2226.py

```

class Solution:
    def maximumCandies(self, candies: list[int], k: int) -> int:
        l = 1
        r = sum(candies) // k

        def numChildren(m: int) -> bool:
            return sum(c // m for c in candies)

        while l < r:
            m = (l + r) // 2
            if numChildren(m) < k:
                r = m
            else:
                l = m + 1

        return l if numChildren(l) >= k else l - 1

```

1170. 2227-2.py

Path: LeetCode-main/solutions/2227. Encrypt and Decrypt Strings/2227-2.py

```
class Encrypter:
    def __init__(self, keys: list[str], values: list[str], dictionary: list[str]):
        self.keyToValue = {k: v for k, v in zip(keys, values)}
        self.decrypt = collections.Counter(self.encrypt(word)
                                           for word in dictionary).__getitem__

    def encrypt(self, word1: str) -> str:
        return ''.join(self.keyToValue[c] for c in word1)
```

1171. 2227.py

Path: LeetCode-main/solutions/2227. Encrypt and Decrypt Strings/2227.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Encrypter:
    def __init__(self, keys: list[str], values: list[str], dictionary: list[str]):
        self.keyToValue = {k: v for k, v in zip(keys, values)}
        self.valueToKeys = collections.defaultdict(list)
        self.root = TrieNode()
        for k, v in zip(keys, values):
            self.valueToKeys[v].append(k)
        for word in dictionary:
            self._insert(word)

    def encrypt(self, word1: str) -> str:
        return ''.join(self.keyToValue[c] for c in word1)

    def decrypt(self, word2: str) -> int:
        return self._find(word2, 0, self.root)

    def _insert(self, word: str) -> None:
        node = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

    def _find(self, word: str, i: int, node: TrieNode) -> int:
        value = word[i:i + 2]
        if value not in self.valueToKeys:
            return 0

        ans = 0
        if i + 2 == len(word):
            for key in self.valueToKeys[value]:
                if key in node.children and node.children[key].isWord:
                    ans += 1
            return ans

        for key in self.valueToKeys[value]:
            if key not in node.children:
                continue
            ans += self._find(word, i + 2, node.children[key])

        return ans

```

1172. 2229.py

Path: LeetCode-main/solutions/2229. Check if an Array Is Consecutive/2229.py

```

class Solution:
    def isConsecutive(self, nums: list[int]) -> bool:
        return max(nums) - min(nums) + 1 == len(set(nums)) == len(nums)

```

1173. 223.py

Path: LeetCode-main/solutions/223. Rectangle Area/223.py

```

class Solution:
    def computeArea(self,
                    A: int, B: int, C: int, D: int,
                    E: int, F: int, G: int, H: int) -> int:
        x = min(C, G) - max(A, E) if max(A, E) < min(C, G) else 0
        y = min(D, H) - max(B, F) if max(B, F) < min(D, H) else 0
        return (C - A) * (D - B) + (G - E) * (H - F) - x * y

```

1174. 2231.py

Path: LeetCode-main/solutions/2231. Largest Number After Digit Swaps by Parity/2231.py

```

class Solution:
    def largestInteger(self, num: int) -> int:
        s = str(num)
        ans = 0
        # maxHeap[0] := the odd digits
        # maxHeap[1] := the even digits
        maxHeap = [[] for _ in range(2)]

        for c in s:
            digit = int(c)
            heapq.heappush(maxHeap[digit % 2], -digit)

        for c in s:
            i = int(c) & 1
            ans = (ans * 10 - heapq.heappop(maxHeap[i]))

        return ans

```

1175. 2232.py

Path: LeetCode-main/solutions/2232. Minimize Result by Adding Parentheses to Expression/2232.py

```

class Solution:
    def minimizeResult(self, expression: str) -> str:
        plusIndex = expression.index('+')
        left = expression[:plusIndex]
        right = expression[plusIndex + 1:]
        ans = ''
        mn = math.inf

        # the expression -> a * (b + c) * d
        for i in range(len(left)):
            for j in range(len(right)):
                a = 1 if i == 0 else int(left[:i])
                b = int(left[i:])
                c = int(right[0:j + 1])
                d = 1 if j == len(right) - 1 else int(right[j + 1:])
                val = a * (b + c) * d
                if val < mn:
                    mn = val
                    ans = (('' if i == 0 else str(a)) +
                        '(' + str(b) + '+' + str(c) + ')' +
                        ('' if j == len(right) - 1 else str(d)))

        return ans

```

1176. 2233.py

Path: LeetCode-main/solutions/2233. Maximum Product After K Increments/2233.py

```
class Solution:
    def maximumProduct(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        ans = 1
        minHeap = nums.copy()
        heapq.heapify(minHeap)

        for _ in range(k):
            minNum = heapq.heappop(minHeap)
            heapq.heappush(minHeap, minNum + 1)

        while minHeap:
            ans *= heapq.heappop(minHeap)
            ans %= MOD

        return ans
```

1177. 2234.py

Path: LeetCode-main/solutions/2234. Maximum Total Beauty of the Gardens/2234.py

```

class Solution:
    def maximumBeauty(
        self,
        flowers: list[int],
        newFlowers: int,
        target: int,
        full: int,
        partial: int,
    ) -> int:
        n = len(flowers)

        # If a garden is already complete, clamp it to the target.
        flowers = [min(flower, target) for flower in flowers]
        flowers.sort()

        # All gardens are complete, so nothing we can do.
        if flowers[0] == target:
            return n * full

        # Having many new flowers maximizes the beauty value.
        if newFlowers >= n * target - sum(flowers):
            return max(n * full, (n - 1) * full + (target - 1) * partial)

        ans = 0
        leftFlowers = newFlowers
        # cost[i] := the cost to make flowers[0..i] the same
        cost = [0] * n

        for i in range(1, n):
            # Plant (flowers[i] - flowers[i - 1]) flowers for flowers[0..i - 1].
            cost[i] = cost[i - 1] + i * (flowers[i] - flowers[i - 1])

        i = n - 1 # flowers' index (flowers[i + 1..n) are complete)
        while flowers[i] == target:
            i -= 1

        while leftFlowers >= 0:
            # To maximize the minimum number of incomplete flowers, we find the first
            # index j that we can't make flowers[0..j] equal to flowers[j], then we
            # know we can make flowers[0..j - 1] equal to flowers[j - 1]. In the
            # meantime, evenly increase each of them to seek a bigger minimum value.
            j = min(i + 1, bisect_right(cost, leftFlowers))
            minIncomplete = flowers[j - 1] + (leftFlowers - cost[j - 1]) // j
            ans = max(ans, (n - 1 - i) * full + minIncomplete * partial)
            leftFlowers -= max(0, target - flowers[i])
            i -= 1

        return ans

```

1178. 2235.py

Path: LeetCode-main/solutions/2235. Add Two Integers/2235.py

```

class Solution:
    sum = operator.add

```

1179. 2236.py

Path: LeetCode-main/solutions/2236. Root Equals Sum of Children/2236.py


```
class Solution:
    def checkTree(self, root: TreeNode | None) -> bool:
        return root.val == root.left.val + root.right.val
```

1180. 2237.py

Path: LeetCode-main/solutions/2237. Count Positions on Street With Required Brightness/2237.py

```
class Solution:
    def meetRequirement(
        self,
        n: int,
        lights: list[list[int]],
        requirement: list[int],
    ) -> int:
        ans = 0
        currBrightness = 0
        change = [0] * (n + 1)

        for position, rg in lights:
            change[max(0, position - rg)] += 1
            change[min(n, position + rg + 1)] -= 1

        for i in range(n):
            currBrightness += change[i]
            if currBrightness >= requirement[i]:
                ans += 1

        return ans
```

1181. 2239.py

Path: LeetCode-main/solutions/2239. Find Closest Number to Zero/2239.py

```
class Solution:
    def findClosestNumber(self, nums: list[int]) -> int:
        nums.sort(key=lambda x: (abs(x), -x))
        return nums[0]
```

1182. 224.py

Path: LeetCode-main/solutions/224. Basic Calculator/224.py

```

class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        num = 0
        sign = 1
        stack = [sign] # stack[-1]: the current environment's sign

        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '(':
                stack.append(sign)
            elif c == ')':
                stack.pop()
            elif c == '+' or c == '-':
                ans += sign * num
                sign = (1 if c == '+' else -1) * stack[-1]
                num = 0

        return ans + sign * num

```

1183. 2240.py

Path: LeetCode-main/solutions/2240. Number of Ways to Buy Pens and Pencils/2240.py

```

class Solution:
    def waysToBuyPensPencils(self, total: int, cost1: int, cost2: int) -> int:
        maxPen = total // cost1
        return sum((total - i * cost1) // cost2
                    for i in range(maxPen + 1)) + maxPen + 1

```

1184. 2241.py

Path: LeetCode-main/solutions/2241. Design an ATM Machine/2241.py

```

class ATM:
    def __init__(self):
        self.banknotes = [20, 50, 100, 200, 500]
        self.bank = [0] * 5

    def deposit(self, banknotesCount: list[int]) -> None:
        for i in range(5):
            self.bank[i] += banknotesCount[i]

    def withdraw(self, amount: int) -> list[int]:
        withdrew = [0] * 5

        for i in reversed(range(5)):
            withdrew[i] = min(self.bank[i], amount // self.banknotes[i])
            amount -= withdrew[i] * self.banknotes[i]

        if amount:
            return [-1]

        for i in range(5):
            self.bank[i] -= withdrew[i]
        return withdrew

```

1185. 2242.py

Path: LeetCode-main/solutions/2242. Maximum Score of a Node Sequence/2242.py

```
class Solution:
    def maximumScore(self, scores: list[int], edges: list[list[int]]) -> int:
        n = len(scores)
        ans = -1
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append((scores[v], v))
            graph[v].append((scores[u], u))

        for i in range(n):
            graph[i] = heapq.nlargest(3, graph[i])

        # To find the target sequence: a - u - v - b, enumerate each edge (u, v),
        # and find a (u's child) and b (v's child). That's why we find the 3
        # children that have the highest scores because one of the 3 children is
        # guaranteed to be valid.
        for u, v in edges:
            for scoreA, a in graph[u]:
                for scoreB, b in graph[v]:
                    if a != b and a != v and b != u:
                        ans = max(ans, scoreA + scores[u] + scores[v] + scoreB)

        return ans
```

1186. 2243.py

Path: LeetCode-main/solutions/2243. Calculate Digit Sum of a String/2243.py

```
class Solution:
    def digitSum(self, s: str, k: int) -> str:
        while len(s) > k:
            next = []
            for i in range(0, len(s), k):
                summ = 0
                for j in range(i, min(len(s), i + k)):
                    summ += int(s[j])
                next.append(str(summ))
            s = ''.join(next)
        return s
```

1187. 2244.py

Path: LeetCode-main/solutions/2244. Minimum Rounds to Complete All Tasks/2244.py

```
class Solution:
    def minimumRounds(self, tasks: list[int]) -> int:
        freqs = collections.Counter(tasks).values()
        return -1 if 1 in freqs else sum((f + 2) // 3 for f in freqs)
```

1188. 2245.py

Path: LeetCode-main/solutions/2245. Maximum Trailing Zeros in a Cornered Path/2245.py

```

class Solution:
    def maxTrailingZeros(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        # leftPrefix2[i][j] := the number of 2 in grid[i][0..j]
        # leftPrefix5[i][j] := the number of 5 in grid[i][0..j]
        # topPrefix2[i][j] := the number of 2 in grid[0..i][j]
        # topPrefix5[i][j] := the number of 5 in grid[0..i][j]
        leftPrefix2 = [[0] * n for _ in range(m)]
        leftPrefix5 = [[0] * n for _ in range(m)]
        topPrefix2 = [[0] * n for _ in range(m)]
        topPrefix5 = [[0] * n for _ in range(m)]

        def getCount(num: int, factor: int) -> int:
            count = 0
            while num % factor == 0:
                num //= factor
                count += 1
            return count

        for i in range(m):
            for j in range(n):
                leftPrefix2[i][j] = getCount(grid[i][j], 2)
                leftPrefix5[i][j] = getCount(grid[i][j], 5)
                if j:
                    leftPrefix2[i][j] += leftPrefix2[i][j - 1]
                    leftPrefix5[i][j] += leftPrefix5[i][j - 1]

        for j in range(n):
            for i in range(m):
                topPrefix2[i][j] = getCount(grid[i][j], 2)
                topPrefix5[i][j] = getCount(grid[i][j], 5)
                if i:
                    topPrefix2[i][j] += topPrefix2[i - 1][j]
                    topPrefix5[i][j] += topPrefix5[i - 1][j]

        ans = 0
        for i in range(m):
            for j in range(n):
                curr2 = getCount(grid[i][j], 2)
                curr5 = getCount(grid[i][j], 5)
                l2 = leftPrefix2[i][j]
                l5 = leftPrefix5[i][j]
                r2 = leftPrefix2[i][n - 1] - (0 if j == 0 else leftPrefix2[i][j - 1])
                r5 = leftPrefix5[i][n - 1] - (0 if j == 0 else leftPrefix5[i][j - 1])
                t2 = topPrefix2[i][j]
                t5 = topPrefix5[i][j]
                d2 = topPrefix2[m - 1][j] - (0 if i == 0 else topPrefix2[i - 1][j])
                d5 = topPrefix5[m - 1][j] - (0 if i == 0 else topPrefix5[i - 1][j])
                ans = max(ans,
                           min(l2 + t2 - curr2, l5 + t5 - curr5),
                           min(r2 + t2 - curr2, r5 + t5 - curr5),
                           min(l2 + d2 - curr2, l5 + d5 - curr5),
                           min(r2 + d2 - curr2, r5 + d5 - curr5))

        return ans

```

1189. 2246.py

Path: LeetCode-main/solutions/2246. Longest Path With Different Adjacent Characters/2246.py

```
class Solution:
    def longestPath(self, parent: list[int], s: str) -> int:
        n = len(parent)
        ans = 0
        graph = [[] for _ in range(n)]

        for i in range(1, n):
            graph[parent[i]].append(i)

        def longestPathDownFrom(u: int) -> int:
            nonlocal ans
            max1 = 0
            max2 = 0

            for v in graph[u]:
                res = longestPathDownFrom(v)
                if s[u] == s[v]:
                    continue
                if res > max1:
                    max2 = max1
                    max1 = res
                elif res > max2:
                    max2 = res

            ans = max(ans, 1 + max1 + max2)
            return 1 + max1

        longestPathDownFrom(0)
        return ans
```

1190. 2247.py

Path: LeetCode-main/solutions/2247. Maximum Cost of Trip With K Highways/2247.py

```

class Solution:
    def maximumCost(self, n: int, highways: list[list[int]], k: int) -> int:
        if k + 1 > n:
            return -1

        graph = [[] for _ in range(n)]

        for u, v, w in highways:
            graph[u].append((v, w))
            graph[v].append((u, w))

        @functools.lru_cache(None)
        def dp(u: int, mask: int) -> int:
            """
            Returns the maximum cost of trip starting from u, where `mask` is the
            bitmask of the visited cities.
            """
            if mask.bit_count() == k + 1:
                return 0

            res = -1
            for v, w in graph[u]:
                if mask >> v & 1:
                    continue
                nextCost = dp(v, mask | 1 << v)
                if nextCost != -1:
                    res = max(res, w + nextCost)
            return res

        return max(dp(i, 1 << i) for i in range(n))

```

1191. 2248.py

Path: LeetCode-main/solutions/2248. Intersection of Multiple Arrays/2248.py

```

class Solution:
    def intersection(self, nums: list[list[int]]) -> list[int]:
        count = [0] * 1001

        for row in nums:
            for a in row:
                count[a] += 1

        return [i for i, c in enumerate(count)
                if c == len(nums)]

```

1192. 2249-2.py

Path: LeetCode-main/solutions/2249. Count Lattice Points Inside a Circle/2249-2.py

```
class Solution:
    def countLatticePoints(self, circles: list[list[int]]) -> int:
        points = set()

        # dx := relative to x
        # dy := relative to y
        # So, dx^2 + dy^2 = r^2.
        for x, y, r in circles:
            for dx in range(-r, r + 1):
                yMax = int((r**2 - dx**2)**0.5)
                for dy in range(-yMax, yMax + 1):
                    points.add((x + dx, y + dy))

        return len(points)
```

1193. 2249.py

Path: LeetCode-main/solutions/2249. Count Lattice Points Inside a Circle/2249.py

```
class Solution:
    def countLatticePoints(self, circles: list[list[int]]) -> int:
        return sum(any((xc - x)**2 + (yc - y)**2 <= r**2 for xc, yc, r in circles)
                    for x in range(201)
                    for y in range(201))
```

1194. 225.py

Path: LeetCode-main/solutions/225. Implement Stack using Queues/225.py

```
class MyStack:
    def __init__(self):
        self.q = collections.deque()

    def push(self, x: int) -> None:
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self) -> int:
        return self.q.popleft()

    def top(self) -> int:
        return self.q[0]

    def empty(self) -> bool:
        return not self.q
```

1195. 2250.py

Path: LeetCode-main/solutions/2250. Count Number of Rectangles Containing Each Point/2250.py

```
class Solution:
    def countRectangles(
        self,
        rectangles: list[list[int]],
        points: list[list[int]],
    ) -> list[int]:
        ans = []
        yToXs = [[] for _ in range(101)]

        for l, h in rectangles:
            yToXs[h].append(l)

        for xs in yToXs:
            xs.sort()

        for xi, yi in points:
            count = 0
            for y in range(yi, 101):
                xs = yToXs[y]
                count += len(xs) - bisect.bisect_left(xs, xi)
            ans.append(count)

        return ans
```

1196. 2251.py

Path: LeetCode-main/solutions/2251. Number of Flowers in Full Bloom/2251.py

```
class Solution:
    def fullBloomFlowers(
        self,
        flowers: list[list[int]],
        persons: list[int],
    ) -> list[int]:
        starts = sorted(s for s, _ in flowers)
        ends = sorted(e for _, e in flowers)
        return [bisect.bisect_right(starts, person) -
                bisect.bisect_left(ends, person)
                for person in persons]
```

1197. 2254.py

Path: LeetCode-main/solutions/2254. Design Video Sharing Platform/2254.py


```

class VideoSharingPlatform:
    def __init__(self):
        self.currVideoId = 0
        self.usedIds = []
        self.videoIdToVideo = {}
        self.videoIdToViews = collections.Counter()
        self.videoIdToLikes = collections.Counter()
        self.videoIdToDislikes = collections.Counter()

    def upload(self, video: str) -> int:
        videoId = self._getVideoId()
        self.videoIdToVideo[videoId] = video
        return videoId

    def remove(self, videoId: int) -> None:
        if videoId in self.videoIdToVideo:
            heapq.heappush(self.usedIds, videoId)
            del self.videoIdToVideo[videoId]
            del self.videoIdToViews[videoId]
            del self.videoIdToLikes[videoId]
            del self.videoIdToDislikes[videoId]

    def watch(self, videoId: int, startMinute: int, endMinute: int) -> str:
        if videoId not in self.videoIdToVideo:
            return '-1'
        self.videoIdToViews[videoId] += 1
        video = self.videoIdToVideo[videoId]
        return video[startMinute:min(endMinute + 1, len(video))]

    def like(self, videoId: int) -> None:
        if videoId in self.videoIdToVideo:
            self.videoIdToLikes[videoId] += 1

    def dislike(self, videoId: int) -> None:
        if videoId in self.videoIdToVideo:
            self.videoIdToDislikes[videoId] += 1

    def getLikesAndDislikes(self, videoId: int) -> list[int]:
        if videoId in self.videoIdToVideo:
            return [self.videoIdToLikes[videoId], self.videoIdToDislikes[videoId]]
        return [-1]

    def getViews(self, videoId: int) -> int:
        if videoId in self.videoIdToVideo:
            return self.videoIdToViews[videoId]
        return -1

    def _getVideoId(self) -> int:
        if not self.usedIds:
            self.currVideoId += 1
            return self.currVideoId - 1
        return heapq.heappop(self.usedIds)

```

1198. 2255.py

Path: LeetCode-main/solutions/2255. Count Prefixes of a Given String/2255.py

```

class Solution:
    def countPrefixes(self, words: list[str], s: str) -> int:
        return sum(map(s.startswith, words))

```

1199. 2256.py

Path: LeetCode-main/solutions/2256. Minimum Average Difference/2256.py

```
class Solution:
    def minimumAverageDifference(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        minDiff = inf
        prefix = 0
        suffix = sum(nums)

        for i, num in enumerate(nums):
            prefix += num
            suffix -= num
            prefixAvg = prefix // (i + 1)
            suffixAvg = 0 if i == n - 1 else suffix // (n - 1 - i)
            diff = abs(prefixAvg - suffixAvg)
            if diff < minDiff:
                ans = i
                minDiff = diff

        return ans
```

1200. 2257.py

Path: LeetCode-main/solutions/2257. Count Unguarded Cells in the Grid/2257.py

```

class Solution:
    def countUnguarded(
        self,
        m: int,
        n: int,
        guards: list[list[int]],
        walls: list[list[int]],
    ) -> int:
        ans = 0
        grid = [[0] * n for _ in range(m)]
        left = [[0] * n for _ in range(m)]
        right = [[0] * n for _ in range(m)]
        up = [[0] * n for _ in range(m)]
        down = [[0] * n for _ in range(m)]

        for row, col in guards:
            grid[row][col] = 'G'

        for row, col in walls:
            grid[row][col] = 'W'

        for i in range(m):
            lastCell = 0
            for j in range(n):
                if grid[i][j] == 'G' or grid[i][j] == 'W':
                    lastCell = grid[i][j]
                else:
                    left[i][j] = lastCell
            lastCell = 0
            for j in range(n - 1, -1, -1):
                if grid[i][j] == 'G' or grid[i][j] == 'W':
                    lastCell = grid[i][j]
                else:
                    right[i][j] = lastCell

        for j in range(n):
            lastCell = 0
            for i in range(m):
                if grid[i][j] == 'G' or grid[i][j] == 'W':
                    lastCell = grid[i][j]
                else:
                    up[i][j] = lastCell
            lastCell = 0
            for i in range(m - 1, -1, -1):
                if grid[i][j] == 'G' or grid[i][j] == 'W':
                    lastCell = grid[i][j]
                else:
                    down[i][j] = lastCell

        for i in range(m):
            for j in range(n):
                if (grid[i][j] == 0 and left[i][j] != 'G' and right[i][j] != 'G' and
                    up[i][j] != 'G' and down[i][j] != 'G'):
                    ans += 1

        return ans

```

1201. 2258.py

Path: LeetCode-main/solutions/2258. Escape the Spreading Fire/2258.py

```

class Solution:
    def escapeFire(self, grid: list[list[str]], fire: list[list[int]], person: list[int]) -> int:

```

```

def maximumMinutes(self, grid: list[list[int]]) -> int:
    DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
    MAX = len(grid) * len(grid[0])
    fireGrid = [[-1] * len(grid[0]) for _ in range(len(grid[0]))]
    self._buildFireGrid(grid, fireGrid, DIRS)

    ans = -1
    l = 0
    r = MAX

    while l <= r:
        m = (l + r) // 2
        if self._canStayFor(grid, fireGrid, m, DIRS):
            ans = m
            l = m + 1
        else:
            r = m - 1

    return 1e9 if ans == MAX else ans

def _buildFireGrid(
    self,
    grid: list[list[int]],
    fireMinute: list[list[int]],
    DIRS: list[int],
) -> None:
    minuteFromFire = 0
    q = collections.deque()

    for i in range(len(grid)):
        for j in range(len(grid[0])):
            if grid[i][j] == 1: # the fire
                q.append((i, j))
                fireMinute[i][j] = 0

    while q:
        minuteFromFire += 1
        for _ in range(len(q)):
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == len(grid) or y < 0 or y == len(grid[0]):
                    continue
                if grid[x][y] == 2: # the wall
                    continue
                if fireMinute[x][y] != -1:
                    continue
                fireMinute[x][y] = minuteFromFire
                q.append((x, y))

def _canStayFor(
    self,
    grid: list[list[int]],
    fireMinute: list[list[int]],
    minute: int, DIRS: list[int],
) -> bool:
    q = collections.deque([(0, 0)]) # the start position
    seen = {(0, 0)}

    while q:
        minute += 1
        for _ in range(len(q)):
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx

```

```

y = j + dy
if x < 0 or x == len(grid) or y < 0 or y == len(grid[0]):
    continue
if grid[x][y] == 2: # the wall
    continue
if x == len(grid) - 1 and y == len(grid[0]) - 1:
    if fireMinute[x][y] != -1 and fireMinute[x][y] < minute:
        continue
    return True
if fireMinute[x][y] != -1 and fireMinute[x][y] <= minute:
    continue
if seen[x][y]:
    continue
q.append((x, y))
seen.add((x, y))

return False

```

1202. 2259.py

Path: LeetCode-main/solutions/2259. Remove Digit From Number to Maximize Result/2259.py

1203. 226.py

Path: LeetCode-main/solutions/226. Invert Binary Tree/226.py

```

class Solution:
    def invertTree(self, root: TreeNode | None) -> TreeNode | None:
        if not root:
            return None

        left = root.left
        right = root.right
        root.left = self.invertTree(right)
        root.right = self.invertTree(left)
        return root

```

1204. 2260.py

Path: LeetCode-main/solutions/2260. Minimum Consecutive Cards to Pick Up/2260.py

```

class Solution:
    def minimumCardPickup(self, cards: list[int]) -> int:
        ans = math.inf
        lastSeen = {}

        for i, card in enumerate(cards):
            if card in lastSeen:
                ans = min(ans, i - lastSeen[card] + 1)
                lastSeen[card] = i

        return -1 if ans == math.inf else ans

```

1205. 2261.py

Path: LeetCode-main/solutions/2261. K Divisible Elements Subarrays/2261.py

```
class TrieNode:
    def __init__(self):
        self.children: dict[int, TrieNode] = {}
        self.count = 0

class Solution:
    def countDistinct(self, nums: list[int], k: int, p: int) -> int:
        ans = 0
        root = TrieNode()

        def insert(node: TrieNode, i: int, k: int):
            nonlocal ans
            if i == len(nums) or k - (nums[i] % p == 0) < 0:
                return
            if nums[i] not in node.children:
                node.children[nums[i]] = TrieNode()
                ans += 1
            insert(node.children[nums[i]], i + 1, k - (nums[i] % p == 0))

        for i in range(len(nums)):
            insert(root, i, k)

        return ans
```

1206. 2262-2.py

Path: LeetCode-main/solutions/2262. Total Appeal of A String/2262-2.py

```
class Solution:
    def appealSum(self, s: str) -> int:
        ans = 0
        lastSeen = {}

        for i, c in enumerate(s):
            ans += (i - lastSeen.get(c, -1)) * (len(s) - i)
            lastSeen[c] = i

        return ans
```

1207. 2262.py

Path: LeetCode-main/solutions/2262. Total Appeal of A String/2262.py

```

class Solution:
    def appealSum(self, s: str) -> int:
        ans = 0
        # the total appeal of all substrings ending in the index so far
        dp = 0
        lastSeen = {}

        for i, c in enumerate(s):
            # the total appeal of all substrings ending in s[i]
            # = the total appeal of all substrings ending in s[i - 1]
            # + the number of substrings ending in s[i] that contain only this s[i]
            dp += i - lastSeen.get(c, -1)
            ans += dp
            lastSeen[c] = i

        return ans

```

1208. 2263.py

Path: LeetCode-main/solutions/2263. Make Array Non-decreasing or Non-increasing/2263.py

```

class Solution:
    def convertArray(self, nums: list[int]) -> int:
        def cost(nums: list[int]) -> int:
            ans = 0
            minHeap = []

            # Greedily make `nums` non-increasing.
            for num in nums:
                if minHeap and minHeap[0] < num:
                    ans += num - heapq.heappushpop(minHeap, num)
                    heapq.heappush(minHeap, num)

            return ans

        return min(cost(nums), cost([-num for num in nums]))

```

1209. 2264.py

Path: LeetCode-main/solutions/2264. Largest 3-Same-Digit Number in String/2264.py

```

class Solution:
    def largestGoodInteger(self, num: str) -> str:
        return max(num[i - 2:i + 1]
                    if num[i] == num[i - 1] == num[i - 2]
                    else '' for i in range(2, len(num)))

```

1210. 2265.py

Path: LeetCode-main/solutions/2265. Count Nodes Equal to Average of Subtree/2265.py

```

class Solution:
    def averageOfSubtree(self, root: TreeNode | None) -> int:
        ans = 0

        def dfs(root: TreeNode | None) -> tuple[int, int]:
            nonlocal ans
            if not root:
                return (0, 0)
            leftSum, leftCount = dfs(root.left)
            rightSum, rightCount = dfs(root.right)
            summ = root.val + leftSum + rightSum
            count = 1 + leftCount + rightCount
            if summ // count == root.val:
                ans += 1
            return (summ, count)

        dfs(root)
        return ans

```

1211. 2266.py

Path: LeetCode-main/solutions/2266. Count Number of Texts/2266.py

```

class Solution:
    def countTexts(self, pressedKeys: str) -> int:
        MOD = 1_000_000_007
        n = len(pressedKeys)
        # dp[i] := the number of possible text messages of pressedKeys[i..n)
        dp = [0] * n + [1]

        def isSame(s: str, i: int, k: int) -> bool:
            """Returns True if s[i..i + k) are the same digits."""
            if i + k > len(s):
                return False
            for j in range(i + 1, i + k):
                if s[j] != s[i]:
                    return False
            return True

        for i in reversed(range(n)):
            dp[i] = dp[i + 1]
            if isSame(pressedKeys, i, 2):
                dp[i] += dp[i + 2]
            if isSame(pressedKeys, i, 3):
                dp[i] += dp[i + 3]
            if ((pressedKeys[i] == '7' or pressedKeys[i] == '9') and
                isSame(pressedKeys, i, 4)):
                dp[i] += dp[i + 4]
            dp[i] %= MOD

        return dp[0]

```

1212. 2267.py

Path: LeetCode-main/solutions/2267. Check if There Is a Valid Parentheses String Path/2267.py


```

class Solution:
    def isValidPath(self, grid: list[list[str]]) -> bool:
        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> bool:
            """
            Returns True if there's a path from grid[i][j] to grid[m - 1][n - 1],
            where the number of '(' - the number of ')' == k.
            """
            if i == len(grid) or j == len(grid[0]):
                return False
            k += 1 if grid[i][j] == '(' else -1
            if k < 0:
                return False
            if i == len(grid) - 1 and j == len(grid[0]) - 1:
                return k == 0
            return dp(i + 1, j, k) | dp(i, j + 1, k)

        return dp(0, 0, 0)

```

1213. 2268.py

Path: LeetCode-main/solutions/2268. Minimum Number of Keypresses/2268.py

```

class Solution:
    def minimumKeypresses(self, s: str) -> int:
        return sum(c * (i // 9 + 1)
                    for i, c in enumerate(sorted(Counter(s).values(), reverse=True)))

```

1214. 2269.py

Path: LeetCode-main/solutions/2269. Find the K-Beauty of a Number/2269.py

```

class Solution:
    def divisorSubstrings(self, num: int, k: int) -> int:
        s = str(num)
        ans = 0

        for i in range(len(s) - k + 1):
            x = int(s[i:i + k])
            if x != 0 and num % x == 0:
                ans += 1

        return ans

```

1215. 227-2.py

Path: LeetCode-main/solutions/227. Basic Calculator II/227-2.py

```

class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        prevNum = 0
        currNum = 0
        op = '+'

        for i, c in enumerate(s):
            if c.isdigit():
                currNum = currNum * 10 + int(c)
            if not c.isdigit() and c != ' ' or i == len(s) - 1:
                if op == '+' or op == '-':
                    ans += prevNum
                    prevNum = (currNum if op == '+' else -currNum)
                elif op == '*':
                    prevNum *= currNum
                elif op == '/':
                    prevNum = int(prevNum / currNum)
                op = c
                currNum = 0

        return ans + prevNum

```

1216. 227.py

Path: LeetCode-main/solutions/227. Basic Calculator II/227.py

```

class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        prevNum = 0
        currNum = 0
        op = '+'

        for i, c in enumerate(s):
            if c.isdigit():
                currNum = currNum * 10 + int(c)
            if not c.isdigit() and c != ' ' or i == len(s) - 1:
                if op == '+' or op == '-':
                    ans += prevNum
                    prevNum = currNum if op == '+' else -currNum
                elif op == '*':
                    prevNum = prevNum * currNum
                elif op == '/':
                    if prevNum < 0:
                        prevNum = math.ceil(prevNum / currNum)
                    else:
                        prevNum = prevNum // currNum
                op = c
                currNum = 0

        return ans + prevNum

```

1217. 2270.py

Path: LeetCode-main/solutions/2270. Number of Ways to Split Array/2270.py

```

class Solution:
    def waysToSplitArray(self, nums: list[int]) -> int:
        ans = 0
        prefix = 0
        suffix = sum(nums)

        for i in range(len(nums) - 1):
            prefix += nums[i]
            suffix -= nums[i]
            if prefix >= suffix:
                ans += 1

        return ans

```

1218. 2271.py

Path: LeetCode-main/solutions/2271. Maximum White Tiles Covered by a Carpet/2271.py

```

class Solution:
    def maximumWhiteTiles(self, tiles: list[list[int]], carpetLen: int) -> int:
        if any(tile[1] - tile[0] + 1 >= carpetLen for tile in tiles):
            return carpetLen

        ans = 0
        prefix = [0] * (len(tiles) + 1)

        tiles.sort()
        starts = [tile[0] for tile in tiles]

        for i, tile in enumerate(tiles):
            length = tile[1] - tile[0] + 1
            prefix[i + 1] = prefix[i] + length

        for i, (s, _) in enumerate(tiles):
            carpetEnd = s + carpetLen - 1
            endIndex = bisect_right(starts, carpetEnd) - 1
            notCover = max(0, tiles[endIndex][1] - carpetEnd)
            ans = max(ans, prefix[endIndex + 1] - prefix[i] - notCover)

        return ans

```

1219. 2272.py

Path: LeetCode-main/solutions/2272. Substring With Largest Variance/2272.py

```

class Solution:
    def largestVariance(self, s: str) -> int:
        # a := the letter with the higher frequency
        # b := the letter with the lower frequency
        def kadane(a: str, b: str) -> int:
            ans = 0
            countA = 0
            countB = 0
            canExtendPrevB = False

            for c in s:
                if c != a and c != b:
                    continue
                if c == a:
                    countA += 1
                else:
                    countB += 1
                if countB > 0:
                    # An interval should contain at least one b.
                    ans = max(ans, countA - countB)
                elif countB == 0 and canExtendPrevB:
                    # edge case: consider the previous b.
                    ans = max(ans, countA - 1)
                # Reset if the number of b > the number of a.
                if countB > countA:
                    countA = 0
                    countB = 0
                    canExtendPrevB = True

            return ans

        return max(kadane(a, b)
                   for a in string.ascii_lowercase
                   for b in string.ascii_lowercase
                   if a != b)

```

1220. 2273.py

Path: LeetCode-main/solutions/2273. Find Resultant Array After Removing Anagrams/2273.py

```

class Solution:
    def removeAnagrams(self, words: list[str]) -> list[str]:
        ans = []

        def isAnagram(a: str, b: str) -> bool:
            count = collections.Counter(a)
            count.subtract(collections.Counter(b))
            return all(value == 0 for value in count.values())

        i = 0
        while i < len(words):
            j = i + 1
            while j < len(words) and isAnagram(words[i], words[j]):
                j += 1
            ans.append(words[i])
            i = j

        return ans

```

1221. 2274.py

Path: LeetCode-main/solutions/2274. Maximum Consecutive Floors Without Special Floors/2274.py

```
class Solution:
    def maxConsecutive(self, bottom: int, top: int, special: list[int]) -> int:
        ans = 0

        special.sort()

        for a, b in zip(special, special[1:]):
            ans = max(ans, b - a - 1)

        return max(ans, special[0] - bottom, top - special[-1])
```

1222. 2275.py

Path: LeetCode-main/solutions/2275. Largest Combination With Bitwise AND Greater Than Zero/2275.py

```
class Solution:
    def largestCombination(self, candidates: list[int]) -> int:
        return max(sum(c >> i & 1 for c in candidates) for i in range(24))
```

1223. 2276.py

Path: LeetCode-main/solutions/2276. Count Integers in Intervals/2276.py

```
from sortedcontainers import SortedDict

class CountIntervals:
    def __init__(self):
        self.intervals = SortedDict()
        self.cnt = 0

    def add(self, left: int, right: int) -> None:
        while self._isOverlapped(left, right):
            i = self.intervals.bisect_right(right) - 1
            l, r = self.intervals.popitem(i)
            left = min(left, l)
            right = max(right, r)
            self.cnt -= r - l + 1

        self.intervals[left] = right
        self.cnt += right - left + 1

    def count(self) -> int:
        return self.cnt

    def _isOverlapped(self, left: int, right: int) -> bool:
        i = self.intervals.bisect_right(right)
        return i > 0 and self.intervals.peekitem(i - 1)[1] >= left
```

1224. 2277.py

Path: LeetCode-main/solutions/2277. Closest Node to Path in Tree/2277.py

```

class Solution:
    def closestNode(
        self,
        n: int,
        edges: list[list[int]],
        query: list[list[int]],
    ) -> list[int]:
        tree = [[] for _ in range(n)]
        dist = [[-1] * n for _ in range(n)]

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def fillDist(start: int, u: int, d: int) -> None:
            dist[start][u] = d
            for v in tree[u]:
                if dist[start][v] == -1:
                    fillDist(start, v, d + 1)

        for i in range(n):
            fillDist(i, i, 0)

        def findClosest(u: int, end: int, node: int, ans: int) -> int:
            for v in tree[u]:
                if dist[v][end] < dist[u][end]:
                    return findClosest(
                        v, end, node, ans if dist[ans][node] < dist[v][node] else v)
            return ans

        return [findClosest(start, end, node, start)
                for start, end, node in query]

```

1225. 2278.py

Path: LeetCode-main/solutions/2278. Percentage of Letter in String/2278.py

```

class Solution:
    def percentageLetter(self, s: str, letter: str) -> int:
        return 100 * s.count(letter) // len(s)

```

1226. 2279.py

Path: LeetCode-main/solutions/2279. Maximum Bags With Full Capacity of Rocks/2279.py

```

class Solution:
    def maximumBags(
        self,
        capacity: list[int],
        rocks: list[int],
        additionalRocks: int,
    ) -> int:
        for i, d in enumerate(sorted([c - r for c, r in zip(capacity, rocks)])):
            if d > additionalRocks:
                return i
            additionalRocks -= d
        return len(capacity)

```

1227. 228.py

Path: LeetCode-main/solutions/228. Summary Ranges/228.py

```
class Solution:
    def summaryRanges(self, nums: list[int]) -> list[str]:
        ans = []

        i = 0
        while i < len(nums):
            begin = nums[i]
            while i < len(nums) - 1 and nums[i] == nums[i + 1] - 1:
                i += 1
            end = nums[i]
            if begin == end:
                ans.append(str(begin))
            else:
                ans.append(str(begin) + "->" + str(end))
            i += 1

        return ans
```

1228. 2280.py

Path: LeetCode-main/solutions/2280. Minimum Lines to Represent a Line Chart/2280.py

```
class Solution:
    def minimumLines(self, stockPrices: list[list[int]]) -> int:
        ans = 0

        stockPrices.sort()

        def getSlope(p: list[int], q: list[int]) -> tuple[int, int]:
            dx = p[0] - q[0]
            dy = p[1] - q[1]
            if dx == 0:
                return (0, p[0])
            if dy == 0:
                return (p[1], 0)
            d = gcd(dx, dy)
            return (dx // d, dy // d)

        for i in range(2, len(stockPrices)):
            a = getSlope(stockPrices[i - 2], stockPrices[i - 1])
            b = getSlope(stockPrices[i - 1], stockPrices[i])
            if a != b:
                ans += 1

        return ans + (len(stockPrices) > 1)
```

1229. 2281.py

Path: LeetCode-main/solutions/2281. Sum of Total Strength of Wizards/2281.py

```

class Solution:
    def totalStrength(self, strength: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(strength)
        # left[i] := the next index on the left (if any)
        #           s.t. nums[left[i]] <= nums[i]
        left = [-1] * n
        # right[i] := the next index on the right (if any)
        #           s.t. nums[right[i]] < nums[i]
        right = [n] * n
        stack = []

        for i in reversed(range(n)):
            while stack and strength[stack[-1]] >= strength[i]:
                left[stack.pop()] = i
            stack.append(i)

        stack = []

        for i in range(n):
            while stack and strength[stack[-1]] > strength[i]:
                right[stack.pop()] = i
            stack.append(i)

        ans = 0
        prefixOfPrefix = list(itertools.accumulate(
            itertools.accumulate(strength), initial=0))

        # For each strength[i] as the minimum, calculate sum.
        for i, (l, r) in enumerate(zip(left, right)):
            leftSum = prefixOfPrefix[i] - prefixOfPrefix[max(0, l)]
            rightSum = prefixOfPrefix[r] - prefixOfPrefix[i]
            leftLen = i - l
            rightLen = r - i
            ans += strength[i] * (rightSum * leftLen - leftSum * rightLen) % MOD

        return ans % MOD

```

1230. 2282.py

Path: LeetCode-main/solutions/2282. Number of People That Can Be Seen in a Grid/2282.py


```

class Solution:
    def seePeople(self, heights: list[list[int]]) -> list[list[int]]:
        m = len(heights)
        n = len(heights[0])
        ans = [[0] * n for _ in range(m)]

        for i, row in enumerate(heights):
            stack = []
            for j, height in enumerate(row):
                hasEqualHeight = False
                while stack and row[stack[-1]] <= height:
                    if row[stack[-1]] == height:
                        # edge case: [4, 2, 1, 1, 3]
                        hasEqualHeight = True
                    ans[i][stack.pop()] += 1
                if stack and not hasEqualHeight:
                    ans[i][stack[-1]] += 1
                stack.append(j)

        for j, col in enumerate(zip(*heights)):
            stack = []
            for i, height in enumerate(col):
                hasEqualHeight = False
                while stack and col[stack[-1]] <= height:
                    if col[stack[-1]] == height:
                        hasEqualHeight = True
                    ans[stack.pop()][j] += 1
                if stack and not hasEqualHeight:
                    ans[stack[-1]][j] += 1
                stack.append(i)

        return ans

```

1231. 2283.py

Path: LeetCode-main/solutions/2283. Check if Number Has Equal Digit Count and Digit Value/2283.py

```

class Solution:
    def digitCount(self, num: str) -> bool:
        count = collections.Counter(num)
        return all(count[str(i)] == int(digit)
                    for i, digit in enumerate(num))

```

1232. 2284.py

Path: LeetCode-main/solutions/2284. Sender With Largest Word Count/2284.py

```

class Solution:
    def largestWordCount(self, messages: list[str], senders: list[str]) -> str:
        n = len(messages)
        ans = ''
        maxWordsSent = 0
        count = collections.Counter() # [sender, # Words sent]

        for message, sender in zip(messages, senders):
            wordsCount = message.count(' ') + 1
            count[sender] += wordsCount
            numWordsSent = count[sender]
            if numWordsSent > maxWordsSent:
                ans = sender
                maxWordsSent = numWordsSent
            elif numWordsSent == maxWordsSent and sender > ans:
                ans = sender

        return ans

```

1233. 2285.py

Path: LeetCode-main/solutions/2285. Maximum Total Importance of Roads/2285.py

```

class Solution:
    def maximumImportance(self, n: int, roads: list[list[int]]) -> int:
        count = [0] * n

        for u, v in roads:
            count[u] += 1
            count[v] += 1

        count.sort()
        return sum((i + 1) * c for i, c in enumerate(count))

```

1234. 2287.py

Path: LeetCode-main/solutions/2287. Rearrange Characters to Make Target String/2287.py

```

class Solution:
    def rearrangeCharacters(self, s: str, target: str) -> int:
        countS = collections.Counter(s)
        countT = collections.Counter(target)
        return min(countS[c] // countT[c] for c in target)

```

1235. 2288.py

Path: LeetCode-main/solutions/2288. Apply Discount to Prices/2288.py

```

class Solution:
    def discountPrices(self, sentence: str, discount: int) -> str:
        PRECISION = 2
        ans = []

        for word in sentence.split():
            if word[0] == '$' and len(word) > 1:
                digits = word[1:]
                if all(digit.isdigit() for digit in digits):
                    val = float(digits) * (100 - discount) / 100
                    s = f'{val:.2f}'
                    trimmed = s[:s.index('.') + PRECISION + 1]
                    ans.append('$' + trimmed)
                else:
                    ans.append(word)
            else:
                ans.append(word)

        return ' '.join(ans)

```

1236. 2289.py

Path: LeetCode-main/solutions/2289. Steps to Make Array Non-decreasing/2289.py

```

class Solution:
    def totalSteps(self, nums: list[int]) -> int:
        # dp[i] := the number of steps to remove nums[i]
        dp = [0] * len(nums)
        stack = []

        for i, num in enumerate(nums):
            step = 1
            while stack and nums[stack[-1]] <= num:
                step = max(step, dp[stack.pop()] + 1)
            if stack:
                dp[i] = step
            stack.append(i)

        return max(dp)

```

1237. 229.py

Path: LeetCode-main/solutions/229. Majority Element II/229.py

```

class Solution:
    def majorityElement(self, nums: list[int]) -> list[int]:
        ans1 = 0
        ans2 = 1
        count1 = 0
        count2 = 0

        for num in nums:
            if num == ans1:
                count1 += 1
            elif num == ans2:
                count2 += 1
            elif count1 == 0:
                ans1 = num
                count1 = 1
            elif count2 == 0:
                ans2 = num
                count2 = 1
            else:
                count1 -= 1
                count2 -= 1

        return [ans for ans in (ans1, ans2) if nums.count(ans) > len(nums) // 3]

```

1238. 2290.py

Path: LeetCode-main/solutions/2290. Minimum Obstacle Removal to Reach Corner/2290.py

```

class Solution:
    def minimumObstacles(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        minHeap = [(grid[0][0], 0, 0)] # (d, i, j)
        dist = [[math.inf] * n for _ in range(m)]
        dist[0][0] = grid[0][0]

        while minHeap:
            d, i, j = heapq.heappop(minHeap)
            if i == m - 1 and j == n - 1:
                return d
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                newDist = d + grid[i][j]
                if newDist < dist[x][y]:
                    dist[x][y] = newDist
                    heapq.heappush(minHeap, (newDist, x, y))

        return dist[m - 1][n - 1]

```

1239. 2291-2.py

Path: LeetCode-main/solutions/2291. Maximum Profit From Trading Stocks/2291-2.py

```

class Solution:
    def maximumProfit(
        self,
        present: list[int],
        future: list[int],
        budget: int,
    ) -> int:
        # dp[i] := the maximum profit of buying present so far with i budget
        dp = [0] * (budget + 1)

        for p, f in zip(present, future):
            for j in range(budget, p - 1, -1):
                dp[j] = max(dp[j], f - p + dp[j - p])

        return dp[budget]

```

1240. 2291.py

Path: LeetCode-main/solutions/2291. Maximum Profit From Trading Stocks/2291.py

```

class Solution:
    def maximumProfit(
        self,
        present: list[int],
        future: list[int],
        budget: int,
    ) -> int:
        n = len(present)
        # dp[i][j] := the maximum profit of buying present[0..i) with j budget
        dp = [[0] * (budget + 1) for _ in range(n + 1)]

        for i in range(1, n + 1):
            profit = future[i - 1] - present[i - 1]
            for j in range(budget + 1):
                if j < present[i - 1]:
                    dp[i][j] = dp[i - 1][j]
                else:
                    dp[i][j] = max(dp[i - 1][j], profit + dp[i - 1][j - present[i - 1]])

        return dp[n][budget]

```

1241. 2293.py

Path: LeetCode-main/solutions/2293. Min Max Game/2293.py

```

class Solution:
    def minMaxGame(self, nums: list[int]) -> int:
        if len(nums) == 1:
            return nums[0]

        nextNums = []
        for i in range(len(nums) // 2):
            nextNums.append(min(nums[2 * i], nums[2 * i + 1]) if i % 2 == 0 else
                             max(nums[2 * i], nums[2 * i + 1]))
        return self.minMaxGame(nextNums)

```

1242. 2294.py

Path: LeetCode-main/solutions/2294. Partition Array Such That Maximum Difference Is K/2294.py

```
class Solution:
    def partitionArray(self, nums: list[int], k: int) -> int:
        nums.sort()

        ans = 1
        mn = nums[0]

        for i in range(1, len(nums)):
            if mn + k < nums[i]:
                ans += 1
                mn = nums[i]

        return ans
```

1243. 2295.py

Path: LeetCode-main/solutions/2295. Replace Elements in an Array/2295.py

```
class Solution:
    def arrayChange(
        self,
        nums: list[int],
        operations: list[list[int]],
    ) -> list[int]:
        numToIndex = {num: i for i, num in enumerate(nums)}

        for original, replaced in operations:
            index = numToIndex[original]
            nums[index] = replaced
            del numToIndex[original]
            numToIndex[replaced] = index

        return nums
```

1244. 2296.py

Path: LeetCode-main/solutions/2296. Design a Text Editor/2296.py

```

class TextEditor:
    def __init__(self):
        self.s = []
        self.stack = []

    def addText(self, text: str) -> None:
        for c in text:
            self.s.append(c)

    def deleteText(self, k: int) -> int:
        numDeleted = min(k, len(self.s))
        for _ in range(numDeleted):
            self.s.pop()
        return numDeleted

    def cursorLeft(self, k: int) -> str:
        while self.s and k > 0:
            self.stack.append(self.s.pop())
            k -= 1
        return self._getString()

    def cursorRight(self, k: int) -> str:
        while self.stack and k > 0:
            self.s.append(self.stack.pop())
            k -= 1
        return self._getString()

    def _getString(self) -> str:
        if len(self.s) < 10:
            return ''.join(self.s)
        return ''.join(self.s[-10:])

```

1245. 2297.py

Path: LeetCode-main/solutions/2297. Jump Game IX/2297.py

```

class Solution:
    def minCost(self, nums: list[int], costs: list[int]) -> int:
        # dp[i] := the minimum cost to jump to i
        dp = [math.inf] * len(nums)
        maxStack = []
        minStack = []

        dp[0] = 0

        for i, num in enumerate(nums):
            while maxStack and num >= nums[maxStack[-1]]:
                dp[i] = min(dp[i], dp[maxStack.pop()] + costs[i])
            while minStack and num < nums[minStack[-1]]:
                dp[i] = min(dp[i], dp[minStack.pop()] + costs[i])
            maxStack.append(i)
            minStack.append(i)

        return dp[-1]

```

1246. 2299.py

Path: LeetCode-main/solutions/2299. Strong Password Checker II/2299.py

```
class Solution:
    def strongPasswordCheckerII(self, password: str) -> bool:
        if len(password) < 8:
            return False
        if not any(c.islower() for c in password):
            return False
        if not any(c.isupper() for c in password):
            return False
        if not any(c.isdigit() for c in password):
            return False
        if not any("!@#%^&*()-+.".find(c) != -1 for c in password):
            return False
        return all(a != b for a, b in zip(password, password[1:]))
```

1247. 23.py

Path: LeetCode-main/solutions/23. Merge k Sorted Lists/23.py

```
from queue import PriorityQueue

class Solution:
    def mergeKLists(self, lists: list[ListNode]) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        pq = PriorityQueue()

        for i, lst in enumerate(lists):
            if lst:
                pq.put((lst.val, i, lst))

        while not pq.empty():
            _, i, minNode = pq.get()
            if minNode.next:
                pq.put((minNode.next.val, i, minNode.next))
            curr.next = minNode
            curr = curr.next

        return dummy.next
```

1248. 230-2.py

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230-2.py


```
class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        rank = 0
        ans = 0

        def traverse(root: TreeNode | None) -> None:
            nonlocal rank
            nonlocal ans
            if not root:
                return

            traverse(root.left)
            rank += 1
            if rank == k:
                ans = root.val
                return
            traverse(root.right)

        traverse(root)
        return ans
```

1249. 230-3.py

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230-3.py

```
class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        stack = []

        while root:
            stack.append(root)
            root = root.left

        for _ in range(k - 1):
            root = stack.pop()
            root = root.right
            while root:
                stack.append(root)
                root = root.left

        return stack[-1].val
```

1250. 230.py

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230.py

```

class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        def countNodes(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + countNodes(root.left) + countNodes(root.right)

        leftCount = countNodes(root.left)

        if leftCount == k - 1:
            return root.val
        if leftCount >= k:
            return self.kthSmallest(root.left, k)
        return self.kthSmallest(root.right, k - 1 - leftCount) # leftCount < k

```

1251. 2300.py

Path: LeetCode-main/solutions/2300. Successful Pairs of Spells and Potions/2300.py

```

class Solution:
    def successfulPairs(
        self,
        spells: list[int],
        potions: list[int],
        success: int,
    ) -> list[int]:
        potions.sort()

        def firstIndexSuccess(spell: int):
            """Returns the first index i s.t. spell * potions[i] >= success."""
            l = 0
            r = len(potions)
            while l < r:
                m = (l + r) // 2
                if spell * potions[m] >= success:
                    r = m
                else:
                    l = m + 1
            return l

        return [len(potions) - firstIndexSuccess(spell) for spell in spells]

```

1252. 2301.py

Path: LeetCode-main/solutions/2301. Match Substring After Replacement/2301.py

```

class Solution:
    def matchReplacement(
        self,
        s: str,
        sub: str,
        mappings: list[list[str]],
    ) -> bool:
        isMapped = [[False] * 128 for _ in range(128)]

        for old, new in mappings:
            isMapped[ord(old)][ord(new)] = True

        for i in range(len(s)):
            if self._canTransform(s, i, sub, isMapped):
                return True

        return False

    def _canTransform(
        self,
        s: str,
        start: int,
        sub: str,
        isMapped: list[list[bool]],
    ) -> bool:
        if start + len(sub) > len(s):
            return False

        for i in range(len(sub)):
            a = sub[i]
            b = s[start + i]
            if a != b and not isMapped[ord(a)][ord(b)]:
                return False

        return True

```

1253. 2302.py

Path: LeetCode-main/solutions/2302. Count Subarrays With Score Less Than K/2302.py

```

class Solution:
    def countSubarrays(self, nums: list[int], k: int) -> int:
        ans = 0
        summ = 0

        l = 0
        for r, num in enumerate(nums):
            summ += num
            while summ * (r - l + 1) >= k:
                summ -= nums[l]
                l += 1
            ans += r - l + 1

        return ans

```

1254. 2303.py

Path: LeetCode-main/solutions/2303. Calculate Amount Paid in Taxes/2303.py

```

class Solution:
    def calculateTax(self, brackets: list[list[int]], income: int) -> float:
        ans = 0
        prev = 0

        for upper, percent in brackets:
            if income < upper:
                return ans + (income - prev) * percent / 100.0
            ans += (upper - prev) * percent / 100.0
            prev = upper

        return ans

```

1255. 2304.py

Path: LeetCode-main/solutions/2304. Minimum Path Cost in a Grid/2304.py

```

class Solution:
    def minPathCost(
        self,
        grid: list[list[int]],
        moveCost: list[list[int]],
    ) -> int:
        m = len(grid)
        n = len(grid[0])
        # dp[i][j] := the minimum cost to reach grid[i][j]
        dp = [[math.inf] * n for _ in range(m)]
        dp[0] = grid[0]

        for i in range(1, m):
            for j in range(n):
                for k in range(n):
                    dp[i][j] = min(dp[i][j], dp[i - 1][k] +
                                   moveCost[grid[i - 1][k]][j] + grid[i][j])

        return min(dp[-1])

```

1256. 2305.py

Path: LeetCode-main/solutions/2305. Fair Distribution of Cookies/2305.py

```

class Solution:
    def distributeCookies(self, cookies: list[int], k: int) -> int:
        ans = math.inf

        def dfs(s: int, children: list[int]) -> None:
            nonlocal ans
            if s == len(cookies):
                ans = min(ans, max(children))
                return

            for i in range(k):
                children[i] += cookies[s]
                dfs(s + 1, children)
                children[i] -= cookies[s]

        dfs(0, [0] * k)
        return ans

```

1257. 2306.py

Path: LeetCode-main/solutions/2306. Naming a Company/2306.py

```
class Solution:
    def distinctNames(self, ideas: list[str]) -> int:
        ans = 0
        # suffixes[i] := the set of strings omitting the first letter, where the
        # first letter is ('a' + i)
        suffixes = [set() for _ in range(26)]

        for idea in ideas:
            suffixes[ord(idea[0]) - ord('a')].add(idea[1:])

        for i, j in itertools.combinations(range(26), 2):
            count = len(suffixes[i] & suffixes[j])
            ans += 2 * (len(suffixes[i]) - count) * (len(suffixes[j]) - count)

        return ans
```

1258. 2307.py

Path: LeetCode-main/solutions/2307. Check for Contradictions in Equations/2307.py

```
class Solution:
    def checkContradictions(
        self,
        equations: list[list[str]],
        values: list[float],
    ) -> bool:
        # Convert `string` to `int` for a better performance.
        strToInt = {}

        for u, v in equations:
            strToInt.setdefault(u, len(strToInt))
            strToInt.setdefault(v, len(strToInt))

        graph = [[] for _ in range(len(strToInt))]
        seen = [0.0] * len(graph)

        for i, ((A, B), value) in enumerate(zip(equations, values)):
            u = strToInt[A]
            v = strToInt[B]
            graph[u].append((v, value))
            graph[v].append((u, 1 / value))

        def dfs(u: int, val: float) -> bool:
            if seen[u]:
                return abs(val / seen[u] - 1) > 1e-5

            seen[u] = val
            return any(dfs(v, val / w) for v, w in graph[u])

        for i in range(len(graph)):
            if not seen[i] and dfs(i, 1.0):
                return True

        return False
```

1259. 2309.py

Path: LeetCode-main/solutions/2309. Greatest English Letter in Upper and Lower Case/2309.py

```
class Solution:
    def greatestLetter(self, s: str) -> str:
        seen = set(s)

        for i in range(25, -1, -1):
            if (chr(ord('a') + i) in seen and
                chr(ord('A') + i) in seen):
                return chr(ord('A') + i)

        return ''
```

1260. 231.py

Path: LeetCode-main/solutions/231. Power of Two/231.py

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n >= 0 and n.bit_count() == 1
```

1261. 2310.py

Path: LeetCode-main/solutions/2310. Sum of Numbers With Units Digit K/2310.py

```
class Solution:
    def minimumNumbers(self, num: int, k: int) -> int:
        if num == 0:
            return 0

        # Assume the size of the set is n, and the numbers in the set are X1, X2,
        # ..., Xn. Since the units digit of each number is k, X1 + X2 + ... + Xn =
        # N * k + 10 * (x1 + x2 + ... + xn) = num. Therefore, the goal is to find
        # the n s.t. n * k % 10 = num % 10
        for i in range(1, 11):
            if i * k > num + 1:
                break
            if i * k % 10 == num % 10:
                return i

        return -1
```

1262. 2311.py

Path: LeetCode-main/solutions/2311. Longest Binary Subsequence Less Than or Equal to K/2311.py

```
class Solution:
    def longestSubsequence(self, s: str, k: int) -> int:
        oneCount = 0
        num = 0
        pow = 1

        # Take as many 1s as possible from the right.
        for i in reversed(range(len(s))):
            if num + pow > k:
                break
            if s[i] == '1':
                oneCount += 1
                num += pow
                pow *= 2

        return s.count('0') + oneCount
```

1263. 2312.py

Path: LeetCode-main/solutions/2312. Selling Pieces of Wood/2312.py

```
class Solution:
    def sellingWood(self, m: int, n: int, prices: list[list[int]]) -> int:
        # dp[i][j] := the maximum money of cutting i x j piece of wood
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for h, w, price in prices:
            dp[h][w] = price

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                for h in range(1, i // 2 + 1):
                    dp[i][j] = max(dp[i][j], dp[h][j] + dp[i - h][j])
                for w in range(1, j // 2 + 1):
                    dp[i][j] = max(dp[i][j], dp[i][w] + dp[i][j - w])

        return dp[m][n]
```

1264. 2313.py

Path: LeetCode-main/solutions/2313. Minimum Flips in Binary Tree to Get Result/2313.py

```

class Solution:
    def minimumFlips(self, root: TreeNode | None, result: bool) -> int:
        @functools.lru_cache(None)
        def dp(root: TreeNode | None, target: bool) -> int:
            """Returns the minimum flips to make the subtree root become target."""
            if root.val in (0, 1): # the leaf
                return 0 if root.val == target else 1
            if root.val == 5: # NOT
                return dp(root.left or root.right, not target)
            if root.val == 2: # OR
                nextTargets = [(0, 1), (1, 0), (1, 1)] if target else [(0, 0)]
            elif root.val == 3: # AND
                nextTargets = [(1, 1)] if target else [(0, 0), (0, 1), (1, 0)]
            else: # root.val == 4 XOR
                nextTargets = [(0, 1), (1, 0)] if target else [(0, 0), (1, 1)]
            return min(dp(root.left, leftTarget) + dp(root.right, rightTarget)
                       for leftTarget, rightTarget in nextTargets)

        return dp(root, result)

```

1265. 2315.py

Path: LeetCode-main/solutions/2315. Count Asterisks/2315.py

```

class Solution:
    def countAsterisks(self, s: str) -> int:
        ans = 0
        bars = 0

        for c in s:
            if c == '|':
                bars += 1
            elif c == '*' and bars % 2 == 0:
                ans += 1

        return ans

```

1266. 2316.py

Path: LeetCode-main/solutions/2316. Count Unreachable Pairs of Nodes in an Undirected Graph/2316.py


```

class Solution:
    def countPairs(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        seen = [0] * n
        unreached = n

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        for i in range(n):
            reached = self._dfs(graph, i, seen)
            unreached -= reached
            ans += unreached * reached

        return ans

    def _dfs(self, graph: list[list[int]], u: int, seen: list[bool]) -> int:
        if seen[u]:
            return 0
        seen[u] = True
        return functools.reduce(lambda subtotal, v:
                                subtotal + self._dfs(graph, v, seen), graph[u], 1)

```

1267. 2317.py

Path: LeetCode-main/solutions/2317. Maximum XOR After Operations/2317.py

```

class Solution:
    def maximumXOR(self, nums: list[int]) -> int:
        # 1. nums[i] & (nums[i] ^ x) enables you to turn 1-bit to 0-bit from
        #    nums[i] since x is arbitrary.
        # 2. The i-th bit of the XOR of all the elements is 1 if the i-th bit is 1
        #    for an odd number of elements.
        # 3. Therefore, the question is equivalent to: if you can convert any digit
        #    from 1 to 0 for any number, what is the maximum for XOR(nums[i]).
        # 4. The maximum we can get is of course to make every digit of the answer
        #    to be 1 if possible
        # 5. Therefore, OR(nums[i]) is an approach.
        return functools.reduce(operator.ior, nums)

```

1268. 2318.py

Path: LeetCode-main/solutions/2318. Number of Distinct Roll Sequences/2318.py

```

class Solution:
    def distinctSequences(self, n: int) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(n: int, prev: int, prevPrev: int) -> int:
            """
            Returns the number of distinct sequences for n dices with `prev` and
            `prevPrev`.
            """
            if n == 0:
                return 1
            res = 0
            for dice in range(1, 7):
                if (dice not in (prev, prevPrev) and
                    (prev == 0 or math.gcd(dice, prev) == 1)):
                    res += dp(n - 1, dice, prev)
            res %= MOD
            return res

        return dp(n, 0, 0)

```

1269. 2319.py

Path: LeetCode-main/solutions/2319. Check if Matrix Is X-Matrix/2319.py

```

class Solution:
    def checkXMatrix(self, grid: list[list[int]]) -> bool:
        n = len(grid)

        for i in range(n):
            for j in range(n):
                if i == j or i + j == n - 1: # in diagonal
                    if grid[i][j] == 0:
                        return False
                elif grid[i][j]: # not in diagonal
                    return False

        return True

```

1270. 232.py

Path: LeetCode-main/solutions/232. Implement Queue using Stacks/232.py

```
class MyQueue:
    def __init__(self):
        self.input = []
        self.output = []

    def push(self, x: int) -> None:
        self.input.append(x)

    def pop(self) -> int:
        self.peak()
        return self.output.pop()

    def peek(self) -> int:
        if not self.output:
            while self.input:
                self.output.append(self.input.pop())
        return self.output[-1]

    def empty(self) -> bool:
        return not self.input and not self.output
```

1271. 2320.py

Path: LeetCode-main/solutions/2320. Count Number of Ways to Place Houses/2320.py

```
class Solution:
    def countHousePlacements(self, n: int) -> int:
        MOD = 1_000_000_007
        house = 1 # the number of ways ending in a house
        space = 1 # the number of ways ending in a space
        total = house + space

        for _ in range(2, n + 1):
            house = space
            space = total
            total = (house + space) % MOD

        return total**2 % MOD
```

1272. 2321.py

Path: LeetCode-main/solutions/2321. Maximum Score Of Spliced Array/2321.py

```
class Solution:
    def maximumSplicedArray(self, nums1: list[int], nums2: list[int]) -> int:
        def kadane(nums1: list[int], nums2: list[int]) -> int:
            """
            Returns the maximum gain of swapping some numbers in `nums1` with some
            numbers in `nums2`.
            """
            gain = 0
            maxGain = 0

            for num1, num2 in zip(nums1, nums2):
                gain = max(0, gain + num2 - num1)
                maxGain = max(maxGain, gain)

            return maxGain + sum(nums1)

        return max(kadane(nums1, nums2), kadane(nums2, nums1))
```

1273. 2322.py

Path: LeetCode-main/solutions/2322. Minimum Score After Removals on a Tree/2322.py

```

class Solution:
    def minimumScore(self, nums: list[int], edges: list[list[int]]) -> int:
        n = len(nums)
        xors = functools.reduce(operator.xor, nums)
        subXors = nums[:]
        tree = [[] for _ in range(n)]
        children = [{i} for i in range(n)]

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def dfs(u: int, prev: int) -> tuple[int, set[int]]:
            for v in tree[u]:
                if v == prev:
                    continue
                vXor, vChildren = dfs(v, u)
                subXors[u] ^= vXor
                children[u] |= vChildren
            return subXors[u], children[u]

        dfs(0, -1)

        ans = math.inf
        for i in range(len(edges)):
            a, b = edges[i]
            if b in children[a]:
                a, b = b, a
            for j in range(i):
                c, d = edges[j]
                if d in children[c]:
                    c, d = d, c

                if c in children[a] and a != c:
                    cands = [subXors[c], subXors[a] ^ subXors[c], xors ^ subXors[a]]
                elif a in children[c] and a != c:
                    cands = [subXors[a], subXors[c] ^ subXors[a], xors ^ subXors[c]]
                else:
                    cands = [subXors[a], subXors[c], xors ^ subXors[a] ^ subXors[c]]
                ans = min(ans, max(cands) - min(cands))

        return ans

```

1274. 2323.py

Path: LeetCode-main/solutions/2323. Find Minimum Time to Finish All Jobs II/2323.py

```

class Solution:
    def minimumTime(self, jobs: list[int], workers: list[int]) -> int:
        ans = 0

        jobs.sort()
        workers.sort()

        for job, worker in zip(jobs, workers):
            ans = max(ans, (job - 1) // worker + 1)

        return ans

```

1275. 2325.py

Path: LeetCode-main/solutions/2325. Decode the Message/2325.py

```
class Solution:
    def decodeMessage(self, key: str, message: str) -> str:
        keyToActual = {' ': ' '}
        currChar = 'a'

        for c in key:
            if c not in keyToActual:
                keyToActual[c] = currChar
                currChar = chr(ord(currChar) + 1)

        return ''.join(keyToActual[c] for c in message)
```

1276. 2326.py

Path: LeetCode-main/solutions/2326. Spiral Matrix IV/2326.py

```
class Solution:
    def spiralMatrix(self, m: int, n: int, head: ListNode | None) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        ans = [[-1] * n for _ in range(m)]
        x = 0 # the current x position
        y = 0 # the current y position
        d = 0

        curr = head
        while curr:
            ans[x][y] = curr.val
            if (x + DIRS[d][0] < 0 or x + DIRS[d][0] == m or y + DIRS[d][1] < 0 or
                y + DIRS[d][1] == n or ans[x + DIRS[d][0]][y + DIRS[d][1]] != -1):
                d = (d + 1) % 4
            x += DIRS[d][0]
            y += DIRS[d][1]
            curr = curr.next

        return ans
```

1277. 2327.py

Path: LeetCode-main/solutions/2327. Number of People Aware of a Secret/2327.py

```

class Solution:
    def peopleAwareOfSecret(self, n: int, delay: int, forget: int) -> int:
        MOD = 1_000_000_007
        share = 0
        # dp[i] := the number of people know the secret at day i
        dp = [0] * n # Maps day i to i + 1.
        dp[0] = 1

        for i in range(1, n):
            if i - delay >= 0:
                share += dp[i - delay]
            if i - forget >= 0:
                share -= dp[i - forget]
            share += MOD
            share %= MOD
            dp[i] = share

        # People before day `n - forget - 1` already forget the secret.
        return sum(dp[-forget:]) % MOD

```

1278. 2328.py

Path: LeetCode-main/solutions/2328. Number of Increasing Paths in a Grid/2328.py

```

class Solution:
    def countPaths(self, grid: list[list[int]]) -> int:
        MOD = 1_000_000_007
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns the number of increasing paths starting from (i, j)."""
            ans = 1 # The current cell contributes 1 length.
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if grid[x][y] <= grid[i][j]:
                    continue
                ans += dp(x, y)
            ans %= MOD
            return ans

        return sum(dp(i, j)
                    for i in range(m)
                    for j in range(n)) % MOD

```

1279. 233.py

Path: LeetCode-main/solutions/233. Number of Digit One/233.py

```

class Solution:
    def countDigitOne(self, n: int) -> int:
        ans = 0

        pow10 = 1
        while pow10 <= n:
            divisor = pow10 * 10
            quotient = n // divisor
            remainder = n % divisor
            if quotient > 0:
                ans += quotient * pow10
            if remainder >= pow10:
                ans += min(remainder - pow10 + 1, pow10)
            pow10 *= 10

        return ans

```

1280. 2330.py

Path: LeetCode-main/solutions/2330. Valid Palindrome IV/2330.py

```

class Solution:
    def makePalindrome(self, s: str) -> bool:
        change = 0
        l = 0
        r = len(s) - 1

        while l < r:
            if s[l] != s[r]:
                change += 1
                if change > 2:
                    return False
            l += 1
            r -= 1

        return True

```

1281. 2331.py

Path: LeetCode-main/solutions/2331. Evaluate Boolean Binary Tree/2331.py

```

class Solution:
    def evaluateTree(self, root: TreeNode | None) -> bool:
        if root.val < 2:
            return root.val
        if root.val == 2: # OR
            return self.evaluateTree(root.left) or self.evaluateTree(root.right)
        # AND
        return self.evaluateTree(root.left) and self.evaluateTree(root.right)

```

1282. 2332.py

Path: LeetCode-main/solutions/2332. The Latest Time to Catch a Bus/2332.py


```
class Solution:
    def latestTimeCatchTheBus(
        self,
        buses: list[int],
        passengers: list[int],
        capacity: int,
    ) -> int:
        buses.sort()
        passengers.sort()

        if passengers[0] > buses[-1]:
            return buses[-1]

        ans = passengers[0] - 1
        i = 0 # buses' index
        j = 0 # passengers' index

        while i < len(buses):
            # Greedily make passengers catch `buses[i]`.
            arrived = 0
            while arrived < capacity and j < len(passengers) and passengers[j] <= buses[i]:
                if j > 0 and passengers[j] != passengers[j - 1] + 1:
                    ans = passengers[j] - 1
                j += 1
                arrived += 1
            # There's room for `buses[i]` to carry a passenger arriving at the
            # `buses[i]`.
            if arrived < capacity and j > 0 and passengers[j - 1] != buses[i]:
                ans = buses[i]
                i += 1
            i += 1

        return ans
```

1283. 2334.py

Path: LeetCode-main/solutions/2334. Subarray With Elements Greater Than Varying Threshold/2334.py

```

class Solution:
    # Similar to 907. Sum of Subarray Minimums
    def validSubarraySize(self, nums: list[int], threshold: int) -> int:
        n = len(nums)
        ans = 0
        # prev[i] := the index k s.t. nums[k] is the previous minimum in nums[0..n)
        prev = [-1] * n
        # next[i] := the index k s.t. nums[k] is the next minimum in nums[i + 1..n)
        next = [n] * n
        stack = []

        for i, a in enumerate(nums):
            while stack and nums[stack[-1]] > a:
                index = stack.pop()
                next[index] = i
            if stack:
                prev[i] = stack[-1]
            stack.append(i)

        for i, (num, prevIndex, nextIndex) in enumerate(zip(nums, prev, next)):
            k = (i - prevIndex) + (nextIndex - i) - 1
            if num > threshold / k:
                return k

        return -1

```

1284. 2335.py

Path: LeetCode-main/solutions/2335. Minimum Amount of Time to Fill Cups/2335.py

```

class Solution:
    def fillCups(self, amount: list[int]) -> int:
        return max(max(amount), (sum(amount) + 1) // 2)

```

1285. 2337.py

Path: LeetCode-main/solutions/2337. Move Pieces to Obtain a String/2337.py

```

class Solution:
    def canChange(self, start: str, target: str) -> bool:
        n = len(start)
        i = 0 # start's index
        j = 0 # target's index

        while i <= n and j <= n:
            while i < n and start[i] == '_':
                i += 1
            while j < n and target[j] == '_':
                j += 1
            if i == n or j == n:
                return i == n and j == n
            if start[i] != target[j]:
                return False
            if start[i] == 'R' and i > j:
                return False
            if start[i] == 'L' and i < j:
                return False
            i += 1
            j += 1

        return True

```

1286. 234.py

Path: LeetCode-main/solutions/234. Palindrome Linked List/234.py

```

class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        def reverseList(head: ListNode) -> ListNode:
            prev = None
            curr = head
            while curr:
                next = curr.next
                curr.next = prev
                prev = curr
                curr = next
            return prev

        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        if fast:
            slow = slow.next
        slow = reverseList(slow)

        while slow:
            if slow.val != head.val:
                return False
            slow = slow.next
            head = head.next

        return True

```

1287. 2340.py

Path: LeetCode-main/solutions/2340. Minimum Adjacent Swaps to Make a Valid Array/2340.py

```
class Solution:
    def minimumSwaps(self, nums: list[int]) -> int:
        minIndex = self._getLeftmostMinIndex(nums)
        maxIndex = self._getRightmostMaxIndex(nums)
        swaps = minIndex + (len(nums) - 1 - maxIndex)
        return swaps if minIndex <= maxIndex else swaps - 1

    def _getLeftmostMinIndex(self, nums: list[int]) -> int:
        mn = nums[0]
        minIndex = 0
        for i in range(1, len(nums)):
            if nums[i] < mn:
                mn = nums[i]
                minIndex = i
        return minIndex

    def _getRightmostMaxIndex(self, nums: list[int]) -> int:
        mx = nums[-1]
        maxIndex = len(nums) - 1
        for i in range(len(nums) - 2, -1, -1):
            if nums[i] > mx:
                mx = nums[i]
                maxIndex = i
        return maxIndex
```

1288. 2341.py

Path: LeetCode-main/solutions/2341. Maximum Number of Pairs in Array/2341.py

```
class Solution:
    def numberOfPairs(self, nums: list[int]) -> list[int]:
        ans = [0] * 2
        count = collections.Counter(nums)

        for i in range(101):
            ans[0] += count[i] // 2
            ans[1] += count[i] & 1

        return ans
```

1289. 2342.py

Path: LeetCode-main/solutions/2342. Max Sum of a Pair With Equal Sum of Digits/2342.py

```

class Solution:
    def maximumSum(self, nums: list[int]) -> int:
        MAX = 9 * 9 # 999,999,999
        ans = -1
        count = [[] for _ in range(MAX + 1)]

        for num in nums:
            count[self._getDigitSum(num)].append(num)

        for groupNums in count:
            if len(groupNums) < 2:
                continue
            groupNums.sort(reverse=True)
            ans = max(ans, groupNums[0] + groupNums[1])

        return ans

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))

```

1290. 2344.py

Path: LeetCode-main/solutions/2344. Minimum Deletions to Make Array Divisible/2344.py

```

class Solution:
    def minOperations(self, nums: list[int], numsDivide: list[int]) -> int:
        gcd = functools.reduce(math.gcd, numsDivide)

        for i, num in enumerate(sorted(nums)):
            if gcd % num == 0:
                return i

        return -1

```

1291. 2345.py

Path: LeetCode-main/solutions/2345. Finding the Number of Visible Mountains/2345.py

```

class Solution:
    def visibleMountains(self, peaks: list[list[int]]) -> int:
        ans = 0
        maxRightFoot = 0

        peaks.sort(key=lambda x: (x[0] - x[1], -x[0]))

        for i, peak in enumerate(peaks):
            overlapWithNext = i + 1 < len(peaks) and peak == peaks[i + 1]
            currRightFoot = peak[0] + peak[1]
            if currRightFoot > maxRightFoot:
                if not overlapWithNext:
                    ans += 1
                maxRightFoot = currRightFoot

        return ans

```

1292. 2347.py

Path: LeetCode-main/solutions/2347. Best Poker Hand/2347.py

```
class Solution:
    def bestHand(self, ranks: list[int], suits: list[str]) -> str:
        if all(suit == suits[0] for suit in suits):
            return 'Flush'

        match max(Counter(ranks).values()):
            case 5 | 4 | 3:
                return 'Three of a Kind'
            case 2:
                return 'Pair'
            case _:
                return 'High Card'
```

1293. 2348.py

Path: LeetCode-main/solutions/2348. Number of Zero-Filled Subarrays/2348.py

```
class Solution:
    def zeroFilledSubarray(self, nums: list[int]) -> int:
        ans = 0
        indexBeforeZero = -1

        for i, num in enumerate(nums):
            if num:
                indexBeforeZero = i
            else:
                ans += i - indexBeforeZero

        return ans
```

1294. 2349.py

Path: LeetCode-main/solutions/2349. Design a Number Container System/2349.py

```
from sortedcontainers import SortedSet

class NumberContainers:
    def __init__(self):
        self.numberToIndices = collections.defaultdict(SortedSet)
        self.indexToNumber = {}

    def change(self, index: int, number: int) -> None:
        if index in self.indexToNumber:
            originalNumber = self.indexToNumber[index]
            self.numberToIndices[originalNumber].remove(index)
            if len(self.numberToIndices[originalNumber]) == 0:
                del self.numberToIndices[originalNumber]
            self.indexToNumber[index] = number
            self.numberToIndices[number].add(index)

    def find(self, number: int) -> int:
        if number in self.numberToIndices:
            return self.numberToIndices[number][0]
        return -1
```

1295. 235.py

Path: LeetCode-main/solutions/235. Lowest Common Ancestor of a Binary Search Tree/235.py

```
class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        if root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)
        return root
```

1296. 2350.py

Path: LeetCode-main/solutions/2350. Shortest Impossible Sequence of Rolls/2350.py

```
class Solution:
    def shortestSequence(self, rolls: list[int], k: int) -> int:
        ans = 1 # the the next target length
        seen = set()

        for roll in rolls:
            seen.add(roll)
            if len(seen) == k:
                # Have all combinations that form `ans` length, and we are going to
                # extend the sequence to `ans + 1` length.
                ans += 1
                seen.clear()

        return ans
```

1297. 2351.py

Path: LeetCode-main/solutions/2351. First Letter to Appear Twice/2351.py

```
class Solution:
    def repeatedCharacter(self, s: str) -> str:
        seen = [False] * 26

        for c in s:
            if seen[ord(c) - ord('a')]:
                return c
            seen[ord(c) - ord('a')] = True
```

1298. 2352.py

Path: LeetCode-main/solutions/2352. Equal Row and Column Pairs/2352.py

```

class Solution:
    def equalPairs(self, grid: list[list[int]]) -> int:
        n = len(grid)
        ans = 0

        for i in range(n):
            for j in range(n):
                k = 0
                while k < n:
                    if grid[i][k] != grid[k][j]:
                        break
                    k += 1
                if k == n: # R[i] == C[j]
                    ans += 1

        return ans

```

1299. 2353.py

Path: LeetCode-main/solutions/2353. Design a Food Rating System/2353.py

```

from sortedcontainers import SortedSet

class FoodRatings:
    def __init__(self, foods: list[str], cuisines: list[str], ratings: list[int]):
        self.cuisineToRatingAndFoods = collections.defaultdict(
            lambda: SortedSet(key=lambda x: (-x[0], x[1])))
        self.foodToCuisine = {}
        self.foodToRating = {}

        for food, cuisine, rating in zip(foods, cuisines, ratings):
            self.cuisineToRatingAndFoods[cuisine].add((rating, food))
            self.foodToCuisine[food] = cuisine
            self.foodToRating[food] = rating

    def changeRating(self, food: str, newRating: int) -> None:
        cuisine = self.foodToCuisine[food]
        oldRating = self.foodToRating[food]
        ratingAndFoods = self.cuisineToRatingAndFoods[cuisine]
        ratingAndFoods.remove((oldRating, food))
        ratingAndFoods.add((newRating, food))
        self.foodToRating[food] = newRating

    def highestRated(self, cuisine: str) -> str:
        return self.cuisineToRatingAndFoods[cuisine][0][1]

```

1300. 2354.py

Path: LeetCode-main/solutions/2354. Number of Excellent Pairs/2354.py

```

class Solution:
    def countExcellentPairs(self, nums: list[int], k: int) -> int:
        count = collections.Counter(map(int.bit_count, set(nums)))
        return sum(count[i] * count[j]
                    for i in count
                    for j in count
                    if i + j >= k)

```

1301. 2355.py

Path: LeetCode-main/solutions/2355. Maximum Number of Books You Can Take/2355.py

```
class Solution:
    def maximumBooks(self, books: list[int]) -> int:
        # dp[i] := the maximum the number of books we can take from books[0..i] with taking all of
        # books[i]
        dp = [0] * len(books)
        stack = [] # the possible indices we can reach

        for i, book in enumerate(books):
            # We may take all of books[j], where books[j] < books[i] - (i - j).
            while stack and books[stack[-1]] >= book - (i - stack[-1]):
                stack.pop()
            # We can now take books[j + 1..i].
            j = stack[-1] if stack else -1
            lastPicked = book - (i - j) + 1
            if lastPicked > 1:
                # book + (book - 1) + ... + (book - (i - j) + 1)
                dp[i] = (book + lastPicked) * (i - j) // 2
            else:
                # 1 + 2 + ... + book
                dp[i] = book * (book + 1) // 2
            if j >= 0:
                dp[i] += dp[j]
            stack.append(i)

        return max(dp)
```

1302. 2357.py

Path: LeetCode-main/solutions/2357. Make Array Zero by Subtracting Equal Amounts/2357.py

```
class Solution:
    def minimumOperations(self, nums: list[int]) -> int:
        return len(set(nums) - {0})
```

1303. 2358.py

Path: LeetCode-main/solutions/2358. Maximum Number of Groups Entering a Competition/2358.py

```
class Solution:
    def maximumGroups(self, grades: list[int]) -> int:
        # Sort grades, then we can separate the students into groups of sizes 1, 2,
        # 3, ..., k, s.t. the i-th group < the (i + 1)-th group for both sum and
        # size. So, we can rephrase the problem into:
        # Find the maximum k s.t. 1 + 2 + 3 + ... + k <= n

        # 1 + 2 + 3 + ... + k <= n
        # k(k + 1) // 2 <= n
        # k^2 + k <= 2n
        # (k + 0.5)^2 - 0.25 <= 2n
        # (k + 0.5)^2 <= 2n + 0.25
        # k <= sqrt(2n + 0.25) - 0.5
        return int(math.sqrt(len(grades) * 2 + 0.25) - 0.5)
```

1304. 2359.py

Path: LeetCode-main/solutions/2359. Find Closest Node to Given Two Nodes/2359.py

```
class Solution:
    def closestMeetingNode(self, edges: list[int], node1: int, node2: int) -> int:
        MAX = 10000
        dist1 = self._getDist(edges, node1)
        dist2 = self._getDist(edges, node2)
        minDist = MAX
        ans = -1

        for i, (d1, d2) in enumerate(zip(dist1, dist2)):
            if min(d1, d2) >= 0:
                maxDist = max(d1, d2)
                if maxDist < minDist:
                    minDist = maxDist
                    ans = i

        return ans

    def _getDist(self, edges: list[int], u: int) -> list[int]:
        dist = [-1] * len(edges)
        d = 0
        while u != -1 and dist[u] == -1:
            dist[u] = d
            d += 1
            u = edges[u]
        return dist
```

1305. 236-2.py

Path: LeetCode-main/solutions/236. Lowest Common Ancestor of a Binary Tree/236-2.py

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: TreeNode | None,
        p: TreeNode | None,
        q: TreeNode | None,
    ) -> TreeNode | None:
        q_ = collections.deque([root])
        parent = {root: None}
        ancestors = set() # p's ancestors

        # Iterate until we find both p and q.
        while p not in parent or q not in parent:
            root = q_.popleft()
            if root.left:
                parent[root.left] = root
                q_.append(root.left)
            if root.right:
                parent[root.right] = root
                q_.append(root.right)

        # Insert all the p's ancestors.
        while p:
            ancestors.add(p)
            p = parent[p] # `p` becomes None in the end.

        # Go up from q until we meet any of p's ancestors.
        while q not in ancestors:
            q = parent[q]

        return q

```

1306. 236.py

Path: LeetCode-main/solutions/236. Lowest Common Ancestor of a Binary Tree/236.py

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if not root or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        if left and right:
            return root
        return left or right

```

1307. 2360.py

Path: LeetCode-main/solutions/2360. Longest Cycle in a Graph/2360.py

```

class Solution:
    def longestCycle(self, edges: list[int]) -> int:
        ans = -1
        time = 1
        timeVisited = [0] * len(edges)

        for i, edge in enumerate(edges):
            if timeVisited[i]:
                continue
            startTime = time
            u = i
            while u != -1 and not timeVisited[u]:
                timeVisited[u] = time
                time += 1
                u = edges[u] # Move to the next node.
            if u != -1 and timeVisited[u] >= startTime:
                ans = max(ans, time - timeVisited[u])

        return ans

```

1308. 2361.py

Path: LeetCode-main/solutions/2361. Minimum Costs Using the Train Line/2361.py

```

class Solution:
    def minimumCosts(
        self,
        regular: list[int],
        express: list[int],
        expressCost: int,
    ) -> list[int]:
        n = len(regular)
        ans = [0] * n
        # the minimum cost to reach the current stop in a regular route
        dpReg = 0
        # the minimum cost to reach the current stop in an express route
        dpExp = expressCost

        for i in range(n):
            prevReg = dpReg
            prevExp = dpExp
            dpReg = min(prevReg + regular[i], prevExp + 0 + regular[i])
            dpExp = min(prevReg + expressCost + express[i], prevExp + express[i])
            ans[i] = min(dpReg, dpExp)

        return ans

```

1309. 2363.py

Path: LeetCode-main/solutions/2363. Merge Similar Items/2363.py

```

class Solution:
    def mergeSimilarItems(self, items1: list[list[int]],
                          items2: list[list[int]]) -> list[list[int]]:
        return sorted(
            (Counter(dict(items1)) + collections.Counter(dict(items2))).items()
        )

```

1310. 2364.py

Path: LeetCode-main/solutions/2364. Count Number of Bad Pairs/2364.py

```
class Solution:
    def countBadPairs(self, nums: list[int]) -> int:
        ans = 0
        count = collections.Counter() # (nums[i] - i)

        for i, num in enumerate(nums):
            # count[nums[i] - i] := the number of good pairs
            # i - count[nums[i] - i] := the number of bad pairs
            ans += i - count[num - i]
            count[num - i] += 1

        return ans
```

1311. 2365.py

Path: LeetCode-main/solutions/2365. Task Scheduler II/2365.py

```
class Solution:
    def taskSchedulerII(self, tasks: list[int], space: int) -> int:
        taskToNextAvailable = collections.defaultdict(int)
        ans = 0

        for task in tasks:
            ans = max(ans + 1, taskToNextAvailable[task])
            taskToNextAvailable[task] = ans + space + 1

        return ans
```

1312. 2366.py

Path: LeetCode-main/solutions/2366. Minimum Replacements to Sort the Array/2366.py

```
class Solution:
    def minimumReplacement(self, nums: list[int]) -> int:
        ans = 0
        mx = nums[-1]

        for i in range(len(nums) - 2, -1, -1):
            ops = (nums[i] - 1) // mx
            ans += ops
            mx = nums[i] // (ops + 1)

        return ans
```

1313. 2367.py

Path: LeetCode-main/solutions/2367. Number of Arithmetic Triplets/2367.py

```
class Solution:
    def arithmeticTriplets(self, nums: list[int], diff: int) -> int:
        MAX = 200
        ans = 0
        count = [False] * (MAX + 1)

        for num in nums:
            if num >= 2 * diff and count[num - diff] and count[num - 2 * diff]:
                ans += 1
            count[num] = True

        return ans
```

1314. 2368.py

Path: LeetCode-main/solutions/2368. Reachable Nodes With Restrictions/2368.py

```
class Solution:
    def reachableNodes(
        self,
        n: int,
        edges: list[list[int]],
        restricted: list[int],
    ) -> int:
        tree = [[] for _ in range(n)]
        seen = set(restricted)

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def dfs(u: int) -> int:
            if u in seen:
                return 0
            seen.add(u)
            return 1 + sum(dfs(v) for v in tree[u])

        return dfs(0)
```

1315. 2369.py

Path: LeetCode-main/solutions/2369. Check if There is a Valid Partition For The Array/2369.py

```

class Solution:
    def validPartition(self, nums: list[int]) -> bool:
        n = len(nums)
        # dp[i] := True if there's a valid partition for the first i numbers
        dp = [False] * (n + 1)
        dp[0] = True
        dp[2] = nums[0] == nums[1]

        for i in range(3, n + 1):
            dp[i] = (
                dp[i - 2] and nums[i - 2] == nums[i - 1]) or (
                dp[i - 3]
                and (
                    (nums[i - 3] == nums[i - 2] and nums[i - 2] == nums[i - 1])
                    or (
                        nums[i - 3] + 1 == nums[i - 2] and nums[i - 2] + 1 == nums
                        [i - 1]))))

        return dp[n]

```

1316. 237.py

Path: LeetCode-main/solutions/237. Delete Node in a Linked List/237.py

```

class Solution:
    def deleteNode(self, node):
        node.val = node.next.val
        node.next = node.next.next

```

1317. 2370.py

Path: LeetCode-main/solutions/2370. Longest Ideal Subsequence/2370.py

```

class Solution:
    def longestIdealString(self, s: str, k: int) -> int:
        # dp[i] := the longest subsequence that ends in ('a' + i)
        dp = [0] * 26

        for c in s:
            i = ord(c) - ord('a')
            dp[i] = 1 + self._getMaxReachable(dp, i, k)

        return max(dp)

    def _getMaxReachable(self, dp: list[int], i: int, k: int) -> int:
        first = max(0, i - k)
        last = min(25, i + k)
        maxReachable = 0
        for j in range(first, last + 1):
            maxReachable = max(maxReachable, dp[j])
        return maxReachable

```

1318. 2371.py

Path: LeetCode-main/solutions/2371. Minimize Maximum Value in a Grid/2371.py

```

class Solution:
    def minScore(self, grid: list[list[int]]) -> list[list[int]]:
        m = len(grid)
        n = len(grid[0])
        ans = [[0] * n for _ in range(m)]
        valAndIndices = []
        rows = [0] * m # rows[i] := the maximum used number so far
        cols = [0] * n # cols[j] := the maximum used number so far

        for i in range(m):
            for j in range(n):
                valAndIndices.append((grid[i][j], i, j))

        valAndIndices.sort()

        for _, i, j in valAndIndices:
            nextAvailable = max(rows[i], cols[j]) + 1
            ans[i][j] = nextAvailable
            rows[i] = nextAvailable
            cols[j] = nextAvailable

        return ans

```

1319. 2373.py

Path: LeetCode-main/solutions/2373. Largest Local Values in a Matrix/2373.py

```

class Solution:
    def largestLocal(self, grid: list[list[int]]) -> list[list[int]]:
        n = len(grid)
        ans = [[0] * (n - 2) for _ in range(n - 2)]

        for i in range(n - 2):
            for j in range(n - 2):
                for x in range(i, i + 3):
                    for y in range(j, j + 3):
                        ans[i][j] = max(ans[i][j], grid[x][y])

        return ans

```

1320. 2374.py

Path: LeetCode-main/solutions/2374. Node With Highest Edge Score/2374.py

```

class Solution:
    def edgeScore(self, edges: list[int]) -> int:
        scores = [0] * len(edges)
        for i, edge in enumerate(edges):
            scores[edge] += i
        return scores.index(max(scores))

```

1321. 2375.py

Path: LeetCode-main/solutions/2375. Construct Smallest Number From DI String/2375.py


```

class Solution:
    def smallestNumber(self, pattern: str) -> str:
        ans = []
        stack = ['1']

        for c in pattern:
            maxSorFar = stack[-1]
            if c == 'I':
                while stack:
                    maxSorFar = max(maxSorFar, stack[-1])
                    ans.append(stack.pop())
                stack.append(chr(ord(maxSorFar) + 1))

        while stack:
            ans.append(stack.pop())

        return ''.join(ans)

```

1322. 2376.py

Path: LeetCode-main/solutions/2376. Count Special Integers/2376.py

```

class Solution:
    # Same as 1012. Numbers With Repeated Digits
    def countSpecialNumbers(self, n: int) -> int:
        s = str(n)

        @functools.lru_cache(None)
        def dp(i: int, used: int, tight: bool) -> int:
            """
            Returns the number of special integers, considering the i-th digit, where
            `used` is the bitmask of the used digits, and `tight` indicates if the
            current digit is tightly bound.
            """
            if i == len(s):
                return 1

            res = 0
            maxDigit = int(s[i]) if tight else 9

            for d in range(maxDigit + 1):
                # `d` is used.
                if used >> d & 1:
                    continue
                # Use `d` now.
                nextTight = tight and (d == maxDigit)
                if used == 0 and d == 0: # Don't count leading 0s as used.
                    res += dp(i + 1, used, nextTight)
                else:
                    res += dp(i + 1, used | 1 << d, nextTight)

            return res

        return dp(0, 0, True) - 1 # - 0

```

1323. 2378.py

Path: LeetCode-main/solutions/2378. Choose Edges to Maximize Score in a Tree/2378.py

```

class Solution:
    def maxScore(self, edges: list[list[int]]) -> int:
        n = len(edges)
        graph = [[] for _ in range(n)]

        for i, (parent, weight) in enumerate(edges):
            if parent != -1:
                graph[parent].append((i, weight))

        takeRoot, notTakeRoot = self._dfs(graph, 0)
        return max(takeRoot, notTakeRoot)

    def _dfs(self, graph: list[list[int]], u: int) -> tuple[int, int]:
        """
        Returns (the maximum sum at u if we take one u->v edge,
                 the maximum sum at u if we don't take any child edge).
        """
        bestEdge = 0
        notTakeU = 0

        for v, w in graph[u]:
            takeV, notTakeV = self._dfs(graph, v)
            bestEdge = max(bestEdge, w + notTakeV - takeV)
            notTakeU += takeV

        return (bestEdge + notTakeU, notTakeU)

```

1324. 2379.py

Path: LeetCode-main/solutions/2379. Minimum Recolors to Get K Consecutive Black Blocks/2379.py

```

class Solution:
    def minimumRecolors(self, blocks: str, k: int) -> int:
        countB = 0
        maxCountB = 0

        for i, block in enumerate(blocks):
            if block == 'B':
                countB += 1
            if i >= k and blocks[i - k] == 'B':
                countB -= 1
            maxCountB = max(maxCountB, countB)

        return k - maxCountB

```

1325. 238-2.py

Path: LeetCode-main/solutions/238. Product of Array Except Self/238-2.py

```

class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [1] * n

        # Use ans as the prefix product array.
        for i in range(1, n):
            ans[i] = ans[i - 1] * nums[i - 1]

        suffix = 1 # suffix product
        for i, num in reversed(list(enumerate(nums))):
            ans[i] *= suffix
            suffix *= num

        return ans

```

1326. 238.py

Path: LeetCode-main/solutions/238. Product of Array Except Self/238.py

```

class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        prefix = [1] * n # prefix product
        suffix = [1] * n # suffix product

        for i in range(1, n):
            prefix[i] = prefix[i - 1] * nums[i - 1]

        for i in reversed(range(n - 1)):
            suffix[i] = suffix[i + 1] * nums[i + 1]

        return [prefix[i] * suffix[i] for i in range(n)]

```

1327. 2380.py

Path: LeetCode-main/solutions/2380. Time Needed to Rearrange a Binary String/2380.py

```

class Solution:
    def secondsToRemoveOccurrences(self, s: str) -> int:
        ans = 0
        zeros = 0

        for c in s:
            if c == '0':
                zeros += 1
            elif zeros > 0: # c == '1'
                ans = max(ans + 1, zeros)

        return ans

```

1328. 2381.py

Path: LeetCode-main/solutions/2381. Shifting Letters II/2381.py

```

class Solution:
    def shiftingLetters(self, s: str, shifts: list[list[int]]) -> str:
        ans = []
        currShift = 0
        line = [0] * (len(s) + 1)

        for start, end, direction in shifts:
            diff = 1 if direction else -1
            line[start] += diff
            line[end + 1] -= diff

        for i, c in enumerate(s):
            currShift = (currShift + line[i]) % 26
            num = (ord(c) - ord('a') + currShift + 26) % 26
            ans.append(chr(ord('a') + num))

        return ''.join(ans)

```

1329. 2382.py

Path: LeetCode-main/solutions/2382. Maximum Segment Sum After Removals/2382.py

```

class Solution:
    def maximumSegmentSum(
        self,
        nums: list[int],
        removeQueries: list[int],
    ) -> list[int]:
        n = len(nums)
        maxSum = 0
        ans = [0] * n
        # For the segment [l, r], record its sum in summ[l] and summ[r]
        summ = [0] * n
        # For the segment [l, r], record its count in count[l] and count[r]
        count = [0] * n

        for i in reversed(range(n)):
            ans[i] = maxSum
            j = removeQueries[i]

            # Calculate `segmentSum`.
            leftSum = summ[j - 1] if j > 0 else 0
            rightSum = summ[j + 1] if j + 1 < n else 0
            segmentSum = nums[j] + leftSum + rightSum

            # Calculate `segmentCount`.
            leftCount = count[j - 1] if j > 0 else 0
            rightCount = count[j + 1] if j + 1 < n else 0
            segmentCount = 1 + leftCount + rightCount

            # Update `summ` and `count` of the segment [l, r].
            l = j - leftCount
            r = j + rightCount
            summ[l] = segmentSum
            summ[r] = segmentSum
            count[l] = segmentCount
            count[r] = segmentCount
            maxSum = max(maxSum, segmentSum)

        return ans

```

1330. 2383.py

Path: LeetCode-main/solutions/2383. Minimum Hours of Training to Win a Competition/2383.py

```
class Solution:
    def minNumberOfHours(
        self,
        initialEnergy: int,
        initialExperience: int,
        energy: list[int],
        experience: list[int],
    ) -> int:
        return (self._getRequiredEnergy(initialEnergy, energy) +
                self._getRequiredExperience(initialExperience, experience))

    def _getRequiredEnergy(self, initialEnergy: int, energy: list[int]) -> int:
        return max(0, sum(energy) + 1 - initialEnergy)

    def _getRequiredExperience(
        self,
        currentExperience: int,
        experience: list[int],
    ) -> int:
        requiredExperience = 0
        for e in experience:
            if e >= currentExperience:
                requiredExperience += e + 1 - currentExperience
                currentExperience += e + 1 - currentExperience
            currentExperience += e
        return requiredExperience
```

1331. 2384.py

Path: LeetCode-main/solutions/2384. Largest Palindromic Number/2384.py

```
class Solution:
    def largestPalindromic(self, num: str) -> str:
        count = collections.Counter(num)
        firstHalf = ''.join(count[i] // 2 * i for i in '9876543210').rstrip('0')
        mid = self._getMid(count)
        return (firstHalf + mid + firstHalf[::-1]) or '0'

    def _getMid(self, count: dict[str, int]) -> str:
        for c in '9876543210':
            if count[c] & 1:
                return c
        return ''
```

1332. 2385.py

Path: LeetCode-main/solutions/2385. Amount of Time for Binary Tree to Be Infected/2385.py

```

class Solution:
    def amountOfTime(self, root: TreeNode | None, start: int) -> int:
        ans = -1
        graph = self._getGraph(root)
        q = collections.deque([start])
        seen = {start}

        while q:
            ans += 1
            for _ in range(len(q)):
                u = q.popleft()
                if u not in graph:
                    continue
                for v in graph[u]:
                    if v in seen:
                        continue
                    q.append(v)
                    seen.add(v)

        return ans

    def _getGraph(self, root: TreeNode | None) -> dict[int, list[int]]:
        graph = collections.defaultdict(list)
        q = collections.deque([(root, -1)]) # (node, parent)

        while q:
            node, parent = q.popleft()
            if parent != -1:
                graph[parent].append(node.val)
                graph[node.val].append(parent)
            if node.left:
                q.append((node.left, node.val))
            if node.right:
                q.append((node.right, node.val))

        return graph

```

1333. 2386.py

Path: LeetCode-main/solutions/2386. Find the K-Sum of an Array/2386.py

```

class Solution:
    def kSum(self, nums: list[int], k: int) -> int:
        maxSum = sum(num for num in nums if num > 0)
        absNums = sorted(abs(num) for num in nums)
        # (the next maximum sum, the next index i)
        maxHeap = [(-(maxSum - absNums[0]), 0)]
        nextMaxSum = maxSum

        for _ in range(k - 1):
            nextMaxSum, i = heapq.heappop(maxHeap)
            nextMaxSum *= -1
            if i + 1 < len(absNums):
                heapq.heappush(maxHeap, (-(nextMaxSum - absNums[i + 1]), i + 1))
                heapq.heappush(
                    maxHeap, (-(nextMaxSum - absNums[i + 1] + absNums[i]), i + 1))

        return nextMaxSum

```

1334. 2387.py

Path: LeetCode-main/solutions/2387. Median of a Row Wise Sorted Matrix/2387.py

```
class Solution:
    def matrixMedian(self, grid: list[list[int]]) -> int:
        noGreaterThanMedianCount = len(grid) * len(grid[0]) // 2 + 1
        l = 1
        r = 1_000_000

        while l < r:
            m = (l + r) // 2
            if (sum(bisect.bisect_right(row, m) for row in grid) >=
                noGreaterThanMedianCount):
                r = m
            else:
                l = m + 1

        return l
```

1335. 2389.py

Path: LeetCode-main/solutions/2389. Longest Subsequence With Limited Sum/2389.py

```
class Solution:
    def answerQueries(self, nums: list[int], queries: list[int]) -> list[int]:
        nums.sort()

        def numOfElementsLessThan(query: int) -> int:
            summ = 0
            for i, num in enumerate(nums):
                summ += num
                if summ > query:
                    return i
            return len(nums)

        return [numOfElementsLessThan(query) for query in queries]
```

1336. 239.py

Path: LeetCode-main/solutions/239. Sliding Window Maximum/239.py

```
class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        ans = []
        maxQ = collections.deque()

        for i, num in enumerate(nums):
            while maxQ and maxQ[-1] < num:
                maxQ.pop()
            maxQ.append(num)
            if i >= k and nums[i - k] == maxQ[0]: # out-of-bounds
                maxQ.popleft()
            if i >= k - 1:
                ans.append(maxQ[0])

        return ans
```

1337. 2390.py

Path: LeetCode-main/solutions/2390. Removing Stars From a String/2390.py

```
class Solution:
    def removeStars(self, s: str) -> str:
        ans = []
        for c in s:
            if c == '*':
                ans.pop()
            else:
                ans.append(c)
        return ''.join(ans)
```

1338. 2391.py

Path: LeetCode-main/solutions/2391. Minimum Amount of Time to Collect Garbage/2391.py

```
class Solution:
    def garbageCollection(self, garbage: list[str], travel: list[int]) -> int:
        prefix = list(itertools.accumulate(travel))

        def getTime(c: str) -> int:
            characterCount = 0
            lastIndex = -1
            for i, s in enumerate(garbage):
                if any(g == c for g in s):
                    lastIndex = i
            characterCount += s.count(c)
            return characterCount + (0 if lastIndex <= 0 else prefix[lastIndex - 1])

        return getTime('M') + getTime('P') + getTime('G')
```

1339. 2392.py

Path: LeetCode-main/solutions/2392. Build a Matrix With Conditions/2392.py


```

class Solution:
    def buildMatrix(self, k: int, rowConditions: list[list[int]],
                   colConditions: list[list[int]]) -> list[list[int]]:
        rowOrder = self._topologicalSort(rowConditions, k)
        if not rowOrder:
            return []

        colOrder = self._topologicalSort(colConditions, k)
        if not colOrder:
            return []

        ans = [[0] * k for _ in range(k)]
        nodeToRowIndex = [0] * (k + 1)

        for i, node in enumerate(rowOrder):
            nodeToRowIndex[node] = i

        for j, node in enumerate(colOrder):
            i = nodeToRowIndex[node]
            ans[i][j] = node

        return ans

    def _topologicalSort(self, conditions: list[list[int]], n: int) -> list[int]:
        order = []
        graph = [[] for _ in range(n + 1)]
        inDegrees = [0] * (n + 1)

        # Build the graph.
        for u, v in conditions:
            graph[u].append(v)
            inDegrees[v] += 1

        # Perform topological sorting.
        q = collections.deque([i for i in range(1, n + 1) if inDegrees[i] == 0])

        while q:
            u = q.popleft()
            order.append(u)
            for v in graph[u]:
                inDegrees[v] -= 1
                if inDegrees[v] == 0:
                    q.append(v)

        return order if len(order) == n else []

```

1340. 2393.py

Path: LeetCode-main/solutions/2393. Count Strictly Increasing Subarrays/2393.py

```

class Solution:
    def countSubarrays(self, nums: list[int]) -> int:
        ans = 0

        j = -1
        for i, num in enumerate(nums):
            if i > 0 and num <= nums[i - 1]:
                j = i - 1
            ans += i - j

        return ans

```

1341. 2395.py

Path: LeetCode-main/solutions/2395. Find Subarrays With Equal Sum/2395.py

```
class Solution:
    def findSubarrays(self, nums: list[int]) -> bool:
        seen = set()

        for a, b in zip(nums, nums[1:]):
            summ = a + b
            if summ in seen:
                return True
            seen.add(summ)

        return False
```

1342. 2396.py

Path: LeetCode-main/solutions/2396. Strictly Palindromic Number/2396.py

```
class Solution:
    def isStrictlyPalindromic(self, n: int) -> bool:
        return False
```

1343. 2397.py

Path: LeetCode-main/solutions/2397. Maximum Rows Covered by Columns/2397.py

```

class Solution:
    def maximumRows(self, matrix: list[list[int]], numSelect: int) -> int:
        ans = 0

        def dfs(colIndex: int, leftColsCount: int, mask: int):
            nonlocal ans
            if leftColsCount == 0:
                ans = max(ans, self._getAllZerosRowCount(matrix, mask))
                return

            if colIndex == len(matrix[0]):
                return

            # Choose this column.
            dfs(colIndex + 1, leftColsCount - 1, mask | 1 << colIndex)
            # Don't choose this column.
            dfs(colIndex + 1, leftColsCount, mask)

        dfs(0, numSelect, 0)
        return ans

    def _getAllZerosRowCount(self, matrix: list[list[int]], mask: int) -> int:
        count = 0
        for row in matrix:
            isAllZeros = True
            for i, num in enumerate(row):
                if num == 1 and (mask >> i & 1) == 0:
                    isAllZeros = False
                    break
            if isAllZeros:
                count += 1
        return count

```

1344. 2398.py

Path: LeetCode-main/solutions/2398. Maximum Number of Robots Within Budget/2398.py

```

class Solution:
    def maximumRobots(
        self,
        chargeTimes: list[int],
        runningCosts: list[int],
        budget: int,
    ) -> int:
        cost = 0
        maxQ = collections.deque() # Stores `chargeTimes[i]`.

        j = 0 # window's range := [i..j], so k = i - j + 1
        for i, (chargeTime, runningCost) in enumerate(
            zip(chargeTimes, runningCosts)):
            cost += runningCost
            while maxQ and maxQ[-1] < chargeTime:
                maxQ.pop()
            maxQ.append(chargeTime)
            if maxQ[0] + (i - j + 1) * cost > budget:
                if maxQ[0] == chargeTimes[j]:
                    maxQ.popleft()
                cost -= runningCosts[j]
                j += 1

        return len(chargeTimes) - j

```

1345. 2399.py

Path: LeetCode-main/solutions/2399. Check Distances Between Same Letters/2399.py

```
class Solution:
    def checkDistances(self, s: str, distance: list[int]) -> bool:
        firstSeenIndex = [-1] * 26

        for i, c in enumerate(s):
            j = ord(c) - ord('a')
            prevIndex = firstSeenIndex[j]
            if prevIndex != -1 and i - prevIndex - 1 != distance[j]:
                return False
            firstSeenIndex[j] = i

        return True
```

1346. 24.py

Path: LeetCode-main/solutions/24. Swap Nodes in Pairs/24.py

```
class Solution:
    def swapPairs(self, head: ListNode) -> ListNode:
        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next
            return length

        length = getLength(head)
        dummy = ListNode(0, head)
        prev = dummy
        curr = head

        for _ in range(length // 2):
            next = curr.next
            curr.next = next.next
            next.next = prev.next
            prev.next = next
            prev = curr
            curr = curr.next

        return dummy.next
```

1347. 240.py

Path: LeetCode-main/solutions/240. Search a 2D Matrix II/240.py

```

class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        r = 0
        c = len(matrix[0]) - 1

        while r < len(matrix) and c >= 0:
            if matrix[r][c] == target:
                return True
            if target < matrix[r][c]:
                c -= 1
            else:
                r += 1

        return False

```

1348. 2400.py

Path: LeetCode-main/solutions/2400. Number of Ways to Reach a Position After Exactly k Steps/2400.py

```

class Solution:
    def numberOfWays(self, startPos: int, endPos: int, k: int) -> int:
        # leftStep + rightStep = k
        # rightStep - leftStep = endPos - startPos
        #      2 * rightStep = k + endPos - startPos
        #      rightStep = (k + endPos - startPos) // 2
        val = k + endPos - startPos
        if val < 0 or val % 2 == 1:
            return 0
        rightStep = val // 2
        leftStep = k - rightStep
        if leftStep < 0:
            return 0
        return self._nCk(leftStep + rightStep, min(leftStep, rightStep))

    # C(n, k) = C(n - 1, k) + C(n - 1, k - 1)
    def _nCk(self, n: int, k: int) -> int:
        MOD = 1_000_000_007
        # dp[i] := C(n so far, i)
        dp = [1] + [0] * k

        for _ in range(n): # Calculate n times.
            for j in range(k, 0, -1):
                dp[j] += dp[j - 1]
                dp[j] %= MOD

        return dp[k]

```

1349. 2401.py

Path: LeetCode-main/solutions/2401. Longest Nice Subarray/2401.py

```

class Solution:
    def longestNiceSubarray(self, nums: list[int]) -> int:
        ans = 0
        used = 0

        l = 0
        for r, num in enumerate(nums):
            while used & num:
                used ^= nums[l]
                l += 1
            used |= num
            ans = max(ans, r - l + 1)

        return ans

```

1350. 2402.py

Path: LeetCode-main/solutions/2402. Meeting Rooms III/2402.py

```

class Solution:
    def mostBooked(self, n: int, meetings: list[list[int]]) -> int:
        count = [0] * n

        meetings.sort()

        occupied = [] # (endTime, roomId)
        availableRoomIds = [i for i in range(n)]
        heapq.heapify(availableRoomIds)

        for start, end in meetings:
            # Push meetings ending before this `meeting` in occupied to the
            # `availableRoomIds`.
            while occupied and occupied[0][0] <= start:
                heapq.heappush(availableRoomIds, heapq.heappop(occupied)[1])
            if availableRoomIds:
                roomId = heapq.heappop(availableRoomIds)
                count[roomId] += 1
                heapq.heappush(occupied, (end, roomId))
            else:
                newStart, roomId = heapq.heappop(occupied)
                count[roomId] += 1
                heapq.heappush(occupied, (newStart + (end - start), roomId))

        return count.index(max(count))

```

1351. 2403.py

Path: LeetCode-main/solutions/2403. Minimum Time to Kill All Monsters/2403.py

```

class Solution:
    def minimumTime(self, power: list[int]) -> int:
        n = len(power)
        maxMask = 1 << n
        # dp[i] := the minimum number of days needed to defeat the monsters, where
        # i is the bitmask of the monsters
        dp = [math.inf] * maxMask
        dp[0] = 0

        for mask in range(1, maxMask):
            currentGain = mask.bit_count()
            for i in range(n):
                if mask >> i & 1:
                    dp[mask] = min(dp[mask], dp[mask & ~(1 << i)] +
                                   int(math.ceil(power[i] / currentGain)))

        return dp[-1]

```

1352. 2404.py

Path: LeetCode-main/solutions/2404. Most Frequent Even Element/2404.py

```

class Solution:
    def mostFrequentEven(self, nums: list[int]) -> int:
        ans = -1
        count = collections.Counter()

        for num in nums:
            if num % 2 == 1:
                continue
            count[num] += 1
            newCount = count[num]
            maxCount = count[ans]
            if newCount > maxCount or newCount == maxCount and num < ans:
                ans = num

        return ans

```

1353. 2405.py

Path: LeetCode-main/solutions/2405. Optimal Partition of String/2405.py

```

class Solution:
    def partitionString(self, s: str) -> int:
        ans = 1
        used = 0

        for c in s:
            i = ord(c) - ord('a')
            if used >> i & 1:
                used = 1 << i
                ans += 1
            else:
                used |= 1 << i

        return ans

```

1354. 2406.py

Path: LeetCode-main/solutions/2406. Divide Intervals Into Minimum Number of Groups/2406.py

```
class Solution:
    # Similar to 253. Meeting Rooms II
    def minGroups(self, intervals: list[list[int]]) -> int:
        minHeap = [] # Stores `right`s.

        for left, right in sorted(intervals):
            # There's no overlap, so we can reuse the same group.
            if minHeap and left > minHeap[0]:
                heapq.heappop(minHeap)
                heapq.heappush(minHeap, right)

        return len(minHeap)
```

1355. 2408.py

Path: LeetCode-main/solutions/2408. Design SQL/2408.py

```
class SQL:
    def __init__(self, names: list[str], columns: list[int]):
        self.db: dict[str, list[list[str]]] = collections.defaultdict(list)

    def insertRow(self, name: str, row: list[str]) -> None:
        self.db[name].append(row)

    def deleteRow(self, name: str, rowId: int) -> None:
        pass

    def selectCell(self, name: str, rowId: int, columnId: int) -> str:
        return self.db[name][rowId - 1][columnId - 1]
```

1356. 2409.py

Path: LeetCode-main/solutions/2409. Count Days Spent Together/2409.py


```

class Solution:
    def countDaysTogether(
        self,
        arriveAlice: str,
        leaveAlice: str,
        arriveBob: str,
        leaveBob: str,
    ) -> int:
        days = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

        def toDays(s: str) -> int:
            month = int(s[:2])
            day = int(s[3:])
            prevDays = 0
            for m in range(1, month):
                prevDays += days[m]
            return prevDays + day

        arriveA = toDays(arriveAlice)
        leaveA = toDays(leaveAlice)
        arriveB = toDays(arriveBob)
        leaveB = toDays(leaveBob)
        ans = 0

        for day in range(1, 366):
            if arriveA <= day and day <= leaveA and arriveB <= day and day <= leaveB:
                ans += 1

        return ans

```

1357. 241.py

Path: LeetCode-main/solutions/241. Different Ways to Add Parentheses/241.py

```

class Solution:
    @functools.lru_cache(None)
    def diffWaysToCompute(self, expression: str) -> list[int]:
        ans = []

        for i, c in enumerate(expression):
            if c in '+-*':
                for a in self.diffWaysToCompute(expression[:i]):
                    for b in self.diffWaysToCompute(expression[i + 1:]):
                        ans.append(eval(str(a) + c + str(b)))

        return ans or [int(expression)]

```

1358. 2410.py

Path: LeetCode-main/solutions/2410. Maximum Matching of Players With Trainers/2410.py

```
class Solution:
    def matchPlayersAndTrainers(
        self,
        players: list[int],
        trainers: list[int],
    ) -> int:
        ans = 0

        players.sort()
        trainers.sort()

        for i, trainer in enumerate(trainers):
            if players[ans] <= trainer:
                ans += 1
            if ans == len(players):
                return ans

        return ans
```

1359. 2411.py

Path: LeetCode-main/solutions/2411. Smallest Subarrays With Maximum Bitwise OR/2411.py

```
class Solution:
    def smallestSubarrays(self, nums: list[int]) -> list[int]:
        MAX_BIT = 30
        ans = [1] * len(nums)
        # closest[j] := the closest index i s.t. the j-th bit of nums[i] is 1
        closest = [0] * MAX_BIT

        for i in reversed(range(len(nums))):
            for j in range(MAX_BIT):
                if nums[i] >> j & 1:
                    closest[j] = i
            ans[i] = max(ans[i], closest[j] - i + 1)

        return ans
```

1360. 2412.py

Path: LeetCode-main/solutions/2412. Minimum Money Required Before Transactions/2412.py

```
class Solution:
    def minimumMoney(self, transactions: list[list[int]]) -> int:
        ans = 0
        losses = 0

        # Before picking the final transaction, perform any transaction that raises
        # the required money.
        for cost, cashback in transactions:
            losses += max(0, cost - cashback)

        # Now, pick a transaction to be the final one.
        for cost, cashback in transactions:
            if cost > cashback:
                # The losses except this transaction: losses - (cost - cashback), so
                # add the cost of this transaction = losses - (cost - cashback) + cost.
                ans = max(ans, losses + cashback)
            else:
                # The losses except this transaction: losses, so add the cost of this
                # transaction = losses + cost.
                ans = max(ans, losses + cost)

        return ans
```

1361. 2413.py

Path: LeetCode-main/solutions/2413. Smallest Even Multiple/2413.py

```
class Solution:
    def smallestEvenMultiple(self, n: int) -> int:
        return n * (n % 2 + 1)
```

1362. 2414.py

Path: LeetCode-main/solutions/2414. Length of the Longest Alphabetical Continuous Substring/2414.py

```
class Solution:
    def longestContinuousSubstring(self, s: str) -> int:
        ans = 1
        runningLen = 1

        for a, b in zip(s, s[1:]):
            if ord(a) + 1 == ord(b):
                runningLen += 1
                ans = max(ans, runningLen)
            else:
                runningLen = 1

        return ans
```

1363. 2415.py

Path: LeetCode-main/solutions/2415. Reverse Odd Levels of Binary Tree/2415.py

```

class Solution:
    def reverseOddLevels(self, root: TreeNode | None) -> TreeNode | None:
        def dfs(left: TreeNode | None, right: TreeNode | None, isOddLevel: bool) -> None:
            if not left:
                return
            if isOddLevel:
                left.val, right.val = right.val, left.val
            dfs(left.left, right.right, not isOddLevel)
            dfs(left.right, right.left, not isOddLevel)

        dfs(root.left, root.right, True)
        return root

```

1364. 2416.py

Path: LeetCode-main/solutions/2416. Sum of Prefix Scores of Strings/2416.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.count = 0

class Solution:
    def sumPrefixScores(self, words: list[str]) -> list[int]:
        root = TrieNode()

        def insert(word: str) -> None:
            node: TrieNode = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
                node.count += 1

        for word in words:
            insert(word)

        def getScore(word: str) -> int:
            node: TrieNode = root
            score = 0
            for c in word:
                node = node.children[c]
                score += node.count
            return score

        return [getScore(word) for word in words]

```

1365. 2417.py

Path: LeetCode-main/solutions/2417. Closest Fair Integer/2417.py

```

class Solution:
    def closestFair(self, n: int) -> int:
        digitsCount = len(str(n))
        return (self._getEvenDigits(n) if digitsCount % 2 == 0 else
                self._getOddDigits(digitsCount))

    def _getOddDigits(self, digitsCount: int) -> int:
        zeros = (digitsCount + 1) // 2
        ones = (digitsCount - 1) // 2
        return int('1' + '0' * zeros + '1' * ones)

    def _getEvenDigits(self, n: int) -> int:
        digitsCount = len(str(n))
        maxNum = int('1' + '0' * digitsCount)
        for num in range(n, maxNum):
            if self._isValidNum(num):
                return num
        return self._getOddDigits(digitsCount + 1)

    def _isValidNum(self, num: int) -> bool:
        count = 0
        for c in str(num):
            count += 1 if int(c) % 2 == 0 else -1
        return count == 0

```

1366. 2418.py

Path: LeetCode-main/solutions/2418. Sort the People/2418.py

```

class Solution:
    def sortPeople(self, names: list[str], heights: list[int]) -> list[str]:
        return [height for _, height in
                sorted([(height, name) for name, height in zip(names, heights)], reverse=True)]

```

1367. 2419.py

Path: LeetCode-main/solutions/2419. Longest Subarray With Maximum Bitwise AND/2419.py

```

class Solution:
    def longestSubarray(self, nums: list[int]) -> int:
        ans = 0
        maxIndex = 0
        sameNumLength = 0

        for i, num in enumerate(nums):
            if nums[i] == nums[maxIndex]:
                sameNumLength += 1
                ans = max(ans, sameNumLength)
            elif nums[i] > nums[maxIndex]:
                maxIndex = i
                sameNumLength = 1
                ans = 1
            else:
                sameNumLength = 0

        return ans

```

1368. 242.py

Path: LeetCode-main/solutions/242. Valid Anagram/242.py

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        count = collections.Counter(s)
        count.subtract(collections.Counter(t))
        return all(freq == 0 for freq in count.values())
```

1369. 2420.py

Path: LeetCode-main/solutions/2420. Find All Good Indices/2420.py

```
class Solution:
    # Same as 2100. Find Good Days to Rob the Bank
    def goodIndices(self, nums: list[int], k: int) -> list[int]:
        n = len(nums)
        dec = [1] * n # 1 + the number of continuous decreasing numbers before i
        inc = [1] * n # 1 + the number of continuous increasing numbers after i

        for i in range(1, n):
            if nums[i - 1] >= nums[i]:
                dec[i] = dec[i - 1] + 1

        for i in range(n - 2, -1, -1):
            if nums[i] <= nums[i + 1]:
                inc[i] = inc[i + 1] + 1

        return [i for i in range(k, n - k)
                if dec[i - 1] >= k and inc[i + 1] >= k]
```

1370. 2422.py

Path: LeetCode-main/solutions/2422. Merge Operations to Turn Array Into a Palindrome/2422.py

```
class Solution:
    def minimumOperations(self, nums: list[int]) -> int:
        ans = 0
        l = 0
        r = len(nums) - 1
        leftSum = nums[0]
        rightSum = nums[-1]

        while l < r:
            if leftSum < rightSum:
                l += 1
                leftSum += nums[l]
                ans += 1
            elif leftSum > rightSum:
                r -= 1
                rightSum += nums[r]
                ans += 1
            else: # leftSum == rightSum
                l += 1
                r -= 1
                leftSum = nums[l]
                rightSum = nums[r]

        return ans
```

1371. 2423.py

Path: LeetCode-main/solutions/2423. Remove Letter To Equalize Frequency/2423.py

```
class Solution:
    def equalFrequency(self, word: str) -> bool:
        count = collections.Counter(word)

        # Try to remove each letter, then check if the frequency of all the letters
        # in `word` are equal.
        for c in word:
            count[c] -= 1
            if count[c] == 0:
                del count[c]
            if min(count.values()) == max(count.values()):
                return True
            count[c] += 1

        return False
```

1372. 2424.py

Path: LeetCode-main/solutions/2424. Longest Uploaded Prefix/2424.py

```
class LUPrefix:
    def __init__(self, n: int):
        self.seen = set()
        self.longestPrefix = 0

    def upload(self, video: int) -> None:
        self.seen.add(video)
        while self.longestPrefix + 1 in self.seen:
            self.longestPrefix += 1

    def longest(self) -> int:
        return self.longestPrefix
```

1373. 2425.py

Path: LeetCode-main/solutions/2425. Bitwise XOR of All Pairings/2425.py

```
class Solution:
    def xorAllNums(self, nums1: list[int], nums2: list[int]) -> int:
        xors1 = functools.reduce(operator.xor, nums1)
        xors2 = functools.reduce(operator.xor, nums2)
        # If the size of nums1 is m and the size of nums2 is n, then each number in
        # nums1 is repeated n times and each number in nums2 is repeated m times.
        return (len(nums1) % 2 * xors2) ^ (len(nums2) % 2 * xors1)
```

1374. 2427.py

Path: LeetCode-main/solutions/2427. Number of Common Factors/2427.py

```
class Solution:
    def commonFactors(self, a: int, b: int) -> int:
        gcd = math.gcd(a, b)
        return sum(a % i == 0 and b % i == 0
                    for i in range(1, gcd + 1))
```

1375. 2428.py

Path: LeetCode-main/solutions/2428. Maximum Sum of an Hourglass/2428.py

```
class Solution:
    def maxSum(self, grid: list[list[int]]) -> int:
        return max(
            grid[i - 1][j - 1] + grid[i - 1][j] + grid[i - 1][j + 1] + grid[i][j] +
            grid[i + 1][j - 1] + grid[i + 1][j] + grid[i + 1][j + 1]
            for i in range(1, len(grid) - 1) for j in range(1, len(grid[0]) - 1))
```

1376. 2429.py

Path: LeetCode-main/solutions/2429. Minimize XOR/2429.py


```

class Solution:
    def minimizeXor(self, num1: int, num2: int) -> int:
        MAX_BIT = 30
        bits = num2.bit_count()
        # Can turn off all the bits in `num1`.
        if num1.bit_count() == bits:
            return num1

        ans = 0

        # Turn off the MSB if we have `bits` quota.
        for i in reversed(range(MAX_BIT)):
            if num1 >> i & 1:
                ans |= 1 << i
                bits -= 1
                if bits == 0:
                    return ans

        # Turn on the LSB if we still have `bits`.
        for i in range(MAX_BIT):
            if (num1 >> i & 1) == 0:
                ans |= 1 << i
                bits -= 1
                if bits == 0:
                    return ans

        return ans

```

1377. 243.py

Path: LeetCode-main/solutions/243. Shortest Word Distance/243.py

```

class Solution:
    def shortestDistance(
        self,
        wordsDict: list[str],
        word1: str,
        word2: str,
    ) -> int:
        ans = len(wordsDict)
        index1 = -1 # wordsdict[index1] == word1
        index2 = -1 # wordsdict[index2] == word2

        for i, word in enumerate(wordsDict):
            if word == word1:
                index1 = i
                if index2 != -1:
                    ans = min(ans, index1 - index2)
            if word == word2:
                index2 = i
                if index1 != -1:
                    ans = min(ans, index2 - index1)

        return ans

```

1378. 2430.py

Path: LeetCode-main/solutions/2430. Maximum Deletions on a String/2430.py

```

class Solution:
    def deleteString(self, s: str) -> int:
        n = len(s)
        # lcs[i][j] := the number of the same letters of s[i..n) and s[j..n)
        lcs = [[0] * (n + 1) for _ in range(n + 1)]
        # dp[i] := the maximum number of operations needed to delete s[i..n)
        dp = [1] * n

        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    lcs[i][j] = lcs[i + 1][j + 1] + 1
                if lcs[i][j] >= j - i:
                    dp[i] = max(dp[i], dp[j] + 1)

        return dp[0]

```

1379. 2431-2.py

Path: LeetCode-main/solutions/2431. Maximize Total Tastiness of Purchased Fruits/2431-2.py

```

class Solution:
    def maxTastiness(
        self,
        price: list[int],
        tastiness: list[int],
        maxAmount: int,
        maxCoupons: int,
    ) -> int:
        # dp[j][k] := the maximum tastiness of price so far with j amount of money and k coupons
        dp = [[0] * (maxCoupons + 1) for _ in range(maxAmount + 1)]

        for p, t in zip(price, tastiness):
            for j in range(maxAmount, p // 2 - 1, -1):
                for k in range(maxCoupons, -1, -1):
                    buyWithCoupon = 0 if k == 0 else dp[j - p // 2][k - 1] + t
                    buyWithoutCoupon = 0 if j < p else dp[j - p][k] + t
                    dp[j][k] = max(dp[j][k], buyWithCoupon, buyWithoutCoupon)

        return dp[maxAmount][maxCoupons]

```

1380. 2431.py

Path: LeetCode-main/solutions/2431. Maximize Total Tastiness of Purchased Fruits/2431.py

```

class Solution:
    def maxTastiness(
        self,
        price: list[int],
        tastiness: list[int],
        maxAmount: int,
        maxCoupons: int,
    ) -> int:
        n = len(price)
        # dp[i][j][k] := the maximum tastiness of the first i price with j amount of
        # money and k coupons
        dp = [[0] * (maxCoupons + 1)
              for j in range(maxAmount + 1)]
              for i in range(n + 1)]

        for i in range(1, n + 1):
            # 1-indexed
            currPrice = price[i - 1]
            currTastiness = tastiness[i - 1]
            for amount in range(maxAmount + 1):
                for coupon in range(maxCoupons + 1):
                    # 1. Don't buy, the tastiness will be the same as the first i - 1
                    # price.
                    dp[i][amount][coupon] = dp[i - 1][amount][coupon]

                    # 2. Buy without coupon if have enough money.
                    if amount >= currPrice:
                        dp[i][amount][coupon] = max(
                            dp[i][amount][coupon],
                            dp[i - 1][amount - currPrice][coupon] + currTastiness)

                    # 3. Buy with coupon if have coupon and enough money.
                    if coupon > 0 and amount >= currPrice // 2:
                        dp[i][amount][coupon] = max(
                            dp[i][amount][coupon],
                            dp[i - 1][amount - currPrice // 2][coupon - 1] + currTastiness)

        return dp[n][maxAmount][maxCoupons]

```

1381. 2432.py

Path: LeetCode-main/solutions/2432. The Employee That Worked on the Longest Task/2432.py

```

class Solution:
    def hardestWorker(self, n: int, logs: list[list[int]]) -> int:
        ans = logs[0][0]
        maxWorkingTime = logs[0][1]

        for (_, prevLeaveTime), (id, leaveTime) in zip(logs, logs[1:]):
            workingTime = leaveTime - prevLeaveTime
            if workingTime > maxWorkingTime:
                ans = id
                maxWorkingTime = workingTime
            elif workingTime == maxWorkingTime:
                ans = min(ans, id)

        return ans

```

1382. 2433.py

Path: LeetCode-main/solutions/2433. Find The Original Array of Prefix Xor/2433.py

```
class Solution:
    def findArray(self, pref: list[int]) -> list[int]:
        ans = [0] * len(pref)

        ans[0] = pref[0]
        for i in range(1, len(ans)):
            ans[i] = pref[i] ^ pref[i - 1]

        return ans
```

1383. 2434.py

Path: LeetCode-main/solutions/2434. Using a Robot to Print the Lexicographically Smallest String/2434.py

```
class Solution:
    def robotWithString(self, s: str) -> str:
        ans = []
        count = collections.Counter(s)
        stack = []

        for c in s:
            stack.append(c)
            count[c] -= 1
            minChar = self._getMinChar(count)
            while stack and stack[-1] <= minChar:
                ans.append(stack.pop())

        return ''.join(ans + stack[::-1])

    def _getMinChar(self, count: list[int]) -> str:
        for c in string.ascii_lowercase:
            if count[c]:
                return c
        return 'a'
```

1384. 2435-2.py

Path: LeetCode-main/solutions/2435. Paths in Matrix Whose Sum Is Divisible by K/2435-2.py

```

class Solution:
    def numberOfPaths(self, grid: list[list[int]], k: int) -> int:
        MOD = 1_000_000_007
        m = len(grid)
        n = len(grid[0])
        # dp[i][j][sum] := the number of paths to (i, j), where the sum / k == sum
        dp = [[[0] * k for j in range(n)] for i in range(m)]
        dp[0][0][grid[0][0] % k] = 1

        for i in range(m):
            for j in range(n):
                for summ in range(k):
                    newSum = (summ + grid[i][j]) % k
                    if i > 0:
                        dp[i][j][newSum] += dp[i - 1][j][summ]
                    if j > 0:
                        dp[i][j][newSum] += dp[i][j - 1][summ]
                    dp[i][j][newSum] %= MOD

        return dp[m - 1][n - 1][0]

```

1385. 2435.py

Path: LeetCode-main/solutions/2435. Paths in Matrix Whose Sum Is Divisible by K/2435.py

```

class Solution:
    def numberOfPaths(self, grid: list[list[int]], k: int) -> int:
        MOD = 1_000_000_007
        m = len(grid)
        n = len(grid[0])

        @functools.lru_cache(None)
        def dp(i: int, j: int, summ: int) -> int:
            """
            Returns the number of paths to (i, j), where the sum / k == `summ`.
            """
            if i == m or j == n:
                return 0
            if i == m - 1 and j == n - 1:
                return 1 if (summ + grid[i][j]) % k == 0 else 0
            newSum = (summ + grid[i][j]) % k
            return (dp(i + 1, j, newSum) + dp(i, j + 1, newSum)) % MOD

        return dp(0, 0, 0)

```

1386. 2436.py

Path: LeetCode-main/solutions/2436. Minimum Split Into Subarrays With GCD Greater Than One/2436.py

```
class Solution:
    def minimumSplits(self, nums: list[int]) -> int:
        ans = 1
        gcd = nums[0]

        for num in nums:
            newGcd = math.gcd(gcd, num)
            if newGcd > 1:
                gcd = newGcd
            else:
                gcd = num
                ans += 1

        return ans
```

1387. 2437.py

Path: LeetCode-main/solutions/2437. Number of Valid Clock Times/2437.py

```
class Solution:
    def countTime(self, time: str) -> int:
        ans = 1
        if time[3] == '?':
            ans *= 6
        if time[4] == '?':
            ans *= 10

        if time[0] == '?' and time[1] == '?':
            return ans * 24
        if time[0] == '?':
            return ans * 3 if time[1] < '4' else ans * 2
        if time[1] == '?':
            return ans * 4 if time[0] == '2' else ans * 10
        return ans
```

1388. 2438.py

Path: LeetCode-main/solutions/2438. Range Product Queries of Powers/2438.py

```
class Solution:
    def productQueries(self, n: int, queries: list[list[int]]) -> list[int]:
        MOD = 1_000_000_007
        MAX_BIT = 30
        ans = []
        pows = [1 << i for i in range(MAX_BIT) if n >> i & 1]

        for left, right in queries:
            prod = 1
            for i in range(left, right + 1):
                prod *= pows[i]
            prod %= MOD
            ans.append(prod)

        return ans
```

1389. 2439.py

Path: LeetCode-main/solutions/2439. Minimize Maximum of Array/2439.py

```
class Solution:
    def minimizeArrayValue(self, nums: list[int]) -> int:
        ans = 0
        prefix = 0

        for i, num in enumerate(nums):
            prefix += num
            prefixAvg = math.ceil(prefix / (i + 1))
            ans = max(ans, prefixAvg)

        return ans
```

1390. 244.py

Path: LeetCode-main/solutions/244. Shortest Word Distance II/244.py

```
class WordDistance:
    def __init__(self, wordsDict: list[str]):
        self.wordToIndices = collections.defaultdict(list)
        for i, word in enumerate(wordsDict):
            self.wordToIndices[word].append(i)

    def shortest(self, word1: str, word2: str) -> int:
        indices1 = self.wordToIndices[word1]
        indices2 = self.wordToIndices[word2]
        ans = math.inf

        i = 0
        j = 0
        while i < len(indices1) and j < len(indices2):
            ans = min(ans, abs(indices1[i] - indices2[j]))
            if indices1[i] < indices2[j]:
                i += 1
            else:
                j += 1

        return ans
```

1391. 2440.py

Path: LeetCode-main/solutions/2440. Create Components With Same Value/2440.py

```

class Solution:
    def componentValue(self, nums: list[int], edges: list[list[int]]) -> int:
        MAX = 1_000_000_000
        n = len(nums)
        summ = sum(nums)
        tree = [[] for _ in range(n)]

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def dfs(u: int, target: int, seen: set[bool]) -> int:
            """
            Returns the sum of the subtree rooted at u subtracting the sum of the
            deleted subtrees.
            """
            summ = nums[u]
            seen.add(u)

            for v in tree[u]:
                if v in seen:
                    continue
                summ += dfs(v, target, seen)
                if summ > target:
                    return MAX

            # Delete the tree that has sum == target.
            if summ == target:
                return 0
            return summ

        for i in range(n, 1, -1):
            # Split the tree into i parts, i.e. delete (i - 1) edges.
            if summ % i == 0 and dfs(0, summ // i, set()) == 0:
                return i - 1

        return 0

```

1392. 2441.py

Path: LeetCode-main/solutions/2441. Largest Positive Integer That Exists With Its Negative/2441.py

```

class Solution:
    def findMaxK(self, nums: list[int]) -> int:
        ans = -1
        seen = set()

        for num in nums:
            if -num in seen:
                ans = max(ans, abs(num))
            else:
                seen.add(num)

        return ans

```

1393. 2444.py

Path: LeetCode-main/solutions/2444. Count Subarrays With Fixed Bounds/2444.py


```

class Solution:
    def countSubarrays(self, nums: list[int], minK: int, maxK: int) -> int:
        ans = 0
        j = -1
        prevMinKIndex = -1
        prevMaxKIndex = -1

        for i, num in enumerate(nums):
            if num < minK or num > maxK:
                j = i
            if num == minK:
                prevMinKIndex = i
            if num == maxK:
                prevMaxKIndex = i
            # Any index k in [j + 1, min(prevMinKIndex, prevMaxKIndex)] can be the
            # start of the subarray s.t. nums[k..i] satisfies the conditions.
            ans += max(0, min(prevMinKIndex, prevMaxKIndex) - j)

        return ans

```

1394. 2445.py

Path: LeetCode-main/solutions/2445. Number of Nodes With Value One/2445.py

```

class Solution:
    def numberOfNodes(self, n: int, queries: list[int]) -> int:
        # flipped[i] := True if we should flip all the values in the subtree rooted
        # at i
        flipped = [False] * (n + 1)

        for query in queries:
            flipped[query] = flipped[query] ^ True

        def dfs(label: int, value: int) -> int:
            if label > n:
                return 0
            value ^= flipped[label]
            return value + dfs(label * 2, value) + dfs(label * 2 + 1, value)

        return dfs(1, 0)

```

1395. 2446.py

Path: LeetCode-main/solutions/2446. Determine if Two Events Have Conflict/2446.py

1396. 2447.py

Path: LeetCode-main/solutions/2447. Number of Subarrays With GCD Equal to K/2447.py

```

class Solution:
    def subarrayGCD(self, nums: list[int], k: int) -> int:
        ans = 0
        gcds = collections.Counter()

        for num in nums:
            if num % k == 0:
                nextGcds = collections.defaultdict(int)
                nextGcds[num] += 1
                for prevGcd, count in gcds.items():
                    nextGcds[math.gcd(prevGcd, num)] += count
                ans += nextGcds.get(k, 0)
                gcds = nextGcds
            else:
                # The GCD streak stops, so fresh start from the next number.
                gcds.clear()

        return ans

```

1397. 2448.py

Path: LeetCode-main/solutions/2448. Minimum Cost to Make Array Equal/2448.py

```

class Solution:
    def minCost(self, nums: list[int], cost: list[int]) -> int:
        ans = 0
        l = min(nums)
        r = max(nums)

        def getCost(target: int) -> int:
            return sum(abs(num - target) * c for num, c in zip(nums, cost))

        while l < r:
            m = (l + r) // 2
            cost1 = getCost(m)
            cost2 = getCost(m + 1)
            ans = min(cost1, cost2)
            if cost1 < cost2:
                r = m
            else:
                l = m + 1

        return ans

```

1398. 2449.py

Path: LeetCode-main/solutions/2449. Minimum Number of Operations to Make Arrays Similar/2449.py

```

class Solution:
    def makeSimilar(self, nums: list[int], target: list[int]) -> int:
        nums.sort(key=lambda x: (x % 2, x))
        target.sort(key=lambda x: (x % 2, x))
        return sum(abs(a - b) for a, b in zip(nums, target)) // 4

```

1399. 245.py

Path: LeetCode-main/solutions/245. Shortest Word Distance III/245.py

```

class Solution:
    def shortestWordDistance(
        self,
        wordsDict: list[str],
        word1: str,
        word2: str,
    ) -> int:
        isSame = word1 == word2
        ans = math.inf
        # If word1 == word2, index1 is the newest index.
        index1 = len(wordsDict)
        # If word1 == word2, index2 is the previous index.
        index2 = -len(wordsDict)

        for i, word in enumerate(wordsDict):
            if word == word1:
                if isSame:
                    index2 = index1
                index1 = i
            elif word == word2:
                index2 = i
            ans = min(ans, abs(index1 - index2))

        return ans

```

1400. 2450.py

Path: LeetCode-main/solutions/2450. Number of Distinct Binary Strings After Applying Operations/2450.py

```

class Solution:
    def countDistinctStrings(self, s: str, k: int) -> int:
        # Since the content of `s` doesn't matter, for each i in [0, n - k], we can
        # flip s[i..i + k] or don't flip it. Therefore, there's 2^(n - k + 1) ways.
        return pow(2, len(s) - k + 1, 1_000_000_007)

```

1401. 2451.py

Path: LeetCode-main/solutions/2451. Odd String Difference/2451.py

```

class Solution:
    def oddString(self, words: list[str]) -> str:
        def getDiff(s: str) -> list[int]:
            return [ord(b) - ord(a) for a, b in zip(s, s[1:])]

        wordAndDiffTuples = [(word, tuple(getDiff(word))) for word in words]
        diffTupleCount = collections.Counter()

        for _, diffTuple in wordAndDiffTuples:
            diffTupleCount[diffTuple] += 1

        for word, diffTuple in wordAndDiffTuples:
            if diffTupleCount[diffTuple] == 1:
                return word

```

1402. 2452.py

Path: LeetCode-main/solutions/2452. Words Within Two Edits of Dictionary/2452.py

```
class Solution:
    def twoEditWords(
        self,
        queries: list[str],
        dictionary: list[str],
    ) -> list[str]:
        return [query for query in queries
                if any(sum(a != b for a, b in zip(query, word)) < 3
                       for word in dictionary)]
```

1403. 2453.py

Path: LeetCode-main/solutions/2453. Destroy Sequential Targets/2453.py

```
class Solution:
    def destroyTargets(self, nums: list[int], space: int) -> int:
        count = collections.Counter([num % space for num in nums])
        maxCount = max(count.values())
        return min(num for num in nums if count[num % space] == maxCount)
```

1404. 2454.py

Path: LeetCode-main/solutions/2454. Next Greater Element IV/2454.py

```
class Solution:
    def secondGreaterElement(self, nums: list[int]) -> list[int]:
        ans = [-1] * len(nums)
        # a decreasing stack that stores indices that met the first greater number.
        prevStack = []
        # a decreasing stack that stores indices.
        currStack = []

        for i, num in enumerate(nums):
            # Indices in prevStack meet the second greater num.
            while prevStack and nums[prevStack[-1]] < num:
                ans[prevStack.pop()] = num
            # Push indices that meet the first greater number from `currStack` to
            # `prevStack`. We need a temporary array to make the indices in the
            # `prevStack` increasing.
            decreasingIndices = []
            while currStack and nums[currStack[-1]] < num:
                decreasingIndices.append(currStack.pop())
            while decreasingIndices:
                prevStack.append(decreasingIndices.pop())
            currStack.append(i)

        return ans
```

1405. 2455.py

Path: LeetCode-main/solutions/2455. Average Value of Even Numbers That Are Divisible by Three/2455.py

```

class Solution:
    def averageValue(self, nums: list[int]) -> int:
        summ = 0
        count = 0

        for num in nums:
            if num % 6 == 0:
                summ += num
                count += 1

        return 0 if count == 0 else summ // count

```

1406. 2456.py

Path: LeetCode-main/solutions/2456. Most Popular Video Creator/2456.py

```

class Creator:
    def __init__(self, popularity: int, videoId: str, maxView: int):
        self.popularity = popularity # the popularity sum
        self.videoId = videoId # the video id that has the maximum view
        self.maxView = maxView # the maximum view of the creator

class Solution:
    def mostPopularCreator(self, creators: list[str],
                          ids: list[str],
                          views: list[int]) -> list[list[str]]:

        ans = []
        maxPopularity = 0
        nameToCreator = {}

        for name, id, view in zip(creators, ids, views):
            if name not in nameToCreator:
                nameToCreator[name] = Creator(view, id, view)
                maxPopularity = max(maxPopularity, view)
                continue
            creator = nameToCreator[name]
            creator.popularity += view
            maxPopularity = max(maxPopularity, creator.popularity)
            if (creator.maxView < view or
                creator.maxView == view and creator.videoId > id):
                creator.videoId = id
                creator.maxView = view

        for name, creator in nameToCreator.items():
            if creator.popularity == maxPopularity:
                ans.append([name, creator.videoId])

        return ans

```

1407. 2457.py

Path: LeetCode-main/solutions/2457. Minimum Addition to Make Integer Beautiful/2457.py

```

class Solution:
    def makeIntegerBeautiful(self, n: int, target: int) -> int:
        ans = 0
        power = 1

        # e.g. n = 123. After turning off the last bit by adding 7, n = 130.
        # Effectively, we can think n as 13. That's why we do n = (n / 10) + 1.
        while sum(map(int, str(n))) > target:
            # the cost to turn off the last digit
            ans += power * (10 - n % 10)
            n = n // 10 + 1
            power *= 10

        return ans

```

1408. 2458.py

Path: LeetCode-main/solutions/2458. Height of Binary Tree After Subtree Removal Queries/2458.py

```

class Solution:
    def treeQueries(self, root: TreeNode | None, queries: list[int]) -> list[int]:
        @lru_cache(None)
        def height(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + max(height(root.left), height(root.right))

        # valToMaxHeight[val] := the maximum height without the node with `val`
        valToMaxHeight = {}

        # maxHeight := the maximum height without the current node `root`
        def dfs(root: TreeNode | None, depth: int, maxHeight: int) -> None:
            if not root:
                return
            valToMaxHeight[root.val] = maxHeight
            dfs(root.left, depth + 1, max(maxHeight, depth + height(root.right)))
            dfs(root.right, depth + 1, max(maxHeight, depth + height(root.left)))

        dfs(root, 0, 0)
        return [valToMaxHeight[query] for query in queries]

```

1409. 2459.py

Path: LeetCode-main/solutions/2459. Sort Array by Moving Items to Empty Space/2459.py

```

class Solution:
    def sortArray(self, nums: list[int]) -> int:
        n = len(nums)
        numToIndex = [0] * n

        for i, num in enumerate(nums):
            numToIndex[num] = i

        def minOps(numToIndex: list[int], zeroInBeginning: bool) -> int:
            ops = 0
            num = 1
            # If zeroInBeginning, the correct index of each num is num.
            # If not zeroInBeginning, the correct index of each num is num - 1.
            offset = 0 if zeroInBeginning else 1
            while num < n:
                # 0 is in the correct index, so swap 0 with the first `numInWrongIndex`.
                if (zeroInBeginning and numToIndex[0] == 0 or
                    not zeroInBeginning and numToIndex[0] == n - 1):
                    while numToIndex[num] == num - offset: # num is in correct position
                        num += 1
                    if num == n:
                        return ops
                    numInWrongIndex = num
                # 0 is in the wrong index. e.g. numToIndex[0] == 2, that means 2 is not
                # in nums[2] because nums[2] == 0.
                else:
                    numInWrongIndex = numToIndex[0] + offset
                    numToIndex[0], numToIndex[numInWrongIndex] = (
                        numToIndex[numInWrongIndex], numToIndex[0])
                    ops += 1

            return min(minOps(numToIndex.copy(), True),
                       minOps(numToIndex.copy(), False))

```

1410. 246.py

Path: LeetCode-main/solutions/246. Strobogrammatic Number/246.py

```

class Solution:
    def isStrobogrammatic(self, num: str) -> bool:
        rotated = {'0': '0', '1': '1', '6': '9', '8': '8', '9': '6'}
        l = 0
        r = len(num) - 1

        while l <= r:
            if num[r] not in rotated:
                return False
            if num[l] != rotated[num[r]]:
                return False
            l += 1
            r -= 1

        return True

```

1411. 2460-2.py

Path: LeetCode-main/solutions/2460. Apply Operations to an Array/2460-2.py

```
class Solution:
    def applyOperations(self, nums: list[int]) -> list[int]:
        j = 0
        for i in range(len(nums)):
            if i + 1 < len(nums) and nums[i] == nums[i + 1]:
                nums[i] *= 2
                nums[i + 1] = 0
            if nums[i] > 0:
                nums[i], nums[j] = nums[j], nums[i]
                j += 1
        return nums
```

1412. 2460.py

Path: LeetCode-main/solutions/2460. Apply Operations to an Array/2460.py

```
class Solution:
    def applyOperations(self, nums: list[int]) -> list[int]:
        ans = [0] * len(nums)

        for i in range(len(nums) - 1):
            if nums[i] == nums[i + 1]:
                nums[i] *= 2
                nums[i + 1] = 0

        i = 0
        for num in nums:
            if num > 0:
                ans[i] = num
                i += 1

        return ans
```

1413. 2461.py

Path: LeetCode-main/solutions/2461. Maximum Sum of Distinct Subarrays With Length K/2461.py

```
class Solution:
    def maximumSubarraySum(self, nums: list[int], k: int) -> int:
        ans = 0
        summ = 0
        distinct = 0
        count = collections.Counter()

        for i, num in enumerate(nums):
            summ += num
            count[num] += 1
            if count[num] == 1:
                distinct += 1
            if i >= k:
                count[nums[i - k]] -= 1
                if count[nums[i - k]] == 0:
                    distinct -= 1
                summ -= nums[i - k]
            if i >= k - 1 and distinct == k:
                ans = max(ans, summ)

        return ans
```

1414. 2462.py

Path: LeetCode-main/solutions/2462. Total Cost to Hire K Workers/2462.py

```
class Solution:
    def totalCost(self, costs: list[int], k: int, candidates: int) -> int:
        ans = 0
        i = 0
        j = len(costs) - 1
        minHeapL = [] # First half
        minHeapR = [] # Second half

        for _ in range(k):
            while len(minHeapL) < candidates and i <= j:
                heapq.heappush(minHeapL, costs[i])
                i += 1
            while len(minHeapR) < candidates and i <= j:
                heapq.heappush(minHeapR, costs[j])
                j -= 1
            if not minHeapL:
                ans += heapq.heappop(minHeapR)
            elif not minHeapR:
                ans += heapq.heappop(minHeapL)
            # Both `minHeapL` and `minHeapR` are not empty.
            elif minHeapL[0] <= minHeapR[0]:
                ans += heapq.heappop(minHeapL)
            else:
                ans += heapq.heappop(minHeapR)

        return ans
```

1415. 2463.py

Path: LeetCode-main/solutions/2463. Minimum Total Distance Traveled/2463.py

```

class Solution:
    def minimumTotalDistance(
        self,
        robot: list[int],
        factory: list[list[int]],
    ) -> int:
        robot.sort()
        factory.sort()

        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> int:
            """
            Returns the minimum distance to fix robot[i..n) with factory[j..n), where
            factory[j] already fixed k robots.
            """
            if i == len(robot):
                return 0
            if j == len(factory):
                return math.inf
            skipFactory = dp(i, j + 1, 0)
            position, limit = factory[j]
            useFactory = (dp(i + 1, j, k + 1) + abs(robot[i] - position)
                          if limit > k else math.inf)
            return min(skipFactory, useFactory)

        return dp(0, 0, 0)

```

1416. 2464.py

Path: LeetCode-main/solutions/2464. Minimum Subarrays in a Valid Split/2464.py

```

class Solution:
    def validSubarraySplit(self, nums: list[int]) -> int:
        # dp[i] := the minimum number of subarrays to validly split nums[0..i]
        dp = [math.inf] * len(nums)

        for i, num in enumerate(nums):
            for j in range(i + 1):
                if math.gcd(nums[j], num) > 1:
                    dp[i] = min(dp[i], 1 if j == 0 else dp[j - 1] + 1)

        return -1 if dp[-1] == math.inf else dp[-1]

```

1417. 2465.py

Path: LeetCode-main/solutions/2465. Number of Distinct Averages/2465.py

```

class Solution:
    def distinctAverages(self, nums: list[int]) -> int:
        n = len(nums)
        sums = set()

        nums.sort()

        for i in range(n // 2):
            sums.add(nums[i] + nums[n - 1 - i])

        return len(sums)

```

1418. 2466.py

Path: LeetCode-main/solutions/2466. Count Ways To Build Good Strings/2466.py

```
class Solution:
    def countGoodStrings(self, low: int, high: int, zero: int, one: int) -> int:
        MOD = 1_000_000_007
        ans = 0
        # dp[i] := the number of good strings with length i
        dp = [1] + [0] * high

        for i in range(1, high + 1):
            if i >= zero:
                dp[i] = (dp[i] + dp[i - zero]) % MOD
            if i >= one:
                dp[i] = (dp[i] + dp[i - one]) % MOD
            if i >= low:
                ans = (ans + dp[i]) % MOD

        return ans
```

1419. 2467.py

Path: LeetCode-main/solutions/2467. Most Profitable Path in a Tree/2467.py

```

class Solution:
    def mostProfitablePath(
        self,
        edges: list[list[int]],
        bob: int,
        amount: list[int],
    ) -> int:
        n = len(amount)
        tree = [[] for _ in range(n)]
        parent = [0] * n
        aliceDist = [-1] * n

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        # Fills `parent` and `aliceDist`.
        def dfs(u: int, prev: int, d: int) -> None:
            parent[u] = prev
            aliceDist[u] = d
            for v in tree[u]:
                if aliceDist[v] == -1:
                    dfs(v, u, d + 1)

        dfs(0, -1, 0)

        # Modify amount at the path from node bob to node 0.
        # For each node,
        # 1. If Bob reaches earlier than Alice does, change the amount to 0.
        # 2. If Bob and Alice reach simultaneously, divide the amount by 2.
        u = bob
        bobDist = 0
        while u != 0:
            if bobDist < aliceDist[u]:
                amount[u] = 0
            elif bobDist == aliceDist[u]:
                amount[u] //= 2
            u = parent[u]
            bobDist += 1

        return self._getMoney(tree, 0, -1, amount)

    def _getMoney(
        self,
        tree: list[list[int]],
        u: int,
        prev: int,
        amount: list[int],
    ) -> int:
        # a leaf node
        if len(tree[u]) == 1 and tree[u][0] == prev:
            return amount[u]

        maxPath = -math.inf
        for v in tree[u]:
            if v != prev:
                maxPath = max(maxPath, self._getMoney(tree, v, u, amount))

        return amount[u] + maxPath

```

Path: LeetCode-main/solutions/2468. Split Message Based on Limit/2468.py

```
class Solution:
    def splitMessage(self, message: str, limit: int) -> list[str]:
        MESSAGE_LENGTH = len(message)

        def sz(num: int):
            return len(str(num))

        b = 1
        # the total length of a: initialized with the length of "1"
        aLength = sz(1)

        # the total length of b := b * sz(b)
        # The total length of "</>" := b * 3
        while b * limit < b * (sz(b) + 3) + aLength + MESSAGE_LENGTH:
            # If the length of the last suffix "<b/b>" := sz(b) * 2 + 3 >= limit,
            # then it's impossible that the length of "*<b/b>" <= limit.
            if sz(b) * 2 + 3 >= limit:
                return []
            b += 1
            aLength += sz(b)

        ans = []

        i = 0
        for a in range(1, b + 1):
            # the length of "<a/b>" := sz(a) + sz(b) + 3
            j = limit - (sz(a) + sz(b) + 3)
            ans.append(f'{message[i:i + j]}<a>/{b}>')
            i += j

        return ans
```

1421. 2469.py

Path: LeetCode-main/solutions/2469. Convert the Temperature/2469.py

```
class Solution:
    def convertTemperature(self, celsius: float) -> list[float]:
        return [celsius + 273.15, celsius * 1.8 + 32]
```

1422. 247.py

Path: LeetCode-main/solutions/247. Strobogrammatic Number II/247.py

```

class Solution:
    def findStrobogrammatic(self, n: int) -> list[str]:
        def helper(n: int, k: int) -> list[str]:
            if n == 0:
                return ['']
            if n == 1:
                return ['0', '1', '8']

            ans = []

            for inner in helper(n - 2, k):
                if n < k:
                    ans.append('0' + inner + '0')
                    ans.append('1' + inner + '1')
                    ans.append('6' + inner + '9')
                    ans.append('8' + inner + '8')
                    ans.append('9' + inner + '6')

            return ans

        return helper(n, n)

```

1423. 2470.py

Path: LeetCode-main/solutions/2470. Number of Subarrays With LCM Equal to K/2470.py

```

class Solution:
    def subarrayLCM(self, nums: list[int], k: int) -> int:
        ans = 0

        for i, runningLcm in enumerate(nums):
            for j in range(i, len(nums)):
                runningLcm = math.lcm(runningLcm, nums[j])
                if runningLcm > k:
                    break
                if runningLcm == k:
                    ans += 1

        return ans

```

1424. 2471.py

Path: LeetCode-main/solutions/2471. Minimum Number of Operations to Sort a Binary Tree by Level/2471.py

```

class Solution:
    def minimumOperations(self, root: TreeNode | None) -> int:
        ans = 0
        q = collections.deque([root])

        # e.g. vals = [7, 6, 8, 5]
        # [2, 1, 3, 0]: Initialize the ids based on the order of vals.
        # [3, 1, 2, 0]: Swap 2 with 3, so 2 is in the right place (i == ids[i]).
        # [0, 1, 2, 3]: Swap 3 with 0, so 3 is in the right place.
        while q:
            vals = []
            for _ in range(len(q)):
                node = q.popleft()
                vals.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            # O(n^2 log n), which is not great and leads to TLE.
            ids = [sorted(vals).index(val) for val in vals]
            for i in range(len(ids)):
                while ids[i] != i:
                    j = ids[i]
                    ids[i] = ids[j]
                    ids[j] = i
                    ans += 1

        return ans

```

1425. 2472.py

Path: LeetCode-main/solutions/2472. Maximum Number of Non-overlapping Palindrome Substrings/2472.py

```

class Solution:
    def maxPalindromes(self, s: str, k: int) -> int:
        n = len(s)
        # dp[i] := the maximum number of substrings in the first i chars of s
        dp = [0] * (n + 1)

        def isPalindrome(l: int, r: int) -> bool:
            """Returns True if s[l..r] is a palindrome."""
            if l < 0:
                return False
            while l < r:
                if s[l] != s[r]:
                    return False
                l += 1
                r -= 1
            return True

        # If a palindrome is a subof another palindrome, then considering
        # the longer palindrome won't increase the number of non-overlapping
        # palindromes. So, we only need to consider the shorter one. Also,
        # considering palindromes with both k length and k + 1 length ensures that
        # we look for both even and odd length palindromes.
        for i in range(k, n + 1):
            dp[i] = dp[i - 1]
            # Consider palindrome with length k.
            if isPalindrome(i - k, i - 1):
                dp[i] = max(dp[i], 1 + dp[i - k])
            # Consider palindrome with length k + 1.
            if isPalindrome(i - k - 1, i - 1):
                dp[i] = max(dp[i], 1 + dp[i - k - 1])

        return dp[n]

```

1426. 2473.py

Path: LeetCode-main/solutions/2473. Minimum Cost to Buy Apples/2473.py


```

class Solution:
    def minCost(
        self,
        n: int,
        roads: list[list[int]],
        appleCost: list[int],
        k: int,
    ) -> list[int]:
        graph = [[] for _ in range(n)]

        for u, v, w in roads:
            graph[u - 1].append((v - 1, w))
            graph[v - 1].append((u - 1, w))

        return [self._dijkstra(graph, i, appleCost, k) for i in range(n)]

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        appleCost: list[int],
        k: int
    ) -> int:
        ans = math.inf
        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)]  # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            ans = min(ans, appleCost[u] + (k + 1) * d)
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        return ans

```

1427. 2475.py

Path: LeetCode-main/solutions/2475. Number of Unequal Triplets in Array/2475.py

```

# Assume that we have 4 kinds of numbers a, b, c, and d in the count map.
#
# What we want is:
# cnt[a] * cnt[b] * cnt[c]
# cnt[a] * cnt[b] * cnt[d]
# cnt[a] * cnt[c] * cnt[d]
# cnt[b] * cnt[c] * cnt[d]
#
# The above combinations can be reduced as:
#
# prev          | curr   | next
#
# (0)           * cnt[a] * (cnt[b] + cnt[c] + cnt[d])
# (cnt[a])      * cnt[b] * (cnt[c] + cnt[d])
# (cnt[a] + cnt[b]) * cnt[c] * (cnt[d])
# (cnt[a] + cnt[b] + cnt[c]) * cnt[d] * (0)

```

```

class Solution:
    def unequalTriplets(self, nums: list[int]) -> int:
        ans = 0
        prev = 0
        next = len(nums)

        for freq in collections.Counter(nums).values():
            next -= freq
            ans += prev * freq * next
            prev += freq

        return ans

```

1428. 2476.py

Path: LeetCode-main/solutions/2476. Closest Nodes Queries in a Binary Search Tree/2476.py

```

class Solution:
    def closestNodes(self, root: TreeNode | None, queries: list[int]) -> list[list[int]]:
        sortedVals = []
        self._inorder(root, sortedVals)

        def getClosestPair(query: int) -> list[int]:
            i = bisect_left(sortedVals, query)
            # query is presented in the tree, so just use [query, query].
            if i != len(sortedVals) and sortedVals[i] == query:
                return [query, query]
            # query isn't presented in the tree, so find the closest one if possible.
            return [-1 if i == 0 else sortedVals[i - 1],
                    -1 if i == len(sortedVals) else sortedVals[i]]

        return [getClosestPair(query) for query in queries]

    def _inorder(self, root: TreeNode | None, sortedVals: list[int]) -> None:
        """Walks the BST to collect the sorted numbers."""
        if not root:
            return
        self._inorder(root.left, sortedVals)
        sortedVals.append(root.val)
        self._inorder(root.right, sortedVals)

```

1429. 2477.py

Path: LeetCode-main/solutions/2477. Minimum Fuel Cost to Report to the Capital/2477.py

```
class Solution:
    def minimumFuelCost(self, roads: list[list[int]], seats: int) -> int:
        ans = 0
        tree = [[] for _ in range(len(roads) + 1)]

        for u, v in roads:
            tree[u].append(v)
            tree[v].append(u)

        def dfs(u: int, prev: int) -> int:
            nonlocal ans
            people = 1 + sum(dfs(v, u) for v in tree[u] if v != prev)
            if u > 0:
                # the number of cars needed
                ans += int(math.ceil(people / seats))
            return people

        dfs(0, -1)
        return ans
```

1430. 2478.py

Path: LeetCode-main/solutions/2478. Number of Beautiful Partitions/2478.py

```
class Solution:
    def beautifulPartitions(self, s: str, k: int, minLength: int) -> int:
        def isPrime(c: str) -> bool:
            return c in '2357'

        if not isPrime(s[0]) or isPrime(s[-1]):
            return 0

        MOD = 1_000_000_007

        @lru_cache(None)
        def dp(i: int, k: int) -> int:
            """
            Returns the number of beautiful partitions of s[i..n) with k bars (|)
            left.
            """
            if i <= len(s) and k == 0:
                return 1
            if i >= len(s):
                return 0

            # Don't split between s[i - 1] and s[i].
            ans = dp(i + 1, k) % MOD

            # Split between s[i - 1] and s[i].
            if isPrime(s[i]) and not isPrime(s[i - 1]):
                ans += dp(i + minLength, k - 1)

            return ans % MOD

        return dp(minLength, k - 1)
```

1431. 2479.py

```

class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2

class BitTrie:
    def __init__(self, maxBit: int):
        self.maxBit = maxBit
        self.root = TrieNode()

    def insert(self, num: int) -> None:
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = num >> i & 1
            if not node.children[bit]:
                node.children[bit] = TrieNode()
            node = node.children[bit]

    def getMaxXor(self, num: int) -> int:
        maxXor = 0
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = num >> i & 1
            toggleBit = bit ^ 1
            if node.children[toggleBit]:
                maxXor = maxXor | 1 << i
                node = node.children[toggleBit]
            elif node.children[bit]:
                node = node.children[bit]
            else: # There's nothing in the Bit Trie.
                return 0
        return maxXor

class Solution:
    def maxXor(self, n: int, edges: list[list[int]], values: list[int]) -> int:
        ans = 0
        tree = [[] for _ in range(n)]
        treeSums = [0] * n

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        # Gets the tree sum rooted at node u.
        def getTreeSum(u: int, prev: int) -> int:
            treeSum = values[u] + sum(getTreeSum(v, u) for v in tree[u] if v != prev)
            treeSums[u] = treeSum
            return treeSum

        def dfs(u: int, prev: int, bitTrie: BitTrie) -> None:
            nonlocal ans
            for v in tree[u]:
                if v == prev:
                    continue
                # Preorder to get the ans.
                ans = max(ans, bitTrie.getMaxXor(treeSums[v]))
                # Recursively call on the subtree rooted at node v.
                dfs(v, u, bitTrie)
                # Postorder to insert the tree sum rooted at node v.
                bitTrie.insert(treeSums[v])

        getTreeSum(0, -1)
        maxBit = int(math.log2(max(treeSums[1:n])))

```

```
maxBit = int(math.log2(max(treesums[1:])))
# Similar to 421. Maximum XOR of Two Numbers in an Array
dfs(0, -1, BitTrie(maxBit))
return ans
```

1432. 248.py

Path: LeetCode-main/solutions/248. Strobogrammatic Number III/248.py

```
class Solution:
    def strobogrammaticInRange(self, low: str, high: str) -> int:
        pairs = [['0', '0'], ['1', '1'], ['6', '9'], ['8', '8'], ['9', '6']]
        ans = 0

        def dfs(s: list[str], l: int, r: int) -> None:
            nonlocal ans
            if l > r:
                if len(s) == len(low) and ''.join(s) < low:
                    return
                if len(s) == len(high) and ''.join(s) > high:
                    return
                ans += 1
                return

            for leftDigit, rightDigit in pairs:
                if l == r and leftDigit != rightDigit:
                    continue
                s[l] = leftDigit
                s[r] = rightDigit
                if len(s) > 1 and s[0] == '0':
                    continue
                dfs(s, l + 1, r - 1)

        for n in range(len(low), len(high) + 1):
            dfs([' ']*n, 0, n - 1)

        return ans
```

1433. 2481.py

Path: LeetCode-main/solutions/2481. Minimum Cuts to Divide a Circle/2481.py

```
class Solution:
    def numberOfCuts(self, n: int) -> int:
        if n == 1:
            return 0
        return n // 2 if n % 2 == 0 else n
```

1434. 2482.py

Path: LeetCode-main/solutions/2482. Difference Between Ones and Zeros in Row and Column/2482.py

```

class Solution:
    def onesMinusZeros(self, grid: list[list[int]]) -> list[list[int]]:
        m = len(grid)
        n = len(grid[0])
        ans = [[0] * n for _ in range(m)]
        onesRow = [row.count(1) for row in grid]
        onesCol = [col.count(1) for col in zip(*grid)]

        for i in range(m):
            for j in range(n):
                ans[i][j] = (onesRow[i] + onesCol[j] -
                             (n - onesRow[i]) - (m - onesCol[j]))

        return ans

```

1435. 2483.py

Path: LeetCode-main/solutions/2483. Minimum Penalty for a Shop/2483.py

```

class Solution:
    def bestClosingTime(self, customers: str) -> int:
        # Instead of computing the minimum penalty, we can compute the maximum profit.
        ans = 0
        profit = 0
        maxProfit = 0

        for i, customer in enumerate(customers):
            profit += 1 if customer == 'Y' else -1
            if profit > maxProfit:
                maxProfit = profit
                ans = i + 1

        return ans

```

1436. 2484.py

Path: LeetCode-main/solutions/2484. Count Palindromic Subsequences/2484.py

```

class Solution:
    def countPalindromes(self, s: str) -> int:
        MOD = 1_000_000_007
        ans = 0

        for a in range(10):
            for b in range(10):
                pattern = f'{a}{b}.{b}{a}'
                # dp[i] := the number of subsequences of pattern[i..n] in s, where
                # pattern[2] can be any character
                dp = [0] * 5 + [1]
                for c in s:
                    for i, p in enumerate(pattern):
                        if p == '.' or p == c:
                            dp[i] += dp[i + 1]
                    ans += dp[0]
                ans %= MOD

        return ans

```

1437. 2485.py

Path: LeetCode-main/solutions/2485. Find the Pivot Integer/2485.py

```
class Solution:
    def pivotInteger(self, n: int) -> int:
        # 1 + 2 + ... + x = x + ... + n
        # (1 + x) * x // 2 = (x + n) * (n - x + 1) // 2
        # x + x^2 = nx - x^2 + x + n^2 - nx + n
        # 2 * x^2 = n^2 + n
        # x = sqrt((n^2 + n) // 2)
        y = (n * n + n) // 2
        x = math.isqrt(y)
        return x if x * x == y else -1
```

1438. 2486.py

Path: LeetCode-main/solutions/2486. Append Characters to String to Make Subsequence/2486.py

```
class Solution:
    def appendCharacters(self, s: str, t: str) -> int:
        i = 0 # t's index

        for c in s:
            if c == t[i]:
                i += 1
            if i == len(t):
                return 0

        return len(t) - i
```

1439. 2487.py

Path: LeetCode-main/solutions/2487. Remove Nodes From Linked List/2487.py

```
class Solution:
    def removeNodes(self, head: ListNode | None) -> ListNode | None:
        if not head:
            return None
        head.next = self.removeNodes(head.next)
        return head.next if head.next and head.val < head.next.val else head
```

1440. 2488.py

Path: LeetCode-main/solutions/2488. Count Subarrays With Median K/2488.py

```

class Solution:
    def countSubarrays(self, nums: list[int], k: int) -> int:
        INDEX = nums.index(k)
        ans = 0
        count = collections.Counter()

        balance = 0
        for i in range(INDEX, -1, -1):
            if nums[i] < k:
                balance -= 1
            elif nums[i] > k:
                balance += 1
            count[balance] += 1

        balance = 0
        for i in range(INDEX, len(nums)):
            if nums[i] < k:
                balance -= 1
            elif nums[i] > k:
                balance += 1
            # The subarray that has balance == 0 or 1 having median equal to k.
            # So, add count[0 - balance] and count[1 - balance] to `ans`.
            ans += count[-balance] + count[1 - balance]

        return ans

```

1441. 2489.py

Path: LeetCode-main/solutions/2489. Number of Substrings With Fixed Ratio/2489.py

```

class Solution:
    def fixedRatio(self, s: str, num1: int, num2: int) -> int:
        # Let x := the number of 0s and y := the number of 1s in the subarray.
        # We want x : y = num1 : num2, so our goal is to find number of subarrays
        # with x * num2 - y * num1 = 0. To achieve this, we can use a prefix count
        # map to record the count of the running x * num2 - y * num1. If the
        # running x * num2 - y * num1 = prefix, then add count[prefix] to the
        # `ans`.
        ans = 0
        prefix = 0
        prefixCount = collections.Counter({0: 1})

        for c in s:
            if c == '0':
                prefix += num2
            else: # c == '1'
                prefix -= num1
            ans += prefixCount[prefix]
            prefixCount[prefix] += 1

        return ans

```

1442. 249.py

Path: LeetCode-main/solutions/249. Group Shifted Strings/249.py


```

class Solution:
    def groupStrings(self, strings: list[str]) -> list[list[str]]:
        keyToStrings = collections.defaultdict(list)

        def getKey(s: str) -> str:
            """
            Returns the key of 's' by pairwise calculation of differences.
            e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
            """
            diffs = []

            for i in range(1, len(s)):
                diff = (ord(s[i]) - ord(s[i - 1]) + 26) % 26
                diffs.append(str(diff))

            return ','.join(diffs)

        for s in strings:
            keyToStrings[getKey(s)].append(s)

        return keyToStrings.values()

```

1443. 2490.py

Path: LeetCode-main/solutions/2490. Circular Sentence/2490.py

```

class Solution:
    def isCircularSentence(self, sentence: str) -> bool:
        for i, c in enumerate(sentence):
            if c == ' ' and sentence[i - 1] != sentence[i + 1]:
                return False
        return sentence[0] == sentence[-1]

```

1444. 2491.py

Path: LeetCode-main/solutions/2491. Divide Players Into Teams of Equal Skill/2491.py

```

class Solution:
    def dividePlayers(self, skill: list[int]) -> int:
        n = len(skill)
        teamSkill = sum(skill) // (n // 2)
        ans = 0
        count = collections.Counter(skill)

        for s, freq in count.items():
            requiredSkill = teamSkill - s
            if count[requiredSkill] != freq:
                return -1
            ans += s * requiredSkill * freq

        return ans // 2

```

1445. 2492.py

Path: LeetCode-main/solutions/2492. Minimum Score of a Path Between Two Cities/2492.py

```

class Solution:
    def minScore(self, n: int, roads: list[list[int]]) -> int:
        ans = math.inf
        graph = [[] for _ in range(n + 1)] # graph[u] := [(v, distance)]
        q = collections.deque([1])
        seen = {1}

        for u, v, distance in roads:
            graph[u].append((v, distance))
            graph[v].append((u, distance))

        while q:
            u = q.popleft()
            for v, d in graph[u]:
                ans = min(ans, d)
                if v in seen:
                    continue
                q.append(v)
                seen.add(v)

        return ans

```

1446. 2495.py

Path: LeetCode-main/solutions/2495. Number of Subarrays Having Even Product/2495.py

```

class Solution:
    def evenProduct(self, nums: list[int]) -> int:
        ans = 0
        numsBeforeEven = 0 # inclusively

        # e.g. nums = [1, 0, 1, 1, 0].
        # After meeting the first 0, set `numsBeforeEven` to 2. So, the number
        # between index 1 to index 3 (the one before next 0) will contribute 2 to
        # `ans`.
        for i, num in enumerate(nums):
            if num % 2 == 0:
                numsBeforeEven = i + 1
                ans += numsBeforeEven

        return ans

```

1447. 2496.py

Path: LeetCode-main/solutions/2496. Maximum Value of a String in an Array/2496.py

```

class Solution:
    def maximumValue(self, strs: list[str]) -> int:
        return max(len(s) if any(c.isalpha() for c in s) else int(s)
                    for s in strs)

```

1448. 2497.py

Path: LeetCode-main/solutions/2497. Maximum Star Sum of a Graph/2497.py

```

class Solution:
    def maxStarSum(self, vals: list[int], edges: list[list[int]], k: int) -> int:
        n = len(vals)
        ans = -math.inf
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append((v, vals[v]))
            graph[v].append((u, vals[u]))

        for i, starSum in enumerate(vals):
            maxHeap = []
            for _, val in graph[i]:
                if val > 0:
                    heapq.heappush(maxHeap, -val)
            j = 0
            while j < k and maxHeap:
                starSum -= heapq.heappop(maxHeap)
                j += 1
            ans = max(ans, starSum)

        return ans

```

1449. 2498.py

Path: LeetCode-main/solutions/2498. Frog Jump II/2498.py

```

class Solution:
    def maxJump(self, stones: list[int]) -> int:
        # Let's denote the forwarding path as F and the backwarding path as B.
        # 'F1 B2 B1 F2' is no better than 'F1 B2 F2 B1' since the distance between
        # F1 and F2 increase, resulting a larger `ans`.
        if len(stones) == 2:
            return stones[1] - stones[0]
        return max(stones[i] - stones[i - 2]
                    for i in range(2, len(stones)))

```

1450. 2499.py

Path: LeetCode-main/solutions/2499. Minimum Total Cost to Make Arrays Unequal/2499.py

```

class Solution:
    def minimumTotalCost(self, nums1: list[int], nums2: list[int]) -> int:
        n = len(nums1)
        ans = 0
        maxFreq = 0
        maxFreqNum = 0
        shouldBeSwapped = 0
        conflictedNumCount = [0] * (n + 1)

        # Collect the indices i s.t. num1 == num2 and record their `maxFreq`
        # and `maxFreqNum`.
        for i, (num1, num2) in enumerate(zip(nums1, nums2)):
            if num1 == num2:
                conflictedNum = num1
                conflictedNumCount[conflictedNum] += 1
                if conflictedNumCount[conflictedNum] > maxFreq:
                    maxFreq = conflictedNumCount[conflictedNum]
                    maxFreqNum = conflictedNum
                shouldBeSwapped += 1
                ans += i

        # Collect the indices with num1 != num2 that contribute less cost.
        # This can be greedily achieved by iterating from 0 to n - 1.
        for i, (num1, num2) in enumerate(zip(nums1, nums2)):
            # Since we have over `maxFreq * 2` spaces, `maxFreqNum` can be
            # successfully distributed, so no need to collect extra spaces.
            if maxFreq * 2 <= shouldBeSwapped:
                break
            if num1 == num2:
                continue
            # The numbers == `maxFreqNum` worsen the result since they increase the
            # `maxFreq`.
            if num1 == maxFreqNum or num2 == maxFreqNum:
                continue
            shouldBeSwapped += 1
            ans += i

        return -1 if maxFreq * 2 > shouldBeSwapped else ans

```

1451. 25-2.py

Path: LeetCode-main/solutions/25. Reverse Nodes in k-Group/25-2.py

```
class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        if not head or k == 1:
            return head

        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next
            return length

        length = getLength(head)
        dummy = ListNode(0, head)
        prev = dummy
        curr = head

        for _ in range(length // k):
            for _ in range(k - 1):
                next = curr.next
                curr.next = next.next
                next.next = prev.next
                prev.next = next
            prev = curr
            curr = curr.next

        return dummy.next
```

1452. 25.py

Path: LeetCode-main/solutions/25. Reverse Nodes in k-Group/25.py

```

class Solution:
    def reverseKGroup(self, head: ListNode | None, k: int) -> ListNode | None:
        if not head:
            return None

        tail = head

        for _ in range(k):
            # There are less than k nodes in the list, do nothing.
            if not tail:
                return head
            tail = tail.next

        newHead = self._reverse(head, tail)
        head.next = self.reverseKGroup(tail, k)
        return newHead

    def _reverse(
        self,
        head: ListNode | None,
        tail: ListNode | None,
    ) -> ListNode | None:
        """Reverses [head, tail)."""
        prev = None
        curr = head
        while curr != tail:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next
        return prev

```

1453. 250.py

Path: LeetCode-main/solutions/250. Count Univalue Subtrees/250.py

```

class Solution:
    def countUnivalSubtrees(self, root: TreeNode | None) -> int:
        ans = 0

        def isUnival(root: TreeNode | None, val: int) -> bool:
            nonlocal ans
            if not root:
                return True

            if isUnival(root.left, root.val) & isUnival(root.right, root.val):
                ans += 1
                return root.val == val

            return False

        isUnival(root, math.inf)
        return ans

```

1454. 2500.py

Path: LeetCode-main/solutions/2500. Delete Greatest Value in Each Row/2500.py

```
class Solution:
    def deleteGreatestValue(self, grid: list[list[int]]) -> int:
        for row in grid:
            row.sort()
        return sum(max(col) for col in zip(*grid))
```

1455. 2502.py

Path: LeetCode-main/solutions/2502. Design Memory Allocator/2502.py

```
class Allocator:
    def __init__(self, n: int):
        self.memory = [0] * n
        self.mIDToIndices = [[] for _ in range(1001)]

    def allocate(self, size: int, mID: int) -> int:
        consecutiveFree = 0
        for i, m in enumerate(self.memory):
            consecutiveFree = consecutiveFree + 1 if m == 0 else 0
            if consecutiveFree == size:
                for j in range(i - consecutiveFree + 1, i + 1):
                    self.memory[j] = mID
                    self.mIDToIndices[mID].append(j)
                return i - consecutiveFree + 1
        return -1

    def free(self, mID: int) -> int:
        indices = self.mIDToIndices[mID]
        freedUnits = len(indices)
        for index in indices:
            self.memory[index] = 0
        indices.clear()
        return freedUnits
```

1456. 2503.py

Path: LeetCode-main/solutions/2503. Maximum Number of Points From Grid Queries/2503.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class IndexedQuery:
    queryIndex: int
    query: int

    def __iter__(self):
        yield self.queryIndex
        yield self.query

class Solution:
    def maxPoints(self, grid: list[list[int]], queries: list[int]) -> list[int]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        ans = [0] * len(queries)
        minHeap = [(grid[0][0], 0, 0)] # (grid[i][j], i, j)
        seen = {(0, 0)}
        accumulate = 0

        for queryIndex, query in sorted([IndexedQuery(i, query)
                                         for i, query in enumerate(queries)],
                                         key=lambda x: x.query):
            while minHeap:
                val, i, j = heapq.heappop(minHeap)
                if val >= query:
                    # The smallest neighbor is still larger than `query`, so no need to
                    # keep exploring. Re-push (i, j, grid[i][j]) back to the `minHeap`.
                    heapq.heappush(minHeap, (val, i, j))
                    break
                accumulate += 1
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if (x, y) in seen:
                        continue
                    heapq.heappush(minHeap, (grid[x][y], x, y))
                    seen.add((x, y))
                ans[queryIndex] = accumulate

        return ans

```

1457. 2505.py

Path: LeetCode-main/solutions/2505. Bitwise OR of All Subsequence Sums/2505.py

```

class Solution:
    def subsequenceSumOr(self, nums: list[int]) -> int:
        ans = 0
        prefix = 0

        for num in nums:
            prefix += num
            ans |= num | prefix

        return ans

```

1458. 2506-2.py

Path: LeetCode-main/solutions/2506. Count Pairs Of Similar Strings/2506-2.py

```
class Solution:
    def similarPairs(self, words: list[str]) -> int:
        return sum(set(words[i]) == set(words[j])
                    for i, j in itertools.combinations(range(len(words)), 2))
```

1459. 2506.py

Path: LeetCode-main/solutions/2506. Count Pairs Of Similar Strings/2506.py

```
class Solution:
    def similarPairs(self, words: list[str]) -> int:
        masks = [self._getMask(word) for word in words]
        return sum(masks[i] == masks[j]
                    for i, j in itertools.combinations(range(len(masks)), 2))

    def _getMask(self, word: str) -> int:
        mask = 0
        for c in word:
            mask |= 1 << ord(c) - ord('a')
        return mask
```

1460. 2507.py

Path: LeetCode-main/solutions/2507. Smallest Value After Replacing With Sum of Prime Factors/2507.py

```
class Solution:
    def smallestValue(self, n: int) -> int:
        def getPrimeSum(n: int) -> int:
            primeSum = 0
            for i in range(2, n + 1):
                while n % i == 0:
                    n //= i
                    primeSum += i
            return primeSum

        primeSum = getPrimeSum(n)
        while n != primeSum:
            n = primeSum
            primeSum = getPrimeSum(n)
        return n
```

1461. 2508.py

Path: LeetCode-main/solutions/2508. Add Edges to Make Degrees of All Nodes Even/2508.py

```

class Solution:
    def isPossible(self, n: int, edges: list[list[int]]) -> bool:
        graph = [set() for _ in range(n)]

        for u, v in edges:
            graph[u - 1].add(v - 1)
            graph[v - 1].add(u - 1)

        oddNodes = [i for i, neighbor in enumerate(
            graph) if len(neighbor) % 2 == 1]
        if not oddNodes:
            return True
        if len(oddNodes) == 2:
            a, b = oddNodes
            return any(a not in graph[i] and b not in graph[i] for i in range(n))
        if len(oddNodes) == 4:
            a, b, c, d = oddNodes
            return ((b not in graph[a] and d not in graph[c]) or
                    (c not in graph[a] and d not in graph[b]) or
                    (d not in graph[a] and c not in graph[b]))
        return False

```

1462. 2509.py

Path: LeetCode-main/solutions/2509. Cycle Length Queries in a Tree/2509.py

```

class Solution:
    def cycleLengthQueries(self, n: int, queries: list[list[int]]) -> list[int]:
        def getCycleLength(a: int, b: int):
            cycleLength = 1
            while a != b:
                if a > b:
                    a //= 2
                else:
                    b //= 2
                cycleLength += 1
            return cycleLength

        return [getCycleLength(*query) for query in queries]

```

1463. 251.py

Path: LeetCode-main/solutions/251. Flatten 2D Vector/251.py

```

class Vector2D:
    def __init__(self, vec: list[list[int]]):
        self.vec = []
        self.i = 0

        for arr in vec:
            self.vec += arr

    def next(self) -> int:
        ans = self.vec[self.i]
        self.i += 1
        return ans

    def hasNext(self) -> bool:
        return self.i < len(self.vec)

```

1464. 2510.py

Path: LeetCode-main/solutions/2510. Check if There is a Path With Equal Number of 0's And 1's/2510.py

```
class Solution:
    def isThereAPath(self, grid: list[list[int]]) -> bool:
        m = len(grid)
        n = len(grid[0])
        if m + n - 1 & 1:
            return False

        @functools.lru_cache(None)
        def dp(i: int, j: int, summ: int) -> bool:
            """
            Returns 1 if there's a path to grid[i][j] s.t.
            `summ` = (the number of 0s - the number of 1s).
            """
            if i == m or j == n:
                return False
            summ += 1 if grid[i][j] == 0 else -1
            if i == m - 1 and j == n - 1:
                return summ == 0
            return dp(i + 1, j, summ) or dp(i, j + 1, summ)

        return dp(0, 0, 0)
```

1465. 2511.py

Path: LeetCode-main/solutions/2511. Maximum Enemy Forts That Can Be Captured/2511.py

```
class Solution:
    def captureForts(self, forts: list[int]) -> int:
        ans = 0

        j = 0
        for i, fort in enumerate(forts):
            if fort != 0: # -1 or 1
                if fort == -forts[j]:
                    ans = max(ans, i - j - 1)
                    j = i

        return ans
```

1466. 2512.py

Path: LeetCode-main/solutions/2512. Reward Top K Students/2512.py

```

class Solution:
    def topStudents(
        self,
        positive_feedback: list[str],
        negative_feedback: list[str],
        report: list[str],
        student_id: list[int],
        k: int,
    ) -> list[int]:
        scoreAndIds = []
        pos = set(positive_feedback)
        neg = set(negative_feedback)

        for sid, r in zip(student_id, report):
            score = 0
            for word in r.split():
                if word in pos:
                    score += 3
                if word in neg:
                    score -= 1
            scoreAndIds.append((-score, sid))

        return [sid for _, sid in sorted(scoreAndIds)[:k]]

```

1467. 2513.py

Path: LeetCode-main/solutions/2513. Minimize the Maximum of Two Arrays/2513.py

```

class Solution:
    def minimizeSet(
        self,
        divisor1: int,
        divisor2: int,
        uniqueCnt1: int,
        uniqueCnt2: int,
    ) -> int:
        divisorLcm = math.lcm(divisor1, divisor2)
        l = 0
        r = 2**31 - 1

        def isPossible(m: int) -> bool:
            """
            Returns True if we can take uniqueCnt1 integers from [1..m] to arr1 and
            take uniqueCnt2 integers from [1..m] to arr2.
            """
            cnt1 = m - m // divisor1
            cnt2 = m - m // divisor2
            totalCnt = m - m // divisorLcm
            return (cnt1 >= uniqueCnt1 and
                    cnt2 >= uniqueCnt2 and
                    totalCnt >= uniqueCnt1 + uniqueCnt2)

        while l < r:
            m = (l + r) // 2
            if isPossible(m):
                r = m
            else:
                l = m + 1

        return l

```

1468. 2514.py

Path: LeetCode-main/solutions/2514. Count Anagrams/2514.py

```
class Solution:
    def countAnagrams(self, s: str) -> int:
        ans = 1

        for word in s.split():
            ans = ans * math.factorial(len(word))
            count = collections.Counter(word)
            for freq in count.values():
                ans //= math.factorial(freq)

        return ans % 1_000_000_007
```

1469. 2515.py

Path: LeetCode-main/solutions/2515. Shortest Distance to Target String in a Circular Array/2515.py

```
class Solution:
    def closetTarget(self, words: list[str], target: str, startIndex: int) -> int:
        n = len(words)

        for i in range(n):
            if words[(startIndex + i + n) % n] == target:
                return i
            if words[(startIndex - i + n) % n] == target:
                return i

        return -1
```

1470. 2516.py

Path: LeetCode-main/solutions/2516. Take K of Each Character From Left and Right/2516.py

```
class Solution:
    def takeCharacters(self, s: str, k: int) -> int:
        n = len(s)
        ans = n
        count = collections.Counter(s)
        if any(count[c] < k for c in 'abc'):
            return -1

        l = 0
        for r, c in enumerate(s):
            count[c] -= 1
            while count[c] < k:
                count[s[l]] += 1
                l += 1
            ans = min(ans, n - (r - l + 1))

        return ans
```

1471. 2517.py

Path: LeetCode-main/solutions/2517. Maximum Tastiness of Candy Basket/2517.py

```

class Solution:
    def maximumTastiness(self, price: list[int], k: int) -> int:
        price.sort()

        def numBaskets(m: int) -> int:
            """Returns the number of baskets we can pick for m tastiness."""
            baskets = 0
            prevPrice = -m
            for p in price:
                if p >= prevPrice + m:
                    prevPrice = p
                    baskets += 1
            return baskets

        l = bisect.bisect_left(range(max(price) - min(price) + 1), True,
                               key=lambda m: numBaskets(m) < k)

        return l - 1

```

1472. 2518.py

Path: LeetCode-main/solutions/2518. Number of Great Partitions/2518.py

```

class Solution:
    def countPartitions(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        summ = sum(nums)
        ans = pow(2, len(nums), MOD) # 2^n % MOD
        dp = [1] + [0] * k

        for num in nums:
            for i in range(k, num - 1, -1):
                dp[i] += dp[i - num]
                dp[i] %= MOD

        # Subtract the cases that're not satisfied.
        for i in range(k):
            if summ - i < k: # Both group1 and group2 < k.
                ans -= dp[i]
            else:
                ans -= dp[i] * 2

        return ans % MOD

```

1473. 2519.py

Path: LeetCode-main/solutions/2519. Count the Number of K-Big Indices/2519.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    def kBigIndices(self, nums: list[int], k: int) -> int:
        n = len(nums)
        leftTree = FenwickTree(n)
        rightTree = FenwickTree(n)
        # left[i] := the number of `nums` < nums[i] with index < i
        left = [0] * n
        # right[i] := the number of `nums` < nums[i] with index > i
        right = [0] * n

        for i, num in enumerate(nums):
            left[i] = leftTree.get(num - 1)
            leftTree.add(num, 1)

        for i in range(n - 1, -1, -1):
            right[i] = rightTree.get(nums[i] - 1)
            rightTree.add(nums[i], 1)

        return sum(1 >= k and r >= k for l, r in zip(left, right))

```

1474. 252.py

Path: LeetCode-main/solutions/252. Meeting Rooms/252.py

```

class Solution:
    def canAttendMeetings(self, intervals: list[list[int]]) -> bool:
        intervals.sort()

        for i in range(1, len(intervals)):
            if intervals[i - 1][1] > intervals[i][0]:
                return False

        return True

```

1475. 2520.py

Path: LeetCode-main/solutions/2520. Count the Digits That Divide a Number/2520.py

```
class Solution:
    def countDigits(self, num: int) -> int:
        return sum(num % int(d) == 0 for d in str(num))
```

1476. 2521.py

Path: LeetCode-main/solutions/2521. Distinct Prime Factors of Product of Array/2521.py

```
class Solution:
    def distinctPrimeFactors(self, nums: list[int]) -> int:
        primes = set()

        for num in nums:
            self._addPrimeFactors(primes, num)

        return len(primes)

    def _addPrimeFactors(self, primes: set[int], num: int) -> None:
        for divisor in range(2, num + 1):
            if num % divisor == 0:
                primes.add(divisor)
                while num % divisor == 0:
                    num //= divisor
```

1477. 2522.py

Path: LeetCode-main/solutions/2522. Partition String Into Substrings With Values at Most K/2522.py

```
class Solution:
    def minimumPartition(self, s: str, k: int) -> int:
        ans = 1
        curr = 0

        for c in s:
            curr = curr * 10 + int(c)
            if curr > k:
                curr = int(c)
                ans += 1
            if curr > k:
                return -1

        return ans
```

1478. 2523.py

Path: LeetCode-main/solutions/2523. Closest Prime Numbers in Range/2523.py


```
class Solution:
    def closestPrimes(self, left: int, right: int) -> list[int]:
        isPrime = self._sieveEratosthenes(right + 1)
        primes = [i for i in range(left, right + 1) if isPrime[i]]

        if len(primes) < 2:
            return [-1, -1]

        minDiff = math.inf
        num1 = -1
        num2 = -1

        for a, b in zip(primes, primes[1:]):
            diff = b - a
            if diff < minDiff:
                minDiff = diff
                num1 = a
                num2 = b

        return [num1, num2]

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime
```

1479. 2524.py

Path: LeetCode-main/solutions/2524. Maximum Frequency Score of a Subarray/2524.py

```

class Solution:
    def maxFrequencyScore(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        count = collections.Counter(nums[:k])
        summ = self._getInitialSumm(count, MOD)
        ans = summ

        for i in range(k, len(nums)):
            # Remove the leftmost number that's out-of-window.
            leftNum = nums[i - k]
            summ = (summ - pow(leftNum, count[leftNum], MOD) + MOD) % MOD
            # After decreasing its frequency, if it's still > 0, then add it back.
            count[leftNum] -= 1
            if count[leftNum] > 0:
                summ = (summ + pow(leftNum, count[leftNum], MOD)) % MOD
            # Otherwise, remove it from the count map.
            else:
                del count[leftNum]
            # Add the current number. Similarly, remove the current score like above.
            rightNum = nums[i]
            if count[rightNum] > 0:
                summ = (summ - pow(rightNum, count[rightNum], MOD) + MOD) % MOD
            count[rightNum] += 1
            summ = (summ + pow(rightNum, count[rightNum], MOD)) % MOD
            ans = max(ans, summ)

        return ans

    def _getInitialSumm(self, count: dict[int, int], MOD: int) -> int:
        summ = 0
        for num, freq in count.items():
            summ = (summ + pow(num, freq, MOD)) % MOD
        return summ

```

1480. 2525.py

Path: LeetCode-main/solutions/2525. Categorize Box According to Criteria/2525.py

```

class Solution:
    def categorizeBox(
        self,
        length: int,
        width: int,
        height: int,
        mass: int,
    ) -> str:
        isBulky = (length >= 10000 or
                   width >= 10000 or height >= 10000 or
                   length * width * height >= 1_000_000_000)
        isHeavy = mass >= 100
        if isBulky and isHeavy:
            return 'Both'
        if isBulky:
            return 'Bulky'
        if isHeavy:
            return 'Heavy'
        return 'Neither'

```

1481. 2526.py

Path: LeetCode-main/solutions/2526. Find Consecutive Integers from a Data Stream/2526.py

```
class DataStream:
    def __init__(self, value: int, k: int):
        self.value = value
        self.k = k
        self.q = deque()
        self.count = 0

    def consec(self, num: int) -> bool:
        if len(self.q) == self.k and self.q.popleft() == self.value:
            self.count -= 1
        if num == self.value:
            self.count += 1
        self.q.append(num)
        return self.count == self.k
```

1482. 2527.py

Path: LeetCode-main/solutions/2527. Find Xor-Beauty of Array/2527.py

```
class Solution:
    def xorBeauty(self, nums: list[int]) -> int:
        return functools.reduce(operator.xor, nums)
```

1483. 2528.py

Path: LeetCode-main/solutions/2528. Maximize the Minimum Powered City/2528.py

```

class Solution:
    def maxPower(self, stations: list[int], r: int, k: int) -> int:
        n = len(stations)
        left = min(stations)
        right = sum(stations) + k + 1

        def check(
            stations: list[int],
            additionalStations: int, minPower: int) -> bool:
            """Returns True if each city can have at least `minPower`."""
            # Initilaize `power` as the 0-th city's power - stations[r].
            power = sum(stations[:r])

            for i in range(n):
                if i + r < n:
                    power += stations[i + r] # `power` = sum(stations[i - r..i + r]).
                if power < minPower:
                    requiredPower = minPower - power
                    # There're not enough stations to plant.
                    if requiredPower > additionalStations:
                        return False
                    # Greedily plant `requiredPower` power stations in the farthest place
                    # to cover as many cities as possible.
                    stations[min(n - 1, i + r)] += requiredPower
                    additionalStations -= requiredPower
                    power += requiredPower
                if i - r >= 0:
                    power -= stations[i - r]

            return True

        while left < right:
            mid = (left + right) // 2
            if check(stations.copy(), k, mid):
                left = mid + 1
            else:
                right = mid

        return left - 1

```

1484. 2529.py

Path: LeetCode-main/solutions/2529. Maximum Count of Positive Integer and Negative Integer/2529.py

```

class Solution:
    def maximumCount(self, nums: list[int]) -> int:
        return max(sum(num > 0 for num in nums), sum(num < 0 for num in nums))

```

1485. 253-2.py

Path: LeetCode-main/solutions/253. Meeting Rooms II/253-2.py

```

class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        n = len(intervals)
        ans = 0
        starts = []
        ends = []

        for start, end in intervals:
            starts.append(start)
            ends.append(end)

        starts.sort()
        ends.sort()

        j = 0
        for i in range(n):
            if starts[i] < ends[j]:
                ans += 1
            else:
                j += 1

        return ans

```

1486. 253.py

Path: LeetCode-main/solutions/253. Meeting Rooms II/253.py

```

class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        minHeap = [] # Store the end times of each room.

        for start, end in sorted(intervals):
            # There's no overlap, so we can reuse the same room.
            if minHeap and start >= minHeap[0]:
                heapq.heappop(minHeap)
            heapq.heappush(minHeap, end)

        return len(minHeap)

```

1487. 2530.py

Path: LeetCode-main/solutions/2530. Maximal Score After Applying K Operations/2530.py

```

class Solution:
    def maxKelements(self, nums: list[int], k: int) -> int:
        ans = 0
        maxHeap = [-num for num in nums]
        heapq.heapify(maxHeap)

        for _ in range(k):
            num = -heapq.heappop(maxHeap)
            ans += num
            heapq.heappush(maxHeap, -math.ceil(num / 3))

        return ans

```

1488. 2531.py

Path: LeetCode-main/solutions/2531. Make Number of Distinct Characters Equal/2531.py

```
class Solution:
    def isItPossible(self, word1: str, word2: str) -> bool:
        count1 = collections.Counter(word1)
        count2 = collections.Counter(word2)
        distinct1 = len(count1)
        distinct2 = len(count2)

        for a in count1:
            for b in count2:
                if a == b:
                    # Swapping the same letters won't change the number of distinct
                    # letters in each string, so just check if `distinct1 == distinct2`.
                    if distinct1 == distinct2:
                        return True
                    continue
                # The calculation is meaningful only when a != b
                # Swap a in word1 with b in word2.
                distinctAfterSwap1 = distinct1 - (count1[a] == 1) + (count1[b] == 0)
                distinctAfterSwap2 = distinct2 - (count2[b] == 1) + (count2[a] == 0)
                if distinctAfterSwap1 == distinctAfterSwap2:
                    return True

        return False
```

1489. 2532.py

Path: LeetCode-main/solutions/2532. Time to Cross a Bridge/2532.py

```

class Solution:
    def findCrossingTime(self, n: int, k: int, time: list[list[int]]) -> int:
        ans = 0
        # (leftToRight + rightToLeft, i)
        leftBridgeQueue = [
            (-leftToRight - rightToLeft, -i) for i,
            (leftToRight, pickOld, rightToLeft, pickNew) in enumerate(time)]
        rightBridgeQueue = []
        # (time to be idle, i)
        leftWorkers = []
        rightWorkers = []

        heapq.heapify(leftBridgeQueue)

        while n > 0 or rightBridgeQueue or rightWorkers:
            # Idle left workers get on the left bridge.
            while leftWorkers and leftWorkers[0][0] <= ans:
                i = heapq.heappop(leftWorkers)[1]
                leftWorkers.pop()
                heapq.heappush(leftBridgeQueue, (-time[i][0] - time[i][2], -i))
            # Idle right workers get on the right bridge.
            while rightWorkers and rightWorkers[0][0] <= ans:
                i = heapq.heappop(rightWorkers)[1]
                heapq.heappush(rightBridgeQueue, (-time[i][0] - time[i][2], -i))
            if rightBridgeQueue:
                # If the bridge is free, the worker waiting on the right side of the
                # bridge gets to cross the bridge. If more than one worker is waiting
                # on the right side, the one with the lowest efficiency crosses first.
                i = -heapq.heappop(rightBridgeQueue)[1]
                ans += time[i][2]
                heapq.heappush(leftWorkers, (ans + time[i][3], i))
            elif leftBridgeQueue and n > 0:
                # If the bridge is free and no worker is waiting on the right side, and
                # at least one box remains at the old warehouse, the worker on the left
                # side of the river gets to cross the bridge. If more than one worker
                # is waiting on the left side, the one with the lowest efficiency
                # crosses first.
                i = -heapq.heappop(leftBridgeQueue)[1]
                ans += time[i][0]
                heapq.heappush(rightWorkers, (ans + time[i][1], i))
                n -= 1
            else:
                # Advance the time of the last crossing worker.
                ans = min(leftWorkers[0][0] if leftWorkers and n > 0 else math.inf,
                        rightWorkers[0][0] if rightWorkers else math.inf)

        return ans

```

1490. 2533.py

Path: LeetCode-main/solutions/2533. Number of Good Binary Strings/2533.py

```

class Solution:
    def goodBinaryStrings(
        self,
        minLength: int,
        maxLength: int,
        oneGroup: int,
        zeroGroup: int,
    ) -> int:
        MOD = 1_000_000_007

        # dp[i] := the number of good binary strings with length i
        dp = [1] + [0] * maxLength

        for i in range(maxLength + 1):
            # There are good binary strings with length i, so we can append
            # consecutive 0s or 1s after it.
            if dp[i] > 0:
                appendZeros = i + zeroGroup
                if appendZeros <= maxLength:
                    dp[appendZeros] += dp[i]
                    dp[appendZeros] %= MOD
                appendOnes = i + oneGroup
                if appendOnes <= maxLength:
                    dp[appendOnes] += dp[i]
                    dp[appendOnes] %= MOD

        return sum(dp[minLength:]) % MOD

```

1491. 2535.py

Path: LeetCode-main/solutions/2535. Difference Between Element Sum and Digit Sum of an Array/2535.py

```

class Solution:
    def differenceOfSum(self, nums: list[int]) -> int:
        elementSum = sum(nums)
        digitSum = self._getAllDigitSum(nums)
        return abs(elementSum - digitSum)

    def _getAllDigitSum(self, nums: list[int]) -> int:
        return sum(self._getDigitSum(num) for num in nums)

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))

```

1492. 2538.py

Path: LeetCode-main/solutions/2538. Difference Between Maximum and Minimum Price Sum/2538.py


```

class Solution:
    def maxOutput(self, n: int, edges: list[list[int]], price: list[int]) -> int:
        ans = 0
        tree = [[] for _ in range(n)]
        maxSums = [0] * n # maxSums[i] := the maximum the sum of path rooted at i

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def maxSum(u: int, prev: int) -> int:
            maxChildSum = 0
            for v in tree[u]:
                if v != prev:
                    maxChildSum = max(maxChildSum, maxSum(v, u))
            maxSums[u] = price[u] + maxChildSum
            return maxSums[u]

        # Precalculate `maxSums`.
        maxSum(0, -1)

        def reroot(u: int, prev: int, parentSum: int) -> None:
            nonlocal ans
            # Get the top two subtree sums and the top one node index.
            maxSubtreeSum1 = 0
            maxSubtreeSum2 = 0
            maxNode = -1
            for v in tree[u]:
                if v == prev:
                    continue
                if maxSums[v] > maxSubtreeSum1:
                    maxSubtreeSum2 = maxSubtreeSum1
                    maxSubtreeSum1 = maxSums[v]
                    maxNode = v
                elif maxSums[v] > maxSubtreeSum2:
                    maxSubtreeSum2 = maxSums[v]

            if len(tree[u]) == 1:
                ans = max(ans, parentSum, maxSubtreeSum1)

            for v in tree[u]:
                if v == prev:
                    continue
                nextParentSum = (
                    price[u] + max(parentSum, maxSubtreeSum2) if v == maxNode else
                    price[u] + max(parentSum, maxSubtreeSum1))
                reroot(v, u, nextParentSum)

        reroot(0, -1, 0)
        return ans

```

1493. 2539.py

Path: LeetCode-main/solutions/2539. Count the Number of Good Subsequences/2539.py

```

class Solution:
    def countGoodSubsequences(self, s: str) -> int:
        MOD = 1_000_000_007
        ans = 0
        count = collections.Counter(s)

        @functools.lru_cache(None)
        def fact(i: int) -> int:
            return 1 if i <= 1 else i * fact(i - 1) % MOD

        @functools.lru_cache(None)
        def inv(i: int) -> int:
            return pow(i, MOD - 2, MOD)

        @functools.lru_cache(None)
        def nCk(n: int, k: int) -> int:
            return fact(n) * inv(fact(k)) * inv(fact(n - k)) % MOD

        for freq in range(1, max(count.values()) + 1):
            numSubseqs = 1 # ""
            for charFreq in count.values():
                if charFreq >= freq:
                    numSubseqs = numSubseqs * (1 + nCk(charFreq, freq)) % MOD
            ans += numSubseqs - 1 # Minus "".
            ans %= MOD

        return ans

```

1494. 254.py

Path: LeetCode-main/solutions/254. Factor Combinations/254.py

```

class Solution:
    def getFactors(self, n: int) -> list[list[int]]:
        ans = []

        def dfs(n: int, s: int, path: list[int]) -> None:
            if n <= 1:
                if len(path) > 1:
                    ans.append(path.copy())
                return

            for i in range(s, n + 1):
                if n % i == 0:
                    path.append(i)
                    dfs(n // i, i, path)
                    path.pop()

        dfs(n, 2, []) # The minimum factor is 2.
        return ans

```

1495. 2540.py

Path: LeetCode-main/solutions/2540. Minimum Common Value/2540.py

```
class Solution:
    def getCommon(self, nums1: list[int], nums2: list[int]) -> int:
        i = 0 # nums1's index
        j = 0 # nums2's index

        while i < len(nums1) and j < len(nums2):
            if nums1[i] == nums2[j]:
                return nums1[i]
            if nums1[i] < nums2[j]:
                i += 1
            else:
                j += 1

        return -1
```

1496. 2541.py

Path: LeetCode-main/solutions/2541. Minimum Operations to Make Array Equal II/2541.py

```
class Solution:
    def minOperations(self, nums1: list[int], nums2: list[int], k: int) -> int:
        if k == 0:
            return 0 if nums1 == nums2 else -1

        ans = 0
        opsDiff = 0 # the number of increments - number of decrements

        for num1, num2 in zip(nums1, nums2):
            diff = num1 - num2
            if diff == 0:
                continue
            if diff % k != 0:
                return -1
            ops = diff // k
            opsDiff += ops
            ans += abs(ops)

        return ans // 2 if opsDiff == 0 else -1
```

1497. 2542.py

Path: LeetCode-main/solutions/2542. Maximum Subsequence Score/2542.py

```

class Solution:
    # Same as 1383. Maximum Performance of a Team
    def maxScore(self, nums1: list[int], nums2: list[int], k: int) -> int:
        ans = 0
        summ = 0
        # (nums2[i], nums1[i]) sorted by nums2[i] in descending order
        A = sorted([(num2, num1)
                     for num1, num2 in zip(nums1, nums2)], reverse=True)

        minHeap = []

        for num2, num1 in A:
            heapq.heappush(minHeap, num1)
            summ += num1
            if len(minHeap) > k:
                summ -= heapq.heappop(minHeap)
            if len(minHeap) == k:
                ans = max(ans, summ * num2)

        return ans

```

1498. 2543.py

Path: LeetCode-main/solutions/2543. Check if Point Is Reachable/2543.py

```

class Solution:
    def isReachable(self, targetX: int, targetY: int) -> bool:
        return math.gcd(targetX, targetY).bit_count() == 1

```

1499. 2544.py

Path: LeetCode-main/solutions/2544. Alternating Digit Sum/2544.py

```

class Solution:
    def alternateDigitSum(self, n: int) -> int:
        ans = 0
        sign = 1

        while n > 0:
            sign *= -1
            ans += n % 10 * sign
            n //= 10

        return sign * ans

```

1500. 2545.py

Path: LeetCode-main/solutions/2545. Sort the Students by Their Kth Score/2545.py

```

class Solution:
    def sortTheStudents(self, score: list[list[int]], k: int) -> list[list[int]]:
        return sorted(score, key=lambda x: -x[k])

```

1501. 2546.py

Path: LeetCode-main/solutions/2546. Apply Bitwise Operations to Make Strings Equal/2546.py

```
class Solution:
    def makeStringsEqual(self, s: str, target: str) -> bool:
        return ('1' in s) == ('1' in target)
```

1502. 2547.py

Path: LeetCode-main/solutions/2547. Minimum Cost to Split an Array/2547.py

```
class Solution:
    def minCost(self, nums: list[int], k: int) -> int:
        MAX = 1001
        n = len(nums)
        # trimmedLength[i][j] := trimmed(nums[i..j]).length
        trimmedLength = [[0] * n for _ in range(n)]
        # dp[i] := the minimum cost to split nums[i..n)
        dp = [math.inf] * n + [0]

        for i in range(n):
            length = 0
            count = [0] * MAX
            for j in range(i, n):
                count[nums[j]] += 1
                if count[nums[j]] == 2:
                    length += 2
                elif count[nums[j]] > 2:
                    length += 1
                trimmedLength[i][j] = length

        dp[n] = 0

        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                dp[i] = min(dp[i], k + trimmedLength[i][j] + dp[j + 1])

        return dp[0]
```

1503. 2548.py

Path: LeetCode-main/solutions/2548. Maximum Price to Fill a Bag/2548.py

```
class Solution:
    def maxPrice(self, items: list[list[int]], capacity: int) -> float:
        ans = 0

        # Sort items based on price//weight.
        for price, weight in sorted(items, key=lambda x: -x[0] / x[1]):
            # The bag is filled.
            if capacity <= weight:
                return ans + price * capacity / weight
            ans += price
            capacity -= weight

        return -1
```

1504. 2549.py

Path: LeetCode-main/solutions/2549. Count Distinct Numbers on Board/2549.py

```
class Solution:
    def distinctIntegers(self, n: int) -> int:
        return max(n - 1, 1)
```

1505. 255-2.py

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255-2.py

```
class Solution:
    def verifyPreorder(self, preorder: list[int]) -> list[int]:
        low = -math.inf
        stack = []

        for p in preorder:
            if p < low:
                return False
            while stack and stack[-1] < p:
                low = stack.pop()
            stack.append(p)

        return True
```

1506. 255-3.py

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255-3.py

```
class Solution:
    def verifyPreorder(self, preorder: list[int]) -> bool:
        low = -math.inf
        i = -1

        for p in preorder:
            if p < low:
                return False
            while i >= 0 and preorder[i] < p:
                low = preorder[i]
                i -= 1
            i += 1
            preorder[i] = p

        return True
```

1507. 255.py

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255.py

```

class Solution:
    def verifyPreorder(self, preorder: list[int]) -> bool:
        i = 0

        def dfs(min: int, max: int) -> None:
            nonlocal i
            if i == len(preorder):
                return
            if preorder[i] < min or preorder[i] > max:
                return

            val = preorder[i]
            i += 1
            dfs(min, val)
            dfs(val, max)

        dfs(-math.inf, math.inf)
        return i == len(preorder)

```

1508. 2550.py

Path: LeetCode-main/solutions/2550. Count Collisions of Monkeys on a Polygon/2550.py

```

class Solution:
    def monkeyMove(self, n: int) -> int:
        MOD = 1_000_000_007
        res = pow(2, n, MOD) - 2
        return res + MOD if res < 0 else res

```

1509. 2551.py

Path: LeetCode-main/solutions/2551. Put Marbles in Bags/2551.py

```

class Solution:
    def putMarbles(self, weights: list[int], k: int) -> int:
        # To distribute marbles into k bags, there will be k - 1 cuts. If there's a
        # cut after weights[i], then weights[i] and weights[i + 1] will be added to
        # the cost. Also, no matter how we cut, weights[0] and weights[n - 1] will
        # be counted. So, the goal is to find the max//min k - 1 weights[i] +
        # weights[i + 1].

        # weights[i] + weights[i + 1]
        arr = [a + b for a, b in itertools.pairwise(weights)]
        return sum(heapq.nlargest(k - 1, arr)) - sum(heapq.nsmallest(k - 1, arr))

```

1510. 2552.py

Path: LeetCode-main/solutions/2552. Count Increasing Quadruplets/2552.py

```

class Solution:
    def countQuadruplets(self, nums: list[int]) -> int:
        ans = 0
        # dp[j] := the number of triplets (i, j, k) where i < j < k and nums[i] < nums[k] <
        # nums[j]. Keep this information for l to use later.
        dp = [0] * len(nums)

        # k can be treated as l.
        for k in range(2, len(nums)):
            numLessThanK = 0
            # j can be treated as i.
            for j in range(k):
                if nums[j] < nums[k]:
                    numLessThanK += 1 # nums[i] < nums[k]
                    # nums[j] < nums[l], so we should add dp[j] since we find a new
                    # quadruplets for (i, j, k, l).
                    ans += dp[j]
                elif nums[j] > nums[k]:
                    dp[j] += numLessThanK

        return ans

```

1511. 2553.py

Path: LeetCode-main/solutions/2553. Separate the Digits in an Array/2553.py

```

class Solution:
    def separateDigits(self, nums: list[int]) -> list[int]:
        return [int(c) for num in nums for c in str(num)]

```

1512. 2554.py

Path: LeetCode-main/solutions/2554. Maximum Number of Integers to Choose From a Range I/2554.py

```

class Solution:
    def maxCount(self, banned: list[int], n: int, maxSum: int) -> int:
        ans = 0
        summ = 0
        bannedSet = set(banned)

        for i in range(1, n + 1):
            if i not in bannedSet and summ + i <= maxSum:
                ans += 1
                summ += i

        return ans

```

1513. 2555.py

Path: LeetCode-main/solutions/2555. Maximize Win From Two Segments/2555.py


```

class Solution:
    def maximizeWin(self, prizePositions: list[int], k: int) -> int:
        ans = 0
        # dp[i] := the maximum number of prizes to choose the first i
        # `prizePositions`
        dp = [0] * (len(prizePositions) + 1)

        j = 0
        for i, prizePosition in enumerate(prizePositions):
            while prizePosition - prizePositions[j] > k:
                j += 1
            covered = i - j + 1
            dp[i + 1] = max(dp[i], covered)
            ans = max(ans, dp[j] + covered)

        return ans

```

1514. 2556.py

Path: LeetCode-main/solutions/2556. Disconnect Path in a Binary Matrix by at Most One Flip/2556.py

```

class Solution:
    def isPossibleToCutPath(self, grid: list[list[int]]) -> bool:
        # Returns True is there's a path from (0, 0) to (m - 1, n - 1).
        # Also marks the visited path as 0 except (m - 1, n - 1).
        def hasPath(i: int, j: int) -> bool:
            if i == len(grid) or j == len(grid[0]):
                return False
            if i == len(grid) - 1 and j == len(grid[0]) - 1:
                return True
            if grid[i][j] == 0:
                return False

            grid[i][j] = 0
            # Go down first. Since we use OR logic, we'll only mark one path.
            return hasPath(i + 1, j) or hasPath(i, j + 1)

        if not hasPath(0, 0):
            return True
        # Reassign (0, 0) as 1.
        grid[0][0] = 1
        return not hasPath(0, 0)

```

1515. 2557.py

Path: LeetCode-main/solutions/2557. Maximum Number of Integers to Choose From a Range II/2557.py

```

class Solution:
    def maxCount(self, banned: list[int], n: int, maxSum: int) -> int:
        bannedSet = set(banned)
        l = 1
        r = n

        while l < r:
            m = (l + r + 1) // 2
            if self._getSum(bannedSet, m) > maxSum:
                r = m - 1
            else:
                l = m

        return l - sum(b <= l for b in banned)

    # Returns sum([1..m]) - sum(bannedSet).
    def _getSum(self, bannedSet: set[int], m: int) -> int:
        return m * (m + 1) // 2 - sum(b for b in bannedSet if b <= m)

```

1516. 2558.py

Path: LeetCode-main/solutions/2558. Take Gifts From the Richest Pile/2558.py

```

class Solution:
    def pickGifts(self, gifts: list[int], k: int) -> int:
        maxHeap = [-gift for gift in gifts]
        heapq.heapify(maxHeap)

        for _ in range(k):
            squaredMax = math.isqrt(-heapq.heappop(maxHeap))
            heapq.heappush(maxHeap, -squaredMax)

        return -sum(maxHeap)

```

1517. 2559.py

Path: LeetCode-main/solutions/2559. Count Vowel Strings in Ranges/2559.py

```

class Solution:
    def vowelStrings(
        self,
        words: list[str],
        queries: list[list[int]],
    ) -> list[int]:
        VOWELS = 'aeiou'
        # prefix[i] := the number of the first i words that start with and end in a vowel
        prefix = [0] * (len(words) + 1)

        for i, word in enumerate(words):
            prefix[i + 1] += prefix[i] + (word[0] in VOWELS and word[-1] in VOWELS)

        return [prefix[r + 1] - prefix[l]
                for l, r in queries]

```

1518. 256.py

Path: LeetCode-main/solutions/256. Paint House/256.py

```

class Solution:
    def minCost(self, costs: list[list[int]]) -> list[list[int]]:
        for i in range(1, len(costs)):
            costs[i][0] += min(costs[i - 1][1], costs[i - 1][2])
            costs[i][1] += min(costs[i - 1][0], costs[i - 1][2])
            costs[i][2] += min(costs[i - 1][0], costs[i - 1][1])

        return min(costs[-1])

```

1519. 2560.py

Path: LeetCode-main/solutions/2560. House Robber IV/2560.py

```

class Solution:
    def minCapability(self, nums: list[int], k: int) -> int:
        def numStolenHouses(capacity: int) -> int:
            stolenHouses = 0
            i = 0
            while i < len(nums):
                if nums[i] <= capacity:
                    stolenHouses += 1
                    i += 1
                i += 1
            return stolenHouses

        return bisect.bisect_left(range(max(nums)), k, key=numStolenHouses)

```

1520. 2561.py

Path: LeetCode-main/solutions/2561. Rearranging Fruits/2561.py

```

class Solution:
    def minCost(self, basket1: list[int], basket2: list[int]) -> int:
        swapped = []
        count = collections.Counter(basket1)
        count.subtract(collections.Counter(basket2))

        for num, freq in count.items():
            if freq % 2 != 0:
                return -1
            swapped += [num] * abs(freq // 2)

        swapped.sort()
        minNum = min(min(basket1), min(basket2))
        # Other than directly swap basket1[i] and basket2[j], we can swap basket1[i]
        # with `minNum` first then swap `minNum` with basket2[j], and vice versa.
        # That's why we take min(2 * minNum, num) in the below.
        return sum(min(2 * minNum, num) for num in swapped[0:len(swapped) // 2])

```

1521. 2563.py

Path: LeetCode-main/solutions/2563. Count the Number of Fair Pairs/2563.py

```

class Solution:
    def countFairPairs(self, nums: list[int], lower: int, upper: int) -> int:
        # nums[i] + nums[j] == nums[j] + nums[i], so the condition that i < j
        # degrades to i != j and we can sort the array.
        nums.sort()

        def countLess(summ: int) -> int:
            res = 0
            i = 0
            j = len(nums) - 1
            while i < j:
                while i < j and nums[i] + nums[j] > summ:
                    j -= 1
                res += j - i
                i += 1
            return res

        return countLess(upper) - countLess(lower - 1)

```

1522. 2564.py

Path: LeetCode-main/solutions/2564. Substring XOR Queries/2564.py

```

class Solution:
    def substringXorQueries(self, s: str, queries: list[list[int]]) -> list[list[int]]:
        MAX_BIT = 30
        # {val: [left, right]} := s[left..right]'s decimal value = val
        valToLeftAndRight = collections.defaultdict(lambda: [-1, -1])

        for left, c in enumerate(s):
            val = 0
            if c == '0':
                # edge case: Save the index of the first 0.
                if 0 not in valToLeftAndRight:
                    valToLeftAndRight[0] = [left, left]
                continue
            for right in range(left, min(len(s), left + MAX_BIT)):
                val = val * 2 + int(s[right])
                if val not in valToLeftAndRight:
                    valToLeftAndRight[val] = [left, right]

        return [valToLeftAndRight[first, right]
                for first, right in queries]

```

1523. 2565.py

Path: LeetCode-main/solutions/2565. Subsequence With the Minimum Score/2565.py

```

class Solution:
    def minimumScore(self, s: str, t: str) -> int:
        # leftmost[j] := the minimum index i s.t. t[0..j] is a subsequence of s[0..i].
        #           -1 := impossible
        leftmost = [-1] * len(t)
        # rightmost[j] := the maximum index i s.t. t[j:] is a subsequence of s[i..n).
        #           -1 := impossible
        rightmost = [-1] * len(t)

        j = 0 # t's index
        for i in range(len(s)):
            if s[i] == t[j]:
                leftmost[j] = i
                j += 1
            if j == len(t):
                break

        j = len(t) - 1 # t's index
        for i in reversed(range(len(s))):
            if s[i] == t[j]:
                rightmost[j] = i
                j -= 1
            if j == -1:
                break

        # The worst case is to delete t[0:j] since t[j:] is a subsequence of s. (deduced
        # from the above loop).
        ans = j + 1

        j = 0
        for i in range(len(t)):
            # It's impossible that t[0..i] is a subsequence of s. So, stop the loop since
            # no need to consider any larger i.
            if leftmost[i] == -1:
                break
            # While t[0..i] + t[j:] is not a subsequence of s, increase j.
            while j < len(t) and leftmost[i] >= rightmost[j]:
                j += 1
            # Now, leftmost[i] < rightmost[j], so t[0..i] + t[j:n] is a subsequence of s.
            # If i == j that means t is a subsequence of s, so just return 0.
            if i == j:
                return 0
            # Delete t[i + 1..j - 1] and that's a total of j - i - 1 letters.
            ans = min(ans, j - i - 1)

        return ans

```

1524. 2566.py

Path: LeetCode-main/solutions/2566. Maximum Difference by Remapping a Digit/2566.py

```
class Solution:
    def minMaxDifference(self, num: int) -> int:
        s = str(num)
        to9 = s[self._firstNotNineIndex(s)]
        to0 = s[0]
        return int(s.replace(to9, '9')) - int(s.replace(to0, '0'))

    def _firstNotNineIndex(self, s: str) -> int:
        for i, c in enumerate(s):
            if c != '9':
                return i
        return 0
```

1525. 2567.py

Path: LeetCode-main/solutions/2567. Minimum Score by Changing Two Elements/2567.py

```
class Solution:
    def minimizeSum(self, nums: list[int]) -> int:
        nums.sort()
        # Can always change the number to any other number in `nums`, so `low` becomes 0.
        # Thus, rephrase the problem as finding the minimum `high`.
        highOfChangingTwoMins = nums[-1] - nums[2]
        highOfChangingTwoMaxs = nums[-3] - nums[0]
        highOfChangingMinAndMax = nums[-2] - nums[1]
        return min(highOfChangingTwoMins, highOfChangingTwoMaxs,
                    highOfChangingMinAndMax)
```

1526. 2568.py

Path: LeetCode-main/solutions/2568. Minimum Impossible OR/2568.py

```
class Solution:
    def minImpossibleOR(self, nums: list[int]) -> int:
        ans = 1
        numsSet = set(nums)

        while ans in numsSet:
            ans <<= 1

        return ans
```

1527. 257.py

Path: LeetCode-main/solutions/257. Binary Tree Paths/257.py

```
class Solution:
    def binaryTreePaths(self, root: TreeNode | None) -> list[str]:
        ans = []

        def dfs(root: TreeNode | None, path: list[str]) -> None:
            if not root:
                return
            if not root.left and not root.right:
                ans.append(''.join(path) + str(root.val))
                return

            path.append(str(root.val) + '->')
            dfs(root.left, path)
            dfs(root.right, path)
            path.pop()

        dfs(root, [])
        return ans
```

1528. 2570.py

Path: LeetCode-main/solutions/2570. Merge Two 2D Arrays by Summing Values/2570.py

```
class Solution:
    def mergeArrays(self, nums1: list[list[int]],
                    nums2: list[list[int]]) -> list[list[int]]:
        count = [0] * (1001)
        self._addCount(nums1, count)
        self._addCount(nums2, count)
        return [[i, c] for i, c in enumerate(count) if c > 0]

    def _addCount(self, nums: list[list[int]], count: list[int]) -> None:
        for id_, val in nums:
            count[id_] += val
```

1529. 2571.py

Path: LeetCode-main/solutions/2571. Minimum Operations to Reduce an Integer to 0/2571.py

```
class Solution:
    def minOperations(self, n: int) -> int:
        # The strategy is that when the end of n is
        # 1. consecutive 1s, add 1 ( $2^0$ ).
        # 2. single 1, subtract 1 ( $2^0$ ).
        # 3. 0, subtract  $2^k$  to omit the last 1. Equivalently,  $n \gg 1$ .
        #
        # e.g.
        #
        #         n = 0b101
        # n -=  $2^0$  -> 0b100
        # n -=  $2^2$  -> 0b0
        #         n = 0b1011
        # n +=  $2^0$  -> 0b1100
        # n -=  $2^2$  -> 0b1000
        # n -=  $2^3$  -> 0b0
        ans = 0

        while n > 0:
            if (n & 3) == 3:
                n += 1
                ans += 1
            elif n % 2 == 1:
                n -= 1
                ans += 1
            else:
                n >>= 1

        return ans
```

1530. 2572.py

Path: LeetCode-main/solutions/2572. Count the Number of Square-Free Subsets/2572.py


```

class Solution:
    def squareFreeSubsets(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

        def getMask(num: int) -> int:
            """
            e.g. num = 10 = 2 * 5, so mask = 0b101 . 0b1010 (append a 0)
                  num = 15 = 3 * 5, so mask = 0b110 . 0b1100 (append a 0)
                  num = 25 = 5 * 5, so mask = (-1)2 . (1..1)2 (invalid)
            """
            mask = 0
            for i, prime in enumerate(primes):
                rootCount = 0
                while num % prime == 0:
                    num //= prime
                    rootCount += 1
                if rootCount >= 2:
                    return -1
                if rootCount == 1:
                    mask |= 1 << i
            return mask << 1

        masks = [getMask(num) for num in nums]

        @functools.lru_cache(None)
        def dp(i: int, used: int) -> int:
            if i == len(masks):
                return 1
            pick = dp(i + 1, used | masks[i]) if (masks[i] & used) == 0 else 0
            skip = dp(i + 1, used)
            return (pick + skip) % MOD

        # -1 means that we take no number.
        # `used` is initialized to 1 so that -1 & 1 = 1 instead of 0.
        return (dp(0, 1) - 1 + MOD) % MOD

```

1531. 2574-2.py

Path: LeetCode-main/solutions/2574. Left and Right Sum Differences/2574-2.py

```

class Solution:
    def leftRigthDifference(self, nums: list[int]) -> list[int]:
        ans = []
        leftSum = 0
        rightSum = sum(nums)

        for num in nums:
            rightSum -= num
            ans.append(abs(leftSum - rightSum))
            leftSum += num

        return ans

```

1532. 2574.py

Path: LeetCode-main/solutions/2574. Left and Right Sum Differences/2574.py

```

class Solution:
    def leftRigthDifference(self, nums: list[int]) -> list[int]:
        n = len(nums)
        leftSum = [0] * n
        rightSum = [0] * n
        prefix = 0
        suffix = 0

        for i in range(n):
            if i > 0:
                prefix += nums[i - 1]
            leftSum[i] = prefix

        for i in range(n - 1, -1, -1):
            if i + 1 < n:
                suffix += nums[i + 1]
            rightSum[i] = suffix

        return [abs(l - r) for l, r in zip(leftSum, rightSum)]

```

1533. 2575.py

Path: LeetCode-main/solutions/2575. Find the Divisibility Array of a String/2575.py

```

class Solution:
    def divisibilityArray(self, word: str, m: int) -> list[int]:
        ans = []
        prevRemainder = 0

        for c in word:
            remainder = (prevRemainder * 10 + int(c)) % m
            ans.append(1 if remainder == 0 else 0)
            prevRemainder = remainder

        return ans

```

1534. 2576-2.py

Path: LeetCode-main/solutions/2576. Find the Maximum Number of Marked Indices/2576-2.py

```

class Solution:
    def maxNumOfMarkedIndices(self, nums: list[int]) -> int:
        nums.sort()

        i = 0
        for j in range(len(nums) // 2, len(nums)):
            if 2 * nums[i] <= nums[j]:
                i += 1
            if i == len(nums) // 2:
                break

        return i * 2

```

1535. 2576.py

Path: LeetCode-main/solutions/2576. Find the Maximum Number of Marked Indices/2576.py

```

class Solution:
    def maxNumOfMarkedIndices(self, nums: list[int]) -> int:
        nums.sort()

        def isPossible(m: int) -> bool:
            for i in range(m):
                if 2 * nums[i] > nums[-m + i]:
                    return False
            return True

        l = bisect.bisect_left(range(len(nums) // 2 + 1), True,
                               key=lambda m: not isPossible(m))

        return (l - 1) * 2

```

1536. 2577.py

Path: LeetCode-main/solutions/2577. Minimum Time to Visit a Cell In a Grid/2577.py

```

class Solution:
    def minimumTime(self, grid: list[list[int]]) -> int:
        if grid[0][1] > 1 and grid[1][0] > 1:
            return -1

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        minHeap = [(0, 0, 0)] # (time, i, j)
        seen = {(0, 0)}

        while minHeap:
            time, i, j = heapq.heappop(minHeap)
            if i == m - 1 and j == n - 1:
                return time
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if (x, y) in seen:
                    continue
                extraWait = 1 if (grid[x][y] - time) % 2 == 0 else 0
                nextTime = max(time + 1, grid[x][y] + extraWait)
                heapq.heappush(minHeap, (nextTime, x, y))
                seen.add((x, y))

```

1537. 2578.py

Path: LeetCode-main/solutions/2578. Split With Minimum Sum/2578.py

```

class Solution:
    def splitNum(self, num: int) -> int:
        s = ''.join(sorted(str(num)))
        return sum(map(int, (s[:len(s)//2], s[len(s)//2:])))

```

1538. 2579.py

Path: LeetCode-main/solutions/2579. Count Total Number of Colored Cells/2579.py

```
class Solution:
    def coloredCells(self, n: int) -> int:
        return n**2 + (n - 1)**2
```

1539. 258.py

Path: LeetCode-main/solutions/258. Add Digits/258.py

```
class Solution:
    def addDigits(self, num: int) -> int:
        return 0 if num == 0 else 1 + (num - 1) % 9
```

1540. 2580.py

Path: LeetCode-main/solutions/2580. Count Ways to Group Overlapping Ranges/2580.py

```
class Solution:
    def countWays(self, ranges: list[list[int]]) -> int:
        MOD = 1_000_000_007
        ans = 1
        prevEnd = -1

        for start, end in sorted(ranges):
            if start > prevEnd:
                ans = ans * 2 % MOD
            prevEnd = max(prevEnd, end)

        return ans
```

1541. 2581.py

Path: LeetCode-main/solutions/2581. Count Number of Possible Root Nodes/2581.py

```

class Solution:
    def rootCount(
        self,
        edges: list[list[int]],
        guesses: list[list[int]],
        k: int,
    ) -> int:
        ans = 0
        n = len(edges) + 1
        graph = [[] for _ in range(n)]
        guessGraph = [set() for _ in range(n)]
        parent = [0] * n

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        for u, v in guesses:
            guessGraph[u].add(v)

        def dfs(u: int, prev: int) -> None:
            parent[u] = prev
            for v in graph[u]:
                if v != prev:
                    dfs(v, u)

        # Precalculate `parent`.
        dfs(0, -1)

        # Calculate `correctGuess` for tree rooted at 0.
        correctGuess = sum(i in guessGraph[parent[i]] for i in range(1, n))

        def reroot(u: int, prev: int, correctGuess: int) -> None:
            nonlocal ans
            if u != 0:
                # The tree is rooted at u, so a guess edge (u, prev) will match the new
                # `parent` relationship.
                if prev in guessGraph[u]:
                    correctGuess += 1
                # A guess edge (prev, u) matching the old `parent` relationship will no
                # longer be True.
                if u in guessGraph[prev]:
                    correctGuess -= 1
            if correctGuess >= k:
                ans += 1
            for v in graph[u]:
                if v != prev:
                    reroot(v, u, correctGuess)

        reroot(0, -1, correctGuess)
        return ans

```

1542. 2582.py

Path: LeetCode-main/solutions/2582. Pass the Pillow/2582.py

```
class Solution:
    def passThePillow(self, n: int, time: int) -> int:
        # Repeat every (n - 1) * 2 seconds.
        time %= (n - 1) * 2
        if time < n: # Go forward from 1.
            return 1 + time
        return n - (time - (n - 1)) # Go backward from n.
```

1543. 2583.py

Path: LeetCode-main/solutions/2583. Kth Largest Sum in a Binary Tree/2583.py

```
class Solution:
    def kthLargestLevelSum(self, root: TreeNode | None, k: int) -> int:
        levelSums = []

        def dfs(root: TreeNode | None, level: int) -> None:
            if not root:
                return
            if len(levelSums) == level:
                levelSums.append(0)
            levelSums[level] += root.val
            dfs(root.left, level + 1)
            dfs(root.right, level + 1)

        dfs(root, 0)
        if len(levelSums) < k:
            return -1

        return sorted(levelSums, reverse=True)[k - 1]
```

1544. 2584.py

Path: LeetCode-main/solutions/2584. Split the Array to Make Coprime Products/2584.py

```

class Solution:
    def findValidSplit(self, nums: list[int]) -> int:
        leftPrimeFactors = collections.Counter()
        rightPrimeFactors = collections.Counter()

        def getPrimeFactors(num: int) -> list[int]:
            """Gets the prime factors under sqrt(10^6)."""
            primeFactors = []
            for divisor in range(2, min(1000, num) + 1):
                if num % divisor == 0:
                    primeFactors.append(divisor)
                    while num % divisor == 0:
                        num //= divisor
            # Handle the case that `num` contains a prime factor > 1000.
            if num > 1:
                primeFactors.append(num)
            return primeFactors

        for num in nums:
            for primeFactor in getPrimeFactors(num):
                rightPrimeFactors[primeFactor] += 1

        for i in range(len(nums) - 1):
            for primeFactor in getPrimeFactors(nums[i]):
                rightPrimeFactors[primeFactor] -= 1
                if rightPrimeFactors[primeFactor] == 0:
                    # rightPrimeFactors[primeFactor] == 0, so no need to track
                    # leftPrimeFactors[primeFactor].
                    del rightPrimeFactors[primeFactor]
                    del leftPrimeFactors[primeFactor]
                else:
                    # Otherwise, need to track leftPrimeFactors[primeFactor].
                    leftPrimeFactors[primeFactor] += 1
            if not leftPrimeFactors:
                return i

        return -1

```

1545. 2585-2.py

Path: LeetCode-main/solutions/2585. Number of Ways to Earn Points/2585-2.py

```

class Solution:
    def waysToReachTarget(self, target: int, types: list[list[int]]) -> int:
        MOD = 1_000_000_007
        # dp[j] := the number of ways to earn j points with the types so far
        dp = [1] + [0] * target

        for count, mark in types:
            for j in range(target, -1, -1):
                for solved in range(1, count + 1):
                    if j - solved * mark >= 0:
                        dp[j] += dp[j - solved * mark]
                        dp[j] %= MOD

        return dp[target]

```

1546. 2585.py

Path: LeetCode-main/solutions/2585. Number of Ways to Earn Points/2585.py

```
class Solution:
    def waysToReachTarget(self, target: int, types: list[list[int]]) -> int:
        MOD = 1_000_000_007
        # dp[i][j] := the number of ways to earn j points with the first i types
        dp = [[0] * (target + 1) for _ in range(len(types) + 1)]
        dp[0][0] = 1

        for i in range(1, len(types) + 1):
            count = types[i - 1][0]
            mark = types[i - 1][1]
            for j in range(target + 1):
                for solved in range(count + 1):
                    if j - solved * mark >= 0:
                        dp[i][j] += dp[i - 1][j - solved * mark]
                        dp[i][j] %= MOD

        return dp[len(types)][target]
```

1547. 2586.py

Path: LeetCode-main/solutions/2586. Count the Number of Vowel Strings in Range/2586.py

```
class Solution:
    def vowelStrings(self, words: list[str], left: int, right: int) -> int:
        VOWELS = 'aeiou'
        return sum(word[0] in VOWELS and word[-1] in VOWELS
                    for word in words[left:right + 1])
```

1548. 2587.py

Path: LeetCode-main/solutions/2587. Rearrange Array to Maximize Prefix Score/2587.py

```
class Solution:
    def maxScore(self, nums: list[int]) -> int:
        return sum(num > 0
                    for num in itertools.accumulate(sorted(nums, reverse=True)))
```

1549. 2588.py

Path: LeetCode-main/solutions/2588. Count the Number of Beautiful Subarrays/2588.py

```
class Solution:
    def beautifulSubarrays(self, nums: list[int]) -> int:
        # A subarray is beautiful if xor(subarray) = 0.
        ans = 0
        prefix = 0
        prefixCount = collections.Counter({0: 1})

        for num in nums:
            prefix ^= num
            ans += prefixCount[prefix]
            prefixCount[prefix] += 1

        return ans
```

1550. 2589.py

Path: LeetCode-main/solutions/2589. Minimum Time to Complete All Tasks/2589.py

```
class Solution:
    def findMinimumTime(self, tasks: list[list[int]]) -> int:
        MAX = 2000
        running = [False] * (MAX + 1)

        # Sort tasks by end.
        for start, end, duration in sorted(tasks, key=lambda x: x[1]):
            neededDuration = (duration -
                              sum(running[i] for i in range(start, end + 1)))
            # Greedily run the task as late as possible so that later tasks can run
            # simultaneously.
            i = end
            while neededDuration > 0:
                if not running[i]:
                    running[i] = True
                    neededDuration -= 1
                i -= 1

        return sum(running)
```

1551. 259.py

Path: LeetCode-main/solutions/259. 3Sum Smaller/259.py

```
class Solution:
    def threeSumSmaller(self, nums: list[int], target: int) -> int:
        if len(nums) < 3:
            return 0

        ans = 0

        nums.sort()

        for i in range(len(nums) - 2):
            l = i + 1
            r = len(nums) - 1
            while l < r:
                if nums[i] + nums[l] + nums[r] < target:
                    # (nums[i], nums[l], nums[r])
                    # (nums[i], nums[l], nums[r - 1])
                    # ...,
                    # (nums[i], nums[l], nums[l + 1])
                    ans += r - l
                    l += 1
                else:
                    r -= 1

        return ans
```

1552. 2590.py

Path: LeetCode-main/solutions/2590. Design a Todo List/2590.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Task:
    taskDescription: str
    dueDate: int
    tags: list[str]

class TodoList:
    def __init__(self):
        self.taskId = 0
        self.taskIds = set()
        self.userIdToTaskIdToTasks: dict[int, dict[int, list[Task]]] = {}

    def addTask(self, userId: int, taskDescription: str, dueDate: int,
               tags: list[str]) -> int:
        self.taskId += 1
        taskIdToTasks = self.userIdToTaskIdToTasks.setdefault(userId, {})
        taskIdToTasks[self.taskId] = Task(taskDescription, dueDate, tags)
        self.taskIds.add(self.taskId)
        return self.taskId

    def getAllTasks(self, userId: int) -> list[str]:
        return [task.taskDescription
                for task in self._getTasksSortedByDueDate(userId)]

    def getTasksForTag(self, userId: int, tag: str) -> list[str]:
        return [task.taskDescription
                for task in self._getTasksSortedByDueDate(userId)
                if tag in task.tags]

    def completeTask(self, userId: int, taskId: int) -> None:
        if taskId not in self.taskIds:
            return
        if userId not in self.userIdToTaskIdToTasks:
            return
        taskIdToTasks = self.userIdToTaskIdToTasks[userId]
        if taskId not in taskIdToTasks:
            return
        del taskIdToTasks[taskId]

    def _getTasksSortedByDueDate(self, userId: int) -> list[Task]:
        if userId not in self.userIdToTaskIdToTasks:
            return []
        taskIdToTasks = self.userIdToTaskIdToTasks[userId]
        return sorted(
            [task for task in taskIdToTasks.values()],
            key=lambda x: x.dueDate)

```

1553. 2591.py

Path: LeetCode-main/solutions/2591. Distribute Money to Maximum Children/2591.py

```

class Solution:
    def distMoney(self, money: int, children: int) -> int:
        # Everyone must receive at least 1 dollar.
        money -= children
        if money < 0:
            return -1

        count7 = money // 7
        remaining = money % 7

        # Distribute 8 dollars to every child.
        if count7 == children and remaining == 0:
            return count7

        # Need to move 1 dollar from the last child with 4 dollars to one of other
        # children. That's why we need to subtract 1.
        if count7 == children - 1 and remaining == 3:
            return count7 - 1

        # Though there might be child with 4 dollars, since count7 < children - 1,
        # we have 'extra' spot to move money to if needed.
        return min(children - 1, count7)

```

1554. 2592.py

Path: LeetCode-main/solutions/2592. Maximize Greatness of an Array/2592.py

```

class Solution:
    def maximizeGreatness(self, nums: list[int]) -> int:
        ans = 0

        nums.sort()

        for num in nums:
            if num > nums[ans]:
                ans += 1

        return ans

```

1555. 2593.py

Path: LeetCode-main/solutions/2593. Find Score of an Array After Marking All Elements/2593.py

```

class Solution:
    def findScore(self, nums: list[int]) -> int:
        ans = 0
        seen = set()

        for num, i in sorted([(num, i) for i, num in enumerate(nums)]):
            if i in seen:
                continue
            seen.add(i - 1)
            seen.add(i + 1)
            seen.add(i)
            ans += num

        return ans

```

1556. 2594.py

Path: LeetCode-main/solutions/2594. Minimum Time to Repair Cars/2594.py

```
class Solution:
    def repairCars(self, ranks: list[int], cars: int) -> int:
        def numCarsFixed(minutes: int) -> int:
            # r * n^2 = minutes
            # -> n = sqrt(minutes / r)
            return sum(math.isqrt(minutes // rank) for rank in ranks)

        return bisect.bisect_left(range(min(ranks) * cars**2), cars,
                                   key=numCarsFixed)
```

1557. 2595.py

Path: LeetCode-main/solutions/2595. Number of Even and Odd Bits/2595.py

```
class Solution:
    def evenOddBit(self, n: int) -> list[int]:
        ans = [0] * 2
        i = 0 # 0 := even, 1 := odd

        while n > 0:
            ans[i] += n & 1
            n >>= 1
            i ^= 1

        return ans
```

1558. 2596.py

Path: LeetCode-main/solutions/2596. Check Knight Tour Configuration/2596.py

```

class Solution:
    def checkValidGrid(self, grid: list[list[int]]) -> bool:
        if grid[0][0] != 0:
            return False

        DIRS = ((1, 2), (2, 1), (2, -1), (1, -2),
                (-1, -2), (-2, -1), (-2, 1), (-1, 2))

        n = len(grid)
        i = 0
        j = 0

        def nextGrid(i: int, j: int, target: int) -> tuple[int, int]:
            """
            Returns (x, y), where grid[x][y] == target if (i, j) can reach target.
            """
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x >= n or y < 0 or y >= n:
                    continue
                if grid[x][y] == target:
                    return (x, y)
            return (-1, -1)

        for target in range(1, n * n):
            x, y = nextGrid(i, j, target)
            if x == -1 and y == -1:
                return False
            # Move (x, y) to (i, j).
            i = x
            j = y

        return True

```

1559. 2597.py

Path: LeetCode-main/solutions/2597. The Number of Beautiful Subsets/2597.py

```

# e.g. nums = [2, 3, 4, 4], k = 2
#
# subset[0] = [2, 4, 4']
# subset[1] = [1]
# count = {2: 1, 4: 2, 1: 1}
#
# Initially, skip = len([]) = 0, pick = len([]) = 0
#
# * For values in subset[0]:
#   After 2:
#     skip = skip + pick = len([]) = 0
#     pick = (2^count[2] - 1) * (1 + skip + pick)
#           = len([[2]]) * len([])
#           = len([[2]]) = 1
#   After 4:
#     skip = skip + pick = len([[2]]) = 1
#     pick = (2^count[4] - 1) * (1 + skip)
#           = len([[4], [4'], [4, 4']]) * len([])
#           = len([[4], [4'], [4, 4']]) = 3
#
# * For values in subset[1]:
#   After 1:
#     skip = skip + pick
#           = len([[2], [4], [4'], [4, 4']]) = 4
#     pick = (2^count[1] - 1) * (1 + skip + pick)
#           = len([[1]]) * len([[2], [4], [4'], [4, 4']])
#           = len([[1], [1, 2], [1, 4], [1, 4'], [1, 4, 4']]) = 5
#
# So, ans = skip + pick = 9

class Solution:
    def beautifulSubsets(self, nums: list[int], k: int) -> int:
        count = collections.Counter(nums)
        modToSubset = collections.defaultdict(set)

        for num in nums:
            modToSubset[num % k].add(num)

        prevNum = -k
        skip = 0
        pick = 0

        for subset in modToSubset.values():
            for num in sorted(subset):
                nonEmptyCount = 2**count[num] - 1
                skip, pick = (skip + pick,
                             nonEmptyCount *
                             (1 + skip + (0 if num - prevNum == k else pick)))
                prevNum = num

        return skip + pick

```

1560. 2598.py

Path: LeetCode-main/solutions/2598. Smallest Missing Non-negative Integer After Operations/2598.py

```
class Solution:
    def findSmallestInteger(self, nums: list[int], value: int) -> int:
        count = collections.Counter([num % value for num in nums])

        for i in range(len(nums)):
            if count[i % value] == 0:
                return i
            count[i % value] -= 1

        return len(nums)
```

1561. 2599.py

Path: LeetCode-main/solutions/2599. Make the Prefix Sum Non-negative/2599.py

```
class Solution:
    def makePrefSumNonNegative(self, nums: list[int]) -> int:
        ans = 0
        prefix = 0
        minHeap = []

        for num in nums:
            prefix += num
            if num < 0:
                heapq.heappush(minHeap, num)
            while prefix < 0:
                prefix -= heapq.heappop(minHeap)
                ans += 1

        return ans
```

1562. 26.py

Path: LeetCode-main/solutions/26. Remove Duplicates from Sorted Array/26.py

```
class Solution:
    def removeDuplicates(self, nums: list[int]) -> int:
        i = 0

        for num in nums:
            if i < 1 or num > nums[i - 1]:
                nums[i] = num
                i += 1

        return i
```

1563. 260.py

Path: LeetCode-main/solutions/260. Single Number III/260.py

```
class Solution:
    def singleNumber(self, nums: list[int]) -> list[int]:
        xors = functools.reduce(operator.xor, nums)
        lowbit = xors & -xors
        ans = [0, 0]

        # Seperate `nums` into two groups by `lowbit`.
        for num in nums:
            if num & lowbit:
                ans[0] ^= num
            else:
                ans[1] ^= num

        return ans
```

1564. 2600.py

Path: LeetCode-main/solutions/2600. K Items With the Maximum Sum/2600.py

```
class Solution:
    def kItemsWithMaximumSum(
        self,
        numOnes: int,
        numZeros: int,
        numNegOnes: int,
        k: int,
    ) -> int:
        if k <= numOnes:
            return k
        if k <= numOnes + numZeros:
            return numOnes
        return numOnes - (k - numOnes - numZeros)
```

1565. 2601.py

Path: LeetCode-main/solutions/2601. Prime Subtraction Operation/2601.py


```

class Solution:
    def primeSubOperation(self, nums: list[int]) -> bool:
        MAX = 1000
        primes = self._sieveEratosthenes(MAX)

        prevNum = 0
        for num in nums:
            # Make nums[i] the smallest as possible and still > nums[i - 1].
            i = bisect.bisect_left(primes, num - prevNum)
            if i > 0:
                num -= primes[i - 1]
            if num <= prevNum:
                return False
            prevNum = num

        return True

    def _sieveEratosthenes(self, n: int) -> list[int]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return [i for i in range(n) if isPrime[i]]

```

1566. 2602.py

Path: LeetCode-main/solutions/2602. Minimum Operations to Make All Array Elements Equal/2602.py

```

class Solution:
    def minOperations(self, nums: list[int], queries: list[int]) -> list[int]:
        n = len(nums)
        nums.sort()
        prefix = list(itertools.accumulate(nums, initial=0))
        splits = [(query, bisect.bisect_right(nums, query)) for query in queries]
        return [(query * i - prefix[i]) +
                (prefix[-1] - prefix[i] - query * (n - i))
                for query, i in splits]

```

1567. 2603.py

Path: LeetCode-main/solutions/2603. Collect Coins in a Tree/2603.py

```

class Solution:
    def collectTheCoins(self, coins: list[int], edges: list[list[int]]) -> int:
        n = len(coins)
        tree = [set() for _ in range(n)]
        leavesToBeRemoved = collections.deque()

        for u, v in edges:
            tree[u].add(v)
            tree[v].add(u)

        for u in range(n):
            # Remove the leaves that don't have coins.
            while len(tree[u]) == 1 and coins[u] == 0:
                v = tree[u].pop()
                tree[v].remove(u)
                u = v # Walk up to its parent.
            # After trimming leaves without coins, leaves with coins may satisfy
            # `leavesToBeRemoved`.
            if len(tree[u]) == 1: # coins[u] must be 1.
                leavesToBeRemoved.append(u)

        # Remove each remaining leaf node and its parent. The remaining nodes are
        # the ones that must be visited.
        for _ in range(2):
            for _ in range(len(leavesToBeRemoved)):
                u = leavesToBeRemoved.popleft()
                if tree[u]:
                    v = tree[u].pop()
                    tree[v].remove(u)
                    if len(tree[v]) == 1: # It's a leaf.
                        leavesToBeRemoved.append(v)

        return sum(len(children) for children in tree)

```

1568. 2604.py

Path: LeetCode-main/solutions/2604. Minimum Time to Eat All Grains/2604.py

```

class Solution:
    def minimumTime(self, hens: list[int], grains: list[int]) -> int:
        hens.sort()
        grains.sort()

    def canEat(time: int) -> bool:
        """Returns True if `hens` can eat all `grains` within `time`."""
        i = 0 # grains[i] := next grain to be ate
        for hen in hens:
            rightMoves = time
            if grains[i] < hen:
                # `hen` needs go back to eat `grains[i]`.
                leftMoves = hen - grains[i]
                if leftMoves > time:
                    return False
                leftThenRight = time - 2 * leftMoves
                rightThenLeft = (time - leftMoves) // 2
                rightMoves = max(0, leftThenRight, rightThenLeft)
            i = bisect.bisect_right(grains, hen + rightMoves)
        if i == len(grains):
            return True
        return False

    maxMoves = int(1.5 * (max(hens + grains) - min(hens + grains)))
    return bisect.bisect_left(range(maxMoves), True, key=canEat)

```

1569. 2605.py

Path: LeetCode-main/solutions/2605. Form Smallest Number From Two Digit Arrays/2605.py

```

class Solution:
    def minNumber(self, nums1: list[int], nums2: list[int]) -> int:
        return min(a if a == b else min(a, b) * 10 + max(a, b)
                    for a in nums1
                    for b in nums2)

```

1570. 2606.py

Path: LeetCode-main/solutions/2606. Find the Substring With Maximum Cost/2606.py

```

class Solution:
    def maximumCostSubstring(self, s: str, chars: str, vals: list[int]) -> int:
        ans = 0
        cost = 0
        costs = [i for i in range(1, 27)] # costs[i] := the cost of 'a' + i

        for c, val in zip(chars, vals):
            costs[ord(c) - ord('a')] = val

        for c in s:
            cost = max(0, cost + costs[ord(c) - ord('a')])
            ans = max(ans, cost)

        return ans

```

1571. 2607.py

Path: LeetCode-main/solutions/2607. Make K-Subarray Sums Equal/2607.py

```
class Solution:
    def makeSubKSumEqual(self, arr: list[int], k: int) -> int:
        # If the sum of each subarray of length k is equal, then `arr` must have a
        # repeated pattern of size k. e.g. arr = [1, 2, 3, ...] and k = 3, to have
        # sum([1, 2, 3]) == sum([2, 3, x]), x must be 1. Therefore, arr[i] ==
        # arr[(i + k) % n] for every i.
        n = len(arr)
        ans = 0
        seen = [0] * n

        for i in range(n):
            groups = []
            j = i
            while not seen[j]:
                groups.append(arr[j])
                seen[j] = True
                j = (j + k) % n
            groups.sort()
            for num in groups:
                ans += abs(num - groups[len(groups) // 2])

        return ans
```

1572. 2608.py

Path: LeetCode-main/solutions/2608. Shortest Cycle in a Graph/2608.py

```
class Solution:
    def findShortestCycle(self, n: int, edges: list[list[int]]) -> int:
        INF = 1001
        ans = INF
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def bfs(i: int) -> int:
            """Returns the length of the minimum cycle by starting BFS from node `i` .

            Returns `INF` if there's no cycle.
            """
            dist = [INF] * n
            q = collections.deque([i])
            dist[i] = 0
            while q:
                u = q.popleft()
                for v in graph[u]:
                    if dist[v] == INF:
                        dist[v] = dist[u] + 1
                        q.append(v)
                    elif dist[v] + 1 != dist[u]: # v is not a parent u.
                        return dist[v] + dist[u] + 1
            return INF

        ans = min(map(bfs, range(n)))
        return -1 if ans == INF else ans
```

1573. 2609-2.py

Path: LeetCode-main/solutions/2609. Find the Longest Balanced Substring of a Binary String/2609-2.py

```
class Solution:
    def findTheLongestBalancedSubstring(self, s: str) -> int:
        ans = 0
        zeros = 0
        ones = 0

        for c in s:
            if c == '0':
                zeros = 1 if ones > 0 else zeros + 1
                ones = 0
            else: # c == '1'
                ones += 1
            if zeros >= ones:
                ans = max(ans, ones)

        return ans * 2
```

1574. 2609.py

Path: LeetCode-main/solutions/2609. Find the Longest Balanced Substring of a Binary String/2609.py

```
class Solution:
    def findTheLongestBalancedSubstring(self, s: str) -> int:
        ans = 0

        i = 0
        while i < len(s):
            zeros = 0
            ones = 0
            while i < len(s) and s[i] == '0':
                zeros += 1
                i += 1
            while i < len(s) and s[i] == '1':
                ones += 1
                i += 1
            ans = max(ans, min(zeros, ones))

        return ans * 2
```

1575. 261-2.py

Path: LeetCode-main/solutions/261. Graph Valid Tree/261-2.py

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

        uf = UnionFind(n)

        for u, v in edges:
            uf.unionByRank(u, v)

        return uf.count == 1
```

1576. 261.py

Path: LeetCode-main/solutions/261. Graph Valid Tree/261.py

```

class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

        graph = [[] for _ in range(n)]
        q = collections.deque([0])
        seen = {0}

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        while q:
            u = q.popleft()
            for v in graph[u]:
                if v not in seen:
                    q.append(v)
                    seen.add(v)

        return len(seen) == n

```

1577. 2610.py

Path: LeetCode-main/solutions/2610. Convert an Array Into a 2D Array With Conditions/2610.py

```

class Solution:
    def findMatrix(self, nums: list[int]) -> list[list[int]]:
        # The number of rows we need equals the maximum frequency.
        ans = []
        count = [0] * (len(nums) + 1)

        for num in nums:
            count[num] += 1
            # Construct `ans` on demand.
            if count[num] > len(ans):
                ans.append([])
            ans[count[num] - 1].append(num)

        return ans

```

1578. 2611.py

Path: LeetCode-main/solutions/2611. Mice and Cheese/2611.py

```

class Solution:
    def miceAndCheese(
        self,
        reward1: list[int],
        reward2: list[int],
        k: int,
    ) -> int:
        return (sum(reward2) +
                sum(heapq.nlargest(k, (a - b for a, b in zip(reward1, reward2)))))

```

1579. 2612.py

Path: LeetCode-main/solutions/2612. Minimum Reverse Operations/2612.py

```
from sortedcontainers import SortedList

class Solution:
    def minReverseOperations(
        self,
        n: int,
        p: int,
        banned: list[int],
        k: int,
    ) -> list[int]:
        bannedSet = set(banned)
        ans = [-1] * n
        # unseen[i] := the unseen numbers that % 2 == i
        unseen = [SortedList(), SortedList()]

        for num in range(n):
            if num != p and num not in bannedSet:
                unseen[num % 2].add(num)

        # Perform BFS from `p`.
        q = collections.deque([p])
        ans[p] = 0

        while q:
            u = q.popleft()
            lo = max(u - k + 1, k - 1 - u)
            hi = min(u + k - 1, n - 1 - (u - (n - k)))
            # Choose the correct set of numbers.
            nums = unseen[lo % 2]
            i = nums.bisect_left(lo)
            while i < len(nums) and nums[i] <= hi:
                num = nums[i]
                ans[num] = ans[u] + 1
                q.append(num)
                nums.pop(i)

        return ans
```

1580. 2614.py

Path: LeetCode-main/solutions/2614. Prime In Diagonal/2614.py

```
class Solution:
    def diagonalPrime(self, nums: list[list[int]]) -> int:
        def isPrime(n: int) -> bool:
            if n <= 1:
                return False
            for i in range(2, int(n**0.5) + 1):
                if n % i == 0:
                    return False
            return True

        primes1 = [row[i]
                    for i, row in enumerate(nums) if isPrime(row[i])]
        primes2 = [row[-1 - i]
                    for i, row in enumerate(nums) if isPrime(row[-1 - i])]
        return max(max(primes1) if primes1 else 0,
                   max(primes2) if primes2 else 0)
```

1581. 2615.py

Path: LeetCode-main/solutions/2615. Sum of Distances/2615.py

```
class Solution:
    def distance(self, nums: list[int]) -> list[int]:
        ans = [0] * len(nums)
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums):
            numToIndices[num].append(i)

        for indices in numToIndices.values():
            n = len(indices)
            if n == 1:
                continue
            sumSoFar = sum(indices)
            prevIndex = 0
            for i in range(n):
                sumSoFar += (i - 1) * (indices[i] - prevIndex)
                sumSoFar -= (n - 1 - i) * (indices[i] - prevIndex)
                ans[indices[i]] = sumSoFar
                prevIndex = indices[i]

        return ans
```

1582. 2616.py

Path: LeetCode-main/solutions/2616. Minimize the Maximum Difference of Pairs/2616.py

```
class Solution:
    def minimizeMax(self, nums: list[int], p: int) -> int:
        nums.sort()

        def numPairs(maxDiff: int) -> int:
            """
            Returns the number of pairs that can be obtained if the difference between
            each pair <= `maxDiff`.
            """
            pairs = 0
            i = 1
            while i < len(nums):
                # Greedily pair nums[i] with nums[i - 1].
                if nums[i] - nums[i - 1] <= maxDiff:
                    pairs += 1
                    i += 2
                else:
                    i += 1
            return pairs

        return bisect.bisect_left(range(nums[-1] - nums[0]), p, key=numPairs)
```

1583. 263.py

Path: LeetCode-main/solutions/263. Ugly Number/263.py

```
class Solution:
    def isUgly(self, n: int) -> bool:
        if n == 0:
            return False

        for prime in 2, 3, 5:
            while n % prime == 0:
                n //= prime

        return n == 1
```

1584. 2638.py

Path: LeetCode-main/solutions/2638. Count the Number of K-Free Subsets/2638.py

```
class Solution:
    def countTheNumOfKFreeSubsets(self, nums: list[int], k: int) -> int:
        modToSubset = collections.defaultdict(set)

        for num in nums:
            modToSubset[num % k].add(num)

        prevNum = -k
        skip = 0
        pick = 0

        for subset in modToSubset.values():
            for num in sorted(subset):
                skip, pick = (skip + pick,
                             1 + skip + (0 if num - prevNum == k else pick))
                prevNum = num

        return 1 + skip + pick
```

1585. 2639.py

Path: LeetCode-main/solutions/2639. Find the Width of Columns of a Grid/2639.py

```
class Solution:
    def findColumnWidth(self, grid: list[list[int]]) -> list[int]:
        return [max(map(len, map(str, col))) for col in zip(*grid)]
```

1586. 264.py

Path: LeetCode-main/solutions/264. Ugly Number II/264.py

```
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        nums = [1]
        i2 = 0
        i3 = 0
        i5 = 0

        while len(nums) < n:
            next2 = nums[i2] * 2
            next3 = nums[i3] * 3
            next5 = nums[i5] * 5
            next = min(next2, next3, next5)
            if next == next2:
                i2 += 1
            if next == next3:
                i3 += 1
            if next == next5:
                i5 += 1
            nums.append(next)

        return nums[-1]
```

1587. 2640.py

Path: LeetCode-main/solutions/2640. Find the Score of All Prefixes of an Array/2640.py

```
class Solution:
    def findPrefixScore(self, nums: list[int]) -> list[int]:
        conver = []
        mx = 0

        for num in nums:
            mx = max(mx, num)
            conver.append(num + mx)

        return itertools.accumulate(conver)
```

1588. 2641.py

Path: LeetCode-main/solutions/2641. Cousins in Binary Tree II/2641.py

```

class Solution:
    def replaceValueInTree(self, root: TreeNode | None) -> TreeNode | None:
        levelSums = []

        def dfs(root: TreeNode | None, level: int) -> None:
            if not root:
                return
            if len(levelSums) == level:
                levelSums.append(0)
            levelSums[level] += root.val
            dfs(root.left, level + 1)
            dfs(root.right, level + 1)

        def replace(
            root: TreeNode | None,
            level: int, curr: TreeNode | None,
        ) -> TreeNode | None:
            nextLevel = level + 1
            nextLevelCousinsSum = (
                (levelSums[nextLevel] if nextLevel < len(levelSums) else 0) -
                (root.left.val if root.left else 0) -
                (root.right.val if root.right else 0))
            if root.left:
                curr.left = TreeNode(nextLevelCousinsSum)
                replace(root.left, level + 1, curr.left)
            if root.right:
                curr.right = TreeNode(nextLevelCousinsSum)
                replace(root.right, level + 1, curr.right)
            return curr

        dfs(root, 0)
        return replace(root, 0, TreeNode(0))

```

1589. 2642.py

Path: LeetCode-main/solutions/2642. Design Graph With Shortest Path Calculator/2642.py

```

class Graph:
    def __init__(self, n: int, edges: list[list[int]]):
        self.graph = [[] for _ in range(n)]
        for edge in edges:
            self.addEdge(edge)

    def addEdge(self, edge: list[int]):
        u, v, w = edge
        self.graph[u].append((v, w))

    def shortestPath(self, node1: int, node2: int) -> int:
        dist = [math.inf] * len(self.graph)

        dist[node1] = 0
        minHeap = [(dist[node1], node1)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if u == node2:
                return d
            for v, w in self.graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        return -1

```

1590. 2643.py

Path: LeetCode-main/solutions/2643. Row With Maximum Ones/2643.py

```

class Solution:
    def rowAndMaximumOnes(self, mat: list[list[int]]) -> list[int]:
        ans = [0, 0]

        for i, row in enumerate(mat):
            ones = row.count(1)
            if ones > ans[1]:
                ans[0] = i
                ans[1] = ones

        return ans

```

1591. 2644.py

Path: LeetCode-main/solutions/2644. Find the Maximum Divisibility Score/2644.py

```
class Solution:
    def maxDivScore(self, nums: list[int], divisors: list[int]) -> int:
        ans = -1
        maxScore = -1

        for divisor in divisors:
            score = sum([1 for num in nums if num % divisor == 0])
            if score > maxScore:
                ans = divisor
                maxScore = score
            elif score == maxScore:
                ans = min(ans, divisor)

        return ans
```

1592. 2645.py

Path: LeetCode-main/solutions/2645. Minimum Additions to Make Valid String/2645.py

```
class Solution:
    def addMinimum(self, word: str) -> int:
        letters = ['a', 'b', 'c']
        ans = 0
        i = 0

        while i < len(word):
            for c in letters:
                if i < len(word) and word[i] == c:
                    i += 1
            else:
                ans += 1

        return ans
```

1593. 2646.py

Path: LeetCode-main/solutions/2646. Minimize the Total Price of the Trips/2646.py

```

class Solution:
    def minimumTotalPrice(self, n: int, edges: list[list[int]], price: list[int],
                        trips: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        # count[i] := the number of times i is traversed
        count = [0] * n

        def dfsCount(u: int, prev: int, end: int, path: list[int]) -> None:
            path.append(u)
            if u == end:
                for i in path:
                    count[i] += 1
                return
            for v in graph[u]:
                if v != prev:
                    dfsCount(v, u, end, path)
            path.pop()

        for start, end in trips:
            dfsCount(start, -1, end, [])

        @functools.lru_cache(None)
        def dfs(u: int, prev: int, parentHalved: bool) -> int:
            """
            Returns the minimum price sum for the i-th node, where its parent is
            halved parent or not halved not.
            """
            sumWithFullNode = price[u] * count[u] + sum(dfs(v, u, False)
                                                         for v in graph[u]
                                                         if v != prev)

            if parentHalved: # Can't halve this node if its parent was halved.
                return sumWithFullNode

            sumWithHalvedNode = (price[u] // 2) * count[u] + sum(dfs(v, u, True)
                                                                  for v in graph[u]
                                                                  if v != prev)

            return min(sumWithFullNode, sumWithHalvedNode)

        return dfs(0, -1, False)

```

1594. 2647.py

Path: LeetCode-main/solutions/2647. Color the Triangle Red/2647.py

```

class Solution:
    def colorRed(self, n: int) -> list[list[int]]:
        ans = []
        tipSize = n % 4

        # The tip of the triangle is always painted red.
        if tipSize >= 1:
            ans.append([1, 1])

        # Pamost right and most left elements at the following rows.
        for i in range(2, tipSize + 1):
            ans.append([i, 1])
            ans.append([i, 2 * i - 1])

        # Pa4-row chunks.
        for i in range(tipSize + 1, n, 4):
            # Fill the first row of the chunk.
            ans.append([i, 1])
            # Fill the second row.
            for j in range(1, i + 1):
                ans.append([i + 1, 2 * j + 1])
            # Fill the third row.
            ans.append([i + 2, 2])
            # Fill the fourth row.
            for j in range(i + 2 + 1):
                ans.append([i + 3, 2 * j + 1])

        return ans

```

1595. 265.py

Path: LeetCode-main/solutions/265. Paint House II/265.py

```

class Solution:
    def minCostII(self, costs: list[list[int]]) -> int:
        prevIndex = -1 # the previous minimum index
        prevMin1 = 0 # the minimum cost so far
        prevMin2 = 0 # the second minimum cost so far

        for cost in costs: # O(n)
            # the painted index that will achieve the minimum cost after painting the
            # current house
            index = -1
            # the minimum cost after painting the current house
            min1 = math.inf
            # the second minimum cost after painting the current house
            min2 = math.inf
            for i, cst in enumerate(cost): # O(k)
                theCost = cst + (prevMin2 if i == prevIndex else prevMin1)
                if theCost < min1:
                    index = i
                    min2 = min1
                    min1 = theCost
                elif theCost < min2: # min1 <= theCost < min2
                    min2 = theCost

            prevIndex = index
            prevMin1 = min1
            prevMin2 = min2

        return prevMin1

```

1596. 2651.py

Path: LeetCode-main/solutions/2651. Calculate Delayed Arrival Time/2651.py

```
class Solution:
    def findDelayedArrivalTime(self, arrivalTime: int, delayedTime: int) -> int:
        return (arrivalTime + delayedTime) % 24
```

1597. 2652-2.py

Path: LeetCode-main/solutions/2652. Sum Multiples/2652-2.py

```
class Solution:
    def sumOfMultiples(self, n: int) -> int:
        def sumOfMultiples(value: int) -> int:
            """Returns the sum of multiples of value in [1, n]."""
            lo = value
            hi = (n // value) * value
            count = (hi - lo) // value + 1
            return (lo + hi) * count // 2

        return (sumOfMultiples(3) + sumOfMultiples(5) + sumOfMultiples(7) -
                (sumOfMultiples(15) + sumOfMultiples(21) + sumOfMultiples(35)) +
                sumOfMultiples(105))
```

1598. 2652.py

Path: LeetCode-main/solutions/2652. Sum Multiples/2652.py

```
class Solution:
    def sumOfMultiples(self, n: int) -> int:
        ans = 0
        for i in range(1, n + 1):
            if i % 3 == 0 or i % 5 == 0 or i % 7 == 0:
                ans += i
        return ans
```

1599. 2653.py

Path: LeetCode-main/solutions/2653. Sliding Subarray Beauty/2653.py

```

class Solution:
    def getSubarrayBeauty(self, nums: list[int], k: int, x: int) -> list[int]:
        ans = []
        count = [0] * 50 # count[i] := the frequency of (i + 50)

        for i, num in enumerate(nums):
            if num < 0:
                count[num + 50] += 1
            if i - k >= 0 and nums[i - k] < 0:
                count[nums[i - k] + 50] -= 1
            if i + 1 >= k:
                ans.append(self._getXthSmallestNum(count, x))

        return ans

    def _getXthSmallestNum(self, count: list[int], x: int) -> int:
        prefix = 0
        for i in range(50):
            prefix += count[i]
            if prefix >= x:
                return i - 50
        return 0

```

1600. 2654.py

Path: LeetCode-main/solutions/2654. Minimum Number of Operations to Make All Array Elements Equal to 1/2654.py

```

class Solution:
    def minOperations(self, nums: list[int]) -> int:
        n = len(nums)
        ones = nums.count(1)
        if ones > 0:
            return n - ones

        # the minimum operations to make the shortest subarray with a gcd == 1
        minOps = math.inf

        for i, g in enumerate(nums):
            for j in range(i + 1, n):
                g = math.gcd(g, nums[j])
                if g == 1: # gcd(nums[i..j]) == 1
                    minOps = min(minOps, j - i)
                    break

        # After making the shortest subarray with `minOps`, need additional n - 1
        # operations to make the other numbers to 1.
        return -1 if minOps == math.inf else minOps + n - 1

```

1601. 2655.py

Path: LeetCode-main/solutions/2655. Find Maximal Uncovered Ranges/2655.py

```
class Solution:
    def findMaximalUncoveredRanges(self, n: int, ranges: list[list[int]]) -> list[list[int]]:
        ans = []
        start = 0

        for l, r in sorted(ranges):
            if start < l:
                ans.append([start, l - 1])
            if start <= r:
                start = r + 1

        if start < n:
            ans.append([start, n - 1])

        return ans
```

1602. 2656.py

Path: LeetCode-main/solutions/2656. Maximum Sum With Exactly K Elements/2656.py

```
class Solution:
    def maximizeSum(self, nums: list[int], k: int) -> int:
        return max(nums) * k + k * (k - 1) // 2
```

1603. 2657.py

Path: LeetCode-main/solutions/2657. Find the Prefix Common Array of Two Arrays/2657.py

```
class Solution:
    def findThePrefixCommonArray(self, A: list[int], B: list[int]) -> list[int]:
        n = len(A)
        prefixCommon = 0
        ans = []
        count = [0] * (n + 1)

        for a, b in zip(A, B):
            count[a] += 1
            if count[a] == 2:
                prefixCommon += 1
            count[b] += 1
            if count[b] == 2:
                prefixCommon += 1
            ans.append(prefixCommon)

        return ans
```

1604. 2658.py

Path: LeetCode-main/solutions/2658. Maximum Number of Fish in a Grid/2658.py

```

class Solution:
    def findMaxFish(self, grid: list[list[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return 0
            if grid[i][j] == 0:
                return 0
            caughtFish = grid[i][j]
            grid[i][j] = 0 # Mark 0 as visited
            return (caughtFish +
                    dfs(i + 1, j) + dfs(i - 1, j) +
                    dfs(i, j + 1) + dfs(i, j - 1))

        return max(dfs(i, j)
                   for i in range(len(grid))
                   for j in range(len(grid[0])))

```

1605. 2659.py

Path: LeetCode-main/solutions/2659. Make Array Empty/2659.py

```

class Solution:
    def countOperationsToEmptyArray(self, nums: list[int]) -> int:
        n = len(nums)
        ans = n
        numToIndex = {}

        for i, num in enumerate(nums):
            numToIndex[num] = i

        nums.sort()

        for i in range(1, n):
            # On the i-th step we've already removed the i - 1 smallest numbers and
            # can ignore them. If an element nums[i] has smaller index in origin
            # array than nums[i - 1], we should rotate the whole left array n - i
            # times to set nums[i] element on the first position.
            if numToIndex[nums[i]] < numToIndex[nums[i - 1]]:
                ans += n - i

        return ans

```

1606. 266.py

Path: LeetCode-main/solutions/266. Palindrome Permutation/266.py

```

class Solution:
    def canPermutePalindrome(self, s: str) -> bool:
        seen = set()

        for c in s:
            if c in seen:
                seen.remove(c)
            else:
                seen.add(c)

        return len(seen) <= 1

```

1607. 2660.py

Path: LeetCode-main/solutions/2660. Determine the Winner of a Bowling Game/2660.py

```
class Solution:
    def isWinner(self, player1: list[int], player2: list[int]) -> int:
        def getScore(player: list[int]) -> int:
            INVALID = -3
            score = 0
            last10 = INVALID
            for i, p in enumerate(player):
                score += p if i - last10 > 2 else p * 2
                if p == 10:
                    last10 = i
            return score

        score1 = getScore(player1)
        score2 = getScore(player2)
        if score1 > score2:
            return 1
        if score2 > score1:
            return 2
        return 0
```

1608. 2661.py

Path: LeetCode-main/solutions/2661. First Completely Painted Row or Column/2661.py

```
class Solution:
    def firstCompleteIndex(self, arr: list[int], mat: list[list[int]]) -> int:
        m = len(mat)
        n = len(mat[0])
        # rows[i] := the number of painted grid in the i-th row
        rows = [0] * m
        # cols[j] := the number of painted grid in the j-th column
        cols = [0] * n
        # numToRow[num] := the i-th row of `num` in `mat`
        numToRow = [0] * (m * n + 1)
        # numToCol[num] := the j-th column of `num` in `mat`
        numToCol = [0] * (m * n + 1)

        for i, row in enumerate(mat):
            for j, num in enumerate(row):
                numToRow[num] = i
                numToCol[num] = j

        for i, a in enumerate(arr):
            rows[numToRow[a]] += 1
            if rows[numToRow[a]] == n:
                return i
            cols[numToCol[a]] += 1
            if cols[numToCol[a]] == m:
                return i
```

1609. 2662.py

Path: LeetCode-main/solutions/2662. Minimum Cost of a Path With Special Roads/2662.py

```

class Solution:
    def minimumCost(
        self,
        start: list[int],
        target: list[int],
        specialRoads: list[list[int]],
    ) -> int:
        return self.dijkstra(specialRoads, *start, *target)

    def dijkstra(
        self,
        specialRoads: list[list[int]],
        srcX: int,
        srcY: int,
        dstX: int,
        dstY: int,
    ) -> int:
        n = len(specialRoads)
        # dist[i] := the minimum distance of (srcX, srcY) to specialRoads[i](x2, y2)
        dist = [math.inf] * n
        minHeap = [] # (d, u), where u := the i-th specialRoads

        # (srcX, srcY) -> (x1, y1) to cost -> (x2, y2)
        for u, (x1, y1, _, _, cost) in enumerate(specialRoads):
            d = abs(x1 - srcX) + abs(y1 - srcY) + cost
            dist[u] = d
            heapq.heappush(minHeap, (dist[u], u))

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            _, _, ux2, uy2, _ = specialRoads[u]
            for v in range(n):
                if v == u:
                    continue
                vx1, vy1, _, _, vcost = specialRoads[v]
                # (ux2, uy2) -> (vx1, vy1) to vcost -> (vx2, vy2)
                newDist = d + abs(vx1 - ux2) + abs(vy1 - uy2) + vcost
                if newDist < dist[v]:
                    dist[v] = newDist
                    heapq.heappush(minHeap, (dist[v], v))

        ans = abs(dstX - srcX) + abs(dstY - srcY)
        for u in range(n):
            _, _, x2, y2, _ = specialRoads[u]
            # (srcX, srcY) -> (x2, y2) -> (dstX, dstY).
            ans = min(ans, dist[u] + abs(dstX - x2) + abs(dstY - y2))

        return ans

```

1610. 2663.py

Path: LeetCode-main/solutions/2663. Lexicographically Smallest Beautiful String/2663.py

```

class Solution:
    def smallestBeautifulString(self, s: str, k: int) -> str:
        chars = list(s)

        for i in reversed(range(len(chars))):
            chars[i] = chr(ord(chars[i]) + 1)
            while self._containsPalindrome(chars, i):
                chars[i] = chr(ord(chars[i]) + 1)
            if chars[i] < chr(ord('a') + k):
                # If s[i] is among the first k letters, then change the letters after
                # s[i] to the smallest ones that don't form any palindrome substring.
                return self._changeSuffix(chars, i + 1)

        return ''

    def _containsPalindrome(self, chars: list[str], i: int) -> bool:
        """Returns True if chars[0..i] contains palindrome."""
        return ((i > 0 and chars[i] == chars[i - 1]) or
                (i > 1 and chars[i] == chars[i - 2]))

    def _changeSuffix(self, chars: list[str], i: int) -> str:
        """
        Returns a string, where replacing sb[i..n) with the smallest possible
        letters don't form any palindrome substring.
        """
        for j in range(i, len(chars)):
            chars[j] = 'a'
            while self._containsPalindrome(chars, j):
                chars[j] = chr(ord(chars[j]) + 1)
        return ''.join(chars)

```

1611. 2664.py

Path: LeetCode-main/solutions/2664. The Knight's Tour/2664.py

```

class Solution:
    def tourOfKnight(self, m: int, n: int, r: int, c: int) -> list[list[int]]:
        DIRS = ((1, 2), (2, 1), (2, -1), (1, -2),
                (-1, -2), (-2, -1), (-2, 1), (-1, 2))
        ans = [[-1] * n for _ in range(m)]

        def dfs(i: int, j: int, step: int) -> bool:
            if step == m * n:
                return True
            if i < 0 or i >= m or j < 0 or j >= n:
                return False
            if ans[i][j] != -1:
                return False
            ans[i][j] = step
            for dx, dy in DIRS:
                if dfs(i + dx, j + dy, step + 1):
                    return True
            ans[i][j] = -1
            return False

        dfs(r, c, 0)
        return ans

```

1612. 267.py

Path: LeetCode-main/solutions/267. Palindrome Permutation II/267.py

```
class Solution:
    def generatePalindromes(self, s: str) -> list[str]:
        count = collections.Counter(s)

        # Count odd ones.
        odd = sum(value & 1 for value in count.values())

        # Can't form any palindrome.
        if odd > 1:
            return []

        ans = []
        candidates = []
        mid = ''

        # Get the mid and the candidates characters.
        for key, value in count.items():
            if value % 2 == 1:
                mid += key
            for _ in range(value // 2):
                candidates.append(key)

        def dfs(used: list[bool], path: list[str]) -> None:
            """Generates all the unique palindromes from the candidates."""
            if len(path) == len(candidates):
                ans.append(''.join(path) + mid + ''.join(reversed(path)))
                return

            for i, candidate in enumerate(candidates):
                if used[i]:
                    continue
                if i > 0 and candidate == candidates[i - 1] and not used[i - 1]:
                    continue
                used[i] = True
                path.append(candidate)
                dfs(used, path)
                path.pop()
                used[i] = False

        # Backtrack to generate the ans strings.
        dfs([False] * len(candidates), [])
        return ans
```

1613. 2670.py

Path: LeetCode-main/solutions/2670. Find the Distinct Difference Array/2670.py


```

class Solution:
    def distinctDifferenceArray(self, nums: list[int]) -> list[int]:
        MAX = 50
        ans = []
        prefixCount = [0] * (MAX + 1)
        suffixCount = [0] * (MAX + 1)
        distinctPrefix = 0
        distinctSuffix = 0

        for num in nums:
            if suffixCount[num] == 0:
                distinctSuffix += 1
            suffixCount[num] += 1

        for num in nums:
            if prefixCount[num] == 0:
                distinctPrefix += 1
            prefixCount[num] += 1
            if suffixCount[num] == 1:
                distinctSuffix -= 1
            suffixCount[num] -= 1
            ans.append(distinctPrefix - distinctSuffix)

        return ans

```

1614. 2671.py

Path: LeetCode-main/solutions/2671. Frequency Tracker/2671.py

```

class FrequencyTracker:
    def __init__(self):
        self.count = collections.Counter()
        self.freqCount = collections.Counter()

    def add(self, number: int) -> None:
        if self.count[number] > 0:
            self.freqCount[self.count[number]] -= 1
        self.count[number] += 1
        self.freqCount[self.count[number]] += 1

    def deleteOne(self, number: int) -> None:
        if self.count[number] == 0:
            return
        self.freqCount[self.count[number]] -= 1
        self.count[number] -= 1
        self.freqCount[self.count[number]] += 1

    def hasFrequency(self, frequency: int) -> bool:
        return self.freqCount[frequency] > 0

```

1615. 2672.py

Path: LeetCode-main/solutions/2672. Number of Adjacent Elements With the Same Color/2672.py

```

class Solution:
    def colorTheArray(self, n: int, queries: list[list[int]]) -> list[int]:
        ans = []
        arr = [0] * n
        sameColors = 0

        for i, color in queries:
            if i + 1 < n:
                if arr[i + 1] > 0 and arr[i + 1] == arr[i]:
                    sameColors -= 1
                if arr[i + 1] == color:
                    sameColors += 1
            if i > 0:
                if arr[i - 1] > 0 and arr[i - 1] == arr[i]:
                    sameColors -= 1
                if arr[i - 1] == color:
                    sameColors += 1
            arr[i] = color
            ans.append(sameColors)

        return ans

```

1616. 2673.py

Path: LeetCode-main/solutions/2673. Make Costs of Paths Equal in a Binary Tree/2673.py

```

class Solution:
    def minIncrements(self, n: int, cost: list[int]) -> int:
        ans = 0

        for i in range(n // 2 - 1, -1, -1):
            l = i * 2 + 1
            r = i * 2 + 2
            ans += abs(cost[l] - cost[r])
            # Record the information in the parent from the children. So, there's need to actually
            # update the values in the children.
            cost[i] += max(cost[l], cost[r])

        return ans

```

1617. 2674.py

Path: LeetCode-main/solutions/2674. Split a Circular Linked List/2674.py

```

class Solution:
    def splitCircularLinkedList(self, list: ListNode | None) -> list[ListNode | None]:
        slow = list
        fast = list

        # Point `slow` to the last node in the first half.
        while fast.next != list and fast.next.next != list:
            slow = slow.next
            fast = fast.next.next

        # Circle back the second half.
        secondHead = slow.next
        if fast.next == list:
            fast.next = secondHead
        else:
            fast.next.next = secondHead

        # Circle back the first half.
        slow.next = list

        return [list, secondHead]

```

1618. 2678.py

Path: LeetCode-main/solutions/2678. Number of Senior Citizens/2678.py

```

class Solution:
    def countSeniors(self, details: list[str]) -> int:
        return sum(int(detail[11:13]) > 60 for detail in details)

```

1619. 2679.py

Path: LeetCode-main/solutions/2679. Sum in a Matrix/2679.py

```

class Solution:
    def matrixSum(self, nums: list[list[int]]) -> int:
        for row in nums:
            row.sort()

        return sum(max(col) for col in zip(*nums))

```

1620. 268.py

Path: LeetCode-main/solutions/268. Missing Number/268.py

```

class Solution:
    def missingNumber(self, nums: list[int]) -> int:
        ans = len(nums)

        for i, num in enumerate(nums):
            ans ^= i ^ num

        return ans

```

1621. 2680.py

Path: LeetCode-main/solutions/2680. Maximum OR/2680.py

```
class Solution:
    def maximumOr(self, nums: list[int], k: int) -> int:
        n = len(nums)
        # prefix[i] := nums[0] | nums[1] | ... | nums[i - 1]
        prefix = [0] * n
        # suffix[i] := nums[i + 1] | nums[i + 2] | ... | nums[n - 1]
        suffix = [0] * n

        for i in range(1, n):
            prefix[i] = prefix[i - 1] | nums[i - 1]

        for i in range(n - 2, -1, -1):
            suffix[i] = suffix[i + 1] | nums[i + 1]

        # For each num, greedily shift it left by k bits.
        return max(p | num << k | s for num, p, s in zip(nums, prefix, suffix))
```

1622. 2681.py

Path: LeetCode-main/solutions/2681. Power of Heroes/2681.py

```
class Solution:
    def sumOfPower(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        ans = 0
        summ = 0

        for num in sorted(nums):
            ans += (num + summ) * num**2
            ans %= MOD
            summ = (summ * 2 + num) % MOD

        return ans
```

1623. 2682.py

Path: LeetCode-main/solutions/2682. Find the Losers of the Circular Game/2682.py

```
class Solution:
    def circularGameLosers(self, n: int, k: int) -> list[int]:
        seen = [False] * n
        friendIndex = 0
        turn = 1

        while not seen[friendIndex]:
            seen[friendIndex] = True
            friendIndex += turn * k
            friendIndex %= n
            turn += 1

        return [friendIndex + 1
                for friendIndex in range(n)
                if not seen[friendIndex]]
```

1624. 2683.py

Path: LeetCode-main/solutions/2683. Neighboring Bitwise XOR/2683.py

```
class Solution:
    def doesValidArrayExist(self, derived: list[int]) -> bool:
        return functools.reduce(operator.xor, derived) == 0
```

1625. 2684.py

Path: LeetCode-main/solutions/2684. Maximum Number of Moves in a Grid/2684.py

```
class Solution:
    def maxMoves(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        # dp[i][j] := the maximum number of moves you can perform from (i, j)
        dp = [[0] * n for _ in range(m)]

        for j in range(n - 2, -1, -1):
            for i in range(m):
                if grid[i][j + 1] > grid[i][j]:
                    dp[i][j] = 1 + dp[i][j + 1]
                if i > 0 and grid[i - 1][j + 1] > grid[i][j]:
                    dp[i][j] = max(dp[i][j], 1 + dp[i - 1][j + 1])
                if i + 1 < m and grid[i + 1][j + 1] > grid[i][j]:
                    dp[i][j] = max(dp[i][j], 1 + dp[i + 1][j + 1])

        return max(dp[i][0] for i in range(m))
```

1626. 2685.py

Path: LeetCode-main/solutions/2685. Count the Number of Complete Components/2685.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n
        self.nodeCount = [1] * n
        self.edgeCount = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        self.edgeCount[i] += 1
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
            self.edgeCount[j] += self.edgeCount[i]
            self.nodeCount[j] += self.nodeCount[i]
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
            self.edgeCount[i] += self.edgeCount[j]
            self.nodeCount[i] += self.nodeCount[j]
        else:
            self.id[i] = j
            self.edgeCount[j] += self.edgeCount[i]
            self.nodeCount[j] += self.nodeCount[i]
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

    def isComplete(self, u):
        return self.nodeCount[u] * (self.nodeCount[u] - 1) // 2 == self.edgeCount[u]

class Solution:
    def countCompleteComponents(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        uf = UnionFind(n)
        parents = set()

        for u, v in edges:
            uf.unionByRank(u, v)

        for i in range(n):
            parent = uf.find(i)
            if parent not in parents and uf.isComplete(parent):
                ans += 1
                parents.add(parent)

        return ans

```

1627. 2589-2.py

Path: LeetCode-main/solutions/2689. Extract Kth Character From The Rope Tree/2589-2.py

```
class Solution:
    def getKthCharacter(self, root: object | None, k: int) -> str:
        """type root: RopeTreeNode | None"""
        while root.len > 0:
            leftLen = (0 if not root.left
                       else max(root.left.len, len(root.left.val)))
            if leftLen >= k:
                root = root.left
            else:
                root = root.right
                k -= leftLen
        return root.val[k - 1]
```

1628. 2689.py

Path: LeetCode-main/solutions/2689. Extract Kth Character From The Rope Tree/2689.py

```
class Solution:
    def getKthCharacter(self, root: object | None, k: int) -> str:
        """type root: RopeTreeNode | None"""
        if root.len == 0:
            return root.val[k - 1]
        leftLen = (0 if not root.left
                   else max(root.left.len, len(root.left.val)))
        if leftLen >= k:
            return self.getKthCharacter(root.left, k)
        return self.getKthCharacter(root.right, k - leftLen)
```

1629. 269.py

Path: LeetCode-main/solutions/269. Alien Dictionary/269.py

```

class Solution:
    def alienOrder(self, words: list[str]) -> str:
        graph = {}
        inDegrees = [0] * 26

        self._buildGraph(graph, words, inDegrees)
        return self._topology(graph, inDegrees)

    def _buildGraph(
        self,
        graph: dict[str, set[str]],
        words: list[str],
        inDegrees: list[int],
    ) -> None:
        # Create a node for each character in each word.
        for word in words:
            for c in word:
                if c not in graph:
                    graph[c] = set()

        for first, second in zip(words, words[1:]):
            length = min(len(first), len(second))
            for j in range(length):
                u = first[j]
                v = second[j]
                if u != v:
                    if v not in graph[u]:
                        graph[u].add(v)
                        inDegrees[ord(v) - ord('a')] += 1
                    break # The order of characters after this are meaningless.
            # First = 'ab', second = 'a' . invalid
            if j == length - 1 and len(first) > len(second):
                graph.clear()
            return

    def _topology(self, graph: dict[str, set[str]], inDegrees: list[int]) -> str:
        s = ''
        q = collections.deque()

        for c in graph:
            if inDegrees[ord(c) - ord('a')] == 0:
                q.append(c)

        while q:
            u = q.pop()
            s += u
            for v in graph[u]:
                inDegrees[ord(v) - ord('a')] -= 1
                if inDegrees[ord(v) - ord('a')] == 0:
                    q.append(v)

        # Words = ['z', 'x', 'y', 'x']
        return s if len(s) == len(graph) else ''

```

1630. 2696.py

Path: LeetCode-main/solutions/2696. Minimum String Length After Removing Substrings/2696.py


```
class Solution:
    def minLength(self, s: str) -> int:
        stack = []

        def match(c: str) -> bool:
            return stack and stack[-1] == c

        for c in s:
            if c == 'B' and match('A'):
                stack.pop()
            elif c == 'D' and match('C'):
                stack.pop()
            else:
                stack.append(c)

        return len(stack)
```

1631. 2697.py

Path: LeetCode-main/solutions/2697. Lexicographically Smallest Palindrome/2697.py

```
class Solution:
    def makeSmallestPalindrome(self, s: str) -> str:
        chars = list(s)
        i = 0
        j = len(s) - 1

        while i < j:
            minChar = min(chars[i], chars[j])
            chars[i] = minChar
            chars[j] = minChar
            i += 1
            j -= 1

        return ''.join(chars)
```

1632. 2698.py

Path: LeetCode-main/solutions/2698. Find the Punishment Number of an Integer/2698.py

```
class Solution:
    def punishmentNumber(self, n: int) -> int:
        def isPossible(
            accumulate: int, running: int, numChars: list[str],
            s: int, target: int) -> bool:
            """
            Returns True if the sum of any split of `numChars` equals to the target.
            """
            if s == len(numChars):
                return target == accumulate + running
            d = int(numChars[s])
            return (
                # Keep growing `running`.
                isPossible(accumulate, running * 10 + d, numChars, s + 1, target) or
                # Start a new `running`.
                isPossible(accumulate + running, d, numChars, s + 1, target)
            )

        return sum(i * i
                    for i in range(1, n + 1)
                    if isPossible(0, 0, str(i * i), 0, i))
```

1633. 2699.py

Path: LeetCode-main/solutions/2699. Modify Graph Edge Weights/2699.py

```

class Solution:
    def modifiedGraphEdges(self, n: int, edges: list[list[int]], source: int, destination: int, target: int) -> list[list[int]]:
        MAX = 2_000_000_000
        graph = [[] for _ in range(n)]

        for u, v, w in edges:
            if w == -1:
                continue
            graph[u].append((v, w))
            graph[v].append((u, w))

        distToDestination = self._dijkstra(graph, source, destination)
        if distToDestination < target:
            return []
        if distToDestination == target:
            # Change the weights of negative edges to an impossible value.
            for edge in edges:
                if edge[2] == -1:
                    edge[2] = MAX
            return edges

        for i, (u, v, w) in enumerate(edges):
            if w != -1:
                continue
            edges[i][2] = 1
            graph[u].append((v, 1))
            graph[v].append((u, 1))
            distToDestination = self._dijkstra(graph, source, destination)
            if distToDestination <= target:
                edges[i][2] += target - distToDestination
                # Change the weights of negative edges to an impossible value.
                for j in range(i + 1, len(edges)):
                    if edges[j][2] == -1:
                        edges[j][2] = MAX
                return edges

        return []

    def _dijkstra(self, graph: list[list[int]], src: int, dst: int) -> int:
        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        return dist[dst]

```

1634. 27.py

Path: LeetCode-main/solutions/27. Remove Element/27.py

```
class Solution:
    def removeElement(self, nums: list[int], val: int) -> int:
        i = 0

        for num in nums:
            if num != val:
                nums[i] = num
                i += 1

        return i
```

1635. 270.py

Path: LeetCode-main/solutions/270. Closest Binary Search Tree Value/270.py

```
class Solution:
    def closestValue(self, root: TreeNode | None, target: float) -> int:
        # If target < root.val, search the left subtree.
        if target < root.val and root.left:
            left = self.closestValue(root.left, target)
            if abs(left - target) <= abs(root.val - target):
                return left

        # If target > root.val, search the right subtree.
        if target > root.val and root.right:
            right = self.closestValue(root.right, target)
            if abs(right - target) < abs(root.val - target):
                return right

        return root.val
```

1636. 2702.py

Path: LeetCode-main/solutions/2702. Minimum Operations to Make Numbers Non-positive/2702.py

```
class Solution:
    def minOperations(self, nums: list[int], x: int, y: int) -> int:
        def isPossible(m: int) -> bool:
            """
            Returns True if it's possible to make all `nums` <= 0 using m operations.
            """
            # If we want m operations, first decrease all the numbers by y * m. Then
            # we have m operations to select indices to decrease them by x - y.
            return sum(max(0, math.ceil((num - y * m) / (x - y)))
                       for num in nums) <= m

        return bisect.bisect_left(range(max(nums)), True, key=isPossible)
```

1637. 2706.py

Path: LeetCode-main/solutions/2706. Buy Two Chocolates/2706.py

```
class Solution:
    def buyChoco(self, prices: list[int], money: int) -> int:
        min1 = math.inf
        min2 = math.inf

        for price in prices:
            if price <= min1:
                min2 = min1
                min1 = price
            elif price < min2:
                min2 = price

        minCost = min1 + min2
        return money if minCost > money else money - minCost
```

1638. 2707.py

Path: LeetCode-main/solutions/2707. Extra Characters in a String/2707.py

```
class Solution:
    # Similar to 139. Word Break
    def minExtraChar(self, s: str, dictionary: list[str]) -> int:
        n = len(s)
        dictionarySet = set(dictionary)
        # dp[i] := the minimum extra letters if breaking up s[0..i) optimally
        dp = [0] + [n] * n

        for i in range(1, n + 1):
            for j in range(i):
                if s[j:i] in dictionarySet:
                    dp[i] = min(dp[i], dp[j])
                else:
                    dp[i] = min(dp[i], dp[j] + i - j)

        return dp[n]
```

1639. 2708.py

Path: LeetCode-main/solutions/2708. Maximum Strength of a Group/2708.py

```
class Solution:
    def maxStrength(self, nums: list[int]) -> int:
        posProd = 1
        negProd = 1
        maxNeg = -math.inf
        negCount = 0
        hasPos = False
        hasZero = False

        for num in nums:
            if num > 0:
                posProd *= num
                hasPos = True
            elif num < 0:
                negProd *= num
                maxNeg = max(maxNeg, num)
                negCount += 1
            else: # num == 0
                hasZero = True

        if negCount == 0 and not hasPos:
            return 0
        if negCount % 2 == 0:
            return negProd * posProd
        if negCount >= 3:
            return negProd // maxNeg * posProd
        if hasPos:
            return posProd
        if hasZero:
            return 0
        return maxNeg
```

1640. 2709.py

Path: LeetCode-main/solutions/2709. Greatest Common Divisor Traversal/2709.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.sz = [1] * n

    def unionBySize(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.sz[i] < self.sz[j]:
            self.sz[j] += self.sz[i]
            self.id[i] = j
        else:
            self.sz[i] += self.sz[j]
            self.id[j] = i

    def getSize(self, i: int) -> int:
        return self.sz[i]

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def canTraverseAllPairs(self, nums: list[int]) -> bool:
        n = len(nums)
        mx = max(nums)
        maxPrimeFactor = self._sieveEratosthenes(mx + 1)
        primeToFirstIndex = collections.defaultdict(int)
        uf = UnionFind(n)

        for i, num in enumerate(nums):
            for prime_factor in self._getPrimeFactors(num, maxPrimeFactor):
                if prime_factor in primeToFirstIndex:
                    uf.unionBySize(primeToFirstIndex[prime_factor], i)
                else:
                    primeToFirstIndex[prime_factor] = i

        return any(uf.getSize(i) == n for i in range(n))

    def _sieveEratosthenes(self, n: int) -> list[int]:
        """Gets the minimum prime factor of i, where 1 < i <= n."""
        minPrimeFactors = [i for i in range(n + 1)]
        for i in range(2, int(n**0.5) + 1):
            if minPrimeFactors[i] == i: # `i` is prime.
                for j in range(i * i, n, i):
                    minPrimeFactors[j] = min(minPrimeFactors[j], i)
        return minPrimeFactors

    def _getPrimeFactors(self, num: int, minPrimeFactors: list[int]) -> list[int]:
        primeFactors = []
        while num > 1:
            divisor = minPrimeFactors[num]
            primeFactors.append(divisor)
            while num % divisor == 0:
                num //= divisor
        return primeFactors

```

Path: LeetCode-main/solutions/271. Encode and Decode Strings/271.py

```
class Codec:
    def encode(self, strs: list[str]) -> str:
        """Encodes a list of strings to a single string."""
        return ''.join(str(len(s)) + '/' + s for s in strs)

    def decode(self, s: str) -> list[str]:
        """Decodes a single string to a list of strings."""
        decoded = []

        i = 0
        while i < len(s):
            slash = s.find('/', i)
            length = int(s[i:slash])
            i = slash + length + 1
            decoded.append(s[slash + 1:i])

        return decoded
```

1642. 2710.py

Path: LeetCode-main/solutions/2710. Remove Trailing Zeros From a String/2710.py

```
class Solution:
    def removeTrailingZeros(self, num: str) -> str:
        return num.rstrip('0')
```

1643. 2711.py

Path: LeetCode-main/solutions/2711. Difference of Number of Distinct Values on Diagonals/2711.py


```

class Solution:
    def differenceOfDistinctValues(self, grid: list[list[int]]) -> list[list[int]]:
        m = len(grid)
        n = len(grid[0])
        ans = [[0] * n for _ in range(m)]

        def fillInDiagonal(i: int, j: int) -> None:
            topLeft = set()
            bottomRight = set()

            # Fill in the diagonal from the top-left to the bottom-right.
            while i < len(grid) and j < len(grid[0]):
                ans[i][j] = len(topLeft)
                # Post-addition, so this information can be utilized in subsequent cells.
                topLeft.add(grid[i][j])
                i += 1
                j += 1

            i -= 1
            j -= 1

            # Fill in the diagonal from the bottom-right to the top-left.
            while i >= 0 and j >= 0:
                ans[i][j] = abs(ans[i][j] - len(bottomRight))
                # Post-addition, so this information can be utilized in subsequent cells.
                bottomRight.add(grid[i][j])
                i -= 1
                j -= 1

        for i in range(m):
            fillInDiagonal(i, 0)

        for j in range(1, n):
            fillInDiagonal(0, j)

        return ans

```

1644. 2712.py

Path: LeetCode-main/solutions/2712. Minimum Cost to Make All Characters Equal/2712.py

```

class Solution:
    def minimumCost(self, s: str) -> int:
        n = len(s)
        ans = 0

        for i in range(1, n):
            if s[i] != s[i - 1]:
                # Invert s[0..i - 1] or s[i..n - 1].
                ans += min(i, n - i)

        return ans

```

1645. 2713.py

Path: LeetCode-main/solutions/2713. Maximum Strictly Increasing Cells in a Matrix/2713.py

```
class Solution:
    def maxIncreasingCells(self, mat: list[list[int]]) -> int:
        m = len(mat)
        n = len(mat[0])
        rows = [0] * m # rows[i] := the maximum path length for the i-th row
        cols = [0] * n # cols[j] := the maximum path length for the j-th column
        valToIndices = collections.defaultdict(list)
        # maxPathLength[i][j] := the maximum path length from mat[i][j]
        maxPathLength = [[0] * n for _ in range(m)]
        # Sort all the unique values in the matrix in non-increasing order.
        decreasingSet = set()

        for i in range(m):
            for j in range(n):
                val = mat[i][j]
                valToIndices[val].append((i, j))
                decreasingSet.add(val)

        for val in sorted(decreasingSet, reverse=True):
            for i, j in valToIndices[val]:
                maxPathLength[i][j] = max(rows[i], cols[j]) + 1
            for i, j in valToIndices[val]:
                rows[i] = max(rows[i], maxPathLength[i][j])
                cols[j] = max(cols[j], maxPathLength[i][j])

        return max(max(rows), max(cols))
```

1646. 2714.py

Path: LeetCode-main/solutions/2714. Find Shortest Path with K Hops/2714.py

```

class Solution:
    # Similar to 787. Cheapest Flights Within K Stops
    def shortestPathWithHops(
        self,
        n: int,
        edges: list[list[int]],
        s: int,
        d: int,
        k: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        return self._dijkstra(graph, s, d, k)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        dist = [[math.inf for _ in range(k + 1)] for _ in range(len(graph))]

        dist[src][k] = 0
        minHeap = [(dist[src][k], src, k)] # (d, u, hops)

        while minHeap:
            d, u, hops = heapq.heappop(minHeap)
            if u == dst:
                return d
            if dist[u][hops] < d:
                continue
            for v, w in graph[u]:
                # Go from u -> v with w cost.
                if d + w < dist[v][hops]:
                    dist[v][hops] = d + w
                    heapq.heappush(minHeap, (dist[v][hops], v, hops))
                # Hop from u -> v with 0 cost.
                if hops > 0 and d < dist[v][hops - 1]:
                    dist[v][hops - 1] = d
                    heapq.heappush(minHeap, (dist[v][hops - 1], v, hops - 1))

```

1647. 2716.py

Path: LeetCode-main/solutions/2716. Minimize String Length/2716.py

```

class Solution:
    def minimizedStringLength(self, s: str) -> int:
        return len(set(s))

```

1648. 2717.py

Path: LeetCode-main/solutions/2717. Semi-Ordered Permutation/2717.py

```
class Solution:
    def semiOrderedPermutation(self, nums: list[int]) -> int:
        n = len(nums)
        index1 = nums.index(1)
        indexN = nums.index(n)
        return index1 + (n - 1 - indexN) - int(index1 > indexN)
```

1649. 2718.py

Path: LeetCode-main/solutions/2718. Sum of Matrix After Queries/2718.py

```
class Solution:
    def matrixSumQueries(self, n: int, queries: list[list[int]]) -> int:
        ans = 0
        # seen[0] := row, seen[1] := col
        seen = [[False] * n for _ in range(2)]
        # notSet[0] = row, notSet[1] := col
        notSet = [n] * 2

        # Late queries dominate.
        for type, index, val in reversed(queries):
            if not seen[type][index]:
                ans += val * notSet[type ^ 1]
                seen[type][index] = True
                notSet[type] -= 1

        return ans
```

1650. 272.py

Path: LeetCode-main/solutions/272. Closest Binary Search Tree Value II/272.py

```
class Solution:
    def closestKValues(
        self,
        root: TreeNode | None,
        target: float,
        k: int,
    ) -> list[int]:
        dq = collections.deque()

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return

            inorder(root.left)
            dq.append(root.val)
            inorder(root.right)

        inorder(root)

        while len(dq) > k:
            if abs(dq[0] - target) > abs(dq[-1] - target):
                dq.popleft()
            else:
                dq.pop()

        return list(dq)
```

1651. 2728.py

Path: LeetCode-main/solutions/2728. Count Houses in a Circular Street/2728.py

```
# Definition for a street.
# class Street:
#     def openDoor(self):
#         pass
#     def closeDoor(self):
#         pass
#     def isDoorOpen(self):
#         pass
#     def moveRight(self):
#         pass
#     def moveLeft(self):
#         pass

class Solution:
    def houseCount(self, street: Optional['Street'], k: int) -> int:
        for _ in range(k):
            if street.isDoorOpen():
                street.closeDoor()
                street.moveRight()

        for ans in range(k + 1):
            if street.isDoorOpen():
                return ans
            street.openDoor()
            street.moveRight()
```

1652. 2729.py

Path: LeetCode-main/solutions/2729. Check if The Number is Fascinating/2729.py

```
class Solution:
    def isFascinating(self, n):
        s = str(n) + str(2 * n) + str(3 * n)
        return ''.join(sorted(s)) == '123456789'
```

1653. 273.py

Path: LeetCode-main/solutions/273. Integer to English Words/273.py

```

class Solution:
    def numberToWords(self, num: int) -> str:
        if num == 0:
            return 'Zero'

        belowTwenty = ['', 'One', 'Two', 'Three',
                        'Four', 'Five', 'Six', 'Seven',
                        'Eight', 'Nine', 'Ten', 'Eleven',
                        'Twelve', 'Thirteen', 'Fourteen', 'Fifteen',
                        'Sixteen', 'Seventeen', 'Eighteen', 'Nineteen']
        tens = ['', 'Ten', 'Twenty', 'Thirty', 'Forty',
                'Fifty', 'Sixty', 'Seventy', 'Eighty', 'Ninety']

        def helper(num: int) -> str:
            if num < 20:
                s = belowTwenty[num]
            elif num < 100:
                s = tens[num // 10] + ' ' + belowTwenty[num % 10]
            elif num < 1000:
                s = helper(num // 100) + ' Hundred ' + helper(num % 100)
            elif num < 1000000:
                s = helper(num // 1000) + ' Thousand ' + helper(num % 1000)
            elif num < 1000000000:
                s = helper(num // 1000000) + ' Million ' + helper(num % 1000000)
            else:
                s = helper(num // 1000000000) + ' Billion ' + helper(num % 1000000000)
            return s.strip()

        return helper(num)

```

1654. 2730.py

Path: LeetCode-main/solutions/2730. Find the Longest Semi-Repetitive Substring/2730.py

```

class Solution:
    def longestSemiRepetitiveSubstring(self, s: str) -> int:
        ans = 1
        prevStart = 0
        start = 0

        for i in range(1, len(s)):
            if s[i] == s[i - 1]:
                if prevStart > 0:
                    start = prevStart
                prevStart = i
            ans = max(ans, i - start + 1)

        return ans

```

1655. 2731.py

Path: LeetCode-main/solutions/2731. Movement of Robots/2731.py

```

class Solution:
    def sumDistance(self, nums: list[int], s: str, d: int) -> int:
        MOD = 1_000_000_007
        ans = 0
        prefix = 0
        pos = sorted([num - d if c == 'L' else num + d
                      for num, c in zip(nums, s)])

        for i, p in enumerate(pos):
            ans = ((ans + i * p - prefix) % MOD + MOD) % MOD
            prefix = ((prefix + p) % MOD + MOD) % MOD

        return ans

```

1656. 2732.py

Path: LeetCode-main/solutions/2732. Find a Good Subset of the Matrix/2732.py

```

class Solution:
    def goodSubsetofBinaryMatrix(self, grid: list[list[int]]) -> list[int]:
        MAX_BIT = 30
        maskToIndex = {}

        def getMask(row: list[int]) -> int:
            mask = 0
            for i, num in enumerate(row):
                if num == 1:
                    mask |= 1 << i
            return mask

        for i, row in enumerate(grid):
            mask = getMask(row)
            if mask == 0:
                return [i]
            for prevMask in range(1, MAX_BIT):
                if (mask & prevMask) == 0 and prevMask in maskToIndex:
                    return [maskToIndex[prevMask], i]
            maskToIndex[mask] = i

        return []

```

1657. 2733.py

Path: LeetCode-main/solutions/2733. Neither Minimum nor Maximum/2733.py

```

class Solution:
    def findNonMinOrMax(self, nums: list[int]) -> int:
        return -1 if len(nums) < 3 else sorted(nums[:3])[1]

```

1658. 2734.py

Path: LeetCode-main/solutions/2734. Lexicographically Smallest String After Substring Operation/2734.py

```

class Solution:
    def smallestString(self, s: str) -> str:
        chars = list(s)
        n = len(s)
        i = 0

        while i < n and chars[i] == 'a':
            i += 1
        if i == n:
            chars[-1] = 'z'
            return ''.join(chars)

        while i < n and s[i] != 'a':
            chars[i] = chr(ord(chars[i]) - 1)
            i += 1

        return ''.join(chars)

```

1659. 2735.py

Path: LeetCode-main/solutions/2735. Collecting Chocolates/2735.py

```

class Solution:
    def minCost(self, nums: list[int], x: int) -> int:
        n = len(nums)
        ans = math.inf
        # minCost[i] := the minimum cost to collect the i-th type
        minCost = [math.inf] * n

        for rotate in range(n):
            for i in range(n):
                minCost[i] = min(minCost[i], nums[(i - rotate + n) % n])
            ans = min(ans, sum(minCost) + rotate * x)

        return ans

```

1660. 2736.py

Path: LeetCode-main/solutions/2736. Maximum Sum Queries/2736.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Pair:
    x: int
    y: int

    def __iter__(self):
        yield self.x
        yield self.y

@dataclass(frozen=True)
class IndexedQuery:
    queryIndex: int
    minX: int
    minY: int

    def __iter__(self):

```



```

yield self.queryIndex
yield self.minX
yield self.minY

class Solution:
    def maximumSumQueries(
        self,
        nums1: list[int],
        nums2: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        pairs = sorted([Pair(nums1[i], nums2[i])
                        for i in range(len(nums1))], key=lambda x: x.x, reverse=True)

        ans = [0] * len(queries)
        stack = [] # [(y, x + y)]

        pairsIndex = 0
        for queryIndex, minX, minY in sorted([IndexedQuery(i, query[0], query[1])
                                             for i, query in enumerate(queries)],
                                             key=lambda x: -x.minX):
            while pairsIndex < len(pairs) and pairs[pairsIndex].x >= minX:
                # x + y is a better candidate. Given that x is decreasing, the
                # condition "x + y >= stack[-1][1]" suggests that y is relatively
                # larger, thereby making it a better candidate.
                x, y = pairs[pairsIndex]
                while stack and x + y >= stack[-1][1]:
                    stack.pop()
                if not stack or y > stack[-1][0]:
                    stack.append((y, x + y))
                pairsIndex += 1
            j = self._firstGreaterEqual(stack, minY)
            ans[queryIndex] = -1 if j == len(stack) else stack[j][1]

        return ans

    def _firstGreaterEqual(self, A: list[tuple[int, int]], target: int) -> int:
        l = 0
        r = len(A)
        while l < r:
            m = (l + r) // 2
            if A[m][0] >= target:
                r = m
            else:
                l = m + 1
        return l

```

1661. 2737.py

Path: LeetCode-main/solutions/2737. Find the Closest Marked Node/2737.py

```

class Solution:
    def minimumDistance(
        self,
        n: int,
        edges: list[list[int]],
        s: int,
        marked: list[int],
    ) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))

        dist = self._dijkstra(graph, s)
        ans = min(dist[u] for u in marked)
        return -1 if ans == math.inf else ans

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
    ) -> list[int]:
        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        return dist

```

1662. 2739.py

Path: LeetCode-main/solutions/2739. Total Distance Traveled/2739.py

```

class Solution:
    def distanceTraveled(self, mainTank: int, additionalTank: int) -> int:
        # M M M M M A M M M M A
        # 1 [2 3 4 5] 6 [7 8 9 10] 11
        return (mainTank + min((mainTank - 1) // 4, additionalTank)) * 10

```

1663. 274-2.py

Path: LeetCode-main/solutions/274. H-Index/274-2.py

```
class Solution:
    def hIndex(self, citations: list[int]) -> int:
        n = len(citations)

        citations.sort()

        for i, citation in enumerate(citations):
            if citation >= n - i:
                return n - i

        return 0
```

1664. 274.py

Path: LeetCode-main/solutions/274. H-Index/274.py

```
class Solution:
    def hIndex(self, citations: list[int]) -> int:
        n = len(citations)
        accumulate = 0
        count = [0] * (n + 1)

        for citation in citations:
            count[min(citation, n)] += 1

        # To find the maximum h-index, loop from the back to the front.
        # i := the candidate's h-index
        for i, c in reversed(list(enumerate(count))):
            accumulate += c
            if accumulate >= i:
                return i
```

1665. 2740.py

Path: LeetCode-main/solutions/2740. Find the Value of the Partition/2740.py

```
class Solution:
    def findValueOfPartition(self, nums: list[int]) -> int:
        return min(b - a for a, b in itertools.pairwise(sorted(nums)))
```

1666. 2741.py

Path: LeetCode-main/solutions/2741. Special Permutations/2741.py

```

class Solution:
    def specialPerm(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        maxMask = 1 << len(nums)

        @functools.lru_cache(None)
        def dp(prev: int, mask: int) -> int:
            """
            Returns the number of special permutations, where the previous number is
            nums[i] and `mask` is the bitmask of the used numbers.
            """
            if mask == maxMask - 1:
                return 1

            res = 0

            for i, num in enumerate(nums):
                if mask >> i & 1:
                    continue
                if num % nums[prev] == 0 or nums[prev] % num == 0:
                    res += dp(i, mask | 1 << i)
                    res %= MOD

            return res

        return sum(dp(i, 1 << i)
                    for i in range(len(nums))) % MOD

```

1667. 2742-2.py

Path: LeetCode-main/solutions/2742. Painting the Walls/2742-2.py

```

class Solution:
    def paintWalls(self, cost: list[int], time: list[int]) -> int:
        MAX = 500_000_000
        n = len(cost)
        # dp[i] := the minimum cost to paint i walls by the painters so far
        dp = [0] + [MAX] * n

        for c, t in zip(cost, time):
            for walls in range(n, 0, -1):
                dp[walls] = min(dp[walls], dp[max(walls - t - 1, 0)] + c)

        return dp[n]

```

1668. 2742.py

Path: LeetCode-main/solutions/2742. Painting the Walls/2742.py

```

class Solution:
    def paintWalls(self, cost: list[int], time: list[int]) -> int:
        n = len(cost)

        @functools.lru_cache(None)
        def dp(i: int, walls: int) -> int:
            """Returns the minimum cost to paint j walls by painters[i..n)."""
            if walls <= 0:
                return 0
            if i == n:
                return math.inf
            pick = cost[i] + dp(i + 1, walls - time[i] - 1)
            skip = dp(i + 1, walls)
            return min(pick, skip)

        return dp(0, n)

```

1669. 2743.py

Path: LeetCode-main/solutions/2743. Count Substrings Without Repeating Character/2743.py

```

class Solution:
    def numberOfSpecialSubstrings(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] == 2:
                count[s[l]] -= 1
                l += 1
            ans += r - l + 1

        return ans

```

1670. 2744.py

Path: LeetCode-main/solutions/2744. Find Maximum Number of String Pairs/2744.py

```

class Solution:
    def maximumNumberOfStringPairs(self, words: list[str]) -> int:
        ans = 0
        seen = [False] * (26 * 26)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        for word in words:
            if seen[val(word[1]) * 26 + val(word[0])]:
                ans += 1
            seen[val(word[0]) * 26 + val(word[1])] = True

        return ans

```

1671. 2745.py

Path: LeetCode-main/solutions/2745. Construct the Longest New String/2745.py

```
class Solution:
    def longestString(self, x: int, y: int, z: int) -> int:
        # 'AB' can always be easily appended within the string.
        # Alternating 'AA' and 'BB' can be appended, creating a pattern like 'AABB'
        # If x == y, we repeat the pattern 'AABBAABB...AABB'.
        # If x != y, the pattern becomes 'AABBAABB...AABBAA' or 'BBAABBAABB...AABB'
        mn = min(x, y)
        if x == y:
            return (mn * 2 + z) * 2
        return (mn * 2 + 1 + z) * 2
```

1672. 2746.py

Path: LeetCode-main/solutions/2746. Decremental String Concatenation/2746.py

```
class Solution:
    def minimizeConcatenatedLength(self, words: list[str]) -> int:
        @functools.lru_cache(None)
        def dp(i: int, first: str, last: str) -> int:
            """
            Returns the minimum concatenated length of the first i words starting with
            `first` and ending in `last`.
            """
            if i == len(words):
                return 0
            nextFirst = words[i][0]
            nextLast = words[i][-1]
            return len(words[i]) + min(
                # join(words[i - 1], words[i])
                dp(i + 1, first, nextLast) - (last == nextFirst),
                # join(words[i], words[i - 1])
                dp(i + 1, nextFirst, last) - (first == nextLast)
            )

        return len(words[0]) + dp(1, words[0][0], words[0][-1])
```

1673. 2747.py

Path: LeetCode-main/solutions/2747. Count Zero Request Servers/2747.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class IndexedQuery:
    queryIndex: int
    query: int

    def __iter__(self):
        yield self.queryIndex
        yield self.query

class Solution:
    def countServers(
        self,
        n: int,
        logs: list[list[int]],
        x: int,
        queries: list[int],
    ) -> list[int]:
        ans = [0] * len(queries)
        count = [0] * (n + 1)

        logs.sort(key=lambda x: x[1])

        i = 0
        j = 0
        servers = 0

        # For each query, we care about logs[i..j].
        for queryIndex, query in sorted([IndexedQuery(i, query)
                                         for i, query in enumerate(queries)],
                                         key=lambda x: x.query):
            while j < len(logs) and logs[j][1] <= query:
                count[logs[j][0]] += 1
                if count[logs[j][0]] == 1:
                    servers += 1
                j += 1
            while i < len(logs) and logs[i][1] < query - x:
                count[logs[i][0]] -= 1
                if count[logs[i][0]] == 0:
                    servers -= 1
                i += 1
            ans[queryIndex] = n - servers

        return ans

```

1674. 2748.py

Path: LeetCode-main/solutions/2748. Number of Beautiful Pairs/2748.py

```

class Solution:
    def countBeautifulPairs(self, nums: list[int]) -> int:
        def firstDigit(num: int) -> int:
            return int(str(num)[0])

        def lastDigit(num: int) -> int:
            return num % 10

        return sum(math.gcd(firstDigit(nums[i]), lastDigit(nums[j])) == 1
                    for i, j in itertools.combinations(range(len(nums)), 2))

```

1675. 2749.py

Path: LeetCode-main/solutions/2749. Minimum Operations to Make the Integer Zero/2749.py

```

class Solution:
    def makeTheIntegerZero(self, num1: int, num2: int) -> int:
        # If k operations are used, num1 - [(num2 + 2^{i_1}) + (num2 + 2^{i_2}) +
        # ... + (num2 + 2^{i_k})] = 0. So, num1 - k * num2 = (2^{i_1} + 2^{i_2} +
        # ... + 2^{i_k}), where i_1, i_2, ..., i_k are in the range [0, 60].
        # Note that for any number x, we can use "x's bit count" operations to make
        # x equal to 0. Additionally, we can also use x operations to deduct x by
        # 2^0 (x times), which also results in 0.

        for ops in range(61):
            target = num1 - ops * num2
            if target.bit_count() <= ops <= target:
                return ops

        return -1

```

1676. 275.py

Path: LeetCode-main/solutions/275. H-Index II/275.py

```

class Solution:
    def hIndex(self, citations: list[int]) -> int:
        n = len(citations)
        return n - bisect.bisect_left(range(n), n,
                                     key=lambda m: citations[m] + m)

```

1677. 2750.py

Path: LeetCode-main/solutions/2750. Ways to Split Array Into Good Subarrays/2750.py


```
class Solution:
    def numberOfGoodSubarraySplits(self, nums: list[int]) -> int:
        if 1 not in nums:
            return 0

        MOD = 1_000_000_007
        prev = -1 # the previous index of 1
        ans = 1

        for i, num in enumerate(nums):
            if num == 1:
                if prev != -1:
                    ans *= i - prev
                    ans %= MOD
                prev = i

        return ans
```

1678. 2751.py

Path: LeetCode-main/solutions/2751. Robot Collisions/2751.py

```

from dataclasses import dataclass

@dataclass
class Robot:
    index: int
    position: int
    health: int
    direction: str

class Solution:
    def survivedRobotsHealths(
        self,
        positions: list[int],
        healths: list[int],
        directions: str,
    ) -> list[int]:
        robots = sorted([Robot(index, position, health, direction)
                          for index, (position, health, direction) in
                          enumerate(zip(positions, healths, directions))],
                         key=lambda x: x.position)
        stack: list[Robot] = [] # running robots

        for robot in robots:
            if robot.direction == 'R':
                stack.append(robot)
                continue
            # Collide with robots going right if any.
            while stack and stack[-1].direction == 'R' and robot.health > 0:
                if stack[-1].health == robot.health:
                    stack.pop()
                    robot.health = 0
                elif stack[-1].health < robot.health:
                    stack.pop()
                    robot.health -= 1
                else: # stack[-1].health > robot.health
                    stack[-1].health -= 1
                    robot.health = 0
            if robot.health > 0:
                stack.append(robot)

        stack.sort(key=lambda robot: robot.index)
        return [robot.health for robot in stack]

```

1679. 2753.py

Path: LeetCode-main/solutions/2753. Count Houses in a Circular Street II/2753.py

```

# Definition for a street.
# class Street:
#     def closeDoor(self):
#         pass
#     def isDoorOpen(self):
#         pass
#     def moveRight(self):
#         pass
class Solution:
    def houseCount(self, street: Optional['Street'], k: int) -> int:
        ans = 0

        # Go to the first open door.
        while not street.isDoorOpen():
            street.moveRight()

        street.moveRight()

        for count in range(k):
            # Each time we encounter an open door, there's a possibility that it's the
            # first open door we intentionally left open.
            if street.isDoorOpen():
                ans = count + 1
                street.closeDoor()
                street.moveRight()

        return ans

```

1680. 276.py

Path: LeetCode-main/solutions/276. Paint Fence/276.py

```

class Solution:
    def numWays(self, n: int, k: int) -> int:
        if n == 0:
            return 0
        if n == 1:
            return k
        if n == 2:
            return k * k

        # dp[i] := the number of ways to pan posts with k colors
        dp = [0] * (n + 1)
        dp[0] = 0
        dp[1] = k
        dp[2] = k * k

        for i in range(3, n + 1):
            dp[i] = (dp[i - 1] + dp[i - 2]) * (k - 1)

        return dp[n]

```

1681. 2760.py

Path: LeetCode-main/solutions/2760. Longest Even Odd Subarray With Threshold/2760.py

```

class Solution:
    def longestAlternatingSubarray(self, nums: list[int], threshold: int) -> int:
        ans = 0
        dp = 0

        def isOddEven(a: int, b: int) -> bool:
            return a % 2 != b % 2

        for i, num in enumerate(nums):
            if num > threshold:
                dp = 0
            elif i > 0 and dp > 0 and isOddEven(nums[i - 1], num):
                # Increase the size of the subarray.
                dp += 1
            else:
                # Start a new subarray if the start is valid.
                dp = 1 if num % 2 == 0 else 0
            ans = max(ans, dp)

        return ans

```

1682. 2761.py

Path: LeetCode-main/solutions/2761. Prime Pairs With Target Sum/2761.py

```

class Solution:
    def findPrimePairs(self, n: int) -> list[list[int]]:
        isPrime = self._sieveEratosthenes(n + 1)
        return [[i, n - i] for i in range(2, n // 2 + 1)
                if isPrime[i] and isPrime[n - i]]

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime

```

j

1683. 2762.py

Path: LeetCode-main/solutions/2762. Continuous Subarrays/2762.py

```
class Solution:
    def continuousSubarrays(self, nums: list[int]) -> int:
        ans = 1 # [nums[0]]
        left = nums[0] - 2
        right = nums[0] + 2
        l = 0

        # nums[l..r] is a valid window.
        for r in range(1, len(nums)):
            if left <= nums[r] <= right:
                left = max(left, nums[r] - 2)
                right = min(right, nums[r] + 2)
            else:
                # nums[r] is out-of-bounds, so reconstruct the window.
                left = nums[r] - 2
                right = nums[r] + 2
                l = r
                # If we consistently move leftward in each iteration, it implies that
                # the entire left subarray satisfies the given condition. For every
                # subarray with l in the range [0, r], the condition is met, preventing
                # the code from reaching the final "else" condition. Instead, it stops
                # at the "if" condition.
                while nums[r] - 2 <= nums[l] <= nums[r] + 2:
                    left = max(left, nums[l] - 2)
                    right = min(right, nums[l] + 2)
                    l -= 1
                l += 1
            # nums[l..r], num[l + 1..r], ..., nums[r]
            ans += r - l + 1

        return ans
```

1684. 2763-2.py

Path: LeetCode-main/solutions/2763. Sum of Imbalance Numbers of All Subarrays/2763-2.py

```

class Solution:
    # If sorted(nums)[i + 1] - sorted(nums)[i] > 1, then there's a gap. Instead
    # of determining the number of gaps in each subarray, let's find out how many
    # subarrays contain each gap.
    def sumImbalanceNumbers(self, nums: list[int]) -> int:
        n = len(nums)

        # Note that to avoid double counting, only `left` needs to check nums[i].
        # This adjustment ensures that i represents the position of the leftmost
        # element of nums[i] within the subarray.

        # left[i] := the maximum index l s.t. nums[l] = nums[i] or nums[i] + 1
        left = [0] * n
        # right[i] := the minimum index r s.t. nums[r] = nums[i]
        right = [0] * n

        numToIndex = [-1] * (n + 2)
        for i, num in enumerate(nums):
            left[i] = max(numToIndex[num], numToIndex[num + 1])
            numToIndex[num] = i

        numToIndex = [n] * (n + 2)
        for i in range(n - 1, -1, -1):
            right[i] = numToIndex[nums[i] + 1]
            numToIndex[nums[i]] = i

        # The gap above nums[i] persists until encountering nums[i] or nums[i] + 1.
        # Consider subarrays nums[l..r] with l <= i <= r, where l in [left[i], i]
        # and r in [i, right[i] - 1]. There are (i - left[i]) * (right[i] - i)
        # subarrays satisfying this condition.
        #
        # Subtract n * (n + 1) / 2 to account for the overcounting of elements
        # initially assumed to have a gap. This adjustment is necessary as the
        # maximum element of every subarray does not have a gap.
        return sum((i - left[i]) * (right[i] - i)
                   for i in range(n)) - n * (n + 1) // 2

```

1685. 2764.py

Path: LeetCode-main/solutions/2764. is Array a Preorder of Some Binary Tree/2764.py

```

class Solution:
    def isPreorder(self, nodes: list[list[int]]) -> bool:
        stack = [] # Stores `id`s.

        for id, parentId in nodes:
            if parentId == -1:
                stack.append(id)
                continue
            while stack and stack[-1] != parentId:
                stack.pop()
            if not stack:
                return False
            stack.append(id)

        return True

```

1686. 2765.py

Path: LeetCode-main/solutions/2765. Longest Alternating Subarray/2765.py

```
class Solution:
    def alternatingSubarray(self, nums: list[int]) -> int:
        ans = 1
        dp = 1

        for i in range(1, len(nums)):
            targetDiff = -1 if dp % 2 == 0 else 1
            # Append nums[i] to the current alternating subarray.
            if nums[i] - nums[i - 1] == targetDiff:
                dp += 1
            # Reset the alternating subarray to nums[i - 1..i].
            elif nums[i] - nums[i - 1] == 1:
                dp = 2
            # Reset the alternating subarray to nums[i].
            else:
                dp = 1
            ans = max(ans, dp)

        return -1 if ans == 1 else ans
```

1687. 2766.py

Path: LeetCode-main/solutions/2766. Relocate Marbles/2766.py

```
class Solution:
    def relocateMarbles(
        self,
        nums: list[int],
        moveFrom: list[int],
        moveTo: list[int],
    ) -> list[int]:
        numsSet = set(nums)

        for f, t in zip(moveFrom, moveTo):
            numsSet.remove(f)
            numsSet.add(t)

        return sorted(numsSet)
```

1688. 2767.py

Path: LeetCode-main/solutions/2767. Partition String Into Minimum Beautiful Substrings/2767.py

```

class Solution:
    def minimumBeautifulSubstrings(self, s: str) -> int:
        n = len(s)
        # dp[i] := the minimum number of beautiful substrings for the first i chars
        dp = [0] + [n + 1] * n

        for i in range(1, n + 1):
            if s[i - 1] == '0':
                continue
            num = 0 # the number of s[i - 1..j - 1]
            for j in range(i, n + 1):
                num = (num << 1) + int(s[j - 1])
                if self._isPowerOfFive(num):
                    dp[j] = min(dp[j], dp[i - 1] + 1)

        return -1 if dp[n] == n + 1 else dp[n]

    def _isPowerOfFive(self, num: int) -> bool:
        while num % 5 == 0:
            num //= 5
        return num == 1

```

1689. 2768.py

Path: LeetCode-main/solutions/2768. Number of Black Blocks/2768.py

```

class Solution:
    def countBlackBlocks(
        self,
        m: int,
        n: int,
        coordinates: list[list[int]],
    ) -> list[int]:
        ans = [0] * 5
        # count[i * n + j] := the number of black cells in
        # (i - 1, j - 1), (i - 1, j), (i, j - 1), (i, j)
        count = collections.Counter()

        for x, y in coordinates:
            for i in range(x, x + 2):
                for j in range(y, y + 2):
                    # 2 x 2 submatrix with right-bottom conner being (i, j) contains the
                    # current black cell (x, y).
                    if 0 < i < m and 0 < j < n:
                        count[(i, j)] += 1

        for freq in count.values():
            ans[freq] += 1

        ans[0] = (m - 1) * (n - 1) - sum(ans)
        return ans

```

1690. 2769.py

Path: LeetCode-main/solutions/2769. Find the Maximum Achievable Number/2769.py

```

class Solution:
    def theMaximumAchievableX(self, num: int, t: int) -> int:
        return num + 2 * t

```

1691. 277.py

Path: LeetCode-main/solutions/277. Find the Celebrity/277.py

```
# The knows API is already defined for you.
# Returns a bool, whether a knows b
# Def knows(a: int, b: int) -> bool:

class Solution:
    def findCelebrity(self, n: int) -> int:
        candidate = 0

        # Everyone knows the celebrity.
        for i in range(1, n):
            if knows(candidate, i):
                candidate = i

        # The candidate knows nobody and everyone knows the celebrity.
        for i in range(n):
            if i < candidate and knows(candidate, i) or not knows(i, candidate):
                return -1
            if i > candidate and not knows(i, candidate):
                return -1

        return candidate
```

1692. 2770.py

Path: LeetCode-main/solutions/2770. Maximum Number of Jumps to Reach the Last Index/2770.py

```
class Solution:
    def maximumJumps(self, nums: list[int], target: int) -> int:
        n = len(nums)
        # dp[i] := the maximum number of jumps to reach i from 0
        dp = [-1] * n
        dp[0] = 0

        for j in range(1, n):
            for i in range(j):
                if dp[i] != -1 and abs(nums[j] - nums[i]) <= target:
                    dp[j] = max(dp[j], dp[i] + 1)

        return dp[-1]
```

1693. 2771.py

Path: LeetCode-main/solutions/2771. Longest Non-decreasing Subarray From Two Arrays/2771.py

```

class Solution:
    def maxNonDecreasingLength(self, nums1: list[int], nums2: list[int]) -> int:
        ans = 1
        dp1 = 1 # the longest subarray that ends in nums1[i] so far
        dp2 = 1 # the longest subarray that ends in nums2[i] so far

        for i in range(1, len(nums1)):
            dp11 = dp1 + 1 if nums1[i - 1] <= nums1[i] else 1
            dp21 = dp2 + 1 if nums2[i - 1] <= nums1[i] else 1
            dp12 = dp1 + 1 if nums1[i - 1] <= nums2[i] else 1
            dp22 = dp2 + 1 if nums2[i - 1] <= nums2[i] else 1
            dp1 = max(dp11, dp21)
            dp2 = max(dp12, dp22)
            ans = max(ans, dp1, dp2)

        return ans

```

1694. 2772.py

Path: LeetCode-main/solutions/2772. Apply Operations to Make All Array Elements Equal to Zero/2772.py

```

class Solution:
    def checkArray(self, nums: list[int], k: int) -> bool:
        if k == 1:
            return True

        needDecrease = 0
        # Store nums[i - k + 1..i] with decreasing nums[i - k + 1].
        dq = collections.deque()

        for i, num in enumerate(nums):
            if i >= k:
                needDecrease -= dq.popleft()
            if nums[i] < needDecrease:
                return False
            decreasedNum = nums[i] - needDecrease
            dq.append(decreasedNum)
            needDecrease += decreasedNum

        return dq[-1] == 0

```

1695. 2773.py

Path: LeetCode-main/solutions/2773. Height of Special Binary Tree/2773.py

```

class Solution:
    def heightOfTree(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        # a leaf node
        if root.left and root.left.right == root:
            return 0
        return 1 + max(self.heightOfTree(root.left), self.heightOfTree(root.right))

```

1696. 2778.py

Path: LeetCode-main/solutions/2778. Sum of Squares of Special Elements/2778.py

```
class Solution:
    def sumOfSquares(self, nums: list[int]) -> int:
        return sum(num**2 for i, num in enumerate(nums)
                    if len(nums) % (i + 1) == 0)
```

1697. 2779-2.py

Path: LeetCode-main/solutions/2779. Maximum Beauty of an Array After Applying Operation/2779-2.py

```
class Solution:
    def maximumBeauty(self, nums: list[int], k: int) -> int:
        nums.sort()

        # l and r track the maximum window instead of the valid window.
        l = 0
        for r in range(len(nums)):
            if nums[r] - nums[l] > 2 * k:
                l += 1

        return r - l + 1
```

1698. 2779.py

Path: LeetCode-main/solutions/2779. Maximum Beauty of an Array After Applying Operation/2779.py

```
class Solution:
    def maximumBeauty(self, nums: list[int], k: int) -> int:
        ans = 0

        nums.sort()

        l = 0
        for r in range(len(nums)):
            while nums[r] - nums[l] > 2 * k:
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

1699. 278.py

Path: LeetCode-main/solutions/278. First Bad Version/278.py

```
class Solution:
    def firstBadVersion(self, n: int) -> int:
        l = 1
        r = n

        while l < r:
            m = (l + r) >> 1
            if isBadVersion(m):
                r = m
            else:
                l = m + 1

        return l
```

1700. 2780.py

Path: LeetCode-main/solutions/2780. Minimum Index of a Valid Split/2780.py

```
class Solution:
    def minimumIndex(self, nums: list[int]) -> int:
        count1 = collections.Counter()
        count2 = collections.Counter(nums)

        for i, num in enumerate(nums):
            count1[num] = count1[num] + 1
            count2[num] = count2[num] - 1
            if count1[num] * 2 > i + 1 and count2[num] * 2 > len(nums) - i - 1:
                return i

        return -1
```

1701. 2781-2.py

Path: LeetCode-main/solutions/2781. Length of the Longest Valid Substring/2781-2.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node: TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

    def search(self, word: str, l: int, r: int) -> bool:
        node: TrieNode = self.root
        for i in range(l, r):
            if word[i] not in node.children:
                return False
            node = node.children[word[i]]
        return node.isWord

class Solution:
    def longestValidSubstring(self, word: str, forbidden: list[str]) -> int:
        ans = 0
        trie = Trie()

        for s in forbidden:
            trie.insert(s)

        # r is the rightmost index to make word[l..r] a valid substring.
        r = len(word) - 1
        for l in range(len(word) - 1, -1, -1):
            for end in range(l, min(l + 10, r + 1)):
                if trie.search(word, l, end + 1):
                    r = end - 1
                    break
            ans = max(ans, r - l + 1)

        return ans

```

1702. 2781.py

Path: LeetCode-main/solutions/2781. Length of the Longest Valid Substring/2781.py

```

class Solution:
    def longestValidSubstring(self, word: str, forbidden: list[str]) -> int:
        forbiddenSet = set(forbidden)
        ans = 0
        r = len(word) - 1 # rightmost index of the valid substring

        for l in range(len(word) - 1, -1, -1):
            for end in range(l, min(l + 10, r + 1)):
                if word[l:end + 1] in forbiddenSet:
                    r = end - 1
                    break
            ans = max(ans, r - l + 1)

        return ans

```

1703. 2782.py

Path: LeetCode-main/solutions/2782. Number of Unique Categories/2782.py

```

# Definition for a category handler.
# class CategoryHandler:
#     def haveSameCategory(self, a: int, b: int) -> bool:
#         pass

class Solution:
    def numberOfCategories(
        self,
        n: int,
        categoryHandler: Optional['CategoryHandler'],
    ) -> int:
        ans = 0

        for i in range(n):
            if not any(categoryHandler.haveSameCategory(i, j) for j in range(i)):
                ans += 1

        return ans

```

1704. 2784.py

Path: LeetCode-main/solutions/2784. Check if Array is Good/2784.py

```

class Solution:
    def isGood(self, nums: list[int]) -> bool:
        n = len(nums) - 1
        count = collections.Counter(nums)
        return all(count[i] == 1 for i in range(1, n)) and count[n] == 2

```

1705. 2785.py

Path: LeetCode-main/solutions/2785. Sort Vowels in a String/2785.py

```

class Solution:
    def sortVowels(self, s: str) -> str:
        VOWELS = 'aeiouAEIOU'
        ans = []
        vowels = sorted([c for c in s if c in VOWELS])

        i = 0 # vowels' index
        for c in s:
            if c in VOWELS:
                ans.append(vowels[i])
                i += 1
            else:
                ans.append(c)

        return ''.join(ans)

```

1706. 2786.py

Path: LeetCode-main/solutions/2786. Visit Array Positions to Maximize Score/2786.py

```

class Solution:
    def maxScore(self, nums: list[int], x: int) -> int:
        # Note that we always need to take nums[0], so the initial definition might
        # not hold true.

        # dp0 := the maximum score so far with `nums` ending in an even number
        dp0 = nums[0] - (x if nums[0] % 2 == 1 else 0)
        # dp1 := the maximum score so far with `nums` ending in an odd number
        dp1 = nums[0] - (x if nums[0] % 2 == 0 else 0)

        for i in range(1, len(nums)):
            if nums[i] % 2 == 0:
                dp0 = nums[i] + max(dp0, dp1 - x)
            else:
                dp1 = nums[i] + max(dp1, dp0 - x)

        return max(dp0, dp1)

```

1707. 2787.py

Path: LeetCode-main/solutions/2787. Ways to Express an Integer as Sum of Powers/2787.py

```

class Solution:
    def numberOfWays(self, n: int, x: int) -> int:
        MOD = 1_000_000_007
        # dp[i] := the number of ways to express i
        dp = [1] + [0] * n

        for a in range(1, n + 1):
            ax = a**x
            if ax > n:
                break
            for i in range(n, ax - 1, -1):
                dp[i] += dp[i - ax]
                dp[i] %= MOD

        return dp[n]

```

1708. 2788.py

Path: LeetCode-main/solutions/2788. Split Strings by Separator/2788.py

```
class Solution:
    def splitWordsBySeparator(
        self,
        words: list[str],
        separator: str,
    ) -> list[str]:
        return [splitWord
                for word in words
                for splitWord in word.split(separator)
                if splitWord]
```

1709. 2789.py

Path: LeetCode-main/solutions/2789. Largest Element in an Array after Merge Operations/2789.py

```
class Solution:
    def maxArrayValue(self, nums: list[int]) -> int:
        ans = nums[-1]

        for i in range(len(nums) - 2, -1, -1):
            if nums[i] > ans:
                ans = nums[i]
            else:
                ans += nums[i]

        return ans
```

1710. 279.py

Path: LeetCode-main/solutions/279. Perfect Squares/279.py

```
class Solution:
    def numSquares(self, n: int) -> int:
        dp = [n] * (n + 1) # 1^2 x n
        dp[0] = 0 # no way
        dp[1] = 1 # 1^2

        for i in range(2, n + 1):
            j = 1
            while j * j <= i:
                dp[i] = min(dp[i], dp[i - j * j] + 1)
                j += 1

        return dp[n]
```

1711. 2790.py

Path: LeetCode-main/solutions/2790. Maximum Number of Groups With Increasing Length/2790.py


```

class Solution:
    def maxIncreasingGroups(self, usageLimits: list[int]) -> int:
        ans = 1 # the next target length
        availableLimits = 0

        for usageLimit in sorted(usageLimits):
            availableLimits += usageLimit
            # Can create groups 1, 2, ..., ans.
            if availableLimits >= ans * (ans + 1) // 2:
                ans += 1

        return ans - 1

```

1712. 2791.py

Path: LeetCode-main/solutions/2791. Count Paths That Can Form a Palindrome in a Tree/2791.py

```

class Solution:
    def countPalindromePaths(self, parent: list[int], s: str) -> int:
        # A valid (u, v) has at most 1 letter with odd frequency on its path. The
        # frequency of a letter on the u-v path is equal to the sum of its
        # frequencies on the root-u and root-v paths subtract twice of its
        # frequency on the root-LCA(u, v) path. Considering only the parity
        # (even/odd), the part involving root-LCA(u, v) can be ignored, making it
        # possible to calculate both parts easily using a simple DFS.
        tree = [[] for _ in parent]
        maskToCount = collections.Counter({0: 1})

        for i in range(1, len(parent)):
            tree[parent[i]].append(i)

        # mask := 26 bits that represent the parity of each character in the alphabet
        # on the path from node 0 to node u
        def dfs(u: int, mask: int) -> int:
            res = 0
            if u > 0:
                mask ^= 1 << (ord(s[u]) - ord('a'))
                # Consider any u-v path with 1 bit set.
                for i in range(26):
                    res += maskToCount[mask ^ (1 << i)]
                # Consider u-v path with 0 bit set.
                res += maskToCount[mask ^ 0]
                maskToCount[mask] += 1
            for v in tree[u]:
                res += dfs(v, mask)
            return res

        return dfs(0, 0)

```

1713. 2792.py

Path: LeetCode-main/solutions/2792. Count Nodes That Are Great Enough/2792.py

```

class Solution:
    def countGreatEnoughNodes(self, root: TreeNode | None, k: int) -> int:
        ans = 0

        def dfs(root: TreeNode | None) -> list[int]:
            nonlocal ans
            if not root:
                return []

            kSmallest = sorted(dfs(root.left) + dfs(root.right))[:k]
            if len(kSmallest) == k and root.val > kSmallest[-1]:
                ans += 1

            return kSmallest + [root.val]

        dfs(root)
        return ans

```

1714. 2798.py

Path: LeetCode-main/solutions/2798. Number of Employees Who Met the Target/2798.py

```

class Solution:
    def numberOfEmployeesWhoMetTarget(self, hours: list[int], target: int) -> int:
        return sum(hour >= target for hour in hours)

```

1715. 2799.py

Path: LeetCode-main/solutions/2799. Count Complete Subarrays in an Array/2799.py

```

class Solution:
    def countCompleteSubarrays(self, nums: list[int]) -> int:
        ans = 0
        distinct = len(set(nums))
        count = collections.Counter()

        l = 0
        for num in nums:
            count[num] += 1
            while len(count) == distinct:
                count[nums[l]] -= 1
                if count[nums[l]] == 0:
                    del count[nums[l]]
                l += 1
            # Assume nums[r] = num,
            # nums[0..r], nums[1..r], ..., nums[l - 1..r] have k different values.
            ans += l

        return ans

```

1716. 28.py

Path: LeetCode-main/solutions/28. Implement strStr()/28.py

```
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        m = len(haystack)
        n = len(needle)

        for i in range(m - n + 1):
            if haystack[i:i + n] == needle:
                return i

        return -1
```

1717. 280.py

Path: LeetCode-main/solutions/280. Wiggle Sort/280.py

```
class Solution:
    def wiggleSort(self, nums: list[int]) -> None:
        # 1. If i is even, then nums[i] <= nums[i - 1].
        # 2. If i is odd, then nums[i] >= nums[i - 1].
        for i in range(1, len(nums)):
            if (i % 2 == 0 and nums[i] > nums[i - 1] or
                i % 2 == 1 and nums[i] < nums[i - 1]):
                nums[i], nums[i - 1] = nums[i - 1], nums[i]
```

1718. 2800.py

Path: LeetCode-main/solutions/2800. Shortest String That Contains Three Strings/2800.py

```

class Solution:
    def minimumString(self, a: str, b: str, c: str) -> str:
        def merge(a: str, b: str) -> str:
            """Merges a and b."""
            if a in b: # a is a substring of b.
                return b
            for i in range(len(a)):
                aSuffix = a[i:]
                bPrefix = b[:len(aSuffix)]
                if aSuffix == bPrefix:
                    return a + b[len(bPrefix):]
            return a + b

        abc = merge(a, merge(b, c))
        acb = merge(a, merge(c, b))
        bac = merge(b, merge(a, c))
        bca = merge(b, merge(c, a))
        cab = merge(c, merge(a, b))
        cba = merge(c, merge(b, a))
        return self._getMin([abc, acb, bac, bca, cab, cba])

    def _getMin(self, words: list[str]) -> str:
        """Returns the lexicographically smallest string."""

        def getMin(a: str, b: str) -> str:
            """Returns the lexicographically smaller string."""
            return a if len(a) < len(b) or (len(a) == len(b) and a < b) else b

        res = words[0]
        for i in range(1, len(words)):
            res = getMin(res, words[i])
        return res

```

1719. 2801.py

Path: LeetCode-main/solutions/2801. Count Stepping Numbers in Range/2801.py

```

class Solution:
    def countSteppingNumbers(self, low: str, high: str) -> int:
        MOD = 1_000_000_007
        low = '0' * (len(high) - len(low)) + low

        @functools.lru_cache(None)
        def dp(
            i: int,
            prevDigit: int,
            isLeadingZero: bool,
            tight1: bool,
            tight2: bool,
        ) -> int:
            """
            Returns the number of valid integers, considering the i-th digit, where
            `prevDigit` is the previous digit, `tight1` indicates if the current
            digit is tightly bound for `low`, and `tight2` indicates if the current
            digit is tightly bound for `high`.
            """
            if i == len(high):
                return 1

            res = 0
            minDigit = int(low[i]) if tight1 else 0
            maxDigit = int(high[i]) if tight2 else 9

            for d in range(minDigit, maxDigit + 1):
                nextTight1 = tight1 and (d == minDigit)
                nextTight2 = tight2 and (d == maxDigit)
                if isLeadingZero:
                    # Can place any digit in [minDigit, maxDigit].
                    res += dp(i + 1, d, isLeadingZero and d == 0, nextTight1, nextTight2)
                elif abs(d - prevDigit) == 1:
                    res += dp(i + 1, d, False, nextTight1, nextTight2)
            res %= MOD

            return res

        return dp(0, -1, True, True, True)

```

1720. 2802.py

Path: LeetCode-main/solutions/2802. Find The K-th Lucky Number/2802.py

```

class Solution:
    def kthLuckyNumber(self, k: int) -> str:
        return bin(k + 1)[3:].replace('0', '4').replace('1', '7')

```

1721. 2806.py

Path: LeetCode-main/solutions/2806. Account Balance After Rounded Purchase/2806.py

```

class Solution:
    def accountBalanceAfterPurchase(self, purchaseAmount: int) -> int:
        return 100 - ((purchaseAmount + 5) // 10) * 10

```

1722. 2807.py

Path: LeetCode-main/solutions/2807. Insert Greatest Common Divisors in Linked List/2807.py

```
class Solution:
    def insertGreatestCommonDivisors(
        self, head: ListNode | None
    ) -> ListNode | None:
        curr = head
        while curr.next:
            inserted = ListNode(math.gcd(curr.val, curr.next.val), curr.next)
            curr.next = inserted
            curr = inserted.next
        return head
```

1723. 2808.py

Path: LeetCode-main/solutions/2808. Minimum Seconds to Equalize a Circular Array/2808.py

```
class Solution:
    def minimumSeconds(self, nums: list[int]) -> int:
        n = len(nums)
        ans = n
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums):
            numToIndices[num].append(i)

        def getSeconds(i: int, j: int) -> int:
            """Returns the number of seconds required to make nums[i..j] the same."""
            return (i - j) // 2

        for indices in numToIndices.values():
            seconds = getSeconds(indices[0] + n, indices[-1])
            for i in range(1, len(indices)):
                seconds = max(seconds, getSeconds(indices[i], indices[i - 1]))
            ans = min(ans, seconds)

        return ans
```

1724. 2809-2.py

Path: LeetCode-main/solutions/2809. Minimum Time to Make Array Sum At Most x/2809-2.py

```

class Solution:
    def minimumTime(self, nums1: list[int], nums2: list[int], x: int) -> int:
        n = len(nums1)
        # dp[j] := the maximum reduced value if we do j operations on the numbers
        # so far
        dp = [0] * (n + 1)
        sum1 = sum(nums1)
        sum2 = sum(nums2)

        for i, (num2, num1) in enumerate(sorted(zip(nums2, nums1)), 1):
            for j in range(i, 0, -1):
                dp[j] = max(
                    # the maximum reduced value if we do j operations on the first
                    # i - 1 numbers
                    dp[j],
                    # the maximum reduced value if we do j - 1 operations on the first
                    # i - 1 numbers + making the i-th number of `nums1` to 0 at the
                    # j-th operation
                    dp[j - 1] + num2 * j + num1
                )

            for op in range(n + 1):
                if sum1 + sum2 * op - dp[op] <= x:
                    return op

        return -1

```

1725. 2809.py

Path: LeetCode-main/solutions/2809. Minimum Time to Make Array Sum At Most x/2809.py

```

class Solution:
    def minimumTime(self, nums1: list[int], nums2: list[int], x: int) -> int:
        n = len(nums1)
        # dp[i][j] := the maximum reduced value if we do j operations on the first
        # i numbers
        dp = [[0] * (n + 1) for _ in range(n + 1)]
        sum1 = sum(nums1)
        sum2 = sum(nums2)

        for i, (num2, num1) in enumerate(sorted(zip(nums2, nums1)), 1):
            for j in range(1, i + 1):
                dp[i][j] = max(
                    # the maximum reduced value if we do j operations on the first
                    # i - 1 numbers
                    dp[i - 1][j],
                    # the maximum reduced value if we do j - 1 operations on the first
                    # i - 1 numbers + making the i-th number of `nums1` to 0 at the
                    # j-th operation
                    dp[i - 1][j - 1] + num2 * j + num1
                )

            for op in range(n + 1):
                if sum1 + sum2 * op - dp[n][op] <= x:
                    return op

        return -1

```

1726. 281.py

Path: LeetCode-main/solutions/281. Zigzag Iterator/281.py

```
class ZigzagIterator:
    def __init__(self, v1: list[int], v2: list[int]):
        def vals():
            for i in itertools.count():
                for v in v1, v2:
                    if i < len(v):
                        yield v[i]
        self.vals = vals()
        self.n = len(v1) + len(v2)

    def next(self):
        self.n -= 1
        return next(self.vals)

    def hasNext(self):
        return self.n > 0
```

1727. 2810.py

Path: LeetCode-main/solutions/2810. Faulty Keyboard/2810.py

```
class Solution:
    def finalString(self, s: str) -> str:
        dq = collections.deque()
        inversed = False

        for c in s:
            if c == 'i':
                inversed = not inversed
            elif inversed:
                dq.appendleft(c)
            else:
                dq.append(c)

        return ''.join(reversed(dq)) if inversed else ''.join(dq)
```

1728. 2811.py

Path: LeetCode-main/solutions/2811. Check if it is Possible to Split Array/2811.py

```
class Solution:
    def canSplitArray(self, nums: list[int], m: int) -> bool:
        return len(nums) < 3 or any(a + b >= m for a, b in itertools.pairwise(nums))
```

1729. 2812.py

Path: LeetCode-main/solutions/2812. Find the Safest Path in a Grid/2812.py


```

class Solution:
    def maximumSafenessFactor(self, grid: list[list[int]]) -> int:
        self.DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        n = len(grid)
        distToThief = self._getDistToThief(grid)

        def hasValidPath(safeness: int) -> bool:
            if distToThief[0][0] < safeness:
                return False

            q = collections.deque([(0, 0)])
            seen = {(0, 0)}

            while q:
                i, j = q.popleft()
                if distToThief[i][j] < safeness:
                    continue
                if i == n - 1 and j == n - 1:
                    return True
                for dx, dy in self.DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == n or y < 0 or y == n:
                        continue
                    if (x, y) in seen:
                        continue
                    q.append((x, y))
                    seen.add((x, y))

            return False

        return bisect.bisect_left(range(n * 2), True,
                                   key=lambda m: not hasValidPath(m)) - 1

    def _getDistToThief(self, grid: list[list[int]]) -> list[list[int]]:
        n = len(grid)
        distToThief = [[0] * n for _ in range(n)]
        q = collections.deque()
        seen = set()

        for i in range(n):
            for j in range(n):
                if grid[i][j] == 1:
                    q.append((i, j))
                    seen.add((i, j))

        dist = 0
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                distToThief[i][j] = dist
                for dx, dy in self.DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == n or y < 0 or y == n:
                        continue
                    if (x, y) in seen:
                        continue
                    q.append((x, y))
                    seen.add((x, y))
            dist += 1

        return distToThief

```

1730. 2813.py

Path: LeetCode-main/solutions/2813. Maximum Elegance of a K-Length Subsequence/2813.py

```
class Solution:
    def findMaximumElegance(self, items: list[list[int]], k: int) -> int:
        ans = 0
        totalProfit = 0
        seenCategories = set()
        decreasingDuplicateProfits = []

        items.sort(reverse=True)

        for i in range(k):
            profit, category = items[i]
            totalProfit += profit
            if category in seenCategories:
                decreasingDuplicateProfits.append(profit)
            else:
                seenCategories.add(category)

        ans = totalProfit + len(seenCategories)**2

        for i in range(k, len(items)):
            profit, category = items[i]
            if category not in seenCategories and decreasingDuplicateProfits:
                # If this is a new category we haven't seen before, it's worth
                # considering taking it and replacing the one with the least profit
                # since it will increase the distinct_categories and potentially result
                # in a larger total_profit + distinct_categories^2.
                totalProfit -= decreasingDuplicateProfits.pop()
                totalProfit += profit
                seenCategories.add(category)
                ans = max(ans, totalProfit + len(seenCategories)**2)

        return ans
```

1731. 2814.py

Path: LeetCode-main/solutions/2814. Minimum Time Takes to Reach Destination Without Drowning/2814.py

```
class Solution:
    def minimumSeconds(self, land: list[list[str]]) -> int:
        self.DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(land)
        n = len(land[0])
        floodDist = self._getFloodDist(land)
        startPos = self._getStartPos(land, 'S')

        q = collections.deque([startPos])
        seen = {startPos}

        step = 1
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                for dx, dy in self.DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if land[x][y] == 'D':
```

```

        return step
    if floodDist[x][y] <= step or land[x][y] == 'X' or (x, y) in seen:
        continue
    q.append((x, y))
    seen.add((x, y))
    step += 1

return -1

def _getFloodDist(self, land: list[list[str]]) -> list[list[int]]:
    m = len(land)
    n = len(land[0])
    dist = [[math.inf] * n for _ in range(m)]
    q = collections.deque()
    seen = set()

    for i, row in enumerate(land):
        for j, cell in enumerate(row):
            if cell == '*':
                q.append((i, j))
                seen.add((i, j))

    d = 0
    while q:
        for _ in range(len(q)):
            i, j = q.popleft()
            dist[i][j] = d
            for dx, dy in self.DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if land[x][y] in 'XD' or (x, y) in seen:
                    continue
                q.append((x, y))
                seen.add((x, y))
            d += 1

    return dist

def _getStartPos(self, land: list[list[str]], c: str) -> tuple[int, int]:
    for i, row in enumerate(land):
        for j, cell in enumerate(row):
            if cell == c:
                return i, j

```

1732. 2815.py

Path: LeetCode-main/solutions/2815. Max Pair Sum in an Array/2815.py

```

class Solution:
    def maxSum(self, nums: list[int]) -> int:
        ans = 0
        # maxNum[i] := the maximum num we met so far with the maximum digit i
        maxNum = [0] * 10

        def getMaxDigit(num: int) -> int:
            maxDigit = 0
            while num > 0:
                maxDigit = max(maxDigit, num % 10)
                num //= 10
            return maxDigit

        for num in nums:
            d = getMaxDigit(num)
            if maxNum[d] > 0:
                ans = max(ans, num + maxNum[d])
            maxNum[d] = max(maxNum[d], num)

        return -1 if ans == 0 else ans

```

1733. 2816-2.py

Path: LeetCode-main/solutions/2816. Double a Number Represented as a Linked List/2816-2.py

```

class Solution:
    def doubleIt(self, head: ListNode | None) -> ListNode | None:
        if head.val >= 5:
            head = ListNode(0, head)

        curr = head

        while curr:
            curr.val *= 2
            curr.val %= 10
            if curr.next and curr.next.val >= 5:
                curr.val += 1
            curr = curr.next

        return head

```

1734. 2816.py

Path: LeetCode-main/solutions/2816. Double a Number Represented as a Linked List/2816.py

```

class Solution:
    def doubleIt(self, head: ListNode | None) -> ListNode | None:
        def getCarry(node: ListNode | None) -> ListNode | None:
            val = node.val * 2
            if node.next:
                val += getCarry(node.next)
            node.val = val % 10
            return val // 10

        if getCarry(head) == 1:
            return ListNode(1, head)
        return head

```

1735. 2817.py

Path: LeetCode-main/solutions/2817. Minimum Absolute Difference Between Elements With Constraint/2817.py

```
from sortedcontainers import SortedSet

class Solution:
    def minAbsoluteDifference(self, nums: list[int], x: int) -> int:
        ans = math.inf
        seen = SortedSet()

        for i in range(x, len(nums)):
            seen.add(nums[i - x])
            it = seen.bisect_left(nums[i])
            if it != len(seen):
                ans = min(ans, seen[it] - nums[i])
            if it != 0:
                ans = min(ans, nums[i] - seen[it - 1])

        return ans
```

1736. 2818.py

Path: LeetCode-main/solutions/2818. Apply Operations to Maximize Score/2818.py

```
class Solution:
    def maximumScore(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        ans = 1
        minPrimeFactors = self._sieveEratosthenes(max(nums) + 1)
        primeScores = [self._getPrimeScore(num, minPrimeFactors) for num in nums]
        # left[i] := the next index on the left (if any)
        #           s.t. primeScores[left[i]] >= primeScores[i]
        left = [-1] * n
        # right[i] := the next index on the right (if any)
        #           s.t. primeScores[right[i]] > primeScores[i]
        right = [n] * n
        stack = []

        # Find the next indices on the left where `primeScores` are greater or equal.
        for i in reversed(range(n)):
            while stack and primeScores[stack[-1]] <= primeScores[i]:
                left[stack.pop()] = i
            stack.append(i)

        stack = []

        # Find the next indices on the right where `primeScores` are greater.
        for i in range(n):
            while stack and primeScores[stack[-1]] < primeScores[i]:
                right[stack.pop()] = i
            stack.append(i)

        numAndIndexes = [(num, i) for i, num in enumerate(nums)]

        def modPow(x: int, n: int) -> int:
            if n == 0:
                return 1
            if n % 2 == 1:
                return x * modPow(x, n - 1) % MOD
```

```

        return modPow(x * x % MOD, n // 2)

for num, i in sorted(numAndIndexes, key=lambda x: (-x[0], x[1])):
    # nums[i] is the maximum value in the range [left[i] + 1, right[i] - 1]
    # So, there are (i - left[i]) * (right[i] - 1) ranges where nums[i] will
    # be chosen.
    rangeCount = (i - left[i]) * (right[i] - i)
    actualCount = min(rangeCount, k)
    k -= actualCount
    ans *= modPow(num, actualCount)
    ans %= MOD

return ans

def _sieveEratosthenes(self, n: int) -> list[int]:
    """Gets the minimum prime factor of i, where 2 <= i <= n."""
    minPrimeFactors = [i for i in range(n + 1)]
    for i in range(2, int(n**0.5) + 1):
        if minPrimeFactors[i] == i: # `i` is prime.
            for j in range(i * i, n, i):
                minPrimeFactors[j] = min(minPrimeFactors[j], i)
    return minPrimeFactors

def _getPrimeScore(self, num: int, minPrimeFactors: list[int]) -> int:
    primeFactors = set()
    while num > 1:
        divisor = minPrimeFactors[num]
        primeFactors.add(divisor)
        while num % divisor == 0:
            num //= divisor
    return len(primeFactors)

```

1737. 2819.py

Path: LeetCode-main/solutions/2819. Minimum Relative Loss After Buying Chocolates/2819.py

```

class Solution:
    def minimumRelativeLosses(
        self,
        prices: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        ans = []

        prices.sort()

        prefix = list(itertools.accumulate(prices, initial=0))

        for k, m in queries:
            countFront = self._getCountFront(k, m, prices)
            countBack = m - countFront
            ans.append(self._getRelativeLoss(countFront, countBack, k, prefix))

        return ans

    def _getCountFront(
        self,
        k: int,
        m: int,
        prices: list[int],
    ) -> int:
        """Returns `countFront` for query (k, m).

```

Returns `countFront` for query (k, m) s.t. picking the first `countFront` and the last `m - countFront` chocolates is optimal.

Define `loss[i]` := the relative loss of picking `prices[i]`.

1. For `prices[i] <= k`, Bob pays `prices[i]` while Alice pays 0.
Thus, `loss[i] = prices[i] - 0 = prices[i]`.
2. For `prices[i] > k`, Bob pays `k` while Alice pays `prices[i] - k`.
Thus, `loss[i] = k - (prices[i] - k) = 2 * k - prices[i]`.

By observation, we deduce that it is always better to pick from the front or the back since `loss[i]` is increasing for 1. and is decreasing for 2.

Assume that picking `left` chocolates from the left and `right = m - left` chocolates from the right is optimal. Therefore, we are selecting chocolates from `prices[0..left - 1]` and `prices[n - right..n - 1]`.

To determine the optimal `left` in each iteration, we simply compare `loss[left]` with `loss[n - right]` if `loss[left] < loss[n - right]`, it's worth increasing `left`.

"""

```
n = len(prices)
countNoGreaterThanOrK = bisect.bisect_right(prices, k)
l = 0
r = min(countNoGreaterThanOrK, m)
```

```
while l < r:
    mid = (l + r) // 2
    right = m - mid
    # Picking prices[mid] is better than picking prices[n - right].
    if prices[mid] < 2 * k - prices[n - right]:
        l = mid + 1
    else:
        r = mid
```

```
return l
```

```
def _getRelativeLoss(
    self,
    countFront: int,
    countBack: int,
    k: int,
    prefix: list[int],
) -> int:
    """
    Returns the relative loss of picking `countFront` and `countBack`
    chocolates.
    """
    lossFront = prefix[countFront]
    lossBack = 2 * k * countBack - (prefix[-1] - prefix[-countBack - 1])
    return lossFront + lossBack
```

1738. 282.py

Path: `LeetCode-main/solutions/282. Expression Add Operators/282.py`

```

class Solution:
    def addOperators(self, num: str, target: int) -> list[str]:
        ans = []

        def dfs(start: int, prev: int, eval: int, path: list[str]) -> None:
            if start == len(num):
                if eval == target:
                    ans.append(''.join(path))
                return

            for i in range(start, len(num)):
                if i > start and num[start] == '0':
                    return
                s = num[start:i + 1]
                curr = int(s)
                if start == 0:
                    path.append(s)
                    dfs(i + 1, curr, curr, path)
                    path.pop()
                else:
                    for op in ['+', '-', '*']:
                        path.append(op + s)
                        if op == '+':
                            dfs(i + 1, curr, eval + curr, path)
                        elif op == '-':
                            dfs(i + 1, -curr, eval - curr, path)
                        else:
                            dfs(i + 1, prev * curr, eval - prev + prev * curr, path)
                        path.pop()

        dfs(0, 0, 0, [])
        return ans

```

1739. 2824.py

Path: LeetCode-main/solutions/2824. Count Pairs Whose Sum is Less than Target/2824.py

```

class Solution:
    def countPairs(self, nums: list[int], target: int) -> int:
        return sum(nums[i] + nums[j] < target
                    for i in range(len(nums))
                    for j in range(i + 1, len(nums)))

```

1740. 2825.py

Path: LeetCode-main/solutions/2825. Make String a Subsequence Using Cyclic Increments/2825.py

```

class Solution:
    def canMakeSubsequence(self, str1: str, str2: str) -> bool:
        i = 0 # str2's index

        for c in str1:
            if c == str2[i] or chr(
                ord('a') + (ord(c) - ord('a') + 1) % 26) == str2[i]:
                i += 1
            if i == len(str2):
                return True

        return False

```

1741. 2826.py

Path: LeetCode-main/solutions/2826. Sorting Three Groups/2826.py

```
class Solution:
    def minimumOperations(self, nums: list[int]) -> int:
        # dp[i] := the longest non-decreasing subsequence so far with numbers in [1..i]
        dp = [0] * 4

        for num in nums:
            dp[num] += 1 # Append num to the sequence so far.
            dp[2] = max(dp[2], dp[1])
            dp[3] = max(dp[3], dp[2])

        return len(nums) - dp[3]
```

1742. 2828.py

Path: LeetCode-main/solutions/2828. Check if a String Is an Acronym of Words/2828.py

```
class Solution:
    def isAcronym(self, words: list[str], s: str) -> bool:
        return (len(words) == len(s) and
                all(word[0] == c for word, c in zip(words, s)))
```

1743. 2829.py

Path: LeetCode-main/solutions/2829. Determine the Minimum Sum of a k-avoiding Array/2829.py

```
class Solution:
    def minimumSum(self, n: int, k: int) -> int:
        # These are the unique pairs that sum up to k:
        # (1, k - 1), (2, k - 2), ..., (ceil(k // 2), floor(k // 2)).
        # Our optimal strategy is to select 1, 2, ..., floor(k // 2), and then
        # choose k, k + 1, ... if necessary, as selecting any number in the range
        # [ceil(k // 2), k - 1] will result in a pair summing up to k.

        def trapezoid(a: int, b: int) -> int:
            """Returns sum(a..b)."""
            return (a + b) * (b - a + 1) // 2

        mid = k // 2 # floor(k // 2)
        if n <= mid:
            return trapezoid(1, n)
        return trapezoid(1, mid) + trapezoid(k, k + (n - mid - 1))
```

1744. 283.py

Path: LeetCode-main/solutions/283. Move Zeroes/283.py

```

class Solution:
    def moveZeroes(self, nums: list[int]) -> None:
        j = 0
        for num in nums:
            if num != 0:
                nums[j] = num
                j += 1

        for i in range(j, len(nums)):
            nums[i] = 0

```

1745. 2830.py

Path: LeetCode-main/solutions/2830. Maximize the Profit as the Salesman/2830.py

```

class Solution:
    def maximizeTheProfit(self, n: int, offers: list[list[int]]) -> int:
        # dp[i] := the maximum amount of gold of selling the first i houses
        dp = [0] * (n + 1)
        endToStartAndGolds = [[] for _ in range(n)]

        for start, end, gold in offers:
            endToStartAndGolds[end].append((start, gold))

        for end in range(1, n + 1):
            # Get at least the same gold as selling the first `end - 1` houses.
            dp[end] = dp[end - 1]
            for start, gold in endToStartAndGolds[end - 1]:
                dp[end] = max(dp[end], dp[start] + gold)

        return dp[n]

```

1746. 2831-2.py

Path: LeetCode-main/solutions/2831. Find the Longest Equal Subarray/2831-2.py

```

class Solution:
    def longestEqualSubarray(self, nums: list[int], k: int) -> int:
        ans = 0
        count = collections.Counter()

        # l and r track the maximum window instead of the valid window.
        l = 0
        for r, num in enumerate(nums):
            count[num] += 1
            ans = max(ans, count[num])
            if r - l + 1 - k > ans:
                count[nums[l]] -= 1
                l += 1

        return ans

```

1747. 2831.py

Path: LeetCode-main/solutions/2831. Find the Longest Equal Subarray/2831.py

```

class Solution:
    def longestEqualSubarray(self, nums: list[int], k: int) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, num in enumerate(nums):
            count[num] += 1
            ans = max(ans, count[num])
            while r - l + 1 - k > ans:
                count[nums[l]] -= 1
                l += 1

        return ans

```

1748. 2832.py

Path: LeetCode-main/solutions/2832. Maximal Range That Each Element Is Maximum in It/2832.py

```

class Solution:
    def maximumLengthOfRanges(self, nums: list[int]) -> list[int]:
        ans = [0] * len(nums)
        stack = [] # a decreasing stack

        for i in range(len(nums) + 1):
            while stack and (i == len(nums) or nums[stack[-1]] < nums[i]):
                index = stack.pop()
                left = stack[-1] if stack else -1
                ans[index] = i - left - 1
            stack.append(i)

        return ans

```

1749. 2833.py

Path: LeetCode-main/solutions/2833. Furthest Point From Origin/2833.py

```

class Solution:
    def furthestDistanceFromOrigin(self, moves: str) -> int:
        return abs(moves.count('L') - moves.count('R')) + moves.count('_')

```

1750. 2834.py

Path: LeetCode-main/solutions/2834. Find the Minimum Possible Sum of a Beautiful Array/2834.py

```

class Solution:
    # Same as 2829. Determine the Minimum Sum of a k-avoiding Array
    def minimumPossibleSum(self, n: int, target: int) -> int:
        # These are the unique pairs that sum up to k (target):
        # (1, k - 1), (2, k - 2), ..., (ceil(k // 2), floor(k // 2)).
        # Our optimal strategy is to select 1, 2, ..., floor(k // 2), and then
        # choose k, k + 1, ... if necessary, as selecting any number in the range
        # [ceil(k // 2), k - 1] will result in a pair summing up to k.
        MOD = 1_000_000_007

        def trapezoid(a: int, b: int) -> int:
            """Returns sum(a..b)."""
            return (a + b) * (b - a + 1) // 2

        mid = target // 2 # floor(k // 2)
        if n <= mid:
            return trapezoid(1, n)
        return (trapezoid(1, mid) + trapezoid(target, target + (n - mid - 1))) % MOD

```

1751. 2835.py

Path: LeetCode-main/solutions/2835. Minimum Operations to Form Subsequence With Target Sum/2835.py

```

class Solution:
    def minOperations(self, nums: list[int], target: int) -> int:
        NO_MISSING_BIT = 31
        maxBit = 31
        ans = 0
        minMissingBit = NO_MISSING_BIT
        # count[i] := the number of occurrences of 2^i
        count = collections.Counter(int(math.log2(num)) for num in nums)

        for bit in range(maxBit):
            # Check if `bit` is in the target.
            if target >> bit & 1:
                # If there are available bits, use one bit.
                if count[bit] > 0:
                    count[bit] -= 1
                else:
                    minMissingBit = min(minMissingBit, bit)
            # If we previously missed a bit and there are available bits.
            if minMissingBit != NO_MISSING_BIT and count[bit] > 0:
                count[bit] -= 1
                # Count the operations to break `bit` into `minMissingBit`.
                ans += bit - minMissingBit
                minMissingBit = NO_MISSING_BIT # Set it to an the invalid value.
            # Combining smaller numbers costs nothing.
            count[bit + 1] += count[bit] // 2

        # Check if all target bits have been covered, otherwise return -1.
        return ans if minMissingBit == maxBit else -1

```

1752. 2836.py

Path: LeetCode-main/solutions/2836. Maximize Value of Function in a Ball Passing Game/2836.py

```

class Solution:
    def getMaxFunctionValue(self, receiver: list[int], k: int) -> int:
        n = len(receiver)
        m = int(math.log2(k)) + 1
        ans = 0
        # jump[i][j] := the the node you reach after jumping 2^j steps from i
        jump = [[0] * m for _ in range(n)]
        # summ[i][j] := the sum of the first 2^j nodes you reach when jumping from i
        summ = [[0] * m for _ in range(n)]

        for i in range(n):
            jump[i][0] = receiver[i]
            summ[i][0] = receiver[i]

        # Calculate binary lifting.
        for j in range(1, m):
            for i in range(n):
                midNode = jump[i][j - 1]
                # the the node you reach after jumping 2^j steps from i
                # = the node you reach after jumping 2^(j - 1) steps from i
                # + the node you reach after jumping another 2^(j - 1) steps
                jump[i][j] = jump[midNode][j - 1]
                # the sum of the first 2^j nodes you reach when jumping from i
                # = the sum of the first 2^(j - 1) nodes you reach when jumping from i
                # + the sum of another 2^(j - 1) nodes you reach
                summ[i][j] = summ[i][j - 1] + summ[midNode][j - 1]

        for i in range(n):
            currSum = i
            currPos = i
            for j in range(m):
                if (k >> j) & 1 == 1:
                    currSum += summ[currPos][j]
                    currPos = jump[currPos][j]
            ans = max(ans, currSum)

        return ans

```

1753. 2838.py

Path: LeetCode-main/solutions/2838. Maximum Coins Heroes Can Collect/2838.py

```

class Solution:
    def maximumCoins(
        self,
        heroes: list[int],
        monsters: list[int],
        coins: list[int]
    ) -> list[int]:
        monsterAndCoins = sorted(list(zip(monsters, coins)))
        coinsPrefix = list(itertools.accumulate(
            (coin for _, coin in monsterAndCoins),
            initial=0))
        return [coinsPrefix[self._firstGreaterEqual(monsterAndCoins, hero)]
                for hero in heroes]

    def _firstGreaterEqual(
        self,
        monsterAndCoins: list[tuple[int, int]],
        hero: int,
    ) -> int:
        l, r = 0, len(monsterAndCoins)
        while l < r:
            m = (l + r) // 2
            if monsterAndCoins[m][0] > hero:
                r = m
            else:
                l = m + 1
        return l

```

1754. 2839.py

Path: LeetCode-main/solutions/2839. Check if Strings Can be Made Equal With Operations I/2839.py

```

class Solution:
    def canBeEqual(self, s1: str, s2: str) -> bool:
        def swappedStrings(s: str) -> list[str]:
            chars = list(s)
            return [chars,
                    ''.join([chars[2], chars[1], chars[0], chars[3]]),
                    ''.join([chars[0], chars[3], chars[2], chars[1]]),
                    ''.join([chars[2], chars[3], chars[0], chars[1]])]

        return any(a == b
                   for a in swappedStrings(s1)
                   for b in swappedStrings(s2))

```

1755. 284.py

Path: LeetCode-main/solutions/284. Peeking Iterator/284.py

```

class PeekingIterator:
    def __init__(self, iterator: Iterator):
        self.iterator = iterator
        self.buffer = self.iterator.next() if self.iterator.hasNext() else None

    def peek(self) -> int:
        """
        Returns the next element in the iteration without advancing the iterator.
        """
        return self.buffer

    def next(self) -> int:
        next = self.buffer
        self.buffer = self.iterator.next() if self.iterator.hasNext() else None
        return next

    def hasNext(self) -> bool:
        return self.buffer is not None

```

1756. 2840.py

Path: LeetCode-main/solutions/2840. Check if Strings Can be Made Equal With Operations II/2840.py

```

class Solution:
    def checkStrings(self, s1: str, s2: str) -> bool:
        count = [collections.Counter() for _ in range(2)]

        for i, (a, b) in enumerate(zip(s1, s2)):
            count[i % 2][a] += 1
            count[i % 2][b] -= 1

        return (all(freq == 0 for freq in count[0].values()) and
                all(freq == 0 for freq in count[1].values()))

```

1757. 2841.py

Path: LeetCode-main/solutions/2841. Maximum Sum of Almost Unique Subarray/2841.py

```

class Solution:
    def maxSum(self, nums: list[int], m: int, k: int) -> int:
        ans = 0
        summ = 0
        count = collections.Counter()

        for i, num in enumerate(nums):
            summ += num
            count[num] += 1
            if i >= k:
                numToRemove = nums[i - k]
                summ -= numToRemove
                count[numToRemove] -= 1
                if count[numToRemove] == 0:
                    del count[numToRemove]
            if len(count) >= m:
                ans = max(ans, summ)

        return ans

```

1758. 2842.py

Path: LeetCode-main/solutions/2842. Count K-Subsequences of a String With Maximum Beauty Solved/2842.py

```
class Solution:
    def countKSubsequencesWithMaxBeauty(self, s: str, k: int) -> int:
        MOD = 1_000_000_007
        count = collections.Counter(s)
        if len(count) < k:
            return 0

        ans = 1
        # freqCount := (f(c), # of chars with f(c))
        freqCount = collections.Counter(count.values())

        for fc, numOfChars in list(sorted(freqCount.items(), reverse=True)):
            if numOfChars >= k:
                ans *= math.comb(numOfChars, k) * pow(fc, k, MOD)
                return ans % MOD
            ans *= pow(fc, numOfChars, MOD)
            ans %= MOD
            k -= numOfChars
```

1759. 2843.py

Path: LeetCode-main/solutions/2843. Count Symmetric Integers/2843.py

```
class Solution:
    def countSymmetricIntegers(self, low: int, high: int) -> int:
        def isSymmetricInteger(num: int) -> bool:
            if num >= 10 and num <= 99:
                return num // 10 == num % 10
            if num >= 1000 and num <= 9999:
                left = num // 100
                right = num % 100
                return left // 10 + left % 10 == right // 10 + right % 10
            return False

        return sum(isSymmetricInteger(num) for num in range(low, high + 1))
```

1760. 2844.py

Path: LeetCode-main/solutions/2844. Minimum Operations to Make a Special Number/2844.py


```

class Solution:
    def minimumOperations(self, num: str) -> int:
        n = len(num)
        seenFive = False
        seenZero = False

        for i in range(n - 1, -1, -1):
            if seenZero and num[i] == '0': # '00'
                return n - i - 2
            if seenZero and num[i] == '5': # '50'
                return n - i - 2
            if seenFive and num[i] == '2': # '25'
                return n - i - 2
            if seenFive and num[i] == '7': # '75'
                return n - i - 2
            seenZero = seenZero or num[i] == '0'
            seenFive = seenFive or num[i] == '5'

        return n - 1 if seenZero else n

```

1761. 2845.py

Path: LeetCode-main/solutions/2845. Count of Interesting Subarrays/2845.py

```

class Solution:
    def countInterestingSubarrays(
        self,
        nums: list[int],
        modulo: int,
        k: int,
    ) -> int:
        ans = 0
        prefix = 0 # (number of nums[i] % modulo == k so far) % modulo
        prefixCount = collections.Counter({0: 1})

        for num in nums:
            if num % modulo == k:
                prefix = (prefix + 1) % modulo
            ans += prefixCount[(prefix - k + modulo) % modulo]
            prefixCount[prefix] += 1

        return ans

```

1762. 2846.py

Path: LeetCode-main/solutions/2846. Minimum Edge Weight Equilibrium Queries in a Tree/2846.py

```

class Solution:
    def minOperationsQueries(
        self,
        n: int,
        edges: list[list[int]],
        queries: list[list[int]],
    ) -> list[int]:
        MAX = 26
        m = math.ceil(math.log2(n))
        graph = [[] for _ in range(n)]
        # jump[i][j] := the 2^j-th ancestor of i
        jump = [[0] * m for _ in range(n)]
        # depth[i] := the depth of i

```

```

depth = [0] * n
# count[i][j] := the count of j from root to i, where 1 <= j <= 26
count = [[] for _ in range(n)]

for u, v, w in edges:
    graph[u].append((v, w))
    graph[v].append((u, w))

count[0] = [0] * (MAX + 1)
self._dfs(graph, 0, -1, jump, depth, count)

for j in range(1, m):
    for i in range(n):
        jump[i][j] = jump[jump[i][j - 1]][j - 1]

def getMinOperations(u: int, v: int) -> int:
    """
    Returns the minimum number of operations to make the edge weight
    equilibrium between (u, v).
    """
    lca = self._getLCA(u, v, jump, depth)
    # the number of edges between (u, v).
    numEdges = depth[u] + depth[v] - 2 * depth[lca]
    # the maximum frequency of edges between (u, v)
    maxFreq = max(count[u][j] + count[v][j] - 2 * count[lca][j]
                  for j in range(1, MAX + 1))
    return numEdges - maxFreq

return [getMinOperations(u, v) for u, v in queries]

def _dfs(
    self,
    graph: list[list[tuple[int, int]]],
    u: int,
    prev: int,
    jump: list[list[int]],
    depth: list[int],
    count: list[list[int]]
) -> None:
    for v, w in graph[u]:
        if v == prev:
            continue
        jump[v][0] = u
        depth[v] = depth[u] + 1
        count[v] = count[u][:]
        count[v][w] += 1
        self._dfs(graph, v, u, jump, depth, count)

def _getLCA(
    self,
    u: int,
    v: int,
    jump: list[list[int]],
    depth: list[int]
) -> int:
    """Returns the lca(u, v) by binary jump."""
    # v is always deeper than u.
    if depth[u] > depth[v]:
        return self._getLCA(v, u, jump, depth)
    # Jump v to the same height of u.
    for j in range(len(jump[0])):
        if depth[v] - depth[u] >> j & 1:
            v = jump[v][j]
    if u == v:
        return u

```

```
# Jump u and v to the node right below the lca.
for j in range(len(jump[0]) - 1, -1, -1):
    if jump[u][j] != jump[v][j]:
        u = jump[u][j]
        v = jump[v][j]
return jump[u][0]
```

1763. 2847.py

Path: LeetCode-main/solutions/2847. Smallest Number With Given Digit Product/2847.py

```
class Solution:
    def smallestNumber(self, n: int) -> str:
        if n <= 9:
            return str(n)

        ans = []

        for divisor in range(9, 1, -1):
            while n % divisor == 0:
                ans.append(str(divisor))
                n //= divisor

        return '-1' if n > 1 else ''.join(reversed(ans))
```

1764. 2848.py

Path: LeetCode-main/solutions/2848. Points That Intersect With Cars/2848.py

```
class Solution:
    def numberOfPoints(self, nums: list[list[int]]) -> int:
        MAX = 100
        ans = 0
        runningSum = 0
        count = [0] * (MAX + 2)

        for start, end in nums:
            count[start] += 1
            count[end + 1] -= 1

        for i in range(1, MAX + 1):
            runningSum += count[i]
            if runningSum > 0:
                ans += 1

        return ans
```

1765. 2849.py

Path: LeetCode-main/solutions/2849. Determine if a Cell Is Reachable at a Given Time/2849.py

```

class Solution:
    def isReachableAtTime(
        self,
        sx: int,
        sy: int,
        fx: int,
        fy: int,
        t: int,
    ) -> bool:
        minStep = max(abs(sx - fx), abs(sy - fy))
        return t != 1 if minStep == 0 else minStep <= t

```

1766. 285.py

Path: LeetCode-main/solutions/285. Inorder Successor in BST/285.py

```

class Solution:
    def inorderSuccessor(
        self,
        root: TreeNode | None,
        p: TreeNode | None,
    ) -> TreeNode | None:
        if not root:
            return None
        if root.val <= p.val:
            return self.inorderSuccessor(root.right, p)
        return self.inorderSuccessor(root.left, p) or root

```

1767. 2850.py

Path: LeetCode-main/solutions/2850. Minimum Moves to Spread Stones Over Grid/2850.py

```

class Solution:
    def minimumMoves(self, grid: list[list[int]]) -> int:
        if sum(row.count(0) for row in grid) == 0:
            return 0

        ans = math.inf

        for i in range(3):
            for j in range(3):
                if grid[i][j] == 0:
                    for x in range(3):
                        for y in range(3):
                            if grid[x][y] > 1:
                                grid[x][y] -= 1
                                grid[i][j] += 1
                                ans = min(ans, abs(x - i) + abs(y - j) +
                                           self.minimumMoves(grid))
                                grid[x][y] += 1
                                grid[i][j] -= 1

        return ans

```

1768. 2851.py

Path: LeetCode-main/solutions/2851. String Transformation/2851.py

```

class Solution:
    # This dynamic programming table dp[k][i] represents the number of ways to
    # rearrange the String s after k steps such that it starts with s[i].
    # A String can be rotated from 1 to n - 1 times. The transition rule is
    # dp[k][i] = sum(dp[k - 1][j]) for all j != i. For example, when n = 4 and
    # k = 3, the table looks like this:
    #
    # -----
    # |      | i = 0 | i = 1 | i = 2 | i = 3 | sum = (n - 1)^k |
    # -----
    # | k = 0 |   1   |   0   |   0   |   0   |         1         |
    # | k = 1 |   0   |   1   |   1   |   1   |         3         |
    # | k = 2 |   3   |   2   |   2   |   2   |         9         |
    # | k = 3 |   6   |   7   |   7   |   7   |        27         |
    # -----
    #
    # By observation, we have
    # * dp[k][!0] = ((n - 1)^k - (-1)^k) / n
    # * dp[k][0] = dp[k][!0] + (-1)^k
    def numberOfWays(self, s: str, t: str, k: int) -> int:
        MOD = 1_000_000_007
        n = len(s)
        negOnePowK = 1 if k % 2 == 0 else -1 # (-1)^k
        z = self._zFunction(s + t + t)
        # indices in `s` s.t. for each `i` in the returned indices,
        # `s[i..n) + s[0..i) = t`.
        indices = [i - n for i in range(n, n + n) if z[i] >= n]
        dp = [0] * 2 # dp[0] := dp[k][0]; dp[1] := dp[k][!0]
        dp[1] = (pow(n - 1, k, MOD) - negOnePowK) * pow(n, MOD - 2, MOD)
        dp[0] = dp[1] + negOnePowK
        return sum(dp[0] if index == 0 else dp[1] for index in indices) % MOD

    def _zFunction(self, s: str) -> list[int]:
        """
        Returns the z array, where z[i] is the length of the longest prefix of
        s[i..n) which is also a prefix of s.

        https://cp-algorithms.com/string/z-function.html#implementation
        """
        n = len(s)
        z = [0] * n
        l = 0
        r = 0
        for i in range(1, n):
            if i < r:
                z[i] = min(r - i, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] > r:
                l = i
                r = i + z[i]
        return z

```

1769. 2852.py

Path: LeetCode-main/solutions/2852. Sum of Remoteness of All Cells/2852.py

```

class Solution:
    def sumRemoteness(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        n = len(grid)
        summ = sum(max(0, cell) for row in grid for cell in row)
        ans = 0

        def dfs(i: int, j: int) -> tuple[int, int]:
            """
            Returns the (count, componentSum) of the connected component that contains
            (x, y).
            """
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return (0, 0)
            if grid[i][j] == -1:
                return (0, 0)

            count = 1
            componentSum = grid[i][j]
            grid[i][j] = -1 # Mark as visited.

            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                nextCount, nextComponentSum = dfs(x, y)
                count += nextCount
                componentSum += nextComponentSum

            return (count, componentSum)

        for i in range(n):
            for j in range(n):
                if grid[i][j] > 0:
                    count, componentSum = dfs(i, j)
                    ans += (summ - componentSum) * count

        return ans

```

1770. 2855.py

Path: LeetCode-main/solutions/2855. Minimum Right Shifts to Sort the Array/2855.py

```

class Solution:
    def minimumRightShifts(self, nums: list[int]) -> int:
        count = 0

        for i, (a, b) in enumerate(itertools.pairwise(nums)):
            if a > b:
                count += 1
                pivot = i

        if count == 0:
            return 0
        if count > 1 or nums[-1] > nums[0]:
            return -1
        return len(nums) - pivot - 1

```

1771. 2856.py

Path: LeetCode-main/solutions/2856. Minimum Array Length After Pair Removals/2856.py

```
class Solution:
    def minLengthAfterRemovals(self, nums: list[int]) -> int:
        n = len(nums)
        count = collections.Counter(nums)
        maxFreq = max(count.values())

        # The number with the maximum frequency cancel all the other numbers.
        if maxFreq <= n / 2:
            return n % 2

        # The number with the maximum frequency cancel all the remaining numbers.
        return maxFreq - (n - maxFreq)
```

1772. 2857.py

Path: LeetCode-main/solutions/2857. Count Pairs of Points With Distance k/2857.py

```
class Solution:
    def countPairs(self, coordinates: list[list[int]], k: int) -> int:
        ans = 0

        for x in range(k + 1):
            y = k - x
            count = collections.Counter()
            for xi, yi in coordinates:
                ans += count[(xi ^ x, yi ^ y)]
                count[(xi, yi)] += 1

        return ans
```

1773. 2858.py

Path: LeetCode-main/solutions/2858. Minimum Edge Reversals So Every Node Is Reachable/2858.py

```

class Solution:
    def minEdgeReversals(self, n: int, edges: list[list[int]]) -> list[int]:
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append((v, True)) # 1 means (u -> v)
            graph[v].append((u, False)) # 0 means (v <- u)

        seen = {0}

        @functools.lru_cache(None)
        def dp(u: int) -> int:
            """
            Returns the minimum number of edge reversals so node u can reach every
            node in its subtree.
            """
            res = 0
            for v, isForward in graph[u]:
                if v in seen:
                    continue
                seen.add(v)
                res += dp(v) + (0 if isForward else 1)
            return res

        ans = [0] * n
        ans[0] = dp(0)

        def dfs(u: int) -> None:
            for v, isForward in graph[u]:
                if v in seen:
                    continue
                seen.add(v)
                ans[v] = ans[u] + (1 if isForward else -1)
                dfs(v)

        seen = {0}
        dfs(0)
        return ans

```

1774. 2859.py

Path: LeetCode-main/solutions/2859. Sum of Values at Indices With K Set Bits/2859.py

```

class Solution:
    def sumIndicesWithKSetBits(self, nums: list[int], k: int) -> int:
        return sum(num for i, num in enumerate(nums)
                    if i.bit_count() == k)

```

1775. 286.py

Path: LeetCode-main/solutions/286. Walls and Gates/286.py


```

class Solution:
    def wallsAndGates(self, rooms: list[list[int]]) -> None:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        INF = 2**31 - 1
        m = len(rooms)
        n = len(rooms[0])
        q = collections.deque((i, j)
                               for i in range(m)
                               for j in range(n)
                               if rooms[i][j] == 0)

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if rooms[x][y] != INF:
                    continue
                rooms[x][y] = rooms[i][j] + 1
                q.append((x, y))

```

1776. 2860.py

Path: LeetCode-main/solutions/2860. Happy Students/2860.py

```

class Solution:
    def countWays(self, nums: list[int]) -> int:
        return sum(a < i < b
                   for i, (a, b) in # i := the number of the selected numbers
                   enumerate(itertools.pairwise([-1] + sorted(nums) + [math.inf])))

```

1777. 2861.py

Path: LeetCode-main/solutions/2861. Maximum Number of Alloys/2861.py

```

class Solution:
    def maxNumberOfAlloys(self, n: int, k: int, budget: int,
                        composition: list[list[int]], stock: list[int],
                        costs: list[int]) -> int:

        l = 1
        r = 1_000_000_000

        def isPossible(m: int) -> bool:
            """Returns True if it's possible to create `m` alloys by using any machine."""
            # Try all the possible machines.
            for machine in composition:
                requiredMoney = 0
                for j in range(n):
                    requiredUnits = max(0, machine[j] * m - stock[j])
                    requiredMoney += requiredUnits * costs[j]
                if requiredMoney <= budget:
                    return True
            return False

        while l < r:
            m = (l + r) // 2
            if isPossible(m):
                l = m + 1
            else:
                r = m

        return l - 1

```

1778. 2862-2.py

Path: LeetCode-main/solutions/2862. Maximum Element-Sum of a Complete Subset of Indices/2862-2.py

```

class Solution:
    def maximumSum(self, nums: list[int]) -> int:
        ans = 0

        for oddPower in range(1, len(nums) + 1):
            summ = 0
            for num in range(1, len(nums) + 1):
                if num * num * oddPower > len(nums):
                    break
                summ += nums[oddPower * num * num - 1]
            ans = max(ans, summ)

        return ans

```

1779. 2862.py

Path: LeetCode-main/solutions/2862. Maximum Element-Sum of a Complete Subset of Indices/2862.py

```

class Solution:
    def maximumSum(self, nums: list[int]) -> int:
        ans = 0
        oddPowerToSum = collections.Counter()

        def divideSquares(val: int) -> int:
            for num in range(2, val + 1):
                while val % (num * num) == 0:
                    val //= (num * num)
            return val

        for i, num in enumerate(nums):
            oddPower = divideSquares(i + 1)
            oddPowerToSum[oddPower] += num
            ans = max(ans, oddPowerToSum[oddPower])

        return ans

```

1780. 2863.py

Path: LeetCode-main/solutions/2863. Maximum Length of Semi-Decreasing Subarrays/2863.py

```

class Solution:
    def maxSubarrayLength(self, nums: list[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(nums) - 1, -1, -1):
            # If nums[stack[-1]] <= nums[i], stack[-1] is better than i.
            # So, no need to append it.
            if not stack or nums[stack[-1]] > nums[i]:
                stack.append(i)

        for i, num in enumerate(nums):
            while stack and num > nums[stack[-1]]:
                ans = max(ans, stack.pop() - i + 1)

        return ans

```

1781. 2864.py

Path: LeetCode-main/solutions/2864. Maximum Odd Binary Number/2864.py

```

class Solution:
    def maximumOddBinaryNumber(self, s: str) -> str:
        return '1' * (s.count('1') - 1) + '0' * s.count('0') + '1'

```

1782. 2865.py

Path: LeetCode-main/solutions/2865. Beautiful Towers I/2865.py

```
class Solution:
    def maximumSumOfHeights(self, heights: list[int]) -> int:
        n = len(heights)
        maxSum = [0] * n # maxSum[i] := the maximum sum with peak i

        def process(stack: list[int], i: int, summ: int) -> int:
            while len(stack) > 1 and heights[stack[-1]] > heights[i]:
                j = stack.pop()
                # The last abs(j - stack[-1]) heights are heights[j].
                summ -= abs(j - stack[-1]) * heights[j]
            # Put abs(i - stack[-1]) `maxHeight` in heights.
            summ += abs(i - stack[-1]) * heights[i]
            stack.append(i)
            return summ

        stack = [-1]
        summ = 0
        for i in range(len(heights)):
            summ = process(stack, i, summ)
            maxSum[i] = summ

        stack = [n]
        summ = 0
        for i in range(n - 1, -1, -1):
            summ = process(stack, i, summ)
            maxSum[i] += summ - heights[i]

        return max(maxSum)
```

1783. 2866.py

Path: LeetCode-main/solutions/2866. Beautiful Towers II/2866.py

```

class Solution:
    # Same as 2865. Beautiful Towers I
    def maximumSumOfHeights(self, maxHeights: list[int]) -> int:
        n = len(maxHeights)
        maxSum = [0] * n # maxSum[i] := the maximum sum with peak i

        def process(stack: list[int], i: int, summ: int) -> int:
            while len(stack) > 1 and maxHeights[stack[-1]] > maxHeights[i]:
                j = stack.pop()
                # The last abs(j - stack[-1]) heights are maxHeights[j].
                summ -= abs(j - stack[-1]) * maxHeights[j]
            # Put abs(i - stack[-1]) `maxHeight` in heights.
            summ += abs(i - stack[-1]) * maxHeights[i]
            stack.append(i)
            return summ

        stack = [-1]
        summ = 0
        for i in range(len(maxHeights)):
            summ = process(stack, i, summ)
            maxSum[i] = summ

        stack = [n]
        summ = 0
        for i in range(n - 1, -1, -1):
            summ = process(stack, i, summ)
            maxSum[i] += summ - maxHeights[i]

        return max(maxSum)

```

1784. 2867.py

Path: LeetCode-main/solutions/2867. Count Valid Paths in a Tree/2867.py

```

class Solution:
    def countPaths(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        isPrime = self._sieveEratosthenes(n + 1)
        graph = [[] for _ in range(n + 1)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def dfs(u: int, prev: int) -> tuple[int, int]:
            nonlocal ans
            countZeroPrimePath = int(not isPrime[u])
            countOnePrimePath = int(isPrime[u])

            for v in graph[u]:
                if v == prev:
                    continue
                countZeroPrimeChildPath, countOnePrimeChildPath = dfs(v, u)
                ans += (countZeroPrimePath * countOnePrimeChildPath +
                        countOnePrimePath * countZeroPrimeChildPath)
                if isPrime[u]:
                    countOnePrimePath += countZeroPrimeChildPath
                else:
                    countZeroPrimePath += countZeroPrimeChildPath
                    countOnePrimePath += countOnePrimeChildPath

            return countZeroPrimePath, countOnePrimePath

        dfs(1, -1)
        return ans

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime

```

1785. 2868.py

Path: LeetCode-main/solutions/2868. The Wording Game/2868.py

```

class Solution:
    def canAliceWin(self, a: list[str], b: list[str]) -> bool:
        # words[0][i] := the biggest word starting with ('a' + i) for Alice
        # words[1][i] := the biggest word starting with ('a' + i) for Bob
        words = [[''] * 26 for _ in range(2)]

        # For each letter, only the biggest word is useful.
        for word in a:
            words[0][ord(word[0]) - ord('a')] = word

        for word in b:
            words[1][ord(word[0]) - ord('a')] = word

        # Find Alice's smallest word.
        i = 0
        while not words[0][i]:
            i += 1

        # 0 := Alice, 1 := Bob
        # Start with Alice, so it's Bob's turn now.
        turn = 1

        # Iterate through each letter until we find a winner.
        while True:
            # If the current player has a word that having the letter that is greater
            # than the opponent's word, choose it.
            if words[turn][i] and words[turn][i] > words[1 - turn][i]:
                # Choose the current words[turn][i].
                pass
            elif words[turn][i + 1]:
                # Choose the next words[turn][i + 1].
                i += 1
            else:
                # Game over. If it's Bob's turn, Alice wins, and vice versa.
                return turn == 1
            turn = 1 - turn

```

1786. 2869.py

Path: LeetCode-main/solutions/2869. Minimum Operations to Collect Elements/2869.py

```

class Solution:
    def minOperations(self, nums: list[int], k: int) -> int:
        seen = set()

        for i, num in enumerate(reversed(nums)):
            if num > k:
                continue
            seen.add(num)
            if len(seen) == k:
                return i + 1

```

1787. 287.py

Path: LeetCode-main/solutions/287. Find the Duplicate Number/287.py

```
class Solution:
    def findDuplicate(self, nums: list[int]) -> int:
        slow = nums[nums[0]]
        fast = nums[nums[nums[0]]]

        while slow != fast:
            slow = nums[slow]
            fast = nums[nums[fast]]

        slow = nums[0]

        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]

        return slow
```

1788. 2870.py

Path: LeetCode-main/solutions/2870. Minimum Number of Operations to Make Array Empty/2870.py

```
class Solution:
    def minOperations(self, nums: list[int]) -> int:
        count = collections.Counter(nums)
        if 1 in count.values():
            return -1
        return sum((freq + 2) // 3 for freq in count.values())
```

1789. 2871.py

Path: LeetCode-main/solutions/2871. Split Array Into Maximum Number of Subarrays/2871.py

```
class Solution:
    def maxSubarrays(self, nums: list[int]) -> int:
        ans = 0
        score = 0

        for num in nums:
            score = num if score == 0 else score & num
            if score == 0:
                ans += 1

        return max(1, ans)
```

1790. 2872.py

Path: LeetCode-main/solutions/2872. Maximum Number of K-Divisible Components/2872.py


```

class Solution:
    def maxKDivisibleComponents(
        self,
        n: int,
        edges: list[list[int]],
        values: list[int],
        k: int,
    ) -> int:
        ans = 0
        graph = [[] for _ in range(n)]

        def dfs(u: int, prev: int) -> int:
            nonlocal ans
            treeSum = values[u]

            for v in graph[u]:
                if v != prev:
                    treeSum += dfs(v, u)

            if treeSum % k == 0:
                ans += 1
            return treeSum

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        dfs(0, -1)
        return ans

```

1791. 2873.py

Path: LeetCode-main/solutions/2873. Maximum Value of an Ordered Triplet I/2873.py

```

class Solution:
    def maximumTripletValue(self, nums: list[int]) -> int:
        ans = 0
        maxDiff = 0 # max(nums[i] - nums[j])
        maxNum = 0 # max(nums[i])

        for num in nums:
            ans = max(ans, maxDiff * num) # num := nums[k]
            maxDiff = max(maxDiff, maxNum - num) # num := nums[j]
            maxNum = max(maxNum, num) # num := nums[i]

        return ans

```

1792. 2874.py

Path: LeetCode-main/solutions/2874. Maximum Value of an Ordered Triplet II/2874.py

```

class Solution:
    # Same as 2873. Maximum Value of an Ordered Triplet I
    def maximumTripletValue(self, nums: list[int]) -> int:
        ans = 0
        maxDiff = 0 # max(nums[i] - nums[j])
        maxNum = 0 # max(nums[i])

        for num in nums:
            ans = max(ans, maxDiff * num) # num := nums[k]
            maxDiff = max(maxDiff, maxNum - num) # num := nums[j]
            maxNum = max(maxNum, num) # num := nums[i]

        return ans

```

1793. 2875.py

Path: LeetCode-main/solutions/2875. Minimum Size Subarray in Infinite Array/2875.py

```

class Solution:
    def minSizeSubarray(self, nums: list[int], target: int) -> int:
        summ = sum(nums)
        n = len(nums)
        remainingTarget = target % summ
        repeatLength = (target // summ) * n
        if remainingTarget == 0:
            return repeatLength

        suffixPlusPrefixLength = n
        prefix = 0
        prefixToIndex = {0: -1}

        for i in range(2 * n):
            prefix += nums[i % n]
            if prefix - remainingTarget in prefixToIndex:
                suffixPlusPrefixLength = min(
                    suffixPlusPrefixLength,
                    i - prefixToIndex[prefix - remainingTarget])
            prefixToIndex[prefix] = i

        return -1 if suffixPlusPrefixLength == n else suffixPlusPrefixLength + repeatLength

```

1794. 2876.py

Path: LeetCode-main/solutions/2876. Count Visited Nodes in a Directed Graph/2876.py

```

class Solution:
    def countVisitedNodes(self, edges: list[int]) -> list[int]:
        n = len(edges)
        ans = [0] * n
        inDegrees = [0] * n
        seen = [False] * n
        stack = []

        for v in edges:
            inDegrees[v] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        # Push non-cyclic nodes to stack.
        while q:
            u = q.popleft()
            inDegrees[edges[u]] -= 1
            if inDegrees[edges[u]] == 0:
                q.append(edges[u])
            stack.append(u)
            seen[u] = True

        # Fill the length of cyclic nodes.
        for i in range(n):
            if not seen[i]:
                self._fillCycle(edges, i, seen, ans)

        # Fill the length of non-cyclic nodes.
        while stack:
            u = stack.pop()
            ans[u] = ans[edges[u]] + 1

        return ans

    def _fillCycle(
        self,
        edges: list[int],
        start: int,
        seen: list[bool],
        ans: list[int],
    ) -> None:
        cycleLength = 0
        u = start
        while not seen[u]:
            cycleLength += 1
            seen[u] = True
            u = edges[u]
        ans[start] = cycleLength
        u = edges[start]
        while u != start:
            ans[u] = cycleLength
            u = edges[u]

```

1795. 2877.py

Path: LeetCode-main/solutions/2877. Create a DataFrame from List/2877.py

```
import pandas as pd

def createDataFrame(student_data: list[list[int]]) -> pd.DataFrame:
    return pd.DataFrame(student_data, columns=['student_id', 'age'])
```

1796. 2878.py

Path: LeetCode-main/solutions/2878. Get the Size of a DataFrame/2878.py

```
import pandas as pd

def getDataframeSize(players: pd.DataFrame) -> list[int]:
    return [*players.shape]
```

1797. 2879.py

Path: LeetCode-main/solutions/2879. Display the First Three Rows/2879.py

```
import pandas as pd

def selectFirstRows(employees: pd.DataFrame) -> pd.DataFrame:
    return employees.head(3)
```

1798. 288.py

Path: LeetCode-main/solutions/288. Unique Word Abbreviation/288.py

```
class ValidWordAbbr:
    def __init__(self, dictionary: list[str]):
        self.dict = set(dictionary)
        # T := unique, F := not unique
        self.abbrUnique = {}

        for word in self.dict:
            abbr = self._getAbbr(word)
            self.abbrUnique[abbr] = abbr not in self.abbrUnique

    def isUnique(self, word: str) -> bool:
        abbr = self._getAbbr(word)
        return abbr not in self.abbrUnique or self.abbrUnique[abbr] and word in self.dict

    def _getAbbr(self, s: str) -> str:
        n = len(s)
        if n <= 2:
            return s
        return s[0] + str(n - 2) + s[-1]
```

1799. 2880.py

Path: LeetCode-main/solutions/2880. Select Data/2880.py

```
import pandas as pd

def selectData(students: pd.DataFrame) -> pd.DataFrame:
    return students.loc[students['student_id'] == 101, ['name', 'age']]
```

1800. 2881.py

Path: LeetCode-main/solutions/2881. Create a New Column/2881.py

```
import pandas as pd

def createBonusColumn(employees: pd.DataFrame) -> pd.DataFrame:
    employees['bonus'] = employees['salary'] * 2
    return employees
```

1801. 2882.py

Path: LeetCode-main/solutions/2882. Drop Duplicate Rows/2882.py

```
import pandas as pd

def dropDuplicateEmails(customers: pd.DataFrame) -> pd.DataFrame:
    customers.drop_duplicates(subset='email', keep='first', inplace=True)
    return customers
```

1802. 2883.py

Path: LeetCode-main/solutions/2883. Drop Missing Data/2883.py

```
import pandas as pd

def dropMissingData(students: pd.DataFrame) -> pd.DataFrame:
    students.dropna(subset=['name'], inplace=True)
    return students
```

1803. 2884.py

Path: LeetCode-main/solutions/2884. Modify Columns/2884.py

```
import pandas as pd

def modifySalaryColumn(employees: pd.DataFrame) -> pd.DataFrame:
    employees['salary'] = employees['salary'] * 2
    return employees
```

1804. 2885.py

Path: LeetCode-main/solutions/2885. Rename Columns/2885.py

```
import pandas as pd

def renameColumns(students: pd.DataFrame) -> pd.DataFrame:
    return students.rename(
        columns={
            "id": "student_id",
            "first": "first_name",
            "last": "last_name",
            "age": "age_in_years",
        }
    )
```

1805. 2886.py

Path: LeetCode-main/solutions/2886. Change Data Type/2886.py

```
import pandas as pd

def changeDatatype(students: pd.DataFrame) -> pd.DataFrame:
    return students.astype({'grade': int})
```

1806. 2887.py

Path: LeetCode-main/solutions/2887. Fill Missing Data/2887.py

```
import pandas as pd

def fillMissingValues(products: pd.DataFrame) -> pd.DataFrame:
    products['quantity'].fillna(0, inplace=True)
    return products
```

1807. 2888.py

Path: LeetCode-main/solutions/2888. Reshape Data: Concatenate/2888.py

```
import pandas as pd

def concatenateTables(df1: pd.DataFrame, df2: pd.DataFrame) -> pd.DataFrame:
    return pd.concat([df1, df2], axis=0)
```

1808. 2889.py

Path: LeetCode-main/solutions/2889. Reshape Data: Pivot/2889.py

```
import pandas as pd

def pivotTable(weather: pd.DataFrame) -> pd.DataFrame:
    return weather.pivot_table(
        index='month',
        columns='city',
        values='temperature',
        aggfunc='max',
    )
```

1809. 289.py

Path: LeetCode-main/solutions/289. Game of Life/289.py

```
class Solution:
    def gameOfLife(self, board: list[list[int]]) -> None:
        m = len(board)
        n = len(board[0])

        for i in range(m):
            for j in range(n):
                ones = 0
                for x in range(max(0, i - 1), min(m, i + 2)):
                    for y in range(max(0, j - 1), min(n, j + 2)):
                        ones += board[x][y] & 1
                # Any live cell with two or three live neighbors lives on to the next
                # generation.
                if board[i][j] == 1 and (ones == 3 or ones == 4):
                    board[i][j] |= 0b10
                # Any dead cell with exactly three live neighbors becomes a live cell,
                # as if by reproduction.
                if board[i][j] == 0 and ones == 3:
                    board[i][j] |= 0b10

        for i in range(m):
            for j in range(n):
                board[i][j] >>= 1
```

1810. 2890.py

Path: LeetCode-main/solutions/2890. Reshape Data: Melt/2890.py

```
import pandas as pd

def meltTable(report: pd.DataFrame) -> pd.DataFrame:
    return pd.melt(
        report,
        id_vars=['product'],
        var_name='quarter',
        value_name='sales',
    )
```

1811. 2891.py

Path: LeetCode-main/solutions/2891. Method Chaining/2891.py

```
import pandas as pd

def findHeavyAnimals(animals: pd.DataFrame) -> pd.DataFrame:
    return animals[animals['weight'] > 100].sort_values(
        by='weight',
        ascending=False
    )[['name']]
```

1812. 2892.py

Path: LeetCode-main/solutions/2892. Minimizing Array After Replacing Pairs With Their Product/2892.py

```
class Solution:
    def minArrayLength(self, nums: list[int], k: int) -> int:
        count = 0
        prod = -1

        for num in nums:
            if num == 0:
                return 1
            if prod != -1 and prod * num <= k:
                prod *= num
            else:
                prod = num
                count += 1

        return count
```

1813. 2894.py

Path: LeetCode-main/solutions/2894. Divisible and Non-divisible Sums Difference/2894.py

```
class Solution:
    def differenceOfSums(self, n: int, m: int) -> int:
        summ = (1 + n) * n // 2
        num2 = self._getDivisibleSum(n, m)
        num1 = summ - num2
        return num1 - num2

    def _getDivisibleSum(self, n: int, m: int) -> int:
        """Returns the sum of all the integers in [1, n] that are divisible by m."""
        last = n // m * m
        if last == 0:
            return 0
        first = m
        count = (last - first) // m + 1
        return (first + last) * count // 2
```

1814. 2895.py

Path: LeetCode-main/solutions/2895. Minimum Processing Time/2895.py


```
class Solution:
    def minProcessingTime(
        self,
        processorTime: list[int],
        tasks: list[int],
    ) -> int:
        return max(time + task
                    for (time, task) in zip(
                        sorted(processorTime),
                        sorted(tasks)[: -4]))
```

1815. 2896-2.py

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896-2.py

```
class Solution:
    def minOperations(self, s1: str, s2: str, x: int) -> int:
        diffIndices = [i for i, (a, b) in enumerate(zip(s1, s2))
                       if a != b]

        if not diffIndices:
            return 0

        # It's impossible to make two strings equal if there are odd number of
        # differences.
        if len(diffIndices) & 1:
            return -1

        # dp[i] := the minimum cost to correct diffIndices[i:]
        dp = [math.inf] * len(diffIndices) + [0]
        dp[-2] = x / 2

        for i in reversed(range(len(diffIndices) - 1)):
            dp[i] = min(dp[i + 1] + x / 2,
                       dp[i + 2] + diffIndices[i + 1] - diffIndices[i])

        return int(dp[0])
```

1816. 2896-3.py

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896-3.py

```

class Solution:
    def minOperations(self, s1: str, s2: str, x: int) -> int:
        diffIndices = [i for i, (a, b) in enumerate(zip(s1, s2))
                        if a != b]

        if not diffIndices:
            return 0

        # It's impossible to make two strings equal if there are odd number of
        # differences.
        if len(diffIndices) & 1:
            return -1

        # dp := the minimum cost to correct diffIndices[i:]
        # dpNext := the minimum cost to correct diffIndices[i + 1:]
        # dpNextNext := the minimum cost to correct diffIndices[i + 2:]
        dpNext = x / 2
        dpNextNext = 0

        for i in reversed(range(len(diffIndices) - 1)):
            dp = min(dpNext + x / 2,
                    dpNextNext + diffIndices[i + 1] - diffIndices[i])
            dpNextNext = dpNext
            dpNext = dp

        return int(dp)

```

1817. 2896.py

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896.py

```

class Solution:
    def minOperations(self, s1: str, s2: str, x: int) -> int:
        diffIndices = [i for i, (a, b) in enumerate(zip(s1, s2))
                        if a != b]

        if not diffIndices:
            return 0

        # It's impossible to make two strings equal if there are odd number of
        # differences.
        if len(diffIndices) & 1:
            return -1

        @functools.lru_cache(None)
        def dp(i: int) -> int:
            """Returns the minimum cost to correct diffIndices[i..n)."""
            if i == len(diffIndices):
                return 0
            if i == len(diffIndices) - 1:
                return x / 2
            return min(dp(i + 1) + x / 2,
                    dp(i + 2) + diffIndices[i + 1] - diffIndices[i])

        return int(dp(0))

```

1818. 2897.py

Path: LeetCode-main/solutions/2897. Apply Operations on Array to Maximize Sum of Squares/2897.py

```

class Solution:
    def maxSum(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        MAX_BIT = 30
        ans = 0
        # minIndices[i] := the minimum index in `optimalNums` that the i-th bit
        # should be moved to
        minIndices = [0] * MAX_BIT
        optimalNums = [0] * len(nums)

        for num in nums:
            for i in range(MAX_BIT):
                if num >> i & 1:
                    optimalNums[minIndices[i]] |= 1 << i
                    minIndices[i] += 1

        for i in range(k):
            ans += optimalNums[i]**2
            ans %= MOD

        return ans

```

1819. 2898.py

Path: LeetCode-main/solutions/2898. Maximum Linear Stock Score/2898.py

```

class Solution:
    def maxScore(self, prices: list[int]) -> int:
        groupIdToSum = collections.defaultdict(int)

        for i, price in enumerate(prices):
            groupIdToSum[price - i] += price

        return max(groupIdToSum.values())

```

1820. 2899.py

Path: LeetCode-main/solutions/2899. Last Visited Integers/2899.py

```

class Solution:
    def lastVisitedIntegers(self, words: list[str]) -> list[int]:
        ans = []
        nums = []
        k = 0

        for word in words:
            if word == 'prev':
                k += 1
                ans.append(-1 if k > len(nums) else nums[-k])
            else:
                k = 0
                nums.append(int(word))

        return ans

```

1821. 29.py

Path: LeetCode-main/solutions/29. Divide Two Integers/29.py

```
class Solution:
    def divide(self, dividend: int, divisor: int) -> int:
        # -2^{31} / -1 = 2^{31} will overflow, so return 2^{31} - 1.
        if dividend == -2**31 and divisor == -1:
            return 2**31 - 1

        sign = -1 if (dividend > 0) ^ (divisor > 0) else 1
        ans = 0
        dvd = abs(dividend)
        dvs = abs(divisor)

        while dvd >= dvs:
            k = 1
            while k * 2 * dvs <= dvd:
                k <<= 1
            dvd -= k * dvs
            ans += k

        return sign * ans
```

1822. 290.py

Path: LeetCode-main/solutions/290. Word Pattern/290.py

```
class Solution:
    def wordPattern(self, pattern: str, str: str) -> bool:
        t = str.split()
        return [*map(pattern.index, pattern)] == [*map(t.index, t)]
```

1823. 2900.py

Path: LeetCode-main/solutions/2900. Longest Unequal Adjacent Groups Subsequence I/2900.py

```
class Solution:
    def getWordsInLongestSubsequence(
        self,
        n: int,
        words: list[str],
        groups: list[int],
    ) -> list[str]:
        ans = []
        groupId = -1

        for word, group in zip(words, groups):
            if group != groupId:
                groupId = group
                ans.append(word)

        return ans
```

1824. 2901.py

Path: LeetCode-main/solutions/2901. Longest Unequal Adjacent Groups Subsequence II/2901.py

```

class Solution:
    def getWordsInLongestSubsequence(
        self,
        n: int,
        words: list[str],
        groups: list[int],
    ) -> list[str]:
        ans = []
        # dp[i] := the length of the longest subsequence ending in `words[i]`
        dp = [1] * n
        # prev[i] := the best index of words[i]
        prev = [-1] * n

        for i in range(1, n):
            for j in range(i):
                if groups[i] == groups[j]:
                    continue
                if len(words[i]) != len(words[j]):
                    continue
                if sum(a != b for a, b in zip(words[i], words[j])) != 1:
                    continue
                if dp[i] < dp[j] + 1:
                    dp[i] = dp[j] + 1
                    prev[i] = j

        # Find the last index of the subsequence.
        index = dp.index(max(dp))
        while index != -1:
            ans.append(words[index])
            index = prev[index]

        return ans[::-1]

```

1825. 2902.py

Path: LeetCode-main/solutions/2902. Count of Sub-Multisets With Bounded Sum/2902.py

```

class Solution:
    def countSubMultisets(self, nums: list[int], l: int, r: int) -> int:
        MOD = 1_000_000_007
        # dp[i] := the number of submultisets of `nums` with sum i
        dp = [1] + [0] * r
        count = collections.Counter(nums)
        zeros = count.pop(0, 0)

        for num, freq in count.items():
            # stride[i] := dp[i] + dp[i - num] + dp[i - 2 * num] + ...
            stride = dp.copy()
            for i in range(num, r + 1):
                stride[i] += stride[i - num]
            for i in range(r, 0, -1):
                if i >= num * (freq + 1):
                    # dp[i] + dp[i - num] + dp[i - freq * num]
                    dp[i] = stride[i] - stride[i - num * (freq + 1)]
                else:
                    dp[i] = stride[i]

        return (zeros + 1) * sum(dp[l:r + 1]) % MOD

```

1826. 2903.py

Path: LeetCode-main/solutions/2903. Find Indices With Index and Value Difference I/2903.py

```
class Solution:
    def findIndices(
        self,
        nums: list[int],
        indexDifference: int,
        valueDifference: int,
    ) -> list[int]:
        # nums[minIndex] := the minimum number with enough index different from the
        # current number
        minIndex = 0
        # nums[maxIndex] := the maximum number with enough index different from the
        # current number
        maxIndex = 0

        for i in range(indexDifference, len(nums)):
            if nums[i - indexDifference] < nums[minIndex]:
                minIndex = i - indexDifference
            if nums[i - indexDifference] > nums[maxIndex]:
                maxIndex = i - indexDifference
            if nums[i] - nums[minIndex] >= valueDifference:
                return [i, minIndex]
            if nums[maxIndex] - nums[i] >= valueDifference:
                return [i, maxIndex]

        return [-1, -1]
```

1827. 2904.py

Path: LeetCode-main/solutions/2904. Shortest and Lexicographically Smallest Beautiful String/2904.py

```
class Solution:
    # Same as 76. Minimum Window Substring
    def shortestBeautifulSubstring(self, s: str, k: int) -> str:
        bestLeft = -1
        minLength = len(s) + 1
        ones = 0

        l = 0
        for r, c in enumerate(s):
            if c == '1':
                ones += 1
            while ones == k:
                if r - l + 1 < minLength:
                    bestLeft = l
                    minLength = r - l + 1
                elif r - l + 1 == minLength and s[l:l + minLength] < s[bestLeft:bestLeft + minLength]:
                    bestLeft = l
                if s[l] == '1':
                    ones -= 1
                l += 1
            else:
                l += 1

        return "" if bestLeft == -1 else s[bestLeft:bestLeft + minLength]
```

1828. 2905.py

Path: LeetCode-main/solutions/2905. Find Indices With Index and Value Difference II/2905.py

```
class Solution:
    def findIndices(
        self,
        nums: list[int],
        indexDifference: int,
        valueDifference: int,
    ) -> list[int]:
        # nums[minIndex] := the minimum number with enough index different from the current number
        minIndex = 0
        # nums[maxIndex] := the maximum number with enough index different from the current number
        maxIndex = 0

        for i in range(indexDifference, len(nums)):
            if nums[i - indexDifference] < nums[minIndex]:
                minIndex = i - indexDifference
            if nums[i - indexDifference] > nums[maxIndex]:
                maxIndex = i - indexDifference
            if nums[i] - nums[minIndex] >= valueDifference:
                return [i, minIndex]
            if nums[maxIndex] - nums[i] >= valueDifference:
                return [i, maxIndex]

        return [-1, -1]
```

1829. 2906.py

Path: LeetCode-main/solutions/2906. Construct Product Matrix/2906.py

```
class Solution:
    def constructProductMatrix(self, grid: list[list[int]]) -> list[list[int]]:
        MOD = 12345
        m = len(grid)
        n = len(grid[0])
        ans = [[0] * n for _ in range(m)]
        prefix = [1]
        suffix = 1

        for row in grid:
            for num in row:
                prefix.append(prefix[-1] * num % MOD)

        for i in reversed(range(m)):
            for j in reversed(range(n)):
                ans[i][j] = prefix[i * n + j] * suffix % MOD
                suffix = suffix * grid[i][j] % MOD

        return ans
```

1830. 2907.py

Path: LeetCode-main/solutions/2907. Maximum Profitable Triplets With Increasing Prices I/2907.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.vals = [0] * (n + 1)

    def maximize(self, i: int, val: int) -> None:
        while i < len(self.vals):
            self.vals[i] = max(self.vals[i], val)
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        res = 0
        while i > 0:
            res = max(res, self.vals[i])
            i -= FenwickTree.lowbit(i)
        return res

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    def maxProfit(self, prices: list[int], profits: list[int]) -> int:
        ans = -1
        maxPrice = max(prices)
        maxProfitTree1 = FenwickTree(maxPrice)
        maxProfitTree2 = FenwickTree(maxPrice)

        for price, profit in zip(prices, profits):
            # max(proftis[i])
            maxProfit1 = maxProfitTree1.get(price - 1)
            # max(proftis[i]) + max(profits[j])
            maxProfit2 = maxProfitTree2.get(price - 1)
            maxProfitTree1.maximize(price, profit)
            if maxProfit1 > 0:
                maxProfitTree2.maximize(price, profit + maxProfit1)
            if maxProfit2 > 0:
                ans = max(ans, profit + maxProfit2)

        return ans

```

1831. 2908.py

Path: LeetCode-main/solutions/2908. Minimum Sum of Mountain Triplets I/2908.py

```

class Solution:
    # Same as 2908. Minimum Sum of Mountain Triplets I
    def minimumSum(self, nums: list[int]) -> int:
        ans = math.inf
        minPrefix = list(itertools.accumulate(nums, min))
        minSuffix = list(itertools.accumulate(reversed(nums), min))[::-1]

        for i, num in enumerate(nums):
            if num > minPrefix[i] and num > minSuffix[i]:
                ans = min(ans, num + minPrefix[i] + minSuffix[i])

        return -1 if ans == math.inf else ans

```

1832. 2909.py

Path: LeetCode-main/solutions/2909. Minimum Sum of Mountain Triplets II/2909.py

```
class Solution:
    # Same as 2908. Minimum Sum of Mountain Triplets I
    def minimumSum(self, nums: list[int]) -> int:
        ans = math.inf
        minPrefix = list(itertools.accumulate(nums, min))
        minSuffix = list(itertools.accumulate(reversed(nums), min))[::-1]

        for i, num in enumerate(nums):
            if num > minPrefix[i] and num > minSuffix[i]:
                ans = min(ans, num + minPrefix[i] + minSuffix[i])

        return -1 if ans == math.inf else ans
```

1833. 291.py

Path: LeetCode-main/solutions/291. Word Pattern II/291.py

```
class Solution:
    def wordPatternMatch(self, pattern: str, s: str) -> bool:
        def isMatch(
            i: int, j: int, charToString: dict[str, str],
            seen: set[str]) -> bool:
            if i == len(pattern) and j == len(s):
                return True
            if i == len(pattern) or j == len(s):
                return False

            c = pattern[i]

            if c in charToString:
                t = charToString[c]
                # See if we can match t with s[j..n).
                if t not in s[j:]:
                    return False

            # If there's a match, continue to match the rest.
            return isMatch(i + 1, j + len(t), charToString, seen)

            for k in range(j, len(s)):
                t = s[j:k + 1]

                # This string is mapped by another character.
                if t in seen:
                    continue

                charToString[c] = t
                seen.add(t)

                if isMatch(i + 1, k + 1, charToString, seen):
                    return True

            # Backtrack.
            del charToString[c]
            seen.remove(t)

        return False

    return isMatch(0, 0, {}, set())
```

1834. 2910.py

Path: LeetCode-main/solutions/2910. Minimum Number of Groups to Create a Valid Assignment/2910.py

```
class Solution:
    def minGroupsForValidAssignment(self, nums: list[int]) -> int:
        count = collections.Counter(nums)
        minFreq = min(count.values())

        for groupSize in range(minFreq, 0, -1):
            numGroups = self.getNumGroups(count, groupSize)
            if numGroups > 0:
                return numGroups

        raise ValueError("Invalid argument")

    def getNumGroups(self, count: dict[int, int], groupSize: int) -> int:
        """Returns the number of groups if each group's size is `groupSize` or `groupSize + 1`."""
        numGroups = 0
        for freq in count.values():
            a = freq // (groupSize + 1)
            b = freq % (groupSize + 1)
            if b == 0:
                # Assign 1 number from `groupSize - b` out of `a` groups to this group,
                # so we'll have `a - (groupSize - b)` groups of size `groupSize + 1`
                # and `groupSize - b + 1` groups of size `groupSize`. In total, we have
                # `a + 1` groups.
                numGroups += a
            elif groupSize - b <= a:
                numGroups += a + 1
            else:
                return 0
        return numGroups
```

1835. 2911.py

Path: LeetCode-main/solutions/2911. Minimum Changes to Make K Semi-palindromes/2911.py

```

class Solution:
    def minimumChanges(self, s: str, k: int) -> int:
        n = len(s)
        # factors[i] := factors of i
        factors = self._getFactors(n)
        # cost[i][j] := changes to make s[i..j] a semi-palindrome
        cost = self._getCost(s, n, factors)
        # dp[i][j] := the minimum changes to split s[i:] into j valid parts
        dp = [[n] * (k + 1) for _ in range(n + 1)]

        dp[n][0] = 0

        for i in range(n - 1, -1, -1):
            for j in range(1, k + 1):
                for l in range(i + 1, n):
                    dp[i][j] = min(dp[i][j], dp[l + 1][j - 1] + cost[i][l])

        return dp[0][k]

    def _getFactors(self, n: int) -> list[list[int]]:
        factors = [[1] for _ in range(n + 1)]
        for d in range(2, n):
            for i in range(d * 2, n + 1, d):
                factors[i].append(d)
        return factors

    def _getCost(self, s: str, n: int, factors: list[list[int]]) -> list[list[int]]:
        cost = [[0] * n for _ in range(n)]
        for i, j in itertools.combinations(range(n), 2):
            length = j - i + 1
            minCost = length
            for d in factors[length]:
                minCost = min(minCost, self._getCostD(s, i, j, d))
            cost[i][j] = minCost
        return cost

    def _getCostD(self, s: str, i: int, j: int, d: int) -> int:
        """Returns the cost to make s[i..j] a semi-palindrome of `d`."""
        cost = 0
        for offset in range(d):
            l = i + offset
            r = j - d + 1 + offset
            while l < r:
                if s[l] != s[r]:
                    cost += 1
                l += d
                r -= d
        return cost

```

1836. 2912-2.py

Path: LeetCode-main/solutions/2912. Number of Ways to Reach Destination in the Grid/2912-2.py

```

class Solution:
    def numberOfWays(
        self,
        n: int,
        m: int,
        k: int,
        source: list[int],
        dest: list[int],
    ) -> int:
        MOD = 1_000_000_007

        # the number of ways of `source` to `dest` using steps so far
        ans = int(source == dest)

        # the number of ways of `source` to dest's row using steps so far
        row = int(source[0] == dest[0] and source[1] != dest[1])

        # the number of ways of `source` to dest's col using steps so far
        col = int(source[0] != dest[0] and source[1] == dest[1])

        # the number of ways of `source` to others using steps so far
        others = int(source[0] != dest[0] and source[1] != dest[1])

        for _ in range(k):
            nextAns = (row + col) % MOD
            nextRow = (ans * (m - 1) + # -self
                      row * (m - 2) + # -self, -center
                      others) % MOD
            nextCol = (ans * (n - 1) + # -self
                     col * (n - 2) + # -self, -center
                     others) % MOD
            nextOthers = (row * (n - 1) + # -self
                        col * (m - 1) + # -self
                        others * (m + n - 1 - 3)) % MOD # -self, -row, -col

            ans = nextAns
            row = nextRow
            col = nextCol
            others = nextOthers

        return ans

```

1837. 2912.py

Path: LeetCode-main/solutions/2912. Number of Ways to Reach Destination in the Grid/2912.py

```

class Solution:
    def numberOfWays(
        self,
        n: int,
        m: int,
        k: int,
        source: list[int],
        dest: list[int],
    ) -> int:
        MOD = 1_000_000_007

        # dp[i][0] := the the number of ways of `source` to `dest` using i steps
        # dp[i][1] := the the number of ways of `source` to dest's row using i steps
        # dp[i][2] := the the number of ways of `source` to dest's col using i steps
        # dp[i][3] := the the number of ways of `source` to others using i steps
        dp = [[0] * 4 for _ in range(k + 1)]
        if source == dest:
            dp[0][0] = 1
        elif source[0] == dest[0]:
            dp[0][1] = 1
        elif source[1] == dest[1]:
            dp[0][2] = 1
        else:
            dp[0][3] = 1

        for i in range(1, k + 1):
            dp[i][0] = (dp[i - 1][1] + dp[i - 1][2]) % MOD
            dp[i][1] = (dp[i - 1][0] * (m - 1) + # -self
                        dp[i - 1][1] * (m - 2) + # -self, -center
                        dp[i - 1][3]) % MOD
            dp[i][2] = (dp[i - 1][0] * (n - 1) + # -self
                        dp[i - 1][2] * (n - 2) + # -self, -center
                        dp[i - 1][3]) % MOD
            dp[i][3] = (dp[i - 1][1] * (n - 1) + # -self
                        dp[i - 1][2] * (m - 1) + # -self
                        dp[i - 1][3] * (m + n - 1 - 3)) % MOD # -self, -row, -col

        return dp[k][0]

```

1838. 2914.py

Path: LeetCode-main/solutions/2914. Minimum Number of Changes to Make Binary String Beautiful/2914.py

```

class Solution:
    def minChanges(self, s: str) -> int:
        return sum(a != b for a, b in zip(s[::2], s[1::2]))

```

1839. 2915-2.py

Path: LeetCode-main/solutions/2915. Length of the Longest Subsequence That Sums to Target/2915-2.py

```

class Solution:
    def lengthOfLongestSubsequence(self, nums: list[int], target: int) -> int:
        # dp[i] := the maximum length of any subsequence of numbers so far that
        # sum to j
        dp = [0] * (target + 1)

        for num in nums:
            for i in range(target, num - 1, -1):
                if i == num or dp[i - num] > 0:
                    dp[i] = max(dp[i], 1 + dp[i - num])

        return dp[target] if dp[target] > 0 else -1

```

1840. 2915.py

Path: LeetCode-main/solutions/2915. Length of the Longest Subsequence That Sums to Target/2915.py

```

class Solution:
    def lengthOfLongestSubsequence(self, nums: list[int], target: int) -> int:
        n = len(nums)
        # dp[i][j] := the maximum length of any subsequence of the first i numbers
        # that sum to j
        dp = [[-1] * (target + 1) for _ in range(n + 1)]

        for i in range(n + 1):
            dp[i][0] = 0

        for i in range(1, n + 1):
            num = nums[i - 1]
            for j in range(1, target + 1):
                # 1. Skip `num`.
                if j < num or dp[i - 1][j - num] == -1:
                    dp[i][j] = dp[i - 1][j]
                # 2. Skip `num` or pick `num`.
                else:
                    dp[i][j] = max(dp[i - 1][j], 1 + dp[i - 1][j - num])

        return dp[n][target]

```

1841. 2917.py

Path: LeetCode-main/solutions/2917. Find the K-or of an Array/2917.py

```

class Solution:
    def findKOr(self, nums: list[int], k: int) -> int:
        MAX_BIT = 30
        return sum(2**i
                    for i in range(MAX_BIT + 1)
                    if sum(num >> i & 1 for num in nums) >= k)

```

1842. 2918.py

Path: LeetCode-main/solutions/2918. Minimum Equal Sum of Two Arrays After Replacing Zeros/2918.py

```
class Solution:
    def minSum(self, nums1: list[int], nums2: list[int]) -> int:
        sum1 = sum(nums1)
        sum2 = sum(nums2)
        zero1 = nums1.count(0)
        zero2 = nums2.count(0)
        if zero1 == 0 and sum1 < sum2 + zero2:
            return -1
        if zero2 == 0 and sum2 < sum1 + zero1:
            return -1
        return max(sum1 + zero1, sum2 + zero2)
```

1843. 2919.py

Path: LeetCode-main/solutions/2919. Minimum Increment Operations to Make Array Beautiful/2919.py

```
class Solution:
    def minIncrementOperations(self, nums: list[int], k: int) -> int:
        # the minimum operations to increase nums[i - 3] and nums[0..i - 3]
        prev3 = 0
        # the minimum operations to increase nums[i - 2] and nums[0..i - 2]
        prev2 = 0
        # the minimum operations to increase nums[i - 1] and nums[0..i - 1]
        prev1 = 0

        for num in nums:
            dp = min(prev1, prev2, prev3) + max(0, k - num)
            prev3 = prev2
            prev2 = prev1
            prev1 = dp

        return min(prev1, prev2, prev3)
```

1844. 292.py

Path: LeetCode-main/solutions/292. Nim Game/292.py

```
class Solution:
    def canWinNim(self, n: int) -> bool:
        return n % 4 != 0
```

1845. 2920.py

Path: LeetCode-main/solutions/2920. Maximum Points After Collecting Coins From All Nodes/2920.py

```

class Solution:
    def maximumPoints(
        self,
        edges: list[list[int]],
        coins: list[int],
        k: int,
    ) -> int:
        MAX_COIN = 10000
        MAX_HALVED = int(MAX_COIN).bit_length()
        n = len(coins)
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        @functools.lru_cache(None)
        def dfs(u: int, prev: int, halved: int) -> int:
            # All the children will be 0, so no need to explore.
            if halved > MAX_HALVED:
                return 0

            val = coins[u] // (1 << halved)
            takeAll = val - k
            takeHalf = math.floor(val / 2)

            for v in graph[u]:
                if v == prev:
                    continue
                takeAll += dfs(v, u, halved)
                takeHalf += dfs(v, u, halved + 1)

            return max(takeAll, takeHalf)

        return dfs(0, -1, 0)

```

1846. 2921.py

Path: LeetCode-main/solutions/2921. Maximum Profitable Triplets With Increasing Prices II/2921.py


```

class FenwickTree:
    def __init__(self, n: int):
        self.vals = [0] * (n + 1)

    def maximize(self, i: int, val: int) -> None:
        while i < len(self.vals):
            self.vals[i] = max(self.vals[i], val)
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        res = 0
        while i > 0:
            res = max(res, self.vals[i])
            i -= FenwickTree.lowbit(i)
        return res

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    # Same as 2907. Maximum Profitable Triplets With Increasing Prices I
    def maxProfit(self, prices: list[int], profits: list[int]) -> int:
        ans = -1
        maxPrice = max(prices)
        maxProfitTree1 = FenwickTree(maxPrice)
        maxProfitTree2 = FenwickTree(maxPrice)

        for price, profit in zip(prices, profits):
            # max(proftis[i])
            maxProfit1 = maxProfitTree1.get(price - 1)
            # max(proftis[i]) + max(profits[j])
            maxProfit2 = maxProfitTree2.get(price - 1)
            maxProfitTree1.maximize(price, profit)
            if maxProfit1 > 0:
                maxProfitTree2.maximize(price, profit + maxProfit1)
            if maxProfit2 > 0:
                ans = max(ans, profit + maxProfit2)

        return ans

```

1847. 2923-2.py

Path: LeetCode-main/solutions/2923. Find Champion I/2923-2.py

```

class Solution:
    def findChampion(self, grid: list[list[int]]) -> int:
        return max(range(len(grid)), key=lambda x: sum(grid[x]))

```

1848. 2923.py

Path: LeetCode-main/solutions/2923. Find Champion I/2923.py

```
class Solution:
    def findChampion(self, grid: list[list[int]]) -> int:
        n = len(grid)
        inDegrees = [0] * n

        for i in range(n):
            for j in range(n):
                if i == j:
                    continue
                if grid[i][j] == 1:
                    inDegrees[j] += 1
                else:
                    inDegrees[i] += 1

        return (-1 if inDegrees.count(0) > 1
                else inDegrees.index(0))
```

1849. 2924.py

Path: LeetCode-main/solutions/2924. Find Champion II/2924.py

```
class Solution:
    def findChampion(self, n: int, edges: list[list[int]]) -> int:
        inDegrees = [0] * n

        for _, v in edges:
            inDegrees[v] += 1

        return (-1 if inDegrees.count(0) > 1
                else inDegrees.index(0))
```

1850. 2925.py

Path: LeetCode-main/solutions/2925. Maximum Score After Applying Operations on a Tree/2925.py

```
class Solution:
    def maximumScoreAfterOperations(
        self,
        edges: list[list[int]],
        values: list[int],
    ) -> int:
        tree = [[] for _ in values]

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def dfs(u: int, prev: int) -> None:
            if u > 0 and len(tree[u]) == 1:
                return values[u]
            childrenSum = sum(dfs(v, u)
                             for v in tree[u]
                             if v != prev)
            return min(childrenSum, values[u])

        return sum(values) - dfs(0, -1)
```

1851. 2926.py

Path: LeetCode-main/solutions/2926. Maximum Balanced Subsequence Sum/2926.py

```
class FenwickTree:
    def __init__(self, n: int):
        self.vals = [0] * (n + 1)

    def maximize(self, i: int, val: int) -> None:
        """Updates the maximum sum of subsequence ending in (i - 1) with `val`."""
        while i < len(self.vals):
            self.vals[i] = max(self.vals[i], val)
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        """Returns the maximum sum of subsequence ending in (i - 1)."""
        res = 0
        while i > 0:
            res = max(res, self.vals[i])
            i -= FenwickTree.lowbit(i)
        return res

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i


class Solution:
    def maxBalancedSubsequenceSum(self, nums: list[int]) -> int:
        # Let's define maxSum[i] := subsequence with the maximum sum ending in i
        # By observation:
        #   nums[i] - nums[j] >= i - j
        # => nums[i] - i >= nums[j] - j
        # So, if nums[i] - i >= nums[j] - j, where i > j,
        # maxSum[i] = max(maxSum[j], maxSum[j] + nums[i])
        ans = -math.inf
        tree = FenwickTree(len(nums))

        for _, i in sorted([(num - i, i) for i, num in enumerate(nums)]):
            subseqSum = tree.get(i) + nums[i]
            tree.maximize(i + 1, subseqSum)
            ans = max(ans, subseqSum)

        return ans
```

1852. 2927.py

Path: LeetCode-main/solutions/2927. Distribute Candies Among Children III/2927.py

```

class Solution:
    def distributeCandies(self, n: int, limit: int) -> int:
        def ways(n: int) -> int:
            """Returns the number of ways to distribute n candies to 3 children."""
            if n < 0:
                return 0
            # Stars and bars method:
            # e.g. '**|**|*' means to distribute 5 candies to 3 children, where
            # stars (*) := candies and bars (|) := dividers between children.
            return math.comb(n + 2, 2)

        limitPlusOne = limit + 1
        oneChildExceedsLimit = ways(n - limitPlusOne)
        twoChildrenExceedLimit = ways(n - 2 * limitPlusOne)
        threeChildrenExceedLimit = ways(n - 3 * limitPlusOne)
        # Principle of Inclusion-Exclusion (PIE)
        return (ways(n)
                - 3 * oneChildExceedsLimit
                + 3 * twoChildrenExceedLimit
                - threeChildrenExceedLimit)

```

1853. 2928.py

Path: LeetCode-main/solutions/2928. Distribute Candies Among Children I/2928.py

```

class Solution:
    def distributeCandies(self, n: int, limit: int) -> int:
        def ways(n: int) -> int:
            """Returns the number of ways to distribute n candies to 3 children."""
            if n < 0:
                return 0
            # Stars and bars method:
            # e.g. '**|**|*' means to distribute 5 candies to 3 children, where
            # stars (*) := candies and bars (|) := dividers between children.
            return math.comb(n + 2, 2)

        limitPlusOne = limit + 1
        oneChildExceedsLimit = ways(n - limitPlusOne)
        twoChildrenExceedLimit = ways(n - 2 * limitPlusOne)
        threeChildrenExceedLimit = ways(n - 3 * limitPlusOne)
        # Principle of Inclusion-Exclusion (PIE)
        return (ways(n)
                - 3 * oneChildExceedsLimit
                + 3 * twoChildrenExceedLimit
                - threeChildrenExceedLimit)

```

1854. 2929.py

Path: LeetCode-main/solutions/2929. Distribute Candies Among Children II/2929.py

```

class Solution:
    def distributeCandies(self, n: int, limit: int) -> int:
        def ways(n: int) -> int:
            """Returns the number of ways to distribute n candies to 3 children."""
            if n < 0:
                return 0
            # Stars and bars method:
            # e.g. '**|**|*' means to distribute 5 candies to 3 children, where
            # stars (*) := candies and bars (|) := dividers between children.
            return math.comb(n + 2, 2)

        limitPlusOne = limit + 1
        oneChildExceedsLimit = ways(n - limitPlusOne)
        twoChildrenExceedLimit = ways(n - 2 * limitPlusOne)
        threeChildrenExceedLimit = ways(n - 3 * limitPlusOne)
        # Principle of Inclusion-Exclusion (PIE)
        return (ways(n)
                - 3 * oneChildExceedsLimit
                + 3 * twoChildrenExceedLimit
                - threeChildrenExceedLimit)

```

1855. 293.py

Path: LeetCode-main/solutions/293. Flip Game/293.py

```

class Solution:
    def generatePossibleNextMoves(self, currentState: str) -> list[str]:
        return [currentState[:i] + '--' + currentState[i + 2:]
                for i, (a, b) in enumerate(zip(currentState, currentState[1:]))
                if a == '+' and b == '+']

```

1856. 2930.py

Path: LeetCode-main/solutions/2930. Number of Strings Which Can Be Rearranged to Contain Substring/2930.py

```

class Solution:
    def stringCount(self, n: int) -> int:
        # There're three invalid conditions:
        # a. count('l') == 0
        # b. count('e') < 2
        # c. count('t') == 0
        #
        # By Principle of Inclusion-Exclusion (PIE):
        # ans = allCount - a - b - c + ab + ac + bc - abc
        MOD = 1_000_000_007
        allCount = pow(26, n, MOD)
        a = pow(25, n, MOD)
        b = pow(25, n, MOD)
        c = pow(25, n, MOD) + n * pow(25, n - 1, MOD)
        ab = pow(24, n, MOD) + n * pow(24, n - 1, MOD)
        ac = pow(24, n, MOD)
        bc = pow(24, n, MOD) + n * pow(24, n - 1, MOD)
        abc = pow(23, n, MOD) + n * pow(23, n - 1, MOD)
        return (allCount - a - b - c + ab + ac + bc - abc) % MOD

```

1857. 2931.py

Path: LeetCode-main/solutions/2931. Maximum Spending After Buying Items/2931.py

```
class Solution:
    def maxSpending(self, values: list[list[int]]) -> int:
        items = sorted(item for shop in values for item in shop)
        return sum(item * d for d, item in enumerate(items, 1))
```

1858. 2932.py

Path: LeetCode-main/solutions/2932. Maximum Strong Pair XOR I/2932.py

```

class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2
        self.mn = math.inf
        self.mx = -math.inf

class BitTrie:
    def __init__(self, maxBit: int):
        self.maxBit = maxBit
        self.root = TrieNode()

    def insert(self, num: int) -> None:
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = num >> i & 1
            if not node.children[bit]:
                node.children[bit] = TrieNode()
            node = node.children[bit]
            node.mn = min(node.mn, num)
            node.mx = max(node.mx, num)

    def getMaxXor(self, x: int) -> int:
        """Returns max(x ^ y) where |x - y| <= min(x, y).

        If x <= y, |x - y| <= min(x, y) can be written as y - x <= x.
        So, y <= 2 * x.
        """
        maxXor = 0
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = x >> i & 1
            toggleBit = bit ^ 1
            # If `node.children[toggleBit].mx > x`, it means there's a number in the
            # node that satisfies the condition to ensure that x <= y among x and y.
            # If `node.children[toggleBit].mn <= 2 * x`, it means there's a number in
            # the node that satisfies the condition for a valid y.
            if (node.children[toggleBit] and
                node.children[toggleBit].mx > x and
                node.children[toggleBit].mn <= 2 * x):
                maxXor = maxXor | 1 << i
                node = node.children[toggleBit]
            elif node.children[bit]:
                node = node.children[bit]
            else: # There's nothing in the Bit Trie.
                return 0
        return maxXor

class Solution:
    # Similar to 421. Maximum XOR of Two Numbers in an Array
    def maximumStrongPairXor(self, nums: list[int]) -> int:
        maxNum = max(nums)
        maxBit = int(math.log2(maxNum))
        bitTrie = BitTrie(maxBit)

        for num in nums:
            bitTrie.insert(num)

        return max(bitTrie.getMaxXor(num) for num in nums)

```

Path: LeetCode-main/solutions/2933. High-Access Employees/2933.py

```
class Solution:
    def findHighAccessEmployees(self, access_times: list[list[str]]) -> list[str]:
        ans = set()

        access_times.sort()

        for i in range(len(access_times) - 2):
            name = access_times[i][0]
            if name in ans:
                continue
            if name != access_times[i + 2][0]:
                continue
            if int(access_times[i + 2][1]) - int(access_times[i][1]) < 100:
                ans.add(name)

        return list(ans)
```

1860. 2934.py

Path: LeetCode-main/solutions/2934. Minimum Operations to Maximize Last Elements in Arrays/2934.py

```
class Solution:
    def minOperations(self, nums1: list[int], nums2: list[int]) -> int:
        n = len(nums1)
        mn = min(nums1[-1], nums2[-1])
        mx = max(nums1[-1], nums2[-1])
        # the number of the minimum operations, where nums1[n - 1] is not swapped
        # with nums2[n - 1]
        dp1 = 0
        # the number of the minimum operations, where nums1[n - 1] is swapped with
        # nums2[n - 1]
        dp2 = 0

        for a, b in zip(nums1, nums2):
            if min(a, b) > mn:
                return -1
            if max(a, b) > mx:
                return -1
            if a > nums1[-1] or b > nums2[-1]:
                dp1 += 1
            if a > nums2[-1] or b > nums1[-1]:
                dp2 += 1

        return min(dp1, dp2)
```

1861. 2935.py

Path: LeetCode-main/solutions/2935. Maximum Strong Pair XOR II/2935.py


```

class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2
        self.mn = math.inf
        self.mx = -math.inf

class BitTrie:
    def __init__(self, maxBit: int):
        self.maxBit = maxBit
        self.root = TrieNode()

    def insert(self, num: int) -> None:
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = num >> i & 1
            if not node.children[bit]:
                node.children[bit] = TrieNode()
            node = node.children[bit]
            node.mn = min(node.mn, num)
            node.mx = max(node.mx, num)

    def getMaxXor(self, x: int) -> int:
        """Returns max(x ^ y) where |x - y| <= min(x, y).

        If x <= y, |x - y| <= min(x, y) can be written as y - x <= x.
        So, y <= 2 * x.
        """
        maxXor = 0
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = x >> i & 1
            toggleBit = bit ^ 1
            # If `node.children[toggleBit].mx > x`, it means there's a number in the
            # node that satisfies the condition to ensure that x <= y among x and y.
            # If `node.children[toggleBit].mn <= 2 * x`, it means there's a number in
            # the node that satisfies the condition for a valid y.
            if (node.children[toggleBit] and
                node.children[toggleBit].mx > x and
                node.children[toggleBit].mn <= 2 * x):
                maxXor = maxXor | 1 << i
                node = node.children[toggleBit]
            elif node.children[bit]:
                node = node.children[bit]
            else: # There's nothing in the Bit Trie.
                return 0
        return maxXor

class Solution:
    # Same as 2932. Maximum Strong Pair XOR I
    def maximumStrongPairXor(self, nums: list[int]) -> int:
        maxNum = max(nums)
        maxBit = int(math.log2(maxNum))
        bitTrie = BitTrie(maxBit)

        for num in nums:
            bitTrie.insert(num)

        return max(bitTrie.getMaxXor(num) for num in nums)

```

Path: LeetCode-main/solutions/2936. Number of Equal Numbers Blocks/2936.py

```
# Definition for BigArray.
# class BigArray:
#     def at(self, index: long) -> int:
#         pass
#     def size(self) -> long:
#         pass

class Solution(object):
    def countBlocks(self, nums: Optional['BigArray']) -> int:
        def countBlocks(l: int, r: int, leftValue: int, rightValue: int) -> int:
            """Returns the number of maximal blocks in nums[l..r]."""
            if leftValue == rightValue:
                return 1
            if l + 1 == r:
                return 2
            m = (l + r) // 2
            midValue = nums.at(m)
            return (countBlocks(l, m, leftValue, midValue) +
                    countBlocks(m, r, midValue, rightValue) - 1)
        # Subtract nums[m], which will be counted twice.
        return countBlocks(0, nums.size() - 1,
                            nums.at(0), nums.at(nums.size() - 1))
```

1863. 2937.py

Path: LeetCode-main/solutions/2937. Make Three Strings Equal/2937.py

```
class Solution:
    def findMinimumOperations(self, s1: str, s2: str, s3: str) -> int:
        minLength = min(map(len, [s1, s2, s3]))
        i = 0
        while i < minLength and s1[i] == s2[i] and s2[i] == s3[i]:
            i += 1
        return -1 if i == 0 else len(s1) + len(s2) + len(s3) - i * 3
```

1864. 2938.py

Path: LeetCode-main/solutions/2938. Separate Black and White Balls/2938.py

```
class Solution:
    def minimumSteps(self, s: str) -> int:
        ans = 0
        ones = 0

        for c in s:
            if c == '1':
                ones += 1
            else: # Move 1s to the front of the current '0'.
                ans += ones

        return ans
```

1865. 2939.py

Path: LeetCode-main/solutions/2939. Maximum XOR Product/2939.py


```

if a == b or heights[a] < heights[b]:
    # 1. Alice and Bob are already in the same index (a == b) or
    # 2. Alice can jump from a -> b (heights[a] < heights[b]).
    ans[queryIndex] = b
else:
    # Now, a < b and heights[a] >= heights[b].
    # Gradually add heights with an index > b to the monotonic stack.
    while heightsIndex > b:
        # heights[heightsIndex] is a better candidate, given that
        # heightsIndex is smaller than the indices in the stack and
        # heights[heightsIndex] is larger or equal to the heights mapped in
        # the stack.
        while stack and heights[stack[-1]] <= heights[heightsIndex]:
            stack.pop()
        stack.append(heightsIndex)
        heightsIndex -= 1
    # Binary search to find the smallest index j such that j > b and
    # heights[j] > heights[a], thereby ensuring heights[j] > heights[b].
    j = self._lastGreater(stack, a, heights)
    if j != -1:
        ans[queryIndex] = stack[j]

return ans

def _lastGreater(self, A: list[int], target: int, heights: list[int]):
    """
    Returns the last index i in A s.t. heights[A.get(i)] is > heights[target].
    """
    l = -1
    r = len(A) - 1
    while l < r:
        m = (l + r + 1) // 2
        if heights[A[m]] > heights[target]:
            l = m
        else:
            r = m - 1
    return l

```

1868. 2941.py

Path: LeetCode-main/solutions/2941. Maximum GCD-Sum of a Subarray/2941.py

```

class Solution:
    def maxGcdSum(self, nums: list[int], k: int) -> int:
        ans = 0
        # [(startIndex, gcd of subarray starting at startIndex)]
        startIndexAndGcds = []
        prefix = list(itertools.accumulate(nums, initial=0))

        for i, num in enumerate(nums):
            nextStartIndexAndGcds = []
            for startIndex, gcd in startIndexAndGcds:
                nextGcd = math.gcd(gcd, nums[i])
                if (not nextStartIndexAndGcds or
                    nextStartIndexAndGcds[-1][1] != nextGcd): # Skip duplicates.
                    nextStartIndexAndGcds.append((startIndex, nextGcd))
            startIndexAndGcds = nextStartIndexAndGcds
            startIndexAndGcds.append((i, nums[i]))
            for startIndex, gcd in startIndexAndGcds:
                if i - startIndex + 1 >= k:
                    ans = max(ans, (prefix[i + 1] - prefix[startIndex]) * gcd)

        return ans

```

1869. 2944-2.py

Path: LeetCode-main/solutions/2944. Minimum Number of Coins for Fruits/2944-2.py

```

class Solution:
    def minimumCoins(self, prices: list[int]) -> int:
        n = len(prices)
        # Stores (dp[i], i), where dp[i] is the minimum number of coins to acquire
        # fruits[i:] (0-indexed).
        minHeap = [(0, n)]
        ans = 0

        for i in range(n - 1, -1, -1):
            while minHeap and minHeap[0][1] > (i + 1) * 2:
                heapq.heappop(minHeap)
            ans = prices[i] + minHeap[0][0]
            heapq.heappush(minHeap, (ans, i))

        return ans

```

1870. 2944-3.py

Path: LeetCode-main/solutions/2944. Minimum Number of Coins for Fruits/2944-3.py

```

class Solution:
    def minimumCoins(self, prices: list[int]) -> int:
        n = len(prices)
        ans = math.inf
        # Stores (dp[i], i), where dp[i] := the minimum number of coins to acquire
        # fruits[i:] (0-indexed) in ascending order.
        minQ = collections.deque([(0, n)])

        for i in range(n - 1, -1, -1):
            while minQ and minQ[0][1] > (i + 1) * 2:
                minQ.popleft()
            ans = prices[i] + minQ[0][0]
            while minQ and minQ[-1][0] >= ans:
                minQ.pop()
            minQ.append((ans, i))

        return ans

```

1871. 2944.py

Path: LeetCode-main/solutions/2944. Minimum Number of Coins for Fruits/2944.py

```

class Solution:
    def minimumCoins(self, prices: list[int]) -> int:
        n = len(prices)
        # Convert to 0-indexed for easy computation.
        # dp[i] := the minimum number of coins to acquire fruits[i:]
        dp = [math.inf] * n + [0]

        for i in range(n - 1, -1, -1):
            # Convert back to 1-indexed.
            for j in range(i + 1, min((i + 1) * 2 + 1, n + 1)):
                dp[i] = min(dp[i], prices[i] + dp[j])

        return dp[0]

```

1872. 2945-2.py

Path: LeetCode-main/solutions/2945. Find Maximum Non-decreasing Array Length/2945-2.py

```

class Solution:
    def findMaximumLength(self, nums: list[int]) -> int:
        n = len(nums)
        # prefix[i] := the sum of the first i nums
        prefix = list(itertools.accumulate(nums, initial=0))
        # dp[i] := the maximum number of elements in the increasing
        # sequence after processing the first i nums
        dp = [0] * (n + 1)
        # bestLeft[i] := the index l s.t. merging nums[l..i) is the
        # optimal strategy among processing the first i nums
        bestLeft = [0] * (n + 2)

        for i in range(1, n + 1):
            bestLeft[i] = max(bestLeft[i], bestLeft[i - 1])
            # When merging nums[l, i), consider the next segment as [i, r).
            # Find the minimum `r` where sum(nums[l, i)) <= sum(nums[i, r)).
            # Equivalently, prefix[i] - prefix[l] <= prefix[r] - prefix[i].
            # => prefix[r] >= prefix[i] * 2 - prefix[l]
            # Therefore, we can binary search `prefix` to find the minimum `r`.
            l = bestLeft[i]
            r = bisect.bisect_left(prefix, 2 * prefix[i] - prefix[l])
            dp[i] = dp[l] + 1
            bestLeft[r] = i

        return dp[n]

```

1873. 2945.py

Path: LeetCode-main/solutions/2945. Find Maximum Non-decreasing Array Length/2945.py

```

class Solution:
    def findMaximumLength(self, nums: list[int]) -> int:
        n = len(nums)
        INF = 10_000_000_000
        # prefix[i] := the sum of the first i nums
        prefix = list(itertools.accumulate(nums, initial=0))
        # dp[i] := the maximum number of elements in the increasing
        # sequence after processing the first i nums
        dp = [0] * (n + 1)
        # last[i] := the last sum after processing the first i nums
        last = [0] + [INF] * n

        for i in range(n):
            j = self._findIndex(i, prefix, last)
            dp[i + 1] = max(dp[i], dp[j] + 1)
            last[i + 1] = prefix[i + 1] - prefix[j]

        return dp[n]

    def _findIndex(self, i: int, prefix: list[int], last: list[int]) -> int:
        """Returns the index in [0..i].

        Returns the maximum index j in [0..i] s.t.
        prefix[i + 1] - prefix[j] >= last[j].
        """
        for j in range(i, -1, -1):
            if prefix[i + 1] - prefix[j] >= last[j]:
                return j

```

1874. 2946.py

Path: LeetCode-main/solutions/2946. Matrix Similarity After Cyclic Shifts/2946.py

```
class Solution:
    def areSimilar(self, mat: list[list[int]], k: int) -> bool:
        n = len(mat[0])
        for row in mat:
            for j in range(n):
                if row[j] != row[(j + k) % n]:
                    return False
        return True
```

1875. 2947.py

Path: LeetCode-main/solutions/2947. Count Beautiful Substrings I/2947.py

```
class Solution:
    def beautifulSubstrings(self, s: str, k: int) -> int:
        VOWELS = 'aeiou'
        root = self._getRoot(k)
        ans = 0
        vowels = 0
        vowelsMinusConsonants = 0
        # {(vowels, vowelsMinusConsonants): count}
        prefixCount = collections.Counter({(0, 0): 1})

        for c in s:
            if c in VOWELS:
                vowelsMinusConsonants += 1
                vowels = (vowels + 1) % root
            else:
                vowelsMinusConsonants -= 1
            ans += prefixCount[(vowels, vowelsMinusConsonants)]
            prefixCount[(vowels, vowelsMinusConsonants)] += 1

        return ans

    def _getRoot(self, k: int) -> int:
        for i in range(1, k + 1):
            if i * i % k == 0:
                return i
```

1876. 2948.py

Path: LeetCode-main/solutions/2948. Make Lexicographically Smallest Array by Swapping Elements/2948.py


```

class Solution:
    def lexicographicallySmallestArray(
        self,
        nums: list[int],
        limit: int,
    ) -> list[int]:
        ans = [0] * len(nums)
        numAndIndexes = sorted([(num, i) for i, num in enumerate(nums)])
        # [[(num, index)]], where the difference between in each pair in each
        # `[(num, index)]` group <= `limit`
        numAndIndexesGroups: list[list[tuple[int, int]]] = []

        for numAndIndex in numAndIndexes:
            if (not numAndIndexesGroups or
                numAndIndex[0] - numAndIndexesGroups[-1][-1][0] > limit):
                # Start a new group.
                numAndIndexesGroups.append([numAndIndex])
            else:
                # Append to the existing group.
                numAndIndexesGroups[-1].append(numAndIndex)

        for numAndIndexesGroup in numAndIndexesGroups:
            sortedNums = [num for num, _ in numAndIndexesGroup]
            sortedIndices = sorted([index for _, index in numAndIndexesGroup])
            for num, index in zip(sortedNums, sortedIndices):
                ans[index] = num

        return ans

```

1877. 2949.py

Path: LeetCode-main/solutions/2949. Count Beautiful Substrings II/2949.py

```

class Solution:
    # Same as 2947. Count Beautiful Substrings I
    def beautifulSubstrings(self, s: str, k: int) -> int:
        VOWELS = 'aeiou'
        root = self._getRoot(k)
        ans = 0
        vowels = 0
        vowelsMinusConsonants = 0
        # {(vowels, vowelsMinusConsonants): count}
        prefixCount = collections.Counter({(0, 0): 1})

        for c in s:
            if c in VOWELS:
                vowelsMinusConsonants += 1
                vowels = (vowels + 1) % root
            else:
                vowelsMinusConsonants -= 1
            ans += prefixCount[(vowels, vowelsMinusConsonants)]
            prefixCount[(vowels, vowelsMinusConsonants)] += 1

        return ans

    def _getRoot(self, k: int) -> int:
        for i in range(1, k + 1):
            if i * i % k == 0:
                return i

```

1878. 295.py

Path: LeetCode-main/solutions/295. Find Median from Data Stream/295.py

```
class MedianFinder:
    def __init__(self):
        self.maxHeap = []
        self.minHeap = []

    def addNum(self, num: int) -> None:
        if not self.maxHeap or num <= -self.maxHeap[0]:
            heapq.heappush(self.maxHeap, -num)
        else:
            heapq.heappush(self.minHeap, num)

        # Balance the two heaps s.t.
        # |maxHeap| >= |minHeap| and |maxHeap| - |minHeap| <= 1.
        if len(self.maxHeap) < len(self.minHeap):
            heapq.heappush(self.maxHeap, -heapq.heappop(self.minHeap))
        elif len(self.maxHeap) - len(self.minHeap) > 1:
            heapq.heappush(self.minHeap, -heapq.heappop(self.maxHeap))

    def findMedian(self) -> float:
        if len(self.maxHeap) == len(self.minHeap):
            return (-self.maxHeap[0] + self.minHeap[0]) / 2.0
        return -self.maxHeap[0]
```

1879. 2950.py

Path: LeetCode-main/solutions/2950. Number of Divisible Substrings/2950.py

```
class Solution:
    def countDivisibleSubstrings(self, word: str) -> int:
        # Let f(c) = d, where d = 1, 2, ..., 9.
        # Rephrase the question to return the number of substrings that satisfy
        # f(c1) + f(c2) + ... + f(ck) // k = avg
        # => f(c1) + f(c2) + ... + f(ck) - k * avg, where avg in [1, 9].
        ans = 0

        def f(c: str) -> int:
            return 9 - (ord('z') - ord(c)) // 3

        for avg in range(1, 10):
            prefix = 0
            prefixCount = collections.Counter({0: 1})
            for c in word:
                prefix += f(c) - avg
                ans += prefixCount[prefix]
                prefixCount[prefix] += 1

        return ans
```

1880. 2951.py

Path: LeetCode-main/solutions/2951. Find the Peaks/2951.py

```
class Solution:
    def findPeaks(self, mountain: list[int]) -> list[int]:
        return [i for i in range(1, len(mountain) - 1)
                if mountain[i - 1] < mountain[i] > mountain[i + 1]]
```

1881. 2952.py

Path: LeetCode-main/solutions/2952. Minimum Number of Coins to be Added/2952.py

```
class Solution:
    # Same as 330. Patching Array
    def minimumAddedCoins(self, coins: list[int], target: int) -> int:
        ans = 0
        i = 0 # coins' index
        miss = 1 # the minimum sum in [1, n] we might miss

        coins.sort()

        while miss <= target:
            if i < len(coins) and coins[i] <= miss:
                miss += coins[i]
                i += 1
            else:
                # Greedily add `miss` itself to increase the range from
                # [1, miss) to [1, 2 * miss).
                miss += miss
                ans += 1

        return ans
```

1882. 2953.py

Path: LeetCode-main/solutions/2953. Count Complete Substrings/2953.py

```
class Solution:
    def countCompleteSubstrings(self, word: str, k: int) -> int:
        uniqueLetters = len(set(word))
        return sum(self._countCompleteStrings(word, k, windowSize)
                    for windowSize in range(k, k * uniqueLetters + 1, k))

    def _countCompleteStrings(self, word: str, k: int, windowSize: int) -> int:
        """
        Returns the number of complete substrings of `windowSize` of `word`.
        """
        res = 0
        countLetters = 0 # the number of letters in the running substring
        count = collections.Counter()

        for i, c in enumerate(word):
            count[c] += 1
            countLetters += 1
            if i > 0 and abs(ord(c) - ord(word[i - 1])) > 2:
                count = collections.Counter()
                # Start a new substring starting at word[i].
                count[c] += 1
                countLetters = 1
            if countLetters == windowSize + 1:
                count[word[i - windowSize]] -= 1
                countLetters -= 1
            if countLetters == windowSize:
                res += all(freq == 0 or freq == k for freq in count.values())

        return res
```

1883. 2954.py

Path: LeetCode-main/solutions/2954. Count the Number of Infection Sequences/2954.py

```

class Solution:
    def numberOfSequence(self, n: int, sick: list[int]) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def fact(i: int) -> int:
            return 1 if i <= 1 else i * fact(i - 1) % MOD

        @functools.lru_cache(None)
        def inv(i: int) -> int:
            return pow(i, MOD - 2, MOD)

        ans = fact(n - len(sick)) # the number of infected children
        prevSick = -1

        for i, s in enumerate(sick):
            # The segment [prevSick + 1, sick - 1] are the current non-infected
            # children.
            nonInfected = sick[i] - prevSick - 1
            prevSick = sick[i]
            if nonInfected == 0:
                continue
            ans *= inv(fact(nonInfected))
            ans %= MOD
            if i > 0:
                # There're two choices per second since the children at the two
                # endpoints can both be the infect candidates. So, there are
                # 2^[nonInfected - 1] ways to infect all children in the current
                # segment.
                ans *= pow(2, nonInfected - 1, MOD)

        nonInfected = n - sick[-1] - 1
        return ans * inv(fact(nonInfected)) % MOD

```

1884. 2955.py

Path: LeetCode-main/solutions/2955. Number of Same-End Substrings/2955.py

```

class Solution:
    def sameEndSubstringCount(
        self,
        s: str,
        queries: list[list[int]],
    ) -> list[int]:
        count = collections.Counter()
        # counts[i] := the count of s[0..i]
        counts = [count.copy()]

        for c in s:
            count[c] += 1
            counts.append(count.copy())

        ans = []

        for l, r in queries:
            sameEndCount = 0
            for c in string.ascii_lowercase:
                # the count of s[0..r] - the count of s[0..l - 1]
                # = the count of s[l..r]
                freq = counts[r + 1][c] - counts[l][c]
                # C(freq, 2) + freq
                # = freq * (freq - 1) / 2 + freq
                # = freq * (freq + 1) / 2
                sameEndCount += freq * (freq + 1) // 2
            ans.append(sameEndCount)

        return ans

```

1885. 2956.py

Path: LeetCode-main/solutions/2956. Find Common Elements Between Two Arrays/2956.py

```

class Solution:
    def findIntersectionValues(
        self,
        nums1: list[int],
        nums2: list[int],
    ) -> list[int]:
        nums1Set = set(nums1)
        nums2Set = set(nums2)
        return [sum(num in nums2Set for num in nums1),
                sum(num in nums1Set for num in nums2)]

```

1886. 2957.py

Path: LeetCode-main/solutions/2957. Remove Adjacent Almost-Equal Characters/2957.py

```
class Solution:
    def removeAlmostEqualCharacters(self, word: str) -> int:
        ans = 0
        i = 1
        while i < len(word):
            if abs(ord(word[i]) - ord(word[i - 1])) <= 1:
                ans += 1
                i += 2
            else:
                i += 1
        return ans
```

1887. 2958.py

Path: LeetCode-main/solutions/2958. Length of Longest Subarray With at Most K Frequency/2958.py

```
class Solution:
    def maxSubarrayLength(self, nums: list[int], k: int) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, num in enumerate(nums):
            count[num] += 1
            while count[num] == k + 1:
                count[nums[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

1888. 2959.py

Path: LeetCode-main/solutions/2959. Number of Possible Sets of Closing Branches/2959.py

```

class Solution:
    def numberOfSets(
        self,
        n: int,
        maxDistance: int,
        roads: list[list[int]],
    ) -> int:
        return sum(self._floydWarshall(n, maxDistance, roads, mask) <= maxDistance
                    for mask in range(1 << n))

    def _floydWarshall(
        self,
        n: int,
        maxDistanceThreshold: int,
        roads: list[list[int]],
        mask: int,
    ) -> list[list[int]]:
        """
        Returns the maximum distance between any two branches, where the mask
        represents the selected branches.
        """
        maxDistance = 0
        dist = [[maxDistanceThreshold + 1] * n for _ in range(n)]

        for i in range(n):
            if mask >> i & 1:
                dist[i][i] = 0

        for u, v, w in roads:
            if mask >> u & 1 and mask >> v & 1:
                dist[u][v] = min(dist[u][v], w)
                dist[v][u] = min(dist[v][u], w)

        for k in range(n):
            if mask >> k & 1:
                for i in range(n):
                    if mask >> i & 1:
                        for j in range(n):
                            if mask >> j & 1:
                                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

        for i in range(n):
            if mask >> i & 1:
                for j in range(i + 1, n):
                    if mask >> j & 1:
                        maxDistance = max(maxDistance, dist[i][j])

        return maxDistance

```

1889. 296.py

Path: `LeetCode-main/solutions/296. Best Meeting Point/296.py`


```

class Solution:
    def minTotalDistance(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        # i indices s.t. grid[i][j] == 1
        I = [i for i in range(m) for j in range(n) if grid[i][j]]
        # j indices s.t. grid[i][j] == 1
        J = [j for j in range(n) for i in range(m) if grid[i][j]]

        def minTotalDistance(grid: list[int]) -> int:
            summ = 0
            i = 0
            j = len(grid) - 1
            while i < j:
                summ += grid[j] - grid[i]
                i += 1
                j -= 1
            return summ

        # sum(i - median(I)) + sum(j - median(J))
        return minTotalDistance(I) + minTotalDistance(J)

```

1890. 2960.py

Path: LeetCode-main/solutions/2960. Count Tested Devices After Test Operations/2960.py

```

class Solution:
    def countTestedDevices(self, batteryPercentages: list[int]) -> int:
        ans = 0

        for batteryPercentage in batteryPercentages:
            if batteryPercentage - ans > 0:
                ans += 1

        return ans

```

1891. 2961.py

Path: LeetCode-main/solutions/2961. Double Modular Exponentiation/2961.py

```

class Solution:
    def getGoodIndices(
        self,
        variables: list[list[int]],
        target: int,
    ) -> list[int]:
        return [i for i, (a, b, c, m) in enumerate(variables)
                if pow(pow(a, b, 10), c, m) == target]

```

1892. 2962.py

Path: LeetCode-main/solutions/2962. Count Subarrays Where Max Element Appears at Least K Times/2962.py

```

class Solution:
    def countSubarrays(self, nums: list[int], k: int) -> int:
        maxNum = max(nums)
        ans = 0
        count = 0

        l = 0
        for r, num in enumerate(nums):
            if num == maxNum:
                count += 1

            # Keep the window to include k - 1 times of the maxNum number.
            while count == k:
                if nums[l] == maxNum:
                    count -= 1
                l += 1

            # If l > 0, nums[l:r+1] has k - 1 times of the maxNum number. For any
            # subarray nums[i:r+1], where i < l, it will have at least k times of the
            # maxNum number, since nums[l - 1] equals the maxNum number.
            ans += l

        return ans

```

1893. 2963.py

Path: LeetCode-main/solutions/2963. Count the Number of Good Partitions/2963.py

```

class Solution:
    def numberOfGoodPartitions(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        ans = 1
        # lastSeen[num] := the index of the last time `num` appeared
        lastSeen = {}

        for i, num in enumerate(nums):
            lastSeen[num] = i

        # Track the maximum right index of each running partition by ensuring that
        # the first and last occurrences of a number fall within the same partition.
        maxRight = 0
        for i, num in enumerate(nums):
            if i > maxRight:
                # Start a new partition that starts from nums[i].
                # Each partition doubles the total number of good partitions.
                ans = ans * 2 % MOD
                maxRight = max(maxRight, lastSeen[num])

        return ans

```

1894. 2964.py

Path: LeetCode-main/solutions/2964. Number of Divisible Triplet Sums/2964.py

```
class Solution:
    # Similar to 1995. Count Special Quadruplets
    def divisibleTripletCount(self, nums: list[int], d: int) -> int:
        ans = 0
        count = collections.Counter()

        for j in range(len(nums) - 1, 0, -1): # `j` also represents k.
            for i in range(j - 1, -1, -1):
                ans += count[-(nums[i] + nums[j]) % d]
                count[nums[j] % d] += 1 # j := k

        return ans
```

1895. 2965.py

Path: LeetCode-main/solutions/2965. Find Missing and Repeated Values/2965.py

```
class Solution:
    def findMissingAndRepeatedValues(self, grid: list[list[int]]) -> list[int]:
        count = [1] + [0] * len(grid)**2 # padding for 1-indexed

        for row in grid:
            for num in row:
                count[num] += 1

        return [count.index(2), count.index(0)]
```

1896. 2966.py

Path: LeetCode-main/solutions/2966. Divide Array Into Arrays With Max Difference/2966.py

```
class Solution:
    def divideArray(self, nums: list[int], k: int) -> list[list[int]]:
        ans = []

        nums.sort()

        for i in range(2, len(nums), 3):
            if nums[i] - nums[i - 2] > k:
                return []
            ans.append([nums[i - 2], nums[i - 1], nums[i]])

        return ans
```

1897. 2967.py

Path: LeetCode-main/solutions/2967. Minimum Cost to Make Array Equalindromic/2967.py

```

class Solution:
    def minimumCost(self, nums: list[int]) -> int:
        nums.sort()
        median = nums[len(nums) // 2]
        nextPalindrome = self._getPalindrome(median, delta=1)
        prevPalindrome = self._getPalindrome(median, delta=-1)
        return min(self._cost(nums, nextPalindrome),
                    self._cost(nums, prevPalindrome))

    def _cost(self, nums: list[int], palindrome: int) -> int:
        """Returns the cost to change all the numbers to `palindrome`."""
        return sum(abs(palindrome - num) for num in nums)

    def _getPalindrome(self, num: int, delta: int) -> int:
        """Returns the palindrome `p`, where  $p = num + a * delta$  and  $a > 0$ ."""
        while not self._isPalindrome(num):
            num += delta
        return num

    def _isPalindrome(self, num: int) -> int:
        original = str(num)
        return original == original[::-1]

```

1898. 2968.py

Path: LeetCode-main/solutions/2968. Apply Operations to Maximize Frequency Score/2968.py

```

class Solution:
    def maxFrequencyScore(self, nums: list[int], k: int) -> int:
        nums.sort()
        ans = 0
        cost = 0

        l = 0
        for r, num in enumerate(nums):
            cost += num - nums[(l + r) // 2]
            while cost > k:
                cost -= nums[(l + r + 1) // 2] - nums[l]
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

1899. 2969-2.py

Path: LeetCode-main/solutions/2969. Minimum Number of Coins for Fruits II/2969-2.py

```

class Solution:
    # Same as 2944. Minimum Number of Coins for Fruits
    def minimumCoins(self, prices: list[int]) -> int:
        n = len(prices)
        # Stores (dp[i], i), where dp[i] is the minimum number of coins to acquire
        # fruits[i:] (0-indexed).
        minHeap = [(0, n)]
        ans = 0

        for i in range(n - 1, -1, -1):
            while minHeap and minHeap[0][1] > (i + 1) * 2:
                heapq.heappop(minHeap)
            ans = prices[i] + minHeap[0][0]
            heapq.heappush(minHeap, (ans, i))

        return ans

```

1900. 2969-3.py

Path: LeetCode-main/solutions/2969. Minimum Number of Coins for Fruits II/2969-3.py

```

class Solution:
    # Same as 2944. Minimum Number of Coins for Fruits
    def minimumCoins(self, prices: list[int]) -> int:
        n = len(prices)
        ans = math.inf
        # Stores (dp[i], i), where dp[i] := the minimum number of coins to acquire
        # fruits[i:] (0-indexed) in ascending order.
        minQ = collections.deque([(0, n)])

        for i in range(n - 1, -1, -1):
            while minQ and minQ[0][1] > (i + 1) * 2:
                minQ.popleft()
            ans = prices[i] + minQ[0][0]
            while minQ and minQ[-1][0] >= ans:
                minQ.pop()
            minQ.append((ans, i))

        return ans

```

1901. 2969.py

Path: LeetCode-main/solutions/2969. Minimum Number of Coins for Fruits II/2969.py

```

class Solution:
    # Same as 2944. Minimum Number of Coins for Fruits
    def minimumCoins(self, prices: list[int]) -> int:
        n = len(prices)
        # Convert to 0-indexed for easy computation.
        # dp[i] := the minimum number of coins to acquire fruits[i:]
        dp = [math.inf] * n + [0]

        for i in range(n - 1, -1, -1):
            # Convert back to 1-indexed.
            for j in range(i + 1, min((i + 1) * 2 + 1, n + 1)):
                dp[i] = min(dp[i], prices[i] + dp[j])

        return dp[0]

```

1902. 297-2.py

Path: LeetCode-main/solutions/297. Serialize and Deserialize Binary Tree/297-2.py

```
class Codec:
    def serialize(self, root: 'TreeNode') -> str:
        """Encodes a tree to a single string."""
        s = []

        def preorder(root: 'TreeNode') -> None:
            if not root:
                s.append('n')
                return

            s.append(str(root.val))
            preorder(root.left)
            preorder(root.right)

        preorder(root)
        return ' '.join(s)

    def deserialize(self, data: str) -> 'TreeNode':
        """Decodes your encoded data to tree."""
        q = collections.deque(data.split())

        def preorder() -> 'TreeNode':
            s = q.popleft()
            if s == 'n':
                return None

            root = TreeNode(s)
            root.left = preorder()
            root.right = preorder()
            return root

        return preorder()
```

1903. 297.py

Path: LeetCode-main/solutions/297. Serialize and Deserialize Binary Tree/297.py

```

class Codec:
    def serialize(self, root: 'TreeNode') -> str:
        """Encodes a tree to a single string."""
        if not root:
            return ''

        s = ''
        q = collections.deque([root])

        while q:
            node = q.popleft()
            if node:
                s += str(node.val) + ' '
                q.append(node.left)
                q.append(node.right)
            else:
                s += 'n '

        return s

    def deserialize(self, data: str) -> 'TreeNode':
        """Decodes your encoded data to tree."""
        if not data:
            return None

        vals = data.split()
        root = TreeNode(vals[0])
        q = collections.deque([root])

        for i in range(1, len(vals), 2):
            node = q.popleft()
            if vals[i] != 'n':
                node.left = TreeNode(vals[i])
                q.append(node.left)
            if vals[i + 1] != 'n':
                node.right = TreeNode(vals[i + 1])
                q.append(node.right)

        return root

```

1904. 2970-2.py

Path: LeetCode-main/solutions/2970. Count the Number of Incremovable Subarrays I/2970-2.py

```

class Solution:
    def incremovableSubarrayCount(self, nums: list[int]) -> int:
        n = len(nums)
        startIndex = self._getStartIndexofSuffix(nums)
        # If the complete array is strictly increasing, the total number of ways we
        # can remove elements equals the total number of possible subarrays.
        if startIndex == 0:
            return n * (n + 1) // 2

        # The valid removals starting from nums[0] include nums[0..startIndex - 1],
        # nums[0..startIndex], ..., nums[0..n].
        ans = n - startIndex + 1

        # Enumerate each prefix subarray that is strictly increasing.
        j = startIndex
        for i in range(startIndex):
            if i > 0 and nums[i] <= nums[i - 1]:
                break
            # Since nums[0..i] is strictly increasing, move j to the place such that
            # nums[j] > nums[i]. The valid removals will then be nums[i + 1..j - 1],
            # nums[i + 1..j], ..., nums[i + 1..n].
            while j < n and nums[i] >= nums[j]:
                j += 1
            ans += n - j + 1

        return ans

    def _getStartIndexofSuffix(self, nums: list[int]) -> int:
        for i in range(len(nums) - 2, -1, -1):
            if nums[i] >= nums[i + 1]:
                return i + 1
        return 0

```

1905. 2970.py

Path: LeetCode-main/solutions/2970. Count the Number of Incremovable Subarrays I/2970.py


```

class Solution:
    def incremovableSubarrayCount(self, nums: list[int]) -> int:
        n = len(nums)
        startIndex = self._getStartIndexofSuffix(nums)
        # If the complete array is strictly increasing, the total number of ways we
        # can remove elements equals the total number of possible subarrays.
        if startIndex == 0:
            return n * (n + 1) // 2

        # The valid removals starting from nums[0] include nums[0..startIndex - 1],
        # nums[0..startIndex], ..., nums[0..n).
        ans = n - startIndex + 1

        # Enumerate each prefix subarray that is strictly increasing.
        for i in range(startIndex):
            if i > 0 and nums[i] <= nums[i - 1]:
                break
            # Since nums[0..i] is strictly increasing, find the first index j in
            # nums[startIndex..n) such that nums[j] > nums[i]. The valid removals
            # will then be nums[i + 1..j - 1], nums[i + 1..j], ..., nums[i + 1..n).
            ans += n - bisect.bisect_right(nums, nums[i], startIndex) + 1

        return ans

    def _getStartIndexofSuffix(self, nums: list[int]) -> int:
        for i in range(len(nums) - 2, -1, -1):
            if nums[i] >= nums[i + 1]:
                return i + 1
        return 0

```

1906. 2971.py

Path: LeetCode-main/solutions/2971. Find Polygon With the Largest Perimeter/2971.py

```

class Solution:
    def largestPerimeter(self, nums: list[int]) -> int:
        prefix = sum(nums)

        for num in sorted(nums, reverse=True):
            prefix -= num
            # Let `num` be the longest side. Check if the sum of all the edges with
            # length no longer than `num` > `num`.
            if prefix > num:
                return prefix + num

        return -1

```

1907. 2972-2.py

Path: LeetCode-main/solutions/2972. Count the Number of Incremovable Subarrays II/2972-2.py

```

class Solution:
    # Same as 2970. Count the Number of Incremovable Subarrays I
    def incremovableSubarrayCount(self, nums: list[int]) -> int:
        n = len(nums)
        startIndex = self._getStartIndexofSuffix(nums)
        # If the complete array is strictly increasing, the total number of ways we
        # can remove elements equals the total number of possible subarrays.
        if startIndex == 0:
            return n * (n + 1) // 2

        # The valid removals starting from nums[0] include nums[0..startIndex - 1],
        # nums[0..startIndex], ..., nums[0..n].
        ans = n - startIndex + 1

        # Enumerate each prefix subarray that is strictly increasing.
        j = startIndex
        for i in range(startIndex):
            if i > 0 and nums[i] <= nums[i - 1]:
                break
            # Since nums[0..i] is strictly increasing, move j to the place such that
            # nums[j] > nums[i]. The valid removals will then be nums[i + 1..j - 1],
            # nums[i + 1..j], ..., nums[i + 1..n].
            while j < n and nums[i] >= nums[j]:
                j += 1
            ans += n - j + 1

        return ans

    def _getStartIndexofSuffix(self, nums: list[int]) -> int:
        for i in range(len(nums) - 2, -1, -1):
            if nums[i] >= nums[i + 1]:
                return i + 1
        return 0

```

1908. 2972.py

Path: LeetCode-main/solutions/2972. Count the Number of Incremovable Subarrays II/2972.py

```

class Solution:
    # Same as 2970. Count the Number of Incremovable Subarrays I
    def incremovableSubarrayCount(self, nums: list[int]) -> int:
        n = len(nums)
        startIndex = self._getStartIndexOfSuffix(nums)
        # If the complete array is strictly increasing, the total number of ways we
        # can remove elements equals the total number of possible subarrays.
        if startIndex == 0:
            return n * (n + 1) // 2

        # The valid removals starting from nums[0] include nums[0..startIndex - 1],
        # nums[0..startIndex], ..., nums[0..n).
        ans = n - startIndex + 1

        # Enumerate each prefix subarray that is strictly increasing.
        for i in range(startIndex):
            if i > 0 and nums[i] <= nums[i - 1]:
                break
            # Since nums[0..i] is strictly increasing, find the first index j in
            # nums[startIndex..n) such that nums[j] > nums[i]. The valid removals
            # will then be nums[i + 1..j - 1], nums[i + 1..j], ..., nums[i + 1..n).
            ans += n - bisect.bisect_right(nums, nums[i], startIndex) + 1

        return ans

    def _getStartIndexOfSuffix(self, nums: list[int]) -> int:
        for i in range(len(nums) - 2, -1, -1):
            if nums[i] >= nums[i + 1]:
                return i + 1
        return 0

```

1909. 2973.py

Path: LeetCode-main/solutions/2973. Find Number of Coins to Place in Tree Nodes/2973.py

```

class ChildCost:
    def __init__(self, cost: int):
        self.numNodes = 1
        self.maxPosCosts = [cost] if cost > 0 else []
        self.minNegCosts = [cost] if cost < 0 else []

    def update(self, childCost: 'ChildCost') -> None:
        self.numNodes += childCost.numNodes
        self.maxPosCosts.extend(childCost.maxPosCosts)
        self.minNegCosts.extend(childCost.minNegCosts)
        self.maxPosCosts.sort(reverse=True)
        self.minNegCosts.sort()
        self.maxPosCosts = self.maxPosCosts[:3]
        self.minNegCosts = self.minNegCosts[:2]

    def maxProduct(self) -> int:
        if self.numNodes < 3:
            return 1
        if not self.maxPosCosts:
            return 0
        res = 0
        if len(self.maxPosCosts) == 3:
            res = self.maxPosCosts[0] * self.maxPosCosts[1] * self.maxPosCosts[2]
        if len(self.minNegCosts) == 2:
            res = max(res,
                      self.minNegCosts[0] * self.minNegCosts[1] * self.maxPosCosts[0])
        return res

class Solution:
    def placedCoins(self, edges: list[list[int]], cost: list[int]) -> list[int]:
        n = len(cost)
        ans = [0] * n
        tree = [[] for _ in range(n)]

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        def dfs(u: int, prev: int) -> None:
            res = ChildCost(cost[u])
            for v in tree[u]:
                if v != prev:
                    res.update(dfs(v, u))
            ans[u] = res.maxProduct()
            return res

        dfs(0, -1)
        return ans

```

1910. 2974.py

Path: LeetCode-main/solutions/2974. Minimum Number Game/2974.py

```

class Solution:
    def numberGame(self, nums: list[int]) -> list[int]:
        nums.sort()
        return [nums[i + 1] if i % 2 == 0
                else nums[i - 1]
                for i in range(len(nums))]

```

1911. 2975.py

Path: LeetCode-main/solutions/2975. Maximum Square Area by Removing Fences From a Field/2975.py

```
class Solution:
    def maximizeSquareArea(
        self,
        m: int,
        n: int,
        hFences: list[int],
        vFences: list[int],
    ) -> int:
        hFences = sorted(hFences + [1, m])
        vFences = sorted(vFences + [1, n])
        hGaps = {hFences[i] - hFences[j]
                  for i in range(len(hFences))
                  for j in range(i)}
        vGaps = {vFences[i] - vFences[j]
                  for i in range(len(vFences))
                  for j in range(i)}
        maxGap = next((hGap
                       for hGap in sorted(hGaps, reverse=True)
                       if hGap in vGaps), -1)
        return -1 if maxGap == -1 else maxGap**2 % (10**9 + 7)
```

1912. 2976.py

Path: LeetCode-main/solutions/2976. Minimum Cost to Convert String I/2976.py

```

class Solution:
    def minimumCost(
        self,
        source: str,
        target: str,
        original: list[str],
        changed: list[str],
        cost: list[int],
    ) -> int:
        ans = 0
        # dist[u][v] := the minimum distance to change ('a' + u) to ('a' + v)
        dist = [[math.inf] * 26 for _ in range(26)]

        for a, b, c in zip(original, changed, cost):
            u = ord(a) - ord('a')
            v = ord(b) - ord('a')
            dist[u][v] = min(dist[u][v], c)

        for k in range(26):
            for i in range(26):
                if dist[i][k] < math.inf:
                    for j in range(26):
                        if dist[k][j] < math.inf:
                            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

        for s, t in zip(source, target):
            if s == t:
                continue
            u = ord(s) - ord('a')
            v = ord(t) - ord('a')
            if dist[u][v] == math.inf:
                return -1
            ans += dist[u][v]

        return ans

```

1913. 2977.py

Path: LeetCode-main/solutions/2977. Minimum Cost to Convert String II/2977.py

```

class Solution:
    def minimumCost(
        self,
        source: str,
        target: str,
        original: list[str],
        changed: list[str],
        cost: list[int],
    ) -> int:
        subLengths = set(len(s) for s in original)
        subToId = self._getSubToId(original, changed)
        subCount = len(subToId)
        # dist[u][v] := the minimum distance to change the substring with id u to
        # the substring with id v
        dist = [[math.inf for _ in range(subCount)] for _ in range(subCount)]
        # dp[i] := the minimum cost to change the first i letters of `source` into
        # `target`, leaving the suffix untouched
        dp = [math.inf for _ in range(len(source) + 1)]

        for a, b, c in zip(original, changed, cost):
            u = subToId[a]
            v = subToId[b]
            dist[u][v] = min(dist[u][v], c)

        for k in range(subCount):
            for i in range(subCount):
                if dist[i][k] < math.inf:
                    for j in range(subCount):
                        if dist[k][j] < math.inf:
                            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

        dp[0] = 0

        for i, (s, t) in enumerate(zip(source, target)):
            if dp[i] == math.inf:
                continue
            if s == t:
                dp[i + 1] = min(dp[i + 1], dp[i])
            for subLength in subLengths:
                if i + subLength > len(source):
                    continue
                subSource = source[i:i + subLength]
                subTarget = target[i:i + subLength]
                if subSource not in subToId or subTarget not in subToId:
                    continue
                u = subToId[subSource]
                v = subToId[subTarget]
                if dist[u][v] != math.inf:
                    dp[i + subLength] = min(dp[i + subLength], dp[i] + dist[u][v])

        return -1 if dp[len(source)] == math.inf else dp[len(source)]

    def _getSubToId(self, original: str, changed: str) -> dict[str, int]:
        subToId = {}
        for s in original + changed:
            if s not in subToId:
                subToId[s] = len(subToId)
        return subToId

```

1914. 2979.py

Path: LeetCode-main/solutions/2979. Most Expensive Item That Can Not Be Bought/2979.py

```
class Solution:
    def mostExpensiveItem(self, primeOne: int, primeTwo: int) -> int:
        # https://en.wikipedia.org/wiki/Coin_problem
        return primeOne * primeTwo - primeOne - primeTwo
```

1915. 298.py

Path: LeetCode-main/solutions/298. Binary Tree Longest Consecutive Sequence/298.py

```
class Solution:
    def longestConsecutive(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        def dfs(root: TreeNode | None, target: int, length: int, maxLength: int) -> int:
            if not root:
                return maxLength
            if root.val == target:
                length += 1
                maxLength = max(maxLength, length)
            else:
                length = 1
            return max(dfs(root.left, root.val + 1, length, maxLength),
                      dfs(root.right, root.val + 1, length, maxLength))

        return dfs(root, root.val, 0, 0)
```

1916. 2980.py

Path: LeetCode-main/solutions/2980. Check if Bitwise OR Has Trailing Zeros/2980.py

```
class Solution:
    def hasTrailingZeros(self, nums: list[int]) -> bool:
        return sum(num % 2 == 0 for num in nums) >= 2
```

1917. 2981.py

Path: LeetCode-main/solutions/2981. Find Longest Special Substring That Occurs Thrice I/2981.py


```

class Solution:
    def maximumLength(self, s: str) -> int:
        n = len(s)
        runningLen = 0
        prevLetter = '@'
        # counts[i][j] := the frequency of ('a' + i) repeating j times
        counts = [[0] * (n + 1) for _ in range(26)]

        for c in s:
            if c == prevLetter:
                runningLen += 1
                counts[ord(c) - ord('a')][runningLen] += 1
            else:
                runningLen = 1
                counts[ord(c) - ord('a')][runningLen] += 1
                prevLetter = c

        def getMaxFreq(count: list[int]) -> int:
            """Returns the maximum frequency that occurs more than three times."""
            times = 0
            for freq in range(n, 0, -1):
                times += count[freq]
                if times >= 3:
                    return freq
            return -1

        return max(getMaxFreq(count) for count in counts)

```

1918. 2982.py

Path: LeetCode-main/solutions/2982. Find Longest Special Substring That Occurs Thrice II/2982.py

```

class Solution:
    def maximumLength(self, s: str) -> int:
        n = len(s)
        runningLen = 0
        prevLetter = '@'
        # counts[i][j] := the frequency of ('a' + i) repeating j times
        counts = [[0] * (n + 1) for _ in range(26)]

        for c in s:
            if c == prevLetter:
                runningLen += 1
                counts[ord(c) - ord('a')][runningLen] += 1
            else:
                runningLen = 1
                counts[ord(c) - ord('a')][runningLen] += 1
                prevLetter = c

        def getMaxFreq(count: list[int]) -> int:
            """Returns the maximum frequency that occurs more than three times."""
            times = 0
            for freq in range(n, 0, -1):
                times += count[freq]
                if times >= 3:
                    return freq
            return -1

        return max(getMaxFreq(count) for count in counts)

```

1919. 2983.py

Path: LeetCode-main/solutions/2983. Palindrome Rearrangement Queries/2983.py

```
class Solution:
    def canMakePalindromeQueries(
        self,
        s: str,
        queries: list[list[int]],
    ) -> list[bool]:
        n = len(s)
        # mirroredDiffs[i] := the number of different letters between the first i
        # letters of s[0..n / 2) and the first i letters of s[n / 2..n][::-1]
        mirroredDiffs = self._getMirroredDiffs(s)
        # counts[i] := the count of s[0..i)
        counts = self._getCounts(s)
        ans = []

        def subtractArrays(a: list[int], b: list[int]):
            return [x - y for x, y in zip(a, b)]

        for a, b, c, d in queries:
            # Use left-closed, right-open intervals to facilitate the calculation.
            # ..... [a, b) ...|... [rb, ra) .....
            # .... [rd, rc) .....|..... [c, d) ....
            b += 1
            d += 1
            ra = n - a # the reflected index of a in s[n / 2..n)
            rb = n - b # the reflected index of b in s[n / 2..n)
            rc = n - c # the reflected index of c in s[n / 2..n)
            rd = n - d # the reflected index of d in s[n / 2..n)
            # No difference is allowed outside the query ranges.
            if ((min(a, rd) > 0 and mirroredDiffs[min(a, rd)] > 0) or
                (n // 2 > max(b, rc) and
                 mirroredDiffs[n // 2] - mirroredDiffs[max(b, rc)] > 0) or
                (rd > b and mirroredDiffs[rd] - mirroredDiffs[b] > 0) or
                (a > rc and mirroredDiffs[a] - mirroredDiffs[rc] > 0)):
                ans.append(False)
            else:
                # The `count` map of the intersection of [a, b) and [rd, rc) in
                # s[0..n / 2) must equate to the `count` map of the intersection of
                # [c, d) and [rb, ra) in s[n / 2..n).
                leftRangeCount = subtractArrays(counts[b], counts[a])
                rightRangeCount = subtractArrays(counts[d], counts[c])
                if a > rd:
                    rightRangeCount = subtractArrays(
                        rightRangeCount, subtractArrays(counts[min(a, rc)], counts[rd]))
                if rc > b:
                    rightRangeCount = subtractArrays(
                        rightRangeCount, subtractArrays(counts[rc], counts[max(b, rd)]))
                if c > rb:
                    leftRangeCount = subtractArrays(
                        leftRangeCount, subtractArrays(counts[min(c, ra)], counts[rb]))
                if ra > d:
                    leftRangeCount = subtractArrays(
                        leftRangeCount, subtractArrays(counts[ra], counts[max(d, rb)]))
                ans.append(min(leftRangeCount) >= 0
                           and min(rightRangeCount) >= 0
                           and leftRangeCount == rightRangeCount)

        return ans

    def _getMirroredDiffs(self, s: str) -> list[int]:
        diffs = [0]
        for i, j in zip(range(len(s)), reversed(range(len(s)))):
```

```

        if i >= j:
            break
        diffs.append(diffs[-1] + (s[i] != s[j]))
    return diffs

def _getCounts(self, s: str) -> list[list[int]]:
    count = [0] * 26
    counts = [count.copy()]
    for c in s:
        count[ord(c) - ord('a')] += 1
        counts.append(count.copy())
    return counts

```

1920. 299.py

Path: LeetCode-main/solutions/299. Bulls and Cows/299.py

```

class Solution:
    def getHint(self, secret: str, guess: str) -> str:
        bulls = sum(map(operator.eq, secret, guess))
        bovine = sum(min(secret.count(x), guess.count(x)) for x in set(guess))
        return '%dA%dB' % (bulls, bovine - bulls)

```

1921. 2992.py

Path: LeetCode-main/solutions/2992. Number of Self-Divisible Permutations/2992.py

```

class Solution:
    def selfDivisiblePermutationCount(self, n: int) -> int:
        def dfs(num: int, used: int) -> int:
            if num > n:
                return 1

            count = 0
            for i in range(1, n + 1):
                if (used >> i & 1) == 0 and (num % i == 0 or i % num == 0):
                    count += dfs(num + 1, used | 1 << i)

            return count

        return dfs(1, 0)

```

1922. 2996.py

Path: LeetCode-main/solutions/2996. Smallest Missing Integer Greater Than Sequential Prefix Sum/2996.py

```
class Solution:
    def missingInteger(self, nums: list[int]) -> int:
        numsSet = set(nums)
        ans = nums[0]

        for i in range(1, len(nums)):
            if nums[i] != nums[i - 1] + 1:
                break
            ans += nums[i]

        while ans in numsSet:
            ans += 1

        return ans
```

1923. 2997.py

Path: LeetCode-main/solutions/2997. Minimum Number of Operations to Make Array XOR Equal to K/2997.py

```
class Solution:
    def minOperations(self, nums: list[int], k: int) -> int:
        return functools.reduce(operator.xor, nums, k).bit_count()
```

1924. 2998.py

Path: LeetCode-main/solutions/2998. Minimum Number of Operations to Make X and Y Equal/2998.py

```
class Solution:
    def minimumOperationsToMakeEqual(self, x, y):
        if x <= y:
            return y - x

        queue = collections.deque([x])
        seen = set()

        ans = 0
        while queue:
            for _ in range(len(queue)):
                num = queue.popleft()
                if num == y:
                    return ans
                if num in seen:
                    continue
                seen.add(num)
                if num % 11 == 0:
                    queue.append(num // 11)
                if num % 5 == 0:
                    queue.append(num // 5)
                queue.append(num - 1)
                queue.append(num + 1)
            ans += 1
```

1925. 3-2.py

Path: LeetCode-main/solutions/3. Longest Substring Without Repeating Characters/3-2.py

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        # The substring s[j + 1..i] has no repeating characters.
        j = -1
        # lastSeen[c] := the index of the last time c appeared
        lastSeen = {}

        for i, c in enumerate(s):
            # Update j to lastSeen[c], so the window must start from j + 1.
            j = max(j, lastSeen.get(c, -1))
            ans = max(ans, i - j)
            lastSeen[c] = i

        return ans
```

1926. 3.py

Path: LeetCode-main/solutions/3. Longest Substring Without Repeating Characters/3.py

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] > 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

1927. 30.py

Path: LeetCode-main/solutions/30. Substring with Concatenation of All Words/30.py

```

class Solution:
    def findSubstring(self, s: str, words: list[str]) -> list[int]:
        if len(s) == 0 or words == []:
            return []

        k = len(words)
        n = len(words[0])
        ans = []
        count = collections.Counter(words)

        for i in range(len(s) - k * n + 1):
            seen = collections.defaultdict(int)
            j = 0
            while j < k:
                word = s[i + j * n: i + (j + 1) * n]
                seen[word] += 1
                if seen[word] > count[word]:
                    break
                j += 1
            if j == k:
                ans.append(i)

        return ans

```

1928. 300-2.py

Path: LeetCode-main/solutions/300. Longest Increasing Subsequence/300-2.py

```

class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        # tails[i] := the minimum tails of all the increasing subsequences having
        # length i + 1
        tails = []

        for num in nums:
            if not tails or num > tails[-1]:
                tails.append(num)
            else:
                tails[bisect.bisect_left(tails, num)] = num

        return len(tails)

```

1929. 300.py

Path: LeetCode-main/solutions/300. Longest Increasing Subsequence/300.py

```

class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        if not nums:
            return 0

        # dp[i] := the length of LIS ending in nums[i]
        dp = [1] * len(nums)

        for i in range(1, len(nums)):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)

```

1930. 3000.py

Path: LeetCode-main/solutions/3000. Maximum Area of Longest Diagonal Rectangle/3000.py

```

class Solution:
    def areaOfMaxDiagonal(self, dimensions: list[list[int]]) -> int:
        a, b = max(dimensions, key=lambda x: (x[0]**2 + x[1]**2, x[0] * x[1]))
        return a * b

```

1931. 3001.py

Path: LeetCode-main/solutions/3001. Minimum Moves to Capture The Queen/3001.py

```

class Solution:
    def minMovesToCaptureTheQueen(
        self, a: int, b: int, c: int, d: int, e: int, f: int,
    ) -> int:
        # The rook is in the same row as the queen.
        if a == e:
            # The bishop blocks the rook or not.
            return 2 if c == a and (b < d < f or b > d > f) else 1
        # The rook is in the same column as the queen.
        if b == f:
            # The bishop blocks the rook or not.
            return 2 if d == f and (a < c < e or a > c > e) else 1
        # The bishop is in the same up-diagonal as the queen.
        if c + d == e + f:
            # The rook blocks the bishop or not.
            return 2 if a + b == c + d and (c < a < e or c > a > e) else 1
        # The bishop is in the same down-diagonal as the queen.
        if c - d == e - f:
            # The rook blocks the bishop or not.
            return 2 if a - b == c - d and (c < a < e or c > a > e) else 1
        # The rook can always get the queen in two steps.
        return 2

```

1932. 3002.py

Path: LeetCode-main/solutions/3002. Maximum Size of a Set After Removals/3002.py

```

class Solution:
    def maximumSetSize(self, nums1: list[int], nums2: list[int]) -> int:
        set1 = set(nums1)
        set2 = set(nums2)
        common = set1.intersection(set2)

        n = len(nums1)
        n1 = len(set1)
        n2 = len(set2)
        nc = len(common)
        maxUniqueNums1 = min(n1 - nc, n // 2)
        maxUniqueNums2 = min(n2 - nc, n // 2)
        return min(n, maxUniqueNums1 + maxUniqueNums2 + nc)

```

1933. 3003.py

Path: LeetCode-main/solutions/3003. Maximize the Number of Partitions After Operations/3003.py

```

class Solution:
    def maxPartitionsAfterOperations(self, s: str, k: int) -> int:
        @functools.lru_cache(None)
        def dp(i: int, canChange: bool, mask: int) -> int:
            """
            Returns the maximum number of partitions of s[i..n), where `canChange` is
            True if we can still change a letter, and `mask` is the bitmask of the
            letters we've seen.
            """
            if i == len(s):
                return 0

            def getRes(newBit: int, nextCanChange: bool) -> int:
                newMask = mask | newBit
                if newMask.bit_count() > k:
                    return 1 + dp(i + 1, nextCanChange, newBit)
                return dp(i + 1, nextCanChange, newMask)

            # Initialize the result based on the current letter.
            res = getRes(1 << (ord(s[i]) - ord('a')), canChange)

            # If allowed, explore the option to change the current letter.
            if canChange:
                for j in range(26):
                    res = max(res, getRes(1 << j, False))
            return res

        return dp(0, True, 0) + 1

```

1934. 3004.py

Path: LeetCode-main/solutions/3004. Maximum Subtree of the Same Color/3004.py


```

class Solution:
    def maximumSubtreeSize(
        self,
        edges: list[list[int]],
        colors: list[int],
    ) -> int:
        ans = 1
        tree = [[] for _ in range(len(colors))]

        for u, v in edges:
            tree[u].append(v)

        def dfs(u: int) -> int:
            """
            Returns the size of subtree of u if every node in the subtree has the same
            color. Otherwise, returns -1.
            """
            nonlocal ans
            res = 1
            for v in tree[u]:
                if colors[v] != colors[u]:
                    res = -1
                # If any node in the subtree of v has a different color, the result of
                # the subtree of u will be -1 as well.
                subtreeSize = dfs(v)
                if subtreeSize == -1:
                    res = -1
                elif res != -1:
                    res += subtreeSize
            ans = max(ans, res)
            return res

        dfs(0)
        return ans

```

1935. 3005.py

Path: LeetCode-main/solutions/3005. Count Elements With Maximum Frequency/3005.py

```

class Solution:
    def maxFrequencyElements(self, nums: list[int]) -> int:
        count = collections.Counter(nums)
        maxFreq = max(count.values())
        return sum(freq == maxFreq for freq in count.values()) * maxFreq

```

1936. 3006.py

Path: LeetCode-main/solutions/3006. Find Beautiful Indices in the Given Array I/3006.py

```

class Solution:
    def beautifulIndices(self, s: str, a: str, b: str, k: int) -> list[int]:
        ans = []
        indicesA = self._kmp(s, a)
        indicesB = self._kmp(s, b)
        indicesBIndex = 0 # indicesB' index

        for i in indicesA:
            # The constraint is: |j - i| <= k. So, -k <= j - i <= k. So, move
            # `indicesBIndex` s.t. j - i >= -k, where j := indicesB[indicesBIndex].
            while indicesBIndex < len(indicesB) and indicesB[indicesBIndex] - i < -k:
                indicesBIndex += 1
            if indicesBIndex < len(indicesB) and indicesB[indicesBIndex] - i <= k:
                ans.append(i)

        return ans

    def _kmp(self, s: str, pattern: str) -> list[int]:
        """Returns the starting indices of all occurrences of the pattern in `s`."""

        def getLPS(pattern: str) -> list[int]:
            """
            Returns the lps array, where lps[i] is the length of the longest prefix of
            pattern[0..i] which is also a suffix of this substring.
            """
            lps = [0] * len(pattern)
            j = 0
            for i in range(1, len(pattern)):
                while j > 0 and pattern[j] != pattern[i]:
                    j = lps[j - 1]
                if pattern[i] == pattern[j]:
                    lps[i] = j + 1
                    j += 1
            return lps

        res = []
        lps = getLPS(pattern)
        i = 0 # s' index
        j = 0 # pattern's index
        while i < len(s):
            if s[i] == pattern[j]:
                i += 1
                j += 1
                if j == len(pattern):
                    res.append(i - j)
                    j = lps[j - 1]
            elif j != 0: # Mismatch after j matches.
                # Don't match lps[0..lps[j - 1]] since they will match anyway.
                j = lps[j - 1]
            else:
                i += 1
        return res

```

1937. 3007.py

Path: LeetCode-main/solutions/3007. Maximum Number That Sum of the Prices Is Less Than or Equal to K/3007.py

```
class Solution:
    def findMaximumNumber(self, k: int, x: int) -> int:
        def getSumPrices(num: int) -> int:
            """Returns the sum of prices of all numbers from 1 to `num`."""
            sumPrices = 0
            # Increment `num` to account the 0-th row in the count of groups.
            num += 1
            for i in range(num.bit_length(), 0, -1):
                if i % x == 0:
                    groupSize = 1 << i
                    halfGroupSize = 1 << i - 1
                    sumPrices += num // groupSize * halfGroupSize
                    sumPrices += max(0, (num % groupSize) - halfGroupSize)
            return sumPrices

        l = 1
        r = 10**15
        return bisect.bisect_right(range(l, r + 1), k, key=getSumPrices) - 1 + 1
```

1938. 3008.py

Path: LeetCode-main/solutions/3008. Find Beautiful Indices in the Given Array II/3008.py

```

class Solution:
    # Same as 3006. Find Beautiful Indices in the Given Array I
    def beautifulIndices(self, s: str, a: str, b: str, k: int) -> list[int]:
        ans = []
        indicesA = self._kmp(s, a)
        indicesB = self._kmp(s, b)
        indicesBIndex = 0 # indicesB' index

        for i in indicesA:
            # The constraint is: |j - i| <= k. So, -k <= j - i <= k. So, move
            # `indicesBIndex` s.t. j - i >= -k, where j := indicesB[indicesBIndex].
            while indicesBIndex < len(indicesB) and indicesB[indicesBIndex] - i < -k:
                indicesBIndex += 1
            if indicesBIndex < len(indicesB) and indicesB[indicesBIndex] - i <= k:
                ans.append(i)

        return ans

    def _kmp(self, s: str, pattern: str) -> list[int]:
        """Returns the starting indices of all occurrences of the pattern in `s`."""

        def getLPS(pattern: str) -> list[int]:
            """
            Returns the lps array, where lps[i] is the length of the longest prefix of
            pattern[0..i] which is also a suffix of this substring.
            """
            lps = [0] * len(pattern)
            j = 0
            for i in range(1, len(pattern)):
                while j > 0 and pattern[j] != pattern[i]:
                    j = lps[j - 1]
                if pattern[i] == pattern[j]:
                    lps[i] = j + 1
                    j += 1
            return lps

        lps = getLPS(pattern)
        res = []
        i = 0 # s' index
        j = 0 # pattern's index
        while i < len(s):
            if s[i] == pattern[j]:
                i += 1
                j += 1
                if j == len(pattern):
                    res.append(i - j)
                    j = lps[j - 1]
            elif j != 0: # Mismatch after j matches.
                # Don't match lps[0..lps[j - 1]] since they will match anyway.
                j = lps[j - 1]
            else:
                i += 1
        return res

```

1939. 3009.py

Path: LeetCode-main/solutions/3009. Maximum Number of Intersections on the Chart/3009.py

```
class Solution:
    def maxIntersectionCount(self, y: list[int]) -> int:
        ans = 0
        intersectionCount = 0
        line = collections.Counter()

        for i, (a, b) in enumerate(itertools.pairwise(y)):
            start = 2 * a
            end = 2 * b + (0 if i == len(y) - 2 else -1 if b > a else 1)
            line[min(start, end)] += 1
            line[max(start, end) + 1] -= 1

        for count in sorted(line):
            intersectionCount += line[count]
            ans = max(ans, intersectionCount)

        return ans
```

1940. 301.py

Path: LeetCode-main/solutions/301. Remove Invalid Parentheses/301.py

```

class Solution:
    def removeInvalidParentheses(self, s: str) -> list[str]:
        # Similar to 921. Minimum Add to Make Parentheses Valid
        def getLeftAndRightCounts(s: str) -> tuple[int, int]:
            """Returns how many '(' and ')' need to be deleted."""
            l = 0
            r = 0

            for c in s:
                if c == '(':
                    l += 1
                elif c == ')':
                    if l == 0:
                        r += 1
                    else:
                        l -= 1

            return l, r

        def isValid(s: str):
            opened = 0 # the number of '(' - # of ')'
            for c in s:
                if c == '(':
                    opened += 1
                elif c == ')':
                    opened -= 1
                if opened < 0:
                    return False
            return True # opened == 0

        ans = []

        def dfs(s: str, start: int, l: int, r: int) -> None:
            if l == 0 and r == 0 and isValid(s):
                ans.append(s)
                return

            for i in range(start, len(s)):
                if i > start and s[i] == s[i - 1]:
                    continue
                if r > 0 and s[i] == ')': # Delete s[i]
                    dfs(s[:i] + s[i + 1:], i, l, r - 1)
                elif l > 0 and s[i] == '(': # Delete s[i]
                    dfs(s[:i] + s[i + 1:], i, l - 1, r)

        l, r = getLeftAndRightCounts(s)
        dfs(s, 0, l, r)
        return ans

```

1941. 3010.py

Path: LeetCode-main/solutions/3010. Divide an Array Into Subarrays With Minimum Cost I/3010.py

```
class Solution:
    def minimumCost(self, nums: list[int]) -> int:
        MAX = 50
        min1 = MAX
        min2 = MAX

        for i in range(1, len(nums)):
            if nums[i] < min1:
                min2 = min1
                min1 = nums[i]
            elif nums[i] < min2:
                min2 = nums[i]

        return nums[0] + min1 + min2
```

1942. 3011.py

Path: LeetCode-main/solutions/3011. Find if Array Can Be Sorted/3011.py

```
class Solution:
    def canSortArray(self, nums: list[int]) -> int:
        # Divide the array into distinct segments where each segment is comprised
        # of consecutive elements sharing an equal number of set bits. Ensure that
        # for each segment, when moving from left to right, the maximum of a
        # preceding segment is less than the minimum of the following segment.
        prevSetBits = 0
        prevMax = -math.inf # the maximum of the previous segment
        currMax = -math.inf # the maximum of the current segment
        currMin = math.inf # the minimum of the current segment

        for num in nums:
            setBits = num.bit_count()
            if setBits != prevSetBits: # Start a new segment.
                if prevMax > currMin:
                    return False
                prevSetBits = setBits
                prevMax = currMax
                currMax = num
                currMin = num
            else: # Continue with the current segment.
                currMax = max(currMax, num)
                currMin = min(currMin, num)

        return prevMax <= currMin
```

1943. 3012.py

Path: LeetCode-main/solutions/3012. Minimize Length of Array Using Operations/3012.py

```
class Solution:
    def minimumArrayLength(self, nums: list[int]) -> int:
        # Let the minimum number in the array `nums` be x.
        # * If there exists any element nums[i] where nums[i] % x > 0, a new
        #   minimum can be generated and all other numbers can be removed.
        # * If not, count the frequency of x in `nums`. For each pair of x, a 0 is
        #   generated which cannot be removed. Therefore, the result will be
        #   (frequency of x + 1) / 2.
        minNum = min(nums)
        if any(num % minNum > 0 for num in nums):
            return 1
        return (nums.count(minNum) + 1) // 2
```

1944. 3013.py

Path: LeetCode-main/solutions/3013. Divide an Array Into Subarrays With Minimum Cost II/3013.py


```

from sortedcontainers import SortedList

class Solution:
    def minimumCost(self, nums: list[int], k: int, dist: int) -> int:
        # Equivalently, the problem is to find nums[0] + the minimum sum of the top
        # k - 1 numbers in nums[i..i + dist], where i > 0 and i + dist < n.
        windowSum = sum(nums[i] for i in range(1, dist + 2))
        selected = SortedList(nums[i] for i in range(1, dist + 2))
        candidates = SortedList()

        def balance() -> int:
            """
            Returns the updated `windowSum` by balancing the multiset `selected` to
            keep the top k - 1 numbers.
            """
            nonlocal windowSum
            while len(selected) < k - 1:
                minCandidate = candidates[0]
                windowSum += minCandidate
                selected.add(minCandidate)
                candidates.remove(minCandidate)
            while len(selected) > k - 1:
                maxSelected = selected[-1]
                windowSum -= maxSelected
                selected.remove(maxSelected)
                candidates.add(maxSelected)
            return windowSum

        windowSum = balance()
        minWindowSum = windowSum

        for i in range(dist + 2, len(nums)):
            outOfScope = nums[i - dist - 1]
            if outOfScope in selected:
                windowSum -= outOfScope
                selected.remove(outOfScope)
            else:
                candidates.remove(outOfScope)
            if nums[i] < selected[-1]: # nums[i] is a better number.
                windowSum += nums[i]
                selected.add(nums[i])
            else:
                candidates.add(nums[i])
            windowSum = balance()
            minWindowSum = min(minWindowSum, windowSum)

        return nums[0] + minWindowSum

```

1945. 3014.py

Path: LeetCode-main/solutions/3014. Minimum Number of Pushes to Type Word I/3014.py

```

class Solution:
    def minimumPushes(self, word: str) -> int:
        freqs = sorted(collections.Counter(word).values(), reverse=True)
        return sum(freq * (i // 8 + 1) for i, freq in enumerate(freqs))

```

1946. 3015.py

```

class Solution:
    def countOfPairs(self, n: int, x: int, y: int) -> list[int]:
        if x > y:
            x, y = y, x

        def bothInRing(ringLen: int) -> list[int]:
            """
            Returns the contribution from the scenario where two houses are located
            in the ring.
            """
            res = [0] * n
            for k in range(1, (ringLen - 1) // 2 + 1):
                res[k - 1] += ringLen
            if ringLen % 2 == 0:
                res[ringLen // 2 - 1] += ringLen // 2
            return res

        def bothInTheSameLine(lineLen: int) -> list[int]:
            """
            Returns the contribution from the scenario where two houses are either
            located in the left line [1, x) or the right line (y, n].
            """
            res = [0] * n
            for k in range(1, lineLen + 1):
                res[k - 1] += lineLen - k
            return res

        def lineToRing(lineLen: int, ringLen: int) -> list[int]:
            """
            Returns the contribution from the scenario where one house is either
            located in the left line [1, x) or the right line (y, n] and the
            other house is located in the cycle.
            """
            res = [0] * n
            for k in range(1, lineLen + ringLen):
                # min(
                #   at most k - 1 since we need to give 1 to the line,
                #   at most ringLen / 2 since for length > ringLen / 2, it can always be
                #   calculated as ringLen - ringLen / 2
                # )
                maxInRingLen = min(k - 1, ringLen // 2)
                # max(at least 0, at least k - lineLen)
                minInRingLen = max(0, k - lineLen)
                if minInRingLen <= maxInRingLen:
                    # Each ring length contributes 2 to the count due to the split of
                    # paths when entering the ring: One path traverses the upper half of
                    # the ring, and the other traverses the lower half.
                    # This is illustrated as follows:
                    #   Path 1: ... -- x -- (upper half of the ring)
                    #   Path 2: ... -- x -- (lower half of the ring)
                    res[k - 1] += (maxInRingLen - minInRingLen + 1) * 2
                if minInRingLen == 0:
                    # Subtract 1 since there's no split.
                    res[k - 1] -= 1
                if maxInRingLen * 2 == ringLen:
                    # Subtract 1 since the following case only contribute one:
                    #   ... -- x -- (upper half of the ring) -- middle point
                    #   ... -- x -- (upper half of the ring) -- middle point
                    res[k - 1] -= 1
            return res

        def lineToLine(leftLineLen: int, rightLineLen: int) -> list[int]:
            """
            Returns the contribution from the scenario where two houses are in the left

```

```

Returns the contribution from the scenario where one house is in the left
line [1, x) and the other house is in the right line (y, n].
"""

res = [0] * n
for k in range(leftLineLen + rightLineLen + 2):
    # min(
    #   at most leftLineLen,
    #   at most k - 1 - (x < y) since we need to give 1 to the right line
    #   and if x < y we need to give another 1 to "x - y".
    # )
    maxInLeft = min(leftLineLen, k - 1 - (x < y))
    # max(at least 1, at least k - rightLineLen - (x < y))
    minInLeft = max(1, k - rightLineLen - (x < y))
    if minInLeft <= maxInLeft:
        res[k - 1] += maxInLeft - minInLeft + 1
return res

ringLen = y - x + 1
leftLineLen = x - 1
rightLineLen = (n - y)

ans = [0] * n
ans = list(map(operator.add, ans, bothInRing(ringLen)))
ans = list(map(operator.add, ans, bothInTheSameLine(leftLineLen)))
ans = list(map(operator.add, ans, bothInTheSameLine(rightLineLen)))
ans = list(map(operator.add, ans, lineToRing(leftLineLen, ringLen)))
ans = list(map(operator.add, ans, lineToRing(rightLineLen, ringLen)))
ans = list(map(operator.add, ans, lineToLine(leftLineLen, rightLineLen)))
return [freq * 2 for freq in ans]

```

1947. 3016.py

Path: LeetCode-main/solutions/3016. Minimum Number of Pushes to Type Word II/3016.py

```

class Solution:
    # Same as 3014. Minimum Number of Pushes to Type Word I
    def minimumPushes(self, word: str) -> int:
        freqs = sorted(collections.Counter(word).values(), reverse=True)
        return sum(freq * (i // 8 + 1) for i, freq in enumerate(freqs))

```

1948. 3017.py

Path: LeetCode-main/solutions/3017. Count the Number of Houses at a Certain Distance II/3017.py

```

class Solution:
    # Same as 3015. Count the Number of Houses at a Certain Distance I
    def countOfPairs(self, n: int, x: int, y: int) -> list[int]:
        if x > y:
            x, y = y, x

        def bothInRing(ringLen: int) -> list[int]:
            """
            Returns the contribution from the scenario where two houses are located
            in the ring.
            """
            res = [0] * n
            for k in range(1, (ringLen - 1) // 2 + 1):
                res[k - 1] += ringLen
            if ringLen % 2 == 0:
                res[ringLen // 2 - 1] += ringLen // 2
            return res

```

```

def bothInTheSameLine(lineLen: int) -> list[int]:
    """
    Returns the contribution from the scenario where two houses are either
    located in the left line [1, x) or the right line (y, n].
    """
    res = [0] * n
    for k in range(1, lineLen + 1):
        res[k - 1] += lineLen - k
    return res

def lineToRing(lineLen: int, ringLen: int) -> list[int]:
    """
    Returns the contribution from the scenario where one house is either
    located in the left line [1, x) or the right line (y, n] and the
    other house is located in the cycle.
    """
    res = [0] * n
    for k in range(1, lineLen + ringLen):
        # min(
        #   at most k - 1 since we need to give 1 to the line,
        #   at most ringLen / 2 since for length > ringLen / 2, it can always be
        #   calculated as ringLen - ringLen / 2
        # )
        maxInRingLen = min(k - 1, ringLen // 2)
        # max(at least 0, at least k - lineLen)
        minInRingLen = max(0, k - lineLen)
        if minInRingLen <= maxInRingLen:
            # Each ring length contributes 2 to the count due to the split of
            # paths when entering the ring: One path traverses the upper half of
            # the ring, and the other traverses the lower half.
            # This is illustrated as follows:
            #   Path 1: ... -- x -- (upper half of the ring)
            #   Path 2: ... -- x -- (lower half of the ring)
            res[k - 1] += (maxInRingLen - minInRingLen + 1) * 2
            if minInRingLen == 0:
                # Subtract 1 since there's no split.
                res[k - 1] -= 1
            if maxInRingLen * 2 == ringLen:
                # Subtract 1 since the following case only contribute one:
                #   ... -- x -- (upper half of the ring) -- middle point
                #   ... -- x -- (lower half of the ring) -- middle point
                res[k - 1] -= 1
    return res

def lineToLine(leftLineLen: int, rightLineLen: int) -> list[int]:
    """
    Returns the contribution from the scenario where one house is in the left
    line [1, x) and the other house is in the right line (y, n].
    """
    res = [0] * n
    for k in range(leftLineLen + rightLineLen + 2):
        # min(
        #   at most leftLineLen,
        #   at most k - 1 - (x < y) since we need to give 1 to the right line
        #   and if x < y we need to give another 1 to "x - y".
        # )
        maxInLeft = min(leftLineLen, k - 1 - (x < y))
        # max(at least 1, at least k - rightLineLen - (x < y))
        minInLeft = max(1, k - rightLineLen - (x < y))
        if minInLeft <= maxInLeft:
            res[k - 1] += maxInLeft - minInLeft + 1
    return res

ringLen = y - x + 1

```

```

leftLineLen = x - 1
rightLineLen = (n - y)

ans = [0] * n
ans = list(map(operator.add, ans, bothInRing(ringLen)))
ans = list(map(operator.add, ans, bothInTheSameLine(leftLineLen)))
ans = list(map(operator.add, ans, bothInTheSameLine(rightLineLen)))
ans = list(map(operator.add, ans, lineToRing(leftLineLen, ringLen)))
ans = list(map(operator.add, ans, lineToRing(rightLineLen, ringLen)))
ans = list(map(operator.add, ans, lineToLine(leftLineLen, rightLineLen)))
return [freq * 2 for freq in ans]

```

1949. 3018.py

Path: LeetCode-main/solutions/3018. Maximum Number of Removal Queries That Can Be Processed I/3018.py

```

class Solution:
    def maximumProcessableQueries(
        self,
        nums: list[int],
        queries: list[int],
    ) -> int:
        n = len(nums)
        # dp[i][j] := the maximum number of queries processed if nums[i..j] are not
        # removed after processing dp[i][j] queries
        dp = [[0] * n for _ in range(n)]

        for d in range(n - 1, -1, -1):
            for i in range(n):
                j = i + d
                if j >= n:
                    continue
                if i > 0:
                    # Remove nums[i - 1] from nums[i - 1..j] if possible.
                    dp[i][j] = max(dp[i][j], dp[i - 1][j] +
                                   (nums[i - 1] >= queries[dp[i - 1][j]]))
                if j + 1 < n:
                    # Remove nums[j + 1] from nums[i..j + 1] if possible.
                    dp[i][j] = max(dp[i][j], dp[i][j + 1] +
                                   (nums[j + 1] >= queries[dp[i][j + 1]]))
                if dp[i][j] == len(queries):
                    return len(queries)

        return max(dp[i][i] + (nums[i] >= queries[dp[i][i]]))
            for i in range(n))

```

1950. 3019.py

Path: LeetCode-main/solutions/3019. Number of Changing Keys/3019.py

```

class Solution:
    def countKeyChanges(self, s: str) -> int:
        return sum(a.lower() != b.lower()
                   for a, b in itertools.pairwise(s))

```

1951. 302-2.py

Path: LeetCode-main/solutions/302. Smallest Rectangle Enclosing Black Pixels/302-2.py

```
class Solution:
    def minArea(self, image: list[list[str]], x: int, y: int) -> int:
        def firstAnyOne(l: int, r: int, allZeros: Callable[[int], bool]) -> int:
            while l < r:
                m = (l + r) // 2
                if allZeros(m):
                    l = m + 1
                else:
                    r = m
            return l

        def firstAllZeros(l: int, r: int, allZeros: Callable[[int], bool]) -> int:
            while l < r:
                m = (l + r) // 2
                if allZeros(m):
                    r = m
                else:
                    l = m + 1
            return l

        def colAllZeros(colIndex: int) -> bool:
            return all(pixel == '0' for pixel in list(zip(*image))[colIndex])

        def rowAllZeros(rowIndex: int) -> bool:
            return all(pixel == '0' for pixel in image[rowIndex])

        x1 = firstAnyOne(0, x, rowAllZeros)
        x2 = firstAllZeros(x + 1, len(image), rowAllZeros)
        y1 = firstAnyOne(0, y, colAllZeros)
        y2 = firstAllZeros(y + 1, len(image[0]), colAllZeros)
        return (x2 - x1) * (y2 - y1)
```

1952. 302.py

Path: LeetCode-main/solutions/302. Smallest Rectangle Enclosing Black Pixels/302.py

```

class Solution:
    def minArea(self, image: list[list[str]], x: int, y: int) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(image)
        n = len(image[0])
        topLeft = [x, y]
        bottomRight = [x, y]
        q = collections.deque([(x, y)])
        image[x][y] = '2' # Mark as visited.

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                r = i + dx
                c = j + dy
                if r < 0 or r == m or c < 0 or c == n:
                    continue
                if image[r][c] != '1':
                    continue
                topLeft[0] = min(topLeft[0], r)
                topLeft[1] = min(topLeft[1], c)
                bottomRight[0] = max(bottomRight[0], r)
                bottomRight[1] = max(bottomRight[1], c)
                q.append((r, c))
                image[r][c] = '2'

        width = bottomRight[1] - topLeft[1] + 1
        height = bottomRight[0] - topLeft[0] + 1
        return width * height

```

1953. 3020.py

Path: LeetCode-main/solutions/3020. Find the Maximum Number of Elements in Subset/3020.py

```

class Solution:
    def maximumLength(self, nums: list[int]) -> int:
        maxNum = max(nums)
        count = collections.Counter(nums)
        ans = count[1] - (count[1] % 2 == 0) if 1 in count else 1

        for num in nums:
            if num == 1:
                continue
            length = 0
            x = num
            while x <= maxNum and x in count and count[x] >= 2:
                length += 2
                x *= x
            # x is now x^k, and the pattern is [x, ..., x^(k/2), x^(k/2), ..., x].
            # The goal is to determine if we can insert x^k in the middle of the
            # pattern to increase the length by 1. If not, we make x^(k/2) the middle
            # and decrease the length by 1.
            ans = max(ans, length + (1 if x in count else -1))

        return ans

```

1954. 3021.py

Path: LeetCode-main/solutions/3021. Alice and Bob Playing Flower Game/3021.py

```

class Solution:
    def flowerGame(self, n: int, m: int) -> int:
        # Alice wins if x + y is odd, occurring when:
        # 1. x is even and y is odd, or
        # 2. y is even and x is odd.
        xEven = n // 2
        yEven = m // 2
        xOdd = (n + 1) // 2
        yOdd = (m + 1) // 2
        return xEven * yOdd + yEven * xOdd

```

1955. 3022.py

Path: LeetCode-main/solutions/3022. Minimize OR of Remaining Elements Using Operations/3022.py

```

class Solution:
    def minOrAfterOperations(self, nums: list[int], k: int) -> int:
        MAX_BIT = 30
        ans = 0
        prefixMask = 0 # Grows like: 10000 -> 11000 -> ... -> 11111

        for i in range(MAX_BIT, -1, -1):
            # Add the i-th bit to `prefixMask` and attempt to "turn off" the
            # currently added bit within k operations. If it's impossible, then we
            # add the i-th bit to the answer.
            prefixMask |= 1 << i
            if self._getMergeOps(nums, prefixMask, ans) > k:
                ans |= 1 << i

        return ans

    def _getMergeOps(self, nums: list[int], prefixMask: int, target: int) -> int:
        """
        Returns the number of merge operations to turn `prefixMask` to the target
        by ANDing `nums`.
        """
        mergeOps = 0
        ands = prefixMask
        for num in nums:
            ands &= num
            if (ands | target) == target:
                ands = prefixMask
            else:
                mergeOps += 1 # Keep merging the next num
        return mergeOps

```

1956. 3023.py

Path: LeetCode-main/solutions/3023. Find Pattern in Infinite Stream I/3023.py


```

# Definition for an infinite stream.
# class InfiniteStream:
#     def next(self) -> int:
#         pass

class Solution:
    def findPattern(
        self,
        stream: Optional['InfiniteStream'],
        pattern: list[int],
    ) -> int:
        lps = self._getLPS(pattern)
        i = 0 # stream's index
        j = 0 # pattern's index
        bit = 0 # the bit in the stream
        readNext = False
        while True:
            if not readNext:
                bit = stream.next()
                readNext = True
            if bit == pattern[j]:
                i += 1
                readNext = False
                j += 1
                if j == len(pattern):
                    return i - j
            # Mismatch after j matches.
            elif j > 0:
                # Don't match lps[0..lps[j - 1]] since they will match anyway.
                j = lps[j - 1]
            else:
                i += 1
                readNext = False

    def _getLPS(self, pattern: list[int]) -> list[int]:
        """
        Returns the lps array, where lps[i] is the length of the longest prefix of
        pattern[0..i] which is also a suffix of this substring.
        """
        lps = [0] * len(pattern)
        j = 0
        for i in range(1, len(pattern)):
            while j > 0 and pattern[j] != pattern[i]:
                j = lps[j - 1]
            if pattern[i] == pattern[j]:
                j += 1
            lps[i] = j
        return lps

```

1957. 3024.py

Path: LeetCode-main/solutions/3024. Type of Triangle II Easy/3024.py

```
class Solution:
    def triangleType(self, nums: list[int]) -> str:
        nums.sort()
        if nums[0] + nums[1] <= nums[2]:
            return 'none'
        if nums[0] == nums[1] and nums[1] == nums[2]:
            return 'equilateral'
        if nums[0] == nums[1] or nums[1] == nums[2]:
            return 'isosceles'
        return 'scalene'
```

1958. 3025.py

Path: LeetCode-main/solutions/3025. Find the Number of Ways to Place People I/3025.py

```
class Solution:
    def numberOfPairs(self, points: list[list[int]]) -> int:
        ans = 0

        points.sort(key=lambda x: (x[0], -x[1]))

        for i, (_, yi) in enumerate(points):
            maxY = -math.inf
            for j in range(i + 1, len(points)):
                _, yj = points[j]
                # Chisato is in the upper-left corner at (xi, yi), and Takina is in the
                # lower-right corner at (xj, yj). Also, if yj > maxY, it means that
                # nobody other than Chisato and Takina is inside or on the fence.
                if yi >= yj > maxY:
                    ans += 1
                    maxY = yj

        return ans
```

1959. 3026.py

Path: LeetCode-main/solutions/3026. Maximum Good Subarray Sum/3026.py

```
class Solution:
    def maximumSubarraySum(self, nums: list[int], k: int) -> int:
        ans = -math.inf
        prefix = 0
        numToMinPrefix = {}

        for num in nums:
            if num not in numToMinPrefix or numToMinPrefix[num] > prefix:
                numToMinPrefix[num] = prefix
            prefix += num
            if num + k in numToMinPrefix:
                ans = max(ans, prefix - numToMinPrefix[num + k])
            if num - k in numToMinPrefix:
                ans = max(ans, prefix - numToMinPrefix[num - k])

        return 0 if ans == -math.inf else ans
```

1960. 3027.py

Path: LeetCode-main/solutions/3027. Find the Number of Ways to Place People II/3027.py

```
class Solution:
    # Same as 3025. Find the Number of Ways to Place People I
    def numberOfPairs(self, points: list[list[int]]) -> int:
        ans = 0

        points.sort(key=lambda x: (x[0], -x[1]))

        for i, (_, yi) in enumerate(points):
            maxY = -math.inf
            for j in range(i + 1, len(points)):
                _, yj = points[j]
                # Chisato is in the upper-left corner at (xi, yi), and Takina is in the
                # lower-right corner at (xj, yj). Also, if yj > maxY, it means that
                # nobody other than Chisato and Takina is inside or on the fence.
                if yi >= yj > maxY:
                    ans += 1
                    maxY = yj

        return ans
```

1961. 3028.py

Path: LeetCode-main/solutions/3028. Ant on the Boundary/3028.py

```
class Solution:
    def returnToBoundaryCount(self, nums: list[int]) -> int:
        return sum(prefix == 0 for prefix in itertools.accumulate(nums))
```

1962. 3029.py

Path: LeetCode-main/solutions/3029. Minimum Time to Revert Word to Initial State I/3029.py

```

class Solution:
    # Same as 3029. Minimum Time to Revert Word to Initial State I
    def minimumTimeToInitialState(self, word: str, k: int) -> int:
        n = len(word)
        maxOps = (n - 1) // k + 1
        z = self._zFunction(word)

        for ans in range(1, maxOps):
            if z[ans * k] >= n - ans * k:
                return ans

        return maxOps

    def _zFunction(self, s: str) -> list[int]:
        """
        Returns the z array, where z[i] is the length of the longest prefix of
        s[i..n) which is also a prefix of s.

        https://cp-algorithms.com/string/z-function.html#implementation
        """
        n = len(s)
        z = [0] * n
        l = 0
        r = 0
        for i in range(1, n):
            if i < r:
                z[i] = min(r - i, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] > r:
                l = i
                r = i + z[i]
        return z

```

1963. 303.py

Path: LeetCode-main/solutions/303. Range Sum Query - Immutable/303.py

```

class NumArray:
    def __init__(self, nums: list[int]):
        self.prefix = list(itertools.accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) -> int:
        return self.prefix[right + 1] - self.prefix[left]

```

1964. 3030.py

Path: LeetCode-main/solutions/3030. Find the Grid of Region Average/3030.py

```

class Solution:
    def resultGrid(
        self,
        image: list[list[int]],
        threshold: int,
    ) -> list[list[int]]:
        m = len(image)
        n = len(image[0])
        sums = [[0] * n for _ in range(m)]
        counts = [[0] * n for _ in range(m)]

        for i in range(m - 2):
            for j in range(n - 2):
                if self._isRegion(image, i, j, threshold):
                    subgridSum = sum(image[x][y]
                                     for x in range(i, i + 3)
                                     for y in range(j, j + 3))
                    for x in range(i, i + 3):
                        for y in range(j, j + 3):
                            sums[x][y] += subgridSum // 9
                            counts[x][y] += 1

        for i in range(m):
            for j in range(n):
                if counts[i][j] > 0:
                    image[i][j] = sums[i][j] // counts[i][j]

        return image

    def _isRegion(
        self,
        image: list[list[int]],
        i: int,
        j: int,
        threshold: int,
    ) -> bool:
        """Returns True if image[i..i + 2][j..j + 2] is a region."""
        for x in range(i, i + 3):
            for y in range(j, j + 3):
                if x > i and abs(image[x][y] - image[x - 1][y]) > threshold:
                    return False
                if y > j and abs(image[x][y] - image[x][y - 1]) > threshold:
                    return False
        return True

```

1965. 3031.py

Path: LeetCode-main/solutions/3031. Minimum Time to Revert Word to Initial State II/3031.py

```

class Solution:
    # Same as 3029. Minimum Time to Revert Word to Initial State I
    def minimumTimeToInitialState(self, word: str, k: int) -> int:
        n = len(word)
        maxOps = (n - 1) // k + 1
        z = self._zFunction(word)

        for ans in range(1, maxOps):
            if z[ans * k] >= n - ans * k:
                return ans

        return maxOps

    def _zFunction(self, s: str) -> list[int]:
        """
        Returns the z array, where z[i] is the length of the longest prefix of
        s[i..n) which is also a prefix of s.

        https://cp-algorithms.com/string/z-function.html#implementation
        """
        n = len(s)
        z = [0] * n
        l = 0
        r = 0
        for i in range(1, n):
            if i < r:
                z[i] = min(r - i, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] > r:
                l = i
                r = i + z[i]
        return z

```

1966. 3032.py

Path: LeetCode-main/solutions/3032. Count Numbers With Unique Digits II/3032.py

```

class Solution:
    def numberCount(self, a: int, b: int) -> int:
        return sum(len(set(str(num))) == len(str(num))
                    for num in range(a, b + 1))

```

1967. 3033.py

Path: LeetCode-main/solutions/3033. Modify the Matrix/3033.py

```
class Solution:
    def modifiedMatrix(self, matrix: list[list[int]]) -> list[list[int]]:
        m = len(matrix)
        n = len(matrix[0])
        ans = matrix.copy()

        for j in range(n):
            mx = max(matrix[i][j] for i in range(m))
            for i in range(m):
                if matrix[i][j] == -1:
                    ans[i][j] = mx

        return ans
```

1968. 3034.py

Path: LeetCode-main/solutions/3034. Number of Subarrays That Match a Pattern I/3034.py

```

class Solution:
    def countMatchingSubarrays(self, nums: list[int], pattern: list[int]) -> int:
        def getNum(a: int, b: int) -> int:
            if a < b:
                return 1
            if a > b:
                return -1
            return 0

        numsPattern = [getNum(a, b) for a, b in itertools.pairwise(nums)]
        return self._kmp(numsPattern, pattern)

    def _kmp(self, nums: list[int], pattern: list[int]) -> int:
        """Returns the number of occurrences of the pattern in `nums`."""

        def getLPS(nums: list[int]) -> list[int]:
            """
            Returns the lps array, where lps[i] is the length of the longest prefix of
            nums[0..i] which is also a suffix of this substring.
            """
            lps = [0] * len(nums)
            j = 0
            for i in range(1, len(nums)):
                while j > 0 and nums[j] != nums[i]:
                    j = lps[j - 1]
                if nums[i] == nums[j]:
                    lps[i] = j + 1
                    j += 1
            return lps

        lps = getLPS(pattern)
        res = 0
        i = 0 # s' index
        j = 0 # pattern's index
        while i < len(nums):
            if nums[i] == pattern[j]:
                i += 1
                j += 1
                if j == len(pattern):
                    res += 1
                    j = lps[j - 1]
            elif j != 0: # Mismatch after j matches.
                # Don't match lps[0..lps[j - 1]] since they will match anyway.
                j = lps[j - 1]
            else:
                i += 1
        return res

```

1969. 3035.py

Path: LeetCode-main/solutions/3035. Maximum Palindromes After Operations/3035.py


```
class Solution:
    def maxPalindromesAfterOperations(self, words: list[str]) -> int:
        ans = 0
        count = collections.Counter(''.join(words))
        pairs = sum(value // 2 for value in count.values())

        for length in sorted(len(word) for word in words):
            needPairs = length // 2
            if pairs < needPairs:
                return ans
            ans += 1
            pairs -= needPairs

        return ans
```

1970. 3036.py

Path: LeetCode-main/solutions/3036. Number of Subarrays That Match a Pattern II/3036.py

```

class Solution:
    # Same as 3034. Number of Subarrays That Match a Pattern I
    def countMatchingSubarrays(self, nums: list[int], pattern: list[int]) -> int:
        def getNum(a: int, b: int) -> int:
            if a < b:
                return 1
            if a > b:
                return -1
            return 0

        numsPattern = [getNum(a, b) for a, b in itertools.pairwise(nums)]
        return self._kmp(numsPattern, pattern)

    def _kmp(self, nums: list[int], pattern: list[int]) -> int:
        """Returns the number of occurrences of the pattern in `nums`."""

        def getLPS(nums: list[int]) -> list[int]:
            """
            Returns the lps array, where lps[i] is the length of the longest prefix of
            nums[0..i] which is also a suffix of this substring.
            """
            lps = [0] * len(nums)
            j = 0
            for i in range(1, len(nums)):
                while j > 0 and nums[j] != nums[i]:
                    j = lps[j - 1]
                if nums[i] == nums[j]:
                    lps[i] = j + 1
                    j += 1
            return lps

        lps = getLPS(pattern)
        res = 0
        i = 0 # s' index
        j = 0 # pattern's index
        while i < len(nums):
            if nums[i] == pattern[j]:
                i += 1
                j += 1
                if j == len(pattern):
                    res += 1
                    j = lps[j - 1]
            elif j != 0: # Mismatch after j matches.
                # Don't match lps[0..lps[j - 1]] since they will match anyway.
                j = lps[j - 1]
            else:
                i += 1
        return res

```

1971. 3037.py

Path: LeetCode-main/solutions/3037. Find Pattern in Infinite Stream II/3037.py

```

# Definition for an infinite stream.
# class InfiniteStream:
#     def next(self) -> int:
#         pass

class Solution:
    # Same as 3023. Find Pattern in Infinite Stream I
    def findPattern(
        self,
        stream: Optional['InfiniteStream'],
        pattern: list[int]
    ) -> int:
        lps = self._getLPS(pattern)
        i = 0 # stream's index
        j = 0 # pattern's index
        bit = 0 # the bit in the stream
        readNext = False
        while True:
            if not readNext:
                bit = stream.next()
                readNext = True
            if bit == pattern[j]:
                i += 1
                readNext = False
                j += 1
                if j == len(pattern):
                    return i - j
            # Mismatch after j matches.
            elif j > 0:
                # Don't match lps[0..lps[j - 1]] since they will match anyway.
                j = lps[j - 1]
            else:
                i += 1
                readNext = False

    def _getLPS(self, pattern: list[int]) -> list[int]:
        """
        Returns the lps array, where lps[i] is the length of the longest prefix of
        pattern[0..i] which is also a suffix of this substring.
        """
        lps = [0] * len(pattern)
        j = 0
        for i in range(1, len(pattern)):
            while j > 0 and pattern[j] != pattern[i]:
                j = lps[j - 1]
            if pattern[i] == pattern[j]:
                j += 1
            lps[i] = j
        return lps

```

1972. 3038.py

Path: LeetCode-main/solutions/3038. Maximum Number of Operations With the Same Score I/3038.py

```

class Solution:
    def maxOperations(self, nums: list[int]) -> int:
        ans = 1
        summ = nums[0] + nums[1]

        for i in range(2, len(nums) - 1, 2):
            if nums[i] + nums[i + 1] == summ:
                ans += 1
            else:
                break

        return ans

```

1973. 3039.py

Path: LeetCode-main/solutions/3039. Apply Operations to Make String Empty/3039.py

```

class Solution:
    def lastNonEmptyString(self, s: str) -> str:
        ans = []
        count = collections.Counter(s)
        maxFreq = max(count.values())

        for c in reversed(s):
            if count[c] == maxFreq:
                ans.append(c)
                count[c] -= 1

        return ''.join(reversed(ans))

```

1974. 304.py

Path: LeetCode-main/solutions/304. Range Sum Query 2D - Immutable/304.py

```

class NumMatrix:
    def __init__(self, matrix: list[list[int]]):
        if not matrix:
            return

        m = len(matrix)
        n = len(matrix[0])
        # prefix[i][j] := the sum of matrix[0..i][0..j]
        self.prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                self.prefix[i + 1][j + 1] = (matrix[i][j] + self.prefix[i][j + 1] +
                                              self.prefix[i + 1][j] - self.prefix[i][j])

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (self.prefix[row2 + 1][col2 + 1] - self.prefix[row1][col2 + 1] -
                self.prefix[row2 + 1][col1] + self.prefix[row1][col1])

```

1975. 3040.py

Path: LeetCode-main/solutions/3040. Maximum Number of Operations With the Same Score II/3040.py

```

class Solution:
    def maxOperations(self, nums: list[int]) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int, score: int) -> int:
            """
            Returns the maximum number of operations that can be performed for
            nums[i..j], s.t. all operations have the same `score`.
            """
            if i >= j:
                return 0
            deleteFirstTwo = (1 + dp(i + 2, j, score)
                               if nums[i] + nums[i + 1] == score else 0)
            deleteLastTwo = (1 + dp(i, j - 2, score)
                              if nums[j] + nums[j - 1] == score else 0)
            deleteFirstAndLast = (1 + dp(i + 1, j - 1, score)
                                  if nums[i] + nums[j] == score else 0)
            return max(deleteFirstTwo, deleteLastTwo, deleteFirstAndLast)

        n = len(nums)
        return max(dp(0, n - 1, nums[0] + nums[1]),
                   dp(0, n - 1, nums[-1] + nums[-2]),
                   dp(0, n - 1, nums[0] + nums[-1]))

```

1976. 3041-2.py

Path: LeetCode-main/solutions/3041. Maximize Consecutive Elements in an Array After Modification/3041-2.py

```

class Solution:
    def maxSelectedElements(self, nums: list[int]) -> int:
        ans = 1
        prev = -math.inf
        # the length of the longest consecutive elements (seq0) ending in the
        # previous number
        dp0 = 1
        # the length of the longest consecutive elements (seq1) ending in the
        # previous number + 1
        dp1 = 1

        for num in sorted(nums):
            if num == prev:
                dp1 = dp0 + 1 # Append `num + 1` to seq0.
            elif num == prev + 1:
                dp0 += 1 # Append `num` to seq0.
                dp1 += 1 # Add 1 to every number in seq0 and append `num + 1` to seq0.
            elif num == prev + 2:
                dp0 = dp1 + 1 # Append `num` to seq1.
                dp1 = 1 # Start a new sequence [ `num + 1` ].
            else:
                dp0 = 1 # Start a new sequence [ `num` ].
                dp1 = 1 # Start a new sequence [ `num + 1` ].
            ans = max(ans, dp0, dp1)
            prev = num

        return ans

```

1977. 3041.py

Path: LeetCode-main/solutions/3041. Maximize Consecutive Elements in an Array After Modification/3041.py

```
class Solution:
    def maxSelectedElements(self, nums: list[int]) -> int:
        ans = 0
        # {num: the length of the longest consecutive elements ending in num}
        dp = {}

        for num in sorted(nums):
            dp[num + 1] = dp.get(num, 0) + 1
            dp[num] = dp.get(num - 1, 0) + 1
            ans = max(ans, dp[num], dp[num + 1])

        return ans
```

1978. 3042.py

Path: LeetCode-main/solutions/3042. Count Prefix and Suffix Pairs I/3042.py

```
class TrieNode:
    def __init__(self):
        self.children: dict[tuple[str, str], TrieNode] = {}
        self.count = 0

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> int:
        node = self.root
        count = 0
        for i, prefix in enumerate(word):
            suffix = word[-1 - i]
            node = node.children.setdefault((prefix, suffix), TrieNode())
            count += node.count
        node.count += 1
        return count

class Solution:
    def countPrefixSuffixPairs(self, words: list[str]) -> int:
        trie = Trie()
        return sum(trie.insert(word) for word in words)
```

1979. 3043.py

Path: LeetCode-main/solutions/3043. Find the Length of the Longest Common Prefix/3043.py

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node: TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

    def search(self, word: str) -> int:
        prefixLength = 0
        node = self.root
        for c in word:
            if c not in node.children:
                break
            node = node.children[c]
            prefixLength += 1
        return prefixLength

class Solution:
    def longestCommonPrefix(self, arr1: list[int], arr2: list[int]) -> int:
        trie = Trie()

        for num in arr1:
            trie.insert(str(num))

        return max(trie.search(str(num)) for num in arr2)
```

1980. 3044.py

Path: LeetCode-main/solutions/3044. Most Frequent Prime/3044.py

```

class Solution:
    def mostFrequentPrime(self, mat: list[list[int]]) -> int:
        DIRS = ((1, 0), (1, -1), (0, -1), (-1, -1),
                (-1, 0), (-1, 1), (0, 1), (1, 1))

        m = len(mat)
        n = len(mat[0])
        count = collections.Counter()

        def isPrime(num: int) -> bool:
            return not any(num % i == 0 for i in range(2, int(num**0.5 + 1)))

        for i in range(m):
            for j in range(n):
                for dx, dy in DIRS:
                    num = 0
                    x = i
                    y = j
                    while 0 <= x < m and 0 <= y < n:
                        num = num * 10 + mat[x][y]
                        if num > 10 and isPrime(num):
                            count[num] += 1
                        x += dx
                        y += dy

        if not count.items():
            return -1
        return max(count.items(), key=lambda x: (x[1], x[0]))[0]

```

1981. 3045.py

Path: LeetCode-main/solutions/3045. Count Prefix and Suffix Pairs II/3045.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[tuple[str, str], TrieNode] = {}
        self.count = 0

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> int:
        node = self.root
        count = 0
        for i, prefix in enumerate(word):
            suffix = word[-1 - i]
            node = node.children.setdefault((prefix, suffix), TrieNode())
            count += node.count
        node.count += 1
        return count

class Solution:
    # Same as 3045. Count Prefix and Suffix Pairs II
    def countPrefixSuffixPairs(self, words: list[str]) -> int:
        trie = Trie()
        return sum(trie.insert(word) for word in words)

```


1982. 3046.py

Path: LeetCode-main/solutions/3046. Split the Array/3046.py

```
class Solution:
    def isPossibleToSplit(self, nums: list[int]) -> bool:
        return all(freq <= 2 for freq in collections.Counter(nums).values())
```

1983. 3047.py

Path: LeetCode-main/solutions/3047. Find the Largest Area of Square Inside Two Rectangles/3047.py

```
class Solution:
    def largestSquareArea(
        self,
        bottomLeft: list[list[int]],
        topRight: list[list[int]],
    ) -> int:
        minSide = 0

        for ((ax1, ay1), (ax2, ay2)), ((bx1, by1), (bx2, by2)) in (
            itertools.combinations(zip(bottomLeft, topRight), 2)):
            overlapX = min(ax2, bx2) - max(ax1, bx1)
            overlapY = min(ay2, by2) - max(ay1, by1)
            minSide = max(minSide, min(overlapX, overlapY))

        return minSide**2
```

1984. 3048.py

Path: LeetCode-main/solutions/3048. Earliest Second to Mark Indices I/3048.py

```

class Solution:
    def earliestSecondToMarkIndices(
        self,
        nums: list[int],
        changeIndices: list[int],
    ) -> int:
        def canMark(second: int) -> bool:
            """
            Returns True if all indices of `nums` can be marked within `second`.
            """
            numMarked = 0
            decrement = 0
            indexToLastSecond = {}

            for i in range(second):
                indexToLastSecond[changeIndices[i] - 1] = i

            for i in range(second):
                index = changeIndices[i] - 1 # Convert to 0-indexed
                if i == indexToLastSecond[index]:
                    # Reach the last occurrence of the number.
                    # So, the current second will be used to mark the index.
                    if nums[index] > decrement:
                        # The decrement is less than the number to be marked.
                        return False
                    decrement -= nums[index]
                    numMarked += 1
                else:
                    decrement += 1

            return numMarked == len(nums)

        l = 1
        r = len(changeIndices) + 1
        ans = bisect.bisect_left(range(l, r), True, key=canMark) + 1
        return ans if ans <= len(changeIndices) else -1

```

1985. 3049.py

Path: LeetCode-main/solutions/3049. Earliest Second to Mark Indices II/3049.py

```

class Solution:
    def earliestSecondToMarkIndices(
        self,
        nums: list[int],
        changeIndices: list[int],
    ) -> int:
        # {the second: the index of nums can be zeroed at the current second}
        secondToIndex = self._getSecondToIndex(nums, changeIndices)
        numsSum = sum(nums)

        def canMark(maxSecond: int) -> bool:
            """
            Returns True if all indices of `nums` can be marked within `maxSecond`.
            """
            # Use a min-heap to greedily pop out the minimum number, which yields the
            # least saving.
            minHeap = []
            marks = 0

            for second in range(maxSecond - 1, -1, -1):
                if second in secondToIndex:
                    # The number mapped by the index is a candidate to be zeroed out.
                    index = secondToIndex[second]
                    heapq.heappush(minHeap, nums[index])
                    if marks == 0:
                        # Running out of marks, so need to pop out the minimum number.
                        # So, the current second will be used to mark an index.
                        heapq.heappop(minHeap)
                        marks += 1
                    else:
                        # There're enough marks.
                        # So, the current second will be used to zero out a number.
                        marks -= 1
                else:
                    # There's no candidate to be zeroed out.
                    # So, the current second will be used to mark an index.
                    marks += 1

            decrementAndMarkCost = ((numsSum - sum(minHeap)) +
                                    (len(nums) - len(minHeap)))
            zeroAndMarkCost = len(minHeap) + len(minHeap)
            return decrementAndMarkCost + zeroAndMarkCost <= maxSecond

        l = 0
        r = len(changeIndices) + 1
        ans = bisect.bisect_left(range(l, r), True, key=canMark) + 1
        return ans if ans <= len(changeIndices) else -1

    def _getSecondToIndex(
        self,
        nums: list[int],
        changeIndices: list[int],
    ) -> dict[int, int]:
        # {the `index` of nums: the earliest second to zero out nums[index]}
        indexToFirstSecond = {}
        for zeroIndexedSecond, oneIndexedIndex in enumerate(changeIndices):
            index = oneIndexedIndex - 1 # Convert to 0-indexed.
            if nums[index] > 0 and index not in indexToFirstSecond:
                indexToFirstSecond[index] = zeroIndexedSecond
        return {second: index for index, second in indexToFirstSecond.items()}

```

Path: LeetCode-main/solutions/305. Number of Islands II/305.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = [-1] * n
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def numIslands2(
        self,
        m: int,
        n: int,
        positions: list[list[int]],
    ) -> list[int]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        ans = []
        seen = [[False] * n for _ in range(m)]
        uf = UnionFind(m * n)
        count = 0

        def getId(i: int, j: int, n: int) -> int:
            return i * n + j

        for i, j in positions:
            if seen[i][j]:
                ans.append(count)
                continue
            seen[i][j] = True
            id = getId(i, j, n)
            uf.id[id] = id
            count += 1
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                neighborId = getId(x, y, n)
                if uf.id[neighborId] == -1: # water
                    continue
                currentParent = uf.find(id)
                neighborParent = uf.find(neighborId)
                if currentParent != neighborParent:
                    uf.unionByRank(currentParent, neighborParent)
                    count -= 1
            ans.append(count)

        return ans

```

1987. 306.py

Path: LeetCode-main/solutions/306. Additive Number/306.py

```
class Solution:
    def isAdditiveNumber(self, num: str) -> bool:
        n = len(num)

        def dfs(firstNum: int, secondNum: int, s: int) -> bool:
            if s == len(num):
                return True

            thirdNum = firstNum + secondNum
            thirdNumStr = str(thirdNum)

            return (num.find(thirdNumStr, s) == s and
                    dfs(secondNum, thirdNum, s + len(thirdNumStr)))

        # num[0..i] = firstNum
        for i in range(n // 2):
            if i > 0 and num[0] == '0':
                return False
            firstNum = int(num[:i + 1])
            # num[i + 1..j] = secondNum
            # |thirdNum| >= max(|firstNum|, |secondNum|)
            j = i + 1
            while max(i, j - i) < n - j:
                if j > i + 1 and num[i + 1] == '0':
                    break
                secondNum = int(num[i + 1:j + 1])
                if dfs(firstNum, secondNum, j + 1):
                    return True
                j += 1

        return False
```

1988. 3062.py

Path: LeetCode-main/solutions/3062. Winner of the Linked List Game/3062.py

```
class Solution:
    def gameResult(self, head: ListNode | None) -> str:
        even = 0
        odd = 0

        while head:
            if head.val > head.next.val:
                even += 1
            elif head.val < head.next.val:
                odd += 1
            head = head.next.next

        if even > odd:
            return 'Even'
        if even < odd:
            return 'Odd'
        return 'Tie'
```

1989. 3063.py

Path: LeetCode-main/solutions/3063. Linked List Frequency/3063.py

```
class Solution:
    def frequenciesOfElements(self, head: ListNode | None) -> ListNode | None:
        count = collections.Counter()
        curr = head

        while curr:
            count[curr.val] += 1
            curr = curr.next

        dummy = ListNode(0)
        tail = dummy

        for freq in count.values():
            tail.next = ListNode(freq)
            tail = tail.next

        return dummy.next
```

1990. 3064.py

Path: LeetCode-main/solutions/3064. Guess the Number Using Bitwise Questions I/3064.py

```
# Definition of commonSetBits API.
# def commonSetBits(num: int) -> int:

class Solution:
    def findNumber(self) -> int:
        return sum(1 << i for i in range(31)
                    if commonSetBits(1 << i) == 1)
```

1991. 3065.py

Path: LeetCode-main/solutions/3065. Minimum Operations to Exceed Threshold Value I/3065.py

```
class Solution:
    def minOperations(self, nums: list[int], k: int) -> int:
        return sum(num < k for num in nums)
```

1992. 3066.py

Path: LeetCode-main/solutions/3066. Minimum Operations to Exceed Threshold Value II/3066.py

```

class Solution:
    def minOperations(self, nums: list[int], k: int) -> int:
        ans = 0
        minHeap = nums.copy()
        heapq.heapify(minHeap)

        while len(minHeap) > 1 and minHeap[0] < k:
            x = heapq.heappop(minHeap)
            y = heapq.heappop(minHeap)
            heapq.heappush(minHeap, min(x, y) * 2 + max(x, y))
            ans += 1

        return ans

```

1993. 3067.py

Path: LeetCode-main/solutions/3067. Count Pairs of Connectable Servers in a Weighted Tree Network/3067.py

```

class Solution:
    def countPairsOfConnectableServers(
        self,
        edges: list[list[int]],
        signalSpeed: int,
    ) -> list[int]:
        n = len(edges) + 1
        tree = [[] for _ in range(n)]

        for u, v, w in edges:
            tree[u].append((v, w))
            tree[v].append((u, w))

        def connectablePairsRootedAt(u: int) -> int:
            pairs = 0
            count = 0
            for v, w in tree[u]:
                childCount = dfs(v, u, w)
                pairs += count * childCount
                count += childCount
            return pairs

        def dfs(u: int, prev: int, dist: int) -> int:
            return (int(dist % signalSpeed == 0) +
                    sum(dfs(v, u, dist + w)
                        for v, w in tree[u]
                        if v != prev))

        return [connectablePairsRootedAt(i) for i in range(n)]

```

1994. 3068.py

Path: LeetCode-main/solutions/3068. Find the Maximum Sum of Node Values/3068.py


```
class Solution:
    def maximumValueSum(
        self,
        nums: list[int],
        k: int,
        edges: list[list[int]],
    ) -> int:
        maxSum = sum(max(num, num ^ k) for num in nums)
        changedCount = sum((num ^ k) > num for num in nums)
        if changedCount % 2 == 0:
            return maxSum
        minChangeDiff = min(abs(num - (num ^ k)) for num in nums)
        return maxSum - minChangeDiff
```

1995. 3069.py

Path: LeetCode-main/solutions/3069. Distribute Elements Into Two Arrays I/3069.py

```
class Solution:
    def resultArray(self, nums: list[int]) -> list[int]:
        arr1 = [nums[0]]
        arr2 = [nums[1]]

        for i in range(2, len(nums)):
            if arr1[-1] > arr2[-1]:
                arr1.append(nums[i])
            else:
                arr2.append(nums[i])

        return arr1 + arr2
```

1996. 307.py

Path: LeetCode-main/solutions/307. Range Sum Query - Mutable/307.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class NumArray:
    def __init__(self, nums: list[int]):
        self.nums = nums
        self.tree = FenwickTree(len(nums))
        for i, num in enumerate(nums):
            self.tree.add(i + 1, num)

    def update(self, index: int, val: int) -> None:
        self.tree.add(index + 1, val - self.nums[index])
        self.nums[index] = val

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.get(right + 1) - self.tree.get(left)

```

1997. 3070.py

Path: LeetCode-main/solutions/3070. Count Submatrices with Top-Left Element and Sum Less Than k/3070.py

```

class Solution:
    def countSubmatrices(self, grid: list[list[int]], k: int) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = 0
        # prefix[i][j] := the sum of matrix[0..i][0..j]
        prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                prefix[i + 1][j + 1] = (grid[i][j] + prefix[i][j + 1] +
                                         prefix[i + 1][j] - prefix[i][j])
                if prefix[i + 1][j + 1] <= k:
                    ans += 1

        return ans

```

1998. 3071.py

Path: LeetCode-main/solutions/3071. Minimum Operations to Write the Letter Y on a Grid/3071.py

```
class Solution:
    def minimumOperationsToWriteY(self, grid: list[list[int]]) -> int:
        n = len(grid)
        mid = n // 2

        def getOperations(a: int, b: int) -> int:
            """Returns the number of operations to turn Y into a and non-Y into b."""
            operations = 0
            for i, row in enumerate(grid):
                for j, num in enumerate(row):
                    # For the 'Y' pattern, before the midpoint, check the diagonal and
                    # anti-diagonal. After the midpoint, check the middle column.
                    if (i < mid and (i == j or i + j == n - 1)) or i >= mid and j == mid:
                        if num != a:
                            operations += 1
                    elif num != b:
                        operations += 1
            return operations

        return min(getOperations(0, 1), getOperations(0, 2),
                   getOperations(1, 0), getOperations(1, 2),
                   getOperations(2, 0), getOperations(2, 1))
```

1999. 3072.py

Path: LeetCode-main/solutions/3072. Distribute Elements Into Two Arrays II/3072.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    def resultArray(self, nums: list[int]) -> list[int]:
        arr1 = []
        arr2 = []
        ranks = self._getRanks(nums)
        tree1 = FenwickTree(len(ranks))
        tree2 = FenwickTree(len(ranks))

        def add(num: int, arr: list[int], tree: FenwickTree) -> None:
            arr.append(num)
            tree.add(ranks[num], 1)

        add(nums[0], arr1, tree1)
        add(nums[1], arr2, tree2)

        for i in range(2, len(nums)):
            greaterCount1 = len(arr1) - tree1.get(ranks[nums[i]])
            greaterCount2 = len(arr2) - tree2.get(ranks[nums[i]])
            if greaterCount1 > greaterCount2:
                add(nums[i], arr1, tree1)
            elif greaterCount1 < greaterCount2:
                add(nums[i], arr2, tree2)
            elif len(arr1) > len(arr2):
                add(nums[i], arr2, tree2)
            else:
                add(nums[i], arr1, tree1)

        return arr1 + arr2

    def _getRanks(self, nums: list[int]) -> dict[int, int]:
        ranks = collections.Counter()
        rank = 0
        for num in sorted(set(nums)):
            rank += 1
            ranks[num] = rank
        return ranks

```

2000. 3073.py

Path: LeetCode-main/solutions/3073. Maximum Increasing Triplet Value/3073.py

```

from sortedcontainers import SortedSet

class Solution:
    def maximumTripletValue(self, nums: list[int]) -> int:
        ans = 0
        rightMax = [0] * len(nums) # rightMax[i] := max(nums[i + 1..n])
        leftSortedSet = SortedSet([nums[0]])

        for i in range(len(nums) - 2, -1, -1):
            rightMax[i] = max(nums[i + 1], rightMax[i + 1])

        for j in range(1, len(nums) - 1):
            i = bisect.bisect_left(leftSortedSet, nums[j])
            if i > 0 and rightMax[j] > nums[j]:
                ans = max(ans, leftSortedSet[i - 1] - nums[j] + rightMax[j])
            leftSortedSet.add(nums[j])

        return ans

```

2001. 3074.py

Path: LeetCode-main/solutions/3074. Apple Redistribution into Boxes/3074.py

```

class Solution:
    def minimumBoxes(self, apple: list[int], capacity: list[int]) -> int:
        appleSum = sum(apple)
        capacitySum = 0

        for i, c in enumerate(sorted(capacity, reverse=True)):
            capacitySum += c
            if capacitySum >= appleSum:
                return i + 1

        return len(capacity)

```

2002. 3075.py

Path: LeetCode-main/solutions/3075. Maximize Happiness of Selected Children/3075.py

```

class Solution:
    def maximumHappinessSum(self, happiness: list[int], k: int) -> int:
        ans = 0
        decremented = 0

        happiness.sort(reverse=True)

        for i in range(k):
            ans += max(0, happiness[i] - decremented)
            decremented += 1

        return ans

```

2003. 3076.py

Path: LeetCode-main/solutions/3076. Shortest Uncommon Substring in an Array/3076.py

```
class Solution:
    def shortestSubstrings(self, arr: list[str]) -> list[str]:
        ans = []
        count = collections.Counter()

        def getSubstrings(s: str) -> Iterator[str]:
            for i in range(len(s)):
                for j in range(i + 1, len(s) + 1):
                    yield s[i:j]

        def add(s: str) -> None:
            """Adds all substrings of s to `count`."""
            for sub in getSubstrings(s):
                count[sub] += 1

        def remove(s: str) -> None:
            """Removes all substrings of s from `count`."""
            for sub in getSubstrings(s):
                count[sub] -= 1

        def getMinSub(s: str) -> str:
            minSub = ''
            for sub in getSubstrings(s):
                if count[sub] > 0:
                    continue
                if minSub == '' or
                    len(sub) < len(minSub) or
                    len(sub) == len(minSub) and sub < minSub:
                    minSub = sub
            return minSub

        for s in arr:
            add(s)

        for s in arr:
            remove(s)
            ans.append(getMinSub(s))
            add(s)

        return ans
```

2004. 3077.py

Path: LeetCode-main/solutions/3077. Maximum Strength of K Disjoint Subarrays/3077.py

```

class Solution:
    def maximumStrength(self, nums: list[int], k: int) -> int:

        @functools.lru_cache(None)
        def dp(i: int, k: int, fresh: bool) -> int:
            """
            Returns the maximum strength of nums[i..n) with k operations left, where
            `fresh` means we're starting a new subarray.
            """
            if len(nums) - i < k:
                return -math.inf
            if k == 0:
                return 0
            if i == len(nums):
                return 0 if k == 0 else -math.inf
            # If it's not fresh, we can't skip the current number and consider it as a
            # fresh start, since the case where it's fresh is already covered by
            # `includeAndFreshStart`.
            skip = dp(i + 1, k, True) if fresh else -math.inf
            gain = (-1 if k % 2 == 0 else 1) * nums[i] * k
            includeAndContinue = dp(i + 1, k, False) + gain
            includeAndFreshStart = dp(i + 1, k - 1, True) + gain
            return max(skip, includeAndContinue, includeAndFreshStart)

        return dp(0, k, True)

```

2005. 3078.py

Path: LeetCode-main/solutions/3078. Match Alphanumerical Pattern in Matrix I/3078.py

```

class Solution:
    def findPattern(
        self,
        board: list[list[int]],
        pattern: list[str],
    ) -> list[int]:
        def isMatch(x: int, y: int) -> bool:
            digitToLetter = {}
            letterToDigit = {}
            for i, row in enumerate(pattern):
                for j, c in enumerate(row):
                    digit = board[i + x][j + y]
                    if c.isdigit():
                        if int(c) != digit:
                            return False
                    else:
                        if digitToLetter.get(digit, c) != c:
                            return False
                        if letterToDigit.get(c, digit) != digit:
                            return False
                        digitToLetter[digit] = c
                        letterToDigit[c] = digit
            return True

        for x in range(len(board) - len(pattern) + 1):
            for y in range(len(board[0]) - len(pattern[0]) + 1):
                if isMatch(x, y):
                    return [x, y]

        return [-1, -1]

```

2006. 3079.py

Path: LeetCode-main/solutions/3079. Find the Sum of Encrypted Integers/3079.py

```
class Solution:
    def sumOfEncryptedInt(self, nums: list[int]) -> int:
        def getEncrypted(num: int) -> int:
            maxDigit = 0
            base = 0
            while num > 0:
                maxDigit = max(maxDigit, num % 10)
                base = base * 10 + 1
                num //= 10
            return base * maxDigit

        return sum(getEncrypted(num) for num in nums)
```

2007. 308.py

Path: LeetCode-main/solutions/308. Range Sum Query 2D - Mutable/308.py


```

class FenwickTree:
    def __init__(self, m: int, n: int):
        self.sums = [[0] * (n + 1) for _ in range(m + 1)]

    def add(self, row: int, col: int, delta: int) -> None:
        i = row
        while i < len(self.sums):
            j = col
            while j < len(self.sums[0]):
                self.sums[i][j] += delta
                j += FenwickTree.lowbit(j)
            i += FenwickTree.lowbit(i)

    def get(self, row: int, col: int) -> int:
        summ = 0
        i = row
        while i > 0:
            j = col
            while j > 0:
                summ += self.sums[i][j]
                j -= FenwickTree.lowbit(j)
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class NumMatrix:
    def __init__(self, matrix: list[list[int]]):
        self.matrix = matrix
        self.tree = FenwickTree(len(matrix), len(matrix[0]))

        for i in range(len(matrix)):
            for j, val in enumerate(matrix[i]):
                self.tree.add(i + 1, j + 1, val)

    def update(self, row: int, col: int, val: int) -> None:
        self.tree.add(row + 1, col + 1, val - self.matrix[row][col])
        self.matrix[row][col] = val

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (self.tree.get(row2 + 1, col2 + 1) - self.tree.get(row1, col2 + 1) -
                self.tree.get(row2 + 1, col1) + self.tree.get(row1, col1))

```

2008. 3080.py

Path: LeetCode-main/solutions/3080. Mark Elements on Array by Performing Queries/3080.py

```
class Solution:
    def unmarkedSumArray(
        self,
        nums: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        ans = []
        marked = set()
        summ = sum(nums)
        minHeap = [(num, i) for i, num in enumerate(nums)]
        heapq.heapify(minHeap)

        for index, k in queries:
            if index not in marked:
                marked.add(index)
                summ -= nums[index]
            popped = 0
            while popped < k and minHeap:
                num, i = heapq.heappop(minHeap)
                if i not in marked:
                    marked.add(i)
                    summ -= num
                    popped += 1
            ans.append(summ)

        return ans
```

2009. 3081.py

Path: LeetCode-main/solutions/3081. Replace Question Marks in String to Minimize Its Value/3081.py

```

class Solution:
    def minimizeStringValue(self, s: str) -> str:
        ans = []
        count = collections.Counter(s)
        letters = []

        del count['?']

        def getMinFreqLetter(count: dict[str, int]) -> str:
            minFreqLetter = 'a'
            for c in string.ascii_lowercase:
                if count[c] < count[minFreqLetter]:
                    minFreqLetter = c
            return minFreqLetter

        for c in s:
            if c == '?':
                minFreqLetter = getMinFreqLetter(count)
                letters.append(minFreqLetter)
                count[minFreqLetter] += 1

        letters.sort()
        i = 0 # letters' index

        for c in s:
            if c == '?':
                ans.append(letters[i])
                i += 1
            else:
                ans.append(c)

        return ''.join(ans)

```

2010. 3082-2.py

Path: LeetCode-main/solutions/3082. Find the Sum of the Power of All Subsequences/3082-2.py

```

class Solution:
    def sumOfPower(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        # dp[i][j] := the number of subsequences in nums[0..i] that sums to k
        dp = [[0] * (k + 1) for _ in range(n + 1)]
        dp[0][0] = 1

        for i in range(1, n + 1):
            num = nums[i - 1]
            for j in range(k + 1):
                if j < num:
                    # 1. Exclude nums[i] in the subsequence.
                    # 2. Include nums[i] in the subsequence and skip it.
                    dp[i][j] = (dp[i - 1][j] * 2) % MOD
                else:
                    # 1. Exclude nums[i] in the subsequence.
                    # 2. Include nums[i] in the subsequence and skip it.
                    # 3. Include nums[i] in the subsequence and pick it.
                    dp[i][j] = (dp[i - 1][j] * 2 + dp[i - 1][j - num]) % MOD

        return dp[n][k]

```

2011. 3082-3.py

Path: LeetCode-main/solutions/3082. Find the Sum of the Power of All Subsequences/3082-3.py

```
class Solution:
    def sumOfPower(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        # dp[i] := the number of subsequences in nums so far that sums to k
        dp = [1] + [0] * k

        for num in nums:
            for i in range(k, -1, -1):
                if i < num:
                    dp[i] = (dp[i] * 2) % MOD
                else:
                    dp[i] = (dp[i] * 2 + dp[i - num]) % MOD

        return dp[k]
```

2012. 3082.py

Path: LeetCode-main/solutions/3082. Find the Sum of the Power of All Subsequences/3082.py

```
class Solution:
    def sumOfPower(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns the number of subsequences in nums[i..n) that sums to j."""
            if j == 0:
                # For each of the remaining number, we can either pick it or skip it.
                return pow(2, len(nums) - i, MOD)
            if i == len(nums) or j < 0:
                return 0
            # 1. Include nums[i] in the subsequence and pick it.
            # 2. Include nums[i] in the subsequence and skip it.
            # 3. Exclude nums[i] in the subsequence.
            return (dp(i + 1, j - nums[i]) + 2 * dp(i + 1, j)) % MOD

        return dp(0, k)
```

2013. 3083.py

Path: LeetCode-main/solutions/3083. Existence of a Substring in a String and Its Reverse/3083.py

```
class Solution:
    def isSubstringPresent(self, s: str) -> bool:
        return any(s[i:i + 2] in s[::-1] for i in range(len(s) - 1))
```

2014. 3084.py

Path: LeetCode-main/solutions/3084. Count Substrings Starting and Ending with Given Character/3084.py

```
class Solution:
    def countSubstrings(self, s: str, c: str) -> int:
        freq = s.count(c)
        return freq * (freq + 1) // 2
```

2015. 3085.py

Path: LeetCode-main/solutions/3085. Minimum Deletions to Make String K-Special/3085.py

```
class Solution:
    def minimumDeletions(self, word: str, k: int) -> int:
        ans = math.inf
        count = collections.Counter(word)

        for minFreq in count.values():
            deletions = 0
            for freq in count.values():
                if freq < minFreq:
                    deletions += freq
                else:
                    deletions += max(0, freq - (minFreq + k))
            ans = min(ans, deletions)

        return ans
```

2016. 3086.py

Path: LeetCode-main/solutions/3086. Minimum Moves to Pick K Ones/3086.py

```

class Solution:
    def minimumMoves(self, nums: list[int], k: int, maxChanges: int) -> int:
        # Dylan has two actions for collecting '1's in a sequence:
        # Action 1: Put a '1' next to him and pick it up.
        #           The cost is 2.
        # Action 2: Swap a '1' towards him and collect it.
        #           The cost equals the distance to the '1'.
        #
        # To minimize the swapping cost, Dylan can use a sliding window strategy,
        # selecting the optimal position (middle '1' in the window) for efficient
        # collection. The window's size is crucial:
        #
        # The minimum window size: min(0, k - maxChanges), ensuring the window
        # isn't too small.
        # The maximum window size: min(k, minOnesByTwo + 3, the number of ones),
        # preventing overly ambitious swaps.
        #
        # Note that if needing to move a '1' beyond `minOnesByTwo + 3`, it's
        # cheaper to use Action 1.
        #
        # At most three indices, (dylanIndex - 1, dylanIndex, dylanIndex + 1), have
        # a distance <= 1 from dylanIndex, implying that we'll be taking at most
        # `maxOnesByTwo + 3` using Action 2. Any more Action 2 is not optimal and
        # should be replaced with Action 1.
        NUM_OF_INDICES_WITHIN_ONE_DISTANCE = 3
        ans = math.inf
        oneIndices = [i for i, num in enumerate(nums) if num == 1]
        prefix = list(itertools.accumulate(oneIndices, initial=0))

        minOnesByTwo = max(0, k - maxChanges)
        maxOnesByTwo = min(
            k, minOnesByTwo + NUM_OF_INDICES_WITHIN_ONE_DISTANCE, len(oneIndices))

        for onesByTwo in range(minOnesByTwo, maxOnesByTwo + 1):
            for l in range(len(prefix) - onesByTwo):
                r = l + onesByTwo # Collect 1s in oneIndices[l - 1..r - 1].
                cost1 = (k - onesByTwo) * 2
                cost2 = ((prefix[r] - prefix[(l + r) // 2]) -
                        (prefix[(l + r + 1) // 2] - prefix[l]))
                ans = min(ans, cost1 + cost2)

        return ans

```

2017. 3088.py

Path: LeetCode-main/solutions/3088. Make String Anti-palindrome/3088.py

```

class Solution:
    def makeAntiPalindrome(self, s: str) -> str:
        n = len(s)
        i = n // 2
        chars = sorted(list(s))
        if chars[i] != chars[n - 1 - i]:
            return ''.join(chars)

        j = self._getFirstDiffIndexInSecondHalf(chars)
        while chars[i] == chars[n - 1 - i]:
            if j == n:
                return '-1'
            chars[i], chars[j] = chars[j], chars[i]
            i += 1
            j += 1

        return ''.join(chars)

    def _getFirstDiffIndexInSecondHalf(self, chars: list[str]) -> int:
        """
        Returns the first index in chars[n / 2..n) that is different from the first
        letter of the second half, chars[n / 2].
        """
        n = len(chars)
        firstLetter = chars[n // 2]
        firstDiffIndex = n // 2
        while firstDiffIndex < n and chars[firstDiffIndex] == firstLetter:
            firstDiffIndex += 1
        return firstDiffIndex

```

2018. 309.py

Path: LeetCode-main/solutions/309. Best Time to Buy and Sell Stock with Cooldown/309.py

```

class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sell = 0
        hold = -math.inf
        prev = 0

        for price in prices:
            cache = sell
            sell = max(sell, hold + price)
            hold = max(hold, prev - price)
            prev = cache

        return sell

```

2019. 3090.py

Path: LeetCode-main/solutions/3090. Maximum Length Substring With Two Occurrences/3090.py

```

class Solution:
    def maximumLengthSubstring(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] > 2:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

2020. 3091-2.py

Path: LeetCode-main/solutions/3091. Apply Operations to Make Sum of Array Greater Than or Equal to k/3091-2.py

```

class Solution:
    def minOperations(self, k: int) -> int:
        # The required operations are
        # 1. Increase `1` to `x`
        # 2. Duplicate `x`, `y` times, to `sum` s.t.  $x * (1 + y) \geq k$ .
        # The number of operations used would be  $(x - 1) + y$ . Equivalently, the
        # problem can be rephrased as finding  $\min(x - 1 + y)$  s.t.  $x * (1 + y) \geq k$ .
        # Optimally, `x` should equal to `1 + y`, implying that  $x^2 \geq k$ , and
        # hence,  $x \geq \sqrt{k}$  and  $y = \lceil k / x \rceil - 1$ .
        x = math.isqrt(k)
        y = (k - 1) // x + 1 - 1 #  $\lceil k / x \rceil - 1$ 
        return x - 1 + y

```

2021. 3091.py

Path: LeetCode-main/solutions/3091. Apply Operations to Make Sum of Array Greater Than or Equal to k/3091.py

```

class Solution:
    def minOperations(self, k: int) -> int:
        # The required operations are
        # 1. Increase `1` to `x`
        # 2. Duplicate `x`, `y` times, to `sum` s.t.  $x * (1 + y) \geq k$ .
        # The number of operations used would be  $(x - 1) + y$ . Equivalently, the
        # problem can be rephrased as finding  $\min(x - 1 + y)$  s.t.  $x * (1 + y) \geq k$ .
        # Optimally, `x` should equal to `1 + y`, implying that  $x^2 \geq k$ , and
        # hence,  $x \geq \sqrt{k}$  and  $y = \lceil k / x \rceil - 1$ .
        x = math.ceil(math.sqrt(k))
        y = (k - 1) // x + 1 - 1 #  $\lceil k / x \rceil - 1$ 
        return x - 1 + y

```

2022. 3092.py

Path: LeetCode-main/solutions/3092. Most Frequent IDs/3092.py


```
from sortedcontainers import SortedDict

class Solution:
    def mostFrequentIDs(self, nums: list[int], freq: list[int]) -> list[int]:
        ans = []
        numCount = collections.Counter() # {num: freq}
        freqCount = SortedDict() # {num's freq: freq}

        for num, f in zip(nums, freq):
            if numCount[num] > 0:
                numFreq = numCount[num]
                freqCount[numFreq] -= 1
                if freqCount[numFreq] == 0:
                    del freqCount[numFreq]
                newFreq = numCount[num] + f
                if newFreq == 0:
                    del numCount[num]
                else:
                    numCount[num] = newFreq
                    freqCount[newFreq] = freqCount.get(newFreq, 0) + 1
            ans.append(freqCount.peekitem(-1)[0] if freqCount else 0)

        return ans
```

2023. 3093.py

Path: LeetCode-main/solutions/3093. Longest Common Suffix Queries/3093.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False
        self.length = math.inf
        self.index = -1

class Solution:
    def stringIndices(
        self,
        wordsContainer: list[str],
        wordsQuery: list[str],
    ) -> list[int]:
        ans = []
        root = TrieNode()
        minIndex = min(enumerate(wordsContainer), key=lambda x: len(x[1]))[0]

        def insert(word: str, index: int) -> None:
            node = root
            for c in reversed(word):
                node = node.children.setdefault(c, TrieNode())
                if node.length > len(word):
                    node.length = len(word)
                node.index = index

        def search(word: str) -> int:
            node = root
            for c in reversed(word):
                if c not in node.children:
                    return node.index
                node = node.children[c]
            return node.index

        for i, word in enumerate(wordsContainer):
            insert(word, i)

        for query in wordsQuery:
            index = search(query)
            ans.append(minIndex if index == -1 else index)

        return ans

```

2024. 3094-2.py

Path: LeetCode-main/solutions/3094. Guess the Number Using Bitwise Questions II/3094-2.py

```

# Definition of commonBits API.
# def commonBits(num: int) -> int:

class Solution:
    def findNumber(self) -> int:
        return functools.reduce(lambda x, i: x | (1 << i)
                                if commonBits(1 << i) > commonBits(1 << i)
                                else x, range(31), 0)

```

2025. 3094.py

Path: LeetCode-main/solutions/3094. Guess the Number Using Bitwise Questions II/3094.py

```
# Definition of commonBits API.
# def commonBits(num: int) -> int:

class Solution:
    def findNumber(self) -> int:
        ans = 0
        sameCount = commonBits(0)

        for i in range(31):
            if commonBits(1 << i) > sameCount:
                ans |= 1 << i
                commonBits(1 << i) # Revert the XOR.

        return ans
```

2026. 3095.py

Path: LeetCode-main/solutions/3095. Shortest Subarray With OR at Least K I/3095.py

```
class Solution:
    def minimumSubarrayLength(self, nums: list[int], k: int) -> int:
        ans = len(nums) + 1
        ors = 0
        count = collections.Counter()

        l = 0
        for r, num in enumerate(nums):
            ors = self._orNum(ors, num, count)
            while ors >= k and l <= r:
                ans = min(ans, r - l + 1)
                ors = self._undoOrNum(ors, nums[l], count)
                l += 1

        return -1 if ans == len(nums) + 1 else ans

    def _orNum(self, ors: int, num: int, count: dict[int, int]) -> int:
        for i in range(30):
            if num >> i & 1:
                count[i] += 1
                if count[i] == 1:
                    ors += 1 << i
        return ors

    def _undoOrNum(self, ors: int, num: int, count: dict[int, int]) -> int:
        for i in range(30):
            if num >> i & 1:
                count[i] -= 1
                if count[i] == 0:
                    ors -= 1 << i
        return ors
```

2027. 3096.py

Path: LeetCode-main/solutions/3096. Minimum Levels to Gain More Points/3096.py

```

class Solution:
    def minimumLevels(self, possible: list[int]) -> int:
        n = len(possible)
        nums = [num if num == 1 else -1 for num in possible]
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(1, n):
            if prefix[i] > prefix[n] - prefix[i]:
                return i

        return -1

```

2028. 3097.py

Path: LeetCode-main/solutions/3097. Shortest Subarray With OR at Least K II/3097.py

```

class Solution:
    # Same as 3095. Shortest Subarray With OR at Least K I
    def minimumSubarrayLength(self, nums: list[int], k: int) -> int:
        ans = len(nums) + 1
        ors = 0
        count = collections.Counter()

        l = 0
        for r, num in enumerate(nums):
            ors = self._orNum(ors, num, count)
            while ors >= k and l <= r:
                ans = min(ans, r - l + 1)
                ors = self._undoOrNum(ors, nums[l], count)
                l += 1

        return -1 if ans == len(nums) + 1 else ans

    def _orNum(self, ors: int, num: int, count: dict[int, int]) -> int:
        for i in range(30):
            if num >> i & 1:
                count[i] += 1
            if count[i] == 1:
                ors += 1 << i
        return ors

    def _undoOrNum(self, ors: int, num: int, count: dict[int, int]) -> int:
        for i in range(30):
            if num >> i & 1:
                count[i] -= 1
            if count[i] == 0:
                ors -= 1 << i
        return ors

```

2029. 3098-2.py

Path: LeetCode-main/solutions/3098. Find the Sum of Subsequence Powers/3098-2.py

```

class Solution:
    def sumOfPowers(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        n = len(nums)

        nums.sort()

        @functools.lru_cache(None)
        def dp(i: int, k: int, lastPickIndex: int, minDiff: int) -> int:
            if k == 0:
                return minDiff
            if i == n:
                return 0
            newMinDiff = (minDiff if lastPickIndex == - 1
                          else min(minDiff, nums[i] - nums[lastPickIndex]))
            pick = dp(i + 1, k - 1, i, newMinDiff)
            skip = dp(i + 1, k, lastPickIndex, minDiff)
            return (pick + skip) % MOD

        return dp(0, k, -1, math.inf)

```

2030. 3098.py

Path: LeetCode-main/solutions/3098. Find the Sum of Subsequence Powers/3098.py

```

class Solution:
    def sumOfPowers(self, nums: list[int], k: int) -> int:
        MOD = 1_000_000_007
        nums.sort()

        @functools.lru_cache(None)
        def dp(
            i: int,
            k: int,
            lastPickedIndex: int,
            firstIndex: int,
            secondIndex: int
        ) -> int:
            if k == 0:
                return nums[secondIndex] - nums[firstIndex]
            if i == len(nums):
                return 0
            newFirstIndex = firstIndex
            newSecondIndex = secondIndex
            if firstIndex == -1:
                newFirstIndex = i
            elif secondIndex == -1:
                newSecondIndex = i
            elif nums[i] - nums[lastPickedIndex] < nums[secondIndex] - nums[firstIndex]:
                newFirstIndex = lastPickedIndex
                newSecondIndex = i
            pick = dp(i + 1, k - 1, i, newFirstIndex, newSecondIndex)
            skip = dp(i + 1, k, lastPickedIndex, firstIndex, secondIndex)
            return (pick + skip) % MOD

        return dp(0, k, -1, -1, -1)

```

2031. 3099.py

Path: LeetCode-main/solutions/3099. Harshad Number/3099.py

```
class Solution:
    def sumOfTheDigitsOfHarshadNumber(self, x: int) -> int:
        digitSum = self._getDigitSum(x)
        return digitSum if x % digitSum == 0 else -1

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))
```

2032. 31.py

Path: LeetCode-main/solutions/31. Next Permutation/31.py

```
class Solution:
    def nextPermutation(self, nums: list[int]) -> None:
        n = len(nums)

        # From back to front, find the first number < nums[i + 1].
        i = n - 2
        while i >= 0:
            if nums[i] < nums[i + 1]:
                break
            i -= 1

        # From back to front, find the first number > nums[i], swap it with nums[i].
        if i >= 0:
            for j in range(n - 1, i, -1):
                if nums[j] > nums[i]:
                    nums[i], nums[j] = nums[j], nums[i]
                    break

        def reverse(nums: list[int], l: int, r: int) -> None:
            while l < r:
                nums[l], nums[r] = nums[r], nums[l]
                l += 1
                r -= 1

        # Reverse nums[i + 1..n - 1].
        reverse(nums, i + 1, len(nums) - 1)
```

2033. 310.py

Path: LeetCode-main/solutions/310. Minimum Height Trees/310.py

```

class Solution:
    def findMinHeightTrees(self, n: int, edges: list[list[int]]) -> list[int]:
        if n == 1 or not edges:
            return [0]

        ans = []
        graph = collections.defaultdict(set)

        for u, v in edges:
            graph[u].add(v)
            graph[v].add(u)

        for label, children in graph.items():
            if len(children) == 1:
                ans.append(label)

        while n > 2:
            n -= len(ans)
            nextLeaves = []
            for leaf in ans:
                u = next(iter(graph[leaf]))
                graph[u].remove(leaf)
                if len(graph[u]) == 1:
                    nextLeaves.append(u)
            ans = nextLeaves

        return ans

```

2034. 3100.py

Path: LeetCode-main/solutions/3100. Water Bottles II/3100.py

```

class Solution:
    def maxBottlesDrunk(self, numBottles: int, numExchange: int) -> int:
        ans = numBottles

        while numBottles >= numExchange:
            numBottles = numBottles - numExchange + 1
            numExchange += 1
            ans += 1

        return ans

```

2035. 3101.py

Path: LeetCode-main/solutions/3101. Count Alternating Subarrays/3101.py

```

class Solution:
    def countAlternatingSubarrays(self, nums: list[int]) -> int:
        # dp[i] := the number of alternating subarrays ending in index i
        dp = [1] * len(nums)

        for i in range(1, len(nums)):
            if nums[i] != nums[i - 1]:
                dp[i] += dp[i - 1]

        return sum(dp)

```

2036. 3102.py

Path: LeetCode-main/solutions/3102. Minimize Manhattan Distances/3102.py

```
class Solution:
    def minimumDistance(self, points: list[list[int]]) -> int:
        i, j = self._maxManhattanDistance(points, -1)
        xi, yi = self._maxManhattanDistance(points, i)
        xj, yj = self._maxManhattanDistance(points, j)
        return min(self._manhattan(points, xi, yi),
                   self._manhattan(points, xj, yj))

    def _maxManhattanDistance(
        self,
        points: list[list[int]],
        excludedIndex: int,
    ) -> int:
        minSum = math.inf
        maxSum = -math.inf
        minDiff = math.inf
        maxDiff = -math.inf
        minSumIndex = -1
        maxSumIndex = -1
        minDiffIndex = -1
        maxDiffIndex = -1

        for i, (x, y) in enumerate(points):
            if i == excludedIndex:
                continue
            summ = x + y
            diff = x - y
            if summ < minSum:
                minSum = summ
                minSumIndex = i
            if summ > maxSum:
                maxSum = summ
                maxSumIndex = i
            if diff < minDiff:
                minDiff = diff
                minDiffIndex = i
            if diff > maxDiff:
                maxDiff = diff
                maxDiffIndex = i

        return ([minSumIndex, maxSumIndex] if maxSum - minSum >= maxDiff - minDiff
                else [minDiffIndex, maxDiffIndex])

    def _manhattan(self, points: list[list[int]], i: int, j: int) -> int:
        return abs(points[i][0] - points[j][0]) + abs(points[i][1] - points[j][1])
```

2037. 3104.py

Path: LeetCode-main/solutions/3104. Find Longest Self-Contained Substring/3104.py


```

class Solution:
    def maxSubStringLength(self, s: str) -> int:
        allCount = collections.Counter(s)

        # Similar to 395. Longest Substring with At Least K Repeating Characters
        def maxSubStringLengthWithNUniqueLetters(n: int) -> int:
            res = -1
            # the number of unique letters
            uniqueLetters = 0
            # the number of letters that have all their frequency in the substring
            lettersHavingAllFreq = 0
            count = collections.Counter()

            l = 0
            for r, c in enumerate(s):
                count[c] += 1
                if count[c] == 1:
                    uniqueLetters += 1
                if count[c] == allCount[c]:
                    lettersHavingAllFreq += 1
                while uniqueLetters > n:
                    if count[s[l]] == allCount[s[l]]:
                        lettersHavingAllFreq -= 1
                    count[s[l]] -= 1
                    if count[s[l]] == 0:
                        uniqueLetters -= 1
                    l += 1
                # Since both the number of unique letters and the number of letters
                # having all their frequency are equal to n, this is a valid window.
                # Implicit: uniqueLetters == n
                if lettersHavingAllFreq == n and r - l + 1 < len(s):
                    res = max(res, r - l + 1)

            return res

        return max(maxSubStringLengthWithNUniqueLetters(n)
                   for n in range(1, 27))

```

2038. 3105.py

Path: LeetCode-main/solutions/3105. Longest Strictly Increasing or Strictly Decreasing Subarray/3105.py

```

class Solution:
    # Similar to 978. Longest Turbulent Subarray
    def longestMonotonicSubarray(self, nums: list[int]) -> int:
        ans = 1
        increasing = 1
        decreasing = 1

        for i in range(1, len(nums)):
            if nums[i] > nums[i - 1]:
                increasing += 1
                decreasing = 1
            elif nums[i] < nums[i - 1]:
                decreasing += 1
                increasing = 1
            else:
                increasing = 1
                decreasing = 1
            ans = max(ans, increasing, decreasing)

        return ans

```

2039. 3106.py

Path: LeetCode-main/solutions/3106. Lexicographically Smallest String After Operations With Constraint/3106.py

```

class Solution:
    def getSmallestString(self, s: str, k: int) -> str:
        ans = list(s)

        for i, c in enumerate(s):
            if k == 0:
                break
            distToA = min(ord(c) - ord('a'), ord('z') - ord(c) + 1)
            if k >= distToA:
                k -= distToA
                ans[i] = 'a'
            else:
                # k is not enough to change the current letter to 'a', so move as closer
                # to 'a' as possible.
                ans[i] = chr(ord(c) - k)
                k = 0

        return ''.join(ans)

```

2040. 3107.py

Path: LeetCode-main/solutions/3107. Minimum Operations to Make Median of Array Equal to K/3107.py

```
class Solution:
    def minOperationsToMakeMedianK(self, nums: list[int], k: int) -> int:
        n = len(nums)
        ans = 0

        nums.sort()

        for i in range(n // 2 + 1):
            ans += max(0, nums[i] - k)

        for i in range(n // 2, n):
            ans += max(0, k - nums[i])

        return ans
```

2041. 3108.py

Path: LeetCode-main/solutions/3108. Minimum Cost Walk in Weighted Graph/3108.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n
        # 2^17 - 1 is the minimum number in the form 2^x - 1 > 10^5.
        self.weight = [(1 << 17) - 1] * n

    def unionByRank(self, u: int, v: int, w: int) -> None:
        i = self._find(u)
        j = self._find(v)
        newWeight = self.weight[i] & self.weight[j] & w
        self.weight[i] = newWeight
        self.weight[j] = newWeight
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def getMinCost(self, u: int, v: int) -> int:
        if u == v:
            return 0
        i = self._find(u)
        j = self._find(v)
        return self.weight[i] if i == j else -1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

```

```

class Solution:
    def minimumCost(
        self,
        n: int,
        edges: list[list[int]],
        query: list[list[int]],
    ) -> list[int]:
        uf = UnionFind(n)

        for u, v, w in edges:
            uf.unionByRank(u, v, w)

        return [uf.getMinCost(u, v) for u, v in query]

```

2042. 3109.py

Path: LeetCode-main/solutions/3109. Find the Index of Permutation/3109.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    def getPermutationIndex(self, perm: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(perm)
        ans = 0
        tree = FenwickTree(n)
        fact = [1] * (n + 1) # fact[i] := i!

        for i in range(2, n + 1):
            fact[i] = (fact[i - 1] * i) % MOD

        for i, num in enumerate(perm):
            # the number of unused numbers less than `num`
            unusedNums = num - 1 - tree.get(num - 1)
            suffixLength = fact[n - 1 - i]
            ans += unusedNums * suffixLength
            ans %= MOD
            tree.add(num, 1)

        return ans

```

2043. 311-2.py

Path: LeetCode-main/solutions/311. Sparse Matrix Multiplication/311-2.py

```

class Solution:
    def multiply(self, mat1: list[list[int]],
                 mat2: list[list[int]]) -> list[list[int]]:
        m = len(mat1)
        n = len(mat2)
        l = len(mat2[0])
        ans = [[0] * l for _ in range(m)]
        nonZeroColIndicesInMat2 = [
            [j for j, a in enumerate(row) if a]
            for row in mat2
        ]

        for i in range(m):
            for j, a in enumerate(mat1[i]):
                if a == 0:
                    continue
                # mat1's j-th column matches mat2's j-th row
                for colIndex in nonZeroColIndicesInMat2[j]:
                    ans[i][colIndex] += a * mat2[j][colIndex]

        return ans

```

2044. 311.py

Path: LeetCode-main/solutions/311. Sparse Matrix Multiplication/311.py

```

class Solution:
    def multiply(self, mat1: list[list[int]],
                 mat2: list[list[int]]) -> list[list[int]]:
        m = len(mat1)
        n = len(mat2)
        l = len(mat2[0])
        ans = [[0] * l for _ in range(m)]

        for i in range(m):
            for j in range(l):
                for k in range(n):
                    ans[i][j] += mat1[i][k] * mat2[k][j]

        return ans

```

2045. 3110.py

Path: LeetCode-main/solutions/3110. Score of a String/3110.py

```

class Solution:
    def scoreOfString(self, s: str) -> int:
        return sum(abs(ord(a) - ord(b))
                    for a, b in itertools.pairwise(s))

```

2046. 3111.py

Path: LeetCode-main/solutions/3111. Minimum Rectangles to Cover Points/3111.py

```

class Solution:
    def minRectanglesToCoverPoints(self, points: list[list[int]], w: int) -> int:
        ans = 0
        prevX = -w - 1
        xs = sorted([x for x, _ in points])

        for x in xs:
            if x > prevX + w:
                ans += 1
                prevX = x

        return ans

```

2047. 3112.py

Path: LeetCode-main/solutions/3112. Minimum Time to Visit Disappearing Nodes/3112.py

```

class Solution:
    def minimumTime(
        self,
        n: int,
        edges: list[list[int]],
        disappear: list[int],
    ) -> list[int]:
        graph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        return self._dijkstra(graph, 0, disappear)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        disappear: list[int],
    ) -> list[int]:
        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < disappear[v] and d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        return [d if d != math.inf else -1
                for d in dist]

```

2048. 3113.py

Path: LeetCode-main/solutions/3113. Find the Number of Subarrays Where Boundary Elements Are Maximum/3113.py

```
class Solution:
    def numberOfSubarrays(self, nums: list[int]) -> int:
        ans = 0
        stack = []

        for num in nums:
            while stack and stack[-1][0] < num:
                stack.pop()
            if not stack or stack[-1][0] != num:
                stack.append([num, 0])
            stack[-1][1] += 1
            ans += stack[-1][1]

        return ans
```

2049. 3114.py

Path: LeetCode-main/solutions/3114. Latest Time You Can Obtain After Replacing Characters/3114.py

```
class Solution:
    # Similar to 1736. Latest Time by Replacing Hidden Digits
    def findLatestTime(self, s: str) -> str:
        ans = list(s)
        if s[0] == '?':
            ans[0] = '1' if s[1] == '?' or s[1] < '2' else '0'
        if s[1] == '?':
            ans[1] = '1' if ans[0] == '1' else '9'
        if s[3] == '?':
            ans[3] = '5'
        if s[4] == '?':
            ans[4] = '9'
        return ''.join(ans)
```

2050. 3115.py

Path: LeetCode-main/solutions/3115. Maximum Prime Difference/3115.py


```

class Solution:
    def maximumPrimeDifference(self, nums: list[int]) -> int:
        MAX = 100
        isPrime = self._sieveEratosthenes(MAX + 1)
        minPrimeIndex = -1
        maxPrimeIndex = -1

        for i, num in enumerate(nums):
            if isPrime[num]:
                if minPrimeIndex == -1:
                    minPrimeIndex = i
                maxPrimeIndex = i

        return maxPrimeIndex - minPrimeIndex

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime

```

2051. 3116.py

Path: LeetCode-main/solutions/3116. Kth Smallest Amount With Single Denomination Combination/3116.py

```

class Solution:
    def findKthSmallest(self, coins: list[int], k: int) -> int:
        sizeToLcms = self._getSizeToLcms(coins)

        def count(m: int) -> int:
            """Returns the number of denominations <= m."""
            res = 0
            for sz, lcms in enumerate(sizeToLcms):
                for lcm in lcms:
                    # Principle of Inclusion-Exclusion (PIE)
                    res += m // lcm * pow(-1, sz + 1)
            return res

        return bisect.bisect_left(range(k * min(coins)), k, key=count)

    def _getSizeToLcms(self, coins: list[int]) -> list[list[int]]:
        # Returns the LCMs for each number of combination of coins.
        sizeToLcms = [[] for _ in range(len(coins) + 1)]
        for sz in range(1, len(coins) + 1):
            for combination in itertools.combinations(coins, sz):
                sizeToLcms[sz].append(math.lcm(*combination))
        return sizeToLcms

```

2052. 3117.py

Path: LeetCode-main/solutions/3117. Minimum Sum of Values by Dividing Array/3117.py

```

class Solution:
    def minimumValueSum(self, nums: list[int], andValues: list[int]) -> int:
        n = len(nums)
        m = len(andValues)

        @functools.lru_cache(None)
        def dp(i: int, j: int, mask: int) -> int:
            """
            Returns the minimum value sum of nums[i..n) and andValues[j..m), where
            `mask` is the running value of the current subarray.
            """
            if i == n and j == m:
                return 0
            if i == n or j == m:
                return math.inf
            mask &= nums[i]
            if mask < andValues[j]:
                return math.inf
            if mask == andValues[j]:
                # 1. Keep going.
                # 2. End the subarray here and pick nums[i], then fresh start.
                return min(dp(i + 1, j, mask),
                           nums[i] + dp(i + 1, j + 1, -1))
            return dp(i + 1, j, mask) # Keep going.

        ans = dp(0, 0, -1)
        return ans if ans < math.inf else -1

```

2053. 3119.py

Path: LeetCode-main/solutions/3119. Maximum Number of Potholes That Can Be Fixed/3119.py

```

class Solution:
    def maxPotholes(self, road: str, budget: int) -> int:
        ans = 0

        for length in sorted(map(len, road.split('.')), reverse=True):
            canRepair = max(0, budget - 1)
            if length > canRepair:
                return ans + canRepair
            ans += length
            budget -= length + 1

        return ans

```

2054. 312-2.py

Path: LeetCode-main/solutions/312. Burst Balloons/312-2.py

```
class Solution:
    def maxCoins(self, nums: list[int]) -> int:
        n = len(nums)
        # dp[i][j] := maxCoins(nums[i..j])
        dp = [[0] * (n + 2) for _ in range(n + 2)]

        nums = [1] + nums + [1]

        for d in range(n):
            for i in range(1, n - d + 1):
                j = i + d
                for k in range(i, j + 1):
                    dp[i][j] = max(
                        dp[i][j],
                        dp[i][k - 1] +
                        dp[k + 1][j] +
                        nums[i - 1] * nums[k] * nums[j + 1])

        return dp[1][n]
```

Path: LeetCode-main/solutions/312. Burst Balloons/312.py

Path: LeetCode-main/solutions/3120. Count the Number of Special Characters I/3120.py

2057. 3121.py

Path: LeetCode-main/solutions/3121. Count the Number of Special Characters II/3121.py

```
class Solution:
    def numberOfSpecialChars(self, word: str) -> int:
        lower = collections.defaultdict(bool)
        upper = collections.defaultdict(bool)

        for c in word:
            if c.islower():
                lower[c] = not upper[c.upper()]
            else:
                upper[c] = True

        return sum(lower[a] and upper[b]
                    for a, b in zip(string.ascii_lowercase,
                                    string.ascii_uppercase))
```

2058. 3122.py

Path: LeetCode-main/solutions/3122. Minimum Number of Operations to Satisfy Conditions/3122.py

```
class Solution:
    def minimumOperations(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        count = [[0] * 10 for _ in range(n)]

        for row in grid:
            for j, num in enumerate(row):
                count[j][num] += 1

        @functools.lru_cache(None)
        def dp(i: int, prev: int) -> int:
            """
            Returns the number of minimum operations needed to make grid[:,j..n)
            satisfy the conditions, where the (j - 1)-th column is filled with `prev`.
            """
            if i == n:
                return 0

            res = math.inf

            for num in range(10):
                if i == 0 or num != prev:
                    res = min(res, m - count[i][num] + dp(i + 1, num))

            return res

        return dp(0, 0)
```

2059. 3123.py

Path: LeetCode-main/solutions/3123. Find Edges in Shortest Paths/3123.py

```

class Solution:
    # Similar to 2203. Minimum Weighted Subgraph With the Required Paths
    def findAnswer(self, n: int, edges: list[list[int]]) -> list[bool]:
        graph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        from0 = self._dijkstra(graph, 0)
        from1 = self._dijkstra(graph, n - 1)
        return [from0[u] + w + from1[v] == from0[-1] or
                from0[v] + w + from1[u] == from0[-1]
                for u, v, w in edges]

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
    ) -> list[int]:
        dist = [10**9] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        return dist

```

2060. 3125.py

Path: LeetCode-main/solutions/3125. Maximum Number That Makes Result of Bitwise AND Zero/3125.py

```

class Solution:
    def maxNumber(self, n: int) -> int:
        # assume n = 0b00...11???
        #       x = 0b00...01111
        # since y = 0b00...10000 is in [x, n]
        #   and x & y = 0
        return (1 << n.bit_length() - 1) - 1

```

2061. 3127.py

Path: LeetCode-main/solutions/3127. Make a Square with the Same Color/3127.py

```
class Solution:
    def canMakeSquare(self, grid: list[list[str]]) -> bool:
        for i in range(2):
            for j in range(2):
                black = 0
                white = 0
                for x in range(2):
                    for y in range(2):
                        if grid[i + x][j + y] == 'B':
                            black += 1
                        else:
                            white += 1
                if black >= 3 or white >= 3:
                    return True
        return False
```

2062. 3128.py

Path: LeetCode-main/solutions/3128. Right Triangles/3128.py

```
class Solution:
    def numberOfRightTriangles(self, grid: list[list[int]]) -> int:
        rows = [0] * len(grid)
        cols = [0] * len(grid[0])

        for i, row in enumerate(grid):
            for j, num in enumerate(row):
                if num == 1:
                    rows[i] += 1
                    cols[j] += 1

        return sum((rows[i] - 1) * (cols[j] - 1)
                    for i, row in enumerate(grid)
                    for j, num in enumerate(row)
                    if num == 1)
```

2063. 3129.py

Path: LeetCode-main/solutions/3129. Find All Possible Stable Binary Arrays I/3129.py

```

class Solution:
    # Same as 3129. Find All Possible Stable Binary Arrays I
    def numberOfStableArrays(self, zero: int, one: int, limit: int) -> int:
        MOD = 1_000_000_007
        # dp[i][j][k] := the number of stable arrays, where the number of
        # occurrences of 0 is i and the number of occurrences of 1 is j and the last
        # number is k (0/1)
        dp = [[[0] * 2
                for _ in range(one + 1)]
               for _ in range(zero + 1)]

        for i in range(min(zero, limit) + 1):
            dp[i][0][0] = 1

        for j in range(min(one, limit) + 1):
            dp[0][j][1] = 1

        for i in range(1, zero + 1):
            for j in range(1, one + 1):
                dp[i][j][0] = (
                    dp[i - 1][j][0] + dp[i - 1][j][1] -
                    (dp[i - limit - 1][j][1] if i - limit >= 1 else 0) + MOD) % MOD
                dp[i][j][1] = (
                    dp[i][j - 1][0] + dp[i][j - 1][1] -
                    (dp[i][j - limit - 1][0] if j - limit >= 1 else 0) + MOD) % MOD

        return (dp[zero][one][0] + dp[zero][one][1]) % MOD

```

2064. 313-2.py

Path: LeetCode-main/solutions/313. Super Ugly Number/313-2.py

```

class UglyNum:
    def __init__(self, prime: int, index: int, value: int):
        self.prime = prime
        self.index = index # Point the next index of uglyNums.
        self.value = value # prime * uglyNums[index]

class Solution:
    def nthSuperUglyNumber(self, n: int, primes: list[int]) -> int:
        minHeap = [] # (value, prime, index)
        uglyNums = [1]

        for prime in primes:
            heapq.heappush(minHeap, (prime * uglyNums[0], prime, 1))

        while len(uglyNums) < n:
            uglyNums.append(minHeap[0][0])
            while minHeap[0][0] == uglyNums[-1]:
                _, prime, index = heapq.heappop(minHeap)
                heapq.heappush(minHeap, (prime * uglyNums[index], prime, index + 1))

        return uglyNums[-1]

```

2065. 313.py

Path: LeetCode-main/solutions/313. Super Ugly Number/313.py

```

class Solution:
    def nthSuperUglyNumber(self, n: int, primes: list[int]) -> int:
        k = len(primes)
        nums = [1]
        indices = [0] * k

        while len(nums) < n:
            nexts = [0] * k
            for i in range(k):
                nexts[i] = nums[indices[i]] * primes[i]
            next = min(nexts)
            for i in range(k):
                if next == nexts[i]:
                    indices[i] += 1
            nums.append(next)

        return nums[-1]

```

2066. 3130.py

Path: LeetCode-main/solutions/3130. Find All Possible Stable Binary Arrays II/3130.py

```

class Solution:
    # Same as 3129. Find All Possible Stable Binary Arrays I
    def numberOfStableArrays(self, zero: int, one: int, limit: int) -> int:
        MOD = 1_000_000_007
        # dp[i][j][k] := the number of stable arrays, where the number of
        # occurrences of 0 is i and the number of occurrences of 1 is j and the last
        # number is k (0/1)
        dp = [[[0] * 2
                for _ in range(one + 1)
                for _ in range(zero + 1)]

            for i in range(min(zero, limit) + 1):
                dp[i][0][0] = 1

            for j in range(min(one, limit) + 1):
                dp[0][j][1] = 1

            for i in range(1, zero + 1):
                for j in range(1, one + 1):
                    dp[i][j][0] = (
                        dp[i - 1][j][0] + dp[i - 1][j][1] -
                        (dp[i - limit - 1][j][1] if i - limit >= 1 else 0) + MOD) % MOD
                    dp[i][j][1] = (
                        dp[i][j - 1][0] + dp[i][j - 1][1] -
                        (dp[i][j - limit - 1][0] if j - limit >= 1 else 0) + MOD) % MOD

            return (dp[zero][one][0] + dp[zero][one][1]) % MOD

```

2067. 3131.py

Path: LeetCode-main/solutions/3131. Find the Integer Added to Array I/3131.py

```

class Solution:
    def addedInteger(self, nums1: list[int], nums2: list[int]) -> int:
        return min(nums2) - min(nums1)

```


2068. 3132.py

Path: LeetCode-main/solutions/3132. Find the Integer Added to Array II/3132.py

```
class Solution:
    def minimumAddedInteger(self, nums1: list[int], nums2: list[int]) -> int:
        # After removing two elements from nums1, either nums1[0], nums1[1], or
        # nums1[2] will persist. Therefore, the difference between nums1 (with two
        # elements removed) and nums2 is represented by nums2[0] - nums1[i], where
        # 0 <= i <= 2.
        ans = math.inf

        nums1.sort()
        nums2.sort()

        for i in range(3):
            inc = nums2[0] - nums1[i]
            if self._isValidDiff(nums1, nums2, inc):
                ans = min(ans, inc)

        return ans

    def _isValidDiff(self, nums1: list[int], nums2: list[int], inc: int) -> bool:
        """
        Returns True if it's possible to increase nums1 (with two elements removed)
        by `inc` to nums2.
        """
        removed = 0
        i = 0 # nums2's index

        for num in nums1:
            if num + inc == nums2[i]:
                i += 1
            if i == len(nums2):
                break
            else:
                removed += 1

        return removed <= 2
```

2069. 3133.py

Path: LeetCode-main/solutions/3133. Minimum Array End/3133.py

```
class Solution:
    def minEnd(self, n: int, x: int) -> int:
        # Set x's 0s with (n - 1)'s LSB-to-MSb bits, preserving x's 1s. This
        # operation increase x for (n - 1) iterations while preserving x's 1s.
        MAX_BIT = n.bit_length() + x.bit_length()
        k = n - 1
        BINARY_INDEX = 0

        for i in range(MAX_BIT):
            if x >> i & 1 == 0:
                # Set x's 0 with k's bit if the running bit of k is 1.
                if k >> BINARY_INDEX & 1:
                    x |= 1 << i
                BINARY_INDEX += 1

        return x
```

2070. 3134.py

Path: LeetCode-main/solutions/3134. Find the Median of the Uniqueness Array/3134.py

```
class Solution:
    def medianOfUniquenessArray(self, nums: list[int]):
        n = len(nums)
        subarrayCount = n * (n + 1) // 2
        medianCount = (subarrayCount + 1) // 2

        # Similar to 992. Subarrays with K Different Integers
        def subarraysWithAtMostKDistinct(k: int) -> int:
            res = 0
            count = collections.Counter()

            l = 0
            for r, num in enumerate(nums):
                count[num] += 1
                if count[num] == 1:
                    k -= 1
                while k < 0:
                    count[nums[l]] -= 1
                    if count[nums[l]] == 0:
                        k += 1
                    l += 1
                res += r - l + 1 # nums[l..r], nums[l + 1..r], ..., nums[r]

            return res

        l = 1
        r = n
        return bisect.bisect_left(range(l, r), medianCount,
                                   key=subarraysWithAtMostKDistinct) + 1
```

2071. 3135.py

Path: LeetCode-main/solutions/3135. Equalize Strings by Adding or Removing Characters at Ends/3135.py

```
class Solution:
    def minOperations(self, initial: str, target: str) -> int:
        m = len(initial)
        n = len(target)
        # dp[i][j] := the length of LCS(initial[0..i], target[0..j])
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if initial[i - 1] == target[j - 1]:
                    dp[i][j] = 1 + dp[i - 1][j - 1]

        return m + n - 2 * max(map(max, dp))
```

2072. 3136.py

Path: LeetCode-main/solutions/3136. Valid Word/3136.py

```
class Solution:
    def isValid(self, word: str) -> bool:
        VOWELS = 'aeiouAEIOU'

        def isConsonant(c: str) -> bool:
            return c.isalpha() and c not in VOWELS

        return (len(word) >= 3 and
                all(c.isalnum() for c in word) and
                any(c in VOWELS for c in word) and
                any(isConsonant(c) for c in word))
```

2073. 3137.py

Path: LeetCode-main/solutions/3137. Minimum Number of Operations to Make Word K-Periodic/3137.py

```
class Solution:
    def minimumOperationsToMakeKPeriodic(self, word: str, k: int) -> int:
        count = collections.Counter(word[i:i + k] for i in range(0, len(word), k))
        return len(word) // k - max(count.values())
```

2074. 3138.py

Path: LeetCode-main/solutions/3138. Minimum Length of Anagram Concatenation/3138.py

```
class Solution:
    def minAnagramLength(self, s: str) -> int:
        n = len(s)
        for k in range(1, n + 1):
            if n % k == 0 and self._canFormAnagram(s, k):
                return k
        return n

    def _canFormAnagram(self, s: str, k: int) -> bool:
        """Returns True if we can concatenate an anagram of length k to s."""
        anagramCount = collections.Counter(s[:k])
        return all(collections.Counter(s[i:i + k]) == anagramCount
                  for i in range(k, len(s), k))
```

2075. 3139.py

Path: LeetCode-main/solutions/3139. Minimum Cost to Equalize Array/3139.py

```
class Solution:
    def minCostToEqualizeArray(
        self,
        nums: list[int],
        cost1: int,
        cost2: int,
    ) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        minNum = min(nums)
        maxNum = max(nums)
        summ = sum(nums)

        if cost1 * 2 <= cost2 or n < 3:
            totalGap = maxNum * n - summ
            return (cost1 * totalGap) % MOD

        def getMinCost(target: int) -> int:
            """Returns the minimum cost to make all numbers equal to `target`."""
            maxGap = target - minNum
            totalGap = target * n - summ
            # Pair one shallowest number with one non-shallowest number, so the worst
            # case is that we have `totalGap - maxGap` non-shallowest numbers to pair.
            pairs = min(totalGap // 2, totalGap - maxGap)
            return cost1 * (totalGap - 2 * pairs) + cost2 * pairs

        return min(getMinCost(target)
                    for target in range(maxNum, 2 * maxNum)) % MOD
```

2076. 314.py

Path: LeetCode-main/solutions/314. Binary Tree Vertical Order Traversal/314.py

```

class Solution:
    def verticalOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        range_ = [0] * 2

        def getRange(root: TreeNode | None, x: int) -> None:
            if not root:
                return

            range_[0] = min(range_[0], x)
            range_[1] = max(range_[1], x)

            getRange(root.left, x - 1)
            getRange(root.right, x + 1)

        getRange(root, 0) # Get the leftmost and the rightmost x index.

        ans = [[] for _ in range(range_[1] - range_[0] + 1)]
        q = collections.deque([(root, -range_[0])]) # (TreeNode, x)

        while q:
            node, x = q.popleft()
            ans[x].append(node.val)
            if node.left:
                q.append((node.left, x - 1))
            if node.right:
                q.append((node.right, x + 1))

        return ans

```

2077. 3141.py

Path: LeetCode-main/solutions/3141. Maximum Hamming Distances/3141.py

```

class Solution:
    def maxHammingDistances(self, nums: list[int], m: int) -> list[int]:
        MAX_MASK = 1 << m
        # dp[i] := the maximum hamming distance from i to any number in `nums`
        dp = [-math.inf] * MAX_MASK

        for num in nums:
            dp[num] = 0

        for bit in range(m):
            newDp = [0] * MAX_MASK
            for mask in range(MAX_MASK):
                newDp[mask] = max(dp[mask], dp[mask ^ (1 << bit)] + 1)
            dp = newDp

        return [dp[num] for num in nums]

```

2078. 3142.py

Path: LeetCode-main/solutions/3142. Check if Grid Satisfies Conditions/3142.py

```

class Solution:
    def satisfiesConditions(self, grid: list[list[int]]) -> bool:
        m = len(grid)
        n = len(grid[0])
        return (all(grid[i][j] == grid[i + 1][j]
                    for i in range(m - 1)
                    for j in range(n)) and
                all(grid[i][j] != grid[i][j + 1]
                    for i in range(m)
                    for j in range(n - 1)))

```

2079. 3143.py

Path: LeetCode-main/solutions/3143. Maximum Points Inside the Square/3143.py

```

class Solution:
    def maxPointsInsideSquare(self, points: list[list[int]], s: str) -> int:
        secondMinSize = math.inf
        minSizes = {}

        for (x, y), c in zip(points, s):
            sz = max(abs(x), abs(y))
            if c not in minSizes:
                minSizes[c] = sz
            elif sz < minSizes[c]:
                # This is because minSizes[j] is about to be replaced by a smaller
                # value, so it becomes a candidate for the second minimum size.
                secondMinSize = min(secondMinSize, minSizes[c])
                minSizes[c] = sz
            else:
                # `sz` is not smaller than the current minimum size, but it could be
                # smaller than the current second minimum size.
                secondMinSize = min(secondMinSize, sz)

        return sum(sz < secondMinSize for sz in minSizes.values())

```

2080. 3144.py

Path: LeetCode-main/solutions/3144. Minimum Substring Partition of Equal Character Frequency/3144.py

```

class Solution:
    def minimumSubstringsInPartition(self, s: str) -> int:
        n = len(s)
        # dp[i] := the minimum number of substrings in s[0..i]
        dp = [n] * n

        for i in range(n):
            count = collections.Counter()
            for j in range(i, -1, -1):
                count[s[j]] += 1
                # word[j..i] is balanced.
                if min(count.values()) == max(count.values()):
                    dp[i] = min(dp[i], 1 + dp[j - 1] if j > 0 else 1)

        return dp[-1]

```

2081. 3145.py

Path: LeetCode-main/solutions/3145. Find Products of Elements of Big Array/3145.py

```
class Solution:
    def findProductsOfElements(self, queries: list[list[int]]) -> list[int]:
        def sumBitsTill(x: int) -> int:
            """Returns sum(i.bit_count()), where 1 <= i <= x."""
            sumBits = 0
            powerOfTwo = 1
            while powerOfTwo <= x:
                sumBits += (x // (2 * powerOfTwo)) * powerOfTwo
                sumBits += max(0, x % (2 * powerOfTwo) + 1 - powerOfTwo)
                powerOfTwo *= 2
            return sumBits

        def sumPowersTill(x: int) -> int:
            """Returns sum(all powers of i), where 1 <= i <= x."""
            sumPowers = 0
            powerOfTwo = 1
            for power in range(x.bit_length()):
                sumPowers += (x // (2 * powerOfTwo)) * powerOfTwo * power
                sumPowers += max(0, x % (2 * powerOfTwo) + 1 - powerOfTwo) * power
                powerOfTwo *= 2
            return sumPowers

        def sumPowersFirstKBigNums(k: int) -> int:
            """Returns the sum of powers of the first k numbers in `big_nums`."""
            # Find the first number in [1, k] that has sumBitsTill(num) >= k.
            num = bisect.bisect_left(range(k), k, key=sumBitsTill)
            sumPowers = sumPowersTill(num - 1)
            remainingCount = k - sumBitsTill(num - 1)
            for power in range(num.bit_length()):
                if num >> power & 1:
                    sumPowers += power
                    remainingCount -= 1
                    if remainingCount == 0:
                        break
            return sumPowers

        return [pow(2,
                    sumPowersFirstKBigNums(b + 1) -
                    sumPowersFirstKBigNums(a), mod)
                for a, b, mod in queries]
```

2082. 3146.py

Path: LeetCode-main/solutions/3146. Permutation Difference between Two Strings/3146.py

```
class Solution:
    def findPermutationDifference(self, s: str, t: str) -> int:
        indices = {c: i for i, c in enumerate(s)}
        return sum([abs(indices[c] - i) for i, c in enumerate(t)])
```

2083. 3147.py

Path: LeetCode-main/solutions/3147. Taking Maximum Energy From the Mystic Dungeon/3147.py

```
class Solution:
    def maximumEnergy(self, energy: list[int], k: int) -> int:
        # dp[i] := the sum of energy starting at i
        dp = energy.copy()
        for i in range(len(energy) - 1 - k, -1, -1):
            dp[i] += dp[i + k]
        return max(dp)
```

2084. 3148.py

Path: LeetCode-main/solutions/3148. Maximum Difference Score in a Grid/3148.py

```
class Solution:
    def maxScore(self, grid: list[list[int]]) -> int:
        MAX = 200000
        ans = -MAX

        for i, row in enumerate(grid):
            for j, num in enumerate(row):
                prevMin = min(grid[i - 1][j] if i > 0 else MAX,
                              grid[i][j - 1] if j > 0 else MAX)
                ans = max(ans, num - prevMin)
                grid[i][j] = min(num, prevMin)

        return ans
```

2085. 3149.py

Path: LeetCode-main/solutions/3149. Find the Minimum Cost Array Permutation/3149.py


```

class Solution:
    def findPermutation(self, nums: list[int]) -> list[int]:
        n = len(nums)
        bestPick = [[0] * (1 << n) for _ in range(n)]

        @functools.lru_cache(None)
        def getScore(last: int, mask: int) -> int:
            if mask.bit_count() == len(nums):
                return abs(last - nums[0])

            minScore = math.inf
            for i in range(1, len(nums)):
                if mask >> i & 1:
                    continue
                nextMinScore = abs(last - nums[i]) + getScore(i, mask | (1 << i))
                if nextMinScore < minScore:
                    minScore = nextMinScore
                    bestPick[last][mask] = i

            return minScore

        getScore(0, 1)
        return self._construct(bestPick)

    def _construct(self, bestPick: list[list[int]]) -> list[int]:
        ans = []
        last = 0
        mask = 1
        for _ in range(len(bestPick)):
            ans.append(last)
            last = bestPick[last][mask]
            mask |= 1 << last
        return ans

```

2086. 315-4.py

Path: LeetCode-main/solutions/315. Count of Smaller Numbers After Self/315-4.py

```

from dataclasses import dataclass

@dataclass
class Item:
    num: int = 0
    index: int = 0

class Solution:
    def countSmaller(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [0] * n
        items = [Item(num, i) for i, num in enumerate(nums)]

        self._mergeSort(items, 0, n - 1, ans)
        return ans

    def _mergeSort(
        self,
        items: list[Item],
        l: int,
        r: int,
        ans: list[int],
    ):
        if l < r:
            m = (l + r) // 2
            self._mergeSort(items, l, m, ans)
            self._mergeSort(items, m + 1, r, ans)
            self._merge(items, l, m, r, ans)

```

```

) -> None:
    if l >= r:
        return

    m = (l + r) // 2
    self._mergeSort(items, l, m, ans)
    self._mergeSort(items, m + 1, r, ans)
    self._merge(items, l, m, r, ans)

def _merge(
    self,
    items: list[Item],
    l: int,
    m: int,
    r: int,
    ans: list[int],
) -> None:
    sorted = [Item()] * (r - l + 1)
    k = 0 # sorted's index
    i = l # left's index
    j = m + 1 # right's index
    rightCount = 0 # the number of numbers < items[i].num

    while i <= m and j <= r:
        if items[i].num > items[j].num:
            rightCount += 1
            sorted[k] = items[j]
            k += 1
            j += 1
        else:
            ans[items[i].index] += rightCount
            sorted[k] = items[i]
            k += 1
            i += 1

    # Put the possible remaining left part into the sorted array.
    while i <= m:
        ans[items[i].index] += rightCount
        sorted[k] = items[i]
        k += 1
        i += 1

    # Put the possible remaining right part into the sorted array.
    while j <= r:
        sorted[k] = items[j]
        k += 1
        j += 1

    items[l:l + len(sorted)] = sorted

```

2087. 315.py

Path: LeetCode-main/solutions/315. Count of Smaller Numbers After Self/315.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class Solution:
    def countSmaller(self, nums: list[int]) -> list[int]:
        ans = []
        ranks = self._getRanks(nums)
        tree = FenwickTree(len(ranks))

        for num in reversed(nums):
            ans.append(tree.get(ranks[num] - 1))
            tree.add(ranks[num], 1)

        return ans[::-1]

    def _getRanks(self, nums: list[int]) -> dict[int, int]:
        ranks = collections.Counter()
        rank = 0
        for num in sorted(set(nums)):
            rank += 1
            ranks[num] = rank
        return ranks

```

2088. 3151.py

Path: LeetCode-main/solutions/3151. Special Array I/3151.py

```

class Solution:
    def isArraySpecial(self, nums: list[int]) -> bool:
        return all(a % 2 != b % 2 for a, b in itertools.pairwise(nums))

```

2089. 3152.py

Path: LeetCode-main/solutions/3152. Special Array II/3152.py

```

class Solution:
    def isArraySpecial(
        self,
        nums: list[int],
        queries: list[list[int]],
    ) -> list[bool]:
        ans = []
        id = 0
        # parityIds[i] := the id of the parity group that nums[i] belongs to
        parityIds = [id]

        for a, b in itertools.pairwise(nums):
            if a % 2 == b % 2:
                id += 1
            parityIds.append(id)

        for _from, to in queries:
            ans.append(parityIds[_from] == parityIds[to])

        return ans

```

2090. 3153.py

Path: LeetCode-main/solutions/3153. Sum of Digit Differences of All Pairs/3153.py

```

class Solution:
    def sumDigitDifferences(self, nums: list[int]) -> int:
        n = len(nums)
        digitSize = len(str(nums[0]))
        ans = 0

        denominator = 1
        for _ in range(digitSize):
            count = [0] * 10
            for num in nums:
                count[num // denominator % 10] += 1
            ans += sum(freq * (n - freq) for freq in count)
            denominator *= 10

        return ans // 2

```

2091. 3154.py

Path: LeetCode-main/solutions/3154. Find Number of Ways to Reach the K-th Stair/3154.py

```

class Solution:
    def waysToReachStair(self, k: int) -> int:
        # Let's say we have `down` operation 1 and `jump` operation 2.
        # The final stair is  $1 + (2^0 + 2^1 + \dots + 2^{(\text{jump} - 1)}) - \text{down} = k$ .
        #  $\Rightarrow 1 + (2^{\text{jump}} - 1) - \text{down} = k$ .
        #  $\Rightarrow \text{down} = 2^{\text{jump}} - k$ .
        # Since `down` operations cannot be used consecutively, there're jump + 1
        # positions (before and after each `jump`) for `down`. The maximum jump is
        # 29, as it satisfies the condition  $\text{down} = 2^{\text{jump}} - k \leq \text{jump} + 1$ , with k
        # being the maximum value of  $10^9$ .
        MAX_JUMP = 29
        ans = 0

        for jump in range(MAX_JUMP + 1):
            down = (1 << jump) - k
            if down < 0 or down > jump + 1:
                continue
            ans += math.comb(jump + 1, down)

        return ans

```

2092. 3155.py

Path: LeetCode-main/solutions/3155. Maximum Number of Upgradable Servers/3155.py

```

class Solution:
    def maxUpgrades(
        self,
        count: list[int],
        upgrade: list[int],
        sell: list[int],
        money: list[int],
    ) -> list[int]:
        # If there's enough money, upgrade all servers; otherwise, optimize by
        # upgrading x servers. We have  $x * \text{upgrade} \leq \text{money} + (\text{count} - x) * \text{sell}$ .
        # Therefore,  $x = (\text{money} + \text{count} * \text{sell}) / (\text{sell} + \text{upgrade})$ .
        return [min(c, (m + c * s) // (s + u))
                for c, u, s, m in zip(count, upgrade, sell, money)]

```

2093. 3157-2.py

Path: LeetCode-main/solutions/3157. Find the Level of Tree with Minimum Sum/3157-2.py

```

class Solution:
    # Similar to 1161. Maximum Level Sum of a Binary Tree
    def minimumLevel(self, root: TreeNode | None) -> int:
        # levelSums[i] := the sum of level (i + 1) (1-indexed)
        levelSums = []

        def dfs(root: TreeNode | None, level: int) -> None:
            if not root:
                return
            if len(levelSums) == level:
                levelSums.append(0)
            levelSums[level] += root.val
            dfs(root.left, level + 1)
            dfs(root.right, level + 1)

        dfs(root, 0)
        return 1 + levelSums.index(min(levelSums))

```

2094. 3157.py

Path: LeetCode-main/solutions/3157. Find the Level of Tree with Minimum Sum/3157.py

```

class Solution:
    # Similar to 1161. Maximum Level Sum of a Binary Tree
    def minimumLevel(self, root: TreeNode | None) -> int:
        ans = 0
        minLevelSum = math.inf
        q = collections.deque([root])

        level = 1
        while q:
            levelSum = 0
            for _ in range(len(q)):
                node = q.popleft()
                levelSum += node.val
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            if levelSum < minLevelSum:
                minLevelSum = levelSum
                ans = level
            level += 1

        return ans

```

2095. 3158.py

Path: LeetCode-main/solutions/3158. Find the XOR of Numbers Which Appear Twice/3158.py

```

class Solution:
    def duplicateNumbersXOR(self, nums):
        count = collections.Counter(nums)
        return functools.reduce(
            operator.xor, [num for num, freq in count.items() if freq == 2],
            0)

```

2096. 3159.py

Path: LeetCode-main/solutions/3159. Find Occurrences of an Element in an Array/3159.py

```
class Solution:
    def occurrencesOfElement(
        self,
        nums: list[int],
        queries: list[int],
        x: int,
    ) -> list[int]:
        indices = [i for i, num in enumerate(nums) if num == x]
        return [indices[query - 1] if query <= len(indices) else -1
                for query in queries]
```

2097. 316.py

Path: LeetCode-main/solutions/316. Remove Duplicate Letters/316.py

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        ans = []
        count = collections.Counter(s)
        used = [False] * 26

        for c in s:
            count[c] -= 1
            if used[ord(c) - ord('a')]:
                continue
            while ans and ans[-1] > c and count[ans[-1]] > 0:
                used[ord(ans[-1]) - ord('a')] = False
                ans.pop()
            ans.append(c)
            used[ord(ans[-1]) - ord('a')] = True

        return ''.join(ans)
```

2098. 3160.py

Path: LeetCode-main/solutions/3160. Find the Number of Distinct Colors Among the Balls/3160.py

```
class Solution:
    def queryResults(self, limit: int, queries: list[list[int]]) -> list[int]:
        ans = []
        ballToColor = {}
        colorCount = collections.Counter()

        for ball, color in queries:
            if ball in ballToColor:
                prevColor = ballToColor[ball]
                colorCount[prevColor] -= 1
                if colorCount[prevColor] == 0:
                    del colorCount[prevColor]
            ballToColor[ball] = color
            colorCount[color] += 1
            ans.append(len(colorCount))

        return ans
```

2099. 3161.py

Path: LeetCode-main/solutions/3161. Block Placement Queries/3161.py

```
from sortedcontainers import SortedList

class FenwickTree:
    def __init__(self, n: int):
        self.vals = [0] * (n + 1)

    def maximize(self, i: int, val: int) -> None:
        while i < len(self.vals):
            self.vals[i] = max(self.vals[i], val)
            i += FenwickTree.lowtree(i)

    def get(self, i: int) -> int:
        res = 0
        while i > 0:
            res = max(res, self.vals[i])
            i -= FenwickTree.lowtree(i)
        return res

    @staticmethod
    def lowtree(i: int) -> int:
        return i & -i

class Solution:
    def getResults(self, queries: list[list[int]]) -> list[bool]:
        n = min(50000, len(queries) * 3)
        ans = []
        tree = FenwickTree(n + 1)
        obstacles = SortedList([0, n]) # sentinel values

        for query in queries:
            type = query[0]
            if type == 1:
                x = query[1]
                obstacles.add(x)

        for x1, x2 in itertools.pairwise(obstacles):
            tree.maximize(x2, x2 - x1)

        for query in reversed(queries):
            type = query[0]
            x = query[1]
            if type == 1:
                i = obstacles.index(x)
                next = obstacles[i + 1]
                prev = obstacles[i - 1]
                obstacles.remove(x)
                tree.maximize(next, next - prev)
            else:
                sz = query[2]
                i = obstacles.bisect_right(x)
                prev = obstacles[i - 1]
                ans.append(tree.get(prev) >= sz or x - prev >= sz)

        return ans[::-1]
```

2100. 3162.py

Path: LeetCode-main/solutions/3162. Find the Number of Good Pairs I/3162.py

```
class Solution:
    def numberOfPairs(self, nums1: list[int], nums2: list[int], k: int) -> int:
        return sum(num1 % (num2 * k) == 0
                    for num1 in nums1
                    for num2 in nums2)
```

2101. 3163.py

Path: LeetCode-main/solutions/3163. String Compression III/3163.py

```
class Solution:
    def compressedString(self, word: str) -> str:
        n = len(word)
        ans = []
        i = 0
        j = 0

        while i < n:
            count = 0
            while j < n and word[j] == word[i] and count < 9:
                j += 1
                count += 1
            ans.append(str(count) + word[i])
            i = j

        return ''.join(ans)
```

2102. 3164.py

Path: LeetCode-main/solutions/3164. Find the Number of Good Pairs II/3164.py

```
class Solution:
    def numberOfPairs(self, nums1: list[int], nums2: list[int], k: int) -> int:
        count = collections.Counter(num * k for num in nums2)
        ans = 0

        for num in nums1:
            for divisor in range(1, int(num ** 0.5) + 1):
                if num % divisor == 0:
                    ans += count[divisor]
                    if num // divisor != divisor:
                        ans += count[num // divisor]

        return ans
```

2103. 3167.py

Path: LeetCode-main/solutions/3167. Better Compression of String/3167.py

```

class Solution:
    def betterCompression(self, compressed: str) -> str:
        count = collections.Counter()
        i = 0

        while i < len(compressed):
            c = compressed[i]
            i += 1
            freq = 0
            while i < len(compressed) and compressed[i].isdigit():
                freq = freq * 10 + int(compressed[i])
                i += 1
            count[c] += freq

        return ''.join([c + str(count[c])
                        for c in sorted(count.keys())])

```

2104. 3168.py

Path: LeetCode-main/solutions/3168. Minimum Number of Chairs in a Waiting Room/3168.py

```

class Solution:
    def minimumChairs(self, s: str) -> int:
        ans = 0
        chairs = 0

        for c in s:
            chairs += 1 if c == 'E' else -1
            ans = max(ans, chairs)

        return ans

```

2105. 3169.py

Path: LeetCode-main/solutions/3169. Count Days Without Meetings/3169.py

```

class Solution:
    def countDays(self, days: int, meetings: list[list[int]]) -> int:
        freeDays = 0
        prevEnd = 0

        for start, end in sorted(meetings):
            if start > prevEnd:
                freeDays += start - prevEnd - 1
            prevEnd = max(prevEnd, end)

        return freeDays + max(0, days - prevEnd)

```

2106. 317.py

Path: LeetCode-main/solutions/317. Shortest Distance from All Buildings/317.py

```

class Solution:
    def shortestDistance(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        nBuildings = sum(a == 1 for row in grid for a in row)
        ans = math.inf
        # dist[i][j] := the total distance of grid[i][j] (0) to reach all the
        # buildings (1)
        dist = [[0] * n for _ in range(m)]
        # reachCount[i][j] := the number of buildings (1) grid[i][j] (0) can reach
        reachCount = [[0] * n for _ in range(m)]

        def bfs(row: int, col: int) -> bool:
            q = collections.deque([(row, col)])
            seen = {(row, col)}
            seenBuildings = 1

            step = 1
            while q:
                for _ in range(len(q)):
                    i, j = q.popleft()
                    for dx, dy in DIRS:
                        x = i + dx
                        y = j + dy
                        if x < 0 or x == m or y < 0 or y == n:
                            continue
                        if (x, y) in seen:
                            continue
                        seen.add((x, y))
                        if not grid[x][y]:
                            dist[x][y] += step
                            reachCount[x][y] += 1
                            q.append((x, y))
                        elif grid[x][y] == 1:
                            seenBuildings += 1
                    step += 1

            # True if all the buildings (1) are connected
            return seenBuildings == nBuildings

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1: # BFS from this building.
                    if not bfs(i, j):
                        return -1

        for i in range(m):
            for j in range(n):
                if reachCount[i][j] == nBuildings:
                    ans = min(ans, dist[i][j])

        return -1 if ans == math.inf else ans

```

2107. 3170.py

Path: LeetCode-main/solutions/3170. Lexicographically Minimum String After Removing Stars/3170.py

```

class Solution:
    def clearStars(self, s: str) -> str:
        ans = list(s)
        buckets = [[] for _ in range(26)]

        for i, c in enumerate(s):
            if c == '*':
                ans[i] = ''
                j = next(j for j, bucket in enumerate(buckets) if bucket)
                ans[buckets[j].pop()] = ''
            else:
                buckets[ord(c) - ord('a')].append(i)

        return ''.join(ans)

```

2108. 3171.py

Path: LeetCode-main/solutions/3171. Find Subarray With Bitwise AND Closest to K/3171.py

```

class Solution:
    # Similar to 1521. Find a Value of a Mysterious Function Closest to Target
    def minimumDifference(self, nums: list[int], k: int) -> int:
        ans = math.inf
        dp = set() # all the values of subarrays that end in the current number

        for num in nums:
            # Extend each subarray that ends in the dpious number. Due to
            # monotonicity of the OR operation, the size of `next_set` will be at most
            # bin(num).count('1') + 1.
            dp = {num} | {val | num for val in dp}
            ans = min(ans, min(abs(k - val) for val in dp))

        return ans

```

2109. 3173.py

Path: LeetCode-main/solutions/3173. Bitwise OR of Adjacent Elements/3173.py

```

class Solution:
    def orArray(self, nums: list[int]) -> list[int]:
        return [a | b for a, b in itertools.pairwise(nums)]

```

2110. 3174.py

Path: LeetCode-main/solutions/3174. Clear Digits/3174.py

```

class Solution:
    def clearDigits(self, s: str) -> str:
        ans = []

        for c in s:
            if c.isdigit():
                # Since `ans` only contains non-digit characters, removing the last
                # character is equivalent to deleting the closest non-digit character.
                ans.pop()
            else:
                ans.append(c)

        return ''.join(ans)

```

2111. 3175.py

Path: LeetCode-main/solutions/3175. Find The First Player to win K Games in a Row/3175.py

```

class Solution:
    # Similar to 1535. Find the Winner of an Array Game
    def findWinningPlayer(self, skills: list[int], k: int) -> int:
        ans = 0
        wins = 0

        i = 1
        while i < len(skills) and wins < k:
            if skills[i] > skills[ans]:
                ans = i
                wins = 1
            else:
                wins += 1
            i += 1

        return ans

```

2112. 3176.py

Path: LeetCode-main/solutions/3176. Find the Maximum Length of a Good Subsequence I/3176.py

```

class Solution:
    def maximumLength(self, nums: list[int], k: int) -> int:
        # dp[count][num] := the maximum length of a good subsequence with at most
        # `count` indices where seq[i] != seq[i + 1] and it ends in `num`.
        dp = [collections.Counter() for _ in range(k + 1)]
        # maxlen[count] := the maximum length of a good subsequence with `count`
        # indices where seq[i] != seq[i + 1]
        maxlen = [0] * (k + 1)

        for num in nums:
            for count in range(k, -1, -1):
                # Append `num` to the subsequence.
                dp[count][num] += 1
                if count > 0:
                    dp[count][num] = max(dp[count][num], maxlen[count - 1] + 1)
                maxlen[count] = max(maxlen[count], dp[count][num])

        return maxlen[k]

```

2113. 3177.py

Path: LeetCode-main/solutions/3177. Find the Maximum Length of a Good Subsequence II/3177.py

```
class Solution:
    # Same as 3176. Find the Maximum Length of a Good Subsequence I
    def maximumLength(self, nums: list[int], k: int) -> int:
        # dp[count][num] := the maximum length of a good subsequence with at most
        # `count` indices where seq[i] != seq[i + 1] and it ends in `num`.
        dp = [collections.Counter() for _ in range(k + 1)]
        # maxlen[count] := the maximum length of a good subsequence with `count`
        # indices where seq[i] != seq[i + 1]
        maxlen = [0] * (k + 1)

        for num in nums:
            for count in range(k, -1, -1):
                # Append `num` to the subsequence.
                dp[count][num] += 1
                if count > 0:
                    dp[count][num] = max(dp[count][num], maxlen[count - 1] + 1)
                maxlen[count] = max(maxlen[count], dp[count][num])

        return maxlen[k]
```

2114. 3178.py

Path: LeetCode-main/solutions/3178. Find the Child Who Has the Ball After K Seconds/3178.py

```
class Solution:
    def numberOfChild(self, n: int, k: int) -> int:
        # the time for the ball to return to 0
        roundTime = 2 * (n - 1)
        pos = k % roundTime
        return pos if pos < n else roundTime - pos
```

2115. 3179.py

Path: LeetCode-main/solutions/3179. Find the N-th Value After K Seconds/3179.py

```
class Solution:
    def valueAfterKSeconds(self, n: int, k: int) -> int:
        return math.comb(n + k - 1, n - 1) % 1_000_000_007
```

2116. 318.py

Path: LeetCode-main/solutions/318. Maximum Product of Word Lengths/318.py

```

class Solution:
    def maxProduct(self, words: list[str]) -> int:
        ans = 0

        def getMask(word: str) -> int:
            mask = 0
            for c in word:
                mask |= 1 << ord(c) - ord('a')
            return mask

        masks = [getMask(word) for word in words]

        for i in range(len(words)):
            for j in range(i):
                if not (masks[i] & masks[j]):
                    ans = max(ans, len(words[i]) * len(words[j]))

        return ans

```

2117. 3180.py

Path: LeetCode-main/solutions/3180. Maximum Total Reward Using Operations I/3180.py

```

# According to the constraint rewardValues[i] <= 5 * 10^4, the maximum total
# reward < 2 * (5 * 10^4) = 10^5. We can use bitset to record whether each
# `rewardValue` is achievable in O(1).
#
# Let's use `rewardValues = [1, 3, 4]` as an example.
#
# The maximum reward is 4, so the maximum possible total < 2 * 4 = 8.
# Therefore, we can set the size of the bitset to 8 to represent possible
# total rewards from 0 to 7.
#
# Let's define a bitset `dp` to record whether each total reward is
# achievable. dp[num] = true if reward `num` is achievable.
#
# Initially, dp = 0b00000001 := reward 0 is achievable.
#
# * rewardValues[0] = 1, for each dp[i] = 1, where i + 1 < 10, dp[i + 1] = 1.
#   => dp = 0b00000011 := rewards 0 and 1 are achievable.
#
# * rewardValues[1] = 3, for each dp[i] = 1, where i + 3 < 10, dp[i + 3] = 1.
#   => dp = 0b00011011 := rewards 0, 1, 3, and 4 are achievable.
#
# * rewardValues[2] = 4, for each dp[i] = 1, where i + 4 < 10, dp[i + 4] = 1.
#   => dp = 0b10011011 := rewards 0, 1, 3, 4, 5, and 7 are achievable.
#
# Therefore, the maximum total reward is 7.

```

```

class Solution:
    def maxTotalReward(self, rewardValues: list[int]) -> int:
        dp = 1 # the possible rewards (initially, 0 is achievable)

        for num in sorted(rewardValues):
            # Remove the numbers >= the current number.
            smallerNums = dp & ((1 << num) - 1)
            dp |= smallerNums << num

        return dp.bit_length() - 1

```

2118. 3181.py

Path: LeetCode-main/solutions/3181. Maximum Total Reward Using Operations II/3181.py

```
class Solution:
    # Same as 3180. Maximum Total Reward Using Operations I
    def maxTotalReward(self, rewardValues: list[int]) -> int:
        dp = 1 # the possible rewards (initially, 0 is achievable)

        for num in sorted(rewardValues):
            # Remove the numbers >= the current number.
            smallerNums = dp & ((1 << num) - 1)
            dp |= smallerNums << num

        return dp.bit_length() - 1
```

2119. 3183.py

Path: LeetCode-main/solutions/3183. The Number of Ways to Make the Sum/3183.py

```
class Solution:
    def numberOfWays(self, n: int) -> int:
        MOD = 1_000_000_007
        # dp[i] := the number of ways to make the sum of i using coins 1, 2, and 6
        dp = [1] + [0] * n

        for coin in (1, 2, 6):
            for i in range(coin, n + 1):
                dp[i] = (dp[i] + dp[i - coin]) % MOD

        ans = dp[n]
        if n - 4 >= 0:
            ans = (ans + dp[n - 4]) % MOD
        if n - 8 >= 0:
            ans = (ans + dp[n - 8]) % MOD
        return ans
```

2120. 3184.py

Path: LeetCode-main/solutions/3184. Count Pairs That Form a Complete Day I/3184.py

```
class Solution:
    def countCompleteDayPairs(self, hours: list[int]) -> int:
        ans = 0
        count = [0] * 24

        for hour in hours:
            ans += count[(24 - hour % 24) % 24]
            count[hour % 24] += 1

        return ans
```

2121. 3185.py

Path: LeetCode-main/solutions/3185. Count Pairs That Form a Complete Day II/3185.py


```

class Solution:
    # Same as 3184. Count Pairs That Form a Complete Day I
    def countCompleteDayPairs(self, hours: list[int]) -> int:
        ans = 0
        count = [0] * 24

        for hour in hours:
            ans += count[(24 - hour % 24) % 24]
            count[hour % 24] += 1

        return ans

```

2122. 3186.py

Path: LeetCode-main/solutions/3186. Maximum Total Damage With Spell Casting/3186.py

```

class Solution:
    def maximumTotalDamage(self, power: list[int]) -> int:
        count = collections.Counter(power)
        uniqueDamages = sorted(count.keys())
        # dp[i][k] := the maximum damage using uniqueDamages[0..i], where k
        # indicates if the i-th damage is used
        dp = [[0] * 2 for _ in range(len(uniqueDamages))]

        for i, damage in enumerate(uniqueDamages):
            if i == 0:
                dp[0] = [0, damage * count[damage]]
                continue
            dp[i][0] = max(dp[i - 1])
            dp[i][1] = damage * count[damage]
            if i >= 1 and uniqueDamages[i - 1] not in (damage - 1, damage - 2):
                dp[i][1] += max(dp[i - 1])
            elif i >= 2 and uniqueDamages[i - 2] != damage - 2:
                dp[i][1] += max(dp[i - 2])
            elif i >= 3:
                dp[i][1] += max(dp[i - 3])

        return max(dp[-1])

```

2123. 3187.py

Path: LeetCode-main/solutions/3187. Peaks in Array/3187.py

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod

```

```

def lowbit(i: int) -> int:
    return i & -i

class Solution:
    def countOfPeaks(
        self,
        nums: list[int],
        queries:
            list[list[int]],
    ) -> list[int]:
        ans = []
        peak = [0] + [int(a < b > c)
                        for a, b, c in zip(nums[:-2], nums[1:-1], nums[2:])] + [0]
        tree = FenwickTree(len(peak))

        for i, p in enumerate(peak):
            tree.add(i + 1, p)

        def update(i: int) -> None:
            """
            Update the peak array and Fenwick tree if the peak status of nums[i]
            changes.
            """
            newPeak = self._isPeak(nums, i)
            if newPeak != peak[i]:
                tree.add(i + 1, newPeak - peak[i])
                peak[i] = newPeak

        for query in queries:
            if query[0] == 1:
                l = query[1]
                r = query[2]
                ans.append(0 if r - l < 2 else tree.get(r) - tree.get(l + 1))
            elif query[0] == 2:
                index = query[1]
                val = query[2]
                nums[index] = val
                update(index)
                if index > 0:
                    update(index - 1)
                if index + 1 < len(nums):
                    update(index + 1)

        return ans

    def _isPeak(self, nums: list[int], i: int) -> bool:
        return i > 0 and i + 1 < len(nums) and nums[i - 1] < nums[i] > nums[i + 1]

```

2124. 3189.py

Path: LeetCode-main/solutions/3189. Minimum Moves to Get a Peaceful Board/3189.py

```

class Solution:
    def minMoves(self, rooks: list[list[int]]) -> int:
        n = len(rooks)
        sortedByRow = sorted(rooks, key=lambda x: x[0])
        sortedByCol = sorted(rooks, key=lambda x: x[1])
        return (sum(abs(i - row) for (i, _), row in zip(sortedByRow, range(n))) +
                sum(abs(j - col) for (_, j), col in zip(sortedByCol, range(n))))

```

2125. 319.py

Path: LeetCode-main/solutions/319. Bulb Switcher/319.py

```
class Solution:
    def bulbSwitch(self, n: int) -> int:
        # The k-th bulb can only be switched when k % i == 0.
        # So, we can rephrase the problem:
        # To find number of numbers <= n that have odd factors.
        # Obviously, only square numbers have odd factor(s).
        # e.g. n = 10, only 1, 4, and 9 are square numbers that <= 10
        return math.isqrt(n)
```

2126. 3190.py

Path: LeetCode-main/solutions/3190. Find Minimum Operations to Make All Elements Divisible by Three/3190.py

```
class Solution:
    def minimumOperations(self, nums: list[int]) -> int:
        return sum(num % 3 != 0 for num in nums)
```

2127. 3191.py

Path: LeetCode-main/solutions/3191. Minimum Operations to Make Binary Array Elements Equal to One I/3191.py

```
class Solution:
    def minOperations(self, nums: list[int]) -> int:
        ans = 0

        for i in range(len(nums) - 2):
            if nums[i] == 0:
                nums[i + 1] ^= 1
                nums[i + 2] ^= 1
                ans += 1

        return -1 if nums[-1] == 0 or nums[-2] == 0 else ans
```

2128. 3192.py

Path: LeetCode-main/solutions/3192. Minimum Operations to Make Binary Array Elements Equal to One II/3192.py

```
class Solution:
    def minOperations(self, nums: list[int]) -> int:
        ans = 0
        target = 1

        for num in nums:
            if num != target:
                ans += 1
                target ^= 1

        return ans
```

2129. 3193.py

Path: LeetCode-main/solutions/3193. Count the Number of Inversions/3193.py

```
class Solution:
    def numberOfPermutations(self, n: int, requirements: list[list[int]]) -> int:
        MOD = 1_000_000_007
        MAX_INVERSIONS = 400
        # dp[i][j] := the number of ways to arrange the first i numbers of the
        # permutation s.t. there are j inversions
        dp = [[0] * (MAX_INVERSIONS + 1) for _ in range(n + 1)]
        endToCnt = {end + 1: cnt for end, cnt in requirements}

        # There's only one way to arrange a single number with zero inversions.
        dp[1][0] = 1

        for i in range(2, n + 1):
            for newInversions in range(i):
                for j in range(MAX_INVERSIONS - newInversions + 1):
                    inversionsAfterInsertion = j + newInversions
                    if i in endToCnt and inversionsAfterInsertion != endToCnt[i]:
                        continue
                    dp[i][inversionsAfterInsertion] += dp[i - 1][j]
                    dp[i][inversionsAfterInsertion] %= MOD

        return dp[n][endToCnt[n]]
```

2130. 3194.py

Path: LeetCode-main/solutions/3194. Minimum Average of Smallest and Largest Elements/3194.py

```
class Solution:
    def minimumAverage(self, nums: list[int]) -> float:
        nums.sort()
        return min((nums[i] + nums[~i]) / 2
                   for i in range(len(nums) // 2 + 1))
```

2131. 3195.py

Path: LeetCode-main/solutions/3195. Find the Minimum Area to Cover All Ones I/3195.py

```
class Solution:
    def minimumArea(self, grid: list[list[int]]) -> int:
        x1 = math.inf
        y1 = math.inf
        x2 = 0
        y2 = 0

        for i, row in enumerate(grid):
            for j, num in enumerate(row):
                if num == 1:
                    x1 = min(x1, i)
                    y1 = min(y1, j)
                    x2 = max(x2, i)
                    y2 = max(y2, j)

        return (x2 - x1 + 1) * (y2 - y1 + 1)
```

2132. 3196-2.py

Path: LeetCode-main/solutions/3196. Maximize Total Cost of Alternating Subarrays/3196-2.py

```
class Solution:
    def maximumTotalCost(self, nums: list[int]) -> int:
        # A small trick so that we don't need to handle the edge case and can use
        # ranged-based for loop.
        keep = -math.inf # the maximum cost if the last number is kept
        flip = 0 # the maximum cost if the last number is flipped

        for num in nums:
            keep, flip = max(keep, flip) + num, keep - num

        return max(keep, flip)
```

2133. 3196.py

Path: LeetCode-main/solutions/3196. Maximize Total Cost of Alternating Subarrays/3196.py

```
class Solution:
    def maximumTotalCost(self, nums: list[int]) -> int:
        keep = nums[0] # the maximum cost if the last number is kept
        flip = nums[0] # the maximum cost if the last number is flipped

        for i in range(1, len(nums)):
            keepCurr = max(keep, flip) + nums[i]
            flipCurr = keep - nums[i]
            keep = keepCurr
            flip = flipCurr

        return max(keep, flip)
```

2134. 3197.py

Path: LeetCode-main/solutions/3197. Find the Minimum Area to Cover All Ones II/3197.py

```
class Solution:
    def minimumSum(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = m * n

        for i in range(m):
            top = self._minimumArea(grid, 0, i, 0, n - 1)
            for j in range(n):
                ans = min(ans, top +
                           self._minimumArea(grid, i + 1, m - 1, 0, j) +
                           self._minimumArea(grid, i + 1, m - 1, j + 1, n - 1))

        for i in range(m):
            bottom = self._minimumArea(grid, i, m - 1, 0, n - 1)
            for j in range(n):
                ans = min(ans, bottom +
                           self._minimumArea(grid, 0, i - 1, 0, j) +
                           self._minimumArea(grid, 0, i - 1, j + 1, n - 1))

        for j in range(n):
            left = self._minimumArea(grid, 0, m - 1, 0, j)
            for i in range(m):
                ans = min(ans, left +
```

```

        self._minimumArea(grid, 0, i, j + 1, n - 1) +
        self._minimumArea(grid, i + 1, m - 1, j + 1, n - 1))

for j in range(n):
    right = self._minimumArea(grid, 0, m - 1, j, n - 1)
    for i in range(m):
        ans = min(ans, right +
            self._minimumArea(grid, 0, i, 0, j - 1) +
            self._minimumArea(grid, i + 1, m - 1, 0, j - 1))

for i1 in range(m):
    for i2 in range(i1 + 1, m):
        ans = min(ans, self._minimumArea(grid, 0, i1, 0, n - 1) +
            self._minimumArea(grid, i1 + 1, i2, 0, n - 1) +
            self._minimumArea(grid, i2 + 1, m - 1, 0, n - 1))

for j1 in range(n):
    for j2 in range(j1 + 1, n):
        ans = min(ans, self._minimumArea(grid, 0, m - 1, 0, j1) +
            self._minimumArea(grid, 0, m - 1, j1 + 1, j2) +
            self._minimumArea(grid, 0, m - 1, j2 + 1, n - 1))

return ans

def _minimumArea(
    self,
    grid: list[list[int]],
    si: int,
    ei: int,
    sj: int,
    ej: int,
) -> int:
    x1 = math.inf
    y1 = math.inf
    x2 = 0
    y2 = 0
    for i in range(si, ei + 1):
        for j in range(sj, ej + 1):
            if grid[i][j] == 1:
                x1 = min(x1, i)
                y1 = min(y1, j)
                x2 = max(x2, i)
                y2 = max(y2, j)
    return 0 if x1 == math.inf else (x2 - x1 + 1) * (y2 - y1 + 1)

```

2135. 3199.py

Path: LeetCode-main/solutions/3199. Count Triplets with Even XOR Set Bits I/3199.py

```

class Solution:
    def tripletCount(self, a: list[int], b: list[int], c: list[int]) -> int:
        evenA, oddA = self._getEvenOddBitCount(a)
        evenB, oddB = self._getEvenOddBitCount(b)
        evenC, oddC = self._getEvenOddBitCount(c)
        return evenA * oddB * oddC + oddA * evenB * oddC + oddA * oddB * evenC + evenA * evenB * evenC

    def _getEvenOddBitCount(self, nums: list[int]) -> tuple[int, int]:
        """
        Returns the count of numbers in the `nums` arrays that have even number of
        ones and odd number of ones in their binary representation.
        """
        even = sum(num.bit_count() % 2 == 0 for num in nums)
        return (even, len(nums) - even)

```

2136. 32.py

Path: LeetCode-main/solutions/32. Longest Valid Parentheses/32.py

```

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        s2 = ')' + s
        # dp[i] := the length of the longest valid parentheses in the substring
        # s2[1..i]
        dp = [0] * len(s2)

        for i in range(1, len(s2)):
            if s2[i] == ')' and s2[i - dp[i - 1] - 1] == '(':
                dp[i] = dp[i - 1] + dp[i - dp[i - 1] - 2] + 2

        return max(dp)

```

2137. 320.py

Path: LeetCode-main/solutions/320. Generalized Abbreviation/320.py

```

class Solution:
    def generateAbbreviations(self, word: str) -> list[str]:
        ans = []

        def getCountString(count: int) -> str:
            return str(count) if count > 0 else ''

        def dfs(i: int, count: int, path: list[str]) -> None:
            if i == len(word):
                ans.append(''.join(path) + getCountString(count))
                return

            # Abbreviate the word[i].
            dfs(i + 1, count + 1, path)
            # Keep the word[i], so consume the count as a string.
            path.append(getCountString(count) + word[i])
            # Reset the count to 0.
            dfs(i + 1, 0, path)
            path.pop()

        dfs(0, 0, [])
        return ans

```

2138. 3200.py

Path: LeetCode-main/solutions/3200. Maximum Height of a Triangle/3200.py

```
class Solution:
    def maxHeightOfTriangle(self, red: int, blue: int) -> int:
        return max(self._maxHeight(red, blue),
                   self._maxHeight(blue, red))

    def _maxHeight(self, n1: int, n2: int) -> int:
        """
        Returns the maximum height of a triangle with the odd levels having `n1`
        balls and the even levels having `n2` balls.
        """
        #          1 + 3 + ... + h <= n1
        # ((1 + h) * (n + 1) / 2) / 2 <= n1
        #          h <= sqrt(4 * n1) - 1
        oddHeight = math.isqrt(4 * n1) - 1
        #          2 + 4 + ... + h <= n2
        # ((2 + h) * h / 2) / 2 <= n2
        #          h <= sqrt(4 * n2 + 1) - 1
        evenHeight = math.isqrt(4 * n2 + 1) - 1
        # If the difference between the odd and even heights is >= 1, we can add an
        # extra level to the minimum height.
        return min(oddHeight, evenHeight) + (1 if abs(oddHeight - evenHeight) >= 1
                                              else 0)
```

2139. 3201.py

Path: LeetCode-main/solutions/3201. Find the Maximum Length of Valid Subsequence I/3201.py

```
class Solution:
    def maximumLength(self, nums: list[int]) -> int:
        # dp[i][j] := the maximum length of a valid subsequence, where the last
        # number mod k equal to i and the next desired number mod k equal to j
        dp = [[0] * 2 for _ in range(2)]

        # Extend the pattern xyxy...xy.
        for x in nums:
            for y in range(2):
                dp[x % 2][y] = dp[y][x % 2] + 1

        return max(map(max, dp))
```

2140. 3202.py

Path: LeetCode-main/solutions/3202. Find the Maximum Length of Valid Subsequence II/3202.py


```
class Solution:
    # Similar to 3201. Find the Maximum Length of Valid Subsequence I
    def maximumLength(self, nums: list[int], k: int) -> int:
        # dp[i][j] := the maximum length of a valid subsequence, where the last
        # number mod k equal to i and the next desired number mod k equal to j
        dp = [[0] * k for _ in range(k)]

        # Extend the pattern xyxyxy...xy.
        for x in nums:
            for y in range(k):
                dp[x % k][y] = dp[y][x % k] + 1

        return max(map(max, dp))
```

2141. 3203.py

Path: LeetCode-main/solutions/3203. Find Minimum Diameter After Merging Two Trees/3203.py

```

class Solution:
    def minimumDiameterAfterMerge(
        self,
        edges1: list[list[int]],
        edges2: list[list[int]],
    ) -> int:
        diameter1 = self._getDiameter(edges1)
        diameter2 = self._getDiameter(edges2)
        combinedDiameter = (diameter1 + 1) // 2 + (diameter2 + 1) // 2 + 1
        return max(diameter1, diameter2, combinedDiameter)

    def _getDiameter(self, edges: list[list[int]]) -> int:
        n = len(edges) + 1
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        maxDiameter = [0]
        self._maxDepth(graph, 0, -1, maxDiameter)
        return maxDiameter[0]

# Similar to 1522. Diameter of N-Ary Tree
def _maxDepth(
    self,
    graph: list[list[int]],
    u: int,
    prev: int,
    maxDiameter: list[int],
) -> int:
    """Returns the maximum depth of the subtree rooted at u."""
    maxSubDepth1 = 0
    maxSubDepth2 = 0
    for v in graph[u]:
        if v == prev:
            continue
        maxSubDepth = self._maxDepth(graph, v, u, maxDiameter)
        if maxSubDepth > maxSubDepth1:
            maxSubDepth2 = maxSubDepth1
            maxSubDepth1 = maxSubDepth
        elif maxSubDepth > maxSubDepth2:
            maxSubDepth2 = maxSubDepth
    maxDiameter[0] = max(maxDiameter[0], maxSubDepth1 + maxSubDepth2)
    return 1 + maxSubDepth1

```

2142. 3205-2.py

Path: LeetCode-main/solutions/3205. Maximum Array Hopping Score I/3205-2.py

```

class Solution:
    def maxScore(self, nums: list[int]) -> int:
        # The optimal jump is the maximum number in the remaining suffix.
        return sum(itertools.accumulate(nums[:0:-1], max))

```

2143. 3205.py

Path: LeetCode-main/solutions/3205. Maximum Array Hopping Score I/3205.py

```
class Solution:
    def maxScore(self, nums: list[int]) -> int:
        n = len(nums)
        # dp[i] := the maximum score to jump from index i to n - 1
        dp = [0] * n

        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                # Jump from i to j, and then jump from j to n - 1.
                dp[i] = max(dp[i], (j - i) * nums[j] + dp[j])

        return dp[0]
```

2144. 3206.py

Path: LeetCode-main/solutions/3206. Alternating Groups I/3206.py

```
class Solution:
    def numberOfAlternatingGroups(self, colors: list[int]) -> int:
        n = len(colors)
        return sum(colors[i] != colors[i - 1] and
                    colors[i] != colors[(i + 1) % n]
                    for i in range(n))
```

2145. 3207.py

Path: LeetCode-main/solutions/3207. Maximum Points After Enemy Battles/3207.py

```
class Solution:
    def maximumPoints(self, enemyEnergies: list[int], currentEnergy: int) -> int:
        minEnergy = min(enemyEnergies)
        return (0 if currentEnergy < minEnergy
                else (currentEnergy + sum(enemyEnergies) - minEnergy) // minEnergy)
```

2146. 3208.py

Path: LeetCode-main/solutions/3208. Alternating Groups II/3208.py

```
class Solution:
    def numberOfAlternatingGroups(self, colors: list[int], k: int) -> int:
        n = len(colors)
        ans = 0
        alternating = 1

        for i in range(n + k - 2):
            alternating = (1 if colors[i % n] == colors[(i - 1) % n]
                           else alternating + 1)
            if alternating >= k:
                ans += 1

        return ans
```

2147. 3209.py

Path: LeetCode-main/solutions/3209. Number of Subarrays With AND Value of K/3209.py

```
class Solution:
    # Similar to 1521. Find a Value of a Mysterious Function Closest to Target
    def countSubarrays(self, nums: list[int], k: int) -> int:
        ans = 0
        # the counter of all the values of subarrays that end in the previous
        # number
        prev = collections.Counter()

        for num in nums:
            # Extend each subarray that ends in the previous number. Due to
            # monotonicity of the AND operation, the size of `curr` will be at most
            # num.bit_count() + 1.
            curr = collections.Counter({num: 1})
            for val, freq in prev.items():
                curr[val & num] += freq
            ans += curr[k]
            prev = curr

        return ans
```

2148. 321.py

Path: LeetCode-main/solutions/321. Create Maximum Number/321.py

```
class Solution:
    def maxNumber(self, nums1: list[int], nums2: list[int], k: int) -> list[int]:
        def maxArray(nums: list[int], k: int) -> list[int]:
            res = []
            toTop = len(nums) - k
            for num in nums:
                while res and res[-1] < num and toTop > 0:
                    res.pop()
                    toTop -= 1
                res.append(num)
            return res[:k]

        def merge(nums1: list[int], nums2: list[int]) -> list[int]:
            return [max(nums1, nums2).pop(0) for _ in nums1 + nums2]

        return max(merge(maxArray(nums1, i), maxArray(nums2, k - i))
                    for i in range(k + 1)
                    if i <= len(nums1) and k - i <= len(nums2))
```

2149. 3210.py

Path: LeetCode-main/solutions/3210. Find the Encrypted String/3210.py

```
class Solution:
    def getEncryptedString(self, s: str, k: int) -> str:
        k %= len(s)
        return s[k:] + s[0:k]
```

2150. 3211.py

Path: LeetCode-main/solutions/3211. Generate Binary Strings Without Adjacent Zeros/3211.py

```

class Solution:
    def validStrings(self, n: int) -> list[str]:
        ans = []

        def dfs(n: int, s: list[str]) -> None:
            if n == 0:
                ans.append(''.join(s))
                return
            if not s or s[-1] == '1':
                s.append('0')
                dfs(n - 1, s)
                s.pop()
            s.append('1')
            dfs(n - 1, s)
            s.pop()

        dfs(n, [])
        return ans

```

2151. 3212.py

Path: LeetCode-main/solutions/3212. Count Submatrices With Equal Frequency of X and Y/3212.py

```

class Solution:
    def numberOfSubmatrices(self, grid: list[list[str]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = 0
        # x[i][j] := the number of 'X' in grid[0..i][0..j]
        x = [[0] * (n + 1) for _ in range(m + 1)]
        # y[i][j] := the number of 'Y' in grid[0..i][0..j]
        y = [[0] * (n + 1) for _ in range(m + 1)]

        for i, row in enumerate(grid):
            for j, cell in enumerate(row):
                x[i + 1][j + 1] = (cell == 'X') + x[i][j + 1] + x[i + 1][j] - x[i][j]
                y[i + 1][j + 1] = (cell == 'Y') + y[i][j + 1] + y[i + 1][j] - y[i][j]
                if x[i + 1][j + 1] > 0 and x[i + 1][j + 1] == y[i + 1][j + 1]:
                    ans += 1

        return ans

```

2152. 3213.py

Path: LeetCode-main/solutions/3213. Construct String with Minimum Cost/3213.py

```

class Solution:
    def minimumCost(self, target: str, words: list[str], costs: list[int]) -> int:
        n = len(target)
        # dp[i] := the minimum cost to construct target[0..i)
        dp = [0] + [math.inf] * n
        # minCost[c][word] := the minimum cost to construct word starting with `c`
        minCost: dict[str, dict[str, int]] = collections.defaultdict(dict)

        for word, cost in zip(words, costs):
            c = word[0]
            minCost[c][word] = min(minCost[c].get(word, math.inf), cost)

        for i, c in enumerate(target):
            for word, cost in minCost[c].items():
                j = i + len(word)
                if j <= n and cost + dp[i] < dp[j] and target[i:j] == word:
                    dp[j] = cost + dp[i]

        return -1 if dp[n] == math.inf else dp[n]

```

2153. 3215.py

Path: LeetCode-main/solutions/3215. Count Triplets with Even XOR Set Bits II/3215.py

```

class Solution:
    # Same as 3199. Count Triplets with Even XOR Set Bits I
    def tripletCount(self, a: list[int], b: list[int], c: list[int]) -> int:
        evenA, oddA = self._getEvenOddBitCount(a)
        evenB, oddB = self._getEvenOddBitCount(b)
        evenC, oddC = self._getEvenOddBitCount(c)
        return evenA * oddB * oddC + oddA * evenB * oddC + oddA * oddB * evenC + evenA * evenB * evenC

    def _getEvenOddBitCount(self, nums: list[int]) -> tuple[int, int]:
        """
        Returns the count of numbers in the `nums` arrays that have even number of
        ones and odd number of ones in their binary representation.
        """
        even = sum(num.bit_count() % 2 == 0 for num in nums)
        return (even, len(nums) - even)

```

2154. 3216.py

Path: LeetCode-main/solutions/3216. Lexicographically Smallest String After a Swap/3216.py

```

class Solution:
    def getSmallestString(self, s: str) -> str:
        chars = list(s)
        for i, (a, b) in enumerate(itertools.pairwise(chars)):
            if ord(a) % 2 == ord(b) % 2 and a > b:
                chars[i], chars[i + 1] = chars[i + 1], chars[i]
        return ''.join(chars)

    return s

```

2155. 3217.py

Path: LeetCode-main/solutions/3217. Delete Nodes From Linked List Present in Array/3217.py

```

class Solution:
    def modifiedList(
        self,
        nums: list[int],
        head: ListNode | None,
    ) -> ListNode | None:
        dummy = ListNode(0, head)
        numsSet = set(nums)

        curr = dummy
        while curr.next:
            if curr.next.val in numsSet:
                curr.next = curr.next.next
            else:
                curr = curr.next

        return dummy.next

```

2156. 3218.py

Path: LeetCode-main/solutions/3218. Minimum Cost for Cutting Cake I/3218.py

```

class Solution:
    def minimumCost(
        self,
        m: int,
        n: int,
        horizontalCut: list[int],
        verticalCut: list[int],
    ) -> int:
        ans = 0
        sumH = sum(horizontalCut)
        sumV = sum(verticalCut)

        horizontalCut.sort()
        verticalCut.sort()

        while horizontalCut and verticalCut:
            if horizontalCut[-1] > verticalCut[-1]:
                ans += horizontalCut[-1] + sumV
                sumH -= horizontalCut.pop()
            else:
                ans += verticalCut[-1] + sumH
                sumV -= verticalCut.pop()

        return ans + sumH + sumV

```

2157. 3219.py

Path: LeetCode-main/solutions/3219. Minimum Cost for Cutting Cake II/3219.py

```

class Solution:
    # Same as 3218. Minimum Cost for Cutting Cake I
    def minimumCost(
        self,
        m: int,
        n: int,
        horizontalCut: list[int],
        verticalCut: list[int],
    ) -> int:
        ans = 0
        sumH = sum(horizontalCut)
        sumV = sum(verticalCut)

        horizontalCut.sort()
        verticalCut.sort()

        while horizontalCut and verticalCut:
            if horizontalCut[-1] > verticalCut[-1]:
                ans += horizontalCut[-1] + sumV
                sumH -= horizontalCut.pop()
            else:
                ans += verticalCut[-1] + sumH
                sumV -= verticalCut.pop()

        return ans + sumH + sumV

```

2158. 322.py

Path: LeetCode-main/solutions/322. Coin Change/322.py

```

class Solution:
    def coinChange(self, coins: list[int], amount: int) -> int:
        # dp[i] := the minimum number Of coins to make up i
        dp = [0] + [amount + 1] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)

        return -1 if dp[amount] == amount + 1 else dp[amount]

```

2159. 3221.py

Path: LeetCode-main/solutions/3221. Maximum Array Hopping Score II/3221.py

```

class Solution:
    # Same as 3205. Maximum Array Hopping Score I
    def maxScore(self, nums: list[int]) -> int:
        # The optimal jump is the maximum number in the remaining suffix.
        return sum(itertools.accumulate(nums[:0:-1], max))

```

2160. 3222.py

Path: LeetCode-main/solutions/3222. Find the Winning Player in Coin Game/3222.py


```
class Solution:
    def losingPlayer(self, x: int, y: int) -> str:
        return 'Bob' if min(x, y // 4) % 2 == 0 else 'Alice'
```

2161. 3223.py

Path: LeetCode-main/solutions/3223. Minimum Length of String After Operations/3223.py

```
class Solution:
    def minimumLength(self, s: str) -> int:
        count = collections.Counter(s)
        return sum(2 if freq % 2 == 0 else 1 for freq in count.values())
```

2162. 3224.py

Path: LeetCode-main/solutions/3224. Minimum Array Changes to Make Differences Equal/3224.py

```
class Solution:
    def minChanges(self, nums: list[int], k: int) -> int:
        pairSize = len(nums) // 2
        diffCount = collections.Counter() # {nums[-1 - i] - nums[i]: freq}
        # oneChangeCount[i] := the number of pairs that need only one change to
        # to achieve a difference of `i`
        oneChangeCount = [0] * (k + 1)

        for i in range(pairSize):
            a = nums[i]
            b = nums[-1 - i]
            diffCount[abs(a - b)] += 1
            oneChangeCount[max(a, b, k - a, k - b)] += 1

        # prefixOneChangeCount[i] := the number of pairs that need only one change
        # to achieve a difference >= `i`
        # prefixOneChangeCount[i] = sum(oneChangeCount[i..k])
        prefixOneChangeCount = list(
            itertools.accumulate(reversed(oneChangeCount)))[::-1]

        return min(prefixOneChangeCount[diff] - freq + # one change
                    (pairSize - prefixOneChangeCount[diff]) * 2 # two changes
                    for diff, freq in diffCount.items())
```

2163. 3225.py

Path: LeetCode-main/solutions/3225. Maximum Score From Grid Operations/3225.py

```

class Solution:
    def maximumScore(self, grid: list[list[int]]) -> int:
        n = len(grid)
        # prefix[j][i] := the sum of the first i elements in the j-th column
        prefix = [[0] * (n + 1) for _ in range(n)]
        # prevPick[i] := the maximum score up to the previous column, where the
        # bottommost selected element in the previous column is in row (i - 1)
        prevPick = [0] * (n + 1)
        # prevSkip[i] := the maximum score up to the previous column, where the
        # bottommost selected element in the column before the previous one is in
        # row (i - 1)
        prevSkip = [0] * (n + 1)

        for j in range(n):
            for i in range(n):
                prefix[j][i + 1] = prefix[j][i] + grid[i][j]

        for j in range(1, n):
            currPick = [0] * (n + 1)
            currSkip = [0] * (n + 1)
            # Consider all possible combinations of the number of current and
            # previous selected elements.
            for curr in range(n + 1): # the number of current selected elements
                for prev in range(n + 1): # the number of previous selected elements
                    if curr > prev:
                        # 1. The current bottom is deeper than the previous bottom.
                        # Get the score of grid[prev..curr][j - 1] for both pick and skip.
                        score = prefix[j - 1][curr] - prefix[j - 1][prev]
                        currPick[curr] = max(currPick[curr], prevSkip[prev] + score)
                        currSkip[curr] = max(currSkip[curr], prevSkip[prev] + score)
                    else:
                        # 2. The previous bottom is deeper than the current bottom.
                        # Get the score of grid[curr..prev][j] for pick only.
                        score = prefix[j][prev] - prefix[j][curr]
                        currPick[curr] = max(currPick[curr], prevPick[prev] + score)
                        currSkip[curr] = max(currSkip[curr], prevPick[prev])
            prevPick = currPick
            prevSkip = currSkip

        return max(prevPick)

```

2164. 3226.py

Path: LeetCode-main/solutions/3226. Number of Bit Changes to Make Two Integers Equal/3226.py

```

class Solution:
    def minChanges(self, n: int, k: int) -> int:
        # n needs to be a superset of k.
        return (n ^ k).bit_count() if (n & k) == k else -1

```

2165. 3227.py

Path: LeetCode-main/solutions/3227. Vowels Game in a String/3227.py

```

class Solution:
    def doesAliceWin(self, s: str) -> bool:
        # Let k be the number of vowels in s.
        # 1. If k == 0, Bob wins since Alice has no vowels to pick.
        # 2. If k % 2 == 1, Alice wins since Alice can pick the entire string.
        # 3. If k % 2 == 0, Alice wins since Alice can pick (k - 1) vowels,
        # then Bob will either pick a substring containing 0 vowels, resulting in
        # Alice picking the remaining entire string, or Bob couldn't pick at all
        # (the last vowel).
        VOWELS = 'aeiou'
        return any(c in VOWELS for c in s)

```

2166. 3228.py

Path: LeetCode-main/solutions/3228. Maximum Number of Operations to Move Ones to the End/3228.py

```

class Solution:
    def maxOperations(self, s: str) -> int:
        ans = 0
        ones = 0

        for i, c in enumerate(s):
            if c == '1':
                ones += 1
            elif i + 1 == len(s) or s[i + 1] == '1':
                ans += ones

        return ans

```

2167. 3229.py

Path: LeetCode-main/solutions/3229. Minimum Operations to Make Array Equal to Target/3229.py

```

class Solution:
    # Similar to 1526. Minimum Number of Increments on Subarrays to Form a Target Array
    def minimumOperations(self, nums: list[int], target: list[int]) -> int:
        ans = abs(nums[0] - target[0])

        for (prevNum, prevTarget), (currNum, currTarget) in (
            itertools.pairwise(zip(nums, target))
        ):
            currDiff = currTarget - currNum
            prevDiff = prevTarget - prevNum
            if currDiff >= 0 and prevDiff >= 0:
                ans += max(0, currDiff - prevDiff)
            elif currDiff <= 0 and prevDiff <= 0:
                ans += max(0, abs(currDiff) - abs(prevDiff))
            else:
                ans += abs(currDiff)

        return ans

```

2168. 323-2.py

Path: LeetCode-main/solutions/323. Number of Connected Components in an Undirected Graph/323-2.py

```
class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        seen = set()

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def dfs(u: int, seen: set[int]) -> None:
            for v in graph[u]:
                if v not in seen:
                    seen.add(v)
                    dfs(v, seen)

        for i in range(n):
            if i not in seen:
                seen.add(i)
                dfs(graph, i, seen)
                ans += 1

        return ans
```

2169. 323-3.py

Path: LeetCode-main/solutions/323. Number of Connected Components in an Undirected Graph/323-3.py

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        uf = UnionFind(n)

        for u, v in edges:
            uf.unionByRank(u, v)

        return uf.count
```

2170. 323.py

Path: LeetCode-main/solutions/323. Number of Connected Components in an Undirected Graph/323.py

```

class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        seen = set()

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def bfs(node: int, seen: set[int]) -> None:
            q = collections.deque([node])
            seen.add(node)

            while q:
                u = q.pop()
                for v in graph[u]:
                    if v not in seen:
                        q.append(v)
                        seen.add(v)

        for i in range(n):
            if i not in seen:
                bfs(i, seen)
                ans += 1

        return ans

```

2171. 3231.py

Path: LeetCode-main/solutions/3231. Minimum Number of Increasing Subsequence to Be Removed/3231.py

```

class Solution:
    def minOperations(self, nums: list[int]) -> int:
        return self._lengthOfLIS(nums[::-1])

    def _lengthOfLIS(self, nums: list[int]) -> int:
        # tails[i] := the minimum tail of all the increasing subsequences having
        # length i + 1
        tails = []
        for num in nums:
            if not tails or num >= tails[-1]:
                tails.append(num)
            else:
                tails[bisect.bisect_right(tails, num)] = num
        return len(tails)

```

2172. 3232.py

Path: LeetCode-main/solutions/3232. Find if Digit Game Can Be Won/3232.py

```

class Solution:
    def canAliceWin(self, nums: list[int]) -> bool:
        return sum(num if num < 10 else -num for num in nums) != 0

```

2173. 3233.py

Path: LeetCode-main/solutions/3233. Find the Count of Numbers Which Are Not Special/3233.py

```
class Solution:
    def nonSpecialCount(self, l: int, r: int) -> int:
        maxRoot = math.isqrt(r)
        isPrime = self._sieveEratosthenes(maxRoot + 1)
        specialCount = 0

        for num in range(2, math.isqrt(r) + 1):
            if isPrime[num] and l <= num**2 <= r:
                specialCount += 1

        return r - l + 1 - specialCount

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime
```

2174. 3234.py

Path: LeetCode-main/solutions/3234. Count the Number of Substrings With Dominant Ones/3234.py

```
class Solution:
    def numberOfSubstrings(self, s: str) -> int:
        ans = 0
        #  $z^2 + z = n$ .
        #  $\Rightarrow z^2 + z - n = 0$ .
        #  $\Rightarrow z = (-1 + \sqrt{1 + 4n}) / 2$ .
        maxZero = (-1 + math.sqrt(1 + 4 * len(s))) // 2

        # Iterate through all possible number of 0s.
        for zero in range(int(maxZero) + 1):
            lastInvalidPos = -1
            count = [0, 0]
            l = 0
            for r, c in enumerate(s):
                count[int(c)] += 1
                # Try to shrink the window to maintain the "minimum" length of the
                # valid substring.
                while l < r:
                    if s[l] == '0' and count[0] > zero:
                        count[0] -= 1 # Remove an extra '0'.
                        lastInvalidPos = l
                        l += 1
                    elif s[l] == '1' and count[1] - 1 >= zero * zero:
                        count[1] -= 1 # Remove an extra '1'.
                        l += 1
                    else:
                        break # Cannot remove more characters.
            if count[0] == zero and count[1] >= zero * zero:
                # Add valid substrings ending in s[r] to the answer. They are
                # s[lastInvalidPos + 1..r], s[lastInvalidPos + 2..r], ..., s[l..r].
                ans += l - lastInvalidPos

        return ans
```

2175. 3235.py

Path: LeetCode-main/solutions/3235. Check if the Rectangle Corner Is Reachable/3235.py

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def canReachCorner(self, X: int, Y: int, circles: list[list[int]]) -> bool:
        n = len(circles)
        # Add two virtual nodes, where node n represents (0, 0) and node n + 1
        # represents (X, Y).
        uf = UnionFind(n + 2)

        # Iterate through each circle.
        for i, (x, y, r) in enumerate(circles):
            # Union the current circle with the node (0, 0) if the circle overlaps
            # with the left or top edges.
            if x - r <= 0 or y + r >= Y:
                uf.unionByRank(i, n)
            # Union the current circle with the node (X, Y) if the circle overlaps
            # with the right or bottom edges.
            if x + r >= X or y - r <= 0:
                uf.unionByRank(i, n + 1)
            # Union the current circle with previous circles if they overlap.
            for j in range(i):
                x2, y2, r2 = circles[j]
                if (x - x2)**2 + (y - y2)**2 <= (r + r2)**2:
                    uf.unionByRank(i, j)

        # If nodes (0, 0) and (X, Y) are in the same union set, that means there's
        # a path of overlapping circles that connects the left or top edges to the
        # right or bottom edges, implying that (0, 0) cannot reach (X, Y).
        return uf.find(n) != uf.find(n + 1)
```

2176. 3237.py

Path: LeetCode-main/solutions/3237. Alt and Tab Simulation/3237.py


```

class Solution:
    def simulationResult(
        self,
        windows: list[int],
        queries: list[int],
    ) -> list[int]:
        ans = []
        seen = set()

        for query in reversed(queries):
            if query not in seen:
                ans.append(query)
                seen.add(query)

        for window in windows:
            if window not in seen:
                ans.append(window)
                seen.add(window)

        return ans

```

2177. 3238.py

Path: LeetCode-main/solutions/3238. Find the Number of Winning Players/3238.py

```

class Solution:
    def winningPlayerCount(self, n: int, pick: list[list[int]]) -> int:
        counts = [collections.Counter() for _ in range(n)]
        for player, color in pick:
            counts[player][color] += 1
        return sum(max(count.values(), default=0) > i
                    for i, count in enumerate(counts))

```

2178. 3239.py

Path: LeetCode-main/solutions/3239. Minimum Number of Flips to Make Binary Grid Palindromic I/3239.py

```

class Solution:
    def minFlips(self, grid: list[list[int]]) -> int:
        rowFlips = sum(row[i] != row[-1 - i]
                       for row in grid for i in range(len(row) // 2))
        colFlips = sum(col[i] != col[-1 - i] for col in zip(*grid)
                       for i in range(len(col) // 2))
        return min(rowFlips, colFlips)

```

2179. 324.py

Path: LeetCode-main/solutions/324. Wiggle Sort II/324.py

```

class Solution:
    def wiggleSort(self, nums: list[int]) -> None:
        n = len(nums)
        median = self._findKthLargest(nums, (n + 1) // 2)

        def A(i: int):
            return (1 + 2 * i) % (n | 1)

        i = 0
        j = 0
        k = n - 1

        while i <= k:
            if nums[A(i)] > median:
                nums[A(i)], nums[A(j)] = nums[A(j)], nums[A(i)]
                i, j = i + 1, j + 1
            elif nums[A(i)] < median:
                nums[A(i)], nums[A(k)] = nums[A(k)], nums[A(i)]
                k -= 1
            else:
                i += 1

        # Same as 215. Kth Largest Element in an Array
        def _findKthLargest(self, nums: list[int], k: int) -> int:
            def quickSelect(l: int, r: int, k: int) -> int:
                randIndex = random.randint(0, r - 1) + 1
                nums[randIndex], nums[r] = nums[r], nums[randIndex]
                pivot = nums[r]

                nextSwapped = l
                for i in range(l, r):
                    if nums[i] >= pivot:
                        nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                        nextSwapped += 1
                nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]

                count = nextSwapped - l + 1 # Number of nums >= pivot
                if count == k:
                    return nums[nextSwapped]
                if count > k:
                    return quickSelect(l, nextSwapped - 1, k)
                return quickSelect(nextSwapped + 1, r, k - count)

            return quickSelect(0, len(nums) - 1, k)

```

2180. 3240.py

Path: LeetCode-main/solutions/3240. Minimum Number of Flips to Make Binary Grid Palindromic II/3240.py

```

class Solution:
    def minFlips(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = 0
        middleOnes = 0
        mismatchedPairs = 0

        # Handle top-left, top-right, bottom-left, bottom-right cells.
        for i in range(m // 2):
            for j in range(n // 2):
                ones = (grid[i][j] + grid[i][n - 1 - j] +
                        grid[m - 1 - i][j] + grid[m - 1 - i][n - 1 - j])
                ans += min(ones, 4 - ones)

        # Handle the middle row if the number of m is odd.
        if m % 2 == 1:
            for j in range(n // 2):
                leftCell = grid[m // 2][j]
                rightCell = grid[m // 2][n - 1 - j]
                mismatchedPairs += leftCell ^ rightCell
                middleOnes += leftCell + rightCell

        # Handle the middle column if the number of columns is odd.
        if n % 2 == 1:
            for i in range(m // 2):
                topCell = grid[i][n // 2]
                bottomCell = grid[m - 1 - i][n // 2]
                mismatchedPairs += topCell ^ bottomCell
                middleOnes += topCell + bottomCell

        if mismatchedPairs == 0:
            # Since there's no mismatched pairs, middleOnes % 4 must be 0 or 2.
            if middleOnes % 4 == 2:
                ans += 2 # Flip two 1s to 0s.
        else:
            # Flip every mismatched pair 01 to 00 or 11. It doesn't matter.
            ans += mismatchedPairs

        # Handle the center cell if both dimensions are odd.
        if m % 2 == 1 and n % 2 == 1:
            ans += grid[m // 2][n // 2]

        return ans

```

2181. 3241.py

Path: LeetCode-main/solutions/3241. Time Taken to Mark All Nodes/3241.py

```

from dataclasses import dataclass

@dataclass
class Node:
    node: int = 0 # the node number
    time: int = 0 # the time taken to mark the entire subtree rooted at the node

class Top2:
    def __init__(self, top1: Node = Node(), top2: Node = Node()):
        # the direct child node, where the time taken to mark the entire subtree
        # rooted at the node is the maximum

```

```

self.top1 = top1
# the direct child node, where the time taken to mark the entire subtree
# rooted at the node is the second maximum
self.top2 = top2

class Solution:
    def timeTaken(self, edges: list[list[int]]) -> list[int]:
        n = len(edges) + 1
        ans = [0] * n
        tree = [[] for _ in range(n)]
        # dp[i] := the top two direct child nodes for subtree rooted at node i,
        # where each node contains the time taken to mark the entire subtree rooted
        # at the node itself
        dp = [Top2()] * n

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        self._dfs(tree, 0, -1, dp)
        self._reroot(tree, 0, -1, 0, dp, ans)
        return ans

    def _getTime(self, u: int) -> int:
        """Returns the time taken to mark node u."""
        return 2 if u % 2 == 0 else 1

    def _dfs(
        self,
        tree: list[list[int]],
        u: int,
        prev: int,
        dp: list[Top2]
    ) -> int:
        """
        Performs a DFS traversal of the subtree rooted at node `u`, computes the
        time taken to mark all nodes in the subtree, records the top two direct
        child nodes, where the time taken to mark the subtree rooted at each of the
        child nodes is maximized, and returns the top child node.

        These values are used later in the rerooting process.
        """
        top1 = Node()
        top2 = Node()
        for v in tree[u]:
            if v == prev:
                continue
            time = self._dfs(tree, v, u, dp) + self._getTime(v)
            if time >= top1.time:
                top2 = top1
                top1 = Node(v, time)
            elif time > top2.time:
                top2 = Node(v, time)
        dp[u] = Top2(top1, top2)
        return top1.time

    def _reroot(
        self,
        tree: list[list[int]],
        u: int,
        prev: int,
        maxTime: int,
        dp: list[Top2],
        ans: list[int]
    ) -> None:

```

```

"""
Reroots the tree at node `u` and updates the answer array, where `maxTime`
is the longest path that doesn't go through `u`'s subtree.
"""

ans[u] = max(maxTime, dp[u].top1.time)

for v in tree[u]:
    if v == prev:
        continue
    newMaxTime = self._getTime(u) + max(
        maxTime,
        dp[u].top2.time if dp[u].top1.node == v else dp[u].top1.time
    )
    self._reroot(tree, v, u, newMaxTime, dp, ans)

```

2182. 3242.py

Path: LeetCode-main/solutions/3242. Design Neighbor Sum Service/3242.py

```

class neighborSum:
    def __init__(self, grid: list[list[int]]):
        self.grid = grid
        self.n = len(grid)
        self.numToPos = {num: (i, j)
                           for i, row in enumerate(grid)
                           for j, num in enumerate(row)}

    def adjacentSum(self, value: int) -> int:
        i, j = self.numToPos[value]
        return sum(self.grid[x][y]
                    for x, y in ((i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1))
                    if 0 <= x < self.n and 0 <= y < self.n)

    def diagonalSum(self, value: int) -> int:
        i, j = self.numToPos[value]
        return sum(self.grid[x][y]
                    for x, y in ((i - 1, j - 1), (i - 1, j + 1),
                                (i + 1, j - 1), (i + 1, j + 1))
                    if 0 <= x < self.n and 0 <= y < self.n)

```

2183. 3243.py

Path: LeetCode-main/solutions/3243. Shortest Distance After Road Addition Queries I/3243.py

```

class Solution:
    def shortestDistanceAfterQueries(
        self,
        n: int,
        queries: list[list[int]],
    ) -> list[int]:
        ans = []
        dist = list(range(n))
        graph = [[] for _ in range(n)]

        for i in range(n - 1):
            graph[i].append(i + 1)

        for u, v in queries:
            graph[u].append(v)
            if dist[u] + 1 < dist[v]:
                dist[v] = dist[u] + 1
                self._bfs(graph, v, dist)
            ans.append(dist[n - 1])

        return ans

    def _bfs(self, graph: list[list[int]], start: int, dist: list[int]) -> None:
        """
        Performs a BFS to update the shortest distances from the given `start` node
        to all other reachable nodes in the graph. It updates the `dist` vector
        with the new shortest distances.
        """
        q = collections.deque([start])
        while q:
            u = q.popleft()
            for v in graph[u]:
                if dist[u] + 1 < dist[v]:
                    dist[v] = dist[u] + 1
                    q.append(v)

```

2184. 3244.py

Path: LeetCode-main/solutions/3244. Shortest Distance After Road Addition Queries II/3244.py

```

class Solution:
    def shortestDistanceAfterQueries(
        self,
        n: int,
        queries: list[list[int]],
    ) -> list[int]:
        ans = []
        nodeToFarthestNode = {i: i + 1 for i in range(n - 1)}

        for u, v in queries:
            # If `u` exists in the map and `v` is farther than the current farthest
            # node for `u`, we need to update the map and remove intermediate nodes.
            if u in nodeToFarthestNode and nodeToFarthestNode[u] < v:
                node = nodeToFarthestNode[u]
                while node < v:
                    node = nodeToFarthestNode.pop(node)
                nodeToFarthestNode[u] = v
            ans.append(len(nodeToFarthestNode))

        return ans

```

2185. 3247.py

Path: LeetCode-main/solutions/3247. Number of Subsequences with Odd Sum/3247.py

```
class Solution:
    def subsequenceCount(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        even = 0 # the number of subsequences with even sum
        odd = 0 # the number of subsequences with odd sum

        for num in nums:
            if num % 2 == 0:
                # Appending an even number to a subsequence doesn't change the parity.
                # The even number itself is also a valid subsequence.
                even, odd = even + even + 1, odd + odd
            else:
                # Appending an odd number to a subsequence changes the parity.
                # The odd number itself is also a valid subsequence.
                even, odd = even + odd, odd + even + 1

        return odd % MOD
```

2186. 3248.py

Path: LeetCode-main/solutions/3248. Snake in Matrix/3248.py

```
class Solution:
    def finalPositionOfSnake(self, n: int, commands: list[str]) -> int:
        directions = {
            "UP": (-1, 0),
            "RIGHT": (0, 1),
            "DOWN": (1, 0),
            "LEFT": (0, -1),
        }
        i = 0
        j = 0

        for command in commands:
            dx, dy = directions[command]
            i += dx
            j += dy

        return i * n + j
```

2187. 3249.py

Path: LeetCode-main/solutions/3249. Count the Number of Good Nodes/3249.py

```

class Solution:
    def countGoodNodes(self, edges: list[list[int]]) -> int:
        n = len(edges) + 1
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        ans = 0

        def dfs(u: int, prev: int) -> int:
            """Returns the size of the subtree rooted at u."""
            nonlocal ans
            size = 1
            childrenSizes = []
            for v in graph[u]:
                if v == prev:
                    continue
                child_size = dfs(v, u)
                size += child_size
                childrenSizes.append(child_size)

            if not childrenSizes or all(s == childrenSizes[0]
                                       for s in childrenSizes):
                ans += 1

            return size

        dfs(0, -1)
        return ans

```

2188. 325.py

Path: LeetCode-main/solutions/325. Maximum Size Subarray Sum Equals k/325.py

```

class Solution:
    def maxSubArrayLen(self, nums: list[int], k: int) -> int:
        ans = 0
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += num
            target = prefix - k
            if target in prefixToIndex:
                ans = max(ans, i - prefixToIndex[target])
            if prefix not in prefixToIndex:
                prefixToIndex[prefix] = i

        return ans

```

2189. 3250.py

Path: LeetCode-main/solutions/3250. Find the Count of Monotonic Pairs I/3250.py


```

class Solution:
    def countOfPairs(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        MAX = 1000
        n = len(nums)
        # dp[i][num] := the number of valid ways to fill the arrays up to index i
        # with arr1[i] = num
        dp = [[0] * (MAX + 1) for _ in range(n)]

        for num in range(nums[0] + 1):
            dp[0][num] = 1

        for i in range(1, n):
            ways = 0
            prevNum = 0
            # To satisfy arr1, prevNum <= num.
            # To satisfy arr2, nums[i - 1] - prevNum >= nums[i] - num.
            # => prevNum <= min(num, num - (nums[i] - nums[i - 1])).
            # As we move from `num` to `num + 1`, the range of valid `prevNum` values
            # becomes prevNum <= min(num + 1, num + 1 - (nums[i] - nums[i - 1])).
            # Since the range of `prevNum` can only increase by at most 1, there's
            # no need to iterate through all possible values of `prevNum`. We can
            # simply increment `prevNum` by 1 if it meets the condition.
            for num in range(nums[i] + 1):
                if prevNum <= min(num, num - (nums[i] - nums[i - 1])):
                    ways = (ways + dp[i - 1][prevNum]) % MOD
                    prevNum += 1
            dp[i][num] = ways

        return sum(dp[n - 1]) % MOD

```

2190. 3251.py

Path: LeetCode-main/solutions/3251. Find the Count of Monotonic Pairs II/3251.py

```

class Solution:
    # Same as 3250. Find the Count of Monotonic Pairs I
    def countOfPairs(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        MAX = 1000
        n = len(nums)
        # dp[i][num] := the number of valid ways to fill the arrays up to index i
        # with arr1[i] = num
        dp = [[0] * (MAX + 1) for _ in range(n)]

        for num in range(nums[0] + 1):
            dp[0][num] = 1

        for i in range(1, n):
            ways = 0
            prevNum = 0
            # To satisfy arr1, prevNum <= num.
            # To satisfy arr2, nums[i - 1] - prevNum >= nums[i] - num.
            # => prevNum <= min(num, num - (nums[i] - nums[i - 1])).
            # As we move from `num` to `num + 1`, the range of valid `prevNum` values
            # becomes prevNum <= min(num + 1, num + 1 - (nums[i] - nums[i - 1])).
            # Since the range of `prevNum` can only increase by at most 1, there's
            # no need to iterate through all possible values of `prevNum`. We can
            # simply increment `prevNum` by 1 if it meets the condition.
            for num in range(nums[i] + 1):
                if prevNum <= min(num, num - (nums[i] - nums[i - 1])):
                    ways = (ways + dp[i - 1][prevNum]) % MOD
                    prevNum += 1
            dp[i][num] = ways

        return sum(dp[n - 1]) % MOD

```

2191. 3253.py

Path: LeetCode-main/solutions/3253. Construct String with Minimum Cost (Easy)/3253.py

```

class Solution:
    def minimumCost(self, target: str, words: list[str], costs: list[int]) -> int:
        n = len(target)
        # dp[i] := the minimum cost to construct target[0:i]
        dp = [0] + [math.inf] * n

        for i in range(1, n + 1):
            for j, (word, cost) in enumerate(zip(words, costs)):
                if (i >= len(word) and
                    target[i - len(word):i] == word and
                    dp[i - len(word)] != math.inf):
                    dp[i] = min(dp[i], dp[i - len(word)] + cost)

        return dp[n] if dp[n] != math.inf else -1

```

2192. 3254.py

Path: LeetCode-main/solutions/3254. Find the Power of K-Size Subarrays I/3254.py

```

class Solution:
    def resultsArray(self, nums: list[int], k: int) -> list[int]:
        ans = []
        start = 0

        for i, num in enumerate(nums):
            if i > 0 and num != nums[i - 1] + 1:
                start = i
            if i >= k - 1:
                ans.append(num if i - start + 1 >= k else -1)

        return ans

```

2193. 3255.py

Path: LeetCode-main/solutions/3255. Find the Power of K-Size Subarrays II/3255.py

```

class Solution:
    # Same as 3254. Find the Power of K-Size Subarrays I
    def resultsArray(self, nums: list[int], k: int) -> list[int]:
        ans = []
        start = 0

        for i, num in enumerate(nums):
            if i > 0 and num != nums[i - 1] + 1:
                start = i
            if i >= k - 1:
                ans.append(num if i - start + 1 >= k else -1)

        return ans

```

2194. 3256.py

Path: LeetCode-main/solutions/3256. Maximum Value Sum by Placing Three Rooks I/3256.py

```

class Solution:
    def maximumValueSum(self, board: list[list[int]]) -> int:
        rows = [heapq.nlargest(3, [(val, i, j)
                                   for j, val in enumerate(row)])
                 for i, row in enumerate(board)]
        cols = [heapq.nlargest(3, [(val, i, j)
                                   for i, val in enumerate(col)])
                 for j, col in enumerate(zip(*board))]
        topNine = heapq.nlargest(9,
                                  set(itertools.chain(*rows)) &
                                  set(itertools.chain(*cols)))
        return max(
            (val1 + val2 + val3 for
             (val1, i1, j1),
             (val2, i2, j2),
             (val3, i3, j3) in (itertools.combinations(topNine, 3))
             if len({i1, i2, i3}) == 3 and len({j1, j2, j3}) == 3))

```

2195. 3257.py

Path: LeetCode-main/solutions/3257. Maximum Value Sum by Placing Three Rooks II/3257.py

```

class Solution:
    # Same as 3256. Maximum Value Sum by Placing Three Rooks I
    def maximumValueSum(self, board: list[list[int]]) -> int:
        rows = [heapq.nlargest(3, [(val, i, j)
                                   for j, val in enumerate(row)])
                 for i, row in enumerate(board)]
        cols = [heapq.nlargest(3, [(val, i, j)
                                   for i, val in enumerate(col)])
                 for j, col in enumerate(zip(*board))]
        topNine = heapq.nlargest(9,
                                  set(itertools.chain(*rows)) &
                                  set(itertools.chain(*cols)))

        return max(
            (val1 + val2 + val3 for
             (val1, i1, j1),
             (val2, i2, j2),
             (val3, i3, j3) in (itertools.combinations(topNine, 3))
             if len({i1, i2, i3}) == 3 and len({j1, j2, j3}) == 3))

```

2196. 3258.py

Path: LeetCode-main/solutions/3258. Count Substrings That Satisfy K-Constraint I/3258.py

```

class Solution:
    def countKConstraintSubstrings(self, s: str, k: int) -> int:
        ans = 0
        count = [0, 0]

        l = 0
        for r, c in enumerate(s):
            count[int(c)] += 1
            while min(count) > k:
                count[int(s[l])] -= 1
                l += 1
            ans += r - l + 1

        return ans

```

2197. 3259.py

Path: LeetCode-main/solutions/3259. Maximum Energy Boost From Two Drinks/3259.py

```

class Solution:
    def maxEnergyBoost(
        self,
        energyDrinkA: list[int],
        energyDrinkB: list[int]
    ) -> int:
        dpA = 0 # the maximum energy boost if the last drink is A
        dpB = 0 # the maximum energy boost if the last drink is B

        for a, b in zip(energyDrinkA, energyDrinkB):
            dpA, dpB = max(dpB, dpA + a), max(dpA, dpB + b)

        return max(dpA, dpB)

```

2198. 326.py

Path: LeetCode-main/solutions/326. Power of Three/326.py

```
class Solution:
    def isPowerOfThree(self, n: int) -> bool:
        return n > 0 and 3**19 % n == 0
```

2199. 3260.py

Path: LeetCode-main/solutions/3260. Find the Largest Palindrome Divisible by K/3260.py

```
class Solution:
    def largestPalindrome(self, n: int, k: int) -> str:
        match k:
            case 1:
                return '9' * n
            case 2:
                return '8' * n if n <= 2 else '8' + '9' * (n - 2) + '8'
            case 3 | 9:
                return '9' * n
            case 4:
                return '8' * n if n <= 4 else '88' + '9' * (n - 4) + '88'
            case 5:
                return '5' * n if n <= 2 else '5' + '9' * (n - 2) + '5'
            case 6:
                if n <= 2:
                    return '6' * n
                elif n % 2 == 1:
                    l = n // 2 - 1
                    return '8' + '9' * l + '8' + '9' * l + '8'
                else:
                    l = n // 2 - 2
                    return '8' + '9' * l + '77' + '9' * l + '8'
            case 8:
                return '8' * n if n <= 6 else '888' + '9' * (n - 6) + '888'
            case _:
                middle = {
                    0: '', 1: '7', 2: '77', 3: '959', 4: '9779', 5: '99799',
                    6: '999999', 7: '9994999', 8: '99944999', 9: '999969999',
                    10: '9999449999', 11: '99999499999'
                }
                q, r = divmod(n, 12)
                return '999999' * q + middle[r] + '999999' * q
```

2200. 3261.py

Path: LeetCode-main/solutions/3261. Count Substrings That Satisfy K-Constraint II/3261.py

```

class Solution:
    def countKConstraintSubstrings(
        self,
        s: str,
        k: int,
        queries: list[list[int]]
    ) -> list[int]:
        n = len(s)
        ans = []
        count = [0, 0]
        # leftToRight[l] := the maximum right index r s.t. s[l..r] is valid
        leftToRight = [0] * n
        # rightToLeft[r] := the minimum left index l s.t. s[l..r] is valid
        rightToLeft = [0] * n

        l = 0
        for r in range(n):
            count[int(s[r])] += 1
            while min(count) > k:
                count[int(s[l])] -= 1
                l += 1
            rightToLeft[r] = l

        count = [0, 0]
        r = n - 1
        for l in reversed(range(n)):
            count[int(s[l])] += 1
            while min(count) > k:
                count[int(s[r])] -= 1
                r -= 1
            leftToRight[l] = r

        # prefix[i] := the number of valid substrings ending in [0..i - 1].
        prefix = list(itertools.accumulate((r - l + 1
                                            for r, l in enumerate(rightToLeft)),
                                            initial=0))

        for l, r in queries:
            if r > leftToRight[l]:
                # If r is beyond leftToRight[l], compute the number of valid substrings
                # from l to leftToRight[l] and add the number of valid substrings
                # ending in [leftToRight[l] + 1..r].
                #
                # prefix[r + 1] := the number of valid substrings ending in [0..r].
                # prefix[leftToRight[l] + 1] := the number of valid substrings ending
                # in [0..leftToRight[l]].
                # => prefix[r + 1] - prefix[leftToRight[l] + 1] := the number of valid
                # substrings ending in [leftToRight[l] + 1..r].
                sz = leftToRight[l] - l + 1
                numValidSubstrings = sz * (sz + 1) // 2 + (
                    prefix[r + 1] - prefix[leftToRight[l] + 1])
            else:
                # If r is within the range of leftToRight[l], compute the number of
                # valid substrings directly from l to r.
                sz = r - l + 1
                numValidSubstrings = sz * (sz + 1) // 2
            ans.append(numValidSubstrings)

        return ans

```

Path: LeetCode-main/solutions/3263. Convert Doubly Linked List to Array I/3263.py

```
class Solution:
    def toArray(self, head: 'Optional[Node]') -> list[int]:
        ans = []
        curr = head

        while curr:
            ans.append(curr.val)
            curr = curr.next

        return ans
```

2202. 3264.py

Path: LeetCode-main/solutions/3264. Final Array State After K Multiplication Operations I/3264.py

```
class Solution:
    def getFinalState(
        self,
        nums: list[int],
        k: int,
        multiplier: int
    ) -> list[int]:
        ans = [0] * len(nums)
        minHeap = [(num, i) for i, num in enumerate(nums)]
        heapq.heapify(minHeap)

        for _ in range(k):
            num, i = heapq.heappop(minHeap)
            heapq.heappush(minHeap, (num * multiplier, i))

        for num, i in minHeap:
            ans[i] = num

        return ans
```

2203. 3265.py

Path: LeetCode-main/solutions/3265. Count Almost Equal Pairs I/3265.py

```
class Solution:
    def countPairs(self, nums: list[int]) -> int:
        ans = 0
        count = collections.Counter()
        maxLen = len(str(max(nums)))

        for num in nums:
            digits = list(str(num).zfill(maxLen))
            for swap in self._getSwaps(digits):
                ans += count[swap]
            count[num] += 1

        return ans

    def _getSwaps(self, digits: str) -> set[int]:
        """Returns all possible numbers after 1 swap."""
        n = len(digits)
        swaps = set([int(''.join(digits))])

        for i, j in itertools.combinations(range(n), 2):
            newDigits = digits[:]
            newDigits[i], newDigits[j] = newDigits[j], newDigits[i]
            swaps.add(int(''.join(newDigits)))

        return swaps
```

2204. 3266.py

Path: LeetCode-main/solutions/3266. Final Array State After K Multiplication Operations II/3266.py


```

class Solution:
    def getFinalState(
        self,
        nums: list[int],
        k: int,
        multiplier: int
    ) -> list[int]:
        if multiplier == 1:
            return nums

        MOD = 1_000_000_007
        n = len(nums)
        maxNum = max(nums)
        ans = [0] * n
        minHeap = [(num, i) for i, num in enumerate(nums)]

        heapq.heapify(minHeap)

        # Keep multiplying the minimum number as close as possible to the maximum
        # number in the array. After that, stop multiplying the minimum number
        # because it will be greater than the maximum number in the array and break
        # the circularity.
        while k > 0 and minHeap[0][0] * multiplier <= maxNum:
            num, i = heapq.heappop(minHeap)
            heapq.heappush(minHeap, (num * multiplier, i))
            k -= 1

        sortedIndexedNums = sorted(minHeap)
        multipliesPerNum, remainingK = divmod(k, n)

        # Evenly distribute the remaining multiplications to each number by using
        # fast exponentiation.
        for index, (num, i) in enumerate(sortedIndexedNums):
            sortedIndexedNums[index] = (
                sortedIndexedNums[index][0] *
                pow(multiplier, multipliesPerNum, MOD) % MOD, i)

        # Distribute the remaining multiplications to the minimum `remainingK`
        # numbers.
        for index in range(remainingK):
            sortedIndexedNums[index] = (
                sortedIndexedNums[index][0] * multiplier % MOD,
                sortedIndexedNums[index][1])

        for num, i in sortedIndexedNums:
            ans[i] = num

        return ans

```

2205. 3267.py

Path: LeetCode-main/solutions/3267. Count Almost Equal Pairs II/3267.py

```

class Solution:
    # Similar to 3265. Count Almost Equal Pairs I
    def countPairs(self, nums: list[int]) -> int:
        ans = 0
        count = collections.Counter()
        maxLen = len(str(max(nums)))

        for num in nums:
            digits = list(str(num).zfill(maxLen))
            for swap in self._getSwaps(digits):
                ans += count[swap]
            count[num] += 1

        return ans

    def _getSwaps(self, digits: str) -> set[int]:
        """Returns all possible numbers after 1 or 2 swaps."""
        n = len(digits)
        swaps = set([int(''.join(digits))])

        # Add all numbers after 1 swap.
        for i, j in itertools.combinations(range(n), 2):
            newDigits = digits[:]
            newDigits[i], newDigits[j] = newDigits[j], newDigits[i]
            swaps.add(int(''.join(newDigits)))

        # Add all numbers after 2 swaps.
        for (i1, j1), (i2, j2) in itertools.combinations(
            itertools.combinations(range(n), 2), 2):
            newDigits = digits[:]
            newDigits[i1], newDigits[j1] = newDigits[j1], newDigits[i1]
            newDigits[i2], newDigits[j2] = newDigits[j2], newDigits[i2]
            swaps.add(int(''.join(newDigits)))

        return swaps

```

2206. 3269.py

Path: LeetCode-main/solutions/3269. Constructing Two Increasing Arrays/3269.py

```

class Solution:
    def minLargest(self, nums1: list[int], nums2: list[int]) -> int:
        m = len(nums1)
        n = len(nums2)

        # dp[i][j] := the minimum largest number for the first i nums1 and the
        # first j nums2
        dp = [[math.inf] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = 0

        def f(prev: int, num: int) -> int:
            """
            Returns the next number to fill in the array based on the previous number
            and the current number.
            """
            return prev + (2 if prev % 2 == num else 1)

        for i in range(m + 1):
            for j in range(n + 1):
                if i > 0:
                    dp[i][j] = min(dp[i][j], f(dp[i - 1][j], nums1[i - 1]))
                if j > 0:
                    dp[i][j] = min(dp[i][j], f(dp[i][j - 1], nums2[j - 1]))

        return dp[m][n]

```

2207. 327.py

Path: LeetCode-main/solutions/327. Count of Range Sum/327.py

```

class Solution:
    def countRangeSum(self, nums: list[int], lower: int, upper: int) -> int:
        n = len(nums)
        self.ans = 0
        prefix = list(itertools.accumulate(nums, initial=0))

        self._mergeSort(prefix, 0, n, lower, upper)
        return self.ans

    def _mergeSort(
        self,
        prefix: list[int],
        l: int,
        r: int,
        lower: int,
        upper: int,
    ) -> None:
        if l >= r:
            return

        m = (l + r) // 2
        self._mergeSort(prefix, l, m, lower, upper)
        self._mergeSort(prefix, m + 1, r, lower, upper)
        self._merge(prefix, l, m, r, lower, upper)

    def _merge(
        self,
        prefix: list[int],
        l: int,
        m: int,
        r: int,
        lower: int,
        upper: int,
    ) -> None:
        # Implementation of the merge step for counting range sums
        # This part is partially visible in the image

```

```

) -> None:
lo = m + 1 # the first index s.t. prefix[lo] - prefix[i] >= lower
hi = m + 1 # the first index s.t. prefix[hi] - prefix[i] > upper

# For each index i in range [l, m], add hi - lo to `ans`.
for i in range(l, m + 1):
    while lo <= r and prefix[lo] - prefix[i] < lower:
        lo += 1
    while hi <= r and prefix[hi] - prefix[i] <= upper:
        hi += 1
    self.ans += hi - lo

sorted = [0] * (r - l + 1)
k = 0 # sorted's index
i = l # left's index
j = m + 1 # right's index

while i <= m and j <= r:
    if prefix[i] < prefix[j]:
        sorted[k] = prefix[i]
        k += 1
        i += 1
    else:
        sorted[k] = prefix[j]
        k += 1
        j += 1

# Put the possible remaining left part into the sorted array.
while i <= m:
    sorted[k] = prefix[i]
    k += 1
    i += 1

# Put the possible remaining right part into the sorted array.
while j <= r:
    sorted[k] = prefix[j]
    k += 1
    j += 1

prefix[l:l + len(sorted)] = sorted

```

2208. 3270.py

Path: LeetCode-main/solutions/3270. Find the Key of the Numbers/3270.py

```

class Solution:
    def generateKey(self, num1: int, num2: int, num3: int) -> int:
        return int(''.join(min(a, b, c)
                                for a, b, c in zip(str(num1).zfill(4),
                                                    str(num2).zfill(4),
                                                    str(num3).zfill(4))))

```

2209. 3271.py

Path: LeetCode-main/solutions/3271. Hash Divided String/3271.py

```
class Solution:
    def stringHash(self, s: str, k: int) -> str:
        ans = []

        for i in range(0, len(s), k):
            sumHash = sum(string.ascii_lowercase.index(s[j])
                           for j in range(i, i + k))
            ans.append(string.ascii_lowercase[sumHash % 26])

        return ''.join(ans)
```

2210. 3272.py

Path: LeetCode-main/solutions/3272. Find the Count of Good Integers/3272.py

```
class Solution:
    def countGoodIntegers(self, n: int, k: int) -> int:
        halfLength = (n + 1) // 2
        minHalf = 10**(halfLength - 1)
        maxHalf = 10**halfLength
        ans = 0
        seen = set()

        for num in range(minHalf, maxHalf):
            palindrome = str(num) + str(num)[::-1][n % 2:]
            sortedDigits = ''.join(sorted(palindrome))
            if int(palindrome) % k != 0 or sortedDigits in seen:
                continue
            seen.add(sortedDigits)
            digitCount = collections.Counter(palindrome)
            # Leading zeros are not allowed, so the first digit is special.
            firstDigitChoices = n - digitCount['0']
            permutations = firstDigitChoices * math.factorial(n - 1)
            # For each repeated digit, divide by the factorial of the frequency since
            # permutations that swap identical digits don't create a new number.
            for freq in digitCount.values():
                permutations //= math.factorial(freq)
            ans += permutations

        return ans
```

2211. 3273.py

Path: LeetCode-main/solutions/3273. Minimum Amount of Damage Dealt to Bob/3273.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Enemy:
    damage: int
    timeTakenDown: int

class Solution:
    def minDamage(self, power: int, damage: list[int], health: list[int]) -> int:
        ans = 0
        sumDamage = sum(damage)
        enemies = [Enemy(d, (h + power - 1) // power)
                    for d, h in zip(damage, health)]

        # It's better to take down the enemy i first if the damage dealt of taking
        # down i first is less than the damage dealt of taking down j first. So,
        #   damage[i] * t[i] + (t[i] + t[j]) * damage[j] <
        #   damage[j] * t[j] + (t[i] + t[j]) * damage[i]
        # => damage[i] * t[i] + damage[j] * t[i] + damage[j] * t[j] <
        #   damage[j] * t[j] + damage[i] * t[j] + damage[i] * t[i]
        # => damage[j] * t[i] < damage[i] * t[j]
        # => damage[j] / t[j] < damage[i] / t[i]
        enemies.sort(key=lambda x: -x.damage / x.timeTakenDown)

        for enemy in enemies:
            ans += sumDamage * enemy.timeTakenDown
            sumDamage -= enemy.damage

        return ans

```

2212. 3274.py

Path: LeetCode-main/solutions/3274. Check if Two Chessboard Squares Have the Same Color/3274.py

```

class Solution:
    def checkTwoChessboards(self, coordinate1: str, coordinate2: str) -> bool:
        # Same as 1812. Determine Color of a Chessboard Square
        def squareIsWhite(coordinate: str) -> bool:
            letter, digit = coordinate
            return ord(letter) % 2 != int(digit) % 2

        return squareIsWhite(coordinate1) == squareIsWhite(coordinate2)

```

2213. 3275.py

Path: LeetCode-main/solutions/3275. K-th Nearest Obstacle Queries/3275.py

```

class Solution:
    def resultsArray(self, queries: list[list[int]], k: int) -> list[int]:
        ans = []
        maxHeap = []

        for x, y in queries:
            heapq.heappush(maxHeap, -(abs(x) + abs(y)))
            if len(maxHeap) > k:
                heapq.heappop(maxHeap)
            ans.append(-maxHeap[0] if len(maxHeap) == k else -1)

        return ans

```

2214. 3276.py

Path: LeetCode-main/solutions/3276. Select Cells in Grid With Maximum Score/3276.py

```

class Solution:
    def maxScore(self, grid: list[list[int]]) -> int:
        numToIndices = collections.defaultdict(set)
        for index, row in enumerate(grid):
            for num in row:
                numToIndices[num].add(index)
        numToIndices = list(numToIndices.items())

        @functools.lru_cache(None)
        def dp(i: int, mask: int) -> int:
            """
            Returns the maximum score by selecting numbers from numToIndices[i..],
            where `mask` is the bitmask of the used row indices.
            """
            if i == len(numToIndices):
                return 0
            # Skip numToIndices[i][0].
            res = dp(i + 1, mask)
            for index in numToIndices[i][1]:
                if (mask >> index & 1) == 0:
                    # Take numToIndices[i][0].
                    res = max(res, numToIndices[i][0] + dp(i + 1, mask | 1 << index))
            return res

        return dp(0, 0)

```

2215. 3277.py

Path: LeetCode-main/solutions/3277. Maximum XOR Score Subarray Queries/3277.py

```
class Solution:
    def maximumSubarrayXor(
        self,
        nums: list[int],
        queries: list[list[int]]
    ) -> list[int]:
        n = len(nums)
        # xors[i][j] := the XOR score of nums[i..j]
        xors = [[0] * n for _ in range(n)]
        # dp[i][j] := the maximum XOR score of nums[i..j]
        dp = [[0] * n for _ in range(n)]

        for i, num in enumerate(nums):
            xors[i][i] = num
            dp[i][i] = num

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                xors[i][j] = xors[i][j - 1] ^ xors[i + 1][j]
                dp[i][j] = max(xors[i][j], dp[i][j - 1], dp[i + 1][j])

        return [dp[l][r] for l, r in queries]
```

2216. 3279.py

Path: LeetCode-main/solutions/3279. Maximum Total Area Occupied by Pistons/3279.py


```
from sortedcontainers import SortedDict

class Solution:
    def maxArea(self, height: int, positions: list[int], directions: str) -> int:
        area = sum(positions)
        ans = area
        diffPerSecond = 0
        timeToIndices: SortedDict[int, list[int]] = SortedDict()

        for i, (position, direction) in enumerate(zip(positions, directions)):
            if direction == 'U':
                timeToIndices.setdefault(height - position, []).append(i)
                timeToIndices.setdefault(height - position + height, []).append(i)
                diffPerSecond += 1
            else:
                timeToIndices.setdefault(position, []).append(i)
                timeToIndices.setdefault(position + height, []).append(i)
                diffPerSecond -= 1

        prevTime = 0
        directionsList = list(directions)

        for time, indices in timeToIndices.items():
            area += (time - prevTime) * diffPerSecond
            ans = max(ans, area)
            prevTime = time
            for i in indices:
                if directionsList[i] == 'U':
                    directionsList[i] = 'D'
                    diffPerSecond -= 2
                else:
                    directionsList[i] = 'U'
                    diffPerSecond += 2

        return ans
```

2217. 328.py

Path: LeetCode-main/solutions/328. Odd Even Linked List/328.py

```

class Solution:
    def oddEvenList(self, head: ListNode) -> ListNode:
        oddHead = ListNode(0)
        evenHead = ListNode(0)
        odd = oddHead
        even = evenHead
        isOdd = True

        while head:
            if isOdd:
                odd.next = head
                odd = head
            else:
                even.next = head
                even = head
            head = head.next
            isOdd = not isOdd

        even.next = None
        odd.next = evenHead.next
        return oddHead.next

```

2218. 3280.py

Path: LeetCode-main/solutions/3280. Convert Date to Binary/3280.py

```

class Solution:
    def convertDateToBinary(self, date: str) -> str:
        year, month, day = map(int, date.split('-'))

        def toBinary(value: int) -> str:
            """Converts an integer to binary without leading zeros."""
            return bin(value)[2:]

        return '-'.join([toBinary(year), toBinary(month), toBinary(day)])

```

2219. 3281.py

Path: LeetCode-main/solutions/3281. Maximize Score of Numbers in Ranges/3281.py

```

class Solution:
    def maxPossibleScore(self, start: list[int], d: int) -> int:
        def isPossible(m: int) -> bool:
            lastPick = start[0]
            for i in range(1, len(start)):
                if lastPick + m > start[i] + d:
                    return False
                lastPick = max(lastPick + m, start[i])
            return True

        start.sort()

        maxScore = (start[-1] + d) - start[0] + 1
        l = bisect.bisect_left(range(maxScore), True,
                                key=lambda m: not isPossible(m))

        return l - 1

```

2220. 3282.py

Path: LeetCode-main/solutions/3282. Reach End of Array With Max Score/3282.py

```
class Solution:
    # Similar to 3205. Maximum Array Hopping Score I
    def findMaximumScore(self, nums: list[int]) -> int:
        return sum(itertools.accumulate(nums[:-1], max))
```

2221. 3283.py

Path: LeetCode-main/solutions/3283. Maximum Number of Moves to Kill All Pawns/3283.py

```
class Solution:
    def __init__(self):
        self.SIZE = 50
        self.MAX = 1_000_000
        self.DIRS = ((1, 2), (2, 1), (2, -1), (1, -2),
                     (-1, -2), (-2, -1), (-2, 1), (-1, 2))

    def maxMoves(self, kx: int, ky: int, positions: list[list[int]]) -> int:
        n = len(positions)
        positions.append([kx, ky])
        hashedPositionToIndex = {}
        # dist[i][j] := the minimum distance from positions[i] to positions[j]
        dist = [[0] * (n + 1) for _ in range(n + 1)]

        for i, (x, y) in enumerate(positions):
            hashedPositionToIndex[self._hash(x, y)] = i

        for sourceIndex in range(n + 1):
            self._bfs(positions, sourceIndex, hashedPositionToIndex, dist)

        MAX_MASK = 1 << (n + 1)
        # dp[i][mask][turn] := the maximum (Alice) or the minimum (Bob) cost to
        # kill all pawns, where i is the current pawn, mask is the set of pawns
        # that have been killed, and turn is the current player's turn (0 for Alice
        # and 1 for Bob)
        dp = [[[0, 0]
                for _ in range(1 << (n + 1))]
               for _ in range(n + 1)]

        for i in range(n + 1):
            for mask in range(MAX_MASK - 1):
                dp[i][mask] = [-self.MAX, self.MAX]

        for mask in range(MAX_MASK - 2, -1, -1):
            for i in range(n + 1):
                for turn in range(2):
                    for j in range(n):
                        if mask >> j & 1:
                            continue
                        moves = dist[i][j] + dp[j][mask | 1 << j][1 - turn]
                        dp[i][mask][turn] = (max(dp[i][mask][turn], moves) if turn == 0 else
                                              min(dp[i][mask][turn], moves))

        # Returns the maximum cost to kill all pawns, i.e., the original positions
        # array without the knight (kx, ky).
        return dp[n][1 << n][0]

    def _hash(self, x: int, y: int) -> int:
        return x * self.SIZE + y
```

```

def _bfs(
    self,
    positions: list[list[int]],
    sourceIndex: int,
    hashedPositionToIndex: dict[int, int],
    dist: list[list[int]]
) -> None:
    """
    Computes the distance between positions[sourceIndex] and other positions.
    """

    sx, sy = positions[sourceIndex]
    q = collections.deque([(sx, sy)])
    seen = {(sx, sy)}
    seenPositions = 0

    step = 0
    while q and seenPositions < len(positions):
        for _ in range(len(q)):
            i, j = q.popleft()
            hashedPosition = self._hash(i, j)
            if hashedPosition in hashedPositionToIndex:
                dist[sourceIndex][hashedPositionToIndex[hashedPosition]] = step
                seenPositions += 1
            for dx, dy in self.DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x >= self.SIZE or y < 0 or y >= self.SIZE:
                    continue
                if (x, y) in seen:
                    continue
                q.append((x, y))
                seen.add((x, y))
            step += 1

```

2222. 3284.py

Path: LeetCode-main/solutions/3284. Sum of Consecutive Subarrays/3284.py

```

class Solution:
    def getSum(self, nums: list[int]) -> int:
        def getSum(diff: int) -> int:
            """Returns the sum of all subarrays with a difference of `diff`."""
            res = nums[0]
            summ = nums[0]
            count = 1
            for prev, num in itertools.pairwise(nums):
                if num == prev + diff:
                    count += 1
                    summ += count * num
                else:
                    count = 1
                    summ = num
            res += summ
            return res

        return (getSum(1) + getSum(-1) - sum(nums)) % 1_000_000_007

```

2223. 3285.py

Path: LeetCode-main/solutions/3285. Find Indices of Stable Mountains/3285.py

```
class Solution:
    def stableMountains(self, height: list[int], threshold: int) -> list[int]:
        return [i for i in range(1, len(height))
                if height[i - 1] > threshold]
```

2224. 3286.py

Path: LeetCode-main/solutions/3286. Find a Safe Walk Through a Grid/3286.py

```
class Solution:
    def findSafeWalk(self, grid: list[list[int]], health: int) -> bool:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        initialHealth = health - grid[0][0]
        q = collections.deque([(0, 0, initialHealth)])
        seen = {(0, 0, initialHealth)}

        while q:
            for _ in range(len(q)):
                i, j, h = q.popleft()
                if i == m - 1 and j == n - 1 and h > 0:
                    return True
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    nextHealth = h - grid[x][y]
                    if nextHealth <= 0 or (x, y, nextHealth) in seen:
                        continue
                    q.append((x, y, nextHealth))
                    seen.add((x, y, nextHealth))

        return False
```

2225. 3287.py

Path: LeetCode-main/solutions/3287. Find the Maximum Sequence Value of Array/3287.py

```

class Solution:
    def maxValue(self, nums: list[int], k: int) -> int:
        left = self._getPossibleORs(nums, k)
        right = self._getPossibleORs(nums[::-1], k)[::-1]
        return max(a ^ b
                    for i in range(k - 1, len(nums) - k)
                    for a in range(128 + 1)
                    for b in range(128 + 1)
                    if left[i][k][a] and right[i + 1][k][b])

    def _getPossibleORs(self, nums: list[int], k: int) -> list[list[list[bool]]]:
        dp = [[[False] * (128 + 1)
                for _ in range(k + 1)]
               for _ in range(len(nums))]

        dp[0][1][nums[0]] = True

        for i in range(len(nums)):
            dp[i][0][0] = True

        for i in range(1, len(nums)):
            for j in range(1, k + 1):
                for x in range(128 + 1):
                    if dp[i - 1][j][x]:
                        dp[i][j][x] = True
                    if dp[i - 1][j - 1][x]:
                        dp[i][j][nums[i] | x] = True

        return dp

```

2226. 3288.py

Path: LeetCode-main/solutions/3288. Length of the Longest Increasing Path/3288.py

```

class Solution:
    def maxPathLength(self, coordinates: list[list[int]], k: int) -> int:
        xk, yk = coordinates[k]
        leftCoordinates = [(x, y) for x, y in coordinates if x < xk and y < yk]
        rightCoordinates = [(x, y) for x, y in coordinates if x > xk and y > yk]
        return (1 +
                self._lengthOfLIS(leftCoordinates) +
                self._lengthOfLIS(rightCoordinates))

    # Similar to 300. Longest Increasing Subsequence
    def _lengthOfLIS(self, coordinates: list[tuple[int, int]]) -> int:
        coordinates.sort(key=lambda x: (x[0], -x[1]))
        # tail[i] := the minimum tail of all the increasing subsequences having
        # length i + 1
        tail = []
        for _, y in coordinates:
            if not tail or y > tail[-1]:
                tail.append(y)
            else:
                tail[bisect.bisect_left(tail, y)] = y
        return len(tail)

```

2227. 3289.py

Path: LeetCode-main/solutions/3289. The Two Sneaky Numbers of Digitville/3289.py

```
class Solution:
    def getSneakyNumbers(self, nums: list[int]) -> list[int]:
        return [num for num, freq in collections.Counter(nums).items()
                if freq == 2]
```

2228. 329.py

Path: LeetCode-main/solutions/329. Longest Increasing Path in a Matrix/329.py

```
class Solution:
    def longestIncreasingPath(self, matrix: list[list[int]]) -> int:
        m = len(matrix)
        n = len(matrix[0])

        @functools.lru_cache(None)
        def dfs(i: int, j: int, prev: int) -> int:
            if i < 0 or i == m or j < 0 or j == n:
                return 0
            if matrix[i][j] <= prev:
                return 0

            curr = matrix[i][j]
            return 1 + max(dfs(i + 1, j, curr),
                          dfs(i - 1, j, curr),
                          dfs(i, j + 1, curr),
                          dfs(i, j - 1, curr))

        return max(dfs(i, j, -math.inf) for i in range(m) for j in range(n))
```

2229. 3290-2.py

Path: LeetCode-main/solutions/3290. Maximum Multiplication Score/3290-2.py

```
class Solution:
    def maxScore(self, a: list[int], b: list[int]) -> int:
        # dp[i] := the maximum score of a[0..i]
        dp = [-math.inf] * 4

        for num in b:
            for i in reversed(range(4)):
                # Skip `num` or pair a[i] with `num`.
                dp[i] = max(dp[i], (dp[i - 1] if i > 0 else 0) + a[i] * num)

        return dp[3]
```

2230. 3290.py

Path: LeetCode-main/solutions/3290. Maximum Multiplication Score/3290.py

```

class Solution:
    def maxScore(self, a: list[int], b: list[int]) -> int:
        n = len(b)
        # dp[i][j] := the maximum score by selecting 4 - i numbers from b[j..n - 1]
        # using the corresponding numbers from a[i..3]
        dp = [[0] * (n + 1) for _ in range(5)]

        # Run out of numbers in b but still need to select numbers from a.
        for i in range(4):
            dp[i][n] = -math.inf

        for i in reversed(range(4)):
            for j in reversed(range(n)):
                # Skip b[j] or pair a[i] with b[j].
                dp[i][j] = max(dp[i][j + 1], a[i] * b[j] + dp[i + 1][j + 1])

        return -1 if dp[0][0] == -math.inf else dp[0][0]

```

2231. 3291.py

Path: LeetCode-main/solutions/3291. Minimum Number of Valid Strings to Form Target I/3291.py

```

class Solution:
    def minValidStrings(self, words: list[str], target: str) -> int:
        ans = 0
        unmatchedPrefix = len(target)
        lpsList = [self._getLPS(word + '#' + target) for word in words]

        while unmatchedPrefix > 0:
            # Greedily choose the word that has the longest suffix match with the
            # remaining unmatched prefix.
            maxMatchSuffix = 0
            for lps, word in zip(lpsList, words):
                maxMatchSuffix = max(maxMatchSuffix, lps[len(word) + unmatchedPrefix])
            if maxMatchSuffix == 0:
                return -1
            ans += 1
            unmatchedPrefix -= maxMatchSuffix

        return ans

    def _getLPS(self, pattern: str) -> list[int]:
        """
        Returns the lps array, where lps[i] is the length of the longest prefix of
        pattern[0..i] which is also a suffix of this substring.
        """
        lps = [0] * len(pattern)
        j = 0
        for i in range(1, len(pattern)):
            while j > 0 and pattern[j] != pattern[i]:
                j = lps[j - 1]
            if pattern[i] == pattern[j]:
                lps[i] = j + 1
                j += 1
        return lps

```

2232. 3292.py

Path: LeetCode-main/solutions/3292. Minimum Number of Valid Strings to Form Target II/3292.py


```

class Solution:
    # 3291. Minimum Number of Valid Strings to Form Target I
    def minValidStrings(self, words: list[str], target: str) -> int:
        ans = 0
        unmatchedPrefix = len(target)
        lpsList = [self._getLPS(word + '#' + target) for word in words]

        while unmatchedPrefix > 0:
            # Greedily choose the word that has the longest suffix match with the
            # remaining unmatched prefix.
            maxMatchSuffix = 0
            for lps, word in zip(lpsList, words):
                maxMatchSuffix = max(maxMatchSuffix, lps[len(word) + unmatchedPrefix])
            if maxMatchSuffix == 0:
                return -1
            ans += 1
            unmatchedPrefix -= maxMatchSuffix

        return ans

    def _getLPS(self, pattern: str) -> list[int]:
        """
        Returns the lps array, where lps[i] is the length of the longest prefix of
        pattern[0..i] which is also a suffix of this substring.
        """
        lps = [0] * len(pattern)
        j = 0
        for i in range(1, len(pattern)):
            while j > 0 and pattern[j] != pattern[i]:
                j = lps[j - 1]
            if pattern[i] == pattern[j]:
                lps[i] = j + 1
                j += 1
        return lps

```

2233. 3294.py

Path: LeetCode-main/solutions/3294. Convert Doubly Linked List to Array II/3294.py

```

class Solution:
    def toArray(self, node: 'Optional[Node]') -> list[int]:
        ans = []
        curr = node

        while curr.prev:
            curr = curr.prev

        while curr:
            ans.append(curr.val)
            curr = curr.next

        return ans

```

2234. 3295.py

Path: LeetCode-main/solutions/3295. Report Spam Message/3295.py

```

class Solution:
    def reportSpam(self, message: list[str], bannedWords: list[str]) -> bool:
        bannedWordsSet = set(bannedWords)
        return sum(word in bannedWordsSet for word in message) > 1

```

2235. 3296.py

Path: LeetCode-main/solutions/3296. Minimum Number of Seconds to Make Mountain Height Zero/3296.py

```

class Solution:
    def minNumberOfSeconds(
        self,
        mountainHeight: int,
        workerTimes: list[int]
    ) -> int:
        def getReducedHeight(m: int) -> int:
            """Returns the total height reduced by all workers in `m` seconds."""
            # The height `x` that a worker with working time `w` reduces in `m`
            # seconds.
            #  $w * (1 + 2 + \dots + x) \leq m$ 
            #  $(1 + x) * x / 2 \leq m / w$ 
            #  $x^2 + x - 2 * m / w \leq 0$ 
            #  $x \leq (-1 + \sqrt{1 + 8 * m / w}) / 2$ 
            return sum((-1 + math.sqrt(1 + 8 * m // workerTime)) // 2
                       for workerTime in workerTimes)

        l = 1
        r = min(workerTimes) * mountainHeight * (mountainHeight + 1) // 2
        return bisect.bisect_left(range(l, r), mountainHeight,
                                   key=getReducedHeight) + 1

```

2236. 3297.py

Path: LeetCode-main/solutions/3297. Count Substrings That Can Be Rearranged to Contain a String I/3297.py

```

class Solution:
    def validSubstringCount(self, word1: str, word2: str) -> int:
        ans = 0
        count = collections.Counter(word2)
        required = len(word2)

        l = 0
        for r, c in enumerate(word1):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            while required == 0:
                # Add valid substrings containing word1[l..r] to the answer. They are
                # word1[l..r], word1[l..r + 1], ..., word1[l..n - 1].
                ans += len(word1) - r
                count[word1[l]] += 1
                if count[word1[l]] > 0:
                    required += 1
                l += 1

        return ans

```

2237. 3298.py

Path: LeetCode-main/solutions/3298. Count Substrings That Can Be Rearranged to Contain a String II/3298.py

```
class Solution:
    # Same as 3297. Count Substrings That Can Be Rearranged to Contain a String I
    def validSubstringCount(self, word1: str, word2: str) -> int:
        ans = 0
        count = collections.Counter(word2)
        required = len(word2)

        l = 0
        for r, c in enumerate(word1):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            while required == 0:
                # Add valid substrings containing word1[l..r] to the answer. They are
                # word1[l..r], word1[l..r + 1], ..., word1[l..n - 1].
                ans += len(word1) - r
                count[word1[l]] += 1
                if count[word1[l]] > 0:
                    required += 1
                l += 1

        return ans
```

2238. 3299.py

Path: LeetCode-main/solutions/3299. Sum of Consecutive Subsequences/3299.py

```

class Solution:
    def getSum(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(nums)

    def getSequenceSum(nums: list[int], direction: int) -> int:
        """
        Returns the sum of all sequences in the array that are in consecutive
        increasing order if `direction` is 1, or in consecutive decreasing order
        if `direction` is -1."""
        sequenceSum = 0
        # {num: the number of subsequences ending in `num` so far}
        prefixCount = collections.Counter()
        # {num: the number of subsequences starting from `num` so far}
        suffixCount = collections.Counter()
        # prefixSubseqs[i] := the number of subsequences ending in nums[i]
        prefixSubseqs = [0] * n
        # suffixSubseqs[i] := the number of subsequences starting from nums[i]
        suffixSubseqs = [0] * n

        for i, num in enumerate(nums):
            prevNum = num - direction
            freq = prefixCount[prevNum] + 1
            prefixSubseqs[i] = freq
            prefixCount[num] += freq
            prefixCount[num] %= MOD

        for i, num in reversed(list(enumerate(nums))):
            nextNum = num + direction
            freq = suffixCount[nextNum] + 1
            suffixSubseqs[i] = freq
            suffixCount[num] += freq
            suffixCount[num] %= MOD

        for num, prefixSubseq, suffixSubseq in zip(
            nums, prefixSubseqs, suffixSubseqs):
            sequenceSum += num * prefixSubseq * suffixSubseq
            sequenceSum %= MOD

        return sequenceSum

    increasingSequenceSum = getSequenceSum(nums, 1)
    decreasingSequenceSum = getSequenceSum(nums, -1)
    return (increasingSequenceSum + decreasingSequenceSum - sum(nums) + MOD) % MOD

```

2239. 33.py

Path: LeetCode-main/solutions/33. Search in Rotated Sorted Array/33.py

```

class Solution:
    def search(self, nums: list[int], target: int) -> int:
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[l] <= nums[m]: # nums[l..m] are sorted.
                if nums[l] <= target < nums[m]:
                    r = m - 1
                else:
                    l = m + 1
            else: # nums[m..n - 1] are sorted.
                if nums[m] < target <= nums[r]:
                    l = m + 1
                else:
                    r = m - 1

        return -1

```

2240. 330.py

Path: LeetCode-main/solutions/330. Patching Array/330.py

```

class Solution:
    def minPatches(self, nums: list[int], n: int) -> int:
        ans = 0
        i = 0 # nums' index
        miss = 1 # the minimum sum in [1, n] we might miss

        while miss <= n:
            if i < len(nums) and nums[i] <= miss:
                miss += nums[i]
                i += 1
            else:
                # Greedily add `miss` itself to increase the range from
                # [1, miss) to [1, 2 * miss).
                miss += miss
                ans += 1

        return ans

```

2241. 3300.py

Path: LeetCode-main/solutions/3300. Minimum Element After Replacement With Digit Sum/3300.py

```

class Solution:
    def minElement(self, nums: list[int]) -> int:
        return min(sum(map(int, str(num))) for num in nums)

```

2242. 3301.py

Path: LeetCode-main/solutions/3301. Maximize the Total Height of Unique Towers/3301.py

```

class Solution:
    def maximumTotalSum(self, maximumHeight: list[int]) -> int:
        ans = 0
        mn = math.inf

        for height in sorted(maximumHeight, reverse=True):
            assigned = min(height, mn - 1)
            if assigned == 0:
                return -1
            ans += assigned
            mn = assigned

        return ans

```

2243. 3302.py

Path: LeetCode-main/solutions/3302. Find the Lexicographically Smallest Valid Sequence/3302.py

```

class Solution:
    def validSequence(self, word1: str, word2: str) -> list[int]:
        ans = []
        # last[j] := the index i of the last occurrence in word1, where
        # word1[i] == word2[j]
        last = [-1] * len(word2)

        i = len(word1) - 1
        j = len(word2) - 1
        while i >= 0 and j >= 0:
            if word1[i] == word2[j]:
                last[j] = i
                j -= 1
                i -= 1

        canSkip = True
        j = 0
        for i, c in enumerate(word1):
            if j == len(word2):
                break
            if c == word2[j]:
                ans.append(i)
                j += 1
            elif canSkip and (j == len(word2) - 1 or i < last[j + 1]):
                canSkip = False
                ans.append(i)
                j += 1

        return ans if j == len(word2) else []

```

2244. 3303.py

Path: LeetCode-main/solutions/3303. Find the Occurrence of First Almost Equal Substring/3303.py

```

class Solution:
    def minStartingIndex(self, s: str, pattern: str) -> int:
        z1 = self._zFunction(pattern + s)
        z2 = self._zFunction(pattern[::-1] + s[::-1])

        # Match s[i..i + len(pattern) - 1] with `pattern` from both the prefix and
        # the suffix.
        for i in range(len(s) - len(pattern) + 1):
            if z1[len(pattern) + i] + z2[len(s) - i] >= len(pattern) - 1:
                return i

        return -1

    def _zFunction(self, s: str) -> list[int]:
        """
        Returns the z array, where z[i] is the length of the longest prefix of
        s[i..n) which is also a prefix of s.

        https://cp-algorithms.com/string/z-function.html#implementation
        """
        n = len(s)
        z = [0] * n
        l = 0
        r = 0
        for i in range(1, n):
            if i < r:
                z[i] = min(r - i, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] > r:
                l = i
                r = i + z[i]
        return z

```

2245. 3304.py

Path: LeetCode-main/solutions/3304. Find the K-th Character in String Game I/3304.py

```

class Solution:
    def kthCharacter(self, k: int) -> str:
        return string.ascii_lowercase[(k - 1).bit_count()]

```

2246. 3305.py

Path: LeetCode-main/solutions/3305. Count of Substrings Containing Every Vowel and K Consonants I/3305.py

```

class Solution:
    def countOfSubstrings(self, word: str, k: int) -> int:
        VOWELS = 'aeiou'

        def substringsWithAtMost(k: int) -> int:
            """
            Return the number of substrings containing every vowel with at most k
            consonants.
            """
            if k == -1:
                return 0

            res = 0
            vowels = 0
            uniqueVowels = 0
            vowelLastSeen = {}

            l = 0
            for r, c in enumerate(word):
                if c in VOWELS:
                    vowels += 1
                    if c not in vowelLastSeen or vowelLastSeen[c] < 1:
                        uniqueVowels += 1
                    vowelLastSeen[c] = r
                while r - l + 1 - vowels > k:
                    if word[l] in VOWELS:
                        vowels -= 1
                    if vowelLastSeen[word[l]] == 1:
                        uniqueVowels -= 1
                    l += 1
                if uniqueVowels == 5:
                    # Add substrings containing every vowel with at most k consonants to
                    # the answer. They are
                    # word[l..r], word[l + 1..r], ..., word[min(vowelLastSeen[vowel])..r]
                    res += min(vowelLastSeen[vowel] for vowel in VOWELS) - l + 1

            return res

        return substringsWithAtMost(k) - substringsWithAtMost(k - 1)

```

2247. 3306.py

Path: LeetCode-main/solutions/3306. Count of Substrings Containing Every Vowel and K Consonants II/3306.py


```

class Solution:
    # Same as 3305. Count of Substrings Containing Every Vowel and K Consonants I
    def countOfSubstrings(self, word: str, k: int) -> int:
        VOWELS = 'aeiou'

        def substringsWithAtMost(k: int) -> int:
            """
            Return the number of substrings containing every vowel with at most k
            consonants.
            """
            if k < 0:
                return 0

            res = 0
            vowels = 0
            uniqueVowels = 0
            vowelLastSeen = {}

            l = 0
            for r, c in enumerate(word):
                if c in VOWELS:
                    vowels += 1
                    if c not in vowelLastSeen or vowelLastSeen[c] < 1:
                        uniqueVowels += 1
                        vowelLastSeen[c] = r
                while r - l + 1 - vowels > k:
                    if word[l] in VOWELS:
                        vowels -= 1
                        if vowelLastSeen[word[l]] == 1:
                            uniqueVowels -= 1
                    l += 1
                if uniqueVowels == 5:
                    # Add substrings containing every vowel with at most k consonants to
                    # the answer. They are
                    # word[l..r], word[l + 1..r], ..., word[min(vowelLastSeen[vowel])..r]
                    res += min(vowelLastSeen[vowel] for vowel in VOWELS) - l + 1

            return res

        return substringsWithAtMost(k) - substringsWithAtMost(k - 1)

```

2248. 3307.py

Path: LeetCode-main/solutions/3307. Find the K-th Character in String Game II/3307.py

```

class Solution:
    def kthCharacter(self, k: int, operations: list[int]) -> str:
        operationsCount = math.ceil(math.log2(k))
        increases = 0

        for i in range(operationsCount - 1, -1, -1):
            halfSize = 2**i
            if k > halfSize:
                k -= halfSize # Move k from the right half to the left half.
                increases += operations[i]

        return string.ascii_lowercase[increases % 26]

```

2249. 3309.py

Path: LeetCode-main/solutions/3309. Maximum Possible Number by Binary Concatenation/3309.py

```
class Solution:
    def maxGoodNumber(self, nums: list[int]) -> int:
        ans = 0

        def concat(a: int, b: int) -> int:
            """Returns the concatenation of the binary representations of a and b."""
            return (a << b.bit_length()) + b

        nums.sort(key=functools.cmp_to_key(
            lambda a, b: concat(b, a) - concat(a, b)))

        for num in nums:
            ans = concat(ans, num)

        return ans
```

2250. 331.py

Path: LeetCode-main/solutions/331. Verify Preorder Serialization of a Binary Tree/331.py

```
class Solution:
    def isValidSerialization(self, preorder: str) -> bool:
        degree = 1 # out-degree (children) - in-degree (parent)

        for node in preorder.split(','):
            degree -= 1
            if degree < 0:
                return False
            if node != '#':
                degree += 2

        return degree == 0
```

2251. 3310.py

Path: LeetCode-main/solutions/3310. Remove Methods From Project/3310.py

```

class Solution:
    def remainingMethods(
        self,
        n: int,
        k: int,
        invocations: list[list[int]]
    ) -> list[int]:
        ans = []
        graph = [[] for _ in range(n)]

        for u, v in invocations:
            graph[u].append(v)

        q = collections.deque([k])
        seen = {k}

        while q:
            for _ in range(len(q)):
                for v in graph[q.popleft()]:
                    if v not in seen:
                        q.append(v)
                        seen.add(v)

        for u in range(n):
            if u in seen:
                continue
            for v in graph[u]:
                if v in seen:
                    return list(range(n))
            ans.append(u)

        return ans

```

2252. 3311.py

Path: LeetCode-main/solutions/3311. Construct 2D Grid Matching Graph Layout/3311.py

```

class Solution:
    def constructGridLayout(self, n: int, edges: list[list[int]]) -> list[list[int]]:
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        # Randomly choose a node with the minimum degree as the corner.
        corner = min(range(len(graph)), key=lambda x: len(graph[x]))

        seen = {corner}
        firstRow = self._getFirstRow(graph, corner, seen)
        cols = len(firstRow)
        rows = n // cols

        ans = [[0] * cols for _ in range(rows)]
        ans[0] = firstRow

        for i in range(1, rows):
            for j in range(cols):
                for v in graph[ans[i - 1][j]]:
                    if v not in seen:
                        ans[i][j] = v
                        seen.add(v)
                        break

        return ans

    def _getFirstRow(
        self,
        graph: list[list[int]],
        corner: int,
        seen: set[int]
    ) -> list[int]:
        cornerDegree = len(graph[corner])
        row = [corner]
        # Continue appending neighbors until we hit another corner.
        while len(row) == 1 or len(graph[row[-1]]) == cornerDegree + 1:
            # Sort neighbors by degree to prioritize smaller ones (shortest row built first).
            graph[row[-1]].sort(key=lambda x: len(graph[x]))
            for v in graph[row[-1]]:
                if v not in seen and len(graph[v]) in (cornerDegree, cornerDegree + 1):
                    row.append(v)
                    seen.add(v)
                    break
        return row

```

2253. 3312.py

Path: LeetCode-main/solutions/3312. Sorted GCD Pair Queries/3312.py

```

class Solution:
    def gcdValues(self, nums: list[int], queries: list[int]) -> list[int]:
        maxNum = max(nums)
        # countDivisor[d] := the number of `nums` having `num % d == 0`
        countDivisor = [0] * (maxNum + 1)
        # countGcdPair[g] := the number of pairs having gcd == g
        countGcdPair = [0] * (maxNum + 1)

        for num in nums:
            for i in range(1, math.isqrt(num) + 1):
                if num % i == 0:
                    countDivisor[i] += 1
                    if i != num // i:
                        countDivisor[num // i] += 1

        for gcd in range(maxNum, 0, -1):
            # There are C(countDivisor[gcd], 2) pairs that have a common divisor
            # that's a multiple of `gcd` (including the one that equals to `gcd`).
            # So, subtract the multiples of `gcd` to have the number of pairs with a
            # gcd that's exactly `gcd`.
            countGcdPair[gcd] = countDivisor[gcd] * (countDivisor[gcd] - 1) // 2
            for largerGcd in range(2 * gcd, maxNum + 1, gcd):
                countGcdPair[gcd] -= countGcdPair[largerGcd]

        # prefixCountGcdPair[g] := the number of pairs having gcd <= g
        prefixCountGcdPair = list(itertools.accumulate(countGcdPair))
        return [bisect.bisect_left(prefixCountGcdPair, query + 1)
                for query in queries]

```

2254. 3313.py

Path: LeetCode-main/solutions/3313. Find the Last Marked Nodes in Tree/3313.py

```

from dataclasses import dataclass

@dataclass
class Node:
    node: int = 0 # the node number
    time: int = 0 # the time it got marked

class Last2:
    def __init__(self, last1: Node = Node(), last2: Node = Node()):
        self.last1 = last1 # the last marked node
        self.last2 = last2 # the second last marked node

class Solution:
    # Similar to 3241. Time Taken to Mark All Nodes
    def lastMarkedNodes(self, edges: list[list[int]]) -> list[int]:
        n = len(edges) + 1
        ans = [0] * n
        tree = [[] for _ in range(n)]
        # dp[i] := the last marked two nodes for subtree rooted at node i, where
        # each node contains the time it got marked
        dp = [Last2()] * n

        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

```

```

self._dfs(tree, 0, -1, dp)
self._reroot(tree, 0, -1, Node(), dp, ans)
return ans

def _dfs(
    self,
    tree: list[list[int]],
    u: int,
    prev: int,
    dp: list[Last2]
) -> Node:
    """
    Performs a DFS traversal of the subtree rooted at node `u`, computes the
    time taken to mark all nodes in the subtree, records the last two marked
    nodes, and returns the last marked node.

    These values are used later in the rerooting process.
    """
    last1 = Node(u, 0)
    last2 = Node()
    for v in tree[u]:
        if v == prev:
            continue
        child = self._dfs(tree, v, u, dp)
        time = child.time + 1
        if time > last1.time:
            last2 = last1
            last1 = Node(child.node, time)
        elif time > last2.time:
            last2 = Node(child.node, time)
    dp[u] = Last2(last1, last2)
    return last1

def _reroot(
    self,
    tree: list[list[int]],
    u: int,
    prev: int,
    last: Node,
    dp: list[list[int]],
    ans: list[int]
) -> None:
    """
    Reroots the tree at node `u` and updates the answer array, where `last`
    is the last marked node that doesn't go through `u`'s subtree.
    """
    ans[u] = last.node if last.time > dp[u].last1.time else dp[u].last1.node
    for v in tree[u]:
        if v == prev:
            continue
        newLast = Node(last.node, last.time + 1)
        if dp[u].last1.node == dp[v].last1.node:
            alternativeTime = 1 + dp[u].last2.time
            if alternativeTime > newLast.time:
                newLast = Node(dp[u].last2.node, alternativeTime)
        else:
            alternativeTime = 1 + dp[u].last1.time
            if alternativeTime > newLast.time:
                newLast = Node(dp[u].last1.node, alternativeTime)
        self._reroot(tree, v, u, newLast, dp, ans)

```

Path: LeetCode-main/solutions/3314. Construct the Minimum Bitwise Array I/3314.py

```
class Solution:
    def minBitwiseArray(self, nums: list[int]) -> list[int]:
        return [-1 if num == 2 else num - self._getLeadingOneOfLastGroupOfOnes(num)
                for num in nums]

    def _getLeadingOneOfLastGroupOfOnes(self, num: int) -> int:
        """
        Returns the leading one of the last group of 1s in the binary
        representation of num. For example, if num = 0b10111, the leading one of
        the last group of 1s is 0b100.
        """
        leadingOne = 1
        while (num & leadingOne) > 0:
            leadingOne <<= 1
        return leadingOne >> 1
```

2256. 3315.py

Path: LeetCode-main/solutions/3315. Construct the Minimum Bitwise Array II/3315.py

```
class Solution:
    # Same as 3314. Construct the Minimum Bitwise Array I
    def minBitwiseArray(self, nums: list[int]) -> list[int]:
        return [-1 if num == 2 else num - self._getLeadingOneOfLastGroupOfOnes(num)
                for num in nums]

    def _getLeadingOneOfLastGroupOfOnes(self, num: int) -> int:
        """
        Returns the leading one of the last group of 1s in the binary
        representation of num. For example, if num = 0b10111, the leading one of
        the last group of 1s is 0b100.
        """
        leadingOne = 1
        while (num & leadingOne) > 0:
            leadingOne <<= 1
        return leadingOne >> 1
```

2257. 3316-2.py

Path: LeetCode-main/solutions/3316. Find Maximum Removals From Source String/3316-2.py

```

class Solution:
    def maxRemovals(
        self,
        source: str,
        pattern: str,
        targetIndices: list[int]
    ) -> int:
        m = len(source)
        n = len(pattern)
        target = set(targetIndices)
        # dp[i][j] := the maximum number of operations that can be performed for
        # source[i..m) and pattern[j..n)
        dp = [[-math.inf] * (n + 1) for _ in range(m + 1)]
        dp[m][n] = 0

        for i in reversed(range(m)):
            dp[i][n] = int(i in target) + dp[i + 1][n]
            for j in reversed(range(n)):
                pick = dp[i + 1][j + 1] if source[i] == pattern[j] else -math.inf
                skip = int(i in target) + dp[i + 1][j]
                dp[i][j] = max(pick, skip)

        return 0 if dp[0][0] == -math.inf else dp[0][0]

```

2258. 3316-3.py

Path: LeetCode-main/solutions/3316. Find Maximum Removals From Source String/3316-3.py

```

class Solution:
    def maxRemovals(
        self,
        source: str,
        pattern: str,
        targetIndices: list[int]
    ) -> int:
        m = len(source)
        n = len(pattern)
        target = set(targetIndices)
        # dp[j] := the maximum number of operations that can be performed for
        # source so far and pattern[j..n)
        dp = [-math.inf] * (n + 1)
        dp[n] = 0

        for i in reversed(range(m)):
            newDp = dp[:]
            newDp[n] = int(i in target) + dp[n]
            for j in range(n):
                pick = dp[j + 1] if source[i] == pattern[j] else -math.inf
                skip = int(i in target) + dp[j]
                newDp[j] = max(pick, skip)
            dp = newDp

        return 0 if dp[0] == -math.inf else dp[0]

```

2259. 3316.py

Path: LeetCode-main/solutions/3316. Find Maximum Removals From Source String/3316.py


```

class Solution:
    def maxRemovals(
        self,
        source: str,
        pattern: str,
        targetIndices: list[int]
    ) -> int:
        m = len(source)
        n = len(pattern)
        target = set(targetIndices)

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """
            Returns the maximum number of operations that can be performed for
            source[i..m) and pattern[j..n).
            """
            if i == m:
                return 0 if j == n else -math.inf
            if j == n:
                return int(i in target) + dp(i + 1, j)
            pick = dp(i + 1, j + 1) if source[i] == pattern[j] else -math.inf
            skip = int(i in target) + dp(i + 1, j)
            return max(pick, skip)

        ans = dp(0, 0)
        return 0 if ans == -math.inf else ans

```

2260. 3317.py

Path: LeetCode-main/solutions/3317. Find the Number of Possible Ways for an Event/3317.py

```

class Solution:
    def numberOfWays(self, n: int, x: int, y: int) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def fact(i: int) -> int:
            return 1 if i <= 1 else i * fact(i - 1) % MOD

        @functools.lru_cache(None)
        def inv(i: int) -> int:
            return pow(i, MOD - 2, MOD)

        @functools.lru_cache(None)
        def nCk(n: int, k: int) -> int:
            return fact(n) * inv(fact(k)) * inv(fact(n - k)) % MOD

        @functools.lru_cache(None)
        def stirling(n: int, k: int) -> int:
            """
            Returns the number of ways to partition a set of n objects into k
            non-empty subsets.

            https://en.wikipedia.org/wiki/Stirling_numbers_of_the_second_kind
            """
            if k == 0 or n < k:
                return 0
            if k == 1 or n == k:
                return 1
            return (k * stirling(n - 1, k) + stirling(n - 1, k - 1)) % MOD

        # 1. Choose `k` stages from `x` stages.
        # 2. Partition `n` performers into `k` stages.
        # 3. Permute `k` stages.
        # 4. Score `k` stages with score in the range [1, y], so y^k ways.
        return sum(nCk(x, k) * stirling(n, k) * fact(k) * pow(y, k, MOD) % MOD
                   for k in range(1, min(n, x) + 1)) % MOD

```

2261. 3318.py

Path: LeetCode-main/solutions/3318. Find X-Sum of All K-Long Subarrays I/3318.py

```

from sortedcontainers import SortedList

class Solution:
    def findXSum(self, nums: list[int], k: int, x: int) -> list[int]:
        ans = []
        windowSum = 0
        count = collections.Counter()
        top = SortedList()
        bot = SortedList()

        def update(num: int, freq: int) -> None:
            """Updates the count of num by freq and the window sum accordingly."""
            nonlocal windowSum
            if count[num] > 0: # Clean up old values.
                if [count[num], num] in bot:
                    bot.remove([count[num], num])
                else:
                    top.remove([count[num], num])
                    windowSum -= num * count[num]
            count[num] += freq
            if count[num] > 0:
                bot.add([count[num], num])

        for i, num in enumerate(nums):
            update(num, 1)
            if i >= k:
                update(nums[i - k], -1)
            # Move the bottom element to the top if needed.
            while bot and len(top) < x:
                countB, b = bot.pop()
                top.add([countB, b])
                windowSum += b * countB
            # Swap the bottom and top elements if needed.
            while bot and bot[-1] > top[0]:
                countB, b = bot.pop()
                countT, t = top.pop(0)
                bot.add([countT, t])
                windowSum -= t * countT
                top.add([countB, b])
                windowSum += b * countB
            if i >= k - 1:
                ans.append(windowSum)

        return ans

```

2262. 3319.py

Path: LeetCode-main/solutions/3319. K-th Largest Perfect Subtree Size in Binary Tree/3319.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    isPerfect: bool
    sz: int

class Solution:
    def kthLargestPerfectSubtree(self, root: TreeNode | None, k: int) -> int:
        ans = []
        self._dfs(root, ans)
        if len(ans) < k:
            return -1
        return sorted(ans, reverse=True)[k - 1]

    def _dfs(self, root: TreeNode, ans: list[int]) -> T:
        if not root:
            return T(True, 0)
        l = self._dfs(root.left, ans)
        r = self._dfs(root.right, ans)
        if l.isPerfect and r.isPerfect and l.sz == r.sz:
            sz = 1 + l.sz + r.sz
            ans.append(sz)
            return T(True, sz)
        return T(False, 0)

```

2263. 332.py

Path: LeetCode-main/solutions/332. Reconstruct Itinerary/332.py

```

class Solution:
    def findItinerary(self, tickets: list[list[str]]) -> list[str]:
        ans = []
        graph = collections.defaultdict(list)

        for a, b in reversed(sorted(tickets)):
            graph[a].append(b)

        def dfs(u: str) -> None:
            while u in graph and graph[u]:
                dfs(graph[u].pop())
            ans.append(u)

        dfs('JFK')
        return ans[::-1]

```

2264. 3320.py

Path: LeetCode-main/solutions/3320. Count The Number of Winning Sequences/3320.py

```

class Solution:
    def countWinningSequences(self, s: str) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(i: int, prev: int, bob: int) -> int:
            """
            Returns the number of distinct sequences Bob can use to beat Alice for
            s[i..n), where the previous character is `prev` (0: F, 1: W, 2: E) and the
            number of points that Bob is having is `bob`.
            """
            if i == len(s):
                return int(bob > 0)

            f = 0 # If Bob summons a Fire Dragon at i.
            w = 0 # If Bob summons a Water Serpent at i.
            e = 0 # If Bob summons a Earth Golem at i.

            match s[i]:
                case 'F':
                    if prev != 0:
                        f = dp(i + 1, 0, bob) % MOD
                    if prev != 1:
                        w = dp(i + 1, 1, bob + 1) % MOD
                    if prev != 2:
                        e = dp(i + 1, 2, bob - 1) % MOD
                case 'W':
                    if prev != 0:
                        f = dp(i + 1, 0, bob - 1) % MOD
                    if prev != 1:
                        w = dp(i + 1, 1, bob) % MOD
                    if prev != 2:
                        e = dp(i + 1, 2, bob + 1) % MOD
                case 'E':
                    if prev != 0:
                        f = dp(i + 1, 0, bob + 1) % MOD
                    if prev != 1:
                        w = dp(i + 1, 1, bob - 1) % MOD
                    if prev != 2:
                        e = dp(i + 1, 2, bob) % MOD

            return f + w + e

        return (dp(0, 0, 0) + dp(0, 1, 0) + dp(0, 2, 0)) // 2 % MOD

```

2265. 3321.py

Path: LeetCode-main/solutions/3321. Find X-Sum of All K-Long Subarrays II/3321.py

```

from sortedcontainers import SortedList

class Solution:
    # Same as 3318. Find X-Sum of All K-Long Subarrays I
    def findXSum(self, nums: list[int], k: int, x: int) -> list[int]:
        ans = []
        windowSum = 0
        count = collections.Counter()
        top = SortedList() # the top x elements
        bot = SortedList() # the rest of the elements

        def update(num: int, freq: int) -> None:
            """Updates the count of num by freq and the window sum accordingly."""
            nonlocal windowSum
            if count[num] > 0: # Clean up the old count.
                if [count[num], num] in bot:
                    bot.remove([count[num], num])
                else:
                    top.remove([count[num], num])
                    windowSum -= num * count[num]
            count[num] += freq
            if count[num] > 0:
                bot.add([count[num], num])

        for i, num in enumerate(nums):
            update(num, 1)
            if i >= k:
                update(nums[i - k], -1)
            # Move the bottom elements to the top if needed.
            while bot and len(top) < x:
                countB, b = bot.pop()
                top.add([countB, b])
                windowSum += b * countB
            # Swap the bottom and top elements if needed.
            while bot and bot[-1] > top[0]:
                countB, b = bot.pop()
                countT, t = top.pop(0)
                bot.add([countT, t])
                top.add([countB, b])
                windowSum += b * countB
                windowSum -= t * countT
            if i >= k - 1:
                ans.append(windowSum)

        return ans

```

2266. 3323.py

Path: LeetCode-main/solutions/3323. Minimize Connected Groups by Inserting Interval/3323.py

```

class Solution:
    def minConnectedGroups(self, intervals: list[list[int]], k: int) -> int:
        mergedIntervals = 0
        maxMergedIntervals = 0

        intervals = self._merge(intervals)

        i = 0
        for _, end in intervals:
            while i < len(intervals) and end + k >= intervals[i][0]:
                mergedIntervals += 1
                i += 1
            mergedIntervals -= 1 # Exclude intervals[i].
            maxMergedIntervals = max(maxMergedIntervals, mergedIntervals)

        return len(intervals) - maxMergedIntervals

# Same as 56. Merge Intervals
def _merge(self, intervals: list[list[int]]) -> list[list[int]]:
    res = []
    for interval in sorted(intervals):
        if not res or res[-1][1] < interval[0]:
            res.append(interval)
        else:
            res[-1][1] = max(res[-1][1], interval[1])
    return res

```

2267. 3324.py

Path: LeetCode-main/solutions/3324. Find the Sequence of Strings Appeared on the Screen/3324.py

```

class Solution:
    def stringSequence(self, target: str) -> list[str]:
        ans = []
        s = []

        for targetChar in target:
            s.append('a')
            ans.append(''.join(s))
            for offset in range(ord('b'), ord(targetChar) + 1):
                s[-1] = chr(offset)
                ans.append(''.join(s))

        return ans

```

2268. 3325.py

Path: LeetCode-main/solutions/3325. Count Substrings With K-Frequency Characters I/3325.py

```

class Solution:
    def numberOfSubstrings(self, s: str, k: int) -> int:
        n = len(s)
        ans = n * (n + 1) // 2
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] == k:
                count[s[l]] -= 1
                l += 1
            ans -= r - l + 1

        return ans

```

2269. 3326.py

Path: LeetCode-main/solutions/3326. Minimum Division Operations to Make Array Non Decreasing/3326.py

```

class Solution:
    def minOperations(self, nums: list[int]) -> int:
        ans = 0

        for i in range(len(nums) - 2, -1, -1):
            if nums[i] > nums[i + 1]:
                minDivisor = self._getMinDivisor(nums[i])
                if minDivisor > nums[i + 1]:
                    return -1
                nums[i] = minDivisor
                ans += 1

        return ans

    def _getMinDivisor(self, num: int) -> int:
        for divisor in range(2, math.isqrt(num) + 1):
            if num % divisor == 0:
                return divisor
        return num

```

2270. 3327.py

Path: LeetCode-main/solutions/3327. Check if DFS Strings Are Palindromes/3327.py

```

class Solution:
    def findAnswer(self, parent: list[int], s: str) -> list[bool]:
        n = len(parent)
        tree = [[] for _ in parent]
        start = [0] * n # start[i] := the start index of `dfsStr` of node i
        end = [0] * n # end[i] := the end index of `dfsStr` of node i
        dfsStr = []

        for i in range(1, n):
            tree[parent[i]].append(i)

        self._dfs(tree, 0, 0, s, start, end, dfsStr)
        t = '#'.join('@' + ''.join(dfsStr) + '$')
        p = self._manacher(t)
        return [self._isPalindrome(s, e, p)
                for s, e in zip(start, end)]

```



```

def _dfs(
    self,
    tree: list[list[int]],
    u: int,
    index: int,
    s: str,
    start: list[int],
    end: list[int],
    dfsStr: list[str]
) -> int:
    """Returns the start index of the "DFS string" of u's next node."""
    start[u] = index
    for v in tree[u]:
        index = self._dfs(tree, v, index, s, start, end, dfsStr)
    end[u] = index
    dfsStr.append(s[u])
    return index + 1

def _manacher(self, t: str) -> list[int]:
    """
    Returns an array `p` s.t. `p[i]` is the length of the longest palindrome
    centered at `t[i]`, where `t` is a string with delimiters and sentinels.
    """
    p = [0] * len(t)
    center = 0
    for i in range(1, len(t) - 1):
        rightBoundary = center + p[center]
        mirrorIndex = center - (i - center)
        if rightBoundary > i:
            p[i] = min(rightBoundary - i, p[mirrorIndex])
        # Try to expand the palindrome centered at i.
        while t[i + 1 + p[i]] == t[i - 1 - p[i]]:
            p[i] += 1
        # If a palindrome centered at i expands past `rightBoundary`, adjust
        # the center based on the expanded palindrome.
        if i + p[i] > rightBoundary:
            center = i
    return p

def _isPalindrome(self, s: int, e: int, p: list[int]) -> bool:
    """
    Returns true if `dfsStr[s..e]` is a palindrome by using the precomputed
    array `p` from the Manacher's algorithm.

    The precomputed array `p` is based on the string `t` with delimiters and
    sentinels. Let `t = '#' + dfsStr + '$'`. Then, the center of
    `dfsStr` maps to `t[s + e + 2]` since `dfsStr[s]` maps to `t[2 * s + 2]`
    and `dfsStr[e]` maps to `t[2 * e + 2]`. So, the center of `dfsStr` is
    `t[(2 * s + 2 + 2 * e + 2) / 2] = t[s + e + 2]`.
    """
    length = e - s + 1
    center = s + e + 2
    return p[center] >= length

```

2271. 3329.py

Path: LeetCode-main/solutions/3329. Count Substrings With K-Frequency Characters II/3329.py

```

class Solution:
    def numberOfSubstrings(self, s: str, k: int) -> int:
        n = len(s)
        ans = n * (n + 1) // 2
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] == k:
                count[s[l]] -= 1
                l += 1
            ans -= r - l + 1

        return ans

```

2272. 333.py

Path: LeetCode-main/solutions/333. Largest BST Subtree/333.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    mn: int # the minimum value in the subtree
    mx: int # the maximum value in the subtree
    size: int # the size of the subtree

class Solution:
    def largestBSTSubtree(self, root: TreeNode | None) -> int:
        def dfs(root: TreeNode | None) -> T:
            if not root:
                return T(math.inf, -math.inf, 0)

            l = dfs(root.left)
            r = dfs(root.right)

            if l.mx < root.val < r.mn:
                return T(min(l.mn, root.val), max(r.mx, root.val), 1 + l.size + r.size)

            # Mark one as invalid, but still record the size of children.
            # Return (-inf, inf) because no node will be > inf or < -inf.
            return T(-math.inf, math.inf, max(l.size, r.size))

        return dfs(root).size

```

2273. 3330.py

Path: LeetCode-main/solutions/3330. Find the Original Typed String I/3330.py

```

class Solution:
    def possibleStringCount(self, word: str) -> int:
        return 1 + sum(a == b
                        for a, b in itertools.pairwise(word))

```

2274. 3331.py

Path: LeetCode-main/solutions/3331. Find Subtree Sizes After Changes/3331.py

```
class Solution:
    def findSubtreeSizes(self, parent: list[int], s: str) -> list[int]:
        n = len(parent)
        ans = [0] * n
        newParent = parent.copy()
        tree = [[] for _ in range(n)]

        for i in range(1, n):
            closest = self._findClosestAncestor(i, parent, s)
            if closest != -1:
                newParent[i] = closest

        for i in range(1, n):
            tree[newParent[i]].append(i)

        self._dfs(tree, 0, ans)
        return ans

    def _findClosestAncestor(self, u: int, parent: list[int], s: str) -> int:
        """
        Returns the closest ancestor of node `u` that has the same value as `u`.
        """
        curr = parent[u]
        while curr != -1:
            if s[curr] == s[u]:
                return curr
            curr = parent[curr]
        return -1

    def _dfs(self, tree: list[list[int]], u: int, ans: list[int]) -> int:
        sz = 1
        for v in tree[u]:
            sz += self._dfs(tree, v, ans)
        ans[u] = sz
        return sz
```

2275. 3332-2.py

Path: LeetCode-main/solutions/3332. Maximum Points Tourist Can Earn/3332-2.py

```

class Solution:
    def maxScore(
        self,
        n: int,
        k: int,
        stayScore: list[list[int]],
        travelScore: list[list[int]]
    ) -> int:
        # dp[j] := the maximum score after days so far being at city j
        dp = [0] * n

        for i in range(k):
            newDp = [0] * n
            for dest in range(n):
                # 1. Stay at the current city.
                newDp[dest] = dp[dest] + stayScore[i][dest]
                # 2. Travel from any other city.
                for curr in range(n):
                    if curr != dest:
                        newDp[dest] = max(newDp[dest],
                                           dp[curr] + travelScore[curr][dest])

            dp = newDp

        return max(dp)

```

2276. 3332.py

Path: LeetCode-main/solutions/3332. Maximum Points Tourist Can Earn/3332.py

```

class Solution:
    def maxScore(
        self,
        n: int,
        k: int,
        stayScore: list[list[int]],
        travelScore: list[list[int]]
    ) -> int:
        # dp[i][j] := the maximum score after i days being at city j
        dp = [[0] * n for _ in range(k + 1)]

        for i in range(1, k + 1):
            for dest in range(n):
                # 1. Stay at the current city.
                dp[i][dest] = dp[i - 1][dest] + stayScore[i - 1][dest]
                # 2. Travel from any other city.
                for curr in range(n):
                    if curr != dest:
                        dp[i][dest] = max(dp[i][dest],
                                           dp[i - 1][curr] + travelScore[curr][dest])

        return max(dp[k])

```

2277. 3333.py

Path: LeetCode-main/solutions/3333. Find the Original Typed String II/3333.py

```

class Solution:
    def possibleStringCount(self, word: str, k: int) -> int:
        MOD = 1_000_000_007
        groups = self._getConsecutiveLetters(word)
        totalCombinations = functools.reduce(lambda subtotal, group:
                                              subtotal * group % MOD, groups)

        if k <= len(groups):
            return totalCombinations

        # dp[j] := the number of ways to form strings of length j using groups[0..i]
        dp = [0] * k
        dp[0] = 1 # Base case: empty string

        for i, group in enumerate(groups):
            newDp = [0] * k
            windowSum = 0
            for j in range(i, k):
                newDp[j] = (newDp[j] + windowSum) % MOD
                windowSum = (windowSum + dp[j]) % MOD
                if j >= group:
                    windowSum = (windowSum - dp[j - group] + MOD) % MOD
            dp = newDp

        return (totalCombinations - sum(dp)) % MOD

    def _getConsecutiveLetters(self, word: str) -> list[int]:
        """
        Returns consecutive identical letters in the input string.
        e.g. "aabbbc" -> [2, 3, 1].
        """
        groups = []
        group = 1
        for i in range(1, len(word)):
            if word[i] == word[i - 1]:
                group += 1
            else:
                groups.append(group)
                group = 1
        groups.append(group)
        return groups

```

2278. 3334.py

Path: LeetCode-main/solutions/3334. Find the Maximum Factor Score of Array/3334.py

```

class Solution:
    def maxScore(self, nums: list[int]) -> int:
        n = len(nums)
        # prefixGcd[i] := GCD of nums[0..i]
        # prefixLcm[i] := LCM of nums[0..i]
        prefixGcd, prefixLcm = self._getPrefix(nums)
        # suffixGcd[i] := GCD of nums[i..n - 1]
        # suffixLcm[i] := LCM of nums[i..n - 1]
        suffixGcd, suffixLcm = self._getSuffix(nums)
        ans = suffixGcd[0] * suffixLcm[0]

        for i in range(n):
            gcd1 = prefixGcd[i - 1] if i > 0 else 0
            gcd2 = suffixGcd[i + 1] if i + 1 < n else 0
            lcm1 = prefixLcm[i - 1] if i > 0 else 1
            lcm2 = suffixLcm[i + 1] if i + 1 < n else 1
            score = math.gcd(gcd1, gcd2) * math.lcm(lcm1, lcm2)
            ans = max(ans, score)

        return ans

    def _getPrefix(self, nums: list[int]) -> tuple[list[int], list[int]]:
        """Returns the prefix GCD and LCM arrays."""
        prefixGcd = []
        prefixLcm = []
        currGcd = 0
        currLcm = 1
        for num in nums:
            currGcd = math.gcd(currGcd, num)
            currLcm = math.lcm(currLcm, num)
            prefixGcd.append(currGcd)
            prefixLcm.append(currLcm)
        return prefixGcd, prefixLcm

    def _getSuffix(self, nums: list[int]) -> tuple[list[int], list[int]]:
        """Returns the suffix GCD and LCM arrays."""
        suffixGcd = []
        suffixLcm = []
        currGcd = 0
        currLcm = 1
        for num in reversed(nums):
            currGcd = math.gcd(currGcd, num)
            currLcm = math.lcm(currLcm, num)
            suffixGcd.append(currGcd)
            suffixLcm.append(currLcm)
        return list(reversed(suffixGcd)), list(reversed(suffixLcm))

```

2279. 3335-2.py

Path: LeetCode-main/solutions/3335. Total Characters in String After Transformations I/3335-2.py

```

class Solution:
    def lengthAfterTransformations(self, s: str, t: int) -> int:
        MOD = 1_000_000_007

        def matrixMult(A: list[list[int]], B: list[list[int]]) -> list[list[int]]:
            """Returns A * B."""
            sz = len(A)
            C = [[0] * sz for _ in range(sz)]
            for i in range(sz):
                for j in range(sz):
                    for k in range(sz):
                        C[i][j] += A[i][k] * B[k][j]
                        C[i][j] %= MOD
            return C

        def matrixPow(M: list[list[int]], n: int) -> list[list[int]]:
            """Returns M^n."""
            if n == 0:
                return [[1 if i == j else 0 # identity matrix
                        for j in range(len(M))
                        for i in range(len(M))]]
            if n % 2 == 1:
                return matrixMult(M, matrixPow(M, n - 1))
            return matrixPow(matrixMult(M, M), n // 2)

        # T[i][j] := the number of ways to transform ('a' + i) to ('a' + j)
        T = self._getTransformationMatrix()
        poweredT = matrixPow(T, t)
        count = [0] * 26
        lengths = [0] * 26

        for c in s:
            count[ord(c) - ord('a')] += 1

        for i in range(26):
            for j in range(26):
                lengths[j] += count[i] * poweredT[i][j]
                lengths[j] %= MOD

        return sum(lengths) % MOD

    def _getTransformationMatrix(self) -> list[list[int]]:
        T = [[0] * 26 for _ in range(26)]
        # 'z' -> 'ab'
        T[25][0] = 1
        T[25][1] = 1
        # 'a' -> 'b', 'b' -> 'c', ..., 'y' -> 'z'
        for i in range(25):
            T[i][i + 1] = 1
        return T

```

2280. 3335.py

Path: LeetCode-main/solutions/3335. Total Characters in String After Transformations I/3335.py

```

class Solution:
    def lengthAfterTransformations(self, s: str, t: int) -> int:
        MOD = 1_000_000_007
        count = [0] * 26

        for c in s:
            count[ord(c) - ord('a')] += 1

        for _ in range(t):
            newCount = [0] * 26
            # 'a' -> 'b', 'b' -> 'c', ..., 'y' -> 'z'
            for i in range(25):
                newCount[i + 1] = count[i]
            # 'z' -> 'a'
            newCount[0] = count[25]
            newCount[1] = (newCount[1] + count[25]) % MOD
            count = newCount

        return sum(count) % MOD

```

2281. 3336-2.py

Path: LeetCode-main/solutions/3336. Find the Number of Subsequences With Equal GCD/3336-2.py

```

class Solution:
    def subsequencePairCount(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        maxNum = max(nums)
        # dp[i][x][y] := number of disjoint pairs `seq1` and `seq2` of
        # nums[0..i - 1], where GCD(seq1) == x and GCD(seq2) == y
        dp = [[[0] * (maxNum + 1)
                for _ in range(maxNum + 1)
                for _ in range(len(nums) + 1)]]
        dp[0][0][0] = 1

        for i, num in enumerate(nums):
            for x in range(maxNum + 1):
                for y in range(maxNum + 1):
                    # 1. Skip nums[i].
                    dp[i + 1][x][y] += dp[i][x][y]
                    dp[i + 1][x][y] %= MOD
                    # 2. Pick nums[i] in the first subsequence.
                    newX = math.gcd(x, num)
                    dp[i + 1][newX][y] += dp[i][x][y]
                    dp[i + 1][newX][y] %= MOD
                    # 3. Pick nums[i] in the second subsequence.
                    newY = math.gcd(y, num)
                    dp[i + 1][x][newY] += dp[i][x][y]
                    dp[i + 1][x][newY] %= MOD

        return sum(dp[-1][g][g]
                   for g in range(1, maxNum + 1)) % MOD

```

2282. 3336-3.py

Path: LeetCode-main/solutions/3336. Find the Number of Subsequences With Equal GCD/3336-3.py


```

class Solution:
    def subsequencePairCount(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        maxNum = max(nums)
        # dp[x][y] := number of disjoint pairs `seq1` and `seq2` of
        # nums so far, where GCD(seq1) == x and GCD(seq2) == y
        dp = [[0] * (maxNum + 1) for _ in range(maxNum + 1)]
        dp[0][0] = 1

        for num in nums:
            newDp = [[0] * (maxNum + 1) for _ in range(maxNum + 1)]
            for x in range(maxNum + 1):
                for y in range(maxNum + 1):
                    # 1. Skip `num`.
                    newDp[x][y] += dp[x][y]
                    newDp[x][y] %= MOD
                    # 2. Pick `num` in the first subsequence.
                    newX = math.gcd(x, num)
                    newDp[newX][y] += dp[x][y]
                    newDp[newX][y] %= MOD
                    # 3. Pick `num` in the second subsequence.
                    newY = math.gcd(y, num)
                    newDp[x][newY] += dp[x][y]
                    newDp[x][newY] %= MOD
            dp = newDp

        return sum(dp[g][g]
                   for g in range(1, maxNum + 1)) % MOD

```

2283. 3336.py

Path: LeetCode-main/solutions/3336. Find the Number of Subsequences With Equal GCD/3336.py

```

class Solution:
    def subsequencePairCount(self, nums: list[int]) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(i: int, x: int, y: int) -> int:
            if i == len(nums):
                return int(x > 0 and x == y)
            # 1. Skip nums[i]
            skip = dp(i + 1, x, y)
            # 2. Pick nums[i] in the first subsequence
            take1 = dp(i + 1, math.gcd(x, nums[i]), y)
            # 3. Pick nums[i] in the second subsequence
            take2 = dp(i + 1, x, math.gcd(y, nums[i]))
            return (skip + take1 + take2) % MOD

        return dp(0, 0, 0)

```

2284. 3337.py

Path: LeetCode-main/solutions/3337. Total Characters in String After Transformations II/3337.py

```

class Solution:
    # Similar to 3335. Total Characters in String After Transformations I
    def lengthAfterTransformations(self, s: str, t: int, nums: list[int]) -> int:
        MOD = 1_000_000_007

        def matrixMult(A: list[list[int]], B: list[list[int]]) -> list[list[int]]:
            """Returns A * B."""
            sz = len(A)
            C = [[0] * sz for _ in range(sz)]
            for i in range(sz):
                for j in range(sz):
                    for k in range(sz):
                        C[i][j] += A[i][k] * B[k][j]
                    C[i][j] %= MOD
            return C

        def matrixPow(M: list[list[int]], n: int) -> list[list[int]]:
            """Returns M^n."""
            if n == 0:
                return [[1 if i == j else 0 # identity matrix
                        for j in range(len(M))
                        for i in range(len(M))]]
            if n % 2 == 1:
                return matrixMult(M, matrixPow(M, n - 1))
            return matrixPow(matrixMult(M, M), n // 2)

        # T[i][j] := the number of ways to transform ('a' + i) to ('a' + j)
        T = self._getTransformationMatrix(nums)
        poweredT = matrixPow(T, t)
        count = [0] * 26
        lengths = [0] * 26

        for c in s:
            count[ord(c) - ord('a')] += 1

        for i in range(26):
            for j in range(26):
                lengths[j] += count[i] * poweredT[i][j]
                lengths[j] %= MOD

        return sum(lengths) % MOD

    def _getTransformationMatrix(self, nums: list[int]) -> list[list[int]]:
        T = [[0] * 26 for _ in range(26)]
        for i, steps in enumerate(nums):
            for step in range(1, steps + 1):
                T[i][(i + step) % 26] += 1
        return T

```

2285. 3339-2.py

Path: LeetCode-main/solutions/3339. Find the Number of K-Even Arrays/3339-2.py

```

class Solution:
    def countOfArrays(self, n: int, m: int, k: int) -> int:
        MOD = 10**9 + 7
        even = m // 2 # the number of even numbers in [1, m]
        odd = m - even # the number of odd numbers in [1, m]
        # dp[j][0/1] := the number of arrays of length so far i with j consecutive
        # even number pairs ending in an even number (0) or an odd number (1)
        dp = [[0] * 2 for _ in range(k + 1)]

        # Base case: arrays of length 1
        # For an array of length 1, we can't have any even number pairs yet.
        dp[0][0] = even
        dp[0][1] = odd

        for _ in range(2, n + 1):
            newDp = [[0] * 2 for _ in range(k + 1)]
            for j in range(k + 1):
                # 1. Appending an even number to an array ending in an even number
                #     creates a new consecutive even number pair.
                # 2. Appending an even number to an array ending in an odd number.
                newDp[j][0] = ((dp[j - 1][0] if j > 0 else 0) * even +
                               dp[j][1] * even) % MOD
                # 3. Appending an odd number to an array.
                newDp[j][1] = sum(dp[j]) * odd % MOD
            dp = newDp

        return sum(dp[k]) % MOD

```

2286. 3339.py

Path: LeetCode-main/solutions/3339. Find the Number of K-Even Arrays/3339.py

```

class Solution:
    def countOfArrays(self, n: int, m: int, k: int) -> int:
        MOD = 10**9 + 7
        even = m // 2 # the number of even numbers in [1, m]
        odd = m - even # the number of odd numbers in [1, m]
        # dp[i][j][0/1] := the number of arrays of length i with j consecutive even
        # number pairs ending in an even number (0) or an odd number (1)
        dp = [[[0] * 2
                for _ in range(k + 1)]
               for _ in range(n + 1)]

        # Base case: arrays of length 1
        # For an array of length 1, we can't have any even number pairs yet.
        dp[1][0][0] = even
        dp[1][0][1] = odd

        for i in range(2, n + 1):
            for j in range(k + 1):
                # 1. Appending an even number to an array ending in an even number
                #     creates a new consecutive even number pair.
                # 2. Appending an even number to an array ending in an odd number.
                dp[i][j][0] = ((dp[i - 1][j - 1][0] if j > 0 else 0) * even +
                               dp[i - 1][j][1] * even) % MOD
                # 3. Appending an odd number to an array.
                dp[i][j][1] = sum(dp[i - 1][j]) * odd % MOD

        return sum(dp[n][k]) % MOD

```

2287. 334.py

Path: LeetCode-main/solutions/334. Increasing Triplet Subsequence/334.py

```
class Solution:
    def increasingTriplet(self, nums: list[int]) -> bool:
        first = math.inf
        second = math.inf

        for num in nums:
            if num <= first:
                first = num
            elif num <= second: # first < num <= second
                second = num
            else:
                return True # first < second < num (third)

        return False
```

2288. 3340.py

Path: LeetCode-main/solutions/3340. Check Balanced String/3340.py

```
class Solution:
    def isBalanced(self, num: str) -> bool:
        nums = list(map(int, num))
        return sum(nums[::2]) == sum(nums[1::2])
```

2289. 3341.py

Path: LeetCode-main/solutions/3341. Find Minimum Time to Reach Last Room I/3341.py

```

class Solution:
    def minTimeToReach(self, moveTime: list[list[int]]) -> int:
        return self._dijkstra(moveTime,
                               (0, 0), (len(moveTime) - 1, len(moveTime[0]) - 1))

    def _dijkstra(
        self,
        moveTime: list[list[int]],
        src: tuple[int, int],
        dst: tuple[int, int]
    ) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(moveTime)
        n = len(moveTime[0])
        dist = [[math.inf] * n for _ in range(m)]

        dist[0][0] = 0
        minHeap = [(0, src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if u == dst:
                return d
            i, j = u
            if d > dist[i][j]:
                continue
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                newDist = max(moveTime[x][y], d) + 1
                if newDist < dist[x][y]:
                    dist[x][y] = newDist
                    heapq.heappush(minHeap, (newDist, (x, y)))

        return -1

```

2290. 3342.py

Path: LeetCode-main/solutions/3342. Find Minimum Time to Reach Last Room II/3342.py

```

class Solution:
    # Similar to 3341. Find Minimum Time to Reach Last Room I
    def minTimeToReach(self, moveTime: list[list[int]]) -> int:
        return self._dijkstra(moveTime,
                               (0, 0),
                               (len(moveTime) - 1, len(moveTime[0]) - 1))

    def _dijkstra(
        self,
        moveTime: list[list[int]],
        src: tuple[int, int],
        dst: tuple[int, int]
    ) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(moveTime)
        n = len(moveTime[0])
        dist = [[math.inf] * n for _ in range(m)]

        dist[0][0] = 0
        minHeap = [(0, src)] # (d, (ux, uy))

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if u == dst:
                return d
            i, j = u
            if d > dist[i][j]:
                continue
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                newDist = max(moveTime[x][y], d) + (i + j) % 2 + 1
                if newDist < dist[x][y]:
                    dist[x][y] = newDist
                    heapq.heappush(minHeap, (newDist, (x, y)))

        return -1

```

2291. 3343.py

Path: LeetCode-main/solutions/3343. Count Number of Balanced Permutations/3343.py

```

class Solution:
    def countBalancedPermutations(self, num: str) -> int:
        nums = list(map(int, num))
        summ = sum(nums)
        if summ % 2 == 1:
            return 0

        nums.sort(reverse=True)

        @functools.lru_cache(None)
        def dp(even: int, odd: int, evenBalance: int) -> int:
            """
            Returns the number of permutations where there are `even` even indices
            left, `odd` odd indices left, and `evenBalance` is the target sum of the
            remaining numbers to be placed in even indices.
            """
            if evenBalance < 0:
                return 0
            if even == 0:
                return (evenBalance == 0) * math.factorial(odd)
            if odd == 0:
                return (sum(nums[-(even + odd):]) == evenBalance) * math.factorial(even)
            return (dp(even - 1, odd, evenBalance - nums[-(odd + even)]) * even +
                    dp(even, odd - 1, evenBalance) * odd)

        MOD = 1_000_000_007
        perm = functools.reduce(lambda x, y: x * math.factorial(y),
                                collections.Counter(nums).values(), 1)
        return (dp(even=(len(nums) + 1) // 2,
                    odd=len(nums) // 2,
                    evenBalance=summ // 2) // perm) % MOD

```

2292. 3344.py

Path: LeetCode-main/solutions/3344. Maximum Sized Array/3344.py

```

class Solution:
    def maxSizedArray(self, s: int) -> int:
        def getNumbersWithBitSet(n: int, i: int) -> int:
            """
            Returns the number of integers in [0, n - 1] with the i-th bit set.

            For the i-th bit, numbers in the range [0, n - 1] can be divided into
            groups of  $2^{(i + 1)}$  numbers. In each group, exactly half of the numbers
            have the i-th bit set.
            """
            groupSize = 1 << (i + 1)
            halfGroupSize = 1 << i
            fullGroups = n // groupSize
            remaining = max(0, (n % groupSize) - halfGroupSize)
            return fullGroups * halfGroupSize + remaining

        def getArraySum(n: int) -> int:
            """
            Returns the sum of all  $i * (j \text{ OR } k)$  values in 3D arrays of size  $n^3$ .

             $\text{sum}(i * (j \text{ OR } k))$ , where  $0 \leq i, j, k < n$ 
            =  $0 * (j \text{ OR } k) + 1 * (j \text{ OR } k) + \dots + (n - 1) * (j \text{ OR } k)$ 
            =  $(0 + 1 + \dots + n - 1) * \text{sum}(j \text{ OR } k)$ 
            =  $(n * (n - 1) / 2) * \text{sum}(j \text{ OR } k)$ 
            """
            arithmeticSum = n * (n - 1) // 2 #  $0 + 1 + \dots + n - 1$ 
            orSum = 0 # the sum of  $(j \text{ OR } k)$  values in 2D arrays of size  $n^2$ 
            for i in range(n.bit_length()):
                numbersWithoutBit = n - getNumbersWithBitSet(n, i)
                pairsWithBit = n**2 - numbersWithoutBit**2
                orSum += pairsWithBit * (1 << i) # Add contribution of this bit.
            return arithmeticSum * orSum

        if s == 0:
            return 1
        l = 0
        r = 1196 # when  $s = 10^{15}$ ,  $n = 1196$ 
        return bisect.bisect_right(range(l, r + 1), s, key=getArraySum) - 1 + 1

```

2293. 3345.py

Path: LeetCode-main/solutions/3345. Smallest Divisible Digit Product I/3345.py

```

class Solution:
    def smallestNumber(self, n: int, t: int) -> int:
        return next(num for num in range(n, n + 10)
                    if self._getDigitProd(num) % t == 0)

    def _getDigitProd(self, num: int) -> int:
        digitProd = 1
        while num > 0:
            digitProd *= num % 10
            num //= 10
        return digitProd

```

2294. 3346.py

Path: LeetCode-main/solutions/3346. Maximum Frequency of an Element After Performing Operations I/3346.py


```

from sortedcontainers import SortedDict

class Solution:
    def maxFrequency(self, nums: list[int], k: int, numOperations: int) -> int:
        ans = 1
        adjustable = 0
        count = collections.Counter(nums)
        line = SortedDict()
        candidates = set()

        for num in nums:
            line[num - k] = line.get(num - k, 0) + 1
            line[num + k + 1] = line.get(num + k + 1, 0) - 1
            candidates.add(num)
            candidates.add(num - k)
            candidates.add(num + k + 1)

        for num in sorted(candidates):
            adjustable += line.get(num, 0)
            adjusted = adjustable - count[num]
            ans = max(ans, count[num] + min(numOperations, adjusted))

        return ans

```

2295. 3347.py

Path: LeetCode-main/solutions/3347. Maximum Frequency of an Element After Performing Operations II/3347.py

```

from sortedcontainers import SortedDict

class Solution:
    # Same as 3346. Maximum Frequency of an Element After Performing Operations I
    def maxFrequency(self, nums: list[int], k: int, numOperations: int) -> int:
        ans = 1
        adjustable = 0
        count = collections.Counter(nums)
        line = SortedDict()
        candidates = set()

        for num in nums:
            line[num - k] = line.get(num - k, 0) + 1
            line[num + k + 1] = line.get(num + k + 1, 0) - 1
            candidates.add(num)
            candidates.add(num - k)
            candidates.add(num + k + 1)

        for num in sorted(candidates):
            adjustable += line.get(num, 0)
            adjusted = adjustable - count[num]
            ans = max(ans, count[num] + min(numOperations, adjusted))

        return ans

```

2296. 3348.py

Path: LeetCode-main/solutions/3348. Smallest Divisible Digit Product II/3348.py

```

FACTOR_COUNTS = {

```

```

0: collections.Counter(),
1: collections.Counter(),
2: collections.Counter([2]),
3: collections.Counter([3]),
4: collections.Counter([2, 2]),
5: collections.Counter([5]),
6: collections.Counter([2, 3]),
7: collections.Counter([7]),
8: collections.Counter([2, 2, 2]),
9: collections.Counter([3, 3]),
}

class Solution:
    def smallestNumber(self, num: str, t: int) -> str:
        primeCount, isDivisible = self._getPrimeCount(t)
        if not isDivisible:
            return '-1'

        factorCount = self._getFactorCount(primeCount)
        if sum(factorCount.values()) > len(num):
            return ''.join(factor * freq for factor, freq in factorCount.items())

        primeCountPrefix = sum((FACTOR_COUNTS[int(c)]
                                for c in num), start=collections.Counter())
        firstZeroIndex = next((i for i, d in enumerate(num) if d == '0'), len(num))
        if firstZeroIndex == len(num) and primeCount <= primeCountPrefix:
            return num

        for i, c in reversed(list(enumerate(num))):
            d = int(c)
            # Remove the current digit's factors from primeCountPrefix.
            primeCountPrefix -= FACTOR_COUNTS[d]
            spaceAfterThisDigit = len(num) - 1 - i
            if i <= firstZeroIndex:
                for biggerDigit in range(d + 1, 10):
                    # Compute the required factors after replacing with a larger digit.
                    factorsAfterReplacement = self._getFactorCount(
                        primeCount - primeCountPrefix - FACTOR_COUNTS[biggerDigit]
                    )
                    # Check if the replacement is possible within the available space.
                    if sum(factorsAfterReplacement.values()) <= spaceAfterThisDigit:
                        # Fill extra space with '1', if any, and construct the result.
                        fillOnes = spaceAfterThisDigit - sum(
                            factorsAfterReplacement.values()
                        )
                        return (
                            num[:i] # Keep the prefix unchanged.
                            + str(biggerDigit) # Replace the current digit.
                            + '1' * fillOnes # Fill remaining space with '1'.
                            + ''.join(factor * freq for factor,
                                      freq in factorsAfterReplacement.items())
                        )

            # No solution of the same length exists, so we need to extend the number
            # by prepending '1's and adding the required factors.
            factorCount = self._getFactorCount(primeCount)
            return (
                '1' * (len(num) + 1 - sum(factorCount.values()))
                + ''.join(factor * freq for factor, freq in factorCount.items())
            )

        def _getPrimeCount(self, t: int) -> tuple[dict[int, int], bool]:
            """
            Returns the count of prime factors of t and if t is divisible by 2, 3, 5, 7.
            """

```

```

count = collections.Counter()
for prime in [2, 3, 5, 7]:
    while t % prime == 0:
        t //= prime
        count[prime] += 1
return count, t == 1

def _getFactorCount(self, count: dict[int, int]) -> dict[str, int]:
    """Returns the required factors to form the smallest number."""
    count8, remaining2 = divmod(count[2], 3) # 2^3 = 8
    count9, count3 = divmod(count[3], 2) # 3^2 = 9
    count4, count2 = divmod(remaining2, 2) # 2^2 = 4
    # Combine 2 and 3 to 6 if both are present.
    count2, count3, count6 = ((0, 0, 1) if count2 == 1 and count3 == 1
                               else (count2, count3, 0))
    # Combine 3 and 4 to 2 and 6 if both are present.
    count2, count6, count3, count4 = ((1, 1, 0, 0)
                                       if count3 == 1 and count4 == 1
                                       else (count2, count6, count3, count4))
    return {'2': count2, '3': count3, '4': count4, '5': count[5],
            '6': count6, '7': count[7], '8': count8, '9': count9}

```

2297. 3349.py

Path: LeetCode-main/solutions/3349. Adjacent Increasing Subarrays Detection I/3349.py

```

class Solution:
    def hasIncreasingSubarrays(self, nums: list[int], k: int) -> bool:
        increasing = 1
        prevIncreasing = 0

        for a, b in itertools.pairwise(nums):
            if b > a:
                increasing += 1
            else:
                prevIncreasing = increasing
                increasing = 1
            if increasing // 2 >= k or min(prevIncreasing, increasing) >= k:
                return True

        return False

```

2298. 335.py

Path: LeetCode-main/solutions/335. Self Crossing/335.py

```

class Solution:
    def isSelfCrossing(self, x: list[int]) -> bool:
        if len(x) <= 3:
            return False

        for i in range(3, len(x)):
            if x[i - 2] <= x[i] and x[i - 1] <= x[i - 3]:
                return True
            if i >= 4 and x[i - 1] == x[i - 3] and x[i - 2] <= x[i] + x[i - 4]:
                return True
            if i >= 5 and x[i - 4] <= x[i - 2] and x[i - 2] <= x[i] + x[i - 4] and x[i - 1] <= x[i - 3] and x[i - 3] <= x[i - 1] + x[i - 5]:
                return True

        return False

```

2299. 3350.py

Path: LeetCode-main/solutions/3350. Adjacent Increasing Subarrays Detection II/3350.py

```

class Solution:
    # Similar to 3349. Adjacent Increasing Subarrays Detection I
    def maxIncreasingSubarrays(self, nums: list[int]) -> int:
        ans = 0
        increasing = 1
        prevIncreasing = 0

        for a, b in itertools.pairwise(nums):
            if b > a:
                increasing += 1
            else:
                prevIncreasing = increasing
                increasing = 1
            ans = max(ans, increasing // 2)
            ans = max(ans, min(prevIncreasing, increasing))

        return ans

```

2300. 3351.py

Path: LeetCode-main/solutions/3351. Sum of Good Subsequences/3351.py

```

class Solution:
    def sumOfGoodSubsequences(self, nums: list[int]) -> int:
        MOD = 10**9 + 7
        maxNum = max(nums)
        # endsIn[i] := the number of good subsequences ending in i
        endsIn = [0] * (maxNum + 2)
        # dp[i] := the sum of good subsequences ending in i
        dp = [0] * (maxNum + 2)

        for num in nums:
            seqsToAppend = 1 + endsIn[num - 1] + endsIn[num + 1]
            dp[num] = (seqsToAppend * num +
                      (dp[num] + dp[num - 1] + dp[num + 1])) % MOD
            endsIn[num] += seqsToAppend % MOD

        return sum(dp) % MOD

```

2301. 3352.py

Path: LeetCode-main/solutions/3352. Count K-Reducible Numbers Less Than N/3352.py

```
class Solution:
    def countKReducibleNumbers(self, s: str, k: int) -> int:
        MOD = 1_000_000_007
        ops = self._getOps(s)

        @functools.lru_cache(None)
        def dp(i: int, setBits: int, tight: bool) -> int:
            """
            Returns the number of positive integers less than n that are k-reducible,
            considering the i-th digit, where `setBits` is the number of set bits in
            the current number, and `tight` indicates if the current digit is
            tightly bound.
            """
            if i == len(s):
                return int(ops[setBits] < k and not tight)

            res = 0
            maxDigit = int(s[i]) if tight else 1

            for d in range(maxDigit + 1):
                nextTight = tight and (d == maxDigit)
                res += dp(i + 1, setBits + d, nextTight)
                res %= MOD
            return res

        return dp(0, 0, True) - 1 # - 0

    def _getOps(self, s: str) -> int:
        """Returns the number of operations to reduce a number to 0."""
        ops = [0] * (len(s) + 1)
        for num in range(2, len(s) + 1):
            ops[num] = 1 + ops[num.bit_count()]
        return ops
```

2302. 3353.py

Path: LeetCode-main/solutions/3353. Minimum Total Operations/3353.py

```
class Solution:
    def minOperations(self, nums: list[int]) -> int:
        return sum(a != b for a, b in itertools.pairwise(nums))
```

2303. 3354-2.py

Path: LeetCode-main/solutions/3354. Make Array Elements Equal to Zero/3354-2.py

```

class Solution:
    def countValidSelections(self, nums: list[int]) -> int:
        ans = 0
        prefix = 0
        suffix = sum(nums)

        for i, num in enumerate(nums):
            suffix -= num
            prefix += num
            if num > 0:
                continue
            if prefix == suffix:
                ans += 2 # Go to either direction.
            if abs(prefix - suffix) == 1:
                ans += 1 # Go to the direction with the larger sum.

        return ans

```

2304. 3354.py

Path: LeetCode-main/solutions/3354. Make Array Elements Equal to Zero/3354.py

```

class Solution:
    def countValidSelections(self, nums: list[int]) -> int:
        ans = 0
        prefix = list(itertools.accumulate(nums))
        suffix = list(itertools.accumulate(nums[::-1]))[::-1]

        for i, num in enumerate(nums):
            if num > 0:
                continue
            if prefix[i] == suffix[i]:
                ans += 2 # Go to either direction.
            if abs(prefix[i] - suffix[i]) == 1:
                ans += 1 # Go to the direction with the larger sum.

        return ans

```

2305. 3355.py

Path: LeetCode-main/solutions/3355. Zero Array Transformation I/3355.py

```

class Solution:
    def isZeroArray(self, nums: list[int], queries: list[list[int]]) -> bool:
        line = [0] * (len(nums) + 1)
        decrement = 0

        for l, r in queries:
            line[l] += 1
            line[r + 1] -= 1

        for i, num in enumerate(nums):
            decrement += line[i]
            if decrement < num:
                return False

        return True

```

2306. 3356.py

Path: LeetCode-main/solutions/3356. Zero Array Transformation II/3356.py

```
class Solution:
    def minZeroArray(self, nums: list[int], queries: list[list[int]]) -> int:
        line = [0] * (len(nums) + 1)
        decrement = 0
        k = 0

        for i, num in enumerate(nums):
            while decrement + line[i] < num:
                if k == len(queries):
                    return -1
                l, r, val = queries[k]
                k += 1
                if r < i:
                    continue
                line[max(l, i)] += val
                line[r + 1] -= val
                decrement += line[i]

        return k
```

2307. 3357.py

Path: LeetCode-main/solutions/3357. Minimize the Maximum Adjacent Element Difference/3357.py

```
class Solution:
    def minDifference(self, nums: list[int]) -> int:
        maxPositiveGap = 0
        mn = 1_000_000_000
        mx = 0

        for a, b in itertools.pairwise(nums):
            if (a == -1) != (b == -1):
                positive = max(a, b)
                mn = min(mn, positive)
                mx = max(mx, positive)
            else:
                maxPositiveGap = max(maxPositiveGap, abs(a - b))

        l = maxPositiveGap
        r = (mx - mn + 1) // 2
        return bisect.bisect_left(
            range(l, r), True,
            key=lambda m: self._check(nums, m, mn + m, mx - m)) + 1

    def _check(self, nums: list[int], m: int, x: int, y: int) -> bool:
        """
        Returns True if it's possible have `m` as maximum absolute difference
        between adjacent numbers, where -1s are replaced with `x` or `y`.
        """
        gapLength = 0
        prev = 0

        for num in nums:
            if num == -1:
                gapLength += 1
                continue
            if prev > 0 and gapLength > 0:
                if gapLength == 1 and not self._checkSingleGap(prev, num, m, x, y):
                    return False
            else:
                if num - prev > m:
                    return False
            prev = num
            gapLength = 0

        return True

    def _checkSingleGap(self, prev: int, num: int, m: int, x: int, y: int) -> bool:
        """
        Returns True if it's possible have `m` as maximum absolute difference
        between adjacent numbers, where -1s are replaced with `x` or `y`.
        """
        if num - prev > m:
            return False
        return True
```

```

        return False
    if gapLength > 1 and not self._checkMultipleGaps(prev, num, m, x, y):
        return False
    prev = num
    gapLength = 0

# Check leading gaps
if nums[0] == -1:
    num = next((num for num in nums if num != -1), -1)
    if num != -1 and not self._checkBoundaryGaps(num, m, x, y):
        return False

# Check trailing gaps
if nums[-1] == -1:
    num = next((num for num in reversed(nums) if num != -1), -1)
    if num != -1 and not self._checkBoundaryGaps(num, m, x, y):
        return False

return True

def _checkSingleGap(self, a: int, b: int, m: int, x: int, y: int) -> bool:
    """
    Returns true if it's possible to have at most `m` as the minimized maximum
    difference for a sequence with a single -1 between two numbers.
    e.g. [a, -1, b] can be filled with either x or y.
    """
    gapWithX = max(abs(a - x), abs(b - x)) # [a, x, b]
    gapWithY = max(abs(a - y), abs(b - y)) # [a, y, b]
    return min(gapWithX, gapWithY) <= m

def _checkMultipleGaps(self, a: int, b: int, m: int, x: int, y: int) -> bool:
    """
    Returns true if it's possible to have at most `m` as the minimized maximum
    difference for a sequence with multiple -1s between two numbers.
    e.g. [a, -1, -1, ..., -1, b] can be filled with x and y.
    """
    ax = abs(a - x)
    ay = abs(a - y)
    bx = abs(b - x)
    by = abs(b - y)
    xy = abs(x - y)
    gapAllX = max(ax, bx) # [a, x, x, ..., x, b]
    gapAllY = max(ay, by) # [a, y, y, ..., y, b]
    gapXToY = max(ax, xy, by) # [a, x, ..., y, b]
    gapYToX = max(ay, xy, bx) # [a, y, ..., x, b]
    return min(gapAllX, gapAllY, gapXToY, gapYToX) <= m

def _checkBoundaryGaps(self, a: int, m: int, x: int, y: int) -> bool:
    """
    Returns true if it's possible to have at most `m` as the minimized maximum
    difference for a boundary sequence starting or ending with -1s.
    e.g. [a, -1, -1, ...] or [..., -1, -1, a].
    """
    gapAllX = abs(a - x) # [x, x, ..., x, a]
    gapAllY = abs(a - y) # [y, y, ..., y, a]
    return min(gapAllX, gapAllY) <= m

```

2308. 3359.py

Path: LeetCode-main/solutions/3359. Find Sorted Submatrices With Maximum Element at Most K/3359.py


```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    subarrayWidth: int
    rowIndex: int
    accumulatedSubmatrices: int

class Solution:
    def countSubmatrices(self, grid: list[list[int]], k: int) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = 0
        # dp[i][j] := the number of valid subarrays ending in grid[i][j]
        dp = [[0] * n for _ in range(m)]
        # stacks[j] := the stack of valid
        # (subarray width, row index, number of accumulated submatrices) ending in
        # column j
        stacks: list[T] = [T(0, -1, 0)] for _ in range(n)

        for i, row in enumerate(grid):
            for j, num in enumerate(row):
                if num > k:
                    stacks[j] = T(0, i, 0)
                else:
                    dp[i][j] = 1
                    if j > 0 and row[j - 1] <= k and row[j - 1] >= row[j]:
                        # Extend the valid subarrays to the current number.
                        dp[i][j] += dp[i][j - 1]
                    width = dp[i][j]
                    # Remove subarray widths greater than the current count since they
                    # will become invalid.
                    while stacks[j] and width < stacks[j][-1].subarrayWidth:
                        stacks[j].pop()
                    height = i - stacks[j][-1].rowIndex
                    newSubmatrices = width * height
                    accumulatedSubmatrices = (stacks[j][-1].accumulatedSubmatrices +
                                              newSubmatrices)
                    ans += accumulatedSubmatrices
                    stacks[j].append(T(width, i, accumulatedSubmatrices))

        return ans

```

2309. 336.py

Path: LeetCode-main/solutions/336. Palindrome Pairs/336.py

```
class Solution:
    def palindromePairs(self, words: list[str]) -> list[list[int]]:
        ans = []
        dict = {word[::-1]: i for i, word in enumerate(words)}

        for i, word in enumerate(words):
            if "" in dict and dict[""] != i and word == word[::-1]:
                ans.append([i, dict[""]])

            for j in range(1, len(word) + 1):
                l = word[:j]
                r = word[j:]
                if l in dict and dict[l] != i and r == r[::-1]:
                    ans.append([i, dict[l]])
                if r in dict and dict[r] != i and l == l[::-1]:
                    ans.append([dict[r], i])

        return ans
```

2310. 3360.py

Path: LeetCode-main/solutions/3360. Stone Removal Game/3360.py

```
class Solution:
    def canAliceWin(self, n: int) -> bool:
        for stones in range(10, -1, -1):
            if stones > n:
                return stones % 2 == 1
            n -= stones
```

2311. 3361.py

Path: LeetCode-main/solutions/3361. Shift Distance Between Two Strings/3361.py

```

class Solution:
    def shiftDistance(
        self,
        s: str,
        t: str,
        nextCost: list[int],
        previousCost: list[int]
    ) -> int:
        ans = 0
        # prev[i][j]: the prev cost to shift from ('a' + i) to ('a' + j)
        prev = [[0] * 26 for _ in range(26)]
        # next[i][j]: the next cost to shift from ('a' + i) to ('a' + j)
        next = [[0] * 26 for _ in range(26)]

        for i in range(26):
            cost = 0
            for j in range(26):
                next[i][(i + j) % 26] = cost
                cost += nextCost[(i + j) % 26]

        for i in range(26):
            cost = 0
            for j in range(26):
                prev[i][(i - j + 26) % 26] = cost
                cost += previousCost[(i - j + 26) % 26]

        for a, b in zip(s, t):
            i = ord(a) - ord('a')
            j = ord(b) - ord('a')
            ans += min(next[i][j], prev[i][j])

        return ans

```

2312. 3362.py

Path: LeetCode-main/solutions/3362. Zero Array Transformation III/3362.py

```

from sortedcontainers import SortedList

class Solution:
    def maxRemoval(self, nums: list[int], queries: list[list[int]]) -> int:
        q = collections.deque(sorted(queries))
        available = SortedList() # available `r`s
        running = SortedList() # running `r`s

        for i, num in enumerate(nums):
            while q and q[0][0] <= i:
                available.add(q.popleft()[1])
            while running and running[0] < i:
                running.pop(0)
            while num > len(running):
                if not available or available[-1] < i:
                    return -1
                running.add(available.pop())

        return len(available)

```

2313. 3363.py

Path: LeetCode-main/solutions/3363. Find the Maximum Number of Fruits Collected/3363.py

```
class Solution:
    def maxCollectedFruits(self, fruits: list[list[int]]) -> int:
        n = len(fruits)

        def getTopLeft() -> int:
            return sum(fruits[i][i] for i in range(n))

        def getTopRight() -> int:
            # dp[i][j] := the number of fruits collected from (0, n - 1) to (i, j)
            dp = [[0] * n for _ in range(n)]
            dp[0][n-1] = fruits[0][n-1]
            for x in range(n):
                for y in range(n):
                    if x >= y and (x, y) != (n - 1, n - 1):
                        continue
                    for dx, dy in [(1, -1), (1, 0), (1, 1)]:
                        i = x - dx
                        j = y - dy
                        if i < 0 or i == n or j < 0 or j == n:
                            continue
                        if i < j < n - 1 - i:
                            continue
                        dp[x][y] = max(dp[x][y], dp[i][j] + fruits[x][y])
            return dp[-1][-1]

        def getBottomLeft() -> int:
            # dp[i][j] := the number of fruits collected from (n - 1, 0) to (i, j)
            dp = [[0] * n for _ in range(n)]
            dp[-1][0] = fruits[-1][0]
            for y in range(n):
                for x in range(n):
                    if x <= y and (x, y) != (n - 1, n - 1):
                        continue
                    for dx, dy in [(-1, 1), (0, 1), (1, 1)]:
                        i = x - dx
                        j = y - dy
                        if i < 0 or i == n or j < 0 or j == n:
                            continue
                        if j < i < n - 1 - j:
                            continue
                        dp[x][y] = max(dp[x][y], dp[i][j] + fruits[x][y])
            return dp[-1][-1]

        return getTopLeft() + getTopRight() + getBottomLeft() - 2 * fruits[-1][-1]
```

2314. 3364.py

Path: LeetCode-main/solutions/3364. Minimum Positive Sum Subarray/3364.py

```
class Solution:
    def minimumSumSubarray(self, nums: list[int], l: int, r: int) -> int:
        ans = math.inf

        for windowSize in range(1, r + 1):
            windowSum = sum(nums[:windowSize])
            if windowSum > 0:
                ans = min(ans, windowSum)
            for i in range(windowSize, len(nums)):
                windowSum -= nums[i - windowSize]
                windowSum += nums[i]
                if windowSum > 0:
                    ans = min(ans, windowSum)

        return -1 if ans == math.inf else ans
```

2315. 3365.py

Path: LeetCode-main/solutions/3365. Rearrange K Substrings to Form Target String/3365.py

```
class Solution:
    def isPossibleToRearrange(self, s: str, t: str, k: int) -> bool:
        n = len(s)
        return (collections.Counter(s[i:i + n // k] for i in range(0, n, n // k)) ==
                collections.Counter(t[i:i + n // k] for i in range(0, n, n // k)))
```

2316. 3366-2.py

Path: LeetCode-main/solutions/3366. Minimum Array Sum/3366-2.py

```

class Solution:
    def minArraySum(self, nums: list[int], k: int, op1: int, op2: int) -> int:
        n = len(nums)
        # dp[i][j][k] := the minimum sum of nums[i..n - 1] with j operations of op1
        # and k operations of op2
        dp = [[math.inf] * (op2 + 1)
              for _ in range(op1 + 1)]
              for _ in range(n + 1)]

        # Base case: When index reaches the end of the array, the result is 0.
        for i in range(op1 + 1):
            for j in range(op2 + 1):
                dp[n][i][j] = 0

        for i in range(n - 1, -1, -1):
            for o1 in range(op1 + 1):
                for o2 in range(op2 + 1):
                    summ = nums[i] + dp[i + 1][o1][o2]
                    if o1 > 0:
                        summ = min(summ, (nums[i] + 1) // 2 + dp[i + 1][o1 - 1][o2])
                    if o2 > 0 and nums[i] >= k:
                        summ = min(summ, nums[i] - k + dp[i + 1][o1][o2 - 1])
                    if o1 > 0 and o2 > 0:
                        if (nums[i] + 1) // 2 >= k:
                            summ = min(summ,
                                         (nums[i] + 1) // 2 - k + dp[i + 1][o1 - 1][o2 - 1])
                        if nums[i] >= k:
                            summ = min(summ,
                                         (nums[i] - k + 1) // 2 + dp[i + 1][o1 - 1][o2 - 1])
                    dp[i][o1][o2] = summ

        return dp[0][op1][op2]

```

2317. 3366.py

Path: LeetCode-main/solutions/3366. Minimum Array Sum/3366.py

```

class Solution:
    def minArraySum(self, nums: list[int], k: int, op1: int, op2: int) -> int:
        @functools.lru_cache(None)
        def dp(i: int, op1: int, op2: int) -> int:
            """
            Returns the minimum sum of nums[i..n - 1] with `op1` operations of op1 and
            `op2` operations of op2.
            """
            if i == len(nums):
                return 0
            res = nums[i] + dp(i + 1, op1, op2)
            if op1 > 0:
                res = min(res, (nums[i] + 1) // 2 + dp(i + 1, op1 - 1, op2))
            if op2 > 0 and nums[i] >= k:
                res = min(res, nums[i] - k + dp(i + 1, op1, op2 - 1))
            if op1 > 0 and op2 > 0:
                if (nums[i] + 1) // 2 >= k:
                    res = min(res, (nums[i] + 1) // 2 - k + dp(i + 1, op1 - 1, op2 - 1))
                if nums[i] >= k:
                    res = min(res, (nums[i] - k + 1) // 2 + dp(i + 1, op1 - 1, op2 - 1))
            return res

        return dp(0, op1, op2)

```

2318. 3367.py

Path: LeetCode-main/solutions/3367. Maximize Sum of Weights after Edge Removals/3367.py

```
class Solution:
    def maximizeSumOfWeights(self, edges: list[list[int]], k: int) -> int:
        graph = [[] for _ in range(len(edges) + 1)]

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        def dfs(u: int, prev: int) -> tuple[int, int]:
            """
            Returns
            (the weight sum of the subtree rooted at u with at most k - 1 children,
            the weight sum of the subtree rooted at u with at most k children).
            """
            weightSum = 0
            diffs = []
            for v, w in graph[u]:
                if v == prev:
                    continue
                subK1, subK = dfs(v, u)
                weightSum += subK
                # If picking (u, v) makes the sum larger, we should pick it.
                diffs.append(max(0, subK1 - subK + w))
            return (weightSum + sum(heapq.nlargest(k - 1, diffs)),
                    weightSum + sum(heapq.nlargest(k, diffs)))

        return dfs(0, -1)[1]
```

2319. 3369.py

Path: LeetCode-main/solutions/3369. Design an Array Statistics Tracker/3369.py

```

from sortedcontainers import SortedList

class StatisticsTracker:
    def __init__(self):
        self.q = collections.deque()
        self.count = collections.Counter()
        self.sortedList = SortedList()
        self.modeMaxHeap = [] # (frequency, number)
        self.sum = 0

    def addNumber(self, number: int) -> None:
        self.q.append(number)
        self.count[number] += 1
        self.sortedList.add(number)
        heapq.heappush(self.modeMaxHeap, (-self.count[number], number))
        self.sum += number

    def removeFirstAddedNumber(self) -> None:
        number = self.q.popleft()
        self.count[number] -= 1
        self.sortedList.remove(number)
        # Note: No need to update the heap now; we'll clean up stale entries when
        # getting the mode.
        self.sum -= number

    def getMean(self) -> int:
        return self.sum // len(self.q)

    def getMedian(self) -> int:
        return self.sortedList[len(self.sortedList) // 2]

    def getMode(self) -> int:
        # Removes stale heap entries where frequency no longer matches.
        while self.modeMaxHeap:
            frequency = -self.modeMaxHeap[0][0]
            number = self.modeMaxHeap[0][1]
            if self.count[number] == frequency:
                return number
            heapq.heappop(self.modeMaxHeap)

```

2320. 337.py

Path: LeetCode-main/solutions/337. House Robber III/337.py

```

class Solution:
    def rob(self, root: TreeNode | None) -> int:
        def robOrNot(root: TreeNode | None) -> tuple:
            if not root:
                return (0, 0)

            robLeft, notRobLeft = robOrNot(root.left)
            robRight, notRobRight = robOrNot(root.right)

            return (root.val + notRobLeft + notRobRight,
                    max(robLeft, notRobLeft) + max(robRight, notRobRight))

        return max(robOrNot(root))

```


2321. 3370.py

Path: LeetCode-main/solutions/3370. Smallest Number With All Set Bits/3370.py

```
class Solution:
    def smallestNumber(self, n: int) -> int:
        return (1 << n.bit_length()) - 1
```

2322. 3371.py

Path: LeetCode-main/solutions/3371. Identify the Largest Outlier in an Array/3371.py

```
class Solution:
    def getLargestOutlier(self, nums: list[int]) -> int:
        ans = -math.inf
        summ = sum(nums)
        count = collections.Counter(nums)

        for num in nums:
            withoutNum = summ - num
            if withoutNum % 2 == 0:
                specialSum = withoutNum // 2 # the sum of special numbers
                if count[specialSum] > (1 if num == specialSum else 0):
                    ans = max(ans, num)

        return ans
```

2323. 3372.py

Path: LeetCode-main/solutions/3372. Maximize the Number of Target Nodes After Connecting Trees I/3372.py

```

class Solution:
    def maxTargetNodes(
        self,
        edges1: list[list[int]],
        edges2: list[list[int]],
        k: int
    ) -> list[int]:
        graph1 = self._buildGraph(edges1)
        graph2 = self._buildGraph(edges2)
        maxReachableInGraph2 = 0

        if k > 0:
            for i in range(len(edges2) + 1):
                maxReachableInGraph2 = max(maxReachableInGraph2,
                                           self._dfs(graph2, i, -1, k - 1))

        return [maxReachableInGraph2 + self._dfs(graph1, i, -1, k)
                for i in range(len(edges1) + 1)]

    def _dfs(self, graph: list[list[int]], u: int, prev: int, k: int) -> int:
        """Returns the number of nodes that can be reached from u with k steps."""
        if k == 0:
            return 1
        res = 0
        for v in graph[u]:
            if v != prev:
                res += self._dfs(graph, v, u, k - 1)
        return 1 + res

    def _buildGraph(self, edges: list[list[int]]) -> list[list[int]]:
        graph = [[] for _ in range(len(edges) + 1)]
        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)
        return graph

```

2324. 3373.py

Path: LeetCode-main/solutions/3373. Maximize the Number of Target Nodes After Connecting Trees II/3373.py

```

class Solution:
    def maxTargetNodes(
        self,
        edges1: list[list[int]],
        edges2: list[list[int]]
    ) -> list[int]:
        n = len(edges1) + 1
        m = len(edges2) + 1
        graph1 = self._buildGraph(edges1)
        graph2 = self._buildGraph(edges2)
        parity1 = [False] * n
        parity2 = [False] * m # placeholder (parity2 is not used)
        even1 = self._dfs(graph1, 0, -1, parity1, True)
        even2 = self._dfs(graph2, 0, -1, parity2, True)
        odd1 = n - even1
        odd2 = m - even2

        # Can connect the current node in tree1 to either an even node or an odd
        # node in tree2.
        return [(even1 if parity1[i] else odd1) + max(even2, odd2)
                for i in range(n)]

    def _dfs(
        self,
        graph: list[list[int]],
        u: int,
        prev: int,
        parity: list[bool],
        isEven: bool
    ) -> int:
        """
        Returns the number of nodes that can be reached from u with even steps.
        """
        res = 1 if isEven else 0
        parity[u] = isEven
        for v in graph[u]:
            if v != prev:
                res += self._dfs(graph, v, u, parity, not isEven)
        return res

    def _buildGraph(self, edges: list[list[int]]) -> list[list[int]]:
        graph = [[] for _ in range(len(edges) + 1)]
        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)
        return graph

```

2325. 3375.py

Path: LeetCode-main/solutions/3375. Minimum Operations to Make Array Values Equal to K/3375.py

```

class Solution:
    def minOperations(self, nums: list[int], k: int) -> int:
        numsSet = set(nums)
        mn = min(nums)
        if mn < k:
            return -1
        if mn > k:
            return len(numsSet)
        return len(numsSet) - 1

```

2326. 3377.py

Path: LeetCode-main/solutions/3377. Digit Operations to Make Two Integers Equal/3377.py

```
class Solution:
    def minOperations(self, n: int, m: int) -> int:
        isPrime = self._sieveEratosthenes(10000)
        if isPrime[n] or isPrime[m]:
            return -1
        return self._dijkstra(n, m, isPrime)

    def _dijkstra(self, src: int, dst: int, isPrime: list[bool]) -> int:
        seen = {src}
        minHeap = [(src, src)] # (cost, num)

        while minHeap:
            cost, curr = heapq.heappop(minHeap)
            if curr == dst:
                return cost
            s = list(str(curr))
            for i, c in enumerate(s):
                if c < '9':
                    s[i] = str(int(c) + 1)
                    nextNum = int(''.join(s))
                    if not isPrime[nextNum] and nextNum not in seen:
                        heapq.heappush(minHeap, (cost + nextNum, nextNum))
                        seen.add(nextNum)
                    s[i] = c
                if c > '0' and not (i == 0 and c == '1'):
                    s[i] = str(int(c) - 1)
                    nextNum = int(''.join(s))
                    if not isPrime[nextNum] and nextNum not in seen:
                        heapq.heappush(minHeap, (cost + nextNum, nextNum))
                        seen.add(nextNum)
                    s[i] = c

        return -1

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False
        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime
```

2327. 3378.py

Path: LeetCode-main/solutions/3378. Count Connected Components in LCM Graph/3378.py

```

class UnionFind:
    def __init__(self):
        self.id = {}
        self.rank = collections.Counter()

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if u not in self.id:
            self.id[u] = u
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def countComponents(self, nums: list[int], threshold: int) -> int:
        uf = UnionFind()

        for num in nums:
            for multiple in range(2 * num, threshold + 1, num):
                uf.unionByRank(num, multiple)

        return len(set(uf.find(num) for num in nums))

```

2328. 3379.py

Path: LeetCode-main/solutions/3379. Transformed Array/3379.py

```

class Solution:
    def constructTransformedArray(self, nums: list[int]) -> list[int]:
        n = len(nums)
        return [nums[(i + num % n + n) % n]
                for i, num in enumerate(nums)]

```

2329. 338.py

Path: LeetCode-main/solutions/338. Counting Bits/338.py

```

class Solution:
    def countBits(self, n: int) -> list[int]:
        # f(i) := i's number of 1s in bitmask
        # f(i) = f(i / 2) + i % 2
        ans = [0] * (n + 1)

        for i in range(1, n + 1):
            ans[i] = ans[i // 2] + (i & 1)

        return ans

```

2330. 3381.py

Path: LeetCode-main/solutions/3381. Maximum Subarray Sum With Length Divisible by K/3381.py

```

class Solution:
    def maxSubarraySum(self, nums: list[int], k: int) -> int:
        ans = -math.inf
        prefix = 0
        # minPrefix[i % k] := the minimum prefix sum of the first i numbers
        minPrefix = [math.inf] * k
        minPrefix[k - 1] = 0

        for i, num in enumerate(nums):
            prefix += num
            ans = max(ans, prefix - minPrefix[i % k])
            minPrefix[i % k] = min(minPrefix[i % k], prefix)

        return ans

```

2331. 3383.py

Path: LeetCode-main/solutions/3383. Minimum Runes to Add to Cast Spell/3383.py

```

class Solution:
    def minRunesToAdd(
        self,
        n: int,
        crystals: list[int],
        flowFrom: list[int],
        flowTo: list[int]
    ) -> int:
        graph = [[] for _ in range(n)]
        reversedGraph = [[] for _ in range(n)]

        for u, v in zip(flowFrom, flowTo):
            graph[u].append(v)
            reversedGraph[v].append(u)

        # Identify Strongly Connected Components (SCC) using Kosaraju's Algorithm.
        seen = set()
        orderStack = []
        componentIds = [-1] * n
        componentCount = 0

        for i in range(n):
            if i not in seen:
                self._kosaraju(graph, i, seen, orderStack)

        while orderStack:

```

```

    u = orderStack.pop()
    if componentIds[u] == -1:
        self._identifySCC(reversedGraph, u, componentIds, componentCount)
        componentCount += 1

# Track crystal-containing components and inter-component edges.
hasCrystal = [False] * componentCount
hasInterComponentEdge = [False] * componentCount

for u in crystals:
    hasCrystal[componentIds[u]] = True

for u, v in zip(flowFrom, flowTo):
    id1 = componentIds[u]
    id2 = componentIds[v]
    if id1 != id2: # Edge is inter-component.
        hasInterComponentEdge[id2] = True

return sum(not hasCrystal[i] and not hasInterComponentEdge[i]
           for i in range(componentCount))

def _kosaraju(
    self,
    graph: list[list[int]],
    u: int,
    seen: set[int],
    orderStack: list
) -> None:
    """Creates a topological order stack using Kosaraju's Algorithm."""
    seen.add(u)
    for v in graph[u]:
        if v not in seen:
            self._kosaraju(graph, v, seen, orderStack)
    orderStack.append(u)

def _identifySCC(
    self,
    reversedGraph: list[list[int]],
    u: int,
    componentIds: list[int],
    componentId: int
) -> None:
    """Assigns component IDs during SCC identification in the second DFS."""
    if componentIds[u] != -1:
        return
    componentIds[u] = componentId
    for v in reversedGraph[u]:
        if componentIds[v] == -1:
            self._identifySCC(reversedGraph, v, componentIds, componentId)

```

2332. 3385.py

Path: LeetCode-main/solutions/3385. Minimum Time to Break Locks II/3385.py

```

class Solution:
    def findMinimumTime(self, strength: list[int]) -> int:
        costs = [[(s + turn - 1) // turn
                  for s in strength]
                 for turn in range(1, len(strength) + 1)]
        return self._hungarian(costs)[-1]

    def _hungarian(self, costs):
        """

```

Returns an array `res` of length n (costs.length), with `res[i]` equaling the minimum cost to assign the first (i + 1) turns to the first (i + 1) locks using Hungarian algorithm, where costs[i][j] is the energy required to break j-th lock in i-th turn.

https://en.wikipedia.org/wiki/Hungarian_algorithm

"""

```
numLocks = len(costs)
turnPotentials = [0] * numLocks
lockPotentials = [0] * (numLocks + 1)
lockAssignments = [-1] * (numLocks + 1)
res = []

for currentTurn in range(numLocks):
    currentLock = numLocks
    lockAssignments[currentLock] = currentTurn
    minReducedCosts = [math.inf] * (numLocks + 1)
    previousLockAssignments = [-1] * (numLocks + 1)
    locksInOptimalPath = [False] * (numLocks + 1)

    while lockAssignments[currentLock] != -1:
        locksInOptimalPath[currentLock] = True
        assignedTurn = lockAssignments[currentLock]
        minCostDelta = math.inf
        nextLock = None

        for lock in range(numLocks):
            if not locksInOptimalPath[lock]:
                reducedCost = (
                    costs[assignedTurn][lock] -
                    turnPotentials[assignedTurn] -
                    lockPotentials[lock]
                )
                oldMin = minReducedCosts[lock]
                minReducedCosts[lock] = min(oldMin, reducedCost)
                if minReducedCosts[lock] < oldMin:
                    previousLockAssignments[lock] = currentLock
                if minReducedCosts[lock] < minCostDelta:
                    minCostDelta = minReducedCosts[lock]
                    nextLock = lock

        for lock in range(numLocks + 1):
            if locksInOptimalPath[lock]:
                turnPotentials[lockAssignments[lock]] += minCostDelta
                lockPotentials[lock] -= minCostDelta
            else:
                minReducedCosts[lock] -= minCostDelta

        currentLock = nextLock

    while currentLock != numLocks:
        lockAssignments[currentLock] = lockAssignments[previousLockAssignments[currentLock]]
        currentLock = previousLockAssignments[currentLock]

    res.append(-lockPotentials[numLocks])

return res
```

2333. 3386.py

Path: LeetCode-main/solutions/3386. Button with Longest Push Time/3386.py


```

class Solution:
    def buttonWithLongestTime(self, events: list[list[int]]) -> int:
        ans = 0
        maxTimeTaken = 0
        prevTime = 0

        for index, time in events:
            timeTaken = time - prevTime
            if timeTaken > maxTimeTaken or timeTaken == maxTimeTaken and index < ans:
                maxTimeTaken = timeTaken
                ans = index
            prevTime = time

        return ans

```

2334. 3387.py

Path: LeetCode-main/solutions/3387. Maximize Amount After Two Days of Conversions/3387.py

```

class Solution:
    def maxAmount(
        self,
        initialCurrency: str,
        pairs1: list[list[str]],
        rates1: list[float],
        pairs2: list[list[str]],
        rates2: list[float]
    ) -> float:
        # dp[currency] := the maximum amount of money to convert to `currency`
        dp: dict[str, float] = collections.defaultdict(float)
        dp[initialCurrency] = 1.0
        self._bellman(dp, pairs1, rates1)
        self._bellman(dp, pairs2, rates2)
        return dp[initialCurrency]

    def _bellman(
        self,
        dp: dict[str, float],
        pairs: list[list[str]],
        rates: list[float]
    ) -> None:
        for _ in range(len(pairs)):
            for (start, target), rate in zip(pairs, rates):
                dp[target] = max(dp[target], dp[start] * rate)
                dp[start] = max(dp[start], dp[target] / rate)

```

2335. 3388.py

Path: LeetCode-main/solutions/3388. Count Beautiful Splits in an Array/3388.py

```

class Solution:
    def beautifulSplits(self, nums: list[int]) -> int:
        n = len(nums)
        # z[start][i] := the z array of nums[i..n) with respect to nums[start..n)
        z = [self._zFunction(nums, start)
              for start in range(n)]
        # nums1 | nums2 | nums3 = nums[0..i] | nums[i + 1..j] | nums[j + 1..n - 1]
        return sum((j - i >= i + 1 and z[0][i + 1] >= i + 1) # nums1 is a prefix of nums2
                   or z[i + 1][j + 1] >= j - i # nums2 is a suffix of nums3.
                   for i in range(n - 2)
                   for j in range(i + 1, n - 1))

    def _zFunction(self, nums: list[int], start: int) -> list[int]:
        """
        Returns the z array, where z[i] is the length of the longest prefix of
        nums[i..n) which is also a prefix of nums[start..n).
        """
        n = len(nums)
        z = [0] * n
        l = start
        r = start
        for i in range(1 + start, n):
            if i < r:
                z[i] = min(r - i, z[i - l + start])
            while i + z[i] < n and nums[z[i] + start] == nums[i + z[i]]:
                z[i] += 1
            if i + z[i] > r:
                l = i
                r = i + z[i]
        return z

```

2336. 3389.py

Path: LeetCode-main/solutions/3389. Minimum Operations to Make Character Frequencies Equal/3389.py

```

class Solution:
    def makeStringGood(self, s: str) -> int:
        count = [0] * 26
        for c in s:
            count[ord(c) - ord('a')] += 1
        return min(self._getMinOperations(count, target)
                   for target in range(1, max(count) + 1))

    def _getMinOperations(self, count: list[int], target: int) -> int:
        # dp[i] represents the minimum number of operations to make the frequency of
        # (i..25)-th (0-indexed) letters equal to `target`.
        dp = [0] * 27

        for i in range(25, -1, -1):
            # 1. Delete all the i-th letters.
            deleteAllToZero = count[i]
            # 2. Insert/delete the i-th letters to have `target` number of letters.
            deleteOrInsertToTarget = abs(target - count[i])
            dp[i] = min(deleteAllToZero, deleteOrInsertToTarget) + dp[i + 1]
            if i + 1 < 26 and count[i + 1] < target:
                nextDeficit = target - count[i + 1]
                # Make the frequency of the i-th letter equal to the `target` or 0.
                needToChange = count[i] if count[i] <= target else count[i] - target
                changeToTarget = (
                    # 3. Change all the i-th letters to the next letter and then
                    # insert the remaining deficit for the next letter.
                    needToChange + (nextDeficit - needToChange) if nextDeficit > needToChange
                    # 4. Change `nextDeficit` i-th letters to the next letter and
                    # then delete the remaining i-th letters.
                    else nextDeficit + (needToChange - nextDeficit)
                )
            dp[i] = min(dp[i], changeToTarget + dp[i + 2])

        return dp[0]

```

2337. 339-2.py

Path: LeetCode-main/solutions/339. Nested List Weight Sum/339-2.py

```

class Solution:
    def depthSum(self, nestedList: list[NestedInteger]) -> int:
        ans = 0

        def dfs(nestedList: list[NestedInteger], depth: int) -> None:
            nonlocal ans
            for ni in nestedList:
                if ni.isInteger():
                    ans += ni.getInteger() * depth
                else:
                    dfs(ni.getList(), depth + 1)

        dfs(nestedList, 1)
        return ans

```

2338. 339.py

Path: LeetCode-main/solutions/339. Nested List Weight Sum/339.py

```

class Solution:
    def depthSum(self, nestedList: list[NestedInteger]) -> int:
        ans = 0
        depth = 0
        q = collections.deque()

        def addIntegers(nestedList: list[NestedInteger]) -> None:
            for ni in nestedList:
                q.append(ni)

        addIntegers(nestedList)

        while q:
            depth += 1
            for _ in range(len(q)):
                ni = q.popleft()
                if ni.isInteger():
                    ans += ni.getInteger() * depth
                else:
                    addIntegers(ni.getList())

        return ans

```

2339. 3391.py

Path: LeetCode-main/solutions/3391. Design a 3D Binary Matrix with Efficient Layer Tracking/3391.py

```

class Matrix3D:
    def __init__(self, n: int):
        self.isSet = set()
        # count[x] := the number of set cells in the x-th layer
        self.count = collections.Counter()
        # (count[x], x)
        self.pairs: SortedList = SortedList(key=lambda x: (-x[0], -x[1]))
        self.pairs.update((0, x) for x in range(n))

    def setCell(self, x: int, y: int, z: int) -> None:
        if (x, y, z) in self.isSet:
            return
        self.isSet.add((x, y, z))
        self.pairs.remove((self.count[x], x))
        self.count[x] += 1
        self.pairs.add((self.count[x], x))

    def unsetCell(self, x: int, y: int, z: int) -> None:
        if (x, y, z) not in self.isSet:
            return
        self.isSet.remove((x, y, z))
        self.pairs.remove((self.count[x], x))
        self.count[x] -= 1
        self.pairs.add((self.count[x], x))

    def largestMatrix(self) -> int:
        return self.pairs[0][1]

```

2340. 3392.py

Path: LeetCode-main/solutions/3392. Count Subarrays of Length Three With a Condition/3392.py

```
class Solution:
    def countSubarrays(self, nums: list[int]) -> int:
        return sum(b == (a + c) * 2
                    for a, b, c in zip(nums, nums[1:], nums[2:]))
```

2341. 3393-2.py

Path: LeetCode-main/solutions/3393. Count Paths With the Given XOR Value/3393-2.py

```
class Solution:
    def countPathsWithXorValue(self, grid, k):
        MOD = 1_000_000_007
        MAX = 15
        m = len(grid)
        n = len(grid[0])
        # dp[i][j][xors] := the number of paths from (i, j) to (m - 1, n - 1) with
        # XOR value `xors`
        dp = [[[0] * (MAX + 1)
                for _ in range(n)]
               for _ in range(m)]

        dp[-1][-1][grid[-1][-1]] = 1

        for i in reversed(range(m)):
            for j in reversed(range(n)):
                for xors in range(MAX + 1):
                    if i - 1 >= 0:
                        newXor = xors ^ grid[i - 1][j]
                        dp[i - 1][j][newXor] += dp[i][j][xors]
                        dp[i - 1][j][newXor] %= MOD
                    if j - 1 >= 0:
                        newXor = xors ^ grid[i][j - 1]
                        dp[i][j - 1][newXor] += dp[i][j][xors]
                        dp[i][j - 1][newXor] %= MOD

        return dp[0][0][k]
```

2342. 3393-3.py

Path: LeetCode-main/solutions/3393. Count Paths With the Given XOR Value/3393-3.py

```

class Solution:
    def countPathsWithXorValue(self, grid, k):
        MOD = 1_000_000_007
        MAX = 15
        m = len(grid)
        n = len(grid[0])
        # dp[i][j][xors] := the number of paths from (0, 0) to (i, j) with XOR
        # value `xors`
        dp = [[[0] * (MAX + 1)
                for _ in range(n)]
               for _ in range(m)]

        dp[0][0][grid[0][0]] = 1

        for i in range(m):
            for j in range(n):
                for xors in range(MAX + 1):
                    if i + 1 < m:
                        newXor = xors ^ grid[i + 1][j]
                        dp[i + 1][j][newXor] += dp[i][j][xors]
                        dp[i + 1][j][newXor] %= MOD
                    if j + 1 < n:
                        newXor = xors ^ grid[i][j + 1]
                        dp[i][j + 1][newXor] += dp[i][j][xors]
                        dp[i][j + 1][newXor] %= MOD

        return dp[-1][-1][k]

```

2343. 3393.py

Path: LeetCode-main/solutions/3393. Count Paths With the Given XOR Value/3393.py

```

class Solution:
    def countPathsWithXorValue(self, grid: list[list[int]], k: int) -> int:
        MOD = 1_000_000_007
        m = len(grid)
        n = len(grid[0])

        @functools.lru_cache(None)
        def count(i: int, j: int, xors: int) -> int:
            """
            Return the number of paths from (i, j) to (m - 1, n - 1) with XOR value
            `xors`.
            """
            if i == m or j == n:
                return 0
            xors ^= grid[i][j]
            if i == m - 1 and j == n - 1:
                return int(xors == k)
            right = count(i, j + 1, xors)
            down = count(i + 1, j, xors)
            return (right + down) % MOD

        return count(0, 0, 0)

```

2344. 3394.py

Path: LeetCode-main/solutions/3394. Check if Grid can be Cut into Sections/3394.py

```
class Solution:
    def checkValidCuts(self, n: int, rectangles: list[list[int]]) -> bool:
        xs = [(startX, endX) for startX, _, endX, _ in rectangles]
        ys = [(startY, endY) for _, startY, _, endY in rectangles]
        return max(self._countMerged(xs),
                    self._countMerged(ys)) >= 3

    def _countMerged(self, intervals: list[tuple[int, int]]) -> int:
        count = 0
        prevEnd = 0
        for start, end in sorted(intervals):
            if start < prevEnd:
                prevEnd = max(prevEnd, end)
            else:
                prevEnd = end
                count += 1
        return count
```

2345. 3395-2.py

Path: LeetCode-main/solutions/3395. Subsequences with a Unique Middle Mode I/3395-2.py

```

class Solution:
    def subsequencesWithMiddleMode(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        ans = 0
        p = collections.Counter() # prefix counter
        s = collections.Counter(nums) # suffix counter

        def nC2(n: int) -> int:
            return n * (n - 1) // 2

        for i, a in enumerate(nums):
            s[a] -= 1

            l = i
            r = len(nums) - i - 1

            # Start with all possible subsequences with `a` as the middle number.
            ans += nC2(l) * nC2(r)

            # Minus the cases where the frequency of `a` is 1, so it's not a mode.
            ans -= nC2(l - p[a]) * nC2(r - s[a])

            for b in p | s:
                if b == a:
                    continue

                # Minus the cases where the middle number is not a "unique" mode.
                ans -= p[a] * p[b] * s[b] * (r - s[a] - s[b]) # [a b] a [b c]
                ans -= s[a] * s[b] * p[b] * (l - p[a] - p[b]) # [b c] a [a b]
                ans -= nC2(p[b]) * s[a] * (r - s[a] - s[b]) # [b b] a [a c]
                ans -= nC2(s[b]) * p[a] * (l - p[a] - p[b]) # [a c] a [b b]

                # Minus the cases where the middle number is not a mode.
                ans -= nC2(p[b]) * s[a] * s[b] # [b b] a [a b]
                ans -= nC2(s[b]) * p[a] * p[b] # [a b] a [b b]

            ans %= MOD
            p[a] += 1

        return ans

```

2346. 3395-3.py

Path: LeetCode-main/solutions/3395. Subsequences with a Unique Middle Mode I/3395-3.py

```

# Recall from solution 1 that after counting all the subsequences with `a` as
# the middle mode number, we need to subtract the cases where `a` is not a
# unique mode or not a mode.
#
# To avoid the need of looping through all numbers that are not `a`, we can
# maintain the sums that are not related to `a` in the loop.
#
# So, during the simplification of the formula, keep the running sums of
# pss, spp, pp, ss, and ps as the first item.
# (for cleaner notation, abbreviate p[b] and s[b] to just p and s)
#
# sum(b != a) (p[a] * p * s) * (r - s[a] - s)
#               + (s[a] * s * p) * (l - p[a] - p)
#               + (p, 2) * s[a] * (r - s[a])
#               + (s, 2) * p[a] * (l - p[a])
#
# sum(b != a) (p * s) * (p[a] * (r - s[a])) + (p * s^2) * (-p[a])
# ..

```



```

#         + (s * p) * (s[a] * (1 - p[a])) + (s * p^2) * (-s[a])
#         + (p^2 - p) * (s[a] * (r - s[a]) / 2)
#         + (s^2 - s) * (p[a] * (1 - p[a]) / 2)

class Solution:
    def subsequencesWithMiddleMode(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        ans = 0
        p = collections.Counter() # prefix counter
        s = collections.Counter(nums) # suffix counter

        def nC2(n: int) -> int:
            return n * (n - 1) // 2

        pss = 0
        spp = 0
        pp = 0
        ss = sum(freq**2 for freq in s.values())
        ps = 0

        for i, a in enumerate(nums):
            # Update running sums after decrementing s[a].
            pss += p[a] * (-s[a]**2 + (s[a] - 1)**2)
            spp += -p[a]**2 # (-s[a] + (s[a] - 1)) * p[a]**2
            ss += -s[a]**2 + (s[a] - 1)**2
            ps += -p[a] # -p[a] * (-s[a] + (s[a] - 1))

            s[a] -= 1

            l = i
            r = len(nums) - i - 1

            # Start with all possible subsequences with `a` as the middle number.
            ans += nC2(l) * nC2(r)

            # Minus the cases where the frequency of `a` is 1, so it's not a mode.
            ans -= nC2(l - p[a]) * nC2(r - s[a])

            # Minus the values where `b != a`.
            pss_ = pss - p[a] * s[a]**2
            spp_ = spp - s[a] * p[a]**2
            pp_ = pp - p[a]**2
            ss_ = ss - s[a]**2
            ps_ = ps - p[a] * s[a]
            p_ = 1 - p[a]
            s_ = r - s[a]

            # Minus the cases where the `a` is not a "unique" mode or not a mode.
            ans -= ps_ * (p[a] * (r - s[a])) + pss_ * (-p[a])
            ans -= ps_ * (s[a] * (1 - p[a])) + spp_ * (-s[a])
            ans -= (pp_ - p_) * s[a] * (r - s[a]) // 2
            ans -= (ss_ - s_) * p[a] * (1 - p[a]) // 2
            ans %= MOD

            # Update running sums after incrementing p[a].
            pss += s[a]**2 # (-p[a] + (p[a] + 1)) * s[a]**2
            spp += s[a] * (-p[a]**2 + (p[a] + 1)**2)
            pp += -p[a]**2 + (p[a] + 1)**2
            ps += s[a] # (-p[a] + (p[a] + 1)) * s[a]

            p[a] += 1

        return ans

```

2347. 3395.py

Path: LeetCode-main/solutions/3395. Subsequences with a Unique Middle Mode I/3395.py

```
class Solution:
    def __init__(self):
        self.MOD = 1_000_000_007

    def subsequencesWithMiddleMode(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        left = collections.Counter()
        right = collections.Counter()

        for i in range(2):
            left[nums[i]] += 1

        for i in range(2, n):
            right[nums[i]] += 1

        for i in range(2, n - 2):
            num = nums[i]
            right[num] -= 1
            if right[num] == 0:
                del right[num]

            leftCount = left[num]
            rightCount = right[num]
            leftOther = i - leftCount
            rightOther = n - 1 - i - rightCount

            # count[mode] = 5 -- [a a] a [a a]
            ans += math.comb(leftCount, 2) * math.comb(rightCount, 2)

            # count[mode] = 4 -- [a a] a [a ?]
            ans += math.comb(leftCount, 2) * rightCount * rightOther

            # count[mode] = 4 -- [a ?] a [a a]
            ans += leftCount * leftOther * math.comb(rightCount, 2)

            # count[mode] = 3 -- [a a] a [? ?]
            ans += math.comb(leftCount, 2) * math.comb(rightOther, 2)

            # count[mode] = 3 -- [? ?] a [a a]
            ans += math.comb(leftOther, 2) * math.comb(rightCount, 2)

            # count[mode] = 3 -- [a ?] a [a ?]
            ans += leftCount * leftOther * rightCount * rightOther

            # count[mode] = 2 -- [a ?] a [? ?]
            ans += leftCount * self._calc(num, leftOther, rightOther, left, right)

            # count[mode] = 2 -- [? ?] a [a ?]
            ans += rightCount * self._calc(num, rightOther, leftOther, right, left)

        ans %= self.MOD
        left[num] += 1

        return ans

    def _calc(
        self,
        a: int,
        other1: int
```

```

other1: int,
other2: int,
count1: dict[int, int],
count2: dict[int, int]
) -> int:
    """
    Returns the count of subsequences that have `a` as the middle number, where
    invalid subsequences are excluded.
    """
    # [a ?] a [? ?]
    res = (other1 * math.comb(other2, 2)) % self.MOD

    for b, b1 in count1.items():
        if b == a:
            continue
        b2 = count2[b]
        # Exclude triples -- [a b] a [b b].
        res = (res - b1 * math.comb(b2, 2)) % self.MOD
        # Exclude doubles -- [a b] a [b ?].
        res = (res - b1 * b2 * (other2 - b2)) % self.MOD

    for b, b2 in count2.items():
        if b == a:
            continue
        b1 = count1[b]
        # Exclude doubles -- [a ?] a [b b].
        res = (res - (other1 - b1) * math.comb(b2, 2)) % self.MOD

    return (res + self.MOD) % self.MOD

```

2348. 3396.py

Path: LeetCode-main/solutions/3396. Minimum Number of Operations to Make Elements in Array Distinct/3396.py

```

class Solution:
    def minimumOperations(self, nums: list[int]) -> int:
        seen = set()
        for i, num in reversed(list(enumerate(nums))):
            if num in seen:
                return (i + 1 + 2) // 3 # ceil((i + 1) / 3)
            seen.add(num)
        return 0

```

2349. 3397.py

Path: LeetCode-main/solutions/3397. Maximum Number of Distinct Elements After Operations/3397.py

```

class Solution:
    def maxDistinctElements(self, nums: list[int], k: int) -> int:
        ans = 0
        occupied = -math.inf

        for num in sorted(nums):
            if occupied < num + k:
                occupied = max(occupied + 1, num - k)
                ans += 1

        return ans

```

2350. 3398.py

Path: LeetCode-main/solutions/3398. Smallest Substring With Identical Characters I/3398.py

```
class Solution:
    def minLength(self, s: str, numOps: int) -> int:
        def getMinOps(k: int) -> int:
            """
            Returns the minimum number of operations needed to make all groups of
            identical characters of length k or less.
            """
            if k == 1:
                res = sum(1 for i, c in enumerate(s) if int(c) == i % 2)
                return min(res, len(s) - res)

            res = 0
            runningLen = 1

            for a, b in itertools.pairwise(s):
                if a == b:
                    runningLen += 1
                else:
                    res += runningLen // (k + 1)
                    runningLen = 1

            return res + runningLen // (k + 1)

        return bisect_left(range(1, len(s) + 1),
                           True, key=lambda m: getMinOps(m) <= numOps)
```

2351. 3399.py

Path: LeetCode-main/solutions/3399. Smallest Substring With Identical Characters II/3399.py

```
class Solution:
    # Same as 3398. Smallest Substring With Identical Characters I
    def minLength(self, s: str, numOps: int) -> int:
        def getMinOps(k: int) -> int:
            """
            Returns the minimum number of operations needed to make all groups of
            identical characters of length k or less.
            """
            if k == 1:
                res = sum(1 for i, c in enumerate(s) if int(c) == i % 2)
                return min(res, len(s) - res)

            res = 0
            runningLen = 1

            for a, b in itertools.pairwise(s):
                if a == b:
                    runningLen += 1
                else:
                    res += runningLen // (k + 1)
                    runningLen = 1

            return res + runningLen // (k + 1)

        return bisect_left(range(1, len(s) + 1),
                           True, key=lambda m: getMinOps(m) <= numOps)
```

2352. 34.py

Path: LeetCode-main/solutions/34. Find First and Last Position of Element in Sorted Array/34.py

```
class Solution:
    def searchRange(self, nums: list[int], target: int) -> list[int]:
        l = bisect_left(nums, target)
        if l == len(nums) or nums[l] != target:
            return -1, -1
        r = bisect_right(nums, target) - 1
        return l, r
```

2353. 340.py

Path: LeetCode-main/solutions/340. Longest Substring with At Most K Distinct Characters/340.py

```
class Solution:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        ans = 0
        distinct = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            if count[c] == 1:
                distinct += 1
            while distinct == k + 1:
                count[s[l]] -= 1
                if count[s[l]] == 0:
                    distinct -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

2354. 3400.py

Path: LeetCode-main/solutions/3400. Maximum Number of Matching Indices After Right Shifts/3400.py

```
class Solution:
    def maximumMatchingIndices(self, nums1: list[int], nums2: list[int]) -> int:
        n = len(nums1)
        ans = 0

        for shift in range(n):
            matches = 0
            for i, num2 in enumerate(nums2):
                if nums1[(i + shift) % n] == num2:
                    matches += 1
            ans = max(ans, matches)

        return ans
```

2355. 3402.py

Path: LeetCode-main/solutions/3402. Minimum Operations to Make Columns Strictly Increasing/3402.py

```

class Solution:
    def minimumOperations(self, grid: list[list[int]]) -> int:
        ans = 0

        for j in range(len(grid[0])):
            for i in range(1, len(grid)):
                if grid[i][j] <= grid[i - 1][j]:
                    ans += grid[i - 1][j] - grid[i][j] + 1
                    grid[i][j] = grid[i - 1][j] + 1

        return ans

```

2356. 3403.py

Path: LeetCode-main/solutions/3403. Find the Lexicographically Largest String From the Box I/3403.py

```

class Solution:
    def answerString(self, word: str, numFriends: int) -> str:
        if numFriends == 1:
            return word
        s = self._lastSubstring(word)
        sz = len(word) - numFriends + 1
        return s[:min(len(s), sz)]

    # Same as 1163. Last Substring in Lexicographical Order
    def _lastSubstring(self, s: str) -> str:
        i = 0
        j = 1
        k = 0 # the number of the same letters of s[i..n) and s[j..n)

        while j + k < len(s):
            if s[i + k] == s[j + k]:
                k += 1
            elif s[i + k] > s[j + k]:
                # Skip s[j..j + k) and advance to s[j + k + 1] to find a possible
                # lexicographically larger substring since s[i..i + k) == s[j..j + k)
                # and s[i + k] > s[j + k).
                j = j + k + 1
                k = 0
            else:
                # Skip s[i..i + k) and advance to s[i + k + 1] or s[j] to find a
                # possible lexicographically larger substring since
                # s[i..i + k) == s[j..j + k) and s[i + k] < s[j + k).
                # Note that it's unnecessary to explore s[i + k + 1..j) if
                # i + k + 1 < j since they are already explored by j.
                i = max(i + k + 1, j)
                j = i + 1
                k = 0

        return s[i:]

```

2357. 3404.py

Path: LeetCode-main/solutions/3404. Count Special Subsequences/3404.py

```
class Solution:
    def numberOfSubsequences(self, nums: list[int]) -> int:
        n = len(nums)
        mx = max(nums)
        ans = 0
        count = [[0] * (mx + 1) for _ in range(mx + 1)]

        # nums[p] * nums[r] == nums[q] * nums[s]
        # nums[p] / nums[q] == nums[s] / nums[r]
        for r in range(4, n - 1 - 2 + 1):
            q = r - 2
            for p in range(0, q - 2 + 1):
                g = math.gcd(nums[p], nums[q])
                count[nums[p] // g][nums[q] // g] += 1
            for s in range(r + 2, n):
                g = math.gcd(nums[s], nums[r])
                ans += count[nums[s] // g][nums[r] // g]

        return ans
```

2358. 3405.py

Path: LeetCode-main/solutions/3405. Count the Number of Arrays with K Matching Adjacent Elements/3405.py

```
class Solution:
    def countGoodArrays(self, n: int, m: int, k: int) -> int:
        MOD = 1_000_000_007
        return m * pow(m - 1, n - k - 1, MOD) * math.comb(n - 1, k) % MOD
```

2359. 3406.py

Path: LeetCode-main/solutions/3406. Find the Lexicographically Largest String From the Box II/3406.py

```

class Solution:
    def answerString(self, word: str, numFriends: int) -> str:
        if numFriends == 1:
            return word
        s = self._lastSubstring(word)
        sz = len(word) - numFriends + 1
        return s[:min(len(s), sz)]

# Same as 1163. Last Substring in Lexicographical Order
def _lastSubstring(self, s: str) -> str:
    i = 0
    j = 1
    k = 0 # the number of the same letters of s[i..n) and s[j..n)

    while j + k < len(s):
        if s[i + k] == s[j + k]:
            k += 1
        elif s[i + k] > s[j + k]:
            # Skip s[j..j + k) and advance to s[j + k + 1] to find a possible
            # lexicographically larger substring since s[i..i + k) == s[j..j + k)
            # and s[i + k] > s[j + k).
            j = j + k + 1
            k = 0
        else:
            # Skip s[i..i + k) and advance to s[i + k + 1] or s[j] to find a
            # possible lexicographically larger substring since
            # s[i..i + k) == s[j..j + k) and s[i + k] < s[j + k).
            # Note that it's unnecessary to explore s[i + k + 1..j) if
            # i + k + 1 < j since they are already explored by j.
            i = max(i + k + 1, j)
            j = i + 1
            k = 0

    return s[i:]

```

2360. 3407.py

Path: LeetCode-main/solutions/3407. Substring Matching Pattern/3407.py

```

class Solution:
    def hasMatch(self, s: str, p: str) -> bool:
        prefix, suffix = p.split('*')
        i = s.find(prefix)
        return i != -1 and s.find(suffix, i + len(prefix)) != -1

```

2361. 3408.py

Path: LeetCode-main/solutions/3408. Design Task Manager/3408.py


```

from dataclasses import dataclass
from sortedcontainers import SortedDict, SortedSet

@dataclass(frozen=True)
class Task:
    userId: int
    taskId: int
    priority: int

    def __lt__(self, other):
        if self.priority == other.priority:
            return self.taskId > other.taskId
        return self.priority > other.priority

class TaskManager:
    def __init__(self, tasks: list[list[int]]):
        self.taskMap = SortedDict() # {taskId: Task}, keeps tasks sorted by taskId
        self.taskSet = SortedSet() # Stores tasks sorted by priority and taskId
        for task in tasks:
            self.add(task[0], task[1], task[2])

    def add(self, userId: int, taskId: int, priority: int) -> None:
        task = Task(userId, taskId, priority)
        self.taskMap[taskId] = task
        self.taskSet.add(task)

    def edit(self, taskId: int, newPriority: int) -> None:
        task = self.taskMap[taskId]
        self.taskSet.remove(task)
        editedTask = Task(task.userId, taskId, newPriority)
        self.taskSet.add(editedTask)
        self.taskMap[taskId] = editedTask

    def rmv(self, taskId: int) -> None:
        task = self.taskMap[taskId]
        self.taskSet.remove(task)
        del self.taskMap[taskId]

    def execTop(self):
        if not self.taskSet:
            return -1
        task = self.taskSet.pop(0)
        del self.taskMap[task.taskId]
        return task.userId

```

2362. 3409.py

Path: LeetCode-main/solutions/3409. Longest Subsequence With Decreasing Adjacent Difference/3409.py

```

class Solution:
    def longestSubsequence(self, nums: list[int]) -> int:
        mx = max(nums)
        # dp[num][diff] := the length of the longest subsequence ending in `num`
        # s.t. the last absolute difference between consecutive elements is `diff`
        dp = [[0] * (mx + 1) for _ in range(mx + 1)]

        for num in nums:
            for prev in range(1, mx + 1):
                diff = abs(num - prev)
                dp[num][diff] = max(dp[num][diff], dp[prev][diff] + 1)
            # dp[num][diff] := max(dp[num][j]) for j >= diff
            for j in range(mx - 1, -1, -1):
                dp[num][j] = max(dp[num][j], dp[num][j + 1])

        return max(map(max, dp))

```

2363. 341-2.py

Path: LeetCode-main/solutions/341. Flatten Nested List Iterator/341-2.py

```

class NestedIterator:
    def __init__(self, nestedList: list[NestedInteger]):
        self.stack: list[NestedInteger] = []
        self.addInteger(nestedList)

    def next(self) -> int:
        return self.stack.pop().getInteger()

    def hasNext(self) -> bool:
        while self.stack and not self.stack[-1].isInteger():
            self.addInteger(self.stack.pop().getList())
        return self.stack

# addInteger([1, [4, [6]]]) . stack = [[4, [6]], 1]
# addInteger([4, [6]]) . stack = [[6], 4]
# addInteger([6]) . stack = [6]
def addInteger(self, nestedList: list[NestedInteger]) -> None:
    for n in reversed(nestedList):
        self.stack.append(n)

```

2364. 341.py

Path: LeetCode-main/solutions/341. Flatten Nested List Iterator/341.py

```

class NestedIterator:
    def __init__(self, nestedList: list[NestedInteger]):
        self.q = collections.deque()
        self.addInteger(nestedList)

    def next(self) -> int:
        return self.q.popleft()

    def hasNext(self) -> bool:
        return self.q

    def addInteger(self, nestedList: list[NestedInteger]) -> None:
        for ni in nestedList:
            if ni.isInteger():
                self.q.append(ni.getInteger())
            else:
                self.addInteger(ni.getList())

```

2365. 3410-2.py

Path: LeetCode-main/solutions/3410. Maximize Subarray Sum After Removing All Occurrences of One Element/3410-2.py

```

class Solution:
    def maxSubarraySum(self, nums: list[int]) -> int:
        ans = max(nums)
        prefix = 0
        minPrefix = 0
        # the minimum prefix sum that can have a negative number removed
        modifiedMinPrefix = 0
        count = collections.Counter()
        # minPrefixPlusRemoval[num] := the minimum prefix sum plus removed `num`
        minPrefixPlusRemoval = {}

        for num in nums:
            prefix += num
            ans = max(ans, prefix - modifiedMinPrefix)
            if num < 0:
                count[num] += 1
                minPrefixPlusRemoval[num] = (
                    min(minPrefixPlusRemoval.get(num, 0), minPrefix) + num)
                modifiedMinPrefix = min(modifiedMinPrefix,
                                         count[num] * num,
                                         minPrefixPlusRemoval[num])
            minPrefix = min(minPrefix, prefix)
            modifiedMinPrefix = min(modifiedMinPrefix, minPrefix)

        return ans

```

2366. 3411.py

Path: LeetCode-main/solutions/3411. Maximum Subarray With Equal Products/3411.py

```
class Solution:
    def maxLength(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0

        for i in range(n):
            prod = 1
            l = 1
            g = 0
            for j in range(i, n):
                prod *= nums[j]
                l = math.lcm(l, nums[j])
                g = math.gcd(g, nums[j])
                if prod == l * g:
                    ans = max(ans, j - i + 1)

        return ans
```

2367. 3412.py

Path: LeetCode-main/solutions/3412. Find Mirror Score of a String/3412.py

```
class Solution:
    def calculateScore(self, s: str) -> int:
        ans = 0
        indices = [[] for _ in range(26)]

        for i, c in enumerate(s):
            index = ord(c) - ord('a')
            oppositeIndex = 25 - index
            if indices[oppositeIndex]:
                ans += i - indices[oppositeIndex].pop()
            else:
                indices[index].append(i)

        return ans
```

2368. 3413.py

Path: LeetCode-main/solutions/3413. Maximum Coins From K Consecutive Bags/3413.py

```
class Solution:
    def maximumCoins(self, coins: list[list[int]], k: int) -> int:
        return max(self._slide(coins, k),
                    self._slide([[-r, -1, c] for l, r, c in coins], k))

    def _slide(self, coins: list[list[int]], k: int) -> int:
        coins.sort()
        res = 0
        windowSum = 0
        j = 0
        for li, ri, ci in coins: # Consider the number line [li..li + k).
            rightBoundary = li + k

            # [lj, rj] is fully in [li..li + k).
            while j + 1 < len(coins) and coins[j + 1][0] < rightBoundary:
                lj, rj, cj = coins[j]
                windowSum += (rj - lj + 1) * cj
                j += 1

            # [lj, rj] may be partially in [li..li + k).
            last = 0
            if j < len(coins) and coins[j][0] < rightBoundary:
                lj, rj, cj = coins[j]
                last = (min(rightBoundary - 1, rj) - lj + 1) * cj

            res = max(res, windowSum + last)
            windowSum -= (ri - li + 1) * ci
        return res
```

2369. 3414.py

Path: LeetCode-main/solutions/3414. Maximum Score of Non-overlapping Intervals/3414.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    weight: int
    selected: tuple[int]

    def __iter__(self):
        yield self.weight
        yield self.selected

class Solution:
    def maximumWeight(self, intervals: list[list[int]]) -> list[int]:
        intervals = sorted((*interval, i) for i, interval in enumerate(intervals))

        @functools.lru_cache(None)
        def dp(i: int, quota: int) -> T:
            """
            Returns the maximum weight and the selected intervals for intervals[i..n),
            where `quota` is the number of intervals that can be selected.
            """
            if i == len(intervals) or quota == 0:
                return T(0, ())

            skip = dp(i + 1, quota)

            _, r, weight, originalIndex = intervals[i]
            j = bisect.bisect_right(intervals, (r, math.inf))
            nextRes = dp(j, quota - 1)
            pick = T(weight + nextRes.weight,
                    sorted((originalIndex, *nextRes.selected)))
            return (pick if (pick.weight > skip.weight or
                            pick.weight == skip.weight and pick.selected < skip.selected)
                    else skip)

        return list(dp(0, 4).selected)

```

2370. 3416.py

Path: LeetCode-main/solutions/3416. Subsequences with a Unique Middle Mode II/3416.py

```

# Recall from solution 1 that after counting all the subsequences with `a` as
# the middle mode number, we need to subtract the cases where `a` is not a
# unique mode or not a mode.
#
# To avoid the need of looping through all numbers that are not `a`, we can
# maintain the sums that are not related to `a` in the loop.
#
# So, during the simplification of the formula, keep the running sums of
# pss, spp, pp, ss, and ps as the first item.
# (for cleaner notation, abbreviate p[b] and s[b] to just p and s)
#
# sum(b != a) (p[a] * p * s) * (r - s[a] - s)
#              + (s[a] * s * p) * (1 - p[a] - p)
#              + (p, 2) * s[a] * (r - s[a])
#              + (s, 2) * p[a] * (1 - p[a])
#
# sum(b != a) (p * s) * (p[a] * (r - s[a])) + (p * s^2) * (-p[a])
#              + (s * p) * (s[a] * (1 - p[a])) + (s * p^2) * (-s[a])
#              + (p^2 - p) * (s[a] * (r - s[a]) / 2)
#
#
#

```

```
#         + (s^2 - s) * (p[a] * (1 - p[a]) / 2)
```

```
class Solution:
```

```
# Same as 3395. Subsequences with a Unique Middle Mode I
```

```
def subsequencesWithMiddleMode(self, nums: list[int]) -> int:
```

```
    MOD = 1_000_000_007
```

```
    ans = 0
```

```
    p = collections.Counter() # prefix counter
```

```
    s = collections.Counter(nums) # suffix counter
```

```
    def nC2(n: int) -> int:
```

```
        return n * (n - 1) // 2
```

```
    pss = 0
```

```
    spp = 0
```

```
    pp = 0
```

```
    ss = sum(freq**2 for freq in s.values())
```

```
    ps = 0
```

```
    for i, a in enumerate(nums):
```

```
        # Update running sums after decrementing s[a].
```

```
        pss += p[a] * (-s[a]**2 + (s[a] - 1)**2)
```

```
        spp += -p[a]**2 # (-s[a] + (s[a] - 1)) * p[a]**2
```

```
        ss += -s[a]**2 + (s[a] - 1)**2
```

```
        ps += -p[a] # -p[a] * (-s[a] + (s[a] - 1))
```

```
        s[a] -= 1
```

```
        l = i
```

```
        r = len(nums) - i - 1
```

```
        # Start with all possible subsequences with `a` as the middle number.
```

```
        ans += nC2(l) * nC2(r)
```

```
        # Minus the cases where the frequency of `a` is 1, so it's not a mode.
```

```
        ans -= nC2(l - p[a]) * nC2(r - s[a])
```

```
        # Minus the values where `b != a`.
```

```
        pss_ = pss - p[a] * s[a]**2
```

```
        spp_ = spp - s[a] * p[a]**2
```

```
        pp_ = pp - p[a]**2
```

```
        ss_ = ss - s[a]**2
```

```
        ps_ = ps - p[a] * s[a]
```

```
        p_ = 1 - p[a]
```

```
        s_ = r - s[a]
```

```
        # Minus the cases where the `a` is not a "unique" mode or not a mode.
```

```
        ans -= ps_ * (p[a] * (r - s[a])) + pss_ * (-p[a])
```

```
        ans -= ps_ * (s[a] * (1 - p[a])) + spp_ * (-s[a])
```

```
        ans -= (pp_ - p_) * s[a] * (r - s[a]) // 2
```

```
        ans -= (ss_ - s_) * p[a] * (1 - p[a]) // 2
```

```
        ans %= MOD
```

```
        # Update running sums after incrementing p[a].
```

```
        pss += s[a]**2 # (-p[a] + (p[a] + 1)) * s[a]**2
```

```
        spp += s[a] * (-p[a]**2 + (p[a] + 1)**2)
```

```
        pp += -p[a]**2 + (p[a] + 1)**2
```

```
        ps += s[a] # (-p[a] + (p[a] + 1)) * s[a]
```

```
        p[a] += 1
```

```
    return ans
```

2371. 3417.py

Path: LeetCode-main/solutions/3417. Zigzag Grid Traversal With Skip/3417.py

```
class Solution:
    def zigzagTraversal(self, grid: list[list[int]]) -> list[int]:
        zigzag = [row[::-1] if i % 2 else row for i, row in enumerate(grid)]
        return [num for row in zigzag for num in row][::2]
```

2372. 3418.py

Path: LeetCode-main/solutions/3418. Maximum Amount of Money Robot Can Earn/3418.py

```
class Solution:
    def maximumAmount(self, coins: list[list[int]]) -> int:
        m = len(coins)
        n = len(coins[0])
        # dp[i][j][k] := the maximum profit at position (i, j) with k remaining
        # neutralizations
        dp = [[[-math.inf] * 4 for _ in range(n)] for _ in range(m)]

        # Base case: the robot starts at the top-left corner.
        dp[0][0][2] = coins[0][0]
        if coins[0][0] < 0:
            dp[0][0][1] = 0 # Neutralize the robber.

        for i in range(m):
            for j in range(n):
                for k in range(3): # for each number of remaining neutralizations
                    if i > 0:
                        dp[i][j][k] = max(dp[i][j][k],
                                           dp[i - 1][j][k] + coins[i][j],
                                           dp[i - 1][j][k + 1])
                    if j > 0:
                        dp[i][j][k] = max(dp[i][j][k],
                                           dp[i][j - 1][k] + coins[i][j],
                                           dp[i][j - 1][k + 1])

        return max(dp[-1][-1])
```

2373. 3419.py

Path: LeetCode-main/solutions/3419. Minimize the Maximum Edge Weight of Graph/3419.py


```

class Solution:
    def minMaxWeight(self, n: int, edges: list[list[int]], threshold: int) -> int:
        MAX = 1000000
        reversedGraph = [[] for _ in range(n)]

        for u, v, w in edges:
            reversedGraph[v].append((u, w))

        l = 1
        r = MAX + 1

        while l < r:
            m = (l + r) // 2
            if self._dfs(reversedGraph, 0, m, set()) == n:
                r = m
            else:
                l = m + 1

        return -1 if l == MAX + 1 else l

    def _dfs(
        self,
        reversedGraph: list[list[tuple]],
        u: int,
        maxWeight: int,
        seen: set[int]
    ) -> int:
        """Returns the number of nodes reachable from u with weight <= maxWeight."""
        res = 1
        seen.add(u)
        for v, w in reversedGraph[u]:
            if w > maxWeight or v in seen:
                continue
            res += self._dfs(reversedGraph, v, maxWeight, seen)
        return res

```

2374. 342.py

Path: LeetCode-main/solutions/342. Power of Four/342.py

```

class Solution:
    def isPowerOfFour(self, n: int) -> bool:
        # Why (4^n - 1) % 3 == 0?
        # (4^n - 1) = (2^n - 1)(2^n + 1) and 2^n - 1, 2^n, 2^n + 1 are
        # three consecutive numbers; among one of them, there must be a multiple
        # of 3, and that can't be 2^n, so it must be either 2^n - 1 or 2^n + 1.
        # Therefore, 4^n - 1 is a multiple of 3.
        return n > 0 and n.bit_count() == 1 and (n - 1) % 3 == 0

```

2375. 3420-2.py

Path: LeetCode-main/solutions/3420. Count Non-Decreasing Subarrays After K Operations/3420-2.py

```

class Solution:
    def countNonDecreasingSubarrays(self, nums: list[int], k: int) -> int:
        ans = 0
        cost = 0
        # Store indices (i) of nums with nums[i] in non-increasing order.
        dq = collections.deque()

        j = len(nums) - 1
        for i, num in reversed(list(enumerate(nums))):
            while dq and nums[dq[-1]] < num:
                l = dq.pop()
                r = dq[-1] if dq else j + 1
                cost += (r - l) * (num - nums[l]) # Adjust `nums[l]` to `num`.
            dq.append(i)
            while cost > k: # Remove the rightmost number.
                cost -= nums[dq[0]] - nums[j]
                if dq[0] == j:
                    dq.popleft()
                j -= 1
            ans += j - i + 1

        return ans

```

2376. 3420.py

Path: LeetCode-main/solutions/3420. Count Non-Decreasing Subarrays After K Operations/3420.py

```

class Solution:
    def countNonDecreasingSubarrays(self, nums: list[int], k: int) -> int:
        ans = 0
        cost = 0
        # Store (number, count) pairs in non-increasing order. The numbers in the
        # queue represent what nums[i..j] look like after adjustments.
        dq = collections.deque()

        j = len(nums) - 1
        for i, num in reversed(list(enumerate(nums))):
            count = 1
            while dq and dq[-1][0] < num:
                nextNum, nextCount = dq.pop()
                count += nextCount
                cost += (num - nextNum) * nextCount # Adjust `nextNum`s to `num`.
            dq.append((num, count))
            while cost > k: # Remove the rightmost number.
                rightmostNum, rightmostCount = dq.popleft()
                cost -= (rightmostNum - nums[j])
                j -= 1
                if rightmostCount > 1:
                    dq.appendleft((rightmostNum, rightmostCount - 1))
            ans += j - i + 1

        return ans

```

2377. 3422.py

Path: LeetCode-main/solutions/3422. Minimum Operations to Make Subarray Elements Equal/3422.py

```

class Solution:
    def minOperations(self, nums: list[int], k: int) -> int:
        window = SortedList(nums[:k])
        median = window[(k - 1) // 2]
        ops = sum(abs(median - nums[j]) for j in range(k))
        ans = ops

        for i in range(k, len(nums)):
            window.remove(nums[i - k])
            window.add(nums[i])
            ops -= abs(median - nums[i - k])
            ops += abs(median - nums[i])
            newMedian = window[(k - 1) // 2]
            medianMultiplier = (
                2
                if k % 2 == 0 and median <= newMedian <= window[k // 2]
                else k % 2)
            ops -= abs(newMedian - median) * medianMultiplier
            median = newMedian
            ans = min(ans, ops)

        return ans

```

2378. 3423.py

Path: LeetCode-main/solutions/3423. Maximum Difference Between Adjacent Elements in a Circular Array/3423.py

```

class Solution:
    def maxAdjacentDistance(self, nums: list[int]) -> int:
        return max(abs(nums[i] - nums[i - 1])
                    for i in range(len(nums)))

```

2379. 3424.py

Path: LeetCode-main/solutions/3424. Minimum Cost to Make Arrays Identical/3424.py

```

class Solution:
    def minCost(self, arr: list[int], brr: list[int], k: int) -> int:
        def cost(arr: list[int], brr: list[int]) -> int:
            return sum(abs(a - b) for a, b in zip(arr, brr))
        return min(cost(arr, brr), cost(sorted(arr), sorted(brr)) + k)

```

2380. 3425.py

Path: LeetCode-main/solutions/3425. Longest Special Path/3425.py

```

class Solution:
    def longestSpecialPath(
        self,
        edges: list[list[int]],
        nums: list[int]
    ) -> list[int]:
        maxLength = 0
        minNodes = 1
        graph = [[] for _ in range(len(nums))]

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        prefix = [0]
        lastSeenDepth = {}

        def dfs(
            u: int,
            prev: int,
            leftBoundary: int,
        ) -> None:
            nonlocal maxLength, minNodes
            prevDepth = lastSeenDepth.get(nums[u], 0)
            lastSeenDepth[nums[u]] = len(prefix)
            leftBoundary = max(leftBoundary, prevDepth)

            length = prefix[-1] - prefix[leftBoundary]
            nodes = len(prefix) - leftBoundary
            if length > maxLength or (length == maxLength and nodes < minNodes):
                maxLength = length
                minNodes = nodes

            for v, w in graph[u]:
                if v == prev:
                    continue
                prefix.append(prefix[-1] + w)
                dfs(v, u, leftBoundary)
                prefix.pop()

            lastSeenDepth[nums[u]] = prevDepth

        dfs(0, -1, leftBoundary=0)
        return [maxLength, minNodes]

```

2381. 3426.py

Path: LeetCode-main/solutions/3426. Manhattan Distances of All Arrangements of Pieces/3426.py

```

class Solution:
    def distanceSum(self, m: int, n: int, k: int) -> int:
        # For each distance d, where 1 < d < m, there are `m - d` ways to choose
        # the two columns that the two pieces are on. For each of the two pieces,
        # there are `n` ways to choose the row that the piece is on.
        # Therefore, the contribution of row differences is
        # sum(d * (m - d) * n^2), where 1 < d <= m - 1
        # = n^2 * sum(d * m - d^2)
        # = n^2 * (d * m * (m - 1) / 2 - m * (m - 1) * (2m - 1) / 6)
        # = n^2 * (m^3 - m) / 6
        # Similarly, the contribution of column differences is
        # m^2 * (n^3 - n) / 6
        MOD = 1_000_000_007
        return (n**2 * (m**3 - m) // 6 +
                m**2 * (n**3 - n) // 6) * math.comb(m * n - 2, k - 2) % MOD

```

2382. 3427.py

Path: LeetCode-main/solutions/3427. Sum of Variable Length Subarrays/3427.py

```

class Solution:
    def subarraySum(self, nums: list[int]) -> int:
        prefix = list(itertools.accumulate(nums, initial=0))
        return sum(prefix[i + 1] - prefix[max(0, i - num)]
                    for i, num in enumerate((nums)))

```

2383. 3428.py

Path: LeetCode-main/solutions/3428. Maximum and Minimum Sums of at Most Size K Subsequences/3428.py

```

class Solution:
    def minMaxSums(self, nums: list[int], k: int) -> int:
        # In a sorted array, nums[i] will be
        # 1. The maximum for subsequences formed by nums[0..i].
        # 2. The minimum for subsequences formed by nums[i..n - 1].
        #
        # The number of times nums[i] is the maximum is the same as the number of
        # times nums[n - 1 - i] is the minimum, due to the symmetry in subsequences
        # derived from the sorted order.
        #
        # To calculate the contribution of nums[i], we need to find the number of
        # ways to select at most (k - 1) elements from the range of indices where
        # nums[i] is the smallest or nums[n - 1 - i] is the largest.
        MOD = 1_000_000_007
        n = len(nums)

        def getComb(n: int, k: int) -> list[list[int]]:
            """C(n, k) = C(n - 1, k) + C(n - 1, k - 1)"""
            comb = [[0] * (k + 1) for _ in range(n + 1)]
            for i in range(n + 1):
                comb[i][0] = 1
            for i in range(1, n + 1):
                for j in range(1, k + 1):
                    comb[i][j] = (comb[i - 1][j] + comb[i - 1][j - 1]) % MOD
            return comb

        comb = getComb(n, k - 1)
        ans = 0

        nums.sort()

        # i: available numbers from the left of nums[i] or
        # available numbers from the right of nums[-1 - i]
        for i in range(n):
            count = 0
            for j in range(k): # selected numbers
                count = (count + comb[i][j]) % MOD
            ans += nums[i] * count
            ans += nums[-1 - i] * count
            ans %= MOD

        return ans

```

2384. 3429.py

Path: LeetCode-main/solutions/3429. Paint House IV/3429.py

```

class Solution:
    def minCost(self, n: int, costs: list[list[int]]) -> int:
        INVALID_COLOR = 3

        def getValidColors(prevColor: int) -> list[int]:
            return [color for color in range(3) if color != prevColor]

        @functools.lru_cache(None)
        def minCost(i: int, prevLeftColor: int, prevRightColor: int) -> int:
            if i == len(costs) // 2:
                return 0
            res = math.inf
            for leftColor in getValidColors(prevLeftColor):
                for rightColor in getValidColors(prevRightColor):
                    if leftColor == rightColor:
                        continue
                    leftCost = costs[i][leftColor]
                    rightCost = costs[-1 - i][rightColor]
                    res = min(res, leftCost + rightCost +
                             minCost(i + 1, leftColor, rightColor))
            return res

        return minCost(0, INVALID_COLOR, INVALID_COLOR)

```

2385. 343.py

Path: LeetCode-main/solutions/343. Integer Break/343.py

```

class Solution:
    def integerBreak(self, n: int) -> int:
        # If an optimal product contains a factor f >= 4, then we can replace it
        # with 2 and f - 2 without losing optimality. As 2(f - 2) = 2f - 4 >= f,
        # we never need a factor >= 4, meaning we only need factors 1, 2, and 3
        # (and 1 is wasteful).
        # Also, 3 * 3 is better than 2 * 2 * 2, so we never use 2 more than twice.
        if n == 2: # 1 * 1
            return 1
        if n == 3: # 1 * 2
            return 2

        ans = 1

        while n > 4:
            n -= 3
            ans *= 3
        ans *= n

        return ans

```

2386. 3430.py

Path: LeetCode-main/solutions/3430. Maximum and Minimum Sums of at Most Size K Subarrays/3430.py

```

class Solution:
    # Similar to 2104. Sum of Subarray Ranges
    def minMaxSubarraySum(self, nums: list[int], k: int) -> int:
        prevGt, nextGt = self._getPrevNext(nums, operator.lt)
        prevLt, nextLt = self._getPrevNext(nums, operator.gt)
        return (self._subarraySum(nums, prevGt, nextGt, k) +
                self._subarraySum(nums, prevLt, nextLt, k))

    def _subarraySum(
        self,
        nums: list[int],
        prev: list[int],
        next: list[int],
        k: int
    ) -> int:
        """
        Returns the sum of all subarrays with a size <= k, The `prev` and `next`
        arrays are used to store the indices of the nearest numbers that are
        smaller or larger than the current number, respectively.
        """
        res = 0
        for i, num in enumerate(nums):
            l = min(i - prev[i], k)
            r = min(next[i] - i, k)
            extra = max(0, l + r - 1 - k)
            res += num * (l * r - extra * (extra + 1) // 2)
        return res

    def _getPrevNext(
        self,
        nums: list[int],
        op: callable
    ) -> tuple[list[int], list[int]]:
        """
        Returns `prev` and `next`, that store the indices of the nearest numbers
        that are smaller or larger than the current number depending on `op`.
        """
        n = len(nums)
        prev = [-1] * n
        next = [n] * n
        stack = []
        for i, num in enumerate(nums):
            while stack and op(nums[stack[-1]], num):
                index = stack.pop()
                next[index] = i
            if stack:
                prev[i] = stack[-1]
            stack.append(i)
        return prev, next

```

2387. 3431.py

Path: LeetCode-main/solutions/3431. Minimum Unlocked Indices to Sort Nums/3431.py


```
class Solution:
    def minUnlockedIndices(self, nums: list[int], locked: list[int]) -> int:
        first2 = next((i for i, x in enumerate(nums) if x == 2), -1)
        first3 = next((i for i, x in enumerate(nums) if x == 3), -1)
        last1 = next((i for i, x in reversed(list(enumerate(nums))) if x == 1), -1)
        last2 = next((i for i, x in reversed(list(enumerate(nums))) if x == 2), -1)
        if first3 != -1 and last1 != -1 and first3 < last1:
            return -1
        return (sum(locked[i] == 1 for i in range(first2, last1)
                    if first2 != -1 and last1 != -1) +
                sum(locked[i] == 1 for i in range(first3, last2)
                    if first3 != -1 and last2 != -1))
```

2388. 3432.py

Path: LeetCode-main/solutions/3432. Count Partitions with Even Sum Difference/3432.py

```
class Solution:
    def countPartitions(self, nums: list[int]) -> int:
        # If we add the same number in the left subarray and remove it from the
        # right subarray, then the difference remains the same parity. So, just
        # return the number of ways to partition the array into two subarrays when
        # the array sum is even.
        return len(nums) - 1 if sum(nums) % 2 == 0 else 0
```

2389. 3433.py

Path: LeetCode-main/solutions/3433. Count Mentions Per User/3433.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class OfflineUser:
    returnTimestamp: int
    userId: int

    def __lt__(self, other):
        return self.returnTimestamp < other.returnTimestamp

class Solution:
    def countMentions(
        self,
        numberOfUsers: int,
        events: list[list[str]]
    ) -> list[int]:
        ans = [0] * numberOfUsers
        online = [True] * numberOfUsers
        offlineQueue = [] # min-heap to track users that are offline
        allMentionsCount = 0

        events.sort(key=lambda x: (int(x[1]), -ord(x[0][0])))

        for eventType, t, messageContent in events:
            timestamp = int(t)
            # Bring users back online if their offline period has ended.
            while offlineQueue and offlineQueue[0].returnTimestamp <= timestamp:
                user = heapq.heappop(offlineQueue)
                online[user.userId] = True
            if eventType == "MESSAGE":
                match messageContent:
                    case "ALL":
                        allMentionsCount += 1
                    case "HERE":
                        for userId in range(numberOfUsers):
                            if online[userId]:
                                ans[userId] += 1
                    case _:
                        for userId in [int(part[2:]) for part in messageContent.split()]:
                            ans[userId] += 1
            elif eventType == "OFFLINE":
                userId = int(messageContent)
                online[userId] = False
                # Add to queue to bring back online after 60 units.
                heapq.heappush(offlineQueue, OfflineUser(timestamp + 60, userId))

        # Add the "ALL" mentions to all users.
        for userId in range(numberOfUsers):
            ans[userId] += allMentionsCount
        return ans

```

2390. 3434.py

Path: LeetCode-main/solutions/3434. Maximum Frequency After Subarray Operation/3434.py

```

class Solution:
    def maxFrequency(self, nums: list[int], k: int) -> int:
        return nums.count(k) + max(self._kadane(nums, target, k)
                                   for target in range(1, 51)
                                   if target != k)

    def _kadane(self, nums: list[int], target: int, k: int) -> int:
        """
        Returns the maximum achievable frequency of `k` by Kadane's algorithm,
        where each `target` in subarrays is transformed to `k`.
        """
        maxSum = 0
        sum = 0
        for num in nums:
            if num == target:
                sum += 1
            elif num == k:
                sum -= 1
            if sum < 0: # Reset sum if it becomes negative (Kadane's spirit).
                sum = 0
            maxSum = max(maxSum, sum)
        return maxSum

```

2391. 3435.py

Path: LeetCode-main/solutions/3435. Frequencies of Shortest Supersequences/3435.py

```

from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def supersequences(self, words: list[str]) -> list[list[int]]:
        ans = []
        edges = [(string.ascii_lowercase.index(words[0]),
                  string.ascii_lowercase.index(words[1]))
                  for words in words]
        nodes = sorted({u for u, _ in edges} | {v for _, v in edges})
        letterToIndex = {letter: i for i, letter in enumerate(nodes)}
        graph = [[] for _ in range(len(nodes))]

        for u, v in edges:
            graph[letterToIndex[u]].append(letterToIndex[v])

        for doubledSubset in self._getMinimumSubsets(graph):
            freq = [0] * 26
            for letter in nodes:
                freq[letter] = 1
            for index in doubledSubset:
                freq[nodes[index]] = 2
            ans.append(freq)

        return ans

    def _getMinimumSubsets(self, graph: list[list[int]]) -> list[tuple[int]]:
        """
        Returns a list of the minimum subsets of nodes that do not create a cycle

```

```

when skipped.
"""

n = len(graph)
for subsetSize in range(n + 1):
    doubleSubsets = []
    for doubledSubset in itertools.combinations(range(n), subsetSize):
        if not self._hasCycleSkipping(graph, set(doubledSubset)):
            doubleSubsets.append(doubledSubset)
    if doubleSubsets:
        return doubleSubsets
return []

def _hasCycleSkipping(
    self,
    graph: list[list[int]],
    doubledSubset: set[int]
) -> bool:
    """
    Returns True if there is a cycle in the `graph` when skipping any edges
    whose both endpoints are in `doubledSubset`.
    """

    states = [State.INIT] * len(graph)

    def hasCycle(u: int) -> bool:
        if states[u] == State.VISITING:
            return True
        if states[u] == State.VISITED:
            return False
        states[u] = State.VISITING
        if u not in doubledSubset:
            for v in graph[u]:
                if v in doubledSubset:
                    continue
                if hasCycle(v):
                    return True
        states[u] = State.VISITED
        return False

    return any(hasCycle(i) for i in range(len(graph)))

```

2392. 3437.py

Path: LeetCode-main/solutions/3437. Permutations III/3437.py

```

class Solution:
    def permute(self, n: int) -> list[list[int]]:
        ans = []
        used = [False] * (n + 1)

        def dfs(path: list[int]) -> None:
            if len(path) == n:
                ans.append(path.copy())
                return
            for num in range(1, n + 1):
                if used[num]:
                    continue
                if path and path[-1] % 2 == num % 2:
                    continue
                used[num] = True
                path.append(num)
                dfs(path)
                path.pop()
                used[num] = False

        dfs([])
        return ans

```

2393. 3438.py

Path: LeetCode-main/solutions/3438. Find Valid Pair of Adjacent Digits in String/3438.py

```

class Solution:
    def findValidPair(self, s: str) -> str:
        count = collections.Counter(s)
        return next((a + b
                     for a, b in itertools.pairwise(s)
                     if a != b and count[a] == int(a) and count[b] == int(b)), '')

```

2394. 3439.py

Path: LeetCode-main/solutions/3439. Reschedule Meetings for Maximum Free Time I/3439.py

```

class Solution:
    def maxFreeTime(
        self,
        eventTime: int,
        k: int,
        startTime: list[int],
        endTime: list[int]
    ) -> int:
        gaps = ([startTime[0]] +
                [startTime[i] - endTime[i - 1] for i in range(1, len(startTime))] +
                [eventTime - endTime[-1]])
        windowSum = sum(gaps[:k + 1])
        ans = windowSum

        for i in range(k + 1, len(gaps)):
            windowSum += gaps[i] - gaps[i - k - 1]
            ans = max(ans, windowSum)

        return ans

```

2395. 344.py

Path: LeetCode-main/solutions/344. Reverse String/344.py

```
class Solution:
    def reverseString(self, s: list[str]) -> None:
        l = 0
        r = len(s) - 1

        while l < r:
            s[l], s[r] = s[r], s[l]
            l += 1
            r -= 1
```

2396. 3440.py

Path: LeetCode-main/solutions/3440. Reschedule Meetings for Maximum Free Time II/3440.py

```
class Solution:
    def maxFreeTime(
        self,
        eventTime: int,
        startTime: list[int],
        endTime: list[int]
    ) -> int:
        n = len(startTime)
        gaps = ([startTime[0]] +
                [startTime[i] - endTime[i - 1] for i in range(1, len(startTime))] +
                [eventTime - endTime[-1]])

        ans = 0
        maxLeft = [gaps[0]] + [0] * n # maxLeft[i] := max(gaps[0..i])
        maxRight = [0] * n + [gaps[n]] # maxRight[i] := max(gaps[i..n])

        for i in range(1, n + 1):
            maxLeft[i] = max(gaps[i], maxLeft[i - 1])

        for i in range(n - 1, -1, -1):
            maxRight[i] = max(gaps[i], maxRight[i + 1])

        for i, (start, end) in enumerate(zip(startTime, endTime)):
            currMeetingTime = end - start
            adjacentGapsSum = gaps[i] + gaps[i + 1]
            canMoveMeeting = currMeetingTime <= max(
                maxLeft[i - 1] if i > 0 else 0,
                maxRight[i + 2] if i + 2 < n + 1 else 0
            )
            ans = max(ans, adjacentGapsSum +
                      (currMeetingTime if canMoveMeeting else 0))

        return ans
```

2397. 3441.py

Path: LeetCode-main/solutions/3441. Minimum Cost Good Caption/3441.py

```
class Solution:

    def minCostGoodCaption(self, caption: str) -> str:
        n = len(caption)
        if n < 3:
```

```

        return ''

MAX_COST = 1_000_000_000
# dp[i][j][k] := the minimum cost of caption[i..n - 1], where j is the last
# letter used, and k is the count of consecutive letters
dp = [[[MAX_COST] * 3 for _ in range(26)] for _ in range(n)]

for c in range(26):
    dp[-1][c][0] = abs(string.ascii_lowercase.index(caption[-1]) - c)

minCost = MAX_COST

for i in range(n - 2, -1, -1):
    newMinCost = MAX_COST
    for c in range(26):
        changeCost = abs(string.ascii_lowercase.index(caption[i]) - c)
        dp[i][c][0] = changeCost + minCost
        dp[i][c][1] = changeCost + dp[i + 1][c][0]
        dp[i][c][2] = changeCost + min(dp[i + 1][c][1], dp[i + 1][c][2])
        newMinCost = min(newMinCost, dp[i][c][2])
    minCost = newMinCost

# Reconstruct the string.
ans = []
cost = MAX_COST
letter = -1

# Find the initial best letter.
for c in range(25, -1, -1):
    if dp[0][c][2] <= cost:
        letter = c
        cost = dp[0][c][2]

# Add the initial triplet.
cost -= self._appendLetter(caption, 0, chr(ord('a') + letter), ans)
cost -= self._appendLetter(caption, 1, chr(ord('a') + letter), ans)
cost -= self._appendLetter(caption, 2, chr(ord('a') + letter), ans)

# Build the rest of the string.
i = 3
while i < n:
    nextLetter = self._getNextLetter(dp, i, cost)
    if nextLetter < letter or min(dp[i][letter]) > cost:
        letter = nextLetter
        cost -= self._appendLetter(caption, i, chr(ord('a') + letter), ans)
        cost -= self._appendLetter(caption, i + 1, chr(ord('a') + letter), ans)
        cost -= self._appendLetter(caption, i + 2, chr(ord('a') + letter), ans)
        i += 3
    else:
        cost -= self._appendLetter(caption, i, chr(ord('a') + letter), ans)
        i += 1

return ''.join(ans)

def _getNextLetter(self, dp: list[list[list[int]]], i: int, cost: int) -> int:
    nextLetter = 26
    for c in range(25, -1, -1):
        if cost == dp[i][c][2]:
            nextLetter = c
    return nextLetter

def _appendLetter(
    self,
    caption: str,
    i: int,

```

```
    letter: str,
    ans: list[str]
) -> int:
    ans.append(letter)
    return abs(ord(caption[i]) - ord(letter))
```

2398. 3442.py

Path: LeetCode-main/solutions/3442. Maximum Difference Between Even and Odd Frequency I/3442.py

```
class Solution:
    def maxDifference(self, s: str) -> int:
        count = collections.Counter(s)
        maxOdd = max((freq for freq in count.values()
                      if freq % 2 == 1), default=0)
        minEven = min((freq for freq in count.values()
                      if freq % 2 == 0), default=len(s))
        return maxOdd - minEven
```

2399. 3443.py

Path: LeetCode-main/solutions/3443. Maximum Manhattan Distance After K Changes/3443.py

```
class Solution:
    def maxDistance(self, s: str, k: int) -> int:
        return max(self._flip(s, k, 'NE'), self._flip(s, k, 'NW'),
                  self._flip(s, k, 'SE'), self._flip(s, k, 'SW'))

    def _flip(self, s: str, k: int, direction: str) -> int:
        res = 0
        pos = 0
        opposite = 0

        for c in s:
            if c in direction:
                pos += 1
            else:
                pos -= 1
                opposite += 1
            res = max(res, pos + 2 * min(k, opposite))

        return res
```

2400. 3444.py

Path: LeetCode-main/solutions/3444. Minimum Increments for Target Multiples in an Array/3444.py


```

class Solution:
    def minimumIncrements(self, nums: list[int], target: list[int]) -> int:
        maxMask = 1 << len(target)
        maskToLcm = {}

        for mask in range(1, maxMask):
            subset = [num for i, num in enumerate(target) if mask >> i & 1]
            maskToLcm[mask] = functools.reduce(math.lcm, subset, 1)

        # dp[mask] := the minimum number of increments to make each number in the
        # subset of target have at least one number that is a multiple in `num`,
        # where `mask` is the bitmask of the subset of target
        dp = [math.inf] * maxMask
        dp[0] = 0

        for num in nums:
            # maskToCost := (mask, cost), where `mask` is the bitmask of the subset
            # of target and `cost` is the minimum number of increments to make each
            # number in the subset of target have at least one number that is a
            # multiple in `num`
            maskToCost = [
                (mask, 0 if (remainder := num % lcm) == 0 else lcm - remainder) for mask,
                lcm in maskToLcm.items()]
            newDp = dp[: ]
            for prevMask in range(maxMask):
                if dp[prevMask] == float('inf'):
                    continue
                for mask, cost in maskToCost:
                    newMask = prevMask | mask
                    newDp[newMask] = min(newDp[newMask], dp[prevMask] + cost)
            dp = newDp

        return -1 if dp[-1] == math.inf else dp[-1]

```

2401. 3445.py

Path: LeetCode-main/solutions/3445. Maximum Difference Between Even and Odd Frequency II/3445.py

```

class Solution:
    def maxDifference(self, s: str, k: int) -> int:
        ans = -math.inf
        permutations = [(a, b) for a in '01234' for b in '01234' if a != b]

        for a, b in permutations:
            # minDiff[(parityA, parityB)] := min(a - b) of all valid windows with
            # parityA and parityB
            minDiff = collections.defaultdict(lambda: math.inf)
            prefixA = [0] # prefixA[i] := the number of 'a's in s[0..i)
            prefixB = [0] # prefixB[i] := the number of 'b's in s[0..i)

            l = 0
            for r, c in enumerate(s):
                prefixA.append(prefixA[-1] + int(c == a))
                prefixB.append(prefixB[-1] + int(c == b))
                while (r - l + 1 >= k and # the window size >= k
                    prefixA[l] < prefixA[-1] and # the number of 'a's > 0
                    prefixB[l] < prefixB[-1]): # the number of 'b's > 0
                    paritiesKey = (prefixA[l] % 2, prefixB[l] % 2)
                    minDiff[paritiesKey] = min(minDiff[paritiesKey],
                                                prefixA[l] - prefixB[l])

                    l += 1
            ans = max(ans, (prefixA[-1] - prefixB[-1]) -
                        minDiff[(1 - prefixA[-1] % 2, prefixB[-1] % 2)])

        return ans

```

2402. 3446.py

Path: LeetCode-main/solutions/3446. Sort Matrix by Diagonals/3446.py

```

class Solution:
    def sortMatrix(self, grid: list[list[int]]) -> list[list[int]]:
        n = len(grid)
        ans = [[0] * n for _ in range(n)]
        diag = collections.defaultdict(list)

        for i, row in enumerate(grid):
            for j, num in enumerate(row):
                diag[i - j].append(num)

        for key in diag:
            diag[key].sort(reverse=key < 0)

        for i in range(n):
            for j in range(n):
                ans[i][j] = diag[i - j].pop()

        return ans

```

2403. 3447.py

Path: LeetCode-main/solutions/3447. Assign Elements to Groups with Constraints/3447.py

```

class Solution:
    def assignElements(self, groups: list[int], elements: list[int]) -> list[int]:
        ans = []
        elementToMinIndex = {}

        for i, element in enumerate(elements):
            if element not in elementToMinIndex:
                elementToMinIndex[element] = i

        for num in groups:
            ans.append(self._getMinIndex(num, elementToMinIndex))

        return ans

    def _getMinIndex(self, num: int, elementToMinIndex: dict[int, int]) -> int:
        res = math.inf
        i = 1
        while i * i <= num:
            if num % i != 0:
                continue
            if i in elementToMinIndex:
                res = min(res, elementToMinIndex[i])
            if num // i != i and (num // i) in elementToMinIndex:
                res = min(res, elementToMinIndex[num // i])
            i += 1
        return -1 if res == math.inf else res

```

2404. 3448-2.py

Path: LeetCode-main/solutions/3448. Count Substrings Divisible By Last Digit/3448-2.py

```

class Solution:
    def countSubstrings(self, s: str) -> int:
        ans = 0
        # dp[num][rem] := the number of substrings so far that have a remainder of
        # `rem` when divided by `num`
        dp = [[0] * 10 for _ in range(10)]

        for c in s:
            digit = int(c)
            newDp = [[0] * 10 for _ in range(10)]
            for num in range(1, 10):
                for rem in range(num):
                    newDp[num][(rem * 10 + digit) % num] += dp[num][rem]
                    newDp[num][digit % num] += 1
            dp = newDp
            ans += dp[digit][0]

        return ans

```

2405. 3448.py

Path: LeetCode-main/solutions/3448. Count Substrings Divisible By Last Digit/3448.py

```

class Solution:
    def countSubstrings(self, s: str) -> int:
        ans = 0
        # dp[i][num][rem] := the number of first `i` digits of s that have a
        # remainder of `rem` when divided by `num`
        dp = [[[0] * 10 for _ in range(10)] for _ in range(len(s) + 1)]

        for i in range(1, len(s) + 1):
            digit = int(s[i - 1])
            for num in range(1, 10):
                for rem in range(num):
                    dp[i][num][(rem * 10 + digit) % num] += dp[i - 1][num][rem]
                    dp[i][num][digit % num] += 1
                ans += dp[i][digit][0]

        return ans

```

2406. 3449.py

Path: LeetCode-main/solutions/3449. Maximize the Minimum Game Score/3449.py

```

class Solution:
    def maxScore(self, points: list[int], m: int) -> int:
        def isPossible(minVal: int, m: int) -> bool:
            """
            Returns True if it is possible to achieve the maximum minimum value `x`
            with `m` number of moves.
            """

            moves = 0
            prevMoves = 0 # to track remaining moves from the previous point
            for i, point in enumerate(points):
                required = (minVal + point - 1) // point # ceil(minVal / point)
                required = max(0, required - prevMoves)
                if required > 0:
                    moves += 2 * required - 1
                    prevMoves = required - 1
                elif i + 1 < len(points):
                    moves += 1
                    prevMoves = 0
                if moves > m:
                    return False
            return True

            l = 0
            r = (m + 1) // 2 * points[0] + 1

            while l < r:
                mid = (l + r + 1) // 2
                if isPossible(mid, m):
                    l = mid
                else:
                    r = mid - 1

            return l

```

2407. 345.py

Path: LeetCode-main/solutions/345. Reverse Vowels of a String/345.py

```
class Solution:
    def reverseVowels(self, s: str) -> str:
        chars = list(s)
        VOWELS = 'aeiouAEIOU'
        l = 0
        r = len(s) - 1

        while l < r:
            while l < r and chars[l] not in VOWELS:
                l += 1
            while l < r and chars[r] not in VOWELS:
                r -= 1
            chars[l], chars[r] = chars[r], chars[l]
            l += 1
            r -= 1

        return ''.join(chars)
```

2408. 3450.py

Path: LeetCode-main/solutions/3450. Maximum Students on a Single Bench/3450.py

```
class Solution:
    def maxStudentsOnBench(self, students: list[list[int]]) -> int:
        benchToStudents = collections.defaultdict(set)
        for studentId, benchId in students:
            benchToStudents[benchId].add(studentId)
        return max(map(len, benchToStudents.values()), default=0)
```

2409. 3452.py

Path: LeetCode-main/solutions/3452. Sum of Good Numbers/3452.py

```
class Solution:
    def sumOfGoodNumbers(self, nums: list[int], k: int) -> int:
        return sum(num for i, num in enumerate(nums)
                    if (i - k < 0 or num > nums[i - k])
                    and (i + k >= len(nums) or num > nums[i + k]))
```

2410. 3453.py

Path: LeetCode-main/solutions/3453. Separate Squares I/3453.py

```

class Solution:
    def separateSquares(self, squares: list[list[int]]) -> float:
        halfArea = sum((l**2 for _, _, l in squares)) / 2
        events = sorted([(y, True, l) for _, y, l in squares] +
                        [(y + l, False, l) for _, y, l in squares])

        area = 0
        width = 0
        prevY = 0

        for y, isStart, l in events:
            areaGain = width * (y - prevY)
            if area + areaGain >= halfArea:
                return prevY + (halfArea - area) / width
            area += areaGain
            width += l if isStart else -l
            prevY = y

```

2411. 3455.py

Path: LeetCode-main/solutions/3455. Shortest Matching Substring/3455.py

```

class Solution:
    def shortestMatchingSubstring(self, s: str, p: str) -> int:
        n = len(s)
        a, b, c = p.split('*')
        lpsA = self._getLPS(a + '#' + s)[len(a) + 1:]
        lpsB = self._getLPS(b + '#' + s)[len(b) + 1:]
        lpsC = self._getLPS(c + '#' + s)[len(c) + 1:]
        ans = math.inf

        i = 0 # lpsA's index
        j = 0 # lpsB's index
        k = 0 # lpsC's index
        while i + len(b) + len(c) < n:
            while i < n and lpsA[i] != len(a):
                i += 1
            while j < n and (j < i + len(b) or lpsB[j] != len(b)):
                j += 1
            while k < n and (k < j + len(c) or lpsC[k] != len(c)):
                k += 1
            if k == n:
                break
            ans = min(ans, k - i + len(a))
            i += 1

        return -1 if ans == math.inf else ans

    def _getLPS(self, pattern: str) -> list[int]:
        """
        Returns the lps array, where lps[i] is the length of the longest prefix of
        pattern[0..i] which is also a suffix of this substring.
        """
        lps = [0] * len(pattern)
        j = 0
        for i in range(1, len(pattern)):
            while j > 0 and pattern[j] != pattern[i]:
                j = lps[j - 1]
            if pattern[i] == pattern[j]:
                lps[i] = j + 1
                j += 1
        return lps

```

2412. 3456.py

Path: LeetCode-main/solutions/3456. Find Special Substring of Length K/3456.py

```
class Solution:
    def hasSpecialSubstring(self, s: str, k: int) -> bool:
        return any(len(list(group)) == k for _, group in groupby(s))
```

2413. 3457.py

Path: LeetCode-main/solutions/3457. Eat Pizzas!/3457.py

```
class Solution:
    def maxWeight(self, pizzas: list[int]) -> int:
        eat = len(pizzas) // 4
        odd = math.ceil(eat / 2)
        even = eat - odd
        pizzas.sort(reverse=True)
        return (sum(pizzas[:odd]) +
                sum(pizzas[odd + 1:odd + 1 + even * 2:2]))
```

2414. 3458.py

Path: LeetCode-main/solutions/3458. Select K Disjoint Special Substrings/3458.py

```
class Solution:
    def maxSubStringLength(self, s: str, k: int) -> bool:
        n = len(s)
        first = [n] * 26
        last = [-1] * 26
        # dp[i] := the maximum disjoint special substrings for the first i letters
        dp = [0] * (n + 1)
        seenOrder = []

        for i, c in enumerate(s):
            a = ord(c) - ord('a')
            if first[a] == n:
                first[a] = i
                seenOrder.append(c)
                last[a] = i

        for c in seenOrder:
            a = ord(c) - ord('a')
            for j in range(first[a], last[a]):
                b = ord(s[j]) - ord('a')
                first[a] = min(first[a], first[b])
                last[a] = max(last[a], last[b])

        for i, c in enumerate(s):
            a = ord(c) - ord('a')
            if last[a] != i or (first[a] == 0 and i == n - 1):
                dp[i + 1] = dp[i]
            else: # Start a new special substring.
                dp[i + 1] = max(dp[i], 1 + dp[first[a]])

        return dp[n] >= k
```

2415. 3459.py

Path: LeetCode-main/solutions/3459. Length of Longest V-Shaped Diagonal Segment/3459.py

```
class Solution:
    def lenOfVDiagonal(self, grid: list[list[int]]) -> int:
        DIRS = ((-1, 1), (1, 1), (1, -1), (-1, -1))

        @functools.lru_cache(None)
        def dfs(i: int, j: int, turned: bool, num: int, dir: int) -> int:
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return 0
            if grid[i][j] != num:
                return 0

            nextNum = 0 if num == 2 else 2
            dx, dy = DIRS[dir]
            res = 1 + dfs(i + dx, j + dy, turned, nextNum, dir)

            if not turned:
                nextDir = (dir + 1) % 4
                nextDx, nextDy = DIRS[nextDir]
                res = max(res, 1 + dfs(i + nextDx, j + nextDy, 1, nextNum, nextDir))

            return res

        return max((1 + dfs(i + dx, j + dy, 0, 2, d)
                     for i, row in enumerate(grid)
                     for j, num in enumerate(row)
                     if num == 1
                     for d, (dx, dy) in enumerate(DIRS)),
                    default=0)
```

2416. 346.py

Path: LeetCode-main/solutions/346. Moving Average from Data Stream/346.py

```
class MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.sum = 0
        self.q = collections.deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.size:
            self.sum -= self.q.popleft()
        self.sum += val
        self.q.append(val)
        return self.sum / len(self.q)
```

2417. 3460.py

Path: LeetCode-main/solutions/3460. Longest Common Prefix After at Most One Removal/3460.py


```

class Solution:
    def longestCommonPrefix(self, s: str, t: str) -> int:
        i = 0 # s's index
        j = 0 # t's index
        canSkip = True

        while i < len(s) and j < len(t):
            if s[i] == t[j]:
                i += 1
                j += 1
            elif canSkip:
                i += 1
                canSkip = False
            else:
                return j

        return j

```

2418. 3461.py

Path: LeetCode-main/solutions/3461. Check If Digits Are Equal in String After Operations I/3461.py

```

class Solution:
    def hasSameDigits(self, s: str) -> bool:
        n = len(s)
        num1 = 0
        num2 = 0

        for i in range(n - 1):
            coefficient = self._nCMOD10(n - 2, i)
            num1 += (coefficient * (int(s[i]) - 0)) % 10
            num1 %= 10
            num2 += (coefficient * (int(s[i + 1]) - 0)) % 10
            num2 %= 10

        return num1 == num2

    def _nCMOD10(self, n: int, k: int) -> int:
        """Returns (n, k) % 10."""
        mod2 = self._lucasTheorem(n, k, 2)
        mod5 = self._lucasTheorem(n, k, 5)
        lookup = [
            [0, 6, 2, 8, 4], # mod2 == 0
            [5, 1, 7, 3, 9]  # mod2 == 1
        ]
        return lookup[mod2][mod5]

    def _lucasTheorem(self, n: int, k: int, prime: int) -> int:
        """Returns (n, k) % prime."""
        res = 1
        while n > 0 or k > 0:
            nMod = n % prime
            MOD = k % prime
            res *= math.comb(nMod, MOD)
            res %= prime
            n //= prime
            k //= prime
        return res

```

2419. 3462.py

Path: LeetCode-main/solutions/3462. Maximum Sum With at Most K Elements/3462.py

```
class Solution:
    def maxSum(self, grid: list[list[int]], limits: list[int], k: int) -> int:
        minHeap = []

        for row, limit in zip(grid, limits):
            row.sort(reverse=True)
            for i in range(limit):
                heapq.heappush(minHeap, row[i])
                if len(minHeap) == k + 1:
                    heapq.heappop(minHeap)

        return sum(minHeap)
```

2420. 3463.py

Path: LeetCode-main/solutions/3463. Check If Digits Are Equal in String After Operations II/3463.py

```
class Solution:
    # Same as 3461. Check If Digits Are Equal in String After Operations I
    def hasSameDigits(self, s: str) -> bool:
        n = len(s)
        num1 = 0
        num2 = 0

        for i in range(n - 1):
            coefficient = self._nCMOD10(n - 2, i)
            num1 += (coefficient * (int(s[i]) - 0)) % 10
            num1 %= 10
            num2 += (coefficient * (int(s[i + 1]) - 0)) % 10
            num2 %= 10

        return num1 == num2

    def _nCMOD10(self, n: int, k: int) -> int:
        """Returns (n, k) % 10."""
        mod2 = self._lucasTheorem(n, k, 2)
        mod5 = self._lucasTheorem(n, k, 5)
        lookup = [
            [0, 6, 2, 8, 4], # mod2 == 0
            [5, 1, 7, 3, 9] # mod2 == 1
        ]
        return lookup[mod2][mod5]

    def _lucasTheorem(self, n: int, k: int, prime: int) -> int:
        """Returns (n, k) % prime."""
        res = 1
        while n > 0 or k > 0:
            nMod = n % prime
            MOD = k % prime
            res *= math.comb(nMod, MOD)
            res %= prime
            n //= prime
            k //= prime
        return res
```

2421. 3464.py

Path: LeetCode-main/solutions/3464. Maximize the Distance Between Points on a Square/3464.py

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Sequence:
    startX: int
    startY: int
    endX: int
    endY: int
    length: int

    def __iter__(self):
        yield self.startX
        yield self.startY
        yield self.endX
        yield self.endY
        yield self.length

class Solution:
    def maxDistance(self, side: int, points: list[list[int]], k: int) -> int:
        ordered = self._getOrderedPoints(side, points)

        def isValidDistance(m: int) -> bool:
            """
            Returns True if we can select `k` points such that the minimum Manhattan
            distance between any two consecutive chosen points is at least `m`.
            """
            dq = collections.deque([Sequence(*ordered[0], *ordered[0], 1)])
            maxLength = 1

            for i in range(1, len(ordered)):
                x, y = ordered[i]
                startX, startY = ordered[i]
                length = 1
                while dq and abs(x - dq[0].endX) + abs(y - dq[0].endY) >= m:
                    if (abs(x - dq[0].startX) + abs(y - dq[0].startY) >= m
                        and dq[0].length + 1 >= length):
                        startX = dq[0].startX
                        startY = dq[0].startY
                        length = dq[0].length + 1
                        maxLength = max(maxLength, length)
                    dq.popleft()
                dq.append(Sequence(startX, startY, x, y, length))

            return maxLength >= k

        l = 0
        r = side

        while l < r:
            m = (l + r + 1) // 2
            if isValidDistance(m):
                l = m
            else:
                r = m - 1

        return l

    def _getOrderedPoints(self, side: int, points: list[list[int]]) -> list[list[int]]:
        """
```

```
Returns the ordered points on the perimeter of a square of side length
`side`, starting from left, top, right, and bottom boundaries.
```

```
"""
left = sorted([(x, y) for x, y in points if x == 0 and y > 0])
top = sorted([(x, y) for x, y in points if x > 0 and y == side])
right = sorted([(x, y) for x, y in points if x == side and y < side],
               reverse=True)
bottom = sorted([(x, y) for x, y in points if y == 0], reverse=True)
return left + top + right + bottom
```

2422. 3466-2.py

Path: LeetCode-main/solutions/3466. Maximum Coin Collection/3466-2.py

```
class Solution:
    def maxCoins(self, lane1: list[int], lane2: list[int]) -> int:
        # dp[k] := the maximum number of coins at mile so far with k switches
        dp = (lane1[0], lane2[0], -math.inf)
        ans = max(dp)

        for i in range(1, len(lane1)):
            dp = (max(0, dp[0]) + lane1[i],
                  max(0, dp[0], dp[1]) + lane2[i],
                  max(dp[1], dp[2]) + lane1[i])
            ans = max(ans, max(dp))

        return ans
```

2423. 3466.py

Path: LeetCode-main/solutions/3466. Maximum Coin Collection/3466.py

```
class Solution:
    def maxCoins(self, lane1: list[int], lane2: list[int]) -> int:
        n = len(lane1)
        # dp[i][k] := the maximum number of coins at i-th mile with k switches
        dp = [[-math.inf] * 3 for _ in range(n)]
        dp[0][0] = lane1[0]
        dp[0][1] = lane2[0]

        for i in range(1, n):
            dp[i][0] = max(0, dp[i - 1][0]) + lane1[i]
            dp[i][1] = max(0, dp[i - 1][0], dp[i - 1][1]) + lane2[i]
            dp[i][2] = max(dp[i - 1][1], dp[i - 1][2]) + lane1[i]

        return max(map(max, dp))
```

2424. 3467.py

Path: LeetCode-main/solutions/3467. Transform Array by Parity/3467.py

```
class Solution:
    def transformArray(self, nums: list[int]) -> list[int]:
        return ([0] * sum(num % 2 == 0 for num in nums) +
                [1] * sum(num % 2 == 1 for num in nums))
```

2425. 3468.py

Path: LeetCode-main/solutions/3468. Find the Number of Copy Arrays/3468.py

```
class Solution:
    def countArrays(self, original: list[int], bounds: list[list[int]]) -> int:
        mn, mx = bounds[0]

        for i in range(1, len(original)):
            diff = original[i] - original[i - 1]
            mn = max(mn + diff, bounds[i][0])
            mx = min(mx + diff, bounds[i][1])

        return max(0, mx - mn + 1)
```

2426. 3469.py

Path: LeetCode-main/solutions/3469. Find Minimum Cost to Remove Array Elements/3469.py

```
class Solution:
    def minCost(self, nums: list[int]) -> int:
        n = len(nums)

        @functools.lru_cache(None)
        def dp(last: int, i: int) -> int:
            if i == n: # Single element left.
                return nums[last]
            if i == n - 1: # Two elements left.
                return max(nums[last], nums[i])
            a = max(nums[i], nums[i + 1]) + dp(last, i + 2)
            b = max(nums[last], nums[i]) + dp(i + 1, i + 2)
            c = max(nums[last], nums[i + 1]) + dp(i, i + 2)
            return min(a, b, c)

        return dp(0, 1)
```

2427. 347-2.py

Path: LeetCode-main/solutions/347. Top K Frequent Elements/347-2.py

```
class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        ans = []
        bucket = [[] for _ in range(len(nums) + 1)]

        for num, freq in collections.Counter(nums).items():
            bucket[freq].append(num)

        for b in reversed(bucket):
            ans += b
            if len(ans) == k:
                return ans
```

2428. 3470.py

Path: LeetCode-main/solutions/3470. Permutations IV/3470.py

```

class Solution:
    def permute(self, n: int, k: int) -> list[int]:
        ans = []
        isLookingForEven = True
        remainingNumbers = list(range(1, n + 1))

        for turn in range(n):
            remainingPermutations = (math.factorial((n - 1 - turn) // 2) *
                                     math.factorial((n - turn) // 2))

            found = False
            for index, number in enumerate(remainingNumbers):
                if number % 2 != isLookingForEven and (turn > 0 or n % 2 == 1):
                    continue
                if k <= remainingPermutations:
                    ans.append(remainingNumbers.pop(index))
                    isLookingForEven = ans[-1] % 2 == 0
                    found = True
                    break
                k -= remainingPermutations
            if not found:
                return []

        return ans

```

2429. 3471.py

Path: LeetCode-main/solutions/3471. Find the Largest Almost Missing Integer/3471.py

```

class Solution:
    def largestInteger(self, nums: list[int], k: int) -> int:
        if k == len(nums):
            return max(nums)
        count = collections.Counter(nums)
        if k == 1:
            return max([num for num in nums if count[num] == 1], default=-1)
        return max(
            nums[0] if count[nums[0]] == 1 else -1,
            nums[-1] if count[nums[-1]] == 1 else -1
        )

```

2430. 3472-2.py

Path: LeetCode-main/solutions/3472. Longest Palindromic Subsequence After at Most K Operations/3472-2.py

```

class Solution:
    # Similar to 516. Longest Palindromic Subsequence
    def longestPalindromicSubsequence(self, s: str, k: int) -> int:
        n = len(s)
        # dp[i][j][op] := the length of LPS(s[i..j]) with at most `op` operations
        dp = [[[0] * (k + 1) for _ in range(n)] for _ in range(n)]

        for i in range(n):
            for op in range(k + 1):
                dp[i][i][op] = 1

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                for op in range(k + 1):
                    if s[i] == s[j]:
                        dp[i][j][op] = 2 + dp[i + 1][j - 1][op]
                    else:
                        dp[i][j][op] = max(dp[i + 1][j][op], dp[i][j - 1][op])
                        cost = self._getCost(s[i], s[j])
                        if cost <= op:
                            dp[i][j][op] = max(dp[i][j][op], 2 + dp[i + 1][j - 1][op - cost])

        return dp[0][n - 1][k]

    def _getCost(self, a: str, b: str) -> int:
        dist = abs(ord(a) - ord(b))
        return min(dist, 26 - dist)

```

2431. 3472.py

Path: LeetCode-main/solutions/3472. Longest Palindromic Subsequence After at Most K Operations/3472.py

```

class Solution:
    # Similar to 516. Longest Palindromic Subsequence
    def longestPalindromicSubsequence(self, s: str, k: int) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int, op: int) -> int:
            """Returns the length of LPS(s[i..j]) with at most `op` operations."""
            if i > j:
                return 0
            if i == j:
                return 1
            if s[i] == s[j]:
                return 2 + dp(i + 1, j - 1, op)
            res = max(dp(i + 1, j, op), dp(i, j - 1, op))
            cost = self._getCost(s[i], s[j])
            if cost <= op:
                res = max(res, 2 + dp(i + 1, j - 1, op - cost))
            return res

        return dp(0, len(s) - 1, k)

    def _getCost(self, a: str, b: str) -> int:
        dist = abs(ord(a) - ord(b))
        return min(dist, 26 - dist)

```

2432. 3473-2.py

Path: LeetCode-main/solutions/3473. Sum of K Subarrays With Length at Least M/3473-2.py

```
class Solution:
    def maxSum(self, nums: list[int], k: int, m: int) -> int:
        INF = 20_000_000
        n = len(nums)
        prefix = list(itertools.accumulate(nums, initial=0))

        # dp[i][ongoing][r] := the maximum sum of nums[i:], with `ongoing`
        # indicating if a subarray is currently being extended (1) or not (0),
        # and `r` segments left to choose
        dp = [[[-INF] * (k + 1) for _ in range(2)] for _ in range(n + 1)]

        # Base case: At the end of the array, if no segments are left, score is 0
        dp[n][0][0] = dp[n][1][0] = 0

        for i in range(n - 1, -1, -1):
            for rem in range(k + 1):
                # When no subarray is ongoing:
                # 1. Skip nums[i]
                dp[i][0][rem] = dp[i + 1][0][rem]
                # 2. Start a new segment of length m (only if rem > 0 and there're enough elements)
                if rem > 0 and i + m <= n:
                    dp[i][0][rem] = max(
                        dp[i][0][rem],
                        dp[i + m][1][rem - 1] + (prefix[i + m] - prefix[i]))
                # When a subarray is ongoing:
                # 1. End the current subarray (transition to state 0, same index i)
                # 2. Extend the current subarray by picking nums[i] and move to i + 1
                dp[i][1][rem] = max(dp[i][0][rem], dp[i + 1][1][rem] + nums[i])

        return dp[0][0][k]
```

2433. 3473.py

Path: LeetCode-main/solutions/3473. Sum of K Subarrays With Length at Least M/3473.py


```
class Solution:
    def maxSum(self, nums: list[int], k: int, m: int) -> int:
        INF = 20_000_000
        n = len(nums)
        prefix = list(itertools.accumulate(nums, initial=0))

        @functools.lru_cache(None)
        def dp(i: int, ongoing: int, k: int) -> int:
            if k < 0:
                return -INF
            if i == n:
                return 0 if k == 0 else -INF
            if ongoing == 1:
                # 1. End the current subarray (transition to state 0, same index i)
                # 2. Extend the current subarray by picking nums[i] and move to i + 1
                return max(dp(i, 0, k),
                           dp(i + 1, 1, k) + nums[i])
            # ongoing == 0
            # 1. Skip nums[i]
            # 2. Pick nums[i:i+m] (only if k > 0 and there're enough elements)
            res = dp(i + 1, 0, k)
            if i + m <= n: # If we have enough elements for a new segment
                res = max(res,
                           dp(i + m, 1, k - 1) + (prefix[i + m] - prefix[i]))
            return res

        return dp(0, 0, k)
```

2434. 3474.py

Path: LeetCode-main/solutions/3474. Lexicographically Smallest Generated String/3474.py

```

class Solution:
    def generateString(self, str1: str, str2: str) -> str:
        n = len(str1)
        m = len(str2)
        sz = n + m - 1
        ans = [None] * sz
        modifiable = [True] * sz

        # 1. Handle all 'T' positions first.
        for i, tf in enumerate(str1):
            if tf == 'T':
                for j, c in enumerate(str2):
                    pos = i + j
                    if ans[pos] and ans[pos] != c:
                        return ''
                    ans[pos] = c
                    modifiable[pos] = False

        # 2. Fill all remaining positions with 'a'.
        for i in range(sz):
            if not ans[i]:
                ans[i] = 'a'

        # 3. Handle all 'F' positions.
        for i in range(n):
            if str1[i] == 'F' and self._match(ans, i, str2):
                modifiablePos = self._lastModifiablePosition(i, m, modifiable)
                if modifiablePos == -1:
                    return ''
                ans[modifiablePos] = 'b'
                modifiable[modifiablePos] = False

        return ''.join(ans)

    def _match(self, ans: list, i: int, s: str) -> bool:
        """Returns True if the substring of ans starting at `i` matches `s`."""
        for j, c in enumerate(s):
            if ans[i + j] != c:
                return False
        return True

    def _lastModifiablePosition(self, i: int, m: int, modifiable: list) -> int:
        """
        Finds the last modifiable position in the substring of ans starting at `i`.
        """
        modifiablePos = -1
        for j in range(m):
            pos = i + j
            if modifiable[pos]:
                modifiablePos = pos
        return modifiablePos

```

2435. 3476.py

Path: LeetCode-main/solutions/3476. Maximize Profit from Task Assignment/3476.py

```

class Solution:
    def maxProfit(self, workers: list[int], tasks: list[list[int]]) -> int:
        totalProfit = 0
        skillToProfits = collections.defaultdict(list)

        for skill, profit in tasks:
            skillToProfits[skill].append(profit)

        for skill in skillToProfits:
            skillToProfits[skill].sort(reverse=True)

        for workerSkill in workers:
            if workerSkill in skillToProfits and skillToProfits[workerSkill]:
                profit = skillToProfits[workerSkill][0]
                skillToProfits[workerSkill].pop(0)
                totalProfit += profit

        return totalProfit + max(max(profits, default=0)
                                for profits in skillToProfits.values())

```

2436. 3478.py

Path: LeetCode-main/solutions/3478. Choose K Elements With Maximum Sum/3478.py

```

class Solution:
    def findMaxSum(self, nums1: list[int], nums2: list[int], k: int) -> list[int]:
        ans = [0] * len(nums1)
        numAndIndexes = sorted([(num, i) for i, num in enumerate(nums1)])
        minHeap = []

        firstIndex = numAndIndexes[0][1]
        heapq.heappush(minHeap, nums2[firstIndex])
        summ = nums2[firstIndex]

        for (prevNum, prevIndex), (currNum, currIndex) in itertools.pairwise(numAndIndexes):
            if currNum == prevNum:
                ans[currIndex] = ans[prevIndex]
            else:
                ans[currIndex] = summ
                heapq.heappush(minHeap, nums2[currIndex])
                summ += nums2[currIndex]
            if len(minHeap) == k + 1:
                summ -= heapq.heappop(minHeap)

        return ans

```

2437. 348.py

Path: LeetCode-main/solutions/348. Design Tic-Tac-Toe/348.py

```

class TicTacToe:
    def __init__(self, n: int):
        self.n = n
        # Record count('X') - count('O').
        self.rows = [0] * n
        self.cols = [0] * n
        self.diag = 0
        self.antiDiag = 0

    """ Player {player} makes a move at ({row}, {col}).

    @param row    The row of the board.
    @param col    The column of the board.
    @param player The player, can be either 1 or 2.
    @return The current winning condition, can be either:
        0: No one wins.
        1: Player 1 wins.
        2: Player 2 wins.
    """

    def move(self, row: int, col: int, player: int) -> int:
        toAdd = 1 if player == 1 else -1
        target = self.n if player == 1 else -self.n

        if row == col:
            self.diag += toAdd
            if self.diag == target:
                return player

        if row + col == self.n - 1:
            self.antiDiag += toAdd
            if self.antiDiag == target:
                return player

        self.rows[row] += toAdd
        if self.rows[row] == target:
            return player

        self.cols[col] += toAdd
        if self.cols[col] == target:
            return player

        return 0

```

2438. 3480.py

Path: LeetCode-main/solutions/3480. Maximize Subarrays After Removing One Conflicting Pair/3480.py

```

class Solution:
    def maxSubarrays(self, n: int, conflictingPairs: list[list[int]]) -> int:
        validSubarrays = 0
        maxLeft = 0
        secondMaxLeft = 0
        # gains[i] := the number of additional valid subarrays we can gain if the
        # restriction at index `i` is removed
        gains = [0] * (n + 1)
        # conflicts[r] := left endpoints that conflict with subarrays ending in r
        conflicts = [[] for _ in range(n + 1)]

        for a, b in conflictingPairs:
            conflicts[max(a, b)].append(min(a, b))

        for right in range(1, n + 1):
            for left in conflicts[right]:
                if left > maxLeft:
                    secondMaxLeft = maxLeft
                    maxLeft = left
                elif left > secondMaxLeft:
                    secondMaxLeft = left
            # Subarrays [maxLeft + 1..right],
            # [maxLeft + 2..right],
            # ...
            # [right..right] are valid.
            validSubarrays += right - maxLeft
            # If we remove `maxLeft` (the most restrictive conflict), we gain
            # `maxLeft - secondMaxLeft` new subarrays:
            # [secondMaxLeft + 1..right],
            # [secondMaxLeft + 2..right],
            # ...
            # [maxLeft..right].
            gains[maxLeft] += maxLeft - secondMaxLeft

        return validSubarrays + max(gains)

```

2439. 3481.py

Path: LeetCode-main/solutions/3481. Apply Substitutions/3481.py

```

class Solution:
    def applySubstitutions(self, replacements: list[list[str]], text: str) -> str:
        replaceMap = {key: value for key, value in replacements}

        def evaluate(text: str) -> str:
            """
            Evaluates the text and replaces the placeholders with the values
            from the replace_map recursively.
            """
            res = []
            i = 0
            while i < len(text):
                if text[i] == '%':
                    j = i + 1 + text[i + 1:].find('%')
                    key = text[i + 1:j]
                    value = replaceMap[key]
                    res.append(evaluate(value))
                    i = j + 1
                else:
                    res.append(text[i])
                    i += 1
            return ''.join(res)

        return evaluate(text)

```

2440. 3483.py

Path: LeetCode-main/solutions/3483. Unique 3-Digit Even Numbers/3483.py

```

class Solution:
    def totalNumbers(self, digits: list[int]) -> int:
        nums = set()

        for a, b, c in itertools.permutations(digits, 3):
            if a != 0 and c % 2 == 0:
                nums.add(a * 100 + b * 10 + c)

        return len(nums)

```

2441. 3484.py

Path: LeetCode-main/solutions/3484. Design Spreadsheet/3484.py

```

class Spreadsheet:
    def __init__(self, rows: int) -> None:
        self.spreadsheet = {}

    def setCell(self, cell: str, value: int) -> None:
        self.spreadsheet[cell] = value

    def resetCell(self, cell: str) -> None:
        self.spreadsheet[cell] = 0

    def getValue(self, formula: str) -> int:
        i = formula.find('+')
        return self._getToken(formula[1:i]) + self._getToken(formula[i+1:])

    def _getToken(self, token: str) -> int:
        return int(token) if token[0].isdigit() else self.spreadsheet.get(token, 0)

```

2442. 3485.py

Path: LeetCode-main/solutions/3485. Longest Common Prefix of K Strings After Removal/3485.py

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.count = 0

class Trie:
    def __init__(self, k: int):
        self.k = k
        self.root = TrieNode()
        self.prefixLengthsCount = collections.Counter()
        self.prefixLengths = SortedList()

    def insert(self, word: str) -> None:
        node = self.root
        for i, c in enumerate(word):
            sz = i + 1
            node = node.children.setdefault(c, TrieNode())
            node.count += 1
            if node.count >= self.k:
                self.prefixLengthsCount[sz] += 1
                if self.prefixLengthsCount[sz] == 1:
                    self.prefixLengths.add(-sz)

    def erase(self, word: str) -> None:
        node = self.root
        for i, c in enumerate(word):
            sz = i + 1
            node = node.children[c]
            if node.count == self.k:
                self.prefixLengthsCount[sz] -= 1
                if self.prefixLengthsCount[sz] == 0:
                    self.prefixLengths.remove(-sz)
            node.count -= 1

    def getLongestCommonPrefix(self) -> int:
        return 0 if not self.prefixLengths else -self.prefixLengths[0]

class Solution:
    def longestCommonPrefix(self, words: list[str], k: int) -> list[int]:
        ans = []
        trie = Trie(k)

        for word in words:
            trie.insert(word)

        for word in words:
            trie.erase(word)
            ans.append(trie.getLongestCommonPrefix())
            trie.insert(word)

        return ans
```

2443. 3486.py

Path: LeetCode-main/solutions/3486. Longest Special Path II/3486.py

```

class Solution:
    # Similar to 3425. Longest Special Path
    def longestSpecialPath(
        self,
        edges: list[list[int]],
        nums: list[int]
    ) -> list[int]:
        maxLength = 0
        minNodes = 1
        graph = [[] for _ in range(len(nums))]

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        prefix = [0]
        lastSeenDepth = {}

        def dfs(
            u: int,
            prev: int,
            leftBoundary: list[int],
        ) -> None:
            nonlocal maxLength, minNodes
            prevDepth = lastSeenDepth.get(nums[u], 0)
            lastSeenDepth[nums[u]] = len(prefix)

            if prevDepth != 0:
                leftBoundary = sorted(leftBoundary + [prevDepth])[-2:]

            length = prefix[-1] - prefix[leftBoundary[0]]
            nodes = len(prefix) - leftBoundary[0]
            if length > maxLength or (length == maxLength and nodes < minNodes):
                maxLength = length
                minNodes = nodes

            for v, w in graph[u]:
                if v == prev:
                    continue
                prefix.append(prefix[-1] + w)
                dfs(v, u, leftBoundary)
                prefix.pop()

            lastSeenDepth[nums[u]] = prevDepth

        dfs(0, -1, leftBoundary=[0, 0])
        return [maxLength, minNodes]

```

2444. 3487.py

Path: LeetCode-main/solutions/3487. Maximum Unique Subarray Sum After Deletion/3487.py

```

class Solution:
    def maxSum(self, nums: list[int]) -> int:
        mx = max(nums)
        if mx <= 0:
            return mx
        return sum(max(0, num) for num in set(nums))

```


2445. 3488.py

Path: LeetCode-main/solutions/3488. Closest Equal Element Queries/3488.py

```
class Solution:
    def solveQueries(self, nums: list[int], queries: list[int]) -> list[int]:
        n = len(nums)
        # minDist[i] := the minimum distance between nums[i], and any other index j
        # in the circular array, where nums[j] == nums[i]
        minDist = [n] * n
        lastSeen = {}

        for i in range(n * 2):
            index = i % n
            num = nums[index]
            if num in lastSeen:
                prevIndex = lastSeen[num] % n
                d = i - prevIndex
                minDist[index] = min(minDist[index], d)
                minDist[prevIndex] = min(minDist[prevIndex], d)
            lastSeen[num] = i

        return [-1 if minDist[query] == n
                else minDist[query]
                for query in queries]
```

2446. 3489.py

Path: LeetCode-main/solutions/3489. Zero Array Transformation IV/3489.py

```
class Solution:
    def minZeroArray(self, nums: list[int], queries: list[list[int]]) -> int:
        if all(num == 0 for num in nums):
            return 0

        n = len(nums)
        subsetSums = [{0} for _ in range(n)]

        for k, (l, r, val) in enumerate(queries):
            for i in range(l, r + 1):
                newSums = {subsetSum + val for subsetSum in subsetSums[i]}
                subsetSums[i].update(newSums)
            if all(nums[i] in subsetSums[i] for i in range(n)):
                return k + 1

        return -1
```

2447. 349.py

Path: LeetCode-main/solutions/349. Intersection of Two Arrays/349.py

```

class Solution:
    def intersection(self, nums1: list[int], nums2: list[int]) -> list[int]:
        ans = []
        nums1 = set(nums1)

        for num in nums2:
            if num in nums1:
                ans.append(num)
                nums1.remove(num)

        return ans

```

2448. 3490.py

Path: LeetCode-main/solutions/3490. Count Beautiful Numbers/3490.py

```

class Solution:
    def beautifulNumbers(self, l: int, r: int) -> int:
        @functools.lru_cache(None)
        def dp(
            s: str,
            i: int,
            tight: bool,
            isLeadingZero: bool,
            hasZero: bool,
            sum: int,
            prod: int,
        ) -> int:
            if i == len(s):
                if isLeadingZero:
                    return 0
                return 1 if hasZero or prod % sum == 0 else 0
            if not isLeadingZero and hasZero and not tight:
                return 10 ** (len(s) - i)

            res = 0
            maxDigit = int(s[i]) if tight else 9

            for d in range(maxDigit + 1):
                nextTight = tight and (d == maxDigit)
                nextIsLeadingZero = isLeadingZero and d == 0
                nextHasZero = not nextIsLeadingZero and d == 0
                nextProd = 1 if nextIsLeadingZero else prod * d
                res += dp(s, i + 1, nextTight, nextIsLeadingZero,
                        nextHasZero, sum + d, nextProd)

            return res

        return (dp(str(r), 0, tight=True, isLeadingZero=True, hasZero=False, sum=0, prod=1) -
                dp(str(l - 1), 0, tight=True, isLeadingZero=True, hasZero=False, sum=0, prod=1))

```

2449. 3491-2.py

Path: LeetCode-main/solutions/3491. Phone Number Prefix/3491-2.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, number: str) -> None:
        node: TrieNode = self.root
        for d in number:
            node = node.children.setdefault(d, TrieNode())
        node.isWord = True

    def search(self, number: str) -> bool:
        """
        Returns True if the number is a prefix of any number in the trie or any
        number in the trie is a prefix of the number.
        """
        node: TrieNode = self.root
        for d in number:
            if d not in node.children:
                return False
            node = node.children[d]
            if node.isWord:
                return True
        return True # The number is a prefix of some number in the trie.

class Solution:
    def phonePrefix(self, numbers: list[str]) -> bool:
        trie = Trie()

        for number in numbers:
            if trie.search(number):
                return False
            trie.insert(number)

        return True

```

2450. 3491.py

Path: LeetCode-main/solutions/3491. Phone Number Prefix/3491.py

```

class Solution:
    def phonePrefix(self, numbers: list[str]) -> bool:
        return not any(b.startswith(a)
                        for a, b in itertools.pairwise(sorted(numbers)))

```

2451. 3492.py

Path: LeetCode-main/solutions/3492. Maximum Containers on a Ship/3492.py

```

class Solution:
    def maxContainers(self, n: int, w: int, maxWeight: int) -> int:
        return min(n * n, maxWeight // w)

```

2452. 3493.py

Path: LeetCode-main/solutions/3493. Properties Graph/3493.py

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def getCount(self) -> int:
        return self.count

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def numberOfComponents(self, properties: list[list[int]], k: int) -> int:
        n = len(properties)
        uf = UnionFind(n)
        propertySets = [set(property) for property in properties]

        for i, j in itertools.combinations(range(n), 2):
            if len(propertySets[i] & propertySets[j]) >= k:
                uf.unionByRank(i, j)

        return uf.getCount()
```

2453. 3494.py

Path: LeetCode-main/solutions/3494. Find the Minimum Amount of Time to Brew Potions/3494.py

```

class Solution:
    def minTime(self, skill: list[int], mana: list[int]) -> int:
        sumSkill = sum(skill)
        prevWizardDone = sumSkill * mana[0]

        for j in range(1, len(mana)):
            prevPotionDone = prevWizardDone
            for i in range(len(skill) - 2, -1, -1):
                # start time for wizard i brewing potion j
                # = max(end time for wizard i brewing potion j - 1,
                #       the earliest start time for wizard i + 1 brewing potion j
                #       (coming from previous iteration)
                #       - time for wizard i brewing potion j)
                prevPotionDone -= skill[i + 1] * mana[j - 1]
                prevWizardDone = max(prevPotionDone,
                                     prevWizardDone - skill[i] * mana[j])
            prevWizardDone += sumSkill * mana[j]

        return prevWizardDone

```

2454. 3495.py

Path: LeetCode-main/solutions/3495. Minimum Operations to Make Array Elements Zero/3495.py

```

class Solution:
    def minOperations(self, queries: list[list[int]]) -> int:
        return sum((self._getOperations(r) - self._getOperations(l - 1) + 1) // 2
                    for l, r in queries)

    def _getOperations(self, n: int) -> int:
        """Returns the number of operations required for [1, n]."""
        res = 0
        ops = 0
        powerOfFour = 1
        while powerOfFour <= n:
            l = powerOfFour
            r = min(n, powerOfFour * 4 - 1)
            ops += 1
            res += (r - l + 1) * ops
            powerOfFour *= 4
        return res

```

2455. 3496.py

Path: LeetCode-main/solutions/3496. Maximize Score After Pair Deletions/3496.py

```

class Solution:
    def maxScore(self, nums: list[int]) -> int:
        summ = sum(nums)
        if len(nums) % 2 == 1:
            return summ - min(nums)
        return summ - min(a + b for a, b in itertools.pairwise(nums))

```

2456. 3498.py

Path: LeetCode-main/solutions/3498. Reverse Degree of a String/3498.py

```
class Solution:
    def reverseDegree(self, s: str) -> int:
        return sum((26 - (ord(c) - ord('a')))) * (i + 1)
                   for i, c in enumerate(s))
```

2457. 3499.py

Path: LeetCode-main/solutions/3499. Maximize Active Section with Trade I/3499.py

```
class Solution:
    def maxActiveSectionsAfterTrade(self, s: str) -> int:
        zeroGroups = [len(list(g)) for c, g in itertools.groupby(s) if c == '0']
        return s.count('1') + max(map(sum, pairwise(zeroGroups)), default=0)
```

2458. 35.py

Path: LeetCode-main/solutions/35. Search Insert Position/35.py

```
class Solution:
    def searchInsert(self, nums: list[int], target: int) -> int:
        l = 0
        r = len(nums)

        while l < r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[m] < target:
                l = m + 1
            else:
                r = m

        return l
```

2459. 350.py

Path: LeetCode-main/solutions/350. Intersection of Two Arrays II/350.py

```
class Solution:
    def intersect(self, nums1: list[int], nums2: list[int]) -> list[int]:
        if len(nums1) > len(nums2):
            return self.intersect(nums2, nums1)

        ans = []
        count = collections.Counter(nums1)

        for num in nums2:
            if count[num] > 0:
                ans.append(num)
                count[num] -= 1

        return ans
```

2460. 3500.py

Path: LeetCode-main/solutions/3500. Minimum Cost to Divide Array Into Subarrays/3500.py

```
class Solution:
    def minimumCost(self, nums: list[int], cost: list[int], k: int) -> int:
        n = len(nums)
        prefixNums = list(itertools.accumulate(nums, initial=0))
        prefixCost = list(itertools.accumulate(cost, initial=0))
        # dp[i] := the minimum cost to divide nums[i..n - 1] into subarrays
        dp = [math.inf] * n + [0]

        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                dp[i] = min(dp[i],
                            prefixNums[j + 1] * (prefixCost[j + 1] - prefixCost[i]) +
                            k * (prefixCost[n] - prefixCost[i]) + dp[j + 1])

        return dp[0]
```

2461. 3501.py

Path: LeetCode-main/solutions/3501. Maximize Active Section with Trade II/3501.py

```
from dataclasses import dataclass

@dataclass
class Group:
    start: int
    length: int

class SparseTable:
    def __init__(self, nums: list[int]):
        self.n = len(nums)
        # st[i][j] := max(nums[j..j + 2^i - 1])
        self.st = [[0] * (self.n + 1) for _ in range(self.n.bit_length() + 1)]
        self.st[0] = nums.copy()
        for i in range(1, self.n.bit_length() + 1):
            for j in range(self.n - (1 << i) + 1):
                self.st[i][j] = max(
                    self.st[i - 1][j],
                    self.st[i - 1][j + (1 << (i - 1))])

    def query(self, l: int, r: int) -> int:
        """Returns max(nums[l..r])."""
        i = (r - l + 1).bit_length() - 1
        return max(self.st[i][l], self.st[i][r - (1 << i) + 1])

class Solution:
    def maxActiveSectionsAfterTrade(
        self,
        s: str,
        queries: list[list[int]]
    ) -> list[int]:
        ones = s.count('1')
        zeroGroups, zeroGroupIndex = self._getZeroGroups(s)
        if not zeroGroups:
            return [ones] * len(queries)

        st = SparseTable(self._getZeroMergeLengths(zeroGroups))

        def getMaxActiveSections(l: int, r: int) -> int:
```

```

left = (-1 if zeroGroupIndex[l] == -1
        else (zeroGroups[zeroGroupIndex[l]].length -
              (1 - zeroGroups[zeroGroupIndex[l]].start)))
right = (-1 if zeroGroupIndex[r] == -1
         else (r - zeroGroups[zeroGroupIndex[r]].start + 1))
startAdjacentGroupIndex, endAdjacentGroupIndex = self._mapToAdjacentGroupIndices(
    zeroGroupIndex[l] + 1, zeroGroupIndex[r] if s[r] == '1' else zeroGroupIndex[r] - 1)
activeSections = ones
if (s[l] == '0' and s[r] == '0' and
    zeroGroupIndex[l] + 1 == zeroGroupIndex[r]):
    activeSections = max(activeSections, ones + left + right)
elif startAdjacentGroupIndex <= endAdjacentGroupIndex:
    activeSections = max(
        activeSections,
        ones + st.query(startAdjacentGroupIndex, endAdjacentGroupIndex))
if (s[l] == '0' and
    zeroGroupIndex[l] + 1 <= (zeroGroupIndex[r]
                              if s[r] == '1' else zeroGroupIndex[r] - 1)):
    activeSections = max(activeSections, ones + left +
                          zeroGroups[zeroGroupIndex[l] + 1].length)
if (s[r] == '0' and zeroGroupIndex[l] < zeroGroupIndex[r] - 1):
    activeSections = max(activeSections, ones + right +
                          zeroGroups[zeroGroupIndex[r] - 1].length)
return activeSections

return [getMaxActiveSections(l, r) for l, r in queries]

def _getZeroGroups(self, s: str) -> tuple[list[Group], list[int]]:
    """
    Returns the zero groups and the index of the zero group that contains the
    i-th character.
    """
    zeroGroups = []
    zeroGroupIndex = []
    for i in range(len(s)):
        if s[i] == '0':
            if i > 0 and s[i - 1] == '0':
                zeroGroups[-1].length += 1
            else:
                zeroGroups.append(Group(i, 1))
            zeroGroupIndex.append(len(zeroGroups) - 1)
    return zeroGroups, zeroGroupIndex

def _getZeroMergeLengths(self, zeroGroups: list[Group]) -> list[int]:
    """Returns the sums of the lengths of the adjacent groups."""
    return [a.length + b.length for a, b in itertools.pairwise(zeroGroups)]

def _mapToAdjacentGroupIndices(
    self,
    startGroupIndex: int,
    endGroupIndex: int
) -> tuple[int, int]:
    """
    Returns the indices of the adjacent groups that contain l and r completely.

    e.g.    groupIndices = [0, 1, 2, 3]
    adjacentGroupIndices = [0 (0, 1), 1 (1, 2), 2 (2, 3)]
    map(startGroupIndex = 1, endGroupIndex = 3) -> (1, 2)
    """
    return startGroupIndex, endGroupIndex - 1

```


Path: LeetCode-main/solutions/3502. Minimum Cost to Reach Every Position/3502.py

```
class Solution:
    def minCosts(self, cost: list[int]) -> list[int]:
        return list(itertools.accumulate(cost, min))
```

2463. 3503.py

Path: LeetCode-main/solutions/3503. Longest Palindrome After Substring Concatenation I/3503.py

```
class Solution:
    def longestPalindrome(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        suffix = self._getPalindromeLengths(s, True)
        prefix = self._getPalindromeLengths(t, False)
        ans = max(max(suffix), max(prefix))
        # dp[i][j] := the longest length of palindrome starting in s[i] and ending
        # in t[j]
        dp = [[0] * n for _ in range(m)]

        for i in range(m):
            for j in range(n - 1, -1, -1):
                if s[i] == t[j]:
                    dp[i][j] = 2 + (dp[i - 1][j + 1] if i > 0 and j < n - 1 else 0)
                    extend = max(suffix[i + 1] if i + 1 < m else 0,
                                prefix[j - 1] if j > 0 else 0)
                    ans = max(ans, dp[i][j] + extend)

        return ans

    def _getPalindromeLengths(self, s: str, isSuffix: bool) -> list[int]:
        n = len(s)
        # dp[i][j] := True if s[i..j] is a palindrome
        dp = [[False] * n for _ in range(n)]
        # lengths[i] := length of longest palindrome in s[i..n - 1]
        lengths = [0] * n
        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                if s[i] == s[j] and (j - i < 2 or dp[i + 1][j - 1]):
                    dp[i][j] = True
                    index = i if isSuffix else j
                    lengths[index] = max(lengths[index], j - i + 1)
        return lengths
```

2464. 3504.py

Path: LeetCode-main/solutions/3504. Longest Palindrome After Substring Concatenation II/3504.py

```

class Solution:
    # 3503. Longest Palindrome After Substring Concatenation I
    def longestPalindrome(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        suffix = self._getPalindromeLengths(s, True)
        prefix = self._getPalindromeLengths(t, False)
        ans = max(max(suffix), max(prefix))
        # dp[i][j] := the longest length of palindrome starting in s[i] and ending
        # in t[j]
        dp = [[0] * n for _ in range(m)]

        for i in range(m):
            for j in range(n - 1, -1, -1):
                if s[i] == t[j]:
                    dp[i][j] = 2 + (dp[i - 1][j + 1] if i > 0 and j < n - 1 else 0)
                    extend = max(suffix[i + 1] if i + 1 < m else 0,
                                prefix[j - 1] if j > 0 else 0)
                    ans = max(ans, dp[i][j] + extend)

        return ans

    def _getPalindromeLengths(self, s: str, isSuffix: bool) -> list[int]:
        n = len(s)
        # dp[i][j] := True if s[i..j] is a palindrome
        dp = [[False] * n for _ in range(n)]
        # lengths[i] := length of longest palindrome in s[i..n - 1]
        lengths = [0] * n
        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                if s[i] == s[j] and (j - i < 2 or dp[i + 1][j - 1]):
                    dp[i][j] = True
                    index = i if isSuffix else j
                    lengths[index] = max(lengths[index], j - i + 1)
        return lengths

```

2465. 3506.py

Path: LeetCode-main/solutions/3506. Find Time Required to Eliminate Bacterial Strains/3506.py

```

class Solution:
    def minEliminationTime(self, timeReq: list[int], splitTime: int) -> int:
        minHeap = timeReq.copy()
        heapq.heapify(minHeap)
        heapq.heappop(minHeap)

        while True:
            bacterial = splitTime + heapq.heappop(minHeap)
            if not minHeap:
                return bacterial
            heapq.heappushpop(minHeap, bacterial)

```

2466. 3507.py

Path: LeetCode-main/solutions/3507. Minimum Pair Removal to Sort Array I/3507.py

```
class Solution:
    def minimumPairRemoval(self, nums: list[int]) -> int:
        ans = 0

        while any(x > y for x, y in itertools.pairwise(nums)):
            pairSums = [x + y for x, y in itertools.pairwise(nums)]
            minPairSum = min(pairSums)
            minPairIndex = pairSums.index(minPairSum)
            nums[minPairIndex] = minPairSum
            nums.pop(minPairIndex + 1)
            ans += 1

        return ans
```

2467. 3508.py

Path: LeetCode-main/solutions/3508. Implement Router/3508.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Packet:
    source: int
    destination: int
    timestamp: int

class Router:
    def __init__(self, memoryLimit: int):
        self.memoryLimit = memoryLimit
        self.uniquePackets: set[Packet] = set()
        self.packetQueue: collections.deque[Packet] = collections.deque()
        self.destinationTimestamps = collections.defaultdict(list)
        self.processedPacketIndex = collections.Counter()

    def addPacket(self, source: int, destination: int, timestamp: int) -> bool:
        packet = Packet(source, destination, timestamp)
        if packet in self.uniquePackets:
            return False
        if len(self.packetQueue) == self.memoryLimit:
            self.forwardPacket()
        self.packetQueue.append(packet)
        self.uniquePackets.add(packet)
        if destination not in self.destinationTimestamps:
            self.destinationTimestamps[destination] = []
        self.destinationTimestamps[destination].append(timestamp)
        return True

    def forwardPacket(self) -> list[int]:
        if not self.packetQueue:
            return []
        nextPacket = self.packetQueue.popleft()
        self.uniquePackets.remove(nextPacket)
        self.processedPacketIndex[nextPacket.destination] += 1
        return [nextPacket.source, nextPacket.destination, nextPacket.timestamp]

    def getCount(self, destination: int, startTime: int, endTime: int) -> int:
        if destination not in self.destinationTimestamps:
            return 0
        timestamps = self.destinationTimestamps[destination]
        startIndex = self.processedPacketIndex.get(destination, 0)
        lowerBound = bisect.bisect_left(timestamps, startTime, startIndex)
        upperBound = bisect.bisect_right(timestamps, endTime, startIndex)
        return upperBound - lowerBound

```

2468. 3509.py

Path: LeetCode-main/solutions/3509. Maximum Product of Subsequences With an Alternating Sum Equal to K/3509.py

```

from enum import Enum

class State(Enum):
    FIRST = 0 # first element - add to sum and start product
    SUBTRACT = 1 # second element - subtract from sum and multiply product
    ADD = 2 # third element - add to sum and multiply product

class Solution:
    def maxProduct(self, nums: list[int], k: int, limit: int) -> int:
        MIN = -5000
        if abs(k) > sum(nums):
            return -1

        @functools.lru_cache(None)
        def dp(i: int, product: int, state: State, k: int) -> int:
            if i == len(nums):
                return (product if k == 0 and state != State.FIRST and product <= limit
                        else MIN)
            res = dp(i + 1, product, state, k)
            if state == State.FIRST:
                res = max(res, dp(i + 1, nums[i], State.SUBTRACT, k - nums[i]))
            if state == State.SUBTRACT:
                res = max(res, dp(i + 1, min(product * nums[i], limit + 1),
                                           State.ADD, k + nums[i]))
            if state == State.ADD:
                res = max(res, dp(i + 1, min(product * nums[i], limit + 1),
                                           State.SUBTRACT, k - nums[i]))
            return res

        ans = dp(0, 1, State.FIRST, k)
        return -1 if ans == MIN else ans

```

2469. 351.py

Path: LeetCode-main/solutions/351. Android Unlock Patterns/351.py

```

class Solution:
    def numberOfPatterns(self, m: int, n: int) -> int:
        seen = set()
        accross = [[0] * 10 for _ in range(10)]

        accross[1][3] = accross[3][1] = 2
        accross[1][7] = accross[7][1] = 4
        accross[3][9] = accross[9][3] = 6
        accross[7][9] = accross[9][7] = 8
        accross[1][9] = accross[9][1] = accross[2][8] = accross[8][2] = \
            accross[3][7] = accross[7][3] = accross[4][6] = accross[6][4] = 5

    def dfs(u: int, depth: int) -> int:
        if depth > n:
            return 0

        seen.add(u)
        ans = 1 if depth >= m else 0

        for v in range(1, 10):
            if v == u or v in seen:
                continue
            accrossed = accross[u][v]
            if not accrossed or accrossed in seen:
                ans += dfs(v, depth + 1)

        seen.remove(u)
        return ans

# 1, 3, 7, 9 are symmetric
# 2, 4, 6, 8 are symmetric
return dfs(1, 1) * 4 + dfs(2, 1) * 4 + dfs(5, 1)

```

2470. 3510.py

Path: LeetCode-main/solutions/3510. Minimum Pair Removal to Sort Array II/3510.py

```

from sortedcontainers import SortedList

class Solution:
    def minimumPairRemoval(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        inversionsCount = sum(nums[i + 1] < nums[i] for i in range(n - 1))
        nextIndices = [i + 1 for i in range(n)]
        prevIndices = [i - 1 for i in range(n)]
        pairSums = SortedList((a + b, i)
                               for i, (a, b) in enumerate(itertools.pairwise(nums)))

        while inversionsCount > 0:
            ans += 1
            smallestPair = pairSums.pop(0)
            pairSum, currIndex = smallestPair
            nextIndex = nextIndices[currIndex]
            prevIndex = prevIndices[currIndex]

            if prevIndex >= 0:
                oldPairSum = nums[prevIndex] + nums[currIndex]
                newPairSum = nums[prevIndex] + pairSum
                pairSums.remove((oldPairSum, prevIndex))
                pairSums.add((newPairSum, prevIndex))
                if nums[prevIndex] > nums[currIndex]:
                    inversionsCount -= 1
                if nums[prevIndex] > pairSum:
                    inversionsCount += 1

            if nums[nextIndex] < nums[currIndex]:
                inversionsCount -= 1

            nextNextIndex = nextIndices[nextIndex] if nextIndex < n else n
            if nextNextIndex < n:
                oldPairSum = nums[nextIndex] + nums[nextNextIndex]
                newPairSum = pairSum + nums[nextNextIndex]
                pairSums.remove((oldPairSum, nextIndex))
                pairSums.add((newPairSum, currIndex))
                if nums[nextNextIndex] < nums[nextIndex]:
                    inversionsCount -= 1
                if nums[nextNextIndex] < pairSum:
                    inversionsCount += 1
                prevIndices[nextNextIndex] = currIndex

            nextIndices[currIndex] = nextNextIndex
            nums[currIndex] = pairSum

        return ans

```

2471. 3511.py

Path: LeetCode-main/solutions/3511. Make a Positive Array/3511.py

```
class Solution:
    def makeArrayPositive(self, nums: list[int]) -> int:
        MAX = 10**18
        ans = 0
        minSum = nums[0] + nums[1]

        for i in range(2, len(nums)):
            a, b, c = nums[i - 2], nums[i - 1], nums[i]
            minSum = min(minSum + c, a + b + c)
            if minSum <= 0:
                nums[i] = MAX
                minSum = MAX
                ans += 1

        return ans
```

2472. 3512.py

Path: LeetCode-main/solutions/3512. Minimum Operations to Make Array Sum Divisible by K/3512.py

```
class Solution:
    def minOperations(self, nums: list[int], k: int) -> int:
        return sum(nums) % k
```

2473. 3513.py

Path: LeetCode-main/solutions/3513. Number of Unique XOR Triplets I/3513.py

```
class Solution:
    def uniqueXorTriplets(self, nums: list[int]) -> int:
        n = len(nums)
        if n < 3:
            return n
        return 1 << (int(math.log2(n)) + 1)
```

2474. 3515.py

Path: LeetCode-main/solutions/3514. Number of Unique XOR Triplets II/3515.py

```
class Solution:
    def uniqueXorTriplets(self, nums: list[int]) -> int:
        n = len(nums)
        if n == 1:
            return 1
        pairs = set(nums[i] ^ nums[j]
                     for i, j in itertools.combinations(range(n), 2))
        return len(set(pair ^ num
                       for pair in pairs
                       for num in nums))
```

2475. 3516.py

Path: LeetCode-main/solutions/3516. Find Closest Person/3516.py


```
class Solution:
    def findClosest(self, x: int, y: int, z: int) -> int:
        xz = abs(x - z)
        yz = abs(y - z)
        if xz == yz:
            return 0
        return 1 if xz < yz else 2
```

2476. 3517.py

Path: LeetCode-main/solutions/3517. Smallest Palindromic Rearrangement I/3517.py

```
class Solution:
    def smallestPalindrome(self, s: str) -> str:
        n = len(s)
        sortedHalf = sorted(s[:n // 2])
        return ''.join(sortedHalf +
                        ([s[n // 2]] if n % 2 else []) +
                        sortedHalf[::-1])
```

2477. 3518.py

Path: LeetCode-main/solutions/3518. Smallest Palindromic Rearrangement II/3518.py

```
class Solution:
    def __init__(self):
        self.MAX = 10**6 + 1

    def smallestPalindrome(self, s: str, k: int) -> str:
        count = collections.Counter(s)
        if not self._isPalindromePossible(count):
            return ''

        halfCount, midLetter = self._getHalfCountAndMidLetter(count)
        totalPerm = self._calculateTotalPermutations(halfCount)
        if k > totalPerm:
            return ''
        leftHalf = self._generateLeftHalf(halfCount, k)
        return ''.join(leftHalf) + midLetter + ''.join(reversed(leftHalf))

    def _isPalindromePossible(self, count: collections.Counter) -> bool:
        oddCount = sum(1 for count in count.values() if count % 2 == 1)
        return oddCount <= 1

    def _getHalfCountAndMidLetter(self, count: collections.Counter) -> tuple[list[int], str]:
        halfCount = [0] * 26
        midLetter = ''
        for c, freq in count.items():
            halfCount[ord(c) - ord('a')] = freq // 2
            if freq % 2 == 1:
                midLetter = c
        return halfCount, midLetter

    def _calculateTotalPermutations(self, halfCount: list[int]) -> int:
        """Calculate the total number of possible permutations."""
        return self._countArrangements(halfCount)

    def _generateLeftHalf(self, halfCount: list[int], k: int) -> list[str]:
        """Generate the left half of the palindrome based on k."""
        halfLen = sum(halfCount)
```

```

left = []
for _ in range(halfLen):
    for i, freq in enumerate(halfCount):
        if freq == 0:
            continue
        halfCount[i] -= 1
        arrangements = self._countArrangements(halfCount)
        if arrangements >= k:
            left.append(chr(i + ord('a')))
            break
        else:
            k -= arrangements
            halfCount[i] += 1
return left

def _countArrangements(self, count: list[int]) -> int:
    """Calculate the number of possible arrangements of characters."""
    total = sum(count)
    res = 1
    for freq in count:
        res *= self._nCk(total, freq)
        if res >= self.MAX:
            return self.MAX
        total -= freq
    return res

def _nCk(self, n: int, k: int) -> int:
    res = 1
    for i in range(1, min(k, n - k) + 1):
        res = res * (n - i + 1) // i
        if res >= self.MAX:
            return self.MAX
    return res

```

2478. 352.py

Path: LeetCode-main/solutions/352. Data Stream as Disjoint Intervals/352.py

```

from sortedcontainers import SortedDict


class SummaryRanges:
    def __init__(self):
        self.intervals = SortedDict() # {start: (start, end)}

    def addNum(self, val: int) -> None:
        if val in self.intervals:
            return

        lo = self._lowerKey(val)
        hi = self._higherKey(val)

        # {lo, map[lo][1]} + val + {hi, map[hi][1]} = {lo, map[hi][1]}
        if lo >= 0 and hi >= 0 and self.intervals[lo][1] + 1 == val and val + 1 == hi:
            self.intervals[lo][1] = self.intervals[hi][1]
            del self.intervals[hi]
        # {lo, map[lo][1]} + val = {lo, val}
        # Prevent adding duplicate entry by using '>=' instead of '=='.
        elif lo >= 0 and self.intervals[lo][1] + 1 >= val:
            self.intervals[lo][1] = max(self.intervals[lo][1], val)
        elif hi >= 0 and val + 1 == hi:
            # val + {hi, map[hi][1]} = {val, map[hi][1]}
            self.intervals[val] = [val, self.intervals[hi][1]]
            del self.intervals[hi]
        else:
            self.intervals[val] = [val, val]

    def getIntervals(self) -> list[list[int]]:
        return list(self.intervals.values())

    def _lowerKey(self, key: int):
        """Returns the maximum key in `self.intervals` < `key`."""
        i = self.intervals.bisect_left(key)
        if i == 0:
            return -1
        return self.intervals.peekitem(i - 1)[0]

    def _higherKey(self, key: int):
        """Returns the minimum key in `self.intervals` < `key`."""
        i = self.intervals.bisect_right(key)
        if i == len(self.intervals):
            return -1
        return self.intervals.peekitem(i)[0]

```

2479. 3522.py

Path: LeetCode-main/solutions/3522. Calculate Score After Performing Instructions/3522.py

```

class Solution:
    def calculateScore(self, instructions: list[str], values: list[int]) -> int:
        n = len(instructions)
        ans = 0
        i = 0
        seen = set()

        while 0 <= i < n and i not in seen:
            seen.add(i)
            if instructions[i] == 'add':
                ans += values[i]
                i += 1
            elif instructions[i] == 'jump':
                i += values[i]

        return ans

```

2480. 3523.py

Path: LeetCode-main/solutions/3523. Make Array Non-decreasing/3523.py

```

class Solution:
    def maximumPossibleSize(self, nums: list[int]) -> int:
        ans = 0
        prev = 0

        for num in nums:
            if num >= prev:
                prev = num
                ans += 1

        return ans

```

2481. 3524.py

Path: LeetCode-main/solutions/3524. Find X Value of Array I/3524.py

```

class Solution:
    def resultArray(self, nums: list[int], k: int) -> list[int]:
        ans = [0] * k
        # dp[r] := the number of subarrays ending at current position with
        # product % k == r
        dp = [0] * k

        for num in nums:
            newDp = [0] * k
            numMod = num % k
            # Start new subarray with only `num`
            newDp[numMod] = 1
            # Extend all previous subarrays
            for i in range(k):
                newMod = (i * numMod) % k
                newDp[newMod] += dp[i]
            # Accumulate counts into ans
            for i in range(k):
                ans[i] += newDp[i]
            dp = newDp

        return ans

```

2482. 3527.py

Path: LeetCode-main/solutions/3527. Find the Most Common Response/3527.py

```
class Solution:
    def findCommonResponse(self, responses: list[list[str]]) -> str:
        count = collections.Counter()

        for response in responses:
            for response in set(response):
                count[response] += 1

        maxFreq = max(count.values())
        return min([response
                    for response, count in count.items()
                    if count == maxFreq])
```

2483. 3528.py

Path: LeetCode-main/solutions/3528. Unit Conversion I/3528.py

```
class Solution:
    def baseUnitConversions(self, conversions: list[list[int]]) -> list[int]:
        MOD = 1_000_000_007
        n = len(conversions) + 1
        ans = [0] * n
        ans[0] = 1
        q = collections.deque([0])
        graph = [[] for _ in range(n)]

        for u, v, factor in conversions:
            graph[u].append((v, factor))

        while q:
            u = q.popleft()
            for v, factor in graph[u]:
                ans[v] = (ans[u] * factor) % MOD
                q.append(v)

        return ans
```

2484. 3529.py

Path: LeetCode-main/solutions/3529. Count Cells in Overlapping Horizontal and Vertical Substrings/3529.py

```

class Solution:
    def countCells(self, grid: list[list[str]], pattern: str) -> int:
        BASE = 13
        HASH = 1_000_000_007
        m = len(grid)
        n = len(grid[0])

        def markMatchedCells(flattenedGrid: str, isHorizontal: bool) -> list[list[bool]]:
            matchMatrix = [[False] * n for _ in range(m)]
            matchPrefix = [0] * (len(flattenedGrid) + 1)
            pows = [1] # pows[i] := BASE^i % HASH
            patternHash = 0
            runningHash = 0

            for i in range(1, len(pattern)):
                pows.append((pows[-1] * BASE) % HASH)

            for c in pattern:
                patternHash = (patternHash * BASE + (ord(c) - ord('a')) % HASH)

            for i in range(len(flattenedGrid)):
                runningHash = (
                    runningHash * BASE + (ord(flattenedGrid[i]) - ord('a')) % HASH
                )
                if i >= len(pattern) - 1:
                    if runningHash == patternHash: # Match found.
                        matchPrefix[i - len(pattern) + 1] += 1
                        matchPrefix[i + 1] -= 1
                    # Remove the contribution of the oldest letter.
                    oldestLetterHash = (
                        pows[len(pattern) - 1] *
                        (ord(flattenedGrid[i - len(pattern) + 1]) - ord('a')) % HASH
                    )
                    runningHash = (runningHash - oldestLetterHash + HASH) % HASH

            for k in range(len(flattenedGrid)):
                if k > 0:
                    matchPrefix[k] += matchPrefix[k - 1]
                if matchPrefix[k] > 0:
                    i = k // n if isHorizontal else k % m
                    j = k % n if isHorizontal else k // m
                    matchMatrix[i][j] = True

            return matchMatrix

        # Find matching positions.
        flattenedGridRow = ''.join(cell for row in grid for cell in row)
        flattenedGridCol = ''.join(cell for col in zip(*grid) for cell in col)
        horizontalMatches = markMatchedCells(flattenedGridRow, True)
        verticalMatches = markMatchedCells(flattenedGridCol, False)
        return sum(horizontalMatches[i][j] and verticalMatches[i][j]
                    for i in range(m)
                    for j in range(n))

```

2485. 353.py

Path: LeetCode-main/solutions/353. Design Snake Game/353.py

```

class SnakeGame:
    def __init__(self, width: int, height: int, food: list[list[int]]):
        """
        Initialize your data structure here.
        @param width - screen width
        @param height - screen height
        @param food - list of food positions

```

```

@param food - A list of food positions
E.g food = [[1,1], [1,0]] means the first food is positioned at [1,1], the second is at [1,0].
"""

self.width = width
self.height = height
self.food = food
self.score = 0
self.k = 0 # food's index
self.lookup = set([self.getId(0, 0)])
self.body = collections.deque([self.getId(0, 0)]) # snake's body

def move(self, direction: str) -> int:
    """
    Moves the snake.
    @param direction - 'U' = Up, 'L' = Left, 'R' = Right, 'D' = Down
    @return The game's score after the move. Return -1 if game over.
    Game over when snake crosses the screen boundary or bites its body.
    """

    # the old head's position
    i = self.body[0] // self.width
    j = self.body[0] % self.width

    # Update the head's position and check if it's out-of-bounds.
    if direction == "U":
        i -= 1
        if i < 0:
            return -1
    if direction == "L":
        j -= 1
        if j < 0:
            return -1
    if direction == "R":
        j += 1
        if j == self.width:
            return -1
    if direction == "D":
        i += 1
        if i == self.height:
            return -1

    newHead = self.getId(i, j)

    # 1. Eat food and increase the size by 1.
    if self.k < len(self.food) and i == self.food[self.k][0] and j == self.food[self.k][1]:
        self.lookup.add(newHead)
        self.body.appendleft(newHead)
        self.k += 1
        self.score += 1
        return self.score

    # 2. new head != old tail and eat body!
    if newHead != self.body[-1] and newHead in self.lookup:
        return -1

    # 3. normal case
    # Remove the old tail first, then add new head because new head may be in
    # old tail's position.
    self.lookup.remove(self.body[-1])
    self.lookup.add(newHead)
    self.body.pop()
    self.body.appendleft(newHead)

    return self.score

def getId(self, i: int, j: int) -> int:
    return i * self.width + j

```

2486. 3530.py

Path: LeetCode-main/solutions/3530. Maximum Profit from Valid Topological Order in DAG/3530.py

```
class Solution:
    def maxProfit(self, n: int, edges: list[list[int]], score: list[int]) -> int:
        # need[i] := the bitmask representing all nodes that must be placed before
        # node i
        need = [0] * n
        # dp[mask] := the maximum profit achievable by placing the set of nodes
        # represented by `mask`
        dp = [-1] * (1 << n)
        dp[0] = 0

        for u, v in edges:
            need[v] |= 1 << u

        # Iterate over all subsets of nodes (represented by bitmask `mask`)
        for mask in range(1 << n):
            if dp[mask] == -1:
                continue
            # Determine the position of the next node to be placed (1-based).
            pos = mask.bit_count() + 1
            # Try to place each node `i` that hasn't been placed yet.
            for i in range(n):
                if mask >> i & 1:
                    continue
                # Check if all dependencies of node `i` are already placed.
                if (mask & need[i]) == need[i]:
                    newMask = mask | 1 << i # Mark node `i` as placed.
                    dp[newMask] = max(dp[newMask], dp[mask] + score[i] * pos)

        return dp[-1]
```

2487. 3531.py

Path: LeetCode-main/solutions/3531. Count Covered Buildings/3531.py

```
class Solution:
    def countCoveredBuildings(self, n: int, buildings: list[list[int]]) -> int:
        northernmost = [math.inf] * (n + 1)
        southernmost = [0] * (n + 1)
        westernmost = [math.inf] * (n + 1)
        easternmost = [0] * (n + 1)

        for x, y in buildings:
            northernmost[x] = min(northernmost[x], y)
            southernmost[x] = max(southernmost[x], y)
            westernmost[y] = min(westernmost[y], x)
            easternmost[y] = max(easternmost[y], x)

        return sum(northernmost[x] < y < southernmost[x]
                    and westernmost[y] < x < easternmost[y]
                    for x, y in buildings)
```

2488. 3532.py

Path: LeetCode-main/solutions/3532. Path Existence Queries in a Graph I/3532.py

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def pathExistenceQueries(
        self,
        n: int,
        nums: list[int],
        maxDiff: int,
        queries: list[list[int]]
    ) -> list[bool]:
        uf = UnionFind(n)

        for i in range(1, n):
            if abs(nums[i] - nums[i - 1]) <= maxDiff:
                uf.unionByRank(i, i - 1)

        return [uf.find(u) == uf.find(v)
                for u, v in queries]
```

2489. 3533.py

Path: LeetCode-main/solutions/3533. Concatenated Divisibility/3533.py

```

class Solution:
    def concatenatedDivisibility(self, nums: list[int], k: int) -> list[int]:
        n = len(nums)
        nums.sort()
        lengths = [len(str(num)) for num in nums]
        pows = [pow(10, length, k) for length in lengths]

        @functools.lru_cache(None)
        def dp(mask: int, mod: int) -> bool:
            """
            Returns True if there is a way to form a number divisible by `k` using the
            numbers in `nums`, where nums[i] is used iff `mask & (1 << i)`.
            """
            if mask == (1 << n) - 1:
                return mod == 0
            for i in range(n):
                if (mask >> i & 1) == 0:
                    newMod = (mod * pows[i] + nums[i]) % k
                    if dp(mask | 1 << i, newMod):
                        return True
            return False

        def reconstruct(mask: int, mod: int) -> list[int]:
            """
            Reconstructs the numbers that form a number divisible by `k` using the
            numbers in `nums`, where nums[i] is used iff `mask & (1 << i)`.
            """
            for i in range(n):
                if (mask >> i & 1) == 0:
                    newMod = (mod * pows[i] + nums[i]) % k
                    if dp(mask | 1 << i, newMod):
                        return [nums[i]] + reconstruct(mask | 1 << i, newMod)
            return []

        return reconstruct(0, 0) if dp(0, 0) else []

```

2490. 3534.py

Path: LeetCode-main/solutions/3534. Path Existence Queries in a Graph II/3534.py

```

class Solution:
    def pathExistenceQueries(
        self,
        n: int,
        nums: list[int],
        maxDiff: int,
        queries: list[list[int]],
    ) -> list[int]:
        sortedNumAndIndexes = sorted((num, i) for i, num in enumerate(nums))
        sortedNums = [num for num, _ in sortedNumAndIndexes]
        indexMap = {originalIndex: sortedIndex for sortedIndex,
                     (_, originalIndex) in enumerate(sortedNumAndIndexes)}
        maxLevel = n.bit_length() + 1
        # jump[i][j] is the index of the j-th ancestor of i
        jump = [[0] * maxLevel for _ in range(n)]

        right = 0
        for i in range(n):
            while right + 1 < n and sortedNums[right + 1] - sortedNums[i] <= maxDiff:
                right += 1
            jump[i][0] = right

        for level in range(1, maxLevel):
            for i in range(n):
                prevJump = jump[i][level - 1]
                jump[i][level] = jump[prevJump][level - 1]

        def minJumps(start: int, end: int, level: int) -> int:
            """
            Returns the minimum number of jumps from `start` to `end` using binary
            lifting.
            """
            if start == end:
                return 0
            if jump[start][0] >= end:
                return 1
            if jump[start][level] < end:
                return math.inf
            for j in range(level, -1, -1):
                if jump[start][j] < end:
                    break
            return (1 << j) + minJumps(jump[start][j], end, j)

        def minDist(u: int, v: int) -> int:
            uIndex = indexMap[u]
            vIndex = indexMap[v]
            start = min(uIndex, vIndex)
            end = max(uIndex, vIndex)
            res = minJumps(start, end, maxLevel - 1)
            return res if res < math.inf else -1

        return [minDist(u, v) for u, v in queries]

```

2491. 3535.py

Path: LeetCode-main/solutions/3535. Unit Conversion II/3535.py

```

class Solution:
    def queryConversions(
        self,
        conversions: list[list[int]],
        queries: list[list[int]]
    ) -> list[int]:
        self.MOD = 1_000_000_007
        units = self._baseUnitConversions(conversions)
        # By Fermat's little theorem.
        return [units[v] * self._modPow(units[u], self.MOD - 2) % self.MOD
                for u, v in queries]

# Same as 3528. Unit Conversion I
def _baseUnitConversions(self, conversions: list[list[int]]) -> list[int]:
    n = len(conversions) + 1
    res = [0] * n
    res[0] = 1
    q = collections.deque([0])
    graph = [[] for _ in range(n)]

    for u, v, factor in conversions:
        graph[u].append((v, factor))

    while q:
        u = q.popleft()
        for v, factor in graph[u]:
            res[v] = (res[u] * factor) % self.MOD
            q.append(v)

    return res

def _modPow(self, x: int, n: int) -> int:
    if n == 0:
        return 1
    if n % 2 == 1:
        return x * self._modPow(x, n - 1) % self.MOD
    return self._modPow(x * x % self.MOD, n // 2)

```

2492. 3536.py

Path: LeetCode-main/solutions/3536. Maximum Product of Two Digits/3536.py

```

class Solution:
    def maxProduct(self, n: int) -> int:
        s = sorted(str(n))
        return int(s[-1]) * int(s[-2])

```

2493. 3537.py

Path: LeetCode-main/solutions/3537. Fill a Special Grid/3537.py

```

class Solution:
    def specialGrid(self, n: int) -> list[list[int]]:
        sz = 1 << n
        grid = [[0] * sz for _ in range(sz)]
        count = 0

        def fill(x1: int, x2: int, y1: int, y2: int) -> None:
            nonlocal count
            if x2 - x1 == 1:
                grid[x1][y1] = count
                count += 1
            return
        midRow = (x1 + x2) // 2
        midCol = (y1 + y2) // 2
        fill(x1, midRow, midCol, y2)
        fill(midRow, x2, midCol, y2)
        fill(midRow, x2, y1, midCol)
        fill(x1, midRow, y1, midCol)

        fill(0, sz, 0, sz)
        return grid

```

2494. 3538.py

Path: LeetCode-main/solutions/3538. Merge Operations for Minimum Travel Time/3538.py

```

class Solution:
    def minTravelTime(
        self,
        l: int,
        n: int,
        k: int,
        position: list[int],
        time: list[int]
    ) -> int:
        prefix = list(itertools.accumulate(time))

        @functools.lru_cache(None)
        def dp(i: int, skips: int, last: int) -> int:
            """
            Returns the minimum travel time to reach the last stop from i-th stop,
            with `skips` skips remaining, and the last stop being `last`.
            """
            if i == n - 1:
                return 0 if skips == 0 else math.inf
            res = math.inf
            rate = prefix[i] - (prefix[last - 1] if last > 0 else 0)
            end = min(n - 1, i + skips + 1)
            for j in range(i + 1, end + 1):
                distance = position[j] - position[i]
                res = min(res, distance * rate + dp(j, skips - (j - i - 1), i + 1))
            return res

        return dp(0, k, 0)

```

2495. 3539.py

Path: LeetCode-main/solutions/3539. Find Sum of Array Product of Magical Sequences/3539.py

```

class Solution:
    def magicalSum(self, m: int, k: int, nums: list[int]) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(m: int, k: int, i: int, carry: int) -> int:
            """
            Returns the number of magical sequences of length `k` that can be formed
            from the first `i` numbers in `nums` with at most `m` elements.
            """
            if m < 0 or k < 0 or (m + carry.bit_count() < k):
                return 0
            if m == 0:
                return int(k == carry.bit_count())
            if i == len(nums):
                return 0
            res = 0
            for count in range(m + 1):
                contribution = math.comb(m, count) * pow(nums[i], count, MOD) % MOD
                newCarry = carry + count
                res += dp(m - count, k - (newCarry % 2),
                        i + 1, newCarry // 2) * contribution
            res %= MOD
            return res

        return dp(m, k, 0, 0)

```

2496. 354.py

Path: LeetCode-main/solutions/354. Russian Doll Envelopes/354.py

```

class Solution:
    def maxEnvelopes(self, envelopes: list[list[int]]) -> int:
        envelopes.sort(key=lambda x: (x[0], -x[1]))
        return self._lengthOfLIS(envelopes)

    def _lengthOfLIS(self, envelopes: list[list[int]]) -> int:
        # tails[i] := the minimum tails of all the increasing subsequences having
        # length i + 1
        tails = []

        for _, h in envelopes:
            if not tails or h > tails[-1]:
                tails.append(h)
            else:
                tails[bisect.bisect_left(tails, h)] = h

        return len(tails)

```

2497. 3540.py

Path: LeetCode-main/solutions/3540. Minimum Time to Visit All Houses/3540.py

```

class Solution:
    def minTotalTime(
        self,
        forward: list[int],
        backward: list[int],
        queries: list[int]
    ) -> int:
        summ = sum(backward)
        ans = 0
        pos = 0
        prefixF = [0] + list(itertools.accumulate(forward))
        prefixB = list(itertools.accumulate(backward)) + [0]

        for q in queries:
            r = int(q < pos) * prefixF[-1] + prefixF[q] - prefixF[pos]
            l = int(q > pos) * summ + prefixB[pos] - prefixB[q]
            ans += min(l, r)
            pos = q

        return ans

```

2498. 3541.py

Path: LeetCode-main/solutions/3541. Find Most Frequent Vowel and Consonant/3541.py

```

class Solution:
    def maxFreqSum(self, s: str) -> int:
        VOWELS = 'aeiou'
        count = collections.Counter(s)
        maxVowel = max((count[c] for c in VOWELS if c in count), default=0)
        maxConsonant = max((count[c] for c in count if c not in VOWELS), default=0)
        return maxVowel + maxConsonant

```

2499. 3542.py

Path: LeetCode-main/solutions/3542. Minimum Operations to Convert All Elements to Zero/3542.py

```

class Solution:
    def minOperations(self, nums: list[int]) -> int:
        ans = 0
        stack = [0]

        for num in nums:
            while stack and stack[-1] > num:
                stack.pop()
            if not stack or stack[-1] < num:
                ans += 1
                stack.append(num)

        return ans

```

2500. 3543.py

Path: LeetCode-main/solutions/3543. Maximum Weighted K-Edge Path/3543.py

```
class Solution:
    def maxWeight(self, n: int, edges: list[list[int]], k: int, t: int) -> int:
        graph = [[] for _ in range(n)]
        # dp[u][i] := the set of possible path sums ending at node u with i edges
        dp = [defaultdict(set) for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))

        for u in range(n):
            dp[u][0].add(0) # zero edges = sum 0

        for i in range(k):
            for u in range(n):
                for currSum in dp[u][i]:
                    for v, w in graph[u]:
                        newSum = currSum + w
                        if newSum < t:
                            dp[v][i + 1].add(newSum)

        ans = -1

        for u in range(n):
            if k in dp[u]:
                ans = max(ans, max(dp[u][k]))

        return ans
```

2501. 3544.py

Path: LeetCode-main/solutions/3544. Subtree Inversion Sum/3544.py


```

class Solution:
    def subtreeInversionSum(
        self,
        edges: list[list[int]],
        nums: list[int],
        k: int
    ) -> int:
        n = len(edges) + 1
        parent = [-1] * n
        graph = [[] for _ in range(n)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        @functools.lru_cache(None)
        def dp(u: int, stepsSinceInversion: int, inverted: bool) -> int:
            """
            Returns the maximum sum for subtree rooted at u, with
            `stepsSinceInversion` steps of inversion and `inverted` is true if the
            subtree is inverted.
            """
            num = -nums[u] if inverted else nums[u]
            negNum = -num
            for v in graph[u]:
                if v == parent[u]:
                    continue
                parent[v] = u
                num += dp(v, min(k, stepsSinceInversion + 1), inverted)
                if stepsSinceInversion == k:
                    negNum += dp(v, 1, not inverted)
            return max(num, negNum) if stepsSinceInversion == k else num

        return dp(0, k, False)

```

2502. 3545.py

Path: LeetCode-main/solutions/3545. Minimum Deletions for At Most K Distinct Characters/3545.py

```

class Solution:
    def minDeletion(self, s: str, k: int) -> int:
        count = collections.Counter(s)
        if len(count) <= k:
            return 0
        freqs = sorted(count.values())
        return sum(freqs[:len(freqs) - k])

```

2503. 3546.py

Path: LeetCode-main/solutions/3546. Equal Sum Grid Partition I/3546.py

```
class Solution:
    def canPartitionGrid(self, grid: list[list[int]]) -> bool:
        totalSum = sum(map(sum, grid))

        def canPartition(grid: list[list[int]]) -> bool:
            runningSum = 0
            for row in grid:
                runningSum += sum(row)
                if runningSum * 2 == totalSum:
                    return True
            return False

        return canPartition(grid) or canPartition(zip(*grid))
```

2504. 3547.py

Path: LeetCode-main/solutions/3547. Maximum Sum of Edge Values in a Graph/3547.py

```

class Solution:
    def maxScore(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        cycleSizes = [] # components where all nodes have degree 2
        pathSizes = [] # components that are not cycleSizes
        seen = set()

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        for i in range(n):
            if i in seen:
                continue
            component = self._getComponent(graph, i, seen)
            if all(len(graph[u]) == 2 for u in component):
                cycleSizes.append(len(component))
            elif len(component) > 1:
                pathSizes.append(len(component))

        for cycleSize in cycleSizes:
            ans += self._calculateScore(n - cycleSize + 1, n, True)
            n -= cycleSize

        for pathSize in sorted(pathSizes, reverse=True):
            ans += self._calculateScore(n - pathSize + 1, n, False)
            n -= pathSize

        return ans

    def _getComponent(
        self,
        graph: list[list[int]],
        start: int,
        seen: set[int],
    ) -> list[int]:
        component = [start]
        seen.add(start)
        for u in component:
            for v in graph[u]:
                if v in seen:
                    continue
                component.append(v)
                seen.add(v)
        return component

    def _calculateScore(self, left: int, right: int, isCycle: bool) -> int:
        window = collections.deque([right, right])
        score = 0
        for value in range(right - 1, left - 1, -1):
            windowValue = window.popleft()
            score += windowValue * value
            window.append(value)
        return score + window[0] * window[1] * isCycle

```

2505. 3548.py

Path: LeetCode-main/solutions/3548. Equal Sum Grid Partition II/3548.py

```

class Solution:
    def canPartitionGrid(self, grid: list[list[int]]) -> bool:
        summ = sum(map(sum, grid))

        def canPartition(grid: list[list[int]]) -> bool:
            topSum = 0
            seen = set()
            for i, row in enumerate(grid):
                topSum += sum(row)
                botSum = summ - topSum
                seen |= set(row)
                if topSum - botSum in (0, grid[0][0], grid[0][-1], row[0]):
                    return True
                if len(grid[0]) > 1 and i > 0 and topSum - botSum in seen:
                    return True
            return False

        return (canPartition(grid) or
                canPartition(grid[::-1]) or
                canPartition(list(zip(*grid))[:,::-1]) or
                canPartition(list(zip(*grid))))

```

2506. 3549.py

Path: LeetCode-main/solutions/3549. Multiply Two Polynomials/3549.py

```

from numpy import array, round
from numpy.fft import fft, ifft

class Solution:
    def multiply(self, poly1: list[int], poly2: list[int]) -> list[int]:
        n1 = len(poly1)
        n2 = len(poly2)
        n = n1 + n2 - 1
        sz = 1 << (n - 1).bit_length()
        arr1 = array(poly1 + [0] * (sz - n1))
        arr2 = array(poly2 + [0] * (sz - n2))
        ans = ifft(fft(arr1) * fft(arr2))
        return round(ans).astype(int).tolist()[:n]

```

2507. 355.py

Path: LeetCode-main/solutions/355. Design Twitter/355.py

```

class Twitter:
    def __init__(self):
        self.timer = itertools.count(step=-1)
        self.tweets = collections.defaultdict(deque)
        self.followees = collections.defaultdict(set)

    def postTweet(self, userId: int, tweetId: int) -> None:
        self.tweets[userId].appendleft((next(self.timer), tweetId))
        if len(self.tweets[userId]) > 10:
            self.tweets[userId].pop()

    def getNewsFeed(self, userId: int) -> list[int]:
        tweets = list(
            heapq.merge(
                *
                (self.tweets[followee]
                 for followee in self.followees[userId] | {userId}))
        )
        return [tweetId for _, tweetId in tweets[:10]]

    def follow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].discard(followeeId)

```

2508. 3550.py

Path: LeetCode-main/solutions/3550. Smallest Index With Digit Sum Equal to Index/3550.py

```

class Solution:
    def smallestIndex(self, nums: List[int]) -> int:
        return next((i
                     for i, num in enumerate(nums)
                     if self._getDigitSum(num) == i), -1)

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))

```

2509. 3551.py

Path: LeetCode-main/solutions/3551. Minimum Swaps to Sort by Digit Sum/3551.py

```
class Solution:
    def minSwaps(self, nums: list[int]) -> int:
        ans = 0
        seen = set()
        sortedNums = sorted(nums, key=lambda x: (self._getDigitSum(x), x))
        numToIndex = {num: i for i, num in enumerate(sortedNums)}

        for i, num in enumerate(nums):
            if i in seen or numToIndex[num] == i:
                continue
            cycleSize = 0
            j = i
            while j not in seen:
                seen.add(j)
                j = numToIndex[nums[j]]
                cycleSize += 1
            ans += max(cycleSize - 1, 0)
        return ans

    def _getDigitSum(self, num: int) -> int:
        return sum(int(digit) for digit in str(num))
```

2510. 3552.py

Path: LeetCode-main/solutions/3552. Grid Teleportation Traversal/3552.py

```

class Solution:
    # Similar to 3341. Find Minimum Time to Reach Last Room I
    def minMoves(self, matrix: list[str]) -> int:
        if matrix[-1][-1] == '#':
            return -1

        teleportPositions = [[] for _ in range(26)]

        for i, row in enumerate(matrix):
            for j, c in enumerate(row):
                if c not in ('.', '#'):
                    teleportPositions[ord(c) - ord('A')].append((i, j))

        return self._dijkstra(matrix, teleportPositions,
                               (0, 0), (len(matrix) - 1, len(matrix[0]) - 1))

    def _dijkstra(
        self,
        matrix: list[str],
        teleportPositions: list[list[tuple[int, int]]],
        src: tuple[int, int],
        dst: tuple[int, int],
    ) -> int:
        DIRS = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        m = len(matrix)
        n = len(matrix[0])
        dist = [[math.inf] * n for _ in range(m)]
        seen = set()

        dist[0][0] = 0
        minHeap = [(dist[0][0], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if u == dst:
                return d
            i, j = u
            if d > dist[i][j]:
                continue
            c = matrix[i][j]
            if c.isupper() and c not in seen:
                seen.add(c)
                for x, y in teleportPositions[ord(c) - ord('A')]:
                    if d < dist[x][y]:
                        dist[x][y] = d
                        heapq.heappush(minHeap, (d, (x, y)))
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if matrix[x][y] == '#':
                    continue
                if d + 1 < dist[x][y]:
                    dist[x][y] = d + 1
                    heapq.heappush(minHeap, (d + 1, (x, y)))

        return -1

```

2511. 3553.py

Path: LeetCode-main/solutions/3553. Minimum Weighted Subgraph With the Required Paths II/3553.py

```

class Solution:
    # Similar to 2846. Minimum Edge Weight Equilibrium Queries in a Tree
    def minimumWeight(
        self,
        edges: list[list[int]],
        queries: list[list[int]]
    ) -> list[int]:
        n = len(edges) + 1
        m = math.ceil(math.log2(n))
        graph = [[] for _ in range(n)]
        jump = [[0] * m for _ in range(n)] # jump[i][j] := the 2^j-th ancestor of i
        depth = [0] * n # depth[i] := the depth of i
        dist = [0] * n # dist[i] := the distance from root to i

        for u, v, w in edges:
            graph[u].append((v, w))
            graph[v].append((u, w))

        self._dfs(graph, 0, -1, jump, depth, dist)

        for j in range(1, m):
            for i in range(n):
                jump[i][j] = jump[jump[i][j - 1]][j - 1]

        return [(self._distance(src1, src2, jump, depth, dist) +
                 self._distance(src1, dest, jump, depth, dist) +
                 self._distance(src2, dest, jump, depth, dist)) // 2
                for src1, src2, dest in queries]

    def _dfs(
        self,
        graph: list[list[tuple[int, int]]],
        u: int,
        prev: int,
        jump: list[list[int]],
        depth: list[int],
        dist: list[int]
    ) -> None:
        for v, w in graph[u]:
            if v == prev:
                continue
            jump[v][0] = u
            depth[v] = depth[u] + 1
            dist[v] = dist[u] + w
            self._dfs(graph, v, u, jump, depth, dist)

    def _getLCA(
        self,
        u: int,
        v: int,
        jump: list[list[int]],
        depth: list[int]
    ) -> int:
        """Returns the lca(u, v) by binary jump."""
        # v is always deeper than u.
        if depth[u] > depth[v]:
            return self._getLCA(v, u, jump, depth)
        # Jump v to the same height of u.
        for j in range(len(jump[0])):
            if depth[v] - depth[u] >> j & 1:
                v = jump[v][j]
        if u == v:
            return u
        # Jump u and v to the node right below the lca.
        for j in range(len(jump[0]) - 1, -1, -1):

```



```

        if jump[u][j] != jump[v][j]:
            u = jump[u][j]
            v = jump[v][j]
        return jump[u][0]

def _distance(
    self,
    u: int,
    v: int,
    jump: list[list[int]],
    depth: list[int],
    dist: list[int]
) -> int:
    """Returns the distance between u and v."""
    lca = self._getLCA(u, v, jump, depth)
    return dist[u] + dist[v] - 2 * dist[lca]

```

2512. 3555.py

Path: LeetCode-main/solutions/3555. Smallest Subarray to Sort in Every Sliding Window/3555.py

```

class Solution:
    def minSubarraySort(self, nums: list[int], k):
        ans = []

        for i in range(len(nums) - k + 1):
            window = nums[i:i+k]
            sortedWindow = sorted(window)
            l = 0
            r = k - 1
            while l < k and window[l] == sortedWindow[l]:
                l += 1
            while r >= 0 and window[r] == sortedWindow[r]:
                r -= 1
            ans.append(0 if l > r else r - l + 1)

        return ans

```

2513. 3556.py

Path: LeetCode-main/solutions/3556. Sum of Largest Prime Substrings/3556.py

```

class Solution:
    def sumOfLargestPrimes(self, s: str) -> int:
        primes = set()
        n = len(s)

        for i in range(n):
            for j in range(i + 1, n + 1):
                num = int(s[i:j])
                if num not in primes and self._isPrime(num):
                    primes.add(num)

        top3 = sorted(primes, reverse=True)[:3]
        return sum(top3)

    def _isPrime(self, n: int) -> bool:
        return n > 1 and all(n % i != 0 for i in range(2, math.isqrt(n) + 1))

```

2514. 3557.py

Path: LeetCode-main/solutions/3557. Find Maximum Number of Non Intersecting Substrings/3557.py

```
class Solution:
    def maxSubstrings(self, word: str) -> int:
        ans = 0
        firstSeen = {}

        for i, c in enumerate(word):
            if c not in firstSeen:
                firstSeen[c] = i
            elif i - firstSeen[c] + 1 >= 4:
                ans += 1
                firstSeen.clear()

        return ans
```

2515. 3558.py

Path: LeetCode-main/solutions/3558. Number of Ways to Assign Edge Weights I/3558.py

```
class Solution:
    def assignEdgeWeights(self, edges: list[list[int]]) -> int:
        MOD = 1_000_000_007
        n = len(edges) + 1
        graph = [[] for _ in range(n + 1)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        q = collections.deque([1])
        seen = {1}

        step = 0
        while q:
            for _ in range(len(q)):
                u = q.popleft()
                for v in graph[u]:
                    if v not in seen:
                        q.append(v)
                        seen.add(v)
            step += 1

        return pow(2, step - 2, MOD) if step > 0 else 0
```

2516. 3559.py

Path: LeetCode-main/solutions/3559. Number of Ways to Assign Edge Weights II/3559.py

```

class Solution:
    def assignEdgeWeights(
        self,
        edges: list[list[int]],
        queries: list[list[int]]
    ) -> list[int]:
        MOD = 1_000_000_007
        LOG = 17 # since 2^17 > 1e5
        n = len(edges) + 1
        ans = []
        depth = [0] * (n + 1)
        graph = [[] for _ in range(n + 1)]
        parent = [[-1] * (n + 1) for _ in range(LOG)]

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def dfs(u: int, p: int) -> None:
            parent[0][u] = p
            for v in graph[u]:
                if v != p:
                    depth[v] = depth[u] + 1
                    dfs(v, u)

        dfs(1, -1)

        for k in range(1, LOG):
            for v in range(1, n + 1):
                if parent[k - 1][v] != -1:
                    parent[k][v] = parent[k - 1][parent[k - 1][v]]

        def lca(u: int, v: int) -> int:
            if depth[u] < depth[v]:
                u, v = v, u

            for k in reversed(range(LOG)):
                if parent[k][u] != -1 and depth[parent[k][u]] >= depth[v]:
                    u = parent[k][u]

            if u == v:
                return u

            for k in reversed(range(LOG)):
                if parent[k][u] != -1 and parent[k][u] != parent[k][v]:
                    u = parent[k][u]
                    v = parent[k][v]

            return parent[0][u]

        for u, v in queries:
            if u == v:
                ans.append(0)
            else:
                a = lca(u, v)
                d = depth[u] + depth[v] - 2 * depth[a]
                ans.append(pow(2, d - 1, MOD))

        return ans

```

Path: LeetCode-main/solutions/356. Line Reflection/356.py

```
class Solution:
    def isReflected(self, points: list[list[int]]) -> bool:
        minX = math.inf
        maxX = -math.inf
        seen = set()

        for x, y in points:
            minX = min(minX, x)
            maxX = max(maxX, x)
            seen.add((x, y))

        summ = minX + maxX
        # (leftX + rightX) / 2 = (minX + maxX) / 2
        # leftX = minX + maxX - rightX
        # rightX = minX + maxX - leftX

        return all((summ - x, y) in seen for x, y in points)
```

2518. 3560.py

Path: LeetCode-main/solutions/3560. Find Minimum Log Transportation Cost/3560.py

```
class Solution:
    def minCuttingCost(self, n: int, m: int, k: int) -> int:
        return max(0, max(n, m) - k) * k
```

2519. 3561.py

Path: LeetCode-main/solutions/3561. Resulting String After Adjacent Removals/3561.py

```
class Solution:
    def resultingString(self, s: str) -> str:
        stack = []

        for c in s:
            if stack and abs(ord(stack[-1]) - ord(c)) in (1, 25):
                stack.pop()
            else:
                stack.append(c)

        return ''.join(stack)
```

2520. 3562.py

Path: LeetCode-main/solutions/3562. Maximum Profit from Trading Stocks with Discounts/3562.py

```

class Solution:
    def maxProfit(
        self,
        n: int,
        present: list[int],
        future: list[int],
        hierarchy: list[list[int]],
        budget: int,
    ) -> int:
        tree = [[] for _ in range(n)]

        for u, v in hierarchy:
            tree[u - 1].append(v - 1)

        @functools.lru_cache(None)
        def dp(u: int, parent: int) -> tuple[list[int], list[int]]:
            noDiscount = [0] * (budget + 1)
            withDiscount = [0] * (budget + 1)

            for v in tree[u]:
                if v == parent:
                    continue
                childNoDiscount, childWithDiscount = dp(v, u)
                noDiscount = self._merge(noDiscount, childNoDiscount)
                withDiscount = self._merge(withDiscount, childWithDiscount)

            newDp0 = noDiscount[:]
            newDp1 = noDiscount[:]

            # 1. Buy current node at full cost (no discount)
            fullCost = present[u]
            for b in range(fullCost, budget + 1):
                profit = future[u] - fullCost
                newDp0[b] = max(newDp0[b], withDiscount[b - fullCost] + profit)

            # 2. Buy current node at half cost (discounted by parent)
            halfCost = present[u] // 2
            for b in range(halfCost, budget + 1):
                profit = future[u] - halfCost
                newDp1[b] = max(newDp1[b], withDiscount[b - halfCost] + profit)

            return newDp0, newDp1

        return max(dp(0, -1)[0])

    def _merge(self, dpA: list[int], dpB: list[int]) -> list[int]:
        merged = [-math.inf] * len(dpA)
        for i, a in enumerate(dpA):
            for j in range(len(dpA) - i):
                merged[i + j] = max(merged[i + j], a + dpB[j])
        return merged

```

2521. 3563.py

Path: LeetCode-main/solutions/3563. Lexicographically Smallest String After Adjacent Removals/3563.py

```

class Solution:
    def lexicographicallySmallestString(self, s: str) -> str:
        n = len(s)
        # dp[i][j]: the lexicographically smallest string by removing adjacent
        # letters from s[i..j)
        dp = [[''] * (n + 1) for _ in range(n + 1)]

        for d in range(1, n + 1):
            for i in range(n - d + 1):
                j = i + d
                # 1. Keep s[i].
                minString = s[i] + dp[i + 1][j]
                # 2. Remove s[i] and s[k] if possible.
                for k in range(i + 1, j):
                    if self._isConsecutive(s[i], s[k]) and dp[i + 1][k] == '':
                        candidate = dp[k + 1][j]
                        if candidate < minString:
                            minString = candidate
                dp[i][j] = minString

        return dp[0][n]

    def _isConsecutive(self, a: str, b: str) -> bool:
        return abs(ord(a) - ord(b)) == 1 or abs(ord(a) - ord(b)) == 25

```

2522. 357.py

Path: LeetCode-main/solutions/357. Count Numbers with Unique Digits/357.py

```

class Solution:
    def countNumbersWithUniqueDigits(self, n: int) -> int:
        if n == 0:
            return 1

        ans = 10
        uniqueDigits = 9
        availableNum = 9

        while n > 1 and availableNum > 0:
            uniqueDigits *= availableNum
            ans += uniqueDigits
            n -= 1
            availableNum -= 1

        return ans

```

2523. 358.py

Path: LeetCode-main/solutions/358. Rearrange String k Distance Apart/358.py

```

class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        n = len(s)
        ans = []
        count = collections.Counter(s)
        # valid[i] := the leftmost index i can appear
        valid = collections.Counter()

        def getBestLetter(index: int) -> str:
            """Returns the valid letter that has the most count."""
            maxCount = -1
            bestLetter = '*'

            for c in string.ascii_lowercase:
                if count[c] > 0 and count[c] > maxCount and index >= valid[c]:
                    bestLetter = c
                    maxCount = count[c]

            return bestLetter

        for i in range(n):
            c = getBestLetter(i)
            if c == '*':
                return ''
            ans.append(c)
            count[c] -= 1
            valid[c] = i + k

        return ''.join(ans)

```

2524. 359-2.py

Path: LeetCode-main/solutions/359. Logger Rate Limiter/359-2.py

```

class Logger:
    def __init__(self):
        self.okTime = {} # {message: ok time}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        if timestamp < self.okTime.get(message, 0):
            return False

        self.okTime[message] = timestamp + 10
        return True

```

2525. 359.py

Path: LeetCode-main/solutions/359. Logger Rate Limiter/359.py

```

class Logger:
    def __init__(self):
        # [(timestamp, message)]
        self.messageQueue = collections.deque()
        self.messageSet = set()

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        # Remove the messages that are 10 seconds from the current timestamp.
        while self.messageQueue:
            headTimestamp, headMessage = self.messageQueue[0]
            if timestamp < headTimestamp + 10:
                break
            self.messageQueue.popleft()
            self.messageSet.remove(headMessage)

        if message in self.messageSet:
            return False

        self.messageQueue.append((timestamp, message))
        self.messageSet.add(message)
        return True

```

2526. 36.py

Path: LeetCode-main/solutions/36. Valid Sudoku/36.py

```

class Solution:
    def isValidSudoku(self, board: list[list[str]]) -> bool:
        seen = set()

        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue
                if (c + '@row ' + str(i) in seen or
                    c + '@col ' + str(j) in seen or
                    c + '@box ' + str(i // 3) + str(j // 3) in seen):
                    return False
                seen.add(c + '@row ' + str(i))
                seen.add(c + '@col ' + str(j))
                seen.add(c + '@box ' + str(i // 3) + str(j // 3))

        return True

```

2527. 360.py

Path: LeetCode-main/solutions/360. Sort Transformed Array/360.py


```
class Solution:
    def sortTransformedArray(
        self,
        nums: list[int],
        a: int,
        b: int,
        c: int,
    ) -> list[int]:
        n = len(nums)
        upward = a > 0
        ans = [0] * n

        # The concavity of f only depends on a's sign.
        def f(x: int, a: int, b: int, c: int) -> int:
            return (a * x + b) * x + c

        quad = [f(num, a, b, c) for num in nums]

        i = n - 1 if upward else 0
        l = 0
        r = n - 1
        while l <= r:
            if upward: # is the maximum in the both ends
                if quad[l] > quad[r]:
                    ans[i] = quad[l]
                    l += 1
                else:
                    ans[i] = quad[r]
                    r -= 1
                i -= 1
            else: # is the minimum in the both ends
                if quad[l] < quad[r]:
                    ans[i] = quad[l]
                    l += 1
                else:
                    ans[i] = quad[r]
                    r -= 1
                i += 1

        return ans
```

2528. 361.py

Path: LeetCode-main/solutions/361. Bomb Enemy/361.py

```

class Solution:
    def maxKilledEnemies(self, grid: list[list[str]]) -> int:
        m = len(grid)
        n = len(grid[0])
        enemyCount = 0
        # dp[i][j] := the maximum enemies grid[i][j] can kill
        dp = [[0] * n for _ in range(m)]

        def update(i: int, j: int) -> None:
            nonlocal enemyCount
            if grid[i][j] == '0':
                dp[i][j] += enemyCount
            elif grid[i][j] == 'E':
                enemyCount += 1
            else: # grid[i][j] == 'W'
                enemyCount = 0

        # Extend the four directions, if meet 'W', need to start over from 0.
        for i in range(m):
            enemyCount = 0
            for j in range(n):
                update(i, j)
            enemyCount = 0
            for j in reversed(range(n)):
                update(i, j)

        for j in range(n):
            enemyCount = 0
            for i in range(m):
                update(i, j)
            enemyCount = 0
            for i in reversed(range(m)):
                update(i, j)

        # Returns sum(map(sum, dp))
        return max(map(max, dp))

```

2529. 362.py

Path: LeetCode-main/solutions/362. Design Hit Counter/362.py

```

class HitCounter:
    def __init__(self):
        self.timestamps = [0] * 300
        self.hits = [0] * 300

    def hit(self, timestamp: int) -> None:
        i = timestamp % 300
        if self.timestamps[i] == timestamp:
            self.hits[i] += 1
        else:
            self.timestamps[i] = timestamp
            self.hits[i] = 1 # Reset the hit count to 1.

    def getHits(self, timestamp: int) -> int:
        return sum(h for t, h in zip(self.timestamps, self.hits)
                    if timestamp - t < 300)

```

2530. 363.py

Path: LeetCode-main/solutions/363. Max Sum of Rectangle No Larger Than K/363.py

```
from sortedcontainers import SortedList

class Solution:
    def maxSumSubmatrix(self, matrix: list[list[int]], k: int) -> int:
        m = len(matrix)
        n = len(matrix[0])
        ans = -math.inf

        for baseCol in range(n):
            # sums[i] := sum(matrix[i][baseCol..j])
            sums = [0] * m
            for j in range(baseCol, n):
                for i in range(m):
                    sums[i] += matrix[i][j]
                # Find the maximum sum <= k of all the subarrays.
                accumulate = SortedList([0])
                prefix = 0
                for summ in sums:
                    prefix += summ
                    it = accumulate.bisect_left(prefix - k)
                    if it != len(accumulate):
                        ans = max(ans, prefix - accumulate[it])
                    accumulate.add(prefix)

        return ans
```

2531. 364.py

Path: LeetCode-main/solutions/364. Nested List Weight Sum II/364.py

```
class Solution:
    def depthSumInverse(self, nestedList: list[NestedInteger]) -> int:
        ans = 0
        prevSum = 0
        q = collections.deque(nestedList)

        while q:
            for _ in range(len(q)):
                ni = q.popleft()
                if ni.isInteger():
                    prevSum += ni.getInteger()
                else:
                    for nextNi in ni.getList():
                        q.append(nextNi)
            ans += prevSum

        return ans
```

2532. 365.py

Path: LeetCode-main/solutions/365. Water and Jug Problem/365.py

```
class Solution:
    def canMeasureWater(
        self,
        jug1Capacity: int,
        jug2Capacity: int,
        targetCapacity: int,
    ) -> bool:
        return (targetCapacity == 0 or
                jug1Capacity + jug2Capacity >= targetCapacity and
                targetCapacity % gcd(jug1Capacity, jug2Capacity) == 0)
```

2533. 366.py

Path: LeetCode-main/solutions/366. Find Leaves of Binary Tree/366.py

```
class Solution:
    def findLeaves(self, root: TreeNode | None) -> list[list[int]]:
        ans = []

        def depth(root: TreeNode | None) -> int:
            """Returns the depth of the root (0-indexed)."""
            if not root:
                return -1

            l = depth(root.left)
            r = depth(root.right)
            h = 1 + max(l, r)

            if len(ans) == h: # Meet a leaf
                ans.append([])

            ans[h].append(root.val)
            return h

        depth(root)
        return ans
```

2534. 367.py

Path: LeetCode-main/solutions/367. Valid Perfect Square/367.py

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        l = bisect.bisect_left(range(num), num, key=lambda m: m * m)
        return l**2 == num
```

2535. 368.py

Path: LeetCode-main/solutions/368. Largest Divisible Subset/368.py

```

class Solution:
    def largestDivisibleSubset(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = []
        count = [1] * n
        prevIndex = [-1] * n
        maxCount = 0
        index = -1

        nums.sort()

        for i, num in enumerate(nums):
            for j in reversed(range(i)):
                if num % nums[j] == 0 and count[i] < count[j] + 1:
                    count[i] = count[j] + 1
                    prevIndex[i] = j
            if count[i] > maxCount:
                maxCount = count[i]
                index = i

        while index != -1:
            ans.append(nums[index])
            index = prevIndex[index]

        return ans

```

2536. 369-2.py

Path: LeetCode-main/solutions/369. Plus One Linked List/369-2.py

```

class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        dummy.next = head

        while head:
            if head.val != 9:
                curr = head
                head = head.next
            # `curr` now points to the rightmost non-9 node.

            curr.val += 1
            while curr.next:
                curr.next.val = 0
                curr = curr.next

            return dummy.next if dummy.val == 0 else dummy

```

2537. 369.py

Path: LeetCode-main/solutions/369. Plus One Linked List/369.py

```

class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        if not head:
            return ListNode(1)
        if self._addOne(head) == 1:
            return ListNode(1, head)
        return head

    def _addOne(self, node: ListNode) -> int:
        carry = self._addOne(node.next) if node.next else 1
        summ = node.val + carry
        node.val = summ % 10
        return summ // 10

```

2538. 37.py

Path: LeetCode-main/solutions/37. Sudoku Solver/37.py

```

class Solution:
    def solveSudoku(self, board: list[list[str]]) -> None:
        def isValid(row: int, col: int, c: str) -> bool:
            for i in range(9):
                if (board[i][col] == c or
                    board[row][i] == c or
                    board[3 * (row // 3) + i // 3][3 * (col // 3) + i % 3] == c):
                    return False
            return True

        def solve(s: int) -> bool:
            if s == 81:
                return True

            i = s // 9
            j = s % 9

            if board[i][j] != '.':
                return solve(s + 1)

            for c in string.digits[1:]:
                if isValid(i, j, c):
                    board[i][j] = c
                    if solve(s + 1):
                        return True
                    board[i][j] = '.'

            return False

        solve(0)

```

2539. 370.py

Path: LeetCode-main/solutions/370. Range Addition/370.py

```
class Solution:
    def getModifiedArray(
        self,
        length: int,
        updates: list[list[int]],
    ) -> list[int]:
        line = [0] * length

        for start, end, inc in updates:
            line[start] += inc
            if end + 1 < length:
                line[end + 1] -= inc

        return itertools.accumulate(line)
```

2540. 371.py

Path: LeetCode-main/solutions/371. Sum of Two Integers/371.py

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        mask = 0xFFFFFFFF
        MAX = 2000

        while b != 0:
            a, b = (a ^ b) & mask, ((a & b) << 1) & mask

        return a if a < MAX else ~(a ^ mask)
```

2541. 372.py

Path: LeetCode-main/solutions/372. Super Pow/372.py

```
class Solution:
    def superPow(self, a: int, b: list[int]) -> int:
        MOD = 1337
        ans = 1

        for i in b:
            ans = pow(ans, 10, MOD) * pow(a, i, MOD)

        return ans % MOD
```

2542. 373.py

Path: LeetCode-main/solutions/373. Find K Pairs with Smallest Sums/373.py

```

class Solution:
    def kSmallestPairs(self, nums1: list[int],
                      nums2: list[int],
                      k: int) -> list[list[int]]:
        minHeap = []

        for i in range(min(k, len(nums1))):
            heapq.heappush(minHeap, (nums1[i] + nums2[0], i, 0))

        ans = []
        while minHeap and len(ans) < k:
            _, i, j = heapq.heappop(minHeap)
            ans.append([nums1[i], nums2[j]])
            if j + 1 < len(nums2):
                heapq.heappush(minHeap, (nums1[i] + nums2[j + 1], i, j + 1))

        return ans

```

2543. 374.py

Path: LeetCode-main/solutions/374. Guess Number Higher or Lower/374.py

```

# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
#         1 if num is lower than the picked number
#         otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        l = 1
        r = n

        # Find the first guess number that >= the target number
        while l < r:
            m = (l + r) // 2
            if guess(m) <= 0: # -1, 0
                r = m
            else:
                l = m + 1

        return l

```

2544. 375-2.py

Path: LeetCode-main/solutions/375. Guess Number Higher or Lower II/375-2.py


```

class Solution:
    def getMoneyAmount(self, n: int) -> int:
        # dp[i][j] := the minimum money you need to guarantee a win of picking i..j
        dp = [[0] * (n + 2) for _ in range(n + 2)]

        for d in range(1, n + 1):
            for i in range(1, n - d + 1):
                j = i + d
                dp[i][j] = math.inf
                for k in range(i, j + 1):
                    dp[i][j] = min(dp[i][j], max(dp[i][k - 1], dp[k + 1][j]) + k)

        return dp[1][n]

```

2545. 375.py

Path: LeetCode-main/solutions/375. Guess Number Higher or Lower II/375.py

```

class Solution:
    def getMoneyAmount(self, n: int) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns the minimum money you need to guarantee a win of picking i..j.
            """
            if i >= j:
                return 0
            return min(max(dp(i, k - 1), dp(k + 1, j)) + k
                        for k in range(i, j + 1))

        return dp(1, n)

```

2546. 376.py

Path: LeetCode-main/solutions/376. Wiggle Subsequence/376.py

```

class Solution:
    def wiggleMaxLength(self, nums: list[int]) -> int:
        increasing = 1
        decreasing = 1

        for a, b in itertools.pairwise(nums):
            if b > a:
                increasing = decreasing + 1
            elif b < a:
                decreasing = increasing + 1

        return max(increasing, decreasing)

```

2547. 377.py

Path: LeetCode-main/solutions/377. Combination Sum IV/377.py

```

class Solution:
    def combinationSum4(self, nums: list[int], target: int) -> int:
        dp = [1] + [-1] * target

        def dfs(target: int) -> int:
            if target < 0:
                return 0
            if dp[target] != -1:
                return dp[target]

            dp[target] = sum(dfs(target - num) for num in nums)
            return dp[target]

        return dfs(target)

```

2548. 378-2.py

Path: LeetCode-main/solutions/378. Kth Smallest Element in a Sorted Matrix/378-2.py

```

class Solution:
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
        def numsNoGreaterThan(m: int) -> int:
            count = 0
            j = len(matrix[0]) - 1
            # For each row, find the first index j s.t. row[j] <= m s.t. the number of
            # numbers <= m for this row will be j + 1.
            for row in matrix:
                while j >= 0 and row[j] > m:
                    j -= 1
                count += j + 1
            return count

        l = matrix[0][0]
        r = matrix[-1][-1]
        return bisect.bisect_left(range(l, r), k, key=numsNoGreaterThan) + 1

```

2549. 378.py

Path: LeetCode-main/solutions/378. Kth Smallest Element in a Sorted Matrix/378.py

```

class Solution:
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
        minHeap = [] # (matrix[i][j], i, j)

        i = 0
        while i < k and i < len(matrix):
            heapq.heappush(minHeap, (matrix[i][0], i, 0))
            i += 1

        while k > 1:
            k -= 1
            _, i, j = heapq.heappop(minHeap)
            if j + 1 < len(matrix[0]):
                heapq.heappush(minHeap, (matrix[i][j + 1], i, j + 1))

        return minHeap[0][0]

```

2550. 379.py

Path: LeetCode-main/solutions/379. Design Phone Directory/379.py

```
class PhoneDirectory:
    def __init__(self, maxNumbers: int):
        # the next available numbers
        self.next = [i + 1 for i in range(maxNumbers - 1)] + [0]
        # the current possible available number
        self.number = 0

    def get(self) -> int:
        if self.next[self.number] == -1:
            return -1
        ans = self.number
        self.number = self.next[self.number]
        self.next[ans] = -1 # Mark as used.
        return ans

    def check(self, number: int) -> bool:
        return self.next[number] != -1

    def release(self, number: int) -> None:
        if self.next[number] != -1:
            return
        self.next[number] = self.number
        self.number = number
```

2551. 38.py

Path: LeetCode-main/solutions/38. Count and Say/38.py

```
class Solution:
    def countAndSay(self, n: int) -> str:
        ans = '1'

        for _ in range(n - 1):
            nxt = ''
            i = 0
            while i < len(ans):
                count = 1
                while i + 1 < len(ans) and ans[i] == ans[i + 1]:
                    count += 1
                    i += 1
                nxt += str(count) + ans[i]
                i += 1
            ans = nxt

        return ans
```

2552. 380.py

Path: LeetCode-main/solutions/380. Insert Delete GetRandom O(1)/380.py

```

class RandomizedSet:
    def __init__(self):
        self.vals = []
        self.valToIndex = collections.defaultdict(int) # {val: index in vals}

    def insert(self, val: int) -> bool:
        if val in self.valToIndex:
            return False
        self.valToIndex[val] = len(self.vals)
        self.vals.append(val)
        return True

    def remove(self, val: int) -> bool:
        if val not in self.valToIndex:
            return False
        index = self.valToIndex[val]
        # The order of the following two lines is important when vals.size() == 1.
        self.valToIndex[self.vals[-1]] = index
        del self.valToIndex[val]
        self.vals[index] = self.vals[-1]
        self.vals.pop()
        return True

    def getRandom(self) -> int:
        index = random.randint(0, len(self.vals) - 1)
        return self.vals[index]

```

2553. 381.py

Path: LeetCode-main/solutions/381. Insert Delete GetRandom O(1) - Duplicates allowed/381.py

```

class RandomizedCollection:
    def __init__(self):
        self.vals = []
        self.valToIndices = collections.defaultdict(list)

    def insert(self, val: int) -> bool:
        self.valToIndices[val].append(len(self.vals))
        self.vals.append([val, len(self.valToIndices[val]) - 1])
        return len(self.valToIndices[val]) == 1

    def remove(self, val: int) -> bool:
        if val not in self.valToIndices or self.valToIndices[val] == []:
            return False

        index = self.valToIndices[val][-1]
        self.valToIndices[self.vals[-1][0]][self.vals[-1][1]] = index
        self.valToIndices[val].pop()
        self.vals[index] = self.vals[-1]
        self.vals.pop()
        return True

    def getRandom(self) -> int:
        index = random.randint(0, len(self.vals) - 1)
        return self.vals[index][0]

```

2554. 382.py

Path: LeetCode-main/solutions/382. Linked List Random Node/382.py

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def __init__(self, head: ListNode | None):
        self.head = head

    def getRandom(self) -> int:
        res = -1
        i = 1
        curr = self.head

        while curr:
            if random.randint(0, i - 1) == 0:
                res = curr.val
                curr = curr.next
                i += 1

        return res
```

2555. 383.py

Path: LeetCode-main/solutions/383. Ransom Note/383.py

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        count1 = collections.Counter(ransomNote)
        count2 = collections.Counter(magazine)
        return all(count1[c] <= count2[c] for c in string.ascii_lowercase)
```

2556. 384.py

Path: LeetCode-main/solutions/384. Shuffle an Array/384.py

```
class Solution:
    def __init__(self, nums: list[int]):
        self.nums = nums

    def reset(self) -> list[int]:
        return self.nums

    def shuffle(self) -> list[int]:
        arr = self.nums.copy()
        for i in range(len(arr) - 1, 0, -1):
            j = random.randint(0, i)
            arr[i], arr[j] = arr[j], arr[i]
        return arr
```

2557. 385.py

Path: LeetCode-main/solutions/385. Mini Parser/385.py

```

class Solution:
    def deserialize(self, s: str) -> NestedInteger:
        if s[0] != '[':
            return NestedInteger(int(s))

        stack = []

        for i, c in enumerate(s):
            if c == '[':
                stack.append(NestedInteger())
                start = i + 1
            elif c == ',':
                if i > start:
                    num = int(s[start:i])
                    stack[-1].add(NestedInteger(num))
                    start = i + 1
            elif c == ']':
                popped = stack.pop()
                if i > start:
                    num = int(s[start:i])
                    popped.add(NestedInteger(num))
                if stack:
                    stack[-1].add(popped)
                else:
                    return popped
                start = i + 1

```

2558. 386.py

Path: LeetCode-main/solutions/386. Lexicographical Numbers/386.py

```

class Solution:
    def lexicalOrder(self, n: int) -> list[int]:
        ans = []
        curr = 1

        while len(ans) < n:
            ans.append(curr)
            if curr * 10 <= n:
                curr *= 10
            else:
                while curr % 10 == 9 or curr == n:
                    curr //= 10
                curr += 1

        return ans

```

2559. 387.py

Path: LeetCode-main/solutions/387. First Unique Character in a String/387.py

```
class Solution:
    def firstUniqChar(self, s: str) -> int:
        count = collections.Counter(s)

        for i, c in enumerate(s):
            if count[c] == 1:
                return i

        return -1
```

2560. 388.py

Path: LeetCode-main/solutions/388. Longest Absolute File Path/388.py

```
class Solution:
    def lengthLongestPath(self, input: str) -> int:
        ans = 0
        stack = [(-1, 0)] # placeholder

        for token in input.split('\n'):
            depth = token.count('\t')
            token = token.replace('\t', '')
            while depth <= stack[-1][0]:
                stack.pop()
            if '.' in token: # `token` is file.
                ans = max(ans, stack[-1][1] + len(token))
            else: # directory + '/'
                stack.append((depth, stack[-1][1] + len(token) + 1))

        return ans
```

2561. 389-2.py

Path: LeetCode-main/solutions/389. Find the Difference/389-2.py

```
class Solution:
    def findTheDifference(self, s: str, t: str) -> str:
        count = collections.Counter(s)

        for c in t:
            if count[c] == 0:
                return c
            count[c] -= 1
```

2562. 389.py

Path: LeetCode-main/solutions/389. Find the Difference/389.py

```
class Solution:
    def findTheDifference(self, s: str, t: str) -> str:
        sXors = chr(functools.reduce(operator.xor, map(ord, s), 0))
        tXors = chr(functools.reduce(operator.xor, map(ord, t), 0))
        return chr(ord(sXors) ^ ord(tXors))
```

2563. 39.py

Path: LeetCode-main/solutions/39. Combination Sum/39.py

```
class Solution:
    def combinationSum(self, candidates: list[int],
                      target: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.clone())
                return

            for i in range(s, len(candidates)):
                path.append(candidates[i])
                dfs(i, target - candidates[i], path)
                path.pop()

        candidates.sort()
        dfs(0, target, [])
        return ans
```

2564. 390.py

Path: LeetCode-main/solutions/390. Elimination Game/390.py

```
class Solution:
    def lastRemaining(self, n: int) -> int:
        return 1 if n == 1 else 2 * (1 + n // 2 - self.lastRemaining(n // 2))
```

2565. 391.py

Path: LeetCode-main/solutions/391. Perfect Rectangle/391.py


```

class Solution:
    def isRectangleCover(self, rectangles: list[list[int]]) -> bool:
        area = 0
        x1 = math.inf
        y1 = math.inf
        x2 = -math.inf
        y2 = -math.inf
        corners: set[tuple[int, int]] = set()

        for x, y, a, b in rectangles:
            area += (a - x) * (b - y)
            x1 = min(x1, x)
            y1 = min(y1, y)
            x2 = max(x2, a)
            y2 = max(y2, b)

            # the four points of the current rectangle
            for point in [(x, y), (x, b), (a, y), (a, b)]:
                if point in corners:
                    corners.remove(point)
                else:
                    corners.add(point)

        if len(corners) != 4:
            return False
        if ((x1, y1) not in corners or
            (x1, y2) not in corners or
            (x2, y1) not in corners or
            (x2, y2) not in corners):
            return False
        return area == (x2 - x1) * (y2 - y1)

```

2566. 392.py

Path: LeetCode-main/solutions/392. Is Subsequence/392.py

```

class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        if not s:
            return True

        i = 0
        for c in t:
            if s[i] == c:
                i += 1
            if i == len(s):
                return True

        return False

```

2567. 393.py

Path: LeetCode-main/solutions/393. UTF-8 Validation/393.py

```

class Solution:
    def validUtf8(self, data: list[int]) -> bool:
        followedBytes = 0

        for d in data:
            if followedBytes == 0:
                if (d >> 3) == 0b11110:
                    followedBytes = 3
                elif (d >> 4) == 0b1110:
                    followedBytes = 2
                elif (d >> 5) == 0b110:
                    followedBytes = 1
                elif (d >> 7) == 0b0:
                    followedBytes = 0
                else:
                    return False
            else:
                if (d >> 6) != 0b10:
                    return False
                followedBytes -= 1

        return followedBytes == 0

```

2568. 394-2.py

Path: LeetCode-main/solutions/394. Decode String/394-2.py

```

class Solution:
    def decodeString(self, s: str) -> str:
        ans = ''

        while self.i < len(s) and s[self.i] != ']':
            if s[self.i].isdigit():
                k = 0
                while self.i < len(s) and s[self.i].isdigit():
                    k = k * 10 + int(s[self.i])
                    self.i += 1
                self.i += 1 # '['
                decodedString = self.decodeString(s)
                self.i += 1 # ']'
                ans += k * decodedString
            else:
                ans += s[self.i]
                self.i += 1

        return ans

i = 0

```

2569. 394.py

Path: LeetCode-main/solutions/394. Decode String/394.py

```

class Solution:
    def decodeString(self, s: str) -> str:
        stack = [] # (prevStr, repeatCount)
        currStr = ''
        currNum = 0

        for c in s:
            if c.isdigit():
                currNum = currNum * 10 + int(c)
            else:
                if c == '[':
                    stack.append((currStr, currNum))
                    currStr = ''
                    currNum = 0
                elif c == ']':
                    prevStr, num = stack.pop()
                    currStr = prevStr + num * currStr
                else:
                    currStr += c

        return currStr

```

2570. 395.py

Path: LeetCode-main/solutions/395. Longest Substring with At Least K Repeating Characters/395.py

```

class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        def longestSubstringWithNUniqueLetters(n: int) -> int:
            res = 0
            uniqueLetters = 0 # the number of unique letters
            lettersHavingKFreq = 0 # the number of letters having frequency >= k
            count = collections.Counter()

            l = 0
            for r, c in enumerate(s):
                count[c] += 1
                if count[c] == 1:
                    uniqueLetters += 1
                if count[c] == k:
                    lettersHavingKFreq += 1
                while uniqueLetters > n:
                    if count[s[l]] == k:
                        lettersHavingKFreq -= 1
                    count[s[l]] -= 1
                    if count[s[l]] == 0:
                        uniqueLetters -= 1
                    l += 1
                # Since both the number of unique letters and the number of letters
                # having frequency >= k are equal to n, this is a valid window.
                if lettersHavingKFreq == n: # Implicit: uniqueLetters == n
                    res = max(res, r - l + 1)

            return res

        return max(longestSubstringWithNUniqueLetters(n)
                    for n in range(1, 27))

```

2571. 396.py

Path: LeetCode-main/solutions/396. Rotate Function/396.py

```
class Solution:
    def maxRotateFunction(self, nums: list[int]) -> int:
        f = sum(i * num for i, num in enumerate(nums))
        ans = f
        summ = sum(nums)

        for a in reversed(nums):
            f += summ - len(nums) * a
            ans = max(ans, f)

        return ans
```

2572. 397.py

Path: LeetCode-main/solutions/397. Integer Replacement/397.py

```
class Solution:
    def integerReplacement(self, n: int) -> int:
        ans = 0

        while n > 1:
            if n % 2 == 0: # `n` ends in 0.
                n >>= 1
            elif n == 3 or (n >> 1 & 1) == 0: # `n` = 3 or ends in 0b01.
                n -= 1
            else: # `n` ends in 0b11.
                n += 1
            ans += 1

        return ans
```

2573. 398.py

Path: LeetCode-main/solutions/398. Random Pick Index/398.py

```
class Solution:
    def __init__(self, nums: list[int]):
        self.nums = nums

    def pick(self, target: int) -> int:
        ans = -1
        rng = 0
        for i, num in enumerate(self.nums):
            if num == target:
                rng += 1
                if random.randint(0, rng - 1) == 0:
                    ans = i
        return ans
```

2574. 399.py

Path: LeetCode-main/solutions/399. Evaluate Division/399.py

```

class Solution:
    def calcEquation(
        self,
        equations: list[list[str]],
        values: list[float],
        queries: list[list[str]],
    ) -> list[float]:
        ans = []
        # graph[A][B] := A / B
        graph = collections.defaultdict(dict)

        for (A, B), value in zip(equations, values):
            graph[A][B] = value
            graph[B][A] = 1 / value

        def deuide(A: str, C: str, seen: set[str]) -> float:
            """Returns A / C."""
            if A == C:
                return 1.0

            seen.add(A)

            # value := A / B
            for B, value in graph[A].items():
                if B in seen:
                    continue
                res = deuide(B, C, seen) # B / C
                if res > 0: # valid result
                    return value * res # (A / B) * (B / C) = A / C

            return -1.0 # invalid result

        for A, C in queries:
            if A not in graph or C not in graph:
                ans.append(-1.0)
            else:
                ans.append(deuide(A, C, set()))

        return ans

```

2575. 4.py

Path: LeetCode-main/solutions/4. Median of Two Sorted Arrays/4.py

```

class Solution:
    def findMedianSortedArrays(self, nums1: list[int], nums2: list[int]) -> float:
        n1 = len(nums1)
        n2 = len(nums2)
        if n1 > n2:
            return self.findMedianSortedArrays(nums2, nums1)

        l = 0
        r = n1

        while l <= r:
            partition1 = (l + r) // 2
            partition2 = (n1 + n2 + 1) // 2 - partition1
            maxLeft1 = -2**31 if partition1 == 0 else nums1[partition1 - 1]
            maxLeft2 = -2**31 if partition2 == 0 else nums2[partition2 - 1]
            minRight1 = 2**31 - 1 if partition1 == n1 else nums1[partition1]
            minRight2 = 2**31 - 1 if partition2 == n2 else nums2[partition2]
            if maxLeft1 <= minRight2 and maxLeft2 <= minRight1:
                return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) * 0.5 if (n1 + n2) % 2 == 0 else max(maxLeft1, maxLeft2)
            elif maxLeft1 > minRight2:
                r = partition1 - 1
            else:
                l = partition1 + 1

```

2576. 40.py

Path: LeetCode-main/solutions/40. Combination Sum II/40.py

```

class Solution:
    def combinationSum2(self, candidates: list[int],
                        target: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.copy())
                return

            for i in range(s, len(candidates)):
                if i > s and candidates[i] == candidates[i - 1]:
                    continue
                path.append(candidates[i])
                dfs(i + 1, target - candidates[i], path)
                path.pop()

        candidates.sort()
        dfs(0, target, [])
        return ans

```

2577. 400.py

Path: LeetCode-main/solutions/400. Nth Digit/400.py

```
class Solution:
    def findNthDigit(self, n: int) -> int:
        def getDigit(num: int, pos: int, digitSize: int):
            if pos == 0:
                return num % 10
            for _ in range(digitSize - pos):
                num //= 10
            return num % 10

        digitSize = 1
        startNum = 1
        count = 9

        while digitSize * count < n:
            n -= digitSize * count
            digitSize += 1
            startNum *= 10
            count *= 10

        targetNum = startNum + (n - 1) // digitSize
        pos = n % digitSize

        return getDigit(targetNum, pos, digitSize)
```

2578. 401-2.py

Path: LeetCode-main/solutions/401. Binary Watch/401-2.py

```
class Solution:
    def readBinaryWatch(self, turnedOn: int) -> list[str]:
        ans = []

        for h in range(12):
            for m in range(60):
                if h.bit_count() + m.bit_count() == turnedOn:
                    ans.append(f'{h}:{m:02d}')

        return ans
```

2579. 401.py

Path: LeetCode-main/solutions/401. Binary Watch/401.py

```

class Solution:
    def readBinaryWatch(self, turnedOn: int) -> list[str]:
        ans = []
        hours = [1, 2, 4, 8]
        minutes = [1, 2, 4, 8, 16, 32]

        def dfs(turnedOn: int, s: int, h: int, m: int) -> None:
            if turnedOn == 0:
                time = str(h) + ":" + (str(m).zfill(2))
                ans.append(time)
                return

            for i in range(s, len(hours) + len(minutes)):
                if i < 4 and h + hours[i] < 12:
                    dfs(turnedOn - 1, i + 1, h + hours[i], m)
                elif i >= 4 and m + minutes[i - 4] < 60:
                    dfs(turnedOn - 1, i + 1, h, m + minutes[i - 4])

        dfs(turnedOn, 0, 0, 0)
        return ans

```

2580. 402.py

Path: LeetCode-main/solutions/402. Remove K Digits/402.py

```

class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        if len(num) == k:
            return '0'

        ans = []
        stack = []

        for i, digit in enumerate(num):
            while k > 0 and stack and stack[-1] > digit:
                stack.pop()
                k -= 1
            stack.append(digit)

        for _ in range(k):
            stack.pop()

        for c in stack:
            if c == '0' and not ans:
                continue
            ans.append(c)

        return ''.join(ans) if ans else '0'

```

2581. 403-2.py

Path: LeetCode-main/solutions/403. Frog Jump/403-2.py


```
class Solution:
    def canCross(self, stones: list[int]) -> bool:
        n = len(stones)
        # dp[i][j] := True if a frog can make a size j jump from stones[i]
        dp = [[False] * (n + 1) for _ in range(n)]
        dp[0][1] = True

        for i in range(1, n):
            for j in range(i):
                k = stones[i] - stones[j]
                if k <= n and dp[j][k]:
                    dp[i][k - 1] = True
                    dp[i][k] = True
                    dp[i][k + 1] = True

        return any(dp[-1])
```

2582. 403.py

Path: LeetCode-main/solutions/403. Frog Jump/403.py

```
class Solution:
    def canCross(self, stones: list[int]) -> bool:
        n = len(stones)
        # dp[i][j] := True if a frog can make a size j jump to stones[i]
        dp = [[False] * (n + 1) for _ in range(n)]
        dp[0][0] = True

        for i in range(1, n):
            for j in range(i):
                k = stones[i] - stones[j]
                if k > n:
                    continue
                for x in (k - 1, k, k + 1):
                    if 0 <= x <= n:
                        dp[i][k] |= dp[j][x]

        return any(dp[-1])
```

2583. 404-2.py

Path: LeetCode-main/solutions/404. Sum of Left Leaves/404-2.py

```
class Solution:
    def sumOfLeftLeaves(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        ans = 0
        stack = [root]

        while stack:
            root = stack.pop()
            if root.left:
                if not root.left.left and not root.left.right:
                    ans += root.left.val
                else:
                    stack.append(root.left)
            if root.right:
                stack.append(root.right)

        return ans
```

2584. 404.py

Path: LeetCode-main/solutions/404. Sum of Left Leaves/404.py

```
class Solution:
    def sumOfLeftLeaves(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        ans = 0

        if root.left:
            if not root.left.left and not root.left.right:
                ans += root.left.val
            else:
                ans += self.sumOfLeftLeaves(root.left)
        ans += self.sumOfLeftLeaves(root.right)

        return ans
```

2585. 405.py

Path: LeetCode-main/solutions/405. Convert a Number to Hexadecimal/405.py

```
class Solution:
    def toHex(self, num: int) -> str:
        if num == 0:
            return '0'

        hex = '0123456789abcdef'
        ans = []

        # Handling negative numbers by using 32-bit unsigned representation Python's
        # bitwise operation works on signed numbers, so we convert to 32-bit
        # unsigned for negative numbers.
        if num < 0:
            num += 2**32

        while num > 0:
            ans.append(hex[num & 0xF])
            num >>= 4

        return ''.join(reversed(ans))
```

2586. 406.py

Path: LeetCode-main/solutions/406. Queue Reconstruction by Height/406.py

```
class Solution:
    def reconstructQueue(self, people: list[list[int]]) -> list[list[int]]:
        ans = []

        people.sort(key=lambda x: (-x[0], x[1]))

        for person in people:
            ans.insert(person[1], person)

        return ans
```

2587. 407.py

Path: LeetCode-main/solutions/407. Trapping Rain Water II/407.py

```

class Solution:
    def trapRainWater(self, heightMap: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heightMap)
        n = len(heightMap[0])
        ans = 0
        minHeap = []
        seen = set()

        for i in range(m):
            heapq.heappush(minHeap, (heightMap[i][0], i, 0))
            heapq.heappush(minHeap, (heightMap[i][n - 1], i, n - 1))
            seen.add((i, 0))
            seen.add((i, n - 1))

        for j in range(1, n - 1):
            heapq.heappush(minHeap, (heightMap[0][j], 0, j))
            heapq.heappush(minHeap, (heightMap[m - 1][j], m - 1, j))
            seen.add((0, j))
            seen.add((m - 1, j))

        while minHeap:
            h, i, j = heapq.heappop(minHeap)
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if (x, y) in seen:
                    continue
                if heightMap[x][y] < h:
                    ans += h - heightMap[x][y]
                    # Fill water in grid[x][y].
                    heapq.heappush(minHeap, (h, x, y))
                else:
                    heapq.heappush(minHeap, (heightMap[x][y], x, y))
                seen.add((x, y))

        return ans

```

2588. 408.py

Path: LeetCode-main/solutions/408. Valid Word Abbreviation/408.py

```

class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        i = 0 # word's index
        j = 0 # abbr's index

        while i < len(word) and j < len(abbr):
            if word[i] == abbr[j]:
                i += 1
                j += 1
                continue
            if not abbr[j].isdigit() or abbr[j] == '0':
                return False
            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1
            i += num

        return i == len(word) and j == len(abbr)

```

2589. 409.py

Path: LeetCode-main/solutions/409. Longest Palindrome/409.py

```

class Solution:
    def longestPalindrome(self, s: str) -> int:
        ans = 0
        count = collections.Counter(s)

        for c in count.values():
            ans += c if c % 2 == 0 else c - 1

        hasOddCount = any(c % 2 == 1 for c in count.values())
        return ans + hasOddCount

```

2590. 41.py

Path: LeetCode-main/solutions/41. First Missing Positive/41.py

```

class Solution:
    def firstMissingPositive(self, nums: list[int]) -> int:
        n = len(nums)

        # Correct slot:
        # nums[i] = i + 1
        # nums[i] - 1 = i
        # nums[nums[i] - 1] = nums[i]
        for i in range(n):
            while nums[i] > 0 and nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]

        for i, num in enumerate(nums):
            if num != i + 1:
                return i + 1

        return n + 1

```

2591. 410-2.py

Path: LeetCode-main/solutions/410. Split Array Largest Sum/410-2.py

```
class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        n = len(nums)
        # dp[i][k] := the minimum of the maximum sum to split the first i numbers
        # into k groups
        dp = [[math.inf] * (k + 1) for _ in range(n + 1)]
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(1, n + 1):
            dp[i][1] = prefix[i]

        for l in range(2, k + 1):
            for i in range(1, n + 1):
                for j in range(1 - 1, i):
                    dp[i][l] = min(dp[i][l], max(dp[j][l - 1], prefix[i] - prefix[j]))

        return dp[n][k]
```

2592. 410-3.py

Path: LeetCode-main/solutions/410. Split Array Largest Sum/410-3.py

```
class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        l = max(nums)
        r = sum(nums) + 1

        def numGroups(maxSumInGroup: int) -> int:
            groupCount = 1
            sumInGroup = 0

            for num in nums:
                if sumInGroup + num <= maxSumInGroup:
                    sumInGroup += num
                else:
                    groupCount += 1
                    sumInGroup = num

            return groupCount

        while l < r:
            m = (l + r) // 2
            if numGroups(m) > k:
                l = m + 1
            else:
                r = m

        return l
```

2593. 410.py

Path: LeetCode-main/solutions/410. Split Array Largest Sum/410.py

```
class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        prefix = list(itertools.accumulate(nums, initial=0))

        @functools.lru_cache(None)
        def dp(i: int, k: int) -> int:
            """
            Returns the minimum of the maximum sum to split the first i numbers into
            k groups.
            """
            if k == 1:
                return prefix[i]
            return min(max(dp(j, k - 1), prefix[i] - prefix[j])
                       for j in range(k - 1, i))

        return dp(len(nums), k)
```

2594. 411.py

Path: LeetCode-main/solutions/411. Minimum Unique Word Abbreviation/411.py

```

class Solution:
    def minAbbreviation(self, target: str, dictionary: list[str]) -> str:
        m = len(target)

        def getMask(word: str) -> int:
            # mask[i] = 0 := target[i] == word[i]
            # mask[i] = 1 := target[i] != word[i]
            # e.g. target = "apple"
            #       word = "blade"
            #       mask = 11110
            mask = 0
            for i, c in enumerate(word):
                if c != target[i]:
                    mask |= 1 << m - 1 - i
            return mask

        masks = [getMask(word) for word in dictionary if len(word) == m]
        if not masks:
            return str(m)

        abbrs = []

        def getAbbr(cand: int) -> str:
            abbr = []
            replacedCount = 0
            for i, c in enumerate(target):
                if cand >> m - 1 - i & 1:
                    # If cand[i] = 1, `abbr` should show the original character.
                    if replacedCount:
                        abbr += str(replacedCount)
                        abbr.append(c)
                        replacedCount = 0
                    else:
                        # If cand[i] = 0, `abbr` can be replaced.
                        replacedCount += 1
                if replacedCount:
                    abbr.append(str(replacedCount))
            return ''.join(abbr)

        # all the candidate representation of the target
        for cand in range(2**m):
            # All the masks have at lease one bit different from the candidate.
            if all(cand & mask for mask in masks):
                abbr = getAbbr(cand)
                abbrs.append(abbr)

        def getAbbrLen(abbr: str) -> int:
            abbrLen = 0
            i = 0
            j = 0
            while i < len(abbr):
                if abbr[j].isalpha():
                    j += 1
                else:
                    while j < len(abbr) and abbr[j].isdigit():
                        j += 1
                    abbrLen += 1
                i = j
            return abbrLen

        return min(abbrs, key=lambda x: getAbbrLen(x))

```


2595. 412.py

Path: LeetCode-main/solutions/412. Fizz Buzz/412.py

```
class Solution:
    def fizzBuzz(self, n: int) -> list[str]:
        d = {3: 'Fizz', 5: 'Buzz'}
        return [''.join([d[k] for k in d if i % k == 0]) or str(i) for i in range(1, n + 1)]
```

2596. 413-2.py

Path: LeetCode-main/solutions/413. Arithmetic Slices/413-2.py

```
class Solution:
    def numberOfArithmeticSlices(self, nums: list[int]) -> int:
        ans = 0
        dp = 0

        for i in range(2, len(nums)):
            if nums[i] - nums[i - 1] == nums[i - 1] - nums[i - 2]:
                dp += 1
                ans += dp
            else:
                dp = 0

        return ans
```

2597. 413.py

Path: LeetCode-main/solutions/413. Arithmetic Slices/413.py

```
class Solution:
    def numberOfArithmeticSlices(self, nums: list[int]) -> int:
        n = len(nums)
        if n < 3:
            return 0

        dp = [0] * n # dp[i] := the number of arithmetic slices ending in index i

        for i in range(2, len(nums)):
            if nums[i] - nums[i - 1] == nums[i - 1] - nums[i - 2]:
                dp[i] = dp[i - 1] + 1

        return sum(dp)
```

2598. 414-2.py

Path: LeetCode-main/solutions/414. Third Maximum Number/414-2.py

```
class Solution:
    def thirdMax(self, nums: list[int]) -> int:
        minHeap = []
        seen = set()

        for num in nums:
            if num not in seen:
                seen.add(num)
                heapq.heappush(minHeap, num)
                if len(minHeap) > 3:
                    heapq.heappop(minHeap)

        if len(minHeap) == 2:
            heapq.heappop(minHeap)

        return minHeap[0]
```

2599. 414.py

Path: LeetCode-main/solutions/414. Third Maximum Number/414.py

```
class Solution:
    def thirdMax(self, nums: list[int]) -> int:
        max1 = -math.inf # the maximum
        max2 = -math.inf # the second maximum
        max3 = -math.inf # the third maximum

        for num in nums:
            if num > max1:
                max3 = max2
                max2 = max1
                max1 = num
            elif max1 > num and num > max2:
                max3 = max2
                max2 = num
            elif max2 > num and num > max3:
                max3 = num

        return max1 if max3 == -math.inf else max3
```

2600. 415.py

Path: LeetCode-main/solutions/415. Add Strings/415.py

```
class Solution:
    def addStrings(self, num1: str, num2: str) -> str:
        ans = []
        carry = 0
        i = len(num1) - 1
        j = len(num2) - 1

        while i >= 0 or j >= 0 or carry:
            if i >= 0:
                carry += int(num1[i])
            if j >= 0:
                carry += int(num2[j])
            ans.append(str(carry % 10))
            carry //= 10
            i -= 1
            j -= 1

        return ''.join(reversed(ans))
```

2601. 416-2.py

Path: LeetCode-main/solutions/416. Partition Equal Subset Sum/416-2.py

```
class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        summ = sum(nums)
        if summ % 2 == 1:
            return False
        return self.knapsack_(nums, summ // 2)

    def knapsack_(self, nums: list[int], subsetSum: int) -> bool:
        # dp[i] := True if i can be formed by nums so far
        dp = [False] * (subsetSum + 1)
        dp[0] = True

        for num in nums:
            for i in range(subsetSum, num - 1, -1):
                dp[i] = dp[i] or dp[i - num]

        return dp[subsetSum]
```

2602. 416.py

Path: LeetCode-main/solutions/416. Partition Equal Subset Sum/416.py

```

class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        summ = sum(nums)
        if summ % 2 == 1:
            return False
        return self.knapsack_(nums, summ // 2)

    def knapsack_(self, nums: list[int], subsetSum: int) -> bool:
        n = len(nums)
        # dp[i][j] := True if j can be formed by nums[0..i]
        dp = [[False] * (subsetSum + 1) for _ in range(n + 1)]
        dp[0][0] = True

        for i in range(1, n + 1):
            num = nums[i - 1]
            for j in range(subsetSum + 1):
                if j < num:
                    dp[i][j] = dp[i - 1][j]
                else:
                    dp[i][j] = dp[i - 1][j] or dp[i - 1][j - num]

        return dp[n][subsetSum]

```

2603. 417-2.py

Path: LeetCode-main/solutions/417. Pacific Atlantic Water Flow/417-2.py

```

class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        m = len(heights)
        n = len(heights[0])
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]

        def dfs(i: int, j: int, h: int, seen: list[list[bool]]) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if seen[i][j] or heights[i][j] < h:
                return

            seen[i][j] = True
            dfs(i + 1, j, heights[i][j], seen)
            dfs(i - 1, j, heights[i][j], seen)
            dfs(i, j + 1, heights[i][j], seen)
            dfs(i, j - 1, heights[i][j], seen)

        for i in range(m):
            dfs(i, 0, 0, seenP)
            dfs(i, n - 1, 0, seenA)

        for j in range(n):
            dfs(0, j, 0, seenP)
            dfs(m - 1, j, 0, seenA)

        return [[i, j]
                for i in range(m)
                for j in range(n)
                if seenP[i][j] and seenA[i][j]]

```

2604. 417.py

Path: LeetCode-main/solutions/417. Pacific Atlantic Water Flow/417.py

```
class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heights)
        n = len(heights[0])
        qP = collections.deque()
        qA = collections.deque()
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]

        for i in range(m):
            qP.append((i, 0))
            qA.append((i, n - 1))
            seenP[i][0] = True
            seenA[i][n - 1] = True

        for j in range(n):
            qP.append((0, j))
            qA.append((m - 1, j))
            seenP[0][j] = True
            seenA[m - 1][j] = True

        def bfs(q: deque, seen: list[list[bool]]):
            while q:
                i, j = q.popleft()
                h = heights[i][j]
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if seen[x][y] or heights[x][y] < h:
                        continue
                    q.append((x, y))
                    seen[x][y] = True

        bfs(qP, seenP)
        bfs(qA, seenA)

        return [[i, j] for i in range(m) for j in range(n) if seenP[i][j] and seenA[i][j]]
```

2605. 418.py

Path: LeetCode-main/solutions/418. Sentence Screen Fitting/418.py

```
class Solution:
    def wordsTyping(self, sentence: list[str], rows: int, cols: int) -> int:
        combined = ' '.join(sentence) + ' '
        n = len(combined)
        i = 0

        for _ in range(rows):
            i += cols
            if combined[i % n] == ' ':
                i += 1
            else:
                while i > 0 and combined[(i - 1) % n] != ' ':
                    i -= 1

        return i // n
```

2606. 419.py

Path: LeetCode-main/solutions/419. Battleships in a Board/419.py

```
class Solution:
    def countBattleships(self, board: list[list[str]]) -> int:
        ans = 0

        for i, row in enumerate(board):
            for j, cell in enumerate(row):
                if cell == '.':
                    continue
                if i > 0 and board[i - 1][j] == 'X':
                    continue
                if j > 0 and board[i][j - 1] == 'X':
                    continue
                ans += 1

        return ans
```

2607. 42-2.py

Path: LeetCode-main/solutions/42. Trapping Rain Water/42-2.py

```
class Solution:
    def trap(self, height: list[int]) -> int:
        if not height:
            return 0

        ans = 0
        l = 0
        r = len(height) - 1
        maxL = height[l]
        maxR = height[r]

        while l < r:
            if maxL < maxR:
                ans += maxL - height[l]
                l += 1
                maxL = max(maxL, height[l])
            else:
                ans += maxR - height[r]
                r -= 1
                maxR = max(maxR, height[r])

        return ans
```

2608. 42.py

Path: LeetCode-main/solutions/42. Trapping Rain Water/42.py

```
class Solution:
    def trap(self, height: list[int]) -> int:
        n = len(height)
        l = [0] * n # l[i] := max(height[0..i])
        r = [0] * n # r[i] := max(height[i..n))

        for i, h in enumerate(height):
            l[i] = h if i == 0 else max(h, l[i - 1])

        for i, h in reversed(list(enumerate(height))):
            r[i] = h if i == n - 1 else max(h, r[i + 1])

        return sum(min(l[i], r[i]) - h
                    for i, h in enumerate(height))
```

2609. 420.py

Path: LeetCode-main/solutions/420. Strong Password Checker/420.py

```

class Solution:
    def strongPasswordChecker(self, password: str) -> int:
        n = len(password)
        missing = self._getMissing(password)
        # the number of replacements to deal with 3 repeating characters
        replaces = 0
        # the number of sequences that can be substituted with 1 deletions,
        # (3k)-seqs
        oneSeq = 0
        # the number of sequences that can be substituted with 2 deletions,
        # (3k + 1)-seqs
        twoSeq = 0

        i = 2
        while i < n:
            if password[i] == password[i - 1] and password[i - 1] == password[i - 2]:
                length = 2 # the length of the repeating password
                while i < n and password[i] == password[i - 1]:
                    length += 1
                    i += 1
                replaces += length // 3 # 'aaaaaaa' -> 'aaxaaxa'
                if length % 3 == 0:
                    oneSeq += 1
                if length % 3 == 1:
                    twoSeq += 1
            else:
                i += 1

        if n < 6:
            return max(6 - n, missing)
        if n <= 20:
            return max(replaces, missing)

        deletes = n - 20
        # Each replacement in (3k)-seqs can be substituted with 1 deletions.
        replaces -= min(oneSeq, deletes)
        # Each replacement in (3k + 1)-seqs can be substituted with 2 deletions.
        replaces -= min(max(deletes - oneSeq, 0), twoSeq * 2) // 2
        # Each replacement in other seqs can be substituted with 3 deletions.
        replaces -= max(deletes - oneSeq - twoSeq * 2, 0) // 3
        return deletes + max(replaces, missing)

    def _getMissing(self, password: str) -> int:
        return (3
            - any(c.isupper() for c in password)
            - any(c.islower() for c in password)
            - any(c.isdigit() for c in password))

```

2610. 421-2.py

Path: LeetCode-main/solutions/421. Maximum XOR of Two Numbers in an Array/421-2.py


```

class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2

class BitTrie:
    def __init__(self, maxBit: int):
        self.maxBit = maxBit
        self.root = TrieNode()

    def insert(self, num: int) -> None:
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = num >> i & 1
            if not node.children[bit]:
                node.children[bit] = TrieNode()
            node = node.children[bit]

    def getMaxXor(self, num: int) -> int:
        maxXor = 0
        node = self.root
        for i in range(self.maxBit, -1, -1):
            bit = num >> i & 1
            toggleBit = bit ^ 1
            if node.children[toggleBit]:
                maxXor = maxXor | 1 << i
                node = node.children[toggleBit]
            elif node.children[bit]:
                node = node.children[bit]
            else: # There's nothing in the Bit Trie.
                return 0
        return maxXor

class Solution:
    def findMaximumXOR(self, nums: list[int]) -> int:
        maxNum = max(nums)
        if maxNum == 0:
            return 0
        maxBit = int(math.log2(maxNum))
        ans = 0
        bitTrie = BitTrie(maxBit)

        for num in nums:
            ans = max(ans, bitTrie.getMaxXor(num))
            bitTrie.insert(num)

        return ans

```

2611. 421.py

Path: LeetCode-main/solutions/421. Maximum XOR of Two Numbers in an Array/421.py

```

class Solution:
    def findMaximumXOR(self, nums: list[int]) -> int:
        maxNum = max(nums)
        if maxNum == 0:
            return 0
        maxBit = int(math.log2(maxNum))
        ans = 0
        prefixMask = 0 # `prefixMask` grows like: 10000 -> 11000 -> ... -> 11111.

        # If ans is 11100 when i = 2, it means that before we reach the last two
        # bits, 11100 is the maximum XOR we have, and we're going to explore if we
        # can get another two 1s and put them into `ans`.
        for i in range(maxBit, -1, -1):
            prefixMask |= 1 << i
            # We only care about the left parts,
            # If i = 2, nums = [1110, 1011, 0111]
            # -> prefixes = [1100, 1000, 0100]
            prefixes = set([num & prefixMask for num in nums])
            # If i = 1 and before this iteration, the ans is 10100, it means that we
            # want to grow the ans to 10100 | 1 << 1 = 10110 and we're looking for
            # XOR of two prefixes = candidate.
            candidate = ans | 1 << i
            for prefix in prefixes:
                if prefix ^ candidate in prefixes:
                    ans = candidate
                    break

        return ans

```

2612. 422.py

Path: LeetCode-main/solutions/422. Valid Word Square/422.py

```

class Solution:
    def validWordSquare(self, words: list[str]) -> bool:
        for i, word in enumerate(words):
            for j, c in enumerate(word):
                if len(words) <= j or len(words[j]) <= i: # out-of-bounds
                    return False
                if c != words[j][i]:
                    return False
        return True

```

2613. 423.py

Path: LeetCode-main/solutions/423. Reconstruct Original Digits from English/423.py

```

class Solution:
    def originalDigits(self, s: str) -> str:
        count = [0] * 10

        for c in s:
            if c == 'z':
                count[0] += 1
            if c == 'o':
                count[1] += 1
            if c == 'w':
                count[2] += 1
            if c == 'h':
                count[3] += 1
            if c == 'u':
                count[4] += 1
            if c == 'f':
                count[5] += 1
            if c == 'x':
                count[6] += 1
            if c == 's':
                count[7] += 1
            if c == 'g':
                count[8] += 1
            if c == 'i':
                count[9] += 1

        count[1] -= count[0] + count[2] + count[4]
        count[3] -= count[8]
        count[5] -= count[4]
        count[7] -= count[6]
        count[9] -= count[5] + count[6] + count[8]
        return ''.join(chr(i + ord('0')) for i, c in enumerate(count)
                        for _ in range(c))

```

2614. 424-2.py

Path: LeetCode-main/solutions/424. Longest Repeating Character Replacement/424-2.py

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        maxCount = 0
        count = collections.Counter()

        # l and r track the maximum window instead of the valid window.
        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])
            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1

        return r - l + 1

```

2615. 424.py

Path: LeetCode-main/solutions/424. Longest Repeating Character Replacement/424.py

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        ans = 0
        maxCount = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])
            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

2616. 425.py

Path: LeetCode-main/solutions/425. Word Squares/425.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.startsWith: list[str] = []

class Trie:
    def __init__(self, words: list[str]):
        self.root = TrieNode()
        for word in words:
            self._insert(word)

    def findBy(self, prefix: str) -> list[str]:
        node = self.root
        for c in prefix:
            if c not in node.children:
                return []
            node = node.children[c]
        return node.startsWith

    def _insert(self, word: str) -> None:
        node = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
            node.startsWith.append(word)

class Solution:
    def wordSquares(self, words: list[str]) -> list[list[str]]:
        if not words:
            return []

        n = len(words[0])
        ans = []
        path = []
        trie = Trie(words)

        for word in words:
            path.append(word)
            self._dfs(trie, n, path, ans)
            path.pop()

```

```

    return ans

def _dfs(self, trie: Trie, n: int, path: list[str], ans: list[list[str]]):
    if len(path) == n:
        ans.append(path.copy())
        return

    prefix = self._getPrefix(path)

    for s in trie.findBy(prefix):
        path.append(s)
        self._dfs(trie, n, path, ans)
        path.pop()

def _getPrefix(self, path: list[str]) -> str:
    """
    e.g. path = ["wall",
                 "area"]
    prefix = "le.."
    """
    prefix = []
    index = len(path)
    for s in path:
        prefix.append(s[index])
    return ''.join(prefix)

```

2617. 426-2.py

Path: LeetCode-main/solutions/426. Convert Binary Search Tree to Sorted Doubly Linked List/426-2.py

```

class Solution:
    def treeToDoublyList(self, root: 'Node | None') -> 'Node | None':
        if not root:
            return None

        stack = []
        first = None
        pred = None

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()
            if not first:
                first = root
            if pred:
                pred.right = root
                root.left = pred
            pred = root
            root = root.right

        pred.right = first
        first.left = pred
        return first

```

2618. 426.py

Path: LeetCode-main/solutions/426. Convert Binary Search Tree to Sorted Doubly Linked List/426.py

```

class Solution:
    def treeToDoublyList(self, root: 'Node | None') -> 'Node | None':
        if not root:
            return None
        leftHead = self.treeToDoublyList(root.left)
        rightHead = self.treeToDoublyList(root.right)
        root.left = root
        root.right = root
        return self._connect(self._connect(leftHead, root), rightHead)

    def _connect(self, node1: 'Node | None', node2: 'Node | None') -> 'Node | None':
        if not node1:
            return node2
        if not node2:
            return node1

        tail1 = node1.left
        tail2 = node2.left

        # Connect node1's tail with node2.
        tail1.right = node2
        node2.left = tail1

        # Connect node2's tail with node1.
        tail2.right = node1
        node1.left = tail2
        return node1

```

2619. 427.py

Path: LeetCode-main/solutions/427. Construct Quad Tree/427.py

```

class Solution:
    def construct(self, grid: list[list[int]]) -> 'Node':
        return self._helper(grid, 0, 0, len(grid))

    def _helper(self, grid: list[list[int]], i: int, j: int, w: int) -> 'Node':
        if self._allSame(grid, i, j, w):
            return Node(grid[i][j] == 1, True)
        half = w // 2
        return Node(True, False,
                    self._helper(grid, i, j, half),
                    self._helper(grid, i, j + half, half),
                    self._helper(grid, i + half, j, half),
                    self._helper(grid, i + half, j + half, half))

    def _allSame(self, grid: list[list[int]], i: int, j: int, w: int) -> bool:
        return all(grid[x][y] == grid[i][j]
                   for x in range(i, i + w)
                   for y in range(j, j + w))

```

2620. 428.py

Path: LeetCode-main/solutions/428. Serialize and Deserialize N-ary Tree/428.py

```

class Codec:
    def serialize(self, root: 'Node') -> str:
        """Encodes a tree to a single string."""
        if not root:
            return ''

        s = []
        q = collections.deque([root])
        s.append(str(root.val) + ' ')

        while q:
            for _ in range(len(q)):
                node = q.popleft()
                if not node.children:
                    s.append('n')
                else:
                    for child in node.children:
                        q.append(child)
                        s.append(str(child.val) + '#')
                    s.append(' ')

            return ''.join(s)

    def deserialize(self, data: str) -> 'Node':
        """Decodes your encoded data to tree."""
        if not data:
            return None

        words = data.split()
        root = Node(int(words[0]))
        q = collections.deque([root])

        for word in words[1:]:
            parent = q.popleft()
            children = []
            for kid in word.split('#'):
                if kid in ('', 'n'):
                    continue
                child = Node(int(kid))
                children.append(child)
                q.append(child)
            parent.children = children

        return root

```

2621. 429.py

Path: LeetCode-main/solutions/429. N-ary Tree Level Order Traversal/429.py

```

class Solution:
    def levelOrder(self, root: 'Node') -> list[list[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            currLevel = []
            for _ in range(len(q)):
                node = q.popleft()
                currLevel.append(node.val)
                for child in node.children:
                    q.append(child)
            ans.append(currLevel)

        return ans

```

2622. 43.py

Path: LeetCode-main/solutions/43. Multiply Strings/43.py

```

class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        s = [0] * (len(num1) + len(num2))

        for i in reversed(range(len(num1))):
            for j in reversed(range(len(num2))):
                mult = int(num1[i]) * int(num2[j])
                summ = mult + s[i + j + 1]
                s[i + j] += summ // 10
                s[i + j + 1] = summ % 10

        for i, c in enumerate(s):
            if c != 0:
                break

        return ''.join(map(str, s[i:]))

```

2623. 430-2.py

Path: LeetCode-main/solutions/430. Flatten a Multilevel Doubly Linked List/430-2.py


```
class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        curr = head

        while curr:
            if curr.child:
                cachedNext = curr.next
                curr.next = curr.child
                curr.child.prev = curr
                curr.child = None
                tail = curr.next
                while tail.next:
                    tail = tail.next
                tail.next = cachedNext
                if cachedNext:
                    cachedNext.prev = tail
                curr = curr.next

        return head
```

2624. 430.py

Path: LeetCode-main/solutions/430. Flatten a Multilevel Doubly Linked List/430.py

```
class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def flatten(head: 'Node', rest: 'Node') -> 'Node':
            if not head:
                return rest

            head.next = flatten(head.child, flatten(head.next, rest))
            if head.next:
                head.next.prev = head
            head.child = None
            return head

        return flatten(head, None)
```

2625. 431-2.py

Path: LeetCode-main/solutions/431. Encode N-ary Tree to Binary Tree/431-2.py

```
class Codec:
    # Encodes an n-ary tree to a binary tree.
    def encode(self, root: 'Node') -> TreeNode | None:
        if not root:
            return None

        rootTreeNode = TreeNode(root.val)
        if root.children:
            rootTreeNode.left = self.encode(root.children[0])

        # The parent for the rest of the children
        currTreeNode = rootTreeNode.left

        # Encode the rest of the children
        for i in range(1, len(root.children)):
            currTreeNode.right = self.encode(root.children[i])
            currTreeNode = currTreeNode.right

        return rootTreeNode

    # Decodes your binary tree to an n-ary tree.
    def decode(self, root: TreeNode | None) -> 'Node':
        if not root:
            return None

        rootNode = Node(root.val, [])
        currTreeNode = root.left

        while currTreeNode:
            rootNode.children.append(self.decode(currTreeNode))
            currTreeNode = currTreeNode.right

        return rootNode
```

2626. 431.py

Path: LeetCode-main/solutions/431. Encode N-ary Tree to Binary Tree/431.py

```

class Codec:
    # Encodes an n-ary tree to a binary tree.
    def encode(self, root: 'Node') -> TreeNode | None:
        if not root:
            return None

        rootTreeNode = TreeNode(root.val)
        q = collections.deque([(root, rootTreeNode)])

        while q:
            parentNode, parentTreeNode = q.popleft()
            prevTreeNode = None
            headTreeNode = None
            for child in parentNode.children:
                currTreeNode = TreeNode(child.val)
                if prevTreeNode:
                    prevTreeNode.right = currTreeNode
                else:
                    headTreeNode = currTreeNode
                prevTreeNode = currTreeNode
                q.append((child, currTreeNode))
            parentTreeNode.left = headTreeNode

        return rootTreeNode

    # Decodes your binary tree to an n-ary tree.
    def decode(self, root: TreeNode | None) -> 'Node':
        if not root:
            return None

        rootNode = Node(root.val, [])
        q = collections.deque([(rootNode, root)])

        while q:
            parentNode, parentTreeNode = q.popleft()
            sibling = parentTreeNode.left
            while sibling:
                currNode = Node(sibling.val, [])
                parentNode.children.append(currNode)
                q.append((currNode, sibling))
                sibling = sibling.right

        return rootNode

```

2627. 432.py

Path: LeetCode-main/solutions/432. All O one Data Structure/432.py`

```

from dataclasses import dataclass

@dataclass
class Node:
    def __init__(self, count: int, key: str | None = None):
        self.count = count
        self.keys: set[str] = {key} if key else set()
        self.prev: Node | None = None
        self.next: Node | None = None

    def __eq__(self, other) -> bool:
        if not isinstance(other, Node):
            return NotImplemented

```

```
return self.count == other.count and self.keys == other.keys
```

```
class AllOne:
    def __init__(self):
        self.keyToNode: dict[str, Node] = {}
        self.head = Node(0)
        self.tail = Node(0)
        self.head.next = self.tail
        self.tail.prev = self.head

    def inc(self, key: str) -> None:
        if key in self.keyToNode:
            self._incrementExistingKey(key)
        else:
            self._addNewKey(key)

    def dec(self, key: str) -> None:
        # It is guaranteed that key exists in the data structure before the
        # decrement.
        self._decrementExistingKey(key)

    def getMaxKey(self) -> str:
        return '' if self.tail.prev == self.head \
            else next(iter(self.tail.prev.keys))

    def getMinKey(self) -> str:
        return '' if self.head.next == self.tail \
            else next(iter(self.head.next.keys))

    def _addNewKey(self, key: str) -> None:
        """Adds a new node with frequency 1."""
        if self.head.next.count == 1:
            self.head.next.keys.add(key)
        else:
            self._insertAfter(self.head, Node(1, key))
        self.keyToNode[key] = self.head.next

    def _incrementExistingKey(self, key: str) -> None:
        """Increments the frequency of the key by 1."""
        node = self.keyToNode[key]
        node.keys.remove(key)
        if node.next == self.tail or node.next.count > node.count + 1:
            self._insertAfter(node, Node(node.count + 1))
        node.next.keys.add(key)
        self.keyToNode[key] = node.next
        if not node.keys:
            self._remove(node)

    def _decrementExistingKey(self, key: str) -> None:
        """Decrements the count of the key by 1."""
        node = self.keyToNode[key]
        node.keys.remove(key)
        if node.count > 1:
            if node.prev == self.head or node.prev.count != node.count - 1:
                self._insertAfter(node.prev, Node(node.count - 1))
            node.prev.keys.add(key)
            self.keyToNode[key] = node.prev
        else:
            del self.keyToNode[key]
        if not node.keys:
            self._remove(node)

    def _insertAfter(self, node: Node, newNode: Node) -> None:
        newNode.prev = node
        newNode.next = node.next
```

```
node.next.prev = newNode
node.next = newNode
```

```
def _remove(self, node: Node) -> None:
    node.prev.next = node.next
    node.next.prev = node.prev
```

2628. 433.py

Path: LeetCode-main/solutions/433. Minimum Genetic Mutation/433.py

```
class Solution:
    def minMutation(self, startGene: str, endGene: str, bank: list[str]) -> int:
        bankSet = set(bank)
        if endGene not in bankSet:
            return -1

        GENES = 'ACGT'
        q = collections.deque([startGene])

        step = 1
        while q:
            for _ in range(len(q)):
                wordList = list(q.popleft())
                for j, cache in enumerate(wordList):
                    for c in GENES:
                        wordList[j] = c
                        word = ''.join(wordList)
                        if word == endGene:
                            return step
                        if word in bankSet:
                            bankSet.remove(word)
                            q.append(word)
                wordList[j] = cache
            step += 1

        return -1
```

2629. 434.py

Path: LeetCode-main/solutions/434. Number of Segments in a String/434.py

```
class Solution:
    def countSegments(self, s: str) -> int:
        return len(s.split())
```

2630. 435.py

Path: LeetCode-main/solutions/435. Non-overlapping Intervals/435.py

```
class Solution:
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
        ans = 0
        currentEnd = -math.inf

        for interval in sorted(intervals, key=lambda x: x[1]):
            if interval[0] >= currentEnd:
                currentEnd = interval[1]
            else:
                ans += 1

        return ans
```

2631. 436.py

Path: LeetCode-main/solutions/436. Find Right Interval/436.py

```
from sortedcontainers import SortedDict

class Solution:
    def findRightInterval(self, intervals: list[list[int]]) -> list[int]:
        ans = []
        startToIndex = SortedDict()

        for i, (start, end) in enumerate(intervals):
            startToIndex[start] = i

        for start, end in intervals:
            i = startToIndex.bisect_left(end)
            ans.append(-1 if i == len(startToIndex) else startToIndex.peekitem(i)[1])

        return ans
```

2632. 437.py

Path: LeetCode-main/solutions/437. Path Sum III/437.py

```
class Solution:
    def pathSum(self, root: TreeNode | None, summ: int) -> int:
        if not root:
            return 0

        def dfs(root: TreeNode, summ: int) -> int:
            if not root:
                return 0
            return (int(summ == root.val) +
                    dfs(root.left, summ - root.val) +
                    dfs(root.right, summ - root.val))

        return (dfs(root, summ) +
                self.pathSum(root.left, summ) +
                self.pathSum(root.right, summ))
```

2633. 438.py

Path: LeetCode-main/solutions/438. Find All Anagrams in a String/438.py

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> list[int]:
        ans = []
        count = collections.Counter(p)
        required = len(p)

        for r, c in enumerate(s):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            if r >= len(p):
                count[s[r - len(p)]] += 1
                if count[s[r - len(p)]] > 0:
                    required += 1
            if required == 0:
                ans.append(r - len(p) + 1)

        return ans
```

2634. 439.py

Path: LeetCode-main/solutions/439. Ternary Expression Parser/439.py

```
class Solution:
    def parseTernary(self, expression: str) -> str:
        c = expression[self.i]

        if self.i + 1 == len(expression) or expression[self.i + 1] == ':':
            self.i += 2
            return str(c)

        self.i += 2
        first = self.parseTernary(expression)
        second = self.parseTernary(expression)

        return first if c == 'T' else second

    i = 0
```

2635. 44.py

Path: LeetCode-main/solutions/44. Wildcard Matching/44.py

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m = len(s)
        n = len(p)
        # dp[i][j] := True if s[0..i) matches p[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return i >= 0 and p[j] == '?' or s[i] == p[j]

        for j, c in enumerate(p):
            if c == '*':
                dp[0][j + 1] = dp[0][j]

        for i in range(m):
            for j in range(n):
                if p[j] == '*':
                    matchEmpty = dp[i + 1][j]
                    matchSome = dp[i][j + 1]
                    dp[i + 1][j + 1] = matchEmpty or matchSome
                elif isMatch(i, j):
                    dp[i + 1][j + 1] = dp[i][j]

        return dp[m][n]

```

2636. 440.py

Path: LeetCode-main/solutions/440. K-th Smallest in Lexicographical Order/440.py

```

class Solution:
    def findKthNumber(self, n: int, k: int) -> int:
        ans = 1

        i = 1
        while i < k:
            gap = self._getGap(ans, ans + 1, n)
            if i + gap <= k:
                i += gap
                ans += 1
            else:
                i += 1
                ans *= 10

        return ans

    def _getGap(self, a: int, b: int, n: int) -> int:
        gap = 0
        while a <= n:
            gap += min(n + 1, b) - a
            a *= 10
            b *= 10
        return gap

```

2637. 441.py

Path: LeetCode-main/solutions/441. Arranging Coins/441.py


```
class Solution:
    def arrangeCoins(self, n: int) -> int:
        return int((-1 + math.sqrt(8 * n + 1)) // 2)
```

2638. 442.py

Path: LeetCode-main/solutions/442. Find All Duplicates in an Array/442.py

```
class Solution:
    def findDuplicates(self, nums: list[int]) -> list[int]:
        ans = []

        for num in nums:
            nums[abs(num) - 1] *= -1
            if nums[abs(num) - 1] > 0:
                ans.append(abs(num))

        return ans
```

2639. 443.py

Path: LeetCode-main/solutions/443. String Compression/443.py

```
class Solution:
    def compress(self, chars: list[str]) -> int:
        ans = 0
        i = 0

        while i < len(chars):
            letter = chars[i]
            count = 0
            while i < len(chars) and chars[i] == letter:
                count += 1
                i += 1
            chars[ans] = letter
            ans += 1
            if count > 1:
                for c in str(count):
                    chars[ans] = c
                    ans += 1

        return ans
```

2640. 444.py

Path: LeetCode-main/solutions/444. Sequence Reconstruction/444.py

```

class Solution:
    def sequenceReconstruction(
        self,
        org: list[int],
        seqs: list[list[int]],
    ) -> bool:
        if not seqs:
            return False

        n = len(org)
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n

        # Build the graph.
        for seq in seqs:
            if len(seq) == 1 and seq[0] < 1 or seq[0] > n:
                return False
            for u, v in zip(seq, seq[1:]):
                if u < 1 or u > n or v < 1 or v > n:
                    return False
                graph[u - 1].append(v - 1)
                inDegrees[v - 1] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])
        i = 0 # org's index

        while q:
            if len(q) > 1:
                return False
            u = q.popleft()
            if u != org[i] - 1:
                return False
            i += 1
            for v in graph[u]:
                inDegrees[v] -= 1
                if inDegrees[v] == 0:
                    q.append(v)

        return i == n

```

2641. 445.py

Path: LeetCode-main/solutions/445. Add Two Numbers II/445.py

```

class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        stack1 = []
        stack2 = []

        while l1:
            stack1.append(l1.val)
            l1 = l1.next

        while l2:
            stack2.append(l2.val)
            l2 = l2.next

        head = None
        carry = 0

        while carry or stack1 or stack2:
            if stack1:
                carry += stack1.pop().val
            if stack2:
                carry += stack2.pop().val
            node = ListNode(carry % 10)
            node.next = head
            head = node
            carry //= 10

        return head

```

2642. 446.py

Path: LeetCode-main/solutions/446. Arithmetic Slices II - Subsequence/446.py

```

class Solution:
    def numberOfArithmeticSlices(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        # dp[i][j] := the number of subsequences end in nums[j] nums[i]
        dp = [[0] * n for _ in range(n)]
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums):
            numToIndices[num].append(i)

        for i in range(n):
            for j in range(i):
                target = nums[j] * 2 - nums[i]
                if target in numToIndices:
                    for k in numToIndices[target]:
                        if k < j:
                            dp[i][j] += dp[j][k] + 1
            ans += dp[i][j]

        return ans

```

2643. 447.py

Path: LeetCode-main/solutions/447. Number of Boomerangs/447.py

```
class Solution:
    def numberOfBoomerangs(self, points: list[list[int]]) -> int:
        ans = 0

        for x1, y1 in points:
            count = collections.Counter()
            for x2, y2 in points:
                ans += 2 * count[(x1 - x2)**2 + (y1 - y2)**2]
                count[(x1 - x2)**2 + (y1 - y2)**2] += 1

        return ans
```

2644. 448.py

Path: LeetCode-main/solutions/448. Find All Numbers Disappeared in an Array/448.py

```
class Solution:
    def findDisappearedNumbers(self, nums: list[int]) -> list[int]:
        for num in nums:
            index = abs(num) - 1
            nums[index] = -abs(nums[index])

        return [i + 1 for i, num in enumerate(nums) if num > 0]
```

2645. 449.py

Path: LeetCode-main/solutions/449. Serialize and Deserialize BST/449.py

```

class Codec:
    def serialize(self, root: TreeNode | None) -> str:
        """Encodes a tree to a single string."""
        if not root:
            return ''
        chars = []
        self._serialize(root, chars)
        return ''.join(chars)

    def deserialize(self, data: str) -> TreeNode | None:
        """Decodes your encoded data to tree."""
        if not data:
            return None
        q = collections.deque(int(val) for val in data.split())
        return self._deserialize(-math.inf, math.inf, q)

    def _serialize(self, root: TreeNode | None, chars: list[str]) -> None:
        if not root:
            return
        chars.append(str(root.val))
        chars.append(' ')
        self._serialize(root.left, chars)
        self._serialize(root.right, chars)

    def _deserialize(
        self,
        mn: int,
        mx: int,
        q: collections.deque[int]
    ) -> TreeNode | None:
        if not q:
            return None

        val = q[0]
        if val < mn or val > mx:
            return None

        q.popleft()
        return TreeNode(val,
                        self._deserialize(mn, val, q),
                        self._deserialize(val, mx, q))

```

2646. 45.py

Path: LeetCode-main/solutions/45. Jump Game II/45.py

```
class Solution:
    def jump(self, nums: list[int]) -> int:
        ans = 0
        end = 0
        farthest = 0

        # Start an implicit BFS.
        for i in range(len(nums) - 1):
            farthest = max(farthest, i + nums[i])
            if farthest >= len(nums) - 1:
                ans += 1
                break
            if i == end:      # Visited all the items on the current level.
                ans += 1     # Increment the level.
                end = farthest # Make the queue size for the next level.

        return ans
```

2647. 450.py

Path: LeetCode-main/solutions/450. Delete Node in a BST/450.py

```
class Solution:
    def deleteNode(self, root: TreeNode | None, key: int) -> TreeNode | None:
        if not root:
            return None
        if root.val == key:
            if not root.left:
                return root.right
            if not root.right:
                return root.left
            minNode = self._getMin(root.right)
            root.right = self.deleteNode(root.right, minNode.val)
            minNode.left = root.left
            minNode.right = root.right
            root = minNode
        elif root.val < key:
            root.right = self.deleteNode(root.right, key)
        else: # root.val > key
            root.left = self.deleteNode(root.left, key)
        return root

    def _getMin(self, node: TreeNode | None) -> TreeNode | None:
        while node.left:
            node = node.left
        return node
```

2648. 451.py

Path: LeetCode-main/solutions/451. Sort Characters By Frequency/451.py

```
class Solution:
    def frequencySort(self, s: str) -> str:
        ans = []
        buckets = [[] for _ in range(len(s) + 1)]

        for c, freq in collections.Counter(s).items():
            buckets[freq].append(c)

        for freq in reversed(range(len(buckets))):
            for c in buckets[freq]:
                ans.append(c * freq)

        return ''.join(ans)
```

2649. 452.py

Path: LeetCode-main/solutions/452. Minimum Number of Arrows to Burst Balloons/452.py

```
class Solution:
    def findMinArrowShots(self, points: list[list[int]]) -> int:
        ans = 0
        arrowX = -math.inf

        for point in sorted(points, key=lambda x: x[1]):
            if point[0] > arrowX:
                ans += 1
                arrowX = point[1]

        return ans
```

2650. 453.py

Path: LeetCode-main/solutions/453. Minimum Moves to Equal Array Elements/453.py

```
class Solution:
    def minMoves(self, nums: list[int]) -> int:
        mn = min(nums)
        return sum(num - mn for num in nums)
```

2651. 454.py

Path: LeetCode-main/solutions/454. 4Sum II/454.py

```
class Solution:
    def fourSumCount(self, nums1: list[int], nums2: list[int],
                     nums3: list[int], nums4: list[int]) -> int:
        count = collections.Counter(a + b for a in nums1 for b in nums2)
        return sum(count[-c - d] for c in nums3 for d in nums4)
```

2652. 455.py

Path: LeetCode-main/solutions/455. Assign Cookies/455.py

```
class Solution:
    def findContentChildren(self, g: list[int], s: list[int]) -> int:
        g.sort()
        s.sort()

        i = 0
        for cookie in s:
            if i < len(g) and g[i] <= cookie:
                i += 1

        return i
```

2653. 456.py

Path: LeetCode-main/solutions/456. 132 Pattern/456.py

```
class Solution:
    def find132pattern(self, nums: list[int]) -> bool:
        stack = [] # a decreasing stack
        ak = -math.inf # Find a seq, where ai < ak < aj.

        for num in reversed(nums):
            # If ai < ak, done because ai must < aj.
            if num < ak:
                return True
            while stack and stack[-1] < num:
                ak = stack[-1]
                stack.pop()
            stack.append(num) # `nums[i]` is a candidate of aj.

        return False
```

2654. 457.py

Path: LeetCode-main/solutions/457. Circular Array Loop/457.py


```

class Solution:
    def circularArrayLoop(self, nums: list[int]) -> bool:
        def advance(i: int) -> int:
            return (i + nums[i]) % len(nums)

        if len(nums) < 2:
            return False

        for i, num in enumerate(nums):
            if num == 0:
                continue

            slow = i
            fast = advance(slow)
            while num * nums[fast] > 0 and num * nums[advance(fast)] > 0:
                if slow == fast:
                    if slow == advance(slow):
                        break
                    return True
                slow = advance(slow)
                fast = advance(advance(fast))

            slow = i
            sign = num
            while sign * nums[slow] > 0:
                next = advance(slow)
                nums[slow] = 0
                slow = next

        return False

```

2655. 458.py

Path: LeetCode-main/solutions/458. Poor Pigs/458.py

```

class Solution:
    def poorPigs(self, buckets: int, minutesToDie: int, minutesToTest: int) -> int:
        base = minutesToTest // minutesToDie + 1
        ans = 0
        x = 1
        while x < buckets:
            ans += 1
            x *= base
        return ans

```

2656. 459.py

Path: LeetCode-main/solutions/459. Repeated Substring Pattern/459.py

```

class Solution:
    def repeatedSubstringPattern(self, s: str) -> bool:
        return s in (s + s)[1:-1]

```

2657. 46.py

Path: LeetCode-main/solutions/46. Permutations/46.py

```
class Solution:
    def permute(self, nums: list[int]) -> list[list[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: list[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

            for i, num in enumerate(nums):
                if used[i]:
                    continue
                used[i] = True
                path.append(num)
                dfs(path)
                path.pop()
                used[i] = False

        dfs([])
        return ans
```

2658. 460.py

Path: LeetCode-main/solutions/460. LFU Cache/460.py

```

class Node:
    def __init__(self, key: int, value: int, freq: int, it):
        self.key = key
        self.value = value
        self.freq = freq
        self.it = it

class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.minFreq = 0
        self.keyToNode = {}
        self.freqToList = {}

    def get(self, key: int) -> int:
        if key not in self.keyToNode:
            return -1

        node = self.keyToNode[key]
        self._touch(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return

        if key in self.keyToNode:
            node = self.keyToNode[key]
            node.value = value
            self._touch(node)
            return

        if len(self.keyToNode) == self.capacity:
            # Evict an LRU key from `minFreq` list
            keyToEvict = self.freqToList[self.minFreq][-1]
            self.freqToList[self.minFreq].pop()
            del self.keyToNode[keyToEvict]

        self.minFreq = 1
        if 1 not in self.freqToList:
            self.freqToList[1] = []
        self.freqToList[1].insert(0, key)
        self.keyToNode[key] = Node(key, value, 1, 0) # Using index 0 as iterator

    def _touch(self, node: Node) -> None:
        # Update the node's frequency
        prevFreq = node.freq
        node.freq += 1
        newFreq = node.freq

        # Remove the key from `prevFreq`'s list
        self.freqToList[prevFreq].remove(node.key)
        if not self.freqToList[prevFreq]:
            del self.freqToList[prevFreq]
        # Update `minFreq` if needed
        if prevFreq == self.minFreq:
            self.minFreq += 1

        # Insert the key to the front of `newFreq`'s list
        if newFreq not in self.freqToList:
            self.freqToList[newFreq] = []
        self.freqToList[newFreq].insert(0, node.key)
        node.it = 0 # Using index 0 as iterator

```

2659. 461.py

Path: LeetCode-main/solutions/461. Hamming Distance/461.py

```
class Solution:
    def hammingDistance(self, x: int, y: int) -> int:
        ans = 0

        while x > 0 or y > 0:
            ans += (x & 1) ^ (y & 1)
            x >>= 1
            y >>= 1

        return ans
```

2660. 462.py

Path: LeetCode-main/solutions/462. Minimum Moves to Equal Array Elements II/462.py

```
import statistics

class Solution:
    def minMoves2(self, nums: list[int]) -> int:
        median = int(statistics.median(nums))
        return sum(abs(num - median) for num in nums)
```

2661. 463.py

Path: LeetCode-main/solutions/463. Island Perimeter/463.py

```
class Solution:
    def islandPerimeter(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        islands = 0
        neighbors = 0

        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    islands += 1
                    if i + 1 < m and grid[i + 1][j] == 1:
                        neighbors += 1
                    if j + 1 < n and grid[i][j + 1] == 1:
                        neighbors += 1

        return islands * 4 - neighbors * 2
```

2662. 464.py

Path: LeetCode-main/solutions/464. Can I Win/464.py

```

class Solution:
    def canIWin(self, maxChoosableInteger: int, desiredTotal: int) -> bool:
        if desiredTotal <= 0:
            return True

        totalSum = maxChoosableInteger * (maxChoosableInteger + 1) // 2
        if totalSum < desiredTotal:
            return False

        @functools.lru_cache(None)
        def dp(total: int, used: int) -> bool:
            """
            Returns true if the first player can win, where `used` represents the used
            numbers.
            """
            if total <= 0:
                return False
            return any((used >> i & 1) == 0
                       and not dp(total - i, used | 1 << i)
                       for i in range(1, maxChoosableInteger + 1))

        return dp(desiredTotal, 0)

```

2663. 465.py

Path: LeetCode-main/solutions/465. Optimal Account Balancing/465.py

```

class Solution:
    def minTransfers(self, transactions: list[list[int]]) -> int:
        balance = [0] * 21

        for u, v, amount in transactions:
            balance[u] -= amount
            balance[v] += amount

        debts = [b for b in balance if b]

        def dfs(s: int) -> int:
            while s < len(debts) and not debts[s]:
                s += 1
            if s == len(debts):
                return 0

            ans = math.inf

            for i in range(s + 1, len(debts)):
                if debts[i] * debts[s] < 0:
                    debts[i] += debts[s] # `debts[s]` is settled.
                    ans = min(ans, 1 + dfs(s + 1))
                    debts[i] -= debts[s] # Backtrack.

            return ans

        return dfs(0)

```

2664. 466.py

Path: LeetCode-main/solutions/466. Count The Repetitions/466.py

```

from dataclasses import dataclass

@dataclass
class Record:
    count: int
    nextIndex: int

class Solution:
    def getMaxRepetitions(self, s1: str, n1: int, s2: str, n2: int) -> int:
        # records[i].count := the number of times that s2 starting from index i has
        # been fully matched with s1
        # records[i].nextIndex := the next index in s2 to be matched after
        # completing a full match starting from index i
        records = []

        for nextIndex in range(len(s2)):
            count = 0
            for c in s1:
                if s2[nextIndex] == c:
                    nextIndex += 1
                    if nextIndex == len(s2): # There's a match.
                        count += 1
                        nextIndex = 0
            records.append(Record(count, nextIndex))

        matches = 0 # the number of matches between `s1` x n1 and `s2`
        i = 0 # the index in `s2` to start matching

        for _ in range(n1):
            matches += records[i].count
            i = records[i].nextIndex

        return matches // n2

```

2665. 467.py

Path: LeetCode-main/solutions/467. Unique Substrings in Wraparound String/467.py

```

class Solution:
    def findSubstringInWraparoundString(self, s: str) -> int:
        maxLength = 1
        # count[i] := the number of substrings ending in ('a' + i)
        count = [0] * 26

        for i in range(len(s)):
            if i > 0 and (ord(s[i]) - ord(s[i - 1]) == 1
                        or ord(s[i - 1]) - ord(s[i]) == 25):
                maxLength += 1
            else:
                maxLength = 1
            index = ord(s[i]) - ord('a')
            count[index] = max(count[index], maxLength)

        return sum(count)

```

2666. 468.py

Path: LeetCode-main/solutions/468. Validate IP Address/468.py

```
class Solution:
    def validIPAddress(self, queryIP: str) -> str:
        if queryIP.count('.') == 3:
            parts = queryIP.split('.')
            if len(parts) == 4 and all(self._isIPv4(part) for part in parts):
                return 'IPv4'

        if queryIP.count(':') == 7:
            parts = queryIP.split(':')
            if len(parts) == 8 and all(self._isIPv6(part) for part in parts):
                return 'IPv6'

        return 'Neither'

    def _isIPv4(self, digit: str) -> bool:
        if not digit or len(digit) > 3:
            return False
        if len(digit) > 1 and digit[0] == '0':
            return False
        if not all(c.isdigit() for c in digit):
            return False
        num = int(digit)
        return 0 <= num <= 255

    def _isIPv6(self, digit: str) -> bool:
        if not digit or len(digit) > 4:
            return False
        validChars = set('0123456789abcdefABCDEF')
        return all(c in validChars for c in digit)
```

2667. 469.py

Path: LeetCode-main/solutions/469. Convex Polygon/469.py

```
class Solution:
    def isConvex(self, points: list[list[int]]) -> bool:
        def getCross(p: list[int], q: list[int], r: list[int]):
            """Returns pq x qr."""
            return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0])

        sign = 0
        for i in range(len(points)):
            cross = getCross(points[i - 2], points[i - 1], points[i])
            if cross == 0: # p, q, r are collinear.
                continue
            if sign == 0: # Find the first cross that's not 0.
                sign = cross
            elif cross * sign < 0:
                return False

        return True
```

2668. 47.py

Path: LeetCode-main/solutions/47. Permutations II/47.py

```

class Solution:
    def permuteUnique(self, nums: list[int]) -> list[list[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: list[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

            for i, num in enumerate(nums):
                if used[i]:
                    continue
                if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                    continue
                used[i] = True
                path.append(num)
                dfs(path)
                path.pop()
                used[i] = False

        nums.sort()
        dfs([])
        return ans

```

2669. 470.py

Path: LeetCode-main/solutions/470. Implement Rand10() Using Rand7()/470.py

```

# The rand7() API is already defined for you.
# def rand7():
# @return a random integer in the range 1 to 7

class Solution:
    def rand10(self) -> int:
        num = 40
        while num >= 40:
            num = (rand7() - 1) * 7 + (rand7() - 1)
        return num % 10 + 1

```

2670. 471-2.py

Path: LeetCode-main/solutions/471. Encode String with Shortest Length/471-2.py


```

class Solution:
    def encode(self, s: str) -> str:
        n = len(s)
        # dp[i][j] := the shortest encoded string of s[i..j]
        dp = [[''] * n for _ in range(n)]

        for d in range(n):
            for i in range(n - d):
                j = i + d
                curr = s[i:j + 1]
                dp[i][j] = curr

                if len(dp[i][j]) < 5:
                    continue

                # Try all the possible partitions.
                for k in range(i, j):
                    if len(dp[i][k]) + len(dp[k + 1][j]) < len(dp[i][j]):
                        dp[i][j] = dp[i][k] + dp[k + 1][j]

                # Try to compress the string.
                # e.g. s = aabaabaab -> 3[aab]
                for k in range(i, j):
                    pattern = s[i:k + 1]
                    if len(curr) % len(pattern) == 0 and pattern * (len(curr) //
                                                                    len(pattern)) == curr:
                        candidate = f"{len(curr) // len(pattern)}[{dp[i][k]}]"
                        if len(candidate) < len(dp[i][j]):
                            dp[i][j] = candidate

        return dp[0][n - 1]

```

2671. 471.py

Path: LeetCode-main/solutions/471. Encode String with Shortest Length/471.py

```

class Solution:
    def encode(self, s: str) -> str:
        n = len(s)

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> str:
            """Returns the shortest encoded string of s[i..j]."""
            curr = s[i:j + 1]
            res = curr

            if len(res) < 5:
                return res

            # Try all possible partitions.
            for k in range(i, j):
                l = dp(i, k)
                r = dp(k + 1, j)
                if len(l) + len(r) < len(res):
                    res = l + r

            # Try to compress the string.
            # e.g. s = aabaabaab -> 3[aab]
            for k in range(i, j):
                pattern = s[i:k + 1]
                if len(curr) % len(pattern) == 0 and pattern * (len(curr) //
                                                                len(pattern)) == curr:
                    candidate = f"{len(curr) // len(pattern)}[{dp(i, k)}]"
                    if len(candidate) < len(res):
                        res = candidate

            return res

        return dp(0, n - 1)

```

2672. 472.py

Path: LeetCode-main/solutions/472. Concatenated Words/472.py

```

class Solution:
    def findAllConcatenatedWordsInADict(self, words: list[str]) -> list[str]:
        wordSet = set(words)

        @functools.lru_cache(None)
        def isConcat(word: str) -> bool:
            for i in range(1, len(word)):
                prefix = word[:i]
                suffix = word[i:]
                if prefix in wordSet and (suffix in wordSet or isConcat(suffix)):
                    return True

            return False

        return [word for word in words if isConcat(word)]

```

2673. 473.py

Path: LeetCode-main/solutions/473. Matchsticks to Square/473.py

```

class Solution:
    def makesquare(self, matchsticks: list[int]) -> bool:
        if len(matchsticks) < 4:
            return False

        perimeter = sum(matchsticks)
        if perimeter % 4 != 0:
            return False

        A = sorted(matchsticks)[::-1]

        def dfs(selected: int, edges: list[int]) -> bool:
            if selected == len(A):
                return all(edge == edges[0] for edge in edges)

            for i, edge in enumerate(edges):
                if A[selected] > edge:
                    continue
                edges[i] -= A[selected]
                if dfs(selected + 1, edges):
                    return True
                edges[i] += A[selected]

            return False

        return dfs(0, [perimeter // 4] * 4)

```

2674. 474.py

Path: LeetCode-main/solutions/474. Ones and Zeroes/474.py

```

class Solution:
    def findMaxForm(self, strs: list[str], m: int, n: int) -> int:
        # dp[i][j] := the maximum size of the subset given i 0s and j 1s are
        # available
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for s in strs:
            zeros = s.count('0')
            ones = len(s) - zeros
            for i in range(m, zeros - 1, -1):
                for j in range(n, ones - 1, -1):
                    dp[i][j] = max(dp[i][j], dp[i - zeros][j - ones] + 1)

        return dp[m][n]

```

2675. 477.py

Path: LeetCode-main/solutions/477. Total Hamming Distance/477.py

```

class Solution:
    def totalHammingDistance(self, nums: list[int]) -> int:
        MAX_BIT = 30
        ans = 0

        for i in range(MAX_BIT):
            ones = sum(num & (1 << i) > 0 for num in nums)
            zeros = len(nums) - ones
            ans += ones * zeros

        return ans

```

2676. 478.py

Path: LeetCode-main/solutions/478. Generate Random Point in a Circle/478.py

```

class Solution:
    def __init__(self, radius: float, x_center: float, y_center: float):
        self.radius = radius
        self.x_center = x_center
        self.y_center = y_center

    def randPoint(self) -> list[float]:
        length = math.sqrt(random.uniform(0, 1)) * self.radius
        degree = random.uniform(0, 1) * 2 * math.pi
        x = self.x_center + length * math.cos(degree)
        y = self.y_center + length * math.sin(degree)
        return [x, y]

```

2677. 479.py

Path: LeetCode-main/solutions/479. Largest Palindrome Product/479.py

```

class Solution:
    def largestPalindrome(self, n: int) -> int:
        if n == 1:
            return 9

        MOD = 1337
        upper = pow(10, n) - 1
        lower = pow(10, n - 1) - 1

        for i in range(upper, lower, -1):
            cand = int(str(i) + str(i)[::-1])
            j = upper
            while j * j >= cand:
                if cand % j == 0:
                    return cand % MOD
                j -= 1

```

2678. 48-2.py

Path: LeetCode-main/solutions/48. Rotate Image/48-2.py

```

class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        for mn in range(len(matrix) // 2):
            mx = len(matrix) - mn - 1
            for i in range(mn, mx):
                offset = i - mn
                top = matrix[mn][i]
                matrix[mn][i] = matrix[mx - offset][mn]
                matrix[mx - offset][mn] = matrix[mx][mx - offset]
                matrix[mx][mx - offset] = matrix[i][mx]
                matrix[i][mx] = top

```

2679. 48.py

Path: LeetCode-main/solutions/48. Rotate Image/48.py

```

class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        matrix.reverse()
        for i, j in itertools.combinations(range(len(matrix)), 2):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

```

2680. 481.py

Path: LeetCode-main/solutions/481. Magical String/481.py

```

class Solution:
    def magicalString(self, n: int) -> int:
        s = [' ', '1', '2', '2']

        for i in range(3, n + 1):
            if i % 2 == 1:
                s.extend(['1'] * (int(s[i])))
            else:
                s.extend(['2'] * (int(s[i])))

        return sum(1 for c in s[:n + 1] if c == '1')

```

2681. 482.py

Path: LeetCode-main/solutions/482. License Key Formatting/482.py

```

class Solution:
    def licenseKeyFormatting(self, s: str, k: int) -> str:
        ans = []
        length = 0

        for i in reversed(range(len(s))):
            if s[i] == '-':
                continue
            if length > 0 and length % k == 0:
                ans += '-'
            ans += s[i].upper()
            length += 1

        return ''.join(reversed(ans))

```

2682. 483.py

Path: LeetCode-main/solutions/483. Smallest Good Base/483.py

```
class Solution:
    def smallestGoodBase(self, n: str) -> str:
        n = int(n)

        for m in range(int(math.log(n, 2)), 1, -1):
            k = int(n**(m+1)-1)
            if (k**(m+1) - 1) // (k - 1) == n:
                return str(k)

        return str(n - 1)
```

2683. 484-2.py

Path: LeetCode-main/solutions/484. Find Permutation/484-2.py

```
class Solution:
    def findPermutation(self, s: str) -> list[int]:
        ans = [i for i in range(1, len(s) + 2)]

        # For each D* group (s[i..j]), reverse ans[i..j + 1].
        i = -1
        j = -1

        def getNextIndex(c: str, start: int) -> int:
            for i in range(start, len(s)):
                if s[i] == c:
                    return i
            return len(s)

        while True:
            i = getNextIndex('D', j + 1)
            if i == len(s):
                break
            j = getNextIndex('I', i + 1)
            ans[i:j + 1] = ans[i:j + 1][::-1]

        return ans
```

2684. 484.py

Path: LeetCode-main/solutions/484. Find Permutation/484.py

```
class Solution:
    def findPermutation(self, s: str) -> list[int]:
        ans = []
        stack = []

        for i, c in enumerate(s):
            stack.append(i + 1)
            if c == 'I':
                while stack: # Consume all decreasing
                    ans.append(stack.pop())
            stack.append(len(s) + 1)

        while stack:
            ans.append(stack.pop())

        return ans
```

2685. 485.py

Path: LeetCode-main/solutions/485. Max Consecutive Ones/485.py

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        ans = 0
        summ = 0

        for num in nums:
            if num == 0:
                summ = 0
            else:
                summ += num
                ans = max(ans, summ)

        return ans
```

2686. 487-2.py

Path: LeetCode-main/solutions/487. Max Consecutive Ones II/487-2.py

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        maxZeros = 1
        ans = 0
        q = collections.deque() # Store indices of zero.

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                q.append(r)
            if len(q) > maxZeros:
                l = q.popleft() + 1
            ans = max(ans, r - l + 1)

        return ans
```

2687. 487-3.py

Path: LeetCode-main/solutions/487. Max Consecutive Ones II/487-3.py

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        ans = 0
        lastZeroIndex = -1

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                l = lastZeroIndex + 1
                lastZeroIndex = r
            ans = max(ans, r - l + 1)

        return ans
```

2688. 487.py

Path: LeetCode-main/solutions/487. Max Consecutive Ones II/487.py

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        ans = 0
        zeros = 0

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                zeros += 1
            while zeros == 2:
                if nums[l] == 0:
                    zeros -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

2689. 488.py

Path: LeetCode-main/solutions/488. Zuma Game/488.py


```

class Solution:
    def findMinStep(self, board: str, hand: str) -> int:
        def deDup(board):
            start = 0 # the start index of a color sequence
            for i, c in enumerate(board):
                if c != board[start]:
                    if i - start >= 3:
                        return deDup(board[:start] + board[i:])
                    start = i # Meet a new sequence.
            return board

        @functools.lru_cache(None)
        def dfs(board: str, hand: str):
            board = deDup(board)
            if board == '#':
                return 0

            boardSet = set(board)
            # hand that is in board
            hand = ''.join(h for h in hand if h in boardSet)
            if not hand: # infeasible
                return math.inf

            ans = math.inf

            for i in range(len(board)):
                for j, h in enumerate(hand):
                    # Place hs[j] in board[i].
                    newHand = hand[:j] + hand[j + 1:]
                    newBoard = board[:i] + h + board[i:]
                    ans = min(ans, 1 + dfs(newBoard, newHand))

            return ans

        ans = dfs(board + '#', hand)
        return -1 if ans == math.inf else ans

```

2690. 49.py

Path: LeetCode-main/solutions/49. Group Anagrams/49.py

```

class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        dict = collections.defaultdict(list)

        for str in strs:
            key = ''.join(sorted(str))
            dict[key].append(str)

        return dict.values()

```

2691. 490-2.py

Path: LeetCode-main/solutions/490. The Maze/490-2.py

```

class Solution:
    def hasPath(
        self,
        maze: list[list[int]],
        start: list[int],
        destination: list[int],
    ) -> bool:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(maze)
        n = len(maze[0])

        seen = set()

        def isValid(x: int, y: int) -> bool:
            return 0 <= x < m and 0 <= y < n and maze[x][y] == 0

        def dfs(i: int, j: int) -> bool:
            if [i, j] == destination:
                return True
            if (i, j) in seen:
                return False

            seen.add((i, j))

            for dx, dy in DIRS:
                x = i
                y = j
                while isValid(x + dx, y + dy):
                    x += dx
                    y += dy
                    if dfs(x, y):
                        return True

            return False

        return dfs(start[0], start[1])

```

2692. 490.py

Path: LeetCode-main/solutions/490. The Maze/490.py

```

class Solution:
    def hasPath(
        self,
        maze: list[list[int]],
        start: list[int],
        destination: list[int],
    ) -> bool:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(maze)
        n = len(maze[0])
        q = collections.deque([(start[0], start[1])])
        seen = {(start[0], start[1])}

        def isValid(x: int, y: int) -> bool:
            return 0 <= x < m and 0 <= y < n and maze[x][y] == 0

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i
                y = j
                while isValid(x + dx, y + dy):
                    x += dx
                    y += dy
                    if [x, y] == destination:
                        return True
                    if (x, y) in seen:
                        continue
                    q.append((x, y))
                    seen.add((x, y))

        return False

```

2693. 491.py

Path: LeetCode-main/solutions/491. Increasing Subsequences/491.py

```

class Solution:
    def findSubsequences(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            if len(path) > 1:
                ans.append(path)

            used = set()

            for i in range(s, len(nums)):
                if nums[i] in used:
                    continue
                if not path or nums[i] >= path[-1]:
                    used.add(nums[i])
                    dfs(i + 1, path + [nums[i]])

        dfs(0, [])
        return ans

```

2694. 494-2.py

Path: LeetCode-main/solutions/494. Target Sum/494-2.py

```
class Solution:
    def findTargetSumWays(self, nums: list[int], target: int) -> int:
        summ = sum(nums)
        if summ < abs(target) or (summ + target) % 2 == 1:
            return 0

        def knapsack(nums: list[int], target: int) -> int:
            # dp[i] := the number of ways to sum to i by nums so far
            dp = [0] * (target + 1)
            dp[0] = 1

            for num in nums:
                for i in range(target, num - 1, -1):
                    dp[i] += dp[i - num]

            return dp[target]

        return knapsack(nums, (summ + target) // 2)
```

2695. 494.py

Path: LeetCode-main/solutions/494. Target Sum/494.py

```
class Solution:
    def findTargetSumWays(self, nums: list[int], target: int) -> int:
        summ = sum(nums)
        if summ < abs(target) or (summ + target) % 2 == 1:
            return 0

        def knapsack(target: int) -> int:
            # dp[i] := the number of ways to sum to i by nums so far
            dp = [1] + [0] * summ

            for num in nums:
                for j in range(summ, num - 1, -1):
                    dp[j] += dp[j - num]

            return dp[target]

        return knapsack((summ + target) // 2)
```

2696. 495.py

Path: LeetCode-main/solutions/495. Teemo Attacking/495.py

```
class Solution:
    def findPoisonedDuration(self, timeSeries: list[int], duration: int) -> int:
        if duration == 0:
            return 0

        ans = 0

        for i in range(0, len(timeSeries) - 1):
            ans += min(timeSeries[i + 1] - timeSeries[i], duration)

        return ans + duration
```

2697. 496.py

Path: LeetCode-main/solutions/496. Next Greater Element I/496.py

```
class Solution:
    def nextGreaterElement(self, nums1: list[int], nums2: list[int]) -> list[int]:
        numToNextGreater = {}
        stack = [] # a decreasing stack

        for num in nums2:
            while stack and stack[-1] < num:
                numToNextGreater[stack.pop()] = num
            stack.append(num)

        return [numToNextGreater.get(num, -1) for num in nums1]
```

2698. 497.py

Path: LeetCode-main/solutions/497. Random Point in Non-overlapping Rectangles/497.py

```
class Solution:
    def __init__(self, rects: list[list[int]]):
        self.rects = rects
        self.areas = list(itertools.accumulate(
            [(x2 - x1 + 1) * (y2 - y1 + 1) for x1, y1, x2, y2 in rects]))

    def pick(self) -> list[int]:
        index = bisect_right(self.areas, random.randint(0, self.areas[-1] - 1))
        x1, y1, x2, y2 = self.rects[index]
        return [random.randint(x1, x2), random.randint(y1, y2)]
```

2699. 499.py

Path: LeetCode-main/solutions/499. The Maze III/499.py

```

class Solution:
    def findShortestWay(
        self,
        maze: list[list[int]],
        ball: list[int],
        hole: list[int],
    ) -> str:
        ans = 'impossible'
        minSteps = math.inf

        def dfs(i: int, j: int, dx: int, dy: int, steps: int, path: str):
            nonlocal ans
            nonlocal minSteps
            if steps >= minSteps:
                return

            if dx != 0 or dy != 0: # Both are zeros for the initial ball position.
                while (0 <= i + dx < len(maze) and 0 <= j + dy < len(maze[0]) and
                    maze[i + dx][j + dy] != 1):
                    i += dx
                    j += dy
                    steps += 1
                    if i == hole[0] and j == hole[1] and steps < minSteps:
                        minSteps = steps
                        ans = path

            if maze[i][j] == 0 or steps + 2 < maze[i][j]:
                maze[i][j] = steps + 2 # +2 because maze[i][j] == 0 || 1.
                if dx == 0:
                    dfs(i, j, 1, 0, steps, path + 'd')
                if dy == 0:
                    dfs(i, j, 0, -1, steps, path + 'l')
                if dy == 0:
                    dfs(i, j, 0, 1, steps, path + 'r')
                if dx == 0:
                    dfs(i, j, -1, 0, steps, path + 'u')

        dfs(ball[0], ball[1], 0, 0, 0, '')
        return ans

```

2700. 5-2.py

Path: LeetCode-main/solutions/5. Longest Palindromic Substring/5-2.py

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        t = '#'.join('@' + s + '$')
        p = self._manacher(t)
        maxPalindromeLength, bestCenter = max((extend, i)
                                              for i, extend in enumerate(p))

        l = (bestCenter - maxPalindromeLength) // 2
        r = (bestCenter + maxPalindromeLength) // 2
        return s[l:r]

    def _manacher(self, t: str) -> list[int]:
        """
        Returns an array `p` s.t. `p[i]` is the length of the longest palindrome
        centered at `t[i]`, where `t` is a string with delimiters and sentinels.
        """
        p = [0] * len(t)
        center = 0
        for i in range(1, len(t) - 1):
            rightBoundary = center + p[center]
            mirrorIndex = center - (i - center)
            if rightBoundary > i:
                p[i] = min(rightBoundary - i, p[mirrorIndex])
            # Try to expand the palindrome centered at i.
            while t[i + 1 + p[i]] == t[i - 1 - p[i]]:
                p[i] += 1
            # If a palindrome centered at i expands past `rightBoundary`, adjust
            # the center based on the expanded palindrome.
            if i + p[i] > rightBoundary:
                center = i
        return p

```

2701. 5.py

Path: LeetCode-main/solutions/5. Longest Palindromic Substring/5.py

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ''

        # (start, end) indices of the longest palindrome in s
        indices = [0, 0]

        def extend(s: str, i: int, j: int) -> tuple[int, int]:
            """
            Returns the (start, end) indices of the longest palindrome extended from
            the substring s[i..j].
            """
            while i >= 0 and j < len(s):
                if s[i] != s[j]:
                    break
                i -= 1
                j += 1
            return i + 1, j - 1

        for i in range(len(s)):
            l1, r1 = extend(s, i, i)
            if r1 - l1 > indices[1] - indices[0]:
                indices = l1, r1
            if i + 1 < len(s) and s[i] == s[i + 1]:
                l2, r2 = extend(s, i, i + 1)
                if r2 - l2 > indices[1] - indices[0]:
                    indices = l2, r2

        return s[indices[0]:indices[1] + 1]

```

2702. 50.py

Path: LeetCode-main/solutions/50. Pow(x, n)/50.py

```

class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n == 0:
            return 1
        if n < 0:
            return 1 / self.myPow(x, -n)
        if n % 2 == 1:
            return x * self.myPow(x, n - 1)
        return self.myPow(x * x, n // 2)

```

2703. 500.py

Path: LeetCode-main/solutions/500. Keyboard Row/500.py


```
class Solution:
    def findWords(self, words: list[str]) -> list[str]:
        ans = []
        rows = [set('qwertyuiop'), set('asdfghjkl'), set('zxcvbnm')]

        for word in words:
            lowerWord = set(word.lower())
            if any(lowerWord <= row for row in rows):
                ans.append(word)

        return ans
```

2704. 501.py

Path: LeetCode-main/solutions/501. Find Mode in Binary Search Tree/501.py

```
class Solution:
    def findMode(self, root: TreeNode | None) -> list[int]:
        self.ans = []
        self.pred = None
        self.count = 0
        self.maxCount = 0

    def updateCount(self, root: TreeNode | None) -> None:
        if self.pred and self.pred.val == root.val:
            self.count += 1
        else:
            self.count = 1

        if self.count > self.maxCount:
            self.maxCount = self.count
            self.ans = [root.val]
        elif self.count == self.maxCount:
            self.ans.append(root.val)

        self.pred = root

    def inorder(self, root: TreeNode | None) -> None:
        if not root:
            return

        inorder(root.left)
        updateCount(root)
        inorder(root.right)

    def findMode(self, root: TreeNode | None) -> list[int]:
        inorder(root)
        return self.ans
```

2705. 503.py

Path: LeetCode-main/solutions/503. Next Greater Element II/503.py

```
class Solution:
    def nextGreaterElements(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [-1] * n
        stack = [] # a decreasing stack storing indices

        for i in range(n * 2):
            num = nums[i % n]
            while stack and nums[stack[-1]] < num:
                ans[stack.pop()] = num
            if i < n:
                stack.append(i)

        return ans
```

2706. 507.py

Path: LeetCode-main/solutions/507. Perfect Number/507.py

```
class Solution:
    def checkPerfectNumber(self, num: int) -> bool:
        return num in {6, 28, 496, 8128, 33550336}
```

2707. 508.py

Path: LeetCode-main/solutions/508. Most Frequent Subtree Sum/508.py

```
class Solution:
    def findFrequentTreeSum(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        count = collections.Counter()

        def dfs(root: TreeNode | None) -> int:
            if not root:
                return 0

            summ = root.val + dfs(root.left) + dfs(root.right)
            count[summ] += 1
            return summ

        dfs(root)
        maxFreq = max(count.values())
        return [summ for summ in count if count[summ] == maxFreq]
```

2708. 509.py

Path: LeetCode-main/solutions/509. Fibonacci Number/509.py

```
class Solution:
    def fib(self, n: int) -> int:
        if n < 2:
            return n

        dp = [0, 0, 1]

        for i in range(2, n + 1):
            dp[0] = dp[1]
            dp[1] = dp[2]
            dp[2] = dp[0] + dp[1]

        return dp[2]
```

2709. 51.py

Path: LeetCode-main/solutions/51. N-Queens/51.py

```
class Solution:
    def solveNQueens(self, n: int) -> list[list[str]]:
        ans = []
        cols = [False] * n
        diag1 = [False] * (2 * n - 1)
        diag2 = [False] * (2 * n - 1)

        def dfs(i: int, board: list[int]) -> None:
            if i == n:
                ans.append(board)
                return

            for j in range(n):
                if cols[j] or diag1[i + j] or diag2[j - i + n - 1]:
                    continue
                cols[j] = diag1[i + j] = diag2[j - i + n - 1] = True
                dfs(i + 1, board + ['. ' * j + 'Q' + ' ' * (n - j - 1)])
                cols[j] = diag1[i + j] = diag2[j - i + n - 1] = False

        dfs(0, [])
        return ans
```

2710. 513-2.py

Path: LeetCode-main/solutions/513. Find Bottom Left Tree Value/513-2.py

```
class Solution:
    def findBottomLeftValue(self, root: TreeNode | None) -> int:
        ans = 0
        maxDepth = 0

        def dfs(root: TreeNode | None, depth: int) -> None:
            nonlocal ans
            nonlocal maxDepth
            if not root:
                return
            if depth > maxDepth:
                maxDepth = depth
                ans = root.val

            dfs(root.left, depth + 1)
            dfs(root.right, depth + 1)

        dfs(root, 1)
        return ans
```

2711. 513.py

Path: LeetCode-main/solutions/513. Find Bottom Left Tree Value/513.py

```
class Solution:
    def findBottomLeftValue(self, root: TreeNode | None) -> int:
        q = collections.deque([root])

        while q:
            root = q.popleft()
            if root.right:
                q.append(root.right)
            if root.left:
                q.append(root.left)

        return root.val
```

2712. 514.py

Path: LeetCode-main/solutions/514. Freedom Trail/514.py

```

class Solution:
    def findRotateSteps(self, ring: str, key: str) -> int:
        @functools.lru_cache(None)
        def dfs(ring: str, index: int) -> int:
            """Returns the number of rotates of ring to match key[index..n)."""
            if index == len(key):
                return 0

            ans = math.inf

            # For each ring[i] == key[index], we rotate the ring to match the ring[i]
            # with the key[index], then recursively match the newRing with the
            # key[index + 1..n).
            for i, r in enumerate(ring):
                if r == key[index]:
                    minRotates = min(i, len(ring) - i)
                    newRing = ring[i:] + ring[:i]
                    remainingRotates = dfs(newRing, index + 1)
                    ans = min(ans, minRotates + remainingRotates)

            return ans

        return dfs(ring, 0) + len(key)

```

2713. 515-2.py

Path: LeetCode-main/solutions/515. Find Largest Value in Each Tree Row/515-2.py

```

class Solution:
    def largestValues(self, root: TreeNode | None) -> list[int]:
        ans = []

        def dfs(root: TreeNode | None, depth: int) -> None:
            if not root:
                return
            if depth + 1 > len(ans):
                ans.append(root.val)
            else:
                ans[depth] = max(ans[depth], root.val)

            dfs(root.left, depth + 1)
            dfs(root.right, depth + 1)

        dfs(root, 0)
        return ans

```

2714. 515.py

Path: LeetCode-main/solutions/515. Find Largest Value in Each Tree Row/515.py

```

class Solution:
    def largestValues(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            mx = -math.inf
            for _ in range(len(q)):
                root = q.popleft()
                mx = max(mx, root.val)
                if root.left:
                    q.append(root.left)
                if root.right:
                    q.append(root.right)
            ans.append(mx)

        return ans

```

2715. 516-2.py

Path: LeetCode-main/solutions/516. Longest Palindromic Subsequence/516-2.py

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        # dp[i][j] := the length of LPS(s[i..j])
        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                if s[i] == s[j]:
                    dp[i][j] = 2 + dp[i + 1][j - 1]
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

        return dp[0][n - 1]

```

2716. 516.py

Path: LeetCode-main/solutions/516. Longest Palindromic Subsequence/516.py

```
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns the length of LPS(s[i..j])."""
            if i > j:
                return 0
            if i == j:
                return 1
            if s[i] == s[j]:
                return 2 + dp(i + 1, j - 1)
            return max(dp(i + 1, j), dp(i, j - 1))

        return dp(0, len(s) - 1)
```

2717. 517.py

Path: LeetCode-main/solutions/517. Super Washing Machines/517.py

```
class Solution:
    def findMinMoves(self, machines: list[int]) -> int:
        dresses = sum(machines)

        if dresses % len(machines) != 0:
            return -1

        ans = 0
        average = dresses // len(machines)
        inout = 0

        for dress in machines:
            inout += dress - average
            ans = max(ans, abs(inout), dress - average)

        return ans
```

2718. 518.py

Path: LeetCode-main/solutions/518. Coin Change 2/518.py

```
class Solution:
    def change(self, amount: int, coins: list[int]) -> int:
        dp = [1] + [0] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] += dp[i - coin]

        return dp[amount]
```

2719. 52.py

Path: LeetCode-main/solutions/52. N-Queens II/52.py

```
class Solution:
    def totalNQueens(self, n: int) -> int:
        ans = 0
        cols = [False] * n
        diag1 = [False] * (2 * n - 1)
        diag2 = [False] * (2 * n - 1)

        def dfs(i: int) -> None:
            nonlocal ans
            if i == n:
                ans += 1
                return

            for j in range(n):
                if cols[j] or diag1[i + j] or diag2[j - i + n - 1]:
                    continue
                cols[j] = diag1[i + j] = diag2[j - i + n - 1] = True
                dfs(i + 1)
                cols[j] = diag1[i + j] = diag2[j - i + n - 1] = False

        dfs(0)
        return ans
```

2720. 520.py

Path: LeetCode-main/solutions/520. Detect Capital/520.py

```
class Solution:
    def detectCapitalUse(self, word: str) -> bool:
        return word.isupper() or word.islower() or word.istitle()
```

2721. 521.py

Path: LeetCode-main/solutions/521. Longest Uncommon Subsequence I/521.py

```
class Solution:
    def findLUSlength(self, a: str, b: str) -> int:
        return -1 if a == b else max(len(a), len(b))
```

2722. 522.py

Path: LeetCode-main/solutions/522. Longest Uncommon Subsequence II/522.py


```

class Solution:
    def findLUSlength(self, strs: list[str]) -> int:
        def isSubsequence(a: str, b: str) -> bool:
            i = 0
            j = 0

            while i < len(a) and j < len(b):
                if a[i] == b[j]:
                    i += 1
                    j += 1

            return i == len(a)

        seen = set()
        duplicates = set()

        for s in strs:
            if s in seen:
                duplicates.add(s)
            seen.add(s)

        strs.sort(key=lambda x: -len(x))

        for i in range(len(strs)):
            if strs[i] in duplicates:
                continue
            isASubsequence = False
            for j in range(i):
                isASubsequence |= isSubsequence(strs[i], strs[j])
            if not isASubsequence:
                return len(strs[i])

        return -1

```

2723. 523.py

Path: LeetCode-main/solutions/523. Continuous Subarray Sum/523.py

```

class Solution:
    def checkSubarraySum(self, nums: list[int], k: int) -> bool:
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += num
            if k != 0:
                prefix %= k
            if prefix in prefixToIndex:
                if i - prefixToIndex[prefix] > 1:
                    return True
            else:
                # Set a new key if it's absent because the previous index is better.
                prefixToIndex[prefix] = i

        return False

```

2724. 524.py

Path: LeetCode-main/solutions/524. Longest Word in Dictionary through Deleting/524.py

```
class Solution:
    def findLongestWord(self, s: str, d: list[str]) -> str:
        ans = ''

        for word in d:
            i = 0
            for c in s:
                if i < len(word) and c == word[i]:
                    i += 1
            if i == len(word):
                if len(word) > len(ans) or len(word) == len(ans) and word < ans:
                    ans = word

        return ans
```

2725. 525.py

Path: LeetCode-main/solutions/525. Contiguous Array/525.py

```
class Solution:
    def findMaxLength(self, nums: list[int]) -> int:
        ans = 0
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += 1 if num else -1
            ans = max(ans, i - prefixToIndex.setdefault(prefix, i))

        return ans
```

2726. 527-2.py

Path: LeetCode-main/solutions/527. Word Abbreviation/527-2.py

```

from dataclasses import dataclass

@dataclass
class IndexedWord:
    word: str
    index: int

class Solution:
    def wordsAbbreviation(self, words: list[str]) -> list[str]:
        n = len(words)
        ans = [''] * n

        def getAbbrev(s: str, prefixIndex: int) -> str:
            n = len(s)
            num = n - (prefixIndex + 1) - 1
            numLength = 1 if num < 10 else (2 if num < 100 else 3)
            abbrevLength = (prefixIndex + 1) + numLength + 1
            if abbrevLength >= n:
                return s
            return s[:prefixIndex + 1] + str(num) + s[-1]

        abbrevToIndexedWords = collections.defaultdict(list)

        for i, word in enumerate(words):
            abbrev = getAbbrev(word, 0)
            abbrevToIndexedWords[abbrev].append(IndexedWord(word, i))

        def longestCommonPrefix(s1: str, s2: str) -> int:
            i = 0
            while i < len(s1) and i < len(s2) and s1[i] == s2[i]:
                i += 1
            return i

        for indexedWords in abbrevToIndexedWords.values():
            indexedWords.sort(key=lambda x: x.word)
            lcp = [0] * len(indexedWords)
            for i, (a, b) in enumerate(zip(indexedWords, indexedWords[1:])):
                k = longestCommonPrefix(a.word, b.word)
                lcp[i] = max(lcp[i], k)
                lcp[i + 1] = k
            for iw, l in zip(indexedWords, lcp):
                ans[iw.index] = getAbbrev(iw.word, l)

        return ans

```

2727. 527-3.py

Path: LeetCode-main/solutions/527. Word Abbreviation/527-3.py

```

from dataclasses import dataclass

@dataclass
class IndexedWord:
    word: str
    index: int

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.count = 0

class Solution:
    def wordsAbbreviation(self, words: list[str]) -> list[str]:
        n = len(words)
        ans = [''] * n

        def getAbbrev(s: str, prefixIndex: int) -> str:
            n = len(s)
            num = n - (prefixIndex + 1) - 1
            numLength = 1 if num < 10 else (2 if num < 100 else 3)
            abbrevLength = (prefixIndex + 1) + numLength + 1
            if abbrevLength >= n:
                return s
            return s[:prefixIndex + 1] + str(num) + s[-1]

        abbrevToIndexedWords = collections.defaultdict(list)

        for i, word in enumerate(words):
            abbrev = getAbbrev(word, 0)
            abbrevToIndexedWords[abbrev].append(IndexedWord(word, i))

        def insertWord(root: TrieNode | None, word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
                node.count += 1

        def firstUniqueIndex(root: TrieNode | None, word: str) -> None:
            node = root
            for i, c in enumerate(word):
                node = node.children[c]
                if node.count == 1:
                    return i
            return len(word)

        for indexedWords in abbrevToIndexedWords.values():
            root = TrieNode()
            for iw in indexedWords:
                insertWord(root, iw.word)
            for iw in indexedWords:
                index = firstUniqueIndex(root, iw.word)
                ans[iw.index] = getAbbrev(iw.word, index)

        return ans

```

2728. 527.py

Path: LeetCode-main/solutions/527. Word Abbreviation/527.py

```

class Solution:
    def wordsAbbreviation(self, words: list[str]) -> list[str]:
        n = len(words)

        def getAbbrev(s: str, prefixIndex: int) -> str:
            n = len(s)
            num = n - (prefixIndex + 1) - 1
            numLength = 1 if num < 10 else (2 if num < 100 else 3)
            abbrevLength = (prefixIndex + 1) + numLength + 1
            if abbrevLength >= n:
                return s
            return s[:prefixIndex + 1] + str(num) + s[-1]

        ans = [getAbbrev(word, 0) for word in words]
        # prefix[i] := ans[i] takes words[i][0..prefix[i]]
        prefix = [0] * n

        for i in range(n):
            while True:
                dupeIndices = []
                for j in range(i + 1, n):
                    if ans[i] == ans[j]:
                        dupeIndices.append(j)
                if not dupeIndices:
                    break
                dupeIndices.append(i)
                for index in dupeIndices:
                    prefix[index] += 1
                    ans[index] = getAbbrev(words[index], prefix[index])

        return ans

```

2729. 528-2.py

Path: LeetCode-main/solutions/528. Random Pick with Weight/528-2.py

```

class Solution:
    def __init__(self, w: list[int]):
        self.prefix = list(itertools.accumulate(w))

    def pickIndex(self) -> int:
        return bisect_left(self.prefix, random.random() * self.prefix[-1])

```

2730. 528.py

Path: LeetCode-main/solutions/528. Random Pick with Weight/528.py

```

class Solution:
    def __init__(self, w: list[int]):
        self.prefix = list(itertools.accumulate(w))

    def pickIndex(self) -> int:
        target = random.randint(0, self.prefix[-1] - 1)
        return bisect.bisect_right(range(len(self.prefix)), target,
                                   key=lambda m: self.prefix[m])

```

2731. 529.py

Path: LeetCode-main/solutions/529. Minesweeper/529.py

```
class Solution:
    def updateBoard(self, board: list[list[str]],
                    click: list[int]) -> list[list[str]]:
        i, j = click
        if board[i][j] == 'M':
            board[i][j] = 'X'
            return board

        DIRS = ((-1, -1), (-1, 0), (-1, 1), (0, -1),
                (0, 1), (1, -1), (1, 0), (1, 1))

        def getMinesCount(i: int, j: int) -> int:
            minesCount = 0
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == len(board) or y < 0 or y == len(board[0]):
                    continue
                if board[x][y] == 'M':
                    minesCount += 1
            return minesCount

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == len(board) or j < 0 or j == len(board[0]):
                return
            if board[i][j] != 'E':
                return

            minesCount = getMinesCount(i, j)
            board[i][j] = 'B' if minesCount == 0 else str(minesCount)

            if minesCount == 0:
                for dx, dy in DIRS:
                    dfs(i + dx, j + dy)

        dfs(i, j)
        return board
```

2732. 53-2.py

Path: LeetCode-main/solutions/53. Maximum Subarray/53-2.py

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        ans = -math.inf
        summ = 0

        for num in nums:
            summ = max(num, summ + num)
            ans = max(ans, summ)

        return ans
```

2733. 53-3.py

Path: LeetCode-main/solutions/53. Maximum Subarray/53-3.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    summ: int
    # the sum of the subarray starting from the first number
    maxSubarraySumLeft: int
    # the sum of the subarray ending in the last number
    maxSubarraySumRight: int
    maxSubarraySum: int

class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        def divideAndConquer(l: int, r: int) -> T:
            if l == r:
                return T(nums[l], nums[l], nums[l], nums[l])
            m = (l + r) // 2
            left = divideAndConquer(l, m)
            right = divideAndConquer(m + 1, r)
            maxSubarraySumLeft = max(left.maxSubarraySumLeft,
                                     left.summ + right.maxSubarraySumLeft)
            maxSubarraySumRight = max(
                left.maxSubarraySumRight + right.summ, right.maxSubarraySumRight)
            maxSubarraySum = max(left.maxSubarraySumRight + right.maxSubarraySumLeft,
                                 left.maxSubarraySum, right.maxSubarraySum)
            summ = left.summ + right.summ
            return T(summ, maxSubarraySumLeft, maxSubarraySumRight, maxSubarraySum)

        return divideAndConquer(0, len(nums) - 1).maxSubarraySum

```

2734. 53.py

Path: LeetCode-main/solutions/53. Maximum Subarray/53.py

```

class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        # dp[i] := the maximum sum subarray ending in i
        dp = [0] * len(nums)

        dp[0] = nums[0]
        for i in range(1, len(nums)):
            dp[i] = max(nums[i], dp[i - 1] + nums[i])

        return max(dp)

```

2735. 531.py

Path: LeetCode-main/solutions/531. Lonely Pixel I/531.py

```

class Solution:
    def findLonelyPixel(self, picture: list[list[str]]) -> int:
        m = len(picture)
        n = len(picture[0])
        ans = 0
        rows = [0] * m # rows[i] := the number of B's in rows i
        cols = [0] * n # cols[i] := the number of B's in cols i

        for i in range(m):
            for j in range(n):
                if picture[i][j] == 'B':
                    rows[i] += 1
                    cols[j] += 1

        for i in range(m):
            if rows[i] == 1: # Only have to examine the rows if rows[i] == 1.
                for j in range(n):
                    # After meeting a 'B' in this rows, break and search the next row.
                    if picture[i][j] == 'B':
                        if cols[j] == 1:
                            ans += 1
                            break

        return ans

```

2736. 532.py

Path: LeetCode-main/solutions/532. K-diff Pairs in an Array/532.py

```

class Solution:
    def findPairs(self, nums: list[int], k: int) -> int:
        ans = 0
        numToIndex = {num: i for i, num in enumerate(nums)}

        for i, num in enumerate(nums):
            target = num + k
            if target in numToIndex and numToIndex[target] != i:
                ans += 1
                del numToIndex[target]

        return ans

```

2737. 533.py

Path: LeetCode-main/solutions/533. Lonely Pixel II/533.py


```

class Solution:
    def findBlackPixel(self, picture: list[list[str]], target: int) -> int:
        m = len(picture)
        n = len(picture[0])
        ans = 0
        rows = [row.count('B') for row in picture]
        cols = [col.count('B') for col in zip(*picture)]
        rowStrings = [''.join(row) for row in picture]
        countRowStrings = collections.Counter(rowStrings)

        for i, (row, stringRow) in enumerate(zip(rows, rowStrings)):
            if row == target and countRowStrings[stringRow] == target:
                for j, col in enumerate(cols):
                    if picture[i][j] == 'B' and col == target:
                        ans += 1

        return ans

```

2738. 535.py

Path: LeetCode-main/solutions/535. Encode and Decode TinyURL/535.py

```

class Codec:
    alphabets = string.ascii_letters + '0123456789'
    urlToCode = {}
    codeToUrl = {}

    def encode(self, longUrl: str) -> str:
        while longUrl not in self.urlToCode:
            code = ''.join(random.choice(self.alphabets) for _ in range(6))
            if code not in self.codeToUrl:
                self.codeToUrl[code] = longUrl
                self.urlToCode[longUrl] = code
        return 'http://tinyurl.com/' + self.urlToCode[longUrl]

    def decode(self, shortUrl: str) -> str:
        return self.codeToUrl[shortUrl[-6:]]

```

2739. 537.py

Path: LeetCode-main/solutions/537. Complex Number Multiplication/537.py

```

class Solution:
    def complexNumberMultiply(self, num1: str, num2: str) -> str:
        a0, a1 = self._getReala0ndImag(num1)
        b0, b1 = self._getReala0ndImag(num2)
        return str(a0 * b0 - a1 * b1) + '+' + str(a0 * b1 + a1 * b0) + 'i'

    def _getReala0ndImag(self, s: str) -> tuple:
        return int(s[:s.index('+')]), int(s[s.index('+') + 1:-1])

```

2740. 538.py

Path: LeetCode-main/solutions/538. Convert BST to Greater Tree/538.py

```
class Solution:
    def convertBST(self, root: TreeNode | None) -> TreeNode | None:
        prefix = 0

        def reversedInorder(root: TreeNode | None) -> None:
            nonlocal prefix
            if not root:
                return

            reversedInorder(root.right)
            prefix += root.val
            root.val = prefix
            reversedInorder(root.left)

        reversedInorder(root)
        return root
```

2741. 539.py

Path: LeetCode-main/solutions/539. Minimum Time Difference/539.py

```
class Solution:
    def findMinDifference(self, timePoints: list[str]) -> int:
        ans = 24 * 60
        nums = sorted([int(timePoint[:2]) * 60 + int(timePoint[3:])
                        for timePoint in timePoints])

        for a, b in zip(nums, nums[1:]):
            ans = min(ans, b - a)

        return min(ans, 24 * 60 - nums[-1] + nums[0])
```

2742. 54.py

Path: LeetCode-main/solutions/54. Spiral Matrix/54.py

```

class Solution:
    def spiralOrder(self, matrix: list[list[int]]) -> list[int]:
        if not matrix:
            return []

        m = len(matrix)
        n = len(matrix[0])
        ans = []
        r1 = 0
        c1 = 0
        r2 = m - 1
        c2 = n - 1

        # Repeatedly add matrix[r1..r2][c1..c2] to `ans`.
        while len(ans) < m * n:
            j = c1
            while j <= c2 and len(ans) < m * n:
                ans.append(matrix[r1][j])
                j += 1
            i = r1 + 1
            while i <= r2 - 1 and len(ans) < m * n:
                ans.append(matrix[i][c2])
                i += 1
            j = c2
            while j >= c1 and len(ans) < m * n:
                ans.append(matrix[r2][j])
                j -= 1
            i = r2 - 1
            while i >= r1 + 1 and len(ans) < m * n:
                ans.append(matrix[i][c1])
                i -= 1
            r1 += 1
            c1 += 1
            r2 -= 1
            c2 -= 1

        return ans

```

2743. 540.py

Path: LeetCode-main/solutions/540. Single Element in a Sorted Array/540.py

```

class Solution:
    def singleNonDuplicate(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if m % 2 == 1:
                m -= 1
            if nums[m] == nums[m + 1]:
                l = m + 2
            else:
                r = m

        return nums[l]

```

2744. 541.py

Path: LeetCode-main/solutions/541. Reverse String II/541.py

```
class Solution:
    def reverseStr(self, s: str, k: int) -> str:
        return s[:k][::-1] + s[k:2 * k] + self.reverseStr(s[2 * k:], k) if s else ""
```

2745. 542-2.py

Path: LeetCode-main/solutions/542. 01 Matrix/542-2.py

```
class Solution:
    def updateMatrix(self, mat: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(mat)
        n = len(mat[0])
        q = collections.deque()

        for i in range(m):
            for j in range(n):
                if mat[i][j] == 0:
                    q.append((i, j))
                else:
                    mat[i][j] = math.inf

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if mat[x][y] <= mat[i][j] + 1:
                    continue
                q.append((x, y))
                mat[x][y] = mat[i][j] + 1

        return mat
```

2746. 542.py

Path: LeetCode-main/solutions/542. 01 Matrix/542.py

```

class Solution:
    def updateMatrix(self, mat: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(mat)
        n = len(mat[0])
        q = collections.deque()
        seen = [[False] * n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                if mat[i][j] == 0:
                    q.append((i, j))
                    seen[i][j] = True

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if seen[x][y]:
                    continue
                mat[x][y] = mat[i][j] + 1
                q.append((x, y))
                seen[x][y] = True

        return mat

```

2747. 543.py

Path: LeetCode-main/solutions/543. Diameter of Binary Tree/543.py

```

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode | None) -> int:
        ans = 0

        def maxDepth(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = maxDepth(root.left)
            r = maxDepth(root.right)
            ans = max(ans, l + r)
            return 1 + max(l, r)

        maxDepth(root)
        return ans

```

2748. 544-2.py

Path: LeetCode-main/solutions/544. Output Contest Matches/544-2.py

```
class Solution:
    def findContestMatch(self, n: int) -> str:
        matches = [str(i + 1) for i in range(n)]

        while n > 1:
            for i in range(n // 2):
                matches[i] = '(' + matches[i] + ',' + matches[n - 1 - i] + ')'
            n //= 2

        return matches[0]
```

2749. 544.py

Path: LeetCode-main/solutions/544. Output Contest Matches/544.py

```
class Solution:
    def findContestMatch(self, n: int) -> str:
        def generateMatches(matches: list[str]) -> str:
            if len(matches) == 1:
                return matches[0]

            nextMatches = []

            for i in range(len(matches) // 2):
                nextMatches.append(
                    '(' + matches[i] + ',' + matches[len(matches) - 1 - i] + ')'
                )

            return generateMatches(nextMatches)

        return generateMatches([str(i + 1) for i in range(n)])
```

2750. 545.py

Path: LeetCode-main/solutions/545. Boundary of Binary Tree/545.py

```

class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = [root.val]

        def dfs(root: TreeNode | None, lb: bool, rb: bool):
            """
            1. root.left is left boundary if root is left boundary.
               root.right if left boundary if root.left is None.
            2. Same applies for right boundary.
            3. If root is left boundary, add it before 2 children - preorder.
               If root is right boundary, add it after 2 children - postorder.
            4. A leaf that is neither left/right boundary belongs to the bottom.
            """
            if not root:
                return
            if lb:
                ans.append(root.val)
            if not lb and not rb and not root.left and not root.right:
                ans.append(root.val)

            dfs(root.left, lb, rb and not root.right)
            dfs(root.right, lb and not root.left, rb)
            if rb:
                ans.append(root.val)

        dfs(root.left, True, False)
        dfs(root.right, False, True)
        return ans

```

2751. 546.py

Path: LeetCode-main/solutions/546. Remove Boxes/546.py

```

class Solution:
    def removeBoxes(self, boxes: list[int]) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> int:
            """
            Returns the maximum score of boxes[i..j] if k boxes equal to boxes[j].
            """
            if i > j:
                return 0

            r = j
            sameBoxes = k + 1
            while r > 0 and boxes[r - 1] == boxes[r]:
                r -= 1
                sameBoxes += 1
            res = dp(i, r - 1, 0) + sameBoxes * sameBoxes

            for p in range(i, r):
                if boxes[p] == boxes[r]:
                    res = max(res, dp(i, p, sameBoxes) + dp(p + 1, r - 1, 0))

            return res

        return dp(0, len(boxes) - 1, 0)

```

2752. 547.py

Path: LeetCode-main/solutions/547. Friend Circles/547.py

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findCircleNum(self, isConnected: list[list[int]]) -> int:
        n = len(isConnected)
        uf = UnionFind(n)

        for i in range(n):
            for j in range(i, n):
                if isConnected[i][j] == 1:
                    uf.unionByRank(i, j)

        return uf.count
```

2753. 55.py

Path: LeetCode-main/solutions/55. Jump Game/55.py

```
class Solution:
    def canJump(self, nums: list[int]) -> bool:
        i = 0
        reach = 0

        while i < len(nums) and i <= reach:
            reach = max(reach, i + nums[i])
            i += 1

        return i == len(nums)
```

2754. 551.py

Path: LeetCode-main/solutions/551. Student Attendance Record I/551.py

```
class Solution:
    def checkRecord(self, s: str) -> bool:
        return s.count('A') <= 1 and 'LLL' not in s
```

2755. 552.py

Path: LeetCode-main/solutions/552. Student Attendance Record II/552.py

```
class Solution:
    def checkRecord(self, n: int) -> int:
        MOD = 1_000_000_007
        # dp[i][j] := the length so far with i A's and the last letters are j L's
        dp = [[0] * 3 for _ in range(2)]
        dp[0][0] = 1

        for _ in range(n):
            prev = [A[:] for A in dp]

            # Append a P.
            dp[0][0] = (prev[0][0] + prev[0][1] + prev[0][2]) % MOD

            # Append an L.
            dp[0][1] = prev[0][0]

            # Append an L.
            dp[0][2] = prev[0][1]

            # Append an A or append a P.
            dp[1][0] = (prev[0][0] + prev[0][1] + prev[0][2] +
                       prev[1][0] + prev[1][1] + prev[1][2]) % MOD

            # Append an L.
            dp[1][1] = prev[1][0]

            # Append an L.
            dp[1][2] = prev[1][1]

        return (sum(dp[0]) + sum(dp[1])) % MOD
```

2756. 553.py

Path: LeetCode-main/solutions/553. Optimal Division/553.py

```
class Solution:
    def optimalDivision(self, nums: list[int]) -> str:
        ans = str(nums[0])

        if len(nums) == 1:
            return ans
        if len(nums) == 2:
            return ans + '/' + str(nums[1])

        ans += '(' + str(nums[1])
        for i in range(2, len(nums)):
            ans += '/' + str(nums[i])
        ans += ')'
        return ans
```

2757. 554.py

Path: LeetCode-main/solutions/554. Brick Wall/554.py

```
class Solution:
    def leastBricks(self, wall: list[list[int]]) -> int:
        maxFreq = 0
        count = collections.defaultdict(int)

        for row in wall:
            prefix = 0
            for i in range(len(row) - 1):
                prefix += row[i]
                count[prefix] += 1
            maxFreq = max(maxFreq, count[prefix])

        return len(wall) - maxFreq
```

2758. 555.py

Path: LeetCode-main/solutions/555. Split Concatenated Strings/555.py

```
class Solution:
    def splitLoopedString(self, strs: list[str]) -> str:
        ans = ''
        sortedStrs = [max(s, s[::-1]) for s in strs]

        for i, sortedStr in enumerate(sortedStrs):
            for s in (sortedStr, sortedStr[::-1]):
                for j in range(len(s) + 1):
                    ans = max(
                        ans, s[j:] + ''.join(sortedStrs[i + 1:] + sortedStrs[:i]) + s[:j])

        return ans
```

2759. 556.py

Path: LeetCode-main/solutions/556. Next Greater Element III/556.py

```

class Solution:
    def nextGreaterElement(self, n: int) -> int:
        def nextPermutation(s: list[str]) -> str:
            i = len(s) - 2
            while i >= 0:
                if s[i] < s[i + 1]:
                    break
                i -= 1

            if i >= 0:
                for j in range(len(s) - 1, i, -1):
                    if s[j] > s[i]:
                        break
                s[i], s[j] = s[j], s[i]

            reverse(s, i + 1, len(s) - 1)
            return ''.join(s)

        def reverse(s: list[str], l: int, r: int):
            while l < r:
                s[l], s[r] = s[r], s[l]
                l += 1
                r -= 1

        s = nextPermutation(list(str(n)))
        ans = int(s)
        return -1 if ans > 2**31 - 1 or ans <= n else ans

```

2760. 559.py

Path: LeetCode-main/solutions/559. Maximum Depth of N-ary Tree/559.py

```

class Solution:
    def maxDepth(self, root: 'Node') -> int:
        if not root:
            return 0
        if not root.children:
            return 1
        return 1 + max(self.maxDepth(child) for child in root.children)

```

2761. 56.py

Path: LeetCode-main/solutions/56. Merge Intervals/56.py

```

class Solution:
    def merge(self, intervals: list[list[int]]) -> list[list[int]]:
        ans = []

        for interval in sorted(intervals):
            if not ans or ans[-1][1] < interval[0]:
                ans.append(interval)
            else:
                ans[-1][1] = max(ans[-1][1], interval[1])

        return ans

```

2762. 560.py

Path: LeetCode-main/solutions/560. Subarray Sum Equals K/560.py

```
class Solution:
    def subarraySum(self, nums: list[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = collections.Counter({0: 1})

        for num in nums:
            prefix += num
            ans += count[prefix - k]
            count[prefix] += 1

        return ans
```

2763. 561.py

Path: LeetCode-main/solutions/561. Array Partition I/561.py

```
class Solution:
    def arrayPairSum(self, nums: list[int]) -> int:
        return sum(sorted(nums)[::2])
```

2764. 562.py

Path: LeetCode-main/solutions/562. Longest Line of Consecutive One in Matrix/562.py

```
class Solution:
    def longestLine(self, mat: list[list[int]]) -> int:
        m = len(mat)
        n = len(mat[0])
        ans = 0
        # dp[i][j][0] := horizontal
        # dp[i][j][1] := vertical
        # dp[i][j][2] := diagonal
        # dp[i][j][3] := anti-diagonal
        dp = [[[0] * 4 for j in range(n)] for _ in range(m)]

        for i in range(m):
            for j in range(n):
                if mat[i][j] == 1:
                    dp[i][j][0] = dp[i][j - 1][0] + 1 if j > 0 else 1
                    dp[i][j][1] = dp[i - 1][j][1] + 1 if i > 0 else 1
                    dp[i][j][2] = dp[i - 1][j - 1][2] + 1 if i > 0 and j > 0 else 1
                    dp[i][j][3] = dp[i - 1][j + 1][3] + 1 if i > 0 and j < n - 1 else 1
                    ans = max(ans, max(dp[i][j]))

        return ans
```

2765. 563.py

Path: LeetCode-main/solutions/563. Binary Tree Tilt/563.py

```

class Solution:
    def findTilt(self, root: TreeNode | None) -> int:
        ans = 0

        def summ(root: TreeNode | None) -> None:
            nonlocal ans
            if not root:
                return 0

            l = summ(root.left)
            r = summ(root.right)
            ans += abs(l - r)
            return root.val + l + r

        summ(root)
        return ans

```

2766. 564.py

Path: LeetCode-main/solutions/564. Find the Closest Palindrome/564.py

```

class Solution:
    def nearestPalindromic(self, n: str) -> str:
        prevPalindrome, nextPalindrome = self._getPalindromes(n)
        return (str(prevPalindrome)
                if abs(prevPalindrome - int(n)) <= abs(nextPalindrome - int(n))
                else str(nextPalindrome))

    def _getPalindromes(self, s: str) -> tuple[str, str]:
        """Returns the two closest palindromes to the given number."""
        num = int(s)
        sz = len(s)
        palindromes = []
        half = s[0:(sz + 1) // 2]
        reversedHalf = half[sz // 2::-1]
        candidate = int(half + reversedHalf)

        if candidate < num:
            palindromes.append(candidate)
        else:
            prevHalf = str(int(half) - 1)
            reversedPrevHalf = prevHalf[sz // 2::-1]
            if sz % 2 == 0 and int(prevHalf) == 0:
                palindromes.append(9)
            elif sz % 2 == 0 and prevHalf == '9':
                palindromes.append(int(prevHalf + '9' + reversedPrevHalf))
            else:
                palindromes.append(int(prevHalf + reversedPrevHalf))

        if candidate > num:
            palindromes.append(candidate)
        else:
            nextHalf = str(int(half) + 1)
            reversedNextHalf = nextHalf[sz // 2::-1]
            palindromes.append(int(nextHalf + reversedNextHalf))

        return palindromes

```

2767. 565.py

Path: LeetCode-main/solutions/565. Array Nesting/565.py

```
class Solution:
    def arrayNesting(self, nums: list[int]) -> int:
        ans = 0

        for num in nums:
            if num == -1:
                continue
            index = num
            count = 0
            while nums[index] != -1:
                cache = index
                index = nums[index]
                nums[cache] = -1
                count += 1
            ans = max(ans, count)

        return ans
```

2768. 566.py

Path: LeetCode-main/solutions/566. Reshape the Matrix/566.py

```
class Solution:
    def matrixReshape(self, nums: list[list[int]],
                      r: int, c: int) -> list[list[int]]:
        if nums == [] or r * c != len(nums) * len(nums[0]):
            return nums

        ans = [[0 for j in range(c)] for i in range(r)]
        k = 0

        for row in nums:
            for num in row:
                ans[k // c][k % c] = num
                k += 1

        return ans
```

2769. 567-2.py

Path: LeetCode-main/solutions/567. Permutation in String/567-2.py

```

class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        count = collections.Counter(s1)
        required = len(s1)

        for r, c in enumerate(s2):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            if r >= len(s1): # The window is oversized.
                count[s2[r - len(s1)]] += 1
                if count[s2[r - len(s1)]] > 0:
                    required += 1
            if required == 0:
                return True

        return False

```

2770. 57.py

Path: LeetCode-main/solutions/57. Insert Interval/57.py

```

class Solution:
    def insert(self, intervals: list[list[int]],
               newInterval: list[int]) -> list[list[int]]:
        n = len(intervals)
        ans = []
        i = 0

        while i < n and intervals[i][1] < newInterval[0]:
            ans.append(intervals[i])
            i += 1

        # Merge overlapping intervals.
        while i < n and intervals[i][0] <= newInterval[1]:
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1

        ans.append(newInterval)

        while i < n:
            ans.append(intervals[i])
            i += 1

        return ans

```

2771. 573.py

Path: LeetCode-main/solutions/573. Squirrel Simulation/573.py

```

class Solution:
    def minDistance(
        self,
        height: int,
        width: int,
        tree: list[int],
        squirrel: list[int],
        nuts: list[list[int]],
    ) -> int:
        def dist(a: list[int], b: list[int]) -> int:
            return abs(a[0] - b[0]) + abs(a[1] - b[1])

        totDist = sum(dist(nut, tree) for nut in nuts) * 2
        maxSave = max(dist(nut, tree) - dist(nut, squirrel) for nut in nuts)
        return totDist - maxSave

```

2772. 575.py

Path: LeetCode-main/solutions/575. Distribute Candies/575.py

```

class Solution:
    def distributeCandies(self, candies: list[int]) -> int:
        return min(len(candies) // 2, len(set(candies)))

```

2773. 576-2.py

Path: LeetCode-main/solutions/576. Out of Boundary Paths/576-2.py

```

class Solution:
    def findPaths(
        self,
        m: int,
        n: int,
        maxMove: int,
        startRow: int,
        startColumn: int,
    ) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        MOD = 1_000_000_007
        ans = 0

        # dp[i][j] := the number of paths to move the ball (i, j) out-of-bounds
        dp = [[0] * n for _ in range(m)]
        dp[startRow][startColumn] = 1

        for _ in range(maxMove):
            newDp = [[0] * n for _ in range(m)]
            for i in range(m):
                for j in range(n):
                    if dp[i][j] > 0:
                        for dx, dy in DIRS:
                            x = i + dx
                            y = j + dy
                            if x < 0 or x == m or y < 0 or y == n:
                                ans = (ans + dp[i][j]) % MOD
                            else:
                                newDp[x][y] = (newDp[x][y] + dp[i][j]) % MOD
            dp = newDp

        return ans

```

2774. 576.py

Path: LeetCode-main/solutions/576. Out of Boundary Paths/576.py

```
class Solution:
    def findPaths(self, m, n, maxMove, startRow, startColumn):
        MOD = 1000000007

        @functools.lru_cache(None)
        def dp(k: int, i: int, j: int) -> int:
            """
            Returns the number of paths to move the ball at (i, j) out-of-bounds with
            k moves.
            """
            if i < 0 or i == m or j < 0 or j == n:
                return 1
            if k == 0:
                return 0
            return (dp(k - 1, i + 1, j) + dp(k - 1, i - 1, j) +
                    dp(k - 1, i, j + 1) + dp(k - 1, i, j - 1)) % MOD

        return dp(maxMove, startRow, startColumn)
```

2775. 58.py

Path: LeetCode-main/solutions/58. Length of Last Word/58.py

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        i = len(s) - 1

        while i >= 0 and s[i] == ' ':
            i -= 1
        lastIndex = i
        while i >= 0 and s[i] != ' ':
            i -= 1

        return lastIndex - i
```

2776. 581.py

Path: LeetCode-main/solutions/581. Shortest Unsorted Continuous Subarray/581.py

```

class Solution:
    def findUnsortedSubarray(self, nums: list[int]) -> int:
        mn = math.inf
        mx = -math.inf
        flag = False

        for i in range(1, len(nums)):
            if nums[i] < nums[i - 1]:
                flag = True
            if flag:
                mn = min(mn, nums[i])

        flag = False

        for i in reversed(range(len(nums) - 1)):
            if nums[i] > nums[i + 1]:
                flag = True
            if flag:
                mx = max(mx, nums[i])

        for l in range(len(nums)):
            if nums[l] > mn:
                break

        for r, num in reversed(list(enumerate(nums))):
            if num < mx:
                break

        return 0 if l >= r else r - l + 1

```

2777. 582.py

Path: LeetCode-main/solutions/582. Kill Process/582.py

```

class Solution:
    def killProcess(
        self,
        pid: list[int],
        ppid: list[int],
        kill: int,
    ) -> list[int]:
        ans = []
        tree = collections.defaultdict(list)

        for v, u in zip(pid, ppid):
            if u == 0:
                continue
            tree[u].append(v)

        def dfs(u: int) -> None:
            ans.append(u)
            for v in tree.get(u, []):
                dfs(v)

        dfs(kill)
        return ans

```

2778. 583.py

Path: LeetCode-main/solutions/583. Delete Operation for Two Strings/583.py

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        k = self._lcs(word1, word2)
        return (len(word1) - k) + (len(word2) - k)

    def _lcs(self, a: str, b: str) -> int:
        m = len(a)
        n = len(b)
        # dp[i][j] := the length of LCS(a[0..i], b[0..j])
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if a[i - 1] == b[j - 1]:
                    dp[i][j] = 1 + dp[i - 1][j - 1]
                else:
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

        return dp[m][n]
```

2779. 587.py

Path: LeetCode-main/solutions/587. Erect the Fence/587.py

```
class Solution:
    def outerTrees(self, trees: list[list[int]]) -> list[list[int]]:
        hull = []

        trees.sort(key=lambda x: (x[0], x[1]))

        def cross(p: list[int], q: list[int], r: list[int]) -> int:
            return (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])

        # Build the lower hull: left-to-right scan.
        for tree in trees:
            while len(hull) > 1 and cross(hull[-1], hull[-2], tree) > 0:
                hull.pop()
            hull.append(tuple(tree))
        hull.pop()

        # Build the upper hull: right-to-left scan.
        for tree in reversed(trees):
            while len(hull) > 1 and cross(hull[-1], hull[-2], tree) > 0:
                hull.pop()
            hull.append(tuple(tree))

        # Remove the redundant elements from the stack.
        return list(set(hull))
```

2780. 59.py

Path: LeetCode-main/solutions/59. Spiral Matrix II/59.py

```
class Solution:
    def generateMatrix(self, n: int) -> list[list[int]]:
        ans = [[0] * n for _ in range(n)]
        count = 1

        for mn in range(n // 2):
            mx = n - mn - 1
            for i in range(mn, mx):
                ans[mn][i] = count
                count += 1
            for i in range(mn, mx):
                ans[i][mx] = count
                count += 1
            for i in range(mx, mn, -1):
                ans[mx][i] = count
                count += 1
            for i in range(mx, mn, -1):
                ans[i][mn] = count
                count += 1

        if n % 2 == 1:
            ans[n // 2][n // 2] = count

        return ans
```

2781. 591.py

Path: LeetCode-main/solutions/591. Tag Validator/591.py

```

class Solution:
    def isValid(self, code: str) -> bool:
        if code[0] != '<' or code[-1] != '>':
            return False

        containsTag = False
        stack = []

        def isValidCdata(s: str) -> bool:
            return s.find('[CDATA[') == 0

        def isValidTagName(tagName: str, isEndTag: bool) -> bool:
            nonlocal containsTag
            if not tagName or len(tagName) > 9:
                return False
            if any(not c.isupper() for c in tagName):
                return False

            if isEndTag:
                return stack and stack.pop() == tagName

            containsTag = True
            stack.append(tagName)
            return True

        i = 0
        while i < len(code):
            if not stack and containsTag:
                return False
            if code[i] == '<':
                # It's inside a tag, so check if it's a cdata.
                if stack and code[i + 1] == '!':
                    closeIndex = code.find(']]>', i + 2)
                    if closeIndex == -1 or not isValidCdata(code[i + 2:closeIndex]):
                        return False
                elif code[i + 1] == '/': # the end tag
                    closeIndex = code.find('>', i + 2)
                    if closeIndex == -1 or not isValidTagName(
                        code[i + 2: closeIndex],
                        True):
                        return False
                else: # the start tag
                    closeIndex = code.find('>', i + 1)
                    if closeIndex == -1 or not isValidTagName(
                        code[i + 1: closeIndex],
                        False):
                        return False
                i = closeIndex
            i += 1

        return not stack and containsTag

```

2782. 592.py

Path: LeetCode-main/solutions/592. Fraction Addition and Subtraction/592.py

```

class Solution:
    def fractionAddition(self, expression: str) -> str:
        ints = list(map(int, re.findall('[+-]?[0-9]+', expression)))
        A = 0
        B = 1

        for a, b in zip(ints[::2], ints[1::2]):
            A = A * b + a * B
            B *= b
            g = math.gcd(A, B)
            A //= g
            B //= g

        return str(A) + '/' + str(B)

```

2783. 593.py

Path: LeetCode-main/solutions/593. Valid Square/593.py

```

class Solution:
    def validSquare(
        self,
        p1: list[int],
        p2: list[int],
        p3: list[int],
        p4: list[int],
    ) -> bool:
        def dist(p1: list[int], p2: list[int]) -> int:
            return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2

        distSet = set([dist(*pair)
                        for pair in list(
                            itertools.combinations([p1, p2, p3, p4], 2))]
        return 0 not in distSet and len(distSet) == 2

```

2784. 594.py

Path: LeetCode-main/solutions/594. Longest Harmonious Subsequence/594.py

```

class Solution:
    def findLHS(self, nums: list[int]) -> int:
        ans = 0
        count = collections.Counter(nums)

        for num, freq in count.items():
            if num + 1 in count:
                ans = max(ans, freq + count[num + 1])

        return ans

```

2785. 598.py

Path: LeetCode-main/solutions/598. Range Addition II/598.py

```
class Solution:
    def maxCount(self, m: int, n: int, ops: list[list[int]]) -> int:
        minY = m
        minX = n

        for y, x in ops:
            minY = min(minY, y)
            minX = min(minX, x)

        return minX * minY
```

2786. 599.py

Path: LeetCode-main/solutions/599. Minimum Index Sum of Two Lists/599.py

```
class Solution:
    def findRestaurant(self, list1: list[str], list2: list[str]) -> list[str]:
        ans = []
        restaurantToIndex = {restaurant: i for i,
                                restaurant in enumerate(list1)}

        minSum = math.inf

        for i, restaurant in enumerate(list2):
            if restaurant in restaurantToIndex:
                summ = restaurantToIndex[restaurant] + i
                if summ < minSum:
                    ans.clear()
                if summ <= minSum:
                    ans.append(restaurant)
                    minSum = summ

        return ans
```

2787. 6.py

Path: LeetCode-main/solutions/6. ZigZag Conversion/6.py

```
class Solution:
    def convert(self, s: str, numRows: int) -> str:
        rows = [''] * numRows
        k = 0
        direction = (numRows == 1) - 1

        for c in s:
            rows[k] += c
            if k == 0 or k == numRows - 1:
                direction *= -1
            k += direction

        return ''.join(rows)
```

2788. 60.py

Path: LeetCode-main/solutions/60. Permutation Sequence/60.py

```

class Solution:
    def getPermutation(self, n: int, k: int) -> str:
        ans = ''
        nums = [i + 1 for i in range(n)]
        fact = [1] * (n + 1) # fact[i] := i!

        for i in range(2, n + 1):
            fact[i] = fact[i - 1] * i

        k -= 1 # 0-indexed

        for i in reversed(range(n)):
            j = k // fact[i]
            k %= fact[i]
            ans += str(nums[j])
            nums.pop(j)

        return ans

```

2789. 604-2.py

Path: LeetCode-main/solutions/604. Design Compressed String Iterator/604-2.py

```

class StringIterator:
    def __init__(self, compressedString: str):
        self.q = collections.deque() # (currentChar, num)

        i = 0 # compressedString's index
        while i < len(compressedString):
            c = compressedString[i]
            i += 1
            num = 0
            while i < len(compressedString) and compressedString[i].isdigit():
                num = num * 10 + int(compressedString[i])
                i += 1
            self.q.append((c, num))

        def next(self) -> str:
            if not self.hasNext():
                return ' '
            c, num = self.q.popleft()
            if num > 1:
                self.q.appendleft((c, num - 1))
            return c

        def hasNext(self) -> bool:
            return self.q

```

2790. 604.py

Path: LeetCode-main/solutions/604. Design Compressed String Iterator/604.py


```

class StringIterator:
    def __init__(self, compressedString: str):
        self.s = compressedString
        self.i = 0    # s' index
        self.num = 0  # currentChar's count
        self.currentChar = ' '

    def next(self) -> str:
        if not self.hasNext():
            return ' '

        if self.num == 0:
            self.currentChar = self.s[self.i]
            self.i += 1
            while self.i < len(self.s) and self.s[self.i].isdigit():
                self.num = self.num * 10 + int(self.s[self.i])
                self.i += 1

            self.num -= 1
            return self.currentChar

    def hasNext(self) -> bool:
        return self.i < len(self.s) or self.num > 0

```

2791. 605.py

Path: LeetCode-main/solutions/605. Can Place Flowers/605.py

```

class Solution:
    def canPlaceFlowers(self, flowerbed: list[int], n: int) -> bool:
        for i, flower in enumerate(flowerbed):
            if flower == 0 and (
                i == 0 or flowerbed[i - 1] == 0) and (
                i == len(flowerbed) - 1 or flowerbed[i + 1] == 0):
                flowerbed[i] = 1
                n -= 1
        if n <= 0:
            return True

        return False

```

2792. 606.py

Path: LeetCode-main/solutions/606. Construct String from Binary Tree/606.py

```

class Solution:
    def tree2str(self, t: TreeNode | None) -> str:
        def dfs(root: TreeNode | None) -> str:
            if not root:
                return ''
            if root.right:
                return str(root.val) + '(' + dfs(root.left) + ')(' + dfs(root.right) + ')'
            if root.left:
                return str(root.val) + '(' + dfs(root.left) + ')'
            return str(root.val)
        return dfs(t)

```

2793. 609.py

Path: LeetCode-main/solutions/609. Find Duplicate File in System/609.py

```
class Solution:
    def findDuplicate(self, paths: list[str]) -> list[list[str]]:
        contentToPathFiles = collections.defaultdict(list)

        for path in paths:
            words = path.split(' ')
            rootPath = words[0] # "root/d1/d2/.../dm"
            for fileAndContent in words[1:]: # "fn.txt(fn_content)"
                l = fileAndContent.find('(')
                r = fileAndContent.find(')')
                # "fn.txt"
                file = fileAndContent[l+1:]
                # "fn_content"
                content = fileAndContent[l+1:r]
                # "root/d1/d2/.../dm/fn.txt"
                filePath = rootPath + '/' + file
                contentToPathFiles[content].append(filePath)

        return [filePath for filePath in contentToPathFiles.values() if len(filePath) > 1]
```

2794. 61.py

Path: LeetCode-main/solutions/61. Rotate List/61.py

```
class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0:
            return head

        tail = head
        length = 1
        while tail.next:
            tail = tail.next
            length += 1
        tail.next = head # Circle the list.

        t = length - k % length
        for _ in range(t):
            tail = tail.next
        newHead = tail.next
        tail.next = None

        return newHead
```

2795. 611.py

Path: LeetCode-main/solutions/611. Valid Triangle Number/611.py

```
class Solution:
    def triangleNumber(self, nums: list[int]) -> int:
        ans = 0

        nums.sort()

        for k in range(len(nums) - 1, 1, -1):
            i = 0
            j = k - 1
            while i < j:
                if nums[i] + nums[j] > nums[k]:
                    ans += j - i
                    j -= 1
                else:
                    i += 1

        return ans
```

2796. 616-2.py

Path: LeetCode-main/solutions/616. Add Bold Tag in String/616-2.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Solution:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        ans = []
        # bold[i] := True if s[i] should be bolded
        bold = [0] * n
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.isWord = True

        def find(s: str, i: int) -> int:
            node = root
            ans = -1
            for j in range(i, len(s)):
                if s[j] not in node.children:
                    node.children[s[j]] = TrieNode()
                node = node.children[s[j]]
                if node.isWord:
                    ans = j
            return ans

        for word in words:
            insert(word)

        boldEnd = -1 # `s[i..boldEnd]` should be bolded.
        for i in range(n):
            boldEnd = max(boldEnd, find(s, i))
            bold[i] = boldEnd >= i

        # Construct the with bold tags
        i = 0
        while i < n:
            if bold[i]:
                j = i
                while j < n and bold[j]:
                    j += 1
                # `s[i..j]` should be bolded.
                ans.append('<b>' + s[i:j] + '</b>')
                i = j
            else:
                ans.append(s[i])
                i += 1

        return ''.join(ans)

```

2797. 616.py

Path: LeetCode-main/solutions/616. Add Bold Tag in String/616.py

```

class Solution:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        ans = []
        # bold[i] := True if s[i] should be bolded
        bold = [0] * n

        boldEnd = -1 # s[i:boldEnd] should be bolded
        for i in range(n):
            for word in words:
                if s[i:].startswith(word):
                    boldEnd = max(boldEnd, i + len(word))
            bold[i] = boldEnd > i

        # Construct the with bold tags
        i = 0
        while i < n:
            if bold[i]:
                j = i
                while j < n and bold[j]:
                    j += 1
                # `s[i..j)` should be bolded.
                ans.append('<b>' + s[i:j] + '</b>')
                i = j
            else:
                ans.append(s[i])
                i += 1

        return ''.join(ans)

```

2798. 617.py

Path: LeetCode-main/solutions/617. Merge Two Binary Trees/617.py

```

class Solution:
    def mergeTrees(
        self,
        root1: TreeNode | None,
        root2: TreeNode | None,
    ) -> TreeNode | None:
        if not root1 and not root2:
            return None
        val = (root1.val if root1 else 0) + (root2.val if root2 else 0)
        root = TreeNode(val)
        root.left = self.mergeTrees(root1.left if root1 else None,
                                    root2.left if root2 else None)
        root.right = self.mergeTrees(root1.right if root1 else None,
                                    root2.right if root2 else None)
        return root

```

2799. 62-2.py

Path: LeetCode-main/solutions/62. Unique Paths/62-2.py

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [1] * n

        for _ in range(1, m):
            for j in range(1, n):
                dp[j] += dp[j - 1]

        return dp[n - 1]
```

2800. 62.py

Path: LeetCode-main/solutions/62. Unique Paths/62.py

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[i][j] := the number of unique paths from (0, 0) to (i, j)
        dp = [[1] * n for _ in range(m)]

        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[-1][-1]
```

2801. 621.py

Path: LeetCode-main/solutions/621. Task Scheduler/621.py

```
class Solution:
    def leastInterval(self, tasks: list[str], n: int) -> int:
        count = collections.Counter(tasks)
        maxFreq = max(count.values())
        # Put the most frequent task in the slot first.
        maxFreqTaskOccupy = (maxFreq - 1) * (n + 1)
        # Get the number of tasks with same frequency as maxFreq, we'll append them after the
        # `maxFreqTaskOccupy`.
        nMaxFreq = sum(value == maxFreq for value in count.values())
        # max(
        #     the most frequent task is frequent enough to force some idle slots,
        #     the most frequent task is not frequent enough to force idle slots
        # )
        return max(maxFreqTaskOccupy + nMaxFreq, len(tasks))
```

2802. 624-2.py

Path: LeetCode-main/solutions/624. Maximum Distance in Arrays/624-2.py

```

class Solution:
    def maxDistance(self, arrays: list[list[int]]) -> int:
        min1, index_min1 = min((A[0], i) for i, A in enumerate(arrays))
        max1, index_max1 = max((A[-1], i) for i, A in enumerate(arrays))
        if index_min1 != index_max1:
            return max1 - min1

        min2, index_min2 = min((A[0], i)
                                for i, A in enumerate(arrays) if i != index_min1)
        max2, index_max2 = max((A[-1], i)
                                for i, A in enumerate(arrays) if i != index_max1)
        return max(max1 - min2, max2 - min1)

```

2803. 624.py

Path: LeetCode-main/solutions/624. Maximum Distance in Arrays/624.py

```

class Solution:
    def maxDistance(self, arrays: list[list[int]]) -> int:
        ans = 0
        mn = 10000
        mx = -10000

        for A in arrays:
            ans = max(ans, A[-1] - mn, mx - A[0])
            mn = min(mn, A[0])
            mx = max(mx, A[-1])

        return ans

```

2804. 625.py

Path: LeetCode-main/solutions/625. Minimum Factorization/625.py

```

class Solution:
    def smallestFactorization(self, num: int) -> int:
        if num == 1:
            return 1

        ans = 0
        base = 1

        for i in range(9, 1, -1):
            while num % i == 0:
                num //= i
                ans = base * i + ans
                base *= 10

        return ans if num == 1 and ans < 2**31 - 1 else 0

```

2805. 628-2.py

Path: LeetCode-main/solutions/628. Maximum Product of Three Numbers/628-2.py

```

class Solution:
    def maximumProduct(self, nums: list[int]) -> int:
        min1 = inf    # the minimum
        min2 = inf    # the second minimum
        max1 = -inf    # the maximum
        max2 = -inf    # the second maximum
        max3 = -inf    # the third maximum

        for num in nums:
            if num <= min1:
                min2 = min1
                min1 = num
            elif num <= min2:
                min2 = num

            if num >= max1:
                max3 = max2
                max2 = max1
                max1 = num
            elif num >= max2:
                max3 = max2
                max2 = num
            elif num >= max3:
                max3 = num

        return max(max1 * min1 * min2, max1 * max2 * max3)

```

2806. 628.py

Path: LeetCode-main/solutions/628. Maximum Product of Three Numbers/628.py

```

class Solution:
    def maximumProduct(self, nums: list[int]) -> int:
        nums.sort()
        return max(nums[-1] * nums[0] * nums[1],
                    nums[-1] * nums[-2] * nums[-3])

```

2807. 629.py

Path: LeetCode-main/solutions/629. K Inverse Pairs Array/629.py

```

class Solution:
    def kInversePairs(self, n: int, k: int) -> int:
        MOD = 1_000_000_007

        # dp[i][j] := the number of permutations of numbers 1..i with j inverse pairs
        dp = [[0] * (k + 1) for _ in range(n + 1)]

        # If there's no inverse pair, the permutation is unique '123..i'
        for i in range(n + 1):
            dp[i][0] = 1

        for i in range(1, n + 1):
            for j in range(1, k + 1):
                dp[i][j] = (dp[i][j - 1] + dp[i - 1][j]) % MOD
                if j - i >= 0:
                    dp[i][j] = (dp[i][j] - dp[i - 1][j - i] + MOD) % MOD

        return dp[n][k]

```

2808. 63-2.py

Path: LeetCode-main/solutions/63. Unique Paths II/63-2.py

```
class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: list[list[int]]) -> int:
        m = len(obstacleGrid)
        n = len(obstacleGrid[0])
        dp = [0] * n
        dp[0] = 1

        for i in range(m):
            for j in range(n):
                if obstacleGrid[i][j]:
                    dp[j] = 0
                elif j > 0:
                    dp[j] += dp[j - 1]

        return dp[n - 1]
```

2809. 63.py

Path: LeetCode-main/solutions/63. Unique Paths II/63.py

```
class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: list[list[int]]) -> int:
        m = len(obstacleGrid)
        n = len(obstacleGrid[0])
        # dp[i][j] := the number of unique paths from (0, 0) to (i, j)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        dp[0][1] = 1 # Can also set dp[1][0] = 1.

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if obstacleGrid[i - 1][j - 1] == 0:
                    dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[m][n]
```

2810. 630.py

Path: LeetCode-main/solutions/630. Course Schedule III/630.py

```
class Solution:
    def scheduleCourse(self, courses: list[list[int]]) -> int:
        time = 0
        maxHeap = []

        for duration, lastDay in sorted(courses, key=lambda x: x[1]):
            heapq.heappush(maxHeap, -duration)
            time += duration
            # If the current course cannot be taken, check if it can be swapped with
            # a previously taken course that has a larger duration to increase the
            # time available to take upcoming courses.
            if time > lastDay:
                time += heapq.heappop(maxHeap)

        return len(maxHeap)
```

2811. 631.py

Path: LeetCode-main/solutions/631. Design Excel Sum Formula/631.py

```

from dataclasses import dataclass

@dataclass
class Cell:
    val: int = 0
    posCount: dict[tuple[int, int], int] | None = None

class Excel:
    def __init__(self, height: int, width: str):
        self.width = ord(width) - ord('A') + 1
        self.sheet = [[Cell() for _ in range(self.width)] for _ in range(height)]

    def set(self, row: int, column: str, val: int) -> None:
        self._getCell(row, column).val = val
        self._getCell(row, column).posCount = {}

    def get(self, row: int, column: str) -> int:
        cell = self._getCell(row, column)
        if not cell.posCount:
            return cell.val
        val = 0
        for pos, count in cell.posCount.items():
            val += self.get(pos // self.width + 1, chr(pos %
                self.width + ord('A'))) * count
        return val

    def sum(self, row: int, column: str, numbers: list[str]) -> int:
        self._getCell(row, column).posCount = self._parse(numbers)
        return self.get(row, column)

    def _getCell(self, row: int, column: str) -> Cell:
        return self.sheet[row - 1][ord(column) - ord('A')]

    def _parse(self, numbers: list[str]) -> dict[int, int]:
        count: dict[int, int] = {}
        for s in numbers:
            startRow, startCol, endRow, endCol = self._parseRange(s)
            for i in range(startRow - 1, endRow):
                for j in range(ord(startCol) - ord('A'), ord(endCol) - ord('A') + 1):
                    pos = i * self.width + j
                    count[pos] = count.get(pos, 0) + 1
        return count

    def _parseRange(self, s: str) -> tuple[int, str, int, str]:
        if ':' not in s:
            return int(s[1:]), s[0], int(s[1:]), s[0]
        l, r = s.split(':')
        return int(l[1:]), l[0], int(r[1:]), r[0]

```

2812. 632.py

Path: `LeetCode-main/solutions/632. Smallest Range Covering Elements from K Lists/632.py`

```

class Solution:
    def smallestRange(self, nums: list[list[int]]) -> list[int]:
        minHeap = [(row[0], i, 0) for i, row in enumerate(nums)]
        heapq.heapify(minHeap)

        maxRange = max(row[0] for row in nums)
        minRange = heapq.nsmallest(1, minHeap)[0][0]
        ans = [minRange, maxRange]

        while len(minHeap) == len(nums):
            num, r, c = heapq.heappop(minHeap)
            if c + 1 < len(nums[r]):
                heapq.heappush(minHeap, (nums[r][c + 1], r, c + 1))
                maxRange = max(maxRange, nums[r][c + 1])
                minRange = heapq.nsmallest(1, minHeap)[0][0]
                if maxRange - minRange < ans[1] - ans[0]:
                    ans[0], ans[1] = minRange, maxRange

        return ans

```

2813. 633.py

Path: LeetCode-main/solutions/633. Sum of Square Numbers/633.py

```

class Solution:
    def judgeSquareSum(self, c: int) -> bool:
        l = 0
        r = math.isqrt(c)

        while l <= r:
            summ = l * l + r * r
            if summ == c:
                return True
            if summ < c:
                l += 1
            else:
                r -= 1

        return False

```

2814. 634-2.py

Path: LeetCode-main/solutions/634. Find the Derangement of An Array/634-2.py

```

class Solution:
    def findDerangement(self, n: int) -> int:
        MOD = 1_000_000_007
        dp = [1] + [0] * n

        for i in range(2, n + 1):
            dp[i] = (i - 1) * (dp[i - 1] + dp[i - 2]) % MOD

        return dp[n]

```

2815. 634.py

Path: LeetCode-main/solutions/634. Find the Derangement of An Array/634.py

```
class Solution:
    def findDerangement(self, n: int) -> int:
        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(i: int) -> int:
            if i == 0:
                return 1
            if i == 1:
                return 0
            return (i - 1) * (dp(i - 1) + dp(i - 2)) % MOD

        return dp(n)
```

2816. 635.py

Path: LeetCode-main/solutions/635. Design Log Storage System/635.py

```
class LogSystem:
    def __init__(self):
        self.granularityToIndices = {'Year': 4, 'Month': 7, 'Day': 10,
                                     'Hour': 13, 'Minute': 16, 'Second': 19}

        self.idAndTimestamps = []

    def put(self, id: int, timestamp: str) -> None:
        self.idAndTimestamps.append((id, timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> list[int]:
        index = self.granularityToIndices[granularity]
        s = start[:index]
        e = end[:index]
        return [id for id, timestamp in self.idAndTimestamps
                if s <= timestamp[:index] <= e]
```

2817. 638.py

Path: LeetCode-main/solutions/638. Shopping Offers/638.py

```

class Solution:
    def shoppingOffers(
        self,
        price: list[int],
        special: list[list[int]],
        needs: list[int]
    ) -> int:
        def dfs(s: int) -> int:
            ans = 0
            for i, need in enumerate(needs):
                ans += need * price[i]

            for i in range(s, len(special)):
                offer = special[i]
                if all(offer[j] <= need for j, need in enumerate(needs)):
                    # Use the special[i].
                    for j in range(len(needs)):
                        needs[j] -= offer[j]
                    ans = min(ans, offer[-1] + dfs(i))
                    # Unuse the special[i] (backtracking).
                    for j in range(len(needs)):
                        needs[j] += offer[j]

            return ans

        return dfs(0)

```

2818. 64.py

Path: LeetCode-main/solutions/64. Minimum Path Sum/64.py

```

class Solution:
    def minPathSum(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        for i in range(m):
            for j in range(n):
                if i > 0 and j > 0:
                    grid[i][j] += min(grid[i - 1][j], grid[i][j - 1])
                elif i > 0:
                    grid[i][0] += grid[i - 1][0]
                elif j > 0:
                    grid[0][j] += grid[0][j - 1]

        return grid[m - 1][n - 1]

```

2819. 640.py

Path: LeetCode-main/solutions/640. Solve the Equation/640.py

```

class Solution:
    def solveEquation(self, equation: str) -> str:
        def calculate(s: str) -> tuple:
            coefficient = 0
            constant = 0
            num = 0
            sign = 1

            for i, c in enumerate(s):
                if c.isdigit():
                    num = num * 10 + int(c)
                elif c in '+-':
                    constant += sign * num
                    sign = 1 if c == '+' else -1
                    num = 0
                else:
                    if i > 0 and num == 0 and s[i - 1] == '0':
                        continue
                    coefficient += sign if num == 0 else sign * num
                    num = 0

            return coefficient, constant + sign * num

        lhsEquation, rhsEquation = equation.split('=')
        lhsCoefficient, lhsConstant = calculate(lhsEquation)
        rhsCoefficient, rhsConstant = calculate(rhsEquation)
        coefficient = lhsCoefficient - rhsCoefficient
        constant = rhsConstant - lhsConstant

        if coefficient == 0 and constant == 0:
            return "Infinite solutions"
        if coefficient == 0 and constant != 0:
            return "No solution"
        return "x=" + str(constant // coefficient)

```

2820. 642.py

Path: LeetCode-main/solutions/642. Design Search Autocomplete System/642.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.s: str | None = None
        self.time = 0
        self.top3: list[TrieNode] = []

    def __lt__(self, other):
        if self.time == other.time:
            return self.s < other.s
        return self.time > other.time

    def update(self, node) -> None:
        if node not in self.top3:
            self.top3.append(node)
        self.top3.sort()
        if len(self.top3) > 3:
            self.top3.pop()

class AutocompleteSystem:
    def __init__(self, sentences: list[str], times: list[int]):
        self.root = TrieNode()
        self.curr = self.root
        self.s: list[str] = []

        for sentence, time in zip(sentences, times):
            self._insert(sentence, time)

    def input(self, c: str) -> list[str]:
        if c == '#':
            self._insert(''.join(self.s), 1)
            self.curr = self.root
            self.s = []
            return []

        self.s.append(c)

        if self.curr:
            self.curr = self.curr.children.get(c, None)
        if not self.curr:
            return []
        return [node.s for node in self.curr.top3]

    def _insert(self, sentence: str, time: int) -> None:
        node = self.root
        for c in sentence:
            node = node.children.setdefault(c, TrieNode())
        node.s = sentence
        node.time += time

        leaf = node
        node: TrieNode = self.root
        for c in sentence:
            node = node.children[c]
            node.update(leaf)

```

2821. 643.py

Path: LeetCode-main/solutions/643. Maximum Average Subarray I/643.py


```

class Solution:
    def findMaxAverage(self, nums: list[int], k: int) -> float:
        summ = sum(nums[:k])
        ans = summ

        for i in range(k, len(nums)):
            summ += nums[i] - nums[i - k]
            ans = max(ans, summ)

        return ans / k

```

2822. 644.py

Path: LeetCode-main/solutions/644. Maximum Average Subarray II/644.py

```

class Solution:
    def findMaxAverage(self, nums: list[int], k: int) -> float:
        ERR = 1e-5
        l = min(nums)
        r = max(nums)

        def check(m: float) -> bool:
            """
            Returns True if there's a subarray, where its length >= k and its average
            sum >= m.
            """
            summ = 0
            prevSum = 0
            minPrevSum = 0

            for i, num in enumerate(nums):
                # Need to subtract m for each `num` so that we can check if the sum of
                # the subarray >= 0.
                summ += num - m
                if i >= k:
                    prevSum += nums[i - k] - m
                    minPrevSum = min(minPrevSum, prevSum)
                if i + 1 >= k and summ >= minPrevSum:
                    return True

            return False

        while r - l > ERR:
            m = (l + r) / 2
            if check(m):
                l = m
            else:
                r = m

        return l

```

2823. 645.py

Path: LeetCode-main/solutions/645. Set Mismatch/645.py

```
class Solution:
    def findErrorNums(self, nums: list[int]) -> list[int]:
        for num in nums:
            if nums[abs(num) - 1] < 0:
                duplicate = abs(num)
            else:
                nums[abs(num) - 1] *= -1

        for i, num in enumerate(nums):
            if num > 0:
                return [duplicate, i + 1]
```

2824. 646.py

Path: LeetCode-main/solutions/646. Maximum Length of Pair Chain/646.py

```
class Solution:
    def findLongestChain(self, pairs: list[list[int]]) -> int:
        ans = 0
        prevEnd = -math.inf

        for s, e in sorted(pairs, key=lambda x: x[1]):
            if s > prevEnd:
                ans += 1
                prevEnd = e

        return ans
```

2825. 647.py

Path: LeetCode-main/solutions/647. Palindromic Substrings/647.py

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        def extendPalindromes(l: int, r: int) -> int:
            count = 0

            while l >= 0 and r < len(s) and s[l] == s[r]:
                count += 1
                l -= 1
                r += 1

            return count

        ans = 0

        for i in range(len(s)):
            ans += extendPalindromes(i, i)
            ans += extendPalindromes(i, i + 1)

        return ans
```

2826. 648.py

Path: LeetCode-main/solutions/648. Replace Words/648.py

```
class Solution:
    def __init__(self):
        self.root = {}

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node:
                node[c] = {}
            node = node[c]
        node['word'] = word

    def search(self, word: str) -> str:
        node = self.root
        for c in word:
            if 'word' in node:
                return node['word']
            if c not in node:
                return word
            node = node[c]
        return word

    def replaceWords(self, dictionary: list[str], sentence: str) -> str:
        for word in dictionary:
            self.insert(word)

        words = sentence.split(' ')
        return ' '.join([self.search(word) for word in words])
```

2827. 65.py

Path: LeetCode-main/solutions/65. Valid Number/65.py

```

class Solution:
    def isNumber(self, s: str) -> bool:
        s = s.strip()
        if not s:
            return False

        seenNum = False
        seenDot = False
        seenE = False

        for i, c in enumerate(s):
            if c == '.':
                if seenDot or seenE:
                    return False
                seenDot = True
            elif c == 'e' or c == 'E':
                if seenE or not seenNum:
                    return False
                seenE = True
                seenNum = False
            elif c in '+-':
                if i > 0 and s[i - 1] not in 'eE':
                    return False
                seenNum = False
            else:
                if not c.isdigit():
                    return False
                seenNum = True

        return seenNum

```

2828. 650.py

Path: LeetCode-main/solutions/650. 2 Keys Keyboard/650.py

```

class Solution:
    def minSteps(self, n: int) -> int:
        if n <= 1:
            return 0

        # dp[i] := the minimum steps to get i 'A's
        # Copy 'A', then paste 'A' i - 1 times.
        dp = [i for i in range(n + 1)]

        for i in range(2, n + 1):
            for j in range(i // 2, 2, -1):
                if i % j == 0:
                    dp[i] = dp[j] + i // j # Paste dp[j] i / j times.
                    break

        return dp[n]

```

2829. 652.py

Path: LeetCode-main/solutions/652. Find Duplicate Subtrees/652.py

```
class Solution:
    def findDuplicateSubtrees(self, root: TreeNode | None) -> list[TreeNode | None]:
        ans = []
        count = collections.Counter()

        def encode(root: TreeNode | None) -> str:
            if not root:
                return ''

            encoded = (str(root.val) + '#' +
                      encode(root.left) + '#' +
                      encode(root.right))
            count[encoded] += 1
            if count[encoded] == 2:
                ans.append(root)
            return encoded

        encode(root)
        return ans
```

2830. 653.py

Path: LeetCode-main/solutions/653. Two Sum IV - Input is a BST/653.py

```

class BSTIterator:
    def __init__(self, root: TreeNode | None, leftToRight: bool):
        self.stack = []
        self.leftToRight = leftToRight
        self._pushUntilNone(root)

    def next(self) -> int:
        node = self.stack.pop()
        if self.leftToRight:
            self._pushUntilNone(node.right)
        else:
            self._pushUntilNone(node.left)
        return node.val

    def _pushUntilNone(self, root: TreeNode | None):
        while root:
            self.stack.append(root)
            root = root.left if self.leftToRight else root.right

class Solution:
    def findTarget(self, root: TreeNode | None, k: int) -> bool:
        if not root:
            return False

        left = BSTIterator(root, True)
        right = BSTIterator(root, False)

        l = left.next()
        r = right.next()
        while l < r:
            summ = l + r
            if summ == k:
                return True
            if summ < k:
                l = left.next()
            else:
                r = right.next()

        return False

```

2831. 654.py

Path: LeetCode-main/solutions/654. Maximum Binary Tree/654.py

```

class Solution:
    def constructMaximumBinaryTree(self, nums: list[int]) -> TreeNode | None:
        def build(i: int, j: int) -> TreeNode | None:
            if i > j:
                return None

            maxNum = max(nums[i:j + 1])
            maxIndex = nums.index(maxNum)

            root = TreeNode(maxNum)
            root.left = build(i, maxIndex - 1)
            root.right = build(maxIndex + 1, j)
            return root

        return build(0, len(nums) - 1)

```

2832. 655.py

Path: LeetCode-main/solutions/655. Print Binary Tree/655.py

```
class Solution:
    def printTree(self, root: TreeNode | None) -> list[list[str]]:
        def maxHeight(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + max(maxHeight(root.left), maxHeight(root.right))

        def dfs(root: TreeNode | None, row: int, left: int, right: int) -> None:
            if not root:
                return

            mid = (left + right) // 2
            ans[row][mid] = str(root.val)
            dfs(root.left, row + 1, left, mid - 1)
            dfs(root.right, row + 1, mid + 1, right)

        m = maxHeight(root)
        n = pow(2, m) - 1
        ans = [[''] * n for _ in range(m)]
        dfs(root, 0, 0, len(ans[0]) - 1)
        return ans
```

2833. 656-2.py

Path: LeetCode-main/solutions/656. Coin Path/656-2.py

```
class Solution:
    def cheapestJump(self, coins: list[int], maxJump: int) -> list[int]:
        if coins[-1] == -1:
            return []

        n = len(coins)
        # dp[i] := the minimum cost to jump from i to n - 1
        dp = [math.inf] * n
        next = [-1] * n

        dp[-1] = coins[-1]

        for i in reversed(range(n - 1)):
            if coins[i] == -1:
                continue
            for j in range(i + 1, min(i + maxJump + 1, n)):
                if dp[j] == math.inf:
                    continue
                cost = coins[i] + dp[j]
                if cost < dp[i]:
                    dp[i] = cost
                    next[i] = j

        if dp[0] == math.inf:
            return []
        return self._constructPath(next, 0)

    def _constructPath(self, next: list[int], i: int) -> list[int]:
        ans = []
        while i != -1:
            ans.append(i + 1) # 1-indexed
            i = next[i]
        return ans
```

2834. 656.py

Path: LeetCode-main/solutions/656. Coin Path/656.py


```

class Solution:
    def cheapestJump(self, coins: list[int], maxJump: int) -> list[int]:
        if coins[-1] == -1:
            return []

        n = len(coins)
        # dp[i] := the minimum cost to jump from i to n - 1
        dp = [math.inf] * n
        next = [-1] * n

        def cheapestJump(i: int) -> int:
            if i == len(coins) - 1:
                dp[i] = coins[i]
                return dp[i]
            if dp[i] < math.inf:
                return dp[i]
            if coins[i] == -1:
                return math.inf

            for j in range(i + 1, min(i + maxJump + 1, n)):
                res = cheapestJump(j)
                if res == math.inf:
                    continue
                cost = coins[i] + res
                if cost < dp[i]:
                    dp[i] = cost
                    next[i] = j

            return dp[i]

        cheapestJump(0)
        if dp[0] == math.inf:
            return []
        return self._constructPath(next, 0)

    def _constructPath(self, next: list[int], i: int) -> list[int]:
        ans = []
        while i != -1:
            ans.append(i + 1) # 1-indexed
            i = next[i]
        return ans

```

2835. 657.py

Path: LeetCode-main/solutions/657. Robot Return to Origin/657.py

```

class Solution:
    def judgeCircle(self, moves: str) -> bool:
        return moves.count('R') == moves.count('L') and moves.count('U') == moves.count('D')

```

2836. 658.py

Path: LeetCode-main/solutions/658. Find K Closest Elements/658.py

```
class Solution:
    def findClosestElements(self, arr: list[int], k: int, x: int) -> list[int]:
        l = 0
        r = len(arr) - k

        while l < r:
            m = (l + r) // 2
            if x - arr[m] <= arr[m + k] - x:
                r = m
            else:
                l = m + 1

        return arr[l:l + k]
```

2837. 66.py

Path: LeetCode-main/solutions/66. Plus One/66.py

```
class Solution:
    def plusOne(self, digits: list[int]) -> list[int]:
        for i, d in reversed(list(enumerate(digits))):
            if d < 9:
                digits[i] += 1
                return digits
            digits[i] = 0

        return [1] + digits
```

2838. 660.py

Path: LeetCode-main/solutions/660. Remove 9/660.py

```
class Solution:
    def newInteger(self, n: int) -> int:
        ans = []
        while n:
            ans.append(str(n % 9))
            n //= 9
        return ''.join(reversed(ans))
```

2839. 661.py

Path: LeetCode-main/solutions/661. Image Smoother/661.py

```

class Solution:
    def imageSmoother(self, M: list[list[int]]) -> list[list[int]]:
        m = len(M)
        n = len(M[0])
        ans = [[0 for j in range(n)] for i in range(m)]

        for i in range(m):
            for j in range(n):
                ones = 0
                count = 0
                for y in range(max(0, i - 1), min(m, i + 2)):
                    for x in range(max(0, j - 1), min(n, j + 2)):
                        ones += M[y][x]
                        count += 1
                ans[i][j] = ones // count

        return ans

```

2840. 663.py

Path: LeetCode-main/solutions/663. Equal Tree Partition/663.py

```

class Solution:
    def checkEqualTree(self, root: TreeNode | None) -> bool:
        if not root:
            return False

        seen = set()

        def dfs(root: TreeNode | None) -> int:
            if not root:
                return 0

            summ = root.val + dfs(root.left) + dfs(root.right)
            seen.add(summ)
            return summ

        summ = root.val + dfs(root.left) + dfs(root.right)
        return summ % 2 == 0 and summ // 2 in seen

```

2841. 665.py

Path: LeetCode-main/solutions/665. Non-decreasing Array/665.py

```

class Solution:
    def checkPossibility(self, nums: list[int]) -> bool:
        j = None

        for i in range(len(nums) - 1):
            if nums[i] > nums[i + 1]:
                if j is not None:
                    return False
                j = i

        return (j is None or j == 0 or j == len(nums) - 2 or
                nums[j - 1] <= nums[j + 1] or nums[j] <= nums[j + 2])

```

2842. 666.py

Path: LeetCode-main/solutions/666. Path Sum IV/666.py

```
class Solution:
    def pathSum(self, nums: list[int]) -> int:
        ans = 0
        tree = [[-1] * 8 for _ in range(4)]

        for num in nums:
            d = num // 100 - 1
            p = (num % 100) // 10 - 1
            v = num % 10
            tree[d][p] = v

        def dfs(i: int, j: int, path: int) -> None:
            nonlocal ans
            if tree[i][j] == -1:
                return
            if i == 3 or max(tree[i + 1][j * 2], tree[i + 1][j * 2 + 1]) == -1:
                ans += path + tree[i][j]
                return

            dfs(i + 1, j * 2, path + tree[i][j])
            dfs(i + 1, j * 2 + 1, path + tree[i][j])

        dfs(0, 0, 0)
        return ans
```

2843. 667.py

Path: LeetCode-main/solutions/667. Beautiful Arrangement II/667.py

```
class Solution:
    def constructArray(self, n: int, k: int) -> list[int]:
        ans = list(range(1, n - k + 1))

        for i in range(k):
            if i % 2 == 0:
                ans.append(n - i // 2)
            else:
                ans.append(n - k + (i + 1) // 2)

        return ans
```

2844. 67.py

Path: LeetCode-main/solutions/67. Add Binary/67.py

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        carry = 0
        i = len(a) - 1
        j = len(b) - 1

        while i >= 0 or j >= 0 or carry:
            if i >= 0:
                carry += int(a[i])
                i -= 1
            if j >= 0:
                carry += int(b[j])
                j -= 1
            ans.append(str(carry % 2))
            carry //= 2

        return ''.join(reversed(ans))
```

2845. 670.py

Path: LeetCode-main/solutions/670. Maximum Swap/670.py

```
class Solution:
    def maximumSwap(self, num: int) -> int:
        s = list(str(num))
        dict = {c: i for i, c in enumerate(s)}

        for i, c in enumerate(s):
            for digit in reversed(string.digits):
                if digit <= c:
                    break
                if digit in dict and dict[digit] > i:
                    s[i], s[dict[digit]] = digit, s[i]
                    return int(''.join(s))

        return num
```

2846. 672.py

Path: LeetCode-main/solutions/672. Bulb Switcher II/672.py

```
class Solution:
    def flipLights(self, n: int, m: int) -> int:
        n = min(n, 3)

        if m == 0:
            return 1
        if m == 1:
            return [2, 3, 4][n - 1]
        if m == 2:
            return [2, 4, 7][n - 1]

        return [2, 4, 8][n - 1]
```

2847. 673.py

Path: LeetCode-main/solutions/673. Number of Longest Increasing Subsequence/673.py

```
class Solution:
    def findNumberOfLIS(self, nums: list[int]) -> int:
        ans = 0
        maxLength = 0
        # length[i] := the length of the LIS ending in nums[i]
        length = [1] * len(nums)
        # count[i] := the number of LIS's ending in nums[i]
        count = [1] * len(nums)

        # Calculate the `length` and `count` arrays.
        for i, num in enumerate(nums):
            for j in range(i):
                if nums[j] < num:
                    if length[i] < length[j] + 1:
                        length[i] = length[j] + 1
                        count[i] = count[j]
                    elif length[i] == length[j] + 1:
                        count[i] += count[j]

        # Get the number of LIS.
        for i, l in enumerate(length):
            if l > maxLength:
                maxLength = l
                ans = count[i]
            elif l == maxLength:
                ans += count[i]

        return ans
```

2848. 674.py

Path: LeetCode-main/solutions/674. Longest Continuous Increasing Subsequence/674.py

```
class Solution:
    def findLengthOfLCIS(self, nums: list[int]) -> int:
        ans = 0
        j = 0

        for i in range(len(nums)):
            if i > 0 and nums[i] <= nums[i - 1]:
                j = i
            ans = max(ans, i - j + 1)

        return ans
```

2849. 676-2.py

Path: LeetCode-main/solutions/676. Implement Magic Dictionary/676-2.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class MagicDictionary:
    def __init__(self):
        self.root = TrieNode()

    def buildDict(self, dictionary: list[str]) -> None:
        for word in dictionary:
            self._insert(word)

    def search(self, searchWord: str) -> bool:
        node: TrieNode = self.root
        for i, c in enumerate(searchWord):
            for letter in string.ascii_lowercase:
                if letter == c:
                    continue
                if letter not in node.children:
                    continue
                child = node.children[letter]
                # Replace the searchWord[i] with `letter`, then check if
                # searchWord[i + 1..n) matches `child`.
                if self._find(child, searchWord, i + 1):
                    return True
            if c not in node.children:
                return False
            node = node.children[c]
        return False

    def _insert(self, word: str) -> None:
        node: TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

    def _find(self, node: TrieNode, word: str, i: int) -> bool:
        for c in word[i:]:
            if c not in node.children:
                return False
            node = node.children[c]
        return node.isWord

```

2850. 676.py

Path: LeetCode-main/solutions/676. Implement Magic Dictionary/676.py

```

class MagicDictionary:
    def __init__(self):
        self.dict = {}

    def buildDict(self, dictionary: list[str]) -> None:
        for word in dictionary:
            for i, c in enumerate(word):
                replaced = self._getReplaced(word, i)
                self.dict[replaced] = '*' if replaced in self.dict else c

    def search(self, searchWord: str) -> bool:
        for i, c in enumerate(searchWord):
            replaced = self._getReplaced(searchWord, i)
            if self.dict.get(replaced, c) != c:
                return True
        return False

    def _getReplaced(self, s: str, i: int) -> str:
        return s[:i] + '*' + s[i + 1:]

```

2851. 677.py

Path: LeetCode-main/solutions/677. Map Sum Pairs/677.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.sum = 0

class MapSum:
    def __init__(self):
        self.root = TrieNode()
        self.keyToVal = {}

    def insert(self, key: str, val: int) -> None:
        diff = val - self.keyToVal.get(key, 0)
        node: TrieNode = self.root
        for c in key:
            node = node.children.setdefault(c, TrieNode())
            node.sum += diff
        self.keyToVal[key] = val

    def sum(self, prefix: str) -> int:
        node: TrieNode = self.root
        for c in prefix:
            if c not in node.children:
                return 0
            node = node.children[c]
        return node.sum

```

2852. 678.py

Path: LeetCode-main/solutions/678. Valid Parenthesis String/678.py


```

class Solution:
    def checkValidString(self, s: str) -> bool:
        low = 0
        high = 0

        for c in s:
            if c == '(':
                low += 1
                high += 1
            elif c == ')':
                if low > 0:
                    low -= 1
                    high -= 1
                else:
                    if low > 0:
                        low -= 1
                    high += 1
            if high < 0:
                return False

        return low == 0

```

2853. 679.py

Path: LeetCode-main/solutions/679. 24 Game/679.py

```

class Solution:
    def judgePoint24(self, nums: list[int]) -> bool:
        def generate(a: float, b: float) -> list[float]:
            return [a * b,
                    math.inf if b == 0 else a / b,
                    math.inf if a == 0 else b / a,
                    a + b, a - b, b - a]

        def dfs(nums: list[float]) -> bool:
            if len(nums) == 1:
                return abs(nums[0] - 24.0) < 0.001

            for i, j in itertools.combinations(range(len(nums)), 2):
                for num in generate(nums[i], nums[j]):
                    nextRound = [num]
                    for k in range(len(nums)):
                        if k == i or k == j:
                            continue
                        nextRound.append(nums[k])
                    if dfs(nextRound):
                        return True

            return False

        return dfs(nums)

```

2854. 68.py

Path: LeetCode-main/solutions/68. Text Justification/68.py

```

class Solution:
    def fullJustify(self, words: list[str], maxWidth: int) -> list[str]:
        ans = []
        row = []
        rowLetters = 0

        for word in words:
            # If we place the word in this row, it will exceed the maximum width.
            # Therefore, we cannot put the word in this row and have to pad spaces
            # for each word in this row.
            if rowLetters + len(word) + len(row) > maxWidth:
                for i in range(maxWidth - rowLetters):
                    row[i % (len(row) - 1 or 1)] += ' '
                ans.append(''.join(row))
                row = []
                rowLetters = 0
            row.append(word)
            rowLetters += len(word)

        return ans + [' '.join(row).ljust(maxWidth)]

```

2855. 680.py

Path: LeetCode-main/solutions/680. Valid Palindrome II/680.py

```

class Solution:
    def validPalindrome(self, s: str) -> bool:
        def validPalindrome(l: int, r: int) -> bool:
            return all(s[i] == s[r - i + 1] for i in range(l, (l + r) // 2 + 1))

        n = len(s)

        for i in range(n // 2):
            if s[i] != s[~i]:
                return validPalindrome(i + 1, n - 1 - i) or validPalindrome(i, n - 2 - i)

        return True

```

2856. 681.py

Path: LeetCode-main/solutions/681. Next Closest Time/681.py

```

class Solution:
    def nextClosestTime(self, time: str) -> str:
        ans = list(time)
        digits = sorted(ans)

        def nextClosest(digit: str, limit: str) -> str:
            next = bisect_right(digits, digit)
            return digits[0] if next == 4 or digits[next] > limit else digits[next]

        ans[4] = nextClosest(ans[4], '9')
        if time[4] < ans[4]:
            return ''.join(ans)

        ans[3] = nextClosest(ans[3], '5')
        if time[3] < ans[3]:
            return ''.join(ans)

        ans[1] = nextClosest(ans[1], '3' if ans[0] == '2' else '9')
        if time[1] < ans[1]:
            return ''.join(ans)

        ans[0] = nextClosest(ans[0], '2')
        return ''.join(ans)

```

2857. 682.py

Path: LeetCode-main/solutions/682. Baseball Game/682.py

```

class Solution:
    def calPoints(self, operations: list[str]) -> int:
        scores = []

        for operation in operations:
            match operation:
                case '+':
                    scores.append(scores[-1] + scores[-2])
                case 'D':
                    scores.append(scores[-1] * 2)
                case 'C':
                    scores.pop()
                case default:
                    scores.append(int(operation))

        return sum(scores)

```

2858. 683.py

Path: LeetCode-main/solutions/683. K Empty Slots/683.py

```
class Solution:
    def kEmptySlots(self, bulbs: list[int], k: int) -> int:
        n = len(bulbs)
        ans = math.inf
        # day[i] := the day when bulbs[i] is turned on
        day = [0] * n

        for i, bulb in enumerate(bulbs):
            day[bulb - 1] = i + 1

        # Find a subarray of day[l..r], where its length is k + 2.
        # For each l < i < r, day[i] > max(day[l], day[r]).
        l = 0
        r = l + k + 1
        i = 1
        while r < n:
            if i == r:
                ans = min(ans, max(day[l], day[r]))
                l = i
                r = i + k + 1
            elif day[i] < max(day[l], day[r]):
                l = i
                r = i + k + 1
            i += 1

        return -1 if ans == math.inf else ans
```

2859. 684.py

Path: LeetCode-main/solutions/684. Redundant Connection/684.py

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findRedundantConnection(self, edges: list[list[int]]) -> list[int]:
        uf = UnionFind(len(edges) + 1)

        for edge in edges:
            u, v = edge
            if not uf.unionByRank(u, v):
                return edge
```

2860. 685.py

Path: LeetCode-main/solutions/685. Redundant Connection II/685.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findRedundantDirectedConnection(
        self, edges: list[list[int]],
    ) -> list[int]:
        ids = [0] * (len(edges) + 1)
        nodeWithTwoParents = 0

        for _, v in edges:
            ids[v] += 1
            if ids[v] == 2:
                nodeWithTwoParents = v

        def findRedundantDirectedConnection(skippedEdgeIndex: int) -> list[int]:
            uf = UnionFind(len(edges) + 1)

            for i, edge in enumerate(edges):
                if i == skippedEdgeIndex:
                    continue
                if not uf.unionByRank(edge[0], edge[1]):
                    return edge

            return []

        # If there is no edge with two ids, don't skip any edge.
        if nodeWithTwoParents == 0:
            return findRedundantDirectedConnection(-1)

        for i in reversed(range(len(edges))):
            _, v = edges[i]
            if v == nodeWithTwoParents:
                # Try to delete the edges[i].
                if not findRedundantDirectedConnection(i):
                    return edges[i]

```

2861. 686.py

Path: LeetCode-main/solutions/686. Repeated String Match/686.py

```

class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        n = math.ceil(len(b) / len(a))
        s = a * n
        if b in s:
            return n
        if b in s + a:
            return n + 1
        return -1

```

2862. 687.py

Path: LeetCode-main/solutions/687. Longest Univalue Path/687.py

```

class Solution:
    def longestUnivaluePath(self, root: TreeNode | None) -> int:
        ans = 0

        def longestUnivaluePathDownFrom(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = longestUnivaluePathDownFrom(root.left)
            r = longestUnivaluePathDownFrom(root.right)
            arrowLeft = l + 1 if root.left and root.left.val == root.val else 0
            arrowRight = r + 1 if root.right and root.right.val == root.val else 0
            ans = max(ans, arrowLeft + arrowRight)
            return max(arrowLeft, arrowRight)

        longestUnivaluePathDownFrom(root)
        return ans

```

2863. 688.py

Path: LeetCode-main/solutions/688. Knight Probability in Chessboard/688.py

```

class Solution:
    def knightProbability(self, n: int, k: int, row: int, column: int) -> float:
        DIRS = ((1, 2), (2, 1), (2, -1), (1, -2),
                (-1, -2), (-2, -1), (-2, 1), (-1, 2))

        # dp[i][j] := the probability to stand on (i, j)
        dp = [[0] * n for _ in range(n)]
        dp[row][column] = 1.0

        for _ in range(k):
            newDp = [[0] * n for _ in range(n)]
            for i in range(n):
                for j in range(n):
                    for dx, dy in DIRS:
                        x = i + dx
                        y = j + dy
                        if 0 <= x < n and 0 <= y < n:
                            newDp[i][j] += dp[x][y]
            dp = newDp

        return sum(map(sum, dp)) / 8**k

```

2864. 689.py

Path: LeetCode-main/solutions/689. Maximum Sum of 3 Non-Overlapping Subarrays/689.py

```
class Solution:
    def maxSumOfThreeSubarrays(self, nums: list[int], k: int) -> list[int]:
        n = len(nums) - k + 1
        # sums[i] := sum(nums[i..i + k))
        sums = [0] * n
        # l[i] := the index in [0..i] that has the maximum sums[i]
        l = [0] * n
        # r[i] := the index in [i..n) that has the maximum sums[i]
        r = [0] * n

        summ = 0
        for i, num in enumerate(nums):
            summ += num
            if i >= k:
                summ -= nums[i - k]
            if i >= k - 1:
                sums[i - k + 1] = summ

        maxIndex = 0
        for i in range(n):
            if sums[i] > sums[maxIndex]:
                maxIndex = i
            l[i] = maxIndex

        maxIndex = n - 1
        for i in range(n - 1, -1, -1):
            if sums[i] >= sums[maxIndex]:
                maxIndex = i
            r[i] = maxIndex

        ans = [-1, -1, -1]

        for i in range(k, n - k):
            if (ans[0] == -1 or
                sums[ans[0]] + sums[ans[1]] + sums[ans[2]] <
                sums[l[i - k]] + sums[i] + sums[r[i + k]]):
                ans[0] = l[i - k]
                ans[1] = i
                ans[2] = r[i + k]

        return ans
```

2865. 69.py

Path: LeetCode-main/solutions/69. Sqrt(x)/69.py

```
class Solution:
    def mySqrt(self, x: int) -> int:
        return bisect.bisect_right(range(x + 1), x,
                                    key=lambda m: m * m) - 1
```

2866. 690.py

Path: LeetCode-main/solutions/690. Employee Importance/690.py


```

class Solution:
    def getImportance(self, employees: list['Employee'], id: int) -> int:
        idToEmployee = {employee.id: employee for employee in employees}

        def dfs(id: int) -> int:
            values = idToEmployee[id].importance
            for subId in idToEmployee[id].subordinates:
                values += dfs(subId)
            return values

        return dfs(id)

```

2867. 691.py

Path: LeetCode-main/solutions/691. Stickers to Spell Word/691.py

```

class Solution:
    def minStickers(self, stickers: list[str], target: str) -> int:
        maxMask = 1 << len(target)
        # dp[i] := the minimum number of stickers to spell out i, where i is the
        # bit mask of target
        dp = [math.inf] * maxMask
        dp[0] = 0

        for mask in range(maxMask):
            if dp[mask] == math.inf:
                continue
            # Try to expand from `mask` by using each sticker.
            for sticker in stickers:
                superMask = mask
                for c in sticker:
                    for i, t in enumerate(target):
                        # Try to apply it on a missing letter.
                        if c == t and not (superMask >> i & 1):
                            superMask |= 1 << i
                            break
                dp[superMask] = min(dp[superMask], dp[mask] + 1)

        return -1 if dp[-1] == math.inf else dp[-1]

```

2868. 692-2.py

Path: LeetCode-main/solutions/692. Top K Frequent Words/692-2.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    word: str
    freq: int

    def __lt__(self, other):
        if self.freq == other.freq:
            return self.word > other.word
        return self.freq < other.freq

class Solution:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        ans = []
        heap = []

        for word, freq in collections.Counter(words).items():
            heapq.heappush(heap, T(word, freq))
            if len(heap) > k:
                heapq.heappop(heap)

        while heap:
            ans.append(heapq.heappop(heap).word)

        return ans[::-1]

```

2869. 692.py

Path: LeetCode-main/solutions/692. Top K Frequent Words/692.py

```

class Solution:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        ans = []
        bucket = [[] for _ in range(len(words) + 1)]

        for word, freq in collections.Counter(words).items():
            bucket[freq].append(word)

        for b in reversed(bucket):
            for word in sorted(b):
                ans.append(word)
                if len(ans) == k:
                    return ans

```

2870. 693.py

Path: LeetCode-main/solutions/693. Binary Number with Alternating Bits/693.py

```

class Solution:
    def hasAlternatingBits(self, n: int) -> bool:
        #         n = 0b010101
        #         n >> 2 = 0b000101
        # n ^ (n >> 2) = 0b010000 = a
        #         a - 1 = 0b001111
        # a & (a - 1) = 0
        a = n ^ (n >> 2)
        return (a & (a - 1)) == 0

```

2871. 694.py

Path: LeetCode-main/solutions/694. Number of Distinct Islands/694.py

```

class Solution:
    def numDistinctIslands(self, grid: list[list[int]]) -> int:
        seen = set()

        def dfs(i: int, j: int, i0: int, j0: int):
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return
            if grid[i][j] == 0 or (i, j) in seen:
                return

            seen.add((i, j))
            island.append((i - i0, j - j0))
            dfs(i + 1, j, i0, j0)
            dfs(i - 1, j, i0, j0)
            dfs(i, j + 1, i0, j0)
            dfs(i, j - 1, i0, j0)

        islands = set() # all the different islands

        for i in range(len(grid)):
            for j in range(len(grid[0])):
                island = []
                dfs(i, j, i, j)
                if island:
                    islands.add(frozenset(island))

        return len(islands)

```

2872. 695.py

Path: LeetCode-main/solutions/695. Max Area of Island/695.py

```

class Solution:
    def maxAreaOfIsland(self, grid: list[list[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return 0
            if grid[i][j] != 1:
                return 0

            grid[i][j] = 2

            return 1 + dfs(i + 1, j) + dfs(i - 1, j) + dfs(i, j + 1) + dfs(i, j - 1)

        return max(dfs(i, j) for i in range(len(grid)) for j in range(len(grid[0])))

```

2873. 696.py

Path: LeetCode-main/solutions/696. Count Binary Substrings/696.py

```
class Solution:
    def countBinarySubstrings(self, s: str) -> int:
        ans = 0
        prevCount = 0
        equals = 1

        for i in range(len(s) - 1):
            if s[i] == s[i + 1]:
                equals += 1
            else:
                ans += min(prevCount, equals)
                prevCount = equals
                equals = 1

        return ans + min(prevCount, equals)
```

2874. 697.py

Path: LeetCode-main/solutions/697. Degree of an Array/697.py

```
class Solution:
    def findShortestSubArray(self, nums: list[int]) -> int:
        ans = 0
        degree = 0
        debut = {}
        count = collections.Counter()

        for i, num in enumerate(nums):
            debut.setdefault(num, i)
            count[num] += 1
            if count[num] > degree:
                degree = count[num]
                ans = i - debut[num] + 1
            elif count[num] == degree:
                ans = min(ans, i - debut[num] + 1)

        return ans
```

2875. 698.py

Path: LeetCode-main/solutions/698. Partition to K Equal Sum Subsets/698.py

```

class Solution:
    def canPartitionKSubsets(self, nums: list[int], k: int) -> bool:
        summ = sum(nums)
        if summ % k != 0:
            return False

        target = summ // k # the target sum of each subset
        if any(num > target for num in nums):
            return False

        def dfs(s: int, remainingGroups: int, currSum: int, used: int) -> bool:
            if remainingGroups == 0:
                return True
            if currSum > target:
                return False
            if currSum == target: # Find a valid group, so fresh start.
                return dfs(0, remainingGroups - 1, 0, used)

            for i in range(s, len(nums)):
                if used >> i & 1:
                    continue
                if dfs(i + 1, remainingGroups, currSum + nums[i], used | 1 << i):
                    return True

            return False

        nums.sort(reverse=True)
        return dfs(0, k, 0, 0)

```

2876. 7.py

Path: LeetCode-main/solutions/7. Reverse Integer/7.py

```

class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        sign = -1 if x < 0 else 1
        x *= sign

        while x:
            ans = ans * 10 + x % 10
            x //= 10

        return 0 if ans < -2**31 or ans > 2**31 - 1 else sign * ans

```

2877. 70-2.py

Path: LeetCode-main/solutions/70. Climbing Stairs/70-2.py

```
class Solution:
    def climbStairs(self, n: int) -> int:
        prev1 = 1 # dp[i - 1]
        prev2 = 1 # dp[i - 2]

        for _ in range(2, n + 1):
            dp = prev1 + prev2
            prev2 = prev1
            prev1 = dp

        return prev1
```

2878. 70.py

Path: LeetCode-main/solutions/70. Climbing Stairs/70.py

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # dp[i] := the number of ways to climb to the i-th stair
        dp = [1, 1] + [0] * (n - 1)

        for i in range(2, n + 1):
            dp[i] = dp[i - 1] + dp[i - 2]

        return dp[n]
```

2879. 700.py

Path: LeetCode-main/solutions/700. Search in a Binary Search Tree/700.py

```
class Solution:
    def searchBST(self, root: TreeNode | None, val: int) -> TreeNode | None:
        if not root:
            return None
        if root.val == val:
            return root
        if root.val > val:
            return self.searchBST(root.left, val)
        return self.searchBST(root.right, val)
```

2880. 701.py

Path: LeetCode-main/solutions/701. Insert into a Binary Search Tree/701.py

```
class Solution:
    def insertIntoBST(self, root: TreeNode | None, val: int) -> TreeNode | None:
        if not root:
            return TreeNode(val)
        if root.val > val:
            root.left = self.insertIntoBST(root.left, val)
        else:
            root.right = self.insertIntoBST(root.right, val)
        return root
```

2881. 702.py

Path: LeetCode-main/solutions/702. Search in a Sorted Array of Unknown Size/702.py

```
# """
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class ArrayReader:
#     def get(self, index: int) -> int:

class Solution:
    def search(self, reader: 'ArrayReader', target: int) -> int:
        l = bisect.bisect_left(range(10**4), target,
                                key=lambda m: reader.get(m))
        return l if reader.get(l) == target else -1
```

2882. 704.py

Path: LeetCode-main/solutions/704. Binary Search/704.py

```
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        i = bisect.bisect_left(nums, target)
        return -1 if i == len(nums) or nums[i] != target else i
```

2883. 705.py

Path: LeetCode-main/solutions/705. Design HashSet/705.py

```
class MyHashSet:
    def __init__(self):
        self.set = [False] * 1000001

    def add(self, key: int) -> None:
        self.set[key] = True

    def remove(self, key: int) -> None:
        self.set[key] = False

    def contains(self, key: int) -> bool:
        return self.set[key]
```

2884. 707.py

Path: LeetCode-main/solutions/707. Design Linked List/707.py

```

from dataclasses import dataclass

@dataclass
class ListNode:
    val: int
    next: ListNode | None = None

class MyLinkedList:
    def __init__(self):
        self.length = 0
        self.dummy = ListNode(0)

    def get(self, index: int) -> int:
        if index < 0 or index >= self.length:
            return -1
        curr = self.dummy.next
        for _ in range(index):
            curr = curr.next
        return curr.val

    def addAtHead(self, val: int) -> None:
        curr = self.dummy.next
        self.dummy.next = ListNode(val)
        self.dummy.next.next = curr
        self.length += 1

    def addAtTail(self, val: int) -> None:
        curr = self.dummy
        while curr.next:
            curr = curr.next
        curr.next = ListNode(val)
        self.length += 1

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.length:
            return
        curr = self.dummy
        for _ in range(index):
            curr = curr.next
        temp = curr.next
        curr.next = ListNode(val)
        curr.next.next = temp
        self.length += 1

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.length:
            return
        curr = self.dummy
        for _ in range(index):
            curr = curr.next
        temp = curr.next
        curr.next = temp.next
        self.length -= 1

```

2885. 709.py

Path: LeetCode-main/solutions/709. To Lower Case/709.py


```
class Solution:
    def toLowerCase(self, str: str) -> str:
        return ''.join(chr(ord(c) + 32) if 'A' <= c <= 'Z' else c for c in str)
```

2886. 71.py

Path: LeetCode-main/solutions/71. Simplify Path/71.py

```
class Solution:
    def simplifyPath(self, path: str) -> str:
        stack = []

        for str in path.split('/'):
            if str in ('', '.'):
                continue
            if str == '..':
                if stack:
                    stack.pop()
            else:
                stack.append(str)

        return '/' + '/'.join(stack)
```

2887. 710.py

Path: LeetCode-main/solutions/710. Random Pick with Blacklist/710.py

```
class Solution:
    def __init__(self, n: int, blacklist: list[int]):
        self.validRange = n - len(blacklist)
        self.dict = {}

        maxAvailable = n - 1

        for b in blacklist:
            self.dict[b] = -1

        for b in blacklist:
            if b < self.validRange:
                # Find the slot that haven't been used.
                while maxAvailable in self.dict:
                    maxAvailable -= 1
                self.dict[b] = maxAvailable
                maxAvailable -= 1

    def pick(self) -> int:
        value = random.randint(0, self.validRange - 1)

        if value in self.dict:
            return self.dict[value]

        return value
```

2888. 711.py

Path: LeetCode-main/solutions/711. Number of Distinct Islands II/711.py

```

class Solution:
    def numDistinctIslands2(self, grid: list[list[int]]) -> int:
        seen = set()

        def dfs(i: int, j: int):
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return
            if grid[i][j] == 0 or (i, j) in seen:
                return

            seen.add((i, j))
            island.append((i, j))
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        def normalize(island: list[tuple]) -> list[tuple]:
            # points[i] := 8 different rotations/reflections of an island
            points = [[] for _ in range(8)]

            for i, j in island:
                points[0].append((i, j))
                points[1].append((i, -j))
                points[2].append((-i, j))
                points[3].append((-i, -j))
                points[4].append((j, i))
                points[5].append((j, -i))
                points[6].append((-j, i))
                points[7].append((-j, -i))

            points = [sorted(p) for p in points]

            # Normalize each p by subtracting p[1..7] with p[0].
            for p in points:
                for i in range(1, len(island)):
                    p[i] = (p[i][0] - p[0][0],
                           p[i][1] - p[0][1])
                p[0] = (0, 0)

            return sorted(points)[0]

        islands = set() # all the islands with different shapes

        for i in range(len(grid)):
            for j in range(len(grid[0])):
                island = []
                dfs(i, j)
                if island:
                    islands.add(frozenset(normalize(island)))

        return len(islands)

```

2889. 713.py

Path: LeetCode-main/solutions/713. Subarray Product Less Than K/713.py

```

class Solution:
    def numSubarrayProductLessThanK(self, nums: list[int], k: int) -> int:
        if k <= 1:
            return 0

        ans = 0
        prod = 1

        j = 0
        for i, num in enumerate(nums):
            prod *= num
            while prod >= k:
                prod /= nums[j]
                j += 1
            ans += i - j + 1

        return ans

```

2890. 714.py

Path: LeetCode-main/solutions/714. Best Time to Buy and Sell Stock with Transaction Fee/714.py

```

class Solution:
    def maxProfit(self, prices: list[int], fee: int) -> int:
        sell = 0
        hold = -math.inf

        for price in prices:
            sell = max(sell, hold + price)
            hold = max(hold, sell - price - fee)

        return sell

```

2891. 715-3.py

Path: LeetCode-main/solutions/715. Range Module/715-3.py

```

class RangeModule:
    def __init__(self):
        self.A = []

    def addRange(self, left: int, right: int) -> None:
        i = bisect_left(self.A, left)
        j = bisect_right(self.A, right)
        self.A[i:j] = [left] * (i % 2 == 0) + [right] * (j % 2 == 0)

    def queryRange(self, left: int, right: int) -> bool:
        i = bisect_right(self.A, left)
        j = bisect_left(self.A, right)
        return i == j and i % 2 == 1

    def removeRange(self, left: int, right: int) -> None:
        i = bisect_left(self.A, left)
        j = bisect_right(self.A, right)
        self.A[i:j] = [left] * (i % 2 == 1) + [right] * (j % 2 == 1)

```

2892. 717.py

Path: LeetCode-main/solutions/717. 1-bit and 2-bit Characters/717.py

```
class Solution:
    def isOneBitCharacter(self, bits: list[int]) -> bool:
        i = 0
        while i < len(bits) - 1:
            i += bits[i] + 1

        return i == len(bits) - 1
```

2893. 718-2.py

Path: LeetCode-main/solutions/718. Maximum Length of Repeated Subarray/718-2.py

```
class Solution:
    def findLength(self, nums1: list[int], nums2: list[int]) -> int:
        ans = 0
        dp = [0] * (len(nums2) + 1)

        for a in reversed(nums1):
            for j, b in enumerate(nums2): # The order is important.
                dp[j] = dp[j + 1] + 1 if a == b else 0
                ans = max(ans, dp[j])

        return ans
```

2894. 718.py

Path: LeetCode-main/solutions/718. Maximum Length of Repeated Subarray/718.py

```
class Solution:
    def findLength(self, nums1: list[int], nums2: list[int]) -> int:
        m = len(nums1)
        n = len(nums2)
        ans = 0
        # dp[i][j] := the maximum length of a subarray that appears in both
        # nums1[i..m) and nums2[j..n)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in reversed(range(m)):
            for j in reversed(range(n)):
                if nums1[i] == nums2[j]:
                    dp[i][j] = dp[i + 1][j + 1] + 1
                    ans = max(ans, dp[i][j])

        return ans
```

2895. 719.py

Path: LeetCode-main/solutions/719. Find K-th Smallest Pair Distance/719.py

```

class Solution:
    def smallestDistancePair(self, nums: list[int], k: int) -> int:
        nums.sort()

        def numPairDistancesNoGreaterThanOr(m: int) -> int:
            count = 0
            j = 1
            # For each index i, find the first index j s.t. nums[j] > nums[i] + m,
            # so numPairDistancesNoGreaterThanOr for the index i will be j - i - 1.
            for i, num in enumerate(nums):
                while j < len(nums) and nums[j] <= num + m:
                    j += 1
                count += j - i - 1
            return count

        return bisect.bisect_left(range(nums[-1] - nums[0]), k,
                                   key=numPairDistancesNoGreaterThanOr)

```

2896. 72.py

Path: LeetCode-main/solutions/72. Edit Distance/72.py

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m = len(word1)
        n = len(word2)
        # dp[i][j] := the minimum number of operations to convert word1[0..i) to
        # word2[0..j)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            dp[i][0] = i

        for j in range(1, n + 1):
            dp[0][j] = j

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1

        return dp[m][n]

```

2897. 720.py

Path: LeetCode-main/solutions/720. Longest Word in Dictionary/720.py

```
class Solution:
    def longestWord(self, words: list[str]) -> str:
        root = {}

        for word in words:
            node = root
            for c in word:
                if c not in node:
                    node[c] = {}
                node = node[c]
            node['word'] = word

        def dfs(node: dict) -> str:
            ans = node['word'] if 'word' in node else ''

            for child in node:
                if 'word' in node[child] and len(node[child]['word']) > 0:
                    childWord = dfs(node[child])
                    if len(childWord) > len(ans) or (
                        len(childWord) == len(ans) and childWord < ans):
                        ans = childWord

            return ans

        return dfs(root)
```

2898. 722.py

Path: LeetCode-main/solutions/722. Remove Comments/722.py

```

class Solution:
    def removeComments(self, source: list[str]) -> list[str]:
        ans = []
        commenting = False
        modified = ''

        for line in source:
            i = 0
            while i < len(line):
                if i + 1 == len(line):
                    if not commenting:
                        modified += line[i]
                    i += 1
                    break
                twoChars = line[i:i + 2]
                if twoChars == '/*' and not commenting:
                    commenting = True
                    i += 2
                elif twoChars == '*/' and commenting:
                    commenting = False
                    i += 2
                elif twoChars == '//':
                    if not commenting:
                        break
                    else:
                        i += 2
                else:
                    if not commenting:
                        modified += line[i]
                    i += 1
            if modified and not commenting:
                ans.append(modified)
                modified = ''

        return ans

```

2899. 724.py

Path: LeetCode-main/solutions/724. Find Pivot Index/724.py

```

class Solution:
    def pivotIndex(self, nums: list[int]) -> int:
        summ = sum(nums)
        prefix = 0

        for i, num in enumerate(nums):
            if prefix == summ - prefix - num:
                return i
            prefix += num

        return -1

```

2900. 725.py

Path: LeetCode-main/solutions/725. Split Linked List in Parts/725.py

```
class Solution:
    def splitListToParts(self, root: ListNode, k: int) -> list[ListNode]:
        ans = [[] for _ in range(k)]
        length = 0
        curr = root
        while curr:
            length += 1
            curr = curr.next
        subLength = length // k
        remainder = length % k

        prev = None
        head = root

        for i in range(k):
            ans[i] = head
            for j in range(subLength + (1 if remainder > 0 else 0)):
                prev = head
                head = head.next
            if prev:
                prev.next = None
            remainder -= 1

        return ans
```

2901. 726.py

Path: LeetCode-main/solutions/726. Number of Atoms/726.py


```

class Solution:
    def countOfAtoms(self, formula: str) -> str:
        def parse() -> dict:
            ans = collections.defaultdict(int)

            nonlocal i
            while i < n:
                if formula[i] == '(':
                    i += 1
                    for elem, freq in parse().items():
                        ans[elem] += freq
                elif formula[i] == ')':
                    i += 1
                    numStart = i
                    while i < n and formula[i].isdigit():
                        i += 1
                    factor = int(formula[numStart:i])
                    for elem, freq in ans.items():
                        ans[elem] *= factor
                    return ans
                elif formula[i].isupper():
                    elemStart = i
                    i += 1
                    while i < n and formula[i].islower():
                        i += 1
                    elem = formula[elemStart:i]
                    numStart = i
                    while i < n and formula[i].isdigit():
                        i += 1
                    num = 1 if i == numStart else int(
                        formula[numStart:i])
                    ans[elem] += num

            return ans

        n = len(formula)

        ans = ""
        i = 0
        count = parse()

        for elem in sorted(count.keys()):
            ans += elem
            if count[elem] > 1:
                ans += str(count[elem])

        return ans

```

2902. 728.py

Path: LeetCode-main/solutions/728. Self Dividing Numbers/728.py

```

class Solution:
    def selfDividingNumbers(self, left: int, right: int) -> list[int]:
        return [num for num in range(left, right + 1) if all(n != 0 and num % n == 0 for n in map(int, str(num)))]

```

2903. 729-3.py

Path: LeetCode-main/solutions/729. My Calendar I/729-3.py

```

from dataclasses import dataclass

@dataclass
class Node:
    start: int
    end: int
    left = None
    right = None

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, node: Node, root: Node = None) -> bool:
        if not root:
            if not self.root:
                self.root = node
                return True
            else:
                root = self.root

        if node.start >= root.end:
            if not root.right:
                root.right = node
                return True
            return self.insert(node, root.right)
        elif node.end <= root.start:
            if not root.left:
                root.left = node
                return True
            return self.insert(node, root.left)
        else:
            return False

class MyCalendar:
    def __init__(self):
        self.tree = Tree()

    def book(self, start: int, end: int) -> bool:
        return self.tree.insert(Node(start, end))

```

2904. 729.py

Path: LeetCode-main/solutions/729. My Calendar I/729.py

```

class MyCalendar:
    def __init__(self):
        self.line = []

    def book(self, startTime: int, endTime: int) -> bool:
        for s, e in self.line:
            if max(startTime, s) < min(endTime, e):
                return False
        self.line.append((startTime, endTime))
        return True

```

2905. 73.py

Path: LeetCode-main/solutions/73. Set Matrix Zeroes/73.py

```
class Solution:
    def setZeroes(self, matrix: list[list[int]]) -> None:
        m = len(matrix)
        n = len(matrix[0])
        shouldFillFirstRow = 0 in matrix[0]
        shouldFillFirstCol = 0 in list(zip(*matrix))[0]

        # Store the information in the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        # Fill 0s for the matrix except the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

        # Fill 0s for the first row if needed.
        if shouldFillFirstRow:
            matrix[0] = [0] * n

        # Fill 0s for the first column if needed.
        if shouldFillFirstCol:
            for row in matrix:
                row[0] = 0
```

2906. 730.py

Path: LeetCode-main/solutions/730. Count Different Palindromic Subsequences/730.py

```

class Solution:
    def countPalindromicSubsequences(self, s: str) -> int:
        MOD = 1_000_000_007
        n = len(s)
        # dp[i][j] := the number of different non-empty palindromic subsequences in
        # s[i..j]
        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                if s[i] == s[j]:
                    lo = i + 1
                    hi = j - 1
                    while lo <= hi and s[lo] != s[hi]:
                        lo += 1
                    while lo <= hi and s[hi] != s[i]:
                        hi -= 1
                    if lo > hi:
                        dp[i][j] = dp[i + 1][j - 1] * 2 + 2
                    elif lo == hi:
                        dp[i][j] = dp[i + 1][j - 1] * 2 + 1
                    else:
                        dp[i][j] = dp[i + 1][j - 1] * 2 - dp[lo + 1][hi - 1]
                else:
                    dp[i][j] = dp[i][j - 1] + dp[i + 1][j] - dp[i + 1][j - 1]
                dp[i][j] = (dp[i][j] + MOD) % MOD

        return dp[0][n - 1]

```

2907. 732.py

Path: LeetCode-main/solutions/732. My Calendar III/732.py

```

from sortedcontainers import SortedDict

class MyCalendarThree:
    def __init__(self):
        self.line = SortedDict()

    def book(self, start: int, end: int) -> int:
        self.line[start] = self.line.get(start, 0) + 1
        self.line[end] = self.line.get(end, 0) - 1

        ans = 0
        activeEvents = 0

        for count in self.line.values():
            activeEvents += count
            ans = max(ans, activeEvents)

        return ans

```

2908. 733.py

Path: LeetCode-main/solutions/733. Flood Fill/733.py

```
class Solution:
    def floodFill(self, image: list[list[int]],
                  sr: int, sc: int, newColor: int) -> list[list[int]]:
        startColor = image[sr][sc]
        seen = set()

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == len(image) or j < 0 or j == len(image[0]):
                return
            if image[i][j] != startColor or (i, j) in seen:
                return

            image[i][j] = newColor
            seen.add((i, j))

            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        dfs(sr, sc)
        return image
```

2909. 734.py

Path: LeetCode-main/solutions/734. Sentence Similarity/734.py

```
class Solution:
    def areSentencesSimilar(
        self,
        sentence1: list[str],
        sentence2: list[str],
        similarPairs: list[list[str]],
    ) -> bool:
        if len(sentence1) != len(sentence2):
            return False

        # map[key] := all the similar words of key
        map = collections.defaultdict(set)

        for a, b in similarPairs:
            map[a].add(b)
            map[b].add(a)

        for word1, word2 in zip(sentence1, sentence2):
            if word1 == word2:
                continue
            if word1 not in map:
                return False
            if word2 not in map[word1]:
                return False

        return True
```

2910. 735.py

Path: LeetCode-main/solutions/735. Asteroid Collision/735.py

```
class Solution:
    def asteroidCollision(self, asteroids: list[int]) -> list[int]:
        stack = []

        for a in asteroids:
            if a > 0:
                stack.append(a)
            else: # a < 0
                # Destroy the previous positive one(s).
                while stack and stack[-1] > 0 and stack[-1] < -a:
                    stack.pop()
                if not stack or stack[-1] < 0:
                    stack.append(a)
                elif stack[-1] == -a:
                    stack.pop() # Both asteroids explode.
                else: # stack[-1] > the current asteroid.
                    pass # Destroy the current asteroid, so do nothing.

        return stack
```

2911. 736.py

Path: `LeetCode-main/solutions/736. Parse Lisp Expression/736.py`

```

class Solution:
    def evaluate(self, expression: str) -> int:
        def evaluate(e: str, prevScope: dict) -> int:
            if e[0].isdigit() or e[0] == '-':
                return int(e)
            if e in prevScope:
                return prevScope[e]

            scope = prevScope.copy()
            nextExpression = e[e.index(' ') + 1:-1]
            tokens = parse(nextExpression)

            if e[1] == 'm': # 'mult'
                return evaluate(tokens[0], scope) * evaluate(tokens[1], scope)
            if e[1] == 'a': # 'add'
                return evaluate(tokens[0], scope) + evaluate(tokens[1], scope)

            # 'let'
            for i in range(0, len(tokens) - 2, 2):
                scope[tokens[i]] = evaluate(tokens[i + 1], scope)

            return evaluate(tokens[-1], scope)

        def parse(e: str):
            tokens = []
            s = ''
            opened = 0

            for c in e:
                if c == '(':
                    opened += 1
                elif c == ')':
                    opened -= 1
                if opened == 0 and c == ' ':
                    tokens.append(s)
                    s = ''
                else:
                    s += c

            if len(s) > 0:
                tokens.append(s)
            return tokens

        return evaluate(expression, {})

```

2912. 737.py

Path: LeetCode-main/solutions/737. Sentence Similarity II/737.py

```

class Solution:
    def areSentencesSimilarTwo(
        self,
        words1: list[str],
        words2: list[str],
        pairs: list[list[str]],
    ) -> bool:
        if len(words1) != len(words2):
            return False

        # graph[key] := all the similar words of key
        graph = collections.defaultdict(set)

        for a, b in pairs:
            graph[a].add(b)
            graph[b].add(a)

        def dfs(word1: str, word2: str, seen: set) -> bool:
            if word1 in graph[word2]:
                return True

            seen.add(word1)

            for child in graph[word1]:
                if child in seen:
                    continue
                if dfs(child, word2, seen):
                    return True

            return False

        for word1, word2 in zip(words1, words2):
            if word1 == word2:
                continue
            if word1 not in graph:
                return False
            if not dfs(word1, word2, set()):
                return False

        return True

```

2913. 739.py

Path: LeetCode-main/solutions/739. Daily Temperatures/739.py

```

class Solution:
    def dailyTemperatures(self, temperatures: list[int]) -> list[int]:
        ans = [0] * len(temperatures)
        stack = [] # a decreasing stack

        for i, temperature in enumerate(temperatures):
            while stack and temperature > temperatures[stack[-1]]:
                index = stack.pop()
                ans[index] = i - index
            stack.append(i)

        return ans

```

2914. 74.py

Path: LeetCode-main/solutions/74. Search a 2D Matrix/74.py

```
class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        if not matrix:
            return False

        m = len(matrix)
        n = len(matrix[0])
        l = 0
        r = m * n

        while l < r:
            mid = (l + r) // 2
            i = mid // n
            j = mid % n
            if matrix[i][j] == target:
                return True
            if matrix[i][j] < target:
                l = mid + 1
            else:
                r = mid

        return False
```

2915. 743.py

Path: LeetCode-main/solutions/743. Network Delay Time/743.py

```
class Solution:
    def networkDelayTime(self, times: list[list[int]], n: int, k: int) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in times:
            graph[u - 1].append((v - 1, w))

        return self._dijkstra(graph, k - 1)

    def _dijkstra(self, graph: list[list[tuple[int, int]]], src: int) -> int:
        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)]  # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(minHeap, (dist[v], v))

        maxDist = max(dist)
        return maxDist if maxDist != math.inf else -1
```

2916. 744.py

Path: LeetCode-main/solutions/744. Find Smallest Letter Greater Than Target/744.py

```
class Solution:
    def nextGreatestLetter(self, letters: list[str], target: str) -> str:
        l = bisect.bisect_right(range(len(letters)), target,
                                key=lambda m: letters[m])
        return letters[l % len(letters)]
```

2917. 746.py

Path: LeetCode-main/solutions/746. Min Cost Climbing Stairs/746.py

```
class Solution:
    def minCostClimbingStairs(self, cost: list[int]) -> int:
        cost.append(0)

        for i in range(2, len(cost)):
            cost[i] += min(cost[i - 1], cost[i - 2])

        return cost[-1]
```

2918. 747.py

Path: LeetCode-main/solutions/747. Largest Number At Least Twice of Others/747.py

```
class Solution:
    def dominantIndex(self, nums: list[int]) -> int:
        mx = 0
        secondMax = 0

        for i, num in enumerate(nums):
            if num > mx:
                secondMax = mx
                mx = num
                ans = i
            elif num > secondMax:
                secondMax = num

        return ans if mx >= 2 * secondMax else -1
```

2919. 748.py

Path: LeetCode-main/solutions/748. Shortest Completing Word/748.py

```

class Solution:
    def shortestCompletingWord(self, licensePlate: str, words: list[str]) -> str:
        def isMatch(word: str) -> bool:
            wordCount = collections.Counter(word)
            return False if any(
                wordCount[i] < count[i] for i in string.ascii_letters) else True

        ans = '*' * 16
        count = collections.defaultdict(int)

        for c in licensePlate:
            if c.isalpha():
                count[c.lower()] += 1

        for word in words:
            if len(word) < len(ans) and isMatch(word):
                ans = word

        return ans

```

2920. 75-2.py

Path: LeetCode-main/solutions/75. Sort Colors/75-2.py

```

class Solution:
    def sortColors(self, nums: list[int]) -> None:
        l = 0 # The next 0 should be placed in l.
        r = len(nums) - 1 # The next 2 should be placed in r.

        i = 0
        while i <= r:
            if nums[i] == 0:
                nums[i], nums[l] = nums[l], nums[i]
                i += 1
                l += 1
            elif nums[i] == 1:
                i += 1
            else:
                # We may swap a 0 to index i, but we're still not sure whether this 0
                # is placed in the correct index, so we can't move pointer i.
                nums[i], nums[r] = nums[r], nums[i]
                r -= 1

```

2921. 75.py

Path: LeetCode-main/solutions/75. Sort Colors/75.py

```
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        zero = -1
        one = -1
        two = -1

        for num in nums:
            if num == 0:
                two += 1
                one += 1
                zero += 1
                nums[two] = 2
                nums[one] = 1
                nums[zero] = 0
            elif num == 1:
                two += 1
                one += 1
                nums[two] = 2
                nums[one] = 1
            else:
                two += 1
                nums[two] = 2
```

2922. 751.py

Path: LeetCode-main/solutions/751. IP to CIDR/751.py

```

class Solution:
    def ipToCIDR(self, ip: str, n: int) -> list[str]:
        ans = []
        num = self._getNum(ip.split('.'))

        while n > 0:
            lowbit = num & -num
            count = self._maxLow(n) if lowbit == 0 else self._firstFit(lowbit, n)
            ans.append(self._getCIDR(num, self._getPrefix(count)))
            n -= count
            num += count

        return ans

    def _getNum(self, x: list[str]) -> int:
        num = 0
        for i in range(4):
            num = num * 256 + int(x[i])
        return num

    def _maxLow(self, n: int) -> int | None:
        """Returns the maximum i s.t. 2^i < n."""
        for i in range(32):
            if 1 << i + 1 > n:
                return 1 << i

    def _firstFit(self, lowbit: int, n: int) -> int:
        while lowbit > n:
            lowbit >>= 1
        return lowbit

    def _getCIDR(self, num: int, prefix: int) -> str:
        d = num & 255
        num >>= 8
        c = num & 255
        num >>= 8
        b = num & 255
        num >>= 8
        a = num & 255
        return '.'.join([str(s) for s in [a, b, c, d]]) + '/' + str(prefix)

    def _getPrefix(self, count: int) -> int | None:
        """
        e.g. count = 8 = 2^3 . prefix = 32 - 3 = 29
            count = 1 = 2^0 . prefix = 32 - 0 = 32
        """
        for i in range(32):
            if count == 1 << i:
                return 32 - i

```

2923. 753.py

Path: LeetCode-main/solutions/753. Cracking the Safe/753.py

```
class Solution:
    def crackSafe(self, n: int, k: int) -> str:
        passwordSize = k**n
        path = '0' * n
        seen = set()
        seen.add(path)

        def dfs(path: str) -> str:
            if len(seen) == passwordSize:
                return path

            for c in map(str, range(k)):
                node = path[-n + 1:] + c if n > 1 else c
                if node not in seen:
                    seen.add(node)
                    res = dfs(path + c)
                    if res:
                        return res
                    seen.remove(node)

        return dfs(path)
```

2924. 754.py

Path: LeetCode-main/solutions/754. Reach a Number/754.py

```
class Solution:
    def reachNumber(self, target: int) -> int:
        ans = 0
        pos = 0
        target = abs(target)

        while pos < target:
            ans += 1
            pos += ans

        while (pos - target) % 2 == 1:
            ans += 1
            pos += ans

        return ans
```

2925. 755.py

Path: LeetCode-main/solutions/755. Pour Water/755.py

```

class Solution:
    def pourWater(self, heights: list[int], volume: int, k: int) -> list[int]:
        i = k

        while volume > 0:
            volume -= 1
            while i > 0 and heights[i] >= heights[i - 1]:
                i -= 1
            while i + 1 < len(heights) and heights[i] >= heights[i + 1]:
                i += 1
            while i > k and heights[i] == heights[i - 1]:
                i -= 1
            heights[i] += 1

        return heights

```

2926. 756.py

Path: LeetCode-main/solutions/756. Pyramid Transition Matrix/756.py

```

class Solution:
    def pyramidTransition(self, bottom: str, allowed: list[str]) -> bool:
        prefixToBlocks = collections.defaultdict(list)

        for a in allowed:
            prefixToBlocks[a[:2]].append(a[2])

        def dfs(row: str, nextRow: str, i: int) -> bool:
            if len(row) == 1:
                return True
            if len(nextRow) + 1 == len(row):
                return dfs(nextRow, '', 0)

            for c in prefixToBlocks[row[i:i + 2]]:
                if dfs(row, nextRow + c, i + 1):
                    return True

            return False

        return dfs(bottom, '', 0)

```

2927. 758-2.py

Path: LeetCode-main/solutions/758. Bold Words in String/758-2.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Solution:
    def boldWords(self, words: list[str], s: str) -> str:
        n = len(s)
        ans = []
        # bold[i] := True if s[i] should be bolded
        bold = [0] * n
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                if c not in node.children:
                    node.children[c] = TrieNode()
                node = node.children[c]
            node.isWord = True

        def find(s: str, i: int) -> int:
            node = root
            ans = -1
            for j in range(i, len(s)):
                node = node.children.setdefault(s[j], TrieNode())
                if node.isWord:
                    ans = j
            return ans

        for word in words:
            insert(word)

        boldEnd = -1 # `s[i..boldEnd]` should be bolded.
        for i in range(n):
            boldEnd = max(boldEnd, find(s, i))
            bold[i] = boldEnd >= i

        # Construct the with bold tags
        i = 0
        while i < n:
            if bold[i]:
                j = i
                while j < n and bold[j]:
                    j += 1
                # `s[i..j]` should be bolded.
                ans.append('<b>' + s[i:j] + '</b>')
                i = j
            else:
                ans.append(s[i])
                i += 1

        return ''.join(ans)

```

2928. 758.py

Path: LeetCode-main/solutions/758. Bold Words in String/758.py


```

class Solution:
    def boldWords(self, words: list[str], s: str) -> str:
        n = len(s)
        ans = []
        # bold[i] := True if s[i] should be bolded
        bold = [0] * n

        boldEnd = -1 # s[i:boldEnd] should be bolded
        for i in range(n):
            for word in words:
                if s[i:].startswith(word):
                    boldEnd = max(boldEnd, i + len(word))
            bold[i] = boldEnd > i

        # Construct the string with the bold tags.
        i = 0
        while i < n:
            if bold[i]:
                j = i
                while j < n and bold[j]:
                    j += 1
                # s[i..j) should be bolded.
                ans.append('<b>' + s[i:j] + '</b>')
                i = j
            else:
                ans.append(s[i])
                i += 1

        return ''.join(ans)

```

2929. 759.py

Path: LeetCode-main/solutions/759. Employee Free Time/759.py

```

class Solution:
    def employeeFreeTime(self, schedule: '[[Interval]]') -> '[Interval]':
        ans = []
        intervals = []

        for s in schedule:
            intervals.extend(s)

        intervals.sort(key=lambda x: x.start)

        prevEnd = intervals[0].end

        for interval in intervals:
            if interval.start > prevEnd:
                ans.append(Interval(prevEnd, interval.start))
            prevEnd = max(prevEnd, interval.end)

        return ans

```

2930. 76.py

Path: LeetCode-main/solutions/76. Minimum Window Substring/76.py

```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        count = collections.Counter(t)
        required = len(t)
        bestLeft = -1
        minLength = len(s) + 1

        l = 0
        for r, c in enumerate(s):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            while required == 0:
                if r - l + 1 < minLength:
                    bestLeft = l
                    minLength = r - l + 1
                count[s[l]] += 1
                if count[s[l]] > 0:
                    required += 1
                l += 1

        return '' if bestLeft == -1 else s[bestLeft: bestLeft + minLength]

```

2931. 760.py

Path: LeetCode-main/solutions/760. Find Anagram Mappings/760.py

```

class Solution:
    def anagramMappings(self, nums1: list[int], nums2: list[int]) -> list[int]:
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums2):
            numToIndices[num].append(i)

        return [numToIndices[num].pop() for num in nums1]

```

2932. 761.py

Path: LeetCode-main/solutions/761. Special Binary String/761.py

```

class Solution:
    def makeLargestSpecial(self, s: str) -> str:
        specials = []
        count = 0

        i = 0
        for j, c in enumerate(s):
            count += 1 if c == '1' else -1
            if count == 0:
                specials.append(
                    '1' + self.makeLargestSpecial(s[i + 1:j]) + '0'
                )
                i = j + 1

        return ''.join(sorted(specials)[::-1])

```

2933. 763.py

Path: LeetCode-main/solutions/763. Partition Labels/763.py

```
class Solution:
    def partitionLabels(self, s: str) -> list[int]:
        ans = []
        letterToRightmostIndex = {c: i for i, c in enumerate(s)}

        l = 0 # the leftmost index of the current running string
        r = 0 # the rightmost index of the current running string

        for i, c in enumerate(s):
            r = max(r, letterToRightmostIndex[c])
            if i == r:
                ans.append(r - l + 1)
                l = r + 1

        return ans
```

2934. 766.py

Path: LeetCode-main/solutions/766. Toeplitz Matrix/766.py

```
class Solution:
    def isToeplitzMatrix(self, matrix: list[list[int]]) -> bool:
        for i in range(len(matrix) - 1):
            for j in range(len(matrix[0]) - 1):
                if matrix[i][j] != matrix[i + 1][j + 1]:
                    return False

        return True
```

2935. 767-2.py

Path: LeetCode-main/solutions/767. Reorganize String/767-2.py

```

class Solution:
    def reorganizeString(self, s: str) -> str:
        n = len(s)
        count = collections.Counter(s)
        maxCount = max(count.values())

        if maxCount > (n + 1) // 2:
            return ''

        if maxCount == (n + 1) // 2:
            maxLetter = max(count, key=count.get)
            ans = [maxLetter if i % 2 == 0 else '' for i in range(n)]
            del count[maxLetter]
            i = 1
        else:
            ans = [''] * n
            i = 0

        for c, freq in count.items():
            for _ in range(freq):
                ans[i] = c
                i += 2
                if i >= n:
                    i = 1

        return ''.join(ans)

```

2936. 767.py

Path: LeetCode-main/solutions/767. Reorganize String/767.py

```

class Solution:
    def reorganizeString(self, s: str) -> str:
        count = collections.Counter(s)
        if max(count.values()) > (len(s) + 1) // 2:
            return ''

        ans = []
        maxHeap = [(-freq, c) for c, freq in count.items()]
        heapq.heapify(maxHeap)
        prevFreq = 0
        prevChar = '@'

        while maxHeap:
            # Get the letter with the maximum frequency.
            freq, c = heapq.heappop(maxHeap)
            ans.append(c)
            # Add the previous letter back s.t. any two adjacent characters are not
            # the same.
            if prevFreq < 0:
                heapq.heappush(maxHeap, (prevFreq, prevChar))
            prevFreq = freq + 1
            prevChar = c

        return ''.join(ans)

```

2937. 768.py

Path: LeetCode-main/solutions/768. Max Chunks To Make Sorted II/768.py

```
class Solution:
    def maxChunksToSorted(self, arr: list[int]) -> int:
        n = len(arr)
        ans = 0
        mx = -math.inf
        mn = [arr[-1]] * n

        for i in reversed(range(n - 1)):
            mn[i] = min(mn[i + 1], arr[i])

        for i in range(n - 1):
            mx = max(mx, arr[i])
            if mx <= mn[i + 1]:
                ans += 1

        return ans + 1
```

2938. 769.py

Path: LeetCode-main/solutions/769. Max Chunks To Make Sorted/769.py

```
class Solution:
    def maxChunksToSorted(self, arr: list[int]) -> int:
        ans = 0
        mx = -math.inf

        for i, a in enumerate(arr):
            mx = max(mx, a)
            if mx == i:
                ans += 1

        return ans
```

2939. 77.py

Path: LeetCode-main/solutions/77. Combinations/77.py

```
class Solution:
    def combine(self, n: int, k: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            if len(path) == k:
                ans.append(path.copy())
                return

            for i in range(s, n + 1):
                path.append(i)
                dfs(i + 1, path)
                path.pop()

        dfs(1, [])
        return ans
```

2940. 770.py

Path: LeetCode-main/solutions/770. Basic Calculator IV/770.py

```

class Poly:
    def __init__(self, term: str = None, coef: int = None):
        if term and coef:
            self.terms = collections.Counter({term: coef})
        else:
            self.terms = collections.Counter()

    def __add__(self, other):
        for term, coef in other.terms.items():
            self.terms[term] += coef
        return self

    def __sub__(self, other):
        for term, coef in other.terms.items():
            self.terms[term] -= coef
        return self

    def __mul__(self, other):
        res = Poly()
        for a, aCoef in self.terms.items():
            for b, bCoef in other.terms.items():
                res.terms[self._merge(a, b)] += aCoef * bCoef
        return res

    # Def __str__(self):
    #     res = []
    #     for term, coef in self.terms.items():
    #         res.append(term + ': ' + str(coef))
    #     return '{' + ', '.join(res) + '}'

    def toList(self) -> list[str]:
        for term in list(self.terms.keys()):
            if not self.terms[term]:
                del self.terms[term]

    def cmp(term: str) -> tuple:
        # the minimum degree is the last
        if term == '1':
            return (0,)
        var = term.split('*')
        # the maximum degree is the first
        # Break ties by their lexicographic orders.
        return (-len(var), term)

    def concat(term: str) -> str:
        if term == '1':
            return str(self.terms[term])
        return str(self.terms[term]) + '*' + term

    terms = list(self.terms.keys())
    terms.sort(key=cmp)
    return [concat(term) for term in terms]

    def _merge(self, a: str, b: str) -> str:
        if a == '1':
            return b
        if b == '1':
            return a
        res = []
        A = a.split('*')
        B = b.split('*')
        i = 0 # A's index
        j = 0 # B's index
        while i < len(A) and j < len(B):
            if A[i] < B[j]:

```

```

        res.append(A[i])
        i += 1
    else:
        res.append(B[j])
        j += 1
    return ''.join(res + A[i:] + B[j:])

```

```
class Solution:
```

```

    def basicCalculatorIV(
        self,
        expression: str,
        evalvars: list[str],
        evalints: list[int],
    ) -> list[str]:
        tokens = list(self._getTokens(expression))
        evalMap = {a: b for a, b in zip(evalvars, evalints)}

```

```

        for i, token in enumerate(tokens):
            if token in evalMap:
                tokens[i] = str(evalMap[token])

```

```

        postfix = self._infixToPostfix(tokens)
        return self._evaluate(postfix).toList()

```

```
def _getTokens(self, s: str) -> Iterator[str]:
```

```

    i = 0
    for j, c in enumerate(s):
        if c == ' ':
            if i < j:
                yield s[i:j]
            i = j + 1
        elif c in '()+-*':
            if i < j:
                yield s[i:j]
            yield c
            i = j + 1
    if i < len(s):
        yield s[i:]

```

```
def _infixToPostfix(self, tokens: list[str]) -> list[str]:
```

```

    postfix = []
    ops = []

```

```
def precedes(prevOp: str, currOp: str) -> bool:
```

```

    if prevOp == '(':
        return False
    return prevOp == '*' or currOp in '+- '

```

```
for token in tokens:
```

```

    if token == '(':
        ops.append(token)
    elif token == ')':
        while ops[-1] != '(':
            postfix.append(ops.pop())
        ops.pop()
    elif token in '+-*': # isOperator(token)
        while ops and precedes(ops[-1], token):
            postfix.append(ops.pop())
        ops.append(token)
    else: # isOperand(token)
        postfix.append(token)

```

```
return postfix + ops[::-1]
```

```
def _evaluate(self, postfix: list[str]) -> Poly:
```

```

    """Evaluate postfix expression"""

```

```
polys: list[Poly] = []
for token in postfix:
    if token in '+-*':
        b = polys.pop()
        a = polys.pop()
        if token == '+':
            polys.append(a + b)
        elif token == '-':
            polys.append(a - b)
        else: # token == '*'
            polys.append(a * b)
    elif token.lstrip('-').isnumeric():
        polys.append(Poly("1", int(token)))
    else:
        polys.append(Poly(token, 1))
return polys[0]
```

2941. 771.py

Path: LeetCode-main/solutions/771. Jewels and Stones/771.py

```
class Solution:
    def numJewelsInStones(self, jewels: str, stones: str) -> int:
        jewelsSet = set(jewels)
        return sum(stone in jewelsSet for stone in stones)
```

2942. 772.py

Path: LeetCode-main/solutions/772. Basic Calculator III/772.py


```

class Solution:
    def calculate(self, s: str) -> int:
        nums = []
        ops = []

        def calc():
            b = nums.pop()
            a = nums.pop()
            op = ops.pop()
            if op == '+':
                nums.append(a + b)
            elif op == '-':
                nums.append(a - b)
            elif op == '*':
                nums.append(a * b)
            else: # op == '/'
                nums.append(int(a / b))

        def precedes(prev: str, curr: str) -> bool:
            """
            Returns True if the previous character is a operator and the priority of
            the previous operator >= the priority of the current character (operator).
            """
            if prev == '(':
                return False
            return prev in '*/' or curr in '+-'

        i = 0
        hasPrevNum = False

        while i < len(s):
            c = s[i]
            if c.isdigit():
                num = int(c)
                while i + 1 < len(s) and s[i + 1].isdigit():
                    num = num * 10 + int(s[i + 1])
                    i += 1
                nums.append(num)
                hasPrevNum = True
            elif c == '(':
                ops.append('(')
                hasPrevNum = False
            elif c == ')':
                while ops[-1] != '(':
                    calc()
                ops.pop() # Pop '('
            elif c in '+-*/*':
                if not hasPrevNum: # Handle input like "-1-(-1)"
                    num.append(0)
                while ops and precedes(ops[-1], c):
                    calc()
                ops.append(c)
            i += 1

        while ops:
            calc()

        return nums.pop()

```

2943. 774.py

Path: LeetCode-main/solutions/774. Minimize Max Distance to Gas Station/774.py

```

class Solution:
    def minmaxGasDist(self, stations: list[int], k: int) -> float:
        ERR = 1e-6
        l = 0
        r = stations[-1] - stations[0]

        def possible(k: int, m: float) -> bool:
            """
            Returns True if can use <= k gas stations to ensure that each adjacent
            distance between gas stations <= m.
            """
            for a, b in zip(stations, stations[1:]):
                diff = b - a
                if diff > m:
                    k -= math.ceil(diff / m) - 1
                    if k < 0:
                        return False
            return True

        while r - l > ERR:
            m = (l + r) / 2
            if possible(k, m):
                r = m
            else:
                l = m

        return l

```

2944. 775-2.py

Path: LeetCode-main/solutions/775. Global and Local Inversions/775-2.py

```

class Solution:
    def isIdealPermutation(self, nums: list[int]) -> bool:
        for i, num in enumerate(nums):
            if abs(num - i) > 1:
                return False
        return True

```

2945. 775.py

Path: LeetCode-main/solutions/775. Global and Local Inversions/775.py

```

class Solution:
    def isIdealPermutation(self, nums: list[int]) -> bool:
        mx = -1 # the number that is most likely > nums[i + 2]

        for i in range(len(nums) - 2):
            mx = max(mx, nums[i])
            if mx > nums[i + 2]:
                return False

        return True

```

2946. 776.py

Path: LeetCode-main/solutions/776. Split BST/776.py

```
class Solution:
    def splitBST(self, root: TreeNode | None, target: int) -> list[TreeNode | None]:
        if not root:
            return None, None
        if root.val > target:
            left, right = self.splitBST(root.left, target)
            root.left = right
            return left, root
        else: # root.val <= target
            left, right = self.splitBST(root.right, target)
            root.right = left
            return root, right
```

2947. 777.py

Path: LeetCode-main/solutions/777. Swap Adjacent in LR String/777.py

```
class Solution:
    def canTransform(self, start: str, end: str) -> bool:
        if start.replace('X', '') != end.replace('X', ''):
            return False

        i = 0 # start's index
        j = 0 # end's index

        while i < len(start) and j < len(end):
            while i < len(start) and start[i] == 'X':
                i += 1
            while j < len(end) and end[j] == 'X':
                j += 1
            if i == len(start) and j == len(end):
                return True
            if i == len(start) or j == len(end):
                return False
            # L can only move to left.
            if start[i] == 'L' and i < j:
                return False
            # R can only move to right.
            if start[i] == 'R' and i > j:
                return False
            i += 1
            j += 1

        return True
```

2948. 78.py

Path: LeetCode-main/solutions/78. Subsets/78.py

```
class Solution:
    def subsets(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            ans.append(path)

            for i in range(s, len(nums)):
                dfs(i + 1, path + [nums[i]])

        dfs(0, [])
        return ans
```

2949. 780.py

Path: LeetCode-main/solutions/780. Reaching Points/780.py

```
class Solution:
    def reachingPoints(self, sx: int, sy: int, tx: int, ty: int) -> bool:
        while sx < tx and sy < ty:
            tx, ty = tx % ty, ty % tx

        return (sx == tx and sy <= ty and (ty - sy) % tx == 0 or
                sy == ty and sx <= tx and (tx - sx) % ty == 0)
```

2950. 781.py

Path: LeetCode-main/solutions/781. Rabbits in Forest/781.py

```
class Solution:
    def numRabbits(self, answers: list[int]) -> int:
        ans = 0
        count = collections.Counter()

        for answer in answers:
            if count[answer] % (answer + 1) == 0:
                ans += answer + 1
            count[answer] += 1

        return ans
```

2951. 782.py

Path: LeetCode-main/solutions/782. Transform to Chessboard/782.py

```

class Solution:
    def movesToChessboard(self, board: list[list[int]]) -> int:
        n = len(board)

        if any(board[0][0] ^ board[i][0] ^ board[0][j] ^ board[i][j]
               for i in range(n) for j in range(n)):
            return -1

        rowSum = sum(board[0])
        colSum = sum(board[i][0] for i in range(n))

        if rowSum != n // 2 and rowSum != (n + 1) // 2:
            return -1
        if colSum != n // 2 and colSum != (n + 1) // 2:
            return -1

        rowSwaps = sum(board[i][0] == (i & 1) for i in range(n))
        colSwaps = sum(board[0][i] == (i & 1) for i in range(n))

        if n % 2 == 1:
            if rowSwaps % 2 == 1:
                rowSwaps = n - rowSwaps
            if colSwaps % 2 == 1:
                colSwaps = n - colSwaps
        else:
            rowSwaps = min(rowSwaps, n - rowSwaps)
            colSwaps = min(colSwaps, n - colSwaps)

        return (rowSwaps + colSwaps) // 2

```

2952. 785-2.py

Path: LeetCode-main/solutions/785. Is Graph Bipartite?/785-2.py

```

from enum import Enum

class Color(Enum):
    WHITE = 0
    RED = 1
    GREEN = 2

class Solution:
    def isBipartite(self, graph: list[list[int]]) -> bool:
        colors = [Color.WHITE] * len(graph)

        def isValidColor(u: int, color: Color) -> bool:
            # The painted color should be same as `color`.
            if colors[u] != Color.WHITE:
                return colors[u] == color

            colors[u] = color

            # All the children should have valid colors.
            childrenColor = Color.RED if colors[u] == Color.GREEN else Color.GREEN
            return all(isValidColor(v, childrenColor) for v in graph[u])

        return all(colors[i] != Color.WHITE or isValidColor(i, Color.RED)
                  for i in range(len(graph)))

```

2953. 785.py

Path: LeetCode-main/solutions/785. Is Graph Bipartite?/785.py

```
from enum import Enum

class Color(Enum):
    WHITE = 0
    RED = 1
    GREEN = 2

class Solution:
    def isBipartite(self, graph: list[list[int]]) -> bool:
        colors = [Color.WHITE] * len(graph)

        for i in range(len(graph)):
            # This node has been colored, so do nothing.
            if colors[i] != Color.WHITE:
                continue
            # Always paint red for a white node.
            colors[i] = Color.RED
            # BFS.
            q = collections.deque([i])
            while q:
                for _ in range(len(q)):
                    u = q.popleft()
                    for v in graph[u]:
                        if colors[v] == colors[u]:
                            return False
                        if colors[v] == Color.WHITE:
                            colors[v] = Color.RED if colors[u] == Color.GREEN else Color.GREEN
                            q.append(v)

        return True
```

2954. 786.py

Path: LeetCode-main/solutions/786. K-th Smallest Prime Fraction/786.py

```
class Solution:
    def kthSmallestPrimeFraction(self, arr: list[int], k: int) -> list[int]:
        n = len(arr)
        ans = [0, 1]
        l = 0
        r = 1

        while True:
            m = (l + r) / 2
            ans[0] = 0
            count = 0
            j = 1

            for i in range(n):
                while j < n and arr[i] > m * arr[j]:
                    j += 1
                count += n - j
                if j == n:
                    break
                if ans[0] * arr[j] < ans[1] * arr[i]:
                    ans[0] = arr[i]
                    ans[1] = arr[j]

            if count < k:
                l = m
            elif count > k:
                r = m
            else:
                return ans
```

2955. 787.py

Path: LeetCode-main/solutions/787. Cheapest Flights Within K Stops/787.py

```

class Solution:
    def findCheapestPrice(
        self,
        n: int,
        flights: list[list[int]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in flights:
            graph[u].append((v, w))

        return self._dijkstra(graph, src, dst, k)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        dist = [[math.inf] * (k + 2) for _ in range(len(graph))]

        dist[src][k + 1] = 0
        minHeap = [(dist[src][k + 1], src, k + 1)] # (d, u, stops)

        while minHeap:
            d, u, stops = heapq.heappop(minHeap)
            if u == dst:
                return d
            if stops == 0 or d > dist[u][stops]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v][stops - 1]:
                    dist[v][stops - 1] = d + w
                    heapq.heappush(minHeap, (dist[v][stops - 1], v, stops - 1))

        return -1

```

2956. 788.py

Path: LeetCode-main/solutions/788. Rotated Digits/788.py

```

class Solution:
    def rotatedDigits(self, n: int) -> int:
        def isGoodNumber(i: int) -> bool:
            isRotated = False

            for c in str(i):
                if c == '0' or c == '1' or c == '8':
                    continue
                if c == '2' or c == '5' or c == '6' or c == '9':
                    isRotated = True
                else:
                    return False

            return isRotated

        return sum(isGoodNumber(i) for i in range(1, n + 1))

```

2957. 789.py

Path: LeetCode-main/solutions/789. Escape The Ghosts/789.py

```
class Solution:
    def escapeGhosts(self, ghosts: list[list[int]], target: list[int]) -> bool:
        ghostSteps = min(abs(x - target[0]) +
                        abs(y - target[1]) for x, y in ghosts)

        return abs(target[0]) + abs(target[1]) < ghostSteps
```

2958. 79.py

Path: LeetCode-main/solutions/79. Word Search/79.py

```
class Solution:
    def exist(self, board: list[list[str]], word: str) -> bool:
        m = len(board)
        n = len(board[0])

        def dfs(i: int, j: int, s: int) -> bool:
            if i < 0 or i == m or j < 0 or j == n:
                return False
            if board[i][j] != word[s] or board[i][j] == '*':
                return False
            if s == len(word) - 1:
                return True

            cache = board[i][j]
            board[i][j] = '*'
            isExist = (dfs(i + 1, j, s + 1) or
                      dfs(i - 1, j, s + 1) or
                      dfs(i, j + 1, s + 1) or
                      dfs(i, j - 1, s + 1))
            board[i][j] = cache

            return isExist

        return any(dfs(i, j, 0)
                   for i in range(m)
                   for j in range(n))
```

2959. 790.py

Path: LeetCode-main/solutions/790. Domino and Tromino Tiling/790.py

```
class Solution:
    def numTilings(self, n: int) -> int:
        MOD = 1_000_000_007
        dp = [0, 1, 2, 5] + [0] * 997

        for i in range(4, n + 1):
            dp[i] = 2 * dp[i - 1] + dp[i - 3]

        return dp[n] % MOD
```

2960. 791.py

Path: LeetCode-main/solutions/791. Custom Sort String/791.py

```
class Solution:
    def customSortString(self, order: str, s: str) -> str:
        ans = ""
        count = [0] * 26

        for c in s:
            count[ord(c) - ord('a')] += 1

        for c in order:
            while count[ord(c) - ord('a')] > 0:
                ans += c
                count[ord(c) - ord('a')] -= 1

        for c in string.ascii_lowercase:
            for _ in range(count[ord(c) - ord('a')]):
                ans += c

        return ans
```

2961. 792-2.py

Path: LeetCode-main/solutions/792. Number of Matching Subsequences/792-2.py

```
class Solution:
    def numMatchingSubseq(self, s: str, words: list[str]) -> int:
        ans = 0
        # [(i, j)] := words[i] and the letter words[i][j] is waiting for
        bucket = [[] for _ in range(26)]

        # For each word, it's waiting for word[0].
        for i, word in enumerate(words):
            bucket[ord(word[0]) - ord('a')].append((i, 0))

        for c in s:
            # Let prevBucket = bucket[c] and clear bucket[c].
            index = ord(c) - ord('a')
            prevBucket = bucket[index]
            bucket[index] = []
            for i, j in prevBucket:
                j += 1
                if j == len(words[i]): # All the letters in words[i] are matched.
                    ans += 1
                else:
                    bucket[ord(words[i][j]) - ord('a')].append((i, j))

        return ans
```

2962. 792.py

Path: LeetCode-main/solutions/792. Number of Matching Subsequences/792.py

```

class Solution:
    def numMatchingSubseq(self, s: str, words: list[str]) -> int:
        root = {}

        def insert(word: str) -> None:
            node = root
            for c in word:
                if c not in node:
                    node[c] = {'count': 0}
                node = node[c]
            node['count'] += 1

        for word in words:
            insert(word)

        def dfs(s: str, i: int, node: dict) -> int:
            ans = node['count'] if 'count' in node else 0

            if i >= len(s):
                return ans

            for c in string.ascii_lowercase:
                if c in node:
                    try:
                        index = s.index(c, i)
                        ans += dfs(s, index + 1, node[c])
                    except ValueError:
                        continue

            return ans

        return dfs(s, 0, root)

```

2963. 794.py

Path: LeetCode-main/solutions/794. Valid Tic-Tac-Toe State/794.py

```

class Solution:
    def validTicTacToe(self, board: list[str]) -> bool:
        def isWin(c: str) -> bool:
            return (any(row.count(c) == 3 for row in board) or
                    any(row.count(c) == 3 for row in list(zip(*board))) or
                    all(board[i][i] == c for i in range(3)) or
                    all(board[i][2 - i] == c for i in range(3)))

        countX = sum(row.count('X') for row in board)
        countO = sum(row.count('O') for row in board)

        if countX < countO or countX - countO > 1:
            return False

        if isWin('X') and countX == countO or isWin('O') and countX != countO:
            return False

        return True

```

2964. 795.py

Path: LeetCode-main/solutions/795. Number of Subarrays with Bounded Maximum/795.py

```
class Solution:
    def numSubarrayBoundedMax(
        self,
        nums: list[int],
        left: int,
        right: int,
    ) -> int:
        ans = 0
        l = -1
        r = -1

        for i, num in enumerate(nums):
            if num > right: # Handle the reset value.
                l = i
            if num >= left: # Handle the reset and the needed value.
                r = i
            ans += r - l

        return ans
```

2965. 796.py

Path: LeetCode-main/solutions/796. Rotate String/796.py

```
class Solution:
    def rotateString(self, s: str, goal: str) -> bool:
        return len(s) == len(goal) and goal in s + s
```

2966. 797.py

Path: LeetCode-main/solutions/797. All Paths From Source to Target/797.py

```
class Solution:
    def allPathsSourceTarget(self, graph: list[list[int]]) -> list[list[int]]:
        ans = []

        def dfs(u: int, path: list[int]) -> None:
            if u == len(graph) - 1:
                ans.append(path)
                return

            for v in graph[u]:
                dfs(v, path + [v])

        dfs(0, [0])
        return ans
```

2967. 8.py

Path: LeetCode-main/solutions/8. String to Integer (atoi)/8.py

```

class Solution:
    def myAtoi(self, s: str) -> int:
        s = s.strip()
        if not s:
            return 0

        sign = -1 if s[0] == '-' else 1
        if s[0] in {'-', '+'}:
            s = s[1:]

        num = 0

        for c in s:
            if not c.isdigit():
                break
            num = num * 10 + int(c)
            if sign * num <= -2**31:
                return -2**31
            if sign * num >= 2**31 - 1:
                return 2**31 - 1

        return sign * num

```

2968. 80.py

Path: LeetCode-main/solutions/80. Remove Duplicates from Sorted Array II/80.py

```

class Solution:
    def removeDuplicates(self, nums: list[int]) -> int:
        i = 0

        for num in nums:
            if i < 2 or num != nums[i - 2]:
                nums[i] = num
                i += 1

        return i

```

2969. 800.py

Path: LeetCode-main/solutions/800. Similar RGB Color/800.py

```

class Solution:
    def similarRGB(self, color: str) -> str:
        SHORTHANDS = ['00', '11', '22', '33', '44', '55', '66', '77', '88', '99',
                      'aa', 'bb', 'cc', 'dd', 'ee', 'ff']
        ans = ['#']

        for i in range(1, len(color), 2):
            currValue = int(color[i:i + 2], 16)
            closestShorthand = min(SHORTHANDS,
                                   key=lambda x: (currValue - int(x, 16))**2)
            ans.append(closestShorthand)

        return ''.join(ans)

```

2970. 801.py

Path: LeetCode-main/solutions/801. Minimum Swaps To Make Sequences Increasing/801.py

```
class Solution:
    def minSwap(self, nums1: list[int], nums2: list[int]) -> int:
        keepAt = [math.inf] * len(nums1)
        swapAt = [math.inf] * len(nums1)
        keepAt[0] = 0
        swapAt[0] = 1

        for i in range(1, len(nums1)):
            if nums1[i] > nums1[i - 1] and nums2[i] > nums2[i - 1]:
                keepAt[i] = keepAt[i - 1]
                swapAt[i] = swapAt[i - 1] + 1
            if nums1[i] > nums2[i - 1] and nums2[i] > nums1[i - 1]:
                keepAt[i] = min(keepAt[i], swapAt[i - 1])
                swapAt[i] = min(swapAt[i], keepAt[i - 1] + 1)

        return min(keepAt[-1], swapAt[-1])
```

2971. 802.py

Path: LeetCode-main/solutions/802. Find Eventual Safe States/802.py

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def eventualSafeNodes(self, graph: list[list[int]]) -> list[int]:
        states = [State.INIT] * len(graph)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            if any(hasCycle(v) for v in graph[u]):
                return True
            states[u] = State.VISITED
            return False

        return [i for i in range(len(graph)) if not hasCycle(i)]
```

2972. 804.py

Path: LeetCode-main/solutions/804. Unique Morse Code Words/804.py

```

class Solution:
    def uniqueMorseRepresentations(self, words: list[str]) -> int:
        morse = [".-.", "-...", "-.-.", "-..", ".", "..-.", "---", "...", "..",
                  ".---", "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.",
                  "...", "-", "-..", "...-", "--", "-.-", "-.--", "--.."]

        transformations = set()

        for word in words:
            transformation = ''.join(
                morse[ord(c) - ord('a')] for c in word)
            transformations.add(transformation)

        return len(transformations)

```

2973. 805.py

Path: LeetCode-main/solutions/805. Split Array With Same Average/805.py

```

class Solution:
    def splitArraySameAverage(self, nums: list[int]) -> bool:
        n = len(nums)
        summ = sum(nums)
        if not any(i * summ % n == 0 for i in range(1, n // 2 + 1)):
            return False

        sums = [set() for _ in range(n // 2 + 1)]
        sums[0].add(0)

        for num in nums:
            for i in range(n // 2, 0, -1):
                for val in sums[i - 1]:
                    sums[i].add(num + val)

        for i in range(1, n // 2 + 1):
            if i * summ % n == 0 and i * summ // n in sums[i]:
                return True

        return False

```

2974. 806.py

Path: LeetCode-main/solutions/806. Number of Lines To Write String/806.py

```

class Solution:
    def numberOfLines(self, widths: list[int], s: str) -> list[int]:
        numLines = 1
        runningWidth = 0

        for c in s:
            width = widths[ord(c) - ord('a')]
            if runningWidth + width <= 100:
                runningWidth += width
            else:
                numLines += 1
                runningWidth = width

        return [numLines, runningWidth]

```

2975. 807.py

Path: LeetCode-main/solutions/807. Max Increase to Keep City Skyline/807.py

```
class Solution:
    def maxIncreaseKeepingSkyline(self, grid: list[list[int]]) -> int:
        rowMax = list(map(max, grid))
        colMax = list(map(max, zip(*grid)))
        return sum(min(i, j) for i in rowMax for j in colMax) - sum(map(sum, grid))
```

2976. 808.py

Path: LeetCode-main/solutions/808. Soup Servings/808.py

```
class Solution:
    def soupServings(self, n: int) -> float:
        @functools.lru_cache(None)
        def dfs(a: int, b: int) -> float:
            if a <= 0 and b <= 0:
                return 0.5
            if a <= 0:
                return 1.0
            if b <= 0:
                return 0.0
            return 0.25 * (dfs(a - 4, b) +
                           dfs(a - 3, b - 1) +
                           dfs(a - 2, b - 2) +
                           dfs(a - 1, b - 3))

        return 1 if n >= 4800 else dfs((n + 24) // 25, (n + 24) // 25)
```

2977. 809.py

Path: LeetCode-main/solutions/809. Expressive Words/809.py

```
class Solution:
    def expressiveWords(self, s: str, words: list[str]) -> int:
        def isStretchy(word: str) -> bool:
            n = len(s)
            m = len(word)

            j = 0
            for i in range(n):
                if j < m and s[i] == word[j]:
                    j += 1
                elif i > 1 and s[i] == s[i - 1] == s[i - 2]:
                    continue
                elif 0 < i < n - 1 and s[i - 1] == s[i] == s[i + 1]:
                    continue
                else:
                    return False

            return j == m

        return sum(isStretchy(word) for word in words)
```

2978. 81.py

Path: LeetCode-main/solutions/81. Search in Rotated Sorted Array II/81.py

```
class Solution:
    def search(self, nums: list[int], target: int) -> bool:
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return True
            if nums[l] == nums[m] == nums[r]:
                l += 1
                r -= 1
            elif nums[l] <= nums[m]: # nums[l..m] are sorted
                if nums[l] <= target < nums[m]:
                    r = m - 1
                else:
                    l = m + 1
            else: # nums[m..n - 1] are sorted
                if nums[m] < target <= nums[r]:
                    l = m + 1
                else:
                    r = m - 1

        return False
```

2979. 810.py

Path: LeetCode-main/solutions/810. Chalkboard XOR Game/810.py

```
class Solution:
    def xorGame(self, nums: list[int]) -> bool:
        return functools.reduce(operator.xor, nums) == 0 or len(nums) % 2 == 0
```

2980. 811.py

Path: LeetCode-main/solutions/811. Subdomain Visit Count/811.py

```
class Solution:
    def subdomainVisits(self, cpdomains: list[str]) -> list[str]:
        ans = []
        count = collections.Counter()

        for cpdomain in cpdomains:
            num, domains = cpdomain.split()
            num, domains = int(num), domains.split('.')
            for i in reversed(range(len(domains))):
                count['.'.join(domains[i:])] += num

        return [str(freq) + ' ' + domain for domain, freq in count.items()]
```

2981. 812.py

Path: LeetCode-main/solutions/812. Largest Triangle Area/812.py

```
class Solution:
    def largestTriangleArea(self, points: list[list[int]]) -> float:
        ans = 0

        for Ax, Ay in points:
            for Bx, By in points:
                for Cx, Cy in points:
                    ans = max(ans, 0.5 * abs((Bx - Ax) * (Cy - Ay) -
                                             (Cx - Ax) * (By - Ay)))

        return ans
```

2982. 814.py

Path: LeetCode-main/solutions/814. Binary Tree Pruning/814.py

```
class Solution:
    def pruneTree(self, root: TreeNode | None) -> TreeNode | None:
        if not root:
            return None
        root.left = self.pruneTree(root.left)
        root.right = self.pruneTree(root.right)
        if not root.left and not root.right and not root.val:
            return None
        return root
```

2983. 815.py

Path: LeetCode-main/solutions/815. Bus Routes/815.py

```

class Solution:
    def numBusesToDestination(
        self,
        routes: list[list[int]],
        source: int,
        target: int,
    ) -> int:
        if source == target:
            return 0

        graph = collections.defaultdict(list)
        usedBuses = set()

        for i in range(len(routes)):
            for route in routes[i]:
                graph[route].append(i)

        q = collections.deque([source])

        step = 1
        while q:
            for _ in range(len(q)):
                for bus in graph[q.popleft()]:
                    if bus in usedBuses:
                        continue
                    usedBuses.add(bus)
                    for nextRoute in routes[bus]:
                        if nextRoute == target:
                            return step
                        q.append(nextRoute)
                    q.append(nextRoute)
            step += 1

        return -1

```

2984. 816.py

Path: LeetCode-main/solutions/816. Ambiguous Coordinates/816.py

```

class Solution:
    def ambiguousCoordinates(self, s: str) -> list[str]:
        def splits(s: str) -> list[str]:
            if not s or len(s) > 1 and s[0] == s[-1] == '0':
                return []
            if s[-1] == '0':
                return [s]
            if s[0] == '0':
                return [s[0] + '.' + s[1:]]
            return [s] + [s[:i] + '.' + s[i:] for i in range(1, len(s))]

        ans = []
        s = s[1:-1]

        for i in range(1, len(s)):
            for x in splits(s[:i]):
                for y in splits(s[i:]):
                    ans.append('(%s, %s)' % (x, y))

        return ans

```

2985. 817.py

Path: LeetCode-main/solutions/817. Linked List Components/817.py

```
class Solution:
    def numComponents(self, head: ListNode | None, nums: list[int]) -> int:
        ans = 0
        numsSet = set(nums)

        while head:
            if head.val in numsSet and (
                head.next == None or head.next.val not in numsSet):
                ans += 1
            head = head.next

        return ans
```

2986. 819.py

Path: LeetCode-main/solutions/819. Most Common Word/819.py

```
class Solution:
    def mostCommonWord(self, paragraph: str, banned: list[str]) -> str:
        banned = set(banned)
        words = re.findall(r'\w+', paragraph.lower())
        return collections.Counter(
            word for word in words if word not in banned).most_common(1)[0][0]
```

2987. 82.py

Path: LeetCode-main/solutions/82. Remove Duplicates from Sorted List II/82.py

```
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        prev = dummy

        while head:
            while head.next and head.val == head.next.val:
                head = head.next
            if prev.next == head:
                prev = prev.next
            else:
                prev.next = head.next
            head = head.next

        return dummy.next
```

2988. 820.py

Path: LeetCode-main/solutions/820. Short Encoding of Words/820.py

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.depth = 0

class Solution:
    def minimumLengthEncoding(self, words: list[str]) -> int:
        root = TrieNode()
        leaves = []

        def insert(word: str) -> TrieNode:
            node = root
            for c in reversed(word):
                node = node.children.setdefault(c, TrieNode())
            node.depth = len(word)
            return node

        for word in set(words):
            leaves.append(insert(word))

        return sum(leaf.depth + 1 for leaf in leaves
                    if not len(leaf.children))

```

2989. 821.py

Path: LeetCode-main/solutions/821. Shortest Distance to a Character/821.py

```

class Solution:
    def shortestToChar(self, s: str, c: str) -> list[int]:
        n = len(s)
        ans = [0] * n
        prev = -n

        for i in range(n):
            if s[i] == c:
                prev = i
                ans[i] = i - prev

        for i in range(prev - 1, -1, -1):
            if s[i] == c:
                prev = i
                ans[i] = min(ans[i], prev - i)

        return ans

```

2990. 822.py

Path: LeetCode-main/solutions/822. Card Flipping Game/822.py

```

class Solution:
    def flipgame(self, fronts: list[int], backs: list[int]) -> int:
        same = {front
                 for front, back in zip(fronts, backs)
                 if front == back}
        return min([num for num in fronts + backs
                    if num not in same] or [0])

```

2991. 823.py

Path: LeetCode-main/solutions/823. Binary Trees With Factors/823.py

```
class Solution:
    def numFactoredBinaryTrees(self, arr: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(arr)
        # dp[i] := the number of binary trees with arr[i] as the root
        dp = [1] * n
        arr.sort()
        numToIndex = {a: i for i, a in enumerate(arr)}

        for i, root in enumerate(arr): # arr[i] is the root
            for j in range(i):
                if root % arr[j] == 0: # arr[j] is the left subtree
                    right = root // arr[j]
                    if right in numToIndex:
                        dp[i] += dp[j] * dp[numToIndex[right]]
                        dp[i] %= MOD

        return sum(dp) % MOD
```

2992. 824.py

Path: LeetCode-main/solutions/824. Goat Latin/824.py

```
class Solution:
    def toGoatLatin(self, sentence: str) -> str:
        ans = []
        VOWELS = 'aeiouAEIOU'

        i = 1
        for word in sentence.split():
            if i > 1:
                ans.append(' ')
            if word[0] in VOWELS:
                ans.append(word)
            else:
                ans.append(word[1:] + word[0])
            ans.append('ma' + 'a' * i)
            i += 1

        return ''.join(ans)
```

2993. 825.py

Path: LeetCode-main/solutions/825. Friends Of Appropriate Ages/825.py

```
class Solution:
    def numFriendRequests(self, ages: list[int]) -> int:
        ans = 0
        count = [0] * 121

        for age in ages:
            count[age] += 1

        for i in range(15, 121):
            ans += count[i] * (count[i] - 1)

        for i in range(15, 121):
            for j in range(i // 2 + 8, i):
                ans += count[i] * count[j]

        return ans
```

2994. 826.py

Path: LeetCode-main/solutions/826. Most Profit Assigning Work/826.py

```
class Solution:
    def maxProfitAssignment(
        self,
        difficulty: list[int],
        profit: list[int],
        worker: list[int],
    ) -> int:
        ans = 0
        jobs = sorted(zip(difficulty, profit))
        worker.sort(reverse=1)

        i = 0
        maxProfit = 0

        for w in sorted(worker):
            while i < len(jobs) and w >= jobs[i][0]:
                maxProfit = max(maxProfit, jobs[i][1])
                i += 1
            ans += maxProfit

        return ans
```

2995. 828-2.py

Path: LeetCode-main/solutions/828. Count Unique Characters of All Substrings of a Given String/828-2.py

```

class Solution:
    def uniqueLetterString(self, s: str) -> int:
        ans = 0
        # lastSeen[c] := the index of the last time ('a' + i) appeared
        lastSeen = collections.defaultdict(lambda: -1)
        # prevSeen[c] := the previous index of the last time ('a' + i) appeared
        prevLastSeen = collections.defaultdict(lambda: -1)

        for i, c in enumerate(s):
            if c in lastSeen:
                ans += (i - lastSeen[c]) * (lastSeen[c] - prevLastSeen[c])
                prevLastSeen[c] = lastSeen[c]
                lastSeen[c] = i

        for c in string.ascii_uppercase:
            ans += (len(s) - lastSeen[c]) * (lastSeen[c] - prevLastSeen[c])

        return ans

```

2996. 828.py

Path: LeetCode-main/solutions/828. Count Unique Characters of All Substrings of a Given String/828.py

```

class Solution:
    def uniqueLetterString(self, s: str) -> int:
        ans = 0
        # the number of unique letters in all the substrings ending in the index so
        # far
        dp = 0
        lastCount = {}
        lastSeen = {}

        for i, c in enumerate(s):
            newCount = i - lastSeen.get(c, -1)
            # Subtract the duplicates.
            dp -= lastCount.get(c, 0)
            # Add count of s[lastSeen[c] + 1..i], s[lastSeen[c] + 2..i], ..., s[i].
            dp += newCount
            lastCount[c] = newCount
            lastSeen[c] = i
            ans += dp

        return ans

```

2997. 829.py

Path: LeetCode-main/solutions/829. Consecutive Numbers Sum/829.py

```

class Solution:
    def consecutiveNumbersSum(self, n: int) -> int:
        ans = 0
        i = 1
        triangleNum = 1
        while triangleNum <= n:
            if (n - triangleNum) % i == 0:
                ans += 1
            i += 1
            triangleNum += i
        return ans

```

2998. 83.py

Path: LeetCode-main/solutions/83. Remove Duplicates from Sorted List/83.py

```
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        curr = head

        while curr:
            while curr.next and curr.val == curr.next.val:
                curr.next = curr.next.next
            curr = curr.next

        return head
```

2999. 830.py

Path: LeetCode-main/solutions/830. Positions of Large Groups/830.py

```
class Solution:
    def largeGroupPositions(self, s: str) -> list[list[int]]:
        n = len(s)
        ans = []
        i = 0
        j = 0

        while i < n:
            while j < n and s[j] == s[i]:
                j += 1
            if j - i >= 3:
                ans.append([i, j - 1])
            i = j

        return ans
```

3000. 831.py

Path: LeetCode-main/solutions/831. Masking Personal Information/831.py

```
class Solution:
    def maskPII(self, s: str) -> str:
        atIndex = s.find('@')
        if atIndex != -1:
            s = s.lower()
            return s[0] + '*' * 5 + s[atIndex - 1:]

        ans = ''.join(c for c in s if c.isdigit())

        if len(ans) == 10:
            return '***-***-' + ans[-4:]
        return '+' + '*' * (len(ans) - 10) + '-***-***-' + ans[-4:]
```

3001. 832.py

Path: LeetCode-main/solutions/832. Flipping an Image/832.py

```
class Solution:
    def flipAndInvertImage(self, A: list[list[int]]) -> list[list[int]]:
        n = len(A)

        for i in range(n):
            for j in range((n + 2) // 2):
                A[i][j], A[i][n - j - 2] = A[i][n - j - 1] ^ 2, A[i][j] ^ 1

        return A
```

3002. 833.py

Path: LeetCode-main/solutions/833. Find And Replace in String/833.py

```
class Solution:
    def findReplaceString(
        self,
        s: str,
        indices: list[int],
        sources: list[str],
        targets: list[str]
    ) -> str:
        for index, source, target in sorted(zip(indices, sources, targets),
                                           reverse=True):
            if s[index:index + len(source)] == source:
                s = s[:index] + target + s[index + len(source):]
        return s
```

3003. 834.py

Path: LeetCode-main/solutions/834. Sum of Distances in Tree/834.py

```

class Solution:
    def sumOfDistancesInTree(self, n: int, edges: list[list[int]]) -> list[int]:
        ans = [0] * n
        count = [1] * n
        tree = [set() for _ in range(n)]

        for u, v in edges:
            tree[u].add(v)
            tree[v].add(u)

        def postorder(u: int, prev: int) -> None:
            for v in tree[u]:
                if v == prev:
                    continue
                postorder(v, u)
                count[u] += count[v]
                ans[u] += ans[v] + count[v]

        def preorder(u: int, prev: int) -> None:
            for v in tree[u]:
                if v == prev:
                    continue
                # count[v] us are 1 step closer from v than prev.
                # (n - count[v]) us are 1 step farther from v than prev.
                ans[v] = ans[u] - count[v] + (n - count[v])
                preorder(v, u)

        postorder(0, -1)
        preorder(0, -1)
        return ans

```

3004. 835.py

Path: LeetCode-main/solutions/835. Image Overlap/835.py

```

class Solution:
    def largestOverlap(self, img1: list[list[int]], img2: list[list[int]]) -> int:
        MAGIC = 100
        ones1 = [(i, j)
                  for i, row in enumerate(img1)
                  for j, num in enumerate(row)
                  if num == 1]
        ones2 = [(i, j)
                  for i, row in enumerate(img2)
                  for j, num in enumerate(row)
                  if num == 1]
        offsetCount = collections.Counter()

        for ax, ay in ones1:
            for bx, by in ones2:
                offsetCount[(ax - bx) * MAGIC + (ay - by)] += 1

        return max(offsetCount.values()) if offsetCount else 0

```

3005. 836.py

Path: LeetCode-main/solutions/836. Rectangle Overlap/836.py

```
class Solution:
    def isRectangleOverlap(self, rec1: list[int], rec2: list[int]) -> bool:
        return rec1[0] < rec2[2] and rec2[0] < rec1[2] and rec1[1] < rec2[3] and rec2[1] < rec1[3]
```

3006. 837.py

Path: LeetCode-main/solutions/837. New 21 Game/837.py

```
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        # When the game ends, the point is in [k..k - 1 maxPts].
        # P = 1, if n >= k - 1 + maxPts
        # P = 0, if n < k (note that the constraints already have k <= n)
        if k == 0 or n >= k - 1 + maxPts:
            return 1.0

        ans = 0.0
        dp = [1.0] + [0] * n # dp[i] := the probability to have i points
        windowSum = dp[0] # P(i - 1) + P(i - 2) + ... + P(i - maxPts)

        for i in range(1, n + 1):
            # The probability to get i points is
            # P(i) = [P(i - 1) + P(i - 2) + ... + P(i - maxPts)] / maxPts
            dp[i] = windowSum / maxPts
            if i < k:
                windowSum += dp[i]
            else: # The game ends.
                ans += dp[i]
            if i - maxPts >= 0:
                windowSum -= dp[i - maxPts]

        return ans
```

3007. 838.py

Path: LeetCode-main/solutions/838. Push Dominoes/838.py

```

class Solution:
    def pushDominoes(self, dominoes: str) -> str:
        ans = list(dominoes)
        L = -1
        R = -1

        for i in range(len(dominoes) + 1):
            if i == len(dominoes) or dominoes[i] == 'R':
                if L < R:
                    while R < i:
                        ans[R] = 'R'
                        R += 1
                    R = i
                elif dominoes[i] == 'L':
                    if R < L or (L, R) == (-1, -1):
                        if (L, R) == (-1, -1):
                            L += 1
                        while L < i:
                            ans[L] = 'L'
                            L += 1
                    else:
                        l = R + 1
                        r = i - 1
                        while l < r:
                            ans[l] = 'R'
                            ans[r] = 'L'
                            l += 1
                            r -= 1
                        L = i
                else:
                    pass

        return ''.join(ans)

```

3008. 84.py

Path: LeetCode-main/solutions/84. Largest Rectangle in Histogram/84.py

```

class Solution:
    def largestRectangleArea(self, heights: list[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(heights) + 1):
            while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                h = heights[stack.pop()]
                w = i - stack[-1] - 1 if stack else i
                ans = max(ans, h * w)
            stack.append(i)

        return ans

```

3009. 840.py

Path: LeetCode-main/solutions/840. Magic Squares In Grid/840.py

```

class Solution:
    def numMagicSquaresInside(self, grid: list[list[int]]) -> int:
        def isMagic(i: int, j: int) -> int:
            s = "".join(str(grid[i + num // 3][j + num % 3])
                        for num in [0, 1, 2, 5, 8, 7, 6, 3])
            return s in "43816729" * 2 or s in "43816729"[::-1] * 2

        ans = 0

        for i in range(len(grid) - 2):
            for j in range(len(grid[0]) - 2):
                if grid[i][j] % 2 == 0 and grid[i + 1][j + 1] == 5:
                    ans += isMagic(i, j)

        return ans

```

3010. 841.py

Path: LeetCode-main/solutions/841. Keys and Rooms/841.py

```

class Solution:
    def canVisitAllRooms(self, rooms: list[list[int]]) -> bool:
        seen = [False] * len(rooms)

        def dfs(node: int) -> None:
            seen[node] = True
            for child in rooms[node]:
                if not seen[child]:
                    dfs(child)

        dfs(0)
        return all(seen)

```

3011. 843.py

Path: LeetCode-main/solutions/843. Guess the Word/843.py

```

# """
# This is Master's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class Master:
#     def guess(self, word: str) -> int:

class Solution:
    def findSecretWord(self, words: list[str], master: 'Master') -> None:
        for _ in range(10):
            guessedWord = words[random.randint(0, len(words) - 1)]
            matches = master.guess(guessedWord)
            if matches == 6:
                break
            words = [
                word for word in words
                if sum(c1 == c2 for c1, c2 in zip(guessedWord, word)) == matches]

```

3012. 844-2.py

Path: LeetCode-main/solutions/844. Backspace String Compare/844-2.py

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i = len(s) - 1 # s' index
        j = len(t) - 1 # t's index

        while True:
            # Delete characters of s if needed.
            backspace = 0
            while i >= 0 and (s[i] == '#' or backspace > 0):
                backspace += 1 if s[i] == '#' else -1
                i -= 1
            # Delete characters of t if needed.
            backspace = 0
            while j >= 0 and (t[j] == '#' or backspace > 0):
                backspace += 1 if t[j] == '#' else -1
                j -= 1
            if i >= 0 and j >= 0 and s[i] == t[j]:
                i -= 1
                j -= 1
            else:
                break

        return i == -1 and j == -1
```

3013. 844.py

Path: LeetCode-main/solutions/844. Backspace String Compare/844.py

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        def backspace(s: str) -> str:
            stack = []
            for c in s:
                if c != '#':
                    stack.append(c)
                elif stack:
                    stack.pop()
            return stack

        return backspace(s) == backspace(t)
```

3014. 845.py

Path: LeetCode-main/solutions/845. Longest Mountain in Array/845.py

```
class Solution:
    def longestMountain(self, arr: list[int]) -> int:
        ans = 0
        i = 0

        while i + 1 < len(arr):
            while i + 1 < len(arr) and arr[i] == arr[i + 1]:
                i += 1

            increasing = 0
            decreasing = 0

            while i + 1 < len(arr) and arr[i] < arr[i + 1]:
                increasing += 1
                i += 1

            while i + 1 < len(arr) and arr[i] > arr[i + 1]:
                decreasing += 1
                i += 1

            if increasing > 0 and decreasing > 0:
                ans = max(ans, increasing + decreasing + 1)

        return ans
```

3015. 846.py

Path: LeetCode-main/solutions/846. Hand of Straights/846.py

```
class Solution:
    def isNStraightHand(self, hand: list[int], groupSize: int) -> bool:
        count = collections.Counter(hand)

        for start in sorted(count):
            value = count[start]
            if value > 0:
                for i in range(start, start + groupSize):
                    count[i] -= value
                    if count[i] < 0:
                        return False

        return True
```

3016. 847.py

Path: LeetCode-main/solutions/847. Shortest Path Visiting All Nodes/847.py


```

class Solution:
    def shortestPathLength(self, graph: list[list[int]]) -> int:
        n = len(graph)
        goal = (1 << n) - 1
        q = collections.deque() # (u, state)
        seen = set()

        for i in range(n):
            q.append((i, 1 << i))

        step = 0
        while q:
            for _ in range(len(q)):
                u, state = q.popleft()
                if state == goal:
                    return step
                if (u, state) in seen:
                    continue
                seen.add((u, state))
                for v in graph[u]:
                    q.append((v, state | 1 << v))
            step += 1

        return -1

```

3017. 848.py

Path: LeetCode-main/solutions/848. Shifting Letters/848.py

```

class Solution:
    def shiftingLetters(self, s: str, shifts: list[int]) -> str:
        ans = []

        for i in reversed(range(len(shifts) - 1)):
            shifts[i] += shifts[i + 1]

        for c, shift in zip(s, shifts):
            ans.append(chr((ord(c) - ord('a') + shift) % 26 + ord('a')))

        return ''.join(ans)

```

3018. 849.py

Path: LeetCode-main/solutions/849. Maximize Distance to Closest Person/849.py

```

class Solution:
    def maxDistToClosest(self, seats: list[int]) -> int:
        n = len(seats)
        ans = 0
        j = -1

        for i in range(n):
            if seats[i] == 1:
                ans = i if j == -1 else max(ans, (i - j) // 2)
                j = i

        return max(ans, n - j - 1)

```

3019. 85.py

Path: LeetCode-main/solutions/85. Maximal Rectangle/85.py

```
class Solution:
    def maximalRectangle(self, matrix: list[list[str]]) -> int:
        if not matrix:
            return 0

        ans = 0
        hist = [0] * len(matrix[0])

        def largestRectangleArea(heights: list[int]) -> int:
            ans = 0
            stack = []

            for i in range(len(heights) + 1):
                while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                    h = heights[stack.pop()]
                    w = i - stack[-1] - 1 if stack else i
                    ans = max(ans, h * w)
                    stack.append(i)

            return ans

        for row in matrix:
            for i, num in enumerate(row):
                hist[i] = 0 if num == '0' else hist[i] + 1
            ans = max(ans, largestRectangleArea(hist))

        return ans
```

3020. 850.py

Path: LeetCode-main/solutions/850. Rectangle Area II/850.py

```

class Solution:
    def rectangleArea(self, rectangles: list[list[int]]) -> int:
        events = []

        for x1, y1, x2, y2 in rectangles:
            events.append((x1, y1, y2, 's'))
            events.append((x2, y1, y2, 'e'))

        events.sort(key=lambda x: x[0])

        ans = 0
        prevX = 0
        yPairs = []

        def getHeight(yPairs: list[tuple[int, int]]) -> int:
            height = 0
            prevY = 0

            for y1, y2 in yPairs:
                prevY = max(prevY, y1)
                if y2 > prevY:
                    height += y2 - prevY
                    prevY = y2

            return height

        for currX, y1, y2, type in events:
            if currX > prevX:
                width = currX - prevX
                ans += width * getHeight(yPairs)
                prevX = currX
            if type == 's':
                yPairs.append((y1, y2))
                yPairs.sort()
            else: # type == 'e'
                yPairs.remove((y1, y2))

        return ans % (10**9 + 7)

```

3021. 851.py

Path: LeetCode-main/solutions/851. Loud and Rich/851.py

```

class Solution:
    def loudAndRich(self, richer: list[list[int]], quiet: list[int]) -> list[int]:
        graph = [[] for _ in range(len(quiet))]

        for v, u in richer:
            graph[u].append(v)

        @functools.lru_cache(None)
        def dfs(u: int) -> int:
            ans = u

            for v in graph[u]:
                res = dfs(v)
                if quiet[res] < quiet[ans]:
                    ans = res

            return ans

        return map(dfs, range(len(graph)))

```

3022. 852.py

Path: LeetCode-main/solutions/852. Peak Index in a Mountain Array/852.py

```

class Solution:
    def peakIndexInMountainArray(self, arr: list[int]) -> int:
        l = 0
        r = len(arr) - 1

        while l < r:
            m = (l + r) // 2
            if arr[m] >= arr[m + 1]:
                r = m
            else:
                l = m + 1

        return l

```

3023. 853.py

Path: LeetCode-main/solutions/853. Car Fleet/853.py

```

class Solution:
    def carFleet(self, target: int, position: list[int], speed: list[int]) -> int:
        ans = 0
        times = [
            float(target - p) / s for p, s in sorted(zip(position, speed),
                                                    reverse=True)]

        maxTime = 0 # the time of the slowest car to reach the target

        for time in times:
            # A car needs more time to reach the target, so it becomes the slowest.
            if time > maxTime:
                maxTime = time
                ans += 1

        return ans

```

3024. 854.py

Path: LeetCode-main/solutions/854. K-Similar Strings/854.py

```
class Solution:
    def kSimilarity(self, s1: str, s2: str) -> int:
        q = collections.deque([s1])
        seen = {s1}

        step = 0
        while q:
            for _ in range(len(q)):
                curr = q.popleft()
                if curr == s2:
                    return step
                for child in self._getChildren(curr, s2):
                    if child in seen:
                        continue
                    q.append(child)
                    seen.add(child)
            step += 1

        return -1

    def _getChildren(self, curr: str, target: str) -> list[str]:
        children = []
        s = list(curr)
        i = 0 # the first index s.t. curr[i] != target[i]
        while curr[i] == target[i]:
            i += 1

        for j in range(i + 1, len(s)):
            if s[j] == target[i]:
                s[i], s[j] = s[j], s[i]
                children.append(''.join(s))
                s[i], s[j] = s[j], s[i]

        return children
```

3025. 856.py

Path: LeetCode-main/solutions/856. Score of Parentheses/856.py

```
class Solution:
    def scoreOfParentheses(self, s: str) -> int:
        ans = 0
        layer = 0

        for a, b in itertools.pairwise(s):
            if a + b == '()':
                ans += 1 << layer
                layer += 1 if a == '(' else -1

        return ans
```

3026. 857.py

Path: LeetCode-main/solutions/857. Minimum Cost to Hire K Workers/857.py

```

class Solution:
    def mincostToHireWorkers(
        self,
        quality: list[int],
        wage: list[int],
        k: int,
    ) -> float:
        ans = math.inf
        qualitySum = 0
        # (wagePerQuality, quality) sorted by wagePerQuality
        workers = sorted((w / q, q) for q, w in zip(quality, wage))
        maxHeap = []

        for wagePerQuality, q in workers:
            heapq.heappush(maxHeap, -q)
            qualitySum += q
            if len(maxHeap) > k:
                qualitySum += heapq.heappop(maxHeap)
            if len(maxHeap) == k:
                ans = min(ans, qualitySum * wagePerQuality)

        return ans

```

3027. 858.py

Path: LeetCode-main/solutions/858. Mirror Reflection/858.py

```

class Solution:
    def mirrorReflection(self, p: int, q: int) -> int:
        while p % 2 == 0 and q % 2 == 0:
            p //= 2
            q //= 2

        if p % 2 == 0:
            return 2
        if q % 2 == 0:
            return 0
        return 1

```

3028. 859.py

Path: LeetCode-main/solutions/859. Buddy Strings/859.py

```

class Solution:
    def buddyStrings(self, s: str, goal: str) -> bool:
        if len(s) != len(goal):
            return False
        if s == goal and len(set(s)) < len(s):
            return True
        diffIndices = [i for i, (a, b) in enumerate(zip(s, goal))
                        if a != b]
        return (len(diffIndices) == 2 and
                s[diffIndices[0]] == goal[diffIndices[1]] and
                s[diffIndices[1]] == goal[diffIndices[0]])

```

3029. 86.py

Path: LeetCode-main/solutions/86. Partition List/86.py

```
class Solution:
    def partition(self, head: ListNode, x: int) -> ListNode:
        beforeHead = ListNode(0)
        afterHead = ListNode(0)
        before = beforeHead
        after = afterHead

        while head:
            if head.val < x:
                before.next = head
                before = head
            else:
                after.next = head
                after = head
            head = head.next

        after.next = None
        before.next = afterHead.next

        return beforeHead.next
```

3030. 860.py

Path: LeetCode-main/solutions/860. Lemonade Change/860.py

```
class Solution:
    def lemonadeChange(self, bills: list[int]) -> bool:
        fives = 0
        tens = 0

        for bill in bills:
            if bill == 5:
                fives += 1
            elif bill == 10:
                fives -= 1
                tens += 1
            else: # bill == 20
                if tens > 0:
                    tens -= 1
                    fives -= 1
                else:
                    fives -= 3
            if fives < 0:
                return False

        return True
```

3031. 861-2.py

Path: LeetCode-main/solutions/861. Score After Flipping Matrix/861-2.py

```

class Solution:
    def matrixScore(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = m # All the cells in the first column are 1.

        for j in range(1, n):
            # The best strategy is flipping the rows with a leading 0..
            onesCount = sum(grid[i][j] == grid[i][0] for i in range(m))
            ans = ans * 2 + max(onesCount, m - onesCount)

        return ans

```

3032. 861.py

Path: LeetCode-main/solutions/861. Score After Flipping Matrix/861.py

```

class Solution:
    def matrixScore(self, grid: list[list[int]]) -> int:
        # Flip the rows with a leading 0.
        for row in grid:
            if row[0] == 0:
                self._flip(row)

        # Flip the columns with 1s < 0s.
        for j, col in enumerate(list(zip(*grid))):
            if sum(col) * 2 < len(grid):
                self._flipCol(grid, j)

        # Add a binary number for each row.
        return sum(self._binary(row) for row in grid)

    def _flip(self, row: list[int]) -> None:
        for i in range(len(row)):
            row[i] ^= 1

    def _flipCol(self, grid: list[list[int]], j: int) -> None:
        for i in range(len(grid)):
            grid[i][j] ^= 1

    def _binary(self, row: list[int]) -> int:
        res = row[0]
        for j in range(1, len(row)):
            res = res * 2 + row[j]
        return res

```

3033. 862.py

Path: LeetCode-main/solutions/862. Shortest Subarray with Sum at Least K/862.py


```

class Solution:
    def shortestSubarray(self, nums: list[int], k: int) -> int:
        n = len(nums)
        ans = n + 1
        dq = collections.deque()
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(n + 1):
            while dq and prefix[i] - prefix[dq[0]] >= k:
                ans = min(ans, i - dq.popleft())
            while dq and prefix[i] <= prefix[dq[-1]]:
                dq.pop()
            dq.append(i)

        return ans if ans <= n else -1

```

3034. 866.py

Path: LeetCode-main/solutions/866. Prime Palindrome/866.py

```

class Solution:
    def primePalindrome(self, n: int) -> int:
        def getPalindromes(n: int) -> int:
            length = n // 2
            for i in range(10**(length - 1), 10**length):
                s = str(i)
                for j in range(10):
                    yield int(s + str(j) + s[::-1])

        def isPrime(num: int) -> bool:
            return not any(num % i == 0 for i in range(2, int(num**0.5 + 1)))

        if n <= 2:
            return 2
        if n == 3:
            return 3
        if n <= 5:
            return 5
        if n <= 7:
            return 7
        if n <= 11:
            return 11

        nLength = len(str(n))

        while True:
            for num in getPalindromes(nLength):
                if num >= n and isPrime(num):
                    return num
            nLength += 1

```

3035. 867.py

Path: LeetCode-main/solutions/867. Transpose Matrix/867.py

```
class Solution:
    def transpose(self, A: list[list[int]]) -> list[list[int]]:
        ans = [[0] * len(A) for _ in range(len(A[0]))]

        for i in range(len(A)):
            for j in range(len(A[0])):
                ans[j][i] = A[i][j]

        return ans
```

3036. 868.py

Path: LeetCode-main/solutions/868. Binary Gap/868.py

```
class Solution:
    def binaryGap(self, n: int) -> int:
        ans = 0
        d = -32 # the distance between any two 1s

        while n:
            if n % 2 == 1:
                ans = max(ans, d)
                d = 0
            n //= 2
            d += 1

        return ans
```

3037. 869.py

Path: LeetCode-main/solutions/869. Reordered Power of 2/869.py

```
class Solution:
    def reorderedPowerOf2(self, n: int) -> bool:
        count = collections.Counter(str(n))
        return any(Counter(str(1 << i)) == count for i in range(30))
```

3038. 87.py

Path: LeetCode-main/solutions/87. Scramble String/87.py

```
class Solution:
    @functools.lru_cache(None)
    def isScramble(self, s1: str, s2: str) -> bool:
        if s1 == s2:
            return True
        if collections.Counter(s1) != collections.Counter(s2):
            return False

        for i in range(1, len(s1)):
            if self.isScramble(s1[:i], s2[:i]) and self.isScramble(s1[i:], s2[i:]):
                return True
            if (self.isScramble(s1[:i], s2[len(s2) - i:]) and
                self.isScramble(s1[i:], s2[: len(s2) - i])):
                return True

        return False
```

3039. 870.py

Path: LeetCode-main/solutions/870. Advantage Shuffle/870.py

```
from sortedcontainers import SortedList

class Solution:
    def advantageCount(self, nums1: list[int], nums2: list[int]) -> list[int]:
        s1 = SortedList(nums1)

        for i, num in enumerate(nums2):
            index = 0 if s1[-1] <= num else s1.bisect_right(num)
            nums1[i] = s1[index]
            del s1[index]

        return nums1
```

3040. 871-2.py

Path: LeetCode-main/solutions/871. Minimum Number of Refueling Stops/871-2.py

```

class Solution:
    def minRefuelStops(
        self,
        target: int,
        startFuel: int,
        stations: list[list[int]],
    ) -> int:
        ans = 0
        i = 0 # station's index
        curr = startFuel
        maxHeap = []

        while curr < target:
            # Add all the reachable stops to maxHeap
            while i < len(stations) and stations[i][0] <= curr:
                heapq.heappush(maxHeap, -stations[i][1])
                i += 1
            if not maxHeap: # Can't be refueled.
                return -1
            curr -= heapq.heappop(maxHeap) # Pop out the largest gas.
            ans += 1 # Then, refuel once.

        return ans

```

3041. 871.py

Path: LeetCode-main/solutions/871. Minimum Number of Refueling Stops/871.py

```

class Solution:
    def minRefuelStops(
        self,
        target: int,
        startFuel: int,
        stations: list[list[int]],
    ) -> int:
        # dp[i] := the farthest position we can reach w / i refuels
        dp = [startFuel] + [0] * len(stations)

        for i, station in enumerate(stations):
            for j in range(i + 1, 0, -1):
                if dp[j - 1] >= station[0]:
                    dp[j] = max(dp[j], dp[j - 1] + station[1])

        for i, d in enumerate(dp):
            if d >= target:
                return i

        return -1

```

3042. 872.py

Path: LeetCode-main/solutions/872. Leaf-Similar Trees/872.py

```
class Solution:
    def leafSimilar(self, root1: TreeNode | None, root2: TreeNode | None) -> bool:
        def dfs(root: TreeNode | None) -> None:
            if not root:
                return
            if not root.left and not root.right:
                yield root.val
                return

            yield from dfs(root.left)
            yield from dfs(root.right)

        return list(dfs(root1)) == list(dfs(root2))
```

3043. 873.py

Path: LeetCode-main/solutions/873. Length of Longest Fibonacci Subsequence/873.py

```
class Solution:
    def lenLongestFibSubseq(self, arr: list[int]) -> int:
        n = len(arr)
        ans = 0
        numToIndex = {a: i for i, a in enumerate(arr)}
        dp = [[2] * n for _ in range(n)]

        for j in range(n):
            for k in range(j + 1, n):
                ai = arr[k] - arr[j]
                if ai < arr[j] and ai in numToIndex:
                    i = numToIndex[ai]
                    dp[j][k] = dp[i][j] + 1
                    ans = max(ans, dp[j][k])

        return ans
```

3044. 874.py

Path: LeetCode-main/solutions/874. Walking Robot Simulation/874.py

```

class Solution:
    def robotSim(self, commands: list[int], obstacles: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        ans = 0
        d = 0 # 0 := north, 1 := east, 2 := south, 3 := west
        x = 0 # the start x
        y = 0 # the start y
        obstaclesSet = {(x, y) for x, y in obstacles}

        for command in commands:
            if command == -1:
                d = (d + 1) % 4
            elif command == -2:
                d = (d + 3) % 4
            else:
                for _ in range(command):
                    if (x + DIRS[d][0], y + DIRS[d][1]) in obstaclesSet:
                        break
                    x += DIRS[d][0]
                    y += DIRS[d][1]
                ans = max(ans, x * x + y * y)

        return ans

```

3045. 875.py

Path: LeetCode-main/solutions/875. Koko Eating Bananas/875.py

```

class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        def eatHours(m: int) -> bool:
            """Returns the hours to eat all the piles with speed m."""
            return sum((pile - 1) // m + 1 for pile in piles)

        l = 1
        r = max(piles)
        return bisect.bisect_left(range(l, r), True,
                                key=lambda m: eatHours(m) <= h) + 1

```

3046. 876.py

Path: LeetCode-main/solutions/876. Middle of the Linked List/876.py

```

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        return slow

```

3047. 877-2.py

Path: LeetCode-main/solutions/877. Stone Game/877-2.py

```

class Solution:
    def stoneGame(self, piles: list[int]) -> bool:
        n = len(piles)
        dp = piles.copy()

        for d in range(1, n):
            for j in range(n - 1, d - 1, -1):
                i = j - d
                dp[j] = max(piles[i] - dp[j], piles[j] - dp[j - 1])

        return dp[n - 1] > 0

```

3048. 877.py

Path: LeetCode-main/solutions/877. Stone Game/877.py

```

class Solution:
    def stoneGame(self, piles: list[int]) -> bool:
        n = len(piles)
        # dp[i][j] := the maximum stones you can get more than your opponent in piles[i..j]
        dp = [[0] * n for _ in range(n)]

        for i, pile in enumerate(piles):
            dp[i][i] = pile

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                dp[i][j] = max(piles[i] - dp[i + 1][j],
                               piles[j] - dp[i][j - 1])

        return dp[0][n - 1] > 0

```

3049. 878.py

Path: LeetCode-main/solutions/878. Nth Magical Number/878.py

```

class Solution:
    def nthMagicalNumber(self, n: int, a: int, b: int) -> int:
        lcm = a * b // math.gcd(a, b)
        l = min(a, b)
        r = min(a, b) * n
        ans = bisect.bisect_left(range(l, r), n,
                                   key=lambda m: m // a + m // b - m // lcm) + 1
        return ans % (10**9 + 7)

```

3050. 88.py

Path: LeetCode-main/solutions/88. Merge Sorted Array/88.py

```
class Solution:
    def merge(self, nums1: list[int], m: int, nums2: list[int], n: int) -> None:
        i = m - 1 # nums1's index (the actual nums)
        j = n - 1 # nums2's index
        k = m + n - 1 # nums1's index (the next filled position)

        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                k -= 1
                i -= 1
            else:
                nums1[k] = nums2[j]
                k -= 1
                j -= 1
```

3051. 880.py

Path: LeetCode-main/solutions/880. Decoded String at Index/880.py

```
class Solution:
    def decodeAtIndex(self, s: str, k: int) -> str:
        size = 0

        for c in s:
            if c.isdigit():
                size *= int(c)
            else:
                size += 1

        for c in reversed(s):
            k %= size
            if k == 0 and c.isalpha():
                return c
            if c.isdigit():
                size //= int(c)
            else:
                size -= 1
```

3052. 881.py

Path: LeetCode-main/solutions/881. Boats to Save People/881.py

```
class Solution:
    def numRescueBoats(self, people: list[int], limit: int) -> int:
        ans = 0
        i = 0
        j = len(people) - 1

        people.sort()

        while i <= j:
            remain = limit - people[j]
            j -= 1
            if people[i] <= remain:
                i += 1
            ans += 1

        return ans
```

3053. 882.py

Path: LeetCode-main/solutions/882. Reachable Nodes In Subdivided Graph/882.py

```
class Solution:
    def reachableNodes(
        self,
        edges: list[list[int]],
        maxMoves: int,
        n: int,
    ) -> int:
        graph = [[] for _ in range(n)]
        dist = [maxMoves + 1] * n

        for u, v, cnt in edges:
            graph[u].append((v, cnt))
            graph[v].append((u, cnt))

        reachableNodes = self._dijkstra(graph, 0, maxMoves, dist)
        reachableSubnodes = 0

        for u, v, cnt in edges:
            # the number of reachable nodes of (u, v) from `u`
            a = 0 if dist[u] > maxMoves else min(maxMoves - dist[u], cnt)
            # the number of reachable nodes of (u, v) from `v`
            b = 0 if dist[v] > maxMoves else min(maxMoves - dist[v], cnt)
            reachableSubnodes += min(a + b, cnt)

        return reachableNodes + reachableSubnodes

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        maxMoves: int,
        dist: list[int],
    ) -> int:
        dist[src] = 0
        minHeap = [(dist[src], src)]  # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            # Already took `maxMoves` to reach `u`, so can't explore anymore.
            if dist[u] >= maxMoves:
                break
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                newDist = d + w + 1
                if newDist < dist[v]:
                    dist[v] = newDist
                    heapq.heappush(minHeap, (newDist, v))

        return sum(d <= maxMoves for d in dist)
```

3054. 883.py

Path: LeetCode-main/solutions/883. Projection Area of 3D Shapes/883.py

```
class Solution:
    def projectionArea(self, grid: list[list[int]]) -> int:
        return sum(
            a > 0 for row in grid for a in row) + sum(
                max(row) for row in grid) + sum(
                    max(col) for col in zip(*grid))
```

3055. 884.py

Path: LeetCode-main/solutions/884. Uncommon Words from Two Sentences/884.py

```
class Solution:
    def uncommonFromSentences(self, A: str, B: str) -> list[str]:
        count = collections.Counter((A + ' ' + B).split())
        return [word for word, freq in count.items() if freq == 1]
```

3056. 885.py

Path: LeetCode-main/solutions/885. Spiral Matrix III/885.py

```
class Solution:
    def spiralMatrixIII(self, rows: int, cols: int, rStart: int, cStart: int) -> list[list[int]]:
        dx = [1, 0, -1, 0]
        dy = [0, 1, 0, -1]
        ans = [[rStart, cStart]]
        i = 0

        while len(ans) < rows * cols:
            for _ in range(i // 2 + 1):
                rStart += dy[i % 4]
                cStart += dx[i % 4]
                if 0 <= rStart < rows and 0 <= cStart < cols:
                    ans.append([rStart, cStart])
            i += 1

        return ans
```

3057. 886.py

Path: LeetCode-main/solutions/886. Possible Bipartition/886.py

```

from enum import Enum

class Color(Enum):
    WHITE = 0
    RED = 1
    GREEN = 2

class Solution:
    def possibleBipartition(self, n: int, dislikes: list[list[int]]) -> bool:
        graph = [[] for _ in range(n + 1)]
        colors = [Color.WHITE] * (n + 1)

        for u, v in dislikes:
            graph[u].append(v)
            graph[v].append(u)

        # Reduce to 785. Is Graph Bipartite?
        def isValidColor(u: int, color: Color) -> bool:
            # Always paint red for a white node.
            if colors[u] != Color.WHITE:
                return colors[u] == color

            colors[u] = color # Always paint the node with `color`.

            # All the children should have valid colors.
            childrenColor = Color.RED if colors[u] == Color.GREEN else Color.GREEN
            return all(isValidColor(v, childrenColor) for v in graph[u])

        return all(colors[i] != Color.WHITE or isValidColor(i, Color.RED)
                    for i in range(1, n + 1))

```

3058. 887-3.py

Path: LeetCode-main/solutions/887. Super Egg Drop/887-3.py

```

class Solution:
    def superEggDrop(self, k: int, n: int) -> int:
        moves = 0
        dp = [[0] * (k + 1) for _ in range(n + 1)]

        while dp[moves][k] < n:
            moves += 1
            for eggs in range(1, k + 1):
                dp[moves][eggs] = (dp[moves - 1][eggs - 1] +
                                   dp[moves - 1][eggs] + 1)

        return moves

```

3059. 888.py

Path: LeetCode-main/solutions/888. Fair Candy Swap/888.py

```

class Solution:
    def fairCandySwap(
        self,
        aliceSizes: list[int],
        bobSizes: list[int],
    ) -> list[int]:
        diff = (sum(aliceSizes) - sum(bobSizes)) // 2
        bobSizesSet = set(bobSizes)

        for aliceSize in aliceSizes:
            target = aliceSize - diff
            if target in bobSizesSet:
                return [aliceSize, target]

```

3060. 889.py

Path: LeetCode-main/solutions/889. Construct Binary Tree from Preorder and Postorder Traversal/889.py

```

class Solution:
    def constructFromPrePost(
        self,
        pre: list[int],
        post: list[int],
    ) -> TreeNode | None:
        postToIndex = {num: i for i, num in enumerate(post)}

        def build(preStart: int, preEnd: int, postStart: int, postEnd: int) -> TreeNode | None:
            if preStart > preEnd:
                return None
            if preStart == preEnd:
                return TreeNode(pre[preStart])

            rootVal = pre[preStart]
            leftRootVal = pre[preStart + 1]
            leftRootPostIndex = postToIndex[leftRootVal]
            leftSize = leftRootPostIndex - postStart + 1

            root = TreeNode(rootVal)
            root.left = build(preStart + 1, preStart + leftSize,
                             postStart, leftRootPostIndex)
            root.right = build(preStart + leftSize + 1, preEnd,
                               leftRootPostIndex + 1, postEnd - 1)

            return root

        return build(0, len(pre) - 1, 0, len(post) - 1)

```

3061. 89.py

Path: LeetCode-main/solutions/89. Gray Code/89.py

```

class Solution:
    def grayCode(self, n: int) -> list[int]:
        ans = [0]

        for i in range(n):
            for j in reversed(range(len(ans))):
                ans.append(ans[j] | 1 << i)

        return ans

```

3062. 890.py

Path: LeetCode-main/solutions/890. Find and Replace Pattern/890.py

```
class Solution:
    def findAndReplacePattern(self, words: list[str], pattern: str) -> list[str]:
        def isIsomorphic(w: str, p: str) -> bool:
            return [*map(w.index, w)] == [*map(p.index, p)]
        return [word for word in words if isIsomorphic(word, pattern)]
```

3063. 891.py

Path: LeetCode-main/solutions/891. Sum of Subsequence Widths/891.py

```
class Solution:
    def sumSubseqWidths(self, nums: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(nums)
        ans = 0
        exp = 1

        nums.sort()

        for i in range(n):
            ans += (nums[i] - nums[n - 1 - i]) * exp
            ans %= MOD
            exp = exp * 2 % MOD

        return ans
```

3064. 892.py

Path: LeetCode-main/solutions/892. Surface Area of 3D Shapes/892.py

```
class Solution:
    def surfaceArea(self, grid: list[list[int]]) -> int:
        ans = 0

        for i in range(len(grid)):
            for j in range(len(grid)):
                if grid[i][j]:
                    ans += grid[i][j] * 4 + 2
                if i > 0:
                    ans -= min(grid[i][j], grid[i - 1][j]) * 2
                if j > 0:
                    ans -= min(grid[i][j], grid[i][j - 1]) * 2

        return ans
```

3065. 893.py

Path: LeetCode-main/solutions/893. Groups of Special-Equivalent Strings/893.py

```
class Solution:
    def numSpecialEquivGroups(self, words: list[str]) -> int:
        return len({''.join(sorted(word[:2])) + ''.join(sorted(word[1::2]))
                     for word in words})
```

3066. 894.py

Path: LeetCode-main/solutions/894. All Possible Full Binary Trees/894.py

```
class Solution:
    @functools.lru_cache(None)
    def allPossibleFBT(self, n: int) -> list[TreeNode | None]:
        if n % 2 == 0:
            return []
        if n == 1:
            return [TreeNode(0)]

        ans = []

        for leftCount in range(n):
            rightCount = n - 1 - leftCount
            for left in self.allPossibleFBT(leftCount):
                for right in self.allPossibleFBT(rightCount):
                    ans.append(TreeNode(0))
                    ans[-1].left = left
                    ans[-1].right = right

        return ans
```

3067. 895.py

Path: LeetCode-main/solutions/895. Maximum Frequency Stack/895.py

```
class FreqStack:
    def __init__(self):
        self.maxFreq = 0
        self.count = collections.Counter()
        self.countToStack = collections.defaultdict(list)

    def push(self, val: int) -> None:
        self.count[val] += 1
        self.countToStack[self.count[val]].append(val)
        self.maxFreq = max(self.maxFreq, self.count[val])

    def pop(self) -> int:
        val = self.countToStack[self.maxFreq].pop()
        self.count[val] -= 1
        if not self.countToStack[self.maxFreq]:
            self.maxFreq -= 1
        return val
```

3068. 896.py

Path: LeetCode-main/solutions/896. Monotonic Array/896.py

```
class Solution:
    def isMonotonic(self, nums: list[int]) -> bool:
        increasing = True
        decreasing = True

        for i in range(1, len(nums)):
            increasing &= nums[i - 1] <= nums[i]
            decreasing &= nums[i - 1] >= nums[i]

        return increasing or decreasing
```

3069. 897.py

Path: LeetCode-main/solutions/897. Increasing Order Search Tree/897.py

```
class Solution:
    def increasingBST(self, root: TreeNode, tail: TreeNode = None) -> TreeNode:
        if not root:
            return tail

        res = self.increasingBST(root.left, root)
        root.left = None
        root.right = self.increasingBST(root.right, tail)
        return res
```

3070. 899.py

Path: LeetCode-main/solutions/899. Orderly Queue/899.py

```
class Solution:
    def orderlyQueue(self, s: str, k: int) -> str:
        return (''.join(sorted(s)) if k > 1
                else min(s[i:] + s[:i] for i in range(len(s))))
```

3071. 9.py

Path: LeetCode-main/solutions/9. Palindrome Number/9.py

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0:
            return False

        rev = 0
        y = x

        while y:
            rev = rev * 10 + y % 10
            y //= 10

        return rev == x
```

3072. 90.py

Path: LeetCode-main/solutions/90. Subsets II/90.py

```
class Solution:
    def subsetsWithDup(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            ans.append(path)
            if s == len(nums):
                return

            for i in range(s, len(nums)):
                if i > s and nums[i] == nums[i - 1]:
                    continue
                dfs(i + 1, path + [nums[i]])

        nums.sort()
        dfs(0, [])
        return ans
```

3073. 900.py

Path: LeetCode-main/solutions/900. RLE Iterator/900.py

```
class RLEIterator:
    def __init__(self, encoding: list[int]):
        self.encoding = encoding
        self.index = 0

    def next(self, n: int) -> int:
        while self.index < len(self.encoding) and self.encoding[self.index] < n:
            n -= self.encoding[self.index]
            self.index += 2

        if self.index == len(self.encoding):
            return -1

        self.encoding[self.index] -= n
        return self.encoding[self.index + 1]
```

3074. 901.py

Path: LeetCode-main/solutions/901. Online Stock Span/901.py

```
class StockSpanner:
    def __init__(self):
        self.stack = [] # (price, span)

    def next(self, price: int) -> int:
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]
        self.stack.append((price, span))
        return span
```

3075. 902.py

Path: LeetCode-main/solutions/902. Numbers At Most N Given Digit Set/902.py

```
class Solution:
    def atMostNGivenDigitSet(self, digits: list[str], n: int) -> int:
        ans = 0
        num = str(n)

        for i in range(1, len(num)):
            ans += pow(len(digits), i)

        for i, c in enumerate(num):
            dHasSameNum = False
            for digit in digits:
                if digit[0] < c:
                    ans += pow(len(digits), len(num) - i - 1)
                elif digit[0] == c:
                    dHasSameNum = True
            if not dHasSameNum:
                return ans

        return ans + 1
```

3076. 904.py

Path: LeetCode-main/solutions/904. Fruit Into Baskets/904.py

```
class Solution:
    def totalFruit(self, fruits: list[int]) -> int:
        ans = 0
        count = collections.defaultdict(int)

        l = 0
        for r, fruit in enumerate(fruits):
            count[fruit] += 1
            while len(count) > 2:
                count[fruits[l]] -= 1
                if count[fruits[l]] == 0:
                    del count[fruits[l]]
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

3077. 905.py

Path: LeetCode-main/solutions/905. Sort Array By Parity/905.py

```

class Solution:
    def sortArrayByParity(self, nums: list[int]) -> list[int]:
        l = 0
        r = len(nums) - 1

        while l < r:
            if nums[l] % 2 == 1 and nums[r] % 2 == 0:
                nums[l], nums[r] = nums[r], nums[l]
            if nums[l] % 2 == 0:
                l += 1
            if nums[r] % 2 == 1:
                r -= 1

        return nums

```

3078. 906.py

Path: LeetCode-main/solutions/906. Super Palindromes/906.py

```

class Solution:
    def superpalindromesInRange(self, left: str, right: str) -> int:
        def nextPalindrome(num: int) -> int:
            s = str(num)
            n = len(s)

            half = s[0:(n + 1) // 2]
            reversedHalf = half[:n // 2][::-1]
            candidate = int(half + reversedHalf)
            if candidate >= num:
                return candidate

            half = str(int(half) + 1)
            reversedHalf = half[:n // 2][::-1]
            return int(half + reversedHalf)

        def isPalindrome(num: int) -> bool:
            s = str(num)
            l = 0
            r = len(s) - 1

            while l < r:
                if s[l] != s[r]:
                    return False
                l += 1
                r -= 1

            return True

        ans = 0
        l = int(left)
        r = int(right)
        i = math.isqrt(l)

        while i * i <= r:
            palindrome = nextPalindrome(i)
            squared = palindrome**2
            if squared <= r and isPalindrome(squared):
                ans += 1
            i = palindrome + 1

        return ans

```

3079. 907.py

Path: LeetCode-main/solutions/907. Sum of Subarray Minimums/907.py

```
class Solution:
    def sumSubarrayMins(self, arr: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(arr)
        ans = 0
        # prevMin[i] := index k s.t. arr[k] is the previous minimum in arr[:i]
        prevMin = [-1] * n
        # nextMin[i] := index k s.t. arr[k] is the next minimum in arr[i + 1:]
        nextMin = [n] * n
        stack = []

        for i, a in enumerate(arr):
            while stack and arr[stack[-1]] > a:
                index = stack.pop()
                nextMin[index] = i
            if stack:
                prevMin[i] = stack[-1]
            stack.append(i)

        for i, a in enumerate(arr):
            ans += a * (i - prevMin[i]) * (nextMin[i] - i)
            ans %= MOD

        return ans
```

3080. 908.py

Path: LeetCode-main/solutions/908. Smallest Range I/908.py

```
class Solution:
    def smallestRangeI(self, nums: list[int], k: int) -> int:
        return max(0, max(nums) - min(nums) - 2 * k)
```

3081. 909.py

Path: LeetCode-main/solutions/909. Snakes and Ladders/909.py

```

class Solution:
    def snakesAndLadders(self, board: list[list[int]]) -> int:
        n = len(board)
        q = collections.deque([1])
        seen = set()
        arr = [0] * (1 + n * n) # 2D -> 1D

        for i in range(n):
            for j in range(n):
                arr[(n - 1 - i) * n + (n - j if (n - i) % 2 == 0 else j + 1)] = board[i][j]

        step = 1
        while q:
            for _ in range(len(q)):
                curr = q.popleft()
                for next in range(curr + 1, min(curr + 6, n * n) + 1):
                    dest = arr[next] if arr[next] > 0 else next
                    if dest == n * n:
                        return step
                    if dest in seen:
                        continue
                    q.append(dest)
                    seen.add(dest)
                step += 1

        return -1

```

3082. 91.py

Path: LeetCode-main/solutions/91. Decode Ways/91.py

```

class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # dp[i] := the number of ways to decode s[i..n)
        dp = [0] * n + [1]

        def isValid(a: str, b=None) -> bool:
            if b:
                return a == '1' or a == '2' and b < '7'
            return a != '0'

        if isValid(s[-1]):
            dp[n - 1] = 1

        for i in reversed(range(n - 1)):
            if isValid(s[i]):
                dp[i] += dp[i + 1]
            if isValid(s[i], s[i + 1]):
                dp[i] += dp[i + 2]

        return dp[0]

```

3083. 910.py

Path: LeetCode-main/solutions/910. Smallest Range II/910.py

```
class Solution:
    def smallestRangeII(self, nums: list[int], k: int) -> int:
        nums.sort()

        ans = nums[-1] - nums[0]
        left = nums[0] + k
        right = nums[-1] - k

        for a, b in itertools.pairwise(nums):
            mn = min(left, b - k)
            mx = max(right, a + k)
            ans = min(ans, mx - mn)

        return ans
```

3084. 911.py

Path: LeetCode-main/solutions/911. Online Election/911.py

```
class TopVotedCandidate:
    def __init__(self, persons: list[int], times: list[int]):
        self.times = times
        self.timeToLead = {}
        count = collections.Counter() # {person: voted}
        lead = -1

        for person, time in zip(persons, times):
            count[person] += 1
            if count[person] >= count[lead]:
                lead = person
            self.timeToLead[time] = lead

    def q(self, t: int) -> int:
        i = bisect_right(self.times, t)
        return self.timeToLead[self.times[i - 1]]
```

3085. 912-2.py

Path: LeetCode-main/solutions/912. Sort an Array/912-2.py

```

class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._heapSort(nums)
        return nums

    def _heapSort(self, nums: list[int]) -> None:
        def maxHeapify(nums: list[int], i: int, heapSize: int) -> None:
            l = 2 * i + 1
            r = 2 * i + 2
            largest = i
            if l < heapSize and nums[largest] < nums[l]:
                largest = l
            if r < heapSize and nums[largest] < nums[r]:
                largest = r
            if largest != i:
                nums[largest], nums[i] = nums[i], nums[largest]
                maxHeapify(nums, largest, heapSize)

        def buildMaxHeap(nums: list[int]) -> None:
            for i in range(len(nums) // 2, -1, -1):
                maxHeapify(nums, i, len(nums))

        buildMaxHeap(nums)
        heapSize = len(nums)
        for i in reversed(range(1, len(nums))):
            nums[i], nums[0] = nums[0], nums[i]
            heapSize -= 1
            maxHeapify(nums, 0, heapSize)

```

3086. 912-3.py

Path: LeetCode-main/solutions/912. Sort an Array/912-3.py

```

class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._quickSort(nums, 0, len(nums) - 1)
        return nums

    def _quickSort(self, nums: list[int], l: int, r: int) -> None:
        if l >= r:
            return

        def partition(nums: list[int], l: int, r: int) -> int:
            randIndex = random.randint(0, r - l) + l
            nums[randIndex], nums[r] = nums[r], nums[randIndex]
            pivot = nums[r]
            nextSwapped = l
            for i in range(l, r):
                if nums[i] <= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1
            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]
            return nextSwapped

        m = partition(nums, l, r)
        self._quickSort(nums, l, m - 1)
        self._quickSort(nums, m + 1, r)

```

3087. 912.py

Path: LeetCode-main/solutions/912. Sort an Array/912.py

```
class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._mergeSort(nums, 0, len(nums) - 1)
        return nums

    def _mergeSort(self, nums: list[int], l: int, r: int) -> None:
        if l >= r:
            return

        def merge(nums: list[int], l: int, m: int, r: int) -> None:
            sorted = [0] * (r - l + 1)
            k = 0 # sorted's index
            i = l # left's index
            j = m + 1 # right's index

            while i <= m and j <= r:
                if nums[i] < nums[j]:
                    sorted[k] = nums[i]
                    k += 1
                    i += 1
                else:
                    sorted[k] = nums[j]
                    k += 1
                    j += 1

            # Put the possible remaining left part into the sorted array.
            while i <= m:
                sorted[k] = nums[i]
                k += 1
                i += 1

            # Put the possible remaining right part into the sorted array.
            while j <= r:
                sorted[k] = nums[j]
                k += 1
                j += 1

            nums[l:l + len(sorted)] = sorted

        m = (l + r) // 2
        self._mergeSort(nums, l, m)
        self._mergeSort(nums, m + 1, r)
        merge(nums, l, m, r)
```

3088. 913.py

Path: LeetCode-main/solutions/913. Cat and Mouse/913.py

```

from enum import IntEnum

class State(IntEnum):
    DRAW = 0
    MOUSE_WIN = 1
    CAT_WIN = 2

class Solution:
    def catMouseGame(self, graph: list[list[int]]) -> int:
        n = len(graph)
        # result of (cat, mouse, move)
        # move := 0 (mouse) // 1 (cat)
        states = [[[0] * 2 for _ in range(n)] for _ in range(n)]
        outDegree = [[[0] * 2 for _ in range(n)] for _ in range(n)]
        q = collections.deque() # (cat, mouse, move, state)

        for cat in range(n):
            for mouse in range(n):
                outDegree[cat][mouse][0] = len(graph[mouse])
                outDegree[cat][mouse][1] = len(graph[cat]) - graph[cat].count(0)

        # Start from the states s.t. the winner can be determined.
        for cat in range(1, n):
            for move in range(2):
                # Mouse is in the hole.
                states[cat][0][move] = int(State.MOUSE_WIN)
                q.append((cat, 0, move, int(State.MOUSE_WIN)))
                # Cat catches mouse.
                states[cat][cat][move] = int(State.CAT_WIN)
                q.append((cat, cat, move, int(State.CAT_WIN)))

        while q:
            cat, mouse, move, state = q.popleft()
            if cat == 2 and mouse == 1 and move == 0:
                return state
            prevMove = move ^ 1
            for prev in graph[cat if prevMove else mouse]:
                prevCat = prev if prevMove else cat
                if prevCat == 0: # invalid
                    continue
                prevMouse = mouse if prevMove else prev
                # The state has been determined.
                if states[prevCat][prevMouse][prevMove]:
                    continue
                if (prevMove == 0 and state == int(State.MOUSE_WIN) or
                    prevMove == 1 and state == int(State.CAT_WIN)):
                    states[prevCat][prevMouse][prevMove] = state
                    q.append((prevCat, prevMouse, prevMove, state))
                else:
                    outDegree[prevCat][prevMouse][prevMove] -= 1
                    if outDegree[prevCat][prevMouse][prevMove] == 0:
                        states[prevCat][prevMouse][prevMove] = state
                        q.append((prevCat, prevMouse, prevMove, state))

        return states[2][1][0]

```

3089. 914.py

Path: LeetCode-main/solutions/914. X of a Kind in a Deck of Cards/914.py


```
class Solution:
    def hasGroupsSizeX(self, deck: list[int]) -> bool:
        count = collections.Counter(deck)
        return functools.reduce(math.gcd, count.values()) >= 2
```

3090. 915.py

Path: LeetCode-main/solutions/915. Partition Array into Disjoint Intervals/915.py

```
class Solution:
    def partitionDisjoint(self, nums: list[int]) -> int:
        n = len(nums)
        mn = [0] * (n - 1) + [nums[-1]]
        mx = -math.inf

        for i in range(n - 2, -1, -1):
            mn[i] = min(mn[i + 1], nums[i])

        for i, num in enumerate(nums):
            mx = max(mx, num)
            if mx <= mn[i + 1]:
                return i + 1
```

3091. 916.py

Path: LeetCode-main/solutions/916. Word Subsets/916.py

```
class Solution:
    def wordSubsets(self, A: list[str], B: list[str]) -> list[str]:
        count = collections.Counter()

        for b in B:
            count = count | collections.Counter(b)

        return [a for a in A if collections.Counter(a) & count == count]
```

3092. 917.py

Path: LeetCode-main/solutions/917. Reverse Only Letters/917.py

```
class Solution:
    def reverseOnlyLetters(self, s: str) -> str:
        ans = list(s)
        i = 0
        j = len(s) - 1

        while i < j:
            while i < j and not s[i].isalpha():
                i += 1
            while i < j and not s[j].isalpha():
                j -= 1
            ans[i], ans[j] = ans[j], ans[i]
            i += 1
            j -= 1

        return ''.join(ans)
```

3093. 918.py

Path: LeetCode-main/solutions/918. Maximum Sum Circular Subarray/918.py

```
class Solution:
    def maxSubarraySumCircular(self, nums: list[int]) -> int:
        totalSum = 0
        currMaxSum = 0
        currMinSum = 0
        maxSum = -math.inf
        minSum = math.inf

        for num in nums:
            totalSum += num
            currMaxSum = max(currMaxSum + num, num)
            currMinSum = min(currMinSum + num, num)
            maxSum = max(maxSum, currMaxSum)
            minSum = min(minSum, currMinSum)

        return maxSum if maxSum < 0 else max(maxSum, totalSum - minSum)
```

3094. 919.py

Path: LeetCode-main/solutions/919. Complete Binary Tree Inserter/919.py

```
class CBTInserter:
    def __init__(self, root: TreeNode | None):
        self.tree = [root]
        for node in self.tree:
            if node.left:
                self.tree.append(node.left)
            if node.right:
                self.tree.append(node.right)

    def insert(self, v: int) -> int:
        n = len(self.tree)
        self.tree.append(TreeNode(v))
        parent = self.tree[(n - 1) // 2]
        if n % 2 == 1:
            parent.left = self.tree[-1]
        else:
            parent.right = self.tree[-1]
        return parent.val

    def get_root(self) -> TreeNode | None:
        return self.tree[0]
```

3095. 92-2.py

Path: LeetCode-main/solutions/92. Reverse Linked List II/92-2.py

```

class Solution:
    def reverseBetween(self, head: ListNode, m: int, n: int) -> ListNode:
        if not head and m == n:
            return head

        dummy = ListNode(0, head)
        prev = dummy

        for _ in range(m - 1):
            prev = prev.next # Point to the node before the sublist [m, n].

        tail = prev.next # Be the tail of the sublist [m, n].

        # Reverse the sublist [m, n] one by one.
        for _ in range(n - m):
            cache = tail.next
            tail.next = cache.next
            cache.next = prev.next
            prev.next = cache

        return dummy.next

```

3096. 92.py

Path: LeetCode-main/solutions/92. Reverse Linked List II/92.py

```

class Solution:
    def reverseBetween(
        self,
        head: ListNode | None,
        left: int,
        right: int,
    ) -> ListNode | None:
        if left == 1:
            return self.reverseN(head, right)

        head.next = self.reverseBetween(head.next, left - 1, right - 1)
        return head

    def reverseN(self, head: ListNode | None, n: int) -> ListNode | None:
        if n == 1:
            return head

        newHead = self.reverseN(head.next, n - 1)
        headNext = head.next
        head.next = headNext.next
        headNext.next = head
        return newHead

```

3097. 921.py

Path: LeetCode-main/solutions/921. Minimum Add to Make Parentheses Valid/921.py

```

class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        l = 0
        r = 0

        for c in s:
            if c == '(':
                l += 1
            else:
                if l == 0:
                    r += 1
                else:
                    l -= 1

        return l + r

```

3098. 922.py

Path: LeetCode-main/solutions/922. Sort Array By Parity II/922.py

```

class Solution:
    def sortArrayByParityII(self, nums: list[int]) -> list[int]:
        n = len(nums)

        i = 0
        j = 1
        while i < n:
            while i < n and nums[i] % 2 == 0:
                i += 2
            while j < n and nums[j] % 2 == 1:
                j += 2
            if i < n:
                nums[i], nums[j] = nums[j], nums[i]

        return nums

```

3099. 923.py

Path: LeetCode-main/solutions/923. 3Sum With Multiplicity/923.py

```

class Solution:
    def threeSumMulti(self, arr: list[int], target: int) -> int:
        MOD = 1_000_000_007
        ans = 0
        count = collections.Counter(arr)

        for i, x in count.items():
            for j, y in count.items():
                k = target - i - j
                if k not in count:
                    continue
                if i == j and j == k:
                    ans = (ans + x * (x - 1) * (x - 2) // 6) % MOD
                elif i == j and j != k:
                    ans = (ans + x * (x - 1) // 2 * count[k]) % MOD
                elif i < j and j < k:
                    ans = (ans + x * y * count[k]) % MOD

        return ans % MOD

```

3100. 925.py

Path: LeetCode-main/solutions/925. Long Pressed Name/925.py

```
class Solution:
    def isLongPressedName(self, name: str, typed: str) -> bool:
        i = 0

        for j, t in enumerate(typed):
            if i < len(name) and name[i] == t:
                i += 1
            elif j == 0 or t != typed[j - 1]:
                return False

        return i == len(name)
```

3101. 926.py

Path: LeetCode-main/solutions/926. Flip String to Monotone Increasing/926.py

```
class Solution:
    def minFlipsMonoIncr(self, s: str) -> int:
        # the number of characters to be flipped to make the substring so far
        # monotone increasing
        dp = 0
        count1 = 0

        for c in s:
            if c == '0':
                # 1. Flip '0'.
                # 2. Keep '0' and flip all the previous 1s.
                dp = min(dp + 1, count1)
            else:
                count1 += 1

        return dp
```

3102. 927.py

Path: LeetCode-main/solutions/927. Three Equal Parts/927.py

```

class Solution:
    def threeEqualParts(self, arr: list[int]) -> list[int]:
        ones = sum(a == 1 for a in arr)

        if ones == 0:
            return [0, len(arr) - 1]
        if ones % 3 != 0:
            return [-1, -1]

        k = ones // 3
        i = 0

        for i in range(len(arr)):
            if arr[i] == 1:
                first = i
                break

        gapOnes = k

        for j in range(i + 1, len(arr)):
            if arr[j] == 1:
                gapOnes -= 1
                if gapOnes == 0:
                    second = j
                    break

        gapOnes = k

        for i in range(j + 1, len(arr)):
            if arr[i] == 1:
                gapOnes -= 1
                if gapOnes == 0:
                    third = i
                    break

        while third < len(arr) and arr[first] == arr[second] == arr[third]:
            first += 1
            second += 1
            third += 1

        if third == len(arr):
            return [first - 1, second]
        return [-1, -1]

```

3103. 929.py

Path: LeetCode-main/solutions/929. Unique Email Addresses/929.py

```

class Solution:
    def numUniqueEmails(self, emails: list[str]) -> int:
        seen = set()

        for email in emails:
            local, domain = email.split('@')
            local = local.split('+')[0].replace('.', '')
            seen.add(local + '@' + domain)

        return len(seen)

```

3104. 93.py

Path: LeetCode-main/solutions/93. Restore IP Addresses/93.py

```
class Solution:
    def restoreIpAddresses(self, s: str) -> list[str]:
        ans = []

        def dfs(start: int, path: list[int]) -> None:
            if len(path) == 4 and start == len(s):
                ans.append(path[0] + '.' + path[1] + '.' + path[2] + '.' + path[3])
                return
            if len(path) == 4 or start == len(s):
                return

            for length in range(1, 4):
                if start + length > len(s):
                    return # out-of-bounds
                if length > 1 and s[start] == '0':
                    return # leading '0'
                num = s[start: start + length]
                if int(num) > 255:
                    return
                dfs(start + length, path + [num])

        dfs(0, [])
        return ans
```

3105. 930-2.py

Path: LeetCode-main/solutions/930. Binary Subarrays With Sum/930-2.py

```
class Solution:
    def numSubarraysWithSum(self, nums: list[int], goal: int) -> int:
        def numSubarraysWithSumAtMost(goal: int) -> int:
            res = 0
            count = 0
            l = 0
            r = 0

            while r < len(nums):
                count += nums[r]
                r += 1
                while l < r and count > goal:
                    count -= nums[l]
                    l += 1
                # nums[l..r), nums[l + 1..r), ..., nums[r - 1]
                res += r - l

            return res

        return numSubarraysWithSumAtMost(goal) - numSubarraysWithSumAtMost(goal - 1)
```

3106. 930.py

Path: LeetCode-main/solutions/930. Binary Subarrays With Sum/930.py

```

class Solution:
    def numSubarraysWithSum(self, nums: list[int], goal: int) -> int:
        ans = 0
        prefix = 0
        # {prefix: number of occurrence}
        count = collections.Counter({0: 1})

        for num in nums:
            prefix += num
            ans += count[prefix - goal]
            count[prefix] += 1

        return ans

```

3107. 931.py

Path: LeetCode-main/solutions/931. Minimum Falling Path Sum/931.py

```

class Solution:
    def minFallingPathSum(self, A: list[list[int]]) -> int:
        n = len(A)

        for i in range(1, n):
            for j in range(n):
                mn = math.inf
                for k in range(max(0, j - 1), min(n, j + 2)):
                    mn = min(mn, A[i - 1][k])
                A[i][j] += mn

        return min(A[-1])

```

3108. 932.py

Path: LeetCode-main/solutions/932. Beautiful Array/932.py

```

class Solution:
    def beautifulArray(self, n: int) -> list[int]:
        arr = [i for i in range(1, n + 1)]

        def partition(l: int, r: int, mask: int) -> int:
            nextSwapped = 1
            for i in range(l, r + 1):
                if arr[i] & mask:
                    arr[i], arr[nextSwapped] = arr[nextSwapped], arr[i]
                    nextSwapped += 1
            return nextSwapped - 1

        def divide(l: int, r: int, mask: int) -> None:
            if l >= r:
                return
            m = partition(l, r, mask)
            divide(l, m, mask << 1)
            divide(m + 1, r, mask << 1)

        divide(0, n - 1, 1)
        return arr

```


3109. 933.py

Path: LeetCode-main/solutions/933. Number of Recent Calls/933.py

```
class RecentCounter:
    def __init__(self):
        self.q = collections.deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

3110. 935.py

Path: LeetCode-main/solutions/935. Knight Dialer/935.py

```
class Solution:
    def knightDialer(self, n: int) -> int:
        DIRS = ((1, 2), (2, 1), (2, -1), (1, -2),
                (-1, -2), (-2, -1), (-2, 1), (-1, 2))
        MOD = 1_000_000_007

        # dp[i][j] := the number of ways stand on (i, j)
        dp = [[1] * 3 for _ in range(4)]
        dp[3][0] = dp[3][2] = 0

        for _ in range(n - 1):
            newDp = [[0] * 3 for _ in range(4)]
            for i in range(4):
                for j in range(3):
                    if (i, j) in ((3, 0), (3, 2)):
                        continue
                    for dx, dy in DIRS:
                        x = i + dx
                        y = j + dy
                        if x < 0 or x >= 4 or y < 0 or y >= 3:
                            continue
                        if (x, y) in ((3, 0), (3, 2)):
                            continue
                        newDp[x][y] = (newDp[x][y] + dp[i][j]) % MOD
            dp = newDp

        return sum(map(sum, dp)) % MOD
```

3111. 936.py

Path: LeetCode-main/solutions/936. Stamping The Sequence/936.py

```

class Solution:
    def movesToStamp(self, stamp: str, target: str) -> list[int]:
        def stampify(s: int) -> int:
            """
            Stamps target[i..i + |stamp|) and returns the number of newly stamped
            characters.
            e.g. stampify("abc", "ababc", 2) returns 3 because target becomes "ab***".
            """
            stampified = len(stamp)

            for i, st in enumerate(stamp):
                if target[s + i] == '*': # It's already been stamped.
                    stampified -= 1
                elif target[s + i] != st: # We can't stamp on the index i.
                    return 0

            for i in range(s, s + len(stamp)):
                target[i] = '*'

            return stampified

        ans = []
        target = list(target)
        # stamped[i] := True if we already stamped target by stamping on index i
        stamped = [False] * len(target)
        stampedCount = 0 # Our goal is to make stampedCount = |target|.

        while stampedCount < len(target):
            isStamped = False
            # Try to stamp target[i..i + |stamp|) for each index.
            for i in range(len(target) - len(stamp) + 1):
                if stamped[i]:
                    continue
                stampified = stampify(i)
                if stampified == 0:
                    continue
                stampedCount += stampified
                isStamped = True
                stamped[i] = True
                ans.append(i)
            # After trying to stamp on each index, we can't find a valid stamp.
            if not isStamped:
                return []

        return ans[::-1]

```

3112. 937.py

Path: LeetCode-main/solutions/937. Reorder Data in Log Files/937.py

```

class Solution:
    def reorderLogFiles(self, logs: list[str]) -> list[str]:
        digitLogs = []
        letterLogs = []

        for log in logs:
            i = log.index(' ')
            if log[i + 1].isdigit():
                digitLogs.append(log)
            else:
                letterLogs.append((log[:i], log[i + 1:]))

        letterLogs.sort(key=lambda x: (x[1], x[0]))
        return [identifier + ' ' + letters for identifier, letters in letterLogs] + digitLogs

```

3113. 939.py

Path: LeetCode-main/solutions/939. Minimum Area Rectangle/939.py

```

class Solution:
    def minAreaRect(self, points: list[list[int]]) -> int:
        ans = math.inf
        xToYs = collections.defaultdict(set)

        for x, y in points:
            xToYs[x].add(y)

        for i in range(len(points)):
            for j in range(i):
                x1, y1 = points[i]
                x2, y2 = points[j]
                if x1 == x2 or y1 == y2:
                    continue
                if y2 in xToYs[x1] and y1 in xToYs[x2]:
                    ans = min(ans, abs(x1 - x2) * abs(y1 - y2))

        return ans if ans < math.inf else 0

```

3114. 94.py

Path: LeetCode-main/solutions/94. Binary Tree Inorder Traversal/94.py

```

class Solution:
    def inorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []
        stack = []

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()
            ans.append(root.val)
            root = root.right

        return ans

```

3115. 940.py

Path: LeetCode-main/solutions/940. Distinct Subsequences II/940.py

```
class Solution:
    def distinctSubseqII(self, s: str) -> int:
        MOD = 1_000_000_007
        # endsIn[i] := the number of subsequence that end in ('a' + i)
        endsIn = [0] * 26

        for c in s:
            endsIn[ord(c) - ord('a')] = (sum(endsIn) + 1) % MOD

        return sum(endsIn) % MOD
```

3116. 941.py

Path: LeetCode-main/solutions/941. Valid Mountain Array/941.py

```
class Solution:
    def validMountainArray(self, arr: list[int]) -> bool:
        if len(arr) < 3:
            return False

        l = 0
        r = len(arr) - 1

        while l + 1 < len(arr) and arr[l] < arr[l + 1]:
            l += 1
        while r > 0 and arr[r] < arr[r - 1]:
            r -= 1

        return l > 0 and r < len(arr) - 1 and l == r
```

3117. 942.py

Path: LeetCode-main/solutions/942. DI String Match/942.py

```
class Solution:
    def diStringMatch(self, s: str) -> list[int]:
        ans = []
        mn = 0
        mx = len(s)

        for c in s:
            if c == 'I':
                ans.append(mn)
                mn += 1
            else:
                ans.append(mx)
                mx -= 1

        return ans + [mn]
```

3118. 944.py

Path: LeetCode-main/solutions/944. Delete Columns to Make Sorted/944.py

```
class Solution:
    def minDeletionSize(self, strs: list[str]) -> int:
        ans = 0

        for j in range(len(strs[0])):
            for i in range(len(strs) - 1):
                if strs[i][j] > strs[i + 1][j]:
                    ans += 1
                    break

        return ans
```

3119. 945.py

Path: LeetCode-main/solutions/945. Minimum Increment to Make Array Unique/945.py

```
class Solution:
    def minIncrementForUnique(self, nums: list[int]) -> int:
        ans = 0
        minAvailable = 0

        for num in sorted(nums):
            ans += max(minAvailable - num, 0)
            minAvailable = max(minAvailable, num) + 1

        return ans
```

3120. 946.py

Path: LeetCode-main/solutions/946. Validate Stack Sequences/946.py

```
class Solution:
    def validateStackSequences(
        self,
        pushed: list[int],
        popped: list[int],
    ) -> bool:
        stack = []
        i = 0 # popped's index

        for x in pushed:
            stack.append(x)
            while stack and stack[-1] == popped[i]:
                stack.pop()
                i += 1

        return not stack
```

3121. 948.py

Path: LeetCode-main/solutions/948. Bag of Tokens/948.py

```

class Solution:
    def bagOfTokensScore(self, tokens: list[int], power: int) -> int:
        ans = 0
        score = 0
        q = collections.deque(sorted(tokens))

        while q and (power >= q[0] or score):
            while q and power >= q[0]:
                # Play the smallest face up.
                power -= q.popleft()
                score += 1
            ans = max(ans, score)
            if q and score:
                # Play the largest face down.
                power += q.pop()
                score -= 1

        return ans

```

3122. 949.py

Path: LeetCode-main/solutions/949. Largest Time for Given Digits/949.py

```

class Solution:
    def largestTimeFromDigits(self, arr: list[int]) -> str:
        for time in itertools.permutations(sorted(arr, reverse=True)):
            if time[:2] < (2, 4) and time[2] < 6:
                return '%d%d:%d%d' % time
        return ''

```

3123. 95.py

Path: LeetCode-main/solutions/95. Unique Binary Search Trees II/95.py

```

class Solution:
    def generateTrees(self, n: int) -> list[TreeNode]:
        if n == 0:
            return []

        def generateTrees(mn: int, mx: int) -> list[int | None]:
            if mn > mx:
                return [None]

            ans = []

            for i in range(mn, mx + 1):
                for left in generateTrees(mn, i - 1):
                    for right in generateTrees(i + 1, mx):
                        ans.append(TreeNode(i))
                        ans[-1].left = left
                        ans[-1].right = right

            return ans

        return generateTrees(1, n)

```

3124. 950.py

Path: LeetCode-main/solutions/950. Reveal Cards In Increasing Order/950.py

```
class Solution:
    def deckRevealedIncreasing(self, deck: list[int]) -> list[int]:
        dq = collections.deque()

        for card in reversed(sorted(deck)):
            dq.rotate()
            dq.appendleft(card)

        return list(dq)
```

3125. 951.py

Path: LeetCode-main/solutions/951. Flip Equivalent Binary Trees/951.py

```
class Solution:
    def flipEquiv(self, root1: TreeNode | None, root2: TreeNode | None) -> bool:
        if not root1:
            return not root2
        if not root2:
            return not root1
        if root1.val != root2.val:
            return False
        return (self.flipEquiv(root1.left, root2.left) and
                self.flipEquiv(root1.right, root2.right) or
                self.flipEquiv(root1.left, root2.right) and
                self.flipEquiv(root1.right, root2.left))
```

3126. 952.py

Path: LeetCode-main/solutions/952. Largest Component Size by Common Factor/952.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def largestComponentSize(self, nums: list[int]) -> int:
        ans = 0
        uf = UnionFind(max(nums) + 1)
        count = collections.Counter()

        for num in nums:
            for x in range(2, math.isqrt(num) + 1):
                if num % x == 0:
                    uf.unionByRank(num, x)
                    uf.unionByRank(num, num // x)

        for num in nums:
            numRoot = uf.find(num)
            count[numRoot] += 1
            ans = max(ans, count[numRoot])

        return ans

```

3127. 953.py

Path: LeetCode-main/solutions/953. Verifying an Alien Dictionary/953.py

```

class Solution:
    def isAlienSorted(self, words: list[str], order: str) -> bool:
        dict = {c: i for i, c in enumerate(order)}
        words = [[dict[c] for c in word] for word in words]
        return all(w1 <= w2 for w1, w2 in zip(words, words[1:]))

```

3128. 954.py

Path: LeetCode-main/solutions/954. Array of Doubled Pairs/954.py


```
class Solution:
    def canReorderDoubled(self, arr: list[int]) -> bool:
        count = collections.Counter(arr)

        for key in sorted(count, key=abs):
            if count[key] > count[2 * key]:
                return False
            count[2 * key] -= count[key]

        return True
```

3129. 957.py

Path: LeetCode-main/solutions/957. Prison Cells After N Days/957.py

```
class Solution:
    def prisonAfterNDays(self, cells: list[int], n: int) -> list[int]:
        nextDayCells = [0] * len(cells)
        day = 0

        while n > 0:
            n -= 1
            for i in range(1, len(cells) - 1):
                nextDayCells[i] = 1 if cells[i - 1] == cells[i + 1] else 0
            if day == 0:
                firstDayCells = nextDayCells.copy()
            elif nextDayCells == firstDayCells:
                n %= day
                cells = nextDayCells.copy()
                day += 1

        return cells
```

3130. 96.py

Path: LeetCode-main/solutions/96. Unique Binary Search Trees/96.py

```
class Solution:
    def numTrees(self, n: int) -> int:
        # dp[i] := the number of unique BST's that store values 1..i
        dp = [1, 1] + [0] * (n - 1)

        for i in range(2, n + 1):
            for j in range(i):
                dp[i] += dp[j] * dp[i - j - 1]

        return dp[n]
```

3131. 961.py

Path: LeetCode-main/solutions/961. N-Repeated Element in Size 2N Array/961.py

```

class Solution:
    def repeatedNTimes(self, nums: list[int]) -> int:
        for i in range(len(nums) - 2):
            if nums[i] == nums[i + 1] or nums[i] == nums[i + 2]:
                return nums[i]
        return nums[-1]

```

3132. 962.py

Path: LeetCode-main/solutions/962. Maximum Width Ramp/962.py

```

class Solution:
    def maxWidthRamp(self, nums: list[int]) -> int:
        ans = 0
        stack = []

        for i, num in enumerate(nums):
            if stack == [] or num <= nums[stack[-1]]:
                stack.append(i)

        for i, num in reversed(list(enumerate(nums))):
            while stack and num >= nums[stack[-1]]:
                ans = max(ans, i - stack.pop())

        return ans

```

3133. 963.py

Path: LeetCode-main/solutions/963. Minimum Area Rectangle II/963.py

```

class Solution:
    def minAreaFreeRect(self, points: list[list[int]]) -> float:
        ans = math.inf

        # For each A, B pair points, {hash(A, B): (ax, ay, bx, by)}.
        centerToPoints = collections.defaultdict(list)

        for ax, ay in points:
            for bx, by in points:
                center = ((ax + bx) / 2, (ay + by) / 2)
                centerToPoints[center].append((ax, ay, bx, by))

        def dist(px: int, py: int, qx: int, qy: int) -> float:
            return (px - qx)**2 + (py - qy)**2

        # For all pair points "that share the same center".
        for points in centerToPoints.values():
            for ax, ay, _, _ in points:
                for cx, cy, dx, dy in points:
                    # AC is perpendicular to AD.
                    # AC dot AD = (cx - ax, cy - ay) dot (dx - ax, dy - ay) == 0.
                    if (cx - ax) * (dx - ax) + (cy - ay) * (dy - ay) == 0:
                        squaredArea = dist(ax, ay, cx, cy) * dist(ax, ay, dx, dy)
                        if squaredArea > 0:
                            ans = min(ans, squaredArea)

        return 0 if ans == math.inf else math.sqrt(ans)

```

3134. 964.py

Path: LeetCode-main/solutions/964. Least Operators to Express Number/964.py

```
class Solution:
    def leastOpsExpressTarget(self, x: int, target: int) -> int:
        @functools.lru_cache(None)
        def dfs(target):
            if x > target:
                return min(2 * target - 1, 2 * (x - target))
            if x == target:
                return 0

            prod = x
            n = 0
            while prod < target:
                prod *= x
                n += 1
            if prod == target:
                return n

            ans = dfs(target - prod // x) + n
            if prod < 2 * target:
                ans = min(ans, dfs(prod - target) + n + 1)
            return ans

        return dfs(target)
```

3135. 965.py

Path: LeetCode-main/solutions/965. Univalued Binary Tree/965.py

```
class Solution:
    def isUnivalTree(self, root: TreeNode | None) -> bool:
        if not root:
            return True
        if root.left and root.left.val != root.val:
            return False
        if root.right and root.right.val != root.val:
            return False
        return self.isUnivalTree(root.left) and self.isUnivalTree(root.right)
```

3136. 966.py

Path: LeetCode-main/solutions/966. Vowel Spellchecker/966.py

```

class Solution:
    def spellchecker(self, wordlist: list[str], queries: list[str]) -> list[str]:
        def lowerKey(word: str) -> str:
            return '$' + ''.join([c.lower() for c in word])

        def vowelKey(word: str) -> str:
            return ''.join(['*' if c.lower() in 'aeiou' else c.lower() for c in word])

        ans = []
        dict = {}

        for word in wordlist:
            dict.setdefault(word, word)
            dict.setdefault(lowerKey(word), word)
            dict.setdefault(vowelKey(word), word)

        for query in queries:
            if query in dict:
                ans.append(dict[query])
            elif lowerKey(query) in dict:
                ans.append(dict[lowerKey(query)])
            elif vowelKey(query) in dict:
                ans.append(dict[vowelKey(query)])
            else:
                ans.append('')

        return ans

```

3137. 969.py

Path: LeetCode-main/solutions/969. Pancake Sorting/969.py

```

class Solution:
    def pancakeSort(self, arr: list[int]) -> list[int]:
        ans = []

        for target in range(len(arr), 0, -1):
            index = arr.index(target)
            arr[:index + 1] = arr[:index + 1][::-1]
            arr[:target] = arr[:target][::-1]
            ans.append(index + 1)
            ans.append(target)

        return ans

```

3138. 97-2.py

Path: LeetCode-main/solutions/97. Interleaving String/97-2.py

```

class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m = len(s1)
        n = len(s2)
        if m + n != len(s3):
            return False

        dp = [False] * (n + 1)

        for i in range(m + 1):
            for j in range(n + 1):
                if i == 0 and j == 0:
                    dp[j] = True
                elif i == 0:
                    dp[j] = dp[j - 1] and s2[j - 1] == s3[j - 1]
                elif j == 0:
                    dp[j] = dp[j] and s1[i - 1] == s3[i - 1]
                else:
                    dp[j] = (dp[j] and s1[i - 1] == s3[i + j - 1] or
                           dp[j - 1] and s2[j - 1] == s3[i + j - 1])

        return dp[n]

```

3139. 97.py

Path: LeetCode-main/solutions/97. Interleaving String/97.py

```

class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m = len(s1)
        n = len(s2)
        if m + n != len(s3):
            return False

        # dp[i][j] := true if s3[0..i + j] is formed by the interleaving of
        # s1[0..i] and s2[0..j]
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        for i in range(1, m + 1):
            dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]

        for j in range(1, n + 1):
            dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1] or
                           dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])

        return dp[m][n]

```

3140. 970.py

Path: LeetCode-main/solutions/970. Powerful Integers/970.py

```
class Solution:
    def powerfulIntegers(self, x: int, y: int, bound: int) -> list[int]:
        xs = {x**i for i in range(20) if x**i < bound}
        ys = {y**i for i in range(20) if y**i < bound}
        return list({i + j for i in xs for j in ys if i + j <= bound})
```

3141. 972.py

Path: LeetCode-main/solutions/972. Equal Rational Numbers/972.py

```
class Solution:
    def isRationalEqual(self, s: str, t: str) -> bool:
        ratios = [1, 1 / 9, 1 / 99, 1 / 999, 1 / 9999]

        def valueOf(s: str) -> float:
            if s.find('(') == -1:
                return float(s)

            # Get the indices.
            leftParenIndex = s.find('(')
            rightParenIndex = s.find(')')
            dotIndex = s.find('.')

            # integerAndNonRepeating := <IntegerPart><.><NonRepeatingPart>
            integerAndNonRepeating = float(s[:leftParenIndex])
            nonRepeatingLength = leftParenIndex - dotIndex - 1

            # repeating := <RepeatingPart>
            repeating = int(s[leftParenIndex + 1:rightParenIndex])
            repeatingLength = rightParenIndex - leftParenIndex - 1
            return integerAndNonRepeating + repeating * 0.1**nonRepeatingLength * ratios[repeatingLength]

        return abs(valueOf(s) - valueOf(t)) < 1e-9
```

3142. 973-2.py

Path: LeetCode-main/solutions/973. K Closest Points to Origin/973-2.py

```
class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        def squareDist(p: list[int]) -> int:
            return p[0] * p[0] + p[1] * p[1]

        def quickSelect(l: int, r: int, k: int) -> None:
            pivot = points[r]

            nextSwapped = l
            for i in range(l, r):
                if squareDist(points[i]) <= squareDist(pivot):
                    points[nextSwapped], points[i] = points[i], points[nextSwapped]
                    nextSwapped += 1
            points[nextSwapped], points[r] = points[r], points[nextSwapped]

            count = nextSwapped - l + 1  # the number of points <= pivot
            if count == k:
                return
            if count > k:
                quickSelect(l, nextSwapped - 1, k)
            else:
                quickSelect(nextSwapped + 1, r, k - count)

        quickSelect(0, len(points) - 1, k)
        return points[0:k]
```

3143. 973-3.py

Path: LeetCode-main/solutions/973. K Closest Points to Origin/973-3.py

```

class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        def squareDist(p: list[int]) -> int:
            return p[0] * p[0] + p[1] * p[1]

        def quickSelect(l: int, r: int, k: int) -> None:
            randIndex = random.randint(0, r - 1 + 1) + 1
            points[randIndex], points[r] = points[r], points[randIndex]
            pivot = points[r]

            nextSwapped = l
            for i in range(l, r):
                if squareDist(points[i]) <= squareDist(pivot):
                    points[nextSwapped], points[i] = points[i], points[nextSwapped]
                    nextSwapped += 1
            points[nextSwapped], points[r] = points[r], points[nextSwapped]

            count = nextSwapped - l + 1  # the number of points <= pivot
            if count == k:
                return
            if count > k:
                quickSelect(l, nextSwapped - 1, k)
            else:
                quickSelect(nextSwapped + 1, r, k - count)

        quickSelect(0, len(points) - 1, k)
        return points[0:k]

```

```

class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        def squareDist(p: list[int]) -> int:
            return p[0] * p[0] + p[1] * p[1]

        def quickSelect(l: int, r: int, k: int) -> None:
            pivot = points[r]

            nextSwapped = l
            for i in range(l, r):
                if squareDist(points[i]) <= squareDist(pivot):
                    points[nextSwapped], points[i] = points[i], points[nextSwapped]
                    nextSwapped += 1
            points[nextSwapped], points[r] = points[r], points[nextSwapped]

            count = nextSwapped - l + 1  # the number of points <= pivot
            if count == k:
                return
            if count > k:
                quickSelect(l, nextSwapped - 1, k)
            else:
                quickSelect(nextSwapped + 1, r, k - count)

        quickSelect(0, len(points) - 1, k)
        return points[0:k]

```

3144. 973.py

Path: LeetCode-main/solutions/973. K Closest Points to Origin/973.py


```
class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        maxHeap = []

        for x, y in points:
            heapq.heappush(maxHeap, (- x * x - y * y, [x, y]))
            if len(maxHeap) > k:
                heapq.heappop(maxHeap)

        return [pair[1] for pair in maxHeap]
```

3145. 974.py

Path: LeetCode-main/solutions/974. Subarray Sums Divisible by K/974.py

```
class Solution:
    def subarraysDivByK(self, nums: list[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = [0] * k
        count[0] = 1

        for num in nums:
            prefix = (prefix + num % k + k) % k
            ans += count[prefix]
            count[prefix] += 1

        return ans
```

3146. 976.py

Path: LeetCode-main/solutions/976. Largest Perimeter Triangle/976.py

```
class Solution:
    def largestPerimeter(self, nums: list[int]) -> int:
        nums = sorted(nums)

        for i in range(len(nums) - 1, 1, -1):
            if nums[i - 2] + nums[i - 1] > nums[i]:
                return nums[i - 2] + nums[i - 1] + nums[i]

        return 0
```

3147. 977.py

Path: LeetCode-main/solutions/977. Squares of a Sorted Array/977.py

```
class Solution:
    def sortedSquares(self, nums: list[int]) -> list[int]:
        n = len(nums)
        l = 0
        r = n - 1
        ans = [0] * n

        while n:
            n -= 1
            if abs(nums[l]) > abs(nums[r]):
                ans[n] = nums[l] * nums[l]
                l += 1
            else:
                ans[n] = nums[r] * nums[r]
                r -= 1

        return ans
```

3148. 978.py

Path: LeetCode-main/solutions/978. Longest Turbulent Subarray/978.py

```
class Solution:
    def maxTurbulenceSize(self, arr: list[int]) -> int:
        ans = 1
        increasing = 1
        decreasing = 1

        for i in range(1, len(arr)):
            if arr[i] > arr[i - 1]:
                increasing = decreasing + 1
                decreasing = 1
            elif arr[i] < arr[i - 1]:
                decreasing = increasing + 1
                increasing = 1
            else:
                increasing = 1
                decreasing = 1
            ans = max(ans, max(increasing, decreasing))

        return ans
```

3149. 98-2.py

Path: LeetCode-main/solutions/98. Validate Binary Search Tree/98-2.py

```

class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        stack = []
        pred = None

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()
            if pred and pred.val >= root.val:
                return False
            pred = root
            root = root.right

        return True

```

3150. 98.py

Path: LeetCode-main/solutions/98. Validate Binary Search Tree/98.py

```

class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        def isValidBST(root: TreeNode | None,
                        minNode: TreeNode | None, maxNode: TreeNode | None) -> bool:
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False

            return (isValidBST(root.left, minNode, root) and
                    isValidBST(root.right, root, maxNode))

        return isValidBST(root, None, None)

```

3151. 981.py

Path: LeetCode-main/solutions/981. Time Based Key-Value Store/981.py

```

class TimeMap:
    def __init__(self):
        self.values = collections.defaultdict(list)
        self.timestamps = collections.defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.values[key].append(value)
        self.timestamps[key].append(timestamp)

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.timestamps:
            return ''
        i = bisect.bisect(self.timestamps[key], timestamp)
        return self.values[key][i - 1] if i > 0 else ''

```

3152. 982.py

Path: LeetCode-main/solutions/982. Triples with Bitwise AND Equal To Zero/982.py

```
class Solution:
    def countTriplets(self, nums: list[int]) -> int:
        MAX = 1 << 16
        ans = 0
        count = [0] * MAX # {nums[i] & nums[j]: times}

        for a in nums:
            for b in nums:
                count[a & b] += 1

        for num in nums:
            for i in range(MAX):
                if (num & i) == 0:
                    ans += count[i]

        return ans
```

3153. 983.py

Path: LeetCode-main/solutions/983. Minimum Cost For Tickets/983.py

```
class Solution:
    def mincostTickets(self, days: list[int], costs: list[int]) -> int:
        ans = 0
        last7 = collections.deque()
        last30 = collections.deque()

        for day in days:
            while last7 and last7[0][0] + 7 <= day:
                last7.popleft()
            while last30 and last30[0][0] + 30 <= day:
                last30.popleft()
            last7.append([day, ans + costs[1]])
            last30.append([day, ans + costs[2]])
            ans = min(ans + costs[0], last7[0][1], last30[0][1])

        return ans
```

3154. 985.py

Path: LeetCode-main/solutions/985. Sum of Even Numbers After Queries/985.py

```

class Solution:
    def sumEvenAfterQueries(
        self,
        nums: list[int],
        queries: list[list[int]],
    ) -> list[int]:
        ans = []
        summ = sum(a for a in nums if a % 2 == 0)

        for val, index in queries:
            if nums[index] % 2 == 0:
                summ -= nums[index]
            nums[index] += val
            if nums[index] % 2 == 0:
                summ += nums[index]
            ans.append(summ)

        return ans

```

3155. 986.py

Path: LeetCode-main/solutions/986. Interval List Intersections/986.py

```

class Solution:
    def intervalIntersection(self, firstList: list[list[int]],
                           secondList: list[list[int]]) -> list[list[int]]:

        ans = []
        i = 0
        j = 0

        while i < len(firstList) and j < len(secondList):
            # lo := the start of the intersection
            # hi := the end of the intersection
            lo = max(firstList[i][0], secondList[j][0])
            hi = min(firstList[i][1], secondList[j][1])
            if lo <= hi:
                ans.append([lo, hi])
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1

        return ans

```

3156. 987.py

Path: LeetCode-main/solutions/987. Vertical Order Traversal of a Binary Tree/987.py

```
class Solution:
    def verticalTraversal(self, root: TreeNode | None) -> list[list[int]]:
        ans = []
        xToNodes = collections.defaultdict(list)

        def dfs(node: TreeNode | None, x: int, y: int) -> None:
            if not node:
                return
            xToNodes[x].append((-y, node.val))
            dfs(node.left, x - 1, y - 1)
            dfs(node.right, x + 1, y - 1)

        dfs(root, 0, 0)

        for _, nodes in sorted(xToNodes.items(), key=lambda x: x[0]):
            ans.append([val for _, val in sorted(nodes)])

        return ans
```

3157. 989.py

Path: LeetCode-main/solutions/989. Add to Array-Form of Integer/989.py

```
class Solution:
    def addToArrayForm(self, num: list[int], k: int) -> list[int]:
        for i in reversed(range(len(num))):
            k, num[i] = divmod(num[i] + k, 10)

        while k > 0:
            num = [k % 10] + num
            k //= 10

        return num
```

3158. 99-2.py

Path: LeetCode-main/solutions/99. Recover Binary Search Tree/99-2.py

```
class Solution:
    def recoverTree(self, root: TreeNode | None) -> None:
        pred = None
        x = None # the first wrong node
        y = None # the second wrong node
        stack = []

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()
            if pred and root.val < pred.val:
                y = root
                if not x:
                    x = pred
            pred = root
            root = root.right

        def swap(x: TreeNode | None, y: TreeNode | None) -> None:
            temp = x.val
            x.val = y.val
            y.val = temp

        swap(x, y)
```

3159. 99-3.py

Path: LeetCode-main/solutions/99. Recover Binary Search Tree/99-3.py

```

class Solution:
    def recoverTree(self, root: TreeNode | None) -> None:
        pred = None
        x = None # the first wrong node
        y = None # the second wrong node

    def findPredecessor(root: TreeNode | None) -> TreeNode | None:
        pred = root.left
        while pred.right and pred.right != root:
            pred = pred.right
        return pred

    while root:
        if root.left:
            morrisPred = findPredecessor(root)
            if morrisPred.right:
                # The node has already been connected before.
                # Start the main logic.
                if pred and root.val < pred.val:
                    y = root
                    if not x:
                        x = pred
                pred = root
                # End of the main logic
                morrisPred.right = None # Break the connection.
                root = root.right
            else:
                morrisPred.right = root # Connect it.
                root = root.left
        else:
            # Start the main logic.
            if pred and root.val < pred.val:
                y = root
                if not x:
                    x = pred
            pred = root
            # End of the main logic.
            root = root.right

    def swap(x: TreeNode | None, y: TreeNode | None) -> None:
        temp = x.val
        x.val = y.val
        y.val = temp

    swap(x, y)

```

3160. 99.py

Path: LeetCode-main/solutions/99. Recover Binary Search Tree/99.py


```

class Solution:
    def recoverTree(self, root: TreeNode | None) -> None:
        def swap(x: TreeNode | None, y: TreeNode | None) -> None:
            temp = x.val
            x.val = y.val
            y.val = temp

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return

            inorder(root.left)

            if self.pred and root.val < self.pred.val:
                self.y = root
                if not self.x:
                    self.x = self.pred
                else:
                    return
                self.pred = root

            inorder(root.right)

        inorder(root)
        swap(self.x, self.y)

    pred = None
    x = None # the first wrong node
    y = None # the second wrong node

```

3161. 990.py

Path: LeetCode-main/solutions/990. Satisfiability of Equality Equations/990.py

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))

    def union(self, u: int, v: int) -> None:
        self.id[self.find(u)] = self.find(v)

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def equationsPossible(self, equations: list[str]) -> bool:
        uf = UnionFind(26)

        for x, op, _, y in equations:
            if op == '=':
                uf.union(ord(x) - ord('a'),
                        ord(y) - ord('a'))

        return all(
            uf.find(ord(x) - ord('a')) !=
            uf.find(ord(y) - ord('a'))
            for x, op, _, y in equations
            if op == '!'
        )

```

3162. 991.py

Path: LeetCode-main/solutions/991. Broken Calculator/991.py

```
class Solution:
    def brokenCalc(self, startValue: int, target: int) -> int:
        ops = 0

        while startValue < target:
            if target % 2 == 0:
                target //= 2
            else:
                target += 1
            ops += 1

        return ops + startValue - target
```

3163. 992.py

Path: LeetCode-main/solutions/992. Subarrays with K Different Integers/992.py

```
class Solution:
    def subarraysWithKDistinct(self, nums: list[int], k: int) -> int:
        def subarraysWithAtMostKDistinct(k: int) -> int:
            res = 0
            count = collections.Counter()

            l = 0
            for r, num in enumerate(nums):
                count[num] += 1
                if count[num] == 1:
                    k -= 1
                while k < 0:
                    count[nums[l]] -= 1
                    if count[nums[l]] == 0:
                        k += 1
                    l += 1
                res += r - l + 1 # nums[l..r], nums[l + 1..r], ..., nums[r]

            return res

        return subarraysWithAtMostKDistinct(k) - subarraysWithAtMostKDistinct(k - 1)
```

3164. 995.py

Path: LeetCode-main/solutions/995. Minimum Number of K Consecutive Bit Flips/995.py

```

class Solution:
    def minKBitFlips(self, nums: list[int], k: int) -> int:
        ans = 0
        flippedTime = 0

        for i, num in enumerate(nums):
            if i >= k and nums[i - k] == 2:
                flippedTime -= 1
            if flippedTime % 2 == num:
                if i + k > len(nums):
                    return -1
                ans += 1
                flippedTime += 1
                nums[i] = 2

        return ans

```

3165. 996.py

Path: LeetCode-main/solutions/996. Number of Squareful Arrays/996.py

```

class Solution:
    def numSquarefulPerms(self, nums: list[int]) -> int:
        ans = 0
        used = [False] * len(nums)

        def isSquare(num: int) -> bool:
            root = math.isqrt(num)
            return root * root == num

        def dfs(path: list[int]) -> None:
            nonlocal ans
            if len(path) > 1 and not isSquare(path[-1] + path[-2]):
                return
            if len(path) == len(nums):
                ans += 1
                return

            for i, a in enumerate(nums):
                if used[i]:
                    continue
                if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                    continue
                used[i] = True
                dfs(path + [a])
                used[i] = False

        nums.sort()
        dfs([])
        return ans

```

3166. 997.py

Path: LeetCode-main/solutions/997. Find the Town Judge/997.py

```
class Solution:
    def findJudge(self, n: int, trust: list[list[int]]) -> int:
        count = [0] * (n + 1)

        for a, b in trust:
            count[a] -= 1
            count[b] += 1

        for i in range(1, n + 1):
            if count[i] == n - 1:
                return i

        return -1
```

3167. 998-2.py

Path: LeetCode-main/solutions/998. Maximum Binary Tree II/998-2.py

```
class Solution:
    def insertIntoMaxTree(
        self,
        root: TreeNode | None,
        val: int,
    ) -> TreeNode | None:
        if root.val < val:
            return TreeNode(val, root, None)
        curr = root
        while curr.right and curr.right.val > val:
            curr = curr.right
        inserted = TreeNode(val, curr.right, None)
        curr.right = inserted
        return root
```

3168. 998.py

Path: LeetCode-main/solutions/998. Maximum Binary Tree II/998.py

```
class Solution:
    def insertIntoMaxTree(
        self,
        root: TreeNode | None,
        val: int,
    ) -> TreeNode | None:
        if not root:
            return TreeNode(val)
        if root.val < val:
            return TreeNode(val, root, None)
        root.right = self.insertIntoMaxTree(root.right, val)
        return root
```

3169. 999.py

Path: LeetCode-main/solutions/999. Available Captures for Rook/999.py

```
class Solution:
    def numRookCaptures(self, board: list[list[str]]) -> int:
        ans = 0

        for i in range(8):
            for j in range(8):
                if board[i][j] == 'R':
                    i0 = i
                    j0 = j

            for d in [[1, 0], [0, 1], [-1, 0], [0, -1]]:
                i = i0 + d[0]
                j = j0 + d[1]
                while 0 <= i < 8 and 0 <= j < 8:
                    if board[i][j] == 'p':
                        ans += 1
                    if board[i][j] != '.':
                        break
                    i += d[0]
                    j += d[1]

        return ans
```
